FAKE RIDER MOTORCYCLE REPAIR SHOP

SYSTEM SOURCE CODE

Thesis Appendix

Document scope: Application source code, database schema and migrations, offline database setup, automation scripts, and essential project configuration.

Excluded: Dependencies, build output, binary assets, diagrams, generated PDFs, logs, documentation, and environment files containing configuration secrets.

Formatting: US Letter; 1.5-inch left margin; 1-inch top, right, and bottom margins; Arial, 12 pt.

Generated: October 28, 2026 at 6:17 AM
Included files: 236

SOURCE FILE INVENTORY

Project configuration

- .prettierrc
- components.json
- eslint.config.js
- package.json
- tsconfig.json
- vite.config.ts

Offline development database

- offline/data/auth-data.sql
- offline/data/auth-users.sql
- offline/data/public-data.sql
- offline/data/schema.sql
- offline/setup.sql
- offline/supabase/config.toml

Application source code

- src/assets/logo-shp.png.asset.json
- src/components/admin/archive-confirmation-dialog.tsx
- src/components/admin/data-list.tsx
- src/components/admin/motorcycle-catalog-manager.tsx
- src/components/admin/page-header.tsx
- src/components/admin/pagination-controls.tsx
- src/components/admin/print-document.tsx
- src/components/admin/product-manager.tsx
- src/components/admin/settings-page.tsx
- src/components/site/appointment-verification-dialog.tsx
- src/components/site/booking-schedule.tsx
- src/components/site/copy-reference-button.tsx
- src/components/site/offline-connection.tsx
- src/components/site/product-card.tsx
- src/components/site/service-booking-row.tsx
- src/components/site/site-footer.tsx
- src/components/site/site-header.tsx
- src/components/site/terms-agreement.tsx
- src/components/site/theme-toggle.tsx
- src/components/site/turnstile-challenge.tsx
- src/components/ui/alert-dialog.tsx
- src/components/ui/alert.tsx
- src/components/ui/badge.tsx
- src/components/ui/button.tsx
- src/components/ui/calendar.tsx
- src/components/ui/card.tsx
- src/components/ui/chart.tsx

- src/components/ui/checkbox.tsx
- src/components/ui/dialog.tsx
- src/components/ui/dropdown-menu.tsx
- src/components/ui/field-error.tsx
- src/components/ui/input.tsx
- src/components/ui/label.tsx
- src/components/ui/popover.tsx
- src/components/ui/select.tsx
- src/components/ui/separator.tsx
- src/components/ui/sheet.tsx
- src/components/ui/sidebar.tsx
- src/components/ui/skeleton.tsx
- src/components/ui/switch.tsx
- src/components/ui/table.tsx
- src/components/ui/tabs.tsx
- src/components/ui/textarea.tsx
- src/components/ui/toggle-group.tsx
- src/components/ui/toggle.tsx
- src/components/ui/tooltip.tsx
- src/hooks/use-mobile.tsx
- src/integrations/supabase/auth-attacher.ts
- src/integrations/supabase/client.server.ts
- src/integrations/supabase/client.ts
- src/integrations/supabase/types.ts
- src/lib/admin-activity.ts
- src/lib/admin-alerts.server.ts
- src/lib/admin-date-range.ts
- src/lib/appointment-confirmation.functions.ts
- src/lib/appointment-reference-recovery.functions.ts
- src/lib/appointment-verification.server.ts
- src/lib/availability.ts
- src/lib/booking-draft.ts
- src/lib/booking.functions.ts
- src/lib/customer-emails.ts
- src/lib/email.server.ts
- src/lib/error-capture.ts
- src/lib/error-page.ts
- src/lib/export-branding.ts
- src/lib/offline-sync.server.ts
- src/lib/print-document.ts
- src/lib/scheduled-request.server.ts
- src/lib/shop.ts
- src/lib/utils.ts
- src/router.tsx
- src/routes/__root.tsx

- src/routes/_authenticated/admin.tsx
- src/routes/_authenticated/admin/activity-log.tsx
- src/routes/_authenticated/admin/appointments.tsx
- src/routes/_authenticated/admin/archive.tsx
- src/routes/_authenticated/admin/availability.tsx
- src/routes/_authenticated/admin/blocked-numbers.tsx
- src/routes/_authenticated/admin/customers.tsx
- src/routes/_authenticated/admin/index.tsx
- src/routes/_authenticated/admin/mechanics.tsx
- src/routes/_authenticated/admin/motorcycles.tsx
- src/routes/_authenticated/admin/notifications.tsx
- src/routes/_authenticated/admin/parts.tsx
- src/routes/_authenticated/admin/prices.tsx
- src/routes/_authenticated/admin/reports.tsx
- src/routes/_authenticated/admin/schedule-blocks.tsx
- src/routes/_authenticated/admin/services.tsx
- src/routes/_authenticated/admin/settings.tsx
- src/routes/_authenticated/route.tsx
- src/routes/api/offline-status.ts
- src/routes/api/offline-sync.ts
- src/routes/api/scheduled-notifications.ts
- src/routes/auth.tsx
- src/routes/book.tsx
- src/routes/index.tsx
- src/routes/my-appointment.tsx
- src/routes/reset-password.tsx
- src/routes/services.tsx
- src/routes/shop.tsx
- src/routeTree.gen.ts
- src/server.ts
- src/start.ts
- src/styles.css

Database schema and migrations

- supabase/.temp/linked-project.json
- supabase/config.toml
- supabase/migrations/20260816152359_1aadbf74-b753-439f-b1eb-4877adbc55dd.sql
- supabase/migrations/20260816152418_d84da632-d75d-4e40-bb9e-ebd93be253cf.sql
- supabase/migrations/20260816152436_f6440707-f2a5-4d75-92b2-eb0b2feb3f33.sql
- supabase/migrations/20260818222449_d76fe9f7-c81f-40c7-915c-1d3658470a09.sql
- supabase/migrations/20260819150000_archive_soft_delete.sql

• supabase/migrations/20260819160000_fix_trigger_permissions.sql
• supabase/migrations/20260819170000_fix_time_slots_duplicates.sql
• supabase/migrations/20260819180000_create_admin_account.sql
• supabase/migrations/20260819190000_price_history_table.sql
• supabase/migrations/20260819193343_expand_ph_motorcycle_catalog.sql
• supabase/migrations/20260819195215_cleanup_duplicate_motorcycles.sql
• supabase/migrations/20260820113112_crew_scheduling_schema.sql
• supabase/migrations/20260820161821_crew_schedules_specific_dates.sql
• supabase/migrations/20260820162150_crew_schedules_note.sql
• supabase/migrations/20260820163024_crew_schedules_nullable_times.sql
• supabase/migrations/20260820190000_crew_schedules_date_override_uniq
ue.sql
• supabase/migrations/20260820214127_remove_seeded_weekly_schedules.
sql
• supabase/migrations/20260820224500_fix_crew_schedules_upsert_conflict.
sql
• supabase/migrations/20260821150500_fix_crew_schedules_updated_at.sql
• supabase/migrations/20260821161200_create_products_storage_bucket.sql
• supabase/migrations/20260821185300_dedupe_services_add_name_uniqu
e.sql
• supabase/migrations/20260821194500_dedupe_products_services.sql
• supabase/migrations/20260822125500_dedupe_crew_members_add_name
_unique.sql
• supabase/migrations/20260822133700_create_blocked_numbers_table.sql
• supabase/migrations/20260823123700_seed_default_crew_schedules.sql
• supabase/migrations/20260823140000_remove_default_weekly_crew_sche
dules.sql
• supabase/migrations/20260823150000_add_booking_duration_capacity_gua
rd.sql
• supabase/migrations/20260824090000_add_blocked_numbers_archiving.sql
• supabase/migrations/20260824100000_create_booking_atomic.sql
• supabase/migrations/20260824110000_normalize_customer_phone_number
s.sql
• supabase/migrations/20260824120000_add_public_endpoint_rate_limits.sql
• supabase/migrations/20260824130000_add_booking_idempotency.sql
• supabase/migrations/20260828120000_fix_create_booking_atomic_referenc
e_ambiguity.sql
• supabase/migrations/20260830120000_separate_deactivation_from_archivin
g.sql
• supabase/migrations/20260902120000_create_admin_activity_logs.sql
• supabase/migrations/20260902130000_harden_activity_log_retention.sql
•
supabase/migrations/20260902140000_expand_admin_activity_coverage.sql
• supabase/migrations/20260904110000_secure_public_data_and_remove_st
aff_role.sql

• supabase/migrations/20260904120000_add_admin_pagination_queries.sql
• supabase/migrations/20260905120000_add_activity_log_security_context.sql
• supabase/migrations/20260905121000_align_booking_duration_buffers.sql
• supabase/migrations/20260906120000_create_shop_settings.sql
• supabase/migrations/20260906130000_expand_default_booking_terms.sql
• supabase/migrations/20260907120000_add_product_stock_quantities.sql
• supabase/migrations/20260908090000_add_services_to_booking_report_rows.sql
• supabase/migrations/20260908120000_add_appointment_rescheduling.sql
• supabase/migrations/20260908130000_rescheduled_booking_starts_pending.sql
• supabase/migrations/20260909110000_create_atomic_admin_appointment_update.sql
• supabase/migrations/20260909120000_enforce_appointment_crew_assignment.sql
• supabase/migrations/20260910100000_review_pending_appointment.sql
• supabase/migrations/20260910110000_add_pending_reschedule_requests.sql
• supabase/migrations/20260910120000_prevent_duplicate_customer_service_appointments.sql
• supabase/migrations/20260910121000_include_duplicate_appointment_details.sql
• supabase/migrations/20260910122000_fix_duplicate_service_trigger_row_types.sql
• supabase/migrations/20260911100000_limit_and_track_appointment_reschedules.sql
• supabase/migrations/20260912090000_add_structured_appointment_names.sql
• supabase/migrations/20260912100000_add_motorcycle_catalog_specifications.sql
• supabase/migrations/20260912110000_correct_legacy_motorcycle_catalog_row.sql
• supabase/migrations/20260912120000_create_service_configurations.sql
• supabase/migrations/20260913110000_add_price_configuration_history_and_archive_dates.sql
• supabase/migrations/20260913130000_add_booking_motorcycle_configuration.sql
• supabase/migrations/20260913140000_seed_complete_service_configuration_matrix.sql
• supabase/migrations/20260913150000_simplify_service_configurations_to_cc_categories.sql
• supabase/migrations/20260913160000_remove_legacy_service_configurations.sql
• supabase/migrations/20260913170000_add_price_history_actor_email.sql

• supabase/migrations/20260914100000_reschedule_approval_creates_linked_booking.sql
• supabase/migrations/20260915000000_add_reschedule_history_tracking.sql
• supabase/migrations/20260915110000_separate_motorcycle_catalog_from_products.sql
• supabase/migrations/20260917100000_harden_booking_capacity_and_reschedule_slots.sql
• supabase/migrations/20260918000000_remove_motorcycle_specifications_from_booking.sql
• supabase/migrations/20260918100000_remove_customer_email_from_bookings.sql
• supabase/migrations/20260918110000_make_booking_middle_name_optional.sql
• supabase/migrations/20260920090000_add_shop_operating_hours.sql
• supabase/migrations/20260920093000_enforce_configured_shop_hours.sql
• supabase/migrations/20260920100000_add_archive_retention.sql
• supabase/migrations/20260921113000_add_customer_cancellation_threshold_notification.sql
• supabase/migrations/20260922090000_fix_reschedule_approval_after_email_removal.sql
• supabase/migrations/20260922091000_restore_reschedule_request_tracker.sql
• supabase/migrations/20260922100000_add_multi_day_booking_continuations.sql
• supabase/migrations/20260923100000_align_admin_appointment_contact_validation.sql
• supabase/migrations/20260924100000_add_appointment_service_progress.sql
• supabase/migrations/20260924110000_harden_service_progress_notification_race.sql
• supabase/migrations/20260924120000_standardize_local_philippine_mobile_numbers.sql
• supabase/migrations/20260925100000_support_customer_name_segments.sql
• supabase/migrations/20260925110000_constrain_service_progress_notification_timeline.sql
• supabase/migrations/20260925120000_transfer_admin_account.sql
• supabase/migrations/20260926100000_add_report_total_value.sql
• supabase/migrations/20260926200000_add_engine_overhaul_500cc_model_overrides.sql
• supabase/migrations/20260927090000_replace_service_catalog_from_updated_details.sql
• supabase/migrations/20260927100000_schedule_durable_admin_notifications.sql
• supabase/migrations/20260927110000_remove_reschedule_loop_variable_s

hadowing.sql
• supabase/migrations/20260928100000_notify_multiple_active_service_bookings.sql
• supabase/migrations/20260928110000_add_customer_no_show_history_warnings.sql
• supabase/migrations/20260928120000_enforce_single_active_admin_session.sql
• supabase/migrations/20260928130000_restore_legacy_booking_rpc_compatibility.sql
• supabase/migrations/20260928140000_fix_uuid_aggregate_in_booking_alert.sql
• supabase/migrations/20260928150000_fix_admin_session_validation.sql
• supabase/migrations/20260929120000_require_service_creation_fields.sql
• supabase/migrations/20261001100000_remove_booking_duration_buffer.sql
• supabase/migrations/20261004090000_add_booking_confirmation_emails.sql
• supabase/migrations/20261004100000_secure_email_reference_recovery.sql
• supabase/migrations/20261006090000_add_motorcycle_cc_categories.sql
• supabase/migrations/20261006100000_replace_service_model_overrides_with_large_bike_pricing.sql
• supabase/migrations/20261007100000_seed_complete_motorcycle_catalog.sql
• supabase/migrations/20261007120000_add_appointment_products_financial_reporting.sql
• supabase/migrations/20261008100000_harden_appointment_crew_availability.sql
• supabase/migrations/20261008110000_add_appointment_confirmation_sms.sql
• supabase/migrations/20261008120000_add_service_completion_customer_notifications.sql
• supabase/migrations/20261008130000_limit_textbee_sms_to_demo_phone.sql
• supabase/migrations/20261008140000_add_completed_financial_report.sql
• supabase/migrations/20261011140000_allow_products_before_service_start.sql
• supabase/migrations/20261014100000_add_appointment_action_verification.sql
• supabase/migrations/20261015100000_protect_completed_appointments_and_lock_down_functions.sql
• supabase/migrations/20261015120000_hold_reschedule_requests_limit_bookings_balance_mechanics.sql
• supabase/migrations/20261018100000_dashboard_summary_and_safe_crew_shift_changes.sql
• supabase/migrations/20261021100000_report_filter_multiple_services.sql

• supabase/migrations/20261026100000_drop_unused_booking_function_versions.sql
• supabase/migrations/20261026110000_appointment_reminders_and_scheduled_notifications.sql

.prettierrc

```
{
  "printWidth": 100,
  "semi": true,
  "singleQuote": false,
  "trailingComma": "all"
}
```

components.json

```
{
  "$schema": "https://ui.shadcn.com/schema.json",
  "style": "new-york",
  "rsc": false,
  "tsx": true,
  "tailwind": {
    "css": "src/styles.css",
    "baseColor": "slate",
    "cssVariables": true,
    "prefix": ""
  },
  "iconLibrary": "lucide",
  "rtl": false,
  "aliases": {
    "components": "@/components",
    "utils": "@/lib/utils",
    "ui": "@/components/ui",
    "lib": "@/lib",
    "hooks": "@/hooks"
  },
  "registries": {}
}
```

eslint.config.js

```
import js from "@eslint/js";
import eslintPluginPrettier from "eslint-plugin-prettier/recommended";
import globals from "globals";
import reactHooks from "eslint-plugin-react-hooks";
import reactRefresh from "eslint-plugin-react-refresh";
import tseslint from "typescript-eslint";

export default tseslint.config(
  { ignores: ["dist", ".output", ".vinxi", ".vercel"] },
  {
    extends: [js.configs.recommended, ...tseslint.configs.recommended],
    files: ["**/*.{ts,tsx}"],
    languageOptions: {
      ecmaVersion: 2020,
      globals: globals.browser,
    },
    plugins: {
      "react-hooks": reactHooks,
      "react-refresh": reactRefresh,
    },
    rules: {
      ...reactHooks.configs.recommended.rules,
      "no-restricted-imports": [
        "error",
        {
          paths: [
            {
              name: "server-only",
              message:
                "TanStack Start does not use the Next.js `server-only` package.
Rename the module to `*.server.ts` or mark it with
`@tanstack/react-start/server-only`.",
            },
          ],
        },
      ],
      "react-refresh/only-export-components": ["warn", { allowConstantExport: true
}],
      "@typescript-eslint/no-unused-vars": "off",
    },
  },
  eslintPluginPrettier,
);
```

offline/data/auth-data.sql

```
SET session_replication_role = replica;

--
-- PostgreSQL database dump
--

-- \restrict r5AgHlgabGRZCwUMFi0zsiTngngY8r3YRtqdDcZ6eN2q3Ok3U312yUx
3b113BbF

-- Dumped from database version 17.6
-- Dumped by pg_dump version 17.6

SET statement_timeout = 0;
SET lock_timeout = 0;
SET idle_in_transaction_session_timeout = 0;
SET transaction_timeout = 0;
SET client_encoding = 'UTF8';
SET standard_conforming_strings = on;
SELECT pg_catalog.set_config('search_path', '', false);
SET check_function_bodies = false;
SET xmloption = content;
SET client_min_messages = warning;
SET row_security = off;

--
-- Data for Name: custom_oauth_providers; Type: TABLE DATA; Schema: auth;
Owner: supabase_auth_admin
--

COPY "auth"."custom_oauth_providers" ("id", "provider_type", "identifier",
"name", "client_id", "client_secret", "acceptable_client_ids", "scopes",
"pkce_enabled", "attribute_mapping", "authorization_params", "enabled",
"email_optional", "issuer", "discovery_url", "skip_nonce_check",
"cached_discovery", "discovery_cached_at", "authorization_url", "token_url",
"userinfo_url", "jwks_uri", "created_at", "updated_at", "custom_claims_allowlist")
FROM stdin;
\.


--
-- Data for Name: users; Type: TABLE DATA; Schema: auth; Owner:
supabase_auth_admin
--
```

COPY "auth"."users" ("instance_id", "id", "aud", "role", "email", "encrypted_password", "email_confirmed_at", "invited_at", "confirmation_token", "confirmation_sent_at", "recovery_token", "recovery_sent_at", "email_change_token_new", "email_change", "email_change_sent_at", "last_sign_in_at", "raw_app_meta_data", "raw_user_meta_data", "is_super_admin", "created_at", "updated_at", "phone", "phone_confirmed_at", "phone_change", "phone_change_token", "phone_change_sent_at", "email_change_token_current", "email_change_confirm_status", "banned_until", "reauthentication_token", "reauthentication_sent_at", "is_sso_user", "deleted_at", "is_anonymous") FROM stdin;
00000000-0000-0000-0000-000000000000 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 authenticated authenticated eapolona0803@gmail.com $2a$10$JODMPeJHysV3BDx8j1mKm.WyG3Zke0E3VZZIWIumYp0uGshb.wMzK 2026-09-25 11:16:13.515718+00 \N \N 9daaa4b8a054fcba2a7be624602a36d01977707a01f2ff6651a3a9ef 2026-10-21 12:37:46.569171+00 \N 2026-10-21 12:36:06.210558+00 {"provider": "email", "providers": ["email"]} {"full_name": "JoeMar tats", "email_verified": true} \N 2026-09-25 11:16:13.493649+00 2026-10-21 12:37:48.356871+00 \N \N \N 0 \N \N f \N f
\.

--
-- Data for Name: identities; Type: TABLE DATA; Schema: auth; Owner: supabase_auth_admin
--

COPY "auth"."identities" ("provider_id", "user_id", "identity_data", "provider", "last_sign_in_at", "created_at", "updated_at", "id") FROM stdin;
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 {"sub": "7a9e497f-8c3f-44a7-ab4e-c9247c50eb74", "email": "eapolona0803@gmail.com", "email_verified": false, "phone_verified": false} email 2026-09-25 11:16:13.50782+00 2026-09-25 11:16:13.507885+00 2026-09-25 11:16:13.507885+00 be414415-e886-4cd4-898f-588e6ff80551
\.

--
-- Data for Name: mfa_recovery_code_sets; Type: TABLE DATA; Schema: auth; Owner: supabase_auth_admin
--

COPY "auth"."mfa_recovery_code_sets" ("id", "user_id", "mfa_factor_id", "failed_verification_count", "verification_locked_until", "created_at", "updated_at") FROM stdin;
\.

--
-- Data for Name: mfa_recovery_codes; Type: TABLE DATA; Schema: auth; Owner: supabase_auth_admin
--

COPY "auth"."mfa_recovery_codes" ("id", "mfa_recovery_code_set_id", "code_hash", "consumed_at", "created_at") FROM stdin;
\.

--
-- Data for Name: oauth_clients; Type: TABLE DATA; Schema: auth; Owner: supabase_auth_admin
--

COPY "auth"."oauth_clients" ("id", "client_secret_hash", "registration_type", "redirect_uris", "grant_types", "client_name", "client_uri", "logo_uri", "created_at", "updated_at", "deleted_at", "client_type", "token_endpoint_auth_method") FROM stdin;
\.

--
-- Data for Name: oauth_authorizations; Type: TABLE DATA; Schema: auth; Owner: supabase_auth_admin
--

COPY "auth"."oauth_authorizations" ("id", "authorization_id", "client_id", "user_id", "redirect_uri", "scope", "state", "resource", "code_challenge", "code_challenge_method", "response_type", "status", "authorization_code", "created_at", "expires_at", "approved_at", "nonce") FROM stdin;
\.

--
-- Data for Name: oauth_client_states; Type: TABLE DATA; Schema: auth; Owner: supabase_auth_admin
--

COPY "auth"."oauth_client_states" ("id", "provider_type", "code_verifier", "created_at") FROM stdin;
\.

--
-- Data for Name: oauth_consents; Type: TABLE DATA; Schema: auth; Owner: supabase_auth_admin
--

COPY "auth"."oauth_consents" ("id", "user_id", "client_id", "scopes", "granted_at", "revoked_at") FROM stdin;
\.

--
-- Data for Name: scim_tokens; Type: TABLE DATA; Schema: auth; Owner: supabase_auth_admin
--

COPY "auth"."scim_tokens" ("id", "sso_provider_id", "token_hash", "prefix", "created_at", "expires_at", "revoked_at", "last_used_at") FROM stdin;
\.

--
-- Data for Name: scim_users; Type: TABLE DATA; Schema: auth; Owner: supabase_auth_admin
--

COPY "auth"."scim_users" ("id", "sso_provider_id", "user_id", "resource", "created_at", "updated_at", "deleted_at") FROM stdin;
\.

--
-- Data for Name: webauthn_challenges; Type: TABLE DATA; Schema: auth; Owner: supabase_auth_admin
--

COPY "auth"."webauthn_challenges" ("id", "user_id", "challenge_type", "session_data", "created_at", "expires_at") FROM stdin;
\.

--
-- Data for Name: webauthn_credentials; Type: TABLE DATA; Schema: auth; Owner: supabase_auth_admin
--

COPY "auth"."webauthn_credentials" ("id", "user_id", "credential_id", "public_key", "attestation_type", "aaguid", "sign_count", "transports", "backup_eligible", "backed_up", "friendly_name", "created_at", "updated_at", "last_used_at") FROM stdin;
\.

--
-- Name: refresh_tokens_id_seq; Type: SEQUENCE SET; Schema: auth; Owner: supabase_auth_admin
--

SELECT pg_catalog.setval('"auth"."refresh_tokens_id_seq"', 530, true);

--
-- PostgreSQL database dump complete
--

-- \unrestrict r5AgHlgabGRZCwUMFi0zsiTngngY8r3YRtqdDcZ6eN2q3Ok3U312yUx3b113BbF

RESET ALL;

offline/data/auth-users.sql

```
SET session_replication_role = replica;

COPY "auth"."users" ("instance_id", "id", "aud", "role", "email",
"encrypted_password", "email_confirmed_at", "invited_at", "confirmation_token",
"confirmation_sent_at", "recovery_token", "recovery_sent_at",
"email_change_token_new", "email_change", "email_change_sent_at",
"last_sign_in_at", "raw_app_meta_data", "raw_user_meta_data",
"is_super_admin", "created_at", "updated_at", "phone", "phone_confirmed_at",
"phone_change", "phone_change_token", "phone_change_sent_at",
"email_change_token_current", "email_change_confirm_status", "banned_until",
"reauthentication_token", "reauthentication_sent_at", "is_sso_user", "deleted_at",
"is_anonymous") FROM stdin;
00000000-0000-0000-0000-000000000000
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74    authenticated    authenticated
eapolona0803@gmail.com
$2a$10$JODMPeJHysV3BDx8j1mKm.WyG3Zke0E3VZZIWIumYp0uGshb.wMzK
   2026-09-25 11:16:13.515718+00    \N    \N
9daaa4b8a054fcba2a7be624602a36d01977707a01f2ff6651a3a9ef    2026-10-21
12:37:46.569171+00    \N    2026-10-21 12:36:06.210558+00    {"provider":
"email", "providers": ["email"]}    {"full_name": "JoeMar tats", "email_verified":
true}    \N    2026-09-25 11:16:13.493649+00    2026-10-21 12:37:48.356871+00
  \N    \N    \N    0    \N    \N    f    \N    f
\.

COPY "auth"."identities" ("provider_id", "user_id", "identity_data", "provider",
"last_sign_in_at", "created_at", "updated_at", "id") FROM stdin;
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74    {"sub":
"7a9e497f-8c3f-44a7-ab4e-c9247c50eb74", "email":
"eapolona0803@gmail.com", "email_verified": false, "phone_verified": false}
email    2026-09-25 11:16:13.50782+00    2026-09-25 11:16:13.507885+00
2026-09-25 11:16:13.507885+00    be414415-e886-4cd4-898f-588e6ff80551
\.

SET session_replication_role = origin;
```

offline/data/public-data.sql

```
SET session_replication_role = replica;

--
-- PostgreSQL database dump
--

-- \restrict dDZn8GvReDx6ALyowb48imcxEX018gr8lhZt3xmclAMCQ6Np0JbImZNa3io5WlD

-- Dumped from database version 17.6
-- Dumped by pg_dump version 17.6

SET statement_timeout = 0;
SET lock_timeout = 0;
SET idle_in_transaction_session_timeout = 0;
SET transaction_timeout = 0;
SET client_encoding = 'UTF8';
SET standard_conforming_strings = on;
SELECT pg_catalog.set_config('search_path', '', false);
SET check_function_bodies = false;
SET xmloption = content;
SET client_min_messages = warning;
SET row_security = off;

--
-- Data for Name: admin_activity_logs; Type: TABLE DATA; Schema: public; Owner: postgres
--

COPY "public"."admin_activity_logs" ("id", "actor_id", "actor_email", "action", "resource_type", "target_label", "summary", "changed_fields", "activity_date", "activity_time", "created_at", "record_id", "ip_address") FROM stdin;
ef55ae07-b883-4a64-ad96-09a0a1473c7c	7a9e497f-8c3f-44a7-ab4e-c9247c50eb74	eapolona0803@gmail.com	signed in	Authentication	Admin console	Administrator signed in to the admin console.	{}	2026-09-25	19:33:03.995899	2026-09-25 11:33:03.995899+00	\N	143.44.164.241
0b941d8e-8596-496b-a98d-8a93b471bea6	7a9e497f-8c3f-44a7-ab4e-c9247c50eb74	eapolona0803@gmail.com	signed in	Authentication	Admin console	Administrator signed in to the admin console.	{}	2026-09-27	09:16:03.167987	2026-09-27 01:16:03.167987+00	\N	143.44.164.184
3f870ed6-0e25-4588-86c9-dade15cc84f6
```

7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-27 19:51:25.934022 2026-09-27 11:51:25.934022+00 \N 143.44.164.184
5be720e6-e8b8-4cc9-94ef-f69a23f3dac0
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-28 16:41:23.98446 2026-09-28 08:41:23.98446+00 \N 143.44.164.66
7650088c-9a7c-41b6-8178-5efe47912de4
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-29 09:30:04.081754 2026-09-29 01:30:04.081754+00 \N 143.44.164.123
caf4fadd-ac80-47ae-8e9c-8cd7cae3aafd
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com exported Reports Booking Report Exported 13 bookings as PDF. {report_type,date_range,export_format} 2026-09-29 09:31:16.445212 2026-09-29 01:31:16.445212+00 \N 143.44.164.123
70f6594c-47fd-4345-ae16-6f2127fd0528
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Price History c9e38be2-2f3d-4ab8-976b-efe352f71406 Price History created: c9e38be2-2f3d-4ab8-976b-efe352f71406 {} 2026-09-29 18:52:29.926217 2026-09-29 10:52:29.926217+00 c9e38be2-2f3d-4ab8-976b-efe352f71406 143.44.164.123
6af0c091-920c-4d42-9341-7ebc4b9d80e7
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-30 08:51:59.232724 2026-09-30 00:51:59.232724+00 \N 49.146.140.233
2706cb0c-5dfe-4c1f-91f1-ddfa3a506ec1
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-07 08:22:14.834585 2026-10-07 00:22:14.834585+00 \N 143.44.164.41
54d7e425-a33b-4c1a-aaea-342bce961cd3
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-10-08 23:39:13.296885 2026-10-08 15:39:13.296885+00 \N 143.44.164.41
6184c500-b7f5-472c-ac47-e3c8d35fd877
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-11 14:27:02.368404 2026-10-11 06:27:02.368404+00 \N 143.44.164.41

2866040c-f406-4567-8ce4-36b47b9481ac
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Services Sprocket Replacement Services updated: Sprocket
Replacement {large_bike_duration_minutes,large_bike_price} 2026-10-14
16:16:21.267017 2026-10-14 08:16:21.267017+00
f9cf4825-20c1-409b-8842-8411540ad30b 143.44.164.41
e1ed524a-16a4-4398-823c-07632fbd29ec
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Services Chain and Sprocket Replacement Services updated:
Chain and Sprocket Replacement
{large_bike_duration_minutes,large_bike_price} 2026-10-14 16:16:33.9627
2026-10-14 08:16:33.9627+00 4ee919a5-884a-4cf4-80af-e6cdfbb97778
143.44.164.41
60fc26ce-f8f7-4e8c-9a41-5f5ae329c18d
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Services Tire Balancing Services updated: Tire Balancing
{large_bike_duration_minutes,large_bike_price} 2026-10-14 16:17:15.178074
2026-10-14 08:17:15.178074+00 7a43f3c4-a01f-476c-8f16-5e6a74fb6833
143.44.164.41
39e79c77-8369-4807-95ac-813a88080e8f
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Services Wheel Alignment Services updated: Wheel Alignment
{large_bike_duration_minutes,large_bike_price} 2026-10-14 16:17:28.344788
2026-10-14 08:17:28.344788+00 f28be846-7db7-422a-b960-1d36cce18e7b
143.44.164.41
7692813e-7fe2-41b0-a5ee-dd69f0c9887c
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Services Wheel Bearing Replacement Services updated: Wheel
Bearing Replacement {large_bike_duration_minutes,large_bike_price}
2026-10-14 16:17:39.307356 2026-10-14 08:17:39.307356+00
9381d3bc-dadb-4e65-9b83-97539b9df723 143.44.164.41
fafb7b99-2954-4d9c-9857-73702f1e35be
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Services Suspension Tuning Services updated: Suspension
Tuning {large_bike_duration_minutes,large_bike_price} 2026-10-14
16:18:29.997157 2026-10-14 08:18:29.997157+00
42490782-6cf8-4db4-86e6-757d96ec9dec 143.44.164.41
5e680e0d-b44b-44a8-8f48-19e8e4fac4ab
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Services Battery Testing Services updated: Battery Testing
{large_bike_duration_minutes,large_bike_price} 2026-10-14 16:19:30.210295
2026-10-14 08:19:30.210295+00 8c384099-4f80-4b61-ab6d-ffea363a6038
143.44.164.41
9d4d48e2-4742-47c9-b032-4df93e30165d
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com

updated Services Electrical Wiring Repair Services updated: Electrical Wiring Repair {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:20:15.008185 2026-10-14 08:20:15.008185+00 cb46dda3-e494-47d3-8f41-ce98287ff9b5 143.44.164.41 bc89c96d-a063-4252-b69d-fea3ddc4d2b9 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Services Spark Plug Replacement Services updated: Spark Plug Replacement {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:21:00.037601 2026-10-14 08:21:00.037601+00 437617a0-ee24-4289-9209-aeaafbaf192c 143.44.164.41 96b92b94-37c5-44c8-a326-ef285dc44025 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Services Carburetor Tuning Services updated: Carburetor Tuning {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:21:48.329003 2026-10-14 08:21:48.329003+00 26b9d1dc-3f41-4466-91af-c53b2c230c6b 143.44.164.41 01cea2db-ea3b-410c-b306-3c6cf27a8541 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Services Throttle Body Cleaning Services updated: Throttle Body Cleaning {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:22:09.879837 2026-10-14 08:22:09.879837+00 3d1f19de-deeb-4b05-b7ae-fc6dc36d14b7 143.44.164.41 ae94389c-c1ce-4103-83a9-79d451cda0f1 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Services Engine Diagnostic Services updated: Engine Diagnostic {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:22:47.490647 2026-10-14 08:22:47.490647+00 625eeaa2-aeb5-443d-8f4e-c612ffcd13b4 143.44.164.41 566fb0ca-cd8d-4e65-b4d4-b46cef7145d0 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Services Starter System Diagnosis & Repair Services updated: Starter System Diagnosis & Repair {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:23:26.099539 2026-10-14 08:23:26.099539+00 4e82dabf-aa10-4846-ae25-1c208f9b68bc 143.44.164.41 23adae43-9dcd-4ff8-ae81-79c0d1898a58 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
archived Services Charging System Testing Services archived: Charging System Testing {archived_at,is_archived} 2026-10-14 16:24:21.963967 2026-10-14 08:24:21.963967+00 ec2b2a81-b3e9-404e-8e1d-d161f25b8a43 143.44.164.41 7af3c699-f36b-4c6e-a8d2-7ce4da9fa6c3 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Services Horn Replacement Services updated: Horn Replacement {large_bike_duration_minutes,large_bike_price} 2026-10-14

16:24:35.10144 2026-10-14 08:24:35.10144+00
cac8ad67-f7d0-438d-baf4-c223d3abd732 143.44.164.41
c39cc094-878e-4996-a377-10515c490b4b
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Services Water Pump Inspection/Repair Services updated: Water
Pump Inspection/Repair {large_bike_duration_minutes,large_bike_price}
2026-10-14 16:25:30.90425 2026-10-14 08:25:30.90425+00
9ab95115-47ef-4011-846a-4af834e115da 143.44.164.41
53004937-6bef-4b82-af6d-c1eae9ab802b
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Services Brake Pad Replacement Services updated: Brake Pad
Replacement {large_bike_duration_minutes,large_bike_price} 2026-10-14
16:26:18.018453 2026-10-14 08:26:18.018453+00
9fa4029c-73e5-4fff-93fc-6f26d8d40785 143.44.164.41
752aa683-3b2a-44b7-8311-7c8cbb5a4ecd
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Services Magneto Cleaning Services updated: Magneto Cleaning
{large_bike_duration_minutes,large_bike_price} 2026-10-14
16:26:29.969134 2026-10-14 08:26:29.969134+00
824a246b-3956-4983-b344-39c3d18001e4 143.44.164.41
305be957-29db-41db-8d80-62e0c3a79eec
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
out Authentication Admin console Administrator was automatically signed
out after 2 minutes of inactivity. {} 2026-09-25 19:35:17.803075
2026-09-25 11:35:17.803075+00 \N 143.44.164.241
eca7e1cb-e96b-462c-99b2-11c475c88664
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
out Authentication Admin console Administrator was automatically signed
out after 2 minutes of inactivity. {} 2026-09-27 09:18:53.160584
2026-09-27 01:18:53.160584+00 \N 143.44.164.184
048c26e2-71d5-4890-860c-823f19d963d3
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-09-28 08:22:27.944601 2026-09-28
00:22:27.944601+00 \N 143.44.164.66
f5233174-7dea-4939-a57a-526ec183c691
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
out Authentication Admin console Administrator signed out before visiting
the public site. {} 2026-09-28 08:22:57.933375 2026-09-28
00:22:57.933375+00 \N 143.44.164.66
7dfdb017-e4f3-40d7-8472-ee829725016b
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-09-28 16:49:50.946724 2026-09-28
08:49:50.946724+00 \N 143.44.164.66

8292da24-6042-4723-8b76-9ce82c087715
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-VVUZEP Appointments status changed: FRM-VVUZEP {status} 2026-09-29 09:30:28.843472 2026-09-29 01:30:28.843472+00 3e260213-1399-48e6-9b69-1e74bf4e0000 143.44.164.123
8b3501e9-0d66-4055-82bb-f841993854e8
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Notifications 504f93e8-929f-4183-8ec6-980a770dc82a Notifications updated: 504f93e8-929f-4183-8ec6-980a770dc82a {is_read} 2026-09-29 09:30:31.278559 2026-09-29 01:30:31.278559+00 504f93e8-929f-4183-8ec6-980a770dc82a 143.44.164.123
4b4ba927-daa6-43e9-88d3-305a8a4499db
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator signed out of the admin console. {} 2026-09-29 09:31:51.446745 2026-09-29 01:31:51.446745+00 \N 143.44.164.123
31ef2040-64c9-4061-893f-da375584f4eb
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Products Braided Brake Hose Products updated: Braided Brake Hose {price} 2026-09-29 18:54:24.430402 2026-09-29 10:54:24.430402+00 c973b83b-ee14-4254-a320-420f75ffb39a 143.44.164.123
c6f3fb76-19f1-4109-95d8-3c42052937c5
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-30 12:22:46.157775 2026-09-30 04:22:46.157775+00 \N 180.193.217.82
a76d0d3c-3038-46bd-9553-88ca2d94f865
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-ULM8GG Appointments status changed: FRM-ULM8GG {service_ended_at,status} 2026-10-07 08:22:28.433925 2026-10-07 00:22:28.433925+00 de1e21dc-53e4-4362-8898-3bd1da2d67fc 143.44.164.41
d2a4c891-3500-4c9b-9c2c-bee638ed7d03
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Notifications 2382a442-21a6-4c8f-b525-d5825aa8f2fe Notifications updated: 2382a442-21a6-4c8f-b525-d5825aa8f2fe {is_read} 2026-10-07 08:22:45.072298 2026-10-07 00:22:45.072298+00 2382a442-21a6-4c8f-b525-d5825aa8f2fe 143.44.164.41
b35cffa7-9312-4bdb-a377-9670f84665a0
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-08 23:41:04.057135 2026-10-08 15:41:04.057135+00 \N 143.44.164.41

db440d9f-af9b-4fe9-b652-3a82aee225b4
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-11 15:08:40.871519 2026-10-11 07:08:40.871519+00 \N 143.44.164.41
25e31c9f-67c6-41ec-ab8f-8945e0c3601d
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Crankshaft Bearing Replacement Services updated: Crankshaft Bearing Replacement {large_bike_price} 2026-10-14 16:27:40.16708 2026-10-14 08:27:40.16708+00
a2e6ef24-2f88-4f18-92b2-180be413a5a7 143.44.164.41
8b0d7134-b8bc-4f3a-aa61-22cb7dfb2a9c
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services CVT Roller / Fly Weight Replacement Services updated: CVT Roller / Fly Weight Replacement {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:28:11.358318 2026-10-14 08:28:11.358318+00 624b8f10-7059-43d0-8557-77b03600cd0f 143.44.164.41
4fe1c210-6902-4445-bb94-ba1d77bee7f9
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Muffler/Exhaust Replacement Services updated: Muffler/Exhaust Replacement {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:28:51.421857 2026-10-14 08:28:51.421857+00
1fd369b6-bdac-4390-b3ba-f9255eeaf3eb 143.44.164.41
541519e2-84d9-4969-910c-a94297f17c31
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Racing Carburetor Installation Services updated: Racing Carburetor Installation {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:29:05.043733 2026-10-14 08:29:05.043733+00
731aac9e-7dd7-496b-b6ed-f8efc1837c36 143.44.164.41
d08c1cd9-5399-469b-87ed-f36d84b6519f
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Frame/Crash Slider Installation Services updated: Frame/Crash Slider Installation {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:29:16.596957 2026-10-14 08:29:16.596957+00
dd863611-9344-4ed4-a516-69f1f6474821 143.44.164.41
5d1e9311-2211-4bad-b60f-7ba0bd861f77
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-16 21:11:22.39827 2026-10-16 13:11:22.39827+00 \N 143.44.164.41
5270b6fc-0ab5-4134-bb83-a1937815347e
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed Reports Financial Report Opened the print dialog for 7 completed transactions. {report_type,date_range,print_format} 2026-10-21

20:36:19.807278 2026-10-21 12:36:19.807278+00 \N 143.44.164.41
291fa510-f7af-4c27-972a-c5e45502e1b0
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-25 20:18:12.297918 2026-09-25 12:18:12.297918+00 \N 143.44.164.241
9f3e1a58-650c-4c65-ae3a-48819fb6ef03
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-27 09:32:50.044805 2026-09-27 01:32:50.044805+00 \N 143.44.164.184
2e494f6a-d58b-4431-a9d3-bdc88c968337
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-28 09:05:17.409536 2026-09-28 01:05:17.409536+00 \N 143.44.164.66
8d84adb0-bae2-4c53-bb38-bc5df9946660
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Crew Schedules 1fce2b2e-177b-4044-86dd-b376706814fc Crew Schedules created: 1fce2b2e-177b-4044-86dd-b376706814fc {} 2026-09-28 16:50:38.655222 2026-09-28 08:50:38.655222+00
1fce2b2e-177b-4044-86dd-b376706814fc 143.44.164.66
586c27fc-f524-479f-bee7-6fbca4706203
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-29 14:09:09.945448 2026-09-29 06:09:09.945448+00 \N 49.146.140.233
de0e1c4c-eb6a-4bdb-9377-91fdb0545ba5
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Price History 94cb692d-ca14-431c-bd00-537222d5550f Price History created: 94cb692d-ca14-431c-bd00-537222d5550f {} 2026-09-29 18:54:24.903124 2026-09-29 10:54:24.903124+00
94cb692d-ca14-431c-bd00-537222d5550f 143.44.164.123
0c36dd03-5520-4f3c-afc7-9dc3d31b801e
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-MPMHT9 Appointments status changed: FRM-MPMHT9 {service_ended_at,status} 2026-09-30 12:23:13.520566 2026-09-30 04:23:13.520566+00 6014afba-21ae-402c-a687-150730e547ea 180.193.217.82
4f68b99c-5460-4e8d-8774-30f87712c02d
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Notifications 011fd0a2-b76f-4f9e-81cf-397fbbf09cd4 Notifications updated: 011fd0a2-b76f-4f9e-81cf-397fbbf09cd4 {is_read} 2026-09-30 12:23:19.284758 2026-09-30 04:23:19.284758+00
011fd0a2-b76f-4f9e-81cf-397fbbf09cd4 180.193.217.82

d7ffdcf3-1995-4d2e-99c6-092abf7f2713
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-HWAVDJ Appointments status changed: FRM-HWAVDJ {service_ended_at,status} 2026-10-07 08:22:30.886674 2026-10-07 00:22:30.886674+00 617073f7-8cfa-4f3f-8352-aff1a76da780 143.44.164.41
e821fdaa-5750-455e-a50b-74d3aa4cccd1
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-10-08 23:43:29.316539 2026-10-08 15:43:29.316539+00 \N 143.44.164.41
26163ebd-aba1-4e4c-a478-6c24844f766e
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed Reports Financial Report (Summary) Opened the print dialog for 3 completed transactions. {report_type,date_range,print_format} 2026-10-11 15:08:50.176804 2026-10-11 07:08:50.176804+00 \N 143.44.164.41
e995bebf-9fa8-410c-a6a1-28f122e99177
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Price History 8b44ed97-7f25-46b8-b430-7dd54119c525 Price History created: 8b44ed97-7f25-46b8-b430-7dd54119c525 {} 2026-10-14 16:27:40.573167 2026-10-14 08:27:40.573167+00 8b44ed97-7f25-46b8-b430-7dd54119c525 143.44.164.41
6facd06e-9ce8-4f68-bddf-22361a667074
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Handlebar Switch Replacement Services updated: Handlebar Switch Replacement {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:28:27.072278 2026-10-14 08:28:27.072278+00 c18718c3-3aa9-40f6-8907-3a7922a07922 143.44.164.41
c8e483f9-cc85-49dc-a4bd-4464aae39c3c
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Throttle Cable Replacement/Adjustment Services updated: Throttle Cable Replacement/Adjustment {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:28:37.773158 2026-10-14 08:28:37.773158+00 289d392a-52a6-4e13-995f-2303faf48fe2 143.44.164.41
588967f1-2905-442a-9464-3c1925351cec
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Fan Spinner Installation Services updated: Fan Spinner Installation {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:29:28.59411 2026-10-14 08:29:28.59411+00 c4c93b1e-7f81-4be3-a748-cc677ff809bc 143.44.164.41
05323376-c12d-4cc5-94ef-f7a056f085cd
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin

console. {} 2026-10-18 16:28:57.168172 2026-10-18 08:28:57.168172+00 \N 143.44.164.41 58bb6ee0-dfdb-44cb-8884-22a9c181de8f 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator signed out before visiting the public site. {} 2026-10-21 20:37:12.224017 2026-10-21 12:37:12.224017+00 \N 143.44.164.41 0561af84-b1ad-4927-86c5-635eb6d789f1 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-E6Y7KF Appointments status changed: FRM-E6Y7KF {service_ended_at,status} 2026-09-25 20:18:27.916329 2026-09-25 12:18:27.916329+00 147297a6-c092-4337-ab4e-2c86b7848da9 143.44.164.241 657f7aa4-1862-4d66-819a-673e64a3a8dc 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Crew Members Jonde Crew Members updated: Jonde {name,role} 2026-09-27 09:40:35.546051 2026-09-27 01:40:35.546051+00 f3c5770f-e485-4efc-83f6-892422d8592e 143.44.164.184 15e614f4-2c3c-4a80-9a24-e736519809d0 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-28 09:07:50.543861 2026-09-28 01:07:50.543861+00 \N 143.44.164.66 02668f59-f5b5-4a45-95a4-075ad927c671 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Crew Schedules 49ead0ec-b812-40f8-947c-d9dab9c6c2a5 Crew Schedules created: 49ead0ec-b812-40f8-947c-d9dab9c6c2a5 {} 2026-09-28 16:50:51.47699 2026-09-28 08:50:51.47699+00 49ead0ec-b812-40f8-947c-d9dab9c6c2a5 143.44.164.66 7c03964a-6ed5-4a2e-a7ad-5b933a712e94 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-746ETT Appointments status changed: FRM-746ETT {service_started_at,status} 2026-09-29 14:09:23.975551 2026-09-29 06:09:23.975551+00 6957127d-9bdd-4173-a856-2f08819ce87f 49.146.140.233 54116a27-2e65-42de-894b-f247f7d11ce3 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Engine Overhaul Services updated: Engine Overhaul {price} 2026-09-29 18:57:28.115338 2026-09-29 10:57:28.115338+00 f8feeecc-86e2-4312-8798-ea5c144f1ad4 143.44.164.123 eaca2609-7d63-45e4-acd1-d60adce66872 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-AQY69P Appointments status changed: FRM-AQY69P {status} 2026-09-30 12:23:17.055034 2026-09-30 04:23:17.055034+00 eda6b8a7-6dc1-4584-8ad5-e918438c0a05

180.193.217.82
9a7c8518-1367-4289-b804-b3cc70b340d6
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-07 19:40:03.338942 2026-10-07 11:40:03.338942+00 \N 143.44.164.41
850c930f-711c-4081-ae20-7ecc76fcfda9
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-09 00:07:09.141715 2026-10-08 16:07:09.141715+00 \N 143.44.164.41
8afc2c89-15fd-4332-8c06-7f238993fb38
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-11 15:37:03.360991 2026-10-11 07:37:03.360991+00 \N 143.44.164.41
a57e3992-0b6a-4a88-95d2-eec79bf9eaf4
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-10-14 16:31:34.740922 2026-10-14 08:31:34.740922+00 \N 143.44.164.41
667a6179-1867-427c-a230-63aa536c0442
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed Activity Log Admin Activity Log Opened the print dialog for 1077 activity records. {date_range,print_format} 2026-10-18 16:45:24.873681 2026-10-18 08:45:24.873681+00 \N 143.44.164.41
c9b9b4b5-0400-4d4a-91cc-7ac235845203 \N reschedule-e2e-1789028922977@example.invalid status changed Appointments E2EC922569 Appointments status changed: E2EC922569 {appointment_date,pending_reschedule_date,pending_reschedule_reason,pending_reschedule_request_id,pending_reschedule_start_time,start_time,status} 2026-09-10 16:28:44.30708 2026-09-10 08:28:44.30708+00
4f926d16-eab1-4676-979b-472b3e55d919 49.146.140.233
fc2cc6a8-8b56-4d59-8c49-dc2dcb622100 \N reschedule-e2e-1789028922977@example.invalid updated Appointments E2ER922569 Appointments updated: E2ER922569 {pending_reschedule_date,pending_reschedule_reason,pending_reschedule_request_id,pending_reschedule_start_time} 2026-09-10 16:28:44.979707 2026-09-10 08:28:44.979707+00 474e2e8e-a4c0-4a99-abcc-bf2785cf9516
49.146.140.233
8f7e9bdf-0c6c-479c-8f97-3974bb3ecd6b
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-26 06:58:13.910665 2026-09-25 22:58:13.910665+00 \N 143.44.164.123

61359943-7ab1-4d65-a2e3-39751d708bf5
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Crew Members Junard Crew Members updated: Junard
{name,role} 2026-09-27 09:40:54.373371 2026-09-27 01:40:54.373371+00
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 143.44.164.184
320b273f-92a6-41f6-84cd-03d5f1df8b6b
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-09-28 12:31:49.599872 2026-09-28
04:31:49.599872+00 \N 49.146.140.233
ed1e91df-abdd-4eee-94f6-49d2ab47bef6
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
out Authentication Admin console Administrator signed out before visiting
the public site. {} 2026-09-28 16:53:53.444091 2026-09-28
08:53:53.444091+00 \N 143.44.164.66
7ed6509b-223e-4fff-ba73-4d567988912c
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Notifications 6ab60877-218d-46a5-b23f-c0e414a23ab6
Notifications updated: 6ab60877-218d-46a5-b23f-c0e414a23ab6 {is_read}
2026-09-29 14:09:39.056647 2026-09-29 06:09:39.056647+00
6ab60877-218d-46a5-b23f-c0e414a23ab6 49.146.140.233
00200990-7d90-42bd-988f-dd2fc506a6a6
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created
Price History 0ce9a20a-6e2a-4407-b0fe-98d10d36677a Price History
created: 0ce9a20a-6e2a-4407-b0fe-98d10d36677a {} 2026-09-29
18:57:28.926507 2026-09-29 10:57:28.926507+00
0ce9a20a-6e2a-4407-b0fe-98d10d36677a 143.44.164.123
8007badc-9363-4965-8731-d6fd857d0b81
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
out Authentication Admin console Administrator was automatically signed
out after 2 minutes of inactivity. {} 2026-09-30 12:27:48.276382
2026-09-30 04:27:48.276382+00 \N 180.193.217.82
f9c19c00-1c49-436e-a088-f76e0f4c48b0
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-10-07 20:17:14.614449 2026-10-07
12:17:14.614449+00 \N 143.44.164.41
6b04c051-0b55-41d8-b83b-45b0ce932335
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed
Reports Financial Report (Summary) Opened the print dialog for 4
completed transactions. {report_type,date_range,print_format} 2026-10-09
00:11:01.94424 2026-10-08 16:11:01.94424+00 \N 143.44.164.41
28af7dc8-5645-4449-b33d-17cb9728e5c5
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed
Reports Financial Report (Summary) Opened the print dialog for 3

completed transactions. {report_type,date_range,print_format} 2026-10-11 15:37:20.232834 2026-10-11 07:37:20.232834+00 \N 143.44.164.41 ee2ba5c3-f1ac-44a5-b1b0-83ec5758c978
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-14 17:29:37.131934 2026-10-14 09:29:37.131934+00 \N 143.44.164.41 6b594c63-988c-48cf-83cc-246ea5508cbf
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed Activity Log Admin Activity Log Opened the print dialog for 203 activity records. {date_range,print_format} 2026-10-18 16:46:08.449319 2026-10-18 08:46:08.449319+00 \N 143.44.164.41 1ecc876e-8f0e-4076-ba8b-20e57791a107
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator signed out of the admin console. {} 2026-09-26 07:00:30.860125 2026-09-25 23:00:30.860125+00 \N 143.44.164.123 ad15bed2-b301-44b9-96bb-d37c44dcb921
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Crew Members Joshua Crew Members updated: Joshua {name} 2026-09-27 09:41:10.496963 2026-09-27 01:41:10.496963+00 03bb877e-57f8-42dc-9c0e-3fcc82b1a207 143.44.164.184 e2c54778-6463-4d8e-9ec8-a058722bf3ac
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator signed out before visiting the public site. {} 2026-09-28 12:32:11.197826 2026-09-28 04:32:11.197826+00 \N 49.146.140.233 82351801-a850-4616-bbb9-b539c40d3d84
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-28 16:55:39.959813 2026-09-28 08:55:39.959813+00 \N 143.44.164.66 1d0d729b-99c6-4efc-9856-337faf38443d
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-P35N26 Appointments status changed: FRM-P35N26 {assigned_crew_id,status} 2026-09-28 16:56:19.100275 2026-09-28 08:56:19.100275+00 4578f9c5-5566-4966-9e56-5f34184be688 143.44.164.66 18ad1865-ecd4-442e-9bc3-25d1b5fd4757
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Notifications 6d187feb-f177-40b2-8def-48a399d04148 Notifications updated: 6d187feb-f177-40b2-8def-48a399d04148 {is_read} 2026-09-28 16:56:21.838513 2026-09-28 08:56:21.838513+00 6d187feb-f177-40b2-8def-48a399d04148 143.44.164.66 188ca0ea-f737-420f-bca9-2fb550ec6848

7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com archived Appointments FRM-F98X7B Appointments archived: FRM-F98X7B {archived_at,is_archived} 2026-09-28 16:58:26.921765 2026-09-28 08:58:26.921765+00 64fe7d0c-3604-4dc4-a072-a3344c02e337 143.44.164.66
47afaec3-555b-417f-8853-53b74c54bbfe
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-P5ZX4N Appointments status changed: FRM-P5ZX4N {status} 2026-09-29 14:12:18.162249 2026-09-29 06:12:18.162249+00 3eac3b68-b97b-46f0-a8eb-ad913f8bc8c6 49.146.140.233
9208a534-6d11-41d1-b2ec-7ec0602995f4
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com deleted Appointments FRM-P35N26 Appointments deleted: FRM-P35N26 {} 2026-09-29 19:00:07.01081 2026-09-29 11:00:07.01081+00 4578f9c5-5566-4966-9e56-5f34184be688 143.44.164.123
16d8f4b4-0a39-4c87-8bbf-b7b1b628c803
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com deleted Appointment Services Brake Service Appointment Services deleted: Brake Service {} 2026-09-29 19:00:07.01081 2026-09-29 11:00:07.01081+00 25265555-80de-4a78-98b7-e7c5dab2aceb 143.44.164.123
e8934c4b-5637-4097-993d-6c9d6193c155
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com deleted Notifications 6d187feb-f177-40b2-8def-48a399d04148 Notifications deleted: 6d187feb-f177-40b2-8def-48a399d04148 {} 2026-09-29 19:00:07.01081 2026-09-29 11:00:07.01081+00 6d187feb-f177-40b2-8def-48a399d04148 143.44.164.123
c599787c-a65d-415f-a83e-ceeaa33e8e41
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-30 12:52:07.360614 2026-09-30 04:52:07.360614+00 \N 180.193.217.82
2f3675ad-df85-4743-9367-1ab71f89618c
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-10-07 20:21:53.603003 2026-10-07 12:21:53.603003+00 \N 143.44.164.41
0104dc0e-7598-4f5b-bb21-883a3c054ed7
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-09 00:30:54.627538 2026-10-08 16:30:54.627538+00 \N 143.44.164.41
e7e88a15-d333-4743-b15d-02927b6cce08
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin

console. {} 2026-10-11 15:40:39.266252 2026-10-11 07:40:39.266252+00 \N 143.44.164.41 67621a08-ec61-44eb-bdc9-adcafd250606 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed Reports Financial Report (Summary) Opened the print dialog for 3 completed transactions. {report_type,date_range,print_format} 2026-10-11 15:40:45.462023 2026-10-11 07:40:45.462023+00 \N 143.44.164.41 613ef281-a874-44df-b7b6-35dd8d9d072a 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator signed out before visiting the public site. {} 2026-10-14 17:29:51.232805 2026-10-14 09:29:51.232805+00 \N 143.44.164.41 2314c6ff-8204-4862-b0ea-cd348aa5b488 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed Reports Financial Report (Summary) Opened the print dialog for 5 completed transactions. {report_type,date_range,print_format} 2026-10-18 16:46:36.321053 2026-10-18 08:46:36.321053+00 \N 143.44.164.41 09ba2ee0-f62d-4e07-904b-b91f39e0e6e7 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-26 07:07:50.582566 2026-09-25 23:07:50.582566+00 \N 143.44.164.123 3d43e9c3-2d96-46e0-ad9f-071de70a4365 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-NNQNP8 Appointments status changed: FRM-NNQNP8 {status} 2026-09-26 07:07:57.445994 2026-09-25 23:07:57.445994+00 2a2de84e-e1d7-42e7-a36d-45905d69df12 143.44.164.123 a9d5661d-d472-4f23-bbb3-a525380ee9cf 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Crew Members Liezel Crew Members updated: Liezel {name,role} 2026-09-27 09:41:27.613539 2026-09-27 01:41:27.613539+00 1700c9e8-37e5-42c2-98e8-48854585ba7d 143.44.164.184 de51a29a-cf2e-4d7d-8496-3468eca54dbc 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-28 12:44:48.584754 2026-09-28 04:44:48.584754+00 \N 49.146.140.233 6942560e-cfc4-4a65-8deb-0e78fc2413a5 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-28 19:37:58.649994 2026-09-28 11:37:58.649994+00 \N 143.44.164.66 aa86695d-1e5a-47db-99d4-5c2d004e95d3 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed

out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-29 14:14:46.973066 2026-09-29 06:14:46.973066+00 \N 49.146.140.233 e4a76b28-519a-4fd8-94ea-8510451a4701 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-29 14:15:09.583732 2026-09-29 06:15:09.583732+00 \N 49.146.140.233 57350400-ad33-4a84-b2fd-78b74e84bd06 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com deleted Appointments FRM-VVUZEP Appointments deleted: FRM-VVUZEP {} 2026-09-29 19:00:34.652612 2026-09-29 11:00:34.652612+00 3e260213-1399-48e6-9b69-1e74bf4e0000 143.44.164.123 5fe0cd85-cd1d-4782-9298-47ab57b97acf 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com deleted Appointment Services Change Oil Appointment Services deleted: Change Oil {} 2026-09-29 19:00:34.652612 2026-09-29 11:00:34.652612+00 88d3ad91-ca34-4370-9948-486a19e4ff0e 143.44.164.123 78da2211-1ec7-436f-ac62-98e89360ef51 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com deleted Appointment Services Brake Service Appointment Services deleted: Brake Service {} 2026-09-29 19:00:34.652612 2026-09-29 11:00:34.652612+00 9d5a7797-4d49-4592-a24b-215455eb9981 143.44.164.123 5c79893d-c144-4d63-ad84-ab7d8039d2be 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com deleted Appointment Services Clutch Adjustment Appointment Services deleted: Clutch Adjustment {} 2026-09-29 19:00:34.652612 2026-09-29 11:00:34.652612+00 608892ba-445c-4b63-aa20-aeabf420521f 143.44.164.123 e507a2da-20ee-4d17-9624-095aa46db84d 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com deleted Appointment Services Chain Cleaning Appointment Services deleted: Chain Cleaning {} 2026-09-29 19:00:34.652612 2026-09-29 11:00:34.652612+00 0e98af61-be12-481e-aeea-5c7fb952601f 143.44.164.123 565b06d4-7e21-45b2-9209-e90415d732ce 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com deleted Appointment Services Chain Lubrication Appointment Services deleted: Chain Lubrication {} 2026-09-29 19:00:34.652612 2026-09-29 11:00:34.652612+00 f24c369a-f11e-489f-b002-6ac9061ad07a 143.44.164.123 4a54c885-492d-4065-9772-c57df430e19e 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com deleted Appointment Services Chain Adjustment Appointment Services deleted: Chain Adjustment {} 2026-09-29 19:00:34.652612 2026-09-29

11:00:34.652612+00 0ab8d18d-3098-47cb-94ce-476410ba6cea
143.44.164.123
ed50a671-a8a4-4f86-95bd-754159a393a1
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com deleted
Appointment Services Wheel Bearing Replacement Appointment Services
deleted: Wheel Bearing Replacement {} 2026-09-29 19:00:34.652612
2026-09-29 11:00:34.652612+00 bcbd6782-034c-4457-b7e6-0cfd9b0a9b34
143.44.164.123
a7ddf055-d413-49c5-9517-298c528f421e
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com deleted
Appointment Services Suspension Rebuild Appointment Services deleted:
Suspension Rebuild {} 2026-09-29 19:00:34.652612 2026-09-29
11:00:34.652612+00 382d7b29-d521-4ab5-ab1a-2f92a156b2a1
143.44.164.123
67cc2fad-ecc0-4513-a8b0-5818b5467fee
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com deleted
Appointment Services Steering Bearing Replacement Appointment
Services deleted: Steering Bearing Replacement {} 2026-09-29
19:00:34.652612 2026-09-29 11:00:34.652612+00
88b7c3e4-7133-4e3e-80d6-69fde94ee9c0 143.44.164.123
d64fe536-680a-4e1e-9c4e-30e90dca08b0
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com deleted
Appointment Services ECU Diagnostic Scan Appointment Services deleted:
ECU Diagnostic Scan {} 2026-09-29 19:00:34.652612 2026-09-29
11:00:34.652612+00 91e2f73d-67ee-4452-ac92-e8b6a4a8ba93
143.44.164.123
60fb3f7d-c292-46c3-b1e1-21ae56101f90
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com deleted
Notifications 504f93e8-929f-4183-8ec6-980a770dc82a Notifications
deleted: 504f93e8-929f-4183-8ec6-980a770dc82a {} 2026-09-29
19:00:34.652612 2026-09-29 11:00:34.652612+00
504f93e8-929f-4183-8ec6-980a770dc82a 143.44.164.123
1a5e60af-9d58-4c86-9275-f95aa031bc62
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status
changed Appointments FRM-4RKQTN Appointments status changed:
FRM-4RKQTN {status} 2026-09-30 12:52:24.594772 2026-09-30
04:52:24.594772+00 2ce47e77-9cfc-49b8-9710-c3b33294eb54
180.193.217.82
261e8626-942c-4719-933e-0150ae5971d2
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-10-07 20:25:19.474556 2026-10-07
12:25:19.474556+00 \N 143.44.164.41
b31afef1-0213-48a5-989c-330b8ac033b9
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed

out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-10-09 00:32:59.190349 2026-10-08 16:32:59.190349+00 \N 143.44.164.41 c493fdaa-894b-4460-91b1-9ab3dc3611df 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-11 15:49:54.635687 2026-10-11 07:49:54.635687+00 \N 143.44.164.41 e3a336f0-488e-4b42-ba0e-98e355328be7 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-14 17:30:47.02837 2026-10-14 09:30:47.02837+00 \N 143.44.164.41 93dd8b54-c796-4f7d-b063-e1252fa54a20 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 15 minutes of inactivity. {} 2026-10-18 17:02:35.997204 2026-10-18 09:02:35.997204+00 \N 143.44.164.41 07dd7a1a-ac70-47e3-9972-f110e28faf81 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Notifications 4cdb7e2f-136f-4fea-9107-7685b2d155c4 Notifications updated: 4cdb7e2f-136f-4fea-9107-7685b2d155c4 {is_read} 2026-09-26 07:08:11.202047 2026-09-25 23:08:11.202047+00 4cdb7e2f-136f-4fea-9107-7685b2d155c4 143.44.164.123 8327aefb-ac91-4962-ba13-64909ea2ee33 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Crew Members Joe Mar Crew Members updated: Joe Mar {name} 2026-09-27 09:41:42.517478 2026-09-27 01:41:42.517478+00 5cb00871-f5ef-4ccf-bd61-4746909237e2 143.44.164.184 09e68542-110b-40be-9c24-1dfae6e698c6 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-28 15:17:53.937013 2026-09-28 07:17:53.937013+00 \N 143.44.164.66 0182206a-c796-48f1-bf62-9779449179a7 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Appointments FRM-EVXKHD Appointments created: FRM-EVXKHD {} 2026-09-28 19:38:09.628656 2026-09-28 11:38:09.628656+00 30c80b92-c789-40fc-936d-5f6567091423 143.44.164.66 ec05ef69-7e6c-4e99-8d96-383480e4f4c2 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Appointment Services Change Oil Appointment Services created: Change Oil {} 2026-09-28 19:38:09.628656 2026-09-28 11:38:09.628656+00 09838e4f-76e7-47bd-8b64-1e76c4e6d0b8 143.44.164.66 f8a807cb-ab2c-4507-9740-3b9734203d55

7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-ZTUG43 Appointments status changed: FRM-ZTUG43 {admin_notes,pending_reschedule_date,pending_reschedule_reason,pending_reschedule_reference_code,pending_reschedule_request_id,pending_reschedule_start_time,reschedule_count,rescheduled_to_appointment_id,status} 2026-09-28 19:38:09.628656 2026-09-28 11:38:09.628656+00 a26950f7-62f0-439a-8f3b-07fa6a6c50d6 143.44.164.66 3c62fc75-aa63-4a53-b78b-07fb45cf84fd
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Notifications ddb0afeb-d56a-4660-99e0-866c2330b7ab Notifications created: ddb0afeb-d56a-4660-99e0-866c2330b7ab {} 2026-09-28 19:38:09.628656 2026-09-28 11:38:09.628656+00 ddb0afeb-d56a-4660-99e0-866c2330b7ab 143.44.164.66 40a0ebf0-cf72-4ac4-9efc-532e33f255b4
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Notifications ddb0afeb-d56a-4660-99e0-866c2330b7ab Notifications updated: ddb0afeb-d56a-4660-99e0-866c2330b7ab {is_read} 2026-09-28 19:38:11.809818 2026-09-28 11:38:11.809818+00 ddb0afeb-d56a-4660-99e0-866c2330b7ab 143.44.164.66 69d09ceb-a510-461c-97c4-5e87407be764
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Notifications e845b120-a027-44cd-9574-6976c0f58394 Notifications updated: e845b120-a027-44cd-9574-6976c0f58394 {is_read} 2026-09-28 19:38:26.797652 2026-09-28 11:38:26.797652+00 e845b120-a027-44cd-9574-6976c0f58394 143.44.164.66 87e57f72-12bf-4f77-9ee8-fc503f1cf4d4
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator signed out before visiting the public site. {} 2026-09-28 19:39:41.224593 2026-09-28 11:39:41.224593+00 \N 143.44.164.66 2dc374dc-332e-4612-abb2-b23937daf8f3
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-29 14:20:34.67852 2026-09-29 06:20:34.67852+00 \N 49.146.140.233 a274dca1-dcde-482d-80c5-22d8dc2c8ff4
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-29 19:04:24.538632 2026-09-29 11:04:24.538632+00 \N 143.44.164.123 2b93885a-433d-48a7-854e-4bb35b72f8a8
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-30 12:56:04.746672 2026-09-30 04:56:04.746672+00 \N 180.193.217.82

ca280d19-99c8-485d-a4c9-0c50881f6855
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-10-07 20:27:48.561009 2026-10-07 12:27:48.561009+00 \N 143.44.164.41
e4a3b8d7-b033-46ee-a178-34d6b4d321c2
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-09 16:55:14.155094 2026-10-09 08:55:14.155094+00 \N 143.44.164.41
27f7d791-cc85-4a45-a548-8459cc9f4b52
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed Reports Financial Report (Summary) Opened the print dialog for 3 completed transactions. {report_type,date_range,print_format} 2026-10-11 15:50:02.028981 2026-10-11 07:50:02.028981+00 \N 143.44.164.41
420bd93b-cf68-4556-88a6-5ca56a5c7258
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-14 18:39:09.916636 2026-10-14 10:39:09.916636+00 \N 143.44.164.41
acaf5322-269f-44af-a210-2a4e756a6870
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-21 18:12:31.7752 2026-10-21 10:12:31.7752+00 \N 143.44.164.41
c7b85a64-5b28-433b-9191-a44a54f53c1a \N admin@gmail.com archived Appointments FRM-NJYMJ5 Appointments archived: FRM-NJYMJ5 {is_archived} 2026-09-15 17:16:28.58863 2026-09-15 09:16:28.58863+00 1c323343-47eb-4454-901a-7f9d179b2142 143.44.164.66
6760eb60-e0a6-4a6a-b328-79539235fa56 \N admin@gmail.com archived Appointments FRM-Y5T8Q5 Appointments archived: FRM-Y5T8Q5 {is_archived} 2026-09-15 17:16:47.155474 2026-09-15 09:16:47.155474+00 72f72f17-5566-46c7-a558-ea331aa75e83 143.44.164.66
9bab7213-59d6-40c5-8fc1-f5e7d9c30700 \N admin@gmail.com archived Appointments FRM-Y5KSUN Appointments archived: FRM-Y5KSUN {is_archived} 2026-09-15 17:16:55.651612 2026-09-15 09:16:55.651612+00 177fbdfa-a85e-4777-8a8c-760b9ad02401 143.44.164.66
11aa953d-f305-4c5d-83a6-d627a749ac26 \N admin@gmail.com archived Appointments FRM-A5AEET Appointments archived: FRM-A5AEET {is_archived} 2026-09-15 17:17:14.285724 2026-09-15 09:17:14.285724+00 39e2435e-9880-4fc5-babb-a5d9c15828b2 143.44.164.66
fd3ba74e-db0f-4765-84eb-e3bdd75219b9 \N admin@gmail.com archived

Appointments FRM-JWSPJG Appointments archived: FRM-JWSPJG {is_archived} 2026-09-15 17:17:20.731314 2026-09-15 09:17:20.731314+00 13b28240-1671-4e50-9bb7-49ab02e21b9a 143.44.164.66 db4fa8b8-3406-42e0-871e-26e2cabb1c93 \N admin@gmail.com deleted
Appointments FRM-V6SZ37 Appointments deleted: FRM-V6SZ37 {} 2026-09-15 17:17:37.009867 2026-09-15 09:17:37.009867+00 5546bc74-3047-4d98-87e4-67a68084a24d 143.44.164.66 845b8e0a-4688-4de4-9616-a37babb82f86 \N admin@gmail.com deleted
Appointment Services Change Oil Appointment Services deleted: Change Oil {} 2026-09-15 17:17:37.009867 2026-09-15 09:17:37.009867+00 d1f97096-18a5-4d1e-a29c-0936c2115c7b 143.44.164.66 9bb75016-96d5-4a21-9b4a-d39f4fa74477 \N admin@gmail.com deleted
Notifications d47980d5-31d1-4e85-857a-bab7e0daab0b Notifications deleted: d47980d5-31d1-4e85-857a-bab7e0daab0b {} 2026-09-15 17:17:37.009867 2026-09-15 09:17:37.009867+00 d47980d5-31d1-4e85-857a-bab7e0daab0b 143.44.164.66 d559595f-aeab-4fdb-ac79-3c813155471a \N admin@gmail.com deleted
Notifications 453d2fcc-7126-4da3-9207-d6ab05fb36c2 Notifications deleted: 453d2fcc-7126-4da3-9207-d6ab05fb36c2 {} 2026-09-15 17:17:37.009867 2026-09-15 09:17:37.009867+00 453d2fcc-7126-4da3-9207-d6ab05fb36c2 143.44.164.66 69e9af5e-3c51-48c0-9e24-ff4db74a0d17 \N admin@gmail.com deleted
Notifications aea5d685-77e3-4fae-b972-db05dc5064ff Notifications deleted: aea5d685-77e3-4fae-b972-db05dc5064ff {} 2026-09-15 17:17:37.009867 2026-09-15 09:17:37.009867+00 aea5d685-77e3-4fae-b972-db05dc5064ff 143.44.164.66 2d1d2c68-f902-40fe-a901-a6fbd2aab86c \N admin@gmail.com deleted
Appointments FRM-A5AEET Appointments deleted: FRM-A5AEET {} 2026-09-15 17:17:46.75602 2026-09-15 09:17:46.75602+00 39e2435e-9880-4fc5-babb-a5d9c15828b2 143.44.164.66 3e86a91c-c106-426a-8d96-b98255ac85a1 \N admin@gmail.com deleted
Appointment Services Motorcycle Safety Inspection Appointment Services deleted: Motorcycle Safety Inspection {} 2026-09-15 17:17:46.75602 2026-09-15 09:17:46.75602+00 16ef159a-e33a-4a1a-b1fd-6a720d6ed9aa 143.44.164.66 4c16d659-6d2b-4d7d-a55d-8b9b66eda758
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-26 08:27:57.584166 2026-09-26 00:27:57.584166+00 \N 143.44.164.123 34c29bb6-2a0b-4516-98e2-753d34e793dd
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-27 09:44:26.974029

2026-09-27 01:44:26.974029+00 \N 143.44.164.184
e5967537-c931-460e-ba1e-c8315ed037b8
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-28 15:22:24.591291 2026-09-28 07:22:24.591291+00 \N 143.44.164.66
fb4eb359-89ed-4023-8ff2-5a1875bf0a7f
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-28 21:17:31.732486 2026-09-28 13:17:31.732486+00 \N 136.158.3.55
9ac1430d-bead-4cf5-822b-9ca7572e9195
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-29 14:24:59.033092 2026-09-29 06:24:59.033092+00 \N 49.146.140.233
0c95572b-d64b-4c9b-9169-ab99e1b0e5c9
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-29 19:05:23.68763 2026-09-29 11:05:23.68763+00 \N 143.44.164.123
ec587824-962e-4c18-bc83-41f33087ce05
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Appointments FRM-ULM8GG Appointments created: FRM-ULM8GG {} 2026-09-30 13:01:21.761562 2026-09-30 05:01:21.761562+00
de1e21dc-53e4-4362-8898-3bd1da2d67fc 180.193.217.82
4ec657ea-e5d8-47dd-b9c8-650bb0e185d1
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Appointment Services Brake Service Appointment Services created: Brake Service {} 2026-09-30 13:01:21.761562 2026-09-30 05:01:21.761562+00
91ac9ff5-f1ce-4dbc-a976-f03173792847 180.193.217.82
3a6f60f0-457f-4e2e-bc86-1f9a665528fa
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-4RKQTN Appointments status changed: FRM-4RKQTN {admin_notes,pending_reschedule_date,pending_reschedule_reason,pending_reschedule_reference_code,pending_reschedule_request_id,pending_reschedule_start_time,reschedule_count,rescheduled_to_appointment_id,status} 2026-09-30 13:01:21.761562 2026-09-30 05:01:21.761562+00
2ce47e77-9cfc-49b8-9710-c3b33294eb54 180.193.217.82
529a681c-ccd6-4be7-b9e8-8656ca3d2311
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Notifications dc396090-b3b7-4506-b00f-9027c731fd5e Notifications created: dc396090-b3b7-4506-b00f-9027c731fd5e {} 2026-09-30 13:01:21.761562 2026-09-30 05:01:21.761562+00
dc396090-b3b7-4506-b00f-9027c731fd5e 180.193.217.82

cf67b4cc-c290-44b1-ac93-31c75ad041b9
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Notifications 8222072d-d208-451b-806a-68afff124dda Notifications created: 8222072d-d208-451b-806a-68afff124dda {} 2026-09-30 13:01:21.761562 2026-09-30 05:01:21.761562+00
8222072d-d208-451b-806a-68afff124dda 180.193.217.82
9030848e-9c41-4eae-99ce-732bd907c037
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-07 20:34:45.412277 2026-10-07 12:34:45.412277+00 \N 143.44.164.41
29037c30-6fe2-4ff0-8fc0-64d253406ab3
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-577L8T Appointments status changed: FRM-577L8T {status} 2026-10-09 16:55:24.539046 2026-10-09 08:55:24.539046+00 dc1c4a1c-1532-4d25-bbd5-b7d8793de6e9
143.44.164.41
169e2eb2-e5b8-4df5-bb0e-64e182b39db6
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-EVXKHD Appointments status changed: FRM-EVXKHD {service_started_at,status} 2026-10-09 16:55:29.629302 2026-10-09 08:55:29.629302+00 30c80b92-c789-40fc-936d-5f6567091423
143.44.164.41
b3a78ad4-971d-4c2d-87ae-672382745e8a
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Notifications 35027590-6e64-4ffa-acb5-fc8abb04b253 Notifications updated: 35027590-6e64-4ffa-acb5-fc8abb04b253 {is_read} 2026-10-09 16:55:32.4368 2026-10-09 08:55:32.4368+00
35027590-6e64-4ffa-acb5-fc8abb04b253 143.44.164.41
1ff8eeaa-0cad-4c0c-9b4d-ea220c07653d
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-13 20:25:30.801317 2026-10-13 12:25:30.801317+00 \N 143.44.164.41
c1f5fbcd-6d46-48e7-90e1-06d5d5ca35e3
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-14 18:45:53.946322 2026-10-14 10:45:53.946322+00 \N 143.44.164.41
18639727-fe35-40e9-8f6f-72acf1f28075
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Notifications a0ed8759-35b5-49d2-9efd-2bcf572610e4 Notifications updated: a0ed8759-35b5-49d2-9efd-2bcf572610e4 {is_read} 2026-10-14 18:46:13.46826 2026-10-14 10:46:13.46826+00
a0ed8759-35b5-49d2-9efd-2bcf572610e4 143.44.164.41

8007a1ed-95ab-4527-ad89-519cb1de648b
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Notifications d4f3f236-2a56-4f4a-b8db-2dc7ced8a84f Notifications
updated: d4f3f236-2a56-4f4a-b8db-2dc7ced8a84f {is_read} 2026-10-14
18:46:13.46826 2026-10-14 10:46:13.46826+00
d4f3f236-2a56-4f4a-b8db-2dc7ced8a84f 143.44.164.41
f0eb313e-1a40-4b2e-a677-66c0b9e233f0
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-10-21 18:18:24.971565 2026-10-21
10:18:24.971565+00 \N 143.44.164.41
2504989a-eb7f-47e1-8a9a-11a810494ed8 \N admin@gmail.com archived
Appointments FRM-QAYJ78 Appointments archived: FRM-QAYJ78
{is_archived} 2026-09-15 17:16:31.909434 2026-09-15
09:16:31.909434+00 6a450dde-e1ad-45d2-b1cb-0c5d13597c0b
143.44.164.66
971d00a5-6f65-4e03-bddc-0fad7b4dd4ff \N admin@gmail.com archived
Appointments FRM-6JQ8FY Appointments archived: FRM-6JQ8FY
{is_archived} 2026-09-15 17:17:01.595655 2026-09-15
09:17:01.595655+00 42fb7740-5457-4460-b968-78c0e4b19027
143.44.164.66
92c14763-477b-4895-9389-4685c2743a7e \N admin@gmail.com archived
Appointments FRM-EZ6AHX Appointments archived: FRM-EZ6AHX
{is_archived} 2026-09-15 17:17:17.463575 2026-09-15
09:17:17.463575+00 cd19fd75-2ed8-4267-bf27-8660dab07d66
143.44.164.66
1aa82de6-10bc-46dd-8ab9-a28bf6bc1420 \N admin@gmail.com deleted
Appointments FRM-JWSPJG Appointments deleted: FRM-JWSPJG {}
2026-09-15 17:17:39.681994 2026-09-15 09:17:39.681994+00
13b28240-1671-4e50-9bb7-49ab02e21b9a 143.44.164.66
4f4b4b3a-67b0-45c0-bff7-b2651d014165 \N admin@gmail.com deleted
Appointment Services Change Oil Appointment Services deleted: Change Oil
{} 2026-09-15 17:17:39.681994 2026-09-15 09:17:39.681994+00
bb68d78a-1457-4177-aa19-1fb8f3bd728e 143.44.164.66
43db9243-d604-4bca-91a0-ab5b93fe6c13 \N admin@gmail.com deleted
Notifications cf028a25-c235-414d-8804-544f9e255c6d Notifications deleted:
cf028a25-c235-414d-8804-544f9e255c6d {} 2026-09-15 17:17:39.681994
2026-09-15 09:17:39.681994+00 cf028a25-c235-414d-8804-544f9e255c6d
143.44.164.66
6afde0d1-89d1-46b7-b8ef-c5d0b77e7d21 \N admin@gmail.com deleted
Notifications fbe99389-297f-407c-902e-840075c75713 Notifications deleted:
fbe99389-297f-407c-902e-840075c75713 {} 2026-09-15 17:17:39.681994
2026-09-15 09:17:39.681994+00 fbe99389-297f-407c-902e-840075c75713
143.44.164.66
07fcdf1e-6c07-4d3a-ac93-be9fd3bde16a \N admin@gmail.com deleted

Notifications 27ac7402-ee29-4711-8cc9-ebe812d08504 Notifications deleted: 27ac7402-ee29-4711-8cc9-ebe812d08504 {} 2026-09-15 17:17:39.681994 2026-09-15 09:17:39.681994+00 27ac7402-ee29-4711-8cc9-ebe812d08504 143.44.164.66
1251d070-2b39-4acd-88f6-4cb22a173deb \N admin@gmail.com updated Appointments FRM-QU6QPF Appointments updated: FRM-QU6QPF {rescheduled_from_appointment_id} 2026-09-15 17:17:39.681994 2026-09-15 09:17:39.681994+00 75a07427-ff65-4d7e-affb-fe489d59346c 143.44.164.66
5df43541-d1e5-4c3d-9cf5-a1e0b5e47d99 \N admin@gmail.com deleted Appointments FRM-QU6QPF Appointments deleted: FRM-QU6QPF {} 2026-09-15 17:17:49.499187 2026-09-15 09:17:49.499187+00 75a07427-ff65-4d7e-affb-fe489d59346c 143.44.164.66
0fd8a51a-abfb-4531-8f13-4a82e0ff80f6 \N admin@gmail.com deleted Appointment Services Change Oil Appointment Services deleted: Change Oil {} 2026-09-15 17:17:49.499187 2026-09-15 09:17:49.499187+00 40f7452f-45c4-499c-a109-489c8ecddcb0 143.44.164.66
6f99304d-0a22-4cff-8d7c-f6b5cf5ec9dc \N admin@gmail.com deleted Notifications 05a9d0cd-d29d-445f-a9b7-791c01c55239 Notifications deleted: 05a9d0cd-d29d-445f-a9b7-791c01c55239 {} 2026-09-15 17:17:49.499187 2026-09-15 09:17:49.499187+00 05a9d0cd-d29d-445f-a9b7-791c01c55239 143.44.164.66
caeb9a81-47be-4c01-9410-30857dad3a2d \N admin@gmail.com deleted Notifications abebbbd8-667f-4b6c-9ae3-59a936668106 Notifications deleted: abebbbd8-667f-4b6c-9ae3-59a936668106 {} 2026-09-15 17:17:49.499187 2026-09-15 09:17:49.499187+00 abebbbd8-667f-4b6c-9ae3-59a936668106 143.44.164.66
4bf08db9-2f58-4bb1-b0a9-e70fdf7dde3d 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-26 08:35:36.637392 2026-09-26 00:35:36.637392+00 \N 143.44.164.123
36665242-882c-40bf-b2ac-a715ec4325a3 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-27 09:58:18.089677 2026-09-27 01:58:18.089677+00 \N 143.44.164.184
ddecf533-9459-491a-9a11-baae40007c30 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-NNQNP8 Appointments status changed: FRM-NNQNP8 {service_started_at,status} 2026-09-28 15:22:36.560525 2026-09-28 07:22:36.560525+00 2a2de84e-e1d7-42e7-a36d-45905d69df12 143.44.164.66
cdfc2afc-f298-40c9-b5ec-47bb3ca73537 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed

out Authentication Admin console Administrator signed out of the admin console. {} 2026-09-28 15:22:55.826018 2026-09-28 07:22:55.826018+00 \N 143.44.164.66 011cd952-683e-4130-b6d0-996429f31027 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-28 21:21:35.1494 2026-09-28 13:21:35.1494+00 \N 136.158.3.55 747db120-e96b-497c-b5f2-2ca743701606 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-29 14:33:07.890695 2026-09-29 06:33:07.890695+00 \N 49.146.140.233 d6c376fb-c6ba-4ba9-a625-16f762a9cc10 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-29 19:08:30.476176 2026-09-29 11:08:30.476176+00 \N 143.44.164.123 bde78525-f85d-4b44-ac89-9671e0b07007 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-HWAVDJ Appointments status changed: FRM-HWAVDJ {status} 2026-09-30 13:03:56.585088 2026-09-30 05:03:56.585088+00 617073f7-8cfa-4f3f-8352-aff1a76da780 180.193.217.82 fdda829e-9ca9-4d4f-be68-67bc3d8d7deb 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-07 21:15:04.992101 2026-10-07 13:15:04.992101+00 \N 143.44.164.41 0c06c9ab-116d-4e79-a5b6-7b7e71d4f720 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator signed out before visiting the public site. {} 2026-10-09 16:56:27.377174 2026-10-09 08:56:27.377174+00 \N 143.44.164.41 77cd84bd-5aa9-4514-90e9-f929459c2c99 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-XQ2XNC Appointments status changed: FRM-XQ2XNC {status} 2026-10-13 20:25:40.460397 2026-10-13 12:25:40.460397+00 e8900d91-8ae4-4bd6-9a33-eb818faa92a2 143.44.164.41 9f97103c-f835-4996-94d6-b30f60ae044f 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-AQY69P Appointments status changed: FRM-AQY69P {service_started_at,status} 2026-10-13 20:25:41.94506 2026-10-13 12:25:41.94506+00 eda6b8a7-6dc1-4584-8ad5-e918438c0a05 143.44.164.41

6d1c3d9d-6a27-428e-abdc-d7b6f621d926
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed Reports Financial Report (Summary) Opened the print dialog for 3 completed transactions. {report_type,date_range,print_format} 2026-10-13 20:27:24.92674 2026-10-13 12:27:24.92674+00 \N 143.44.164.41
94f5f3bf-0014-492a-a310-1b076e535a43
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed Reports Financial Report (Summary) Opened the print dialog for 4 completed transactions. {report_type,date_range,print_format} 2026-10-14 18:46:18.814697 2026-10-14 10:46:18.814697+00 \N 143.44.164.41
9cea987a-7ee9-4303-a99f-e3011ba29273
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-JKNK84 Appointments status changed: FRM-JKNK84 {service_started_at,status} 2026-10-21 18:19:33.517794 2026-10-21 10:19:33.517794+00 6c0de99e-7522-4f4e-ad11-0913682e3ae3 143.44.164.41
8ebef422-0f13-4a1e-836e-f4f594678a90 \N admin@gmail.com archived Appointments FRM-3GK8D3 Appointments archived: FRM-3GK8D3 {is_archived} 2026-09-15 17:16:35.007784 2026-09-15 09:16:35.007784+00 95d9a02d-e47b-4652-8ebd-542cefe276c7 143.44.164.66
6fbe4a82-3663-4b06-97ed-341c009db8f6 \N admin@gmail.com archived Appointments FRM-G4XL75 Appointments archived: FRM-G4XL75 {is_archived} 2026-09-15 17:17:05.068948 2026-09-15 09:17:05.068948+00 4acd6706-a379-4504-b7a6-ae8f896e5dbd 143.44.164.66
8684a254-33de-4f64-bf2f-e2747e2c6045 \N admin@gmail.com deleted Appointments FRM-GAY6TL Appointments deleted: FRM-GAY6TL {} 2026-09-15 17:18:09.158786 2026-09-15 09:18:09.158786+00 5788a190-a4a7-4a96-b460-b29104e00491 143.44.164.66
5b9c1a79-0e1d-4e9e-bd9d-e89208de89d3 \N admin@gmail.com deleted Appointment Services Clutch Adjustment/Repair Appointment Services deleted: Clutch Adjustment/Repair {} 2026-09-15 17:18:09.158786 2026-09-15 09:18:09.158786+00 3406bb12-9d83-47f6-804d-f874e7b4af5e 143.44.164.66
3a50b019-4dfb-41f6-9609-dcd362327b7a
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-26 08:37:31.789657 2026-09-26 00:37:31.789657+00 \N 143.44.164.123
4e88e5ec-5554-4a1c-a84c-10f38ce756c8
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-27 10:00:55.027598 2026-09-27 02:00:55.027598+00 \N 143.44.164.184

50995cc4-aede-4376-8d6b-3dddd4eb7bf5
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-28 15:23:13.68712 2026-09-28 07:23:13.68712+00 \N 143.44.164.66
57b26b35-0f4f-419b-bef8-16a0aeccaeb3
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-28 21:34:19.487028 2026-09-28 13:34:19.487028+00 \N 136.158.3.55
f419181f-7114-4402-8d23-1267c2d5fad8
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-29 14:41:45.003001 2026-09-29 06:41:45.003001+00 \N 49.146.140.233
435cac25-c7be-40ca-9352-710df131696f
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-29 19:19:32.246346 2026-09-29 11:19:32.246346+00 \N 143.44.164.123
5a10df11-b480-467d-acf6-487cb9101180
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-30 13:10:31.182837 2026-09-30 05:10:31.182837+00 \N 180.193.217.82
abd91a08-c41f-4d93-877e-8112768c262b
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-07 21:36:57.342236 2026-10-07 13:36:57.342236+00 \N 143.44.164.41
e90ad241-b0a6-41b0-b1a1-1ba09c581b3c
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-09 16:57:43.360447 2026-10-09 08:57:43.360447+00 \N 143.44.164.41
979d3ded-2d7a-41ef-9559-0f2cffcaeac4
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-ATZJXT Appointments status changed: FRM-ATZJXT {status} 2026-10-09 16:57:48.811002 2026-10-09 08:57:48.811002+00 af37178b-4dfd-455f-8d90-2f7d3f4b4304 143.44.164.41
fbf4ce0c-e50b-424a-a68d-e917acc30272
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-ZBM5AB Appointments status changed: FRM-ZBM5AB {status} 2026-10-13 20:25:46.398361 2026-10-13 12:25:46.398361+00 cfe5e95c-d7d8-4ba5-b6e7-9020b664a0fc

143.44.164.41
6266dde4-86cc-4dd7-9c89-3720414512cd
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Notifications 38c1520d-7a2c-4036-9740-c6b6537a1fa7
Notifications updated: 38c1520d-7a2c-4036-9740-c6b6537a1fa7 {is_read}
2026-10-13 20:25:52.850502 2026-10-13 12:25:52.850502+00
38c1520d-7a2c-4036-9740-c6b6537a1fa7 143.44.164.41
fabcbcc9-02f5-4bcb-aa81-c68239c59901
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Notifications 133a7637-64df-44aa-9a5c-8ee64955659d
Notifications updated: 133a7637-64df-44aa-9a5c-8ee64955659d {is_read}
2026-10-13 20:25:52.850502 2026-10-13 12:25:52.850502+00
133a7637-64df-44aa-9a5c-8ee64955659d 143.44.164.41
c9ea283d-042c-40d8-9785-03debe87f862
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Notifications 0f0f9bfa-d154-4863-8bea-04abeb6e16a3
Notifications updated: 0f0f9bfa-d154-4863-8bea-04abeb6e16a3 {is_read}
2026-10-13 20:25:52.850502 2026-10-13 12:25:52.850502+00
0f0f9bfa-d154-4863-8bea-04abeb6e16a3 143.44.164.41
6390a315-1436-41d4-a5d6-3c44ad16694e
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Notifications 792caf85-daed-4dbb-b2ea-5bc9e82cb1fb
Notifications updated: 792caf85-daed-4dbb-b2ea-5bc9e82cb1fb {is_read}
2026-10-13 20:25:52.850502 2026-10-13 12:25:52.850502+00
792caf85-daed-4dbb-b2ea-5bc9e82cb1fb 143.44.164.41
e446800a-a112-49e9-b4ce-5fc5d10635ad
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-10-14 19:03:17.703751 2026-10-14
11:03:17.703751+00 \N 143.44.164.41
45aa147a-9a66-424c-8238-0072e87dfad8
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status
changed Appointments FRM-XQ2XNC Appointments status changed:
FRM-XQ2XNC {service_started_at,status} 2026-10-21 18:19:34.79011
2026-10-21 10:19:34.79011+00 e8900d91-8ae4-4bd6-9a33-eb818faa92a2
143.44.164.41
1a54fe56-95b6-4ca5-ad35-3090d9c454c8
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status
changed Appointments FRM-JKNK84 Appointments status changed:
FRM-JKNK84 {service_ended_at,status} 2026-10-21 18:19:53.441669
2026-10-21 10:19:53.441669+00 6c0de99e-7522-4f4e-ad11-0913682e3ae3
143.44.164.41
1d425e5c-06bd-4c55-ae85-7ce7dedc84f7
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed
Reports Financial Report (Summary) Opened the print dialog for 7

completed transactions. {report_type,date_range,print_format} 2026-10-21 18:21:37.100958 2026-10-21 10:21:37.100958+00 \N 143.44.164.41 3056e1b2-7aa1-4374-8049-e1c91c673378 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com exported Reports Booking Report Exported 12 bookings as PDF. {report_type,date_range,export_format} 2026-09-26 08:39:27.486245 2026-09-26 00:39:27.486245+00 \N 143.44.164.123 7350ca14-53c6-4f54-b954-f05e632f9713 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-27 10:10:19.196069 2026-09-27 02:10:19.196069+00 \N 143.44.164.184 7dbaa7ab-1e83-4ae2-ab62-b02eb5a600e2 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-28 15:44:12.901363 2026-09-28 07:44:12.901363+00 \N 143.44.164.66 5da55516-6d54-4e91-bc28-ad8d37f979ac \N admin@gmail.com archived Appointments FRM-TR77MJ Appointments archived: FRM-TR77MJ {is_archived} 2026-09-15 17:16:39.157228 2026-09-15 09:16:39.157228+00 09231886-c303-437e-b12d-14f6d13a7e98 143.44.164.66 0a66264f-8f48-4a21-ae6d-a25790a0f586 \N admin@gmail.com archived Appointments FRM-76C4GS Appointments archived: FRM-76C4GS {is_archived} 2026-09-15 17:17:08.270062 2026-09-15 09:17:08.270062+00 49ecb311-2ad3-45fc-b9c1-bca9a53d9b13 143.44.164.66 74c9e0ce-80bc-400b-8bf4-53e7b2f1cba8 \N admin@gmail.com deleted Appointments FRM-SF6XZG Appointments deleted: FRM-SF6XZG {} 2026-09-15 17:17:33.64303 2026-09-15 09:17:33.64303+00 a99afab3-aef7-4875-82fa-a347ce834601 143.44.164.66 11f267cd-f148-4323-815d-68a2362686f6 \N admin@gmail.com deleted Appointment Services Brake Service Appointment Services deleted: Brake Service {} 2026-09-15 17:17:33.64303 2026-09-15 09:17:33.64303+00 62bd7d17-6cc7-4246-9c1c-a50de8acfef9 143.44.164.66 631b0d6a-0c7a-4f23-a634-903815e8a250 \N admin@gmail.com deleted Appointment Services Chain & Sprocket Replacement Appointment Services deleted: Chain & Sprocket Replacement {} 2026-09-15 17:17:33.64303 2026-09-15 09:17:33.64303+00 f04ec88d-222a-4a13-a8bd-0270bf74651c 143.44.164.66 c01406e3-cda8-4d47-84a4-a367c6996936 \N admin@gmail.com deleted Notifications 0b72b9dc-cda7-439b-bc14-b8461decbeb6 Notifications deleted: 0b72b9dc-cda7-439b-bc14-b8461decbeb6 {} 2026-09-15 17:17:33.64303 2026-09-15 09:17:33.64303+00 0b72b9dc-cda7-439b-bc14-b8461decbeb6 143.44.164.66

1b45c8a9-d2b7-4c9f-8f90-08480cc8e07b \N admin@gmail.com deleted Appointments FRM-EZ6AHX Appointments deleted: FRM-EZ6AHX {} 2026-09-15 17:17:42.609721 2026-09-15 09:17:42.609721+00 cd19fd75-2ed8-4267-bf27-8660dab07d66 143.44.164.66
d7ed27b4-a16a-4027-9edf-71b9fcab95f0 \N admin@gmail.com deleted Appointment Services Change Oil Appointment Services deleted: Change Oil {} 2026-09-15 17:17:42.609721 2026-09-15 09:17:42.609721+00 3d859960-862f-453b-9e3b-00c7ac3bc544 143.44.164.66
0b89496a-e8de-45b9-90e9-fcb8f1d63d17 \N admin@gmail.com deleted Notifications ded9c694-d142-4f79-b256-10ca37cf6dbe Notifications deleted: ded9c694-d142-4f79-b256-10ca37cf6dbe {} 2026-09-15 17:17:42.609721 2026-09-15 09:17:42.609721+00 ded9c694-d142-4f79-b256-10ca37cf6dbe 143.44.164.66
8561d504-9b32-48fd-a3e4-d819dd1abe99 \N admin@gmail.com updated Appointments FRM-QU6QPF Appointments updated: FRM-QU6QPF {rescheduled_to_appointment_id} 2026-09-15 17:17:42.609721 2026-09-15 09:17:42.609721+00 75a07427-ff65-4d7e-affb-fe489d59346c 143.44.164.66
a1feec22-011e-4605-a205-03f3615e53e5 \N admin@gmail.com deleted Appointments FRM-Y5KSUN Appointments deleted: FRM-Y5KSUN {} 2026-09-15 17:17:56.463631 2026-09-15 09:17:56.463631+00 177fbdfa-a85e-4777-8a8c-760b9ad02401 143.44.164.66
ffd8d699-971e-4ee6-88e7-a5ff7dba9f37 \N admin@gmail.com deleted Appointment Services Brake Service Appointment Services deleted: Brake Service {} 2026-09-15 17:17:56.463631 2026-09-15 09:17:56.463631+00 775e530f-b4fc-4514-9bc2-788f944d3f28 143.44.164.66
d62da6d5-4b0c-4449-915c-9e0d6ca0a95b \N admin@gmail.com deleted Appointments FRM-EFGUWJ Appointments deleted: FRM-EFGUWJ {} 2026-09-15 17:18:05.807596 2026-09-15 09:18:05.807596+00 34c976d2-745d-4c5e-b994-354d019706e8 143.44.164.66
4d1b340d-aaa8-403d-8fbe-fa94fd829a5a \N admin@gmail.com deleted Appointment Services Clutch Adjustment/Repair Appointment Services deleted: Clutch Adjustment/Repair {} 2026-09-15 17:18:05.807596 2026-09-15 09:18:05.807596+00 965999f0-ff62-4fff-866d-17855b0b0f19 143.44.164.66
2f007b47-8ea5-412a-9c72-d8d7b36f8acc \N admin@gmail.com updated Appointments FRM-GAY6TL Appointments updated: FRM-GAY6TL {rescheduled_to_appointment_id} 2026-09-15 17:18:05.807596 2026-09-15 09:18:05.807596+00 5788a190-a4a7-4a96-b460-b29104e00491 143.44.164.66
64d5caa7-2903-4aac-a043-53aef833d5b0 \N admin@gmail.com deleted Appointments FRM-VJ965M Appointments deleted: FRM-VJ965M {} 2026-09-15 17:18:11.976127 2026-09-15 09:18:11.976127+00 64cf7994-ff35-418a-97f8-9fef8a68b07e 143.44.164.66
18f18a2c-8de5-4e0e-b2b4-7bdfe21f04a2
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed

in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-28 22:05:20.835936 2026-09-28 14:05:20.835936+00 \N 119.93.119.173 e0856095-0b26-4d6e-953b-548ba8edfa36 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-29 14:54:15.924759 2026-09-29 06:54:15.924759+00 \N 49.146.140.233 9767b294-3a0c-40c3-8b98-e489d1abf13d 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Settings Shop information Shop information updated. {"Booking terms & conditions"} 2026-09-29 19:20:23.755656 2026-09-29 11:20:23.755656+00 \N 143.44.164.123 2ef8439d-42a8-4547-b52f-20e610ef022c 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-30 13:16:17.880974 2026-09-30 05:16:17.880974+00 \N 180.193.217.82 3e7d20f2-a6fb-4f3a-b54d-403670fdee0b 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-10-07 21:39:30.42037 2026-10-07 13:39:30.42037+00 \N 143.44.164.41 59533e28-7c41-4c31-b8f1-090f714da814 \N e2e-notification-0446ba81-8783-4506-978e-ca75c8680c91@example.invalid status changed Appointments FRM-228VV4 Appointments status changed: FRM-228VV4 {status} 2026-10-11 13:00:04.981434 2026-10-11 05:00:04.981434+00 53021b0d-5241-4b5c-ab7f-5acd0bbc9823 143.44.164.41 70089ee7-a29e-45c2-a8d7-924fdbf99c52 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-10-13 20:30:35.211863 2026-10-13 12:30:35.211863+00 \N 143.44.164.41 1abb91b5-1792-42ad-adcc-34ff36361981 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed Reports Financial Report (Summary) Opened the print dialog for 4 completed transactions. {report_type,date_range,print_format} 2026-10-14 19:03:27.293723 2026-10-14 11:03:27.293723+00 \N 143.44.164.41 b8c48025-1b8a-409e-b667-6fdfbc682d99 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-XQ2XNC Appointments status changed: FRM-XQ2XNC {service_ended_at,status} 2026-10-21 18:19:43.487597 2026-10-21 10:19:43.487597+00 e8900d91-8ae4-4bd6-9a33-eb818faa92a2 143.44.164.41

9fab33df-4974-40c4-9bbd-da764095b1a7
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-26 08:41:28.984756 2026-09-26 00:41:28.984756+00 \N 143.44.164.123
c9cef667-49ed-4039-acff-44cdd0d5b453
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-27 13:03:13.575205 2026-09-27 05:03:13.575205+00 \N 143.44.164.184
2b64f486-a5d0-422f-b5e4-016655f16b66
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-ZTUG43 Appointments status changed: FRM-ZTUG43 {status} 2026-09-28 15:44:20.963403 2026-09-28 07:44:20.963403+00 a26950f7-62f0-439a-8f3b-07fa6a6c50d6 143.44.164.66
9a3906eb-d3f6-4baa-b3f9-5be5a8fbf886
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Notifications 6cde70de-1009-404d-9ac6-971c1b1e344e Notifications updated: 6cde70de-1009-404d-9ac6-971c1b1e344e {is_read} 2026-09-28 15:44:22.674557 2026-09-28 07:44:22.674557+00 6cde70de-1009-404d-9ac6-971c1b1e344e 143.44.164.66
aa2b4ddb-93e6-4805-9792-d06e0cb101cf
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-29 07:56:09.809028 2026-09-28 23:56:09.809028+00 \N 143.44.164.123
75d31ae0-ad8a-4238-860f-27b54121e947
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-29 18:41:00.407744 2026-09-29 10:41:00.407744+00 \N 143.44.164.123
7144bd7d-913e-4bf3-9aa6-87bab5c512ad
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-29 19:22:28.064758 2026-09-29 11:22:28.064758+00 \N 143.44.164.123
56c7545b-3241-442f-b627-4094c774899d
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com exported Reports Booking Report Exported 7 bookings as PDF. {report_type,date_range,export_format} 2026-09-30 13:20:03.996656 2026-09-30 05:20:03.996656+00 \N 180.193.217.82
b2d7294d-d1ad-4757-a168-ef3fcbb78079
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-07 22:02:05.03628 2026-10-07 14:02:05.03628+00

\N 143.44.164.41
c95eea87-8eac-44ee-b302-d6b680ab7601 \N
e2e-notification-7df22c1d-18a5-4cf4-881e-0951b10de5a6@example.invalid
status changed Appointments FRM-GYSE3U Appointments status
changed: FRM-GYSE3U {status} 2026-10-11 13:25:15.445696
2026-10-11 05:25:15.445696+00 4512cd6e-96e6-443e-8192-1b74c50b8eb1
143.44.164.41
097f1cb6-e7e5-4863-a0e1-dbdf9c30149b
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-10-13 21:00:09.834306 2026-10-13
13:00:09.834306+00 \N 143.44.164.41
23fec549-db8a-4633-9e1f-84d1a417cf89
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-10-15 09:24:53.663352 2026-10-15
01:24:53.663352+00 \N 143.44.164.41
c9380671-7a79-4c2c-b96d-020d71552c8e
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed
Reports Financial Report (Summary) Opened the print dialog for 7
completed transactions. {report_type,date_range,print_format} 2026-10-21
18:25:43.263218 2026-10-21 10:25:43.263218+00 \N 143.44.164.41
2a1457f4-cbe7-4e6f-a61e-57f4b9e91a71 \N admin@gmail.com deleted
Appointment Services Tire Change & Balancing Appointment Services
deleted: Tire Change & Balancing {} 2026-09-15 17:18:47.570759
2026-09-15 09:18:47.570759+00 322dbf3c-d640-4c68-a838-952f7c0598e5
143.44.164.66
1716b929-57fa-4d60-afb7-c7b03e68990d \N admin@gmail.com deleted
Appointments FRM-Y5T8Q5 Appointments deleted: FRM-Y5T8Q5 {}
2026-09-15 17:19:02.105693 2026-09-15 09:19:02.105693+00
72f72f17-5566-46c7-a558-ea331aa75e83 143.44.164.66
b11b29b4-89ea-461b-acfb-1b17a6f30b33 \N admin@gmail.com deleted
Appointment Services Change Oil Appointment Services deleted: Change Oil
{} 2026-09-15 17:19:02.105693 2026-09-15 09:19:02.105693+00
8ebe91ef-1d63-4a3c-aa9a-aaf176b90fbb 143.44.164.66
01cf3cca-d0a0-4980-9806-322a8f6d4d9c
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-09-26 10:29:54.672739 2026-09-26
02:29:54.672739+00 \N 143.44.164.123
4207d8ba-add7-4fcb-8c6b-965bced42520
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status
changed Appointments FRM-MPMHT9 Appointments status changed:
FRM-MPMHT9 {status} 2026-09-27 13:03:27.297226 2026-09-27
05:03:27.297226+00 6014afba-21ae-402c-a687-150730e547ea

143.44.164.184
44c416b0-fb00-4f08-acfe-c91d88092d5a
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Notifications 83043789-5b67-4e6b-96e9-45d7747cbcec
Notifications updated: 83043789-5b67-4e6b-96e9-45d7747cbcec {is_read}
2026-09-27 13:03:34.340926 2026-09-27 05:03:34.340926+00
83043789-5b67-4e6b-96e9-45d7747cbcec 143.44.164.184
e95e03df-59ba-48ca-a9a3-2585a07d7771
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status
changed Appointments FRM-NNQNP8 Appointments status changed:
FRM-NNQNP8 {service_ended_at,status} 2026-09-28 15:46:08.185155
2026-09-28 07:46:08.185155+00 2a2de84e-e1d7-42e7-a36d-45905d69df12
143.44.164.66
c45e5e8e-9b6d-496c-b1ed-d8aa392afcfc
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com deleted
Appointment Services Change Oil Appointment Services deleted: Change
Oil {} 2026-09-28 15:46:08.185155 2026-09-28 07:46:08.185155+00
1e50f42c-def8-425f-a35a-fb4622f78aa0 143.44.164.66
f128eca2-6536-4248-837a-072674c9204e
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created
Appointment Services Change Oil Appointment Services created: Change
Oil {} 2026-09-28 15:46:08.185155 2026-09-28 07:46:08.185155+00
8d8f3384-c578-4611-b845-54acbd18e1b3 143.44.164.66
bb6a1fc6-0445-4c3c-88ca-209ebc0a55aa
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-09-29 08:00:32.754003 2026-09-29
00:00:32.754003+00 \N 143.44.164.123
2a1162dd-eefb-4bf0-b595-8b24f5e2e031
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status
changed Appointments FRM-XUTXNU Appointments status changed:
FRM-XUTXNU {service_ended_at,status} 2026-09-29 18:41:24.158694
2026-09-29 10:41:24.158694+00 2d1379ce-21a4-459a-a1fa-97b2f7cc75db
143.44.164.123
69154990-3828-4f3e-b01c-fd7e2c6d9d11
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-09-29 19:31:47.957258 2026-09-29
11:31:47.957258+00 \N 143.44.164.123
68007f22-6f67-44b2-a392-9e1579cc8c25
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
out Authentication Admin console Administrator was automatically signed
out after 2 minutes of inactivity. {} 2026-09-30 13:22:05.330922
2026-09-30 05:22:05.330922+00 \N 180.193.217.82
11599b43-483d-4787-86cb-689b002fd90c

7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-30 13:22:16.483525 2026-09-30 05:22:16.483525+00 \N 180.193.217.82 d0898859-894c-4a38-8085-75e6eca2a254
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-07 22:59:16.353687 2026-10-07 14:59:16.353687+00 \N 143.44.164.41 53232c87-3309-4767-b1a8-fb33b773d2bd
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-11 13:39:32.828006 2026-10-11 05:39:32.828006+00 \N 143.44.164.41 17f1e875-de18-4455-a812-4e24011dd0dd
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-10-13 21:03:54.409592 2026-10-13 13:03:54.409592+00 \N 143.44.164.41 bda7abf4-d0e1-4d46-b5ed-12478a83916e
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-15 09:44:51.828674 2026-10-15 01:44:51.828674+00 \N 143.44.164.41 5aa3e5e6-b498-46fd-93f0-51c29e3738ba
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-21 18:36:15.26928 2026-10-21 10:36:15.26928+00 \N 143.44.164.41
6a46e755-0ef8-4bc5-9aab-903c5f579c91 \N admin@gmail.com deleted Appointments FRM-TR77MJ Appointments deleted: FRM-TR77MJ {} 2026-09-15 17:18:50.528348 2026-09-15 09:18:50.528348+00 09231886-c303-437e-b12d-14f6d13a7e98 143.44.164.66
4f101264-18cc-455a-8448-9aa38ded2e83 \N admin@gmail.com deleted Appointment Services Electrical Diagnostics Appointment Services deleted: Electrical Diagnostics {} 2026-09-15 17:18:50.528348 2026-09-15 09:18:50.528348+00 8927c20a-9f5e-4b91-b0dc-d73457c67a2a 143.44.164.66
343b2627-0edd-4ccb-afda-19a855ebe1d4 \N admin@gmail.com deleted Appointments FRM-DLBU9Q Appointments deleted: FRM-DLBU9Q {} 2026-09-15 17:18:56.778962 2026-09-15 09:18:56.778962+00 20c522a1-fd55-4a8d-9104-5ba6fad8876a 143.44.164.66
ad5990f4-0d9c-4cec-b17a-14d6fe3ed685 \N admin@gmail.com deleted Appointment Services Brake Service Appointment Services deleted: Brake Service {} 2026-09-15 17:18:56.778962 2026-09-15 09:18:56.778962+00

446ea49b-2cc4-4ad0-8ec6-474fdb95d13c 143.44.164.66
d07a443e-c3b9-41c2-94da-db8c9db57d28
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
exported Activity Log PDF export Exported 10 activity records as PDF.
{export_format} 2026-09-26 10:30:10.049283 2026-09-26
02:30:10.049283+00 \N 143.44.164.123
459a326b-ddaa-40f5-97d8-bd818530d88c
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
out Authentication Admin console Administrator was automatically signed
out after 2 minutes of inactivity. {} 2026-09-27 13:05:56.851325
2026-09-27 05:05:56.851325+00 \N 143.44.164.184
f384ced8-1b9b-4701-9ad2-0b026e5fafce
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
archived Appointments FRM-6VWY2Z Appointments archived:
FRM-6VWY2Z {archived_at,is_archived} 2026-09-28 15:46:42.498402
2026-09-28 07:46:42.498402+00 6a347f55-d34f-42ff-9a73-18f6f0830092
143.44.164.66
15b05a55-f0f4-4b0c-8a80-7d1615d5fa56
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
restored Appointments FRM-6VWY2Z Appointments restored:
FRM-6VWY2Z {archived_at,is_archived} 2026-09-28 15:46:55.597088
2026-09-28 07:46:55.597088+00 6a347f55-d34f-42ff-9a73-18f6f0830092
143.44.164.66
cb6257c7-59a7-4a04-b7f7-4a67f10831e7
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
out Authentication Admin console Administrator was automatically signed
out after 2 minutes of inactivity. {} 2026-09-29 08:04:00.093568
2026-09-29 00:04:00.093568+00 \N 143.44.164.123
19a7924f-6854-44ef-a1bd-bc77b4339ed0
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status
changed Appointments FRM-746ETT Appointments status changed:
FRM-746ETT {service_ended_at,status} 2026-09-29 18:41:26.216923
2026-09-29 10:41:26.216923+00 6957127d-9bdd-4173-a856-2f08819ce87f
143.44.164.123
d7103e82-6671-4dad-a297-1772f8486e6f
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-09-29 19:48:33.330882 2026-09-29
11:48:33.330882+00 \N 143.44.164.123
79307a63-ff19-440c-b965-2d7086e10804
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-10-04 18:36:15.536817 2026-10-04
10:36:15.536817+00 \N 143.44.164.41
a1584eb6-0002-44fa-ae29-5cca43577f76

7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed Reports Booking Report Opened the print dialog for 9 bookings. {report_type,date_range,print_format} 2026-10-07 22:59:28.559096 2026-10-07 14:59:28.559096+00 \N 143.44.164.41 9b3430ef-ac97-4726-92e9-0a066f7be349
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-EVXKHD Appointments status changed: FRM-EVXKHD {service_ended_at,status} 2026-10-11 13:39:40.891101 2026-10-11 05:39:40.891101+00 30c80b92-c789-40fc-936d-5f6567091423 143.44.164.41 338b2c80-002d-4f5d-afc5-ac11bbdf9fdd
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Notifications 2fffee3a-f091-4753-91cd-6a5c99d21a3c Notifications updated: 2fffee3a-f091-4753-91cd-6a5c99d21a3c {is_read} 2026-10-11 13:41:01.118085 2026-10-11 05:41:01.118085+00 2fffee3a-f091-4753-91cd-6a5c99d21a3c 143.44.164.41 ef6fc506-58d6-4a45-86ec-a372e6d7d23c
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Notifications 44f6ff7e-4353-40e4-8021-6cdc86f2d231 Notifications updated: 44f6ff7e-4353-40e4-8021-6cdc86f2d231 {is_read} 2026-10-11 13:41:01.118085 2026-10-11 05:41:01.118085+00 44f6ff7e-4353-40e4-8021-6cdc86f2d231 143.44.164.41 d171a406-d7b6-4d3d-8d5c-25593ce1ec80
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Notifications bb429244-23a2-413d-826d-043f2ca233d8 Notifications updated: bb429244-23a2-413d-826d-043f2ca233d8 {is_read} 2026-10-11 13:41:01.118085 2026-10-11 05:41:01.118085+00 bb429244-23a2-413d-826d-043f2ca233d8 143.44.164.41 881933ff-1756-45a3-a014-b9c896382b90
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-13 21:06:41.764136 2026-10-13 13:06:41.764136+00 \N 143.44.164.41 6adce449-2442-4299-82a6-8b657b00b45a
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-P5ZX4N Appointments status changed: FRM-P5ZX4N {service_started_at,status} 2026-10-15 09:50:42.635524 2026-10-15 01:50:42.635524+00 3eac3b68-b97b-46f0-a8eb-ad913f8bc8c6 143.44.164.41 12bc4d41-aaaa-4b0f-ad2d-fba41b5a7a61
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-21 18:43:00.712393 2026-10-21 10:43:00.712393+00 \N 143.44.164.41 8a2135b8-942a-4799-98b5-5f803813093c \N admin@gmail.com deleted

Appointments FRM-QAYJ78 Appointments deleted: FRM-QAYJ78 {} 2026-09-15 17:18:53.720234 2026-09-15 09:18:53.720234+00 6a450dde-e1ad-45d2-b1cb-0c5d13597c0b 143.44.164.66 106dfb8e-e913-4073-b951-15aaa525b4ab \N admin@gmail.com deleted Appointment Services Brake Service Appointment Services deleted: Brake Service {} 2026-09-15 17:18:53.720234 2026-09-15 09:18:53.720234+00 dacf6c70-b6e2-489c-a08b-e13bd22e6b5d 143.44.164.66 de33a91f-d055-410c-8d78-9188d1426fe0 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com exported Reports Booking Report Exported 12 bookings as PDF. {report_type,date_range,export_format} 2026-09-26 10:32:28.459484 2026-09-26 02:32:28.459484+00 \N 143.44.164.123 5e6c3b2f-54e3-4fff-a419-103e7a382df8 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-27 18:05:20.100007 2026-09-27 10:05:20.100007+00 \N 143.44.164.184 392308e5-f18c-490c-b70a-d654fbf7c45c 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-28 15:51:53.890169 2026-09-28 07:51:53.890169+00 \N 143.44.164.66 d9499ba0-99ad-40e0-9ace-a04d195a40c0 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-29 08:04:17.716877 2026-09-29 00:04:17.716877+00 \N 143.44.164.123 0b3eb7ea-44b9-46d9-8381-41ff66c5e669 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Appointments FRM-P5ZX4N Appointments updated: FRM-P5ZX4N {admin_notes} 2026-09-29 18:44:13.945828 2026-09-29 10:44:13.945828+00 3eac3b68-b97b-46f0-a8eb-ad913f8bc8c6 143.44.164.123 555be4cf-4df7-4adf-bac4-5721b8bce4cb 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com deleted Appointment Services Change Oil Appointment Services deleted: Change Oil {} 2026-09-29 18:44:13.945828 2026-09-29 10:44:13.945828+00 c868d3c7-0db0-4866-a607-5c4cc2dfaebd 143.44.164.123 98f9f5f4-50fa-461a-9be6-a554444b352f 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com deleted Appointment Services Brake Service Appointment Services deleted: Brake Service {} 2026-09-29 18:44:13.945828 2026-09-29 10:44:13.945828+00 843b3f54-1b42-4e0a-85df-bbf03571693b 143.44.164.123 dc5b476e-a540-4453-88de-77d2e8e96527 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created

Appointment Services Change Oil Appointment Services created: Change Oil {} 2026-09-29 18:44:13.945828 2026-09-29 10:44:13.945828+00 e86b68a6-9682-481d-b589-bee470c9f183 143.44.164.123 f886119a-9934-4695-8927-9cb05152e4ce 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Appointment Services Brake Service Appointment Services created: Brake Service {} 2026-09-29 18:44:13.945828 2026-09-29 10:44:13.945828+00 10c152f5-bc08-4e57-a976-6c160548e1a9 143.44.164.123 58015bee-f64f-418d-a4e9-943bf3a52438 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com deleted Appointment Services Change Oil Appointment Services deleted: Change Oil {} 2026-09-29 18:44:50.982917 2026-09-29 10:44:50.982917+00 09838e4f-76e7-47bd-8b64-1e76c4e6d0b8 143.44.164.123 521cc645-b348-4b36-a13e-96e6e8d64dc1 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Appointment Services Change Oil Appointment Services created: Change Oil {} 2026-09-29 18:44:50.982917 2026-09-29 10:44:50.982917+00 5d3655fa-7d00-437e-91a8-7818a57a0073 143.44.164.123 182c83ff-d0a7-4f81-b9a2-99b6ba8ee29f 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Appointments FRM-EVXKHD Appointments updated: FRM-EVXKHD {phone} 2026-09-29 18:44:50.982917 2026-09-29 10:44:50.982917+00 30c80b92-c789-40fc-936d-5f6567091423 143.44.164.123 7981454a-3993-4e65-a29a-ff353b34ebf5 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator signed out of the admin console. {} 2026-09-29 19:53:01.050968 2026-09-29 11:53:01.050968+00 \N 143.44.164.123 03f69231-e6e2-495f-ae83-dd7ab5ff436b 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Notifications d217118f-855d-485e-9f54-124efa2741ff Notifications updated: d217118f-855d-485e-9f54-124efa2741ff {is_read} 2026-10-04 18:36:32.361645 2026-10-04 10:36:32.361645+00 d217118f-855d-485e-9f54-124efa2741ff 143.44.164.41 2f08bff1-8c96-4076-b2cd-ec45c3c944a5 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Notifications 628466d1-e842-43f8-92da-495c3643d264 Notifications updated: 628466d1-e842-43f8-92da-495c3643d264 {is_read} 2026-10-04 18:36:32.361645 2026-10-04 10:36:32.361645+00 628466d1-e842-43f8-92da-495c3643d264 143.44.164.41 bb650b4c-d492-4feb-be9a-81455dc9541e 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Notifications 2ca39f4f-f9b7-4e78-8bc0-2325f5021e14 Notifications updated: 2ca39f4f-f9b7-4e78-8bc0-2325f5021e14 {is_read} 2026-10-04

18:36:32.361645 2026-10-04 10:36:32.361645+00
2ca39f4f-f9b7-4e78-8bc0-2325f5021e14 143.44.164.41
66dd9834-6283-42b6-8831-e025ed8e4b97
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Notifications b7c59277-b8ae-4fb5-b801-021d4a2f2460
Notifications updated: b7c59277-b8ae-4fb5-b801-021d4a2f2460 {is_read}
2026-10-04 18:36:32.361645 2026-10-04 10:36:32.361645+00
b7c59277-b8ae-4fb5-b801-021d4a2f2460 143.44.164.41
eb86c3f2-6fff-4ba9-ad13-197ce600deeb
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Notifications dc396090-b3b7-4506-b00f-9027c731fd5e
Notifications updated: dc396090-b3b7-4506-b00f-9027c731fd5e {is_read}
2026-10-04 18:36:32.361645 2026-10-04 10:36:32.361645+00
dc396090-b3b7-4506-b00f-9027c731fd5e 143.44.164.41
23bbae34-42ee-4686-9af2-7bd5d598009c
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Notifications 8222072d-d208-451b-806a-68afff124dda
Notifications updated: 8222072d-d208-451b-806a-68afff124dda {is_read}
2026-10-04 18:36:32.361645 2026-10-04 10:36:32.361645+00
8222072d-d208-451b-806a-68afff124dda 143.44.164.41
6eb6e577-25e4-4aa8-a50a-9cd391d44e39
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-10-08 09:39:21.349763 2026-10-08
01:39:21.349763+00 \N 143.44.164.41
ee749b3a-694a-4eac-bc2b-1772beb82c13
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status
changed Appointments FRM-APD3BN Appointments status changed:
FRM-APD3BN {status} 2026-10-11 13:39:48.211457 2026-10-11
05:39:48.211457+00 cab87474-af88-46f2-8e66-7acd2e5d2baa
143.44.164.41
184ee3ba-7cb6-455b-98ae-8f01e290ff6a
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
out Authentication Admin console Administrator was automatically signed
out after 2 minutes of inactivity. {} 2026-10-13 21:10:00.501952
2026-10-13 13:10:00.501952+00 \N 143.44.164.41
2ed85987-3015-4fa5-8738-8e137e90732e
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed
Reports Financial Report (Summary) Opened the print dialog for 1
completed transactions. {report_type,date_range,print_format} 2026-10-15
09:53:29.66443 2026-10-15 01:53:29.66443+00 \N 143.44.164.41
dce36e29-371d-4214-a4c9-53aa9f238810
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed
Reports Financial Report (Summary) Opened the print dialog for 7
completed transactions. {report_type,date_range,print_format} 2026-10-21

18:43:18.153116 2026-10-21 10:43:18.153116+00 \N 143.44.164.41
02ef6ae1-639a-4faa-90fb-b10cbe619c87
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
out Authentication Admin console Administrator was automatically signed
out after 2 minutes of inactivity. {} 2026-09-26 10:35:45.536151
2026-09-26 02:35:45.536151+00 \N 143.44.164.123
cd080ca1-f4a4-4756-b55c-9fb5703d1973
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-09-27 18:08:10.193307 2026-09-27
10:08:10.193307+00 \N 143.44.164.184
e8aee5ee-e965-4190-8539-2ebe964c23c2
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-09-28 15:55:12.991413 2026-09-28
07:55:12.991413+00 \N 143.44.164.66
e81cdd2e-105f-4111-9e5c-f32005a6b56d
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
out Authentication Admin console Administrator was automatically signed
out after 2 minutes of inactivity. {} 2026-09-29 08:11:43.495415
2026-09-29 00:11:43.495415+00 \N 143.44.164.123
11d22dee-3f86-4ce5-85af-14f58a675b08
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com deleted
Appointment Services Change Oil Appointment Services deleted: Change
Oil {} 2026-09-29 18:45:31.142156 2026-09-29 10:45:31.142156+00
36739f70-12e7-4ec1-9186-651386754afe 143.44.164.123
27084076-5ac8-412a-be09-b4cf8586cfa0
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created
Appointment Services Change Oil Appointment Services created: Change
Oil {} 2026-09-29 18:45:31.142156 2026-09-29 10:45:31.142156+00
725348ca-f664-42f4-964b-78be9c51116e 143.44.164.123
cbebc2d1-07c5-4d34-92f3-f3232d706496
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Appointments FRM-ZTUG43 Appointments updated:
FRM-ZTUG43 {phone} 2026-09-29 18:45:31.142156 2026-09-29
10:45:31.142156+00 a26950f7-62f0-439a-8f3b-07fa6a6c50d6
143.44.164.123
e038b3e6-81ff-4e84-a298-b6a2bb4fdde2
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-09-29 19:57:37.45247 2026-09-29 11:57:37.45247+00
\N 162.4.58.134
3b4b2231-5dca-4df4-8a51-8c0dc9aa4f75
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
out Authentication Admin console Administrator was automatically signed

out after 2 minutes of inactivity. {} 2026-10-04 18:39:50.532851 2026-10-04 10:39:50.532851+00 \N 143.44.164.41 db68a6a5-e2ea-45b9-ad2f-aaa4f03b6edb 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Crew Schedules 0e9dca6c-7e0f-4a2f-9c7f-b9499c082508 Crew Schedules created: 0e9dca6c-7e0f-4a2f-9c7f-b9499c082508 {} 2026-10-08 09:40:13.199523 2026-10-08 01:40:13.199523+00 0e9dca6c-7e0f-4a2f-9c7f-b9499c082508 143.44.164.41 ea74534b-7263-4fb4-8b96-12137315d84f 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Crew Schedules 52685efe-b1f3-46fc-b53c-fd7f9fb01b86 Crew Schedules created: 52685efe-b1f3-46fc-b53c-fd7f9fb01b86 {} 2026-10-08 09:40:13.199523 2026-10-08 01:40:13.199523+00 52685efe-b1f3-46fc-b53c-fd7f9fb01b86 143.44.164.41 512c0820-f567-4737-9bce-53e83f3da51a 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Crew Schedules e062b6ba-6c07-4b94-9fc6-773002745428 Crew Schedules created: e062b6ba-6c07-4b94-9fc6-773002745428 {} 2026-10-08 09:40:13.199523 2026-10-08 01:40:13.199523+00 e062b6ba-6c07-4b94-9fc6-773002745428 143.44.164.41 b7583af5-6a7f-4df6-9264-f12a1ef880af 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Crew Schedules 56679865-c42c-41e1-8f2d-efcaa028256f Crew Schedules created: 56679865-c42c-41e1-8f2d-efcaa028256f {} 2026-10-08 09:40:13.199523 2026-10-08 01:40:13.199523+00 56679865-c42c-41e1-8f2d-efcaa028256f 143.44.164.41 f8835556-142d-4fdc-b8b8-2d8c4308bc4b 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Crew Schedules cb7c86bd-1a47-4865-a4c4-63d74a2fa10b Crew Schedules created: cb7c86bd-1a47-4865-a4c4-63d74a2fa10b {} 2026-10-08 09:40:13.199523 2026-10-08 01:40:13.199523+00 cb7c86bd-1a47-4865-a4c4-63d74a2fa10b 143.44.164.41 afb3dd61-b1d1-4d9a-9602-9915a5e00bdd 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Crew Schedules 648159b4-7dd0-43d1-8714-f5782393d192 Crew Schedules created: 648159b4-7dd0-43d1-8714-f5782393d192 {} 2026-10-08 09:40:13.199523 2026-10-08 01:40:13.199523+00 648159b4-7dd0-43d1-8714-f5782393d192 143.44.164.41 868b0a77-904e-4bc1-b613-332342c1e8aa 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Crew Schedules b1fdecd7-5966-44b4-bdf6-23fd39be03e6 Crew Schedules created: b1fdecd7-5966-44b4-bdf6-23fd39be03e6 {} 2026-10-08 09:40:13.199523 2026-10-08 01:40:13.199523+00 b1fdecd7-5966-44b4-bdf6-23fd39be03e6 143.44.164.41 a9056fa3-d6e1-4e35-9249-b234051032ad

7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Crew Schedules 51c825a2-1872-4277-81e6-df395fdac414 Crew Schedules created: 51c825a2-1872-4277-81e6-df395fdac414 {} 2026-10-08 09:40:13.199523 2026-10-08 01:40:13.199523+00 51c825a2-1872-4277-81e6-df395fdac414 143.44.164.41 7e4879ea-1423-4efb-9bde-c2c7e993e3dc
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Crew Schedules 4ae42290-a2ae-4593-a39c-8d53c99cb2d1 Crew Schedules created: 4ae42290-a2ae-4593-a39c-8d53c99cb2d1 {} 2026-10-08 09:40:13.199523 2026-10-08 01:40:13.199523+00 4ae42290-a2ae-4593-a39c-8d53c99cb2d1 143.44.164.41 c3a36a51-6a93-45ee-8823-9c8aaf58aaa9
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Crew Schedules 22ba0bdc-8923-4ccd-abb3-5e0161de5507 Crew Schedules created: 22ba0bdc-8923-4ccd-abb3-5e0161de5507 {} 2026-10-08 09:40:13.199523 2026-10-08 01:40:13.199523+00 22ba0bdc-8923-4ccd-abb3-5e0161de5507 143.44.164.41 07e177f2-7df2-4e1b-ba72-17b2206ed8aa
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Crew Schedules c461988d-701e-4874-b316-60fda89c3746 Crew Schedules created: c461988d-701e-4874-b316-60fda89c3746 {} 2026-10-08 09:40:13.199523 2026-10-08 01:40:13.199523+00 c461988d-701e-4874-b316-60fda89c3746 143.44.164.41 89742792-75f1-4086-a95a-ddb2b6024407
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Crew Schedules 559feaf3-f1fc-48de-b3b0-d72c49f48a26 Crew Schedules created: 559feaf3-f1fc-48de-b3b0-d72c49f48a26 {} 2026-10-08 09:40:13.199523 2026-10-08 01:40:13.199523+00 559feaf3-f1fc-48de-b3b0-d72c49f48a26 143.44.164.41 59157cfc-f24f-46c7-97c5-055bd4c488dd
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Crew Schedules 06f52ecc-c4eb-4c4b-b4e1-8e15dd9698ca Crew Schedules created: 06f52ecc-c4eb-4c4b-b4e1-8e15dd9698ca {} 2026-10-08 09:40:13.199523 2026-10-08 01:40:13.199523+00 06f52ecc-c4eb-4c4b-b4e1-8e15dd9698ca 143.44.164.41 a6613ca0-7f2c-490d-af6b-19c412ba87f4
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Crew Schedules d31c7117-abb0-4db0-9ec3-bcf665f9a0aa Crew Schedules created: d31c7117-abb0-4db0-9ec3-bcf665f9a0aa {} 2026-10-08 09:40:13.199523 2026-10-08 01:40:13.199523+00 d31c7117-abb0-4db0-9ec3-bcf665f9a0aa 143.44.164.41 f64d51be-c287-43cb-9fd2-f2ffbfb25e6a
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Crew Schedules cb0c7605-a94a-4e04-8c61-6c0d3ebb0967 Crew Schedules created: cb0c7605-a94a-4e04-8c61-6c0d3ebb0967 {} 2026-10-08

09:40:13.199523 2026-10-08 01:40:13.199523+00
cb0c7605-a94a-4e04-8c61-6c0d3ebb0967 143.44.164.41
43a44374-40ad-41d7-8c2c-d6a5ef2ae868
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created
Crew Schedules 22050d84-172f-479f-af9a-78b70419e9fd Crew Schedules
created: 22050d84-172f-479f-af9a-78b70419e9fd {} 2026-10-08
09:40:13.199523 2026-10-08 01:40:13.199523+00
22050d84-172f-479f-af9a-78b70419e9fd 143.44.164.41
77aeeec7-739b-4a70-aadc-eb4afb83f0d9
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created
Crew Schedules a03cd237-7c85-4fd5-8b23-6d0093adb30f Crew
Schedules created: a03cd237-7c85-4fd5-8b23-6d0093adb30f {} 2026-10-08
09:40:13.199523 2026-10-08 01:40:13.199523+00
a03cd237-7c85-4fd5-8b23-6d0093adb30f 143.44.164.41
b83a9532-5675-44cd-88cd-da516b3e7814
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created
Crew Schedules be7b4593-9deb-46e7-9787-86cd3ae0738d Crew
Schedules created: be7b4593-9deb-46e7-9787-86cd3ae0738d {} 2026-10-08
09:40:13.199523 2026-10-08 01:40:13.199523+00
be7b4593-9deb-46e7-9787-86cd3ae0738d 143.44.164.41
35d81f4b-0d54-42b8-9d0d-a6a5bf5d3239
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
out Authentication Admin console Administrator was automatically signed
out after 2 minutes of inactivity. {} 2026-10-11 13:43:51.285849
2026-10-11 05:43:51.285849+00 \N 143.44.164.41
b9205cc7-b7fd-4b9f-a73e-ff982307a911
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-10-14 14:23:26.004437 2026-10-14
06:23:26.004437+00 \N 143.44.164.41
3ddbf8b4-0153-43dd-bb01-8d9d93fd440c
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-10-15 10:19:18.980315 2026-10-15
02:19:18.980315+00 \N 143.44.164.41
5cf9740d-357d-4db5-b5f0-24cbe30e59dc
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-10-21 18:47:20.317395 2026-10-21
10:47:20.317395+00 \N 143.44.164.41
e6e3f19b-d5e3-4f05-969b-8bf094f216e0
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-09-26 14:46:44.920326 2026-09-26
06:46:44.920326+00 \N 115.147.51.76

23995fec-95c8-49ec-addb-a0e8d4b4c15d
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-27 18:10:45.078776 2026-09-27 10:10:45.078776+00 \N 143.44.164.184
d8602613-b72c-4bc5-a315-2c5fc7d49756
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-28 15:58:32.058341 2026-09-28 07:58:32.058341+00 \N 143.44.164.66
2ad621db-ac26-43e4-b330-048e6b31d989
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-29 08:18:23.807784 2026-09-29 00:18:23.807784+00 \N 143.44.164.123
83bcaf95-1da8-44f3-bed5-be4ff7e673ad
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com archived Appointments FRM-VVUZEP Appointments archived: FRM-VVUZEP {archived_at,is_archived} 2026-09-29 18:46:09.103228 2026-09-29 10:46:09.103228+00 3e260213-1399-48e6-9b69-1e74bf4e0000 143.44.164.123
1320cd0b-6557-48ee-8c24-3cffeec0fb34
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Products Wheel Bearing Products updated: Wheel Bearing {stock_quantity} 2026-09-29 18:47:20.992765 2026-09-29 10:47:20.992765+00 4d351a12-bead-4422-8f85-d6070c853ca8 143.44.164.123
1df66b52-0e25-4be1-a16f-d79825be1d07
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-29 21:35:54.847051 2026-09-29 13:35:54.847051+00 \N 162.4.58.134
267fb005-4556-47da-933c-4a8ee360cf0c
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-06 20:55:36.913274 2026-10-06 12:55:36.913274+00 \N 143.44.164.41
b47ae1dc-96ff-4bec-9623-6d1c2b838ff1
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator signed out before visiting the public site. {} 2026-10-08 09:40:57.184956 2026-10-08 01:40:57.184956+00 \N 143.44.164.41
4cb4a9f2-7edc-4cd7-b3a5-29802066c240 \N product-rpc-test-de0ed6dd-c52f-4e6f-ac04-1cd683a0f4a3@example.invalid updated Appointments TST-BDACA55B Appointments updated:

TST-BDACA55B {total_estimate} 2026-10-11 13:53:13.258199 2026-10-11 05:53:13.258199+00 0c4bd788-bcbf-44c7-83a7-fbca21e1481d 143.44.164.41 cd6ba3cd-a8ad-458b-9b9f-382e40cdc3a7 \N product-rpc-test-de0ed6dd-c52f-4e6f-ac04-1cd683a0f4a3@example.invalid created Appointment Services Change Oil Appointment Services created: Change Oil {} 2026-10-11 13:53:13.258199 2026-10-11 05:53:13.258199+00 119b6845-f891-4151-bbd0-34e39b03d96f 143.44.164.41 38b241e4-3578-488f-ae4e-5a938790bd62 \N product-rpc-test-de0ed6dd-c52f-4e6f-ac04-1cd683a0f4a3@example.invalid updated Appointments TST-BDACA55B Appointments updated: TST-BDACA55B {customer_name} 2026-10-11 13:53:13.258199 2026-10-11 05:53:13.258199+00 0c4bd788-bcbf-44c7-83a7-fbca21e1481d 143.44.164.41 06c4c476-9409-4495-ba00-085d71938c0b \N product-rpc-test-de0ed6dd-c52f-4e6f-ac04-1cd683a0f4a3@example.invalid updated Products Wheel Bearing Products updated: Wheel Bearing {stock_quantity} 2026-10-11 13:53:13.258199 2026-10-11 05:53:13.258199+00 4d351a12-bead-4422-8f85-d6070c853ca8 143.44.164.41 fda1ef1e-a5dc-4aaf-82be-2a5f40a800a3 \N product-rpc-test-de0ed6dd-c52f-4e6f-ac04-1cd683a0f4a3@example.invalid updated Products Wheel Bearing Products updated: Wheel Bearing {stock_quantity} 2026-10-11 13:53:13.895005 2026-10-11 05:53:13.895005+00 4d351a12-bead-4422-8f85-d6070c853ca8 143.44.164.41 a9328d5b-2564-43a1-9a56-3d1c21d28a4c \N product-rpc-test-de0ed6dd-c52f-4e6f-ac04-1cd683a0f4a3@example.invalid status changed Appointments TST-BDACA55B Appointments status changed: TST-BDACA55B {status} 2026-10-11 13:53:13.895005 2026-10-11 05:53:13.895005+00 0c4bd788-bcbf-44c7-83a7-fbca21e1481d 143.44.164.41 011921df-15f4-4766-b5c9-8f703926eef2 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-14 14:44:15.642679 2026-10-14 06:44:15.642679+00 \N 143.44.164.41 d5aa029a-afbe-49c0-8bad-b00cd2985409 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed Reports Financial Report (Summary) Opened the print dialog for 4 completed transactions. {report_type,date_range,print_format} 2026-10-15 10:19:25.613555 2026-10-15 02:19:25.613555+00 \N 143.44.164.41 43801262-13a7-4ae8-8d49-498c21ad7843 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed

Reports Financial Report (Summary) Opened the print dialog for 1 completed transactions. {report_type,date_range,print_format} 2026-10-15 10:20:05.464269 2026-10-15 02:20:05.464269+00 \N 143.44.164.41 bed27014-55ad-40f2-b0fd-f98397e3b67b 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator signed out before visiting the public site. {} 2026-10-15 10:21:39.740221 2026-10-15 02:21:39.740221+00 \N 143.44.164.41 40c70f6b-b9ec-414f-959a-3263a163aab8 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed Reports Financial Report Opened the print dialog for 7 completed transactions. {report_type,date_range,print_format} 2026-10-21 18:47:28.441727 2026-10-21 10:47:28.441727+00 \N 143.44.164.41 087efc01-7738-40f5-9277-76b11add9f86 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-XUTXNU Appointments status changed: FRM-XUTXNU {status} 2026-09-26 14:47:51.392975 2026-09-26 06:47:51.392975+00 2d1379ce-21a4-459a-a1fa-97b2f7cc75db 115.147.51.76 d28581fc-39c9-4b50-a4a2-a4aa71ce45b8 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator signed out of the admin console. {} 2026-09-27 18:59:52.149604 2026-09-27 10:59:52.149604+00 \N 143.44.164.184 58c5ff68-4c6b-4f92-9c8c-0d88a40fe3fe 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-27 19:00:06.550848 2026-09-27 11:00:06.550848+00 \N 143.44.164.184 2fa00341-982a-4921-ba7b-1fea74fe0e09 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-28 16:07:12.721599 2026-09-28 08:07:12.721599+00 \N 143.44.164.66 e7ba606f-1e75-47c3-8be7-824e180dee5e 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-29 08:36:17.551863 2026-09-29 00:36:17.551863+00 \N 143.44.164.123 1f680a1e-f4c7-44ad-ac98-4b269425405d 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com archived Appointments FRM-P35N26 Appointments archived: FRM-P35N26 {archived_at,is_archived} 2026-09-29 18:46:24.884042 2026-09-29 10:46:24.884042+00 4578f9c5-5566-4966-9e56-5f34184be688 143.44.164.123

c4878a49-d2d4-4b1e-af8c-eb52345961f8
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Products Handlebar Grips Products updated: Handlebar Grips
{stock_quantity} 2026-09-29 18:47:11.888182 2026-09-29
10:47:11.888182+00 7403d2b3-6533-4e2a-aa1a-49d0f8babf93
143.44.164.123
c735c79f-b676-47bf-8b67-2564792e1e28
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Products Ohlins Rear Shock Absorber Products updated: Ohlins
Rear Shock Absorber {stock_quantity} 2026-09-29 18:47:29.384998
2026-09-29 10:47:29.384998+00 656db2e0-7cb6-4311-8271-3e15430d31b3
143.44.164.123
ed1c65e0-eee3-4c46-80b9-215c63a30b07
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-09-29 23:44:47.92976 2026-09-29 15:44:47.92976+00
\N 136.158.3.55
5fc06a04-1524-4f04-997d-5c445ae14cda
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-10-06 20:59:36.310333 2026-10-06
12:59:36.310333+00 \N 143.44.164.41
86eecda8-3511-4724-ac9b-3e46daa6e92f
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-10-08 09:41:34.478313 2026-10-08
01:41:34.478313+00 \N 143.44.164.41
5c1315a4-471c-4c51-bff9-51d440a8bc6b
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-10-11 13:56:48.764476 2026-10-11
05:56:48.764476+00 \N 143.44.164.41
0aa0fafb-2513-42fc-93c5-28e0b1511799
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Services Change Oil Services updated: Change Oil
{duration_minutes,large_bike_duration_minutes,large_bike_price} 2026-10-14
14:46:15.238796 2026-10-14 06:46:15.238796+00
97a9714b-20e6-4402-a3ee-4fcdefc1e9db 143.44.164.41
cf3c944d-f6fe-4c53-a7d0-2ee06ed0126d
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-10-15 14:32:09.93748 2026-10-15 06:32:09.93748+00
\N 143.44.164.41
e6abf4f6-bc99-4e46-90c0-b8761f297894
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed

Reports Financial Report Opened the print dialog for 7 completed transactions. {report_type,date_range,print_format} 2026-10-21 18:49:13.035599 2026-10-21 10:49:13.035599+00 \N 143.44.164.41 9ad4f4c6-2563-474b-9c55-e5c4df8b09e9 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-26 14:49:40.412588 2026-09-26 06:49:40.412588+00 \N 143.44.164.123 e7edd003-5a61-4ce5-aab1-b7a9a8f83bd1 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-27 19:02:07.374187 2026-09-27 11:02:07.374187+00 \N 143.44.164.184 89e83173-0e04-4dbc-9c77-3928e4d72165 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Motorcycle General Inspection Services updated: Motorcycle General Inspection {price} 2026-09-28 16:08:58.905061 2026-09-28 08:08:58.905061+00 7f2b49e7-8c12-44d3-9d1f-3cb79c8898ec 143.44.164.66 a5bc2919-80d9-4422-800a-7236bb28297d 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com exported Reports Booking Report Exported 8 bookings as PDF. {report_type,date_range,export_format} 2026-09-29 08:37:49.14162 2026-09-29 00:37:49.14162+00 \N 143.44.164.123 2994d74d-15ab-4cd4-bb82-7d5a641b9136 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Products Brake pads Products updated: Brake pads {stock_quantity} 2026-09-29 18:46:52.521733 2026-09-29 10:46:52.521733+00 9310ef87-578e-4fb2-b95b-725a0900c95a 143.44.164.123 05a79e05-dcfd-4c67-9bcd-96ae303b12a4 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-29 23:48:09.602301 2026-09-29 15:48:09.602301+00 \N 136.158.3.55 24b97dd3-2317-4c25-82dd-e45179fda5d0 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-10-06 21:04:27.572615 2026-10-06 13:04:27.572615+00 \N 143.44.164.41 20de090b-a902-46e1-b3c0-d903ce4c7f62 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-08 09:52:17.360479 2026-10-08 01:52:17.360479+00 \N 143.44.164.41

3c96cb44-64d1-4d52-bc40-64f8e21c1609
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator signed out before visiting the public site. {} 2026-10-08 09:52:31.147791 2026-10-08 01:52:31.147791+00 \N 143.44.164.41
caf4bb5e-ce1c-45ed-b5cd-8cbf23435f99
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com deleted Appointment Services Change Oil Appointment Services deleted: Change Oil {} 2026-10-11 13:58:05.259017 2026-10-11 05:58:05.259017+00
2d3cfe87-fffe-4e58-91a9-9edb845b0e1f 143.44.164.41
1e88e6b6-e3fb-452a-8d45-69e69533bf67
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Appointment Services Change Oil Appointment Services created: Change Oil {} 2026-10-11 13:58:05.259017 2026-10-11 05:58:05.259017+00
c95c7ae4-7459-4e23-9c42-0d80ccbac457 143.44.164.41
411e2d6e-ef6b-4bde-a666-6621c1453679
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Products Wheel Bearing Products updated: Wheel Bearing {stock_quantity} 2026-10-11 13:58:05.259017 2026-10-11 05:58:05.259017+00 4d351a12-bead-4422-8f85-d6070c853ca8 143.44.164.41
2daee5c8-0d9e-4af5-9cbf-4dcf1c4db1c8
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Engine Overhaul Services updated: Engine Overhaul {large_bike_duration_minutes,large_bike_price} 2026-10-14 14:47:26.326545 2026-10-14 06:47:26.326545+00 f8feeecc-86e2-4312-8798-ea5c144f1ad4 143.44.164.41
b6d363c5-2351-44f5-8038-c7bdfa642ca1
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-P5ZX4N Appointments status changed: FRM-P5ZX4N {service_ended_at,status} 2026-10-15 14:32:42.283661 2026-10-15 06:32:42.283661+00 3eac3b68-b97b-46f0-a8eb-ad913f8bc8c6 143.44.164.41
82f09b24-5448-43d4-9220-cbcb16a38584
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator signed out before visiting the public site. {} 2026-10-15 14:34:20.599109 2026-10-15 06:34:20.599109+00 \N 143.44.164.41
f5c60edc-515c-4aea-ae86-dc1a992ed080
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed Reports Financial Report Opened the print dialog for 7 completed transactions. {report_type,date_range,print_format} 2026-10-21 18:49:30.615853 2026-10-21 10:49:30.615853+00 \N 143.44.164.41
f7ebde02-d532-42a5-9317-80ec039b5977
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed

Reports Financial Report Opened the print dialog for 7 completed transactions. {report_type,date_range,print_format} 2026-10-21 18:50:47.009103 2026-10-21 10:50:47.009103+00 \N 143.44.164.41 c99c30ff-969f-46a1-b48b-222e94cb5663 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-26 14:55:09.864132 2026-09-26 06:55:09.864132+00 \N 115.147.51.76 57eff1d2-ec33-43b7-8fce-9754bb1ca422 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-27 19:02:24.585518 2026-09-27 11:02:24.585518+00 \N 143.44.164.184 0adfeb0f-7ca1-4a96-8581-f038542bc630 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created Price History 72b6e542-cdd0-471e-8352-ff00bb4f2b53 Price History created: 72b6e542-cdd0-471e-8352-ff00bb4f2b53 {} 2026-09-28 16:08:59.311974 2026-09-28 08:08:59.311974+00 72b6e542-cdd0-471e-8352-ff00bb4f2b53 143.44.164.66 d21103fd-b678-4196-acbf-dd0404503918 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-29 08:41:54.655688 2026-09-29 00:41:54.655688+00 \N 143.44.164.123 bcea866a-ed7e-4b6e-824e-c0150d3068da 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Products Ohlins Rear Shock Absorber Products updated: Ohlins Rear Shock Absorber {image_url} 2026-09-29 18:47:50.078598 2026-09-29 10:47:50.078598+00 656db2e0-7cb6-4311-8271-3e15430d31b3 143.44.164.123 bb38f411-a917-462e-b08d-9614f4fe55e6 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-30 06:11:50.36508 2026-09-29 22:11:50.36508+00 \N 143.44.164.66 c6a90662-bd62-48d5-8c88-2ed3255cbcfa 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-06 21:15:22.157707 2026-10-06 13:15:22.157707+00 \N 143.44.164.41 794cd664-90f6-4688-a89d-5dda5d1327cb 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-08 10:10:52.042704 2026-10-08 02:10:52.042704+00 \N 143.44.164.41

fe01a73e-6cb9-445b-86ea-2cd5420ba038
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed Reports Financial Report (Summary) Opened the print dialog for 3 completed transactions. {report_type,date_range,print_format} 2026-10-11 13:59:31.464987 2026-10-11 05:59:31.464987+00 \N 143.44.164.41
48652e24-8964-499b-8e10-c63232f6b580
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-10-14 14:49:51.199717 2026-10-14 06:49:51.199717+00 \N 143.44.164.41
39b73f31-812f-4262-90b3-516357b5ec49
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-15 14:34:39.119162 2026-10-15 06:34:39.119162+00 \N 143.44.164.41
2c04e41a-8fd0-4880-93f1-38febc4d5745
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed Reports Financial Report Opened the print dialog for 7 completed transactions. {report_type,date_range,print_format} 2026-10-21 18:51:01.029552 2026-10-21 10:51:01.029552+00 \N 143.44.164.41
2e01f6ed-d72a-4d2d-88a3-5a5b338ef3f7
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-26 14:55:58.096165 2026-09-26 06:55:58.096165+00 \N 115.147.51.76
bd7bfd33-3dad-4ad9-a034-9fff3fad8e85
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-27 19:04:52.051451 2026-09-27 11:04:52.051451+00 \N 143.44.164.184
8ee63587-fc5a-45b6-89d4-f5381a7903a8
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Motorcycle General Inspection Services updated: Motorcycle General Inspection {description} 2026-09-28 16:11:26.619282 2026-09-28 08:11:26.619282+00 7f2b49e7-8c12-44d3-9d1f-3cb79c8898ec 143.44.164.66
aed6be11-6c2a-42ef-80fc-fecdf1693810
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-29 08:50:19.324871 2026-09-29 00:50:19.324871+00 \N 143.44.164.123
dbd3e0db-e416-4ecb-b307-ff08a061b5bc
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-XUTXNU Appointments status changed: FRM-XUTXNU {service_started_at,status} 2026-09-29 08:50:26.944452

2026-09-29 00:50:26.944452+00 2d1379ce-21a4-459a-a1fa-97b2f7cc75db 143.44.164.123 f0a234c6-4c02-481f-a995-75b3b5a35996 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Products Racing Clutch Assembly Products updated: Racing Clutch Assembly {stock_quantity} 2026-09-29 18:49:15.680238 2026-09-29 10:49:15.680238+00 1d42b93e-83ef-46f2-b6dc-7486442bfcca 143.44.164.123 c658647e-f537-46e2-aeab-5e17bb591f42 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Products Ohlins Rear Shock Absorber Products updated: Ohlins Rear Shock Absorber {price} 2026-09-29 18:50:39.532585 2026-09-29 10:50:39.532585+00 656db2e0-7cb6-4311-8271-3e15430d31b3 143.44.164.123 4f137cbd-a620-498d-9a04-6b1a2806b149 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Motorcycle Inspection Services updated: Motorcycle Inspection {duration_minutes,name} 2026-09-30 06:13:06.556512 2026-09-29 22:13:06.556512+00 7f2b49e7-8c12-44d3-9d1f-3cb79c8898ec 143.44.164.66 f3c86074-0d80-442c-854d-4cb9c55013b8 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-ULM8GG Appointments status changed: FRM-ULM8GG {service_started_at,status} 2026-10-06 21:17:21.727957 2026-10-06 13:17:21.727957+00 de1e21dc-53e4-4362-8898-3bd1da2d67fc 143.44.164.41 4ebbbfae-4c51-4517-8464-87598d4d387e 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-08 19:23:25.355884 2026-10-08 11:23:25.355884+00 \N 143.44.164.41 6a4981f4-990f-4229-813b-0e1ea5cfdc90 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-10-11 14:03:53.5081 2026-10-11 06:03:53.5081+00 \N 143.44.164.41 81c220c6-44b0-4d29-b121-8cd54f3891c8 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-14 15:14:24.691926 2026-10-14 07:14:24.691926+00 \N 143.44.164.41 d9e6af7e-ef05-4bcd-863f-7db2aa40ef2d 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-15 15:08:23.29438 2026-10-15 07:08:23.29438+00

\N 143.44.164.41
c590f591-d8dc-4133-b3cf-4ae8aec01323
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed
Reports Financial Report Opened the print dialog for 7 completed
transactions. {report_type,date_range,print_format} 2026-10-21
18:53:38.094984 2026-10-21 10:53:38.094984+00 \N 143.44.164.41
28e29c1b-5321-400a-a5ab-487ecdc325c4 \N admin@gmail.com status
changed Appointments FRM-5YVTRB Appointments status changed:
FRM-5YVTRB {status} 2026-09-22 11:19:40.9908 2026-09-22
03:19:40.9908+00 33fbda0f-318f-41fd-8eac-328e296ea0d3 49.146.140.233
a7830a0a-cda1-45d3-9cb3-6875aa6fe546 \N admin@gmail.com updated
Notifications ec131dba-3891-4e35-ba23-5d121fe7bc96 Notifications
updated: ec131dba-3891-4e35-ba23-5d121fe7bc96 {is_read} 2026-09-22
11:19:53.182947 2026-09-22 03:19:53.182947+00
ec131dba-3891-4e35-ba23-5d121fe7bc96 49.146.140.233
03a463e2-fcea-4064-ae92-e929785cef6b
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Notifications 4eccae64-73ce-4bf4-a4cb-a94763487cac
Notifications updated: 4eccae64-73ce-4bf4-a4cb-a94763487cac {is_read}
2026-09-26 14:56:21.958168 2026-09-26 06:56:21.958168+00
4eccae64-73ce-4bf4-a4cb-a94763487cac 115.147.51.76
1acfb6b6-7978-4e50-8a3f-50d8ac5f5df7
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-09-27 19:32:58.494601 2026-09-27
11:32:58.494601+00 \N 143.44.164.184
f95011cd-efb9-40d4-90e9-076ce97a8500
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
out Authentication Admin console Administrator was automatically signed
out after 2 minutes of inactivity. {} 2026-09-28 16:14:29.911017
2026-09-28 08:14:29.911017+00 \N 143.44.164.66
19e44944-ba1c-466f-bac8-3531ed6f75bc
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
exported Reports Booking Report Exported 13 bookings as PDF.
{report_type,date_range,export_format} 2026-09-29 08:52:14.05372
2026-09-29 00:52:14.05372+00 \N 143.44.164.123
2504dfb4-2451-492d-8164-5672e9267660
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Products Performance Exhaust Pipe Products updated:
Performance Exhaust Pipe {stock_quantity} 2026-09-29 18:49:23.965335
2026-09-29 10:49:23.965335+00 d63ff680-20d7-4a54-92e0-1790fb39446c
143.44.164.123
d2e374e2-7ee1-4d1e-8adf-67069641c868
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Products Motorcycle Tail Bag Products updated: Motorcycle Tail

Bag {in_stock,stock_quantity} 2026-09-29 18:50:02.602372 2026-09-29 10:50:02.602372+00 8247c0ee-a2c2-4e42-81e0-99e7ed002c5d 143.44.164.123 1d8ad505-4ea2-4f51-9319-b5433e198fb6 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator signed out before visiting the public site. {} 2026-09-30 06:16:17.189636 2026-09-29 22:16:17.189636+00 \N 143.44.164.66 cd23a008-f0c8-492e-8183-6d2c0cebce30 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-HWAVDJ Appointments status changed: FRM-HWAVDJ {service_started_at,status} 2026-10-06 21:17:23.289101 2026-10-06 13:17:23.289101+00 617073f7-8cfa-4f3f-8352-aff1a76da780 143.44.164.41 95ecb213-5d4c-4048-9f27-32c9d1d18670 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-2LYRZC Appointments status changed: FRM-2LYRZC {assigned_crew_id,status} 2026-10-08 19:23:42.086301 2026-10-08 11:23:42.086301+00 c5f4d85a-7617-42c7-84db-941d333879bf 143.44.164.41 fc1ba450-115c-478c-9e14-283c824d786e 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-11 14:05:42.069218 2026-10-11 06:05:42.069218+00 \N 143.44.164.41 cff596af-7abd-485c-8afb-8a55b03198ef 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-14 16:12:02.269532 2026-10-14 08:12:02.269532+00 \N 143.44.164.41 57dd6bcf-2620-4fe1-9b87-4f584b49ed0d 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-JKNK84 Appointments status changed: FRM-JKNK84 {status} 2026-10-15 15:09:32.354672 2026-10-15 07:09:32.354672+00 6c0de99e-7522-4f4e-ad11-0913682e3ae3 143.44.164.41 7f092372-aaa1-499e-aa70-558dfa256602 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed Reports Financial Report Opened the print dialog for 7 completed transactions. {report_type,date_range,print_format} 2026-10-21 18:58:33.113855 2026-10-21 10:58:33.113855+00 \N 143.44.164.41 0291af6b-3101-43e1-b6ea-7eefb9d299f1 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-26 15:00:07.676643

2026-09-26 07:00:07.676643+00 \N 115.147.51.76
52e35fe6-8cba-40d7-8e3e-0710f17fe645
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Account Settings Administrator profile Administrator account
settings updated. {"Administrator name"} 2026-09-27 19:39:09.344038
2026-09-27 11:39:09.344038+00 \N 143.44.164.184
1e9ab1e8-53d2-401f-aafe-9d66ab0933e7
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-09-28 16:35:45.770933 2026-09-28
08:35:45.770933+00 \N 143.44.164.66
61077a6d-8d60-47de-aad7-c82cf81aa357
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-09-29 09:01:04.14635 2026-09-29 01:01:04.14635+00
\N 143.44.164.123
12e56a13-3b8e-46ab-98a7-2e8647160d6d
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Products Braided Brake Hose Products updated: Braided Brake
Hose {stock_quantity} 2026-09-29 18:49:35.648601 2026-09-29
10:49:35.648601+00 c973b83b-ee14-4254-a320-420f75ffb39a
143.44.164.123
cc5219db-d33e-4ab6-aaf4-d3186cfe03d5
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Products Motocross Riding Gloves Products updated: Motocross
Riding Gloves {stock_quantity} 2026-09-29 18:49:53.655452 2026-09-29
10:49:53.655452+00 888affb7-4276-41dd-9d6c-5b367e059482
143.44.164.123
970b3ceb-7f01-4222-9b0f-c7f2eff431af
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com created
Price History fcb995ec-bb61-4f57-97d5-2c3c72d7e27b Price History
created: fcb995ec-bb61-4f57-97d5-2c3c72d7e27b {} 2026-09-29
18:50:40.150635 2026-09-29 10:50:40.150635+00
fcb995ec-bb61-4f57-97d5-2c3c72d7e27b 143.44.164.123
7d349df2-07b5-492f-b4c0-bda732adb5d7
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
in Authentication Admin console Administrator signed in to the admin
console. {} 2026-09-30 08:44:24.900045 2026-09-30
00:44:24.900045+00 \N 49.146.140.233
a623e20b-bbfd-435c-bd42-37c663c5744e
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed
out Authentication Admin console Administrator was automatically signed
out after 2 minutes of inactivity. {} 2026-10-06 21:21:12.457597
2026-10-06 13:21:12.457597+00 \N 143.44.164.41
1acc92ee-86dc-4044-848c-0d41fb785558

7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-08 23:34:59.170703 2026-10-08 15:34:59.170703+00 \N 143.44.164.41 22efdbe9-bcb6-4849-a435-61f522847033
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed Reports Financial Report (Summary) Opened the print dialog for 3 completed transactions. {report_type,date_range,print_format} 2026-10-11 14:06:28.856046 2026-10-11 06:06:28.856046+00 \N 143.44.164.41 5b825756-d72e-45cc-990b-333d9c01c171
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Motorcycle Tune-Up Services updated: Motorcycle Tune-Up {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:13:19.760679 2026-10-14 08:13:19.760679+00 d9ecf096-df3b-47f5-a012-80d98aacf9f5 143.44.164.41 0197e822-9b5a-487f-8d9b-a66693bf5acb
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Full CVT Service Services updated: Full CVT Service {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:14:22.711896 2026-10-14 08:14:22.711896+00 a383dd9d-9dbb-4974-b9bf-e8d08ad1d0c5 143.44.164.41 ab519cf2-ed1c-4c8a-8c74-08b25ffc59d3
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Chain Lubrication Services updated: Chain Lubrication {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:15:05.987232 2026-10-14 08:15:05.987232+00 baaf0e56-887d-41c4-b78a-355e5509bc9c 143.44.164.41 a4bf8cb0-da78-4944-a1e6-6a728f4cd4b8
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-AQY69P Appointments status changed: FRM-AQY69P {service_ended_at,status} 2026-10-14 16:15:10.905783 2026-10-14 08:15:10.905783+00 eda6b8a7-6dc1-4584-8ad5-e918438c0a05 143.44.164.41 3c8028a2-4139-42e2-b33e-65d855c465d6
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Notifications 27773a1e-66bf-4172-b0cd-1a9f7d556297 Notifications updated: 27773a1e-66bf-4172-b0cd-1a9f7d556297 {is_read} 2026-10-15 15:10:29.960153 2026-10-15 07:10:29.960153+00 27773a1e-66bf-4172-b0cd-1a9f7d556297 143.44.164.41 e5403095-7f13-44f5-bcb8-6020ba042e6f
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator signed out before visiting the public site. {} 2026-10-21 19:10:56.051363 2026-10-21 11:10:56.051363+00 \N 143.44.164.41 d4c9c2d9-037a-45cc-b078-b1a0557751c5

7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-26 22:45:11.553477 2026-09-26 14:45:11.553477+00 \N 143.44.164.123 61d179df-c878-46e5-bebd-f4098502ccee
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator signed out before visiting the public site. {} 2026-09-27 19:42:46.060309 2026-09-27 11:42:46.060309+00 \N 143.44.164.184 8c61c728-fdbe-45c9-860b-19e8374df3fa
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Notifications 6b0461b8-c19e-495c-82cb-696c1bf5947f Notifications updated: 6b0461b8-c19e-495c-82cb-696c1bf5947f {is_read} 2026-09-27 19:43:50.230667 2026-09-27 11:43:50.230667+00 6b0461b8-c19e-495c-82cb-696c1bf5947f 143.44.164.184 b972a97d-e10c-4137-ae1d-cfd64262383e
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com archived Products Ohlins Rear Shock Absorber Products archived: Ohlins Rear Shock Absorber {archived_at,is_archived} 2026-09-28 16:36:30.130172 2026-09-28 08:36:30.130172+00 656db2e0-7cb6-4311-8271-3e15430d31b3 143.44.164.66 1680822d-5c59-4a9c-a346-94ce6dba19db
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-29 09:05:43.426147 2026-09-29 01:05:43.426147+00 \N 143.44.164.123 dac00743-d550-4a09-ac91-0d470285a842
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Products CNC Adjustable Levers Products updated: CNC Adjustable Levers {stock_quantity} 2026-09-29 18:49:43.720467 2026-09-29 10:49:43.720467+00 281caa1b-f3ac-48c4-a48b-fb02cde092fd 143.44.164.123 f107a2c0-9d8c-4df5-b874-bd8378b9d657
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-30 08:48:21.965337 2026-09-30 00:48:21.965337+00 \N 49.146.140.233 7b57219e-45b8-44f2-920c-815e2bc24248
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-30 08:48:33.72072 2026-09-30 00:48:33.72072+00 \N 49.146.140.233 e74aef41-58be-4ba8-9c9a-4196df46776c
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin

console. {} 2026-10-06 23:35:54.811018 2026-10-06 15:35:54.811018+00 \N 143.44.164.41 bde3cf0e-66c4-427e-b3b3-24c12df52fc0 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed Reports Financial Report (Summary) Opened the print dialog for 9 bookings. {report_type,date_range,print_format} 2026-10-08 23:35:10.791311 2026-10-08 15:35:10.791311+00 \N 143.44.164.41 c89697e3-f631-4ce0-a47d-693db4a7df33 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-11 14:13:41.810154 2026-10-11 06:13:41.810154+00 \N 143.44.164.41 cf2d985e-1b83-4512-a203-636295b8d3f0 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Brake Service Services updated: Brake Service {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:13:43.673664 2026-10-14 08:13:43.673664+00 e5742f78-e0f3-4c14-b4e5-9ad22b3ff85b 143.44.164.41 1c697986-2b96-4d7b-95f3-99c84c846289 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Clutch Adjustment Services updated: Clutch Adjustment {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:14:00.509473 2026-10-14 08:14:00.509473+00 935c4315-72a4-40a4-9a63-6487106bd0fb 143.44.164.41 6f73fe03-1a6f-470b-ab82-c06536254dd6 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Chain Cleaning Services updated: Chain Cleaning {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:14:36.998625 2026-10-14 08:14:36.998625+00 3066e928-4557-488a-831f-f7e0e26ae11f 143.44.164.41 fe72d3a8-efff-4db6-bb0f-11f211b4d791 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-16 21:05:00.816591 2026-10-16 13:05:00.816591+00 \N 143.44.164.41 ba790536-e407-42e4-bb93-a15b41082167 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-10-21 20:36:07.017271 2026-10-21 12:36:07.017271+00 \N 143.44.164.41 f2fd5f1e-09ad-49ae-946c-143ffb2f88f2 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-23 00:57:34.581211 2026-09-22 16:57:34.581211+00 \N 136.158.33.87 e2b51d1a-1e10-4c4f-b449-205effe0edd4

7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-27 09:12:06.681094 2026-09-27 01:12:06.681094+00 \N 143.44.164.184 07d6aa05-4863-465f-9078-757e5f96fc0f
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-27 19:43:41.992642 2026-09-27 11:43:41.992642+00 \N 143.44.164.184 fa252fc9-b397-45be-bbe8-9f114905cc90
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com restored Products Ohlins Rear Shock Absorber Products restored: Ohlins Rear Shock Absorber {archived_at,is_archived} 2026-09-28 16:36:47.73335 2026-09-28 08:36:47.73335+00 656db2e0-7cb6-4311-8271-3e15430d31b3 143.44.164.66 6b9e2886-5127-4771-9883-29f28223ab76
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-29 09:27:28.388749 2026-09-29 01:27:28.388749+00 \N 143.44.164.123 f8eec128-ce27-4212-89b4-8ecf01ddc7c1
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Products Performance Exhaust Pipe Products updated: Performance Exhaust Pipe {price} 2026-09-29 18:52:29.618243 2026-09-29 10:52:29.618243+00 d63ff680-20d7-4a54-92e0-1790fb39446c 143.44.164.123 74416303-1b9a-4c80-b00c-3664879b96cc
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com status changed Appointments FRM-MPMHT9 Appointments status changed: FRM-MPMHT9 {service_started_at,status} 2026-09-30 08:49:55.532526 2026-09-30 00:49:55.532526+00 6014afba-21ae-402c-a687-150730e547ea 49.146.140.233 52590f5a-eb66-4679-a089-07ebef8ea751
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-10-06 23:39:50.725138 2026-10-06 15:39:50.725138+00 \N 143.44.164.41 203285be-fcf5-4cdf-950c-3a697d9ccba9
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com printed Reports Financial Report (Summary) Opened the print dialog for 9 bookings. {report_type,date_range,print_format} 2026-10-08 23:36:02.603573 2026-10-08 15:36:02.603573+00 \N 143.44.164.41 8b9d1c78-d960-4b5d-9b07-3ea0fd22346d
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed out Authentication Admin console Administrator was automatically signed

out after 2 minutes of inactivity. {} 2026-10-11 14:16:06.993584
2026-10-11 06:16:06.993584+00 \N 143.44.164.41
82c8f0d5-8def-42b6-bc42-a8e8fa1d9e3f
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Services Chain Adjustment Services updated: Chain Adjustment
{large_bike_duration_minutes,large_bike_price} 2026-10-14 16:15:56.765738
2026-10-14 08:15:56.765738+00 aad80fe2-5bcc-4a22-864b-8e6fc00531bd
143.44.164.41
0aa47572-1cee-40d1-8895-2c9737898461
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Services Chain Replacement Services updated: Chain
Replacement {large_bike_duration_minutes,large_bike_price} 2026-10-14
16:16:08.69203 2026-10-14 08:16:08.69203+00
e852fde6-f231-4b05-95b4-26e8d6455c97 143.44.164.41
e1625b9c-7574-485f-bab1-ac8e872b1696
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Services Tire Change Services updated: Tire Change
{large_bike_duration_minutes,large_bike_price} 2026-10-14 16:16:48.133378
2026-10-14 08:16:48.133378+00 9bac85e5-f254-43ee-a36d-384b25fcae83
143.44.164.41
3494b294-7c44-4b8b-8655-d347e34514ef
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Services Front Fork Service Services updated: Front Fork Service
{large_bike_duration_minutes,large_bike_price} 2026-10-14
16:17:53.894265 2026-10-14 08:17:53.894265+00
06900445-d8cc-4c22-822c-5b23b4d25b4e 143.44.164.41
9bdbe00c-12fe-48ca-8e91-e01007597905
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Services Rear Shock Service Services updated: Rear Shock
Service {large_bike_duration_minutes,large_bike_price} 2026-10-14
16:18:09.50947 2026-10-14 08:18:09.50947+00
35ba5528-3e82-4df2-9d12-147c813f9295 143.44.164.41
1c486ba1-ffa2-4596-9dd4-88e39fd90840
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Services Suspension Rebuild Services updated: Suspension
Rebuild {large_bike_duration_minutes,large_bike_price} 2026-10-14
16:18:44.637822 2026-10-14 08:18:44.637822+00
951731dd-8412-46b9-8058-7348ea20b9ef 143.44.164.41
36cc19b8-ff6a-4f07-9708-b16a7db86735
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com
updated Services Steering Bearing Replacement Services updated:
Steering Bearing Replacement {large_bike_duration_minutes,large_bike_price}
2026-10-14 16:19:08.206181 2026-10-14 08:19:08.206181+00
f01fd9c7-6a0f-4510-bf61-fd7469d1290a 143.44.164.41
c34587dd-fb51-409e-9c19-e4cb7fd43522

7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Battery Replacement Services updated: Battery Replacement {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:19:47.071494 2026-10-14 08:19:47.071494+00 51d71716-aabb-4e18-80f8-79c8d20d3ed1 143.44.164.41 55587503-9423-4e66-8b79-6ed3073341ba
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Electrical Diagnostics FI Services updated: Electrical Diagnostics FI {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:20:01.474329 2026-10-14 08:20:01.474329+00 b950fc34-4de4-42b6-b3ff-5c0b352103db 143.44.164.41 f43fbb7e-eb63-4581-bcf6-2dbcafa6f110
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services ECU Diagnostic Scan Services updated: ECU Diagnostic Scan {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:20:35.33525 2026-10-14 08:20:35.33525+00 5160c7c2-4cf7-481a-8881-cdfea2519895 143.44.164.41 e12887da-8d46-42a3-bd9c-da6419e242dd
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Air Filter Replacement Services updated: Air Filter Replacement {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:21:21.524449 2026-10-14 08:21:21.524449+00 7dbb4616-2f79-4102-914c-415c0176c05f 143.44.164.41 e00c4116-9c84-4544-ab1e-33d0f2434190
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Carburetor Cleaning Services updated: Carburetor Cleaning {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:21:34.748564 2026-10-14 08:21:34.748564+00 0677835f-d6c7-4151-b33b-9fd0e6b2f14d 143.44.164.41 93825849-c8c3-4605-8bf5-5ea8feec0912
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Fuel System Cleaning Services updated: Fuel System Cleaning {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:22:22.147666 2026-10-14 08:22:22.147666+00 44651d94-93a0-4b63-b2ea-7dd378a57bb0 143.44.164.41 350f8a0b-5b77-4580-b5de-64f122d40d7d
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Radiator Cleaning Services updated: Radiator Cleaning {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:22:34.870151 2026-10-14 08:22:34.870151+00 b6af4e06-1d15-4f61-b7e8-77719cb355de 143.44.164.41 4cd84dac-a9bd-4958-8a30-f3afc864a1ec
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Headlight/Lighting Replacement Services updated: Headlight/Lighting Replacement

{large_bike_duration_minutes,large_bike_price} 2026-10-14 16:23:12.588502 2026-10-14 08:23:12.588502+00 0606e54a-cbfc-4043-b88e-1d1f2d273249 143.44.164.41 1d336449-edfb-442b-892b-48a8eefb4de5 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Charging System Repair Services updated: Charging System Repair {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:23:49.469295 2026-10-14 08:23:49.469295+00 a7b50b13-0e62-4413-ae40-4ecf75b2e54a 143.44.164.41 666a2de8-3c54-49ba-8d7c-ef0aa7b553c8 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Fuel Pump Inspection/Repair Services updated: Fuel Pump Inspection/Repair {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:25:14.080641 2026-10-14 08:25:14.080641+00 9021a319-7ed4-488f-94fe-5a10356259b9 143.44.164.41 28510801-a215-4790-8de1-e3be83e113ff 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Transmission Repair Services updated: Transmission Repair {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:25:46.68475 2026-10-14 08:25:46.68475+00 d621e288-2d04-4ab2-a649-2c22c9256fce 143.44.164.41 3fc3639d-0520-46dd-b330-d3ed66693af1 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Brake Shoe Replacement Services updated: Brake Shoe Replacement {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:26:06.084696 2026-10-14 08:26:06.084696+00 8b41b72f-0a7d-45b9-8053-82ecc20491c1 143.44.164.41 7fefd5d3-0669-4660-90fb-423ac2a3bfa9 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com updated Services Crankshaft Bearing Replacement Services updated: Crankshaft Bearing Replacement {large_bike_duration_minutes,large_bike_price} 2026-10-14 16:26:49.939108 2026-10-14 08:26:49.939108+00 a2e6ef24-2f88-4f18-92b2-180be413a5a7 143.44.164.41 987baa4b-c6d8-4a21-a026-6fa7d8f90cda \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-06 09:00:03.707154 2026-09-06 01:00:03.707154+00 \N 143.44.164.74 4816775f-9626-4a55-a14e-ba05e1939e1e \N admin@gmail.com signed out Authentication Admin console Administrator signed out of the admin console. {} 2026-09-06 13:15:15.369338 2026-09-06 05:15:15.369338+00 \N 143.44.164.74 8c2ff656-abaf-466a-b4aa-88936792d63b \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-06 13:18:56.101959 2026-09-06 05:18:56.101959+00 \N

143.44.164.74
d408e4d1-222f-480e-af37-af584dd4a3c7 \N admin@gmail.com signed out Authentication Admin console Administrator signed out of the admin console. {} 2026-09-06 13:42:09.598999 2026-09-06 05:42:09.598999+00 \N 143.44.164.74
187ad696-87ec-4958-8e3a-5f6523a4d8df \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-06 14:23:28.798632 2026-09-06 06:23:28.798632+00 \N 143.44.164.74
2c2a036b-b1fa-4b42-a3c9-44e88edda49f \N admin@gmail.com exported Reports Booking Volume Report Exported 13 bookings as PDF. {report_type,date_range,export_format} 2026-09-07 07:29:30.119042 2026-09-06 23:29:30.119042+00 \N 122.54.155.126
40e2c44e-a1ca-4434-9488-f9e53f6531ef \N admin@gmail.com exported Reports Booking Volume Report Exported 13 bookings as CSV. {report_type,date_range,export_format} 2026-09-07 09:05:08.305734 2026-09-07 01:05:08.305734+00 \N 180.193.217.82
c4a8c913-e801-4a1d-b069-0318be28bd78 \N admin@gmail.com exported Reports Booking Volume Report Exported 13 bookings as PDF. {report_type,date_range,export_format} 2026-09-07 09:06:35.331213 2026-09-07 01:06:35.331213+00 \N 180.193.217.82
d16e391b-aa35-4ab3-86c6-f77f7a35fa38 \N admin@gmail.com updated Notifications ad2aedc4-9712-4b59-a9eb-9304093c20b5 Notifications updated: ad2aedc4-9712-4b59-a9eb-9304093c20b5 {is_read} 2026-09-07 09:10:34.122044 2026-09-07 01:10:34.122044+00 ad2aedc4-9712-4b59-a9eb-9304093c20b5 180.193.217.82
3251edda-a5bf-4130-894f-b9c9b9fe1d8d \N admin@gmail.com activated Crew Members JR Salazar Crew Members activated: JR Salazar {is_active} 2026-09-07 09:23:00.77261 2026-09-07 01:23:00.77261+00 f3c5770f-e485-4efc-83f6-892422d8592e 180.193.217.82
4a6cf0cc-c676-4c9c-8881-214214ed8382 \N admin@gmail.com deactivated Crew Members JR Salazar Crew Members deactivated: JR Salazar {is_active} 2026-09-07 09:23:03.154341 2026-09-07 01:23:03.154341+00 f3c5770f-e485-4efc-83f6-892422d8592e 180.193.217.82
d7aef75d-ade8-4efe-9783-9603fa6d27b1 \N admin@gmail.com activated Crew Members JR Salazar Crew Members activated: JR Salazar {is_active} 2026-09-07 09:24:14.830701 2026-09-07 01:24:14.830701+00 f3c5770f-e485-4efc-83f6-892422d8592e 180.193.217.82
8a27bbfc-d778-4116-b80f-367d0f556bf2 \N admin@gmail.com updated Products Motorcycle Tail Bag Products updated: Motorcycle Tail Bag {in_stock} 2026-09-07 17:23:28.975519 2026-09-07 09:23:28.975519+00 8247c0ee-a2c2-4e42-81e0-99e7ed002c5d 143.44.164.132
9b5cac52-d63a-45be-96b3-50d34cc91d22 \N admin@gmail.com status changed Appointments FRM-VJ965M Appointments status changed:

FRM-VJ965M {status} 2026-09-07 18:27:21.213654 2026-09-07 10:27:21.213654+00 64cf7994-ff35-418a-97f8-9fef8a68b07e 143.44.164.132 e8e228d3-7404-4146-89b5-e6966421889c \N admin@gmail.com status changed Appointments FRM-U449JR Appointments status changed: FRM-U449JR {status} 2026-09-07 18:27:56.154096 2026-09-07 10:27:56.154096+00 7f56c54a-98d8-4600-8eec-8aca53cedbce 143.44.164.132 6fa89b79-1fcf-41f2-9040-b6f20cd50389 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-07 19:28:24.460092 2026-09-07 11:28:24.460092+00 \N 143.44.164.132 45b5f3c3-8653-45e9-a132-5aabf79c59e3 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-07 19:28:27.281165 2026-09-07 11:28:27.281165+00 \N 143.44.164.132 51efa490-7e25-46d8-9184-e1d576f51f87 \N admin@gmail.com updated Settings Shop information Shop information updated. {"Booking terms & conditions"} 2026-09-08 07:00:00.076727 2026-09-07 23:00:00.076727+00 \N 143.44.164.74 c9a90b54-1947-451f-925c-8da2812fc430 \N admin@gmail.com archived Appointments FRM-VJ965M Appointments archived: FRM-VJ965M {is_archived} 2026-09-08 17:32:59.350769 2026-09-08 09:32:59.350769+00 64cf7994-ff35-418a-97f8-9fef8a68b07e 162.4.58.128 426a8be3-0970-4345-838e-85a2d190c07c \N admin@gmail.com status changed Appointments FRM-LMDFUU Appointments status changed: FRM-LMDFUU {status} 2026-09-08 17:33:46.778499 2026-09-08 09:33:46.778499+00 e25251f4-8f3d-4a8f-aa43-b120117373b9 162.4.58.128 cf003228-9949-49a6-bd4e-b19a35c00ad7 \N admin@gmail.com status changed Appointments FRM-69R7BV Appointments status changed: FRM-69R7BV {status} 2026-09-08 17:33:53.426694 2026-09-08 09:33:53.426694+00 205d1da8-7493-4bf1-994e-cb8534690e54 162.4.58.128 e10624f0-68d9-47be-bd93-9df8efb1b74d \N admin@gmail.com updated Appointments FRM-LMDFUU Appointments updated: FRM-LMDFUU {assigned_crew_id} 2026-09-08 17:33:58.748592 2026-09-08 09:33:58.748592+00 e25251f4-8f3d-4a8f-aa43-b120117373b9 162.4.58.128 09c5e7d0-a64d-46f2-9cfb-d80925fc1213 \N admin@gmail.com exported Reports Booking Volume Report Exported 4 bookings as DOCX. {report_type,date_range,export_format} 2026-09-08 17:56:42.007801 2026-09-08 09:56:42.007801+00 \N 162.4.58.135 080e5045-5872-46fe-86be-ad7d239d99ca \N admin@gmail.com status changed Appointments FRM-EFGUWJ Appointments status changed: FRM-EFGUWJ {status} 2026-09-08 22:24:42.120736 2026-09-08 14:24:42.120736+00 34c976d2-745d-4c5e-b994-354d019706e8 143.44.164.96

b0565205-bcfd-4cca-921e-d69376652809 \N admin@gmail.com created Crew Schedules 166f3f95-ba9b-4f55-bae6-61390dedc0a7 Crew Schedules created: 166f3f95-ba9b-4f55-bae6-61390dedc0a7 {} 2026-09-08 22:38:59.294597 2026-09-08 14:38:59.294597+00 166f3f95-ba9b-4f55-bae6-61390dedc0a7 143.44.164.96
14b16a8a-3121-4992-8b66-7cd975ad38ca \N admin@gmail.com created Crew Schedules 4c12ea86-a747-4fb6-947b-ffb8a38ed876 Crew Schedules created: 4c12ea86-a747-4fb6-947b-ffb8a38ed876 {} 2026-09-08 22:38:59.294597 2026-09-08 14:38:59.294597+00 4c12ea86-a747-4fb6-947b-ffb8a38ed876 143.44.164.96
bf9b4de1-791a-4122-a33f-e5e9c54f6ed3 \N admin@gmail.com created Crew Schedules ba2e1214-1c5e-4f7b-9dc4-3cb898a5ace3 Crew Schedules created: ba2e1214-1c5e-4f7b-9dc4-3cb898a5ace3 {} 2026-09-08 22:38:59.294597 2026-09-08 14:38:59.294597+00 ba2e1214-1c5e-4f7b-9dc4-3cb898a5ace3 143.44.164.96
eb696b05-1564-45c7-8fc8-09a187c6fd3a \N admin@gmail.com created Crew Schedules 8a122bf1-4a77-4289-903d-92290efeb260 Crew Schedules created: 8a122bf1-4a77-4289-903d-92290efeb260 {} 2026-09-08 22:38:59.294597 2026-09-08 14:38:59.294597+00 8a122bf1-4a77-4289-903d-92290efeb260 143.44.164.96
cef5a9d6-60c0-497d-9368-0874c39f3418 \N admin@gmail.com created Crew Schedules d3c6f238-fa98-4c66-89d8-037553caa197 Crew Schedules created: d3c6f238-fa98-4c66-89d8-037553caa197 {} 2026-09-08 22:38:59.294597 2026-09-08 14:38:59.294597+00 d3c6f238-fa98-4c66-89d8-037553caa197 143.44.164.96
5e926dae-2235-44de-8655-d56c07ee0d66 \N admin@gmail.com created Crew Schedules 625f6017-52b4-47c4-ad30-6dbcb35196b1 Crew Schedules created: 625f6017-52b4-47c4-ad30-6dbcb35196b1 {} 2026-09-08 22:38:59.294597 2026-09-08 14:38:59.294597+00 625f6017-52b4-47c4-ad30-6dbcb35196b1 143.44.164.96
ef87433a-69bd-493a-924d-57094ebd6fda \N admin@gmail.com created Crew Schedules 58b66753-3342-42c2-b105-abf6e47b7eb0 Crew Schedules created: 58b66753-3342-42c2-b105-abf6e47b7eb0 {} 2026-09-08 22:38:59.294597 2026-09-08 14:38:59.294597+00 58b66753-3342-42c2-b105-abf6e47b7eb0 143.44.164.96
7b412226-9de8-40c2-abbd-9a6416d2dfda \N admin@gmail.com created Crew Schedules 42e24752-4e51-460b-b4de-43d0492bc39a Crew Schedules created: 42e24752-4e51-460b-b4de-43d0492bc39a {} 2026-09-08 22:38:59.294597 2026-09-08 14:38:59.294597+00 42e24752-4e51-460b-b4de-43d0492bc39a 143.44.164.96
d83b64ec-9f4e-4733-8ed6-5b6837385ef1 \N admin@gmail.com created Crew Schedules 6bdfc093-7322-4525-ace0-f2b6c88d2210 Crew Schedules created: 6bdfc093-7322-4525-ace0-f2b6c88d2210 {} 2026-09-08 22:38:59.294597 2026-09-08 14:38:59.294597+00 6bdfc093-7322-4525-ace0-f2b6c88d2210 143.44.164.96

ee45def0-f2fe-4e6c-8762-864c24c7ef6a \N admin@gmail.com created Crew Schedules 597cb434-ea7b-4081-950b-c592342acd9a Crew Schedules created: 597cb434-ea7b-4081-950b-c592342acd9a {} 2026-09-08 22:38:59.294597 2026-09-08 14:38:59.294597+00 597cb434-ea7b-4081-950b-c592342acd9a 143.44.164.96
a079e28e-0ef3-4c52-b3af-06e70fe36549 \N admin@gmail.com created Crew Schedules 3e4da663-732a-409b-a050-d0393e374fe1 Crew Schedules created: 3e4da663-732a-409b-a050-d0393e374fe1 {} 2026-09-08 22:38:59.294597 2026-09-08 14:38:59.294597+00 3e4da663-732a-409b-a050-d0393e374fe1 143.44.164.96
2ce17580-8fde-48b4-91f1-990d6a2bb504 \N admin@gmail.com created Crew Schedules 818d3a29-0d3f-4f5d-87f2-3172f3f1c839 Crew Schedules created: 818d3a29-0d3f-4f5d-87f2-3172f3f1c839 {} 2026-09-08 22:38:59.294597 2026-09-08 14:38:59.294597+00 818d3a29-0d3f-4f5d-87f2-3172f3f1c839 143.44.164.96
b24587d5-3697-4b46-aed1-ed1d3c5f4c67 \N admin@gmail.com created Crew Schedules f5388a05-6d87-419c-b487-3827b4704498 Crew Schedules created: f5388a05-6d87-419c-b487-3827b4704498 {} 2026-09-08 22:38:59.294597 2026-09-08 14:38:59.294597+00 f5388a05-6d87-419c-b487-3827b4704498 143.44.164.96
ddcc2039-cde1-4cae-82b2-299688cfd21e \N admin@gmail.com created Crew Schedules f661b88a-4288-4aa6-9004-be62e53ac0a5 Crew Schedules created: f661b88a-4288-4aa6-9004-be62e53ac0a5 {} 2026-09-08 22:38:59.294597 2026-09-08 14:38:59.294597+00 f661b88a-4288-4aa6-9004-be62e53ac0a5 143.44.164.96
ca31d900-5c38-47f7-a801-827a2e9692d5 \N admin@gmail.com created Crew Schedules 24c96896-bcdc-4522-8af4-9a7db4fd8f7e Crew Schedules created: 24c96896-bcdc-4522-8af4-9a7db4fd8f7e {} 2026-09-08 22:38:59.294597 2026-09-08 14:38:59.294597+00 24c96896-bcdc-4522-8af4-9a7db4fd8f7e 143.44.164.96
1b83a339-e58c-4877-8e8c-fa5785584d12 \N admin@gmail.com created Crew Schedules 0d81e035-a865-4b27-bd1e-26c432a0b831 Crew Schedules created: 0d81e035-a865-4b27-bd1e-26c432a0b831 {} 2026-09-08 22:39:28.809772 2026-09-08 14:39:28.809772+00 0d81e035-a865-4b27-bd1e-26c432a0b831 143.44.164.96
6685b09c-51a5-4608-ac22-0ac33fd97b97 \N admin@gmail.com created Crew Schedules 942db473-bbe2-4b69-9260-4c4c8bdcf6be Crew Schedules created: 942db473-bbe2-4b69-9260-4c4c8bdcf6be {} 2026-09-08 22:39:28.809772 2026-09-08 14:39:28.809772+00 942db473-bbe2-4b69-9260-4c4c8bdcf6be 143.44.164.96
4abd8e48-9f00-445f-b5c4-ef64ed916eca \N admin@gmail.com created Crew Schedules 2e213cc1-21d5-4ee9-a2f1-ead9d141cd7e Crew Schedules created: 2e213cc1-21d5-4ee9-a2f1-ead9d141cd7e {} 2026-09-08 22:39:28.809772 2026-09-08 14:39:28.809772+00 2e213cc1-21d5-4ee9-a2f1-ead9d141cd7e 143.44.164.96

e8c5d542-d379-4f5c-9a15-ba4bc8dcc4e9 \N admin@gmail.com created Crew Schedules 3dcabaaf-d9c0-4a2f-ae61-00ed0e28e227 Crew Schedules created: 3dcabaaf-d9c0-4a2f-ae61-00ed0e28e227 {} 2026-09-08 22:39:28.809772 2026-09-08 14:39:28.809772+00 3dcabaaf-d9c0-4a2f-ae61-00ed0e28e227 143.44.164.96
0cd58a47-82e9-42f7-81ae-4ed33f87f313 \N admin@gmail.com created Services Radiator Cleaning Services created: Radiator Cleaning {} 2026-09-10 00:33:27.013406 2026-09-09 16:33:27.013406+00 b6af4e06-1d15-4f61-b7e8-77719cb355de 143.44.164.251
2d12d56e-1f29-4e0c-b2c1-4a343b513278 \N admin@gmail.com created Crew Schedules 9d7d394b-f848-47b4-81f6-378833401574 Crew Schedules created: 9d7d394b-f848-47b4-81f6-378833401574 {} 2026-09-08 22:39:28.809772 2026-09-08 14:39:28.809772+00 9d7d394b-f848-47b4-81f6-378833401574 143.44.164.96
aef2ed16-f681-48e5-b786-08e0dacff116 \N admin@gmail.com created Crew Schedules 3a6e2132-f36d-4709-9e5e-abf3c5caaa38 Crew Schedules created: 3a6e2132-f36d-4709-9e5e-abf3c5caaa38 {} 2026-09-08 22:39:28.809772 2026-09-08 14:39:28.809772+00 3a6e2132-f36d-4709-9e5e-abf3c5caaa38 143.44.164.96
0df00a21-d2d2-432e-8510-16a163d668ca \N admin@gmail.com created Crew Schedules 8ff1f3a5-adf0-4f86-8a5b-f1c65ea2960e Crew Schedules created: 8ff1f3a5-adf0-4f86-8a5b-f1c65ea2960e {} 2026-09-08 22:39:28.809772 2026-09-08 14:39:28.809772+00 8ff1f3a5-adf0-4f86-8a5b-f1c65ea2960e 143.44.164.96
1eda4c34-6389-4238-b59f-48a1e1ce1b55 \N admin@gmail.com created Crew Schedules 4286735f-2de9-4720-bd83-794c7456724e Crew Schedules created: 4286735f-2de9-4720-bd83-794c7456724e {} 2026-09-08 22:39:28.809772 2026-09-08 14:39:28.809772+00 4286735f-2de9-4720-bd83-794c7456724e 143.44.164.96
52bb7054-4b4d-4557-890b-fa5f96301b52 \N admin@gmail.com created Crew Schedules 178a6edf-c7fc-4635-99fa-4e08593cbffc Crew Schedules created: 178a6edf-c7fc-4635-99fa-4e08593cbffc {} 2026-09-08 22:39:28.809772 2026-09-08 14:39:28.809772+00 178a6edf-c7fc-4635-99fa-4e08593cbffc 143.44.164.96
a7b3a8ea-f9ed-4187-9c48-4c3c5725e1da \N admin@gmail.com created Crew Schedules 97cc6dcb-2456-47d1-bc62-fc7032536d3d Crew Schedules created: 97cc6dcb-2456-47d1-bc62-fc7032536d3d {} 2026-09-08 22:39:28.809772 2026-09-08 14:39:28.809772+00 97cc6dcb-2456-47d1-bc62-fc7032536d3d 143.44.164.96
65bfabfc-a930-4fed-931e-ffd2062de451 \N admin@gmail.com created Crew Schedules a35b10b8-f3b8-458e-a078-25c3aef6adba Crew Schedules created: a35b10b8-f3b8-458e-a078-25c3aef6adba {} 2026-09-08 22:39:28.809772 2026-09-08 14:39:28.809772+00 a35b10b8-f3b8-458e-a078-25c3aef6adba 143.44.164.96
ca406dbe-8c60-4df3-a898-6a8c0a9efb18 \N admin@gmail.com created

Crew Schedules 312c730e-5d9f-4831-a507-891ae55c2a8b Crew Schedules created: 312c730e-5d9f-4831-a507-891ae55c2a8b {} 2026-09-08 22:39:28.809772 2026-09-08 14:39:28.809772+00 312c730e-5d9f-4831-a507-891ae55c2a8b 143.44.164.96 a81898d4-d59d-4f94-8240-c214a3528362 \N admin@gmail.com created Crew Schedules 144941cf-4f1f-42be-b6d0-385557063eb3 Crew Schedules created: 144941cf-4f1f-42be-b6d0-385557063eb3 {} 2026-09-08 22:39:28.809772 2026-09-08 14:39:28.809772+00 144941cf-4f1f-42be-b6d0-385557063eb3 143.44.164.96 f994eb8a-f8f1-4114-9adc-15ab9431ba10 \N admin@gmail.com created Crew Schedules 05248dc4-f09e-4313-b4c0-b1292c8afdbd Crew Schedules created: 05248dc4-f09e-4313-b4c0-b1292c8afdbd {} 2026-09-08 22:39:28.809772 2026-09-08 14:39:28.809772+00 05248dc4-f09e-4313-b4c0-b1292c8afdbd 143.44.164.96 bf8a8385-edd7-4d44-ac44-423017373a10 \N admin@gmail.com created Crew Schedules 00dec071-a934-4be8-82e1-dd9cced1f2b5 Crew Schedules created: 00dec071-a934-4be8-82e1-dd9cced1f2b5 {} 2026-09-08 22:39:28.809772 2026-09-08 14:39:28.809772+00 00dec071-a934-4be8-82e1-dd9cced1f2b5 143.44.164.96 6f60f184-fdc7-497f-91fc-6523d4aede9c \N admin@gmail.com created Crew Schedules 780314e8-6776-4c4f-b04e-ba3041c81926 Crew Schedules created: 780314e8-6776-4c4f-b04e-ba3041c81926 {} 2026-09-08 22:40:06.579188 2026-09-08 14:40:06.579188+00 780314e8-6776-4c4f-b04e-ba3041c81926 143.44.164.96 d8d74ebb-76eb-44bb-b54a-e3694b6660d9 \N admin@gmail.com created Crew Schedules 52b3ccd4-e0f4-40c5-a08a-f8e992ac0bf2 Crew Schedules created: 52b3ccd4-e0f4-40c5-a08a-f8e992ac0bf2 {} 2026-09-08 22:40:06.579188 2026-09-08 14:40:06.579188+00 52b3ccd4-e0f4-40c5-a08a-f8e992ac0bf2 143.44.164.96 73ba324c-aed0-462e-8785-4f5b86b15f93 \N admin@gmail.com created Crew Schedules daa276b4-a0f7-4509-b932-6c123283f46a Crew Schedules created: daa276b4-a0f7-4509-b932-6c123283f46a {} 2026-09-08 22:40:06.579188 2026-09-08 14:40:06.579188+00 daa276b4-a0f7-4509-b932-6c123283f46a 143.44.164.96 cbe6860d-2738-42e5-bcf7-55b51f2f6dfc \N admin@gmail.com created Crew Schedules 87c0102a-6342-45de-a506-53e6b62df8ee Crew Schedules created: 87c0102a-6342-45de-a506-53e6b62df8ee {} 2026-09-08 22:40:06.579188 2026-09-08 14:40:06.579188+00 87c0102a-6342-45de-a506-53e6b62df8ee 143.44.164.96 90130d56-2d67-4c6d-a0cd-eadcd686a222 \N admin@gmail.com created Crew Schedules f4ed1d97-15c7-42e1-8e84-beeaa63a458d Crew Schedules created: f4ed1d97-15c7-42e1-8e84-beeaa63a458d {} 2026-09-08 22:40:06.579188 2026-09-08 14:40:06.579188+00 f4ed1d97-15c7-42e1-8e84-beeaa63a458d 143.44.164.96 75506a89-d2d4-44f4-98af-ef331552aa6f \N admin@gmail.com created

Crew Schedules 5ce2d683-9b0f-4f92-b008-a05c1d714812 Crew Schedules created: 5ce2d683-9b0f-4f92-b008-a05c1d714812 {} 2026-09-08 22:40:06.579188 2026-09-08 14:40:06.579188+00 5ce2d683-9b0f-4f92-b008-a05c1d714812 143.44.164.96 c0e3c042-4ec3-4b4b-af9a-14744b5b3471 \N admin@gmail.com created Crew Schedules 55b4cf81-5027-4b55-85c5-db9def697c5d Crew Schedules created: 55b4cf81-5027-4b55-85c5-db9def697c5d {} 2026-09-08 22:40:06.579188 2026-09-08 14:40:06.579188+00 55b4cf81-5027-4b55-85c5-db9def697c5d 143.44.164.96 35efb560-6a0b-437b-996b-e166d1ce4d3f \N admin@gmail.com created Crew Schedules 62ad33d8-8b9a-4deb-a0e7-e9f3de1b78d1 Crew Schedules created: 62ad33d8-8b9a-4deb-a0e7-e9f3de1b78d1 {} 2026-09-08 22:40:06.579188 2026-09-08 14:40:06.579188+00 62ad33d8-8b9a-4deb-a0e7-e9f3de1b78d1 143.44.164.96 f18914c8-baae-49f3-8c1f-bcbdec6537f2 \N admin@gmail.com created Crew Schedules 315826f0-84c2-4906-833c-15ecf01a5541 Crew Schedules created: 315826f0-84c2-4906-833c-15ecf01a5541 {} 2026-09-08 22:40:06.579188 2026-09-08 14:40:06.579188+00 315826f0-84c2-4906-833c-15ecf01a5541 143.44.164.96 52c0504e-f2b9-4781-8df9-044c2ffe2250 \N admin@gmail.com created Crew Schedules 7b09228e-174c-4596-b7fe-c9dbf43e092d Crew Schedules created: 7b09228e-174c-4596-b7fe-c9dbf43e092d {} 2026-09-08 22:40:06.579188 2026-09-08 14:40:06.579188+00 7b09228e-174c-4596-b7fe-c9dbf43e092d 143.44.164.96 aa5d4117-0d7e-4f71-8002-9b8f7e4e035b \N admin@gmail.com created Crew Schedules e9e878d7-8c19-47d4-9adb-e004cc002c2f Crew Schedules created: e9e878d7-8c19-47d4-9adb-e004cc002c2f {} 2026-09-08 22:40:06.579188 2026-09-08 14:40:06.579188+00 e9e878d7-8c19-47d4-9adb-e004cc002c2f 143.44.164.96 db57be20-1e62-430b-abfa-3d52a8be26cb \N admin@gmail.com created Crew Schedules 6fa611fd-36fd-40a2-8d72-511cb848044a Crew Schedules created: 6fa611fd-36fd-40a2-8d72-511cb848044a {} 2026-09-08 22:40:06.579188 2026-09-08 14:40:06.579188+00 6fa611fd-36fd-40a2-8d72-511cb848044a 143.44.164.96 5d38425c-a5f3-4116-af1e-2eeb17f58ff9 \N admin@gmail.com created Crew Schedules d31ee468-751e-4983-a8f6-0078d2e295ad Crew Schedules created: d31ee468-751e-4983-a8f6-0078d2e295ad {} 2026-09-08 22:40:06.579188 2026-09-08 14:40:06.579188+00 d31ee468-751e-4983-a8f6-0078d2e295ad 143.44.164.96 079ff2ed-62fe-4e8c-921f-3871bf007688 \N admin@gmail.com created Crew Schedules e6eff3ec-06a6-4c8f-94e6-8e1183c20655 Crew Schedules created: e6eff3ec-06a6-4c8f-94e6-8e1183c20655 {} 2026-09-08 22:40:06.579188 2026-09-08 14:40:06.579188+00 e6eff3ec-06a6-4c8f-94e6-8e1183c20655 143.44.164.96 fa3e9332-f0c7-498e-acdf-455d85c1b506 \N admin@gmail.com updated

Schedule Blocks 2026-08-25 Schedule Blocks updated: 2026-08-25 {reason} 2026-09-09 00:08:59.583053 2026-09-08 16:08:59.583053+00 36e5d275-0d4e-43cf-af25-1abbdc52588a 143.44.164.96 a8848ce6-ea3d-420b-883a-d44c2687d6f9 \N admin@gmail.com updated Products Spark Plug Products updated: Spark Plug {stock_quantity} 2026-09-09 00:09:57.264931 2026-09-08 16:09:57.264931+00 9b221a1e-640e-49d4-8635-3a96d08dee27 143.44.164.96 5a8ddc78-961e-45ff-9d42-d74ab469386d \N admin@gmail.com signed out Authentication Admin console Administrator signed out of the admin console. {} 2026-09-09 20:38:32.671425 2026-09-09 12:38:32.671425+00 \N 143.44.164.251 b14dfa0b-80e7-4ea1-9edd-a0e95e730dc0 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-09 20:38:57.075227 2026-09-09 12:38:57.075227+00 \N 143.44.164.251 c1739a7c-ccd2-4418-8918-ec96523889d6 \N admin@gmail.com created Services Motorcylce Tune-Up Services created: Motorcylce Tune-Up {} 2026-09-10 00:29:30.294761 2026-09-09 16:29:30.294761+00 6a7f36bb-0ec8-47f4-bf32-54c09de9b4b0 143.44.164.251 07c3d8c8-5e58-48a5-9f5e-8798aed97c1e \N admin@gmail.com created Services Chain & Sprocket Replacement Services created: Chain & Sprocket Replacement {} 2026-09-10 00:30:38.684952 2026-09-09 16:30:38.684952+00 1477338f-ec6e-4bf6-b916-b619cc58ac0d 143.44.164.251 2156c563-a0f2-4d04-b9c8-858d36a0842a \N admin@gmail.com created Services Cooling System Service Services created: Cooling System Service {} 2026-09-10 00:32:42.080501 2026-09-09 16:32:42.080501+00 61b18a4e-6804-4701-b035-b8e7913f7911 143.44.164.251 f80afaed-f207-43c0-a301-278be64b1c30 \N admin@gmail.com deactivated Services Clutch Adjustment/Repair Services deactivated: Clutch Adjustment/Repair {is_active} 2026-09-10 00:34:15.299049 2026-09-09 16:34:15.299049+00 f0132c1b-d630-4aa3-a56b-165ce0c408ec 143.44.164.251 6e350fcd-0f43-4e27-b6a5-24a7d861a2af \N admin@gmail.com activated Services Clutch Adjustment/Repair Services activated: Clutch Adjustment/Repair {is_active} 2026-09-10 00:34:24.116938 2026-09-09 16:34:24.116938+00 f0132c1b-d630-4aa3-a56b-165ce0c408ec 143.44.164.251 cf3585a2-1f39-4988-ac16-636d61001e66 \N admin@gmail.com created Services Fork Seal Replacement Services created: Fork Seal Replacement {} 2026-09-10 00:35:39.317517 2026-09-09 16:35:39.317517+00 e724f832-2a1f-4949-bf6f-ad72240d99ea 143.44.164.251 bbec3b92-eaac-4f5d-a681-494a60e82182 \N admin@gmail.com created Services Steering Bearing Replacement Services created: Steering Bearing Replacement {} 2026-09-10 00:36:52.661885 2026-09-09

16:36:52.661885+00 f01fd9c7-6a0f-4510-bf61-fd7469d1290a
143.44.164.251
9f8341cb-f91b-4f97-ae51-42f52bc80161 \N admin@gmail.com created
Services Motorcycle Safety Inspection Services created: Motorcycle Safety
Inspection {} 2026-09-10 00:37:45.999994 2026-09-09
16:37:45.999994+00 e04fec1b-0083-46cf-95d4-5765ba807518
143.44.164.251
8af4d943-ff4c-4d58-a317-6cdd0a0bdc6f \N admin@gmail.com updated
Notifications 5ab536cc-1e96-4687-8b66-0e22b8e45c50 Notifications
updated: 5ab536cc-1e96-4687-8b66-0e22b8e45c50 {is_read} 2026-09-10
00:59:30.495364 2026-09-09 16:59:30.495364+00
5ab536cc-1e96-4687-8b66-0e22b8e45c50 143.44.164.251
25534ac9-1a97-47e9-9ec2-54c01951913c \N admin@gmail.com updated
Notifications fb2ba884-1169-4233-86fb-a763d47af876 Notifications updated:
fb2ba884-1169-4233-86fb-a763d47af876 {is_read} 2026-09-10
01:01:51.884351 2026-09-09 17:01:51.884351+00
fb2ba884-1169-4233-86fb-a763d47af876 143.44.164.251
2218728d-3b65-4723-8c05-cb0d5db05b5b \N admin@gmail.com updated
Notifications 33394fe7-667a-4081-9bf5-d4e7f54f6d7b Notifications updated:
33394fe7-667a-4081-9bf5-d4e7f54f6d7b {is_read} 2026-09-10
01:01:55.549663 2026-09-09 17:01:55.549663+00
33394fe7-667a-4081-9bf5-d4e7f54f6d7b 143.44.164.251
75e13406-dee3-462d-85c4-37126872d4de \N admin@gmail.com updated
Notifications 50d9426f-6718-4cc0-a202-926ed26bb6c0 Notifications
updated: 50d9426f-6718-4cc0-a202-926ed26bb6c0 {is_read} 2026-09-10
01:01:58.528769 2026-09-09 17:01:58.528769+00
50d9426f-6718-4cc0-a202-926ed26bb6c0 143.44.164.251
73ab325d-fa03-4d0f-856a-759f85d445ed \N admin@gmail.com updated
Notifications 6ffc3a7e-63eb-4699-98e6-0504ba9c32e9 Notifications updated:
6ffc3a7e-63eb-4699-98e6-0504ba9c32e9 {is_read} 2026-09-10
01:02:00.668968 2026-09-09 17:02:00.668968+00
6ffc3a7e-63eb-4699-98e6-0504ba9c32e9 143.44.164.251
90274ca6-71e8-42cd-a438-5ac2e4e93818 \N admin@gmail.com updated
Notifications c18e8821-37c1-4ce7-ab5e-c48e7d27004b Notifications
updated: c18e8821-37c1-4ce7-ab5e-c48e7d27004b {is_read} 2026-09-10
01:02:02.205246 2026-09-09 17:02:02.205246+00
c18e8821-37c1-4ce7-ab5e-c48e7d27004b 143.44.164.251
cea2b210-435c-4ce3-8943-d8321e9c6342 \N admin@gmail.com updated
Notifications 7d6320ec-6fd4-4bb0-812e-2ec16359303b Notifications
updated: 7d6320ec-6fd4-4bb0-812e-2ec16359303b {is_read} 2026-09-10
01:02:07.23286 2026-09-09 17:02:07.23286+00
7d6320ec-6fd4-4bb0-812e-2ec16359303b 143.44.164.251
3fe4d869-0e13-483e-8a1b-17cc295eae58 \N admin@gmail.com updated
Notifications d580264a-4763-412f-b30a-d07b4a390138 Notifications
updated: d580264a-4763-412f-b30a-d07b4a390138 {is_read} 2026-09-10

01:02:07.23286 2026-09-09 17:02:07.23286+00
d580264a-4763-412f-b30a-d07b4a390138 143.44.164.251
b699bf6c-c353-43fb-ace9-b821a08e0650 \N admin@gmail.com updated
Notifications f390d004-5143-49b7-93f0-3d667711227f Notifications updated:
f390d004-5143-49b7-93f0-3d667711227f {is_read} 2026-09-10
01:02:07.23286 2026-09-09 17:02:07.23286+00
f390d004-5143-49b7-93f0-3d667711227f 143.44.164.251
051be4bc-37e2-47c2-a928-de7c680b1b5b \N admin@gmail.com updated
Notifications f2c1ffad-41a6-45d5-bd2c-29f572131e28 Notifications updated:
f2c1ffad-41a6-45d5-bd2c-29f572131e28 {is_read} 2026-09-10
01:02:07.23286 2026-09-09 17:02:07.23286+00
f2c1ffad-41a6-45d5-bd2c-29f572131e28 143.44.164.251
1390369c-5773-47c4-ac7e-22acb0bb6a06 \N admin@gmail.com updated
Notifications 524f7119-e2ce-4d5f-970d-d3aaa217ebce Notifications updated:
524f7119-e2ce-4d5f-970d-d3aaa217ebce {is_read} 2026-09-10
01:02:07.23286 2026-09-09 17:02:07.23286+00
524f7119-e2ce-4d5f-970d-d3aaa217ebce 143.44.164.251
5756e526-f7a7-4627-b720-e8268711ce75 \N admin@gmail.com updated
Notifications 06479c06-1079-43a0-aa27-7ffec167e4f6 Notifications updated:
06479c06-1079-43a0-aa27-7ffec167e4f6 {is_read} 2026-09-10
01:02:07.23286 2026-09-09 17:02:07.23286+00
06479c06-1079-43a0-aa27-7ffec167e4f6 143.44.164.251
85aea468-eec4-4d9f-95c3-ca41066e97c1 \N admin@gmail.com activated
Products Motorcycle Tail Bag Products activated: Motorcycle Tail Bag
{is_active} 2026-09-10 09:41:10.627756 2026-09-10 01:41:10.627756+00
8247c0ee-a2c2-4e42-81e0-99e7ed002c5d 49.146.140.233
ba89c612-8d20-4cf4-8b19-34baf99360d7 \N admin@gmail.com signed out
Authentication Admin console Administrator signed out of the admin
console. {} 2026-09-10 12:22:51.713159 2026-09-10
04:22:51.713159+00 \N 49.146.140.233
37549917-0560-484c-bbab-e0477743f73e \N admin@gmail.com signed in
Authentication Admin console Administrator signed in to the admin console.
{} 2026-09-10 12:23:00.302141 2026-09-10 04:23:00.302141+00 \N
49.146.140.233
38b00d67-9377-42bd-a8c8-f0028e8e65bf \N admin@gmail.com status
changed Appointments FRM-6JQ8FY Appointments status changed:
FRM-6JQ8FY {status} 2026-09-10 12:24:28.821687 2026-09-10
04:24:28.821687+00 42fb7740-5457-4460-b968-78c0e4b19027
49.146.140.233
cede34e5-9b9a-47a6-b851-3d2fbfeeb215 \N admin@gmail.com deleted
Appointment Services Change Oil Appointment Services deleted: Change Oil
{} 2026-09-10 12:24:28.821687 2026-09-10 04:24:28.821687+00
c0e1b4c2-91e1-4a04-8cb6-426eb89a54d5 49.146.140.233
4b61c7d3-8547-46c3-9f16-dd381d6c5983 \N admin@gmail.com created
Appointment Services Change Oil Appointment Services created: Change Oil

{} 2026-09-10 12:24:28.821687 2026-09-10 04:24:28.821687+00
12c870c0-557d-4bfe-9972-9d7d90666485 49.146.140.233
ebaf3f2b-0b49-4621-98f7-57a2fa386e50 \N admin@gmail.com updated
Notifications abccc152-1583-4b34-b127-0284dd398a79 Notifications
updated: abccc152-1583-4b34-b127-0284dd398a79 {is_read} 2026-09-10
15:15:40.450474 2026-09-10 07:15:40.450474+00
abccc152-1583-4b34-b127-0284dd398a79 49.146.140.233
9530442c-705f-4e5b-a2ad-b1e6677f02c5 \N admin@gmail.com signed in
Authentication Admin console Administrator signed in to the admin console.
{} 2026-09-10 16:15:31.925623 2026-09-10 08:15:31.925623+00 \N
49.146.140.233
f3cda58e-7b4e-43f5-bf19-fd4a33260d00 \N admin@gmail.com deleted
Notifications c18e8821-37c1-4ce7-ab5e-c48e7d27004b Notifications deleted:
c18e8821-37c1-4ce7-ab5e-c48e7d27004b {} 2026-09-11 18:22:20.112947
2026-09-11 10:22:20.112947+00 c18e8821-37c1-4ce7-ab5e-c48e7d27004b
143.44.164.241
db4e59e6-7d8c-4ca2-b66d-7039714a70f2 \N admin@gmail.com signed in
Authentication Admin console Administrator signed in to the admin console.
{} 2026-09-10 21:29:34.317405 2026-09-10 13:29:34.317405+00 \N
143.44.164.229
63f545a7-745c-4ff5-9368-ae9e8b434566 \N admin@gmail.com archived
Appointments FRM-Y5KSUN Appointments archived: FRM-Y5KSUN
{is_archived} 2026-09-10 21:29:51.968751 2026-09-10
13:29:51.968751+00 177fbdfa-a85e-4777-8a8c-760b9ad02401
143.44.164.229
b03df68d-b7fe-425b-a610-87d02f726984 \N admin@gmail.com status
changed Appointments FRM-76C4GS Appointments status changed:
FRM-76C4GS {status} 2026-09-11 17:38:19.366304 2026-09-11
09:38:19.366304+00 49ecb311-2ad3-45fc-b9c1-bca9a53d9b13
143.44.164.241
0e5ad4b0-8d4a-4c12-9478-c4e1cb4ef2c7 \N admin@gmail.com deleted
Appointment Services Electrical Diagnostics Appointment Services deleted:
Electrical Diagnostics {} 2026-09-11 17:38:19.366304 2026-09-11
09:38:19.366304+00 694f8259-b89b-48a6-aaf7-d95ba148bb5f
143.44.164.241
490d6b15-7286-440b-a3be-2e700f1ef0a9 \N admin@gmail.com created
Appointment Services Electrical Diagnostics Appointment Services created:
Electrical Diagnostics {} 2026-09-11 17:38:19.366304 2026-09-11
09:38:19.366304+00 950eebc7-9449-44cd-8537-e7dc331a46e8
143.44.164.241
9f2d97fa-faa7-477b-9c24-47f4a9901729 \N admin@gmail.com restored
Appointments FRM-Y5KSUN Appointments restored: FRM-Y5KSUN
{is_archived} 2026-09-11 17:39:03.875585 2026-09-11
09:39:03.875585+00 177fbdfa-a85e-4777-8a8c-760b9ad02401
143.44.164.241

e982202f-269b-43b9-a70c-3eaf716cf13c \N admin@gmail.com status changed Appointments FRM-Y5KSUN Appointments status changed: FRM-Y5KSUN {status} 2026-09-11 17:39:11.058738 2026-09-11 09:39:11.058738+00 177fbdfa-a85e-4777-8a8c-760b9ad02401 143.44.164.241
359d6df2-ee9a-48b4-ae20-1e185b19b8ab \N admin@gmail.com updated Notifications 5871a557-ee35-4846-a707-e4742859521c Notifications updated: 5871a557-ee35-4846-a707-e4742859521c {is_read} 2026-09-11 17:39:18.638689 2026-09-11 09:39:18.638689+00 5871a557-ee35-4846-a707-e4742859521c 143.44.164.241
87881b81-d33f-4520-89a1-45d9c47f7d70 \N admin@gmail.com updated Appointments FRM-G4XL75 Appointments updated: FRM-G4XL75 {pendin g_reschedule_date,pending_reschedule_reason,pending_reschedule_request_id ,pending_reschedule_start_time} 2026-09-11 18:05:10.938651 2026-09-11 10:05:10.938651+00 4acd6706-a379-4504-b7a6-ae8f896e5dbd 143.44.164.241
f3f48229-3e9d-4b25-bbbf-1918a3e7deef \N admin@gmail.com status changed Appointments FRM-G4XL75 Appointments status changed: FRM-G4XL75 {status} 2026-09-11 18:05:19.033278 2026-09-11 10:05:19.033278+00 4acd6706-a379-4504-b7a6-ae8f896e5dbd 143.44.164.241
4c1ec5ad-488b-4dac-a8bb-b44394a037c2 \N admin@gmail.com deleted Notifications 5871a557-ee35-4846-a707-e4742859521c Notifications deleted: 5871a557-ee35-4846-a707-e4742859521c {} 2026-09-11 18:17:30.378883 2026-09-11 10:17:30.378883+00 5871a557-ee35-4846-a707-e4742859521c 143.44.164.241
e3ae357d-8254-4bc6-8ec6-719157736d0b \N admin@gmail.com deleted Notifications 10f914eb-d495-4efc-b7ea-79808b655954 Notifications deleted: 10f914eb-d495-4efc-b7ea-79808b655954 {} 2026-09-11 18:22:20.112947 2026-09-11 10:22:20.112947+00 10f914eb-d495-4efc-b7ea-79808b655954 143.44.164.241
d6cf841f-3fbf-4241-8777-46f07c6d5770 \N admin@gmail.com deleted Notifications ee9f6c8f-7910-4942-ac7e-8d476c7e00ae Notifications deleted: ee9f6c8f-7910-4942-ac7e-8d476c7e00ae {} 2026-09-11 18:22:20.112947 2026-09-11 10:22:20.112947+00 ee9f6c8f-7910-4942-ac7e-8d476c7e00ae 143.44.164.241
169f2bd1-f043-4d39-a386-7e2809206ff2 \N admin@gmail.com deleted Notifications 5114fc1b-9073-4282-8004-1724b1f791e6 Notifications deleted: 5114fc1b-9073-4282-8004-1724b1f791e6 {} 2026-09-11 18:22:20.112947 2026-09-11 10:22:20.112947+00 5114fc1b-9073-4282-8004-1724b1f791e6 143.44.164.241
8c9d8678-af0d-4a9d-bbc1-f135a90da381 \N admin@gmail.com deleted Notifications 865a1042-1af3-42ac-bc0c-2a795601d3b5 Notifications deleted: 865a1042-1af3-42ac-bc0c-2a795601d3b5 {} 2026-09-11 18:22:20.112947 2026-09-11 10:22:20.112947+00 865a1042-1af3-42ac-bc0c-2a795601d3b5

143.44.164.241
6847d889-7213-429b-a437-2895eb3c0d23 \N admin@gmail.com deleted Notifications fa1c0fec-747d-41d5-96a4-51dd918a61c2 Notifications deleted: fa1c0fec-747d-41d5-96a4-51dd918a61c2 {} 2026-09-11 18:22:20.112947 2026-09-11 10:22:20.112947+00 fa1c0fec-747d-41d5-96a4-51dd918a61c2
143.44.164.241
cb019416-f3cc-4700-b05e-52f79c58ea37 \N admin@gmail.com deleted Notifications 0a05077f-008b-4fe9-a7b6-d7d951867ae4 Notifications deleted: 0a05077f-008b-4fe9-a7b6-d7d951867ae4 {} 2026-09-11 18:22:20.112947 2026-09-11 10:22:20.112947+00 0a05077f-008b-4fe9-a7b6-d7d951867ae4
143.44.164.241
6a76d1bf-059a-4d21-b89c-70fccb9d87a1 \N admin@gmail.com deleted Notifications 13b0d833-7230-4418-8f78-6b79bc507522 Notifications deleted: 13b0d833-7230-4418-8f78-6b79bc507522 {} 2026-09-11 18:22:20.112947 2026-09-11 10:22:20.112947+00 13b0d833-7230-4418-8f78-6b79bc507522
143.44.164.241
5207711a-3a80-4832-b271-a0de5a356ef1 \N admin@gmail.com deleted Notifications ad2aedc4-9712-4b59-a9eb-9304093c20b5 Notifications deleted: ad2aedc4-9712-4b59-a9eb-9304093c20b5 {} 2026-09-11 18:22:20.112947 2026-09-11 10:22:20.112947+00 ad2aedc4-9712-4b59-a9eb-9304093c20b5
143.44.164.241
41ad78ec-7e7e-42c2-a22a-5b8ec0707570 \N admin@gmail.com deleted Notifications 5ab536cc-1e96-4687-8b66-0e22b8e45c50 Notifications deleted: 5ab536cc-1e96-4687-8b66-0e22b8e45c50 {} 2026-09-11 18:22:20.112947 2026-09-11 10:22:20.112947+00 5ab536cc-1e96-4687-8b66-0e22b8e45c50
143.44.164.241
02d31179-2e19-4cef-a7a1-72921f7bd783 \N admin@gmail.com deleted Notifications fb2ba884-1169-4233-86fb-a763d47af876 Notifications deleted: fb2ba884-1169-4233-86fb-a763d47af876 {} 2026-09-11 18:22:20.112947 2026-09-11 10:22:20.112947+00 fb2ba884-1169-4233-86fb-a763d47af876
143.44.164.241
c6fd3f66-a0b1-43c8-9f72-cba876c30571 \N admin@gmail.com deleted Notifications 33394fe7-667a-4081-9bf5-d4e7f54f6d7b Notifications deleted: 33394fe7-667a-4081-9bf5-d4e7f54f6d7b {} 2026-09-11 18:22:20.112947 2026-09-11 10:22:20.112947+00 33394fe7-667a-4081-9bf5-d4e7f54f6d7b
143.44.164.241
0f368feb-0b5d-4da8-b18c-6b324a8714ae \N admin@gmail.com deleted Notifications 50d9426f-6718-4cc0-a202-926ed26bb6c0 Notifications deleted: 50d9426f-6718-4cc0-a202-926ed26bb6c0 {} 2026-09-11 18:22:20.112947 2026-09-11 10:22:20.112947+00 50d9426f-6718-4cc0-a202-926ed26bb6c0
143.44.164.241
f74cca4c-96bc-40de-805b-3067d23cedd3 \N admin@gmail.com deleted Notifications 6ffc3a7e-63eb-4699-98e6-0504ba9c32e9 Notifications deleted: 6ffc3a7e-63eb-4699-98e6-0504ba9c32e9 {} 2026-09-11 18:22:20.112947 2026-09-11 10:22:20.112947+00 6ffc3a7e-63eb-4699-98e6-0504ba9c32e9

143.44.164.241
42007bab-a405-467c-a3b3-355007c3fca2 \N admin@gmail.com exported Reports Booking Report Exported 22 bookings as DOCX. {report_type,date_range,export_format} 2026-09-14 11:06:33.253836 2026-09-14 03:06:33.253836+00 \N 49.146.140.233
e0886b42-192a-4fe5-bbbd-63bb991ae8dc \N admin@gmail.com exported Reports Booking Report Exported 21 bookings as PDF. {report_type,date_range,export_format} 2026-09-14 13:01:39.395284 2026-09-14 05:01:39.395284+00 \N 49.146.140.233
b0ddb887-c253-4ad1-b7e5-2569660b1315 \N admin@gmail.com status changed Appointments FRM-V6SZ37 Appointments status changed: FRM-V6SZ37 {status} 2026-09-14 13:10:50.835534 2026-09-14 05:10:50.835534+00 5546bc74-3047-4d98-87e4-67a68084a24d 49.146.140.233
9a64b147-10b4-4770-a2c4-0e493a275feb \N admin@gmail.com archived Appointments FRM-8EU6UX Appointments archived: FRM-8EU6UX {is_archived} 2026-09-15 17:16:25.027113 2026-09-15 09:16:25.027113+00 008af0e5-b100-4b1b-af5c-a4ccaf491020 143.44.164.66
780ebed4-4248-4551-8537-b76e52e493e9 \N admin@gmail.com deleted Notifications 7d6320ec-6fd4-4bb0-812e-2ec16359303b Notifications deleted: 7d6320ec-6fd4-4bb0-812e-2ec16359303b {} 2026-09-11 18:22:20.112947 2026-09-11 10:22:20.112947+00 7d6320ec-6fd4-4bb0-812e-2ec16359303b 143.44.164.241
195147d0-fb5b-4f12-8e91-50fe3eb94b80 \N admin@gmail.com deleted Notifications d580264a-4763-412f-b30a-d07b4a390138 Notifications deleted: d580264a-4763-412f-b30a-d07b4a390138 {} 2026-09-11 18:22:20.112947 2026-09-11 10:22:20.112947+00 d580264a-4763-412f-b30a-d07b4a390138 143.44.164.241
5efd3f2d-ad9d-4cb4-8e9b-a3aef5127c26 \N admin@gmail.com deleted Notifications f390d004-5143-49b7-93f0-3d667711227f Notifications deleted: f390d004-5143-49b7-93f0-3d667711227f {} 2026-09-11 18:22:20.112947 2026-09-11 10:22:20.112947+00 f390d004-5143-49b7-93f0-3d667711227f 143.44.164.241
2d3e4f4b-fcdd-4b74-b668-b4d15a90fb66 \N admin@gmail.com deleted Notifications f2c1ffad-41a6-45d5-bd2c-29f572131e28 Notifications deleted: f2c1ffad-41a6-45d5-bd2c-29f572131e28 {} 2026-09-11 18:22:20.112947 2026-09-11 10:22:20.112947+00 f2c1ffad-41a6-45d5-bd2c-29f572131e28 143.44.164.241
c3f10540-d332-48ca-a250-81fba237fa8f \N admin@gmail.com deleted Notifications 524f7119-e2ce-4d5f-970d-d3aaa217ebce Notifications deleted: 524f7119-e2ce-4d5f-970d-d3aaa217ebce {} 2026-09-11 18:22:20.112947 2026-09-11 10:22:20.112947+00 524f7119-e2ce-4d5f-970d-d3aaa217ebce 143.44.164.241
581abda3-99c4-4020-89b6-6a156fa8912b \N admin@gmail.com deleted Notifications 06479c06-1079-43a0-aa27-7ffec167e4f6 Notifications deleted:

06479c06-1079-43a0-aa27-7ffec167e4f6 {} 2026-09-11 18:22:20.112947 2026-09-11 10:22:20.112947+00 06479c06-1079-43a0-aa27-7ffec167e4f6 143.44.164.241
745236e8-9388-436c-9d35-6cd765524e43 \N admin@gmail.com deleted Notifications abccc152-1583-4b34-b127-0284dd398a79 Notifications deleted: abccc152-1583-4b34-b127-0284dd398a79 {} 2026-09-11 18:22:20.112947 2026-09-11 10:22:20.112947+00 abccc152-1583-4b34-b127-0284dd398a79 143.44.164.241
f23e93e1-5515-4200-9898-a824633281da \N admin@gmail.com archived Appointments FRM-GAY6TL Appointments archived: FRM-GAY6TL {is_archived} 2026-09-11 18:40:47.652364 2026-09-11 10:40:47.652364+00 5788a190-a4a7-4a96-b460-b29104e00491 143.44.164.241
9db0fe0f-2b70-41a0-bb6e-37e5a6bc9835 \N admin@gmail.com updated Appointments FRM-6JQ8FY Appointments updated: FRM-6JQ8FY {pendin g_reschedule_date,pending_reschedule_reason,pending_reschedule_request_id ,pending_reschedule_start_time} 2026-09-11 18:51:54.915383 2026-09-11 10:51:54.915383+00 42fb7740-5457-4460-b968-78c0e4b19027 143.44.164.241
a3f2683b-1472-4d66-9511-17b4dc6da558 \N admin@gmail.com signed out Authentication Admin console Administrator signed out of the admin console. {} 2026-09-12 08:52:07.153671 2026-09-12 00:52:07.153671+00 \N 143.44.164.123
86ad005a-d8b7-4ca8-991d-98585679e9e3 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-12 08:53:54.948674 2026-09-12 00:53:54.948674+00 \N 143.44.164.123
a22ae9fc-a7b5-4983-9213-40e97edfd068 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-12 09:36:59.550042 2026-09-12 01:36:59.550042+00 \N 143.44.164.123
01d8e73d-19e8-40de-af2f-693c5b30cdbc \N admin@gmail.com created Schedule Blocks 2026-09-30 Schedule Blocks created: 2026-09-30 {} 2026-09-12 09:40:22.326608 2026-09-12 01:40:22.326608+00 a08c7921-8917-491f-88c4-082f534fc20d 143.44.164.123
901b148b-fc09-4466-9545-838d7f0bd09c \N admin@gmail.com deactivated Schedule Blocks 2026-09-30 Schedule Blocks deactivated: 2026-09-30 {is_active} 2026-09-12 09:40:28.103958 2026-09-12 01:40:28.103958+00 a08c7921-8917-491f-88c4-082f534fc20d 143.44.164.123
15ce3df3-b285-4961-b8d3-7a9ace33a850 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-12 13:08:25.568218 2026-09-12 05:08:25.568218+00 \N 143.44.164.123
c04b84a4-6ef0-488d-bf47-db4c440cb7f5 \N admin@gmail.com updated

Notifications ccc2b0ee-1ef3-4933-a313-9f933a3f7308 Notifications updated: ccc2b0ee-1ef3-4933-a313-9f933a3f7308 {is_read} 2026-09-12 13:34:01.804932 2026-09-12 05:34:01.804932+00 ccc2b0ee-1ef3-4933-a313-9f933a3f7308 143.44.164.123 0572b89b-465b-4e8c-8888-e0d87be4a646 \N admin@gmail.com updated Notifications c61efa22-3358-4ab7-8c61-44fc61e6fb4d Notifications updated: c61efa22-3358-4ab7-8c61-44fc61e6fb4d {is_read} 2026-09-12 13:34:01.804932 2026-09-12 05:34:01.804932+00 c61efa22-3358-4ab7-8c61-44fc61e6fb4d 143.44.164.123 5ffa2ca4-fcb8-4045-8613-19539498d197 \N admin@gmail.com updated Notifications 56d00e3b-efb3-44c4-81fb-931ccbff4e2f Notifications updated: 56d00e3b-efb3-44c4-81fb-931ccbff4e2f {is_read} 2026-09-12 13:34:01.804932 2026-09-12 05:34:01.804932+00 56d00e3b-efb3-44c4-81fb-931ccbff4e2f 143.44.164.123 09055388-75ca-413f-bc1a-b379a1689866 \N admin@gmail.com updated Notifications 49885f8e-1806-450f-b19b-3d495ed1e143 Notifications updated: 49885f8e-1806-450f-b19b-3d495ed1e143 {is_read} 2026-09-12 13:34:01.804932 2026-09-12 05:34:01.804932+00 49885f8e-1806-450f-b19b-3d495ed1e143 143.44.164.123 9147d522-160f-4466-854e-5deffb1036b2 \N admin@gmail.com deleted Notifications 49885f8e-1806-450f-b19b-3d495ed1e143 Notifications deleted: 49885f8e-1806-450f-b19b-3d495ed1e143 {} 2026-09-12 13:34:12.81573 2026-09-12 05:34:12.81573+00 49885f8e-1806-450f-b19b-3d495ed1e143 143.44.164.123 eeb9f472-c513-4b40-842b-2fdf1b4852ae \N admin@gmail.com status changed Appointments FRM-SF6XZG Appointments status changed: FRM-SF6XZG {assigned_crew_id,status} 2026-09-12 17:25:07.57266 2026-09-12 09:25:07.57266+00 a99afab3-aef7-4875-82fa-a347ce834601 143.44.164.123 8a563d32-5a2c-4f21-b56f-d44e780acdb4 \N admin@gmail.com archived Appointments FRM-SF6XZG Appointments archived: FRM-SF6XZG {is_archived} 2026-09-12 17:25:20.581489 2026-09-12 09:25:20.581489+00 a99afab3-aef7-4875-82fa-a347ce834601 143.44.164.123 d3ca3b06-0a76-4c0a-8fb1-afd05dde2da6 \N admin@gmail.com created Products Beat Products created: Beat {} 2026-09-13 09:46:23.834606 2026-09-13 01:46:23.834606+00 1918048b-ca78-42b9-8ea7-6d1a98872e8a 143.44.164.123 1e7b1c05-b8ea-444e-b819-c1cc38b29e46 \N admin@gmail.com archived Products Beat Products archived: Beat {archived_at,is_archived} 2026-09-13 09:47:09.874297 2026-09-13 01:47:09.874297+00 1918048b-ca78-42b9-8ea7-6d1a98872e8a 143.44.164.123 2343566e-a955-48e2-bd5a-f9fdfd26edf8 \N admin@gmail.com updated Services Clutch Adjustment/Repair Services updated: Clutch Adjustment/Repair {price} 2026-09-13 09:49:58.385156 2026-09-13

01:49:58.385156+00 f0132c1b-d630-4aa3-a56b-165ce0c408ec
143.44.164.123
ba559510-3a7a-42b9-80e7-d7564349761a \N admin@gmail.com created
Service Configurations ef3dd1c5-9c9d-4545-9557-725f3f1f5a87 Service
Configurations created: ef3dd1c5-9c9d-4545-9557-725f3f1f5a87 {}
2026-09-13 09:49:58.79015 2026-09-13 01:49:58.79015+00
ef3dd1c5-9c9d-4545-9557-725f3f1f5a87 143.44.164.123
384571b2-544b-46f6-8d73-c245fcd26f3a \N admin@gmail.com deleted
Service Configurations ef3dd1c5-9c9d-4545-9557-725f3f1f5a87 Service
Configurations deleted: ef3dd1c5-9c9d-4545-9557-725f3f1f5a87 {}
2026-09-13 09:50:41.36749 2026-09-13 01:50:41.36749+00
ef3dd1c5-9c9d-4545-9557-725f3f1f5a87 143.44.164.123
12a6afa5-966f-4415-9794-f45a3adb7308 \N admin@gmail.com created
Service Configurations dc4830d7-af0b-4bab-86e8-d36d2922378d Service
Configurations created: dc4830d7-af0b-4bab-86e8-d36d2922378d {}
2026-09-13 09:50:41.55737 2026-09-13 01:50:41.55737+00
dc4830d7-af0b-4bab-86e8-d36d2922378d 143.44.164.123
2b8f3ec1-00d8-4c6c-be73-d4d15e65ef33 \N admin@gmail.com created
Service Model Overrides 29dcd18f-7fa9-4352-886e-b3f1789170f0 Service
Model Overrides created: 29dcd18f-7fa9-4352-886e-b3f1789170f0 {}
2026-09-13 09:50:41.711587 2026-09-13 01:50:41.711587+00
29dcd18f-7fa9-4352-886e-b3f1789170f0 143.44.164.123
cebd2b53-06a8-473f-ad5e-ffd2d3e9a16a \N admin@gmail.com deleted
Service Configurations dc4830d7-af0b-4bab-86e8-d36d2922378d Service
Configurations deleted: dc4830d7-af0b-4bab-86e8-d36d2922378d {}
2026-09-13 09:51:21.413962 2026-09-13 01:51:21.413962+00
dc4830d7-af0b-4bab-86e8-d36d2922378d 143.44.164.123
9d575a7c-64a3-4464-ab26-8354f364e96a \N admin@gmail.com deleted
Service Model Overrides 29dcd18f-7fa9-4352-886e-b3f1789170f0 Service
Model Overrides deleted: 29dcd18f-7fa9-4352-886e-b3f1789170f0 {}
2026-09-13 09:51:21.500943 2026-09-13 01:51:21.500943+00
29dcd18f-7fa9-4352-886e-b3f1789170f0 143.44.164.123
e1cdabd0-e973-47fc-b295-ce048d1c59b6 \N admin@gmail.com created
Service Configurations 980ab8e4-d371-423d-b964-1e9217c8809e Service
Configurations created: 980ab8e4-d371-423d-b964-1e9217c8809e {}
2026-09-13 09:51:21.630943 2026-09-13 01:51:21.630943+00
980ab8e4-d371-423d-b964-1e9217c8809e 143.44.164.123
9fa39b4f-0b4b-46f3-926f-5e7ec8b1f1dc \N admin@gmail.com created
Service Model Overrides 0a5169ff-973a-43f0-b989-050edfa5f2b1 Service
Model Overrides created: 0a5169ff-973a-43f0-b989-050edfa5f2b1 {}
2026-09-13 09:51:21.754258 2026-09-13 01:51:21.754258+00
0a5169ff-973a-43f0-b989-050edfa5f2b1 143.44.164.123
7212d6ab-4e66-4756-ac57-3dcb2076f94b \N admin@gmail.com created
Service Model Overrides 41868c94-45bd-4d15-81ae-13d68cd7d109 Service
Model Overrides created: 41868c94-45bd-4d15-81ae-13d68cd7d109 {}

2026-09-13 09:51:21.754258 2026-09-13 01:51:21.754258+00 41868c94-45bd-4d15-81ae-13d68cd7d109 143.44.164.123 98bf1a64-7bc6-41b7-bc07-b9e332c958dc \N admin@gmail.com updated Services Change Oil Services updated: Change Oil {duration_minutes,price} 2026-09-13 10:30:36.407145 2026-09-13 02:30:36.407145+00 97a9714b-20e6-4402-a3ee-4fcdefc1e9db 143.44.164.123 92371281-e48c-4763-8cca-2511b7ef93f2 \N admin@gmail.com created Service Configurations a70d219b-03a5-4cfc-9990-c14f67f331c7 Service Configurations created: a70d219b-03a5-4cfc-9990-c14f67f331c7 {} 2026-09-13 10:30:36.917617 2026-09-13 02:30:36.917617+00 a70d219b-03a5-4cfc-9990-c14f67f331c7 143.44.164.123 12c21426-3906-43d8-9f57-b3394da71902 \N admin@gmail.com updated Services Carburetor / FI Cleaning Services updated: Carburetor / FI Cleaning {price} 2026-09-13 15:00:22.300175 2026-09-13 07:00:22.300175+00 e5cf562e-7122-440b-9483-033b78aca286 143.44.164.123 711f0370-aeb0-4cda-88ce-9e4f5a3270a3 \N admin@gmail.com created Price History fbc110eb-7ec5-49b8-a1ff-ed5c8cb461ed Price History created: fbc110eb-7ec5-49b8-a1ff-ed5c8cb461ed {} 2026-09-13 15:00:25.430932 2026-09-13 07:00:25.430932+00 fbc110eb-7ec5-49b8-a1ff-ed5c8cb461ed 143.44.164.123 4ba2434e-973c-4793-86b8-77358a27de2e \N admin@gmail.com created Service Model Overrides d26ba842-5852-42af-9404-70edb5c4e0d6 Service Model Overrides created: d26ba842-5852-42af-9404-70edb5c4e0d6 {} 2026-09-13 15:53:17.621127 2026-09-13 07:53:17.621127+00 d26ba842-5852-42af-9404-70edb5c4e0d6 143.44.164.123 4844073d-00d8-4008-89d8-2bbdec34bfda \N admin@gmail.com status changed Appointments FRM-A5AEET Appointments status changed: FRM-A5AEET {status} 2026-09-13 16:31:12.116425 2026-09-13 08:31:12.116425+00 39e2435e-9880-4fc5-babb-a5d9c15828b2 143.44.164.123 fa1d4c42-ccfc-4ece-904c-ead3eaadc254 \N admin@gmail.com updated Notifications 0b72b9dc-cda7-439b-bc14-b8461decbeb6 Notifications updated: 0b72b9dc-cda7-439b-bc14-b8461decbeb6 {is_read} 2026-09-13 16:31:16.589083 2026-09-13 08:31:16.589083+00 0b72b9dc-cda7-439b-bc14-b8461decbeb6 143.44.164.123 4e782d29-f891-4140-86f3-996c973bbca6 \N admin@gmail.com updated Notifications 90a9bcad-8c8d-45ba-b088-52e6af3c349f Notifications updated: 90a9bcad-8c8d-45ba-b088-52e6af3c349f {is_read} 2026-09-13 16:31:16.589083 2026-09-13 08:31:16.589083+00 90a9bcad-8c8d-45ba-b088-52e6af3c349f 143.44.164.123 883169c9-d052-4597-a229-262e964d90d8 \N admin@gmail.com deleted Notifications 90a9bcad-8c8d-45ba-b088-52e6af3c349f Notifications deleted: 90a9bcad-8c8d-45ba-b088-52e6af3c349f {} 2026-09-13 16:31:19.384622 2026-09-13 08:31:19.384622+00 90a9bcad-8c8d-45ba-b088-52e6af3c349f

143.44.164.123
8fbf65b9-76d2-4452-ae9d-b7d10484375c \N admin@gmail.com status changed Appointments FRM-A5AEET Appointments status changed: FRM-A5AEET {status} 2026-09-13 16:33:35.906306 2026-09-13 08:33:35.906306+00 39e2435e-9880-4fc5-babb-a5d9c15828b2
143.44.164.123
19ef047e-4761-41cd-9c69-b0779688e928 \N admin@gmail.com deleted Appointment Services Motorcycle Safety Inspection Appointment Services deleted: Motorcycle Safety Inspection {} 2026-09-13 16:33:35.906306 2026-09-13 08:33:35.906306+00 061cdfa7-c370-4551-82f4-3e85bd820604
143.44.164.123
da8623a1-4e6b-49d7-abb2-990a29f05424 \N admin@gmail.com created Appointment Services Motorcycle Safety Inspection Appointment Services created: Motorcycle Safety Inspection {} 2026-09-13 16:33:35.906306 2026-09-13 08:33:35.906306+00 16ef159a-e33a-4a1a-b1fd-6a720d6ed9aa
143.44.164.123
c2d769f3-e113-4249-a1d4-1ff52557d18d \N admin@gmail.com status changed Appointments FRM-EFGUWJ Appointments status changed: FRM-EFGUWJ {status,total_estimate} 2026-09-13 17:41:10.403112 2026-09-13 09:41:10.403112+00 34c976d2-745d-4c5e-b994-354d019706e8
143.44.164.123
faa2f75d-7351-4baf-8266-cde61f4bff89 \N admin@gmail.com deleted Appointment Services Clutch Adjustment/Repair Appointment Services deleted: Clutch Adjustment/Repair {} 2026-09-13 17:41:10.403112 2026-09-13 09:41:10.403112+00 c0dfc84a-fa16-4fee-947c-c0b313da10c5
143.44.164.123
1b5bb64d-7b2d-4fbc-ac6f-2561b61fd086 \N admin@gmail.com created Appointment Services Clutch Adjustment/Repair Appointment Services created: Clutch Adjustment/Repair {} 2026-09-13 17:41:10.403112 2026-09-13 09:41:10.403112+00 965999f0-ff62-4fff-866d-17855b0b0f19
143.44.164.123
e5dbdfb8-ae45-4f26-810e-4d8edf97794d \N admin@gmail.com updated Appointments FRM-EFGUWJ Appointments updated: FRM-EFGUWJ {customer_name,middle_name} 2026-09-13 17:41:10.403112 2026-09-13 09:41:10.403112+00 34c976d2-745d-4c5e-b994-354d019706e8
143.44.164.123
75e6be40-d010-4b94-901b-d632c9ca74e1 \N admin@gmail.com status changed Appointments FRM-G4XL75 Appointments status changed: FRM-G4XL75 {status} 2026-09-13 17:41:36.256669 2026-09-13 09:41:36.256669+00 4acd6706-a379-4504-b7a6-ae8f896e5dbd
143.44.164.123
6c6eedb7-200b-443e-aada-4ee59af21def \N admin@gmail.com deleted Appointment Services Tire Change & Balancing Appointment Services deleted: Tire Change & Balancing {} 2026-09-13 17:41:36.256669 2026-09-13 09:41:36.256669+00 d685a9a8-6c9c-4dad-94ec-78b1cf9fdefa

143.44.164.123
dce028a2-36c2-489d-be93-79abbe0793d5 \N admin@gmail.com created Appointment Services Tire Change & Balancing Appointment Services created: Tire Change & Balancing {} 2026-09-13 17:41:36.256669 2026-09-13 09:41:36.256669+00 0964606f-2680-4d5a-9acd-85750d04c611 143.44.164.123
7e1d2027-70e2-474f-9065-ca52b92946ee \N admin@gmail.com updated Appointments FRM-G4XL75 Appointments updated: FRM-G4XL75 {customer_name,middle_name} 2026-09-13 17:41:36.256669 2026-09-13 09:41:36.256669+00 4acd6706-a379-4504-b7a6-ae8f896e5dbd 143.44.164.123
aea0f2d9-85a6-4bad-b70d-932d6dea69e8 \N admin@gmail.com deactivated Products Spark Plug Products deactivated: Spark Plug {is_active} 2026-09-13 17:47:09.303789 2026-09-13 09:47:09.303789+00 9b221a1e-640e-49d4-8635-3a96d08dee27 143.44.164.123
381c1b95-63f6-4669-9c4e-f6822cccc1be \N admin@gmail.com activated Products Spark Plug Products activated: Spark Plug {is_active} 2026-09-13 17:48:02.214988 2026-09-13 09:48:02.214988+00 9b221a1e-640e-49d4-8635-3a96d08dee27 143.44.164.123
f19492d3-a3a8-4b85-b74a-172a43cc00f1 \N admin@gmail.com deactivated Crew Members JR Salazar Crew Members deactivated: JR Salazar {is_active} 2026-09-13 17:50:37.44511 2026-09-13 09:50:37.44511+00 f3c5770f-e485-4efc-83f6-892422d8592e 143.44.164.123
8c5b1427-6b79-4718-826d-e216ecc626e8 \N admin@gmail.com activated Crew Members JR Salazar Crew Members activated: JR Salazar {is_active} 2026-09-13 17:50:38.776592 2026-09-13 09:50:38.776592+00 f3c5770f-e485-4efc-83f6-892422d8592e 143.44.164.123
4846654c-38cd-4609-b799-e96b733d3c49 \N admin@gmail.com exported Reports Booking Report Exported 1 bookings as PDF. {report_type,date_range,export_format} 2026-09-13 22:13:08.788983 2026-09-13 14:13:08.788983+00 \N 143.44.164.123
92632dcd-b6db-4247-a87c-a6ad8527eca4 \N admin@gmail.com exported Reports Booking Report Exported 19 bookings as PDF. {report_type,date_range,export_format} 2026-09-14 08:21:09.154367 2026-09-14 00:21:09.154367+00 \N 49.146.140.233
83f690f4-e55f-4854-8736-6ab119b945c6 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-14 09:13:35.531012 2026-09-14 01:13:35.531012+00 \N 49.146.140.233
4950e85a-88ac-4508-96da-d2dd620dd6df \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-14 10:38:26.404399 2026-09-14 02:38:26.404399+00 \N 49.146.140.233
ffc3edd2-04ab-4654-81db-6d438079feb9 \N admin@gmail.com deleted Appointments FRM-3GK8D3 Appointments deleted: FRM-3GK8D3 {}

2026-09-15 17:18:47.570759 2026-09-15 09:18:47.570759+00 95d9a02d-e47b-4652-8ebd-542cefe276c7 143.44.164.66 96b04929-d58f-45bf-a54c-fe573cfac92d \N admin@gmail.com updated Appointments FRM-V6SZ37 Appointments updated: FRM-V6SZ37 {appointment_date,pending_reschedule_date,pending_reschedule_reason,pending_reschedule_request_id,pending_reschedule_start_time,reschedule_count}
2026-09-14 13:11:51.862 2026-09-14 05:11:51.862+00 5546bc74-3047-4d98-87e4-67a68084a24d 49.146.140.233 ea2ab3db-0671-4bc4-8e48-e9cea47ce981 \N admin@gmail.com updated Appointments FRM-V6SZ37 Appointments updated: FRM-V6SZ37 {appointment_date,pending_reschedule_date,pending_reschedule_reason,pending_reschedule_request_id,pending_reschedule_start_time,reschedule_count}
2026-09-14 13:34:41.902494 2026-09-14 05:34:41.902494+00 5546bc74-3047-4d98-87e4-67a68084a24d 49.146.140.233 ebfa4f52-03cc-454d-a2e5-56350e1290b9 \N admin@gmail.com archived Appointments FRM-V6SZ37 Appointments archived: FRM-V6SZ37 {is_archived} 2026-09-14 18:06:25.22285 2026-09-14 10:06:25.22285+00 5546bc74-3047-4d98-87e4-67a68084a24d 143.44.164.184 564bec89-3aac-4577-9359-5a90851f38a7 \N admin@gmail.com status changed Appointments FRM-JWSPJG Appointments status changed: FRM-JWSPJG {status} 2026-09-14 18:09:30.026105 2026-09-14 10:09:30.026105+00 13b28240-1671-4e50-9bb7-49ab02e21b9a 143.44.164.184 e2363b6c-d52d-438d-874c-b732573bb59e \N admin@gmail.com updated Notifications d47980d5-31d1-4e85-857a-bab7e0daab0b Notifications updated: d47980d5-31d1-4e85-857a-bab7e0daab0b {is_read} 2026-09-14 18:09:31.654501 2026-09-14 10:09:31.654501+00 d47980d5-31d1-4e85-857a-bab7e0daab0b 143.44.164.184 c3e54c7e-b221-4657-a9fb-28ed2faaa6d6 \N admin@gmail.com updated Notifications 453d2fcc-7126-4da3-9207-d6ab05fb36c2 Notifications updated: 453d2fcc-7126-4da3-9207-d6ab05fb36c2 {is_read} 2026-09-14 18:09:31.654501 2026-09-14 10:09:31.654501+00 453d2fcc-7126-4da3-9207-d6ab05fb36c2 143.44.164.184 190e6046-263d-4915-9b7e-a379b72cfb0d \N admin@gmail.com updated Notifications aea5d685-77e3-4fae-b972-db05dc5064ff Notifications updated: aea5d685-77e3-4fae-b972-db05dc5064ff {is_read} 2026-09-14 18:09:31.654501 2026-09-14 10:09:31.654501+00 aea5d685-77e3-4fae-b972-db05dc5064ff 143.44.164.184 3572738e-9491-4d9d-9379-236f57b1153a \N admin@gmail.com updated Notifications cf028a25-c235-414d-8804-544f9e255c6d Notifications updated: cf028a25-c235-414d-8804-544f9e255c6d {is_read} 2026-09-14 18:09:31.654501 2026-09-14 10:09:31.654501+00 cf028a25-c235-414d-8804-544f9e255c6d 143.44.164.184 b96cafd1-d21b-40d7-a726-f12c4e24b758 \N admin@gmail.com updated Appointments FRM-JWSPJG Appointments updated: FRM-JWSPJG {appoi

ntment_date,pending_reschedule_date,pending_reschedule_reason,pending_reschedule_request_id,pending_reschedule_start_time,reschedule_count,start_time} 2026-09-14 18:50:04.363037 2026-09-14 10:50:04.363037+00 13b28240-1671-4e50-9bb7-49ab02e21b9a 143.44.164.184 24200c7c-64b9-4ec2-b62f-274d58ae3509 \N admin@gmail.com updated Notifications fbe99389-297f-407c-902e-840075c75713 Notifications updated: fbe99389-297f-407c-902e-840075c75713 {is_read} 2026-09-14 19:34:30.044407 2026-09-14 11:34:30.044407+00 fbe99389-297f-407c-902e-840075c75713 143.44.164.184 ca0eb041-7683-4abe-bd72-b9cc54753a21 \N admin@gmail.com created Appointments FRM-QU6QPF Appointments created: FRM-QU6QPF {} 2026-09-14 19:37:21.955786 2026-09-14 11:37:21.955786+00 75a07427-ff65-4d7e-affb-fe489d59346c 143.44.164.184 d3e8f156-bfe9-4d2a-87f4-e272e208fe18 \N admin@gmail.com created Appointment Services Change Oil Appointment Services created: Change Oil {} 2026-09-14 19:37:21.955786 2026-09-14 11:37:21.955786+00 40f7452f-45c4-499c-a109-489c8ecddcb0 143.44.164.184 c3bcb0d9-17e5-426e-8fe4-69c1b7cbd3b9 \N admin@gmail.com status changed Appointments FRM-JWSPJG Appointments status changed: FRM-JWSPJG {admin_notes,pending_reschedule_date,pending_reschedule_reason,pending_reschedule_reference_code,pending_reschedule_request_id,pending_reschedule_start_time,reschedule_count,rescheduled_to_appointment_id,status} 2026-09-14 19:37:21.955786 2026-09-14 11:37:21.955786+00 13b28240-1671-4e50-9bb7-49ab02e21b9a 143.44.164.184 eda6cf3f-23a6-49fb-b78b-8d0cd63dc5e9 \N admin@gmail.com created Notifications 05a9d0cd-d29d-445f-a9b7-791c01c55239 Notifications created: 05a9d0cd-d29d-445f-a9b7-791c01c55239 {} 2026-09-14 19:37:21.955786 2026-09-14 11:37:21.955786+00 05a9d0cd-d29d-445f-a9b7-791c01c55239 143.44.164.184 05e12a16-ba58-4571-908f-0816a11fecb8 \N admin@gmail.com updated Notifications 05a9d0cd-d29d-445f-a9b7-791c01c55239 Notifications updated: 05a9d0cd-d29d-445f-a9b7-791c01c55239 {is_read} 2026-09-14 23:16:06.17024 2026-09-14 15:16:06.17024+00 05a9d0cd-d29d-445f-a9b7-791c01c55239 143.44.164.184 3fe829ec-3ce1-4e3c-b412-acfdb58a75ba \N admin@gmail.com created Appointments FRM-EZ6AHX Appointments created: FRM-EZ6AHX {} 2026-09-14 23:18:22.22438 2026-09-14 15:18:22.22438+00 cd19fd75-2ed8-4267-bf27-8660dab07d66 143.44.164.184 3a1b7876-19e9-4a86-a17f-b252a95d2fd9 \N admin@gmail.com created Appointment Services Change Oil Appointment Services created: Change Oil {} 2026-09-14 23:18:22.22438 2026-09-14 15:18:22.22438+00 3d859960-862f-453b-9e3b-00c7ac3bc544 143.44.164.184 4aac5d1a-09f9-428d-974b-0efeea71c5fb \N admin@gmail.com status changed Appointments FRM-QU6QPF Appointments status changed: FRM-QU6QPF {admin_notes,pending_reschedule_date,pending_reschedule_r

eason,pending_reschedule_reference_code,pending_reschedule_request_id,pending_reschedule_start_time,reschedule_count,rescheduled_to_appointment_id,status} 2026-09-14 23:18:22.22438 2026-09-14 15:18:22.22438+00 75a07427-ff65-4d7e-affb-fe489d59346c 143.44.164.184 460e5490-dd52-4443-9ec2-dbe926ecef0d \N admin@gmail.com created Notifications ded9c694-d142-4f79-b256-10ca37cf6dbe Notifications created: ded9c694-d142-4f79-b256-10ca37cf6dbe {} 2026-09-14 23:18:22.22438 2026-09-14 15:18:22.22438+00 ded9c694-d142-4f79-b256-10ca37cf6dbe 143.44.164.184
5de17d1c-5cd2-48a9-a1e7-e94ad8786a7b \N admin@gmail.com exported Reports Booking Report Exported 24 bookings as PDF. {report_type,date_range,export_format} 2026-09-15 08:01:56.612398 2026-09-15 00:01:56.612398+00 \N 49.146.140.233
a854bfc4-c781-48d2-84e8-36728320435f \N admin@gmail.com archived Appointments FRM-LMDFUU Appointments archived: FRM-LMDFUU {is_archived} 2026-09-15 17:15:53.585127 2026-09-15 09:15:53.585127+00 e25251f4-8f3d-4a8f-aa43-b120117373b9 143.44.164.66
bc87e8a2-4d83-4c13-a029-c573de5d605b \N admin@gmail.com archived Appointments FRM-69R7BV Appointments archived: FRM-69R7BV {is_archived} 2026-09-15 17:15:58.196318 2026-09-15 09:15:58.196318+00 205d1da8-7493-4bf1-994e-cb8534690e54 143.44.164.66
a88631eb-f0f3-4c3f-96de-5cea61907bd8 \N admin@gmail.com archived Appointments FRM-U449JR Appointments archived: FRM-U449JR {is_archived} 2026-09-15 17:16:01.219591 2026-09-15 09:16:01.219591+00 7f56c54a-98d8-4600-8eec-8aca53cedbce 143.44.164.66
67bdf8e4-ed75-44cc-bfc2-74ffca3e3b5f \N admin@gmail.com archived Appointments FRM-V2MMHU Appointments archived: FRM-V2MMHU {is_archived} 2026-09-15 17:16:03.625701 2026-09-15 09:16:03.625701+00 5a33fc40-19a9-4c2a-ad74-accf186fa8b0 143.44.164.66
8907e354-3e8d-4d52-a316-29f85fba0d2e \N admin@gmail.com archived Appointments FRM-FGC688 Appointments archived: FRM-FGC688 {is_archived} 2026-09-15 17:16:06.498589 2026-09-15 09:16:06.498589+00 b8624165-5c0d-4300-beb9-acced8dcd628 143.44.164.66
36d2c66d-34c6-4510-8431-e1f672a09097 \N admin@gmail.com archived Appointments FRM-3ZSRJY Appointments archived: FRM-3ZSRJY {is_archived} 2026-09-15 17:16:11.922003 2026-09-15 09:16:11.922003+00 352034e8-fc8b-4dfd-b4ff-82be445ecd66 143.44.164.66
81b6659f-4e87-4510-abc2-153f1a70e1d3 \N admin@gmail.com archived Appointments FRM-S46L7V Appointments archived: FRM-S46L7V {is_archived} 2026-09-15 17:16:15.704436 2026-09-15 09:16:15.704436+00 51e41217-bd40-4989-9cfd-978baa0fa0e1

143.44.164.66
c4a1f856-2432-4d15-bd82-fbfac4ecf92d \N admin@gmail.com archived Appointments FRM-LD9RVG Appointments archived: FRM-LD9RVG {is_archived} 2026-09-15 17:16:18.577717 2026-09-15 09:16:18.577717+00 386f5730-6398-493f-8653-732f00518296 143.44.164.66
04b089c2-c373-48d0-85fb-7e0fa2060de9 \N admin@gmail.com archived Appointments FRM-32QQPN Appointments archived: FRM-32QQPN {is_archived} 2026-09-15 17:16:21.747879 2026-09-15 09:16:21.747879+00 0cabcf66-cfb5-4059-a565-82a23aa4087c 143.44.164.66
df5d6512-d405-4c65-b31e-cb20615b4384 \N admin@gmail.com deleted Appointment Services Change Oil Appointment Services deleted: Change Oil {} 2026-09-15 17:18:11.976127 2026-09-15 09:18:11.976127+00 30e488d3-8c05-464c-afa5-f78ad07daec3 143.44.164.66
6876e111-d964-42cd-972d-108e1b2dcd43 \N admin@gmail.com deleted Appointments FRM-69R7BV Appointments deleted: FRM-69R7BV {} 2026-09-15 17:18:17.601207 2026-09-15 09:18:17.601207+00 205d1da8-7493-4bf1-994e-cb8534690e54 143.44.164.66
d998babd-765a-44ff-89aa-f8e7e1b83002 \N admin@gmail.com deleted Appointment Services Electrical Diagnostics Appointment Services deleted: Electrical Diagnostics {} 2026-09-15 17:18:17.601207 2026-09-15 09:18:17.601207+00 3936c05a-715a-477a-9353-7943854bc1e5 143.44.164.66
22c0d1a0-e03c-40ea-958a-67821fba65f6 \N admin@gmail.com archived Appointments FRM-DLBU9Q Appointments archived: FRM-DLBU9Q {is_archived} 2026-09-15 17:16:43.047604 2026-09-15 09:16:43.047604+00 20c522a1-fd55-4a8d-9104-5ba6fad8876a 143.44.164.66
376cfb1c-dc91-47b0-b973-517ba12c08d2 \N admin@gmail.com archived Appointments FRM-EFGUWJ Appointments archived: FRM-EFGUWJ {is_archived} 2026-09-15 17:16:51.769936 2026-09-15 09:16:51.769936+00 34c976d2-745d-4c5e-b994-354d019706e8 143.44.164.66
65598bea-cd5f-43bf-9c2e-9cf367fa4e57 \N admin@gmail.com archived Appointments FRM-QU6QPF Appointments archived: FRM-QU6QPF {is_archived} 2026-09-15 17:17:11.135684 2026-09-15 09:17:11.135684+00 75a07427-ff65-4d7e-affb-fe489d59346c 143.44.164.66
fbe1254e-7730-4577-9b62-86064197872c \N admin@gmail.com deleted Appointments FRM-76C4GS Appointments deleted: FRM-76C4GS {} 2026-09-15 17:17:53.374823 2026-09-15 09:17:53.374823+00 49ecb311-2ad3-45fc-b9c1-bca9a53d9b13 143.44.164.66
a2308bc2-014c-46d7-a3a1-6102fd35d3c4 \N admin@gmail.com deleted Appointment Services Electrical Diagnostics Appointment Services deleted: Electrical Diagnostics {} 2026-09-15 17:17:53.374823 2026-09-15

09:17:53.374823+00 950eebc7-9449-44cd-8537-e7dc331a46e8
143.44.164.66
9408952a-f1a7-4de6-831d-9f4da40dd00c \N admin@gmail.com deleted
Notifications ccc2b0ee-1ef3-4933-a313-9f933a3f7308 Notifications deleted:
ccc2b0ee-1ef3-4933-a313-9f933a3f7308 {} 2026-09-15 17:17:53.374823
2026-09-15 09:17:53.374823+00 ccc2b0ee-1ef3-4933-a313-9f933a3f7308
143.44.164.66
f8a8aa83-66cb-48f1-a3db-420d9c30c365 \N admin@gmail.com deleted
Appointments FRM-6JQ8FY Appointments deleted: FRM-6JQ8FY {}
2026-09-15 17:17:59.755032 2026-09-15 09:17:59.755032+00
42fb7740-5457-4460-b968-78c0e4b19027 143.44.164.66
749a4ba7-4a61-4666-b923-4c119c41f54a \N admin@gmail.com deleted
Appointment Services Change Oil Appointment Services deleted: Change Oil
{} 2026-09-15 17:17:59.755032 2026-09-15 09:17:59.755032+00
12c870c0-557d-4bfe-9972-9d7d90666485 143.44.164.66
5cdb44fd-7df3-482d-a752-19c746de5f35 \N admin@gmail.com deleted
Appointments FRM-G4XL75 Appointments deleted: FRM-G4XL75 {}
2026-09-15 17:18:02.92335 2026-09-15 09:18:02.92335+00
4acd6706-a379-4504-b7a6-ae8f896e5dbd 143.44.164.66
ffe7fc63-523b-460f-bd63-eed80705dfb7 \N admin@gmail.com deleted
Appointment Services Tire Change & Balancing Appointment Services
deleted: Tire Change & Balancing {} 2026-09-15 17:18:02.92335
2026-09-15 09:18:02.92335+00 0964606f-2680-4d5a-9acd-85750d04c611
143.44.164.66
2c551b59-aa92-4558-b688-2cda1f7ecb27 \N admin@gmail.com deleted
Notifications c61efa22-3358-4ab7-8c61-44fc61e6fb4d Notifications deleted:
c61efa22-3358-4ab7-8c61-44fc61e6fb4d {} 2026-09-15 17:18:02.92335
2026-09-15 09:18:02.92335+00 c61efa22-3358-4ab7-8c61-44fc61e6fb4d
143.44.164.66
1319ea01-0e08-4cd7-a392-80af157e0b2d \N admin@gmail.com deleted
Notifications 56d00e3b-efb3-44c4-81fb-931ccbff4e2f Notifications deleted:
56d00e3b-efb3-44c4-81fb-931ccbff4e2f {} 2026-09-15 17:18:02.92335
2026-09-15 09:18:02.92335+00 56d00e3b-efb3-44c4-81fb-931ccbff4e2f
143.44.164.66
be1a15e6-90dc-4f28-a989-72e3d2e8cfb2 \N admin@gmail.com deleted
Appointments FRM-LMDFUU Appointments deleted: FRM-LMDFUU {}
2026-09-15 17:18:14.792749 2026-09-15 09:18:14.792749+00
e25251f4-8f3d-4a8f-aa43-b120117373b9 143.44.164.66
39d64678-4863-49ee-b4c7-f5ede486a383 \N admin@gmail.com deleted
Appointment Services Electrical Diagnostics Appointment Services deleted:
Electrical Diagnostics {} 2026-09-15 17:18:14.792749 2026-09-15
09:18:14.792749+00 83b2524b-7b74-49c4-9079-7f0fe69e29f0
143.44.164.66
d004e6a0-ab30-4ba7-b519-14a774b8c604 \N admin@gmail.com deleted
Appointments FRM-U449JR Appointments deleted: FRM-U449JR {}

2026-09-15 17:18:20.423735 2026-09-15 09:18:20.423735+00 7f56c54a-98d8-4600-8eec-8aca53cedbce 143.44.164.66 19f5ad3d-08f4-4d1a-8000-170c12f8bfc3 \N admin@gmail.com deleted Appointment Services Brake Service Appointment Services deleted: Brake Service {} 2026-09-15 17:18:20.423735 2026-09-15 09:18:20.423735+00 067ea48b-907a-40b1-aa98-6e577f6e88d1 143.44.164.66 d8f98528-a56b-4831-a744-cd73b5403b13 \N admin@gmail.com deleted Appointment Services Electrical Diagnostics Appointment Services deleted: Electrical Diagnostics {} 2026-09-15 17:18:20.423735 2026-09-15 09:18:20.423735+00 c2cbd5c9-6d0a-45da-a84a-2dc3fb0120c0 143.44.164.66 6fea879a-7629-4447-b6ce-2304daed1b96 \N admin@gmail.com deleted Appointments FRM-V2MMHU Appointments deleted: FRM-V2MMHU {} 2026-09-15 17:18:23.403522 2026-09-15 09:18:23.403522+00 5a33fc40-19a9-4c2a-ad74-accf186fa8b0 143.44.164.66 9c903a39-93a2-42d9-887e-09e9ff86e867 \N admin@gmail.com deleted Appointment Services Tire Change & Balancing Appointment Services deleted: Tire Change & Balancing {} 2026-09-15 17:18:23.403522 2026-09-15 09:18:23.403522+00 ac1cc012-5ae7-4593-9ac2-a99b58a35e72 143.44.164.66 fcacf0ce-a1a8-4714-91a8-19f4f2e7a946 \N admin@gmail.com deleted Appointment Services Carburetor / FI Cleaning Appointment Services deleted: Carburetor / FI Cleaning {} 2026-09-15 17:18:23.403522 2026-09-15 09:18:23.403522+00 33a20a9d-c135-4b41-8dcb-ee1a5e74ab08 143.44.164.66 55c65220-6ca3-48fd-881d-ae71541997c2 \N admin@gmail.com deleted Appointments FRM-FGC688 Appointments deleted: FRM-FGC688 {} 2026-09-15 17:18:26.672795 2026-09-15 09:18:26.672795+00 b8624165-5c0d-4300-beb9-acced8dcd628 143.44.164.66 ca1a921b-3036-45a0-bcc6-bcdc28358fe8 \N admin@gmail.com deleted Appointment Services Engine Overhaul Appointment Services deleted: Engine Overhaul {} 2026-09-15 17:18:26.672795 2026-09-15 09:18:26.672795+00 ce5ca77e-ba33-4a5a-b3b6-bb9c57d73d6d 143.44.164.66 1ef2d8ef-91fc-4900-a395-f7694291f963 \N admin@gmail.com deleted Appointments FRM-3ZSRJY Appointments deleted: FRM-3ZSRJY {} 2026-09-15 17:18:29.950879 2026-09-15 09:18:29.950879+00 352034e8-fc8b-4dfd-b4ff-82be445ecd66 143.44.164.66 a790eed4-1b6a-471c-8759-804e162ab4b0 \N admin@gmail.com deleted Appointment Services Change Oil Appointment Services deleted: Change Oil {} 2026-09-15 17:18:29.950879 2026-09-15 09:18:29.950879+00 b664d9fb-6a56-4465-bf30-132f3cdcdf33 143.44.164.66 c4de1b55-9f3d-41bc-a501-fe7164daab6e \N admin@gmail.com deleted Appointments FRM-S46L7V Appointments deleted: FRM-S46L7V {} 2026-09-15 17:18:33.039243 2026-09-15 09:18:33.039243+00

51e41217-bd40-4989-9cfd-978baa0fa0e1 143.44.164.66
adba4a78-ebcd-478a-837a-69e33fe3d4ad \N admin@gmail.com deleted
Appointment Services Brake Service Appointment Services deleted: Brake
Service {} 2026-09-15 17:18:33.039243 2026-09-15 09:18:33.039243+00
fe83104e-2997-45d4-8073-c2358d136c53 143.44.164.66
6be40694-63ed-45a0-9d58-6ddc0c75be25 \N admin@gmail.com deleted
Appointments FRM-LD9RVG Appointments deleted: FRM-LD9RVG {}
2026-09-15 17:18:36.003606 2026-09-15 09:18:36.003606+00
386f5730-6398-493f-8653-732f00518296 143.44.164.66
67450879-7578-4e54-a018-6a926836c7ea \N admin@gmail.com deleted
Appointment Services Change Oil Appointment Services deleted: Change Oil
{} 2026-09-15 17:18:36.003606 2026-09-15 09:18:36.003606+00
b7d66380-a031-4fef-ae93-cdc4d064848a 143.44.164.66
7b7be2a3-3d2b-4631-81d7-76f1f555fcbc \N admin@gmail.com deleted
Appointments FRM-32QQPN Appointments deleted: FRM-32QQPN {}
2026-09-15 17:18:38.656879 2026-09-15 09:18:38.656879+00
0cabcf66-cfb5-4059-a565-82a23aa4087c 143.44.164.66
fc4486a5-171f-4b99-9b3b-ee6d82b92ea9 \N admin@gmail.com deleted
Appointment Services Change Oil Appointment Services deleted: Change Oil
{} 2026-09-15 17:18:38.656879 2026-09-15 09:18:38.656879+00
fae0a7e9-2df5-44cd-a761-d3c46e4bca7a 143.44.164.66
d1bdcbf4-4d10-4f26-8daa-5e247040e904 \N admin@gmail.com deleted
Appointments FRM-8EU6UX Appointments deleted: FRM-8EU6UX {}
2026-09-15 17:18:41.539184 2026-09-15 09:18:41.539184+00
008af0e5-b100-4b1b-af5c-a4ccaf491020 143.44.164.66
a5149e4a-44bb-4e5a-8363-cf7bf2aacd28 \N admin@gmail.com deleted
Appointment Services Clutch Adjustment/Repair Appointment Services
deleted: Clutch Adjustment/Repair {} 2026-09-15 17:18:41.539184
2026-09-15 09:18:41.539184+00 4e7e4754-a4f0-4a50-8654-477f48ddadd3
143.44.164.66
b180133c-5414-4820-990e-913e036334c7 \N admin@gmail.com deleted
Appointments FRM-NJYMJ5 Appointments deleted: FRM-NJYMJ5 {}
2026-09-15 17:18:44.283612 2026-09-15 09:18:44.283612+00
1c323343-47eb-4454-901a-7f9d179b2142 143.44.164.66
40b6c6ba-9b68-48a0-b540-eab2ee521f68 \N admin@gmail.com deleted
Appointment Services Change Oil Appointment Services deleted: Change Oil
{} 2026-09-15 17:18:44.283612 2026-09-15 09:18:44.283612+00
e36e244c-d280-41a2-80b8-67d07997fc15 143.44.164.66
5c2d6e19-5ace-4f57-a504-102bdcef5509 \N admin@gmail.com deleted
Appointment Services Suspension Tuning / Rebuild Appointment Services
deleted: Suspension Tuning / Rebuild {} 2026-09-15 17:18:44.283612
2026-09-15 09:18:44.283612+00 542b52c7-cd96-4ee5-8e6d-22d4a5ec2424
143.44.164.66
df28041d-4c7d-4afe-8825-2d1e04bdcb08 \N admin@gmail.com deleted
Appointments FRM-YGEZ95 Appointments deleted: FRM-YGEZ95 {}

2026-09-15 17:18:59.668587 2026-09-15 09:18:59.668587+00 44060b7e-afee-4d2d-be40-2162d589885b 143.44.164.66 1cb791d0-298e-4d28-bec9-ee1cb0f724ae \N admin@gmail.com deleted Appointment Services overhaul Appointment Services deleted: overhaul {} 2026-09-15 17:18:59.668587 2026-09-15 09:18:59.668587+00 d734dfb1-182b-4b5e-b3fb-774106c5cfd2 143.44.164.66 30b5b7d2-cc1e-43a0-b4fc-32da255ddf77 \N admin@gmail.com created Appointments FRM-77GZQE Appointments created: FRM-77GZQE {} 2026-09-15 17:45:58.46359 2026-09-15 09:45:58.46359+00 d070ed76-55d2-40e1-a55a-fc58fc88bfb8 143.44.164.66 8d8e2fc1-4aeb-406b-8c17-eaf7f87372b5 \N admin@gmail.com created Appointment Services Change Oil Appointment Services created: Change Oil {} 2026-09-15 17:45:58.46359 2026-09-15 09:45:58.46359+00 7a97d46b-5e7c-483b-9edf-84ec2b65201b 143.44.164.66 a40cdd74-d3f9-4ed1-8526-2922ba9aef88 \N admin@gmail.com status changed Appointments FRM-4J2DB9 Appointments status changed: FRM-4J2DB9 {admin_notes,pending_reschedule_date,pending_reschedule_reason,pending_reschedule_reference_code,pending_reschedule_request_id,pending_reschedule_start_time,reschedule_count,rescheduled_to_appointment_id,status} 2026-09-15 17:45:58.46359 2026-09-15 09:45:58.46359+00 71b72a10-644d-4b57-ace3-740377b48bd6 143.44.164.66 1374969d-b8a2-4074-8202-5df6e62456b8 \N admin@gmail.com created Notifications b2f57aab-0b1e-44bd-87d7-bf2da162cfc3 Notifications created: b2f57aab-0b1e-44bd-87d7-bf2da162cfc3 {} 2026-09-15 17:45:58.46359 2026-09-15 09:45:58.46359+00 b2f57aab-0b1e-44bd-87d7-bf2da162cfc3 143.44.164.66 4f0f8496-7d47-406c-8780-bfa87e02f756 \N admin@gmail.com updated Products Beatf1 Products updated: Beatf1 {name} 2026-09-15 19:04:17.757543 2026-09-15 11:04:17.757543+00 be5d79fd-3bf3-41fe-bd2f-362f8a9c8bfa 143.44.164.66 d93aebcc-9076-4b0a-b340-7c2cf77613a1 \N admin@gmail.com deactivated Products Brake pads Products deactivated: Brake pads {is_active} 2026-09-16 12:17:03.684333 2026-09-16 04:17:03.684333+00 9310ef87-578e-4fb2-b95b-725a0900c95a 180.193.217.82 5afbbc8d-4604-4e8d-abc9-773121f7d09c \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-17 22:34:22.630364 2026-09-17 14:34:22.630364+00 \N 143.44.164.184 fb82d58b-cc98-49b5-9458-e2b01dfda9ac \N admin@gmail.com status changed Appointments FRM-3JWTZU Appointments status changed: FRM-3JWTZU {status} 2026-09-18 10:11:05.14514 2026-09-18 02:11:05.14514+00 545503a4-1416-4f5f-83ba-ebc50f5912d7 143.44.164.241 e0221a75-fa27-4563-8093-b1ee2706a7cd \N admin@gmail.com updated Notifications 8696313f-065c-459b-a04b-2b037d49743a Notifications

updated: 8696313f-065c-459b-a04b-2b037d49743a {is_read} 2026-09-18 10:11:22.797259 2026-09-18 02:11:22.797259+00 8696313f-065c-459b-a04b-2b037d49743a 143.44.164.241 fe2a8bd0-88af-4cf7-8099-95684e4c92c1 \N admin@gmail.com updated Notifications dcc90fe3-1c23-44d3-9b78-ffee1ea354b9 Notifications updated: dcc90fe3-1c23-44d3-9b78-ffee1ea354b9 {is_read} 2026-09-18 10:11:22.797259 2026-09-18 02:11:22.797259+00 dcc90fe3-1c23-44d3-9b78-ffee1ea354b9 143.44.164.241 2d865ae6-d874-4934-894b-4156ea82309b \N admin@gmail.com updated Notifications b2f57aab-0b1e-44bd-87d7-bf2da162cfc3 Notifications updated: b2f57aab-0b1e-44bd-87d7-bf2da162cfc3 {is_read} 2026-09-18 10:11:22.797259 2026-09-18 02:11:22.797259+00 b2f57aab-0b1e-44bd-87d7-bf2da162cfc3 143.44.164.241 3728e124-5b4e-4309-8a4c-9cf55489b5c6 \N admin@gmail.com updated Notifications 84429060-5c91-4ade-b15c-45e8d06c3f4e Notifications updated: 84429060-5c91-4ade-b15c-45e8d06c3f4e {is_read} 2026-09-18 10:11:22.797259 2026-09-18 02:11:22.797259+00 84429060-5c91-4ade-b15c-45e8d06c3f4e 143.44.164.241 579fa664-68ee-4885-962c-26543241f09f \N admin@gmail.com updated Notifications cea778ae-ff65-4b6b-a64e-2e443e73661f Notifications updated: cea778ae-ff65-4b6b-a64e-2e443e73661f {is_read} 2026-09-18 10:11:22.797259 2026-09-18 02:11:22.797259+00 cea778ae-ff65-4b6b-a64e-2e443e73661f 143.44.164.241 16bf6295-b356-48fa-8cd4-d4e9c41b212d \N admin@gmail.com updated Notifications db503331-c1d2-41fb-9630-76d25f0dffde Notifications updated: db503331-c1d2-41fb-9630-76d25f0dffde {is_read} 2026-09-18 10:15:07.35421 2026-09-18 02:15:07.35421+00 db503331-c1d2-41fb-9630-76d25f0dffde 143.44.164.241 268e8aa3-83fc-45c7-b4e7-a2f70289b295 \N admin@gmail.com created Appointments FRM-CAAFST Appointments created: FRM-CAAFST {} 2026-09-18 10:15:34.993525 2026-09-18 02:15:34.993525+00 b7424607-9e7d-4fd4-ae4e-73dba6c26b21 143.44.164.241 b1cde685-0a27-4465-90a1-3db4ac452048 \N admin@gmail.com created Appointment Services Change Oil Appointment Services created: Change Oil {} 2026-09-18 10:15:34.993525 2026-09-18 02:15:34.993525+00 1377ff7d-848d-4eca-b8fd-fece474d5255 143.44.164.241 515eae55-ce81-4c12-9f5f-ad6188b578e0 \N admin@gmail.com status changed Appointments FRM-3JWTZU Appointments status changed: FRM-3JWTZU {admin_notes,pending_reschedule_date,pending_reschedule_reason,pending_reschedule_reference_code,pending_reschedule_request_id,pending_reschedule_start_time,reschedule_count,rescheduled_to_appointment_id,status} 2026-09-18 10:15:34.993525 2026-09-18 02:15:34.993525+00 545503a4-1416-4f5f-83ba-ebc50f5912d7 143.44.164.241 97483aab-b9f6-49d2-994c-909086c3e559 \N admin@gmail.com created Notifications a9d5a675-13e9-4439-9c2f-65a451762bdf Notifications created:

a9d5a675-13e9-4439-9c2f-65a451762bdf {} 2026-09-18 10:15:34.993525 2026-09-18 02:15:34.993525+00 a9d5a675-13e9-4439-9c2f-65a451762bdf 143.44.164.241
7873b0b6-2367-4bcb-ab8c-256f04a2ea3d \N admin@gmail.com updated Notifications a9d5a675-13e9-4439-9c2f-65a451762bdf Notifications updated: a9d5a675-13e9-4439-9c2f-65a451762bdf {is_read} 2026-09-18 10:15:41.456003 2026-09-18 02:15:41.456003+00 a9d5a675-13e9-4439-9c2f-65a451762bdf 143.44.164.241
c0e4c4d0-699d-42f5-b3d1-6baa7a54e6cf \N admin@gmail.com exported Reports Booking Report Exported 4 bookings as PDF. {report_type,date_range,export_format} 2026-09-18 10:22:08.732175 2026-09-18 02:22:08.732175+00 \N 143.44.164.241
51275bf3-3762-4524-a044-710f7bffe44c \N admin@gmail.com created Crew Schedules 14e7973d-fe5b-4497-9879-eaf5fe447220 Crew Schedules created: 14e7973d-fe5b-4497-9879-eaf5fe447220 {} 2026-09-18 17:06:58.654864 2026-09-18 09:06:58.654864+00 14e7973d-fe5b-4497-9879-eaf5fe447220 143.44.164.184
12dcb481-eb57-438c-86c6-087ab984a54e \N admin@gmail.com created Crew Schedules 50ce0b38-0e19-42b5-b9d6-0a7c2078030e Crew Schedules created: 50ce0b38-0e19-42b5-b9d6-0a7c2078030e {} 2026-09-18 17:06:58.654864 2026-09-18 09:06:58.654864+00 50ce0b38-0e19-42b5-b9d6-0a7c2078030e 143.44.164.184
c3cc0464-7766-4231-9496-9967d613698b \N admin@gmail.com created Crew Schedules cda3fa42-0c95-4af1-a35f-3c0d7eef9637 Crew Schedules created: cda3fa42-0c95-4af1-a35f-3c0d7eef9637 {} 2026-09-18 17:06:58.654864 2026-09-18 09:06:58.654864+00 cda3fa42-0c95-4af1-a35f-3c0d7eef9637 143.44.164.184
23f8d305-c546-4312-bd33-552e318fe3c0 \N admin@gmail.com created Crew Schedules 346a9955-6a92-4d8f-825d-6178ec167a44 Crew Schedules created: 346a9955-6a92-4d8f-825d-6178ec167a44 {} 2026-09-18 17:06:58.654864 2026-09-18 09:06:58.654864+00 346a9955-6a92-4d8f-825d-6178ec167a44 143.44.164.184
497090b2-9615-4264-802c-54ee746a94aa \N admin@gmail.com created Crew Schedules 87c51185-5cad-4a93-abd6-6c28a7a5f1a6 Crew Schedules created: 87c51185-5cad-4a93-abd6-6c28a7a5f1a6 {} 2026-09-18 17:06:58.654864 2026-09-18 09:06:58.654864+00 87c51185-5cad-4a93-abd6-6c28a7a5f1a6 143.44.164.184
cfba4992-6437-4a78-b235-2040c3e0c884 \N admin@gmail.com created Crew Schedules 7feff085-924f-4950-beaf-0aa2e0409ed9 Crew Schedules created: 7feff085-924f-4950-beaf-0aa2e0409ed9 {} 2026-09-18 17:06:58.654864 2026-09-18 09:06:58.654864+00 7feff085-924f-4950-beaf-0aa2e0409ed9 143.44.164.184
e7731bdd-ac8c-4ea7-bb7d-e5190b75af9b \N admin@gmail.com created Crew Schedules 275eb5cb-e5e1-45ca-914d-4b2b4ef4c68c Crew Schedules created: 275eb5cb-e5e1-45ca-914d-4b2b4ef4c68c {} 2026-09-18

17:06:58.654864 2026-09-18 09:06:58.654864+00
275eb5cb-e5e1-45ca-914d-4b2b4ef4c68c 143.44.164.184
b444b04c-5486-45a8-8f72-a842d75b08a7 \N admin@gmail.com created
Crew Schedules a501111b-8a2c-402f-9ed5-980462d4d2fe Crew Schedules
created: a501111b-8a2c-402f-9ed5-980462d4d2fe {} 2026-09-18
17:06:58.654864 2026-09-18 09:06:58.654864+00
a501111b-8a2c-402f-9ed5-980462d4d2fe 143.44.164.184
69e95860-f052-4396-a2e7-22a647ee3019 \N admin@gmail.com created
Crew Schedules 7c18881f-94ce-44ad-bbf8-9792bf9e2928 Crew Schedules
created: 7c18881f-94ce-44ad-bbf8-9792bf9e2928 {} 2026-09-18
17:06:58.654864 2026-09-18 09:06:58.654864+00
7c18881f-94ce-44ad-bbf8-9792bf9e2928 143.44.164.184
b3a895fa-e3a4-419a-8ab6-aa54a9a452fe \N admin@gmail.com created
Crew Schedules fec0435f-45f8-43f4-ae40-9ca943a8caf7 Crew Schedules
created: fec0435f-45f8-43f4-ae40-9ca943a8caf7 {} 2026-09-18
17:06:58.654864 2026-09-18 09:06:58.654864+00
fec0435f-45f8-43f4-ae40-9ca943a8caf7 143.44.164.184
628dc882-de7e-433a-821a-5eff50742541 \N admin@gmail.com created
Crew Schedules 13e53fc4-ff83-424a-ba92-312376f953dc Crew Schedules
created: 13e53fc4-ff83-424a-ba92-312376f953dc {} 2026-09-18
17:06:58.654864 2026-09-18 09:06:58.654864+00
13e53fc4-ff83-424a-ba92-312376f953dc 143.44.164.184
b65bc621-1cff-4f39-a3a8-5fd264ff0895 \N admin@gmail.com created
Crew Schedules ed492844-9490-425b-b372-31bb964e9af6 Crew Schedules
created: ed492844-9490-425b-b372-31bb964e9af6 {} 2026-09-18
17:06:58.654864 2026-09-18 09:06:58.654864+00
ed492844-9490-425b-b372-31bb964e9af6 143.44.164.184
6ffd19e4-bd41-425e-af66-842177a21447 \N admin@gmail.com created
Crew Schedules 3a82fcdf-cbe4-4b32-b19a-45b5701025ab Crew Schedules
created: 3a82fcdf-cbe4-4b32-b19a-45b5701025ab {} 2026-09-18
17:06:58.654864 2026-09-18 09:06:58.654864+00
3a82fcdf-cbe4-4b32-b19a-45b5701025ab 143.44.164.184
b04f05fc-e258-42cf-b91d-328a64bb0c9d \N admin@gmail.com created
Crew Schedules a2e5d75d-4916-497f-bd2e-8ac3d9a21793 Crew Schedules
created: a2e5d75d-4916-497f-bd2e-8ac3d9a21793 {} 2026-09-18
17:06:58.654864 2026-09-18 09:06:58.654864+00
a2e5d75d-4916-497f-bd2e-8ac3d9a21793 143.44.164.184
86d5cd2d-ef9d-4d20-b10f-51f997e0b8e3 \N admin@gmail.com created
Crew Schedules e9dc3b92-6141-4a93-976f-d7e491cc0606 Crew Schedules
created: e9dc3b92-6141-4a93-976f-d7e491cc0606 {} 2026-09-18
17:06:58.654864 2026-09-18 09:06:58.654864+00
e9dc3b92-6141-4a93-976f-d7e491cc0606 143.44.164.184
56503582-ab27-4230-b1ee-b6bdcae3c50d \N admin@gmail.com created
Crew Schedules d66300ff-9c06-4731-bb75-9b31d760d8cb Crew Schedules
created: d66300ff-9c06-4731-bb75-9b31d760d8cb {} 2026-09-18

17:06:58.654864 2026-09-18 09:06:58.654864+00
d66300ff-9c06-4731-bb75-9b31d760d8cb 143.44.164.184
aeef4906-3922-4fe0-8f03-1b772baba848 \N admin@gmail.com created
Crew Schedules b65b7504-2bf8-4575-aa89-ba514e23b7de Crew Schedules
created: b65b7504-2bf8-4575-aa89-ba514e23b7de {} 2026-09-18
17:06:58.654864 2026-09-18 09:06:58.654864+00
b65b7504-2bf8-4575-aa89-ba514e23b7de 143.44.164.184
543327aa-e67b-440f-8168-76fb4aebad57 \N admin@gmail.com created
Crew Schedules c87846e6-ac90-486b-8ee6-b0c9c4e3f819 Crew Schedules
created: c87846e6-ac90-486b-8ee6-b0c9c4e3f819 {} 2026-09-18
17:06:58.654864 2026-09-18 09:06:58.654864+00
c87846e6-ac90-486b-8ee6-b0c9c4e3f819 143.44.164.184
17fe6e39-6810-4fe2-bcfa-292446ea6875 \N admin@gmail.com created
Crew Schedules 41d0f84e-7ac7-4824-b1ec-e28b175624df Crew Schedules
created: 41d0f84e-7ac7-4824-b1ec-e28b175624df {} 2026-09-18
17:06:58.654864 2026-09-18 09:06:58.654864+00
41d0f84e-7ac7-4824-b1ec-e28b175624df 143.44.164.184
46fcfe1e-3617-45f2-a6d5-22ab479dde3e \N admin@gmail.com created
Crew Schedules cde9e6e9-4870-4539-bbd5-94348e4ef92f Crew Schedules
created: cde9e6e9-4870-4539-bbd5-94348e4ef92f {} 2026-09-18
17:06:58.654864 2026-09-18 09:06:58.654864+00
cde9e6e9-4870-4539-bbd5-94348e4ef92f 143.44.164.184
4328a891-f96a-4a82-93b5-b7079e158247 \N admin@gmail.com created
Crew Schedules 14587a4c-ddc6-4687-ae04-ba2c3c248fe7 Crew Schedules
created: 14587a4c-ddc6-4687-ae04-ba2c3c248fe7 {} 2026-09-18
17:06:58.654864 2026-09-18 09:06:58.654864+00
14587a4c-ddc6-4687-ae04-ba2c3c248fe7 143.44.164.184
303b2706-c44c-440c-bf3c-9d3e73c70d4d \N admin@gmail.com created
Crew Schedules 9e2c7b64-5c1a-4e56-930f-ee2506340f02 Crew Schedules
created: 9e2c7b64-5c1a-4e56-930f-ee2506340f02 {} 2026-09-18
17:06:58.654864 2026-09-18 09:06:58.654864+00
9e2c7b64-5c1a-4e56-930f-ee2506340f02 143.44.164.184
63f7e985-41c3-4b3a-ac6c-83ce2aa3dd11 \N admin@gmail.com created
Crew Schedules 0a61993b-a766-403e-881e-b5d7cd7daa38 Crew Schedules
created: 0a61993b-a766-403e-881e-b5d7cd7daa38 {} 2026-09-18
17:06:58.654864 2026-09-18 09:06:58.654864+00
0a61993b-a766-403e-881e-b5d7cd7daa38 143.44.164.184
2a11d373-5613-4fe4-b15a-e805ecb15f74 \N admin@gmail.com created
Crew Schedules 8bbde87a-a28d-477b-860b-ad2ce939f1bc Crew Schedules
created: 8bbde87a-a28d-477b-860b-ad2ce939f1bc {} 2026-09-18
17:06:58.654864 2026-09-18 09:06:58.654864+00
8bbde87a-a28d-477b-860b-ad2ce939f1bc 143.44.164.184
fc575a58-b2cb-4e75-b443-4914204317e6 \N admin@gmail.com created
Crew Schedules c79d9772-1c51-42b6-ac02-745b49ee7f3d Crew Schedules
created: c79d9772-1c51-42b6-ac02-745b49ee7f3d {} 2026-09-18

17:06:58.654864 2026-09-18 09:06:58.654864+00
c79d9772-1c51-42b6-ac02-745b49ee7f3d 143.44.164.184
c6fb477c-30b6-4511-8d28-13fb75614ec3 \N admin@gmail.com created
Crew Schedules 919629c8-4fea-4725-80b9-120bbb86572f Crew Schedules
created: 919629c8-4fea-4725-80b9-120bbb86572f {} 2026-09-18
17:06:58.654864 2026-09-18 09:06:58.654864+00
919629c8-4fea-4725-80b9-120bbb86572f 143.44.164.184
04bb0292-6408-421d-acae-974400e34b14 \N admin@gmail.com created
Crew Schedules a5e3f683-424b-402b-86d6-2a77844c2e5e Crew Schedules
created: a5e3f683-424b-402b-86d6-2a77844c2e5e {} 2026-09-18
17:06:58.654864 2026-09-18 09:06:58.654864+00
a5e3f683-424b-402b-86d6-2a77844c2e5e 143.44.164.184
9f8986b0-d48d-41dc-a9c8-d27e2ba2cfe5 \N admin@gmail.com created
Crew Schedules 3e02d0ce-89fd-4624-93ed-2f6926cff024 Crew Schedules
created: 3e02d0ce-89fd-4624-93ed-2f6926cff024 {} 2026-09-18
17:06:58.654864 2026-09-18 09:06:58.654864+00
3e02d0ce-89fd-4624-93ed-2f6926cff024 143.44.164.184
e0495c87-5df1-466e-9510-82c39a824ea2 \N admin@gmail.com created
Crew Schedules 9b885e71-465b-4718-9e3a-821ec118ee94 Crew Schedules
created: 9b885e71-465b-4718-9e3a-821ec118ee94 {} 2026-09-18
17:06:58.654864 2026-09-18 09:06:58.654864+00
9b885e71-465b-4718-9e3a-821ec118ee94 143.44.164.184
523065c0-cd34-47e9-8706-2d952b1c1801 \N admin@gmail.com created
Crew Schedules c3928de9-e138-476b-bdd7-ee795f01836e Crew Schedules
created: c3928de9-e138-476b-bdd7-ee795f01836e {} 2026-09-18
17:06:58.654864 2026-09-18 09:06:58.654864+00
c3928de9-e138-476b-bdd7-ee795f01836e 143.44.164.184
34ec9529-60a4-4b7e-add1-4c5385afd75a \N admin@gmail.com created
Crew Schedules ccccf6bb-50f6-461a-a225-5048afbfb8c7 Crew Schedules
created: ccccf6bb-50f6-461a-a225-5048afbfb8c7 {} 2026-09-18
17:06:58.654864 2026-09-18 09:06:58.654864+00
ccccf6bb-50f6-461a-a225-5048afbfb8c7 143.44.164.184
5a1acaae-f444-4ef1-bd3d-8647dcd80b99 \N admin@gmail.com created
Crew Schedules 03d3fea9-596c-45c5-9bdc-7c872d4f75d9 Crew Schedules
created: 03d3fea9-596c-45c5-9bdc-7c872d4f75d9 {} 2026-09-18
17:07:40.287738 2026-09-18 09:07:40.287738+00
03d3fea9-596c-45c5-9bdc-7c872d4f75d9 143.44.164.184
b0c4b3ef-1447-4212-bbbb-6ca437df3478 \N admin@gmail.com created
Crew Schedules 9fabfd43-2755-4d5f-8521-5721ee448c24 Crew Schedules
created: 9fabfd43-2755-4d5f-8521-5721ee448c24 {} 2026-09-18
17:07:40.287738 2026-09-18 09:07:40.287738+00
9fabfd43-2755-4d5f-8521-5721ee448c24 143.44.164.184
54aa8680-64b8-4df3-b3d8-0d4331b313cf \N admin@gmail.com created
Crew Schedules 76e8d7d4-133d-447d-8a4b-98b212ca0f5e Crew Schedules
created: 76e8d7d4-133d-447d-8a4b-98b212ca0f5e {} 2026-09-18

17:07:40.287738 2026-09-18 09:07:40.287738+00
76e8d7d4-133d-447d-8a4b-98b212ca0f5e 143.44.164.184
18aadfb1-08d4-46c1-9c91-3e3d8aee6905 \N admin@gmail.com created
Crew Schedules d381846a-169e-4b1f-abe9-65fd7d8c9f79 Crew Schedules
created: d381846a-169e-4b1f-abe9-65fd7d8c9f79 {} 2026-09-18
17:07:40.287738 2026-09-18 09:07:40.287738+00
d381846a-169e-4b1f-abe9-65fd7d8c9f79 143.44.164.184
a2a9e17d-f38b-4f70-860e-d11fb51c27ea \N admin@gmail.com created
Crew Schedules 479aead5-1432-4cef-9166-80f0236432c0 Crew Schedules
created: 479aead5-1432-4cef-9166-80f0236432c0 {} 2026-09-18
17:07:40.287738 2026-09-18 09:07:40.287738+00
479aead5-1432-4cef-9166-80f0236432c0 143.44.164.184
211c734a-24b8-4d71-8366-04e00934e81e \N admin@gmail.com created
Crew Schedules aae186ea-95d7-4c60-b85b-3bc9a371a821 Crew Schedules
created: aae186ea-95d7-4c60-b85b-3bc9a371a821 {} 2026-09-18
17:07:40.287738 2026-09-18 09:07:40.287738+00
aae186ea-95d7-4c60-b85b-3bc9a371a821 143.44.164.184
4568f3de-ff4b-4553-aa8a-16ba80956202 \N admin@gmail.com created
Crew Schedules 8ae0728d-22fb-4fbe-8062-07a77539abcc Crew Schedules
created: 8ae0728d-22fb-4fbe-8062-07a77539abcc {} 2026-09-18
17:07:40.287738 2026-09-18 09:07:40.287738+00
8ae0728d-22fb-4fbe-8062-07a77539abcc 143.44.164.184
9bc6fffe-a032-48b5-8d0f-e306c2ee641f \N admin@gmail.com created
Crew Schedules d16c5630-32b2-49ac-8171-ec465a628974 Crew Schedules
created: d16c5630-32b2-49ac-8171-ec465a628974 {} 2026-09-18
17:07:40.287738 2026-09-18 09:07:40.287738+00
d16c5630-32b2-49ac-8171-ec465a628974 143.44.164.184
3dfed4ad-a4ab-4645-be25-38725be37a5f \N admin@gmail.com created
Crew Schedules e08830b2-812c-40d0-bc5e-0909b408cf8c Crew Schedules
created: e08830b2-812c-40d0-bc5e-0909b408cf8c {} 2026-09-18
17:07:40.287738 2026-09-18 09:07:40.287738+00
e08830b2-812c-40d0-bc5e-0909b408cf8c 143.44.164.184
ea684b4b-5bf4-4fa7-b9b1-033e48d4a7e9 \N admin@gmail.com created
Crew Schedules ef117950-951e-4c1f-8766-4e77feed0353 Crew Schedules
created: ef117950-951e-4c1f-8766-4e77feed0353 {} 2026-09-18
17:07:40.287738 2026-09-18 09:07:40.287738+00
ef117950-951e-4c1f-8766-4e77feed0353 143.44.164.184
4c30011c-9f3d-4f2c-b4a3-b3178c459dd1 \N admin@gmail.com created
Crew Schedules 806d50e8-60be-451d-a08e-9545c22e9e52 Crew Schedules
created: 806d50e8-60be-451d-a08e-9545c22e9e52 {} 2026-09-18
17:07:40.287738 2026-09-18 09:07:40.287738+00
806d50e8-60be-451d-a08e-9545c22e9e52 143.44.164.184
4ccd3db9-b855-4a99-aee9-f2269e91cf76 \N admin@gmail.com created
Crew Schedules 4de3b298-9cb3-402d-821b-731e62681d82 Crew Schedules
created: 4de3b298-9cb3-402d-821b-731e62681d82 {} 2026-09-18

17:07:40.287738 2026-09-18 09:07:40.287738+00
4de3b298-9cb3-402d-821b-731e62681d82 143.44.164.184
d56ee2ec-2964-4f0c-841b-da25c12c850c \N admin@gmail.com created
Crew Schedules 23dcebea-ad6a-4099-80ea-259029c73252 Crew Schedules
created: 23dcebea-ad6a-4099-80ea-259029c73252 {} 2026-09-18
17:07:40.287738 2026-09-18 09:07:40.287738+00
23dcebea-ad6a-4099-80ea-259029c73252 143.44.164.184
6b05145f-846c-41a0-be5b-835d88ab0cc8 \N admin@gmail.com created
Crew Schedules 31cfdcef-659a-46f7-934a-a578f373cd06 Crew Schedules
created: 31cfdcef-659a-46f7-934a-a578f373cd06 {} 2026-09-18
17:07:40.287738 2026-09-18 09:07:40.287738+00
31cfdcef-659a-46f7-934a-a578f373cd06 143.44.164.184
ab2de649-2b9b-477e-891e-00e4f968a385 \N admin@gmail.com created
Crew Schedules 92373336-fe18-4c82-af45-c60b5d86818e Crew Schedules
created: 92373336-fe18-4c82-af45-c60b5d86818e {} 2026-09-18
17:07:40.287738 2026-09-18 09:07:40.287738+00
92373336-fe18-4c82-af45-c60b5d86818e 143.44.164.184
6fe25973-0f11-4abb-88f6-0df98ae6a6b5 \N admin@gmail.com created
Crew Schedules 1d2c13b1-4838-4bf5-af56-52a011132396 Crew Schedules
created: 1d2c13b1-4838-4bf5-af56-52a011132396 {} 2026-09-18
17:07:40.287738 2026-09-18 09:07:40.287738+00
1d2c13b1-4838-4bf5-af56-52a011132396 143.44.164.184
c04db7fc-3fdc-4e9e-b46f-08f0049b036d \N admin@gmail.com created
Crew Schedules 9751ea38-22e4-444d-9fd4-ca1fb1461619 Crew Schedules
created: 9751ea38-22e4-444d-9fd4-ca1fb1461619 {} 2026-09-18
17:07:40.287738 2026-09-18 09:07:40.287738+00
9751ea38-22e4-444d-9fd4-ca1fb1461619 143.44.164.184
a2c18bc0-4681-4a75-a2d0-d9393fbbee6e \N admin@gmail.com created
Crew Schedules d78bab4e-3338-4f57-a3fa-32c3c5e809ba Crew Schedules
created: d78bab4e-3338-4f57-a3fa-32c3c5e809ba {} 2026-09-18
17:07:40.287738 2026-09-18 09:07:40.287738+00
d78bab4e-3338-4f57-a3fa-32c3c5e809ba 143.44.164.184
14950fdb-4dac-4a9d-ae54-dbfbb6ac88b2 \N admin@gmail.com created
Crew Schedules 09fa2fb5-6053-4ef7-9b81-cd9db171f817 Crew Schedules
created: 09fa2fb5-6053-4ef7-9b81-cd9db171f817 {} 2026-09-18
17:07:40.287738 2026-09-18 09:07:40.287738+00
09fa2fb5-6053-4ef7-9b81-cd9db171f817 143.44.164.184
28eac916-9195-48b8-9f12-f955d3813815 \N admin@gmail.com created
Crew Schedules acc5da6f-d3bb-41cf-ab15-475f25a38d1a Crew Schedules
created: acc5da6f-d3bb-41cf-ab15-475f25a38d1a {} 2026-09-18
17:07:40.287738 2026-09-18 09:07:40.287738+00
acc5da6f-d3bb-41cf-ab15-475f25a38d1a 143.44.164.184
55edba87-1199-46f8-8587-a97320179619 \N admin@gmail.com created
Crew Schedules 311694a8-4f3d-4e18-a2b2-8e16bef1078a Crew Schedules
created: 311694a8-4f3d-4e18-a2b2-8e16bef1078a {} 2026-09-18

17:07:40.287738 2026-09-18 09:07:40.287738+00
311694a8-4f3d-4e18-a2b2-8e16bef1078a 143.44.164.184
1195caad-b6eb-40c2-b002-d85b446b67ea \N admin@gmail.com created
Crew Schedules dc8ced24-3e32-4b6d-bc14-601edfd4430e Crew Schedules
created: dc8ced24-3e32-4b6d-bc14-601edfd4430e {} 2026-09-18
17:07:40.287738 2026-09-18 09:07:40.287738+00
dc8ced24-3e32-4b6d-bc14-601edfd4430e 143.44.164.184
44c80e04-019f-4908-be8a-6fe87b6c8711 \N admin@gmail.com signed in
Authentication Admin console Administrator signed in to the admin console.
{} 2026-09-19 08:54:10.607204 2026-09-19 00:54:10.607204+00 \N
136.158.3.48
8d5b3d81-435d-4018-bc24-78765c937ab3 \N admin@gmail.com signed
out Authentication Admin console Administrator signed out of the admin
console. {} 2026-09-19 17:21:12.796929 2026-09-19
09:21:12.796929+00 \N 143.44.164.123
a6ba2154-c936-4339-918d-af3fb084d2c8 \N admin@gmail.com signed in
Authentication Admin console Administrator signed in to the admin console.
{} 2026-09-19 18:10:40.569665 2026-09-19 10:10:40.569665+00 \N
143.44.164.123
cf60283a-71ca-4b38-9519-10dcc50cacf5 \N admin@gmail.com signed out
Authentication Admin console Administrator signed out of the admin
console. {} 2026-09-19 18:10:54.597317 2026-09-19
10:10:54.597317+00 \N 143.44.164.123
725a4456-e7c5-4397-a6a8-55cb7a5c1b90 \N admin@gmail.com signed in
Authentication Admin console Administrator signed in to the admin console.
{} 2026-09-19 18:11:21.240965 2026-09-19 10:11:21.240965+00 \N
143.44.164.123
8a674169-6b90-4b8c-b856-88ddc6737670 \N admin@gmail.com signed
out Authentication Admin console Administrator signed out of the admin
console. {} 2026-09-19 18:16:37.36321 2026-09-19 10:16:37.36321+00
\N 143.44.164.123
598748c0-e870-40af-bf65-032ba1181167 \N admin@gmail.com signed in
Authentication Admin console Administrator signed in to the admin console.
{} 2026-09-19 18:17:14.357389 2026-09-19 10:17:14.357389+00 \N
143.44.164.123
8fcdeff1-a0b2-4e45-a813-58637572af61 \N admin@gmail.com session
timed out Authentication Admin console Administrator was signed out after
1 minute of inactivity. {} 2026-09-19 18:22:47.95418 2026-09-19
10:22:47.95418+00 \N 143.44.164.123
6d7c0753-77ee-494b-b0f9-847f73b09f29 \N admin@gmail.com signed in
Authentication Admin console Administrator signed in to the admin console.
{} 2026-09-19 18:23:29.699705 2026-09-19 10:23:29.699705+00 \N
143.44.164.123
82bcb1cf-eb75-4c44-9796-bfd06e264d3b \N admin@gmail.com session
timed out Authentication Admin console Administrator was signed out after

1 minute of inactivity. {} 2026-09-19 18:30:46.76141 2026-09-19 10:30:46.76141+00 \N 143.44.164.123
617809cb-e99e-49a9-ac0c-728be6127414 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-19 18:37:14.349247 2026-09-19 10:37:14.349247+00 \N 143.44.164.123
76b873d1-f17d-4410-9e85-20c4933ca437 \N admin@gmail.com session timed out Authentication Admin console Administrator was signed out after 1 minute of inactivity. {} 2026-09-19 18:41:55.918995 2026-09-19 10:41:55.918995+00 \N 143.44.164.123
355d975d-d921-4728-86ce-8603925b2825 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-19 18:52:29.875632 2026-09-19 10:52:29.875632+00 \N 143.44.164.123
e925c5ef-2226-4210-89fd-3c929c4ba09a \N admin@gmail.com session timed out Authentication Admin console Administrator was signed out after 1 minute of inactivity. {} 2026-09-19 18:57:06.820108 2026-09-19 10:57:06.820108+00 \N 143.44.164.123
24a16ac1-6d1d-47b8-b726-d4cf601d56be \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-19 18:58:41.828959 2026-09-19 10:58:41.828959+00 \N 143.44.164.123
8ca4252e-7642-4d7b-a1e9-cc9cdad92df7 \N admin@gmail.com updated Account Settings Administrator profile Administrator account settings updated. {Password} 2026-09-19 18:59:24.707324 2026-09-19 10:59:24.707324+00 \N 143.44.164.123
c8a2dcc1-51b8-4291-8e61-c6ae0be25707 \N admin@gmail.com signed out Authentication Admin console Administrator signed out of the admin console. {} 2026-09-19 18:59:32.718827 2026-09-19 10:59:32.718827+00 \N 143.44.164.123
4263ebaf-6d8d-431c-8dfa-c96dd73a3e69 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-19 18:59:50.792465 2026-09-19 10:59:50.792465+00 \N 143.44.164.123
df4b5549-94c5-443b-9ef1-790ea88ee753 \N admin@gmail.com session timed out Authentication Admin console Administrator was signed out after 1 minute of inactivity. {} 2026-09-19 19:04:28.86557 2026-09-19 11:04:28.86557+00 \N 143.44.164.123
4b8b1375-0e90-4037-8245-52ebf31aad98 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-20 07:16:29.012323 2026-09-19 23:16:29.012323+00 \N 143.44.164.123
4c2df916-79cd-4dbc-a6d5-584edef35891 \N admin@gmail.com updated Account Settings Administrator profile Administrator account settings updated. {Password} 2026-09-20 07:17:03.81252 2026-09-19

23:17:03.81252+00 \N 143.44.164.123
bd9d10a6-cd3f-42ca-bb03-712337308b52 \N admin@gmail.com session timed out Authentication Admin console Administrator was signed out after 1 minute of inactivity. {} 2026-09-20 07:18:51.620255 2026-09-19 23:18:51.620255+00 \N 143.44.164.123
decb5f09-4516-499c-b09e-e78b32c1f4ee \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-20 07:20:00.691492 2026-09-19 23:20:00.691492+00 \N 143.44.164.123
6c5c87a3-a174-471b-98a5-9d97be0f8dd1 \N admin@gmail.com session timed out Authentication Admin console Administrator was signed out after 1 minute of inactivity. {} 2026-09-20 07:21:31.97463 2026-09-19 23:21:31.97463+00 \N 143.44.164.123
654951ca-5455-442e-b820-16340099c80b \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-20 07:22:29.66314 2026-09-19 23:22:29.66314+00 \N 143.44.164.123
0ff8dfe3-cdbc-49bf-8514-23a45ae25f97 \N admin@gmail.com session timed out Authentication Admin console Administrator was signed out after 1 minute of inactivity. {} 2026-09-20 07:23:54.165641 2026-09-19 23:23:54.165641+00 \N 143.44.164.123
5ec3da78-70ab-44f1-9e76-fb68e1535e02 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-20 07:24:32.748521 2026-09-19 23:24:32.748521+00 \N 143.44.164.123
19502cef-1e88-4293-ad59-44f1350a9320 \N admin@gmail.com session timed out Authentication Admin console Administrator was signed out after 1 minute of inactivity. {} 2026-09-20 07:30:25.639492 2026-09-19 23:30:25.639492+00 \N 143.44.164.123
e6efcfb1-beb1-4e58-ae2c-2f96cf2d1cea \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-20 07:34:59.356916 2026-09-19 23:34:59.356916+00 \N 143.44.164.123
ee5f1c9f-34e2-49fa-993b-2a2bea07c705 \N admin@gmail.com signed out Authentication Admin console Administrator signed out of the admin console. {} 2026-09-20 07:35:04.241373 2026-09-19 23:35:04.241373+00 \N 143.44.164.123
8ae88702-50f5-4519-a632-14f28f516660 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-20 07:35:19.886705 2026-09-19 23:35:19.886705+00 \N 143.44.164.123
18a5a1a0-8cf5-4e19-826b-ba32d61c0d4f \N admin@gmail.com signed out Authentication Admin console Administrator signed out of the admin console. {} 2026-09-20 07:35:24.982075 2026-09-19 23:35:24.982075+00 \N 143.44.164.123

005187c9-8865-44e6-a0a6-1589ec2dd7e9 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-20 07:37:25.610116 2026-09-19 23:37:25.610116+00 \N 143.44.164.123
601adc1d-a4dd-45d9-989c-daa7f18620c3 \N admin@gmail.com signed out Authentication Admin console Administrator signed out of the admin console. {} 2026-09-20 07:37:41.994133 2026-09-19 23:37:41.994133+00 \N 143.44.164.123
c8c6779e-5e4c-4d4a-a260-d3ba4b473ef0 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-20 07:39:35.926684 2026-09-19 23:39:35.926684+00 \N 143.44.164.123
89eaa8a9-d041-43fc-9c40-1a08eba3c2a6 \N admin@gmail.com signed out Authentication Admin console Administrator signed out of the admin console. {} 2026-09-20 07:39:39.026053 2026-09-19 23:39:39.026053+00 \N 143.44.164.123
4845e52e-e003-437b-89a2-a58fac0ccd3c \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-20 07:40:32.6456 2026-09-19 23:40:32.6456+00 \N 143.44.164.123
08c5d56d-192f-4561-9b24-73b18aef048f \N admin@gmail.com created Services Motorcycle Tune-Up Services created: Motorcycle Tune-Up {} 2026-09-20 07:50:07.474101 2026-09-19 23:50:07.474101+00 d9ecf096-df3b-47f5-a012-80d98aacf9f5 143.44.164.123
295dde12-0a0b-4201-b12b-969a45eb8a87 \N admin@gmail.com archived Products Full-Face Racing Helmet Products archived: Full-Face Racing Helmet {archived_at,is_archived} 2026-09-20 07:52:18.347451 2026-09-19 23:52:18.347451+00 ce751dec-9abb-4a5f-957c-c3e6e2ac0c61 143.44.164.123
65655688-93b1-4fbb-90be-eed34c8b13b8 \N admin@gmail.com archived Motorcycle Catalog 806e348d-7c95-4374-8555-c91355a56a86 Motorcycle Catalog archived: 806e348d-7c95-4374-8555-c91355a56a86 {archived_at,is_archived} 2026-09-20 08:19:31.013199 2026-09-20 00:19:31.013199+00 806e348d-7c95-4374-8555-c91355a56a86 143.44.164.123
22087c16-4307-4a4c-8adc-dad179e9ae59 \N admin@gmail.com archived Motorcycle Catalog 83728529-1ee4-42c4-a1a3-fd34a1fb9969 Motorcycle Catalog archived: 83728529-1ee4-42c4-a1a3-fd34a1fb9969 {archived_at,is_archived} 2026-09-20 08:19:34.503956 2026-09-20 00:19:34.503956+00 83728529-1ee4-42c4-a1a3-fd34a1fb9969 143.44.164.123
bb7d5263-b2ff-4d69-9fab-71a4ccde0d64 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-20 09:20:45.558557 2026-09-20 01:20:45.558557+00 \N 143.44.164.123

b84b9013-6098-4931-a953-c97154ae47aa \N admin@gmail.com deactivated Services Motorcycle Safety Inspection Services deactivated: Motorcycle Safety Inspection {is_active} 2026-09-20 15:01:48.024912 2026-09-20 07:01:48.024912+00 e04fec1b-0083-46cf-95d4-5765ba807518 143.44.164.123
b5bc4aae-e045-4500-96e4-73060ff1906c \N admin@gmail.com signed out Authentication Admin console Administrator signed out of the admin console. {} 2026-09-20 15:15:04.003159 2026-09-20 07:15:04.003159+00 \N 143.44.164.123
ef6eb568-075f-4b17-bd5d-f02713de26ce \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-20 15:15:46.194184 2026-09-20 07:15:46.194184+00 \N 143.44.164.123
4ed47914-6390-4795-8668-d2b9b68c7dfd \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-20 16:07:43.221471 2026-09-20 08:07:43.221471+00 \N 143.44.164.123
b6b40607-ebe5-4dde-9a1b-bfd66bff3b89 \N admin@gmail.com signed out Authentication Admin console Administrator signed out of the admin console. {} 2026-09-20 17:49:08.054257 2026-09-20 09:49:08.054257+00 \N 143.44.164.123
e989cfd2-87fd-4b93-9566-68725dbac040 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-20 18:45:41.636761 2026-09-20 10:45:41.636761+00 \N 143.44.164.123
7905577a-2a42-44e8-a7c9-f722a4404293 \N admin@gmail.com activated Products Brake pads Products activated: Brake pads {is_active} 2026-09-20 18:46:26.504736 2026-09-20 10:46:26.504736+00 9310ef87-578e-4fb2-b95b-725a0900c95a 143.44.164.123
5039c302-2bb5-411c-a208-88311c45941b \N admin@gmail.com restored Crew Members mark james Crew Members restored: mark james {archived_at,is_archived} 2026-09-20 18:52:48.351788 2026-09-20 10:52:48.351788+00 5cb00871-f5ef-4ccf-bd61-4746909237e2 143.44.164.123
82396422-c1f8-4613-9d50-940fce548bdd \N admin@gmail.com activated Crew Members mark james Crew Members activated: mark james {is_active} 2026-09-20 18:53:43.349855 2026-09-20 10:53:43.349855+00 5cb00871-f5ef-4ccf-bd61-4746909237e2 143.44.164.123
22b18c2b-caf7-446f-8217-3f4fb6cdd84f \N admin@gmail.com signed out Authentication Admin console Administrator signed out of the admin console. {} 2026-09-20 19:30:42.407968 2026-09-20 11:30:42.407968+00 \N 143.44.164.123
bf8984ad-efdf-487e-ab90-01e6cd5e7cf2 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-20 20:04:57.710965 2026-09-20 12:04:57.710965+00 \N

143.44.164.123
2e6a7392-b939-4a3c-a52e-e7263095a6d6 \N admin@gmail.com
deactivated Crew Members JR Salazar Crew Members deactivated: JR
Salazar {is_active} 2026-09-20 22:19:05.816256 2026-09-20
14:19:05.816256+00 f3c5770f-e485-4efc-83f6-892422d8592e
143.44.164.123
45463a75-0e26-4df2-b6ce-867300011379 \N admin@gmail.com activated
Crew Members JR Salazar Crew Members activated: JR Salazar
{is_active} 2026-09-20 22:19:07.109385 2026-09-20 14:19:07.109385+00
f3c5770f-e485-4efc-83f6-892422d8592e 143.44.164.123
2f7d75ea-fa75-4b42-93e2-f319568c973a \N admin@gmail.com deactivated
Crew Members JR Salazar Crew Members deactivated: JR Salazar
{is_active} 2026-09-21 10:26:16.279593 2026-09-21 02:26:16.279593+00
f3c5770f-e485-4efc-83f6-892422d8592e 162.4.58.137
8649257f-74af-4ec4-921f-5c9fc8ad3b02 \N admin@gmail.com activated
Crew Members JR Salazar Crew Members activated: JR Salazar
{is_active} 2026-09-21 10:26:17.148981 2026-09-21 02:26:17.148981+00
f3c5770f-e485-4efc-83f6-892422d8592e 162.4.58.137
f2303f7c-9275-4aa6-8259-55870e087268 \N admin@gmail.com
deactivated Crew Members JR Salazar Crew Members deactivated: JR
Salazar {is_active} 2026-09-21 10:26:18.769846 2026-09-21
02:26:18.769846+00 f3c5770f-e485-4efc-83f6-892422d8592e 162.4.58.137
a4727b6a-9117-4b22-a199-2fc59c2317d4 \N admin@gmail.com activated
Crew Members JR Salazar Crew Members activated: JR Salazar
{is_active} 2026-09-21 10:26:21.431097 2026-09-21 02:26:21.431097+00
f3c5770f-e485-4efc-83f6-892422d8592e 162.4.58.137
98e12d4c-ff69-4d05-b8d7-1fc6f02603f1 \N admin@gmail.com signed out
Authentication Admin console Administrator signed out of the admin console.
{} 2026-09-21 10:31:50.875606 2026-09-21 02:31:50.875606+00 \N
162.4.58.137
d2991920-a2c8-4f4a-9ac6-d0a00d6978a3 \N admin@gmail.com signed in
Authentication Admin console Administrator signed in to the admin console.
{} 2026-09-21 10:40:08.41235 2026-09-21 02:40:08.41235+00 \N
162.4.58.134
6359c76c-b43a-4dd3-a6d3-78b70df7693b \N admin@gmail.com status
changed Appointments FRM-S32DGA Appointments status changed:
FRM-S32DGA {status} 2026-09-21 10:40:16.389707 2026-09-21
02:40:16.389707+00 c2948c1c-8f78-441c-bcc7-1cfa03c0718b 162.4.58.134
d5c9b003-3344-40c9-9679-24978f2cbde8 \N admin@gmail.com updated
Notifications ca41a94d-d4b6-4555-8d03-fe1fbc2db268 Notifications updated:
ca41a94d-d4b6-4555-8d03-fe1fbc2db268 {is_read} 2026-09-21
10:41:38.789468 2026-09-21 02:41:38.789468+00
ca41a94d-d4b6-4555-8d03-fe1fbc2db268 162.4.58.134
1e8485a3-11e0-4d87-9036-aa362eaf90f6 \N admin@gmail.com signed in
Authentication Admin console Administrator signed in to the admin console.

{} 2026-09-21 15:18:49.254327 2026-09-21 07:18:49.254327+00 \N 143.44.164.123
8c54d127-abf5-4c89-8594-d46152ebfcaf \N admin@gmail.com status changed Appointments FRM-F98X7B Appointments status changed: FRM-F98X7B {status} 2026-09-21 15:48:13.55828 2026-09-21 07:48:13.55828+00 64fe7d0c-3604-4dc4-a072-a3344c02e337 143.44.164.123
e3c84b47-c9db-494c-9171-98f6df42ac3e \N admin@gmail.com deactivated Crew Members JR Salazar Crew Members deactivated: JR Salazar {is_active} 2026-09-21 15:49:20.993447 2026-09-21 07:49:20.993447+00 f3c5770f-e485-4efc-83f6-892422d8592e 143.44.164.123
96ae5b40-ec28-4ca4-b679-2381028d2485 \N admin@gmail.com activated Crew Members JR Salazar Crew Members activated: JR Salazar {is_active} 2026-09-21 15:49:22.122197 2026-09-21 07:49:22.122197+00 f3c5770f-e485-4efc-83f6-892422d8592e 143.44.164.123
a5dda86a-30fe-4498-a7e0-df377f188ff5 \N admin@gmail.com deactivated Crew Members JR Salazar Crew Members deactivated: JR Salazar {is_active} 2026-09-21 16:22:48.921311 2026-09-21 08:22:48.921311+00 f3c5770f-e485-4efc-83f6-892422d8592e 143.44.164.123
8154320c-3b5d-4da4-8ae3-d436577060dd \N admin@gmail.com activated Crew Members JR Salazar Crew Members activated: JR Salazar {is_active} 2026-09-21 16:22:54.217869 2026-09-21 08:22:54.217869+00 f3c5770f-e485-4efc-83f6-892422d8592e 143.44.164.123
7f8ade32-5ba3-4234-937d-a2834bcc5992 \N admin@gmail.com signed out Authentication Admin console Administrator signed out of the admin console. {} 2026-09-21 16:31:52.11472 2026-09-21 08:31:52.11472+00 \N 143.44.164.123
aa7c9b98-40ce-4316-89aa-639340b4e04c \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-21 16:32:24.18671 2026-09-21 08:32:24.18671+00 \N 143.44.164.123
4e048c68-a6da-4224-a48c-012fdfad1b3a \N admin@gmail.com signed out Authentication Admin console Administrator signed out of the admin console. {} 2026-09-21 17:33:49.697709 2026-09-21 09:33:49.697709+00 \N 143.44.164.123
e23edfef-9d4c-445c-8fad-d1938e7de0a9 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-21 17:34:17.069556 2026-09-21 09:34:17.069556+00 \N 143.44.164.123
fb55b943-219d-49cd-881c-1e7d3278b58c \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 1 minute of inactivity. {} 2026-09-21 17:47:39.767919 2026-09-21 09:47:39.767919+00 \N 143.44.164.123
d149fa5d-15df-42cd-bb03-5a31f8ccbe54 \N admin@gmail.com signed in

Authentication Admin console Administrator signed in to the admin console. {} 2026-09-21 17:49:45.483221 2026-09-21 09:49:45.483221+00 \N 143.44.164.123
62c6ac7b-1c1f-4d1f-af25-68b5475c9016 \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 1 minute of inactivity. {} 2026-09-21 17:54:01.054432 2026-09-21 09:54:01.054432+00 \N 143.44.164.123
43222ef0-5376-4b3f-a920-606d3abe8f1c \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-21 17:56:12.978739 2026-09-21 09:56:12.978739+00 \N 143.44.164.123
662cb124-7922-46d8-8362-0e44a3af92db \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 1 minute of inactivity. {} 2026-09-21 18:04:48.935261 2026-09-21 10:04:48.935261+00 \N 143.44.164.123
a5027666-abb1-490b-8696-0b0d6f166002 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-21 18:19:22.282674 2026-09-21 10:19:22.282674+00 \N 143.44.164.123
0037c0ed-e448-4a4d-bddb-3c01ffc6ab49 \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 1 minute of inactivity. {} 2026-09-21 18:23:54.936975 2026-09-21 10:23:54.936975+00 \N 143.44.164.123
3eddfb97-d8c7-43f8-aa9a-eac8bdc99681 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-21 18:31:27.747952 2026-09-21 10:31:27.747952+00 \N 143.44.164.123
a04d521f-2cc7-4bc3-b2b5-304f3deb0df4 \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 1 minute of inactivity. {} 2026-09-21 18:33:21.068282 2026-09-21 10:33:21.068282+00 \N 143.44.164.123
1e58c51b-8364-49de-ac53-d66bd972b150 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-21 18:34:11.054106 2026-09-21 10:34:11.054106+00 \N 143.44.164.123
436aff62-621f-4e97-87b3-1dbdbe175f5e \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 1 minute of inactivity. {} 2026-09-21 18:37:35.902608 2026-09-21 10:37:35.902608+00 \N 143.44.164.123
e38c1b18-8209-4b59-a7f3-f64adbcf5e13 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-21 18:41:12.575737 2026-09-21 10:41:12.575737+00 \N 143.44.164.123
4103756a-9eca-4d8d-bccc-0f425710638f \N admin@gmail.com updated Notifications 571b6cd0-3c00-490e-b418-563e6d82628a Notifications

updated: 571b6cd0-3c00-490e-b418-563e6d82628a {is_read} 2026-09-21 18:43:18.667766 2026-09-21 10:43:18.667766+00 571b6cd0-3c00-490e-b418-563e6d82628a 143.44.164.123
431a864b-9e85-4847-901f-99dd9f2fbdca \N admin@gmail.com signed out Authentication Admin console Administrator signed out of the admin console. {} 2026-09-21 18:44:50.31126 2026-09-21 10:44:50.31126+00 \N 143.44.164.123
bebf597c-8622-459f-92ee-1816a7d8763a \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-21 18:45:23.722713 2026-09-21 10:45:23.722713+00 \N 143.44.164.123
cdf1134b-1499-4290-afea-dc0c929e9b25 \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 1 minute of inactivity. {} 2026-09-21 18:48:35.970625 2026-09-21 10:48:35.970625+00 \N 143.44.164.123
c6520bcf-f3b7-4e58-88e7-cae1618d41eb \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-21 20:39:07.834635 2026-09-21 12:39:07.834635+00 \N 143.44.164.123
5724a3c3-2bbe-44fc-943c-aff8c7554df3 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-21 20:46:20.757649 2026-09-21 12:46:20.757649+00 \N 143.44.164.123
d5ac55ac-4c7b-4433-86aa-dbe19cba6a2c \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 1 minute of inactivity. {} 2026-09-21 20:49:20.765992 2026-09-21 12:49:20.765992+00 \N 143.44.164.123
fd80b21a-64ba-41f1-b5e8-af5492b49910 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-22 10:06:33.164613 2026-09-22 02:06:33.164613+00 \N 49.146.140.233
8e687054-8143-48f7-96e2-c1eab6be0e06 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-22 10:06:37.542681 2026-09-22 02:06:37.542681+00 \N 49.146.140.233
4e80bf18-2279-46c0-b0b0-f6115882ec41 \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 1 minute of inactivity. {} 2026-09-22 10:08:48.519378 2026-09-22 02:08:48.519378+00 \N 49.146.140.233
25a79605-e25e-48bc-9e68-2627df15ad47 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-22 10:09:25.931542 2026-09-22 02:09:25.931542+00 \N 49.146.140.233
0cde267a-d170-4abc-b007-48576c201cfe \N admin@gmail.com signed out Authentication Admin console Administrator signed out of the admin

console. {} 2026-09-22 10:10:45.2453 2026-09-22 02:10:45.2453+00 \N 49.146.140.233
a12cf2a7-74cf-4c0f-a853-e4a307d3a35d \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-22 10:17:45.031697 2026-09-22 02:17:45.031697+00 \N 49.146.140.233
e4a3dc86-cfc7-4b7a-80af-5008a40d1e90 \N admin@gmail.com signed out Authentication Admin console Administrator signed out of the admin console. {} 2026-09-22 10:18:33.863333 2026-09-22 02:18:33.863333+00 \N 49.146.140.233
968e76c6-b70b-4c38-a252-99ab55d941e2 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-22 10:28:25.418457 2026-09-22 02:28:25.418457+00 \N 49.146.140.233
4193ef93-bd7f-4fe2-aeb5-aca66f08dad9 \N admin@gmail.com status changed Appointments FRM-R53RU4 Appointments status changed: FRM-R53RU4 {status} 2026-09-22 10:28:35.343668 2026-09-22 02:28:35.343668+00 bb9e341a-1d2f-433b-ae2a-119cba575564 49.146.140.233
fb161cf2-217e-4779-aa46-353b303afc1a \N admin@gmail.com updated Notifications 6d38ea69-05ff-44d1-b6d7-8125b56254ed Notifications updated: 6d38ea69-05ff-44d1-b6d7-8125b56254ed {is_read} 2026-09-22 10:28:51.619402 2026-09-22 02:28:51.619402+00 6d38ea69-05ff-44d1-b6d7-8125b56254ed 49.146.140.233
95c870af-a440-4992-aa13-5c5536000f38 \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 1 minute of inactivity. {} 2026-09-22 10:30:00.161407 2026-09-22 02:30:00.161407+00 \N 49.146.140.233
6a2a30f1-ba37-4185-bfee-41450d300f8d \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-22 10:31:10.914001 2026-09-22 02:31:10.914001+00 \N 49.146.140.233
727eba81-7377-40e7-b9c5-7263f3a4f001 \N admin@gmail.com updated Appointments FRM-R53RU4 Appointments updated: FRM-R53RU4 {last_reschedule_rejected_at,last_reschedule_rejection_message,pending_reschedule_date,pending_reschedule_reason,pending_reschedule_reference_code,pending_reschedule_request_id,pending_reschedule_start_time} 2026-09-22 10:31:38.233765 2026-09-22 02:31:38.233765+00 bb9e341a-1d2f-433b-ae2a-119cba575564 49.146.140.233
f5b4b6ac-7bc6-4621-9f0b-45fe3d047e62 \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 1 minute of inactivity. {} 2026-09-22 10:33:38.338333 2026-09-22 02:33:38.338333+00 \N 49.146.140.233
69945a0a-0467-416f-9532-042ecc61712a \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console.

{} 2026-09-22 10:34:14.961636 2026-09-22 02:34:14.961636+00 \N 49.146.140.233
28b706bc-8267-466f-90c2-f0f9449512aa \N admin@gmail.com updated Notifications 13b8faa3-823d-4473-b215-c919b398f187 Notifications updated: 13b8faa3-823d-4473-b215-c919b398f187 {is_read} 2026-09-22 10:34:23.002848 2026-09-22 02:34:23.002848+00 13b8faa3-823d-4473-b215-c919b398f187 49.146.140.233
deece48d-15fa-4dfb-88e4-be1cc1783ea9 \N admin@gmail.com updated Notifications ebbdab9c-583a-43dc-81ea-042363b3e75d Notifications updated: ebbdab9c-583a-43dc-81ea-042363b3e75d {is_read} 2026-09-22 10:34:23.002848 2026-09-22 02:34:23.002848+00 ebbdab9c-583a-43dc-81ea-042363b3e75d 49.146.140.233
f08decb6-e4fa-41bc-bfe3-0af1c81ea9ad \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 1 minute of inactivity. {} 2026-09-22 10:35:59.090253 2026-09-22 02:35:59.090253+00 \N 49.146.140.233
aa1936c6-e1d9-4666-afb4-1c7486a509f5 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-22 10:51:45.098345 2026-09-22 02:51:45.098345+00 \N 49.146.140.233
0615dbd3-1952-4c55-851c-a601acf90e65 \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 1 minute of inactivity. {} 2026-09-22 10:54:17.08396 2026-09-22 02:54:17.08396+00 \N 49.146.140.233
569bc9ad-2261-4485-b69d-a18e9079fc11 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-22 10:54:48.553419 2026-09-22 02:54:48.553419+00 \N 49.146.140.233
6d734085-abe2-48e2-8916-4f97783e60a7 \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 1 minute of inactivity. {} 2026-09-22 10:56:00.291702 2026-09-22 02:56:00.291702+00 \N 49.146.140.233
6442f5e3-13c8-4719-9e7c-6fe095b49fa0 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-22 10:56:28.340444 2026-09-22 02:56:28.340444+00 \N 49.146.140.233
4d12516e-88f1-4fb7-a58e-2f801a0abac4 \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 1 minute of inactivity. {} 2026-09-22 10:58:40.774712 2026-09-22 02:58:40.774712+00 \N 49.146.140.233
dd303211-73b7-46fa-b1c2-35962e2fa180 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-22 11:08:05.48959 2026-09-22 03:08:05.48959+00 \N 49.146.140.233
c9a74d6b-068d-4da5-ad9b-30f8add7d804 \N admin@gmail.com signed

out Authentication Admin console Administrator was automatically signed out after 1 minute of inactivity. {} 2026-09-22 11:09:18.78257 2026-09-22 03:09:18.78257+00 \N 49.146.140.233
7f762a93-969f-4178-b5a1-3d5a84ac6f3c \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-22 11:19:32.35405 2026-09-22 03:19:32.35405+00 \N 49.146.140.233
4eace517-b28d-4553-850f-b622df5f14d8 \N admin@gmail.com signed out Authentication Admin console Administrator signed out of the admin console. {} 2026-09-22 11:20:48.396598 2026-09-22 03:20:48.396598+00 \N 49.146.140.233
d6946697-7251-4bd3-8528-b1613693bf0f \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-22 14:39:13.927803 2026-09-22 06:39:13.927803+00 \N 143.44.164.152
ffbde592-0c8a-42c2-85f0-d2969906bda4 \N admin@gmail.com signed out Authentication Admin console Administrator signed out of the admin console. {} 2026-09-22 14:39:20.060321 2026-09-22 06:39:20.060321+00 \N 143.44.164.152
b0249b08-21b5-4945-afa0-3eda6efd53cc \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-22 18:33:49.924737 2026-09-22 10:33:49.924737+00 \N 143.44.164.66
03b2a8b0-5b50-449d-8e70-175e2e24c400 \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 1 minute of inactivity. {} 2026-09-22 18:38:06.336299 2026-09-22 10:38:06.336299+00 \N 143.44.164.66
4331c5fe-883f-4ce4-bafc-3bf8fa560e26 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-22 18:38:53.494116 2026-09-22 10:38:53.494116+00 \N 143.44.164.66
ce354dfd-c779-4db1-a6b3-1e1f16b1942f \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-22 18:44:08.453467 2026-09-22 10:44:08.453467+00 \N 143.44.164.66
5d4a219c-4e3c-4a3b-878c-086ccd1219f0 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-22 18:44:11.653718 2026-09-22 10:44:11.653718+00 \N 143.44.164.66
499b6697-d0f3-4b98-a10d-cc2942aa6372 \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 1 minute of inactivity. {} 2026-09-22 18:46:04.502774 2026-09-22 10:46:04.502774+00 \N 143.44.164.66
2d0e456a-3c58-4307-9b2e-f0940bbd6da4 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console.

{} 2026-09-22 22:22:33.746822 2026-09-22 14:22:33.746822+00 \N 143.44.164.66
794a777d-e6a2-4f47-b840-af7feb69ebc3 \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 1 minute of inactivity. {} 2026-09-22 22:25:22.921812 2026-09-22 14:25:22.921812+00 \N 143.44.164.66
7f0dee57-fc5d-4d0b-9f7a-e86ca743ab40 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-22 23:10:35.976715 2026-09-22 15:10:35.976715+00 \N 143.44.164.66
354d5437-88ae-40ea-b935-69e082236490 \N admin@gmail.com status changed Appointments FRM-CAAFST Appointments status changed: FRM-CAAFST {status} 2026-09-22 23:11:05.645983 2026-09-22 15:11:05.645983+00 b7424607-9e7d-4fd4-ae4e-73dba6c26b21 143.44.164.66
3b0678fc-24cd-4ccf-a018-16b5b0660864 \N admin@gmail.com deleted Appointment Services Change Oil Appointment Services deleted: Change Oil {} 2026-09-22 23:11:05.645983 2026-09-22 15:11:05.645983+00 1377ff7d-848d-4eca-b8fd-fece474d5255 143.44.164.66
49e7f3df-76de-4cc2-adaa-b6fa85a8c4a9 \N admin@gmail.com created Appointment Services Change Oil Appointment Services created: Change Oil {} 2026-09-22 23:11:05.645983 2026-09-22 15:11:05.645983+00 7097382e-1382-4768-80df-cfb67b63ba41 143.44.164.66
a833bb77-7a9d-4fca-8016-d0bf5653e03d \N admin@gmail.com created Appointments FRM-E6Y7KF Appointments created: FRM-E6Y7KF {} 2026-09-22 23:11:44.983303 2026-09-22 15:11:44.983303+00 147297a6-c092-4337-ab4e-2c86b7848da9 143.44.164.66
aa550226-bc1d-4340-8377-1c766890d767 \N admin@gmail.com created Appointment Services Change Oil Appointment Services created: Change Oil {} 2026-09-22 23:11:44.983303 2026-09-22 15:11:44.983303+00 f821f4d8-e889-4c99-9199-89d641fbb5b5 143.44.164.66
df892acb-3af3-4709-b2e6-fb250716b430 \N admin@gmail.com created Appointment Services Brake Service Appointment Services created: Brake Service {} 2026-09-22 23:11:44.983303 2026-09-22 15:11:44.983303+00 2e92f279-e0ad-4872-9267-35f694a311d9 143.44.164.66
ab2503aa-ed59-449f-b2c0-d2a1bf0cfaee \N admin@gmail.com status changed Appointments FRM-F98X7B Appointments status changed: FRM-F98X7B {admin_notes,pending_reschedule_date,pending_reschedule_reason,pending_reschedule_reference_code,pending_reschedule_request_id,pending_reschedule_start_time,reschedule_count,rescheduled_to_appointment_id,status} 2026-09-22 23:11:44.983303 2026-09-22 15:11:44.983303+00 64fe7d0c-3604-4dc4-a072-a3344c02e337 143.44.164.66
fd7ed3dc-b450-42a7-9ac9-8122332f3d2c \N admin@gmail.com created Notifications c66beaeb-25b1-4f74-8af6-b9fededcc4d3 Notifications created: c66beaeb-25b1-4f74-8af6-b9fededcc4d3 {} 2026-09-22 23:11:44.983303

2026-09-22 15:11:44.983303+00 c66beaeb-25b1-4f74-8af6-b9fededcc4d3
143.44.164.66
b031d8c1-d6b2-4a7d-bf8e-604f2cecef62 \N admin@gmail.com created
Appointments FRM-5E6MYB Appointments created: FRM-5E6MYB {}
2026-09-22 23:11:47.346985 2026-09-22 15:11:47.346985+00
4974f9ca-6566-491d-8108-0f61dceff13a 143.44.164.66
dccd71f7-63db-4a38-a19e-9d85a4a7d6b1 \N admin@gmail.com created
Appointment Services Motorcycle Tune-Up Appointment Services created:
Motorcycle Tune-Up {} 2026-09-22 23:11:47.346985 2026-09-22
15:11:47.346985+00 e9ec0361-b965-4f1c-8781-f315136cfd01 143.44.164.66
073e4052-95d6-4e10-9673-f1bc09c44d24 \N admin@gmail.com created
Appointment Services Motorcylce Tune-Up Appointment Services created:
Motorcylce Tune-Up {} 2026-09-22 23:11:47.346985 2026-09-22
15:11:47.346985+00 753adf88-c833-4bed-90ce-3b8bef264e84
143.44.164.66
dfd6fe9c-b2e3-47c2-b9cb-140dbd9e7bd8 \N admin@gmail.com created
Appointment Services Clutch Adjustment/Repair Appointment Services
created: Clutch Adjustment/Repair {} 2026-09-22 23:11:47.346985
2026-09-22 15:11:47.346985+00 0c6d47c9-89b6-4735-b110-af242384bf1a
143.44.164.66
a29c0732-d2f6-4d9a-b472-ff7a778eeafe \N admin@gmail.com status
changed Appointments FRM-5YVTRB Appointments status changed:
FRM-5YVTRB {admin_notes,pending_reschedule_date,pending_reschedule_r
eason,pending_reschedule_reference_code,pending_reschedule_request_id,pen
ding_reschedule_start_time,reschedule_count,rescheduled_to_appointment_id,s
tatus} 2026-09-22 23:11:47.346985 2026-09-22 15:11:47.346985+00
33fbda0f-318f-41fd-8eac-328e296ea0d3 143.44.164.66
734e0c95-d73b-4d9e-886b-badbf33e1ea1 \N admin@gmail.com created
Notifications f6125587-0779-422c-b67a-28d3f60e29e8 Notifications created:
f6125587-0779-422c-b67a-28d3f60e29e8 {} 2026-09-22 23:11:47.346985
2026-09-22 15:11:47.346985+00 f6125587-0779-422c-b67a-28d3f60e29e8
143.44.164.66
1e9bcc9b-f4d3-4e17-b1f4-58d09d83d5e8 \N admin@gmail.com updated
Notifications 043dc5a7-0310-447b-ad71-dbc2d7578087 Notifications
updated: 043dc5a7-0310-447b-ad71-dbc2d7578087 {is_read} 2026-09-22
23:12:30.266291 2026-09-22 15:12:30.266291+00
043dc5a7-0310-447b-ad71-dbc2d7578087 143.44.164.66
390aa8e2-2407-405f-887e-ba823b5caa0d \N admin@gmail.com updated
Notifications 9139cc3c-8f07-474c-a7ce-a0f4efc254c8 Notifications updated:
9139cc3c-8f07-474c-a7ce-a0f4efc254c8 {is_read} 2026-09-22
23:12:30.266291 2026-09-22 15:12:30.266291+00
9139cc3c-8f07-474c-a7ce-a0f4efc254c8 143.44.164.66
5be88915-3031-4d70-9794-21bc8eaac8f5 \N admin@gmail.com updated
Notifications c66beaeb-25b1-4f74-8af6-b9fededcc4d3 Notifications updated:
c66beaeb-25b1-4f74-8af6-b9fededcc4d3 {is_read} 2026-09-22

23:12:30.266291 2026-09-22 15:12:30.266291+00
c66beaeb-25b1-4f74-8af6-b9fededcc4d3 143.44.164.66
0f18d5f5-897f-42c1-a174-5fc47f101919 \N admin@gmail.com updated
Notifications f6125587-0779-422c-b67a-28d3f60e29e8 Notifications updated:
f6125587-0779-422c-b67a-28d3f60e29e8 {is_read} 2026-09-22
23:12:30.266291 2026-09-22 15:12:30.266291+00
f6125587-0779-422c-b67a-28d3f60e29e8 143.44.164.66
1c1c2fd8-806d-4a6c-ba1b-ff55feb2d10a \N admin@gmail.com deleted
Notifications 8696313f-065c-459b-a04b-2b037d49743a Notifications deleted:
8696313f-065c-459b-a04b-2b037d49743a {} 2026-09-22 23:12:51.730896
2026-09-22 15:12:51.730896+00 8696313f-065c-459b-a04b-2b037d49743a
143.44.164.66
fb99838f-9d05-4804-82ad-5d1faadbbdee \N admin@gmail.com deleted
Notifications dcc90fe3-1c23-44d3-9b78-ffee1ea354b9 Notifications deleted:
dcc90fe3-1c23-44d3-9b78-ffee1ea354b9 {} 2026-09-22 23:13:12.104003
2026-09-22 15:13:12.104003+00 dcc90fe3-1c23-44d3-9b78-ffee1ea354b9
143.44.164.66
2524bf64-49e6-4307-939a-b01af485a008 \N admin@gmail.com deleted
Notifications b2f57aab-0b1e-44bd-87d7-bf2da162cfc3 Notifications deleted:
b2f57aab-0b1e-44bd-87d7-bf2da162cfc3 {} 2026-09-22 23:13:19.80632
2026-09-22 15:13:19.80632+00 b2f57aab-0b1e-44bd-87d7-bf2da162cfc3
143.44.164.66
5f41779f-b805-4be1-aafa-8a3fa33e094a \N admin@gmail.com deleted
Notifications 84429060-5c91-4ade-b15c-45e8d06c3f4e Notifications deleted:
84429060-5c91-4ade-b15c-45e8d06c3f4e {} 2026-09-22 23:13:36.770076
2026-09-22 15:13:36.770076+00 84429060-5c91-4ade-b15c-45e8d06c3f4e
143.44.164.66
0f1455ef-9ed2-4604-8c64-d3dfa062e6dd \N admin@gmail.com signed out
Authentication Admin console Administrator was automatically signed out
after 1 minute of inactivity. {} 2026-09-22 23:14:46.343244 2026-09-22
15:14:46.343244+00 \N 143.44.164.66
e2517da4-205d-4a2e-bdb3-ed27e17194dd \N admin@gmail.com signed
out Authentication Admin console Administrator was automatically signed
out after 1 minute of inactivity. {} 2026-09-23 01:00:39.517952
2026-09-22 17:00:39.517952+00 \N 136.158.33.87
4979c3d1-764a-47ce-8b44-8dca81addc60 \N admin@gmail.com signed in
Authentication Admin console Administrator signed in to the admin console.
{} 2026-09-23 06:55:23.140397 2026-09-22 22:55:23.140397+00 \N
143.44.164.184
0e353531-8ae8-4bdd-8452-bb512b1d7918 \N admin@gmail.com signed
out Authentication Admin console Administrator was automatically signed
out after 1 minute of inactivity. {} 2026-09-23 06:57:32.131677
2026-09-22 22:57:32.131677+00 \N 143.44.164.184
2ed49a01-494a-4c93-aeda-8cc09ac5ada2 \N admin@gmail.com signed in
Authentication Admin console Administrator signed in to the admin console.

{} 2026-09-23 06:58:57.735054 2026-09-22 22:58:57.735054+00 \N 143.44.164.184
2294ded8-c055-41b6-b3d3-b34f319b6313 \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 1 minute of inactivity. {} 2026-09-23 07:01:03.905503 2026-09-22 23:01:03.905503+00 \N 143.44.164.184
e06e0271-c157-431f-93a9-e95fa38054c7 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-23 07:05:55.071159 2026-09-22 23:05:55.071159+00 \N 143.44.164.184
bfb08379-d1c9-43e7-a0a5-e447ef912834 \N admin@gmail.com updated Notifications 64a67890-58c6-4b5c-b5eb-3a021c86cba4 Notifications updated: 64a67890-58c6-4b5c-b5eb-3a021c86cba4 {is_read} 2026-09-23 07:06:23.357925 2026-09-22 23:06:23.357925+00 64a67890-58c6-4b5c-b5eb-3a021c86cba4 143.44.164.184
4ff7e330-ccc7-4f24-8389-e9bf2c6cf222 \N admin@gmail.com updated Notifications 7cc95602-4033-421c-86a5-11d21d46b502 Notifications updated: 7cc95602-4033-421c-86a5-11d21d46b502 {is_read} 2026-09-23 07:06:23.357925 2026-09-22 23:06:23.357925+00 7cc95602-4033-421c-86a5-11d21d46b502 143.44.164.184
85827cf7-e4d4-44f7-ab61-921e7de503a2 \N admin@gmail.com deleted Notifications cea778ae-ff65-4b6b-a64e-2e443e73661f Notifications deleted: cea778ae-ff65-4b6b-a64e-2e443e73661f {} 2026-09-23 07:07:03.704225 2026-09-22 23:07:03.704225+00 cea778ae-ff65-4b6b-a64e-2e443e73661f 143.44.164.184
d55b3687-abfd-4072-abfe-ca6028b0922b \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 1 minute of inactivity. {} 2026-09-23 07:08:29.185316 2026-09-22 23:08:29.185316+00 \N 143.44.164.184
fce874ec-4cda-41e7-b755-717e3dd06280 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-23 19:47:11.086162 2026-09-23 11:47:11.086162+00 \N 143.44.164.123
1287b473-7d71-4f11-b388-297da22a9adb \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 1 minute of inactivity. {} 2026-09-23 19:49:29.308444 2026-09-23 11:49:29.308444+00 \N 143.44.164.123
dd295fb2-dd73-4e23-b8fb-2baa5fc7aa42 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-23 19:53:31.777089 2026-09-23 11:53:31.777089+00 \N 143.44.164.123
d471c149-6839-405c-b4b3-9f37eda57d67 \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 1 minute of inactivity. {} 2026-09-23 19:55:21.939598 2026-09-23 11:55:21.939598+00 \N 143.44.164.123

6b3ac6b3-a2ba-4eeb-a4d9-f59bb8d21a95 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-23 22:17:59.129315 2026-09-23 14:17:59.129315+00 \N 143.44.164.241
9990cea3-abdd-4354-abff-135515d3ad6b \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-23 22:20:51.000663 2026-09-23 14:20:51.000663+00 \N 143.44.164.241
552601b3-1738-4e43-80aa-911d2ead422d \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-23 22:28:18.704027 2026-09-23 14:28:18.704027+00 \N 143.44.164.241
89faa0e1-6665-422c-894b-855e598693b3 \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-23 22:31:11.964035 2026-09-23 14:31:11.964035+00 \N 143.44.164.241
970d56cb-6335-4190-b3dc-1821f28b2d45 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-23 22:34:13.601179 2026-09-23 14:34:13.601179+00 \N 143.44.164.241
ec102455-ce14-460e-9b4a-eeba91cd5f5d \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-23 22:37:05.148223 2026-09-23 14:37:05.148223+00 \N 143.44.164.241
691330e0-0988-4c1e-aa4e-73e37aee5150 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-24 00:03:16.204847 2026-09-23 16:03:16.204847+00 \N 143.44.164.241
bf8b0843-e7e2-4a08-b866-baeeaf756a40 \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-24 00:05:41.727144 2026-09-23 16:05:41.727144+00 \N 143.44.164.241
0349c60c-1abc-4e34-be3c-103ec3346916 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-24 00:21:24.91945 2026-09-23 16:21:24.91945+00 \N 143.44.164.241
6b2ab9ea-6857-4efe-a836-d2e281466604 \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-24 00:24:14.015752 2026-09-23 16:24:14.015752+00 \N 143.44.164.241
2c2f53dc-32b2-4229-a46d-a22adb919261 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-24 01:04:02.474384 2026-09-23 17:04:02.474384+00 \N 143.44.164.241
251de0ce-618d-427b-af58-b748c2b7e635 \N admin@gmail.com signed out

Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-24 01:07:19.471641 2026-09-23 17:07:19.471641+00 \N 143.44.164.241
7f580fb1-7e69-4d92-90d6-6e6001e40851 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-24 01:22:08.249809 2026-09-23 17:22:08.249809+00 \N 143.44.164.241
57066685-efed-462e-8631-49716a8bf492 \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-24 01:27:26.408933 2026-09-23 17:27:26.408933+00 \N 143.44.164.241
9360650a-a35c-493f-ab2d-b1780da5b191 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-24 07:02:44.260353 2026-09-23 23:02:44.260353+00 \N 143.44.164.241
ddbb18c3-1107-4feb-9109-24ed1d478630 \N admin@gmail.com status changed Appointments FRM-S32DGA Appointments status changed: FRM-S32DGA {service_started_at,status} 2026-09-24 07:02:56.164076 2026-09-23 23:02:56.164076+00 c2948c1c-8f78-441c-bcc7-1cfa03c0718b 143.44.164.241
6a856274-b76d-456b-a9c0-2c99a98433cd \N admin@gmail.com status changed Appointments FRM-S32DGA Appointments status changed: FRM-S32DGA {service_ended_at,status} 2026-09-24 07:04:01.636083 2026-09-23 23:04:01.636083+00 c2948c1c-8f78-441c-bcc7-1cfa03c0718b 143.44.164.241
e19e305a-3661-4f0f-a0b5-89f1e84266c2 \N admin@gmail.com deleted Appointment Services Chain & Sprocket Replacement Appointment Services deleted: Chain & Sprocket Replacement {} 2026-09-24 07:04:01.636083 2026-09-23 23:04:01.636083+00 85a0d1c9-345d-43ac-a823-451feb096a5f 143.44.164.241
6cde19c4-4f9f-4dde-8c0a-23f0fa67ec8c \N admin@gmail.com created Appointment Services Chain & Sprocket Replacement Appointment Services created: Chain & Sprocket Replacement {} 2026-09-24 07:04:01.636083 2026-09-23 23:04:01.636083+00 8a58d33a-f335-4bf6-a0f2-12f00cdd58f5 143.44.164.241
ef05e70f-6a77-4b20-8530-208ebd6898e6 \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-24 07:10:15.106513 2026-09-23 23:10:15.106513+00 \N 143.44.164.241
b983724c-25c7-405b-8297-9c0b24efa1f0 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-24 18:51:47.727255 2026-09-24 10:51:47.727255+00 \N 122.54.182.182
cde25ea3-dd02-44c7-a807-f1d7bde3281e \N admin@gmail.com status changed Appointments FRM-6VWY2Z Appointments status changed:

FRM-6VWY2Z {status} 2026-09-24 18:54:03.098116 2026-09-24 10:54:03.098116+00 6a347f55-d34f-42ff-9a73-18f6f0830092 122.54.182.182 9409c1fd-1b7f-4c1d-ab0b-0a0ba4a41fc0 \N admin@gmail.com signed out Authentication Admin console Administrator signed out of the admin console. {} 2026-09-24 18:54:15.086604 2026-09-24 10:54:15.086604+00 \N 122.54.182.182
836b6fef-fd40-44c0-9cb8-732f6cdd2723 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-24 20:53:19.235281 2026-09-24 12:53:19.235281+00 \N 143.44.164.123
fa6c7ea8-d91e-45a2-bacd-17e8421f731c \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-24 20:56:10.030834 2026-09-24 12:56:10.030834+00 \N 143.44.164.123
3ce17df1-a925-4115-bf8d-8544d1082185 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-25 07:58:06.886349 2026-09-24 23:58:06.886349+00 \N 131.226.109.133
754ab908-9bd2-4cb0-8899-007870b6268f \N admin@gmail.com status changed Appointments FRM-5E6MYB Appointments status changed: FRM-5E6MYB {service_started_at,status} 2026-09-25 08:01:20.919776 2026-09-25 00:01:20.919776+00 4974f9ca-6566-491d-8108-0f61dceff13a 131.226.109.133
a49b7b99-f1cc-4fd2-9032-0238d3577a54 \N admin@gmail.com archived Appointments FRM-5E6MYB Appointments archived: FRM-5E6MYB {archived_at,is_archived} 2026-09-25 08:02:05.032596 2026-09-25 00:02:05.032596+00 4974f9ca-6566-491d-8108-0f61dceff13a 131.226.109.133
603821a8-6593-491e-8081-0a9699acc1ff \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-25 17:35:25.13992 2026-09-25 09:35:25.13992+00 \N 143.44.164.241
5140ddf4-8400-43ba-89a1-d61cf52b878e \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-25 17:35:27.989687 2026-09-25 09:35:27.989687+00 \N 143.44.164.241
8e61f1bf-65e9-455a-8f19-bd958886a2de \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-25 17:39:49.912487 2026-09-25 09:39:49.912487+00 \N 143.44.164.241
c63456c3-2d41-4ae3-80de-cf46608610b0 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-25 17:48:46.661211 2026-09-25 09:48:46.661211+00 \N 143.44.164.241
5d00fd23-6409-4184-a92a-e6603e416b46 \N admin@gmail.com status

changed Appointments FRM-746ETT Appointments status changed: FRM-746ETT {status} 2026-09-25 17:49:12.081898 2026-09-25 09:49:12.081898+00 6957127d-9bdd-4173-a856-2f08819ce87f 143.44.164.241 27179212-8e75-45e5-a07b-3087c6c75cc2 \N admin@gmail.com status changed Appointments FRM-E6Y7KF Appointments status changed: FRM-E6Y7KF {service_started_at,status} 2026-09-25 17:49:34.404998 2026-09-25 09:49:34.404998+00 147297a6-c092-4337-ab4e-2c86b7848da9 143.44.164.241 1849a8b0-1545-454f-875f-821951b4bda8 \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-25 17:53:10.93559 2026-09-25 09:53:10.93559+00 \N 143.44.164.241 52d177cb-7aab-48ba-bd90-575d987a36a6 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-25 18:14:36.259979 2026-09-25 10:14:36.259979+00 \N 143.44.164.241 40b38072-bc8d-49a6-bd38-09d25d7fa4c7 \N admin@gmail.com updated Notifications c00cbae9-79f6-4d96-a962-4a6c35404ae1 Notifications updated: c00cbae9-79f6-4d96-a962-4a6c35404ae1 {is_read} 2026-09-25 18:14:42.30244 2026-09-25 10:14:42.30244+00 c00cbae9-79f6-4d96-a962-4a6c35404ae1 143.44.164.241 29c9a734-2045-43cd-b33e-71960df7c044 \N admin@gmail.com updated Notifications e12f0d5d-9421-47f0-8b15-4c8f60899ae3 Notifications updated: e12f0d5d-9421-47f0-8b15-4c8f60899ae3 {is_read} 2026-09-25 18:14:43.528071 2026-09-25 10:14:43.528071+00 e12f0d5d-9421-47f0-8b15-4c8f60899ae3 143.44.164.241 1a88a56d-69af-40a1-9891-58a45c6b2e35 \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-25 18:20:57.845963 2026-09-25 10:20:57.845963+00 \N 143.44.164.241 73851d1e-ef00-4008-b4a0-f46b9cf3ddf7 \N admin@gmail.com signed in Authentication Admin console Administrator signed in to the admin console. {} 2026-09-25 19:01:25.154492 2026-09-25 11:01:25.154492+00 \N 143.44.164.241 05dda57b-ea5a-4ddb-90fa-abf9013078a6 \N admin@gmail.com updated Account Settings Administrator profile Administrator account settings updated. {Password} 2026-09-25 19:02:55.442564 2026-09-25 11:02:55.442564+00 \N 143.44.164.241 a4033ef6-3f32-4e06-b29a-d4009ede4969 \N admin@gmail.com signed out Authentication Admin console Administrator was automatically signed out after 2 minutes of inactivity. {} 2026-09-25 19:12:58.375561 2026-09-25 11:12:58.375561+00 \N 143.44.164.241 88d17249-0182-45e0-b961-a525dd97ee09 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 eapolona0803@gmail.com signed

out	Authentication	Admin console	Administrator was automatically signed out after 2 minutes of inactivity.	{}	2026-09-27	09:14:31.195613	2026-09-27 01:14:31.195613+00	\N	143.44.164.184
\.

--
-- Data for Name: admin_session_handover_tokens; Type: TABLE DATA; Schema: public; Owner: postgres
--

COPY "public"."admin_session_handover_tokens" ("token", "admin_id", "session_id", "expires_at", "created_at") FROM stdin;
\.

--
-- Data for Name: admin_session_leases; Type: TABLE DATA; Schema: public; Owner: postgres
--

COPY "public"."admin_session_leases" ("admin_id", "session_id", "expires_at", "last_seen_at", "created_at") FROM stdin;
\.

--
-- Data for Name: crew_members; Type: TABLE DATA; Schema: public; Owner: postgres
--

COPY "public"."crew_members" ("id", "name", "role", "phone", "is_active", "created_at", "updated_at", "is_archived", "archived_at") FROM stdin;
5cb00871-f5ef-4ccf-bd61-4746909237e2	Joe Mar	Mechanic	\N	t	2026-08-23 00:54:01.223024+00	2026-09-27 01:41:42.517478+00	f	\N
f3c5770f-e485-4efc-83f6-892422d8592e	Jonde	Mechanic	09181234567	t	2026-08-19 05:14:48.549897+00	2026-09-27 01:40:35.546051+00	f	\N
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b	Junard	Mechanic	09191234567	t	2026-08-19 05:14:48.549897+00	2026-09-27 01:40:54.373371+00	f	\N
03bb877e-57f8-42dc-9c0e-3fcc82b1a207	Joshua	Mechanic	09926067723	t	2026-08-21 08:52:01.842748+00	2026-09-27 01:41:10.496963+00	f	\N
1700c9e8-37e5-42c2-98e8-48854585ba7d	Liezel	Mechanic	09171234567	t	2026-08-19 05:11:46.27827+00	2026-09-27 01:41:27.613539+00	f	\N
\.

--
-- Data for Name: appointments; Type: TABLE DATA; Schema: public; Owner: postgres
--

COPY "public"."appointments" ("id", "reference_code", "customer_name", "phone", "moto_brand", "moto_model", "moto_variant", "moto_year", "plate_number", "appointment_date", "start_time", "status", "notes", "admin_notes", "total_estimate", "assigned_crew_id", "is_archived", "terms_accepted", "cancelled_at", "created_at", "updated_at", "booking_duration_minutes", "booking_request_id", "rescheduled_from_appointment_id", "rescheduled_to_appointment_id", "pending_reschedule_request_id", "pending_reschedule_date", "pending_reschedule_start_time", "pending_reschedule_reason", "reschedule_count", "last_reschedule_rejected_at", "last_reschedule_rejection_message", "first_name", "middle_name", "last_name", "pending_reschedule_reference_code", "archived_at", "service_started_at", "service_ended_at", "arrival_notification_snooze_count", "arrival_notification_snoozed_until", "completion_notification_snoozed_until", "no_show_count_at_booking", "email", "wants_products") FROM stdin;
617073f7-8cfa-4f3f-8352-aff1a76da780 FRM-HWAVDJ Erikson Apolona 09926067723 Honda BeAT Playful \N 2026 NBT23 2026-10-05 08:00:00 completed \N \N 50.00 f3c5770f-e485-4efc-83f6-892422d8592e f t \N 2026-09-30 05:03:54.222561+00 2026-10-07 00:22:30.886674+00 90 2172a06e-52a5-4f9c-a4e4-bcd37e443148 \N \N \N \N \N 0 \N \N Erikson \N Apolona \N \N 2026-10-06 13:17:23.289101+00 2026-10-07 00:22:30.886674+00 0 \N \N 0 \N f c5f4d85a-7617-42c7-84db-941d333879bf FRM-2LYRZC Erik Apolona 09999999999 Honda XRM 125 \N 2026 BBC125 2026-10-12 08:00:00 rejected \N \N 50.00 \N f t \N 2026-10-07 00:21:36.678803+00 2026-10-08 11:23:42.086301+00 60 736431fd-6a5f-4c77-a0a4-d6cfa79a7c5e \N \N \N \N \N \N 0 \N \N Erik \N Apolona \N \N \N \N 0 \N \N 0 eapolona0803@gmail.com f cfe5e95c-d7d8-4ba5-b6e7-9020b664a0fc FRM-ZBM5AB Erik Eri 09242698505 Bajaj CT110 \N 2026 MM45 2026-10-30 13:00:00 confirmed \N \N 70.00 5cb00871-f5ef-4ccf-bd61-4746909237e2 f t \N 2026-10-13 12:24:16.558559+00 2026-10-13 12:25:46.398361+00 90 3a0b5660-0196-4342-8d51-60803a130dd3 \N \N \N \N \N \N 0 \N \N Erik \N Eri \N \N \N \N 0 \N \N 0 eapolona0803@gmail.com t af37178b-4dfd-455f-8d90-2f7d3f4b4304 FRM-ATZJXT Apo Erik 09242698505 Honda XRM 125 \N 2026 POYO234 2026-10-27

08:00:00 confirmed \N \N 50.00
5cb00871-f5ef-4ccf-bd61-4746909237e2 f t \N 2026-10-09
08:57:32.115662+00 2026-10-09 08:57:48.811002+00 90
b1ef4b01-5b9b-41be-ba46-38fac405afa5 \N \N \N \N \N \N 0 \N
\N Apo \N Erik \N \N \N \N 0 \N \N 0
erikson.apolona@bisu.edu.ph t
eda6b8a7-6dc1-4584-8ad5-e918438c0a05 FRM-AQY69P Joshua Swete
Banawan 09094291771 RUSI Rusi Wave125 \N 2026 LVE123
2026-10-12 08:00:00 completed \N \N 50.00
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b f t \N 2026-09-30
00:44:02.356662+00 2026-10-14 08:15:10.905783+00 60
f6b4025b-eeeb-4832-b7cb-c5063fd18c76 \N \N \N \N \N \N 0 \N
\N Joshua Swete Banawan \N \N 2026-10-13 12:25:41.94506+00
2026-10-14 08:15:10.905783+00 0 \N \N 0 \N f
6957127d-9bdd-4173-a856-2f08819ce87f FRM-746ETT ashley chiu
09542154664 KTM 350 EXC \N 2026 BCD1 2026-09-29 10:00:00
completed \N \N 500.00 03bb877e-57f8-42dc-9c0e-3fcc82b1a207 f t
\N 2026-09-25 09:39:41.309607+00 2026-09-29 10:41:26.216923+00 120
4f7b6f92-92ae-43bc-8f10-2e45003fdd18 \N \N \N \N \N \N 0 \N
\N ashley \N chiu \N \N 2026-09-29 06:09:23.975551+00
2026-09-29 10:41:26.216923+00 0 \N \N 0 \N f
c2948c1c-8f78-441c-bcc7-1cfa03c0718b FRM-S32DGA rafa rafa
09090909095 Bajaj Avenger160 \N 2026 23434 2026-09-23
11:00:00 completed \N \N 500.00
03bb877e-57f8-42dc-9c0e-3fcc82b1a207 f t \N 2026-09-21
02:39:06.066492+00 2026-09-24 13:29:34.594168+00 120
35e1d20a-23da-4413-9c90-e5e31a5e94c7 \N \N \N \N \N \N 0 \N
\N rafa \N rafa \N \N 2026-09-23 23:02:56.164076+00 2026-09-23
23:04:01.636083+00 0 \N \N 0 \N f
545503a4-1416-4f5f-83ba-ebc50f5912d7 FRM-3JWTZU lawrence l fuerzas
09926067723 Honda Beatf1 \N 2026 ABC123 2026-09-21 08:00:00
rescheduled \N Rescheduled to FRM-CAAFST\nReason for rescheduling:
mao rani free time nako 50.00 03bb877e-57f8-42dc-9c0e-3fcc82b1a207 f
t \N 2026-09-18 02:10:13.008021+00 2026-09-24 13:29:34.594168+00
90 80325580-1c10-4ca9-bd66-159710e12a7a \N
b7424607-9e7d-4fd4-ae4e-73dba6c26b21 \N \N \N \N 1 \N \N
lawrence l fuerzas \N \N \N \N 0 \N \N 0 \N f
147297a6-c092-4337-ab4e-2c86b7848da9 FRM-E6Y7KF rafa rafa
09926067732 Honda Beat110 \N 2026 ABVE123 2026-09-25
08:00:00 completed \N Rescheduled from FRM-F98X7B\nReason for
rescheduling: free 400.00 03bb877e-57f8-42dc-9c0e-3fcc82b1a207 f t
\N 2026-09-22 15:11:44.983303+00 2026-09-25 12:18:27.916329+00 150
80f471e7-0458-445a-8b42-8fd8a9ff9c38
64fe7d0c-3604-4dc4-a072-a3344c02e337 \N \N \N \N \N 1 \N \N
rafa \N rafa \N \N 2026-09-25 09:49:34.404998+00 2026-09-25

12:18:27.916329+00 0 \N \N 0 \N f
6c0de99e-7522-4f4e-ad11-0913682e3ae3 FRM-JKNK84 Wukong Sun
09242698505 Yamaha Mio i125 \N 2026 PPPT45 2026-10-19
08:00:00 completed \N \N 100.00
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b f t \N 2026-10-15
07:08:09.579389+00 2026-10-21 10:19:53.441669+00 90
670c4887-4fc6-435a-abc1-6ed97026b097 \N \N \N \N \N \N 0 \N
\N Wukong \N Sun \N \N 2026-10-21 10:19:33.517794+00
2026-10-21 10:19:53.441669+00 0 \N \N 0
erikson.apolona@bisu.edu.ph t
2ce47e77-9cfc-49b8-9710-c3b33294eb54 FRM-4RKQTN Ash L Chiu
09543227314 Kymco Like150 \N 2026 SHEFE 2026-10-12
13:00:00 rescheduled \N Rescheduled to FRM-ULM8GG\nReason for
rescheduling: free 50.00 659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b f t \N
2026-09-30 04:51:30.842347+00 2026-09-30 05:01:21.761562+00 120
fcd34bc0-4881-478a-88ce-9718659dfdef \N
de1e21dc-53e4-4362-8898-3bd1da2d67fc \N \N \N \N 1 \N \N Ash
L Chiu \N \N \N \N 0 \N \N 0 \N f
bb9e341a-1d2f-433b-ae2a-119cba575564 FRM-R53RU4 SFHGHJ
ASGYDFYUGH 09128964387 Honda Beat (110cc) \N 2026 ADV343
2026-09-28 08:00:00 cancelled \N \N 50.00
1700c9e8-37e5-42c2-98e8-48854585ba7d f t 2026-09-22
02:33:56.616285+00 2026-09-22 02:27:08.631764+00 2026-10-06
17:07:33.654736+00 90 0e369329-bccf-484b-ad09-7be6b389446f \N \N
\N \N \N \N 0 2026-09-22 02:31:38.233765+00 Reschedule Request
Rejected. Your requested reschedule was not approved. Your original
appointment remains unchanged and reserved. SFHGHJ \N
ASGYDFYUGH \N \N \N \N 0 \N \N 0 \N f
71b72a10-644d-4b57-ace3-740377b48bd6 FRM-4J2DB9 lawrence l fuerzas
09926067723 Honda Beat (110cc) \N 2026 ABC123 2026-09-21
08:00:00 rescheduled \N Rescheduled to FRM-77GZQE\nReason for
rescheduling: free time 50.00 03bb877e-57f8-42dc-9c0e-3fcc82b1a207 f t
\N 2026-09-15 09:43:17.379731+00 2026-10-06 17:07:33.654736+00 90
17b569b4-a36a-412d-954f-2e7a74d25694 \N
d070ed76-55d2-40e1-a55a-fc58fc88bfb8 \N \N \N \N 1 \N \N
lawrence l fuerzas \N \N \N \N 0 \N \N 0 \N f
2334d5bb-94f4-4889-8a06-0caee9004790 FRM-6NLQEM james patac
09926067712 Honda Beat (110cc) \N 2026 NOD123 2026-09-28
08:00:00 cancelled \N \N 8650.00
03bb877e-57f8-42dc-9c0e-3fcc82b1a207 f t 2026-09-22
22:55:05.511671+00 2026-09-22 22:54:19.20688+00 2026-10-06
17:07:33.654736+00 540 ee420e95-e43e-4815-a641-1cbcd7b62fcd \N \N
\N \N \N \N 0 \N \N james \N patac \N \N \N \N 0 \N
\N 0 \N f
2d1379ce-21a4-459a-a1fa-97b2f7cc75db FRM-XUTXNU Lily Cruz

09507704723 Honda CB1000R \N 2026 RGR456 2026-09-29
08:30:00 completed \N \N 2900.00
1700c9e8-37e5-42c2-98e8-48854585ba7d f t \N 2026-09-26
06:43:41.257218+00 2026-10-06 17:07:33.654736+00 450
9c8c2479-4b47-410b-bda3-a22fc0fd097a \N \N \N \N \N \N 0 \N
\N Lily \N Cruz \N \N 2026-09-29 00:50:26.944452+00 2026-09-29
10:41:24.158694+00 0 \N \N 0 \N f
de1e21dc-53e4-4362-8898-3bd1da2d67fc FRM-ULM8GG Ash L Chiu
09543227314 Kymco Like150 \N 2026 SHEFE 2026-10-05
08:00:00 completed \N Rescheduled from FRM-4RKQTN\nReason for
rescheduling: free 50.00 659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b f t \N
2026-09-30 05:01:21.761562+00 2026-10-07 00:22:28.433925+00 120
cbe68a6e-643e-4c50-a690-ba66960aece1
2ce47e77-9cfc-49b8-9710-c3b33294eb54 \N \N \N \N \N 1 \N \N
Ash L Chiu \N \N 2026-10-06 13:17:21.727957+00 2026-10-07
00:22:28.433925+00 0 \N \N 0 \N f
cab87474-af88-46f2-8e66-7acd2e5d2baa FRM-APD3BN Apo Erikson
09242698505 Honda XRM 125 \N 2026 TKO098 2026-10-30
08:00:00 confirmed \N \N 50.00
5cb00871-f5ef-4ccf-bd61-4746909237e2 f t \N 2026-10-11
05:39:06.383071+00 2026-10-11 05:58:05.259017+00 60
7c91df51-0d76-4dfc-b803-a9fd857225fa \N \N \N \N \N \N 0 \N
\N Apo \N Erikson \N \N \N \N 0 \N \N 0
onboarding@resend.dev t
dc1c4a1c-1532-4d25-bbd5-b7d8793de6e9 FRM-577L8T Apolo Erikson
09242698505 Honda XRM 125 \N 2026 POYO234 2026-10-22
08:00:00 confirmed \N \N 50.00
5cb00871-f5ef-4ccf-bd61-4746909237e2 f t \N 2026-10-09
08:54:58.290663+00 2026-10-09 08:55:24.539046+00 60
e8954135-1f5f-472f-9dbd-348e924030a8 \N \N \N \N \N \N 0 \N
\N Apolo \N Erikson \N \N \N \N 0 \N \N 0
erikson.apolona@bisu.edu.ph t
30c80b92-c789-40fc-936d-5f6567091423 FRM-EVXKHD Lara Jessa
Apolona 09468033084 Honda Beat110 FI \N 2026 NMO123
2026-10-09 08:00:00 completed \N Rescheduled from
FRM-ZTUG43\nReason for rescheduling: free 50.00
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b f t \N 2026-09-28
11:38:09.628656+00 2026-10-11 05:39:40.891101+00 60
379ea325-557a-422f-bab5-b786228343cc
a26950f7-62f0-439a-8f3b-07fa6a6c50d6 \N \N \N \N \N 1 \N \N
Lara Jessa \N Apolona \N \N 2026-10-09 08:55:29.629302+00
2026-10-11 05:39:40.891101+00 0 \N \N 0 \N f
6a347f55-d34f-42ff-9a73-18f6f0830092 FRM-6VWY2Z Ana Luz
09507704723 Bristol Veloce650 \N 2026 RJD 478 2026-10-01
11:00:00 cancelled \N \N 850.00

659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b f t 2026-09-27 11:43:30.005031+00 2026-09-24 10:50:58.67746+00 2026-09-28 07:46:55.597088+00 360 68dec4a9-bd0f-4148-a0e3-8cab493f1537 \N \N \N \N \N \N 0 \N \N Ana \N Luz \N \N \N \N 0 \N \N 0 \N f
64fe7d0c-3604-4dc4-a072-a3344c02e337 FRM-F98X7B rafa rafa 09926067732 Honda Beat110 \N 2026 ABVE123 2026-09-28 08:00:00 rescheduled \N Rescheduled to FRM-E6Y7KF\nReason for rescheduling: free 400.00 03bb877e-57f8-42dc-9c0e-3fcc82b1a207 t t \N 2026-09-21 07:18:00.682407+00 2026-09-28 08:58:26.921765+00 150 f4c22c64-42c3-41ce-99c1-835840f82f22 \N 147297a6-c092-4337-ab4e-2c86b7848da9 \N \N \N \N 1 \N \N rafa \N rafa \N 2026-09-28 08:58:26.921765+00 \N \N 0 \N \N 0 \N f
b7424607-9e7d-4fd4-ae4e-73dba6c26b21 FRM-CAAFST lawrence l fuerzas 09926067723 Honda Beatf1 \N 2026 ABC123 2026-09-22 08:00:00 completed \N Rescheduled from FRM-3JWTZU\nReason for rescheduling: mao rani free time nako 50.00 03bb877e-57f8-42dc-9c0e-3fcc82b1a207 f t \N 2026-09-18 02:15:34.993525+00 2026-09-24 13:29:34.594168+00 90 aeb31a40-e551-4c2a-877f-8a8f1cbcb722 545503a4-1416-4f5f-83ba-ebc50f5912d7 \N \N \N \N \N 1 \N \N lawrence l fuerzas \N \N \N \N 0 \N \N 0 \N f
34b4dba4-8ed5-42b8-8d27-ca469116c5d1 FRM-XNS9VJ Erik Eri 09242698505 Honda XRM 125 \N 2026 MM45 2026-10-19 08:00:00 cancelled \N \N 100.00 659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b f t 2026-10-14 09:33:07.787489+00 2026-10-14 09:32:01.545076+00 2026-10-14 09:33:07.787489+00 75 5afb2deb-3cee-4990-b310-ea90ad7a54ec \N \N \N \N \N \N 0 \N \N Erik \N Eri \N \N \N \N 0 \N \N 0 eapolona0803@gmail.com t
2a2de84e-e1d7-42e7-a36d-45905d69df12 FRM-NNQNP8 Glendo Navarez 09469962798 Honda XRM 125 \N 2026 DGH234 2026-09-28 13:00:00 completed \N \N 50.00 03bb877e-57f8-42dc-9c0e-3fcc82b1a207 f t \N 2026-09-25 23:05:41.538332+00 2026-10-06 17:07:33.654736+00 90 02c6d69c-d7d5-44f1-8020-bba3b98afc35 \N \N \N \N \N \N 0 \N \N Glendo \N Navarez \N \N 2026-09-28 07:22:36.560525+00 2026-09-28 07:46:08.185155+00 0 \N \N 0 \N f
3eac3b68-b97b-46f0-a8eb-ad913f8bc8c6 FRM-P5ZX4N Ashley L Chiu 09543227314 Honda Click 160 \N 2026 ABC123 2026-10-15 08:30:00 completed \N hello 100.00 659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b f t \N 2026-09-29 06:07:47.742787+00 2026-10-15 06:32:42.283661+00 150 4e8c440e-2c7d-4c4b-a802-0fb04d2f8389 \N \N \N \N \N \N 0 \N

\N Ashley L Chiu \N \N 2026-10-15 01:50:42.635524+00
2026-10-15 06:32:42.283661+00 0 \N \N 0 \N f
e8900d91-8ae4-4bd6-9a33-eb818faa92a2 FRM-XQ2XNC Erikson Apolona
09242698505 Honda XRM 125 \N 2026 FUBU345 2026-10-19
08:00:00 completed \N \N 100.00
5cb00871-f5ef-4ccf-bd61-4746909237e2 f t \N 2026-10-13
11:14:18.29032+00 2026-10-21 10:19:43.487597+00 150
bd5ae3fd-3b68-4417-8dc0-bef30c323c8d \N \N \N \N \N \N 0 \N
\N Erikson \N Apolona \N \N 2026-10-21 10:19:34.79011+00
2026-10-21 10:19:43.487597+00 0 \N \N 0 onboarding@resend.dev t
33fbda0f-318f-41fd-8eac-328e296ea0d3 FRM-5YVTRB Juann Dela Luz
09578759635 Yamaha FZ150 \N 2026 HTY 783 2026-09-25
10:30:00 rescheduled \N Rescheduled to FRM-5E6MYB\nReason for
rescheduling: Busy 450.00 03bb877e-57f8-42dc-9c0e-3fcc82b1a207 f t
\N 2026-09-22 03:18:49.104551+00 2026-10-25 16:30:00.018859+00 330
8de72051-f088-4569-88ab-1c2f633e2190 \N \N \N \N \N \N 1 \N
\N Juann Dela Luz \N \N \N \N 0 \N \N 0 \N f
6014afba-21ae-402c-a687-150730e547ea FRM-MPMHT9 Christian Josi
Apolona 09346930832 Yamaha Mio i125 \N 2026 BBC123
2026-09-30 08:00:00 completed \N \N 150.00
03bb877e-57f8-42dc-9c0e-3fcc82b1a207 f t \N 2026-09-27
05:01:55.179586+00 2026-10-06 17:07:33.654736+00 150
c2a991b8-5a4b-4d0a-8ab7-3dd436870711 \N \N \N \N \N \N 0 \N
\N Christian Josi Apolona \N \N 2026-09-30 00:49:55.532526+00
2026-09-30 04:23:13.520566+00 0 \N \N 0 \N f
a26950f7-62f0-439a-8f3b-07fa6a6c50d6 FRM-ZTUG43 Lara Jessa Apolona
09468033084 Honda Beat110 FI \N 2026 NMO123 2026-10-05
08:00:00 rescheduled \N Rescheduled to FRM-EVXKHD\nReason for
rescheduling: free 50.00 659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b f t \N
2026-09-28 07:16:07.043301+00 2026-09-29 10:45:31.142156+00 90
07ea1e7b-48e6-436b-b4d8-435cb19107f8 \N
30c80b92-c789-40fc-936d-5f6567091423 \N \N \N \N 1 \N \N Lara
Jessa \N Apolona \N \N \N \N 0 \N \N 0 \N f
d070ed76-55d2-40e1-a55a-fc58fc88bfb8 FRM-77GZQE lawrence l fuerzas
09926067723 Honda Beat (110cc) \N 2026 ABC123 2026-09-22
08:00:00 cancelled \N Rescheduled from FRM-4J2DB9\nReason for
rescheduling: free time 50.00 03bb877e-57f8-42dc-9c0e-3fcc82b1a207 f t
2026-09-15 11:41:12.817+00 2026-09-15 09:45:58.46359+00 2026-10-06
17:07:33.654736+00 90 abadd655-a9ea-4682-ad3d-51274fcc0974
71b72a10-644d-4b57-ace3-740377b48bd6 \N \N \N \N \N 1 \N \N
lawrence l fuerzas \N \N \N \N 0 \N \N 0 \N f
\.

--

-- Data for Name: appointment_action_verifications; Type: TABLE DATA; Schema: public; Owner: postgres
--

COPY "public"."appointment_action_verifications" ("id", "appointment_id", "action", "code_hash", "expires_at", "attempt_count", "verified_at", "token_hash", "token_expires_at", "created_at") FROM stdin;
8ecc3d23-6d68-4347-a820-37f8706a8a43 cab87474-af88-46f2-8e66-7acd2e5d2baa cancel b2a4fe7c8ca99956084e03e6a3f8edc6c8d428a8ec8b169b767fbf6a5cd8bb49 2026-10-14 09:40:13.448+00 0 \N \N \N 2026-10-14 09:30:13.256432+00
1b630f55-7d84-4dfa-8c03-d86a241ebf47 34b4dba4-8ed5-42b8-8d27-ca469116c5d1 cancel bde7ed51873f9a126532b82b80a037ee16dd2f07dd6f0dad63fd86ddfe340f6b 2026-10-14 09:42:19.16+00 0 2026-10-14 09:33:06.984962+00
9749d8708a28f06cbbfd9fe5dfd91b0ed0fa8b3e720eb68612b67c9833a279ba 2026-10-14 10:03:06.984962+00 2026-10-14 09:32:18.969703+00
\.

--
-- Data for Name: appointment_completion_notifications; Type: TABLE DATA; Schema: public; Owner: postgres
--

COPY "public"."appointment_completion_notifications" ("id", "appointment_id", "channel", "recipient", "status", "attempt_count", "last_error", "sent_at", "created_at", "updated_at") FROM stdin;
ff588174-87bf-48dc-9308-6f6bb1065c29 30c80b92-c789-40fc-936d-5f6567091423 sms 09468033084 skipped 1 \N \N 2026-10-11 05:39:40.891101+00 2026-10-11 05:39:41.442+00
5717fd9b-317b-4247-838f-2dc7d82fafa2 eda6b8a7-6dc1-4584-8ad5-e918438c0a05 sms 09094291771 skipped 1 \N \N 2026-10-14 08:15:10.905783+00 2026-10-14 08:15:11.549+00
0dd02acb-2943-418c-8bcb-030973046459 3eac3b68-b97b-46f0-a8eb-ad913f8bc8c6 sms 09543227314 skipped 1 \N \N 2026-10-15 06:32:42.283661+00 2026-10-15 06:32:43.084+00
75bf6b4e-99d9-4c10-a400-26387aa5eb92 e8900d91-8ae4-4bd6-9a33-eb818faa92a2 sms 09242698505 sent 1 \N 2026-10-21 10:19:45.854+00 2026-10-21 10:19:43.487597+00 2026-10-21 10:19:45.854+00
96839669-0522-4676-816b-2e9860f13ebc e8900d91-8ae4-4bd6-9a33-eb818faa92a2 email onboarding@resend.dev sent 1 \N 2026-10-21 10:19:49.656+00 2026-10-21 10:19:43.487597+00

2026-10-21 10:19:49.656+00
ef8c2f6b-7060-480d-8223-6016ca17ee81
6c0de99e-7522-4f4e-ad11-0913682e3ae3 sms 09242698505 sent 1 \N
2026-10-21 10:19:55.642+00 2026-10-21 10:19:53.441669+00 2026-10-21
10:19:55.642+00
e64f8453-9a13-48df-b056-00a85defd1a7
6c0de99e-7522-4f4e-ad11-0913682e3ae3 email
erikson.apolona@bisu.edu.ph sent 1 \N 2026-10-21 10:19:56.727+00
2026-10-21 10:19:53.441669+00 2026-10-21 10:19:56.727+00
\.

--
-- Data for Name: appointment_confirmation_emails; Type: TABLE DATA;
Schema: public; Owner: postgres
--

COPY "public"."appointment_confirmation_emails" ("id", "appointment_id",
"recipient_email", "status", "attempt_count", "last_error", "sent_at", "created_at",
"updated_at") FROM stdin;
bc927fcf-af3a-4c48-824b-2d32933c4dc0
af37178b-4dfd-455f-8d90-2f7d3f4b4304 erikson.apolona@bisu.edu.ph sent
1 \N 2026-10-11 05:39:39.371+00 2026-10-09 08:57:48.811002+00
2026-10-11 05:39:39.371+00
faede63e-59e7-4a4a-8d9e-7a32542ec72d
dc1c4a1c-1532-4d25-bbd5-b7d8793de6e9 erikson.apolona@bisu.edu.ph
sent 1 \N 2026-10-11 05:39:39.732+00 2026-10-09 08:55:24.539046+00
2026-10-11 05:39:39.732+00
54c275d1-29a5-45a9-be8a-1b82647f6681
cab87474-af88-46f2-8e66-7acd2e5d2baa onboarding@resend.dev sent 1
\N 2026-10-11 05:39:49.422+00 2026-10-11 05:39:48.211457+00
2026-10-11 05:39:49.422+00
0846bb2a-720f-42e7-a392-f86451e9e19d
e8900d91-8ae4-4bd6-9a33-eb818faa92a2 onboarding@resend.dev sent 1
\N 2026-10-13 12:25:44.866+00 2026-10-13 12:25:40.460397+00
2026-10-13 12:25:44.866+00
108230a0-2b1e-405c-bba5-576c3815a85c
cfe5e95c-d7d8-4ba5-b6e7-9020b664a0fc eapolona0803@gmail.com sent
1 \N 2026-10-13 12:25:50.526+00 2026-10-13 12:25:46.398361+00
2026-10-13 12:25:50.526+00
f5162377-b284-4bd5-957d-2f87594686bd
6c0de99e-7522-4f4e-ad11-0913682e3ae3 erikson.apolona@bisu.edu.ph
sent 1 \N 2026-10-15 07:09:36.576+00 2026-10-15 07:09:32.354672+00
2026-10-15 07:09:36.576+00
\.

--
-- Data for Name: appointment_confirmation_sms; Type: TABLE DATA; Schema: public; Owner: postgres
--

COPY "public"."appointment_confirmation_sms" ("id", "appointment_id", "recipient_phone", "status", "attempt_count", "last_error", "sent_at", "created_at", "updated_at") FROM stdin;
66c7cb0c-be45-44ba-9e71-29220392bec0 dc1c4a1c-1532-4d25-bbd5-b7d8793de6e9 09242698505 sent 1 \N 2026-10-09 08:55:26.763+00 2026-10-09 08:55:24.539046+00 2026-10-09 08:55:26.763+00
063fc54c-7af6-4dd5-98aa-65779e3ec8fb af37178b-4dfd-455f-8d90-2f7d3f4b4304 09242698505 sent 1 \N 2026-10-09 08:57:50.739+00 2026-10-09 08:57:48.811002+00 2026-10-09 08:57:50.739+00
e00093a3-0c35-404e-bcf6-401b62adcc33 cab87474-af88-46f2-8e66-7acd2e5d2baa 09242698505 sent 1 \N 2026-10-11 05:39:50.632+00 2026-10-11 05:39:48.211457+00 2026-10-11 05:39:50.632+00
847d557b-702d-45ed-93ca-ab20cc1c0839 e8900d91-8ae4-4bd6-9a33-eb818faa92a2 09242698505 sent 1 \N 2026-10-13 12:25:42.805+00 2026-10-13 12:25:40.460397+00 2026-10-13 12:25:42.805+00
01733c4e-f412-46c7-a664-8f43b7826c5e cfe5e95c-d7d8-4ba5-b6e7-9020b664a0fc 09242698505 sent 1 \N 2026-10-13 12:25:48.571+00 2026-10-13 12:25:46.398361+00 2026-10-13 12:25:48.571+00
3c6c96fb-130c-46e8-a8e7-66d33a0ea9cb 6c0de99e-7522-4f4e-ad11-0913682e3ae3 09242698505 sent 1 \N 2026-10-15 07:09:35.05+00 2026-10-15 07:09:32.354672+00 2026-10-15 07:09:35.05+00
\.

--
-- Data for Name: appointment_continuations; Type: TABLE DATA; Schema: public; Owner: postgres
--

COPY "public"."appointment_continuations" ("id", "appointment_id", "segment_number", "appointment_date", "start_time", "booking_duration_minutes", "assigned_crew_id", "created_at") FROM stdin;

dd813ba9-a9f3-4ff0-a5ca-7605f9ed6360
2334d5bb-94f4-4889-8a06-0caee9004790 1 2026-09-29 08:00:00 540
03bb877e-57f8-42dc-9c0e-3fcc82b1a207 2026-09-22 22:54:19.20688+00
27588eae-c64f-4f88-99d4-ad650d9d8e68
2334d5bb-94f4-4889-8a06-0caee9004790 2 2026-09-30 08:00:00 510
03bb877e-57f8-42dc-9c0e-3fcc82b1a207 2026-09-22 22:54:19.20688+00
77d96914-0e65-446f-9567-8b1c28a335df
6a347f55-d34f-42ff-9a73-18f6f0830092 1 2026-10-02 08:00:00 60
f3c5770f-e485-4efc-83f6-892422d8592e 2026-09-24 10:50:58.67746+00
\.

--
-- Data for Name: products; Type: TABLE DATA; Schema: public; Owner: postgres
--

COPY "public"."products" ("id", "name", "category", "brand", "description", "price", "image_url", "in_stock", "is_featured", "is_active", "sort_order", "created_at", "updated_at", "is_archived", "stock_quantity", "archived_at") FROM stdin;
9b221a1e-640e-49d4-8635-3a96d08dee27 Spark Plug part NGK The go-to spark plug brand nationwide. 120.00 https://egqukkwjuagvgyuqdart.supabase.co/storage/v1/object/public/products/3f36ee10-879e-4e02-bff5-f1418ebed70b.jpg t f t 0 2026-08-31 10:33:44.063967+00 2026-09-13 09:48:02.214988+00 f 10 \N
8247c0ee-a2c2-4e42-81e0-99e7ed002c5d Motorcycle Tail Bag accessory Komine Waterproof 20L expandable tail bag. 3400.00 https://egqukkwjuagvgyuqdart.supabase.co/storage/v1/object/public/products/c157d19e-92e7-4e5d-b685-70f4227adfd5.jpg t f t 8 2026-08-19 05:11:46.27827+00 2026-09-29 10:50:02.602372+00 f 7 \N
656db2e0-7cb6-4311-8271-3e15430d31b3 Ohlins Rear Shock Absorber part Ohlins Premium gas-charged rear shock with adjustable preload. 3500.00 https://egqukkwjuagvgyuqdart.supabase.co/storage/v1/object/public/products/9493159e-79f8-4d4d-b8ca-8db99e6a56e0.jpg t f t 1 2026-08-19 05:11:46.27827+00 2026-09-29 10:50:39.532585+00 f 3 \N
9310ef87-578e-4fb2-b95b-725a0900c95a Brake pads part HybridDynamic Widely bought local budget brand for daily-use motorcycles and scooters. 250.00 https://egqukkwjuagvgyuqdart.supabase.co/storage/v1/object/public/products/543bc128-6267-4b3c-878f-d664745bc78d.jpg t f t 0 2026-08-31 10:39:44.04718+00 2026-09-29 10:46:52.521733+00 f 10 \N
7403d2b3-6533-4e2a-aa1a-49d0f8babf93 Handlebar Grips part JRP Upgrade your grip, upgrade your ride — comfort and style in one affordable swap. 200.00 https://egqukkwjuagvgyuqdart.supabase.co/storage/v1/object/public/products/c64283bc-8672-463b-a35c-ec74ef226bc5.jpg t f t 0 2026-08-31 10:46:52.10923+00 2026-09-29 10:47:11.888182+00 f 9 \N

d63ff680-20d7-4a54-92e0-1790fb39446c	Performance Exhaust Pipe	part	Akrapovic	Slip-on stainless exhaust with deep race tone.	900.00	https://egqukkwjuagvgyuqdart.supabase.co/storage/v1/object/public/products/beca01ef-cae3-4448-9fed-f1a38b633c43.jpg	t	t	t	3	2026-08-19 05:11:46.27827+00	2026-09-29 10:52:29.618243+00	f	5	\N
1d42b93e-83ef-46f2-b6dc-7486442bfcca	Racing Clutch Assembly	part	Yoshimura	Heavy duty clutch kit for aggressive riding.	4800.00	https://egqukkwjuagvgyuqdart.supabase.co/storage/v1/object/public/products/098f0c69-2303-45d1-a47a-4b1a9b0853ff.jpg	t	t	t	2	2026-08-19 05:11:46.27827+00	2026-09-29 10:49:15.680238+00	f	11	\N
281caa1b-f3ac-48c4-a48b-fb02cde092fd	CNC Adjustable Levers	accessory	Rizoma	Anodized aluminum brake and clutch levers.	1850.00	https://egqukkwjuagvgyuqdart.supabase.co/storage/v1/object/public/products/29c9a229-72a5-4b6f-aad3-292a509092be.jpg	t	t	t	5	2026-08-19 05:11:46.27827+00	2026-09-29 10:49:43.720467+00	f	10	\N
888affb7-4276-41dd-9d6c-5b367e059482	Motocross Riding Gloves	accessory	Fox	Breathable gloves with silicone grip print.	1250.00	https://egqukkwjuagvgyuqdart.supabase.co/storage/v1/object/public/products/8d39463b-796b-4bf4-9c69-e14e876c32da.jpg	t	f	t	7	2026-08-19 05:11:46.27827+00	2026-09-29 10:49:53.655452+00	f	5	\N
c973b83b-ee14-4254-a320-420f75ffb39a	Braided Brake Hose	part	Galespeed	Stainless braided hose for sharper brake feel.	200.00	https://egqukkwjuagvgyuqdart.supabase.co/storage/v1/object/public/products/602b7207-2721-4fa5-9998-abd6cbfcf5a5.jpg	t	f	t	4	2026-08-19 05:11:46.27827+00	2026-09-29 10:54:24.430402+00	f	13	\N
4d351a12-bead-4422-8f85-d6070c853ca8	Wheel Bearing	part	NACHI	Affordable, reliable bearing brand commonly used for wheel and swingarm replacements.	100.00	https://egqukkwjuagvgyuqdart.supabase.co/storage/v1/object/public/products/258b4837-fa41-4a07-8153-676a306c19ed.png	t	f	t	0	2026-08-31 10:42:46.797463+00	2026-10-11 05:58:05.259017+00	f	19	\N
\.

--
-- Data for Name: appointment_products; Type: TABLE DATA; Schema: public; Owner: postgres
--

COPY "public"."appointment_products" ("id", "appointment_id", "product_id", "product_name", "quantity", "unit_price", "created_at", "updated_at") FROM stdin;
2a4f9609-b475-497b-92a3-41f1165be293	cab87474-af88-46f2-8e66-7acd2e5d2baa	4d351a12-bead-4422-8f85-d6070c853ca8	Wheel Bearing	1	200.00	2026-10-11 05:58:05.259017+00	2026-10-11 05:58:05.259017+00

\.

--
-- Data for Name: appointment_reference_recovery_challenges; Type: TABLE DATA; Schema: public; Owner: postgres
--

COPY "public"."appointment_reference_recovery_challenges" ("id", "email", "code_hash", "expires_at", "attempt_count", "verified_at", "created_at") FROM stdin;
45c7632b-0395-4cdc-a55d-33ebb22c2765	onboarding@resend.dev	c9308f34119804ae60ab1ba19d8181f17e238be86702c12894baa06b8f9417fd	2026-10-13 11:25:26.675+00	0	\N	2026-10-13 11:15:26.629517+00
5cb4eaa3-44df-41ac-a5d5-bbf02c055cba	onboarding@resend.dev	bc8b4a9efab3a9a14d9c06ce44b2de27747cda54406756635211aa98f5692251	2026-10-13 11:29:14.509+00	0	\N	2026-10-13 11:19:14.634762+00
77d6ae36-4b77-47b7-87d0-ab01c85c3de1	eapolona0803@gmail.com	5ce3a6ef19501bf4f5ecfe54a15840a3e8ac0552e665dc40b4fe77f5338002a7	2026-10-13 12:23:32.74+00	0	\N	2026-10-13 12:13:32.680183+00
ca5a45d5-0eca-4c15-8554-ba5fb8ed64f8	eapolona0803@gmail.com	75f053752fda2ca75c649e55831e9ff771a9144ed8856af9d106bb7bb9d35f26	2026-10-13 12:34:30.937+00	0	2026-10-13 12:24:59.642302+00	2026-10-13 12:24:30.769429+00
00030cf1-ba3c-41b7-ba42-c220a8e08ae2	eapolona0803@gmail.com	2b1366f40cbcb679132c5c24916a0cd8ef0eaeb8fdad5db02656f860466db1e9	2026-10-13 12:46:18.689+00	0	2026-10-13 12:37:58.537274+00	2026-10-13 12:36:18.511351+00
\.

--
-- Data for Name: appointment_reschedule_history; Type: TABLE DATA; Schema: public; Owner: postgres
--

COPY "public"."appointment_reschedule_history" ("id", "appointment_id", "new_appointment_id", "reschedule_number", "from_date", "from_start_time", "to_date", "to_start_time", "reason", "decision", "approved_at", "approved_by") FROM stdin;
f25ef95e-f057-48e7-87ae-4aeb0eed7142	545503a4-1416-4f5f-83ba-ebc50f5912d7	b7424607-9e7d-4fd4-ae4e-73dba6c26b21	1	2026-09-21	08:00:00	2026-09-22	08:00:00	mao rani free time nako	confirmed	2026-09-18 02:15:34.993525+00	\N

2183b17a-e65f-4874-b2ef-4cd582fa9191
bb9e341a-1d2f-433b-ae2a-119cba575564 \N 1 2026-09-28 08:00:00
2026-09-29 08:00:00 free time rejected 2026-09-22 02:31:38.233765+00
\N
27da98fa-6ac9-48d6-b2f7-db2aa66f2544
64fe7d0c-3604-4dc4-a072-a3344c02e337
147297a6-c092-4337-ab4e-2c86b7848da9 1 2026-09-28 08:00:00
2026-09-25 08:00:00 free confirmed 2026-09-22 15:11:44.983303+00
\N
b80ad9cd-37f4-4e03-ade8-e6387798a6e7
a26950f7-62f0-439a-8f3b-07fa6a6c50d6
30c80b92-c789-40fc-936d-5f6567091423 1 2026-10-05 08:00:00
2026-10-09 08:00:00 free confirmed 2026-09-28 11:38:09.628656+00
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74
c24f7c9a-bf1a-442f-998c-ad4cf247f3a2
2ce47e77-9cfc-49b8-9710-c3b33294eb54
de1e21dc-53e4-4362-8898-3bd1da2d67fc 1 2026-10-12 13:00:00
2026-10-05 08:00:00 free confirmed 2026-09-30 05:01:21.761562+00
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74
67431951-cd68-41a0-92af-1d2f6d4e8dbf
33fbda0f-318f-41fd-8eac-328e296ea0d3 \N 1 2026-09-25 10:30:00
2026-09-24 11:30:00 Busy confirmed 2026-09-22 15:11:47.346985+00
\N
\.

--
-- Data for Name: services; Type: TABLE DATA; Schema: public; Owner:
postgres
--

COPY "public"."services" ("id", "name", "description", "price", "duration_minutes",
"category", "image_url", "is_active", "sort_order", "created_at", "updated_at",
"is_archived", "archived_at", "large_bike_price", "large_bike_duration_minutes")
FROM stdin;
97a9714b-20e6-4402-a3ee-4fcdefc1e9db Change Oil Drains old engine oil
and refills the engine with the correct oil to reduce friction and protect moving
parts. 50.00 30 maintenance \N t 10 2026-08-19
05:11:46.27827+00 2026-10-14 06:46:15.238796+00 f \N 100.00 60
f8feeecc-86e2-4312-8798-ea5c144f1ad4 Engine Overhaul Top or
bottom-end overhaul with parts assessment. 1500.00 480 engine \N t
340 2026-08-19 05:11:46.27827+00 2026-10-14 06:47:26.326545+00 f
\N 1800.00 480
e5cf562e-7122-440b-9483-033b78aca286 Carburetor / FI Cleaning Deep
clean and tuning for smoother throttle response. 300.00 90 engine \N f

8 2026-08-19 05:11:46.27827+00 2026-09-27 01:26:36.638306+00 t 2026-09-27 01:26:36.638306+00 \N \N
d9ecf096-df3b-47f5-a012-80d98aacf9f5 Motorcycle Tune-Up Checks basic engine and performance items for rough running, difficult starting, reduced power, or rideability concerns. 150.00 150 maintenance \N t 20 2026-09-19 23:50:07.474101+00 2026-10-14 08:13:19.760679+00 f \N 200.00 150
e5742f78-e0f3-4c14-b4e5-9ad22b3ff85b Brake Service Inspects braking concerns and carries out routine brake work where practical; additional confirmed work may require more time or cost. 50.00 90 brakes \N t 30 2026-08-19 05:11:46.27827+00 2026-10-14 08:13:43.673664+00 f \N 100.00 90
e04fec1b-0083-46cf-95d4-5765ba807518 Motorcycle Safety Inspection Basic inspection of brakes, tires, lights, chain, controls, and other important safety-related motorcycle components. 200.00 45 inspection \N f 0 2026-09-09 16:37:45.999994+00 2026-09-27 01:26:36.638306+00 t 2026-09-27 01:26:36.638306+00 \N \N
ab3c3ba9-415a-4f0e-9a42-bffb7d1903c1 Electrical Diagnostics Wiring, charging system and lighting troubleshooting. 350.00 60 electrical \N f 7 2026-08-19 05:11:46.27827+00 2026-09-27 01:26:36.638306+00 t 2026-09-27 01:26:36.638306+00 \N \N
ed2e91b9-1d71-4e29-814d-7b7507751569 Preventive Maintenance Service (PMS) Full check-up: oil change, brake, chain, and fluid inspection. 350.00 90 maintenance \N f 1 2026-08-19 05:11:46.27827+00 2026-09-27 01:26:36.638306+00 t 2026-09-27 01:26:36.638306+00 \N \N
08eef9ae-0750-4ed4-ade6-35aed340eea5 Tire Change & Balancing Tire mounting, balancing and valve replacement. 250.00 60 tires \N f 4 2026-08-19 05:11:46.27827+00 2026-09-27 01:26:36.638306+00 t 2026-09-27 01:26:36.638306+00 \N \N
72a4c5f5-e5f1-4e71-b872-30a3615ac362 Suspension Tuning / Rebuild Ohlins-grade fork and shock service, oil change and re-valving. 2500.00 180 suspension \N f 3 2026-08-19 05:11:46.27827+00 2026-09-27 01:26:36.638306+00 t 2026-09-27 01:26:36.638306+00 \N \N
6a7f36bb-0ec8-47f4-bf32-54c09de9b4b0 Motorcylce Tune-Up General engine and motorcycle inspection with adjustment of idle, throttle response, chain condition, and basic operating components. 300.00 60 maintenance \N f 0 2026-09-09 16:29:30.294761+00 2026-09-27 01:26:36.638306+00 t 2026-09-27 01:26:36.638306+00 \N \N
1477338f-ec6e-4bf6-b916-b619cc58ac0d Chain & Sprocket Replacement Replacement of worn chain and sprocket components followed by proper alignment and tension adjustment. 500.00 90 drivetrain \N f 0 2026-09-09 16:30:38.684952+00 2026-09-27 01:26:36.638306+00 t 2026-09-27 01:26:36.638306+00 \N \N
61b18a4e-6804-4701-b035-b8e7913f7911 Cooling System Service Inspection, cleaning, and basic servicing of the motorcycle cooling system to

maintain proper engine temperature. 400.00 90 engine \N f 0 2026-09-09 16:32:42.080501+00 2026-09-27 01:26:36.638306+00 t 2026-09-27 01:26:36.638306+00 \N \N
e724f832-2a1f-4949-bf6f-ad72240d99ea Fork Seal Replacement Replacement of worn or leaking front fork seals followed by inspection and adjustment of the suspension system. 800.00 120 suspension \N f 0 2026-09-09 16:35:39.317517+00 2026-09-27 01:26:36.638306+00 t 2026-09-27 01:26:36.638306+00 \N \N
f0132c1b-d630-4aa3-a56b-165ce0c408ec Clutch Adjustment/Repair Adjust clutch cable/lever free play or replace worn clutch plates and springs to restore proper engagement and prevent slipping 50.00 60 engine \N f 0 2026-08-31 10:26:49.083565+00 2026-09-27 01:26:36.638306+00 t 2026-09-27 01:26:36.638306+00 \N \N
935c4315-72a4-40a4-9a63-6487106bd0fb Clutch Adjustment Adjusts clutch control and checks operation for difficult shifting, incorrect lever feel, or poor engagement. 50.00 75 drivetrain \N t 40 2026-09-27 01:26:36.638306+00 2026-10-14 08:14:00.509473+00 f \N 100.00 75
a383dd9d-9dbb-4974-b9bf-e8d08ad1d0c5 Full CVT Service Inspects and cleans serviceable automatic-CVT components, checks their condition, and reports worn parts billed separately. 500.00 210 drivetrain \N t 50 2026-09-27 01:26:36.638306+00 2026-10-14 08:14:22.711896+00 f \N 600.00 210
3066e928-4557-488a-831f-f7e0e26ae11f Chain Cleaning Cleans dirt, dust, and sticky buildup from the drive chain and checks its general condition. 50.00 75 drivetrain \N t 60 2026-09-27 01:26:36.638306+00 2026-10-14 08:14:36.998625+00 f \N 100.00 75
baaf0e56-887d-41c4-b78a-355e5509bc9c Chain Lubrication Applies suitable lubricant to a dry, squeaky, or noisy drive chain to support smooth operation and reduce wear. 50.00 70 drivetrain \N t 70 2026-09-27 01:26:36.638306+00 2026-10-14 08:15:05.987232+00 f \N 100.00 70
aad80fe2-5bcc-4a22-864b-8e6fc00531bd Chain Adjustment Adjusts drive-chain slack and checks rear-wheel alignment for a chain that is too loose, too tight, or noisy. 50.00 75 drivetrain \N t 80 2026-09-27 01:26:36.638306+00 2026-10-14 08:15:56.765738+00 f \N 100.00 75
e852fde6-f231-4b05-95b4-26e8d6455c97 Chain Replacement Installs a compatible replacement chain, adjusts tension, and checks operation; the chain itself may be billed separately. 100.00 90 drivetrain \N t 90 2026-09-27 01:26:36.638306+00 2026-10-14 08:16:08.69203+00 f \N 200.00 90
f9cf4825-20c1-409b-8842-8411540ad30b Sprocket Replacement Replaces a worn or damaged sprocket and checks the drive-chain fit. 100.00 90 drivetrain \N t 100 2026-09-27 01:26:36.638306+00 2026-10-14 08:16:21.267017+00 f \N 200.00 90
4ee919a5-884a-4cf4-80af-e6cdfbb97778 Chain and Sprocket Replacement Replaces worn or damaged chain and sprockets as a set, adjusts tension, and

checks alignment; parts may be billed separately. 150.00 120 drivetrain \N t 110 2026-09-27 01:26:36.638306+00 2026-10-14 08:16:33.9627+00 f \N 200.00 120
9bac85e5-f254-43ee-a36d-384b25fcae83 Tire Change Removes a tire and fits a compatible replacement; tires, tubes, and other parts may be billed separately. 70.00 90 tires \N t 120 2026-09-27 01:26:36.638306+00 2026-10-14 08:16:48.133378+00 f \N 100.00 90
7a43f3c4-a01f-476c-8f16-5e6a74fb6833 Tire Balancing Checks and balances the wheel to address vibration caused by uneven weight, where suitable equipment is available. 50.00 75 tires \N t 130 2026-09-27 01:26:36.638306+00 2026-10-14 08:17:15.178074+00 f \N 100.00 75
f28be846-7db7-422a-b960-1d36cce18e7b Wheel Alignment Checks wheel alignment and makes supported adjustments when the motorcycle does not track straight. 50.00 75 tires \N t 140 2026-09-27 01:26:36.638306+00 2026-10-14 08:17:28.344788+00 f \N 100.00 75
9381d3bc-dadb-4e65-9b83-97539b9df723 Wheel Bearing Replacement Checks wheel operation and replaces confirmed worn or damaged bearings. 50.00 120 tires \N t 150 2026-09-27 01:26:36.638306+00 2026-10-14 08:17:39.307356+00 f \N 100.00 120
06900445-d8cc-4c22-822c-5b23b4d25b4e Front Fork Service Inspects and services front forks for leaks, noise, poor damping, or incorrect feel, including seal replacement where appropriate. 150.00 150 suspension \N t 160 2026-09-27 01:26:36.638306+00 2026-10-14 08:17:53.894265+00 f \N 200.00 150
35ba5528-3e82-4df2-9d12-147c813f9295 Rear Shock Service Checks and services the rear shock where its design supports servicing; replacement may be required for some units. 100.00 120 suspension \N t 170 2026-09-27 01:26:36.638306+00 2026-10-14 08:18:09.50947+00 f \N 250.00 120
42490782-6cf8-4db4-86e6-757d96ec9dec Suspension Tuning Checks available suspension settings and adjusts them where the motorcycle supports adjustment. 200.00 90 suspension \N t 180 2026-09-27 01:26:36.638306+00 2026-10-14 08:18:29.997157+00 f \N 300.00 90
951731dd-8412-46b9-8058-7348ea20b9ef Suspension Rebuild Disassembles, checks, and rebuilds a serviceable worn or leaking suspension unit; additional parts or work may be needed. 150.00 210 suspension \N t 190 2026-09-27 01:26:36.638306+00 2026-10-14 08:18:44.637822+00 f \N 200.00 210
f01fd9c7-6a0f-4510-bf61-fd7469d1290a Steering Bearing Replacement Checks steering bearings and replaces worn bearings that cause stiffness, looseness, catching, or knocking. 250.00 210 suspension \N t 200 2026-09-09 16:36:52.661885+00 2026-10-14 08:19:08.206181+00 f \N 300.00 210
8c384099-4f80-4b61-ab6d-ffea363a6038 Battery Testing Tests battery charge retention and identifies whether further charging-system checks are

needed. 20.00 60 electrical \N t 210 2026-09-27 01:26:36.638306+00 2026-10-14 08:19:30.210295+00 f \N 29.97 60
51d71716-aabb-4e18-80f8-79c8d20d3ed1 Battery Replacement Installs a compatible replacement battery after confirming a failed or unreliable battery; the battery cost may be separate. 50.00 75 electrical \N t 220 2026-09-27 01:26:36.638306+00 2026-10-14 08:19:47.071494+00 f \N 70.00 75
b950fc34-4de4-42b6-b3ff-5c0b352103db Electrical Diagnostics FI Checks relevant electrical connections and components for fuel-injected motorcycles with warning lights, starting trouble, or intermittent functions. 150.00 120 electrical \N t 230 2026-09-27 01:26:36.638306+00 2026-10-14 08:20:01.474329+00 f \N 300.00 120
cb46dda3-e494-47d3-8f41-ce98287ff9b5 Electrical Wiring Repair Checks related wiring and connections and repairs accessible electrical faults; difficult faults may take longer. 200.00 150 electrical \N t 240 2026-09-27 01:26:36.638306+00 2026-10-14 08:20:15.008185+00 f \N 300.00 150
5160c7c2-4cf7-481a-8881-cdfea2519895 ECU Diagnostic Scan Scans compatible motorcycles for stored error codes and explains what the results may indicate. 250.00 90 electrical \N t 250 2026-09-27 01:26:36.638306+00 2026-10-14 08:20:35.33525+00 f \N 300.00 90
437617a0-ee24-4289-9209-aeaafbaf192c Spark Plug Replacement Checks and replaces a compatible spark plug for hard starting, uneven running, or misfiring. 50.00 75 engine \N t 260 2026-09-27 01:26:36.638306+00 2026-10-14 08:21:00.037601+00 f \N 100.00 75
7dbb4616-2f79-4102-914c-415c0176c05f Air Filter Replacement Accesses the filter housing and installs a suitable replacement filter when needed. 50.00 75 engine \N t 270 2026-09-27 01:26:36.638306+00 2026-10-14 08:21:21.524449+00 f \N 100.00 75
0677835f-d6c7-4151-b33b-9fd0e6b2f14d Carburetor Cleaning Cleans serviceable carburetor passages and components for hard starting, stalling, uneven idle, or poor running. 100.00 120 engine \N t 280 2026-09-27 01:26:36.638306+00 2026-10-14 08:21:34.748564+00 f \N 250.00 120
26b9d1dc-3f41-4466-91af-c53b2c230c6b Carburetor Tuning Checks and adjusts carburetor settings for rough idle or poor running; worn parts may require separate work. 100.00 90 engine \N t 290 2026-09-27 01:26:36.638306+00 2026-10-14 08:21:48.329003+00 f \N 250.00 90
3d1f19de-deeb-4b05-b7ae-fc6dc36d14b7 Throttle Body Cleaning Cleans the throttle body and accessible air passages on fuel-injected motorcycles; injector cleaning is separate work. 250.00 120 engine \N t 300 2026-09-27 01:26:36.638306+00 2026-10-14 08:22:09.879837+00 f \N 400.00 120
44651d94-93a0-4b63-b2ea-7dd378a57bb0 Fuel System Cleaning Checks and cleans fuel-delivery components included in the shop procedure; injector removal may be separate work. 100.00 120 engine \N t 310 2026-09-27 01:26:36.638306+00 2026-10-14 08:22:22.147666+00 f \N

200.00 120
b6af4e06-1d15-4f61-b7e8-77719cb355de Radiator Cleaning Carefully cleans external radiator fins to remove dirt, leaves, or mud that can restrict airflow. 100.00 90 engine \N t 320 2026-09-09 16:33:27.013406+00 2026-10-14 08:22:34.870151+00 f \N 200.00 90
625eeaa2-aeb5-443d-8f4e-c612ffcd13b4 Engine Diagnostic Performs appropriate checks and tests to identify the cause of engine noise, power loss, rough running, smoke, or similar concerns. 150.00 120 engine \N t 330 2026-09-27 01:26:36.638306+00 2026-10-14 08:22:47.490647+00 f \N 250.00 120
fff41982-2457-426a-b56a-d0fcb3c1be36 Accessory/Upgrade Parts Installation General Fits and secures other customer-supplied accessories or upgrades, such as mirrors, guards, grips, or lighting kits. 0.00 120 upgrades \N t 550 2026-09-27 01:26:36.638306+00 2026-09-27 01:26:36.638306+00 f \N \N \N
7f2b49e7-8c12-44d3-9d1f-3cb79c8898ec Motorcycle Inspection Checks the motorcycle’s key systems and visible condition to identify potential issues and determine which services may be needed. The inspection covers areas such as the brakes, tires, lights, chain, fluid levels, and visible leaks, followed by recommended services or next steps based on the findings. 0.00 120 inspection \N t 350 2026-09-27 01:26:36.638306+00 2026-09-29 22:13:06.556512+00 f \N \N \N
0606e54a-cbfc-4043-b88e-1d1f2d273249 Headlight/Lighting Replacement Checks the bulb and basic connection, then replaces a faulty compatible lighting component when needed; parts may be separate. 250.00 75 electrical \N t 360 2026-09-27 01:26:36.638306+00 2026-10-14 08:23:12.588502+00 f \N 350.00 75
4e82dabf-aa10-4846-ae25-1c208f9b68bc Starter System Diagnosis & Repair Checks battery connections, starter relay, wiring, and starter motor for clicking, no-start, or intermittent-starting concerns. 150.00 120 electrical \N t 370 2026-09-27 01:26:36.638306+00 2026-10-14 08:23:26.099539+00 f \N 250.00 120
a7b50b13-0e62-4413-ae40-4ecf75b2e54a Charging System Repair Checks and repairs a faulty charging-system connection or serviceable component; parts may be separate. 100.00 150 electrical \N t 380 2026-09-27 01:26:36.638306+00 2026-10-14 08:23:49.469295+00 f \N 200.00 150
ec2b2a81-b3e9-404e-8e1d-d161f25b8a43 Charging System Testing Tests the battery and charging output to determine whether the motorcycle is charging correctly. 100.00 75 electrical \N t 390 2026-09-27 01:26:36.638306+00 2026-10-14 08:24:21.963967+00 t 2026-10-14 08:24:21.963967+00 \N \N
cac8ad67-f7d0-438d-baf4-c223d3abd732 Horn Replacement Checks the horn and its connection, then replaces a faulty compatible horn when needed; the replacement part may be separate. 100.00 75 electrical \N t 400 2026-09-27 01:26:36.638306+00 2026-10-14 08:24:35.10144+00 f \N

200.00 75
9021a319-7ed4-488f-94fe-5a10356259b9 Fuel Pump Inspection/Repair Checks the fuel pump and related connections for hard starting, stalling, or suspected fuel-delivery faults. 100.00 120 engine \N t 410 2026-09-27 01:26:36.638306+00 2026-10-14 08:25:14.080641+00 f \N 250.00 120
9ab95115-47ef-4011-846a-4af834e115da Water Pump Inspection/Repair Inspects the water pump and related parts for coolant leaks or overheating and recommends repair or replacement as needed. 200.00 150 engine \N t 420 2026-09-27 01:26:36.638306+00 2026-10-14 08:25:30.90425+00 f \N 300.00 150
d621e288-2d04-4ab2-a649-2c22c9256fce Transmission Repair Diagnoses difficult shifting, gear slipping, and abnormal drivetrain noises; major internal repairs can take longer and cost more. 250.00 270 drivetrain \N t 430 2026-09-27 01:26:36.638306+00 2026-10-14 08:25:46.68475+00 f \N 350.00 270
8b41b72f-0a7d-45b9-8053-82ecc20491c1 Brake Shoe Replacement Removes the wheel, replaces worn drum-brake shoes, and adjusts the drum for proper stopping performance. 70.00 90 brakes \N t 440 2026-09-27 01:26:36.638306+00 2026-10-14 08:26:06.084696+00 f \N 130.00 90
9fa4029c-73e5-4fff-93fc-6f26d8d40785 Brake Pad Replacement Replaces worn disc-brake pads and checks disc condition before reassembly. 50.00 75 brakes \N t 450 2026-09-27 01:26:36.638306+00 2026-10-14 08:26:18.018453+00 f \N 100.00 75
824a246b-3956-4983-b344-39c3d18001e4 Magneto Cleaning Cleans magneto ignition contacts and components and checks ignition output for hard starting, weak spark, or rough idle. 50.00 60 engine \N t 460 2026-09-27 01:26:36.638306+00 2026-10-14 08:26:29.969134+00 f \N 100.00 60
289d392a-52a6-4e13-995f-2303faf48fe2 Throttle Cable Replacement/Adjustment Adjusts or replaces a stiff, sticky, loose, or slow-returning throttle cable and checks smooth operation. 50.00 60 engine \N t 500 2026-09-27 01:26:36.638306+00 2026-10-14 08:28:37.773158+00 f \N 100.00 60
a2e6ef24-2f88-4f18-92b2-180be413a5a7 Crankshaft Bearing Replacement Opens the crankcase, inspects the crankshaft assembly, and replaces worn main bearings; additional internal parts may be needed. 700.00 300 engine \N t 470 2026-09-27 01:26:36.638306+00 2026-10-14 08:27:40.16708+00 f \N 1000.00 300
624b8f10-7059-43d0-8557-77b03600cd0f CVT Roller / Fly Weight Replacement Opens the CVT cover, replaces worn rollers or fly weights, and checks belt condition. 150.00 105 drivetrain \N t 480 2026-09-27 01:26:36.638306+00 2026-10-14 08:28:11.358318+00 f \N 250.00 105
c18718c3-3aa9-40f6-8907-3a7922a07922 Handlebar Switch Replacement Checks switch wiring and replaces a faulty handlebar switch assembly for

affected horn, signal, lighting, or starter controls. 100.00 75 electrical \N t 490 2026-09-27 01:26:36.638306+00 2026-10-14 08:28:27.072278+00 f \N 200.00 75
1fd369b6-bdac-4390-b3ba-f9255eeaf3eb Muffler/Exhaust Replacement Removes and installs a compatible muffler or exhaust, then checks for leaks; the replacement part may be separate. 100.00 90 engine \N t 510 2026-09-27 01:26:36.638306+00 2026-10-14 08:28:51.421857+00 f \N 200.00 90
731aac9e-7dd7-496b-b6ed-f8efc1837c36 Racing Carburetor Installation Installs and base-tunes a customer-supplied performance carburetor; additional fine-tuning may require a follow-up visit. 300.00 150 upgrades \N t 520 2026-09-27 01:26:36.638306+00 2026-10-14 08:29:05.043733+00 f \N 400.00 150
dd863611-9344-4ed4-a516-69f1f6474821 Frame/Crash Slider Installation Fits and secures customer-supplied frame or swingarm sliders to the appropriate mounting points. 100.00 60 upgrades \N t 530 2026-09-27 01:26:36.638306+00 2026-10-14 08:29:16.596957+00 f \N 200.00 60
c4c93b1e-7f81-4be3-a748-cc677ff809bc Fan Spinner Installation Fits and secures a customer-supplied decorative fan spinner on the front-wheel axle. 50.00 30 upgrades \N t 540 2026-09-27 01:26:36.638306+00 2026-10-14 08:29:28.59411+00 f \N 100.00 30
\.

--
-- Data for Name: appointment_services; Type: TABLE DATA; Schema: public; Owner: postgres
--

COPY "public"."appointment_services" ("id", "appointment_id", "service_id", "service_name", "price", "duration_minutes") FROM stdin;
5295b518-782b-42d0-928f-5dd6c6eafa77 2334d5bb-94f4-4889-8a06-0caee9004790 97a9714b-20e6-4402-a3ee-4fcdefc1e9db Change Oil 50.00 60
7927804f-3613-40e2-a1c4-429e4c221c13 2334d5bb-94f4-4889-8a06-0caee9004790 e5cf562e-7122-440b-9483-033b78aca286 Carburetor / FI Cleaning 300.00 90
6d5dec9a-a05e-448e-a46d-8136db4522eb 2334d5bb-94f4-4889-8a06-0caee9004790 ab3c3ba9-415a-4f0e-9a42-bffb7d1903c1 Electrical Diagnostics 350.00 60
8d8f3384-c578-4611-b845-54acbd18e1b3 2a2de84e-e1d7-42e7-a36d-45905d69df12 97a9714b-20e6-4402-a3ee-4fcdefc1e9db Change Oil 50.00 60
5540fe87-9947-421f-aead-699d9fac264a

2334d5bb-94f4-4889-8a06-0caee9004790
f8feeecc-86e2-4312-8798-ea5c144f1ad4 Engine Overhaul 2000.00 480
e86b68a6-9682-481d-b589-bee470c9f183
3eac3b68-b97b-46f0-a8eb-ad913f8bc8c6
97a9714b-20e6-4402-a3ee-4fcdefc1e9db Change Oil 50.00 60
10c152f5-bc08-4e57-a976-6c160548e1a9
3eac3b68-b97b-46f0-a8eb-ad913f8bc8c6
e5742f78-e0f3-4c14-b4e5-9ad22b3ff85b Brake Service 50.00 90
5d3655fa-7d00-437e-91a8-7818a57a0073
30c80b92-c789-40fc-936d-5f6567091423
97a9714b-20e6-4402-a3ee-4fcdefc1e9db Change Oil 50.00 60
725348ca-f664-42f4-964b-78be9c51116e
a26950f7-62f0-439a-8f3b-07fa6a6c50d6
97a9714b-20e6-4402-a3ee-4fcdefc1e9db Change Oil 50.00 60
30b68bd8-b52e-4cd8-a6e9-b3efeb4a831d
eda6b8a7-6dc1-4584-8ad5-e918438c0a05
97a9714b-20e6-4402-a3ee-4fcdefc1e9db Change Oil 50.00 60
5643a04a-e549-4fe1-8701-34c46023936a
2ce47e77-9cfc-49b8-9710-c3b33294eb54
e5742f78-e0f3-4c14-b4e5-9ad22b3ff85b Brake Service 50.00 90
91ac9ff5-f1ce-4dbc-a976-f03173792847
de1e21dc-53e4-4362-8898-3bd1da2d67fc
e5742f78-e0f3-4c14-b4e5-9ad22b3ff85b Brake Service 50.00 90
4f452748-318a-4438-bdf9-e464fdc81986
617073f7-8cfa-4f3f-8352-aff1a76da780
97a9714b-20e6-4402-a3ee-4fcdefc1e9db Change Oil 50.00 60
f2422b78-3f68-4c5a-af04-942bdf89264a
c5f4d85a-7617-42c7-84db-941d333879bf
97a9714b-20e6-4402-a3ee-4fcdefc1e9db Change Oil 50.00 60
8c9a7188-8fe7-4459-b6cc-292259576765
dc1c4a1c-1532-4d25-bbd5-b7d8793de6e9
97a9714b-20e6-4402-a3ee-4fcdefc1e9db Change Oil 50.00 60
4dfac110-2cc0-47dd-b1cf-b3a2c6347828
af37178b-4dfd-455f-8d90-2f7d3f4b4304
e5742f78-e0f3-4c14-b4e5-9ad22b3ff85b Brake Service 50.00 90
c95c7ae4-7459-4e23-9c42-0d80ccbac457
cab87474-af88-46f2-8e66-7acd2e5d2baa
97a9714b-20e6-4402-a3ee-4fcdefc1e9db Change Oil 50.00 60
dabff062-2de3-4bcb-be11-c1c1c5cb3e2e
e8900d91-8ae4-4bd6-9a33-eb818faa92a2
97a9714b-20e6-4402-a3ee-4fcdefc1e9db Change Oil 50.00 60
c7cbab07-696a-4fb7-8589-b1b5a05d0e14
e8900d91-8ae4-4bd6-9a33-eb818faa92a2
e5742f78-e0f3-4c14-b4e5-9ad22b3ff85b Brake Service 50.00 90
b3f84d16-9c5b-49bc-be2e-e2e60e76d287

cfe5e95c-d7d8-4ba5-b6e7-9020b664a0fc
9bac85e5-f254-43ee-a36d-384b25fcae83 Tire Change 70.00 90
d4b65f81-bd3a-4278-b3f4-d6015e97bb87
71b72a10-644d-4b57-ace3-740377b48bd6
97a9714b-20e6-4402-a3ee-4fcdefc1e9db Change Oil 50.00 60
7a97d46b-5e7c-483b-9edf-84ec2b65201b
d070ed76-55d2-40e1-a55a-fc58fc88bfb8
97a9714b-20e6-4402-a3ee-4fcdefc1e9db Change Oil 50.00 60
4f5941ee-fde4-4055-a00c-fd7b7f26910e
545503a4-1416-4f5f-83ba-ebc50f5912d7
97a9714b-20e6-4402-a3ee-4fcdefc1e9db Change Oil 50.00 60
ea104f43-e3b5-4836-bfa3-37abb5f1da73
34b4dba4-8ed5-42b8-8d27-ca469116c5d1
c18718c3-3aa9-40f6-8907-3a7922a07922 Handlebar Switch Replacement
100.00 75
09ac7fea-fa20-43a8-895b-e2a5535bc6ed
6c0de99e-7522-4f4e-ad11-0913682e3ae3
e852fde6-f231-4b05-95b4-26e8d6455c97 Chain Replacement 100.00 90
1e8e6836-c7d6-4493-8732-28f5153ba48c
64fe7d0c-3604-4dc4-a072-a3344c02e337
97a9714b-20e6-4402-a3ee-4fcdefc1e9db Change Oil 50.00 60
81e479a6-fd9d-4aba-81fe-a2dafb2c85b4
64fe7d0c-3604-4dc4-a072-a3344c02e337
e5742f78-e0f3-4c14-b4e5-9ad22b3ff85b Brake Service 350.00 60
7e851d32-86c8-42d8-ac8e-6c210d8f2ff8
bb9e341a-1d2f-433b-ae2a-119cba575564
97a9714b-20e6-4402-a3ee-4fcdefc1e9db Change Oil 50.00 60
400adf69-c7b7-460b-b30e-d44d83390c76
33fbda0f-318f-41fd-8eac-328e296ea0d3
d9ecf096-df3b-47f5-a012-80d98aacf9f5 Motorcycle Tune-Up 100.00 180
15229e96-c71e-4768-a59b-25998e28c030
33fbda0f-318f-41fd-8eac-328e296ea0d3
6a7f36bb-0ec8-47f4-bf32-54c09de9b4b0 Motorcylce Tune-Up 300.00 60
883cf350-d0e1-4cac-acd4-a473e17a812b
33fbda0f-318f-41fd-8eac-328e296ea0d3
f0132c1b-d630-4aa3-a56b-165ce0c408ec Clutch Adjustment/Repair 50.00
60
7097382e-1382-4768-80df-cfb67b63ba41
b7424607-9e7d-4fd4-ae4e-73dba6c26b21
97a9714b-20e6-4402-a3ee-4fcdefc1e9db Change Oil 50.00 60
f821f4d8-e889-4c99-9199-89d641fbb5b5
147297a6-c092-4337-ab4e-2c86b7848da9
97a9714b-20e6-4402-a3ee-4fcdefc1e9db Change Oil 50.00 60
2e92f279-e0ad-4872-9267-35f694a311d9
147297a6-c092-4337-ab4e-2c86b7848da9

e5742f78-e0f3-4c14-b4e5-9ad22b3ff85b Brake Service 350.00 60
8ea3e6cf-ea48-46aa-a727-9dabd59b01a6
2334d5bb-94f4-4889-8a06-0caee9004790
ed2e91b9-1d71-4e29-814d-7b7507751569 Preventive Maintenance Service (PMS) 350.00 90
455fdb02-e558-4548-b2ef-c74cc18a4688
2334d5bb-94f4-4889-8a06-0caee9004790
08eef9ae-0750-4ed4-ade6-35aed340eea5 Tire Change & Balancing 250.00 60
a5f0a1e1-b5ae-4462-a456-41421a11092a
2334d5bb-94f4-4889-8a06-0caee9004790
72a4c5f5-e5f1-4e71-b872-30a3615ac362 Suspension Tuning / Rebuild 2500.00 180
d01e59b9-00a9-4c6a-b9d8-f1c7097860a5
2334d5bb-94f4-4889-8a06-0caee9004790
e5742f78-e0f3-4c14-b4e5-9ad22b3ff85b Brake Service 350.00 60
acc0f702-6942-4e6b-8dd5-20a221c875c1
2334d5bb-94f4-4889-8a06-0caee9004790
6a7f36bb-0ec8-47f4-bf32-54c09de9b4b0 Motorcylce Tune-Up 300.00 60
012e80e8-e13f-4adf-a6ed-38a84df8f22d
2334d5bb-94f4-4889-8a06-0caee9004790
61b18a4e-6804-4701-b035-b8e7913f7911 Cooling System Service 400.00 90
7d32ad11-48d9-4fba-af61-575122ec44f8
2334d5bb-94f4-4889-8a06-0caee9004790
b6af4e06-1d15-4f61-b7e8-77719cb355de Radiator Cleaning 300.00 60
d118d0d1-3a06-47da-8c1a-d8f3229181cc
2334d5bb-94f4-4889-8a06-0caee9004790
e724f832-2a1f-4949-bf6f-ad72240d99ea Fork Seal Replacement 800.00 120
354f388c-fb40-4b02-bb58-7e3bb5f02223
2334d5bb-94f4-4889-8a06-0caee9004790
f01fd9c7-6a0f-4510-bf61-fd7469d1290a Steering Bearing Replacement 700.00 150
8a58d33a-f335-4bf6-a0f2-12f00cdd58f5
c2948c1c-8f78-441c-bcc7-1cfa03c0718b
1477338f-ec6e-4bf6-b916-b619cc58ac0d Chain & Sprocket Replacement 500.00 90
6808e7b7-2bb5-4d26-a3f6-90487be1d883
6a347f55-d34f-42ff-9a73-18f6f0830092
d9ecf096-df3b-47f5-a012-80d98aacf9f5 Motorcycle Tune-Up 100.00 180
d9f6d0c7-dd9b-4789-95b9-80c470f30f43
6a347f55-d34f-42ff-9a73-18f6f0830092
61b18a4e-6804-4701-b035-b8e7913f7911 Cooling System Service 400.00 90

950234e5-ebef-432f-a400-acaea5046c06
6a347f55-d34f-42ff-9a73-18f6f0830092
b6af4e06-1d15-4f61-b7e8-77719cb355de	Radiator Cleaning	300.00	60
7a09496c-9eeb-4268-a60e-92e077bc91a5
6a347f55-d34f-42ff-9a73-18f6f0830092
f0132c1b-d630-4aa3-a56b-165ce0c408ec	Clutch Adjustment/Repair	50.00
60
cb5e6392-f1a3-437a-a06d-96b1062e9c5e
6957127d-9bdd-4173-a856-2f08819ce87f
1477338f-ec6e-4bf6-b916-b619cc58ac0d	Chain & Sprocket Replacement
500.00	90
d4200593-be5e-4633-b9a3-9074b492af82
2d1379ce-21a4-459a-a1fa-97b2f7cc75db
d9ecf096-df3b-47f5-a012-80d98aacf9f5	Motorcycle Tune-Up	100.00	180
c22b7276-15a9-41b3-bc05-95ec9894c507
2d1379ce-21a4-459a-a1fa-97b2f7cc75db
72a4c5f5-e5f1-4e71-b872-30a3615ac362	Suspension Tuning / Rebuild
2500.00	180
546fef7e-00c2-4c0d-b679-ea8e807b5133
2d1379ce-21a4-459a-a1fa-97b2f7cc75db
b6af4e06-1d15-4f61-b7e8-77719cb355de	Radiator Cleaning	300.00	60
2bb3b8cd-8f41-47e3-8b60-1fb8a9fdb28c
6014afba-21ae-402c-a687-150730e547ea
b950fc34-4de4-42b6-b3ff-5c0b352103db	Electrical Diagnostics FI	150.00
120
\.

--
-- Data for Name: blocked_numbers; Type: TABLE DATA; Schema: public;
Owner: postgres
--

COPY "public"."blocked_numbers" ("id", "phone", "reason", "created_at",
"created_by", "is_archived", "archived_at") FROM stdin;
f522a91c-85ce-424a-9044-19f61638fc4e	09123456798	ABUSE
2026-08-23 16:04:05.199649+00	\N	f	\N
\.

--
-- Data for Name: crew_availability_exceptions; Type: TABLE DATA; Schema:
public; Owner: postgres
--

COPY "public"."crew_availability_exceptions" ("id", "crew_id", "start_date", "end_date", "start_time", "end_time", "reason", "note", "is_all_day", "created_at") FROM stdin;
\.

--
-- Data for Name: crew_schedules; Type: TABLE DATA; Schema: public; Owner: postgres
--

COPY "public"."crew_schedules" ("id", "crew_id", "day_of_week", "start_time", "end_time", "is_working", "created_at", "schedule_date", "effective_from", "effective_to", "note", "updated_at") FROM stdin;
57411c5c-70e6-4d99-af37-3f7b49f51141	1700c9e8-37e5-42c2-98e8-48854585ba7d	2	13:00:00	17:00:00	t	2026-08-23 09:24:38.968137+00	2026-09-01	\N	\N	\N	2026-08-23 09:24:38.968137+00
6798a0ec-b98f-4a89-98c5-e0aedc36b795	659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b	2	08:00:00	17:00:00	t	2026-08-23 09:25:01.19064+00	2026-09-01	\N	\N	\N	2026-08-23 09:25:01.19064+00
bfcfc47b-cfe8-481e-83b4-ce865d42fdcf	f3c5770f-e485-4efc-83f6-892422d8592e	1	08:00:00	13:30:00	t	2026-08-23 09:54:44.524006+00	2026-09-14	\N	\N	\N	2026-08-23 09:54:44.524006+00
efec280d-fd77-446e-ad36-c6552eb2d0f5	1700c9e8-37e5-42c2-98e8-48854585ba7d	2	08:00:00	17:00:00	t	2026-08-23 11:00:41.44959+00	2026-09-08	\N	\N	\N	2026-08-23 11:00:41.44959+00
29f7fa1f-4654-4055-9de9-8aeb59ad528b	1700c9e8-37e5-42c2-98e8-48854585ba7d	3	08:00:00	17:00:00	t	2026-08-23 11:00:41.44959+00	2026-09-09	\N	\N	\N	2026-08-23 11:00:41.44959+00
74ddbbf6-7aa1-49c7-8105-5dbed5067fb0	1700c9e8-37e5-42c2-98e8-48854585ba7d	4	08:00:00	17:00:00	t	2026-08-23 11:00:41.44959+00	2026-09-10	\N	\N	\N	2026-08-23 11:00:41.44959+00
3a0c9d8c-9ee2-4d7b-bc4d-42bff3707f9f	1700c9e8-37e5-42c2-98e8-48854585ba7d	5	08:00:00	17:00:00	t	2026-08-23 11:00:41.44959+00	2026-09-11	\N	\N	\N	2026-08-23 11:00:41.44959+00
9abd0027-d29f-4f0a-a67f-2a8a43400c55	1700c9e8-37e5-42c2-98e8-48854585ba7d	6	08:00:00	17:00:00	t	2026-08-23 11:00:41.44959+00	2026-09-12	\N	\N	\N	2026-08-23

11:00:41.44959+00
166f3f95-ba9b-4f55-bae6-61390dedc0a7
03bb877e-57f8-42dc-9c0e-3fcc82b1a207 1 08:00:00 17:00:00 t
2026-09-08 14:38:59.294597+00 2026-09-14 \N \N \N 2026-09-08
14:38:59.294597+00
4c12ea86-a747-4fb6-947b-ffb8a38ed876
03bb877e-57f8-42dc-9c0e-3fcc82b1a207 2 08:00:00 17:00:00 t
2026-09-08 14:38:59.294597+00 2026-09-15 \N \N \N 2026-09-08
14:38:59.294597+00
ba2e1214-1c5e-4f7b-9dc4-3cb898a5ace3
03bb877e-57f8-42dc-9c0e-3fcc82b1a207 3 08:00:00 17:00:00 t
2026-09-08 14:38:59.294597+00 2026-09-16 \N \N \N 2026-09-08
14:38:59.294597+00
8a122bf1-4a77-4289-903d-92290efeb260
03bb877e-57f8-42dc-9c0e-3fcc82b1a207 4 08:00:00 17:00:00 t
2026-09-08 14:38:59.294597+00 2026-09-17 \N \N \N 2026-09-08
14:38:59.294597+00
d3c6f238-fa98-4c66-89d8-037553caa197
03bb877e-57f8-42dc-9c0e-3fcc82b1a207 5 08:00:00 17:00:00 t
2026-09-08 14:38:59.294597+00 2026-09-18 \N \N \N 2026-09-08
14:38:59.294597+00
625f6017-52b4-47c4-ad30-6dbcb35196b1
03bb877e-57f8-42dc-9c0e-3fcc82b1a207 6 08:00:00 17:00:00 t
2026-09-08 14:38:59.294597+00 2026-09-19 \N \N \N 2026-09-08
14:38:59.294597+00
58b66753-3342-42c2-b105-abf6e47b7eb0
03bb877e-57f8-42dc-9c0e-3fcc82b1a207 1 08:00:00 17:00:00 t
2026-09-08 14:38:59.294597+00 2026-09-21 \N \N \N 2026-09-08
14:38:59.294597+00
42e24752-4e51-460b-b4de-43d0492bc39a
03bb877e-57f8-42dc-9c0e-3fcc82b1a207 2 08:00:00 17:00:00 t
2026-09-08 14:38:59.294597+00 2026-09-22 \N \N \N 2026-09-08
14:38:59.294597+00
6bdfc093-7322-4525-ace0-f2b6c88d2210
03bb877e-57f8-42dc-9c0e-3fcc82b1a207 4 08:00:00 17:00:00 t
2026-09-08 14:38:59.294597+00 2026-09-24 \N \N \N 2026-09-08
14:38:59.294597+00
597cb434-ea7b-4081-950b-c592342acd9a
03bb877e-57f8-42dc-9c0e-3fcc82b1a207 5 08:00:00 17:00:00 t
2026-09-08 14:38:59.294597+00 2026-09-25 \N \N \N 2026-09-08
14:38:59.294597+00
3e4da663-732a-409b-a050-d0393e374fe1
03bb877e-57f8-42dc-9c0e-3fcc82b1a207 3 08:00:00 17:00:00 t
2026-09-08 14:38:59.294597+00 2026-09-23 \N \N \N 2026-09-08
14:38:59.294597+00

818d3a29-0d3f-4f5d-87f2-3172f3f1c839
03bb877e-57f8-42dc-9c0e-3fcc82b1a207 6 08:00:00 17:00:00 t
2026-09-08 14:38:59.294597+00 2026-09-26 \N \N \N 2026-09-08
14:38:59.294597+00
f5388a05-6d87-419c-b487-3827b4704498
03bb877e-57f8-42dc-9c0e-3fcc82b1a207 1 08:00:00 17:00:00 t
2026-09-08 14:38:59.294597+00 2026-09-28 \N \N \N 2026-09-08
14:38:59.294597+00
f661b88a-4288-4aa6-9004-be62e53ac0a5
03bb877e-57f8-42dc-9c0e-3fcc82b1a207 2 08:00:00 17:00:00 t
2026-09-08 14:38:59.294597+00 2026-09-29 \N \N \N 2026-09-08
14:38:59.294597+00
24c96896-bcdc-4522-8af4-9a7db4fd8f7e
03bb877e-57f8-42dc-9c0e-3fcc82b1a207 3 08:00:00 17:00:00 t
2026-09-08 14:38:59.294597+00 2026-09-30 \N \N \N 2026-09-08
14:38:59.294597+00
0d81e035-a865-4b27-bd1e-26c432a0b831
1700c9e8-37e5-42c2-98e8-48854585ba7d 1 08:00:00 17:00:00 t
2026-09-08 14:39:28.809772+00 2026-09-14 \N \N \N 2026-09-08
14:39:28.809772+00
942db473-bbe2-4b69-9260-4c4c8bdcf6be
1700c9e8-37e5-42c2-98e8-48854585ba7d 2 08:00:00 17:00:00 t
2026-09-08 14:39:28.809772+00 2026-09-15 \N \N \N 2026-09-08
14:39:28.809772+00
2e213cc1-21d5-4ee9-a2f1-ead9d141cd7e
1700c9e8-37e5-42c2-98e8-48854585ba7d 3 08:00:00 17:00:00 t
2026-09-08 14:39:28.809772+00 2026-09-16 \N \N \N 2026-09-08
14:39:28.809772+00
3dcabaaf-d9c0-4a2f-ae61-00ed0e28e227
1700c9e8-37e5-42c2-98e8-48854585ba7d 4 08:00:00 17:00:00 t
2026-09-08 14:39:28.809772+00 2026-09-17 \N \N \N 2026-09-08
14:39:28.809772+00
9d7d394b-f848-47b4-81f6-378833401574
1700c9e8-37e5-42c2-98e8-48854585ba7d 6 08:00:00 17:00:00 t
2026-09-08 14:39:28.809772+00 2026-09-19 \N \N \N 2026-09-08
14:39:28.809772+00
3a6e2132-f36d-4709-9e5e-abf3c5caaa38
1700c9e8-37e5-42c2-98e8-48854585ba7d 5 08:00:00 17:00:00 t
2026-09-08 14:39:28.809772+00 2026-09-18 \N \N \N 2026-09-08
14:39:28.809772+00
8ff1f3a5-adf0-4f86-8a5b-f1c65ea2960e
1700c9e8-37e5-42c2-98e8-48854585ba7d 1 08:00:00 17:00:00 t
2026-09-08 14:39:28.809772+00 2026-09-21 \N \N \N 2026-09-08
14:39:28.809772+00
4286735f-2de9-4720-bd83-794c7456724e

1700c9e8-37e5-42c2-98e8-48854585ba7d 2 08:00:00 17:00:00 t
2026-09-08 14:39:28.809772+00 2026-09-22 \N \N \N 2026-09-08
14:39:28.809772+00
a575fe70-e13b-4141-baf4-b3afe2c1084a
1700c9e8-37e5-42c2-98e8-48854585ba7d 3 08:00:00 17:00:00 t
2026-08-20 23:46:54.413982+00 2026-09-02 \N \N \N 2026-08-22
06:09:00.658628+00
fb517dd9-f145-4b3a-8bc2-9cc36f72be56
1700c9e8-37e5-42c2-98e8-48854585ba7d 4 08:00:00 17:00:00 t
2026-08-20 23:47:03.300685+00 2026-09-03 \N \N \N 2026-08-22
06:09:00.658628+00
178a6edf-c7fc-4635-99fa-4e08593cbffc
1700c9e8-37e5-42c2-98e8-48854585ba7d 4 08:00:00 17:00:00 t
2026-09-08 14:39:28.809772+00 2026-09-24 \N \N \N 2026-09-08
14:39:28.809772+00
476ed5bd-ba40-4a0a-90a3-6b7f47630ea3
f3c5770f-e485-4efc-83f6-892422d8592e 5 08:00:00 17:00:00 t
2026-08-20 23:48:00.353559+00 2026-09-04 \N \N \N 2026-08-22
06:09:00.658628+00
97cc6dcb-2456-47d1-bc62-fc7032536d3d
1700c9e8-37e5-42c2-98e8-48854585ba7d 3 08:00:00 17:00:00 t
2026-09-08 14:39:28.809772+00 2026-09-23 \N \N \N 2026-09-08
14:39:28.809772+00
a35b10b8-f3b8-458e-a078-25c3aef6adba
1700c9e8-37e5-42c2-98e8-48854585ba7d 5 08:00:00 17:00:00 t
2026-09-08 14:39:28.809772+00 2026-09-25 \N \N \N 2026-09-08
14:39:28.809772+00
02beba23-aaa4-41ca-a9c4-4da6272a6413
f3c5770f-e485-4efc-83f6-892422d8592e 2 08:00:00 17:00:00 t
2026-08-22 04:32:59.626815+00 2026-09-01 \N \N \N 2026-08-22
06:09:00.658628+00
03d3e52c-dd6e-467c-b68c-cfe87608a199
5cb00871-f5ef-4ccf-bd61-4746909237e2 4 08:00:00 12:00:00 t
2026-08-23 00:54:25.215993+00 2026-09-03 \N \N \N 2026-08-23
00:54:25.215993+00
8ba6afbc-1268-4844-a189-97655ea91e52
03bb877e-57f8-42dc-9c0e-3fcc82b1a207 2 08:00:00 17:00:00 t
2026-08-23 04:47:10.113652+00 2026-09-01 \N \N \N 2026-08-23
04:47:10.113652+00
8c24df9e-aa27-476e-abc6-1926af0974a2
5cb00871-f5ef-4ccf-bd61-4746909237e2 2 08:00:00 17:00:00 t
2026-08-23 04:47:19.675453+00 2026-09-01 \N \N \N 2026-08-23
04:47:19.675453+00
7c43100f-84d4-40e4-9b48-68dcef7660b7
5cb00871-f5ef-4ccf-bd61-4746909237e2 3 08:00:00 17:00:00 t

2026-08-23 04:47:33.942706+00 2026-09-02 \N \N \N 2026-08-23 04:47:33.942706+00
02cda5bc-3407-4c81-9c4f-5f54fb2f6997
5cb00871-f5ef-4ccf-bd61-4746909237e2 5 08:00:00 17:00:00 t
2026-08-23 04:47:55.030959+00 2026-09-04 \N \N \N 2026-08-23 04:47:55.030959+00
bb78da3b-285d-4e23-bdf7-f68cab6baef1
03bb877e-57f8-42dc-9c0e-3fcc82b1a207 6 08:00:00 17:00:00 t
2026-08-23 04:48:10.174851+00 2026-09-05 \N \N \N 2026-08-23 04:48:10.174851+00
f2dc1c39-e711-4743-a871-c830771ab83c
03bb877e-57f8-42dc-9c0e-3fcc82b1a207 1 08:00:00 17:00:00 t
2026-08-23 04:48:23.319036+00 2026-09-07 \N \N \N 2026-08-23 04:48:23.319036+00
20e3c8fa-aa8e-427f-9efd-dc7b865c2bf7
5cb00871-f5ef-4ccf-bd61-4746909237e2 1 08:00:00 17:00:00 t
2026-08-23 04:48:34.51034+00 2026-09-07 \N \N \N 2026-08-23 04:48:34.51034+00
312c730e-5d9f-4831-a507-891ae55c2a8b
1700c9e8-37e5-42c2-98e8-48854585ba7d 6 08:00:00 17:00:00 t
2026-09-08 14:39:28.809772+00 2026-09-26 \N \N \N 2026-09-08 14:39:28.809772+00
144941cf-4f1f-42be-b6d0-385557063eb3
1700c9e8-37e5-42c2-98e8-48854585ba7d 1 08:00:00 17:00:00 t
2026-09-08 14:39:28.809772+00 2026-09-28 \N \N \N 2026-09-08 14:39:28.809772+00
05248dc4-f09e-4313-b4c0-b1292c8afdbd
1700c9e8-37e5-42c2-98e8-48854585ba7d 2 08:00:00 17:00:00 t
2026-09-08 14:39:28.809772+00 2026-09-29 \N \N \N 2026-09-08 14:39:28.809772+00
00dec071-a934-4be8-82e1-dd9cced1f2b5
1700c9e8-37e5-42c2-98e8-48854585ba7d 3 08:00:00 17:00:00 t
2026-09-08 14:39:28.809772+00 2026-09-30 \N \N \N 2026-09-08 14:39:28.809772+00
780314e8-6776-4c4f-b04e-ba3041c81926
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 2 08:00:00 17:00:00 t
2026-09-08 14:40:06.579188+00 2026-09-15 \N \N \N 2026-09-08 14:40:06.579188+00
52b3ccd4-e0f4-40c5-a08a-f8e992ac0bf2
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 3 08:00:00 17:00:00 t
2026-09-08 14:40:06.579188+00 2026-09-16 \N \N \N 2026-09-08 14:40:06.579188+00
daa276b4-a0f7-4509-b932-6c123283f46a
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 1 08:00:00 17:00:00 t
2026-09-08 14:40:06.579188+00 2026-09-21 \N \N \N 2026-09-08

14:40:06.579188+00
87c0102a-6342-45de-a506-53e6b62df8ee
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 4 08:00:00 17:00:00 t
2026-09-08 14:40:06.579188+00 2026-09-17 \N \N \N 2026-09-08
14:40:06.579188+00
f4ed1d97-15c7-42e1-8e84-beeaa63a458d
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 5 08:00:00 17:00:00 t
2026-09-08 14:40:06.579188+00 2026-09-18 \N \N \N 2026-09-08
14:40:06.579188+00
5ce2d683-9b0f-4f92-b008-a05c1d714812
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 6 08:00:00 17:00:00 t
2026-09-08 14:40:06.579188+00 2026-09-19 \N \N \N 2026-09-08
14:40:06.579188+00
55b4cf81-5027-4b55-85c5-db9def697c5d
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 6 08:00:00 17:00:00 t
2026-09-08 14:40:06.579188+00 2026-09-26 \N \N \N 2026-09-08
14:40:06.579188+00
62ad33d8-8b9a-4deb-a0e7-e9f3de1b78d1
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 5 08:00:00 17:00:00 t
2026-09-08 14:40:06.579188+00 2026-09-25 \N \N \N 2026-09-08
14:40:06.579188+00
315826f0-84c2-4906-833c-15ecf01a5541
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 4 08:00:00 17:00:00 t
2026-09-08 14:40:06.579188+00 2026-09-24 \N \N \N 2026-09-08
14:40:06.579188+00
7b09228e-174c-4596-b7fe-c9dbf43e092d
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 3 08:00:00 17:00:00 t
2026-09-08 14:40:06.579188+00 2026-09-23 \N \N \N 2026-09-08
14:40:06.579188+00
e9e878d7-8c19-47d4-9adb-e004cc002c2f
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 2 08:00:00 17:00:00 t
2026-09-08 14:40:06.579188+00 2026-09-22 \N \N \N 2026-09-08
14:40:06.579188+00
6fa611fd-36fd-40a2-8d72-511cb848044a
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 1 08:00:00 17:00:00 t
2026-09-08 14:40:06.579188+00 2026-09-28 \N \N \N 2026-09-08
14:40:06.579188+00
d31ee468-751e-4983-a8f6-0078d2e295ad
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 2 08:00:00 17:00:00 t
2026-09-08 14:40:06.579188+00 2026-09-29 \N \N \N 2026-09-08
14:40:06.579188+00
e6eff3ec-06a6-4c8f-94e6-8e1183c20655
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 3 08:00:00 17:00:00 t
2026-09-08 14:40:06.579188+00 2026-09-30 \N \N \N 2026-09-08
14:40:06.579188+00

14e7973d-fe5b-4497-9879-eaf5fe447220
f3c5770f-e485-4efc-83f6-892422d8592e 4 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-01 \N \N \N 2026-09-18
09:06:58.654864+00
50ce0b38-0e19-42b5-b9d6-0a7c2078030e
f3c5770f-e485-4efc-83f6-892422d8592e 5 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-02 \N \N \N 2026-09-18
09:06:58.654864+00
cda3fa42-0c95-4af1-a35f-3c0d7eef9637
f3c5770f-e485-4efc-83f6-892422d8592e 6 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-03 \N \N \N 2026-09-18
09:06:58.654864+00
346a9955-6a92-4d8f-825d-6178ec167a44
f3c5770f-e485-4efc-83f6-892422d8592e 0 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-04 \N \N \N 2026-09-18
09:06:58.654864+00
87c51185-5cad-4a93-abd6-6c28a7a5f1a6
f3c5770f-e485-4efc-83f6-892422d8592e 1 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-05 \N \N \N 2026-09-18
09:06:58.654864+00
7feff085-924f-4950-beaf-0aa2e0409ed9
f3c5770f-e485-4efc-83f6-892422d8592e 2 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-06 \N \N \N 2026-09-18
09:06:58.654864+00
275eb5cb-e5e1-45ca-914d-4b2b4ef4c68c
f3c5770f-e485-4efc-83f6-892422d8592e 3 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-07 \N \N \N 2026-09-18
09:06:58.654864+00
a501111b-8a2c-402f-9ed5-980462d4d2fe
f3c5770f-e485-4efc-83f6-892422d8592e 4 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-08 \N \N \N 2026-09-18
09:06:58.654864+00
7c18881f-94ce-44ad-bbf8-9792bf9e2928
f3c5770f-e485-4efc-83f6-892422d8592e 5 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-09 \N \N \N 2026-09-18
09:06:58.654864+00
fec0435f-45f8-43f4-ae40-9ca943a8caf7
f3c5770f-e485-4efc-83f6-892422d8592e 6 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-10 \N \N \N 2026-09-18
09:06:58.654864+00
13e53fc4-ff83-424a-ba92-312376f953dc
f3c5770f-e485-4efc-83f6-892422d8592e 0 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-11 \N \N \N 2026-09-18
09:06:58.654864+00
ed492844-9490-425b-b372-31bb964e9af6

f3c5770f-e485-4efc-83f6-892422d8592e 1 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-12 \N \N \N 2026-09-18
09:06:58.654864+00
3a82fcdf-cbe4-4b32-b19a-45b5701025ab
f3c5770f-e485-4efc-83f6-892422d8592e 2 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-13 \N \N \N 2026-09-18
09:06:58.654864+00
a2e5d75d-4916-497f-bd2e-8ac3d9a21793
f3c5770f-e485-4efc-83f6-892422d8592e 3 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-14 \N \N \N 2026-09-18
09:06:58.654864+00
e9dc3b92-6141-4a93-976f-d7e491cc0606
f3c5770f-e485-4efc-83f6-892422d8592e 4 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-15 \N \N \N 2026-09-18
09:06:58.654864+00
d66300ff-9c06-4731-bb75-9b31d760d8cb
f3c5770f-e485-4efc-83f6-892422d8592e 5 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-16 \N \N \N 2026-09-18
09:06:58.654864+00
b65b7504-2bf8-4575-aa89-ba514e23b7de
f3c5770f-e485-4efc-83f6-892422d8592e 6 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-17 \N \N \N 2026-09-18
09:06:58.654864+00
c87846e6-ac90-486b-8ee6-b0c9c4e3f819
f3c5770f-e485-4efc-83f6-892422d8592e 0 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-18 \N \N \N 2026-09-18
09:06:58.654864+00
41d0f84e-7ac7-4824-b1ec-e28b175624df
f3c5770f-e485-4efc-83f6-892422d8592e 1 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-19 \N \N \N 2026-09-18
09:06:58.654864+00
cde9e6e9-4870-4539-bbd5-94348e4ef92f
f3c5770f-e485-4efc-83f6-892422d8592e 3 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-21 \N \N \N 2026-09-18
09:06:58.654864+00
14587a4c-ddc6-4687-ae04-ba2c3c248fe7
f3c5770f-e485-4efc-83f6-892422d8592e 2 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-20 \N \N \N 2026-09-18
09:06:58.654864+00
9e2c7b64-5c1a-4e56-930f-ee2506340f02
f3c5770f-e485-4efc-83f6-892422d8592e 4 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-22 \N \N \N 2026-09-18
09:06:58.654864+00
0a61993b-a766-403e-881e-b5d7cd7daa38
f3c5770f-e485-4efc-83f6-892422d8592e 5 08:00:00 17:00:00 t

2026-09-18 09:06:58.654864+00 2026-10-23 \N \N \N 2026-09-18 09:06:58.654864+00
8bbde87a-a28d-477b-860b-ad2ce939f1bc
f3c5770f-e485-4efc-83f6-892422d8592e 6 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-24 \N \N \N 2026-09-18 09:06:58.654864+00
c79d9772-1c51-42b6-ac02-745b49ee7f3d
f3c5770f-e485-4efc-83f6-892422d8592e 0 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-25 \N \N \N 2026-09-18 09:06:58.654864+00
919629c8-4fea-4725-80b9-120bbb86572f
f3c5770f-e485-4efc-83f6-892422d8592e 1 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-26 \N \N \N 2026-09-18 09:06:58.654864+00
a5e3f683-424b-402b-86d6-2a77844c2e5e
f3c5770f-e485-4efc-83f6-892422d8592e 3 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-28 \N \N \N 2026-09-18 09:06:58.654864+00
3e02d0ce-89fd-4624-93ed-2f6926cff024
f3c5770f-e485-4efc-83f6-892422d8592e 2 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-27 \N \N \N 2026-09-18 09:06:58.654864+00
9b885e71-465b-4718-9e3a-821ec118ee94
f3c5770f-e485-4efc-83f6-892422d8592e 4 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-29 \N \N \N 2026-09-18 09:06:58.654864+00
c3928de9-e138-476b-bdd7-ee795f01836e
f3c5770f-e485-4efc-83f6-892422d8592e 5 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-30 \N \N \N 2026-09-18 09:06:58.654864+00
ccccf6bb-50f6-461a-a225-5048afbfb8c7
f3c5770f-e485-4efc-83f6-892422d8592e 6 08:00:00 17:00:00 t
2026-09-18 09:06:58.654864+00 2026-10-31 \N \N \N 2026-09-18 09:06:58.654864+00
03d3fea9-596c-45c5-9bdc-7c872d4f75d9
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 4 08:00:00 17:00:00 t
2026-09-18 09:07:40.287738+00 2026-10-01 \N \N \N 2026-09-18 09:07:40.287738+00
9fabfd43-2755-4d5f-8521-5721ee448c24
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 5 08:00:00 17:00:00 t
2026-09-18 09:07:40.287738+00 2026-10-02 \N \N \N 2026-09-18 09:07:40.287738+00
76e8d7d4-133d-447d-8a4b-98b212ca0f5e
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 6 08:00:00 17:00:00 t
2026-09-18 09:07:40.287738+00 2026-10-03 \N \N \N 2026-09-18

09:07:40.287738+00
d381846a-169e-4b1f-abe9-65fd7d8c9f79
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 0 08:00:00 17:00:00 t
2026-09-18 09:07:40.287738+00 2026-10-04 \N \N \N 2026-09-18
09:07:40.287738+00
479aead5-1432-4cef-9166-80f0236432c0
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 1 08:00:00 17:00:00 t
2026-09-18 09:07:40.287738+00 2026-10-05 \N \N \N 2026-09-18
09:07:40.287738+00
aae186ea-95d7-4c60-b85b-3bc9a371a821
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 2 08:00:00 17:00:00 t
2026-09-18 09:07:40.287738+00 2026-10-06 \N \N \N 2026-09-18
09:07:40.287738+00
8ae0728d-22fb-4fbe-8062-07a77539abcc
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 3 08:00:00 17:00:00 t
2026-09-18 09:07:40.287738+00 2026-10-07 \N \N \N 2026-09-18
09:07:40.287738+00
d16c5630-32b2-49ac-8171-ec465a628974
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 5 08:00:00 17:00:00 t
2026-09-18 09:07:40.287738+00 2026-10-09 \N \N \N 2026-09-18
09:07:40.287738+00
e08830b2-812c-40d0-bc5e-0909b408cf8c
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 4 08:00:00 17:00:00 t
2026-09-18 09:07:40.287738+00 2026-10-08 \N \N \N 2026-09-18
09:07:40.287738+00
ef117950-951e-4c1f-8766-4e77feed0353
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 6 08:00:00 17:00:00 t
2026-09-18 09:07:40.287738+00 2026-10-10 \N \N \N 2026-09-18
09:07:40.287738+00
806d50e8-60be-451d-a08e-9545c22e9e52
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 0 08:00:00 17:00:00 t
2026-09-18 09:07:40.287738+00 2026-10-11 \N \N \N 2026-09-18
09:07:40.287738+00
4de3b298-9cb3-402d-821b-731e62681d82
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 1 08:00:00 17:00:00 t
2026-09-18 09:07:40.287738+00 2026-10-12 \N \N \N 2026-09-18
09:07:40.287738+00
23dcebea-ad6a-4099-80ea-259029c73252
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 2 08:00:00 17:00:00 t
2026-09-18 09:07:40.287738+00 2026-10-13 \N \N \N 2026-09-18
09:07:40.287738+00
31cfdcef-659a-46f7-934a-a578f373cd06
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 3 08:00:00 17:00:00 t
2026-09-18 09:07:40.287738+00 2026-10-14 \N \N \N 2026-09-18
09:07:40.287738+00

92373336-fe18-4c82-af45-c60b5d86818e
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 4 08:00:00 17:00:00 t
2026-09-18 09:07:40.287738+00 2026-10-15 \N \N \N 2026-09-18
09:07:40.287738+00
1d2c13b1-4838-4bf5-af56-52a011132396
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 5 08:00:00 17:00:00 t
2026-09-18 09:07:40.287738+00 2026-10-16 \N \N \N 2026-09-18
09:07:40.287738+00
9751ea38-22e4-444d-9fd4-ca1fb1461619
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 6 08:00:00 17:00:00 t
2026-09-18 09:07:40.287738+00 2026-10-17 \N \N \N 2026-09-18
09:07:40.287738+00
d78bab4e-3338-4f57-a3fa-32c3c5e809ba
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 0 08:00:00 17:00:00 t
2026-09-18 09:07:40.287738+00 2026-10-18 \N \N \N 2026-09-18
09:07:40.287738+00
09fa2fb5-6053-4ef7-9b81-cd9db171f817
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 1 08:00:00 17:00:00 t
2026-09-18 09:07:40.287738+00 2026-10-19 \N \N \N 2026-09-18
09:07:40.287738+00
acc5da6f-d3bb-41cf-ab15-475f25a38d1a
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 3 08:00:00 17:00:00 t
2026-09-18 09:07:40.287738+00 2026-10-21 \N \N \N 2026-09-18
09:07:40.287738+00
311694a8-4f3d-4e18-a2b2-8e16bef1078a
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 4 08:00:00 17:00:00 t
2026-09-18 09:07:40.287738+00 2026-10-22 \N \N \N 2026-09-18
09:07:40.287738+00
dc8ced24-3e32-4b6d-bc14-601edfd4430e
659ab2de-f2d6-4ceb-ba3f-ec0b1f8d316b 2 08:00:00 17:00:00 t
2026-09-18 09:07:40.287738+00 2026-10-20 \N \N \N 2026-09-18
09:07:40.287738+00
1fce2b2e-177b-4044-86dd-b376706814fc
5cb00871-f5ef-4ccf-bd61-4746909237e2 3 08:00:00 17:00:00 t
2026-09-28 08:50:38.655222+00 2026-09-30 \N \N \N 2026-09-28
08:50:38.655222+00
49ead0ec-b812-40f8-947c-d9dab9c6c2a5
f3c5770f-e485-4efc-83f6-892422d8592e 3 08:00:00 17:00:00 t
2026-09-28 08:50:51.47699+00 2026-09-30 \N \N \N 2026-09-28
08:50:51.47699+00
0e9dca6c-7e0f-4a2f-9c7f-b9499c082508
5cb00871-f5ef-4ccf-bd61-4746909237e2 1 08:00:00 17:00:00 t
2026-10-08 01:40:13.199523+00 2026-10-12 \N \N \N 2026-10-08
01:40:13.199523+00
52685efe-b1f3-46fc-b53c-fd7f9fb01b86

5cb00871-f5ef-4ccf-bd61-4746909237e2 3 08:00:00 17:00:00 t
2026-10-08 01:40:13.199523+00 2026-10-14 \N \N \N 2026-10-08
01:40:13.199523+00
e062b6ba-6c07-4b94-9fc6-773002745428
5cb00871-f5ef-4ccf-bd61-4746909237e2 2 08:00:00 17:00:00 t
2026-10-08 01:40:13.199523+00 2026-10-13 \N \N \N 2026-10-08
01:40:13.199523+00
56679865-c42c-41e1-8f2d-efcaa028256f
5cb00871-f5ef-4ccf-bd61-4746909237e2 4 08:00:00 17:00:00 t
2026-10-08 01:40:13.199523+00 2026-10-15 \N \N \N 2026-10-08
01:40:13.199523+00
cb7c86bd-1a47-4865-a4c4-63d74a2fa10b
5cb00871-f5ef-4ccf-bd61-4746909237e2 5 08:00:00 17:00:00 t
2026-10-08 01:40:13.199523+00 2026-10-16 \N \N \N 2026-10-08
01:40:13.199523+00
648159b4-7dd0-43d1-8714-f5782393d192
5cb00871-f5ef-4ccf-bd61-4746909237e2 6 08:00:00 17:00:00 t
2026-10-08 01:40:13.199523+00 2026-10-17 \N \N \N 2026-10-08
01:40:13.199523+00
b1fdecd7-5966-44b4-bdf6-23fd39be03e6
5cb00871-f5ef-4ccf-bd61-4746909237e2 6 08:00:00 17:00:00 t
2026-10-08 01:40:13.199523+00 2026-10-24 \N \N \N 2026-10-08
01:40:13.199523+00
51c825a2-1872-4277-81e6-df395fdac414
5cb00871-f5ef-4ccf-bd61-4746909237e2 1 08:00:00 17:00:00 t
2026-10-08 01:40:13.199523+00 2026-10-19 \N \N \N 2026-10-08
01:40:13.199523+00
4ae42290-a2ae-4593-a39c-8d53c99cb2d1
5cb00871-f5ef-4ccf-bd61-4746909237e2 2 08:00:00 17:00:00 t
2026-10-08 01:40:13.199523+00 2026-10-20 \N \N \N 2026-10-08
01:40:13.199523+00
22ba0bdc-8923-4ccd-abb3-5e0161de5507
5cb00871-f5ef-4ccf-bd61-4746909237e2 3 08:00:00 17:00:00 t
2026-10-08 01:40:13.199523+00 2026-10-21 \N \N \N 2026-10-08
01:40:13.199523+00
c461988d-701e-4874-b316-60fda89c3746
5cb00871-f5ef-4ccf-bd61-4746909237e2 4 08:00:00 17:00:00 t
2026-10-08 01:40:13.199523+00 2026-10-22 \N \N \N 2026-10-08
01:40:13.199523+00
559feaf3-f1fc-48de-b3b0-d72c49f48a26
5cb00871-f5ef-4ccf-bd61-4746909237e2 5 08:00:00 17:00:00 t
2026-10-08 01:40:13.199523+00 2026-10-23 \N \N \N 2026-10-08
01:40:13.199523+00
06f52ecc-c4eb-4c4b-b4e1-8e15dd9698ca
5cb00871-f5ef-4ccf-bd61-4746909237e2 1 08:00:00 17:00:00 t

2026-10-08 01:40:13.199523+00	2026-10-26	\N	\N	\N	2026-10-08 01:40:13.199523+00
d31c7117-abb0-4db0-9ec3-bcf665f9a0aa
5cb00871-f5ef-4ccf-bd61-4746909237e2	2	08:00:00	17:00:00	t
2026-10-08 01:40:13.199523+00	2026-10-27	\N	\N	\N	2026-10-08 01:40:13.199523+00
cb0c7605-a94a-4e04-8c61-6c0d3ebb0967
5cb00871-f5ef-4ccf-bd61-4746909237e2	3	08:00:00	17:00:00	t
2026-10-08 01:40:13.199523+00	2026-10-28	\N	\N	\N	2026-10-08 01:40:13.199523+00
22050d84-172f-479f-af9a-78b70419e9fd
5cb00871-f5ef-4ccf-bd61-4746909237e2	5	08:00:00	17:00:00	t
2026-10-08 01:40:13.199523+00	2026-10-30	\N	\N	\N	2026-10-08 01:40:13.199523+00
a03cd237-7c85-4fd5-8b23-6d0093adb30f
5cb00871-f5ef-4ccf-bd61-4746909237e2	6	08:00:00	17:00:00	t
2026-10-08 01:40:13.199523+00	2026-10-31	\N	\N	\N	2026-10-08 01:40:13.199523+00
be7b4593-9deb-46e7-9787-86cd3ae0738d
5cb00871-f5ef-4ccf-bd61-4746909237e2	4	08:00:00	17:00:00	t
2026-10-08 01:40:13.199523+00	2026-10-29	\N	\N	\N	2026-10-08 01:40:13.199523+00
\.

--
-- Data for Name: customer_service_booking_alert_state; Type: TABLE DATA; Schema: public; Owner: postgres
--

COPY "public"."customer_service_booking_alert_state" ("customer_phone", "service_key", "is_threshold_active", "alert_cycle", "updated_at") FROM stdin;
09926067712	id:e5742f78-e0f3-4c14-b4e5-9ad22b3ff85b	f	0	2026-10-06 17:07:33.654736+00
09578532927	id:97a9714b-20e6-4402-a3ee-4fcdefc1e9db	f	0	2026-09-28 06:55:34.251147+00
09926067712	id:e5cf562e-7122-440b-9483-033b78aca286	f	0	2026-10-06 17:07:33.654736+00
09926067712	id:e724f832-2a1f-4949-bf6f-ad72240d99ea	f	0	2026-10-06 17:07:33.654736+00
09926067712	id:ed2e91b9-1d71-4e29-814d-7b7507751569	f	0	2026-10-06 17:07:33.654736+00
09543227314	id:97a9714b-20e6-4402-a3ee-4fcdefc1e9db	f	0	2026-10-15 06:32:42.283661+00
09999999999	id:f01fd9c7-6a0f-4510-bf61-fd7469d1290a	f	0	2026-09-29

10:46:09.103228+00
09111111111 id:e5742f78-e0f3-4c14-b4e5-9ad22b3ff85b f 0 2026-09-29
10:46:24.884042+00
09507704723 id:61b18a4e-6804-4701-b035-b8e7913f7911 f 0
2026-09-28 07:46:55.597088+00
09507704723 id:f0132c1b-d630-4aa3-a56b-165ce0c408ec f 0
2026-09-28 07:46:55.597088+00
09926067712 id:f01fd9c7-6a0f-4510-bf61-fd7469d1290a f 0 2026-10-06
17:07:33.654736+00
09926067732 id:e5742f78-e0f3-4c14-b4e5-9ad22b3ff85b f 0 2026-09-28
08:58:26.921765+00
09926067732 id:97a9714b-20e6-4402-a3ee-4fcdefc1e9db f 0
2026-09-28 08:58:26.921765+00
09926067712 id:f8feeecc-86e2-4312-8798-ea5c144f1ad4 f 0 2026-10-06
17:07:33.654736+00
09543227314 id:e5742f78-e0f3-4c14-b4e5-9ad22b3ff85b f 1 2026-10-15
06:32:42.283661+00
09507704723 id:72a4c5f5-e5f1-4e71-b872-30a3615ac362 f 0
2026-10-06 17:07:33.654736+00
09542154664 id:1477338f-ec6e-4bf6-b916-b619cc58ac0d f 0 2026-09-29
10:41:26.216923+00
09507704723 id:b6af4e06-1d15-4f61-b7e8-77719cb355de f 0
2026-10-06 17:07:33.654736+00
09507704723 id:d9ecf096-df3b-47f5-a012-80d98aacf9f5 f 0 2026-10-06
17:07:33.654736+00
09242698505 id:97a9714b-20e6-4402-a3ee-4fcdefc1e9db t 1
2026-10-11 04:59:03.846712+00
09468033084 id:97a9714b-20e6-4402-a3ee-4fcdefc1e9db f 0
2026-10-11 05:39:40.891101+00
09242698505 id:e5742f78-e0f3-4c14-b4e5-9ad22b3ff85b f 1 2026-10-21
10:19:43.487597+00
09469962798 id:97a9714b-20e6-4402-a3ee-4fcdefc1e9db f 0
2026-10-06 17:07:33.654736+00
09242698505 id:e852fde6-f231-4b05-95b4-26e8d6455c97 f 0
2026-10-21 10:19:53.441669+00
09346930832 id:b950fc34-4de4-42b6-b3ff-5c0b352103db f 0 2026-10-06
17:07:33.654736+00
09128964387 id:97a9714b-20e6-4402-a3ee-4fcdefc1e9db f 0
2026-10-06 17:07:33.654736+00
09578759635 id:6a7f36bb-0ec8-47f4-bf32-54c09de9b4b0 f 0 2026-10-25
16:30:00.018859+00
09926067712 id:08eef9ae-0750-4ed4-ade6-35aed340eea5 f 0
2026-10-06 17:07:33.654736+00
09926067723 id:97a9714b-20e6-4402-a3ee-4fcdefc1e9db f 0
2026-10-07 00:22:30.886674+00

09999999999 id:97a9714b-20e6-4402-a3ee-4fcdefc1e9db f 0
2026-10-08 11:23:42.086301+00
09926067712 id:61b18a4e-6804-4701-b035-b8e7913f7911 f 0
2026-10-06 17:07:33.654736+00
09926067712 id:6a7f36bb-0ec8-47f4-bf32-54c09de9b4b0 f 0 2026-10-06
17:07:33.654736+00
09926067712 id:72a4c5f5-e5f1-4e71-b872-30a3615ac362 f 0
2026-10-06 17:07:33.654736+00
09926067712 id:97a9714b-20e6-4402-a3ee-4fcdefc1e9db f 0
2026-10-06 17:07:33.654736+00
09111111111 id:97a9714b-20e6-4402-a3ee-4fcdefc1e9db f 0
2026-09-29 10:45:31.142156+00
09999999999 id:3066e928-4557-488a-831f-f7e0e26ae11f f 0 2026-09-29
10:46:09.103228+00
09999999999 id:5160c7c2-4cf7-481a-8881-cdfea2519895 f 0 2026-09-29
10:46:09.103228+00
09999999999 id:935c4315-72a4-40a4-9a63-6487106bd0fb f 0
2026-09-29 10:46:09.103228+00
09999999999 id:9381d3bc-dadb-4e65-9b83-97539b9df723 f 0
2026-09-29 10:46:09.103228+00
09999999999 id:951731dd-8412-46b9-8058-7348ea20b9ef f 0
2026-09-29 10:46:09.103228+00
09123456789 id:97a9714b-20e6-4402-a3ee-4fcdefc1e9db f 0
2026-10-11 05:53:13.895005+00
09999999999 id:aad80fe2-5bcc-4a22-864b-8e6fc00531bd f 0 2026-09-29
10:46:09.103228+00
09999999999 id:baaf0e56-887d-41c4-b78a-355e5509bc9c f 0
2026-09-29 10:46:09.103228+00
09999999999 id:e5742f78-e0f3-4c14-b4e5-9ad22b3ff85b f 0 2026-09-29
10:46:09.103228+00
09926067712 id:ab3c3ba9-415a-4f0e-9a42-bffb7d1903c1 f 0 2026-10-06
17:07:33.654736+00
09926067712 id:b6af4e06-1d15-4f61-b7e8-77719cb355de f 0
2026-10-06 17:07:33.654736+00
09578759635 id:d9ecf096-df3b-47f5-a012-80d98aacf9f5 f 0 2026-10-25
16:30:00.018859+00
09578759635 id:f0132c1b-d630-4aa3-a56b-165ce0c408ec f 0
2026-10-25 16:30:00.018859+00
09242698505 id:9bac85e5-f254-43ee-a36d-384b25fcae83 f 0
2026-10-13 12:25:46.398361+00
09094291771 id:97a9714b-20e6-4402-a3ee-4fcdefc1e9db f 0
2026-10-14 08:15:10.905783+00
09242698505 id:c18718c3-3aa9-40f6-8907-3a7922a07922 f 0
2026-10-14 09:33:07.787489+00
\.

--
-- Data for Name: notifications; Type: TABLE DATA; Schema: public; Owner: postgres
--

COPY "public"."notifications" ("id", "type", "title", "message", "appointment_id", "is_read", "created_at") FROM stdin;
86dc4384-a01e-4c4f-b14a-5c2dfebafcae	service_arrival	Customer expected: FRM-577L8T	Apolo Erikson is scheduled to arrive for service now.	dc1c4a1c-1532-4d25-bbd5-b7d8793de6e9	f	2026-10-22 00:00:00.020646+00
64a67890-58c6-4b5c-b5eb-3a021c86cba4	new_appointment	New booking FRM-6NLQEM	james patac booked Change Oil, Carburetor / FI Cleaning, Electrical Diagnostics, Engine Overhaul, Preventive Maintenance Service (PMS), Tire Change & Balancing, Suspension Tuning / Rebuild, Brake Service, Motorcylce Tune-Up, Cooling System Service, Radiator Cleaning, Fork Seal Replacement, Steering Bearing Replacement on 2026-09-28. Continued across 3 shop days.	2334d5bb-94f4-4889-8a06-0caee9004790	t	2026-09-22 22:54:19.20688+00
7cc95602-4033-421c-86a5-11d21d46b502	cancelled_appointment	Cancelled FRM-6NLQEM	james patac cancelled their Sep 28, 2026 appointment.	2334d5bb-94f4-4889-8a06-0caee9004790	t	2026-09-22 22:55:05.511671+00
32bcb8f6-f5d0-4c6c-af31-231a62f158e9	service_arrival	Customer expected: FRM-S32DGA	rafa rafa is scheduled to arrive for service now.	c2948c1c-8f78-441c-bcc7-1cfa03c0718b	t	2026-09-23 17:22:09.54658+00
6b0461b8-c19e-495c-82cb-696c1bf5947f	cancelled_appointment	Cancelled FRM-6VWY2Z	Ana Luz cancelled their Oct 1, 2026 appointment.	6a347f55-d34f-42ff-9a73-18f6f0830092	t	2026-09-27 11:43:30.005031+00
db503331-c1d2-41fb-9630-76d25f0dffde	reschedule_request	Reschedule request FRM-3JWTZU	lawrence l fuerzas requested Sep 22, 2026 at 08:00 AM. New reference: FRM-CAAFST. Reason: mao rani free time nako	545503a4-1416-4f5f-83ba-ebc50f5912d7	t	2026-09-18 02:13:27.675559+00
a9d5a675-13e9-4439-9c2f-65a451762bdf	rescheduled_appointment	Rescheduled booking FRM-CAAFST	lawrence l fuerzas rescheduled from FRM-3JWTZU to FRM-CAAFST. Reason: mao rani free time nako	b7424607-9e7d-4fd4-ae4e-73dba6c26b21	t	2026-09-18 02:15:34.993525+00
ca41a94d-d4b6-4555-8d03-fe1fbc2db268	new_appointment	New booking FRM-S32DGA	rafa rafa booked Chain & Sprocket Replacement on 2026-09-23.	c2948c1c-8f78-441c-bcc7-1cfa03c0718b	t	2026-09-21 02:39:06.066492+00
571b6cd0-3c00-490e-b418-563e6d82628a	new_appointment	New booking FRM-F98X7B	rafa rafa booked Change Oil, Brake Service on 2026-09-28.	64fe7d0c-3604-4dc4-a072-a3344c02e337	t	2026-09-21 07:18:00.682407+00

6d38ea69-05ff-44d1-b6d7-8125b56254ed new_appointment New booking FRM-R53RU4 SFHGHJ ASGYDFYUGH booked Change Oil on 2026-09-28. bb9e341a-1d2f-433b-ae2a-119cba575564 t 2026-09-22 02:27:08.631764+00
13b8faa3-823d-4473-b215-c919b398f187 reschedule_request Reschedule request FRM-R53RU4 SFHGHJ ASGYDFYUGH requested Sep 29, 2026 at 08:00 AM. New reference: FRM-QF9ZK9. Reason: free time bb9e341a-1d2f-433b-ae2a-119cba575564 t 2026-09-22 02:30:13.151926+00
ebbdab9c-583a-43dc-81ea-042363b3e75d cancelled_appointment Cancelled FRM-R53RU4 SFHGHJ ASGYDFYUGH cancelled their Sep 28, 2026 appointment. bb9e341a-1d2f-433b-ae2a-119cba575564 t 2026-09-22 02:33:56.616285+00
ec131dba-3891-4e35-ba23-5d121fe7bc96 new_appointment New booking FRM-5YVTRB Juann Dela Luz booked Motorcycle Tune-Up, Motorcylce Tune-Up, Clutch Adjustment/Repair on 2026-09-25. 33fbda0f-318f-41fd-8eac-328e296ea0d3 t 2026-09-22 03:18:49.104551+00
043dc5a7-0310-447b-ad71-dbc2d7578087 reschedule_request Reschedule request FRM-5YVTRB Juann Dela Luz requested Sep 24, 2026 at 11:30 AM. New reference: FRM-5E6MYB. Reason: Busy 33fbda0f-318f-41fd-8eac-328e296ea0d3 t 2026-09-22 03:23:23.571084+00
9139cc3c-8f07-474c-a7ce-a0f4efc254c8 reschedule_request Reschedule request FRM-F98X7B rafa rafa requested Sep 25, 2026 at 08:00 AM. New reference: FRM-E6Y7KF. Reason: free 64fe7d0c-3604-4dc4-a072-a3344c02e337 t 2026-09-22 10:38:36.972239+00
c66beaeb-25b1-4f74-8af6-b9fededcc4d3 rescheduled_appointment Rescheduled booking FRM-E6Y7KF rafa rafa rescheduled from FRM-F98X7B to FRM-E6Y7KF. Reason: free 147297a6-c092-4337-ab4e-2c86b7848da9 t 2026-09-22 15:11:44.983303+00
a9ae575f-fbcc-4c5c-aea9-a9858a74c258 service_arrival Customer expected: FRM-E6Y7KF rafa rafa is scheduled to arrive for service now. 147297a6-c092-4337-ab4e-2c86b7848da9 t 2026-09-25 00:00:08.212567+00
c00cbae9-79f6-4d96-a962-4a6c35404ae1 new_appointment New booking FRM-746ETT ashley chiu booked Chain & Sprocket Replacement on 2026-09-29. 6957127d-9bdd-4173-a856-2f08819ce87f t 2026-09-25 09:39:41.309607+00
e12f0d5d-9421-47f0-8b15-4c8f60899ae3 new_appointment New booking FRM-6VWY2Z Ana Luz booked Motorcycle Tune-Up, Cooling System Service, Radiator Cleaning, Clutch Adjustment/Repair on 2026-10-01. Continued across 2 shop days. 6a347f55-d34f-42ff-9a73-18f6f0830092 t 2026-09-24 10:50:58.67746+00
4af62239-fdba-41cc-b966-34d63bf98499 service_completion Service duration ended: FRM-E6Y7KF The expected service duration for rafa rafa has ended. 147297a6-c092-4337-ab4e-2c86b7848da9 t 2026-09-25 12:18:14.034031+00
4cdb7e2f-136f-4fea-9107-7685b2d155c4 new_appointment New booking

FRM-NNQNP8 Glendo Navarez booked Change Oil on 2026-09-28.
2a2de84e-e1d7-42e7-a36d-45905d69df12 t 2026-09-25 23:05:41.538332+00
48c01c79-61e9-4f0a-b366-a6d515e762ed service_arrival Customer expected: FRM-NNQNP8 Glendo Navarez is scheduled to arrive for service now. 2a2de84e-e1d7-42e7-a36d-45905d69df12 t 2026-09-28 05:00:00.030789+00
4eccae64-73ce-4bf4-a4cb-a94763487cac new_appointment New booking FRM-XUTXNU Lily Cruz booked Motorcycle Tune-Up, Suspension Tuning / Rebuild, Radiator Cleaning on 2026-09-29.
2d1379ce-21a4-459a-a1fa-97b2f7cc75db t 2026-09-26 06:43:41.257218+00
6cde70de-1009-404d-9ac6-971c1b1e344e new_appointment New booking FRM-ZTUG43 Lara Jessa Apolona booked Change Oil on 2026-10-05.
a26950f7-62f0-439a-8f3b-07fa6a6c50d6 t 2026-09-28 07:16:07.043301+00
83043789-5b67-4e6b-96e9-45d7747cbcec new_appointment New booking FRM-MPMHT9 Christian Josi Apolona booked Electrical Diagnostics FI on 2026-09-30. 6014afba-21ae-402c-a687-150730e547ea t 2026-09-27 05:01:55.179586+00
ddb0afeb-d56a-4660-99e0-866c2330b7ab rescheduled_appointment Rescheduled booking FRM-EVXKHD Lara Jessa Apolona rescheduled from FRM-ZTUG43 to FRM-EVXKHD. Reason: free
30c80b92-c789-40fc-936d-5f6567091423 t 2026-09-28 11:38:09.628656+00
e845b120-a027-44cd-9574-6976c0f58394 reschedule_request Reschedule request FRM-ZTUG43 Lara Jessa Apolona requested Oct 9, 2026 at 08:00 AM. New reference: FRM-EVXKHD. Reason: free
a26950f7-62f0-439a-8f3b-07fa6a6c50d6 t 2026-09-28 08:59:38.347109+00
ca533f8a-018e-48cd-81f8-51441f9da248 service_arrival Customer expected: FRM-XUTXNU Lily Cruz is scheduled to arrive for service now.
2d1379ce-21a4-459a-a1fa-97b2f7cc75db t 2026-09-29 00:30:00.021914+00
7c1d2ff2-93a4-4447-99a4-15847c1865e6 service_arrival Customer expected: FRM-746ETT ashley chiu is scheduled to arrive for service now.
6957127d-9bdd-4173-a856-2f08819ce87f t 2026-09-29 02:00:00.058794+00
6ab60877-218d-46a5-b23f-c0e414a23ab6 new_appointment New booking FRM-P5ZX4N Ashley L Chiu booked Change Oil, Brake Service on 2026-10-15. 3eac3b68-b97b-46f0-a8eb-ad913f8bc8c6 t 2026-09-29 06:07:47.742787+00
2363e553-d89a-4bb8-907e-8f54e2a9e573 service_completion Service duration ended: FRM-XUTXNU The expected service duration for Lily Cruz has ended. 2d1379ce-21a4-459a-a1fa-97b2f7cc75db t 2026-09-29 07:51:00.028003+00
dd0d56e1-47cc-4aff-af60-e6c04715f7d1 service_completion Service duration ended: FRM-746ETT The expected service duration for ashley chiu has ended. 6957127d-9bdd-4173-a856-2f08819ce87f t 2026-09-29 07:40:00.013955+00
cd4e4501-56c4-426f-a689-1af956df996c service_arrival Customer expected: FRM-MPMHT9 Christian Josi Apolona is scheduled to arrive for service now.

6014afba-21ae-402c-a687-150730e547ea t 2026-09-30 00:00:00.039818+00
0c70778e-5f92-4957-9571-a3f2312d74f9 service_completion Service duration ended: FRM-MPMHT9 The expected service duration for Christian Josi Apolona has ended. 6014afba-21ae-402c-a687-150730e547ea t 2026-09-30 02:50:00.022722+00
011fd0a2-b76f-4f9e-81cf-397fbbf09cd4 new_appointment New booking FRM-AQY69P Joshua Swete Banawan booked Change Oil on 2026-10-12. eda6b8a7-6dc1-4584-8ad5-e918438c0a05 t 2026-09-30 00:44:02.356662+00
dc396090-b3b7-4506-b00f-9027c731fd5e rescheduled_appointment Rescheduled booking FRM-ULM8GG Ash L Chiu rescheduled from FRM-4RKQTN to FRM-ULM8GG. Reason: free
de1e21dc-53e4-4362-8898-3bd1da2d67fc t 2026-09-30 05:01:21.761562+00
8222072d-d208-451b-806a-68afff124dda multiple_active_service_bookings 2 active Brake Service bookings for Ash L Chiu Ash L Chiu (09543227314) has 2 active bookings for Brake Service: Oct 5, 2026 at 8:00 AM; Oct 15, 2026 at 8:30 AM. de1e21dc-53e4-4362-8898-3bd1da2d67fc t 2026-09-30 05:01:21.761562+00
d217118f-855d-485e-9f54-124efa2741ff new_appointment New booking FRM-4RKQTN Ash L Chiu booked Brake Service on 2026-10-12.
2ce47e77-9cfc-49b8-9710-c3b33294eb54 t 2026-09-30 04:51:30.842347+00
628466d1-e842-43f8-92da-495c3643d264 multiple_active_service_bookings 2 active Brake Service bookings for Ash L Chiu Ash L Chiu (09543227314) has 2 active bookings for Brake Service: Oct 12, 2026 at 1:00 PM; Oct 15, 2026 at 8:30 AM. 2ce47e77-9cfc-49b8-9710-c3b33294eb54 t 2026-09-30 04:51:30.842347+00
2ca39f4f-f9b7-4e78-8bc0-2325f5021e14 reschedule_request Reschedule request FRM-4RKQTN Ash L Chiu requested Oct 5, 2026 at 08:00 AM. New reference: FRM-ULM8GG. Reason: free
2ce47e77-9cfc-49b8-9710-c3b33294eb54 t 2026-09-30 05:01:14.554022+00
b7c59277-b8ae-4fb5-b801-021d4a2f2460 new_appointment New booking FRM-HWAVDJ Erikson Apolona booked Change Oil on 2026-10-05.
617073f7-8cfa-4f3f-8352-aff1a76da780 t 2026-09-30 05:03:54.222561+00
f2e47efb-db72-4ee2-a69a-3c0c176b3639 service_arrival Customer expected: FRM-ULM8GG Ash L Chiu is scheduled to arrive for service now.
de1e21dc-53e4-4362-8898-3bd1da2d67fc t 2026-10-05 00:00:00.092288+00
e7d5ad1a-f386-42ee-aa52-914c28617446 service_arrival Customer expected: FRM-HWAVDJ Erikson Apolona is scheduled to arrive for service now. 617073f7-8cfa-4f3f-8352-aff1a76da780 t 2026-10-05 00:00:00.092288+00
1a7c1483-d593-4c47-850a-6858ee840b5d service_completion Service duration ended: FRM-ULM8GG The expected service duration for Ash L Chiu has ended. de1e21dc-53e4-4362-8898-3bd1da2d67fc t 2026-10-06 14:48:00.046784+00
f807b6fb-c5ca-4ccb-8509-0927de962641 service_completion Service

duration ended: FRM-HWAVDJ The expected service duration for Erikson Apolona has ended. 617073f7-8cfa-4f3f-8352-aff1a76da780 t 2026-10-06 14:18:00.05174+00
2382a442-21a6-4c8f-b525-d5825aa8f2fe new_appointment New booking FRM-2LYRZC Erik Apolona booked Change Oil on 2026-10-12. c5f4d85a-7617-42c7-84db-941d333879bf t 2026-10-07 00:21:36.678803+00
cc2d431c-2273-40e5-89af-99c2ca13f744 service_arrival Customer expected: FRM-EVXKHD Lara Jessa Apolona is scheduled to arrive for service now. 30c80b92-c789-40fc-936d-5f6567091423 t 2026-10-09 00:00:00.019729+00
35027590-6e64-4ffa-acb5-fc8abb04b253 new_appointment New booking FRM-577L8T Apolo Erikson booked Change Oil on 2026-10-22. dc1c4a1c-1532-4d25-bbd5-b7d8793de6e9 t 2026-10-09 08:54:58.290663+00
3c55ec5a-a9d5-4925-bd53-53c8e6225b12 service_completion Service duration ended: FRM-EVXKHD The expected service duration for Lara Jessa Apolona has ended. 30c80b92-c789-40fc-936d-5f6567091423 t 2026-10-09 09:56:00.073244+00
2fffee3a-f091-4753-91cd-6a5c99d21a3c new_appointment New booking FRM-ATZJXT Apo Erik booked Brake Service on 2026-10-27. af37178b-4dfd-455f-8d90-2f7d3f4b4304 t 2026-10-09 08:57:32.115662+00
44f6ff7e-4353-40e4-8021-6cdc86f2d231 new_appointment New booking FRM-APD3BN Apo Erikson booked Change Oil on 2026-10-30. cab87474-af88-46f2-8e66-7acd2e5d2baa t 2026-10-11 05:39:06.383071+00
bb429244-23a2-413d-826d-043f2ca233d8 multiple_active_service_bookings 2 active Change Oil bookings for Apo Erikson Apo Erikson (09242698505) has 2 active bookings for Change Oil: Oct 22, 2026 at 8:00 AM; Oct 30, 2026 at 8:00 AM. cab87474-af88-46f2-8e66-7acd2e5d2baa t 2026-10-11 05:39:06.383071+00
93f89040-3872-47d9-b0aa-528fc4ce27df service_arrival Customer expected: FRM-AQY69P Joshua Swete Banawan is scheduled to arrive for service now. eda6b8a7-6dc1-4584-8ad5-e918438c0a05 t 2026-10-12 00:00:00.011816+00
38c1520d-7a2c-4036-9740-c6b6537a1fa7 new_appointment New booking FRM-XQ2XNC Erikson Apolona booked Change Oil, Brake Service on 2026-10-19. e8900d91-8ae4-4bd6-9a33-eb818faa92a2 t 2026-10-13 11:14:18.29032+00
133a7637-64df-44aa-9a5c-8ee64955659d multiple_active_service_bookings 3 active Change Oil bookings for Erikson Apolona Erikson Apolona (09242698505) has 3 active bookings for Change Oil: Oct 19, 2026 at 8:00 AM; Oct 22, 2026 at 8:00 AM; Oct 30, 2026 at 8:00 AM. e8900d91-8ae4-4bd6-9a33-eb818faa92a2 t 2026-10-13 11:14:18.29032+00
0f0f9bfa-d154-4863-8bea-04abeb6e16a3 multiple_active_service_bookings 2 active Brake Service bookings for Erikson Apolona Erikson Apolona (09242698505) has 2 active bookings for Brake Service: Oct 19, 2026 at 8:00 AM; Oct 27, 2026 at 8:00 AM. e8900d91-8ae4-4bd6-9a33-eb818faa92a2 t

2026-10-13 11:14:18.29032+00
792caf85-daed-4dbb-b2ea-5bc9e82cb1fb	new_appointment	New booking	FRM-ZBM5AB	Erik Eri booked Tire Change on 2026-10-30.
cfe5e95c-d7d8-4ba5-b6e7-9020b664a0fc	t	2026-10-13 12:24:16.558559+00
d698dcde-fcb6-4878-ad46-3a3dbe28d03c	service_completion	Service duration ended: FRM-AQY69P	The expected service duration for Joshua Swete Banawan has ended.	eda6b8a7-6dc1-4584-8ad5-e918438c0a05	t	2026-10-13 13:26:00.12237+00
a0ed8759-35b5-49d2-9efd-2bcf572610e4	new_appointment	New booking	FRM-XNS9VJ	Erik Eri booked Handlebar Switch Replacement on 2026-10-19.
34b4dba4-8ed5-42b8-8d27-ca469116c5d1	t	2026-10-14 09:32:01.545076+00
d4f3f236-2a56-4f4a-b8db-2dc7ced8a84f	cancelled_appointment	Cancelled	FRM-XNS9VJ	Erik Eri cancelled their Oct 19, 2026 appointment.
34b4dba4-8ed5-42b8-8d27-ca469116c5d1	t	2026-10-14 09:33:07.787489+00
834d0e6b-7f89-4a04-945d-97278ebc2395	service_arrival	Customer expected: FRM-P5ZX4N	Ashley L Chiu is scheduled to arrive for service now.
3eac3b68-b97b-46f0-a8eb-ad913f8bc8c6	t	2026-10-15 00:30:00.033915+00
e96d1a05-7031-4dac-b653-ab9a2d5fa758	service_completion	Service duration ended: FRM-P5ZX4N	The expected service duration for Ashley L Chiu has ended.	3eac3b68-b97b-46f0-a8eb-ad913f8bc8c6	t	2026-10-15 04:21:00.012033+00
27773a1e-66bf-4172-b0cd-1a9f7d556297	new_appointment	New booking	FRM-JKNK84	Wukong Sun booked Chain Replacement on 2026-10-19.
6c0de99e-7522-4f4e-ad11-0913682e3ae3	t	2026-10-15 07:08:09.579389+00
20fa85c9-9988-4370-b3c0-74954632f5cc	service_arrival	Customer expected: FRM-JKNK84	Wukong Sun is scheduled to arrive for service now.
6c0de99e-7522-4f4e-ad11-0913682e3ae3	t	2026-10-19 00:00:00.011757+00
00409aed-6514-4337-89a5-a78911b89d11	service_arrival	Customer expected: FRM-XQ2XNC	Erikson Apolona is scheduled to arrive for service now.	e8900d91-8ae4-4bd6-9a33-eb818faa92a2	t	2026-10-19 00:00:00.011757+00
\.

--
-- Data for Name: customer_service_booking_alerts; Type: TABLE DATA; Schema: public; Owner: postgres
--

COPY "public"."customer_service_booking_alerts" ("id", "customer_phone", "service_key", "alert_cycle", "active_appointment_ids", "notification_id", "created_at") FROM stdin;
1e3c5cf5-1d2a-4339-aabe-feefda481265	09543227314

id:e5742f78-e0f3-4c14-b4e5-9ad22b3ff85b 1 {2ce47e77-9cfc-49b8-9710-c3b
33294eb54,3eac3b68-b97b-46f0-a8eb-ad913f8bc8c6}
628466d1-e842-43f8-92da-495c3643d264 2026-09-30 04:51:30.842347+00
613a8e01-8cf3-44d9-bca3-546ca676c325 09543227314
id:e5742f78-e0f3-4c14-b4e5-9ad22b3ff85b 1 {3eac3b68-b97b-46f0-a8eb-ad9
13f8bc8c6,de1e21dc-53e4-4362-8898-3bd1da2d67fc}
8222072d-d208-451b-806a-68afff124dda 2026-09-30 05:01:21.761562+00
17411798-8c6d-4958-a17c-26b8b9a3f02f 09242698505
id:97a9714b-20e6-4402-a3ee-4fcdefc1e9db 1 {53021b0d-5241-4b5c-ab7f-5a
cd0bbc9823,dc1c4a1c-1532-4d25-bbd5-b7d8793de6e9} \N 2026-10-11
04:59:03.846712+00
07f3a24e-550f-4cc6-a3ca-96a5a74972ea 09242698505
id:97a9714b-20e6-4402-a3ee-4fcdefc1e9db 1 {4512cd6e-96e6-443e-8192-1
b74c50b8eb1,dc1c4a1c-1532-4d25-bbd5-b7d8793de6e9} \N 2026-10-11
05:24:49.931527+00
cfa3125f-36e9-440b-a257-203288f29d69 09242698505
id:97a9714b-20e6-4402-a3ee-4fcdefc1e9db 1 {cab87474-af88-46f2-8e66-7a
cd2e5d2baa,dc1c4a1c-1532-4d25-bbd5-b7d8793de6e9}
bb429244-23a2-413d-826d-043f2ca233d8 2026-10-11 05:39:06.383071+00
339b2f34-54e0-45fd-bbc9-f4ee8efa46f0 09242698505
id:97a9714b-20e6-4402-a3ee-4fcdefc1e9db 1 {cab87474-af88-46f2-8e66-7a
cd2e5d2baa,dc1c4a1c-1532-4d25-bbd5-b7d8793de6e9,e8900d91-8ae4-4bd6-9
a33-eb818faa92a2} 133a7637-64df-44aa-9a5c-8ee64955659d 2026-10-13
11:14:18.29032+00
fd6fc4b6-24d7-4b3f-b085-ea9c372e6a0d 09242698505
id:e5742f78-e0f3-4c14-b4e5-9ad22b3ff85b 1 {af37178b-4dfd-455f-8d90-2f7d
3f4b4304,e8900d91-8ae4-4bd6-9a33-eb818faa92a2}
0f0f9bfa-d154-4863-8bea-04abeb6e16a3 2026-10-13 11:14:18.29032+00
\.

--
-- Data for Name: motorcycle_catalog; Type: TABLE DATA; Schema: public;
Owner: postgres
--

COPY "public"."motorcycle_catalog" ("id", "brand", "model", "is_active",
"is_archived", "sort_order", "created_at", "updated_at", "archived_at",
"engine_cc") FROM stdin;
6dc4a761-135b-426f-971d-4f48b9253153 Triumph Rocket 3 t f 852
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N
2458.0
78abfb4f-c60b-4a9a-908f-2baf112fbfd0 Triumph Rocket III t f 851
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N
2294.0

39405ede-025d-4796-bb66-7115bd7568e6 Triumph Thruxton 1200 t f 855 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 1200.0
f210db81-dce9-4e71-902d-c4ae97640c06 Triumph Thruxton 900 t f 854 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 865.0
8a5f5a39-b907-4cc5-91b0-50c08bad563f Triumph Tiger 1200 t f 846 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 1160.0
e8012f25-0bdb-46b6-b700-bcc6324c1ecb Triumph Tiger 900 t f 845 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 888.0
ed63677b-7995-41cd-82c6-701c39e4e6bf Triumph Tiger Sport 1050 t f 850 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 1050.0
ce233c67-7ae7-44d7-98cc-78898b2e9780 Triumph Tiger Sport 660 t f 848 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 660.0
1457eb55-2c0e-41ab-8330-fa599902425d Triumph Trident 660 t f 853 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 660.0
a0f31474-e297-49c7-aa3b-2c9c75698c57 Triumph Tiger Explorer1200 f f 847 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
7c9a7f54-2884-423c-b594-3c811f085ec0 Triumph Tiger Sport850 f f 849 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
1df6d165-5b63-4714-94c2-d375295628f9 Triumph SprintST f f 856 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
38833083-c792-41d2-978f-41724fce568d Triumph SprintGT f f 857 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
41a9daec-af7a-4745-96ba-0e08105e0185 Triumph Daytona1200 f f 858 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
2d1c052b-8282-41d7-b16a-5602ce21fe8b Aprilia RS125 f f 860 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
3495ad4e-c3fd-4dbc-8e6c-90ace78d2200 Aprilia RS250 f f 861 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
9c7d9e77-a23f-49d1-a87b-328a254f1ccd Aprilia RS660 f f 862 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
bfb72d6d-1d1f-4618-87e2-0f822610686e Aprilia RS4 125 f f 863 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
7176c8f2-a21c-42d3-b9ef-db27d9d4fe4b Aprilia RSV1000 f f 864 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
23332577-93b7-4664-a1e9-fe029085be3a Aprilia RSV4 f f 865

2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
232991f5-1533-42c3-b808-a4281ef0b104 Aprilia Tuono125 f f 866
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
9afb7117-b2f2-46ca-ba74-95e683095c61 Aprilia Tuono660 f f 867
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
5f97e538-3f9b-4ff4-9ddd-38d2b836b839 Aprilia Tuono V4 f f 868
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
8aa81bbd-facc-4eac-bc48-3de25eb8f03e Aprilia Tuono V4 Factory f f
869 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N
\N
179f9b4b-6e54-417f-8c08-6dc950b246b2 Aprilia Shiver750 f f 870
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
f49ffc6d-2842-477a-bcc2-484505fc2aaf Aprilia Shiver900 f f 871
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
ec79c8de-12b1-4b7c-9c8c-610ace876544 Aprilia Dorsoduro750 f f 872
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
d192a9c7-d3fa-4b6e-a430-82d33a7218f0 Aprilia Dorsoduro900 f f 873
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
8768fdc6-c47d-4a60-82af-0351517c17de Aprilia Mana850 f f 875
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
8c0fe289-c7c3-4a37-99cb-a9f341d0206e Honda CB150X t f 68
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
149.0
134a15ca-3b96-4a08-ac4b-3b1ce267d360 Honda ADV 150 t f 45
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
149.0
176f2675-1c01-416c-b43e-b04d1c0c7ed1 Honda ADV 160 t f 46
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
157.0
e7944c68-2eb7-4595-8893-ac430dbf767b Honda Africa Twin (1084cc) t f
94 2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
1084.0
f0c91070-468e-4147-8a3e-1c8414223bf9 Honda Africa Twin (998cc) t f
0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
998.0
5984de8d-6b03-4e03-abf1-ce8d126f4ef4 Honda AirBlade 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
024b1f37-dc25-4a95-838a-db8c0ad82bd2 Honda AirBlade 150 t f 38
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
149.0
4fe9529e-8423-4b91-beaa-a0ed9b1d1986 Honda AirBlade 160 t f 39
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
157.0
ebd0f0bb-47e4-4dc0-b895-34855d550317 Honda Beat (110cc) t f 23

2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
110.0
71c9ba05-107c-46d7-9dcf-5a6ce5932978 Honda Beat (125cc) t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
fd2489c1-afd5-48b4-88fd-925b37a59dcb Honda CB1000R t f 91
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
998.0
a77674b6-b64b-4b01-acb6-4314ad6c983f Honda CB150R t f 66
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
149.0
59433d48-a5db-4b39-b641-4d64753087f4 Honda CB500F t f 88
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
471.0
2f6573ee-bb8b-4395-a175-9e093c18fd58 Honda CB500X t f 89
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
471.0
fce0a4fb-9ed5-4e8e-ac59-2ae60a84c598 Honda CB650R t f 90
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
649.0
c15a8e81-2524-4d2e-8c23-f7e924bef5d3 Honda CBR1000RR t f 74
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
999.0
779ba4f5-4e93-46ad-ba58-df7aa3e47452 Honda CBR1000RR-R Fireblade
t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00
\N 999.0
70b3e765-3222-43ba-9a40-380812007581 Honda CBR150R t f 69
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
149.0
1147c576-d1d9-4570-96c7-4a29711cf40b Honda CBR500R t f 71
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
471.0
c33ad2d6-c681-435f-9ffe-210a34afcbaa Honda CBR650R t f 73
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
649.0
4a5341ce-fe5a-4d29-81ab-216551fa8381 Honda CL500 t f 80
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
471.0
e9d086a3-cbb9-4eaa-9333-ab2c9d536dfc Honda Click 110i t f 30
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
108.0
cffaba56-d354-44c0-a9ad-67c62e3ec158 Honda Click 125i t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
b9303160-fecc-4e4f-9296-f55a2a025e5e Honda Click 150i t f 33

2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
150.0
73584e28-7d5b-481b-9e6f-a88b2702c19a Honda Click 160 t f 34
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
157.0
3e6e6128-3703-4b04-9fec-a099ab248fba Honda CRF150L t f 84
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
149.0
4916bd2c-1244-4564-89a8-aa64687850eb Honda CRF250L t f 85
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
249.0
898d33d7-2a97-4fb2-8ff0-17751611d516 Honda CRF300 Rally t f 87
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
286.0
5384ce94-4baa-4b56-a091-212f3d8caea0 Honda CRF300L t f 86
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
286.0
254dbd86-03cc-4d17-85e2-e8566503cceb Honda Dio 110 t f 40
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
109.0
80a9e764-6857-40de-89b9-8659cefa4f0e Honda Gold Wing t f 96
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
1833.0
b1775d21-4719-4256-b8c6-a45ac816aa2d Honda NC750X t f 92
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
745.0
e74b4cf8-f770-452f-a763-c16c6b9db26b Honda NX500 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
471.0
6fd39124-68c4-4d42-ba4a-baac8aabb526 Honda PCX 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
4cd3bb36-6a16-4f1a-90c9-40c5dd322677 Honda PCX 150 t f 47
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
149.0
d7982900-18b4-4cbd-9637-d7774e550ea5 Honda PCX 160 t f 48
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
157.0
45fdb49f-fe1d-4a83-883f-83a7e2658578 Honda Rebel 1100 t f 79
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
1084.0
ce0563b9-3c4a-46fd-92ed-39303f1adc81 Honda Rebel 300 t f 77
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
286.0
db043e90-e916-437d-9815-26dc4e316393 Honda Rebel 500 t f 78

2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
471.0
3bd20cdd-572c-4dc0-bcdf-4f83fcd2dbf0 Honda RS125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
93f5f4bb-c1db-421c-b6ae-5595cd5755bb Honda Supra GTR 150 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
149.0
56198728-cce4-49b9-a5ba-a0299491607d Honda TMX 125 t f 60
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
125.0
337e75b2-3584-45ff-be4e-5e63d2a265d3 Honda TMX Supremo t f 62
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
125.0
1df646f7-62ba-4b44-8f72-9b7765c7b2a9 Honda Wave 100 t f 0
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
100.0
94d7a7c4-9425-46aa-9465-009a1fd24679 Honda Wave 110 t f 1
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
110.0
3fd0d623-a965-4432-944d-d7ab1cb33d6a Honda Wave 125 t f 4
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
125.0
29309456-b6da-4a75-8713-e253141dab03 Honda Wave RSX t f 12
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
110.0
482ff94b-53e8-44a8-b49f-a5f419c96de0 Honda X-ADV t f 93
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
745.0
e5df2af2-facb-4068-be25-efb1d019b803 Honda XL750 Transalp t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
755.0
e0162088-d320-48be-aa4f-72578366d203 Honda XR150L t f 82
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
149.0
81254950-191e-4999-8a43-d28e4c98c5ae Honda XRM 110 t f 15
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
109.0
074c7e2c-37c3-420d-a503-428ff289e921 Honda XRM 125 t f 17
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
124.0
30dcf9d8-ab47-451a-8036-4321ea0a34b6 Yamaha Aerox 155 t f 137
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
155.0
f55822b1-9067-4dab-8083-c4b7b4793aac Yamaha Bolt t f 177

2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
942.0
4c6589e9-3671-4cad-8f94-b025099dfc85 Yamaha Fino t f 139
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
125.0
2f7c0153-e4e9-4ca9-b4e9-7fe4025e2068 Yamaha FZ1 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
998.0
db965c5a-f5b3-44b1-8aff-1dca343953bb Yamaha FZ150i t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
149.0
1b352999-4168-4c4b-a14d-e718f6d79856 Yamaha Lexi t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
ccc240db-7659-43d4-bef9-872020ea27f8 Yamaha Mio Fazzio t f 124
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
125.0
73f8e5c6-db7b-43ec-8fe8-b35e04d67d8f Yamaha Mio Gear t f 121
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
125.0
63fe7d5c-56dd-46a1-8729-1866e7f83eca Yamaha Mio Gravis t f 123
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
125.0
8365f6e9-4f60-46ff-aea2-2d3e1ad1dbf7 Yamaha Mio i125 t f 118
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
125.0
3c22fda9-6105-4d4c-85fd-7d9fcfb909d7 Yamaha Mio Soul i t f 116
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
115.0
c50d0b50-f110-4ff9-8179-aeb05a617d30 Yamaha Mio Soul i 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
f8732190-97fb-4ea1-b65b-2214c3d0ffb5 Yamaha Mio Sporty t f 114
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
115.0
3589f56c-5fe7-4c33-a6a5-be11d6388e21 Yamaha MT-03 t f 155
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
321.0
7d221aac-c24b-4c16-bf1d-53d73002c391 Yamaha MT-07 t f 156
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
689.0
ebe92cbd-1f22-4c8e-a8ad-c70a9db2ebb2 Yamaha MT-09 t f 157
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
890.0
fe2b442c-cb6c-4377-9fb7-61b0e37a1560 Yamaha MT-10 t f 158

2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
998.0
b72c37d1-1e22-4e15-8d28-af8540897b4a Yamaha MT-15 t f 154
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
155.0
0c545786-237f-473d-a348-f8f6fa7450d9 Yamaha NMAX t f 131
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
155.0
6d3a66be-6554-472e-9784-60a333e38281 Yamaha R15 t f 150
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
149.0
978f4930-3db9-4903-97f4-ac040e4d08ac Yamaha R15 V3 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
155.0
494cab90-34d0-4c58-8b5a-e2a9bf7eb040 Yamaha R15M t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
155.0
f26a95cb-d60b-4db7-91e1-96924c88178e Yamaha R3 t f 152
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
321.0
bc31dcf3-f5c6-4cbe-a619-86e7df761816 Yamaha Sight t f 111
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
115.0
5e5fc5f3-591c-4edc-b622-902b29e8fe50 Yamaha Sniper 135 t f 145
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
134.0
8dac619b-456d-4c3e-a6f8-b335bccee9a6 Yamaha Sniper 150 t f 146
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
150.0
cd494424-88d0-48e7-a581-0e40b516e41b Yamaha Sniper 155 t f 147
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
155.0
659cedc8-240d-4fbd-9c2f-d7da8a0d4700 Yamaha Tenere 700 t f 167
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
689.0
ceb7ed94-e996-4294-904e-ef6ffbdbcf12 Yamaha TMAX (530cc) t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
530.0
8fdb353a-7e81-42d4-8639-bd331b9f4c18 Yamaha TMAX (560cc) t f
176 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
560.0
91ac36a9-fa08-4917-8179-c79a0601e92e Yamaha Tracer 7 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
689.0
9093c360-cef2-46e2-a0e8-519c4aa930fa Yamaha Tracer 9 t f 0

2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
890.0
7740ddc7-57c2-492a-b7de-0f63967cff2e Yamaha V-Star 250 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
249.0
1f47397b-f637-4920-9ce8-4c3df7210c1a Yamaha XMAX t f 134
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
300.0
fd4447f8-955e-41e0-921c-f1577c0bd234 Yamaha XSR155 t f 172
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
155.0
d9683f0b-ff5c-4f30-88b1-59acb647676f Yamaha XSR700 t f 173
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
689.0
685b43a0-7024-4776-8751-99530d0d2edc Yamaha XSR900 t f 174
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
889.0
cd895894-a9a7-4d6b-80c3-6adad4fcc08c Yamaha YBR 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
8e599cc5-f6ef-4ac0-8178-88465a5f3f98 Yamaha YTX 125 t f 97
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
125.0
7cbf6d39-a96b-4ca5-a92c-d290d2ab01b7 Yamaha YZF-R1 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
998.0
3e7642f8-6b4e-4ad9-a97a-b4093578e553 Yamaha YZF-R6 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
599.0
6510e174-bb77-4654-b443-3aedaee3d248 Yamaha YZF-R7 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
689.0
fc3739dd-5ec0-4922-8714-5a9f5ffbb189 Suzuki Access t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
13be393b-a55a-4dc4-add4-2bfdf2ae62a7 Suzuki Address t f 203
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
113.0
e232307f-e75d-4759-8157-73ba14cc0402 Suzuki Avenis t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
c0110487-0e13-47c4-9859-1fe32c23e135 Suzuki Bandit 1200 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
1157.0
8394f76a-4f36-48a1-b9f0-0cd58f9429a8 Suzuki Bandit 1250 t f 0

2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
1255.0
419eddfe-de4a-4fae-ac4b-557d0ee1c135 Suzuki Bandit 600 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
599.0
be9656b1-c86b-469d-b2e8-9d1b9304985e Suzuki Bandit 650 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
656.0
cf57e0cb-ab50-4709-8981-cf43c9d78b6c Suzuki Best 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
092cf754-06bc-42c6-9f37-d2e762d39bdf Suzuki Boulevard C50 t f 228
 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
805.0
9626802d-b417-4b9b-8687-6917c391dd17 Suzuki Boulevard M109 t f
0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
1783.0
0820b910-e55f-4124-9c89-ab14d56646ad Suzuki Burgman 250 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
249.0
cf248e99-e266-47da-8f42-cca5076630d1 Suzuki Burgman 400 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
400.0
e1101832-29ec-4bac-b935-3e7de1e89e61 Suzuki Burgman 650 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
638.0
355911b1-7cb0-4d74-961b-9d6749132051 Suzuki Burgman Street t f
200 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
 124.0
a00eea24-245e-4666-8e53-6a6977ef2365 Suzuki Burgman Street 125 EX
t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00
\N 124.0
501472d8-4d86-478f-9acb-4d9f5762f4c7 Suzuki DR-Z125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
9b490daa-f73b-4498-bd75-fcab4ec6f854 Suzuki DR-Z400 t f 223
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
398.0
1d75e482-23d4-499d-adbe-7e208bc8c387 Suzuki DR150 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
149.0
306e2c34-07c4-423a-a202-7a6df1168203 Suzuki DR160 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
162.0
6da7b4dc-42d5-4c1e-bc28-164a989d6eb2 Suzuki DR200 t f 0

2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
199.0
7f8e9885-a07c-4ad1-875f-1df5036ae1ae Suzuki DR650 t f 224
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
644.0
24b78f25-fe7f-42b0-b3ad-b8255a9b8d43 Suzuki GD110 t f 192
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
113.0
ddd04f01-ce2c-4103-8dfd-9a3d12c5012a Suzuki Gixxer 155 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
155.0
1ccfedf5-4c5c-499f-a1fc-4d73e8c04d0e Suzuki Gixxer 250 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
249.0
ada8ae93-9981-4333-9347-1ca9ec0447db Suzuki Gixxer SF 155 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
155.0
bd436116-eb55-4664-97d5-49359c8d77c6 Suzuki Gixxer SF250 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
249.0
e64ee616-f51d-478d-ace0-15b0983c860a Suzuki GN125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
178f87f8-d491-4f74-8ff6-de3a0e1afc08 Suzuki GN160 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
162.0
bf2f52f2-44de-4c58-bf02-b2aaa5f09495 Suzuki GSX-8R t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
776.0
ff07d26d-999b-4f4e-8e71-4d9bafb19cd7 Suzuki GSX-8S t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
776.0
80aff18d-ecd8-404b-b101-5de636064532 Suzuki GSX-R1000 t f 216
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
999.0
a4407929-665b-4369-ae4a-42b504a58fc3 Suzuki GSX-R600 t f 214
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
599.0
4ab1557c-cba9-4469-bd3f-1fefad0ca39a Suzuki GSX-R750 t f 215
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
750.0
a9516445-7e54-4461-9e01-245bd2ded452 Suzuki GSX-S1000 t f 232
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
999.0
51fd4aae-7f6f-4647-9e7b-7bde5a4600d1 Suzuki GSX-S1000GT t f 0

2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
999.0
a7cb8baa-821d-401a-b764-f8aa80e8bd6d Suzuki GSX-S750 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
749.0
15dff857-ef4b-41ed-a5a3-c41f5ce09efe Suzuki Hayabusa (1299cc) t f 0
  2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
1299.0
bebfc720-6a05-4479-bc49-fe0668ab95ea Suzuki Hayabusa (1340cc) t f
230 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
 1340.0
c9684837-0c16-48fe-8e30-95bd857e28e2 Suzuki Hayate t f 205
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
113.0
ae8ed335-b9e0-4e6c-be2c-f3e9909de4a4 Suzuki Raider 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
2c93cfa3-b460-401a-b655-df212d1d9f2d Suzuki Raider J Crossover t f
191 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
 113.0
8eed2849-7851-450f-8d2c-3c88c8144cde Suzuki Raider R150 t f 187
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
147.0
8395b833-8231-4c30-aaf5-a780d54788b9 Suzuki Skydrive 125 t f 197
 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
124.0
bf689258-3404-489c-87bf-c2cddbf53570 Suzuki Skydrive Sport t f 199
 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
113.0
3f217ab5-c7f8-427c-914d-7938af4bb569 Suzuki Smash 110 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
113.0
43e47a54-daa5-4e13-bf48-01964c11d998 Suzuki Smash 115 t f 183
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N
115.0
5dd3d2d1-43a2-438c-8ee9-fca79c2265c8 Suzuki SV650 t f 221
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
645.0
d6a1ee7b-99cc-442b-a808-d4cf4648d5fe Suzuki V-Strom 1050 t f 220
 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
1037.0
47fe886d-5cdb-494d-857d-a577aab6cfa8 Suzuki V-Strom 160 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
162.0
5a340ad8-f907-4e6b-994c-5f045c5925ef Suzuki V-Strom 250 SX t f 0

2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
249.0
4163340a-e046-4056-aef7-f93ede0189b1 Suzuki V-Strom 650 t f 218
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
645.0
c2d11740-f394-45f6-a6b9-df53b2fbbbc2 Suzuki V-Strom 800DE t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
776.0
1f118894-177e-4cc4-840f-11579f3f491d Kawasaki Barako t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
175.0
83883e0f-a3af-4101-ab76-8848a89588bc Kawasaki Barako II t f 237
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
177.0
874499bb-357c-4a68-91b9-ae7e6eb0a7a3 Kawasaki Barako III FI t f
238 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
177.0
3d41c224-e45f-4a2c-b011-f245a7752da7 Kawasaki Curve t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
112.0
a2f53dd8-6059-42f4-bb25-225245159f70 Kawasaki ER-6f t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
649.0
d55af9dd-6bfd-4c5f-885c-a58cd97cf1a4 Kawasaki ER-6n t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
649.0
1b6ae0c7-501e-431b-a903-12cbf4d6041c Kawasaki Fury t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
767a3913-3c2a-4166-9448-640c7d2ee7c5 Kawasaki Fury RR t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
6c286266-b008-4313-8346-410942697a0b Kawasaki KLR650 t f 261
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
652.0
bef1b4af-09a1-4f09-af57-b75af8ebb2c2 Kawasaki KLX125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
e44586ea-b4fb-4110-b948-083894d65d28 Kawasaki KLX150 t f 252
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
144.0
8bb7ef99-b85b-4257-a490-d2991e014719 Kawasaki KLX150L t f 253
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
144.0
055ecc7e-5da1-43c7-b3b0-4caadac5d252 Kawasaki KLX230 t f 255

2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
233.0
88b65bba-c0c0-4fe9-b4a9-d42c3051bc54 Kawasaki Ninja 250 t f 262
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
249.0
5aa919af-f18a-4843-aa4a-17134bc8e1a3 Kawasaki Ninja 300 t f 264
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
296.0
ff7dac0c-cc91-4f51-855e-722f7ba7da19 Kawasaki Ninja 400 t f 265
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
399.0
d3c3b07c-59ca-4f42-836d-f59c24fcbbed Kawasaki Ninja 650 t f 267
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
649.0
9636bade-5d82-40a0-bcfc-4da14876f1f8 Kawasaki Ninja ZX-10R t f
271 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
998.0
35d08198-b64a-4667-b97e-4768a231c37f Kawasaki Ninja ZX-14R t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
1441.0
dd1f7980-501b-4128-bfca-e220933b936b Kawasaki Ninja ZX-25R t f
268 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
249.0
f775fd4a-e470-414a-9eb9-6a5245dab7fd Kawasaki Ninja ZX-6R t f 270
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
636.0
5bddbc77-69cb-4586-b71f-53e1205a9444 Kawasaki Versys 1000 t f
286 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
1043.0
662d31fc-e176-4235-878d-aca75a1af000 Kawasaki Versys 650 t f 285
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
649.0
38606a33-9e27-4e60-839b-d01acd67d316 Kawasaki Versys-X 250 t f
283 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
249.0
1026b38e-981c-4770-aff8-5c3437026eae Kawasaki Vulcan 1700 t f
289 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
1700.0
e5714c84-0ead-4a60-bc0c-6d378a0b8879 Kawasaki Vulcan 900 t f
288 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
903.0
ca4ecf58-1451-4b56-a812-bae9241245f5 Kawasaki Vulcan S t f 287
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
649.0
6929bb12-fba6-4c69-a3a2-e9cb30359954 Kawasaki W175 t f 0

2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
177.0
b7758363-2a3b-4950-a215-34f82989cfdf Kawasaki Z1000 t f 282
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
1043.0
706afdc4-4288-4ae4-b480-aa685614e95e Kawasaki Z250 t f 274
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
249.0
e3a3c556-6f7d-43b0-b27c-d04687537e77 Kawasaki Z300 t f 275
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
296.0
1f4e262b-75d2-4a2e-ac4d-67f475983fb6 Kawasaki Z400 t f 276
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
399.0
8b77de0f-4b1f-4b47-8edd-bff53e6600e8 Kawasaki Z650 t f 278
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
649.0
a69ca7f1-e731-4ee0-a1b9-ce2963622a4c Kawasaki Z900 t f 281
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
948.0
703bcd22-39f9-483f-b7ce-52ef6f870c00 Kawasaki ZZR1200 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
1164.0
7708cd06-884e-466a-b89d-7b2c8a0ec8de Kawasaki ZZR600 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
599.0
2ae2cf1e-73ea-424a-92b9-8e2ed1c2e218 RUSI Adventure 110 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
110.0
0a3ec1d8-9227-4fce-9827-fb58d483752d RUSI Classic 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
aca16e3a-15a7-46cb-ab7e-d79748e8aca1 RUSI Classic 250 t f 290
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
249.0
996435d2-eb77-4d42-b960-3afc384e4935 RUSI Euro 100 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
100.0
ea2b79fb-1fe6-4847-8b27-1577fb5a3c7d RUSI Flash 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
cd41b3a4-64aa-42bb-85b8-bb75831d9804 RUSI Gala 150 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
150.0
06cb5bbc-e2b7-4db6-9c2f-ff95ca5d6065 RUSI Gemini 125 t f 0

2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
fc7e6e73-c69d-4d4d-a7ba-5470727bc99b RUSI Magma 110 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
110.0
e666ecd5-4860-4031-9c41-a9e09e14ff67 RUSI Magma 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
15335231-1b98-4376-a174-90bf5fa8d4f5 RUSI Motoscout 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
43f0ec9b-3d10-4d40-a230-3474cb311168 RUSI Pioneer 100 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
100.0
03954894-8cd5-415b-bd4c-9c9acdd34a6d RUSI Rapid 150 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
150.0
29013a07-57f7-43c5-8244-15cf0ed5f70e RUSI Raptor 150 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
150.0
5d92afb3-e01e-4ca8-965b-44308465be75 RUSI RFI 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
59b722ef-8f16-45b1-b7a9-83a4edd46bb4 RUSI RFI 175 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
175.0
142d3718-b0fb-41cb-b7f2-a08198593a7a RUSI Royal 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
fba20d7c-a65f-46aa-8a75-988a2cb02b9d RUSI Saber 150 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
150.0
3eb1a2aa-4de0-492b-9197-58766b729ea0 RUSI Vista 150 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
150.0
0878cc0f-a778-40b3-9123-5f9cfbb436c5 MotorStar Café 400 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
400.0
f9347967-2855-4ac0-a33b-4304bcf6ebe8 MotorStar Magnum 150 t f
341 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
150.0
cb9a91f6-1f2b-4afa-ab8b-9f40b8e5e091 MotorStar Max 200 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
200.0
1792966e-d193-48ca-9d7e-688a27d57ae8 MotorStar Moto R150 t f 0

2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
150.0
347eb457-c8aa-4a6d-a2a3-0eb2dd4e7535 MotorStar Motoposh 110 t f
0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
110.0
c372d9b0-64f6-4db1-8d4b-3b8f5da21def MotorStar Motoposh 125 t f 0
 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
466e9d41-76fc-4519-9cfe-4d24fb1005fc MotorStar MSX100 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
100.0
7c4f3f76-467f-4575-9fc2-e9267c5792d1 MotorStar MSX125 t f 330
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
125.0
7e8ecd35-c7ca-442e-bc10-29ed6486fc6a MotorStar Star X200R t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
200.0
a68557a8-8b97-4e05-a9a0-bd7cf5f8bb90 MotorStar V200 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
200.0
8fa83f62-af74-4f06-8207-2011c70b768c MotorStar X200R II t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
200.0
37b1dc9b-2d12-49d8-8d67-1c7dfdc867ed MotorStar Xplorer 150 t f
335 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
 150.0
889e694d-1cab-4e07-ba0c-5d647246c317 MotorStar Xplorer 200R t f 0
  2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
200.0
23831fcc-2f3e-4e62-b62f-f48677f2afb7 MotorStar Z200R t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
200.0
b729dff4-890b-49f0-b84e-4b83cab01629 MotorStar Z250 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
250.0
8c925d1f-41f5-4baf-a31d-ef1e0bb98a6e Skygo Archer t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
107.0
7560321a-da59-41f8-bf3b-374a63d91f3e Skygo Blink 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
9756b017-bfe3-485e-ab22-788c8aa98763 Skygo Boss 150 t f 344
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
145.0
79aef85a-bafb-4668-ad15-9d78e4b2fe3f Skygo Boss 200 t f 0

2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
200.0
b57f7e53-acbf-424f-a185-0f7a34e7a8eb Skygo Duke t f 0 2026-10-06
17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 107.0
48d6136b-4d22-4cf2-921e-6496a119f06a Skygo Earl 150 t f 351
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
145.0
0d8e59cb-6c40-44a2-a0f4-4edc611781f8 Skygo Earl 250 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
250.0
68b5a01e-d736-4511-8b51-08eebe2045f2 Skygo Hero t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
88e568ec-4d09-4b25-971b-8af0f8aa8c74 Skygo King 150 t f 352
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
145.0
51d27634-ad1b-43c8-89bb-ebb956e51077 Skygo KP V150 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
145.0
a2454703-f216-4a28-9a18-d7076e306bfd Skygo Prince 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
ae3a6ec9-bd58-48a5-beab-9fbb43f9f8b6 Skygo Stallion 150-SM t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
150.0
f862c17a-09fd-4675-9427-874ab4b90be6 Skygo Wizard 125 t f 349
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
124.0
73ce789e-d808-47c1-8edf-6e89c512544e Skygo Wizard 175 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
175.0
ffbbfad7-c085-407d-a239-f66796af8d67 Kymco Active 110 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
111.0
03dd2d77-8806-4bbf-b6a9-9cf14d02b66c Kymco Active 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
36e670ee-012f-4592-be17-c27e3008d858 Kymco Active 50 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 50.0
f833ecf5-8b77-445d-800e-2a74d07f9f30 Kymco Agility 125 t f 365
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
124.0
e917691d-d5a4-4905-bdbf-3743e2d3beb9 Kymco Agility 50 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 49.0
ba131177-f475-49e7-952b-92298f020d93 Kymco AK 550 t f 376

2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
550.0
fd25e07d-f412-4a80-af73-f5f9b17fd7f2 Kymco Dink 150 t f 382
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
149.0
bd04544c-dc11-4b9d-97a9-6718db37e64a Kymco Downtown 300i t f
380 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
298.0
bfadfe16-24c6-41d3-824a-4bed059c793a Kymco Downtown 350i t f
381 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
321.0
ab2537cd-9922-45a0-b3b0-9a590e0bbbd9 Kymco DT X360 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
320.0
6336b661-c6f4-4cdd-8bd9-9253236ca13a Kymco Grand Dink 250 t f
383 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
249.0
a426039d-01cb-4c97-a742-820cb9a7be9f Kymco K-XCT 300i t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
299.0
401968ea-d5ca-4c33-a90b-f0bb3d10b4ac Kymco KRV 180 t f 377
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
175.0
9c93169e-36f5-4e2f-b17e-4f8daaa5b58d Kymco Like 125 t f 361
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
124.0
df7bad0c-fb29-4cb3-bbfa-e91cf6e2bb76 Kymco Like 150i t f 362
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
150.0
d56a488c-0e9b-4514-a53d-576523de40db Kymco People 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
7ef23323-cb7d-426f-8900-75f6640726b7 Kymco People 150 t f 370
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
149.0
d0fd669a-dad8-40fc-8c48-382d50fd0e6d Kymco Super 8 125 t f 368
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
124.0
3fc5e435-01d7-456f-a293-c2a60f9a78cd Kymco Super 8 150 t f 369
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
149.0
84ddcf1a-b187-4345-b1eb-35bdfc7c3dc1 Kymco Venox 250 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
249.0
94260cc7-c502-485b-b948-0567f8831d86 Kymco Xciting 400i t f 375

2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
399.0
ef786f4b-469e-4d26-a967-5aaaaebdb607 Kymco Xciting S 400i t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
399.0
a81960e2-544f-40cd-acb2-c6a6888a60ea CFMOTO 1000MT-X t f 410
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
997.0
e523b366-00a6-4f6d-b9aa-dcd4b6242ab2 CFMOTO 125 ST Papio t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
126.0
099156a6-6a7d-406d-8708-d86fc8e8915b CFMOTO 150 Aura t f 429
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
149.0
bab64e1a-b362-4bbb-9d6d-e40c363e21e7 CFMOTO 150NK t f 387
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
149.0
6cba30bd-211a-4b47-a08b-11550a5ddc49 CFMOTO 150SC t f 428
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
149.0
060d9c74-fb29-4112-a2da-57d124bdd484 CFMOTO 250NK t f 388
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
249.0
c3f1af99-3db0-4d04-b64a-65fa47b7911f CFMOTO 250SR t f 395
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
249.0
6e91879f-b570-4ac3-aa3c-c98c8446e9c7 CFMOTO 300CL-X t f 411
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
292.0
8d481de3-895f-4f6d-b0f8-46b1ef1f6411 CFMOTO 300NK t f 390
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
292.0
13bda9c5-fe37-4468-a21d-30b39a8eb373 CFMOTO 300SR t f 397
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
292.0
f519e571-ce3f-452e-a80e-c415e2947c81 CFMOTO 400GT t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
400.0
538af626-91c4-4121-9377-df06f7d8b248 CFMOTO 400NK t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
400.0
58aadf89-4871-41de-a83c-78c345521675 CFMOTO 450CL-C t f 412
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
449.0
76eac79c-1c2f-4aad-abb2-5e15b2c6751a CFMOTO 450CL-C Bobber t f

414 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
449.0
f82e769e-04e7-474e-96ae-1807c23f6032 CFMOTO 450MT t f 403
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
449.0
19086e38-116b-44f9-b8e8-6c7dabf3f824 CFMOTO 450NK t f 391
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
449.0
9c7c17a8-d2c3-4231-a7a0-5f0107751ee7 CFMOTO 450SR t f 398
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
449.0
25df5d85-22c6-4d9f-90e7-2a1e58a68d3a CFMOTO 450SR-S t f 399
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
449.0
e98c4098-7b6c-47d3-a0c2-1db03f1e6dd6 CFMOTO 500SR VOOM t f
400 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
449.0
76cc7fc4-9d77-4924-b4b1-10a17e11bba3 CFMOTO 650GT t f 419
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
649.0
2c6be9db-fee3-4f64-a7ee-82e0d345f2cf CFMOTO 650MT t f 404
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
649.0
a6bf0a1e-2ef5-40c7-a514-1a776c04b6ee CFMOTO 650NK t f 392
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
649.0
e67015f8-1cb4-4085-8588-5c22a669123e CFMOTO 675SR-R t f 401
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
675.0
abfc6783-ac89-4ac7-81fc-7e91d747ff23 CFMOTO 700CL-X t f 415
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
693.0
51ae2968-9313-466f-82eb-b5c24ad7266d CFMOTO 700MT t f 405
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
693.0
0b00094c-3354-482f-83b3-3eaa026576e9 CFMOTO 750SR-S t f 402
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
749.0
23171209-1922-4895-bf3f-34704b675502 CFMOTO 800MT t f 406
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
799.0
2cfe4492-bd83-4602-8684-4f131e084141 CFMOTO 800MT Explore t f
408 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
799.0
00611244-8cbf-4db6-a966-5811423e2816 CFMOTO 800MT Sport t f

407 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N 799.0
fe705355-d818-4774-8508-722353759ed4 CFMOTO 800MT-X t f 409 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N 799.0
9cfc803e-8e72-4dd2-a32b-20fb638ca53c CFMOTO 800NK t f 394 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N 799.0
eb5e6a67-6ac9-41fb-b283-3547e0d4359e CFMOTO ST Papio t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 126.0
7000824c-be76-4a8f-b2ba-bfb8a2b4468c TVS Apache RR 310 t f 443 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N 312.0
54602bd5-d5c5-4cc7-b3fe-22036c2c355f TVS Apache RTR 160 t f 439 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N 159.0
c5de863d-9e50-4fec-afb2-6b907660aec6 TVS Apache RTR 180 t f 441 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N 177.0
b5f2618e-fccf-4ab1-a8be-be46263175e0 TVS Apache RTR 200 4V t f 442 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N 197.0
caf221e2-215a-498c-a637-669964b2c7cd TVS Jupiter t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 109.0
d419ca7b-218e-49c1-ae07-9b9f7c5495ff TVS Jupiter 125 t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 124.0
d6c21129-1f1e-45e2-90dd-f2f3caeb3f81 TVS NTORQ 125 t f 451 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N 124.0
979856db-777f-4790-9ace-e260ac06c6a5 TVS Radeon t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 109.0
92adc1bd-29f5-48c4-9809-a6df32c039b1 TVS Raider 125 t f 444 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N 124.0
7d7dd041-61dd-46a8-a8d7-3a2f8b0bc3a9 TVS Ronin t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 225.0
09125fa1-b7fb-4270-9a3c-46124ec25cda TVS Sport t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 109.0
afcbea4e-7621-4703-825e-cd376362a81e TVS Star City t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N

109.0
0421c319-2231-4cc2-9df5-64da3448309a TVS Star City Plus t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
109.0
eb63f37d-d6a0-46b3-af6f-a3ac55391c32 TVS Victor t f 0 2026-10-06
17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 109.0
7b56ed44-d20a-4cbe-88cd-e4eee251a8ec TVS Wego t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
109.0
bb59c5c4-0e5e-4848-8c84-9396e7afe736 TVS XL100 t f 432
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N 99.0
577ae811-498f-447d-a963-31e67e5da97f TVS Zest 110 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
109.0
5a07c07f-ead0-4226-839f-3988c5afefc7 Keeway Cafe Racer 152 t f
459 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
149.0
e7091fe8-a6ca-480c-8f5a-99d6eee2bcc4 Keeway CR 152 t f 457
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
149.0
4946e6a7-51ee-4e94-bd43-cb5355cfad85 Keeway K-Light 202 t f 463
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
197.0
08cc13cc-bceb-425d-9002-4e981561bb15 Keeway K-Light 250V t f 464
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
249.0
0fa5adfe-a2ba-4633-a3c5-f46a404388dc Keeway RKF 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
81c7ed02-16f5-423b-bbeb-8f1a1a835b8b Keeway RKF 200 t f 461
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
197.0
3b9e92ee-8756-4eff-97ac-1a6f8af1e1b8 Keeway RKS 125 t f 452
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
124.0
6875ed4c-4475-4e63-8d34-f494469f6edb Keeway RKS 150 t f 453
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
149.0
bd0bc821-ecba-4cd1-a3be-5e96ae1aa82f Keeway RKS 200 t f 454
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
197.0
31cbdeda-223d-4143-b499-eee7516e3ac8 Keeway Sixties 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
79597e51-ba21-40f5-9aee-c9322dd8617b Keeway Superlight 125 t f 0

2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
a49e3dd0-5211-4941-a6e5-0b82b601b0ee Keeway Superlight 200 t f
456 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
197.0
588f8c99-ef0b-48d8-9e45-50e964eb3ff9 Keeway TX 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
ad220309-1850-4d63-8ae2-3ac5669eb685 Keeway TX 200 t f 460
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
197.0
62486139-f0ae-4af8-83c2-55b25f3cae14 Keeway V302C t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
298.0
f7ddae0a-1c3e-4424-ab34-ef189b0343ac Keeway Vieste 125 t f 465
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
125.0
44d5a5fe-7d07-4c0c-8e7d-90c9f5afc1b9 Keeway Vieste 300 t f 466
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
278.0
e9209e06-8c51-458f-a66d-7e58457a6dcb Benelli 180S t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
175.0
c5fe33c4-fb1a-44fa-a144-fd25fd7d3027 Benelli 302R t f 0 2026-10-06
17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 300.0
ccef3e2d-d486-4b33-aab8-b46d537dd257 Benelli 302S t f 491
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
300.0
739fa921-f9ed-45bb-b69a-e38a10e4e6c9 Benelli 502C t f 493
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
500.0
fc4edc7f-ec08-46ac-aee9-30b0ef4cb946 Benelli 752S t f 494
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
754.0
9c524fb9-7c1f-4a7a-b211-506c5e2a4d3e Benelli BN 125 t f 477
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
125.0
732433bb-b9a6-432b-aebc-b1a11053d051 Benelli BN 251 t f 479
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
249.0
e9c74db0-b385-453d-b884-0064fa5ad76c Benelli Imperiale 400 t f 498
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
374.0
37d0a5a7-ef70-4a4a-a5a9-248c01acdf5c Benelli Leoncinco 500 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N

500.0
dd6b6a83-6915-47ad-978e-8be10274a42e Benelli Leoncino 250 t f 482
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
249.0
b9eae694-6fa1-4fe2-b5e0-bf87a1ea7bce Benelli TNT 15 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
148.0
2e83ac9a-764d-4a56-885f-c18cef75a3f2 Benelli TNT 150i t f 471
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
150.0
1b8ea8ce-4e95-41ab-a097-f93ac0a04102 Benelli TNT 180S t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
175.0
22c46273-859f-40fb-9d47-35a63bc08838 Benelli TNT 25 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
249.0
f6e0abdb-5159-4499-b826-18de47142f45 Benelli TNT 600 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
600.0
69d6d11e-47ed-4640-9c51-04973818bc3c Benelli TNT 899 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
898.0
eee1b6ea-5d1e-40a9-83cc-0138cf9adb6b Benelli Tornado 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
e67d428c-f9a3-404d-9e74-0d74f2f773f5 Benelli Tornado 550 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
554.0
ce6489d4-5b8c-419a-9e6d-605e32c6cbe1 Benelli Tornado Tre 1130 t f
0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
1130.0
8413c699-d3b4-49bc-9ecc-c2b726d8a3b7 Benelli TRK 251 t f 486
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
249.0
541b6770-a2a3-476c-8cb5-46e0e988cfa6 Benelli TRK 502 t f 487
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
500.0
9ea84b26-d322-4058-a07d-d28a5c15dda3 Benelli TRK 502 X t f 488
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
500.0
8f536d86-3d12-4375-a3e5-48dc01ded401 Benelli TRK 702 t f 489
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
698.0
4c0afd3c-317f-42f8-9634-7b0b109e8319 Benelli TRK 702 X t f 490
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N

698.0
fd04a820-db23-407e-b328-20b6d432fa1d KTM 125 Duke t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
4285c847-62a6-475b-bb4d-225d7639626e KTM 1290 Super Adventure t
f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
1301.0
deaf772a-a971-4852-8a34-6433eea5e610 KTM 1390 Super Adventure t f
0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
1350.0
afd49a9d-7c70-4ec2-9692-2a2c340def5e KTM 1390 Super Duke R t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
1350.0
e2c82d83-2bbb-426f-b9d8-cabee1a78aef KTM 200 Duke t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
199.0
fa66002e-8608-4c67-a22f-7673915402e8 KTM 250 Duke t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
248.0
8c0bad27-00f1-4124-8214-d9de08eb0141 KTM 350 EXC-F t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
349.0
093bc323-0d81-4c68-ad7c-05d8807b55c9 KTM 390 Adventure t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
373.0
657f75c3-0481-4004-8584-cad9312d33f2 KTM 390 Duke t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
373.0
dc50187d-eb01-444f-9b99-6f80981dbdd0 KTM 390 Enduro R t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
399.0
ae8d0bdf-e1d9-4195-b993-11b5acfef0cd KTM 390 SMC R t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
399.0
e596f0b0-49ca-4a81-9557-8620fd49b32b KTM 450 EXC-F t f 532
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
449.0
7832b70e-ac55-4ef2-94df-416ce6ab5520 KTM 500 EXC-F t f 533
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
510.0
073a1441-2719-4be4-9a34-4377e60c86b6 KTM 690 Duke t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
690.0
df7d2e23-1599-438a-aa22-e853417b6292 KTM 690 SMC R t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N

692.0
91243d05-02bd-474c-a255-ea0f86311b6f KTM 790 Adventure t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
799.0
4f3ed107-8b32-4596-ba0b-d94e1bfcd8e5 KTM 790 Duke t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
799.0
edf8e035-7790-4e4d-8fc8-2154d2a7b502 KTM 890 Adventure t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
889.0
6de766e0-f955-4692-9882-8c7b58f61d9e KTM 890 Duke R t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
889.0
fbc0feca-20b2-4995-a02f-c35737b147dc KTM 990 Duke t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
947.0
e27d0ae3-e744-47c4-b746-101536bceafd KTM 990 RC R t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
947.0
ff2b4205-2a51-4fcd-92ca-2c2280769ade KTM RC 125 t f 510
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
125.0
de6d9367-0b3c-4c07-828c-e782a327ab0c KTM RC 200 t f 511
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
199.0
498d5f66-362b-46f4-a066-f02d900f4bc1 KTM RC 390 t f 513
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
373.0
b0930a5c-e20b-4d07-88b2-91bf74d4e5ca Bristol AD200 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
200.0
d0ccdfc4-d600-4729-b161-d8f294d86c97 Bristol Bobber 250 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
249.0
9d17e6eb-26e8-4046-a56e-701c2889edb9 Bristol BR400i t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
400.0
92d8fce8-79b7-4ca3-967f-17391f8cdcf0 Bristol Cafe Racer 250 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
249.0
f2d35e05-2a87-4fad-9212-add6647bcfc4 Bristol Classic 250 t f 534
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
249.0
33ad98ad-6339-41ba-a598-17f54677c216 Bristol Maxie 400 t f 545
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N

400.0
eb0d1deb-71c7-450b-a8c8-be7b5c7ac54f Bristol Raptor 500 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
500.0
c78e8618-3a7c-4e65-8643-2365288a1a03 Bristol Vantage 400 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
400.0
c706298e-1731-40d7-8c18-0ecd3f02a27d Bristol Venturi 400 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
400.0
017bb139-1dfa-4af0-a47c-e30f69d416e5 Royal Enfield Bear 650 t f 572
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
648.0
27c37ea1-a6cf-48ca-84f8-a0fde8155c99 Royal Enfield Bullet 350 t f
549 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
346.0
71d9a96d-4308-4b7a-ad70-eeb11be13bb9 Royal Enfield Classic 350 t f
551 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
346.0
f381605e-dc95-4078-9bc4-b4fe924b9601 Royal Enfield Classic 650 t f
554 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
648.0
c4f4419f-a572-491d-bc54-65662023e065 Royal Enfield Continental GT 650
t f 568 2026-09-20 00:34:28.972796+00 2026-10-06
17:07:33.654736+00 \N 648.0
ca86b07e-7a2f-4022-9485-eaf45215acb6 Royal Enfield Goan Classic 350 t
f 561 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00
\N 349.0
44fc981f-9adf-4cf7-b923-6deaa927415a Royal Enfield Guerrilla 450 t f
566 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
452.0
89aca05e-a7b3-4355-8ffa-14c985be0b60 Royal Enfield Himalayan 411 t
f 563 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00
\N 411.0
69017af5-3b6e-4f69-928c-0f25af786d36 Royal Enfield Himalayan 450 t f
564 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
452.0
7d5812d3-eb7c-48f0-ac1c-61637a6c826a Royal Enfield Hunter 350 t f
560 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
349.0
0854b852-d192-47dc-b2ab-83438e9b1cf2 Royal Enfield Interceptor 650 t
f 567 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00
\N 648.0
199aa8f9-b678-4f4b-9f1e-51ae523a20ee Royal Enfield Meteor 350 t f
559 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N

349.0
5bac116b-304e-4634-aef7-b0195c62c252 Royal Enfield Scram 411 t f
565 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
411.0
544b492c-4cb6-428b-a457-512ab0de93b5 Royal Enfield Shotgun 650 t f
571 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
648.0
c0cda26a-e72f-4fef-a3ed-4bfd6c193ee6 Royal Enfield Super Meteor 650 t
f 570 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00
\N 648.0
b1a9b95a-e100-4262-b71a-4f0c95fc6824 SYM Allo 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
7d654c0f-c48c-46fc-9529-a0ab31d5eacd SYM Allo 150 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
149.0
1bde3dd3-1199-4512-ab59-0a944e9afc1a SYM Allo 50 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 49.0
a47d2995-00b5-4982-bb30-3e96b22ec579 SYM Bonus 100 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
100.0
d72c29fd-e0d0-4450-a66c-62e077fc923c SYM Bonus 110 t f 576
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
107.0
5454bb96-45a3-44e9-b8ac-455a7d46fc1c SYM Citycom 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
2a92a042-452b-46b7-8d9f-43598afaf5c3 SYM Citycom 300 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
278.0
b4e1597d-295f-4aa2-964a-bdedba4e0a58 SYM Crox 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
613c0d97-a7db-4483-9a9f-6e187b6b035b SYM Crox 50 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 49.0
ddeab70e-c07f-4c4e-9ccd-7d037ab2405a SYM Cruisym 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
5c771e1f-932a-4965-90dd-6100f03a048c SYM Cruisym 250i t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
249.0
4c23a3f9-33c7-4354-b2ed-9ef2d345e8ac SYM Cruisym 300 t f 585
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
278.0
5c0efff3-a38a-49b6-8b7f-164d786650dc SYM Cruisym Alpha 125 t f 0

2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
2b9fe265-9dc7-4aca-b07c-4324f5227925 SYM Jet 14 125 t f 582
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
124.0
5bc3e088-84bd-46d7-bdbe-d09a731f879c SYM Jet X 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
5095213a-5c19-4393-9571-4d04cd90fb78 SYM Maxsym 400 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
399.0
56055409-f341-4a33-bb8f-1aa63744704a SYM Maxsym TL 500 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
465.0
3c3fe695-0231-428b-b639-36bfdc191eab SYM NH-T 200 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
183.0
33568102-4bc4-486d-832e-4d241bd865cc SYM VF3i 185 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
183.0
9871fc3a-72ea-4d0d-9b73-c175e152bf96 SYM Wolf 125 t f 592
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
124.0
99136073-98d3-4cc0-b74f-568c6ebd50f0 Bajaj 125 Classic t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
daa6f6e2-c8d9-467f-b7e0-785036666b1d Bajaj Avenger 180 t f 626
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
178.0
02764117-9c73-497b-898a-d4969216aa62 Bajaj Avenger 200 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
200.0
a64fad16-d706-4151-81db-c2fb9cb1ee78 Bajaj Avenger 220 t f 627
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
220.0
6618814a-8298-4539-8470-9aa1a538a8a6 Bajaj Chetak 145 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
145.0
534f661c-c82d-4f0b-ba03-63199b4fbbe6 Bajaj Chetak Electric f f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N \N
9522489d-a765-4c50-8deb-740ec49daf7e Bajaj CT100 t f 599
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N 99.0
a084788b-a4af-4d37-a2ee-497c62ab5cb7 Bajaj CT110 t f 601
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
115.0

f989f506-e7fe-4e14-b1c2-48305024edf8 Bajaj CT125X t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
f4ee495a-f2f9-4282-b0be-1f85c7f18348 Bajaj Discover 100 t f 606
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N 94.0
0e756996-c082-41d9-b845-c9872cd6d615 Bajaj Discover 110 t f 607
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
115.0
a178d070-6510-4914-9265-2e01b94622a2 Bajaj Discover 125 t f 608
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
124.0
9bdd2d5c-2846-4aee-952c-91d64c9eb580 Bajaj Discover 150 t f 610
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
145.0
c30fba9c-6dcd-4f49-8b97-1628dd10c5b1 Bajaj Dominar 250 t f 622
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
248.0
754f5a97-c0b2-4bf3-a163-6c3a8ec6c99e Bajaj Dominar 400 t f 623
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
373.0
ec675ac2-ae86-45bb-8054-0132f73042be Bajaj Platina 100 t f 629
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
102.0
5c0ebba1-4ede-463d-bae4-2c59022b290e Bajaj Platina 110 t f 630
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
115.0
9d148842-0bf4-43a5-bda9-5695b890faf2 Bajaj Pulsar 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
16653213-be45-4228-9372-875a46e83ad7 Bajaj Pulsar 135 LS t f 611
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
135.0
d9f8796b-d472-4dfc-a221-50bd9734d3c0 Bajaj Pulsar 150 t f 612
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
149.0
1876c6c7-d726-4953-b58f-ec3b02004216 Bajaj Pulsar 180 t f 613
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
178.0
205c5c6e-92d0-40c1-a2d8-a637630dca5c Bajaj Pulsar 220F t f 620
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
220.0
9c88333c-ef3f-4750-b50d-06b7be063cca Bajaj Pulsar NS160 t f 616
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
160.0
36215c23-cd69-40a7-8601-6fafb27dd218 Bajaj Pulsar NS200 t f 617

2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
199.0
eaed6d81-2f68-400e-8099-ca83ac092499 Bajaj Pulsar RS200 t f 619
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
199.0
6c246224-4844-44e0-9a26-d7c488a60498 Bajaj Wind 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
8b53c31e-e804-4582-bf29-552eeff26016 Bajaj XCD 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
8b6c4784-ec51-4cb5-a50c-b8709d9b48f3 Zontes 310 T1 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
312.0
6f482853-9c00-4c14-8c7e-a43e64e5d1e4 Zontes 310 V t f 637
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
312.0
8dc547ed-27a7-46b7-bf0e-c29544e16ab9 Zontes 310 X1 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
312.0
f76705d1-f900-4a2b-b49b-8eaabde6388e Zontes 368G t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
368.0
b291aeca-29f2-407e-b7fb-0a4e58049a03 Zontes 703F t f 646
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
699.0
61ffc3d9-a726-4472-a88e-6aedf6e9f34d Zontes 703RR t f 648
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
699.0
33f5d299-4e82-440f-9f78-41a87c7370b6 Zontes E-Mantis 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
7a864915-53a7-4fa9-8945-d725329a8e8a Zontes GK350 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
348.0
36dc2de5-fee5-4f7c-82af-9027e7dfd119 Zontes RA1 t f 0 2026-10-06
17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 125.0
087a98c3-2ba6-4a2a-ac49-a842539d6981 Zontes ZT125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
1472e3de-12f8-4394-90b6-43d5afeb3d6b Zontes ZT125-G1 Scrambler t
f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
cf13888c-65da-4dc8-83d0-d9667afdfbf5 Zontes ZT125-M t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N

124.0
3244f5b4-af66-4cb3-bebe-4f222a02c0f7 Zontes ZT125-U1 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
a70be04c-4657-4ec8-bb49-1155f0bac3e8 Zontes ZT250 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
249.0
8ff6382a-a29c-411d-817f-05b3d445a4cc Zontes ZT310-M t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
309.0
e2227e48-d599-488a-9a9d-9725a2fc5101 Zontes ZT350-GK t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
348.0
aabe36a0-5c67-4e33-bcca-49aba8380c7d Zontes ZT350-T t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
348.0
c5cc61a4-f200-4747-8b08-df0b27bb3212 Vespa 946 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
711506e3-8bff-44e5-ac17-36f848fe38e0 Vespa ET4 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
0a1eddfe-f8f1-4a11-a452-08cce10dc53e Vespa ET4 150 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
150.0
49c6915b-660d-489c-bbeb-64efcd9acaae Vespa ET4 50 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 49.0
f888d67d-5eb5-41f8-864c-3111c5920493 Vespa GTS 125 t f 664
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
124.0
13f07bbb-f8c6-40ea-b8b5-cbba7a3ff941 Vespa GTS 150 t f 665
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
155.0
d7778a12-cec8-4ca4-9338-d3d4e9222283 Vespa GTS 250 t f 666
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
244.0
af818105-6f30-4b3e-9e8d-2c543fcd6be4 Vespa GTS 300 t f 667
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
278.0
d8cba894-f3ba-4df4-8ac4-b704df285269 Vespa GTV 300 t f 670
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
278.0
10ce5bcf-43e3-4452-a989-b692ba73b683 Vespa LX 125 t f 652
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
124.0

68f32a9e-cb59-45a1-898d-83aa8b9ce57c Vespa LX 150 t f 653
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
150.0
17d4273c-5740-4d61-b28c-a7217bea3384 Vespa LX 50 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 49.0
007bae15-325e-447d-895b-56e6d6b75bba Vespa Primavera 125 t f
658 2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
124.0
d32d1eed-0df9-4504-a7d5-afc72bf6bc8f Vespa Primavera 150 t f 659
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
155.0
2d40506a-d924-4881-bbc8-d8cf9d8784d1 Vespa Primavera 50 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 49.0
dbca902f-686d-4145-a64d-7e3f64918eec Vespa S 125 t f 656
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
124.0
e0cb7b8b-7d2f-4782-9325-eb4642831b15 Vespa S 150 t f 657
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
150.0
77a3d9aa-ad8a-4a81-a28e-954780b210c5 Vespa S 50 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 49.0
3f28696b-a89a-479e-a3b4-7a40d56b3df1 Vespa Sprint 125 t f 661
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
124.0
462d9982-13ec-4b63-997c-1249d8371dab Vespa Sprint 150 t f 662
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
155.0
2fc65244-b7e9-45b7-b3d1-5b93295fbfde Vespa Sprint 50 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 49.0
7c9ac371-a58d-459d-a069-531322a63785 Piaggio Beverly 250 t f 683
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
244.0
5641559e-76d0-4dd1-8d89-89e7522dfa29 Piaggio Beverly 300 t f 684
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
278.0
453f1f09-5a45-4cff-bf17-649ab86fd7cb Piaggio Beverly 350 t f 685
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
330.0
87279638-9a3f-4385-bd47-5295207ff68d Piaggio Beverly 400 t f 686
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
399.0
55ef10e9-d021-4f20-9158-56cef4cfc0bf Piaggio Liberty 125 t f 675
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
124.0
0c9d4003-c9c0-4d74-9f34-6c3d4d09f64b Piaggio Liberty 150 t f 676

2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
155.0
c5b46377-d7fb-4cd6-b2f2-7f2fb34bd501 Piaggio Liberty 50 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 49.0
90ad0687-9a27-486d-905d-c9efc545f30a Piaggio Medley 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
125.0
42de723c-e1fe-4440-b2d7-a4719598e848 Piaggio Medley 150 t f 680
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
155.0
fd5eef23-09a2-4cfa-8a15-063ecce8b5cb Piaggio MP3 300 t f 687
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
278.0
50410d78-7b0b-4d2a-b8a7-005d01d24210 Piaggio MP3 400 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
399.0
7cae0133-3c40-42de-a8c6-98b877ecb23f Piaggio MP3 500 t f 688
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
493.0
099ab9c3-b21a-46cb-bfc9-a2b76f0faeb0 Piaggio MP3 530 t f 689
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
530.0
94b16f5a-6390-4c8f-a507-9664615aaed7 Piaggio X10 125 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
124.0
d149afdd-5fb2-4d50-b17c-cc867a7d821f Piaggio X10 350 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
330.0
efed5041-4734-4e25-86c9-91d3ab31c7f9 Piaggio X9 250 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
244.0
d549353f-7e73-4b5f-9d8f-8e1b2f4087a7 Piaggio X9 500 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
460.0
7db06c45-5045-446d-b245-1480fa434c1c Piaggio Zip 50 t f 0
2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 49.0
8d979928-e1d8-491a-8364-25b49f545cb6 BMW Motorrad C 400 GT t f
735 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N
350.0
da48cbb1-34d8-4a20-8587-84d66234c493 BMW Motorrad C 650 GT t f
0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
647.0
71607f2b-180c-46ea-b366-17c8131e278e BMW Motorrad F 650 CS t f
0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N
652.0

cbdd0525-2747-49a9-87a4-c60e67755539 BMW Motorrad F 650 GS t f 701 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 652.0
6e0e2521-2820-4aa8-8065-420cf016909b BMW Motorrad F 700 GS t f 702 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 798.0
74cdb683-c195-433a-b877-8015de55368e BMW Motorrad F 750 GS t f 703 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 853.0
c70d1d72-906f-4810-aed2-1b9c6721ea85 BMW Motorrad F 800 GS t f 704 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 798.0
38fddc2c-6b26-4ed5-8e74-81f19c260253 BMW Motorrad F 850 GS t f 707 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 853.0
b462e71b-fc7b-473f-bb5d-3ade5293e903 BMW Motorrad F 900 GS t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 895.0
4e164dcd-ae81-427e-900f-ab8def20deca BMW Motorrad F 900 XR t f 710 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 895.0
a0af6579-d5c2-45f4-981b-32ba221a2b28 BMW Motorrad G 310 GS t f 699 2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N 313.0
7ea2e8f6-88ed-44bc-89aa-07cf154b5026 BMW Motorrad G 310 R t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 313.0
8e804ff4-6238-4b23-9d6f-ba13721353fc BMW Motorrad K 1200 S t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 1157.0
71b8f7c0-463d-49e1-be68-8b7d581c0655 BMW Motorrad K 1300 S t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 1293.0
d83c0a1d-bb17-4bb9-bd6e-c9b683bdc5f0 BMW Motorrad K 1600 B t f 732 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 1649.0
2b498e7f-f444-418a-8fa9-3afc15bae15e BMW Motorrad K 1600 GT t f 730 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 1649.0
ea974d7e-7ff1-466f-a0bd-ac0feba3719e BMW Motorrad M 1000 RR t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 999.0
5a4b609e-0cd3-438e-b0a7-fa21d5efb908 BMW Motorrad R 1100 GS t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 1085.0

0b1ed4d4-4862-4e30-95a5-165072b2c765 BMW Motorrad R 1150 GS t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 1130.0
6c6c79d5-33e6-4077-b067-025cd7b55c00 BMW Motorrad R 1200 GS t f 714 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 1170.0
a9636b82-8cc9-4595-8c23-5272c96da92e BMW Motorrad R 1250 GS t f 716 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 1254.0
a1d4e4b0-d825-44a0-9724-c26121d662d9 BMW Motorrad R 1300 GS t f 720 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 1300.0
3703e529-71c1-4d44-a324-86ab92263cce BMW Motorrad R 18 t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 1802.0
9f7ca144-9691-4aee-92dc-5a75afa5b7d3 BMW Motorrad R nineT t f 722 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 1170.0
27ab3f0b-8c73-470d-824c-f888224c5058 BMW Motorrad S 1000 R t f 725 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 999.0
197cecdf-eb01-4e2b-a085-bf77dfb5aa74 BMW Motorrad S 1000 RR t f 724 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 999.0
e14883b1-a2a6-4126-8b3a-d8c13a269d6d BMW Motorrad S 1000 XR t f 726 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 999.0
2f9b29c7-a9ca-4dba-a69b-0b97760d61ed Ducati DesertX t f 779 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 937.0
67685b2c-4f3d-4ee3-9987-bebc4e5b2cb3 Ducati Diavel 1200 t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 1198.0
3ce59fd4-ce88-49b3-baaf-992a78fb699c Ducati Diavel 1260 t f 772 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 1262.0
f2455dc3-ada0-4787-bbcd-8ff31ca6e980 Ducati Diavel V4 t f 773 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 1158.0
aa5a4a1a-b345-4811-8e47-f509f4435c07 Ducati Hypermotard 821 t f 776 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 821.0
46ad7db6-7cc9-44ab-be01-799e49c974c7 Ducati Hypermotard 939 t f 777 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 937.0

006490c7-e34a-4034-88fb-4f5b66eb28ae Ducati Hypermotard 950 t f 778 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 937.0
2067a3f1-9ae0-4fd0-bdda-be4a3597bf4b Ducati Monster 1200 t f 746 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 1198.0
9396994e-d343-4300-b2e6-70c26a921992 Ducati Monster 600 t f 738 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 583.0
efbebb5e-3e9c-49ab-82af-3d7ae23bd059 Ducati Monster 620 t f 739 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 618.0
522f8416-fb8c-40f7-87c7-469f7adde81c Ducati Monster 696 t f 741 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 696.0
87a61535-4094-46f9-8f92-7a68cce4fb71 Ducati Monster 797 t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 803.0
1f3a0500-4fd6-4502-8365-757d46cc9bb3 Ducati Monster 937 t f 744 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 937.0
5f8f4396-6fa2-45dc-b55c-b1e727e3c76b Ducati Multistrada 1200 t f 767 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 1198.0
605c8cb3-fbdd-42df-adf7-62536d3b60dc Ducati Multistrada 1260 t f 768 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 1262.0
aba4907b-e67a-4126-8e14-bd47d98cb6b9 Ducati Multistrada 950 t f 766 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 937.0
7eaf2493-3da8-44f2-b455-d59ae170c4d1 Ducati Multistrada V2 t f 769 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 937.0
973905d9-0189-41ce-aedb-7e0f8a61aac9 Ducati Multistrada V4 t f 770 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 1158.0
cede4d66-c777-4334-b065-f5646a9c112a Ducati Panigale V2 t f 755 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 955.0
4ceca229-4ff7-4602-bc68-b2e7b868f72c Ducati Panigale V4 t f 756 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 1103.0
1bd26d5b-fea3-4847-aef3-dbdf34f89ab4 Ducati Panigale V4 R t f 757 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 998.0

6d657a2e-ff24-4d38-9cb2-4b238c310968 Ducati Scrambler 400 X t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 399.0
55ef7108-4942-4491-a48f-cf9550f9e92a Ducati Scrambler 800 t f 748 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 803.0
8adcf0f1-c313-48e0-9cbf-58d358becfa8 Ducati Streetfighter V2 t f 764 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 955.0
17ea1d9f-3c0b-4366-9d9c-82004f3a3ed3 Ducati Streetfighter V4 t f 765 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 1103.0
8fee7d04-54c3-4062-8791-bd60b338f8f7 Ducati Supersport 939 t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 937.0
58a0076c-26e4-410a-89ee-c2c374d66dc3 Ducati Supersport 950 t f 781 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 937.0
c833698b-e6a0-4ff7-a738-d89de0a573e3 Harley-Davidson Fat Bob 114 t f 799 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 1868.0
e55365f2-2712-4fd0-9eb6-1afae0046cfd Harley-Davidson Fat Boy 114 t f 801 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 1868.0
aff98689-8404-48f8-924d-e661f43a06af Harley-Davidson Forty-Eight t f 787 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 1202.0
5d41d89f-fcf8-4750-8e55-a459b82b2cc3 Harley-Davidson Heritage Classic 114 t f 803 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 1868.0
335940e8-89de-461d-8561-9e13dd402f0a Harley-Davidson Iron 883 t f 785 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 883.0
9e383bd5-0066-4e21-8ea9-1186d942618f Harley-Davidson Low Rider S t f 796 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 1923.0
312bd165-64ba-4d72-8650-fed731ecbea9 Harley-Davidson Low Rider ST t f 797 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 1923.0
d5642876-9844-4753-b3cf-c46cee32383c Harley-Davidson Nightster t f 789 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 975.0
f20d2205-7751-4b38-b148-136e3794fba4 Harley-Davidson Pan America 1250 t f 818 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 1252.0

dae3ddc0-7b3a-41f8-8bfe-6623e732debf Harley-Davidson Road Glide t f 811 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 1868.0
3ac4ebc7-e8ca-4969-96de-08318e0140c8 Harley-Davidson Road King t f 809 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 1746.0
cf7be47b-bb75-4e33-bea4-d5ac18835a19 Harley-Davidson Softail Standard t f 806 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 1746.0
52c6fd9e-9ba5-40e3-90bd-3b8ced6a3a1c Harley-Davidson Sportster 1200 t f 784 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 1202.0
df10cf4b-6720-4017-9b62-ae0c02580f1f Harley-Davidson Sportster 883 t f 783 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 883.0
b708b376-a69f-470c-8514-b71c2b7f01c1 Harley-Davidson Sportster S t f 820 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 1252.0
2dbc3f8b-e65f-473a-b079-1c8ebb68198c Harley-Davidson Street 500 t f 791 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 494.0
81afe1d5-ab21-4554-80ad-dfb8268671ac Harley-Davidson Street 750 t f 792 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 749.0
4f7c1b3e-ba23-432b-bd97-6e88f119353f Harley-Davidson Street Bob 114 t f 794 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N 1868.0
89151bb2-6e0f-4bab-88cc-a11caaa32d91 Harley-Davidson Street Glide t f 814 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 1868.0
fdb13a0b-37c9-4f46-a8c4-34ea31326e56 Harley-Davidson Tri Glide Ultra t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 1868.0
43208fe5-f777-40e8-9f7d-28e2cc5c9517 Harley-Davidson Ultra Limited t f 817 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 1923.0
40bc0620-16c8-4f3d-821a-816194c0a44b Triumph Bonneville T100 t f 823 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 900.0
77abb1e8-4b73-466a-8d14-1660d453bda0 Triumph Bonneville T120 t f 824 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 1200.0
c9935c1d-f362-4961-84da-e4629d30192a Triumph Daytona 660 t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 660.0

ec88fa87-c93a-4cb4-a623-291cf2b52894 Triumph Daytona 675 t f 840 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 675.0
579b9977-648d-4e2f-8966-27f96a6d7054 Triumph Scrambler 400 X t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 398.0
55f4979f-c41f-471e-bc03-baf10663845d Triumph Speed 400 t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 398.0
b616ec6f-cd56-4838-b3ea-21dd62540d17 Triumph Speed Triple 1050 t f 838 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 1050.0
f2a8295b-0619-4745-81cc-aa583bfce870 Triumph Speed Triple 1200 RS t f 0 2026-10-06 17:07:33.654736+00 2026-10-06 17:07:33.654736+00 \N 1160.0
332fdf68-c96c-4816-a696-56ca43a5e21d Triumph Speed Twin 1200 t f 833 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 1200.0
c98db454-1a6c-4926-a16f-3794e6176b70 Triumph Speed Twin 900 t f 832 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 900.0
6048dd2f-ed42-4c75-9416-ebe893543b02 Triumph Street Triple 675 t f 834 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 675.0
e5eac9c2-460e-4453-9dfa-8a1864e0728f Triumph Street Triple 765 t f 835 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 765.0
bf19a565-22a0-4c02-8a37-7f289ac1cd10 Triumph Street Twin t f 831 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 900.0
67f35261-da0d-4afc-b293-4c6915cbe770 Triumph Tiger 800 t f 843 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N 799.0
20f1ef41-e814-4a80-bb29-6fb638faa271 Honda XRM125 Motard f f 21 2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
e5ad2fc0-042b-4002-a572-8ef746565c55 Honda XRM125 Motard FI f f 22 2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
f922431b-0d82-487b-bcb8-287015b74d90 Honda Beat FI f f 24 2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
b80bb4f0-4acd-4dff-a52c-a3a41f2a0838 Honda Beat110 f f 25 2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
c9db41c2-4d55-41e3-8c33-9ef3884d6174 Honda Beat110 FI f f 26 2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
39270f86-0a76-4940-b01a-7a328ff22dd3 Honda BeAT Premium f f 27

2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
1b058270-5cd0-428f-89b6-92b568c8ec7b Honda BeAT Playful f f 28
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
00b8dac4-0609-4a9f-8008-21ab5e72d2dd Honda Click110 f f 29
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
08e5face-83a3-40a9-ba4b-807693ce195f Honda Click125 f f 31
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
ef59e453-c899-4760-9b2b-a425b93c3052 Honda Click125 FI f f 32
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
3c707354-60c4-49a8-abd1-c69fe1fb3d5a Honda Click160 FI f f 35
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
0d89b6fb-b04f-49ef-98f8-525d9354bd07 Honda Click160 ABS f f 36
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
16bcda0a-621e-41ed-bf85-b26c80d7886d Honda Air Blade110 f f 37
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
a40785da-b101-4ac1-84af-778080bc88fb Honda Dio110 FI f f 41
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
5915b1a2-c47c-4f06-97e2-c38281771a27 Honda Scoopy f f 42
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
d9339212-a2de-4d9c-9128-34001cee4048 Honda Scoopy FI f f 43
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
b7c8a83c-b19e-4f22-a2fd-3001eafa95d1 Honda Genio f f 44
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
02ad46ec-d60f-4d86-abac-bb89c060ee6a Honda PCX160 FI f f 49
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
6dad1b20-cf3f-4c34-9c83-b582ecc2887a Honda Vario125 f f 50
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
535c9eb6-41be-4a56-93ee-949fc9de9adb Honda Vario150 f f 51
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
898ebc53-2413-4315-9213-797637f60ed4 Honda Vario160 f f 52
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
7056f8c8-62aa-44c1-9b38-cc1d6f6b859c Honda Navi f f 53
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
13964c15-2b44-465b-a1f1-ee4b188c6735 Honda Giorno f f 54
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
e09365d5-622d-4d46-bfa2-15788bf586ab Honda Giorno+ f f 55
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
6e301a6d-f158-4684-b1d6-bc9c8e136d80 Honda Zoomer-X f f 56
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
a4a69a1c-1ca0-4a76-b5d0-11a3b624975d Honda Spacy f f 57
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
223ec00b-af08-420d-83dd-13cc7b88d4d9 Honda Spacy FI f f 58
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
9518be1b-38a4-460b-a94a-ea9f58ba8354 Honda TMX110 f f 59
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N

076cfecd-c4c0-456b-98cb-6f20401336fb Honda TMX125 Alpha f f 61
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
c17aaef0-4dfb-4ebb-bb08-cd607bb9c708 Honda CB110 f f 63
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
98283f9c-73c0-445f-a91d-d32978742900 Honda CB125 f f 64
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
46304e91-df2e-40f5-9aa1-e21898dced63 Honda CB150 f f 65
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
753b6596-880c-4314-96ab-b9c7d6caa134 Honda CB150F f f 67
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
ab1a2977-1270-4735-be54-a9e1d75ae01b Honda CBR250R f f 70
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
1e948074-9a45-4d84-aa7c-7b737ec6baef Honda CBR600RR f f 72
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
6b1df5e7-367e-490a-b05d-93aa37f4163d Honda CBR1000RR-R f f 75
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
9948650e-7766-4c50-a1a7-83edac4f3480 Honda Rebel250 f f 76
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
2d40412a-bf33-4afc-b61e-bf5e3bc54602 Honda XR125L f f 81
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
7dbca00b-8dc5-41f1-9d8e-f1938e2cd662 Honda XR150L FI f f 83
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
70491ba9-eeb8-470c-85d0-2b0c30f1881d Honda Africa Twin 1100 f f
95 2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
\N
3ba7ffc5-809f-41cb-a456-ab0847081bfb Yamaha YTX125S f f 98
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
6442c5f1-55a5-4101-a890-9b1fcf380aa2 Yamaha YTX125i f f 99
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
ad6310c3-ecf1-4977-b8c2-140db8628483 Yamaha DT125 f f 100
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
2c98de87-3ecc-46f6-a2ce-2b2a559eb449 Yamaha DT175 f f 101
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
65ee2212-20ec-449a-a1fd-7816a299817f Yamaha DT125LC f f 102
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
0489b97f-45a6-412f-995f-1bdfa4e51b4f Yamaha RX100 f f 103
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
41edf655-e89f-45fe-bed4-1651089198ba Aprilia Aprilia RS50 f f 859
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
e2932308-c1c8-477b-8930-134466a7a5aa Honda Wave110 Alpha f f 2
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
ba8f85e8-e163-4eed-87f6-e6a24593da71 Honda Wave110i f f 3
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
9f3ddc2f-dd93-4e50-9007-cbf500267bb5 Honda Wave125i f f 5
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N

309639ba-13c3-431f-93ba-cae98e802d11 Honda Wave125R f f 6
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
14c16b65-335e-4be1-ba0e-777ff8500e40 Honda Wave125S f f 7
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
19de3fda-04b0-4172-beff-e9b6bcf61f52 Honda Wave125X f f 8
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
6a7fffba-5a2f-414e-b085-4c9a18c97987 Honda Wave125X Alpha f f 9
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
3e9c6736-ea64-450e-a127-192d5a79352a Honda Wave Dash110 f f
10 2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N
\N
d9f92a7b-2153-4224-bd3d-d43c7d186134 Honda Wave RS f f 11
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
cf926e81-154e-458d-9f7f-d60074e19963 Honda Wave RSX110 f f 13
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
3d128fe4-e00a-44cd-ae96-16defe580b9d Honda Wave RSX FI f f 14
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
1576509d-3289-4266-baca-719cd140465b Honda XRM110 FI f f 16
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
839529b4-a884-4e0b-972f-cbfaac89a903 Honda XRM125 FI f f 18
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
2db42b22-e6a8-4255-814c-e4efddb2a388 Honda XRM125 DS f f 19
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
5cc01387-4217-4dc0-955f-12a4e1a3ba8d Honda XRM125 DS FI f f 20
2026-09-20 00:34:28.340367+00 2026-10-06 17:07:33.654736+00 \N \N
10087b04-87cc-406a-9c11-e27583649b53 Yamaha RX135 f f 104
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
79d9f18e-ad50-4159-83c9-23180220b6af Yamaha RS100 f f 105
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
da2ab420-620d-4bef-9936-3cb68048a169 Yamaha RS110 f f 106
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
3af92942-a5de-4000-87a4-d5c9c0b78cd1 Yamaha Crypton f f 107
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
1ad34753-49ea-47af-9087-dd9d04d5ccf8 Yamaha Crypton110 f f 108
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
d215cf21-cbac-4d6e-b241-37ece1568632 Yamaha Vega f f 109
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
91cf28af-5407-44bc-93cc-cb756067a6ca Yamaha Vega Force f f 110
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
3db36536-20d7-4687-8826-dc6ae7ef52e3 Yamaha Sight115 f f 112
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
26ee37db-bbfb-4364-9fa7-44f00ba3ad5d Yamaha Mio f f 113
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
0fccc52d-ea2d-4c07-9550-4c107ff5fec9 Yamaha Mio Soul f f 115
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N

f20cd99c-69e7-424b-9b58-b10dd5f932e6 Yamaha Mio MX f f 117 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
77e09383-8060-4dfb-95f6-122962828749 Yamaha Mio Aerox f f 119 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
aab32cd4-038a-4064-ade0-5a0ee98511e8 Yamaha Mio Aerox FI f f 120 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
415d805a-e1b0-4a0c-84a6-06c4e9083326 Yamaha Mio Gear FI f f 122 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
f872edd5-16cc-40d9-99b8-6c36fc94ddab Yamaha Mio Fazzio FI f f 125 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
f70c6c14-6781-48c2-8ae9-40ec1ff6ce6c Yamaha Mio Fascino f f 126 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
a80265f3-e411-4ba3-a43d-b005d04fce33 Yamaha Nouvo f f 127 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
142ac7c2-c2a0-4dd0-9048-de7575d59c19 Yamaha Nouvo Z f f 128 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
c8502087-59e7-4888-b595-d7aaeb8b5f14 Yamaha Nouvo Elegance f f 129 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
f6a5fccc-988e-4638-8338-d4686a5b2958 Yamaha Nouvo FI f f 130 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
20d011f6-9e7c-4690-9fb2-02dd95205537 Yamaha NMAX155 f f 132 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
d4615897-369f-45d5-b22d-ca1f72906c08 Yamaha NMAX FI f f 133 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
c3849178-a730-4d76-94c4-41a6980f13a3 Yamaha XMAX300 f f 135 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
f31f237c-3f33-42fb-b4ee-55588c001d15 Yamaha Aerox f f 136 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
0757a5e2-e976-4789-bc3d-f64fa12459ef Yamaha Aerox FI f f 138 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
1cc6073d-f024-45d7-9dbf-8fa81966689f Yamaha Fino FI f f 140 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
75848947-736b-47b0-8e6c-c93e30b365a2 Yamaha Majesty f f 141 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
984b9425-03b6-4267-a7e1-d120859292aa Yamaha Majesty125 f f 142 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
34a69ec7-187e-471d-b770-719ed97109b3 Yamaha Grand Filano f f 143 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
a9b1ba6b-ec02-4634-8425-3d1a3e212cfe Yamaha Grand Filano Hybrid f f 144 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
a4f3168b-09fa-4660-8707-37c7808dab9c Yamaha Sniper155 FI f f 148

2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
3a643737-17e6-4faa-a552-c9475b93db45 Yamaha YZF-R15 f f 149
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
6ea2799a-8c50-4f52-9a45-d8621385bcc9 Yamaha R15 FI f f 151
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
7064e44c-86eb-4140-9a95-c88ee148edcf Yamaha R25 f f 153
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
7a7fe9d8-e00c-4493-8ee5-d5158b82a9ee Yamaha FZ16 f f 159
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
a641f29e-63c2-4dd4-be16-1e2173f88168 Yamaha FZ25 f f 160
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
62bfd490-a68d-4d6e-b3f3-428fbc17f050 Yamaha FZ150 f f 161
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
2b8b5399-0969-4f5c-ad3d-bafacc5e6fd2 Yamaha FZi f f 162
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
b7f88b71-63ed-40c4-9a9f-db4c82f7e889 Yamaha FZ-S f f 163
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
f369077f-ba39-4e22-b696-a2a133f55e6c Yamaha XTZ125 f f 164
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
0d4543b1-e022-4493-afc4-e6504e7ff83e Yamaha XTZ150 f f 165
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
ae3ef5a0-76cc-415b-84b1-be868f059301 Yamaha XTZ250 f f 166
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
38ce7b0e-9364-46d0-b9d8-554656802c33 Yamaha WR155R f f 168
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
37b106ee-d244-4537-a107-e669a3e385eb Yamaha WR155R FI f f 169
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
ebce8b93-fc29-4f86-add0-fa6d7af8c082 Yamaha WR250R f f 170
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
31e91175-ef5f-463f-a832-ce198d64bc36 Yamaha SR400 f f 171
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
a80bb125-065e-41c4-a863-24df3666bea2 Yamaha VMAX f f 175
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
37e7df16-f611-4d4f-848f-d5f5dc98e974 Yamaha R6 f f 178
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
962c8e0e-3283-4d2d-8d17-c67528da136f Yamaha R1 f f 179
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
77ed868d-11e4-4a6b-a079-38d19cb23671 Suzuki Shogun f f 180
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
a3931086-fa6c-4123-9370-9b689d5e6f58 Suzuki Shogun Pro f f 181
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
595ff348-4d9d-4d48-a397-13a5abd891ff Suzuki Smash f f 182
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
f4117ce2-b5bb-405f-8c8d-cd51e54c7719 Suzuki Smash115 FI f f 184
2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N

dba394cf-388a-472b-95df-eb1731c156b9 Suzuki Raider150 f f 185 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
8c2b66aa-d472-447c-b433-14b9daac7c0c Suzuki Raider150 FI f f 186 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
18755be4-965c-4c7d-85b3-8ab1dc295fa1 Suzuki Raider R150 FI f f 188 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
34f3e065-1fb7-45c5-84da-686347baf847 Suzuki Raider J f f 189 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
554d490c-1fc3-4d1f-a920-dd82b64ecaae Suzuki Raider J115 f f 190 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
6c53fa2b-c33b-4c6a-9e48-476113cb5785 Suzuki GD110 FI f f 193 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
2f8c20fa-b0cd-425d-ac30-c4f8ae5f193f Suzuki Thunder125 f f 194 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
ac319fb0-a19b-48a9-9741-e151a9ea6519 Suzuki Thunder150 f f 195 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
7bbe5417-902c-4197-8f28-61e5be50d41d Suzuki Skydrive f f 196 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
4e8bf4f9-81b5-40f3-86ef-fa1bd734e8de Suzuki Skydrive FI f f 198 2026-09-20 00:34:28.495208+00 2026-10-06 17:07:33.654736+00 \N \N
5742aa98-0ed1-4c7f-927a-d03a8299318b Suzuki Burgman Street FI f f 201 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
df039e97-36f6-4bbe-af2c-8b26085dda9c Suzuki Burgman 125 f f 202 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
e11d771f-555b-4a0f-8b98-79752513a23a Suzuki Address FI f f 204 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
bc8026fa-7b53-47ff-9fcb-b72b3b19982d Suzuki Hayate FI f f 206 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
b11f81df-5c80-44bb-be3c-76d1683d5cd2 Suzuki Raider150X f f 207 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
711de54a-24df-4874-a39b-e5d7e913f063 Suzuki Gixxer150 f f 208 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
cc59807f-703c-44e9-8850-2a4cc91618ee Suzuki Gixxer FI f f 209 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
64af9b10-0e40-4319-b256-8d70b859760d Suzuki Gixxer SF f f 210 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
5b85dbd3-07d1-4298-b81c-78b171866095 Suzuki Gixxer SF FI f f 211 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
80602b66-8801-4aa1-944f-374d986ff009 Suzuki GSX-S150 f f 212 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
6f083a22-22a5-43a4-bd7b-242ab6a231f4 Suzuki GSX-R150 f f 213 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
d7f5e4de-0360-4ad3-a613-7ba6cb363c9f Suzuki V-Strom250 f f 217

2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
cda30375-5411-47ee-8782-31dd7052162d Suzuki V-Strom800 f f 219
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
0b04aec1-e679-443a-8c10-063c8f5adde0 Suzuki SV1000 f f 222
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
34ccffb7-9277-4da5-a2aa-dfa56ddcc607 Suzuki RM-Z250 f f 225
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
cad7cef2-2799-4223-a8d3-db2cc4093f2e Suzuki RM-Z450 f f 226
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
5c9372b0-0fe5-416d-bea8-1203aac433cd Suzuki Boulevard M109R f f
227 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
\N
3f9361c4-c55c-44c5-8b1a-5ccd877d765b Suzuki Boulevard C90 f f 229
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
982d104d-a9d0-4a5f-9590-5a9464313f10 Suzuki Katana f f 231
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
6e714250-cdc2-4d8e-a4cb-c58b99843682 Kawasaki Bajaj CT100 f f
233 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
\N
fb5f8226-69da-4292-ab8c-83b7d77ba53b Kawasaki CT100B f f 234
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
879495aa-cb87-4034-aac9-e9567ed5e72c Kawasaki CT125 f f 235
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
e11283b5-4d3c-4f12-837b-53cc54fed768 Kawasaki CT150 f f 236
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
7c4afe2b-4940-4688-8e2c-16746b20f647 Kawasaki Barako175 f f 239
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
d5638c8e-e0f6-43d6-9d26-254d76ad9068 Kawasaki Bajaj Rouser125 f f
240 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
\N
0b044e8c-3596-4e9b-ba9c-90e991dfc0fe Kawasaki Rouser NS125 f f
241 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
\N
98bdcaf4-95ff-4320-8581-362acf39250a Kawasaki Rouser NS160 f f
242 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
\N
7fdada17-91c7-4db6-b1df-f7e1349f705d Kawasaki Rouser NS200 f f
243 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
\N
0311ba77-3405-4780-b23a-5b5cccd7009a Kawasaki Rouser RS200 f f
244 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
\N
76d734c1-de48-4475-8cdd-b6a353c4868f Kawasaki Rouser 200NS f f
245 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
\N

2c86cbc9-3616-4e21-9965-840a4e0a0f82 Kawasaki Rouser 220 f f 246 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
eef6a35d-4875-4b7e-97eb-ba307fc48cf1 Kawasaki Rouser 400 f f 247 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
6b4397d4-660b-4ca3-b93e-0bc718dc782a Kawasaki Dominar400 f f 248 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
dd0ceb05-fe28-4214-a612-80d36ee5f05d Kawasaki Dominar400 UG f f 249 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
7c39c52e-ce25-4681-adec-38ab51b08219 Kawasaki KLX110 f f 250 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
44007683-b30e-4ce3-aeec-efa361b54d8f Kawasaki KLX140 f f 251 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
a5c881a4-ca26-4696-b37a-1aa8260221fb Kawasaki KLX150BF f f 254 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
ff9b8c66-2c05-4dcf-b486-1336fc258cc3 Kawasaki KLX250 f f 256 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
22ed58e3-a52a-4319-9382-6158e32601a8 Kawasaki KLX300 f f 257 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
5f03d011-e878-4f0b-872f-18211cd0d9a5 Kawasaki KDX125 f f 258 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
e8e7322e-22a6-4756-9f58-f3eb89b92b33 Kawasaki KDX200 f f 259 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
845d1e33-a772-4955-b9dc-2dd2af1cc0f2 Kawasaki KDX250 f f 260 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
697d3ad3-1af2-4333-9131-bb00bb080805 Kawasaki Ninja250 FI f f 263 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
0b7e0fa2-d6c0-42dd-8693-d76cf85be741 Kawasaki Ninja500 f f 266 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
bac7cb15-82fd-43d7-8686-edb87b839e01 Kawasaki Ninja ZX-4R f f 269 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
eb77ba60-f70a-4bb0-8b57-3d5e39fbda37 Kawasaki Ninja H2 f f 272 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
f77e09b4-3fbc-418a-a960-f9c37bc1cbe0 Kawasaki Z125 f f 273 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
d9add240-e14d-4226-b588-d9ecfe587098 Kawasaki Z500 f f 277 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
01fb98c0-c4a4-4930-addb-af666e9bbeee Kawasaki Z750 f f 279 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
ab28e8e8-315a-46cb-aed6-26f20213b951 Kawasaki Z800 f f 280 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
f1392aba-e60c-4807-a611-c26060d948b4 Kawasaki Versys300 f f 284

2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
f3016c00-d5de-4503-93a4-47b278d6133a RUSI Rusi Classic250 FI f f
291 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
\N
807a3cee-b4ae-4669-9407-b58511d485e9 RUSI Rusi Classic400 f f
292 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
\N
c3eebad9-d4a9-49ba-8e4f-74cd7aee325f RUSI Rusi Adventure150 f f
293 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
\N
c13531fd-1e8d-411b-8bc7-0ad150f373ed RUSI Rusi Adventure250 f f
294 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
\N
0f06f48e-82a5-4d08-bbf0-61ea0b1fb292 RUSI Rusi Adventure400 f f
295 2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N
\N
ed6ab506-0885-4d79-94ff-89783a6cb50e RUSI Rusi Alpha150 f f 296
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
ac528558-939e-4c41-a006-016b0210a238 RUSI Rusi Alpha250 f f 297
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
3bc97d00-cadf-41ed-b531-3d27f0616013 RUSI Rusi Sigma250 f f 298
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
e9fdb7f5-d147-49a1-9a9a-1cd5703bf540 RUSI Rusi Sigma400 f f 299
2026-09-20 00:34:28.622103+00 2026-10-06 17:07:33.654736+00 \N \N
b20cfbb2-7da0-485c-bc83-8a55dbbce76d RUSI Rusi Royal250 f f 300
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
21ada317-0b4b-4e48-b694-5bed169e8071 RUSI Rusi Royal400 f f 301
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
4574734f-62e0-4f51-86e0-f7f5d9c8a1f3 RUSI Rusi TC125 f f 302
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
51d1ebfe-f807-44ce-8cdc-f1d4fb0e7e7c RUSI Rusi TC150 f f 303
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
fc05a743-7a9b-4f70-a58a-792a02af58c1 RUSI Rusi TC250 f f 304
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
697573f3-eb62-45ac-a8f9-0eac7396ade0 RUSI Rusi Infinity150 f f 305
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
aca133fa-8db2-4e50-af9e-9e63ed4e862f RUSI Rusi Infinity250 f f 306
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
9a3991b1-5b2c-4864-80b2-6f680860f8bc RUSI Rusi Urban125 f f 307
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
da4e5e3e-4f8a-4af1-8d6a-0ace300e3270 RUSI Rusi Urban150 f f 308
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
f721a9e1-748e-45be-a76f-0694f73cb27e RUSI Rusi Urban200 f f 309
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
e5d809d2-ad10-4692-a2ca-c58cc8c41112 RUSI Rusi Mojo200 f f 310

2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
13d09f58-4db6-4df8-a151-4efc15d0c42f RUSI Rusi Galaxy125 f f 311
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
7280348a-c053-4c4e-815c-ff204f4a3f0b RUSI Rusi Galaxy150 f f 312
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
642ad1c9-b0d5-4553-9b50-ee7ee4259419 RUSI Rusi Wave110 f f 313
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
90710e7e-82f4-4d26-a4af-36127a6679b3 RUSI Rusi Wave125 f f 314
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
b4ff46b7-987e-4af2-a4bf-0e28e31104c4 RUSI Rusi Sigma110 f f 315
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
291c94c5-60a4-4309-ad3b-f715e2ccc943 RUSI Rusi Sigma125 f f 316
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
f0bc12ed-75ee-4ba5-8e5f-ee2c8a1742f4 RUSI Rusi Passion110 f f 317
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
bcef4f0d-befa-46b0-9fc9-c3541859a000 RUSI Rusi Passion125 f f 318
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
9b49e1a5-fc46-403d-87e6-604df3f6787a RUSI Rusi Flash150 f f 319
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
9d0a14b3-487f-4263-a50d-eef9154c9074 RUSI Rusi Flash200 f f 320
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
1ba0d16d-6c50-4b3b-aef4-ec0b8a290685 RUSI Rusi 150FI f f 321
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
f313972c-d97f-4231-b895-e9e2c99e464a RUSI Rusi 175FI f f 322
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
604882d2-c72a-4376-8917-7849292832a8 RUSI Rusi 250FI f f 323
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
85b2657b-38f3-4c3d-9d88-342e025f58a2 RUSI Rusi 300FI f f 324
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
c4e67041-8383-46bd-8c6d-9edbec65e2e7 RUSI Rusi 350FI f f 325
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
ff4b56b7-7168-4a04-88a6-bd8e2b200738 MotorStar MotorStar Zest110 f
f 326 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00
\N \N
0252d357-b1fc-411a-a1bd-1f2d943c9f4f MotorStar Zest X110 f f 327
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
e2483e9c-a122-4969-ad89-13de4e26a455 MotorStar Zest X110 III f f
328 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
\N
08f881c7-5207-4252-8949-53520a194bf0 MotorStar MotorStar Nicess110 f
f 329 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00
\N \N
2d3b5034-5b99-417c-aa76-71a3b7f26a81 MotorStar MSX125S f f 331
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
5cf03183-8928-4b23-bea6-977ee2e664aa MotorStar MSX125S-II f f

332 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
\N
37dcbfa0-4860-4de3-8a28-5252b88ede5c MotorStar MotorStar Star-X125 f
f 333 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00
\N \N
cd0f2d0d-ddcf-42f1-8505-ca19db5da2b1 MotorStar Star-X155 f f 334
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
32567617-0f8f-4979-b9d4-cc3deaa627ed MotorStar Xplorer Z150 f f
336 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
\N
1e2dba4c-97ad-4886-85bc-065895e06215 MotorStar MotorStar Cafe400 f
f 337 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00
\N \N
407cca84-c460-4ec2-b764-e80549d99fd8 MotorStar MotorStar Classic250
f f 338 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00
\N \N
adf83336-d9d5-4128-b932-6effe71f8e45 MotorStar MotorStar Urban125 f
f 339 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00
\N \N
bb3754bf-ceea-4148-95f7-5a08fcf6085f MotorStar MotorStar Easyride150 f
f 340 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00
\N \N
060e77fc-7fb1-4275-80c7-416c41874613 MotorStar MotorStar Moto-X200 f
f 342 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00
\N \N
8bc9399b-8243-4027-8311-4add19366eb0 Skygo Skygo Boss f f 343
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
7ac4fb50-2f07-4091-bee6-3ffa33fcd479 Skygo Skygo Archer110 f f 345
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
a7b8d515-5da9-493e-919b-0c09250718b2 Skygo Archer125 f f 346
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
6961fa7b-fc34-45d1-978f-843c185bd584 Skygo Skygo Duke110 f f 347
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
777d4f59-4daf-4a7c-9ac2-d188eef281c0 Skygo Duke125 f f 348
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
bbb84cc8-3fab-4de2-b3e7-fa0c6b9151eb Skygo Skygo Wizard150 f f
350 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
\N
5c10ebaa-02dc-4680-9da4-3e2fcce34be3 Skygo Skygo Prince150 f f
353 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
\N
02cfdea5-0991-4581-be5b-62bf6f11d7d6 Skygo Skygo Cobra150 f f
354 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
\N
7bbf9897-c081-499c-aa26-94aa66bc7724 Skygo Skygo Stallion150 f f

355 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
\N
2074f738-3a94-4730-a8aa-35d5118a3d14 Skygo Skygo Stallion250 f f
356 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
\N
3fae54d8-b561-459a-a70f-6d747614284c Skygo Skygo Explorer200 f f
357 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
\N
7c979b14-6479-48ef-bcde-38bd40d8988a Skygo Skygo Adventure250 f f
358 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00
\N \N
cb158fd4-54c3-4b39-a1b9-9dd2f76e3b01 Kymco Kymco VISA R110 f f
359 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
\N
5697563f-564f-42a7-8e3f-78ed3948cff4 Kymco VISA R110 FI f f 360
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
401addb0-4f1a-43f9-9e97-659ba0b3bd4a Kymco Like200i f f 363
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
9978059f-ed4c-48a2-9a85-33edddc989de Kymco Like150 f f 364
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
eabc3df5-0404-4636-a694-2e8d14089b33 Kymco Agility150 f f 366
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
92633a97-ce94-4fd2-a186-e3cf52389900 Kymco Agility200 f f 367
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
62e1d99f-e777-479f-a9f7-c943ee7e4ef8 Kymco People S150 f f 371
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
c98ae92c-e4d5-496e-8ed8-d864a5db07dc Kymco People GT300i f f
372 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
\N
92fa9ba3-e33f-4078-959d-2cc1d6bec691 Kymco Kymco Xciting250 f f
373 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
\N
d139838d-bdda-4cbf-84a9-823af7a3e3a6 Kymco Xciting300i f f 374
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
401295d8-2f5b-45b4-a13b-e980f962188b Kymco Kymco KRV200 f f
378 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
\N
26d44169-3401-4f10-b5b5-ee3517249898 Kymco Kymco Downtown200i f
f 379 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00
\N \N
0eaf240c-c5d9-49a6-8472-700cda1653c7 Kymco Kymco Racing King180i f
f 384 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00
\N \N
1c24c686-4b6c-4d4f-80e1-96f11a0ebf99 Kymco Kymco Racing S150i f f
385 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N

\N
669d6daf-9351-402d-9ef3-552575190b8c Kymco Kymco Spade150 f f
386 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
\N
05a63b70-351a-4ece-a2ea-55494fed4a37 CFMOTO 250NK Lite f f 389
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
cc455e48-998d-4ca2-98d4-f77cb3f61192 CFMOTO 675NK f f 393
2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N \N
b6ae1c81-2cbe-46e4-8ce8-40e2ba81d4ad CFMOTO 250SR Lite f f
396 2026-09-20 00:34:28.732175+00 2026-10-06 17:07:33.654736+00 \N
\N
270b6368-8fc4-4160-93f6-fdcfb76d1a53 CFMOTO 450CL-C AMT f f
413 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
\N
0191d005-faf5-46c6-9380-4b04bd102e15 CFMOTO 700CL-X Heritage f f
416 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00
\N \N
4b565b6c-1324-49e7-bfa4-87557ef2a5ba CFMOTO 700CL-X Sport f f
417 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
\N
7e9f0ba8-ee68-4446-988b-d193910a9b6e CFMOTO 700CL-X ADV f f
418 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
\N
985cded9-bbe9-4679-aa72-bb3a9d6f4f9c CFMOTO 650TR f f 420
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
e8b75b76-fa93-4a9d-af78-4aca9661bbe4 CFMOTO 1250TR f f 421
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
02f58763-506a-4645-9709-c4c36a3fddaa CFMOTO Papio125 f f 422
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
3424c67c-3ac8-4074-8a5f-ace843dc3803 CFMOTO Papio ST125 f f
423 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
\N
713753e8-798f-41e8-9358-ab3ee68d9adb CFMOTO Papio XO f f 424
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
43097d4a-0593-4f64-b5a7-ff850e60dd31 CFMOTO Papio XO Racer f f
425 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
\N
b92e1f04-d819-4659-a7cc-aa867f8eed4e CFMOTO Papio XO Racer V2 f
f 426 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00
\N \N
62e10c4a-9be4-49fa-884f-bc432dacf325 CFMOTO Papio XO Trail f f
427 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N
\N
b3e452a7-21b9-4e64-9dec-a9ebbcdea646 CFMOTO 230 Dual f f 430
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N

03ec2115-fd8f-4d94-ac73-cc03456ac386 CFMOTO 250 Dual f f 431 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
75b28dad-3b43-4f63-840b-b1f012da370d TVS XL100 Comfort f f 433 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
7655956a-6fb6-445e-94a5-e88e5b39f909 TVS XL100 Heavy Duty f f 434 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
9cf84fc7-36c0-4b18-8cfb-ecb5f4ec8b5a TVS TVS Neo f f 435 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
d5061be2-26b2-4157-be00-5f9c687dafe0 TVS Neo NX f f 436 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
f70e0c66-154e-4c73-9844-85517c29a317 TVS TVS Rockz125 f f 437 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
04a806e9-e659-447c-8479-610368f59c90 TVS Rockz125 FI f f 438 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
bf38536a-99e0-4c99-8eba-98d4f9129558 TVS Apache RTR160 4V f f 440 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
8bc3b35d-1cea-4a06-99de-22125d538b27 TVS Raider125 FI f f 445 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
80d4bd5d-aba5-4dee-9da6-8c48392322a8 TVS TVS Max125 f f 446 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
c88b2da9-38bf-4385-973a-0cbeaa24a1f1 TVS TVS King f f 447 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
12fb63c9-254a-4b8e-82bb-f90180bd1a67 TVS TVS Dazz110 f f 448 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
753187f4-73e4-468f-a23e-0bb108259d25 TVS TVS Wego110 f f 449 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
3ac9003c-9feb-4284-bf71-e046481ca34d TVS TVS Jupiter110 f f 450 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
01b1d978-fd65-4385-a320-93d8f8d81aa6 Keeway Keeway Superlight150 f f 455 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
a9a28753-3a1f-4667-9881-886c20260115 Keeway CR152 FI f f 458 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
30928463-15f6-4a53-aaca-d7c000d8f3b0 Keeway RKF250 f f 462 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
0b60a384-cf12-4ac0-ad1f-769a2101dca3 Keeway Keeway Versilia150 f f 467 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
79d33f97-136c-4d3f-a8df-b1aef827d0d2 Benelli Benelli RFS150i f f 468 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
7ab40ba2-8905-4d7e-8bd9-67a2fc7c9841 Benelli RFS150S f f 469 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
1715026f-b9a8-465b-9f5f-ecd05b7f57cf Benelli TNT135 f f 470

2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N 757acc3c-fb64-4a80-8f3d-b5b770ba0ed0 Benelli TNT152 f f 472
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N a9a83da9-b2d1-4bee-90aa-5e63b617f6c2 Benelli TNT249S f f 473
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N 3eea1c27-7ce6-475e-bc31-076c9e541b51 Benelli TNT300 f f 474
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N f27bf407-de3b-4aa7-b964-9667693bb022 Benelli TNT600i f f 475
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N 2bfe582b-5ae5-4427-a651-69878e9d92fa Benelli TNT600GT f f 476
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N a988585d-791a-4be9-8544-ae97604b445b Benelli BN150 f f 478
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N b3c10de7-eade-47ce-90f3-3201e370dbde Benelli BN302 f f 480
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N b2e2b9f4-a6b9-4423-8c4e-2440fc52bf23 Benelli BN600 f f 481
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N 25a32d64-eb41-4926-8108-025822fb194a Benelli Leoncino500 f f 483
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N f677802c-54b9-41f3-b2a2-efb94883d567 Benelli Leoncino500 Trail f f 484 2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N
9edf1937-78d3-4d8b-a1d0-d8d78798786e Benelli Leoncino800 f f 485
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N 6c96b5ea-71de-4347-8b84-065d2f3df030 Benelli 302C f f 492
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N 47105088-0a7d-44e5-8d65-53f94c70b706 Benelli 752C f f 495
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N ab0ce322-c4ce-4ca1-8a9d-a996a45fda57 Benelli 1200GT f f 496
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N 10440212-a095-47e8-8f77-f5a05940d249 Benelli 1200GT SE f f 497
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N a73869ad-d4d7-49ac-89da-d7dea1ce834e Benelli Motobi200 f f 499
2026-09-20 00:34:28.857734+00 2026-10-06 17:07:33.654736+00 \N \N 71f0eb0f-7af1-482f-99ec-d226f0ad1018 Benelli Motobi250 f f 500
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N 8598c732-a004-4810-be30-5cfbe8cd032b KTM KTM Duke125 f f 501
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N d4ef738d-8563-4bd1-a6c8-349f007566e2 KTM Duke200 f f 502
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N 0e2b0aad-a9be-4ae3-89a1-d819ee62079d KTM Duke250 f f 503
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N d1998706-d63b-4b17-9330-09b9714e7af8 KTM Duke390 f f 504
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N 3ee834df-3b38-4620-9b0a-73f2292dc0fb KTM Duke690 f f 505

2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
3ca588fb-5bf5-4d69-b62e-251827196d29 KTM Duke790 f f 506
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
dc97b6d0-b28a-4e14-a5f6-36897f356ca6 KTM Duke890 f f 507
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
8a3ae84b-cbb0-45f3-be46-761d300dd5fc KTM Duke990 f f 508
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
7ff50c74-178b-462b-b868-304ae2562796 KTM Duke1290 f f 509
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
e1d14114-7a4d-4329-b9f2-6c09a2bd16f0 KTM RC250 f f 512
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
8cd91935-ffb4-4dee-8a1f-ccafba88dbc7 KTM KTM Adventure250 f f
514 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
\N
ec5d4e28-175e-40b5-a55c-001ab51cd7d1 KTM Adventure390 f f 515
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
f0ea566c-049f-44fb-bd09-e1fe52d7e9d3 KTM Adventure790 f f 516
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
06bdd842-4be8-4bae-a873-b194536fa66e KTM Adventure890 f f 517
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
ef08c21c-a5c5-49fe-a19a-8c01257f3bc7 KTM Adventure1050 f f 518
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
a0481ccf-0dfe-4a18-bba6-b01603828eab KTM Adventure1090 f f 519
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
f3e873b9-eff8-498f-a0d6-7d29e33cdf30 KTM Adventure1190 f f 520
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
5106d563-0d91-458e-8919-709822774e5e KTM Adventure1290 f f 521
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
4f169832-360b-4373-b167-64131c82bee5 KTM KTM 450 EXC f f 522
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
321d90a1-947f-4400-bed1-0ecfe3c847e7 KTM 500 EXC f f 523
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
63d789c0-537d-4953-a282-83dae819bf85 KTM 690 Enduro f f 524
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
ec285de5-a949-4e99-b7e8-a169f05f6c9a KTM 690 SMC f f 525
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
996474b3-6033-433f-a7fc-3c5947ff4025 KTM KTM 125 SX f f 526
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
e2e0068e-be06-4698-be90-ea82d22ee25c KTM 250 SX f f 527
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
2a265220-2612-473a-be34-c626d75a219e KTM 250 EXC f f 528
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
cb9d789d-61d8-40a7-b760-ab5aee2d33e9 KTM 300 EXC f f 529
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
3aebb621-5cba-4910-bad2-43e186159e3a KTM 350 EXC f f 530

2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
22234a12-99e0-4507-82a6-5cf174f55760 KTM 450 SX-F f f 531
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
3a4c5e46-6afa-401e-9977-d675d0810a89 Bristol Classic400 f f 535
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
924e1f1d-ed23-46c4-8e09-0727cfaa0efc Bristol Bobber400 f f 536
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
8aa6d40c-fd8c-430f-bc11-f7eedfa98ddb Bristol Bobber650 f f 537
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
363c40b6-9776-4c52-a939-ef4406d6a7d4 Bristol Cafe400 f f 538
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
febcd655-9f18-4af7-a6c6-57c91402c30b Bristol Cafe500 f f 539
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
7b833ca3-b088-4585-b379-471276835c53 Bristol Scrambler400 f f 540
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
21af9dce-0653-4a78-99b4-63fbc7008d37 Bristol Scrambler500 f f 541
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
3039eefe-667d-4d86-93a6-3652cb5c1f22 Bristol Adventure400 f f 542
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
0dc7fc3b-a73c-4ed5-b1d0-a2e0b24a6d2e Bristol Adventure500 f f 543
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
32facad5-5fcc-4eeb-a03a-6e7043fc495c Bristol Bullet500 f f 544
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
9c4da96e-bbf5-4e76-a6a5-6afb75e5d86b Bristol Maxie500 f f 546
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
2b412a51-e60c-4170-a1fa-59e84dc019c7 Bristol Veloce500 f f 547
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
704a9f6f-5427-4dc6-9eb5-5556a145f083 Bristol Veloce650 f f 548
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
be17a2cf-2e4d-4acf-83ba-ab0502edfdf3 Royal Enfield Bullet500 f f 550
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
5a3e060b-a53a-4028-a1e7-1d2ca1aae469 Royal Enfield Classic500 f f
552 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
\N
acafa2c0-7f60-40de-9f5c-d45569d8820f Royal Enfield Classic500 FI f f
553 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
\N
8c1f2763-099b-40af-89c0-93a29b20950e Royal Enfield Thunderbird350 f
f 555 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00
\N \N
cf224019-9101-40e4-b953-06e86b764cd2 Royal Enfield Thunderbird500 f
f 556 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00
\N \N
7b6dfb84-2d7a-4b7e-9ec7-3b969cb737f5 Royal Enfield Thunderbird X350 f
f 557 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00

\N \N
5f33e92b-e48d-4e4f-a82d-ace711e0a6ae Royal Enfield Thunderbird X500
f f 558 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00
\N \N
5f5ade22-9f42-40a5-9e51-ab5e8d25784e Royal Enfield Himalayan410 f f
562 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00
\N \N
4595198f-cc1c-41e3-853f-f81072f9aca2 Royal Enfield ContinentalGT f f
569 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
\N
3dbfa267-5be0-4901-a672-a110ea68e22d Royal Enfield Electra350 f f
573 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
\N
2679e8f3-52e8-496a-9b44-2e545f62a762 Royal Enfield Electra500 f f
574 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
\N
5d3d48d4-e153-40c0-ae87-74e80ff28719 Royal Enfield ThunderbirdX f f
575 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
\N
49c600e6-c62d-4c8a-85ca-e7e1262d62d8 SYM Bonus110 FI f f 577
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
8fcbf97c-ccdb-49b1-90dd-aa09393ecfb5 SYM Symphony125 f f 578
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
9ca642e8-5f9e-473c-b3a0-100bb6d1cd06 SYM Symphony ST200i f f
579 2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N
\N
bd893389-2adf-4363-965d-0b4c96437d8b SYM Jet4 f f 580
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
b1e1cd28-b242-4e09-93f1-96005d89297f SYM Jet4 125 f f 581
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
6d0e8b89-9b2f-4da1-a46e-26e1846df718 SYM Jet14 200 f f 583
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
34a28b49-0d9d-4c19-9b43-17dc4df3c7c0 SYM Cruisym150 f f 584
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
e19bdddd-7ff8-4d9d-9a28-31340bdf21d0 SYM Maxsym400i f f 586
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
d95d4ab2-8a09-4e9b-897b-f96855c1182f SYM Maxsym600i f f 587
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
3e493bd2-1e90-412c-b2b8-e2c7caf604a0 SYM GTS250i f f 588
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
f387f23d-8e96-4cfd-9468-e2cc8a330ea8 SYM GTS300i f f 589
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
6adea909-87cf-42c5-a4ea-9ed35fa3c994 SYM RV180 f f 590
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
886f9523-3b3d-4160-96f4-1afadc97310a SYM RV200 f f 591

2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
8de64862-4482-4438-bcd3-cdc36a78adf8 SYM Wolf200 f f 593
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
f82aface-1798-4213-bee6-d5d701c9dccb SYM T2 250 f f 594
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
9e7272c9-357f-4679-aa3e-9b7fe42ca534 SYM T3 300 f f 595
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
2e9a4dff-6d06-4baa-99be-6816287c15c9 SYM VF3i f f 596
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
53f0b1e6-eb8f-44b0-a0d5-636bced753f6 SYM VF185 f f 597
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
03bb8be2-13ef-41a1-9f87-f7888bb2cd41 SYM Husky300 f f 598
2026-09-20 00:34:28.972796+00 2026-10-06 17:07:33.654736+00 \N \N
487f2be4-aa99-4b7d-b49b-aa1d16771b5b Bajaj CT100B f f 600
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
9936db04-924f-4e13-8974-4a7790ae7b5e Bajaj CT125 f f 602
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
371366a9-ea2e-4fa4-a2be-295940b217c8 Bajaj CT150 f f 603
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
a0e00fb4-5c93-4da1-b827-cb43888b8691 Bajaj Boxer100 f f 604
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
b7f96575-8cd3-49a1-9abd-652a2c05a48c Bajaj Boxer150 f f 605
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
18d367f1-4fa7-4a4d-8cec-033ec294dd6f Bajaj Discover135 f f 609
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
b40dbec5-8b00-465b-9742-a8b879c27ba9 Bajaj Pulsar200NS f f 614
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
1560496c-8069-47b4-877c-c073bc7819e9 Bajaj Pulsar NS125 f f 615
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
9028ec10-19b6-4cd3-9f26-b933a5b12d76 Bajaj Pulsar NS400 f f 618
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
dbbe779e-0a0f-4806-a7e3-ac68c1c2294d Bajaj Pulsar250 f f 621
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
4d6bb2df-3a50-49f9-9f02-22ac9254be21 Bajaj Avenger150 f f 624
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
e60ae169-91b9-42c0-aa23-ab8149e588e1 Bajaj Avenger160 f f 625
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
d58c4509-9251-47a1-a0dc-1ac13869abcf Bajaj Avenger400 f f 628
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
0c80f572-aab2-4199-8645-ee3d685aa3f9 Zontes Zontes 125U f f 631
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
3f3b0b04-fcab-4176-b733-6b88f554e442 Zontes 150U f f 632
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
c01c4d52-e229-41a0-ade4-4add26ad7459 Zontes 155G f f 633
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N

b38ddf28-43b1-42f6-9b8b-b9a0c9d2dc15 Zontes 250U f f 634
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
3933ac80-194f-426e-b557-80b3900bf7b3 Zontes 310R f f 635
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
89ce3f43-c190-439f-a909-399bdd709209 Zontes 310T f f 636
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
fac2efb3-bc9e-4605-83e2-a8d07f2e208e Zontes 310X f f 638
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
c0be322b-56ac-41ec-9ff4-964bb16a0e7f Zontes 350R f f 639
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
8b6287a5-42be-4670-835a-1f60f0cd039c Zontes 350T f f 640
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
c7aa432b-530e-4692-850a-7f4c202297be Zontes 350D f f 641
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
5b2ae0b3-85db-48eb-8c99-ca011fb8de91 Zontes 350E f f 642
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
e88ff88d-5941-4ae8-ad3d-d281769f743e Zontes 350GK f f 643
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
bfc4a0e2-1916-4d6e-a93e-5782fe8e1ee9 Zontes 350X f f 644
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
be0be564-17b1-4ba4-a922-2f68f6b58e88 Zontes 350S f f 645
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
9064009d-b686-45a6-b4ae-cb13c5ee1525 Zontes 703R f f 647
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
6a946736-8a7a-4cda-8a21-1864a26a2617 Zontes 703T f f 649
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
ace82061-2806-42a6-be4a-ea5067de43da Vespa Vespa ET4 f f 650
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
c080756d-95bd-4c6c-9b59-57dcaf1557e8 Vespa ET8 f f 651
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
89f28af6-9ea4-4e96-9e57-66f9813a4c7c Vespa LXV125 f f 654
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
172c5fb2-839e-4191-a1b0-1cca6dedd090 Vespa LXV150 f f 655
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
70d1f510-10a4-4814-b9fe-0626657dbbd5 Vespa Primavera150 FI f f
660 2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
\N
5fe3ec26-bffb-4e93-8a37-702e1dd8ef61 Vespa Sprint150 FI f f 663
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
a7aa4bfd-e319-45c7-9b10-901035eb339b Vespa GTS300 Super f f
668 2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
\N
505319b9-4d4b-4b25-a6f6-b8a006e50d15 Vespa GTV250 f f 669
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
e8f7fa1c-09e1-4130-88cc-f1a9f37ad016 Vespa Sei Giorni300 f f 671

2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
6241cb50-02f6-4436-8378-e95ab0122b57 Vespa 946 f f 672
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
039cd56f-eb1c-4b78-ac57-ef0c6d8a70ce Piaggio Piaggio Zip100 f f
673 2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
\N
ddeb41fb-7493-40fc-a552-564324cd7618 Piaggio Zip125 f f 674
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
a674bb8f-d970-4be6-bea8-7f05140c1c9d Piaggio Liberty200 f f 677
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
8db1c629-75b0-462d-9563-9057c62739af Piaggio Fly125 f f 678
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
908d6e86-e3d5-4fed-a75b-094281dbd36e Piaggio Fly150 f f 679
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
6183fca7-7f31-4bf2-a2a0-61ed6ab340a4 Piaggio Medley150 FI f f 681
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
c574f0b5-5873-419e-9acc-d3cbb2f97541 Piaggio Medley200 f f 682
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
8ff4a5d1-40c7-4dc8-b84e-c80e49a34159 Piaggio X10 f f 690
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
ffbccfea-587a-4bad-b593-1d5c3dca7ac1 Piaggio X8 f f 691
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
c7d3af02-0ff8-40a6-a19d-ad5e277933b7 Piaggio X9 f f 692
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
4128c418-2f3f-41fe-8b1c-738dc1c51c8c Piaggio Typhoon125 f f 693
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
91988804-7c42-4954-94cd-71818d8911aa Piaggio NRG50 f f 694
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
989f1139-4d08-4581-a686-4c7b4eaf630c Piaggio Ape50 f f 695
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
ce97d3af-f4fd-4c82-8dd6-f881996e978d Piaggio Ape100 f f 696
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
97cb6081-542a-46cc-8d32-9e67cca87bea Piaggio Ape125 f f 697
2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N \N
b54f6699-fc6f-4ff2-b4b4-55c298bca89e BMW Motorrad BMW G310R f f
698 2026-09-20 00:34:29.080771+00 2026-10-06 17:07:33.654736+00 \N
\N
abf6d333-6ed0-4179-a14f-39676d52e234 BMW Motorrad G310RR f f
700 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N
\N
2a60d724-0d6a-4b25-83af-57bdfc0b1333 BMW Motorrad F800R f f
705 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N
\N
7bf29281-8633-4182-91fd-02c7d6af0e4e BMW Motorrad F800GT f f
706 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N

\N
6af2e709-6efa-4937-97ef-59a219ebec95 BMW Motorrad F850GSA f f
708 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N
\N
4e81c687-b959-4512-be2b-52dbdc5f082f BMW Motorrad F900R f f 709
2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
d171c59b-5f0d-4ca0-9bc0-902bcb37c5ce BMW Motorrad R850R f f
711 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N
\N
34672855-a17d-4cb9-9373-b83db1618a1b BMW Motorrad R1100R f f
712 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N
\N
0f1d2c24-5923-46a7-ba86-26f1cd779f5d BMW Motorrad R1150R f f
713 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N
\N
e361a7ed-4556-46ea-8b27-06d9cec1891a BMW Motorrad R1200GSA f f
715 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00
\N \N
e7d60322-2dd0-4557-b338-adc8929d346a BMW Motorrad R1250GSA f f
717 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00
\N \N
03a81ded-1cf5-41a7-8ea3-6387ac108303 BMW Motorrad R1250R f f
718 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N
\N
71143421-25d8-40e7-926c-59bad7fbcab9 BMW Motorrad R1250RS f f
719 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N
\N
19b97885-96ab-4a25-8835-780eb126876b BMW Motorrad R1300GSA f f
721 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00
\N \N
efbbda51-1684-49c5-b093-3c2f28b9fd0c BMW Motorrad R12 f f 723
2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
c3803c82-77ee-4fe8-a93a-9506df17c7a5 BMW Motorrad S1000 f f 727
2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
d1ca18a6-f908-47e1-94a3-da84ae1d7f09 BMW Motorrad K1200R f f
728 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N
\N
eff6be24-f35c-40fd-ac13-5f289421266d BMW Motorrad K1300R f f 729
2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
5eee5c46-1840-41d9-989d-4fbba4c0b726 BMW Motorrad K1600GTL f f
731 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N
\N
978166ff-03d2-4ba1-840a-a1b381556dbc BMW Motorrad K1600Grand
America f f 733 2026-09-20 00:34:29.188869+00 2026-10-06
17:07:33.654736+00 \N \N

ba33ffd2-6b61-4f27-a973-42fef63da92a BMW Motorrad C400X f f 734 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
ee2f8c5e-8f6a-4354-bee5-e5300454864e BMW Motorrad CE04 f f 736 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
53dd4934-675f-41eb-a4fa-42ba4eb881ef BMW Motorrad CE02 f f 737 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
b80d3ee6-153e-4ecf-a617-bd8d1ad7e2cd Ducati Monster695 f f 740 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
8af06134-ef93-4c32-8b21-d2c47c30e394 Ducati Monster796 f f 742 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
4020bb8f-c148-4f36-96d6-803d9f402ab3 Ducati Monster821 f f 743 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
64a48690-1526-4f4e-ad58-420854b31d9f Ducati Monster1100 f f 745 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
16352385-3cee-470f-8c02-3349111aa346 Ducati Monster1200R f f 747 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
8faf40d4-4bc9-4847-a753-19538ba065f1 Ducati Scrambler Icon f f 749 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
b2ab5e56-514e-49c3-889b-8e13cd297140 Ducati Scrambler Sixty2 f f 750 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
10a71f69-8eb0-427b-bca2-160a1aeb457b Ducati Scrambler1100 f f 751 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
2ecb9486-5a5f-46c1-b509-782d6cc13823 Ducati Scrambler1100 Sport f f 752 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
467723fd-4947-4136-b68d-87ab5162b9f2 Ducati Panigale899 f f 753 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
9f6be4ee-fff5-47ca-b3bd-31b9640fb0da Ducati Panigale959 f f 754 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
1cfa02bc-3da1-4526-ac9a-6bea10bc8c85 Ducati 848 f f 758 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
02a3ce01-207d-43c7-8c6f-c43e717ffa02 Ducati 1098 f f 759 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
df8d1f3d-959c-4cb1-a2dc-421744b254b6 Ducati 1198 f f 760 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
06e13ed2-8bdb-4f43-8d35-d8f39108ea45 Ducati 1299 f f 761 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
6b2dfbb8-250e-411f-b567-e25a15849d50 Ducati Streetfighter848 f f 762 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
5af32e2f-cca9-4a6e-8222-00c45d316c25 Ducati Streetfighter1098 f f 763 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N

82bd0ef8-f8b0-47d8-a857-ab703ca578bb Ducati Diavel f f 771 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
256af5b3-679e-42f2-bdcc-4ebab65fde45 Ducati XDiavel f f 774 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
0553bc4d-65da-4294-bcbe-5e8b547d00fe Ducati Hypermotard796 f f 775 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
2f68fce8-b577-45d0-85e3-33c995bae604 Ducati SuperSport f f 780 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
3ecd2105-d346-4212-bef7-3d56b3473258 Ducati MonsterSP f f 782 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
044adfdf-8335-48b9-9b2c-eba02f917335 Harley-Davidson Iron1200 f f 786 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
2e0dd30e-9ec2-4c23-8ff6-4d5fb8d5271c Harley-Davidson Forty-Eight Special f f 788 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
f6a01eec-36f5-4d2b-baba-00bc6fb5b7c7 Harley-Davidson Nightster Special f f 790 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
1aff64f0-264e-4250-bb5c-d0c3ce3770db Harley-Davidson Street Bob f f 793 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
e88250b5-a1fa-42fd-82f6-123281852207 Harley-Davidson Low Rider f f 795 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
4b35b27a-0931-4730-9e10-bfd69b4af7c5 Harley-Davidson Fat Bob f f 798 2026-09-20 00:34:29.188869+00 2026-10-06 17:07:33.654736+00 \N \N
c180efed-2a03-40fb-b545-e4c1710d058d Harley-Davidson Fat Boy f f 800 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
22a5f73f-6d0a-4c26-b8b3-e1b8ab4af047 Harley-Davidson Heritage Classic f f 802 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
01e29c00-a654-46b2-8355-e915629e72bf Harley-Davidson Breakout f f 804 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
dd330512-453d-4108-8789-7456627f6fb3 Harley-Davidson Breakout114 f f 805 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
9ee36fe7-9870-4f2c-be6c-739b290cd106 Harley-Davidson Deluxe f f 807 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
9c43c801-1e41-4b6e-87b0-c0370cb24a3b Harley-Davidson Slim f f

808 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
b14a0d67-78f6-4a92-808c-b54812769dd1 Harley-Davidson Road King Special f f 810 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
7f0ca68d-dd48-48b4-abef-7a87f8efe081 Harley-Davidson Road Glide Special f f 812 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
6ce54708-7d86-4452-8b90-4fb627a50a7b Harley-Davidson Road Glide Limited f f 813 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
9848a961-759e-4156-b2cf-a260a286c419 Harley-Davidson Street Glide Special f f 815 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
6eabe7ac-e353-40fe-8c2b-afae2007ee0c Harley-Davidson Electra Glide f f 816 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
b30876cf-53e7-4bb4-92ad-90bd060fde4d Harley-Davidson Pan America1250 Special f f 819 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
7a7bf9d5-0dd3-4145-81ba-248f961cc4dc Harley-Davidson LiveWire f f 821 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
409b4970-8a5e-4c89-8409-8b6d956f7997 Triumph Triumph Bonneville f f 822 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
af008d86-0f22-4c35-8357-b4b8d913c650 Triumph Bonneville Bobber f f 825 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
76686b75-9515-4800-85f9-ee123a4fa505 Triumph Bobber1200 f f 826 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
5f03b8a9-ea80-45b5-8ec6-7c36de621e15 Triumph Speedmaster f f 827 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
52b5a2ba-477a-4be2-80b5-6cf345a75c53 Triumph Scrambler900 f f 828 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
b3fcba39-50af-4583-94f8-b28031a0c7cc Triumph Scrambler1200 f f 829 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
bb34f264-94d8-451c-8168-83c693f80bc0 Triumph Street Scrambler f f 830 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
1c97e3fa-e9d9-432d-8c32-e033363e3248 Triumph Street Triple R f f 836 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N

\N
a1ae3bf3-2629-4633-9e23-2e6d291be704 Triumph Street Triple RS f f
837 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N
\N
847ae2f1-803a-4456-9aad-2295c078bc35 Triumph Speed Triple1200 f f
839 2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N
\N
8c0bb3da-dc4f-445d-b9a1-71e3a1c5ba88 Triumph Daytona765 f f 841
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
ff71f44e-a8e5-48d8-8ed0-395a0f36fd71 Triumph Daytona955i f f 842
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
048dcb1c-19da-4545-89d9-e8835e10e5fe Triumph Tiger850 f f 844
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
789b8151-aa9b-4e48-adea-8cbe8e9bb7ff Aprilia Caponord1200 f f 874
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
3c38a44e-b47d-4bc1-a8d0-e6108143d9c7 Aprilia Scarabeo100 f f 876
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
6daeef48-db1e-4956-9bdb-7cd44f15c359 Aprilia Scarabeo125 f f 877
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
0f633347-de46-49ae-a275-690696f5382d Aprilia Scarabeo200 f f 878
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
7183b881-aa9d-43cc-873c-e0d6f0430b71 Aprilia Scarabeo250 f f 879
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
cf50b0d2-75e5-4c94-99ee-17569f4d6683 Aprilia SR50 f f 880
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
65eb3527-ed53-418b-a07c-e198b1528b57 Aprilia SR125 f f 881
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
7ef95c8c-0f9d-4765-adaf-82bdba4c06d5 Aprilia SR150 f f 882
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
49ee4e38-2006-4161-831b-646d49e924c4 Aprilia SR160 f f 883
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
c9574597-79f7-4160-9130-6f772a74db7a Aprilia SR GT200 f f 884
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
7452b5c3-92dd-43fb-bad1-73c28e707d21 Aprilia Sportcity125 f f 885
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
422dbed2-cb28-4ebd-b778-aafdf7785aff Aprilia Sportcity200 f f 886
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
806b8daf-98db-40f0-8676-96ed022c3ab7 Aprilia Sportcity250 f f 887
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
74de0e5a-e594-44a2-af74-bd4b3656de39 Aprilia Tuareg660 f f 888
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
8e4d29a8-81bf-4283-a9fb-25e18d8adaba Aprilia Pegaso650 f f 889
2026-09-20 00:34:29.308439+00 2026-10-06 17:07:33.654736+00 \N \N
\.

--
-- Data for Name: price_history; Type: TABLE DATA; Schema: public; Owner: postgres
--

COPY "public"."price_history" ("id", "table_name", "item_id", "item_name", "old_price", "new_price", "reason", "changed_by", "created_at", "configuration_id", "configuration_label", "changed_by_email") FROM stdin;
9dc50737-2638-49c3-a689-16e68af769c9	services	4edb3763-89b5-4f21-afec-fc96ec98e4f7	Brake Service	755.00	760.00	increase shipping	\N	2026-08-19 11:08:33.631305+00	\N	Base service	\N
98ccf049-d46e-49c5-84cb-112e6eaa701a	services	4edb3763-89b5-4f21-afec-fc96ec98e4f7	Brake Service	760.00	770.00	inflation	\N	2026-08-19 11:08:56.680502+00	\N	Base service	\N
b4e1c49c-a51f-4089-aa65-1e95f32f2907	services	97a9714b-20e6-4402-a3ee-4fcdefc1e9db	Change Oil	450.00	500.00	\N	\N	2026-08-21 10:25:58.101307+00	\N	Base service	\N
3dec6fb3-28fb-495e-b184-affecccbf5cc	services	7120d926-7317-4060-9277-580e56b724f4	Change Oil	450.00	500.00	\N	\N	2026-08-21 10:26:04.423134+00	\N	Base service	\N
1d5b0127-588a-4bef-b5b8-c991b6544b96	services	7120d926-7317-4060-9277-580e56b724f4	Change Oil	500.00	400.00	\N	\N	2026-08-21 10:42:21.37524+00	\N	Base service	\N
6755d634-72c0-4b3e-9cbc-bd77d0b2a36e	services	72a4c5f5-e5f1-4e71-b872-30a3615ac362	Suspension Tuning / Rebuild	2500.00	2600.00	tools	\N	2026-08-21 11:29:52.044394+00	\N	Base service	\N
15b2d9f7-efab-4e67-b0d6-0526784508ea	services	9abcd66b-5ea3-4be2-8531-cd06ea4694ef	overhaul	0.00	500.00	\N	\N	2026-08-23 09:56:19.738354+00	\N	Base service	\N
7b63a540-b3da-4999-a8bd-9e1ded37965e	products	806e348d-7c95-4374-8555-c91355a56a86	honda s1000r	0.00	2000.00	\N	\N	2026-08-24 02:50:47.21498+00	\N	Default item	\N
80e362a7-fb13-4f7e-8610-314924f59823	products	a34c0c55-c044-4d23-8c37-b407d87d8139	stock shock	0.00	700.00	\N	\N	2026-08-26 13:03:08.762114+00	\N	Default item	\N
75dd7110-4ef0-4f6d-8983-c01c770f66c2	services	e5742f78-e0f3-4c14-b4e5-9ad22b3ff85b	Brake Service	750.00	300.00	\N	\N	2026-08-31 10:21:57.785234+00	\N	Base service	\N
67a3561b-1e51-439c-81d6-0de365c7f07f	services	97a9714b-20e6-4402-a3ee-4fcdefc1e9db	Change Oil	500.00	150.00	\N	\N	2026-08-31 10:22:08.933301+00	\N	Base service	\N
27a57bd4-ad49-41ea-ae38-050f667499b6	services

ab3c3ba9-415a-4f0e-9a42-bffb7d1903c1 Electrical Diagnostics 500.00 350.00 \N \N 2026-08-31 10:22:35.370621+00 \N Base service \N fa646fc8-65df-47aa-b272-ae93d9035e98 services
f8feeecc-86e2-4312-8798-ea5c144f1ad4 Engine Overhaul 6500.00 5000.00 \N \N 2026-08-31 10:23:02.111168+00 \N Base service \N 8db2516b-8f74-4daa-b368-c9bcd9e3b056 services
f8feeecc-86e2-4312-8798-ea5c144f1ad4 Engine Overhaul 5000.00 2000.00 \N \N 2026-08-31 10:23:25.212711+00 \N Base service \N 2e695780-47f5-4d0a-8b80-e0f7b2153e85 services
ed2e91b9-1d71-4e29-814d-7b7507751569 Preventive Maintenance Service (PMS) 850.00 350.00 \N \N 2026-08-31 10:23:50.57782+00 \N Base service \N
7dbedc69-729d-47d1-adb3-ed49c601db75 services
08eef9ae-0750-4ed4-ade6-35aed340eea5 Tire Change & Balancing 600.00 250.00 \N \N 2026-08-31 10:24:22.034714+00 \N Base service \N 2e8f6fb5-ed48-40f2-a644-c96b54fac01d services
72a4c5f5-e5f1-4e71-b872-30a3615ac362 Suspension Tuning / Rebuild 2600.00 2500.00 \N \N 2026-09-02 21:35:47.785679+00 \N Base service \N
8f9ecfca-4309-423a-a0fb-817bd5d3a5b4 services
e5742f78-e0f3-4c14-b4e5-9ad22b3ff85b Brake Service 300.00 350.00 inflation \N 2026-09-05 09:25:29.323153+00 \N Base service \N fbc110eb-7ec5-49b8-a1ff-ed5c8cb461ed services
e5cf562e-7122-440b-9483-033b78aca286 Carburetor / FI Cleaning 900.00 300.00 new price \N 2026-09-13 07:00:25.430932+00 \N Default Price \N
72b6e542-cdd0-471e-8352-ff00bb4f2b53 services
7f2b49e7-8c12-44d3-9d1f-3cb79c8898ec Motorcycle General Inspection 500.00 0.00 inspection 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 2026-09-28 08:08:59.311974+00 \N Default Price eapolona0803@gmail.com
fcb995ec-bb61-4f57-97d5-2c3c72d7e27b products
656db2e0-7cb6-4311-8271-3e15430d31b3 Ohlins Rear Shock Absorber 24500.00 3500.00 real price 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 2026-09-29 10:50:40.150635+00 656db2e0-7cb6-4311-8271-3e15430d31b3 Part eapolona0803@gmail.com
c9e38be2-2f3d-4ab8-976b-efe352f71406 products
d63ff680-20d7-4a54-92e0-1790fb39446c Performance Exhaust Pipe 13500.00 900.00 price 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 2026-09-29 10:52:29.926217+00 d63ff680-20d7-4a54-92e0-1790fb39446c Part eapolona0803@gmail.com
94cb692d-ca14-431c-bd00-537222d5550f products
c973b83b-ee14-4254-a320-420f75ffb39a Braided Brake Hose 2100.00 200.00 price 7a9e497f-8c3f-44a7-ab4e-c9247c50eb74 2026-09-29 10:54:24.903124+00 c973b83b-ee14-4254-a320-420f75ffb39a Part

eapolona0803@gmail.com
0ce9a20a-6e2a-4407-b0fe-98d10d36677a	services
f8feeecc-86e2-4312-8798-ea5c144f1ad4	Engine Overhaul	2000.00
1500.00	small bike	7a9e497f-8c3f-44a7-ab4e-c9247c50eb74	2026-09-29
10:57:28.926507+00	\N	Default Price	eapolona0803@gmail.com
8b44ed97-7f25-46b8-b430-7dd54119c525	services
a2e6ef24-2f88-4f18-92b2-180be413a5a7	Crankshaft Bearing Replacement
1200.00	1000.00	error input	7a9e497f-8c3f-44a7-ab4e-c9247c50eb74
2026-10-14 08:27:40.573167+00	a2e6ef24-2f88-4f18-92b2-180be413a5a7
Large Bike (126cc and above)	eapolona0803@gmail.com
\.

--
-- Data for Name: public_endpoint_rate_limits; Type: TABLE DATA; Schema: public; Owner: postgres
--

COPY "public"."public_endpoint_rate_limits" ("scope", "subject", "window_started_at", "request_count", "updated_at") FROM stdin;
diagnostic
0000000000000000000000000000000000000000000000000000000000000000
	2026-08-25 12:47:17.38339+00	1	2026-08-25 12:47:17.38339+00
reference_recovery_email_verify
508cc0fe26185d71343b0aaa99a7a8c8a45392f278d5f16b37faef9bf114cc58
2026-10-13 12:24:59.231864+00	2	2026-10-13 12:37:58.109954+00
reference_recovery_email_verify
839beb42bc4a18d4fec925ccdf591dad92b67baebfb65214879a6ebe26de133a
2026-10-13 12:24:59.468699+00	2	2026-10-13 12:37:58.395412+00
reference_recovery_email_request
09df73540303d4fb80b5eac2bcc2ee5b58b117a0ebb351314fe715e983913c72
2026-10-13 11:15:26.368266+00	2	2026-10-13 11:19:14.133924+00
lookup
8de0986747db3c9cfc245ec4403427381c1cbcf54fcb8e2a875167b5b55e5888
2026-10-14 09:29:58.666456+00	1	2026-10-14 09:29:58.666456+00
lookup
51f70868cca94c91b1fb1fd8b5d86c2d42cad63b8002761e6c5f435ea508fa1b
2026-10-13 11:16:53.807491+00	5	2026-10-13 11:20:04.545255+00
verification_request
8de0986747db3c9cfc245ec4403427381c1cbcf54fcb8e2a875167b5b55e5888
2026-10-14 09:30:12.7455+00	1	2026-10-14 09:30:12.7455+00
availability
508cc0fe26185d71343b0aaa99a7a8c8a45392f278d5f16b37faef9bf114cc58
2026-10-21 11:10:32.634122+00	2	2026-10-21 11:10:42.57675+00
reference_recovery_email_request

3805f060f1c31317a65e52c7bd0d6f51d645432237a0e2f63e8cb7cd9fa1813a 2026-10-13 11:38:05.36149+00 2 2026-10-13 11:38:06.333957+00 reference_recovery_email_verify
3805f060f1c31317a65e52c7bd0d6f51d645432237a0e2f63e8cb7cd9fa1813a 2026-10-13 11:38:09.477508+00 1 2026-10-13 11:38:09.477508+00 rescheduling
20752a1a62404edf9781070191ba84b3286df98431f078ffacddd76404d16bb8 2026-09-10 13:32:00.566346+00 5 2026-09-10 13:40:13.169991+00 verification_request
508cc0fe26185d71343b0aaa99a7a8c8a45392f278d5f16b37faef9bf114cc58 2026-10-14 09:30:12.509412+00 2 2026-10-14 09:32:18.205203+00 rescheduling
508cc0fe26185d71343b0aaa99a7a8c8a45392f278d5f16b37faef9bf114cc58 2026-09-10 13:32:00.283219+00 6 2026-09-10 13:45:57.441261+00 verification_request
e7369913523fd14ee1faf6d9408882d89296efd20d44ab05e55f3788261c77cb 2026-10-14 09:32:18.407706+00 1 2026-10-14 09:32:18.407706+00 verification_verify
508cc0fe26185d71343b0aaa99a7a8c8a45392f278d5f16b37faef9bf114cc58 2026-10-14 09:33:06.489895+00 1 2026-10-14 09:33:06.489895+00 verification_verify
e7369913523fd14ee1faf6d9408882d89296efd20d44ab05e55f3788261c77cb 2026-10-14 09:33:06.706388+00 1 2026-10-14 09:33:06.706388+00 cancellation
508cc0fe26185d71343b0aaa99a7a8c8a45392f278d5f16b37faef9bf114cc58 2026-10-14 09:33:07.204968+00 1 2026-10-14 09:33:07.204968+00 cancellation
e7369913523fd14ee1faf6d9408882d89296efd20d44ab05e55f3788261c77cb 2026-10-14 09:33:07.315846+00 1 2026-10-14 09:33:07.315846+00 lookup
508cc0fe26185d71343b0aaa99a7a8c8a45392f278d5f16b37faef9bf114cc58 2026-10-14 09:29:58.36568+00 3 2026-10-14 09:33:07.939692+00 lookup
e7369913523fd14ee1faf6d9408882d89296efd20d44ab05e55f3788261c77cb 2026-10-14 09:32:10.471181+00 2 2026-10-14 09:33:08.06785+00 reference_recovery_email_request
508cc0fe26185d71343b0aaa99a7a8c8a45392f278d5f16b37faef9bf114cc58 2026-10-13 12:36:17.655139+00 1 2026-10-13 12:36:17.655139+00 reference_recovery_email_request
839beb42bc4a18d4fec925ccdf591dad92b67baebfb65214879a6ebe26de133a 2026-10-13 12:36:17.917989+00 1 2026-10-13 12:36:17.917989+00 rescheduling
b476f48d11d51b814cbd0d7e4a2c848a31649df0711aa636bb30e93507991218 2026-09-10 01:43:38.007889+00 2 2026-09-10 01:44:12.651424+00 reference_recovery

508cc0fe26185d71343b0aaa99a7a8c8a45392f278d5f16b37faef9bf114cc58
2026-09-29 06:14:41.903443+00   1   2026-09-29 06:14:41.903443+00
reference_recovery
dfd8271f8b027b1a20ee74bba6c31f97025ef93f529593b286a1ca4c24937023
2026-09-29 06:14:42.257772+00   1   2026-09-29 06:14:42.257772+00
booking
bd1566ecba500671e5fc1225956cf0c24da1cb1b4ef5c74715b8f609bef5245a
2026-10-15 07:08:06.715151+00   1   2026-10-15 07:08:06.715151+00
booking
508cc0fe26185d71343b0aaa99a7a8c8a45392f278d5f16b37faef9bf114cc58
2026-10-15 10:08:53.790155+00   1   2026-10-15 10:08:53.790155+00
booking
6273a0bfb36bbc005dd8a9f171902d9dc283594004dbb5f35d800a7432846da5
2026-10-15 10:08:54.064211+00   1   2026-10-15 10:08:54.064211+00
\.

--
-- Data for Name: schedule_blocks; Type: TABLE DATA; Schema: public;
Owner: postgres
--

COPY "public"."schedule_blocks" ("id", "block_date", "start_time", "reason",
"created_at", "is_active", "archived_at") FROM stdin;
5f0ffc6f-6e8b-4a0e-b821-163bcaa109d9   2026-09-07   08:00:00
RANGE:09:30|event   2026-08-21 09:11:26.657414+00   t   \N
36e5d275-0d4e-43cf-af25-1abbdc52588a   2026-08-25   \N   The shop has
event for it's anniversary   2026-08-20 12:12:05.990924+00   t   \N
\.

--
-- Data for Name: shop_settings; Type: TABLE DATA; Schema: public; Owner:
postgres
--

COPY "public"."shop_settings" ("id", "shop_name", "address", "contact_number",
"contact_email", "booking_terms", "logo_url", "minimum_booking_lead_hours",
"max_advance_booking_days", "default_appointment_duration_minutes",
"allow_same_day_appointments", "cancellation_notice_hours",
"rescheduling_notice_hours", "notify_confirmation", "notify_cancellation",
"notify_reminder", "reminder_hours_before", "notify_admin", "updated_at",
"opening_time", "closing_time") FROM stdin;
t   Fake Rider Motorparts   Purok Bangkal Sta. Cruz, Baclayon, Bohol,
Philippines   0916 126 3317   joemartato4@gmail.com   Terms and Conditions

for Appointment Booking\nLast updated: September 29, 2026\n\nThese Terms and Conditions ("Terms") govern your access to and use of the online appointment booking system ("Booking System" or "Services") operated by Fake Rider Motorparts ("we," "us," "our," or "the Shop"). By submitting an appointment request through the Booking System, you ("you" or "the Customer") acknowledge that you have read, understood, and agree to be bound by these Terms, together with any policies referenced within them. If you do not agree to these Terms, please do not use the Booking System.\n\n1. Using the Booking System\n\na. Eligibility. You must be at least 18 years old, or have the permission of a parent or legal guardian, to submit an appointment request through the Booking System.\n\nb. Accurate Information. You agree to provide accurate, current, and complete information when booking an appointment, including your contact details and motorcycle information. You are responsible for keeping this information up to date, as inaccurate information may delay or prevent us from confirming, contacting you about, or completing your appointment.\n\nc. Acceptable Use. You agree not to misuse the Booking System, including by submitting duplicate, fraudulent, or bad-faith appointment requests, or by attempting to interfere with its normal operation.\n\n2. Appointment Requests and Confirmation\n\nAll appointment requests submitted through the Booking System are requests only and do not guarantee a reserved time slot. An appointment is considered confirmed only once it has been reviewed and approved by the Shop. We reserve the right to decline, limit, or adjust an appointment request based on availability, service capacity, or other reasonable business considerations.\n\n3. Appointment Schedule and Late Arrivals\n\nCustomers are encouraged to arrive at least 15 minutes before their scheduled appointment time to allow for check-in and preparation. If you arrive more than 15 minutes after your scheduled time, the Shop may, at its discretion and depending on availability, reschedule your appointment to a later time or mark it as a no-show.\n\n4. Rescheduling and Cancellation\n\na. Cancellations. Appointments may be cancelled at least 48 hours before the scheduled time. Cancellations made after this period may not be accommodated and may be recorded as a late cancellation under the Shop's scheduling policies.\n\nb. Rescheduling. An appointment may be rescheduled through the Booking System up to three (3) times, subject to availability. Reschedule requests are subject to review and confirmation by the Shop before they take effect; until confirmed, your original appointment date and time remain in place. Once the reschedule limit is reached, the appointment can no longer be rescheduled and must be attended, cancelled, or allowed to lapse under this section.\n\nc. No-Shows. An appointment that is neither attended nor cancelled or rescheduled in accordance with this section may be recorded as a no-show, which may affect your ability to book future appointments.\n\n5. Service Pricing and Additional Charges\n\nPrices shown in the Booking System are starting or estimated rates for the listed services. The final price may vary depending on your motorcycle's condition, the parts required, and the actual work performed. Any additional parts, labor, or services beyond the original request will be communicated to you and must be confirmed by you before the additional work is

carried out, except where immediate action is reasonably necessary to prevent further damage to your motorcycle.\n\n6. Customer Responsibilities and Personal Belongings\n\nYou are responsible for removing all personal belongings from your motorcycle before your appointment. The Shop is not responsible for the loss of, or damage to, personal items left on or in the motorcycle. You are also responsible for ensuring that any information you provide about your motorcycle, such as its make, model, and condition, is accurate, as this information may affect the service performed.\n\n7. Use of Customer Information\n\nInformation you submit through the Booking System, such as your name, contact details, and appointment history, is used only to manage your appointments, communicate with you about your service (including confirmations, reminders, and updates), and provide the Shop's services to you. The Shop does not sell your personal information to third parties. For more information about how your data is handled, please contact the Shop directly.\n\n8. Limitation of Liability\n\nTo the extent permitted by applicable law, the Shop's liability for any claim arising from or relating to an appointment or service booked through the Booking System is limited to the amount you paid for that specific service. The Shop is not liable for indirect, incidental, or consequential losses, including loss of use of your motorcycle, arising from delays, rescheduling, or service outcomes, except where such liability cannot be limited or excluded under applicable law. Nothing in these Terms limits the Shop's liability for gross negligence, willful misconduct, or any liability that cannot lawfully be excluded.\n\n9. Modifications to These Terms\n\nWe may update these Terms from time to time to reflect changes in our services, policies, or applicable law. Updated Terms will be posted through the Booking System or otherwise made available to you, and will apply to appointments booked after the update takes effect. Your continued use of the Booking System after an update takes effect means you agree to the updated Terms.\n\n10. Governing Law\n\nThese Terms are governed by the laws of the Republic of the Philippines. Any dispute arising from these Terms or the use of the Booking System will be resolved in accordance with applicable Philippine law.\n\n11. Acceptance of Terms\n\nBy submitting an appointment request through the Booking System, you confirm that you have read, understood, and agree to these Terms and Conditions in full.	\N	48	30	60	f	48	48	t	t	t	24	t	2026-10-06 17:07:32.071307+00	08:00:00	17:00:00
\.

--
-- Data for Name: time_slots; Type: TABLE DATA; Schema: public; Owner: postgres
--

COPY "public"."time_slots" ("id", "start_time", "end_time", "capacity", "is_active", "created_at") FROM stdin;

```
0dafeaa7-5873-49f4-8492-0e0b34185b47    08:00:00    09:00:00    2    t
2026-08-19 05:11:46.27827+00
920dbe47-d680-4db4-9f4e-ebd752f28b53    09:00:00    10:00:00    2    t
2026-08-19 05:11:46.27827+00
773ba9bd-362d-41f8-8890-e7e8d4d6fb8b    10:00:00    11:00:00    2    t
2026-08-19 05:11:46.27827+00
f93e2e74-6988-461c-8684-2bc6e29c879f    11:00:00    12:00:00    2    t
2026-08-19 05:11:46.27827+00
c936e7b1-4eed-4c86-b170-03c20bf171ac    13:00:00    14:00:00    2    t
2026-08-19 05:11:46.27827+00
d1abb7a7-a62c-4cb7-85b1-0e00e4540ad1    14:00:00    15:00:00    2    t
2026-08-19 05:11:46.27827+00
c8777690-817b-4196-9f8a-340ae005f230    15:00:00    16:00:00    2    t
2026-08-19 05:11:46.27827+00
f94e0c9f-3971-46e7-a006-8c0995ec58d5    16:00:00    17:00:00    2    t
2026-08-19 05:11:46.27827+00
\.


--
-- Data for Name: user_roles; Type: TABLE DATA; Schema: public; Owner:
postgres
--

COPY "public"."user_roles" ("id", "user_id", "role", "created_at") FROM stdin;
f21fba89-7e68-47d7-b130-a375d796c053
7a9e497f-8c3f-44a7-ab4e-c9247c50eb74    admin    2026-09-25
11:28:03.334133+00
\.


--
-- PostgreSQL database dump complete
--

-- \unrestrict dDZn8GvReDx6ALyowb48imcxEX018gr8lhZt3xmclAMCQ6Np0JbIm
ZNa3io5WlD

RESET ALL;
```

```
SET statement_timeout = 0;
SET lock_timeout = 0;
SET idle_in_transaction_session_timeout = 0;
SET client_encoding = 'UTF8';
SET standard_conforming_strings = on;
SELECT pg_catalog.set_config('search_path', '', false);
SET check_function_bodies = false;
SET xmloption = content;
SET client_min_messages = warning;
SET row_security = off;

CREATE EXTENSION IF NOT EXISTS "pg_cron" WITH SCHEMA "pg_catalog";

COMMENT ON SCHEMA "public" IS 'standard public schema';

CREATE EXTENSION IF NOT EXISTS "btree_gist" WITH SCHEMA "public";

CREATE EXTENSION IF NOT EXISTS "pg_stat_statements" WITH SCHEMA
"extensions";

CREATE EXTENSION IF NOT EXISTS "pgcrypto" WITH SCHEMA "extensions";
```

```
CREATE EXTENSION IF NOT EXISTS "supabase_vault" WITH SCHEMA
"vault";




CREATE EXTENSION IF NOT EXISTS "uuid-ossp" WITH SCHEMA
"extensions";




CREATE TYPE "public"."app_role" AS ENUM (
    'admin'
);


ALTER TYPE "public"."app_role" OWNER TO "postgres";


CREATE TYPE "public"."app_role_legacy" AS ENUM (
    'admin',
    'staff'
);


ALTER TYPE "public"."app_role_legacy" OWNER TO "postgres";


CREATE OR REPLACE FUNCTION "public"."admin_activity_request_ip"()
RETURNS "text"
    LANGUAGE "plpgsql" STABLE
    AS $$
DECLARE
  v_headers jsonb;
  v_forwarded_for text;
```

```
BEGIN
  v_headers := COALESCE(
    NULLIF(current_setting('request.headers', true), '')::jsonb,
    '{}'::jsonb
  );
  v_forwarded_for := COALESCE(
    v_headers ->> 'x-vercel-forwarded-for',
    v_headers ->> 'cf-connecting-ip',
    v_headers ->> 'x-real-ip',
    v_headers ->> 'x-forwarded-for'
  );

  RETURN NULLIF(btrim(split_part(COALESCE(v_forwarded_for, ''), ',', 1)), '');
EXCEPTION
  WHEN others THEN
    -- Logging must not block an admin action if a platform omits request headers.
    RETURN NULL;
END;
$$;


ALTER FUNCTION "public"."admin_activity_request_ip"() OWNER TO
"postgres";


CREATE OR REPLACE FUNCTION "public"."assert_no_duplicate_active_custo
mer_service_appointment"("p_appointment_id" "uuid") RETURNS "void"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
DECLARE
  v_plate text;
  v_plate_label text;
  v_appointment_date date;
  v_service_name text;
  v_start_time time;
BEGIN
  SELECT public.normalize_plate_number(appointment.plate_number),
    btrim(appointment.plate_number),
    appointment.appointment_date
  INTO v_plate, v_plate_label, v_appointment_date
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
    AND appointment.rescheduled_to_appointment_id IS NULL
```

```
    AND appointment.status NOT IN ('completed', 'cancelled', 'rejected');

  IF NOT FOUND OR v_plate = '' THEN
    RETURN;
  END IF;

  SELECT current_service.service_name, other_appointment.start_time
  INTO v_service_name, v_start_time
  FROM public.appointment_services AS current_service
  JOIN public.appointments AS other_appointment
    ON public.normalize_plate_number(other_appointment.plate_number) =
v_plate
   AND other_appointment.appointment_date = v_appointment_date
   AND other_appointment.id <> p_appointment_id
   AND other_appointment.is_archived = false
   AND other_appointment.rescheduled_to_appointment_id IS NULL
   AND other_appointment.status NOT IN ('completed', 'cancelled', 'rejected')
  JOIN public.appointment_services AS other_service
    ON other_service.appointment_id = other_appointment.id
   AND (
     other_service.service_id = current_service.service_id
     OR (
       other_service.service_id IS NULL
       AND current_service.service_id IS NULL
       AND other_service.service_name = current_service.service_name
     )
   )
  WHERE current_service.appointment_id = p_appointment_id
  LIMIT 1;

  IF FOUND THEN
    RAISE EXCEPTION
      'The motorcycle with plate number % already has an appointment for "%" on
% at %.',
      v_plate_label,
      v_service_name,
      to_char(v_appointment_date, 'FMDay, FMMonth FMDD, YYYY'),
      to_char(v_start_time, 'FMHH12:MI AM')
      USING ERRCODE = '23505';
  END IF;
END;
$$;


ALTER FUNCTION "public"."assert_no_duplicate_active_customer_service_app
```

```
ointment"("p_appointment_id" "uuid") OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."booking_slot_capacity"("p_start_time" time without time zone)
RETURNS integer
    LANGUAGE "sql" STABLE
    SET "search_path" TO 'public'
    AS $$
  SELECT slot.capacity
  FROM public.time_slots AS slot
  WHERE slot.is_active = true
    AND (
      slot.start_time = p_start_time
      OR slot.start_time = make_time(extract(hour FROM p_start_time)::integer, 0,
0)
    )
  ORDER BY (slot.start_time = p_start_time) DESC
  LIMIT 1;
$$;


ALTER FUNCTION "public"."booking_slot_capacity"("p_start_time" time without
time zone) OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."cancel_public_appointment"("p_appointment_id" "uuid",
"p_cancellation_notice_hours" integer) RETURNS "void"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
DECLARE
  v_appointment public.appointments%ROWTYPE;
  v_month_start timestamptz;
  v_month_end timestamptz;
  v_month_label text;
  v_monthly_cancellation_count integer;
BEGIN
  IF p_appointment_id IS NULL
     OR p_cancellation_notice_hours IS NULL
     OR p_cancellation_notice_hours < 0
     OR p_cancellation_notice_hours > 168 THEN
    RAISE EXCEPTION 'Invalid cancellation request.';
  END IF;
```

```
  SELECT appointment.*
    INTO v_appointment
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
  FOR UPDATE;

  IF NOT FOUND THEN
    RAISE EXCEPTION 'This appointment is no longer available.';
  END IF;
  IF v_appointment.rescheduled_to_appointment_id IS NOT NULL THEN
    RAISE EXCEPTION 'This appointment has already been rescheduled. Please
use the new reference.';
  END IF;
  IF v_appointment.pending_reschedule_request_id IS NOT NULL THEN
    RAISE EXCEPTION 'A reschedule request is already awaiting the shop''s
review.';
  END IF;
  IF v_appointment.status NOT IN ('pending', 'confirmed', 'rescheduled') THEN
    RAISE EXCEPTION 'This appointment can no longer be cancelled online.
Please call the shop.';
  END IF;
  IF (v_appointment.appointment_date + v_appointment.start_time)
       - (now() AT TIME ZONE 'Asia/Manila')
       < make_interval(hours => p_cancellation_notice_hours) THEN
    RAISE EXCEPTION 'Cancellations need % hours notice. Please call the shop
instead.',
      p_cancellation_notice_hours;
  END IF;

  -- Serialize same-customer cancellations for the month so concurrent requests
  -- cannot produce duplicate third-cancellation alerts.
  v_month_start := date_trunc('month', now() AT TIME ZONE 'Asia/Manila') AT
TIME ZONE 'Asia/Manila';
  v_month_end := v_month_start + interval '1 month';
  v_month_label := to_char(now() AT TIME ZONE 'Asia/Manila', 'FMMonth
YYYY');
  PERFORM pg_advisory_xact_lock(
    hashtext(
      'customer-cancellation-threshold:'
      || COALESCE(v_appointment.phone, '')
      || ':'
      || to_char(v_month_start AT TIME ZONE 'Asia/Manila', 'YYYY-MM')
    )
```

```
  );

  UPDATE public.appointments
  SET status = 'cancelled',
      cancelled_at = now()
  WHERE id = v_appointment.id;

  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES (
    'cancelled_appointment',
    'Cancelled ' || v_appointment.reference_code,
    v_appointment.customer_name || ' cancelled their '
      || to_char(v_appointment.appointment_date, 'Mon FMDD, YYYY') || '
appointment.',
    v_appointment.id
  );

  SELECT count(*)
    INTO v_monthly_cancellation_count
  FROM public.appointments AS appointment
  WHERE appointment.phone = v_appointment.phone
    AND appointment.status = 'cancelled'
    AND appointment.cancelled_at >= v_month_start
    AND appointment.cancelled_at < v_month_end;

  -- Alert exactly once when the threshold is reached; later cancellations in
  -- the same month retain normal cancellation notifications without flooding
  -- the admin queue.
  IF v_monthly_cancellation_count = 3 THEN
    INSERT INTO public.notifications (type, title, message, appointment_id)
    VALUES (
      'customer_cancellation_threshold',
      'Customer cancellation threshold reached',
      COALESCE(NULLIF(btrim(v_appointment.customer_name), ''),
v_appointment.phone)
        || ' (' || v_appointment.phone || ') has cancelled 3 appointments in '
        || v_month_label
        || '. Review the customer''s cancellation history and decide whether to block
the customer or take no action.',
      v_appointment.id
    );
  END IF;
END;
$$;
```

```
ALTER FUNCTION "public"."cancel_public_appointment"("p_appointment_id"
"uuid", "p_cancellation_notice_hours" integer) OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."capture_customer_no_show_count_at_booking"() RETURNS "trigger"
    LANGUAGE "plpgsql" SECURITY DEFINER
    SET "search_path" TO 'public'
    AS $$
BEGIN
  SELECT count(*)::integer
  INTO NEW.no_show_count_at_booking
  FROM public.appointments AS appointment
  WHERE appointment.phone = NEW.phone
    -- A row can have only one status, so cancelled and completed appointments
    -- are never included in this recorded no-show history.
    AND appointment.status = 'no_show';

  RETURN NEW;
END;
$$;


ALTER FUNCTION "public"."capture_customer_no_show_count_at_booking"()
OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."change_crew_schedule"("p_schedule_id" "uuid", "p_remove" boolean,
"p_start_time" time without time zone DEFAULT NULL::time without time zone,
"p_end_time" time without time zone DEFAULT NULL::time without time zone)
RETURNS "jsonb"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public', 'auth'
    AS $$
DECLARE
  v_schedule public.crew_schedules%ROWTYPE;
  v_work record;
  v_replacement_id uuid;
  v_replacement_name text;
  v_moved jsonb := '[]'::jsonb;
  v_stuck text[] := ARRAY[]::text[];
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
```

```
    RAISE EXCEPTION 'Not authorized';
  END IF;

  SELECT * INTO v_schedule FROM public.crew_schedules WHERE id =
p_schedule_id;
  IF NOT FOUND THEN
    RAISE EXCEPTION 'This shift no longer exists. Refresh the list and try again.'
      USING ERRCODE = 'P0002';
  END IF;
  IF v_schedule.schedule_date IS NULL THEN
    RAISE EXCEPTION 'Only shifts on a specific date can be changed here.';
  END IF;
  IF NOT p_remove AND (p_start_time IS NULL OR p_end_time IS NULL OR
p_start_time >= p_end_time) THEN
    RAISE EXCEPTION 'The shift must end after it starts.';
  END IF;

  -- The lock every booking change on this date takes, so no booking can claim
  -- this mechanic while the shift changes.
  PERFORM pg_advisory_xact_lock(hashtext('booking-capacity:' ||
v_schedule.schedule_date::text));

  IF p_remove THEN
    DELETE FROM public.crew_schedules WHERE id = v_schedule.id;
  ELSE
    UPDATE public.crew_schedules
    SET start_time = p_start_time,
        end_time = p_end_time,
        is_working = true
    WHERE id = v_schedule.id;
  END IF;

  FOR v_work IN
    SELECT
      'appointment' AS kind,
      appointment.id AS appointment_id,
      NULL::uuid AS continuation_id,
      appointment.reference_code,
      appointment.customer_name,
      appointment.start_time,
      appointment.booking_duration_minutes AS duration_minutes
    FROM public.appointments AS appointment
    WHERE appointment.assigned_crew_id = v_schedule.crew_id
      AND appointment.appointment_date = v_schedule.schedule_date
      AND appointment.is_archived = false
```

```
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
      AND NOT public.is_crew_available_for_appointment(
        v_schedule.crew_id,
        v_schedule.schedule_date,
        appointment.start_time,
        appointment.booking_duration_minutes,
        appointment.id
      )
    UNION ALL
    SELECT
      'continuation',
      appointment.id,
      continuation.id,
      appointment.reference_code,
      appointment.customer_name,
      continuation.start_time,
      continuation.booking_duration_minutes
    FROM public.appointment_continuations AS continuation
    JOIN public.appointments AS appointment ON appointment.id =
continuation.appointment_id
    WHERE continuation.assigned_crew_id = v_schedule.crew_id
      AND continuation.appointment_date = v_schedule.schedule_date
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
      AND NOT public.is_crew_available_for_appointment(
        v_schedule.crew_id,
        v_schedule.schedule_date,
        continuation.start_time,
        continuation.booking_duration_minutes,
        NULL,
        continuation.id
      )
    ORDER BY start_time, reference_code
  LOOP
    SELECT crew.id, crew.name
      INTO v_replacement_id, v_replacement_name
    FROM public.crew_members AS crew
    WHERE crew.id <> v_schedule.crew_id
      AND crew.is_active = true
      AND crew.is_archived = false
      AND public.is_crew_available_for_appointment(
```

```
        crew.id,
        v_schedule.schedule_date,
        v_work.start_time,
        v_work.duration_minutes,
        v_work.appointment_id,
        v_work.continuation_id
      )
    ORDER BY public.crew_daily_workload(crew.id, v_schedule.schedule_date,
v_work.appointment_id),
      crew.name,
      crew.id
    LIMIT 1;

    IF v_replacement_id IS NULL THEN
      v_stuck := array_append(v_stuck, v_work.reference_code);
      CONTINUE;
    END IF;

    IF v_work.kind = 'appointment' THEN
      UPDATE public.appointments
      SET assigned_crew_id = v_replacement_id
      WHERE id = v_work.appointment_id;
    ELSE
      UPDATE public.appointment_continuations
      SET assigned_crew_id = v_replacement_id
      WHERE id = v_work.continuation_id;
    END IF;

    v_moved := v_moved || jsonb_build_array(jsonb_build_object(
      'reference_code', v_work.reference_code,
      'customer_name', v_work.customer_name,
      'start_time', v_work.start_time,
      'mechanic', v_replacement_name
    ));
  END LOOP;

  -- Undo everything, including the shift change, rather than leave a booking
  -- with a mechanic who is not working.
  IF cardinality(v_stuck) > 0 THEN
    RAISE EXCEPTION 'No other mechanic is free for %.',
array_to_string(v_stuck, ', ')
      USING HINT = 'crew_shift_conflict';
  END IF;

  RETURN jsonb_build_object('moved', v_moved);
```

```
END;
$$;


ALTER FUNCTION "public"."change_crew_schedule"("p_schedule_id" "uuid",
"p_remove" boolean, "p_start_time" time without time zone, "p_end_time" time
without time zone) OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."claim_admin_session"("p_handover_token" "uuid" DEFAULT
NULL::"uuid") RETURNS boolean
    LANGUAGE "plpgsql" SECURITY DEFINER
    SET "search_path" TO 'public', 'auth'
    AS $$
DECLARE
  v_admin_id uuid := auth.uid();
  v_session_id uuid := public.current_admin_auth_session_id();
  v_existing_lease public.admin_session_leases%ROWTYPE;
  v_has_valid_handover boolean := false;
BEGIN
  IF v_admin_id IS NULL
     OR v_session_id IS NULL
     OR NOT EXISTS (
       SELECT 1
       FROM public.user_roles AS role
       WHERE role.user_id = v_admin_id
         AND role.role::text = 'admin'
     ) THEN
    RAISE EXCEPTION 'Only administrators can start an admin session.';
  END IF;

  PERFORM pg_advisory_xact_lock(hashtext('admin-session-lease:' ||
v_admin_id::text));
  DELETE FROM public.admin_session_handover_tokens WHERE expires_at
<= now();

  SELECT lease.*
  INTO v_existing_lease
  FROM public.admin_session_leases AS lease
  WHERE lease.admin_id = v_admin_id
  FOR UPDATE;

  IF FOUND
     AND v_existing_lease.session_id <> v_session_id
```

```
     AND v_existing_lease.expires_at > now() THEN
    IF p_handover_token IS NOT NULL THEN
      SELECT true
      INTO v_has_valid_handover
      FROM public.admin_session_handover_tokens AS handover
      WHERE handover.token = p_handover_token
        AND handover.admin_id = v_admin_id
        AND handover.session_id = v_existing_lease.session_id
        AND handover.expires_at > now()
      FOR UPDATE;
    END IF;

    IF NOT COALESCE(v_has_valid_handover, false) THEN
      RAISE EXCEPTION
        'An administrator session is already active. Sign out from the active device
or wait for it to expire before signing in here.';
    END IF;

    DELETE FROM public.admin_session_handover_tokens
    WHERE token = p_handover_token;
  END IF;

  INSERT INTO public.admin_session_leases (
    admin_id,
    session_id,
    expires_at,
    last_seen_at
  )
  VALUES (
    v_admin_id,
    v_session_id,
    now() + interval '3 minutes',
    now()
  )
  ON CONFLICT (admin_id) DO UPDATE
  SET session_id = EXCLUDED.session_id,
      expires_at = EXCLUDED.expires_at,
      last_seen_at = EXCLUDED.last_seen_at;

  RETURN true;
END;
$$;


ALTER FUNCTION "public"."claim_admin_session"("p_handover_token" "uuid")
```

```
OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."consume_reference_recovery_challenge"("p_email" "text",
"p_code_hash" "text") RETURNS "text"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
DECLARE
  v_challenge public.appointment_reference_recovery_challenges%ROWTYPE;
BEGIN
  SELECT challenge.*
    INTO v_challenge
  FROM public.appointment_reference_recovery_challenges AS challenge
  WHERE challenge.email = lower(btrim(p_email))
    AND challenge.verified_at IS NULL
  ORDER BY challenge.created_at DESC
  LIMIT 1
  FOR UPDATE;

  IF NOT FOUND OR v_challenge.expires_at <= now() OR
v_challenge.attempt_count >= 5 THEN
    RETURN 'expired';
  END IF;

  IF v_challenge.code_hash <> p_code_hash THEN
    UPDATE public.appointment_reference_recovery_challenges
    SET attempt_count = attempt_count + 1,
        expires_at = CASE WHEN attempt_count + 1 >= 5 THEN now() ELSE
expires_at END
    WHERE id = v_challenge.id;
    RETURN 'invalid';
  END IF;

  UPDATE public.appointment_reference_recovery_challenges
  SET verified_at = now()
  WHERE id = v_challenge.id;

  RETURN 'verified';
END;
$$;


ALTER FUNCTION "public"."consume_reference_recovery_challenge"("p_email"
```

```
"text", "p_code_hash" "text") OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."create_admin_session_handover"() RETURNS "uuid"
    LANGUAGE "plpgsql" SECURITY DEFINER
    SET "search_path" TO 'public', 'auth'
    AS $$
DECLARE
  v_admin_id uuid := auth.uid();
  v_session_id uuid := public.current_admin_auth_session_id();
  v_token uuid := gen_random_uuid();
BEGIN
  IF NOT public.is_current_admin_session(v_admin_id) THEN
    RAISE EXCEPTION 'Your administrator session is no longer active. Please
sign in again.';
  END IF;

  DELETE FROM public.admin_session_handover_tokens
  WHERE expires_at <= now()
     OR (admin_id = v_admin_id AND session_id = v_session_id);

  INSERT INTO public.admin_session_handover_tokens (token, admin_id,
session_id, expires_at)
  VALUES (v_token, v_admin_id, v_session_id, now() + interval '2 minutes');

  RETURN v_token;
END;
$$;


ALTER FUNCTION "public"."create_admin_session_handover"() OWNER TO
"postgres";


CREATE OR REPLACE FUNCTION
"public"."create_booking_atomic"("p_reference_code" "text",
"p_booking_request_id" "uuid", "p_customer_name" "text", "p_phone" "text",
"p_moto_brand" "text", "p_moto_model" "text", "p_moto_variant" "text",
"p_moto_year" integer, "p_plate_number" "text", "p_appointment_date" "date",
"p_start_time" time without time zone, "p_notes" "text", "p_total_estimate"
numeric, "p_booking_duration_minutes" integer, "p_assigned_crew_id" "uuid",
"p_services" "jsonb", "p_notification_title" "text", "p_notification_message" "text",
"p_first_name" "text", "p_middle_name" "text", "p_last_name" "text") RETURNS
TABLE("appointment_id" "uuid", "reference_code" "text")
```

```
    LANGUAGE "sql"
    SET "search_path" TO 'public'
    AS $$
  SELECT created.appointment_id, created.reference_code
  FROM public.create_booking_atomic(
    p_reference_code,
    p_booking_request_id,
    p_customer_name,
    p_phone,
    p_moto_brand,
    p_moto_model,
    p_moto_variant,
    p_moto_year,
    p_plate_number,
    p_appointment_date,
    p_start_time,
    p_notes,
    p_total_estimate,
    p_booking_duration_minutes,
    p_assigned_crew_id,
    p_services,
    '[]'::jsonb,
    p_notification_title,
    p_notification_message,
    p_first_name,
    p_middle_name,
    p_last_name
  ) AS created;
$$;


ALTER FUNCTION "public"."create_booking_atomic"("p_reference_code" "text",
"p_booking_request_id" "uuid", "p_customer_name" "text", "p_phone" "text",
"p_moto_brand" "text", "p_moto_model" "text", "p_moto_variant" "text",
"p_moto_year" integer, "p_plate_number" "text", "p_appointment_date" "date",
"p_start_time" time without time zone, "p_notes" "text", "p_total_estimate"
numeric, "p_booking_duration_minutes" integer, "p_assigned_crew_id" "uuid",
"p_services" "jsonb", "p_notification_title" "text", "p_notification_message" "text",
"p_first_name" "text", "p_middle_name" "text", "p_last_name" "text") OWNER TO
"postgres";


CREATE OR REPLACE FUNCTION
"public"."create_booking_atomic"("p_reference_code" "text",
"p_booking_request_id" "uuid", "p_customer_name" "text", "p_phone" "text",
```

```
"p_moto_brand" "text", "p_moto_model" "text", "p_moto_variant" "text",
"p_moto_year" integer, "p_plate_number" "text", "p_appointment_date" "date",
"p_start_time" time without time zone, "p_notes" "text", "p_total_estimate"
numeric, "p_booking_duration_minutes" integer, "p_assigned_crew_id" "uuid",
"p_services" "jsonb", "p_continuation_segments" "jsonb", "p_notification_title"
"text", "p_notification_message" "text", "p_first_name" "text", "p_middle_name"
"text", "p_last_name" "text") RETURNS TABLE("appointment_id" "uuid",
"reference_code" "text")
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
DECLARE v_appointment_id uuid; v_reference_code text;
BEGIN
  IF p_booking_request_id IS NULL THEN RAISE EXCEPTION 'A booking
request ID is required.'; END IF;
  IF NOT public.is_valid_customer_name_part(p_first_name)
     OR NOT public.is_valid_customer_name_part(p_last_name) THEN
    RAISE EXCEPTION 'First and last names must contain letters, spaces,
hyphens, or apostrophes only.';
  END IF;
  IF COALESCE(btrim(p_middle_name), '') <> ''
     AND NOT public.is_valid_customer_name_part(p_middle_name) THEN
    RAISE EXCEPTION 'Middle name must contain letters, spaces, hyphens, or
apostrophes only.';
  END IF;
  IF jsonb_typeof(p_services) <> 'array' OR jsonb_array_length(p_services) = 0
THEN
    RAISE EXCEPTION 'A booking must include at least one service.';
  END IF;
  IF jsonb_typeof(COALESCE(p_continuation_segments, '[]'::jsonb)) <> 'array'
THEN
    RAISE EXCEPTION 'Continuation segments must be an array.';
  END IF;

  INSERT INTO public.appointments (
    reference_code, booking_request_id, customer_name, first_name,
middle_name,
    last_name, phone, moto_brand, moto_model, moto_variant, moto_year,
    plate_number, appointment_date, start_time, notes, total_estimate,
    booking_duration_minutes, terms_accepted, assigned_crew_id
  ) VALUES (
    p_reference_code, p_booking_request_id,
    concat_ws(' ', btrim(p_first_name), NULLIF(btrim(p_middle_name), ''),
btrim(p_last_name)),
    btrim(p_first_name), NULLIF(btrim(p_middle_name), ''), btrim(p_last_name),
```

```
p_phone,
    p_moto_brand, p_moto_model, p_moto_variant, p_moto_year,
p_plate_number,
    p_appointment_date, p_start_time, p_notes, p_total_estimate,
    p_booking_duration_minutes, true, p_assigned_crew_id
  ) ON CONFLICT DO NOTHING
  RETURNING appointments.id, appointments.reference_code INTO
v_appointment_id, v_reference_code;

  IF v_appointment_id IS NULL THEN
    SELECT appointments.id, appointments.reference_code INTO
v_appointment_id, v_reference_code
    FROM public.appointments WHERE appointments.booking_request_id =
p_booking_request_id;
    IF FOUND THEN RETURN QUERY SELECT v_appointment_id,
v_reference_code; RETURN; END IF;
    RAISE EXCEPTION 'The requested schedule is no longer available.' USING
ERRCODE = '23P01';
  END IF;

  INSERT INTO public.appointment_services (appointment_id, service_id,
service_name, price, duration_minutes)
  SELECT v_appointment_id, service.service_id, service.service_name,
service.price, service.duration_minutes
  FROM jsonb_to_recordset(p_services) AS service(
    service_id uuid, service_name text, price numeric, duration_minutes integer
  );

  INSERT INTO public.appointment_continuations (
    appointment_id, segment_number, appointment_date, start_time,
    booking_duration_minutes, assigned_crew_id
  )
  SELECT v_appointment_id, segment.segment_number,
segment.appointment_date, segment.start_time,
    segment.booking_duration_minutes, segment.assigned_crew_id
  FROM jsonb_to_recordset(COALESCE(p_continuation_segments, '[]'::jsonb))
AS segment(
    segment_number integer, appointment_date date, start_time time,
    booking_duration_minutes integer, assigned_crew_id uuid
  );

  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES ('new_appointment', p_notification_title, p_notification_message,
v_appointment_id);
  RETURN QUERY SELECT v_appointment_id, v_reference_code;
```

```
END;
$$;


ALTER FUNCTION "public"."create_booking_atomic"("p_reference_code" "text",
"p_booking_request_id" "uuid", "p_customer_name" "text", "p_phone" "text",
"p_moto_brand" "text", "p_moto_model" "text", "p_moto_variant" "text",
"p_moto_year" integer, "p_plate_number" "text", "p_appointment_date" "date",
"p_start_time" time without time zone, "p_notes" "text", "p_total_estimate"
numeric, "p_booking_duration_minutes" integer, "p_assigned_crew_id" "uuid",
"p_services" "jsonb", "p_continuation_segments" "jsonb", "p_notification_title"
"text", "p_notification_message" "text", "p_first_name" "text", "p_middle_name"
"text", "p_last_name" "text") OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."create_booking_atomic"("p_reference_code" "text",
"p_booking_request_id" "uuid", "p_customer_name" "text", "p_phone" "text",
"p_email" "text", "p_moto_brand" "text", "p_moto_model" "text", "p_moto_variant"
"text", "p_moto_year" integer, "p_plate_number" "text", "p_appointment_date"
"date", "p_start_time" time without time zone, "p_notes" "text", "p_total_estimate"
numeric, "p_booking_duration_minutes" integer, "p_assigned_crew_id" "uuid",
"p_services" "jsonb", "p_continuation_segments" "jsonb", "p_notification_title"
"text", "p_notification_message" "text", "p_first_name" "text", "p_middle_name"
"text", "p_last_name" "text") RETURNS TABLE("appointment_id" "uuid",
"reference_code" "text")
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $_$
DECLARE
  v_appointment_id uuid;
  v_reference_code text;
BEGIN
  IF p_booking_request_id IS NULL THEN
    RAISE EXCEPTION 'A booking request ID is required.';
  END IF;
  IF COALESCE(btrim(p_first_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$'
     OR COALESCE(btrim(p_last_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$' THEN
    RAISE EXCEPTION 'First and last names must contain letters only.';
  END IF;
  IF COALESCE(btrim(p_middle_name), '') <> ''
     AND btrim(p_middle_name) !~ '^[[:alpha:]]+( [[:alpha:]]+)*$' THEN
    RAISE EXCEPTION 'Middle name must contain letters only.';
  END IF;
  IF COALESCE(btrim(p_email), '') !~*
```

```
'^[^[:space:]@]+@[^[:space:]@]+\.[^[:space:]@]+$' THEN
    RAISE EXCEPTION 'Enter a valid email address.';
  END IF;
  IF jsonb_typeof(p_services) <> 'array' OR jsonb_array_length(p_services) = 0
THEN
    RAISE EXCEPTION 'A booking must include at least one service.';
  END IF;
  IF jsonb_typeof(COALESCE(p_continuation_segments, '[]'::jsonb)) <> 'array'
THEN
    RAISE EXCEPTION 'Continuation segments must be an array.';
  END IF;

  INSERT INTO public.appointments (
    reference_code, booking_request_id, customer_name, first_name,
middle_name,
    last_name, phone, email, moto_brand, moto_model, moto_variant, moto_year,
    plate_number, appointment_date, start_time, status, notes, total_estimate,
    booking_duration_minutes, terms_accepted, assigned_crew_id
  ) VALUES (
    p_reference_code,
    p_booking_request_id,
    concat_ws(' ', btrim(p_first_name), NULLIF(btrim(p_middle_name), ''),
btrim(p_last_name)),
    btrim(p_first_name),
    NULLIF(btrim(p_middle_name), ''),
    btrim(p_last_name),
    p_phone,
    lower(btrim(p_email)),
    p_moto_brand,
    p_moto_model,
    p_moto_variant,
    p_moto_year,
    p_plate_number,
    p_appointment_date,
    p_start_time,
    'pending',
    p_notes,
    p_total_estimate,
    p_booking_duration_minutes,
    true,
    p_assigned_crew_id
  ) ON CONFLICT DO NOTHING
  RETURNING appointments.id, appointments.reference_code INTO
v_appointment_id, v_reference_code;
```

```
  IF v_appointment_id IS NULL THEN
    SELECT appointments.id, appointments.reference_code
      INTO v_appointment_id, v_reference_code
    FROM public.appointments
    WHERE appointments.booking_request_id = p_booking_request_id;
    IF FOUND THEN
      RETURN QUERY SELECT v_appointment_id, v_reference_code;
      RETURN;
    END IF;
    RAISE EXCEPTION 'The requested schedule is no longer available.' USING
ERRCODE = '23P01';
  END IF;

  INSERT INTO public.appointment_services (
    appointment_id, service_id, service_name, price, duration_minutes
  )
  SELECT v_appointment_id, service.service_id, service.service_name,
service.price,
    service.duration_minutes
  FROM jsonb_to_recordset(p_services) AS service(
    service_id uuid, service_name text, price numeric, duration_minutes integer
  );

  INSERT INTO public.appointment_continuations (
    appointment_id, segment_number, appointment_date, start_time,
    booking_duration_minutes, assigned_crew_id
  )
  SELECT v_appointment_id, segment.segment_number,
segment.appointment_date,
    segment.start_time, segment.booking_duration_minutes,
segment.assigned_crew_id
  FROM jsonb_to_recordset(COALESCE(p_continuation_segments, '[]'::jsonb))
AS segment(
    segment_number integer, appointment_date date, start_time time,
    booking_duration_minutes integer, assigned_crew_id uuid
  );

  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES ('new_appointment', p_notification_title, p_notification_message,
v_appointment_id);

  RETURN QUERY SELECT v_appointment_id, v_reference_code;
END;
$_$;
```

```
ALTER FUNCTION "public"."create_booking_atomic"("p_reference_code" "text",
"p_booking_request_id" "uuid", "p_customer_name" "text", "p_phone" "text",
"p_email" "text", "p_moto_brand" "text", "p_moto_model" "text", "p_moto_variant"
"text", "p_moto_year" integer, "p_plate_number" "text", "p_appointment_date"
"date", "p_start_time" time without time zone, "p_notes" "text", "p_total_estimate"
numeric, "p_booking_duration_minutes" integer, "p_assigned_crew_id" "uuid",
"p_services" "jsonb", "p_continuation_segments" "jsonb", "p_notification_title"
"text", "p_notification_message" "text", "p_first_name" "text", "p_middle_name"
"text", "p_last_name" "text") OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."create_booking_atomic"("p_reference_code" "text",
"p_booking_request_id" "uuid", "p_customer_name" "text", "p_phone" "text",
"p_email" "text", "p_moto_brand" "text", "p_moto_model" "text", "p_moto_variant"
"text", "p_moto_year" integer, "p_plate_number" "text", "p_appointment_date"
"date", "p_start_time" time without time zone, "p_notes" "text", "p_total_estimate"
numeric, "p_booking_duration_minutes" integer, "p_assigned_crew_id" "uuid",
"p_services" "jsonb", "p_continuation_segments" "jsonb", "p_notification_title"
"text", "p_notification_message" "text", "p_first_name" "text", "p_middle_name"
"text", "p_last_name" "text", "p_wants_products" boolean) RETURNS
TABLE("appointment_id" "uuid", "reference_code" "text")
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
DECLARE
  v_appointment_id uuid;
  v_reference_code text;
BEGIN
  IF p_wants_products IS NULL THEN
    RAISE EXCEPTION 'Choose whether you would like to avail shop products.';
  END IF;

  SELECT created.appointment_id, created.reference_code
    INTO v_appointment_id, v_reference_code
  FROM public.create_booking_atomic(
    p_reference_code, p_booking_request_id, p_customer_name, p_phone,
p_email,
    p_moto_brand, p_moto_model, p_moto_variant, p_moto_year,
p_plate_number,
    p_appointment_date, p_start_time, p_notes, p_total_estimate,
    p_booking_duration_minutes, p_assigned_crew_id, p_services,
    p_continuation_segments, p_notification_title, p_notification_message,
    p_first_name, p_middle_name, p_last_name
```

```
  ) AS created;

  UPDATE public.appointments
  SET wants_products = p_wants_products
  WHERE id = v_appointment_id;

  RETURN QUERY SELECT v_appointment_id, v_reference_code;
END;
$$;


ALTER FUNCTION "public"."create_booking_atomic"("p_reference_code" "text",
"p_booking_request_id" "uuid", "p_customer_name" "text", "p_phone" "text",
"p_email" "text", "p_moto_brand" "text", "p_moto_model" "text", "p_moto_variant"
"text", "p_moto_year" integer, "p_plate_number" "text", "p_appointment_date"
"date", "p_start_time" time without time zone, "p_notes" "text", "p_total_estimate"
numeric, "p_booking_duration_minutes" integer, "p_assigned_crew_id" "uuid",
"p_services" "jsonb", "p_continuation_segments" "jsonb", "p_notification_title"
"text", "p_notification_message" "text", "p_first_name" "text", "p_middle_name"
"text", "p_last_name" "text", "p_wants_products" boolean) OWNER TO
"postgres";


CREATE OR REPLACE FUNCTION "public"."crew_daily_workload"("p_crew_id"
"uuid", "p_date" "date", "p_excluded_appointment_id" "uuid" DEFAULT
NULL::"uuid") RETURNS integer[]
    LANGUAGE "sql" STABLE
    SET "search_path" TO 'public'
    AS $$
  SELECT ARRAY[count(*)::integer, COALESCE(sum(work.minutes), 0)::integer]
  FROM (
    SELECT appointment.booking_duration_minutes AS minutes
    FROM public.appointments AS appointment
    WHERE appointment.assigned_crew_id = p_crew_id
      AND appointment.appointment_date = p_date
      AND appointment.id IS DISTINCT FROM p_excluded_appointment_id
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
    UNION ALL
    SELECT continuation.booking_duration_minutes
    FROM public.appointment_continuations AS continuation
    JOIN public.appointments AS appointment ON appointment.id =
continuation.appointment_id
```

```
    WHERE continuation.assigned_crew_id = p_crew_id
      AND continuation.appointment_date = p_date
      AND appointment.id IS DISTINCT FROM p_excluded_appointment_id
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
  ) AS work;
$$;


ALTER FUNCTION "public"."crew_daily_workload"("p_crew_id" "uuid", "p_date"
"date", "p_excluded_appointment_id" "uuid") OWNER TO "postgres";


CREATE OR REPLACE FUNCTION "public"."current_admin_auth_session_id"()
RETURNS "uuid"
    LANGUAGE "sql" STABLE SECURITY DEFINER
    SET "search_path" TO 'public', 'auth'
    AS $$
  SELECT NULLIF(auth.jwt() ->> 'session_id', '')::uuid;
$$;


ALTER FUNCTION "public"."current_admin_auth_session_id"() OWNER TO
"postgres";


CREATE OR REPLACE FUNCTION
"public"."enforce_active_appointment_capacity"() RETURNS "trigger"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
DECLARE
  v_capacity integer;
  v_overlapping_reservations integer;
  v_start_minutes integer;
  v_opening_minutes integer;
  v_closing_minutes integer;
  v_opening_time time;
  v_closing_time time;
  v_hour_start time;
BEGIN
  -- Archived, replaced, and terminal appointments release both their slot
  -- capacity and their crew assignment.
```

```
  IF NEW.is_archived = true
     OR NEW.rescheduled_to_appointment_id IS NOT NULL
     OR NEW.status NOT IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
THEN
    RETURN NEW;
  END IF;

  SELECT opening_time, closing_time
    INTO v_opening_time, v_closing_time
  FROM public.shop_settings
  WHERE id = true;
  v_opening_time := COALESCE(v_opening_time, '08:00:00'::time);
  v_closing_time := COALESCE(v_closing_time, '17:00:00'::time);
  v_opening_minutes := extract(hour FROM v_opening_time)::integer * 60
    + extract(minute FROM v_opening_time)::integer;
  v_closing_minutes := extract(hour FROM v_closing_time)::integer * 60
    + extract(minute FROM v_closing_time)::integer;
  v_start_minutes := extract(hour FROM NEW.start_time)::integer * 60
    + extract(minute FROM NEW.start_time)::integer;

  IF extract(dow FROM NEW.appointment_date) = 0
     OR NEW.booking_duration_minutes IS NULL
     OR NEW.booking_duration_minutes <= 0
     OR v_start_minutes < v_opening_minutes
     OR v_start_minutes >= v_closing_minutes
     OR mod(v_start_minutes - v_opening_minutes, 30) <> 0
     OR v_start_minutes + NEW.booking_duration_minutes > v_closing_minutes
THEN
    RAISE EXCEPTION 'The appointment must fit within the configured shop
hours on a Monday through Saturday.'
      USING ERRCODE = '23P01';
  END IF;

  -- Serialize every active reservation for one date. The lock is deliberately
  -- acquired before both capacity and crew-overlap checks, so a transaction
  -- that waited on it sees a previously committed reservation.
  PERFORM pg_advisory_xact_lock(hashtext('booking-capacity:' ||
NEW.appointment_date::text));

  v_hour_start := make_time(extract(hour FROM NEW.start_time)::integer, 0, 0);
  SELECT slot.capacity
    INTO v_capacity
  FROM public.time_slots AS slot
  WHERE slot.is_active = true
    AND (slot.start_time = NEW.start_time OR slot.start_time = v_hour_start)
```

```
  ORDER BY (slot.start_time = NEW.start_time) DESC
  LIMIT 1
  FOR UPDATE;

  IF COALESCE(v_capacity, 0) <= 0 THEN
    RAISE EXCEPTION 'The requested time slot is not available.' USING
ERRCODE = '23P01';
  END IF;

  IF EXISTS (
    SELECT 1
    FROM public.schedule_blocks AS block
    WHERE block.block_date = NEW.appointment_date
      AND block.is_active = true
      AND (
        block.start_time IS NULL
        OR (
          COALESCE(block.reason, '') !~ '^RANGE:[0-9]{2}:[0-9]{2}'
          AND block.start_time = NEW.start_time
        )
        OR (
          COALESCE(block.reason, '') ~ '^RANGE:[0-9]{2}:[0-9]{2}'
          AND tsrange(
            NEW.appointment_date + NEW.start_time,
            NEW.appointment_date + NEW.start_time
              + make_interval(mins => NEW.booking_duration_minutes),
            '[)'
          ) && tsrange(
            NEW.appointment_date + block.start_time,
            NEW.appointment_date
              + substring(block.reason FROM '^RANGE:([0-9]{2}:[0-9]{2})')::time,
            '[)'
          )
        )
      )
  ) THEN
    RAISE EXCEPTION 'The shop is closed for the requested schedule.' USING
ERRCODE = '23P01';
  END IF;

  IF NEW.assigned_crew_id IS NOT NULL THEN
    IF NOT EXISTS (
      SELECT 1
      FROM public.crew_members AS crew
      JOIN public.crew_schedules AS schedule ON schedule.crew_id = crew.id
```

```
      WHERE crew.id = NEW.assigned_crew_id
        AND crew.is_active = true
        AND crew.is_archived = false
        AND schedule.schedule_date = NEW.appointment_date
        AND schedule.is_working = true
        AND schedule.start_time IS NOT NULL
        AND schedule.end_time IS NOT NULL
        AND schedule.start_time <= NEW.start_time
        AND schedule.end_time >= NEW.start_time
          + make_interval(mins => NEW.booking_duration_minutes)
    ) THEN
      RAISE EXCEPTION 'The assigned crew member is not scheduled for the
requested appointment period.'
        USING ERRCODE = '23P01';
    END IF;

    IF EXISTS (
      SELECT 1
      FROM public.crew_availability_exceptions AS exception
      WHERE exception.crew_id = NEW.assigned_crew_id
        AND exception.start_date <= NEW.appointment_date
        AND exception.end_date >= NEW.appointment_date
        AND (
          exception.is_all_day
          OR (
            exception.start_time IS NOT NULL
            AND exception.end_time IS NOT NULL
            AND tsrange(
              NEW.appointment_date + NEW.start_time,
              NEW.appointment_date + NEW.start_time
                + make_interval(mins => NEW.booking_duration_minutes),
              '[)'
            ) && tsrange(
              NEW.appointment_date + exception.start_time,
              NEW.appointment_date + exception.end_time,
              '[)'
            )
          )
        )
    ) THEN
      RAISE EXCEPTION 'The assigned crew member is unavailable for the
requested appointment period.'
        USING ERRCODE = '23P01';
    END IF;
  END IF;
```

```
  SELECT count(*)
    INTO v_overlapping_reservations
  FROM (
    SELECT appointment.appointment_date, appointment.start_time,
appointment.booking_duration_minutes
    FROM public.appointments AS appointment
    WHERE appointment.id <> NEW.id
      AND appointment.appointment_date = NEW.appointment_date
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
    UNION ALL
    SELECT continuation.appointment_date, continuation.start_time,
continuation.booking_duration_minutes
    FROM public.appointment_continuations AS continuation
    JOIN public.appointments AS appointment ON appointment.id =
continuation.appointment_id
    WHERE continuation.appointment_date = NEW.appointment_date
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
    UNION ALL
    -- A pending reschedule request holds its requested time until it is
    -- reviewed. The appointment's own request, and the request being approved
    -- (when this row is its replacement), do not count against it.
    SELECT hold.pending_reschedule_date, hold.pending_reschedule_start_time,
      COALESCE(hold.booking_duration_minutes, 75)
    FROM public.appointments AS hold
    WHERE hold.pending_reschedule_request_id IS NOT NULL
      AND hold.pending_reschedule_date = NEW.appointment_date
      AND hold.pending_reschedule_start_time IS NOT NULL
      AND hold.id <> NEW.id
      AND hold.id IS DISTINCT FROM NEW.rescheduled_from_appointment_id
      AND hold.is_archived = false
      AND hold.rescheduled_to_appointment_id IS NULL
      AND hold.status IN ('pending', 'confirmed')
  ) AS reservation
  WHERE tsrange(
    reservation.appointment_date + reservation.start_time,
    reservation.appointment_date + reservation.start_time
      + make_interval(mins => reservation.booking_duration_minutes),
    '[)'
```

```
  ) && tsrange(
    NEW.appointment_date + NEW.start_time,
    NEW.appointment_date + NEW.start_time + make_interval(mins =>
NEW.booking_duration_minutes),
    '[)'
  );

  IF v_overlapping_reservations >= v_capacity THEN
    RAISE EXCEPTION 'The requested schedule is no longer available.' USING
ERRCODE = '23P01';
  END IF;

  -- Capacity alone is not enough: two slots can overlap while there is still
  -- aggregate capacity for another crew. A crew ending at exactly the new
  -- start time is allowed because all reservation ranges are [start, end).
  IF NEW.assigned_crew_id IS NOT NULL AND EXISTS (
    SELECT 1
    FROM (
      SELECT appointment.assigned_crew_id, appointment.appointment_date,
        appointment.start_time, appointment.booking_duration_minutes
      FROM public.appointments AS appointment
      WHERE appointment.id <> NEW.id
        AND appointment.assigned_crew_id = NEW.assigned_crew_id
        AND appointment.appointment_date = NEW.appointment_date
        AND appointment.is_archived = false
        AND appointment.rescheduled_to_appointment_id IS NULL
        AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
      UNION ALL
      SELECT continuation.assigned_crew_id, continuation.appointment_date,
        continuation.start_time, continuation.booking_duration_minutes
      FROM public.appointment_continuations AS continuation
      JOIN public.appointments AS appointment ON appointment.id =
continuation.appointment_id
      WHERE continuation.assigned_crew_id = NEW.assigned_crew_id
        AND continuation.appointment_date = NEW.appointment_date
        AND appointment.is_archived = false
        AND appointment.rescheduled_to_appointment_id IS NULL
        AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
    ) AS crew_reservation
    WHERE tsrange(
      crew_reservation.appointment_date + crew_reservation.start_time,
      crew_reservation.appointment_date + crew_reservation.start_time
        + make_interval(mins => crew_reservation.booking_duration_minutes),
```

```
      '[)'
    ) && tsrange(
      NEW.appointment_date + NEW.start_time,
      NEW.appointment_date + NEW.start_time + make_interval(mins =>
NEW.booking_duration_minutes),
      '[)'
    )
  ) THEN
    RAISE EXCEPTION 'The assigned crew member is no longer available for the
requested appointment period.'
      USING ERRCODE = '23P01';
  END IF;

  RETURN NEW;
END;
$$;


ALTER FUNCTION "public"."enforce_active_appointment_capacity"() OWNER
TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."enforce_active_appointment_continuation_capacity"() RETURNS
"trigger"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
DECLARE
  v_parent public.appointments%ROWTYPE;
  v_opening_time time;
  v_closing_time time;
  v_opening_minutes integer;
  v_closing_minutes integer;
  v_start_minutes integer;
  v_hour_start time;
  v_capacity integer;
  v_overlapping_reservations integer;
BEGIN
  SELECT * INTO v_parent FROM public.appointments WHERE id =
NEW.appointment_id;
  IF NOT FOUND THEN
    RAISE EXCEPTION 'The parent appointment no longer exists.' USING
ERRCODE = '23503';
  END IF;
```

```
  IF v_parent.is_archived = true
     OR v_parent.rescheduled_to_appointment_id IS NOT NULL
     OR v_parent.status NOT IN ('pending', 'confirmed', 'in_progress',
'rescheduled') THEN
    RETURN NEW;
  END IF;

  SELECT opening_time, closing_time INTO v_opening_time, v_closing_time
  FROM public.shop_settings WHERE id = true;
  v_opening_time := COALESCE(v_opening_time, '08:00:00'::time);
  v_closing_time := COALESCE(v_closing_time, '17:00:00'::time);
  v_opening_minutes := extract(hour FROM v_opening_time)::integer * 60
    + extract(minute FROM v_opening_time)::integer;
  v_closing_minutes := extract(hour FROM v_closing_time)::integer * 60
    + extract(minute FROM v_closing_time)::integer;
  v_start_minutes := extract(hour FROM NEW.start_time)::integer * 60
    + extract(minute FROM NEW.start_time)::integer;

  IF extract(dow FROM NEW.appointment_date) = 0
     OR NEW.booking_duration_minutes <= 0
     OR v_start_minutes < v_opening_minutes
     OR v_start_minutes >= v_closing_minutes
     OR mod(v_start_minutes - v_opening_minutes, 30) <> 0
     OR v_start_minutes + NEW.booking_duration_minutes > v_closing_minutes
THEN
    RAISE EXCEPTION 'The continuation must fit within the configured shop
hours in 30-minute start intervals.'
      USING ERRCODE = '23P01';
  END IF;

  IF EXISTS (
    SELECT 1
    FROM public.schedule_blocks AS block
    WHERE block.block_date = NEW.appointment_date
      AND block.is_active = true
      AND (
        block.start_time IS NULL
        OR (
          COALESCE(block.reason, '') NOT LIKE 'RANGE:%'
          AND block.start_time = NEW.start_time
        )
        OR (
          COALESCE(block.reason, '') LIKE 'RANGE:%'
          AND tsrange(
```

```
          NEW.appointment_date + NEW.start_time,
          NEW.appointment_date + NEW.start_time
            + make_interval(mins => NEW.booking_duration_minutes),
          '[)'
        ) && tsrange(
          NEW.appointment_date + block.start_time,
          NEW.appointment_date
            + substring(block.reason FROM '^RANGE:([0-9]{2}:[0-9]{2})')::time,
          '[)'
        )
      )
    )
  ) THEN
    RAISE EXCEPTION 'The shop is closed for the continuation schedule.' USING
ERRCODE = '23P01';
  END IF;

  PERFORM pg_advisory_xact_lock(hashtext('booking-capacity:' ||
NEW.appointment_date::text));
  v_hour_start := make_time(extract(hour FROM NEW.start_time)::integer, 0, 0);

  SELECT slot.capacity INTO v_capacity
  FROM public.time_slots AS slot
  WHERE slot.is_active = true
    AND (slot.start_time = NEW.start_time OR slot.start_time = v_hour_start)
  ORDER BY (slot.start_time = NEW.start_time) DESC
  LIMIT 1
  FOR UPDATE;

  IF COALESCE(v_capacity, 0) <= 0 THEN
    RAISE EXCEPTION 'The requested continuation time slot is not available.'
USING ERRCODE = '23P01';
  END IF;

  SELECT count(*) INTO v_overlapping_reservations
  FROM (
    SELECT appointment.appointment_date, appointment.start_time,
appointment.booking_duration_minutes
    FROM public.appointments AS appointment
    WHERE appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
      AND appointment.appointment_date = NEW.appointment_date
    UNION ALL
```

```
    SELECT continuation.appointment_date, continuation.start_time,
continuation.booking_duration_minutes
    FROM public.appointment_continuations AS continuation
    JOIN public.appointments AS appointment ON appointment.id =
continuation.appointment_id
    WHERE continuation.id <> NEW.id
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
      AND continuation.appointment_date = NEW.appointment_date
    UNION ALL
    -- Pending reschedule requests hold their requested times too.
    SELECT hold.pending_reschedule_date, hold.pending_reschedule_start_time,
      COALESCE(hold.booking_duration_minutes, 75)
    FROM public.appointments AS hold
    WHERE hold.pending_reschedule_request_id IS NOT NULL
      AND hold.pending_reschedule_date = NEW.appointment_date
      AND hold.pending_reschedule_start_time IS NOT NULL
      AND hold.is_archived = false
      AND hold.rescheduled_to_appointment_id IS NULL
      AND hold.status IN ('pending', 'confirmed')
  ) AS reservation
  WHERE tsrange(
    reservation.appointment_date + reservation.start_time,
    reservation.appointment_date + reservation.start_time
      + make_interval(mins => reservation.booking_duration_minutes),
    '[)'
  ) && tsrange(
    NEW.appointment_date + NEW.start_time,
    NEW.appointment_date + NEW.start_time + make_interval(mins =>
NEW.booking_duration_minutes),
    '[)'
  );

  IF v_overlapping_reservations >= v_capacity THEN
    RAISE EXCEPTION 'The requested continuation schedule is no longer
available.' USING ERRCODE = '23P01';
  END IF;

  IF NOT public.is_crew_available_for_appointment(
    NEW.assigned_crew_id,
    NEW.appointment_date,
    NEW.start_time,
    NEW.booking_duration_minutes,
```

```
    NULL,
    NEW.id
  ) THEN
    RAISE EXCEPTION 'The assigned mechanic is no longer available for the
continuation schedule.'
      USING ERRCODE = '23P01';
  END IF;

  RETURN NEW;
END;
$$;


ALTER FUNCTION
"public"."enforce_active_appointment_continuation_capacity"() OWNER TO
"postgres";


CREATE OR REPLACE FUNCTION
"public"."enforce_active_appointment_crew_conflict"() RETURNS "trigger"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
BEGIN
  IF NEW.is_archived = true
     OR NEW.rescheduled_to_appointment_id IS NOT NULL
     OR NEW.assigned_crew_id IS NULL
     OR NEW.status NOT IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
THEN
    RETURN NEW;
  END IF;

  PERFORM pg_advisory_xact_lock(hashtext('booking-capacity:' ||
NEW.appointment_date::text));

  IF NOT public.is_crew_available_for_appointment(
    NEW.assigned_crew_id,
    NEW.appointment_date,
    NEW.start_time,
    NEW.booking_duration_minutes,
    NEW.id
  ) THEN
    RAISE EXCEPTION 'The assigned mechanic is no longer available for the
requested schedule.'
      USING ERRCODE = '23P01';
```

```
  END IF;

  RETURN NEW;
END;
$$;


ALTER FUNCTION "public"."enforce_active_appointment_crew_conflict"()
OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."enforce_customer_upcoming_appointment_limit"() RETURNS "trigger"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
DECLARE
  v_limit constant integer := 3;
  v_today date := (now() AT TIME ZONE 'Asia/Manila')::date;
  v_phone text := COALESCE(public.normalize_philippine_mobile(NEW.phone),
btrim(NEW.phone));
  v_email text := NULLIF(lower(btrim(COALESCE(NEW.email, ''))), '');
BEGIN
  -- A replacement created by an approved reschedule takes over an existing
  -- appointment, so it is not limited.
  IF NEW.rescheduled_from_appointment_id IS NOT NULL
     OR NEW.is_archived
     OR NEW.status NOT IN ('pending', 'confirmed', 'in_progress')
     OR NEW.appointment_date < v_today THEN
    RETURN NEW;
  END IF;

  PERFORM
pg_advisory_xact_lock(hashtext('customer-upcoming-appointments:' ||
v_phone));
  IF (
    SELECT count(*)
    FROM public.appointments AS appointment
    WHERE appointment.phone = v_phone
      AND appointment.id <> NEW.id
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress')
      AND appointment.appointment_date >= v_today
  ) >= v_limit THEN
```

```
    RAISE EXCEPTION 'This mobile number already has % upcoming
appointments, the most allowed at one time. Please cancel one or wait until one
is done before booking another.', v_limit
      USING ERRCODE = '23514';
  END IF;

  IF v_email IS NOT NULL THEN
    PERFORM
pg_advisory_xact_lock(hashtext('customer-upcoming-appointments:' || v_email));
    IF (
      SELECT count(*)
      FROM public.appointments AS appointment
      WHERE lower(btrim(appointment.email)) = v_email
        AND appointment.id <> NEW.id
        AND appointment.is_archived = false
        AND appointment.rescheduled_to_appointment_id IS NULL
        AND appointment.status IN ('pending', 'confirmed', 'in_progress')
        AND appointment.appointment_date >= v_today
    ) >= v_limit THEN
      RAISE EXCEPTION 'This email address already has % upcoming
appointments, the most allowed at one time. Please cancel one or wait until one
is done before booking another.', v_limit
        USING ERRCODE = '23514';
    END IF;
  END IF;

  RETURN NEW;
END;
$$;


ALTER FUNCTION "public"."enforce_customer_upcoming_appointment_limit"()
OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."enforce_public_rate_limit"("p_scope" "text", "p_subject" "text", "p_limit"
integer, "p_window_seconds" integer) RETURNS boolean
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
DECLARE
  v_allowed boolean;
BEGIN
  IF p_limit < 1 OR p_window_seconds < 1 THEN
```

```
    RAISE EXCEPTION 'Rate-limit configuration must be positive.';
  END IF;

  -- Keep the small counter table bounded without relying on a scheduled job.
  DELETE FROM public.public_endpoint_rate_limits
  WHERE scope = p_scope
    AND window_started_at < now() - make_interval(secs =>
GREATEST(p_window_seconds * 2, 86400));

  INSERT INTO public.public_endpoint_rate_limits (
    scope,
    subject,
    window_started_at,
    request_count,
    updated_at
  )
  VALUES (p_scope, p_subject, now(), 1, now())
  ON CONFLICT (scope, subject) DO UPDATE
  SET
    window_started_at = CASE
      WHEN public_endpoint_rate_limits.window_started_at
        <= now() - make_interval(secs => p_window_seconds)
        THEN now()
      ELSE public_endpoint_rate_limits.window_started_at
    END,
    request_count = CASE
      WHEN public_endpoint_rate_limits.window_started_at
        <= now() - make_interval(secs => p_window_seconds)
        THEN 1
      ELSE public_endpoint_rate_limits.request_count + 1
    END,
    updated_at = now()
  RETURNING request_count <= p_limit INTO v_allowed;

  RETURN COALESCE(v_allowed, false);
END;
$$;


ALTER FUNCTION "public"."enforce_public_rate_limit"("p_scope" "text",
"p_subject" "text", "p_limit" integer, "p_window_seconds" integer) OWNER TO
"postgres";


CREATE OR REPLACE FUNCTION
```

```
"public"."enforce_reschedule_request_capacity"() RETURNS "trigger"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
DECLARE
  v_date date := NEW.pending_reschedule_date;
  v_start time := NEW.pending_reschedule_start_time;
  v_duration integer := COALESCE(NEW.booking_duration_minutes, 75);
  v_capacity integer;
  v_overlapping integer;
  v_unassigned integer;
  v_free_crew integer;
BEGIN
  IF NEW.pending_reschedule_request_id IS NULL
     OR v_date IS NULL
     OR v_start IS NULL
     OR NEW.is_archived
     OR NEW.rescheduled_to_appointment_id IS NOT NULL
     OR NEW.status NOT IN ('pending', 'confirmed') THEN
    RETURN NEW;
  END IF;
  IF OLD.pending_reschedule_request_id IS NOT DISTINCT FROM
NEW.pending_reschedule_request_id
     AND OLD.pending_reschedule_date IS NOT DISTINCT FROM v_date
     AND OLD.pending_reschedule_start_time IS NOT DISTINCT FROM v_start
THEN
    RETURN NEW;
  END IF;

  PERFORM pg_advisory_xact_lock(hashtext('booking-capacity:' || v_date::text));

  SELECT slot.capacity
    INTO v_capacity
  FROM public.time_slots AS slot
  WHERE slot.is_active = true
    AND (
      slot.start_time = v_start
      OR slot.start_time = make_time(extract(hour FROM v_start)::integer, 0, 0)
    )
  ORDER BY (slot.start_time = v_start) DESC
  LIMIT 1;

  SELECT count(*), count(*) FILTER (WHERE reservation.crew_id IS NULL)
    INTO v_overlapping, v_unassigned
  FROM (
```

```
    SELECT appointment.start_time, appointment.booking_duration_minutes AS
minutes,
      appointment.assigned_crew_id AS crew_id
    FROM public.appointments AS appointment
    WHERE appointment.id <> NEW.id
      AND appointment.appointment_date = v_date
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
    UNION ALL
    SELECT continuation.start_time, continuation.booking_duration_minutes,
      continuation.assigned_crew_id
    FROM public.appointment_continuations AS continuation
    JOIN public.appointments AS parent ON parent.id =
continuation.appointment_id
    WHERE continuation.appointment_date = v_date
      AND parent.id <> NEW.id
      AND parent.is_archived = false
      AND parent.rescheduled_to_appointment_id IS NULL
      AND parent.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
    UNION ALL
    SELECT hold.pending_reschedule_start_time,
COALESCE(hold.booking_duration_minutes, 75),
      NULL::uuid
    FROM public.appointments AS hold
    WHERE hold.id <> NEW.id
      AND hold.pending_reschedule_request_id IS NOT NULL
      AND hold.pending_reschedule_date = v_date
      AND hold.pending_reschedule_start_time IS NOT NULL
      AND hold.is_archived = false
      AND hold.rescheduled_to_appointment_id IS NULL
      AND hold.status IN ('pending', 'confirmed')
  ) AS reservation
  WHERE tsrange(
    v_date + reservation.start_time,
    v_date + reservation.start_time + make_interval(mins => reservation.minutes),
    '[)'
  ) && tsrange(v_date + v_start, v_date + v_start + make_interval(mins =>
v_duration), '[)');

  IF COALESCE(v_capacity, 0) - v_overlapping < 1 THEN
    RAISE EXCEPTION 'The requested time is no longer available. Please
choose another time.'
      USING ERRCODE = '23P01';
```

```
  END IF;

  SELECT count(*)
    INTO v_free_crew
  FROM public.crew_members AS crew
  WHERE crew.is_active = true
    AND crew.is_archived = false
    AND public.is_crew_available_for_appointment(crew.id, v_date, v_start,
v_duration, NEW.id);

  IF v_free_crew - v_unassigned < 1 THEN
    RAISE EXCEPTION 'No mechanic is available at the requested time. Please
choose another time.'
      USING ERRCODE = '23P01';
  END IF;

  RETURN NEW;
END;
$$;


ALTER FUNCTION "public"."enforce_reschedule_request_capacity"() OWNER
TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."enqueue_due_service_progress_notifications"() RETURNS integer
    LANGUAGE "plpgsql" SECURITY DEFINER
    SET "search_path" TO 'public'
    AS $$
DECLARE
  v_created integer := 0;
  v_now timestamptz := now();
BEGIN
  -- Lock appointments before their notification rows so a concurrent admin
  -- action wins without producing a stale or duplicate alert.
  WITH due_appointments AS (
    SELECT appointment.id, appointment.reference_code,
appointment.customer_name
    FROM public.appointments AS appointment
    JOIN LATERAL (
      SELECT sum(service.duration_minutes)::integer AS
service_duration_minutes
      FROM public.appointment_services AS service
      WHERE service.appointment_id = appointment.id
```

```
    ) AS service_duration ON service_duration.service_duration_minutes > 0
    WHERE appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status = 'confirmed'
      AND appointment.service_started_at IS NULL
      AND (appointment.appointment_date + appointment.start_time) AT TIME
ZONE 'Asia/Manila' <= v_now
      AND COALESCE(appointment.arrival_notification_snoozed_until, v_now) <=
v_now
      AND NOT EXISTS (
        SELECT 1
        FROM public.notifications AS notification
        WHERE notification.appointment_id = appointment.id
          AND notification.type = 'service_arrival'
          AND notification.is_read = false
      )
    FOR UPDATE OF appointment SKIP LOCKED
  ),
  inserted AS (
    INSERT INTO public.notifications (type, title, message, appointment_id)
    SELECT
      'service_arrival',
      'Customer expected: ' || due.reference_code,
      due.customer_name || ' is scheduled to arrive for service now.',
      due.id
    FROM due_appointments AS due
    ON CONFLICT DO NOTHING
    RETURNING 1
  )
  SELECT count(*) INTO v_created FROM inserted;

  WITH due_appointments AS (
    SELECT appointment.id, appointment.reference_code,
appointment.customer_name
    FROM public.appointments AS appointment
    JOIN LATERAL (
      SELECT sum(service.duration_minutes)::integer AS
service_duration_minutes
      FROM public.appointment_services AS service
      WHERE service.appointment_id = appointment.id
    ) AS service_duration ON service_duration.service_duration_minutes > 0
    WHERE appointment.is_archived = false
      AND appointment.status = 'in_progress'
      AND appointment.service_started_at IS NOT NULL
      AND appointment.service_ended_at IS NULL
```

```
    AND appointment.service_started_at
      + make_interval(mins => service_duration.service_duration_minutes) <=
v_now
    AND COALESCE(appointment.completion_notification_snoozed_until,
v_now) <= v_now
    AND NOT EXISTS (
      SELECT 1
      FROM public.notifications AS notification
      WHERE notification.appointment_id = appointment.id
        AND notification.type = 'service_completion'
        AND notification.is_read = false
    )
  FOR UPDATE OF appointment SKIP LOCKED
 ),
 inserted AS (
  INSERT INTO public.notifications (type, title, message, appointment_id)
  SELECT
    'service_completion',
    'Service duration ended: ' || due.reference_code,
    'The expected service duration for ' || due.customer_name || ' has ended.',
    due.id
  FROM due_appointments AS due
  ON CONFLICT DO NOTHING
  RETURNING 1
 )
 SELECT v_created + count(*) INTO v_created FROM inserted;

 RETURN v_created;
END;
$$;


ALTER FUNCTION "public"."enqueue_due_service_progress_notifications"()
OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."get_admin_customers_page"("p_search" "text" DEFAULT NULL::"text",
"p_limit" integer DEFAULT 25, "p_offset" integer DEFAULT 0) RETURNS "jsonb"
   LANGUAGE "plpgsql" STABLE
   SET "search_path" TO 'public', 'auth'
   AS $$
DECLARE
 v_limit integer := LEAST(GREATEST(COALESCE(p_limit, 25), 1), 100);
 v_offset integer := GREATEST(COALESCE(p_offset, 0), 0);
```

```
  v_search text := NULLIF(BTRIM(p_search), '');
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;
  RETURN (
    WITH customers AS (
      SELECT a.phone,
        (ARRAY_AGG(a.customer_name ORDER BY a.appointment_date DESC,
a.created_at DESC))[1] AS customer_name,
        COUNT(*)::integer AS visits,
        COALESCE(SUM(a.total_estimate) FILTER (WHERE a.status =
'completed'), 0) AS completed_spend,
        MAX(a.appointment_date) AS last_booking,
        ARRAY_AGG(DISTINCT CONCAT_WS(' ', a.moto_brand, a.moto_model) ||
' (' || a.plate_number || ')') AS units
      FROM public.appointments a GROUP BY a.phone
    ), filtered AS (
      SELECT * FROM customers c
      WHERE v_search IS NULL OR CONCAT_WS(' ', c.customer_name,
c.phone) ILIKE '%' || v_search || '%'
    ), paged AS (
      SELECT * FROM filtered ORDER BY last_booking DESC, customer_name
ASC LIMIT v_limit OFFSET v_offset
    )
    SELECT jsonb_build_object(
      'total', (SELECT COUNT(*) FROM filtered),
      'rows', COALESCE((SELECT jsonb_agg(TO_JSONB(paged)) FROM paged),
'[]'::jsonb)
    )
  );
END;
$$;


ALTER FUNCTION "public"."get_admin_customers_page"("p_search" "text",
"p_limit" integer, "p_offset" integer) OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."get_admin_dashboard_chart"("p_from" "date", "p_to" "date",
"p_by_month" boolean DEFAULT false) RETURNS "jsonb"
    LANGUAGE "plpgsql" STABLE
    SET "search_path" TO 'public', 'auth'
    AS $$
```

```
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;
  IF p_from IS NULL OR p_to IS NULL OR p_from > p_to THEN
    RAISE EXCEPTION 'A valid dashboard date range is required';
  END IF;

  RETURN COALESCE((
    SELECT jsonb_agg(to_jsonb(bucket) ORDER BY bucket.period)
    FROM (
      SELECT
        (
          CASE
            WHEN p_by_month THEN date_trunc('month',
appointment.appointment_date::timestamp)::date
            ELSE appointment.appointment_date
          END
        )::text AS period,
        count(*) FILTER (WHERE appointment.status = 'completed') AS completed,
        count(*) FILTER (WHERE appointment.status = 'confirmed') AS confirmed,
        count(*) FILTER (
          WHERE appointment.status IN ('cancelled', 'rejected', 'no_show')
        ) AS cancelled,
        count(*) FILTER (WHERE appointment.status = 'in_progress') AS
in_progress
      FROM public.appointments AS appointment
      WHERE (appointment.is_archived = false OR appointment.status =
'completed')
        AND appointment.appointment_date BETWEEN p_from AND p_to
      GROUP BY 1
    ) AS bucket
  ), '[]'::jsonb);
END;
$$;


ALTER FUNCTION "public"."get_admin_dashboard_chart"("p_from" "date",
"p_to" "date", "p_by_month" boolean) OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."get_admin_dashboard_summary"("p_from" "date", "p_to" "date",
"p_previous_from" "date", "p_previous_to" "date") RETURNS "jsonb"
    LANGUAGE "plpgsql" STABLE
```

```
    SET "search_path" TO 'public', 'auth'
    AS $$
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;
  IF p_from IS NULL OR p_to IS NULL OR p_from > p_to
     OR p_previous_from IS NULL OR p_previous_to IS NULL OR
p_previous_from > p_previous_to THEN
    RAISE EXCEPTION 'A valid dashboard date range is required';
  END IF;

  RETURN (
    WITH scoped AS (
      SELECT
        appointment.status,
        appointment.phone,
        appointment.appointment_date BETWEEN p_from AND p_to AS in_current,
        appointment.appointment_date BETWEEN p_previous_from AND
p_previous_to AS in_previous,
        -- A rescheduled booking's old record is replaced by its new appointment.
        appointment.status NOT IN ('cancelled', 'rejected', 'no_show')
          AND appointment.rescheduled_to_appointment_id IS NULL AS is_active
      FROM public.appointments AS appointment
      -- Completed appointments always count, as in the financial report.
      WHERE (appointment.is_archived = false OR appointment.status =
'completed')
        AND (
          appointment.appointment_date BETWEEN p_from AND p_to
          OR appointment.appointment_date BETWEEN p_previous_from AND
p_previous_to
        )
    ),
    -- Waiting for the shop whatever their date: new bookings to approve and
    -- reschedule requests to review.
    open_requests AS (
      SELECT
        count(*) FILTER (
          WHERE appointment.status = 'pending'
            AND appointment.pending_reschedule_request_id IS NULL
        ) AS new_bookings,
        count(*) FILTER (
          WHERE appointment.pending_reschedule_request_id IS NOT NULL
        ) AS reschedule_requests
      FROM public.appointments AS appointment
```

```
    WHERE appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND (
        appointment.status = 'pending'
        OR (
          appointment.pending_reschedule_request_id IS NOT NULL
          AND appointment.status IN ('pending', 'confirmed')
        )
      )
  )
  SELECT jsonb_build_object(
    'current', jsonb_build_object(
      'appointments', count(*) FILTER (WHERE in_current AND is_active),
      'customers', count(DISTINCT phone) FILTER (WHERE in_current AND
is_active),
      'revenue', (
        public.get_completed_financial_report_page(p_from, p_to, NULL, NULL, 1,
0)
          -> 'metrics' ->> 'total_revenue'
      )::numeric,
      'statuses', jsonb_build_object(
        'pending', count(*) FILTER (WHERE in_current AND status = 'pending'),
        'confirmed', count(*) FILTER (WHERE in_current AND status =
'confirmed'),
        'in_progress', count(*) FILTER (WHERE in_current AND status =
'in_progress'),
        'completed', count(*) FILTER (WHERE in_current AND status =
'completed'),
        'rescheduled', count(*) FILTER (WHERE in_current AND status =
'rescheduled'),
        'cancelled', count(*) FILTER (WHERE in_current AND status IN
('cancelled', 'rejected')),
        'no_show', count(*) FILTER (WHERE in_current AND status = 'no_show')
      )
    ),
    'previous', jsonb_build_object(
      'appointments', count(*) FILTER (WHERE in_previous AND is_active),
      'customers', count(DISTINCT phone) FILTER (WHERE in_previous AND
is_active),
      'revenue', (
        public.get_completed_financial_report_page(p_previous_from,
p_previous_to, NULL, NULL, 1, 0)
          -> 'metrics' ->> 'total_revenue'
      )::numeric
    ),
```

```
      'needs_action', (
        SELECT jsonb_build_object(
          'new_bookings', open_requests.new_bookings,
          'reschedule_requests', open_requests.reschedule_requests
        )
        FROM open_requests
      )
    )
    FROM scoped
  );
END;
$$;


ALTER FUNCTION "public"."get_admin_dashboard_summary"("p_from" "date",
"p_to" "date", "p_previous_from" "date", "p_previous_to" "date") OWNER TO
"postgres";


CREATE OR REPLACE FUNCTION
"public"."get_admin_report_page"("p_report_kind" "text", "p_from" "date", "p_to"
"date", "p_status" "text" DEFAULT NULL::"text", "p_category" "text" DEFAULT
NULL::"text", "p_service_name" "text" DEFAULT NULL::"text", "p_limit" integer
DEFAULT 25, "p_offset" integer DEFAULT 0) RETURNS "jsonb"
    LANGUAGE "plpgsql" STABLE
    SET "search_path" TO 'public', 'auth'
    AS $$
DECLARE
  v_limit integer := LEAST(GREATEST(COALESCE(p_limit, 25), 1), 250);
  v_offset integer := GREATEST(COALESCE(p_offset, 0), 0);
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;
  IF p_report_kind NOT IN ('bookings', 'services') THEN
    RAISE EXCEPTION 'Unknown report type';
  END IF;
  IF p_from IS NULL OR p_to IS NULL OR p_from > p_to THEN
    RAISE EXCEPTION 'A valid report date range is required';
  END IF;

  IF p_report_kind = 'bookings' THEN
    RETURN (
      WITH filtered AS (
        SELECT
```

```
        appointment.id,
        appointment.reference_code,
        appointment.customer_name,
        appointment.appointment_date,
        appointment.status,
        service_totals.service,
        service_totals.service_subtotal,
        product_totals.products,
        product_totals.product_subtotal,
        service_totals.service_subtotal + product_totals.product_subtotal AS
total_amount,
        service_totals.service_subtotal AS total_estimate
      FROM public.appointments AS appointment
      CROSS JOIN LATERAL (
        SELECT
          COALESCE(string_agg(service.service_name, ', ' ORDER BY
service.service_name), '—') AS service,
          COALESCE(sum(service.price), 0)::numeric AS service_subtotal
        FROM public.appointment_services AS service
        WHERE service.appointment_id = appointment.id
      ) AS service_totals
      CROSS JOIN LATERAL (
        SELECT
          COALESCE(string_agg(product.product_name || ' × ' || product.quantity, ',
' ORDER BY product.product_name), '—') AS products,
          COALESCE(sum(product.quantity * product.unit_price), 0)::numeric AS
product_subtotal
        FROM public.appointment_products AS product
        WHERE product.appointment_id = appointment.id
      ) AS product_totals
      WHERE appointment.is_archived = false
        AND appointment.appointment_date BETWEEN p_from AND p_to
        AND (p_status IS NULL OR appointment.status = p_status)
        AND (
          (p_category IS NULL AND p_service_name IS NULL)
          OR EXISTS (
            SELECT 1
            FROM public.appointment_services AS selected_service
            LEFT JOIN public.services AS catalog_service ON catalog_service.id =
selected_service.service_id
            WHERE selected_service.appointment_id = appointment.id
              AND (p_category IS NULL OR COALESCE(catalog_service.category,
'Uncategorized') = p_category)
              AND (p_service_name IS NULL OR selected_service.service_name =
p_service_name)
```

```
          )
        )
    ),
    grouped_status AS (
      SELECT status, count(*)::integer AS total FROM filtered GROUP BY status
    ),
    paged AS (
      SELECT * FROM filtered
      ORDER BY appointment_date DESC, reference_code ASC
      LIMIT v_limit OFFSET v_offset
    )
    SELECT jsonb_build_object(
      'total', (SELECT count(*) FROM filtered),
      'rows', COALESCE((SELECT jsonb_agg(to_jsonb(paged)) FROM paged),
'[]'::jsonb),
      'metrics', jsonb_build_object(
        'total_bookings', (SELECT count(*) FROM filtered),
        'completed_bookings', (SELECT count(*) FROM filtered WHERE status =
'completed'),
        'completed_value', (SELECT COALESCE(sum(total_amount) FILTER
(WHERE status = 'completed'), 0) FROM filtered),
        'total_value', (SELECT COALESCE(sum(total_amount), 0) FROM filtered),
        'service_total', (SELECT COALESCE(sum(service_subtotal), 0) FROM
filtered),
        'product_total', (SELECT COALESCE(sum(product_subtotal), 0) FROM
filtered),
        'status_counts', COALESCE((SELECT jsonb_object_agg(status, total)
FROM grouped_status), '{}'::jsonb)
      )
    )
  );
 END IF;

 RETURN (
  WITH filtered AS (
    SELECT
      appointment_service.appointment_id,
      appointment_service.service_id,
      appointment_service.service_name,
      appointment_service.price,
      appointment.reference_code,
      appointment.customer_name,
      appointment.appointment_date,
      appointment.status,
      COALESCE(catalog_service.category, 'Uncategorized') AS category
```

```
    FROM public.appointment_services AS appointment_service
    JOIN public.appointments AS appointment ON appointment.id =
appointment_service.appointment_id
    LEFT JOIN public.services AS catalog_service ON catalog_service.id =
appointment_service.service_id
    WHERE appointment.is_archived = false
      AND appointment.appointment_date BETWEEN p_from AND p_to
      AND (p_status IS NULL OR appointment.status = p_status)
      AND (p_category IS NULL OR COALESCE(catalog_service.category,
'Uncategorized') = p_category)
      AND (p_service_name IS NULL OR appointment_service.service_name =
p_service_name)
  ),
  grouped_status AS (
    SELECT status, count(*)::integer AS total FROM filtered GROUP BY status
  ),
  paged AS (
    SELECT * FROM filtered
    ORDER BY appointment_date DESC, reference_code ASC, service_name
ASC
    LIMIT v_limit OFFSET v_offset
  )
  SELECT jsonb_build_object(
    'total', (SELECT count(*) FROM filtered),
    'rows', COALESCE((SELECT jsonb_agg(to_jsonb(paged)) FROM paged),
'[]'::jsonb),
    'metrics', jsonb_build_object(
      'total_bookings', (SELECT count(DISTINCT appointment_id) FROM
filtered),
      'completed_rows', (SELECT count(*) FROM filtered WHERE status =
'completed'),
      'completed_value', (SELECT COALESCE(sum(price) FILTER (WHERE
status = 'completed'), 0) FROM filtered),
      'total_value', (SELECT COALESCE(sum(price), 0) FROM filtered),
      'status_counts', COALESCE((SELECT jsonb_object_agg(status, total)
FROM grouped_status), '{}'::jsonb)
    )
  )
 );
END;
$$;


ALTER FUNCTION "public"."get_admin_report_page"("p_report_kind" "text",
"p_from" "date", "p_to" "date", "p_status" "text", "p_category" "text",
```

```
"p_service_name" "text", "p_limit" integer, "p_offset" integer) OWNER TO
"postgres";


CREATE OR REPLACE FUNCTION
"public"."get_completed_financial_report_page"("p_from" "date", "p_to" "date",
"p_category" "text" DEFAULT NULL::"text", "p_service_names" "text"[] DEFAULT
NULL::"text"[], "p_limit" integer DEFAULT 25, "p_offset" integer DEFAULT 0)
RETURNS "jsonb"
    LANGUAGE "plpgsql" STABLE
    SET "search_path" TO 'public', 'auth'
    AS $$
DECLARE
  v_limit integer := LEAST(GREATEST(COALESCE(p_limit, 25), 1), 250);
  v_offset integer := GREATEST(COALESCE(p_offset, 0), 0);
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;
  IF p_from IS NULL OR p_to IS NULL OR p_from > p_to THEN
    RAISE EXCEPTION 'A valid report date range is required';
  END IF;

  RETURN (
    WITH completed_appointments AS (
      SELECT
        appointment.id,
        appointment.reference_code,
        COALESCE(
          (appointment.service_ended_at AT TIME ZONE 'Asia/Manila')::date,
          appointment.appointment_date
        ) AS transaction_date
      FROM public.appointments AS appointment
      WHERE appointment.status = 'completed'
    ),
    filtered AS (
      SELECT
        appointment.id,
        appointment.reference_code,
        appointment.transaction_date,
        service_totals.services,
        service_totals.service_amount,
        product_totals.products,
        product_totals.product_amount,
        service_totals.service_amount + product_totals.product_amount AS
```

```
total_amount
      FROM completed_appointments AS appointment
      CROSS JOIN LATERAL (
        SELECT
          COALESCE(string_agg(service.service_name, ', ' ORDER BY
service.service_name), '-') AS services,
          COALESCE(sum(service.price), 0)::numeric AS service_amount
        FROM public.appointment_services AS service
        WHERE service.appointment_id = appointment.id
      ) AS service_totals
      CROSS JOIN LATERAL (
        SELECT
          COALESCE(
            string_agg(product.product_name || ' x ' || product.quantity, ', ' ORDER
BY product.product_name),
            '-'
          ) AS products,
          COALESCE(sum(product.quantity * product.unit_price), 0)::numeric AS
product_amount
        FROM public.appointment_products AS product
        WHERE product.appointment_id = appointment.id
      ) AS product_totals
      WHERE appointment.transaction_date BETWEEN p_from AND p_to
        AND (
          (p_category IS NULL AND COALESCE(cardinality(p_service_names), 0) =
0)
          OR EXISTS (
            SELECT 1
            FROM public.appointment_services AS selected_service
            LEFT JOIN public.services AS catalog_service ON catalog_service.id =
selected_service.service_id
            WHERE selected_service.appointment_id = appointment.id
              AND (p_category IS NULL OR COALESCE(catalog_service.category,
'Uncategorized') = p_category)
              AND (
                COALESCE(cardinality(p_service_names), 0) = 0
                OR selected_service.service_name = ANY(p_service_names)
              )
          )
        )
    ),
    scoped_services AS (
      SELECT
        service.appointment_id,
        service.service_name,
```

```
      service.price,
      COALESCE(catalog_service.category, 'Uncategorized') AS category
    FROM public.appointment_services AS service
    JOIN filtered AS appointment ON appointment.id = service.appointment_id
    LEFT JOIN public.services AS catalog_service ON catalog_service.id =
service.service_id
  ),
  scoped_products AS (
    SELECT product.appointment_id, product.product_name, product.quantity,
product.unit_price
    FROM public.appointment_products AS product
    JOIN filtered AS appointment ON appointment.id = product.appointment_id
  ),
  paged AS (
    SELECT *
    FROM filtered
    ORDER BY transaction_date DESC, reference_code ASC
    LIMIT v_limit OFFSET v_offset
  )
  SELECT jsonb_build_object(
    'total', (SELECT count(*) FROM filtered),
    'rows', COALESCE((SELECT jsonb_agg(to_jsonb(paged)) FROM paged),
'[]'::jsonb),
    'metrics', jsonb_build_object(
      'total_revenue', (SELECT COALESCE(sum(total_amount), 0) FROM
filtered),
      'service_revenue', (SELECT COALESCE(sum(service_amount), 0) FROM
filtered),
      'product_revenue', (SELECT COALESCE(sum(product_amount), 0) FROM
filtered),
      'completed_transactions', (SELECT count(*) FROM filtered),
      'total_completed_services', (SELECT count(*) FROM scoped_services),
      'total_products_sold', (SELECT COALESCE(sum(quantity), 0) FROM
scoped_products),
      'average_transaction_amount', (
        SELECT COALESCE(avg(total_amount), 0) FROM filtered
      )
    ),
    'service_performance', COALESCE((
      SELECT jsonb_agg(to_jsonb(performance) ORDER BY
performance.revenue DESC, performance.service_name)
      FROM (
        SELECT
          service_name,
          count(*)::integer AS completed_count,
```

```
          COALESCE(sum(price), 0)::numeric AS revenue
        FROM scoped_services
        GROUP BY service_name
      ) AS performance
    ), '[]'::jsonb),
    'product_sales', COALESCE((
      SELECT jsonb_agg(to_jsonb(sales) ORDER BY sales.total_sales DESC,
sales.product_name, sales.unit_price)
      FROM (
        SELECT
          product_name,
          sum(quantity)::integer AS quantity_sold,
          unit_price,
          COALESCE(sum(quantity * unit_price), 0)::numeric AS total_sales
        FROM scoped_products
        GROUP BY product_name, unit_price
      ) AS sales
    ), '[]'::jsonb),
    'daily_revenue', COALESCE((
      SELECT jsonb_agg(to_jsonb(day_revenue) ORDER BY
day_revenue.period)
      FROM (
        SELECT
          transaction_date::text AS period,
          count(*)::integer AS completed_transactions,
          COALESCE(sum(service_amount), 0)::numeric AS service_revenue,
          COALESCE(sum(product_amount), 0)::numeric AS product_revenue,
          COALESCE(sum(total_amount), 0)::numeric AS total_revenue
        FROM filtered
        GROUP BY transaction_date
      ) AS day_revenue
    ), '[]'::jsonb),
    'monthly_revenue', COALESCE((
      SELECT jsonb_agg(to_jsonb(month_revenue) ORDER BY
month_revenue.period)
      FROM (
        SELECT
          date_trunc('month', transaction_date::timestamp)::date::text AS period,
          count(*)::integer AS completed_transactions,
          COALESCE(sum(service_amount), 0)::numeric AS service_revenue,
          COALESCE(sum(product_amount), 0)::numeric AS product_revenue,
          COALESCE(sum(total_amount), 0)::numeric AS total_revenue
        FROM filtered
        GROUP BY date_trunc('month', transaction_date::timestamp)::date
      ) AS month_revenue
```

```
      ), '[]'::jsonb)
    )
  );
END;
$$;


ALTER FUNCTION "public"."get_completed_financial_report_page"("p_from"
"date", "p_to" "date", "p_category" "text", "p_service_names" "text"[], "p_limit"
integer, "p_offset" integer) OWNER TO "postgres";


CREATE OR REPLACE FUNCTION "public"."has_role"("_user_id" "uuid", "_role"
"public"."app_role") RETURNS boolean
    LANGUAGE "sql" STABLE SECURITY DEFINER
    SET "search_path" TO 'public'
    AS $$
  SELECT EXISTS (
    SELECT 1
    FROM public.user_roles
    WHERE user_id = _user_id
      AND role = _role
  )
  AND public.is_current_admin_session(_user_id);
$$;


ALTER FUNCTION "public"."has_role"("_user_id" "uuid", "_role"
"public"."app_role") OWNER TO "postgres";


CREATE OR REPLACE FUNCTION "public"."has_role_legacy"("_user_id"
"uuid", "_role" "public"."app_role_legacy") RETURNS boolean
    LANGUAGE "sql" STABLE SECURITY DEFINER
    SET "search_path" TO 'public'
    AS $$
  SELECT EXISTS (
    SELECT 1
    FROM public.user_roles
    WHERE user_id = _user_id
      AND role::text = _role::text
  )
  AND public.is_current_admin_session(_user_id);
$$;
```

```
ALTER FUNCTION "public"."has_role_legacy"("_user_id" "uuid", "_role"
"public"."app_role_legacy") OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."inherit_rescheduled_appointment_email"() RETURNS "trigger"
    LANGUAGE "plpgsql" SECURITY DEFINER
    SET "search_path" TO 'public'
    AS $$
BEGIN
  IF NEW.rescheduled_from_appointment_id IS NOT NULL
     AND NULLIF(btrim(NEW.email), '') IS NULL THEN
    SELECT email
      INTO NEW.email
    FROM public.appointments
    WHERE id = NEW.rescheduled_from_appointment_id;
  END IF;
  RETURN NEW;
END;
$$;


ALTER FUNCTION "public"."inherit_rescheduled_appointment_email"() OWNER
TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."is_crew_available_for_appointment"("p_crew_id" "uuid",
"p_appointment_date" "date", "p_start_time" time without time zone,
"p_duration_minutes" integer, "p_excluded_appointment_id" "uuid" DEFAULT
NULL::"uuid", "p_excluded_continuation_id" "uuid" DEFAULT NULL::"uuid")
RETURNS boolean
    LANGUAGE "sql"
    SET "search_path" TO 'public'
    AS $$
  SELECT p_duration_minutes > 0
    AND EXISTS (
      SELECT 1
      FROM public.crew_members AS crew
      JOIN public.crew_schedules AS schedule ON schedule.crew_id = crew.id
      WHERE crew.id = p_crew_id
        AND crew.is_active = true
        AND crew.is_archived = false
        AND schedule.schedule_date = p_appointment_date
```

```
      AND schedule.is_working = true
      AND schedule.start_time IS NOT NULL
      AND schedule.end_time IS NOT NULL
      AND schedule.start_time <= p_start_time
      AND schedule.end_time >= p_start_time + make_interval(mins =>
p_duration_minutes)
  )
  AND NOT EXISTS (
    SELECT 1
    FROM public.crew_availability_exceptions AS exception
    WHERE exception.crew_id = p_crew_id
      AND exception.start_date <= p_appointment_date
      AND exception.end_date >= p_appointment_date
      AND (
        exception.is_all_day = true
        OR (
          exception.start_time IS NOT NULL
          AND exception.end_time IS NOT NULL
          AND tsrange(
            p_appointment_date + p_start_time,
            p_appointment_date + p_start_time + make_interval(mins =>
p_duration_minutes),
            '[)'
          ) && tsrange(
            p_appointment_date + exception.start_time,
            p_appointment_date + exception.end_time,
            '[)'
          )
        )
      )
  )
  AND NOT EXISTS (
    SELECT 1
    FROM public.appointments AS appointment
    WHERE (p_excluded_appointment_id IS NULL OR appointment.id <>
p_excluded_appointment_id)
      AND appointment.assigned_crew_id = p_crew_id
      AND appointment.appointment_date = p_appointment_date
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
      AND tsrange(
        appointment.appointment_date + appointment.start_time,
        appointment.appointment_date + appointment.start_time
```

```
          + make_interval(mins => appointment.booking_duration_minutes),
        '[)'
      ) && tsrange(
        p_appointment_date + p_start_time,
        p_appointment_date + p_start_time + make_interval(mins =>
p_duration_minutes),
        '[)'
      )
  )
  AND NOT EXISTS (
    SELECT 1
    FROM public.appointment_continuations AS continuation
    JOIN public.appointments AS appointment ON appointment.id =
continuation.appointment_id
    WHERE (p_excluded_continuation_id IS NULL OR continuation.id <>
p_excluded_continuation_id)
      AND continuation.assigned_crew_id = p_crew_id
      AND continuation.appointment_date = p_appointment_date
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
      AND tsrange(
        continuation.appointment_date + continuation.start_time,
        continuation.appointment_date + continuation.start_time
          + make_interval(mins => continuation.booking_duration_minutes),
        '[)'
      ) && tsrange(
        p_appointment_date + p_start_time,
        p_appointment_date + p_start_time + make_interval(mins =>
p_duration_minutes),
        '[)'
      )
  );
$$;


ALTER FUNCTION "public"."is_crew_available_for_appointment"("p_crew_id"
"uuid", "p_appointment_date" "date", "p_start_time" time without time zone,
"p_duration_minutes" integer, "p_excluded_appointment_id" "uuid",
"p_excluded_continuation_id" "uuid") OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."is_current_admin_session"("p_admin_id" "uuid") RETURNS boolean
```

```
    LANGUAGE "sql" STABLE SECURITY DEFINER
    SET "search_path" TO 'public', 'auth'
    AS $$
  SELECT
    p_admin_id IS NOT NULL
    AND p_admin_id = auth.uid()
    AND EXISTS (
      SELECT 1
      FROM public.user_roles AS role
      WHERE role.user_id = p_admin_id
        AND role.role::text = 'admin'
    )
    AND EXISTS (
      SELECT 1
      FROM public.admin_session_leases AS lease
      WHERE lease.admin_id = p_admin_id
        AND lease.session_id = public.current_admin_auth_session_id()
        AND lease.expires_at > now()
    );
$$;


ALTER FUNCTION "public"."is_current_admin_session"("p_admin_id" "uuid")
OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."is_valid_customer_name_part"("p_name" "text") RETURNS boolean
    LANGUAGE "sql" IMMUTABLE STRICT
    SET "search_path" TO 'public'
    AS $_$
  SELECT btrim(p_name) ~ '^[[:alpha:]]+(([[:space:]]+|[-''])[[:alpha:]]+)*$';
$_$;


ALTER FUNCTION "public"."is_valid_customer_name_part"("p_name" "text")
OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."manage_appointment_service_progress"("p_appointment_id" "uuid",
"p_action" "text") RETURNS "void"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
```

```
DECLARE
  v_appointment public.appointments%ROWTYPE;
  v_action text := lower(btrim(COALESCE(p_action, '')));
  v_snooze_interval interval := interval '5 minutes';
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Only administrators can update service progress.';
  END IF;

  IF v_action NOT IN (
    'start', 'snooze_arrival', 'no_show', 'complete', 'snooze_completion'
  ) THEN
    RAISE EXCEPTION 'Choose a valid service progress action.';
  END IF;

  SELECT appointment.*
    INTO v_appointment
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
  FOR UPDATE;

  IF NOT FOUND THEN
    RAISE EXCEPTION 'This appointment is no longer available.';
  END IF;

  IF v_action = 'start' THEN
    IF v_appointment.status NOT IN ('pending', 'confirmed')
       OR v_appointment.service_started_at IS NOT NULL THEN
      RAISE EXCEPTION 'This appointment cannot be started.';
    END IF;

    UPDATE public.appointments
    SET status = 'in_progress',
        service_started_at = COALESCE(service_started_at, now()),
        service_ended_at = NULL,
        arrival_notification_snoozed_until = NULL
    WHERE id = v_appointment.id;

    UPDATE public.notifications
    SET is_read = true
    WHERE appointment_id = v_appointment.id
      AND type = 'service_arrival'
      AND is_read = false;
```

```
  ELSIF v_action = 'snooze_arrival' THEN
    IF v_appointment.status <> 'confirmed' OR v_appointment.service_started_at
IS NOT NULL THEN
      RAISE EXCEPTION 'This arrival reminder is no longer available.';
    END IF;
    IF v_appointment.arrival_notification_snooze_count >= 3 THEN
      RAISE EXCEPTION 'The maximum of three arrival snoozes has been
reached.';
    END IF;

    UPDATE public.appointments
    SET arrival_notification_snooze_count = arrival_notification_snooze_count +
1,
        arrival_notification_snoozed_until = now() + v_snooze_interval
    WHERE id = v_appointment.id;

    UPDATE public.notifications
    SET is_read = true
    WHERE appointment_id = v_appointment.id
      AND type = 'service_arrival'
      AND is_read = false;

  ELSIF v_action = 'no_show' THEN
    IF v_appointment.status <> 'confirmed' OR v_appointment.service_started_at
IS NOT NULL THEN
      RAISE EXCEPTION 'This appointment cannot be marked as no-show.';
    END IF;
    IF v_appointment.arrival_notification_snooze_count < 3 THEN
      RAISE EXCEPTION 'The appointment can be marked as no-show after three
arrival snoozes.';
    END IF;

    UPDATE public.appointments
    SET status = 'no_show',
        arrival_notification_snoozed_until = NULL
    WHERE id = v_appointment.id;

    UPDATE public.notifications
    SET is_read = true
    WHERE appointment_id = v_appointment.id
      AND type = 'service_arrival'
      AND is_read = false;

  ELSIF v_action = 'complete' THEN
    IF v_appointment.status <> 'in_progress'
```

```
      OR v_appointment.service_started_at IS NULL
      OR v_appointment.service_ended_at IS NOT NULL THEN
     RAISE EXCEPTION 'This service cannot be marked as completed.';
    END IF;

    UPDATE public.appointments
    SET status = 'completed',
        service_ended_at = COALESCE(service_ended_at, now()),
        completion_notification_snoozed_until = NULL
    WHERE id = v_appointment.id;

    UPDATE public.notifications
    SET is_read = true
    WHERE appointment_id = v_appointment.id
      AND type = 'service_completion'
      AND is_read = false;

  ELSE
    IF v_appointment.status <> 'in_progress'
       OR v_appointment.service_started_at IS NULL
       OR v_appointment.service_ended_at IS NOT NULL THEN
      RAISE EXCEPTION 'This completion reminder is no longer available.';
    END IF;

    UPDATE public.appointments
    SET completion_notification_snoozed_until = now() + v_snooze_interval
    WHERE id = v_appointment.id;

    UPDATE public.notifications
    SET is_read = true
    WHERE appointment_id = v_appointment.id
      AND type = 'service_completion'
      AND is_read = false;
  END IF;
END;
$$;


ALTER FUNCTION
"public"."manage_appointment_service_progress"("p_appointment_id" "uuid",
"p_action" "text") OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."normalize_philippine_mobile"("p_phone" "text") RETURNS "text"
```

```
    LANGUAGE "sql" IMMUTABLE STRICT
    SET "search_path" TO 'public'
    AS $_$
  SELECT CASE
    WHEN value.digits ~ '^09[0-9]{9}$' THEN value.digits
    WHEN value.digits ~ '^639[0-9]{9}$' THEN '0' || substring(value.digits FROM 3)
    ELSE NULL
  END
  FROM (SELECT regexp_replace(p_phone, '[^0-9]', '', 'g') AS digits) AS value;
$_$;


ALTER FUNCTION "public"."normalize_philippine_mobile"("p_phone" "text")
OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."normalize_philippine_mobile_before_write"() RETURNS "trigger"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
BEGIN
  IF NEW.phone IS NULL THEN
    RETURN NEW;
  END IF;

  NEW.phone := public.normalize_philippine_mobile(NEW.phone);
  IF NEW.phone IS NULL THEN
    RAISE EXCEPTION 'Enter a valid Philippine mobile number.' USING
ERRCODE = '22023';
  END IF;

  RETURN NEW;
END;
$$;


ALTER FUNCTION "public"."normalize_philippine_mobile_before_write"()
OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."normalize_plate_number"("p_plate" "text") RETURNS "text"
    LANGUAGE "sql" IMMUTABLE
    SET "search_path" TO 'public'
```

```
    AS $$
  SELECT upper(regexp_replace(COALESCE(p_plate, ''), '[^A-Za-z0-9]', '', 'g'));
$$;


ALTER FUNCTION "public"."normalize_plate_number"("p_plate" "text") OWNER
TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."notify_customer_no_show_threshold"() RETURNS "trigger"
    LANGUAGE "plpgsql" SECURITY DEFINER
    SET "search_path" TO 'public'
    AS $$
DECLARE
  v_no_show_count integer;
BEGIN
  -- This supports both the normal admin workflow and any trusted import that
  -- creates an already-recorded no-show. Repeated updates to a no-show row
do
  -- not create duplicate alerts.
  IF NEW.status <> 'no_show'
     OR (TG_OP = 'UPDATE' AND OLD.status = 'no_show') THEN
    RETURN NEW;
  END IF;

  -- Concurrent status changes for the same customer must see one another
  -- before deciding whether this is the threshold-crossing no-show.
  PERFORM pg_advisory_xact_lock(
    hashtext('customer-no-show-threshold:' || COALESCE(NEW.phone, ''))
  );

  SELECT count(*)::integer
  INTO v_no_show_count
  FROM public.appointments AS appointment
  WHERE appointment.phone = NEW.phone
    AND appointment.status = 'no_show';

  -- Notify once when the customer exceeds two recorded no-shows. If the count
  -- later falls below the threshold and reaches it again, the new crossing is
  -- correctly reported again.
  IF v_no_show_count = 3 THEN
    INSERT INTO public.notifications (type, title, message, appointment_id)
    VALUES (
      'customer_no_show_threshold',
```

```
      'Customer no-show threshold reached',
      COALESCE(NULLIF(btrim(NEW.customer_name), ''), NEW.phone)
        || ' (' || NEW.phone || ') now has ' || v_no_show_count
        || ' recorded no-shows. Review the customer''s booking history before
accepting future work.',
      NEW.id
    );
  END IF;

  RETURN NEW;
END;
$$;


ALTER FUNCTION "public"."notify_customer_no_show_threshold"() OWNER TO
"postgres";


CREATE OR REPLACE FUNCTION
"public"."prevent_duplicate_active_customer_service_appointments"()
RETURNS "trigger"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
DECLARE
  v_appointment_id uuid;
  v_phone text;
  v_appointment_date date;
  v_service_name text;
  v_start_time time;
BEGIN
  v_appointment_id := CASE
    WHEN TG_TABLE_NAME = 'appointments' THEN NEW.id
    ELSE NEW.appointment_id
  END;

  SELECT appointment.phone, appointment.appointment_date
  INTO v_phone, v_appointment_date
  FROM public.appointments AS appointment
  WHERE appointment.id = v_appointment_id
    AND appointment.is_archived = false
    AND appointment.rescheduled_to_appointment_id IS NULL
    AND appointment.status NOT IN ('completed', 'cancelled', 'rejected');

  IF NOT FOUND THEN
```

```
  RETURN NEW;
 END IF;

 SELECT current_service.service_name, other_appointment.start_time
 INTO v_service_name, v_start_time
 FROM public.appointment_services AS current_service
 JOIN public.appointments AS other_appointment
   ON other_appointment.phone = v_phone
  AND other_appointment.appointment_date = v_appointment_date
  AND other_appointment.id <> v_appointment_id
  AND other_appointment.is_archived = false
  AND other_appointment.rescheduled_to_appointment_id IS NULL
  AND other_appointment.status NOT IN ('completed', 'cancelled', 'rejected')
 JOIN public.appointment_services AS other_service
   ON other_service.appointment_id = other_appointment.id
  AND (
    other_service.service_id = current_service.service_id
    OR (
      other_service.service_id IS NULL
      AND current_service.service_id IS NULL
      AND other_service.service_name = current_service.service_name
    )
  )
 WHERE current_service.appointment_id = v_appointment_id
 LIMIT 1;

 IF FOUND THEN
   RAISE EXCEPTION
     'You already have an appointment for "%" on % at %.',
     v_service_name,
     to_char(v_appointment_date, 'FMDay, FMMonth FMDD, YYYY'),
     to_char(v_start_time, 'FMHH12:MI AM')
     USING ERRCODE = '23505';
 END IF;

 RETURN NEW;
END;
$$;


ALTER FUNCTION
"public"."prevent_duplicate_active_customer_service_appointments"() OWNER
TO "postgres";
```

```
CREATE OR REPLACE FUNCTION
"public"."prevent_duplicate_active_customer_service_from_appointment"()
RETURNS "trigger"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
BEGIN
  PERFORM
public.assert_no_duplicate_active_customer_service_appointment(NEW.id);
  RETURN NEW;
END;
$$;


ALTER FUNCTION
"public"."prevent_duplicate_active_customer_service_from_appointment"()
OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."prevent_duplicate_active_customer_service_from_service"() RETURNS
"trigger"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
BEGIN
  PERFORM public.assert_no_duplicate_active_customer_service_appointment(
NEW.appointment_id);
  RETURN NEW;
END;
$$;


ALTER FUNCTION
"public"."prevent_duplicate_active_customer_service_from_service"() OWNER
TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."protect_completed_appointments"() RETURNS "trigger"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
BEGIN
  IF TG_OP = 'DELETE' THEN
```

```
    IF OLD.status = 'completed' THEN
      RAISE EXCEPTION 'Completed appointments are kept for financial records
and cannot be deleted.';
    END IF;
    RETURN OLD;
  END IF;

  IF NEW.is_archived
     AND NOT OLD.is_archived
     AND (OLD.status = 'completed' OR NEW.status = 'completed') THEN
    RAISE EXCEPTION 'Completed appointments are kept for financial records
and cannot be archived.';
  END IF;
  RETURN NEW;
END;
$$;


ALTER FUNCTION "public"."protect_completed_appointments"() OWNER TO
"postgres";


CREATE OR REPLACE FUNCTION
"public"."purge_expired_admin_activity_logs"() RETURNS "void"
    LANGUAGE "plpgsql" SECURITY DEFINER
    SET "search_path" TO 'public', 'auth'
    AS $$
BEGIN
  -- The function is also invoked by a trigger and a scheduled database job,
  -- where auth.uid() is null. Direct signed-in callers must be administrators.
  IF auth.uid() IS NOT NULL AND NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;

  DELETE FROM public.admin_activity_logs
  WHERE created_at < now() - interval '50 days';
END;
$$;


ALTER FUNCTION "public"."purge_expired_admin_activity_logs"() OWNER TO
"postgres";


CREATE OR REPLACE FUNCTION
```

```
"public"."purge_expired_archived_records"() RETURNS "void"
    LANGUAGE "plpgsql" SECURITY DEFINER
    SET "search_path" TO 'public'
    AS $$
BEGIN
  DELETE FROM public.appointments
  WHERE is_archived = true
    AND status <> 'completed'
    AND archived_at < now() - interval '30 days';

  DELETE FROM public.services
  WHERE is_archived = true
    AND archived_at < now() - interval '30 days';

  DELETE FROM public.products
  WHERE is_archived = true
    AND archived_at < now() - interval '30 days';

  DELETE FROM public.motorcycle_catalog
  WHERE is_archived = true
    AND archived_at < now() - interval '30 days';

  DELETE FROM public.crew_members
  WHERE is_archived = true
    AND archived_at < now() - interval '30 days';

  DELETE FROM public.schedule_blocks
  WHERE is_active = false
    AND archived_at < now() - interval '30 days';

  DELETE FROM public.blocked_numbers
  WHERE is_archived = true
    AND archived_at < now() - interval '30 days';
END;
$$;


ALTER FUNCTION "public"."purge_expired_archived_records"() OWNER TO
"postgres";


CREATE OR REPLACE FUNCTION
"public"."queue_appointment_completion_notifications"() RETURNS "trigger"
    LANGUAGE "plpgsql" SECURITY DEFINER
    SET "search_path" TO 'public'
```

```
    AS $$
BEGIN
  IF NEW.status <> 'completed' OR OLD.status IS NOT DISTINCT FROM
'completed' THEN
    RETURN NEW;
  END IF;

  IF NULLIF(btrim(NEW.email), '') IS NOT NULL THEN
    INSERT INTO public.appointment_completion_notifications (appointment_id,
channel, recipient)
    VALUES (NEW.id, 'email', lower(btrim(NEW.email)))
    ON CONFLICT (appointment_id, channel) DO NOTHING;
  END IF;

  IF NULLIF(btrim(NEW.phone), '') IS NOT NULL THEN
    INSERT INTO public.appointment_completion_notifications (appointment_id,
channel, recipient)
    VALUES (NEW.id, 'sms', btrim(NEW.phone))
    ON CONFLICT (appointment_id, channel) DO NOTHING;
  END IF;
  RETURN NEW;
END;
$$;


ALTER FUNCTION "public"."queue_appointment_completion_notifications"()
OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."queue_appointment_confirmation_email"() RETURNS "trigger"
    LANGUAGE "plpgsql" SECURITY DEFINER
    SET "search_path" TO 'public'
    AS $$
BEGIN
  -- Pending booking requests do not send customer email. A row is queued only
  -- when an appointment is actually approved into the confirmed state.
  IF NEW.status <> 'confirmed' OR NULLIF(btrim(NEW.email), '') IS NULL THEN
    RETURN NEW;
  END IF;
  IF TG_OP = 'UPDATE' AND OLD.status IS NOT DISTINCT FROM 'confirmed'
THEN
    RETURN NEW;
  END IF;
```

```
  INSERT INTO public.appointment_confirmation_emails (appointment_id,
recipient_email)
  VALUES (NEW.id, lower(btrim(NEW.email)))
  ON CONFLICT (appointment_id) DO NOTHING;
  RETURN NEW;
END;
$$;


ALTER FUNCTION "public"."queue_appointment_confirmation_email"() OWNER
TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."queue_appointment_confirmation_sms"() RETURNS "trigger"
    LANGUAGE "plpgsql" SECURITY DEFINER
    SET "search_path" TO 'public'
    AS $$
BEGIN
  -- A pending or rejected booking must never notify the customer. For a
  -- reschedule, the replacement appointment is inserted as confirmed, which
  -- queues its own notification after the approval transaction succeeds.
  IF NEW.status <> 'confirmed' OR NULLIF(btrim(NEW.phone), '') IS NULL THEN
    RETURN NEW;
  END IF;
  IF TG_OP = 'UPDATE' AND OLD.status IS NOT DISTINCT FROM 'confirmed'
THEN
    RETURN NEW;
  END IF;

  INSERT INTO public.appointment_confirmation_sms (appointment_id,
recipient_phone)
  VALUES (NEW.id, btrim(NEW.phone))
  ON CONFLICT (appointment_id) DO NOTHING;
  RETURN NEW;
END;
$$;


ALTER FUNCTION "public"."queue_appointment_confirmation_sms"() OWNER
TO "postgres";


CREATE OR REPLACE FUNCTION "public"."record_admin_activity"()
RETURNS "trigger"
```

```
    LANGUAGE "plpgsql" SECURITY DEFINER
    SET "search_path" TO 'public', 'auth'
    AS $_$
DECLARE
  v_actor_id uuid := auth.uid();
  v_claims text := current_setting('request.jwt.claims', true);
  v_actor_email text;
  v_old jsonb := CASE WHEN TG_OP IN ('UPDATE', 'DELETE') THEN
to_jsonb(OLD) ELSE '{}'::jsonb END;
  v_new jsonb := CASE WHEN TG_OP IN ('INSERT', 'UPDATE') THEN
to_jsonb(NEW) ELSE '{}'::jsonb END;
  v_record jsonb;
  v_action text;
  v_target text;
  v_record_id uuid;
  v_changed_fields text[] := ARRAY[]::text[];
BEGIN
  -- Public/server-side activity (such as a customer booking) is not an admin
action.
  IF v_actor_id IS NULL OR NOT public.has_role(v_actor_id, 'admin') THEN
    IF TG_OP = 'DELETE' THEN
      RETURN OLD;
    END IF;
    RETURN NEW;
  END IF;

  IF v_claims IS NOT NULL AND v_claims <> '' THEN
    v_actor_email := nullif(v_claims::jsonb ->> 'email', '');
  END IF;

  IF TG_OP = 'UPDATE' THEN
    SELECT COALESCE(array_agg(changed.key ORDER BY changed.key),
ARRAY[]::text[])
    INTO v_changed_fields
    FROM jsonb_object_keys(v_new) AS changed(key)
    WHERE changed.key NOT IN ('created_at', 'updated_at')
      AND v_old -> changed.key IS DISTINCT FROM v_new -> changed.key;

    -- Do not record a no-op update or an update that only changed a timestamp.
    IF cardinality(v_changed_fields) = 0 THEN
      RETURN NEW;
    END IF;

    IF v_new ->> 'is_archived' IS DISTINCT FROM v_old ->> 'is_archived' THEN
      v_action := CASE WHEN (v_new ->> 'is_archived')::boolean THEN 'archived'
```

```
ELSE 'restored' END;
    ELSIF v_new ->> 'is_active' IS DISTINCT FROM v_old ->> 'is_active' THEN
      v_action := CASE WHEN (v_new ->> 'is_active')::boolean THEN 'activated'
ELSE 'deactivated' END;
    ELSIF v_new ->> 'status' IS DISTINCT FROM v_old ->> 'status' THEN
      v_action := 'status changed';
    ELSE
      v_action := 'updated';
    END IF;
  ELSIF TG_OP = 'INSERT' THEN
    v_action := 'created';
  ELSE
    v_action := 'deleted';
  END IF;

  v_record := CASE WHEN TG_OP = 'DELETE' THEN v_old ELSE v_new END;
  IF COALESCE(v_record ->> 'id', '') ~*
'^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$' THEN
    v_record_id := (v_record ->> 'id')::uuid;
  END IF;
  v_target := COALESCE(
    nullif(v_record ->> 'reference_code', ''),
    nullif(v_record ->> 'name', ''),
    nullif(v_record ->> 'phone', ''),
    nullif(v_record ->> 'service_name', ''),
    nullif(v_record ->> 'block_date', ''),
    nullif(v_record ->> 'id', ''),
    'Record'
  );

  PERFORM public.purge_expired_admin_activity_logs();

  INSERT INTO public.admin_activity_logs (
    actor_id,
    actor_email,
    action,
    resource_type,
    target_label,
    summary,
    changed_fields,
    record_id,
    ip_address
  )
  VALUES (
    v_actor_id,
```

```
    v_actor_email,
    v_action,
    initcap(replace(TG_TABLE_NAME, '_', ' ')),
    v_target,
    format('%s %s: %s', initcap(replace(TG_TABLE_NAME, '_', ' ')), v_action,
v_target),
    v_changed_fields,
    v_record_id,
    public.admin_activity_request_ip()
  );

  IF TG_OP = 'DELETE' THEN
    RETURN OLD;
  END IF;
  RETURN NEW;
END;
$_$;


ALTER FUNCTION "public"."record_admin_activity"() OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."record_admin_activity_event"("p_action" "text", "p_resource_type"
"text", "p_target_label" "text", "p_summary" "text", "p_changed_fields" "text"[]
DEFAULT ARRAY[]::"text"[]) RETURNS "uuid"
    LANGUAGE "plpgsql" SECURITY DEFINER
    SET "search_path" TO 'public', 'auth'
    AS $$
DECLARE
  v_actor_id uuid := auth.uid();
  v_actor_email text;
  v_claims text := current_setting('request.jwt.claims', true);
  v_log_id uuid;
BEGIN
  IF v_actor_id IS NULL OR NOT public.has_role(v_actor_id, 'admin') THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;

  IF v_claims IS NOT NULL AND v_claims <> '' THEN
    v_actor_email := nullif(v_claims::jsonb ->> 'email', '');
  END IF;

  IF btrim(p_action) = '' OR btrim(p_resource_type) = '' OR btrim(p_target_label) =
'' OR btrim(p_summary) = '' THEN
```

```
    RAISE EXCEPTION 'Activity event details are required';
  END IF;

  PERFORM public.purge_expired_admin_activity_logs();

  INSERT INTO public.admin_activity_logs (
    actor_id,
    actor_email,
    action,
    resource_type,
    target_label,
    summary,
    changed_fields,
    ip_address
  )
  VALUES (
    v_actor_id,
    v_actor_email,
    left(btrim(p_action), 80),
    left(btrim(p_resource_type), 120),
    left(btrim(p_target_label), 180),
    left(btrim(p_summary), 500),
    COALESCE(p_changed_fields, ARRAY[]::text[]),
    public.admin_activity_request_ip()
  )
  RETURNING id INTO v_log_id;

  RETURN v_log_id;
END;
$$;


ALTER FUNCTION "public"."record_admin_activity_event"("p_action" "text",
"p_resource_type" "text", "p_target_label" "text", "p_summary" "text",
"p_changed_fields" "text"[]) OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."refresh_customer_service_booking_alert"("p_customer_phone" "text",
"p_service_id" "uuid", "p_service_name" "text", "p_preferred_appointment_id"
"uuid", "p_customer_name" "text") RETURNS "void"
    LANGUAGE "plpgsql" SECURITY DEFINER
    SET "search_path" TO 'public'
    AS $$
DECLARE
```

```
  v_service_key text;
  v_service_label text;
  v_customer_label text;
  v_active_appointment_ids uuid[];
  v_active_booking_count integer;
  v_distinct_booking_date_count integer;
  v_schedule_details text;
  v_notification_appointment_id uuid;
  v_alert_cycle integer;
  v_threshold_was_active boolean;
  v_notification_id uuid;
BEGIN
  IF NULLIF(btrim(p_customer_phone), '') IS NULL
     OR (p_service_id IS NULL AND NULLIF(btrim(p_service_name), '') IS NULL)
THEN
    RETURN;
  END IF;

  v_service_key := COALESCE(
    'id:' || p_service_id::text,
    'name:' || lower(btrim(p_service_name))
  );
  v_service_label := COALESCE(
    NULLIF(btrim(p_service_name), ''),
    (SELECT service.name FROM public.services AS service WHERE service.id
= p_service_id),
    'this service'
  );

  PERFORM pg_advisory_xact_lock(
    hashtext('customer-service-booking-alert:' || btrim(p_customer_phone))
  );

  WITH active_bookings AS (
    SELECT DISTINCT
      appointment.id,
      appointment.appointment_date,
      appointment.start_time,
      appointment.customer_name
    FROM public.appointments AS appointment
    JOIN public.appointment_services AS appointment_service
      ON appointment_service.appointment_id = appointment.id
    WHERE appointment.phone = btrim(p_customer_phone)
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
```

```
    AND appointment.status NOT IN ('completed', 'cancelled', 'rejected')
    AND (
      appointment_service.service_id = p_service_id
      OR (
        p_service_id IS NULL
        AND appointment_service.service_id IS NULL
        AND appointment_service.service_name = p_service_name
      )
    )
  )
  SELECT
    COALESCE(array_agg(appointment.id ORDER BY appointment.id),
ARRAY[]::uuid[]),
    count(*)::integer,
    count(DISTINCT appointment.appointment_date)::integer,
    string_agg(
      to_char(appointment.appointment_date, 'Mon FMDD, YYYY')
        || ' at ' || to_char(appointment.start_time, 'FMHH12:MI AM'),
      '; ' ORDER BY appointment.appointment_date, appointment.start_time,
appointment.id
    ),
    COALESCE(
      max(appointment.customer_name) FILTER (WHERE appointment.id =
p_preferred_appointment_id),
      max(appointment.customer_name),
      NULLIF(btrim(p_customer_name), ''),
      btrim(p_customer_phone)
    ),
    COALESCE(
      (
        array_agg(appointment.id ORDER BY appointment.appointment_date,
appointment.start_time, appointment.id)
          FILTER (WHERE appointment.id = p_preferred_appointment_id)
      )[1],
      (
        array_agg(appointment.id ORDER BY appointment.appointment_date,
appointment.start_time, appointment.id)
      )[1]
    )
  INTO
    v_active_appointment_ids,
    v_active_booking_count,
    v_distinct_booking_date_count,
    v_schedule_details,
    v_customer_label,
```

```
    v_notification_appointment_id
  FROM active_bookings AS appointment;

  INSERT INTO public.customer_service_booking_alert_state (customer_phone,
service_key)
  VALUES (btrim(p_customer_phone), v_service_key)
  ON CONFLICT (customer_phone, service_key) DO NOTHING;

  SELECT state.is_threshold_active, state.alert_cycle
  INTO v_threshold_was_active, v_alert_cycle
  FROM public.customer_service_booking_alert_state AS state
  WHERE state.customer_phone = btrim(p_customer_phone)
    AND state.service_key = v_service_key
  FOR UPDATE;

  IF v_active_booking_count < 2 OR v_distinct_booking_date_count < 2 THEN
    UPDATE public.customer_service_booking_alert_state
    SET is_threshold_active = false,
        updated_at = now()
    WHERE customer_phone = btrim(p_customer_phone)
      AND service_key = v_service_key;
    RETURN;
  END IF;

  IF NOT v_threshold_was_active THEN
    v_alert_cycle := v_alert_cycle + 1;

    UPDATE public.customer_service_booking_alert_state
    SET is_threshold_active = true,
        alert_cycle = v_alert_cycle,
        updated_at = now()
    WHERE customer_phone = btrim(p_customer_phone)
      AND service_key = v_service_key;
  END IF;

  INSERT INTO public.customer_service_booking_alerts (
    customer_phone,
    service_key,
    alert_cycle,
    active_appointment_ids
  )
  VALUES (
    btrim(p_customer_phone),
    v_service_key,
    v_alert_cycle,
```

```
    v_active_appointment_ids
  )
  ON CONFLICT (customer_phone, service_key, alert_cycle,
active_appointment_ids) DO NOTHING
  RETURNING id INTO v_notification_id;

  IF v_notification_id IS NULL THEN
    RETURN;
  END IF;

  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES (
    'multiple_active_service_bookings',
    v_active_booking_count || ' active ' || v_service_label || ' bookings for ' ||
v_customer_label,
    v_customer_label || ' (' || btrim(p_customer_phone) || ') has '
      || v_active_booking_count || ' active bookings for ' || v_service_label || ': '
      || v_schedule_details || '.',
    v_notification_appointment_id
  )
  RETURNING id INTO v_notification_id;

  UPDATE public.customer_service_booking_alerts
  SET notification_id = v_notification_id
  WHERE customer_phone = btrim(p_customer_phone)
    AND service_key = v_service_key
    AND alert_cycle = v_alert_cycle
    AND active_appointment_ids = v_active_appointment_ids;
END;
$$;


ALTER FUNCTION
"public"."refresh_customer_service_booking_alert"("p_customer_phone" "text",
"p_service_id" "uuid", "p_service_name" "text", "p_preferred_appointment_id"
"uuid", "p_customer_name" "text") OWNER TO "postgres";


CREATE OR REPLACE FUNCTION "public"."release_admin_session"()
RETURNS boolean
    LANGUAGE "plpgsql" SECURITY DEFINER
    SET "search_path" TO 'public', 'auth'
    AS $$
DECLARE
  v_admin_id uuid := auth.uid();
```

```
  v_session_id uuid := public.current_admin_auth_session_id();
BEGIN
  IF v_admin_id IS NULL OR v_session_id IS NULL THEN
    RETURN false;
  END IF;

  DELETE FROM public.admin_session_handover_tokens
  WHERE admin_id = v_admin_id
    AND session_id = v_session_id;

  DELETE FROM public.admin_session_leases
  WHERE admin_id = v_admin_id
    AND session_id = v_session_id;

  RETURN FOUND;
END;
$$;


ALTER FUNCTION "public"."release_admin_session"() OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."release_pre_service_product_reservations"() RETURNS "trigger"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
BEGIN
  IF OLD.status IN ('pending', 'confirmed')
     AND NEW.status IN ('cancelled', 'rejected', 'rescheduled', 'no_show')
     AND NEW.status IS DISTINCT FROM OLD.status THEN
    PERFORM 1
    FROM public.products AS product
    JOIN public.appointment_products AS appointment_product
      ON appointment_product.product_id = product.id
    WHERE appointment_product.appointment_id = OLD.id
    ORDER BY product.id
    FOR UPDATE;

    UPDATE public.products AS product
    SET stock_quantity = product.stock_quantity + appointment_product.quantity
    FROM public.appointment_products AS appointment_product
    WHERE appointment_product.appointment_id = OLD.id
      AND appointment_product.product_id = product.id;
```

```
    DELETE FROM public.appointment_products
    WHERE appointment_id = OLD.id;
  END IF;

  RETURN NEW;
END;
$$;


ALTER FUNCTION "public"."release_pre_service_product_reservations"()
OWNER TO "postgres";


CREATE OR REPLACE FUNCTION "public"."renew_admin_session"()
RETURNS boolean
    LANGUAGE "plpgsql" SECURITY DEFINER
    SET "search_path" TO 'public', 'auth'
    AS $$
DECLARE
  v_admin_id uuid := auth.uid();
  v_session_id uuid := public.current_admin_auth_session_id();
BEGIN
  IF v_admin_id IS NULL OR v_session_id IS NULL THEN
    RETURN false;
  END IF;

  UPDATE public.admin_session_leases AS lease
  SET expires_at = now() + interval '3 minutes',
      last_seen_at = now()
  WHERE lease.admin_id = v_admin_id
    AND lease.session_id = v_session_id
    AND lease.expires_at > now();

  RETURN FOUND;
END;
$$;


ALTER FUNCTION "public"."renew_admin_session"() OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."resolve_appointment_service_progress_notifications"() RETURNS
"trigger"
    LANGUAGE "plpgsql" SECURITY DEFINER
```

```
    SET "search_path" TO 'public'
    AS $$
DECLARE
  v_now timestamptz := now();
  v_service_duration_minutes integer := 0;
  v_scheduled_start timestamptz;
  v_expected_completion timestamptz;
BEGIN
  SELECT COALESCE(sum(service.duration_minutes), 0)::integer
    INTO v_service_duration_minutes
  FROM public.appointment_services AS service
  WHERE service.appointment_id = NEW.id;

  v_scheduled_start := (NEW.appointment_date + NEW.start_time) AT TIME
ZONE 'Asia/Manila';
  v_expected_completion := CASE
    WHEN NEW.service_started_at IS NOT NULL
      THEN NEW.service_started_at + make_interval(mins =>
v_service_duration_minutes)
    ELSE NULL
  END;

  UPDATE public.notifications AS notification
  SET is_read = true
  WHERE notification.appointment_id = NEW.id
    AND notification.is_read = false
    AND notification.type IN ('service_arrival', 'service_completion')
    AND (
      NEW.is_archived = true
      OR NEW.rescheduled_to_appointment_id IS NOT NULL
      OR (
        notification.type = 'service_arrival'
        AND (
          NEW.status <> 'confirmed'
          OR NEW.service_started_at IS NOT NULL
          OR v_scheduled_start > v_now
          OR (
            NEW.arrival_notification_snoozed_until IS NULL
            AND (
              v_service_duration_minutes <= 0
              OR v_scheduled_start + make_interval(mins =>
v_service_duration_minutes) <= v_now
            )
          )
        )
```

```
      )
      OR (
        notification.type = 'service_completion'
        AND (
          NEW.status <> 'in_progress'
          OR NEW.service_started_at IS NULL
          OR NEW.service_ended_at IS NOT NULL
          OR v_service_duration_minutes <= 0
          OR v_expected_completion > v_now
        )
      )
    );

  RETURN NEW;
END;
$$;


ALTER FUNCTION
"public"."resolve_appointment_service_progress_notifications"() OWNER TO
"postgres";


CREATE OR REPLACE FUNCTION
"public"."review_pending_appointment"("p_appointment_id" "uuid", "p_decision"
"text") RETURNS "void"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
DECLARE
  v_appointment public.appointments%ROWTYPE;
  v_assigned_crew_id uuid;
  v_duration_minutes integer;
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Only administrators can review appointments.';
  END IF;

  IF p_decision IS NULL OR lower(btrim(p_decision)) NOT IN ('confirmed',
'rejected') THEN
    RAISE EXCEPTION 'Choose either confirmed or rejected.';
  END IF;

  -- Lock the row so two notification actions cannot review the same booking.
  SELECT appointment.*
```

```
    INTO v_appointment
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
  FOR UPDATE;

  IF NOT FOUND THEN
    RAISE EXCEPTION 'This appointment is no longer available.';
  END IF;

  IF v_appointment.status <> 'pending' THEN
    RAISE EXCEPTION 'This appointment has already been reviewed.';
  END IF;

  IF lower(btrim(p_decision)) = 'rejected' THEN
    UPDATE public.appointments
    SET status = 'rejected',
        assigned_crew_id = NULL
    WHERE id = v_appointment.id;
    RETURN;
  END IF;

  v_duration_minutes := COALESCE(v_appointment.booking_duration_minutes,
75);

  -- Prefer the crew member reserved during booking, but fall back to another
  -- currently available member when schedules or availability have changed.
  SELECT crew.id
    INTO v_assigned_crew_id
  FROM public.crew_members AS crew
  WHERE crew.is_active = true
    AND crew.is_archived = false
    AND EXISTS (
      SELECT 1
      FROM public.crew_schedules AS schedule
      WHERE schedule.crew_id = crew.id
        AND schedule.schedule_date = v_appointment.appointment_date
        AND schedule.is_working = true
        AND schedule.start_time IS NOT NULL
        AND schedule.end_time IS NOT NULL
        AND schedule.start_time <= v_appointment.start_time
        AND schedule.end_time >= v_appointment.start_time + make_interval(mins
=> v_duration_minutes)
    )
    AND NOT EXISTS (
```

```
      SELECT 1
      FROM public.crew_availability_exceptions AS exception
      WHERE exception.crew_id = crew.id
        AND exception.start_date <= v_appointment.appointment_date
        AND exception.end_date >= v_appointment.appointment_date
        AND (
          exception.is_all_day = true
          OR (
            exception.start_time IS NOT NULL
            AND exception.end_time IS NOT NULL
            AND tsrange(
              v_appointment.appointment_date + v_appointment.start_time,
              v_appointment.appointment_date + v_appointment.start_time
                + make_interval(mins => v_duration_minutes),
              '[)'
            ) && tsrange(
              v_appointment.appointment_date + exception.start_time,
              v_appointment.appointment_date + exception.end_time,
              '[)'
            )
          )
        )
    )
    AND NOT EXISTS (
      SELECT 1
      FROM public.appointments AS other
      WHERE other.id <> v_appointment.id
        AND other.assigned_crew_id = crew.id
        AND other.appointment_date = v_appointment.appointment_date
        AND other.is_archived = false
        AND other.rescheduled_to_appointment_id IS NULL
        AND other.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
        AND tsrange(
          other.appointment_date + other.start_time,
          other.appointment_date + other.start_time
            + make_interval(mins => COALESCE(other.booking_duration_minutes,
75)),
          '[)'
        ) && tsrange(
          v_appointment.appointment_date + v_appointment.start_time,
          v_appointment.appointment_date + v_appointment.start_time
            + make_interval(mins => v_duration_minutes),
          '[)'
        )
    )
```

```
  ORDER BY (crew.id = v_appointment.assigned_crew_id) DESC,
    public.crew_daily_workload(crew.id, v_appointment.appointment_date,
v_appointment.id),
    crew.name,
    crew.id
  LIMIT 1;

  IF v_assigned_crew_id IS NULL THEN
    RAISE EXCEPTION 'No crew member is available for the selected
appointment period.';
  END IF;

  UPDATE public.appointments
  SET status = 'confirmed',
      assigned_crew_id = v_assigned_crew_id
  WHERE id = v_appointment.id;
END;
$$;


ALTER FUNCTION "public"."review_pending_appointment"("p_appointment_id"
"uuid", "p_decision" "text") OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."review_reschedule_request"("p_appointment_id" "uuid", "p_decision"
"text") RETURNS "void"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $_$
DECLARE
  v_appointment public.appointments%ROWTYPE;
  v_assigned_crew_id uuid;
  v_duration_minutes integer;
  v_new_appointment_id uuid;
  v_new_reference_code text;
  v_reason text;
  v_original_shop_notes text;
  v_new_shop_notes text;
  v_reschedule_number integer;
  v_slot_capacity integer;
  v_overlapping_appointments integer;
  v_start_minutes integer;
  v_opening_time time;
  v_closing_time time;
```

```
  v_opening_minutes integer;
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Only administrators can review reschedule requests.';
  END IF;
  IF lower(btrim(COALESCE(p_decision, ''))) NOT IN ('confirmed', 'rejected')
THEN
    RAISE EXCEPTION 'Choose either confirmed or rejected.';
  END IF;

  SELECT appointment.*
    INTO v_appointment
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
  FOR UPDATE;

  IF NOT FOUND OR v_appointment.pending_reschedule_request_id IS NULL
THEN
    RAISE EXCEPTION 'This reschedule request is no longer available.';
  END IF;

  v_reschedule_number := v_appointment.reschedule_count + 1;

  IF lower(btrim(p_decision)) = 'rejected' THEN
    INSERT INTO public.appointment_reschedule_history (
      appointment_id,
      new_appointment_id,
      reschedule_number,
      from_date,
      from_start_time,
      to_date,
      to_start_time,
      reason,
      decision,
      approved_by
    ) VALUES (
      v_appointment.id,
      NULL,
      v_reschedule_number,
      v_appointment.appointment_date,
      v_appointment.start_time,
      v_appointment.pending_reschedule_date,
      v_appointment.pending_reschedule_start_time,
      btrim(v_appointment.pending_reschedule_reason),
```

```
      'rejected',
      auth.uid()
    );

    UPDATE public.appointments
    SET pending_reschedule_request_id = NULL,
        pending_reschedule_date = NULL,
        pending_reschedule_start_time = NULL,
        pending_reschedule_reference_code = NULL,
        pending_reschedule_reason = NULL
    WHERE id = v_appointment.id;
    RETURN;
  END IF;

  IF v_appointment.reschedule_count >= 3 THEN
    RAISE EXCEPTION 'This appointment has reached the maximum of 3
reschedules.';
  END IF;
  IF v_appointment.pending_reschedule_date IS NULL
     OR v_appointment.pending_reschedule_start_time IS NULL
     OR NULLIF(btrim(v_appointment.pending_reschedule_reason), '') IS NULL
THEN
    RAISE EXCEPTION 'The reschedule request is missing its requested
schedule or reason.';
  END IF;

  v_duration_minutes := COALESCE(v_appointment.booking_duration_minutes,
75);
  v_reason := btrim(v_appointment.pending_reschedule_reason);
  v_start_minutes := extract(hour FROM
v_appointment.pending_reschedule_start_time)::integer * 60
    + extract(minute FROM
v_appointment.pending_reschedule_start_time)::integer;

  SELECT settings.opening_time, settings.closing_time
    INTO v_opening_time, v_closing_time
  FROM public.shop_settings AS settings
  ORDER BY settings.id
  LIMIT 1;
  v_opening_time := COALESCE(v_opening_time, '08:00'::time);
  v_closing_time := COALESCE(v_closing_time, '17:00'::time);
  v_opening_minutes := extract(hour FROM v_opening_time)::integer * 60
    + extract(minute FROM v_opening_time)::integer;

  IF extract(dow FROM v_appointment.pending_reschedule_date) = 0
```

```
    OR v_appointment.pending_reschedule_start_time < v_opening_time
    OR v_appointment.pending_reschedule_start_time >= v_closing_time
    OR mod(v_start_minutes - v_opening_minutes, 30) <> 0
    OR v_appointment.pending_reschedule_start_time + make_interval(mins =>
v_duration_minutes)
       > v_closing_time THEN
   RAISE EXCEPTION 'The requested schedule is outside the shop booking
hours.';
  END IF;

  SELECT
public.booking_slot_capacity(v_appointment.pending_reschedule_start_time)
    INTO v_slot_capacity;
  IF COALESCE(v_slot_capacity, 0) <= 0 THEN
   RAISE EXCEPTION 'The requested time slot is no longer available.';
  END IF;

  SELECT count(*)
   INTO v_overlapping_appointments
  FROM public.appointments AS other
  WHERE other.id <> v_appointment.id
   AND other.appointment_date = v_appointment.pending_reschedule_date
   AND other.is_archived = false
   AND other.rescheduled_to_appointment_id IS NULL
   AND other.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
   AND tsrange(
     other.appointment_date + other.start_time,
     other.appointment_date + other.start_time
       + make_interval(mins => COALESCE(other.booking_duration_minutes,
75)),
     '[)'
   ) && tsrange(
     v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
     v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
       + make_interval(mins => v_duration_minutes),
     '[)'
   );
  -- Other pending reschedule requests hold their requested times too.
  v_overlapping_appointments := v_overlapping_appointments + (
   SELECT count(*)
   FROM public.appointments AS hold
   WHERE hold.id <> v_appointment.id
    AND hold.pending_reschedule_request_id IS NOT NULL
```

```
      AND hold.pending_reschedule_date =
v_appointment.pending_reschedule_date
      AND hold.pending_reschedule_start_time IS NOT NULL
      AND hold.is_archived = false
      AND hold.rescheduled_to_appointment_id IS NULL
      AND hold.status IN ('pending', 'confirmed')
      AND tsrange(
        hold.pending_reschedule_date + hold.pending_reschedule_start_time,
        hold.pending_reschedule_date + hold.pending_reschedule_start_time
          + make_interval(mins => COALESCE(hold.booking_duration_minutes,
75)),
        '[)'
      ) && tsrange(
        v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
        v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time + make_interval(mins =>
v_duration_minutes),
        '[)'
      )
  );
  IF v_overlapping_appointments >= v_slot_capacity THEN
    RAISE EXCEPTION 'The requested time is no longer available.';
  END IF;

  IF EXISTS (
    SELECT 1
    FROM public.schedule_blocks AS block
    WHERE block.block_date = v_appointment.pending_reschedule_date
      AND block.is_active = true
      AND (
        block.start_time IS NULL
        OR (
          block.start_time IS NOT NULL
          AND (
            (COALESCE(block.reason, '') NOT LIKE 'RANGE:%'
              AND block.start_time = v_appointment.pending_reschedule_start_time)
            OR (COALESCE(block.reason, '') LIKE 'RANGE:%'
              AND tsrange(
                v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
                v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
                  + make_interval(mins => v_duration_minutes),
                '[)'
```

```
            ) && tsrange(
              v_appointment.pending_reschedule_date + block.start_time,
              v_appointment.pending_reschedule_date
                + substring(block.reason FROM '^RANGE:([0-9]{2}:[0-9]{2})')::time,
              '[)'
            ))
        )
      )
    )
  ) THEN
    RAISE EXCEPTION 'The requested time is no longer available because the
shop schedule is blocked.';
  END IF;

  SELECT crew.id
    INTO v_assigned_crew_id
  FROM public.crew_members AS crew
  WHERE crew.is_active = true
    AND crew.is_archived = false
    AND EXISTS (
      SELECT 1
      FROM public.crew_schedules AS schedule
      WHERE schedule.crew_id = crew.id
        AND schedule.schedule_date = v_appointment.pending_reschedule_date
        AND schedule.is_working = true
        AND schedule.start_time IS NOT NULL
        AND schedule.end_time IS NOT NULL
        AND schedule.start_time <= v_appointment.pending_reschedule_start_time
        AND schedule.end_time >= v_appointment.pending_reschedule_start_time
          + make_interval(mins => v_duration_minutes)
    )
    AND NOT EXISTS (
      SELECT 1
      FROM public.crew_availability_exceptions AS exception
      WHERE exception.crew_id = crew.id
        AND exception.start_date <= v_appointment.pending_reschedule_date
        AND exception.end_date >= v_appointment.pending_reschedule_date
        AND (
          exception.is_all_day
          OR (
            exception.start_time IS NOT NULL
            AND exception.end_time IS NOT NULL
            AND tsrange(
              v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
```

```
            v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
              + make_interval(mins => v_duration_minutes),
            '[)'
          ) && tsrange(
            v_appointment.pending_reschedule_date + exception.start_time,
            v_appointment.pending_reschedule_date + exception.end_time,
            '[)'
          )
        )
      )
    )
    AND NOT EXISTS (
      SELECT 1
      FROM public.appointments AS other
      WHERE other.id <> v_appointment.id
        AND other.assigned_crew_id = crew.id
        AND other.appointment_date = v_appointment.pending_reschedule_date
        AND other.is_archived = false
        AND other.rescheduled_to_appointment_id IS NULL
        AND other.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
        AND tsrange(
          other.appointment_date + other.start_time,
          other.appointment_date + other.start_time
            + make_interval(mins => COALESCE(other.booking_duration_minutes,
75)),
          '[)'
        ) && tsrange(
          v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
          v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
            + make_interval(mins => v_duration_minutes),
          '[)'
        )
    )
  -- The mechanic with the fewest bookings that day takes the new schedule.
  ORDER BY public.crew_daily_workload(
    crew.id,
    v_appointment.pending_reschedule_date,
    v_appointment.id
  ), crew.name, crew.id
  LIMIT 1;

  IF v_assigned_crew_id IS NULL THEN
```

```
    RAISE EXCEPTION 'No crew member is available for the requested
appointment period.';
  END IF;

  v_new_reference_code := v_appointment.pending_reschedule_reference_code;
  IF v_new_reference_code IS NULL
     OR v_new_reference_code !~ '^FRM-[A-Z0-9]{6}$' THEN
    RAISE EXCEPTION 'The reschedule request is missing its replacement
reference.';
  END IF;

  v_original_shop_notes := concat_ws(
    E'\n',
    NULLIF(btrim(v_appointment.admin_notes), ''),
    'Rescheduled to ' || v_new_reference_code,
    'Reason for rescheduling: ' || v_reason
  );
  v_new_shop_notes := concat_ws(
    E'\n',
    'Rescheduled from ' || v_appointment.reference_code,
    'Reason for rescheduling: ' || v_reason
  );

  INSERT INTO public.appointments AS replacement (
    reference_code,
    booking_request_id,
    customer_name,
    first_name,
    middle_name,
    last_name,
    phone,
    moto_brand,
    moto_model,
    moto_variant,
    moto_year,
    plate_number,
    appointment_date,
    start_time,
    status,
    notes,
    admin_notes,
    total_estimate,
    booking_duration_minutes,
    terms_accepted,
    assigned_crew_id,
```

```
  reschedule_count,
  rescheduled_from_appointment_id
) VALUES (
  v_new_reference_code,
  gen_random_uuid(),
  v_appointment.customer_name,
  v_appointment.first_name,
  v_appointment.middle_name,
  v_appointment.last_name,
  v_appointment.phone,
  v_appointment.moto_brand,
  v_appointment.moto_model,
  v_appointment.moto_variant,
  v_appointment.moto_year,
  v_appointment.plate_number,
  v_appointment.pending_reschedule_date,
  v_appointment.pending_reschedule_start_time,
  'confirmed',
  v_appointment.notes,
  v_new_shop_notes,
  v_appointment.total_estimate,
  v_duration_minutes,
  v_appointment.terms_accepted,
  v_assigned_crew_id,
  v_appointment.reschedule_count + 1,
  v_appointment.id
)
RETURNING replacement.id INTO v_new_appointment_id;

INSERT INTO public.appointment_services (
  appointment_id,
  service_id,
  service_name,
  price,
  duration_minutes
)
SELECT
  v_new_appointment_id,
  service.service_id,
  service.service_name,
  service.price,
  service.duration_minutes
FROM public.appointment_services AS service
WHERE service.appointment_id = v_appointment.id;
```

```
  UPDATE public.appointments
  SET status = 'rescheduled',
      admin_notes = v_original_shop_notes,
      reschedule_count = v_appointment.reschedule_count + 1,
      rescheduled_to_appointment_id = v_new_appointment_id,
      pending_reschedule_request_id = NULL,
      pending_reschedule_date = NULL,
      pending_reschedule_start_time = NULL,
      pending_reschedule_reference_code = NULL,
      pending_reschedule_reason = NULL
  WHERE id = v_appointment.id;

  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES (
    'rescheduled_appointment',
    'Rescheduled booking ' || v_new_reference_code,
    v_appointment.customer_name || ' rescheduled from ' ||
v_appointment.reference_code
      || ' to ' || v_new_reference_code || '. Reason: ' || v_reason,
    v_new_appointment_id
  );

  INSERT INTO public.appointment_reschedule_history (
    appointment_id,
    new_appointment_id,
    reschedule_number,
    from_date,
    from_start_time,
    to_date,
    to_start_time,
    reason,
    decision,
    approved_by
  ) VALUES (
    v_appointment.id,
    v_new_appointment_id,
    v_reschedule_number,
    v_appointment.appointment_date,
    v_appointment.start_time,
    v_appointment.pending_reschedule_date,
    v_appointment.pending_reschedule_start_time,
    v_reason,
    'confirmed',
    auth.uid()
  );
```

```
END;
$_$;


ALTER FUNCTION "public"."review_reschedule_request"("p_appointment_id"
"uuid", "p_decision" "text") OWNER TO "postgres";


CREATE OR REPLACE FUNCTION "public"."set_catalog_archive_timestamp"()
RETURNS "trigger"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
BEGIN
  IF NEW.is_archived = true AND OLD.is_archived = false THEN
    NEW.archived_at = now();
  ELSIF NEW.is_archived = false AND OLD.is_archived = true THEN
    NEW.archived_at = NULL;
  END IF;
  RETURN NEW;
END;
$$;


ALTER FUNCTION "public"."set_catalog_archive_timestamp"() OWNER TO
"postgres";


CREATE OR REPLACE FUNCTION "public"."set_price_history_actor"()
RETURNS "trigger"
    LANGUAGE "plpgsql" SECURITY DEFINER
    SET "search_path" TO 'public'
    AS $$
BEGIN
  NEW.changed_by = COALESCE(auth.uid(), NEW.changed_by);
  NEW.changed_by_email = COALESCE(auth.jwt() ->> 'email',
NEW.changed_by_email);
  RETURN NEW;
END;
$$;


ALTER FUNCTION "public"."set_price_history_actor"() OWNER TO "postgres";
```

```
CREATE OR REPLACE FUNCTION
"public"."set_schedule_block_archive_timestamp"() RETURNS "trigger"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
BEGIN
  IF NEW.is_active = false AND OLD.is_active = true THEN
    NEW.archived_at = now();
  ELSIF NEW.is_active = true AND OLD.is_active = false THEN
    NEW.archived_at = NULL;
  END IF;
  RETURN NEW;
END;
$$;


ALTER FUNCTION "public"."set_schedule_block_archive_timestamp"() OWNER
TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."submit_reschedule_request"("p_appointment_id" "uuid", "p_request_id"
"uuid", "p_appointment_date" "date", "p_start_time" time without time zone,
"p_reason" "text") RETURNS "void"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
DECLARE
  v_appointment public.appointments%ROWTYPE;
  v_reference_code text;
  v_reference_available boolean := false;
  v_reference_alphabet constant text :=
'ABCDEFGHJKLMNPQRSTUVWXYZ23456789';
BEGIN
  IF p_request_id IS NULL OR p_appointment_date IS NULL OR p_start_time IS
NULL
     OR NULLIF(btrim(p_reason), '') IS NULL THEN
    RAISE EXCEPTION 'A date, time, and rescheduling reason are required.';
  END IF;

  SELECT appointment.*
    INTO v_appointment
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
```

```
  FOR UPDATE;

  IF NOT FOUND THEN
    RAISE EXCEPTION 'This appointment is no longer available.';
  END IF;
  IF v_appointment.pending_reschedule_request_id IS NOT NULL
     AND v_appointment.pending_reschedule_request_id <> p_request_id THEN
    RAISE EXCEPTION 'A reschedule request is already awaiting review.';
  END IF;
  IF v_appointment.reschedule_count >= 3 THEN
    RAISE EXCEPTION 'This appointment has reached the maximum of 3
reschedules.';
  END IF;
  IF v_appointment.status NOT IN ('pending', 'confirmed')
     OR v_appointment.rescheduled_to_appointment_id IS NOT NULL THEN
    RAISE EXCEPTION 'This appointment can no longer be rescheduled.';
  END IF;

  v_reference_code := v_appointment.pending_reschedule_reference_code;
  IF v_reference_code IS NULL THEN
    FOR v_attempt IN 1..10 LOOP
      v_reference_code := 'FRM-';
      FOR v_character IN 1..6 LOOP
        v_reference_code := v_reference_code || substr(
          v_reference_alphabet,
          1 + floor(random() * length(v_reference_alphabet))::integer,
          1
        );
      END LOOP;

      SELECT NOT EXISTS (
        SELECT 1
        FROM public.appointments
        WHERE reference_code = v_reference_code
           OR pending_reschedule_reference_code = v_reference_code
      ) INTO v_reference_available;
      EXIT WHEN v_reference_available;
    END LOOP;

    IF NOT v_reference_available THEN
      RAISE EXCEPTION 'Could not generate a unique booking reference. Please
try again.';
    END IF;
  END IF;
```

```
  UPDATE public.appointments
  SET pending_reschedule_request_id = p_request_id,
      pending_reschedule_date = p_appointment_date,
      pending_reschedule_start_time = p_start_time,
      pending_reschedule_reference_code = v_reference_code,
      pending_reschedule_reason = NULLIF(btrim(p_reason), '')
  WHERE id = v_appointment.id;

  IF v_appointment.pending_reschedule_request_id IS NULL THEN
    INSERT INTO public.notifications (type, title, message, appointment_id)
    VALUES (
      'reschedule_request',
      'Reschedule request ' || v_appointment.reference_code,
      v_appointment.customer_name || ' requested ' ||
to_char(p_appointment_date, 'Mon FMDD, YYYY')
        || ' at ' || to_char(p_start_time, 'HH12:MI AM') || '. New reference: '
        || v_reference_code || '. Reason: ' || btrim(p_reason),
      v_appointment.id
    );
  END IF;
END;
$$;


ALTER FUNCTION "public"."submit_reschedule_request"("p_appointment_id"
"uuid", "p_request_id" "uuid", "p_appointment_date" "date", "p_start_time" time
without time zone, "p_reason" "text") OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."sync_customer_service_booking_alert"() RETURNS "trigger"
    LANGUAGE "plpgsql" SECURITY DEFINER
    SET "search_path" TO 'public'
    AS $$
DECLARE
  v_appointment_id uuid;
  v_customer_phone text;
  v_customer_name text;
  v_previous_customer_phone text;
  v_previous_customer_name text;
  v_service record;
BEGIN
  IF TG_TABLE_NAME = 'appointments' THEN
    IF TG_OP = 'DELETE' THEN
      v_appointment_id := OLD.id;
```

```
      v_customer_phone := OLD.phone;
      v_customer_name := OLD.customer_name;
    ELSE
      v_appointment_id := NEW.id;
      v_customer_phone := NEW.phone;
      v_customer_name := NEW.customer_name;
      IF TG_OP = 'UPDATE' AND OLD.phone IS DISTINCT FROM NEW.phone
THEN
        v_previous_customer_phone := OLD.phone;
        v_previous_customer_name := OLD.customer_name;
      END IF;
    END IF;

    FOR v_service IN
      SELECT DISTINCT appointment_service.service_id,
appointment_service.service_name
      FROM public.appointment_services AS appointment_service
      WHERE appointment_service.appointment_id = v_appointment_id
    LOOP
      -- Refresh both customer scopes if an administrator corrects a phone
      -- number. The lexical order prevents two concurrent corrections from
      -- taking the customer advisory locks in opposite directions.
      IF v_previous_customer_phone IS NOT NULL
         AND COALESCE(v_previous_customer_phone, '') <
COALESCE(v_customer_phone, '') THEN
        PERFORM public.refresh_customer_service_booking_alert(
          v_previous_customer_phone,
          v_service.service_id,
          v_service.service_name,
          v_appointment_id,
          v_previous_customer_name
        );
      END IF;

      PERFORM public.refresh_customer_service_booking_alert(
        v_customer_phone,
        v_service.service_id,
        v_service.service_name,
        v_appointment_id,
        v_customer_name
      );

      IF v_previous_customer_phone IS NOT NULL
         AND COALESCE(v_previous_customer_phone, '') >=
COALESCE(v_customer_phone, '') THEN
```

```
      PERFORM public.refresh_customer_service_booking_alert(
        v_previous_customer_phone,
        v_service.service_id,
        v_service.service_name,
        v_appointment_id,
        v_previous_customer_name
      );
    END IF;
  END LOOP;
ELSE
  IF TG_OP = 'DELETE' THEN
    v_appointment_id := OLD.appointment_id;
  ELSE
    v_appointment_id := NEW.appointment_id;
  END IF;

  SELECT appointment.phone, appointment.customer_name
  INTO v_customer_phone, v_customer_name
  FROM public.appointments AS appointment
  WHERE appointment.id = v_appointment_id;

  -- Cascading deletion can remove the parent before this deferred trigger
  -- executes. It does not occur in the normal soft-archive workflow; if it
  -- does, there is no remaining active booking to alert for this row.
  IF NOT FOUND THEN
    IF TG_OP = 'DELETE' THEN
      RETURN OLD;
    END IF;
    RETURN NEW;
  END IF;

  IF TG_OP <> 'DELETE' THEN
    PERFORM public.refresh_customer_service_booking_alert(
      v_customer_phone,
      NEW.service_id,
      NEW.service_name,
      v_appointment_id,
      v_customer_name
    );
  END IF;

  IF TG_OP <> 'INSERT' THEN
    PERFORM public.refresh_customer_service_booking_alert(
      v_customer_phone,
      OLD.service_id,
```

```
        OLD.service_name,
        v_appointment_id,
        v_customer_name
      );
    END IF;
  END IF;

  IF TG_OP = 'DELETE' THEN
    RETURN OLD;
  END IF;
  RETURN NEW;
END;
$$;


ALTER FUNCTION "public"."sync_customer_service_booking_alert"() OWNER
TO "postgres";


CREATE OR REPLACE FUNCTION "public"."sync_product_stock_status"()
RETURNS "trigger"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
BEGIN
  NEW.stock_quantity := GREATEST(COALESCE(NEW.stock_quantity, 0), 0);
  NEW.in_stock := NEW.stock_quantity > 0;
  RETURN NEW;
END;
$$;


ALTER FUNCTION "public"."sync_product_stock_status"() OWNER TO
"postgres";


CREATE OR REPLACE FUNCTION
"public"."sync_service_progress_notifications"() RETURNS integer
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Only administrators can synchronize service progress
notifications.';
```

```
  END IF;

  RETURN public.enqueue_due_service_progress_notifications();
END;
$$;


ALTER FUNCTION "public"."sync_service_progress_notifications"() OWNER TO
"postgres";


CREATE OR REPLACE FUNCTION "public"."track_appointment_reschedule"()
RETURNS "trigger"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
BEGIN
  IF NEW.appointment_date IS DISTINCT FROM OLD.appointment_date
     OR NEW.start_time IS DISTINCT FROM OLD.start_time THEN
    IF OLD.reschedule_count >= 3 THEN
      RAISE EXCEPTION 'This appointment has reached the maximum of 3
reschedules.';
    END IF;

    NEW.reschedule_count := OLD.reschedule_count + 1;
    NEW.last_reschedule_rejected_at := NULL;
    NEW.last_reschedule_rejection_message := NULL;
  ELSIF OLD.pending_reschedule_request_id IS NULL
     AND NEW.pending_reschedule_request_id IS NOT NULL THEN
    NEW.last_reschedule_rejected_at := NULL;
    NEW.last_reschedule_rejection_message := NULL;
  ELSIF OLD.pending_reschedule_request_id IS NOT NULL
     AND NEW.pending_reschedule_request_id IS NULL THEN
    IF NEW.status = 'rescheduled' AND NEW.rescheduled_to_appointment_id IS
NOT NULL THEN
      NEW.last_reschedule_rejected_at := NULL;
      NEW.last_reschedule_rejection_message := NULL;
    ELSE
      NEW.last_reschedule_rejected_at := now();
      NEW.last_reschedule_rejection_message :=
        'Reschedule Request Rejected. Your requested reschedule was not
approved. Your original appointment remains unchanged and reserved.';
    END IF;
  END IF;
```

```
  RETURN NEW;
END;
$$;


ALTER FUNCTION "public"."track_appointment_reschedule"() OWNER TO
"postgres";


CREATE OR REPLACE FUNCTION
"public"."track_appointment_service_progress"() RETURNS "trigger"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
BEGIN
  IF NEW.status IS NOT DISTINCT FROM OLD.status THEN
    RETURN NEW;
  END IF;

  IF NEW.status = 'in_progress' THEN
    IF OLD.service_ended_at IS NOT NULL THEN
      RAISE EXCEPTION 'A completed service cannot be restarted.';
    END IF;

    NEW.service_started_at := COALESCE(NEW.service_started_at, now());
    NEW.service_ended_at := NULL;
  ELSIF NEW.status = 'completed' THEN
    IF NEW.service_started_at IS NULL THEN
      RAISE EXCEPTION 'Start the service before marking the appointment
completed.';
    END IF;

    NEW.service_ended_at := COALESCE(NEW.service_ended_at, now());
  ELSIF NEW.status = 'no_show' THEN
    IF NEW.service_started_at IS NOT NULL THEN
      RAISE EXCEPTION 'An appointment with a recorded service start cannot be
marked as no-show.';
    END IF;
    IF NEW.arrival_notification_snooze_count < 3 THEN
      RAISE EXCEPTION 'An appointment can be marked as no-show after three
arrival snoozes.';
    END IF;
  ELSIF OLD.service_started_at IS NOT NULL
    AND NEW.status IN ('pending', 'confirmed', 'rescheduled', 'cancelled',
'rejected') THEN
```

```
    RAISE EXCEPTION 'A started service cannot return to a pre-service
appointment status.';
  END IF;

  RETURN NEW;
END;
$$;


ALTER FUNCTION "public"."track_appointment_service_progress"() OWNER
TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."update_appointment_details_atomic"("p_appointment_id" "uuid",
"p_service_ids" "uuid"[], "p_appointment_date" "date", "p_start_time" time without
time zone, "p_assigned_crew_id" "uuid", "p_crew_assignment_manual" boolean,
"p_status" "text", "p_admin_notes" "text") RETURNS "void"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
DECLARE
  v_service_count integer;
  v_total_estimate numeric(10,2);
  v_duration_minutes integer;
  v_assigned_crew_id uuid := p_assigned_crew_id;
  v_appointment_id uuid;
  v_motorcycle_category text;
BEGIN
  IF COALESCE(cardinality(p_service_ids), 0) = 0 THEN
    RAISE EXCEPTION 'Select at least one service.';
  END IF;

  IF p_status NOT IN (
    'pending', 'confirmed', 'in_progress', 'completed', 'rescheduled', 'cancelled',
'rejected', 'no_show'
  ) THEN
    RAISE EXCEPTION 'Choose a valid appointment status.';
  END IF;

  SELECT appointment.id, catalog.cc_category
    INTO v_appointment_id, v_motorcycle_category
  FROM public.appointments AS appointment
  LEFT JOIN public.motorcycle_catalog AS catalog
    ON lower(btrim(catalog.brand)) = lower(btrim(appointment.moto_brand))
```

```
   AND lower(btrim(catalog.model)) = lower(btrim(appointment.moto_model))
   AND catalog.is_archived = false
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
  LIMIT 1;

  IF v_appointment_id IS NULL THEN
    RAISE EXCEPTION 'This appointment is no longer available.';
  END IF;
  IF v_motorcycle_category IS NULL
     OR v_motorcycle_category NOT IN ('small_bike', 'big_bike') THEN
    RAISE EXCEPTION 'The appointment motorcycle needs an engine CC
classification before its services can be updated.';
  END IF;

  SELECT count(*)
    INTO v_service_count
  FROM public.services
  WHERE id = ANY(p_service_ids)
    AND is_active = true
    AND is_archived = false;

  IF v_service_count <> cardinality(p_service_ids) THEN
    RAISE EXCEPTION 'One or more selected services are no longer available.';
  END IF;

  IF v_motorcycle_category = 'big_bike' AND EXISTS (
    SELECT 1
    FROM public.services
    WHERE id = ANY(p_service_ids)
      AND (large_bike_price IS NULL OR large_bike_duration_minutes IS NULL)
  ) THEN
    RAISE EXCEPTION 'Every selected service needs a Large Bike price and
duration.';
  END IF;

  SELECT
    COALESCE(sum(CASE WHEN v_motorcycle_category = 'big_bike' THEN
large_bike_price ELSE price END), 0),
    COALESCE(sum(CASE WHEN v_motorcycle_category = 'big_bike' THEN
large_bike_duration_minutes ELSE duration_minutes END), 0)
  INTO v_total_estimate, v_duration_minutes
  FROM public.services
  WHERE id = ANY(p_service_ids);
```

```
  PERFORM pg_advisory_xact_lock(hashtext('booking-capacity:' ||
p_appointment_date::text));

  IF p_status = 'confirmed' AND v_assigned_crew_id IS NULL AND NOT
p_crew_assignment_manual THEN
    SELECT crew.id
      INTO v_assigned_crew_id
    FROM public.crew_members AS crew
    WHERE crew.is_active = true
      AND crew.is_archived = false
      AND public.is_crew_available_for_appointment(
        crew.id, p_appointment_date, p_start_time, v_duration_minutes,
p_appointment_id
      )
    ORDER BY public.crew_daily_workload(crew.id, p_appointment_date,
p_appointment_id),
      crew.name,
      crew.id
    LIMIT 1;

    IF v_assigned_crew_id IS NULL THEN
      RAISE EXCEPTION 'No crew member is available for the selected
appointment period.';
    END IF;
  END IF;

  IF v_assigned_crew_id IS NOT NULL
     AND NOT public.is_crew_available_for_appointment(
       v_assigned_crew_id, p_appointment_date, p_start_time,
v_duration_minutes, p_appointment_id
     ) THEN
    RAISE EXCEPTION 'Choose a crew member who is free for the full
appointment period.';
  END IF;

  UPDATE public.appointments
  SET appointment_date = p_appointment_date,
      start_time = p_start_time,
      assigned_crew_id = v_assigned_crew_id,
      status = p_status,
      admin_notes = NULLIF(btrim(p_admin_notes), ''),
      total_estimate = v_total_estimate,
      booking_duration_minutes = v_duration_minutes
  WHERE id = p_appointment_id
    AND is_archived = false;
```

```
  DELETE FROM public.appointment_services
  WHERE appointment_id = p_appointment_id;

  INSERT INTO public.appointment_services (
    appointment_id, service_id, service_name, price, duration_minutes
  )
  SELECT
    p_appointment_id,
    service.id,
    service.name,
    CASE WHEN v_motorcycle_category = 'big_bike' THEN
service.large_bike_price ELSE service.price END,
    CASE WHEN v_motorcycle_category = 'big_bike' THEN
service.large_bike_duration_minutes ELSE service.duration_minutes END
  FROM public.services AS service
  WHERE service.id = ANY(p_service_ids);
END;
$$;


ALTER FUNCTION
"public"."update_appointment_details_atomic"("p_appointment_id" "uuid",
"p_service_ids" "uuid"[], "p_appointment_date" "date", "p_start_time" time without
time zone, "p_assigned_crew_id" "uuid", "p_crew_assignment_manual" boolean,
"p_status" "text", "p_admin_notes" "text") OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."update_appointment_details_atomic"("p_appointment_id" "uuid",
"p_service_ids" "uuid"[], "p_appointment_date" "date", "p_start_time" time without
time zone, "p_assigned_crew_id" "uuid", "p_crew_assignment_manual" boolean,
"p_status" "text", "p_admin_notes" "text", "p_first_name" "text", "p_middle_name"
"text", "p_last_name" "text", "p_phone" "text") RETURNS "void"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $_$
BEGIN
  IF NOT public.is_valid_customer_name_part(p_first_name)
     OR NOT public.is_valid_customer_name_part(p_last_name) THEN
    RAISE EXCEPTION 'First and last names must contain letters, spaces,
hyphens, or apostrophes only.';
  END IF;

  IF COALESCE(btrim(p_middle_name), '') <> ''
```

```
    AND NOT public.is_valid_customer_name_part(p_middle_name) THEN
   RAISE EXCEPTION 'Middle name must contain letters, spaces, hyphens, or
apostrophes only.';
  END IF;

  IF COALESCE(btrim(p_phone), '') !~ '^09[0-9]{9}$' THEN
    RAISE EXCEPTION 'Enter a valid Philippine mobile number.';
  END IF;

  PERFORM public.update_appointment_details_atomic(
    p_appointment_id,
    p_service_ids,
    p_appointment_date,
    p_start_time,
    p_assigned_crew_id,
    p_crew_assignment_manual,
    p_status,
    p_admin_notes
  );

  UPDATE public.appointments
  SET first_name = btrim(p_first_name),
      middle_name = NULLIF(btrim(p_middle_name), ''),
      last_name = btrim(p_last_name),
      phone = btrim(p_phone),
      customer_name = concat_ws(
        ' ',
        btrim(p_first_name),
        NULLIF(btrim(p_middle_name), ''),
        btrim(p_last_name)
      )
  WHERE id = p_appointment_id
    AND is_archived = false;
END;
$_$;


ALTER FUNCTION
"public"."update_appointment_details_atomic"("p_appointment_id" "uuid",
"p_service_ids" "uuid"[], "p_appointment_date" "date", "p_start_time" time without
time zone, "p_assigned_crew_id" "uuid", "p_crew_assignment_manual" boolean,
"p_status" "text", "p_admin_notes" "text", "p_first_name" "text", "p_middle_name"
"text", "p_last_name" "text", "p_phone" "text") OWNER TO "postgres";
```

```
CREATE OR REPLACE FUNCTION
"public"."update_appointment_financial_details_atomic"("p_appointment_id"
"uuid", "p_service_ids" "uuid"[], "p_appointment_date" "date", "p_start_time" time
without time zone, "p_assigned_crew_id" "uuid", "p_crew_assignment_manual"
boolean, "p_status" "text", "p_admin_notes" "text", "p_first_name" "text",
"p_middle_name" "text", "p_last_name" "text", "p_phone" "text", "p_products"
"jsonb") RETURNS "void"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public', 'auth'
    AS $$
DECLARE
  v_wants_products boolean;
  v_requested_count integer;
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Only administrators can update appointment products.';
  END IF;
  IF jsonb_typeof(COALESCE(p_products, '[]'::jsonb)) <> 'array'
     OR jsonb_array_length(COALESCE(p_products, '[]'::jsonb)) > 50 THEN
    RAISE EXCEPTION 'Products must be a list of up to 50 items.';
  END IF;

  PERFORM public.update_appointment_details_atomic(
    p_appointment_id, p_service_ids, p_appointment_date, p_start_time,
    p_assigned_crew_id, p_crew_assignment_manual, p_status, p_admin_notes,
    p_first_name, p_middle_name, p_last_name, p_phone
  );

  SELECT wants_products
    INTO v_wants_products
  FROM public.appointments
  WHERE id = p_appointment_id
    AND is_archived = false
  FOR UPDATE;
  IF NOT FOUND THEN
    RAISE EXCEPTION 'This appointment is no longer available.';
  END IF;

  WITH requested AS (
    SELECT product_id, quantity, unit_price
    FROM jsonb_to_recordset(COALESCE(p_products, '[]'::jsonb))
      AS item(product_id uuid, quantity integer, unit_price numeric)
  )
  SELECT count(*) INTO v_requested_count FROM requested;
```

```
  IF v_requested_count > 0 AND NOT v_wants_products THEN
    RAISE EXCEPTION 'This customer declined shop products for this
appointment.';
  END IF;
  IF v_requested_count > 0 AND p_status NOT IN ('confirmed', 'in_progress',
'completed') THEN
    RAISE EXCEPTION 'Products can be added after the appointment is
confirmed, before or during service.';
  END IF;

  IF EXISTS (
    WITH requested AS (
      SELECT product_id, quantity, unit_price
      FROM jsonb_to_recordset(COALESCE(p_products, '[]'::jsonb))
        AS item(product_id uuid, quantity integer, unit_price numeric)
    )
    SELECT 1
    FROM requested
    GROUP BY product_id
    HAVING count(*) > 1
       OR min(quantity) IS NULL
       OR min(quantity) < 1
       OR max(quantity) > 999
       OR min(unit_price) IS NULL
       OR min(unit_price) < 0
  ) THEN
    RAISE EXCEPTION 'Each product must be unique and have a valid quantity
and price.';
  END IF;

  -- Lock existing and requested inventory in a stable order. This prevents
  -- parallel appointment edits from overselling a product.
  PERFORM 1
  FROM public.products AS product
  WHERE product.id IN (
    SELECT item.product_id
    FROM jsonb_to_recordset(COALESCE(p_products, '[]'::jsonb))
      AS item(product_id uuid, quantity integer, unit_price numeric)
    UNION
    SELECT appointment_product.product_id
    FROM public.appointment_products AS appointment_product
    WHERE appointment_product.appointment_id = p_appointment_id
  )
  ORDER BY product.id
  FOR UPDATE;
```

```
  IF EXISTS (
    WITH requested AS (
      SELECT product_id, quantity, unit_price
      FROM jsonb_to_recordset(COALESCE(p_products, '[]'::jsonb))
        AS item(product_id uuid, quantity integer, unit_price numeric)
    )
    SELECT 1
    FROM requested
    LEFT JOIN public.products AS product ON product.id = requested.product_id
    LEFT JOIN public.appointment_products AS existing
      ON existing.appointment_id = p_appointment_id
     AND existing.product_id = requested.product_id
    WHERE product.id IS NULL
       OR (existing.id IS NULL AND (product.is_archived OR NOT
product.is_active OR NOT product.in_stock))
       OR product.stock_quantity + COALESCE(existing.quantity, 0) <
requested.quantity
  ) THEN
    RAISE EXCEPTION 'One or more products are unavailable or do not have
enough stock.';
  END IF;

  UPDATE public.products AS product
  SET stock_quantity = product.stock_quantity + existing.quantity
  FROM public.appointment_products AS existing
  WHERE existing.appointment_id = p_appointment_id
    AND existing.product_id = product.id
    AND NOT EXISTS (
      SELECT 1
      FROM jsonb_to_recordset(COALESCE(p_products, '[]'::jsonb))
        AS item(product_id uuid, quantity integer, unit_price numeric)
      WHERE item.product_id = existing.product_id
    );

  UPDATE public.products AS product
  SET stock_quantity = product.stock_quantity + COALESCE(existing.quantity, 0)
- requested.quantity
  FROM jsonb_to_recordset(COALESCE(p_products, '[]'::jsonb))
    AS requested(product_id uuid, quantity integer, unit_price numeric)
  LEFT JOIN public.appointment_products AS existing
    ON existing.appointment_id = p_appointment_id
   AND existing.product_id = requested.product_id
  WHERE product.id = requested.product_id;
```

```
  INSERT INTO public.appointment_products (
    appointment_id, product_id, product_name, quantity, unit_price
  )
  SELECT p_appointment_id, requested.product_id, product.name,
requested.quantity, requested.unit_price
  FROM jsonb_to_recordset(COALESCE(p_products, '[]'::jsonb))
    AS requested(product_id uuid, quantity integer, unit_price numeric)
  JOIN public.products AS product ON product.id = requested.product_id
  ON CONFLICT (appointment_id, product_id) DO UPDATE
    SET quantity = EXCLUDED.quantity,
        unit_price = EXCLUDED.unit_price,
        updated_at = now();

  DELETE FROM public.appointment_products AS existing
  WHERE existing.appointment_id = p_appointment_id
    AND NOT EXISTS (
      SELECT 1
      FROM jsonb_to_recordset(COALESCE(p_products, '[]'::jsonb))
        AS item(product_id uuid, quantity integer, unit_price numeric)
      WHERE item.product_id = existing.product_id
    );
END;
$$;


ALTER FUNCTION
"public"."update_appointment_financial_details_atomic"("p_appointment_id"
"uuid", "p_service_ids" "uuid"[], "p_appointment_date" "date", "p_start_time" time
without time zone, "p_assigned_crew_id" "uuid", "p_crew_assignment_manual"
boolean, "p_status" "text", "p_admin_notes" "text", "p_first_name" "text",
"p_middle_name" "text", "p_last_name" "text", "p_phone" "text", "p_products"
"jsonb") OWNER TO "postgres";


CREATE OR REPLACE FUNCTION "public"."update_updated_at_column"()
RETURNS "trigger"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
BEGIN NEW.updated_at = now(); RETURN NEW; END; $$;


ALTER FUNCTION "public"."update_updated_at_column"() OWNER TO
"postgres";
```

```
CREATE OR REPLACE FUNCTION "public"."validate_admin_session"()
RETURNS boolean
    LANGUAGE "sql" STABLE SECURITY DEFINER
    SET "search_path" TO 'public'
    AS $$
  SELECT public.is_current_admin_session(auth.uid());
$$;


ALTER FUNCTION "public"."validate_admin_session"() OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."validate_appointment_completion_transition"() RETURNS "trigger"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
BEGIN
  IF NEW.status = 'completed' AND OLD.status IS DISTINCT FROM 'completed'
THEN
    IF OLD.status <> 'in_progress' OR OLD.service_started_at IS NULL OR
OLD.service_ended_at IS NOT NULL THEN
      RAISE EXCEPTION 'Start the service before marking the appointment
completed.';
    END IF;
    NEW.service_ended_at := COALESCE(NEW.service_ended_at, now());
  END IF;
  RETURN NEW;
END;
$$;


ALTER FUNCTION "public"."validate_appointment_completion_transition"()
OWNER TO "postgres";


CREATE OR REPLACE FUNCTION
"public"."verify_appointment_action_code"("p_appointment_id" "uuid", "p_action"
"text", "p_code_hash" "text", "p_token_hash" "text", "p_token_ttl_seconds"
integer) RETURNS "text"
    LANGUAGE "plpgsql"
    SET "search_path" TO 'public'
    AS $$
DECLARE
```

```
  v_challenge public.appointment_action_verifications%ROWTYPE;
BEGIN
  IF p_token_ttl_seconds < 60 OR p_token_ttl_seconds > 7200 THEN
    RAISE EXCEPTION 'Verification token lifetime must be between 1 minute and
2 hours.';
  END IF;

  -- Only the newest unverified code for this appointment and action counts.
  SELECT challenge.*
    INTO v_challenge
  FROM public.appointment_action_verifications AS challenge
  WHERE challenge.appointment_id = p_appointment_id
    AND challenge.action = p_action
    AND challenge.verified_at IS NULL
  ORDER BY challenge.created_at DESC
  LIMIT 1
  FOR UPDATE;

  IF NOT FOUND OR v_challenge.expires_at <= now() OR
v_challenge.attempt_count >= 5 THEN
    RETURN 'expired';
  END IF;

  IF v_challenge.code_hash <> p_code_hash THEN
    UPDATE public.appointment_action_verifications
    SET attempt_count = attempt_count + 1,
        expires_at = CASE WHEN attempt_count + 1 >= 5 THEN now() ELSE
expires_at END
    WHERE id = v_challenge.id;
    RETURN 'invalid';
  END IF;

  UPDATE public.appointment_action_verifications
  SET verified_at = now(),
      token_hash = p_token_hash,
      token_expires_at = now() + make_interval(secs => p_token_ttl_seconds)
  WHERE id = v_challenge.id;

  RETURN 'verified';
END;
$$;


ALTER FUNCTION
"public"."verify_appointment_action_code"("p_appointment_id" "uuid", "p_action"
```

```
"text", "p_code_hash" "text", "p_token_hash" "text", "p_token_ttl_seconds"
integer) OWNER TO "postgres";

SET default_tablespace = '';

SET default_table_access_method = "heap";


CREATE TABLE IF NOT EXISTS "public"."admin_activity_logs" (
    "id" "uuid" DEFAULT "gen_random_uuid"() NOT NULL,
    "actor_id" "uuid",
    "actor_email" "text",
    "action" "text" NOT NULL,
    "resource_type" "text" NOT NULL,
    "target_label" "text" NOT NULL,
    "summary" "text" NOT NULL,
    "changed_fields" "text"[] DEFAULT ARRAY[]::"text"[] NOT NULL,
    "activity_date" "date" DEFAULT (("now"() AT TIME ZONE
'Asia/Manila'::"text"))::"date" NOT NULL,
    "activity_time" time without time zone DEFAULT (("now"() AT TIME ZONE
'Asia/Manila'::"text"))::time without time zone NOT NULL,
    "created_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    "record_id" "uuid",
    "ip_address" "text"
);


ALTER TABLE "public"."admin_activity_logs" OWNER TO "postgres";


CREATE TABLE IF NOT EXISTS "public"."admin_session_handover_tokens" (
    "token" "uuid" NOT NULL,
    "admin_id" "uuid" NOT NULL,
    "session_id" "uuid" NOT NULL,
    "expires_at" timestamp with time zone NOT NULL,
    "created_at" timestamp with time zone DEFAULT "now"() NOT NULL
);


ALTER TABLE "public"."admin_session_handover_tokens" OWNER TO
"postgres";


CREATE TABLE IF NOT EXISTS "public"."admin_session_leases" (
    "admin_id" "uuid" NOT NULL,
```

```
    "session_id" "uuid" NOT NULL,
    "expires_at" timestamp with time zone NOT NULL,
    "last_seen_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    "created_at" timestamp with time zone DEFAULT "now"() NOT NULL
);


ALTER TABLE "public"."admin_session_leases" OWNER TO "postgres";


CREATE TABLE IF NOT EXISTS "public"."appointment_action_verifications" (
    "id" "uuid" DEFAULT "gen_random_uuid"() NOT NULL,
    "appointment_id" "uuid" NOT NULL,
    "action" "text" NOT NULL,
    "code_hash" "text" NOT NULL,
    "expires_at" timestamp with time zone NOT NULL,
    "attempt_count" integer DEFAULT 0 NOT NULL,
    "verified_at" timestamp with time zone,
    "token_hash" "text",
    "token_expires_at" timestamp with time zone,
    "created_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    CONSTRAINT "appointment_action_verifications_action_check" CHECK
(("action" = ANY (ARRAY['cancel'::"text", 'reschedule'::"text"]))),
    CONSTRAINT "appointment_action_verifications_attempt_count_check"
CHECK ((("attempt_count" >= 0) AND ("attempt_count" <= 5))),
    CONSTRAINT "appointment_action_verifications_check" CHECK
((("verified_at" IS NULL) = ("token_hash" IS NULL))),
    CONSTRAINT "appointment_action_verifications_check1" CHECK
((("token_hash" IS NULL) = ("token_expires_at" IS NULL))),
    CONSTRAINT "appointment_action_verifications_code_hash_check" CHECK
(("code_hash" ~ '^[a-f0-9]{64}$'::"text")),
    CONSTRAINT "appointment_action_verifications_token_hash_check" CHECK
((("token_hash" IS NULL) OR ("token_hash" ~ '^[a-f0-9]{64}$'::"text")))
);


ALTER TABLE "public"."appointment_action_verifications" OWNER TO
"postgres";


CREATE TABLE IF NOT EXISTS
"public"."appointment_completion_notifications" (
    "id" "uuid" DEFAULT "gen_random_uuid"() NOT NULL,
    "appointment_id" "uuid" NOT NULL,
    "channel" "text" NOT NULL,
```

```
    "recipient" "text" NOT NULL,
    "status" "text" DEFAULT 'pending'::"text" NOT NULL,
    "attempt_count" integer DEFAULT 0 NOT NULL,
    "last_error" "text",
    "sent_at" timestamp with time zone,
    "created_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    "updated_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    CONSTRAINT "appointment_completion_notifications_attempt_count_check"
CHECK (("attempt_count" >= 0)),
    CONSTRAINT "appointment_completion_notifications_channel_check"
CHECK (("channel" = ANY (ARRAY['email'::"text", 'sms'::"text"]))),
    CONSTRAINT "appointment_completion_notifications_status_check" CHECK
(("status" = ANY (ARRAY['pending'::"text", 'sending'::"text", 'sent'::"text",
'skipped'::"text"])))
);


ALTER TABLE "public"."appointment_completion_notifications" OWNER TO
"postgres";


CREATE TABLE IF NOT EXISTS "public"."appointment_confirmation_emails" (
    "id" "uuid" DEFAULT "gen_random_uuid"() NOT NULL,
    "appointment_id" "uuid" NOT NULL,
    "recipient_email" "text" NOT NULL,
    "status" "text" DEFAULT 'pending'::"text" NOT NULL,
    "attempt_count" integer DEFAULT 0 NOT NULL,
    "last_error" "text",
    "sent_at" timestamp with time zone,
    "created_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    "updated_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    CONSTRAINT "appointment_confirmation_emails_attempt_count_check"
CHECK (("attempt_count" >= 0)),
    CONSTRAINT "appointment_confirmation_emails_status_check" CHECK
(("status" = ANY (ARRAY['pending'::"text", 'sending'::"text", 'sent'::"text"])))
);


ALTER TABLE "public"."appointment_confirmation_emails" OWNER TO
"postgres";


CREATE TABLE IF NOT EXISTS "public"."appointment_confirmation_sms" (
    "id" "uuid" DEFAULT "gen_random_uuid"() NOT NULL,
    "appointment_id" "uuid" NOT NULL,
```

```
    "recipient_phone" "text" NOT NULL,
    "status" "text" DEFAULT 'pending'::"text" NOT NULL,
    "attempt_count" integer DEFAULT 0 NOT NULL,
    "last_error" "text",
    "sent_at" timestamp with time zone,
    "created_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    "updated_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    CONSTRAINT "appointment_confirmation_sms_attempt_count_check"
CHECK (("attempt_count" >= 0)),
    CONSTRAINT "appointment_confirmation_sms_status_check" CHECK
(("status" = ANY (ARRAY['pending'::"text", 'sending'::"text", 'sent'::"text",
'skipped'::"text"])))
);


ALTER TABLE "public"."appointment_confirmation_sms" OWNER TO
"postgres";


CREATE TABLE IF NOT EXISTS "public"."appointment_continuations" (
    "id" "uuid" DEFAULT "gen_random_uuid"() NOT NULL,
    "appointment_id" "uuid" NOT NULL,
    "segment_number" integer NOT NULL,
    "appointment_date" "date" NOT NULL,
    "start_time" time without time zone NOT NULL,
    "booking_duration_minutes" integer NOT NULL,
    "assigned_crew_id" "uuid" NOT NULL,
    "created_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    CONSTRAINT
"appointment_continuations_booking_duration_minutes_check" CHECK
(("booking_duration_minutes" > 0)),
    CONSTRAINT "appointment_continuations_segment_number_check" CHECK
(("segment_number" > 0))
);


ALTER TABLE "public"."appointment_continuations" OWNER TO "postgres";


CREATE TABLE IF NOT EXISTS "public"."appointment_products" (
    "id" "uuid" DEFAULT "gen_random_uuid"() NOT NULL,
    "appointment_id" "uuid" NOT NULL,
    "product_id" "uuid" NOT NULL,
    "product_name" "text" NOT NULL,
    "quantity" integer NOT NULL,
```

```
    "unit_price" numeric(10,2) NOT NULL,
    "created_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    "updated_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    CONSTRAINT "appointment_products_product_name_check" CHECK
((("btrim"("product_name") <> ''::"text")),
    CONSTRAINT "appointment_products_quantity_check" CHECK ((("quantity"
>= 1) AND ("quantity" <= 999))),
    CONSTRAINT "appointment_products_unit_price_check" CHECK
(("unit_price" >= (0)::numeric))
);


ALTER TABLE "public"."appointment_products" OWNER TO "postgres";


CREATE TABLE IF NOT EXISTS
"public"."appointment_reference_recovery_challenges" (
    "id" "uuid" DEFAULT "gen_random_uuid"() NOT NULL,
    "email" "text" NOT NULL,
    "code_hash" "text" NOT NULL,
    "expires_at" timestamp with time zone NOT NULL,
    "attempt_count" integer DEFAULT 0 NOT NULL,
    "verified_at" timestamp with time zone,
    "created_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    CONSTRAINT
"appointment_reference_recovery_challenges_attempt_count_check" CHECK
((("attempt_count" >= 0) AND ("attempt_count" <= 5))),
    CONSTRAINT
"appointment_reference_recovery_challenges_code_hash_check" CHECK
(("code_hash" ~ '^[a-f0-9]{64}$'::"text"))
);


ALTER TABLE "public"."appointment_reference_recovery_challenges" OWNER
TO "postgres";


CREATE TABLE IF NOT EXISTS "public"."appointment_reschedule_history" (
    "id" "uuid" DEFAULT "gen_random_uuid"() NOT NULL,
    "appointment_id" "uuid" NOT NULL,
    "new_appointment_id" "uuid",
    "reschedule_number" integer NOT NULL,
    "from_date" "date" NOT NULL,
    "from_start_time" time without time zone NOT NULL,
    "to_date" "date",
```

```
    "to_start_time" time without time zone,
    "reason" "text" NOT NULL,
    "decision" "text" NOT NULL,
    "approved_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    "approved_by" "uuid",
    CONSTRAINT "appointment_reschedule_history_decision_check" CHECK
(("decision" = ANY (ARRAY['confirmed'::"text", 'rejected'::"text"]))),
    CONSTRAINT "appointment_reschedule_history_reschedule_number_check"
CHECK ((("reschedule_number" >= 1) AND ("reschedule_number" <= 3)))
);


ALTER TABLE "public"."appointment_reschedule_history" OWNER TO
"postgres";


CREATE TABLE IF NOT EXISTS "public"."appointment_services" (
    "id" "uuid" DEFAULT "gen_random_uuid"() NOT NULL,
    "appointment_id" "uuid" NOT NULL,
    "service_id" "uuid",
    "service_name" "text" NOT NULL,
    "price" numeric(10,2) DEFAULT 0 NOT NULL,
    "duration_minutes" integer DEFAULT 60 NOT NULL,
    CONSTRAINT "appointment_services_duration_minutes_check" CHECK
(("duration_minutes" > 0))
);


ALTER TABLE "public"."appointment_services" OWNER TO "postgres";


CREATE TABLE IF NOT EXISTS "public"."appointments" (
    "id" "uuid" DEFAULT "gen_random_uuid"() NOT NULL,
    "reference_code" "text" NOT NULL,
    "customer_name" "text" NOT NULL,
    "phone" "text" NOT NULL,
    "moto_brand" "text" NOT NULL,
    "moto_model" "text" NOT NULL,
    "moto_variant" "text",
    "moto_year" integer,
    "plate_number" "text" NOT NULL,
    "appointment_date" "date" NOT NULL,
    "start_time" time without time zone NOT NULL,
    "status" "text" DEFAULT 'pending'::"text" NOT NULL,
    "notes" "text",
```

```
    "admin_notes" "text",
    "total_estimate" numeric(10,2) DEFAULT 0 NOT NULL,
    "assigned_crew_id" "uuid",
    "is_archived" boolean DEFAULT false NOT NULL,
    "terms_accepted" boolean DEFAULT false NOT NULL,
    "cancelled_at" timestamp with time zone,
    "created_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    "updated_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    "booking_duration_minutes" integer DEFAULT 60 NOT NULL,
    "booking_request_id" "uuid",
    "rescheduled_from_appointment_id" "uuid",
    "rescheduled_to_appointment_id" "uuid",
    "pending_reschedule_request_id" "uuid",
    "pending_reschedule_date" "date",
    "pending_reschedule_start_time" time without time zone,
    "pending_reschedule_reason" "text",
    "reschedule_count" integer DEFAULT 0 NOT NULL,
    "last_reschedule_rejected_at" timestamp with time zone,
    "last_reschedule_rejection_message" "text",
    "first_name" "text",
    "middle_name" "text",
    "last_name" "text",
    "pending_reschedule_reference_code" "text",
    "archived_at" timestamp with time zone,
    "service_started_at" timestamp with time zone,
    "service_ended_at" timestamp with time zone,
    "arrival_notification_snooze_count" integer DEFAULT 0 NOT NULL,
    "arrival_notification_snoozed_until" timestamp with time zone,
    "completion_notification_snoozed_until" timestamp with time zone,
    "no_show_count_at_booking" integer DEFAULT 0 NOT NULL,
    "email" "text",
    "wants_products" boolean DEFAULT false NOT NULL,
    CONSTRAINT "appointments_arrival_notification_snooze_count_check"
CHECK ((("arrival_notification_snooze_count" >= 0) AND
("arrival_notification_snooze_count" <= 3))),
    CONSTRAINT "appointments_booking_duration_minutes_check" CHECK
(("booking_duration_minutes" > 0)),
    CONSTRAINT "appointments_email_format_check" CHECK ((("email" IS
NULL) OR ("btrim"("email") ~*
'^[^[:space:]@]+@[^[:space:]@]+\.[^[:space:]@]+$'::"text"))),
    CONSTRAINT "appointments_no_show_count_at_booking_check" CHECK
(("no_show_count_at_booking" >= 0)),
    CONSTRAINT "appointments_reschedule_count_range" CHECK
((("reschedule_count" >= 0) AND ("reschedule_count" <= 3))),
    CONSTRAINT "appointments_service_progress_timestamp_check" CHECK
```

```
((("service_ended_at" IS NULL) OR (("service_started_at" IS NOT NULL) AND
("service_ended_at" >= "service_started_at"))))
);


ALTER TABLE "public"."appointments" OWNER TO "postgres";


COMMENT ON COLUMN "public"."appointments"."booking_duration_minutes"
IS 'Total duration of the selected service work, without an arrival or post-service
buffer.';



COMMENT ON COLUMN "public"."appointments"."service_started_at" IS 'Actual
server-recorded start time of the appointment service.';



COMMENT ON COLUMN "public"."appointments"."service_ended_at" IS 'Actual
server-recorded completion time of the appointment service.';



CREATE TABLE IF NOT EXISTS "public"."blocked_numbers" (
    "id" "uuid" DEFAULT "gen_random_uuid"() NOT NULL,
    "phone" "text" NOT NULL,
    "reason" "text",
    "created_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    "created_by" "uuid",
    "is_archived" boolean DEFAULT false NOT NULL,
    "archived_at" timestamp with time zone
);


ALTER TABLE "public"."blocked_numbers" OWNER TO "postgres";


CREATE TABLE IF NOT EXISTS "public"."crew_availability_exceptions" (
    "id" "uuid" DEFAULT "gen_random_uuid"() NOT NULL,
    "crew_id" "uuid" NOT NULL,
    "start_date" "date" NOT NULL,
    "end_date" "date" NOT NULL,
    "start_time" time without time zone,
    "end_time" time without time zone,
```

```
    "reason" "text" NOT NULL,
    "note" "text",
    "is_all_day" boolean DEFAULT true NOT NULL,
    "created_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    CONSTRAINT "crew_availability_exceptions_reason_check" CHECK
(("reason" = ANY (ARRAY['leave'::"text", 'sick'::"text", 'day_off'::"text",
'emergency'::"text", 'special_schedule'::"text"])))
);


ALTER TABLE "public"."crew_availability_exceptions" OWNER TO "postgres";


CREATE TABLE IF NOT EXISTS "public"."crew_members" (
    "id" "uuid" DEFAULT "gen_random_uuid"() NOT NULL,
    "name" "text" NOT NULL,
    "role" "text" DEFAULT 'Mechanic'::"text" NOT NULL,
    "phone" "text",
    "is_active" boolean DEFAULT true NOT NULL,
    "created_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    "updated_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    "is_archived" boolean DEFAULT false NOT NULL,
    "archived_at" timestamp with time zone
);


ALTER TABLE "public"."crew_members" OWNER TO "postgres";


CREATE TABLE IF NOT EXISTS "public"."crew_schedules" (
    "id" "uuid" DEFAULT "gen_random_uuid"() NOT NULL,
    "crew_id" "uuid" NOT NULL,
    "day_of_week" integer NOT NULL,
    "start_time" time without time zone,
    "end_time" time without time zone,
    "is_working" boolean DEFAULT true NOT NULL,
    "created_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    "schedule_date" "date",
    "effective_from" "date",
    "effective_to" "date",
    "note" "text",
    "updated_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    CONSTRAINT "crew_schedules_day_of_week_check" CHECK
((("day_of_week" >= 0) AND ("day_of_week" <= 6)))
);
```

```
ALTER TABLE "public"."crew_schedules" OWNER TO "postgres";


COMMENT ON COLUMN "public"."crew_schedules"."schedule_date" IS 'Specific
date for one-off schedule override. If NULL, the schedule is a weekly recurring
pattern based on day_of_week.';


COMMENT ON COLUMN "public"."crew_schedules"."effective_from" IS 'Start of
a date range during which this weekly pattern is active. If NULL, always active.';


COMMENT ON COLUMN "public"."crew_schedules"."effective_to" IS 'End of a
date range during which this weekly pattern is active. If NULL, no end date.';


COMMENT ON COLUMN "public"."crew_schedules"."note" IS 'Optional note for
the schedule entry, e.g. reason for unavailability.';


CREATE TABLE IF NOT EXISTS
"public"."customer_service_booking_alert_state" (
    "customer_phone" "text" NOT NULL,
    "service_key" "text" NOT NULL,
    "is_threshold_active" boolean DEFAULT false NOT NULL,
    "alert_cycle" integer DEFAULT 0 NOT NULL,
    "updated_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    CONSTRAINT "customer_service_booking_alert_state_alert_cycle_check"
CHECK (("alert_cycle" >= 0))
);


ALTER TABLE "public"."customer_service_booking_alert_state" OWNER TO
"postgres";


CREATE TABLE IF NOT EXISTS "public"."customer_service_booking_alerts" (
    "id" "uuid" DEFAULT "gen_random_uuid"() NOT NULL,
    "customer_phone" "text" NOT NULL,
```

```
    "service_key" "text" NOT NULL,
    "alert_cycle" integer NOT NULL,
    "active_appointment_ids" "uuid"[] NOT NULL,
    "notification_id" "uuid",
    "created_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    CONSTRAINT
"customer_service_booking_alerts_active_appointment_ids_check" CHECK
(("cardinality"("active_appointment_ids") >= 2)),
    CONSTRAINT "customer_service_booking_alerts_alert_cycle_check" CHECK
(("alert_cycle" > 0))
);


ALTER TABLE "public"."customer_service_booking_alerts" OWNER TO
"postgres";


CREATE TABLE IF NOT EXISTS "public"."motorcycle_catalog" (
    "id" "uuid" DEFAULT "gen_random_uuid"() NOT NULL,
    "brand" "text",
    "model" "text" NOT NULL,
    "is_active" boolean DEFAULT true NOT NULL,
    "is_archived" boolean DEFAULT false NOT NULL,
    "sort_order" integer DEFAULT 0 NOT NULL,
    "created_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    "updated_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    "archived_at" timestamp with time zone,
    "engine_cc" numeric(6,1),
    "cc_category" "text" GENERATED ALWAYS AS (
CASE
    WHEN ("engine_cc" IS NULL) THEN NULL::"text"
    WHEN ("engine_cc" <= (125)::numeric) THEN 'small_bike'::"text"
    ELSE 'big_bike'::"text"
END) STORED,
    CONSTRAINT "motorcycle_catalog_active_requires_engine_cc" CHECK
(((NOT "is_active") OR ("engine_cc" IS NOT NULL))),
    CONSTRAINT "motorcycle_catalog_engine_cc_positive" CHECK
((("engine_cc" IS NULL) OR ("engine_cc" > (0)::numeric)))
);


ALTER TABLE "public"."motorcycle_catalog" OWNER TO "postgres";


COMMENT ON COLUMN "public"."motorcycle_catalog"."cc_category" IS
```

'Derived from engine_cc: small_bike for 125cc and below; Large Bike above 125cc.';

```
CREATE TABLE IF NOT EXISTS "public"."notifications" (
    "id" "uuid" DEFAULT "gen_random_uuid"() NOT NULL,
    "type" "text" DEFAULT 'new_appointment'::"text" NOT NULL,
    "title" "text" NOT NULL,
    "message" "text",
    "appointment_id" "uuid",
    "is_read" boolean DEFAULT false NOT NULL,
    "created_at" timestamp with time zone DEFAULT "now"() NOT NULL
);


ALTER TABLE "public"."notifications" OWNER TO "postgres";


CREATE TABLE IF NOT EXISTS "public"."price_history" (
    "id" "uuid" DEFAULT "gen_random_uuid"() NOT NULL,
    "table_name" "text" NOT NULL,
    "item_id" "uuid" NOT NULL,
    "item_name" "text" NOT NULL,
    "old_price" numeric(10,2) NOT NULL,
    "new_price" numeric(10,2) NOT NULL,
    "reason" "text",
    "changed_by" "uuid",
    "created_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    "configuration_id" "uuid",
    "configuration_label" "text",
    "changed_by_email" "text"
);


ALTER TABLE "public"."price_history" OWNER TO "postgres";


CREATE TABLE IF NOT EXISTS "public"."products" (
    "id" "uuid" DEFAULT "gen_random_uuid"() NOT NULL,
    "name" "text" NOT NULL,
    "category" "text" DEFAULT 'part'::"text" NOT NULL,
    "brand" "text",
    "description" "text",
    "price" numeric(10,2) DEFAULT 0 NOT NULL,
```

```
    "image_url" "text",
    "in_stock" boolean DEFAULT true NOT NULL,
    "is_featured" boolean DEFAULT false NOT NULL,
    "is_active" boolean DEFAULT true NOT NULL,
    "sort_order" integer DEFAULT 0 NOT NULL,
    "created_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    "updated_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    "is_archived" boolean DEFAULT false NOT NULL,
    "stock_quantity" integer DEFAULT 0 NOT NULL,
    "archived_at" timestamp with time zone,
    CONSTRAINT "products_shop_category_check" CHECK (("category" = ANY
(ARRAY['part'::"text", 'accessory'::"text"]))),
    CONSTRAINT "products_stock_quantity_nonnegative" CHECK
(("stock_quantity" >= 0))
);


ALTER TABLE "public"."products" OWNER TO "postgres";


CREATE TABLE IF NOT EXISTS "public"."public_endpoint_rate_limits" (
    "scope" "text" NOT NULL,
    "subject" "text" NOT NULL,
    "window_started_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    "request_count" integer DEFAULT 1 NOT NULL,
    "updated_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    CONSTRAINT "public_endpoint_rate_limits_request_count_check" CHECK
(("request_count" > 0)),
    CONSTRAINT "public_endpoint_rate_limits_scope_check" CHECK
((("char_length"("scope") >= 1) AND ("char_length"("scope") <= 64))),
    CONSTRAINT "public_endpoint_rate_limits_subject_check" CHECK
((("char_length"("subject") >= 1) AND ("char_length"("subject") <= 128)))
);


ALTER TABLE "public"."public_endpoint_rate_limits" OWNER TO "postgres";


CREATE TABLE IF NOT EXISTS "public"."schedule_blocks" (
    "id" "uuid" DEFAULT "gen_random_uuid"() NOT NULL,
    "block_date" "date" NOT NULL,
    "start_time" time without time zone,
    "reason" "text",
    "created_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    "is_active" boolean DEFAULT true NOT NULL,
```

```
    "archived_at" timestamp with time zone
);


ALTER TABLE "public"."schedule_blocks" OWNER TO "postgres";


CREATE TABLE IF NOT EXISTS "public"."services" (
    "id" "uuid" DEFAULT "gen_random_uuid"() NOT NULL,
    "name" "text" NOT NULL,
    "description" "text" NOT NULL,
    "price" numeric(10,2) NOT NULL,
    "duration_minutes" integer NOT NULL,
    "category" "text" NOT NULL,
    "image_url" "text",
    "is_active" boolean DEFAULT true NOT NULL,
    "sort_order" integer DEFAULT 0 NOT NULL,
    "created_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    "updated_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    "is_archived" boolean DEFAULT false NOT NULL,
    "archived_at" timestamp with time zone,
    "large_bike_price" numeric(10,2),
    "large_bike_duration_minutes" integer,
    CONSTRAINT "services_category_required" CHECK
(("char_length"("btrim"("category")) >= 2)),
    CONSTRAINT "services_description_required" CHECK
(("char_length"("btrim"("description")) > 0)),
    CONSTRAINT "services_duration_valid" CHECK ((("duration_minutes" >= 15)
AND ("duration_minutes" <= 480))),
    CONSTRAINT "services_large_bike_configuration_complete" CHECK
((((("large_bike_price" IS NULL) AND ("large_bike_duration_minutes" IS NULL))
OR (("large_bike_price" > "price") AND (("large_bike_duration_minutes" >= 15)
AND ("large_bike_duration_minutes" <= 480)) AND
("large_bike_duration_minutes" >= "duration_minutes")))),
    CONSTRAINT "services_name_required" CHECK
(("char_length"("btrim"("name")) >= 2)),
    CONSTRAINT "services_price_valid" CHECK (("price" >= (0)::numeric))
);


ALTER TABLE "public"."services" OWNER TO "postgres";


COMMENT ON COLUMN "public"."services"."price" IS 'Small Bike service price
for motorcycles at 125cc and below.';
```

COMMENT ON COLUMN "public"."services"."duration_minutes" IS 'Small Bike service duration for motorcycles at 125cc and below.';

COMMENT ON COLUMN "public"."services"."large_bike_price" IS 'Large Bike service price for motorcycles above 125cc; must exceed the Small Bike price.';

COMMENT ON COLUMN "public"."services"."large_bike_duration_minutes" IS 'Large Bike service duration for motorcycles above 125cc.';

CREATE TABLE IF NOT EXISTS "public"."shop_settings" (
    "id" boolean DEFAULT true NOT NULL,
    "shop_name" "text" DEFAULT 'Fake Rider Motorparts'::"text" NOT NULL,
    "address" "text" DEFAULT 'Purok Bangkal Sta. Cruz, Baclayon, Bohol, Philippines'::"text" NOT NULL,
    "contact_number" "text" DEFAULT '0916 126 3317'::"text" NOT NULL,
    "contact_email" "text" DEFAULT 'joemartato4@gmail.com'::"text" NOT NULL,
    "booking_terms" "text" DEFAULT 'Bookings are subject to shop confirmation. Please arrive 15 minutes before your slot. Late arrivals beyond 30 minutes may be rescheduled. Quoted prices are starting rates; parts and additional labor are billed separately. The shop is not liable for personal items left on the unit.

Cancellations must be made at least 48 hours before the schedule.

We use your name, contact details, motorcycle details, selected services, and notes only to manage this booking, contact you about it, and provide shop services. We do not sell your information.'::"text" NOT NULL,
    "logo_url" "text",
    "minimum_booking_lead_hours" integer DEFAULT 48 NOT NULL,
    "max_advance_booking_days" integer DEFAULT 30 NOT NULL,
    "default_appointment_duration_minutes" integer DEFAULT 60 NOT NULL,
    "allow_same_day_appointments" boolean DEFAULT false NOT NULL,
    "cancellation_notice_hours" integer DEFAULT 48 NOT NULL,
    "rescheduling_notice_hours" integer DEFAULT 48 NOT NULL,
    "notify_confirmation" boolean DEFAULT true NOT NULL,
    "notify_cancellation" boolean DEFAULT true NOT NULL,
    "notify_reminder" boolean DEFAULT true NOT NULL,

```
    "reminder_hours_before" integer DEFAULT 24 NOT NULL,
    "notify_admin" boolean DEFAULT true NOT NULL,
    "updated_at" timestamp with time zone DEFAULT "now"() NOT NULL,
    "opening_time" time without time zone DEFAULT '08:00:00'::time without time
zone NOT NULL,
    "closing_time" time without time zone DEFAULT '17:00:00'::time without time
zone NOT NULL,
    CONSTRAINT "shop_settings_cancellation_notice_hours_check" CHECK
((("cancellation_notice_hours" >= 0) AND ("cancellation_notice_hours" <= 168))),
    CONSTRAINT "shop_settings_default_appointment_duration_minutes_check"
CHECK ((("default_appointment_duration_minutes" >= 15) AND
("default_appointment_duration_minutes" <= 600))),
    CONSTRAINT "shop_settings_id_check" CHECK ("id"),
    CONSTRAINT "shop_settings_max_advance_booking_days_check" CHECK
((("max_advance_booking_days" >= 1) AND ("max_advance_booking_days" <=
365))),
    CONSTRAINT "shop_settings_minimum_booking_lead_hours_check" CHECK
((("minimum_booking_lead_hours" >= 0) AND ("minimum_booking_lead_hours"
<= 168))),
    CONSTRAINT "shop_settings_operating_hours_valid" CHECK
((("opening_time" < "closing_time") AND (EXTRACT(minute FROM
"opening_time") = ANY (ARRAY[(0)::numeric, (30)::numeric])) AND
(EXTRACT(minute FROM "closing_time") = ANY (ARRAY[(0)::numeric,
(30)::numeric])) AND (EXTRACT(second FROM "opening_time") = (0)::numeric)
AND (EXTRACT(second FROM "closing_time") = (0)::numeric))),
    CONSTRAINT "shop_settings_reminder_hours_before_check" CHECK
((("reminder_hours_before" >= 1) AND ("reminder_hours_before" <= 168))),
    CONSTRAINT "shop_settings_rescheduling_notice_hours_check" CHECK
((("rescheduling_notice_hours" >= 0) AND ("rescheduling_notice_hours" <=
168)))
);


ALTER TABLE "public"."shop_settings" OWNER TO "postgres";


CREATE TABLE IF NOT EXISTS "public"."time_slots" (
    "id" "uuid" DEFAULT "gen_random_uuid"() NOT NULL,
    "start_time" time without time zone NOT NULL,
    "end_time" time without time zone NOT NULL,
    "capacity" integer DEFAULT 2 NOT NULL,
    "is_active" boolean DEFAULT true NOT NULL,
    "created_at" timestamp with time zone DEFAULT "now"() NOT NULL
);
```

```
ALTER TABLE "public"."time_slots" OWNER TO "postgres";


CREATE TABLE IF NOT EXISTS "public"."user_roles" (
    "id" "uuid" DEFAULT "gen_random_uuid"() NOT NULL,
    "user_id" "uuid" NOT NULL,
    "role" "public"."app_role" NOT NULL,
    "created_at" timestamp with time zone DEFAULT "now"() NOT NULL
);


ALTER TABLE "public"."user_roles" OWNER TO "postgres";


ALTER TABLE ONLY "public"."admin_activity_logs"
    ADD CONSTRAINT "admin_activity_logs_pkey" PRIMARY KEY ("id");



ALTER TABLE ONLY "public"."admin_session_handover_tokens"
    ADD CONSTRAINT "admin_session_handover_tokens_pkey" PRIMARY KEY
("token");



ALTER TABLE ONLY "public"."admin_session_leases"
    ADD CONSTRAINT "admin_session_leases_pkey" PRIMARY KEY
("admin_id");



ALTER TABLE ONLY "public"."appointment_action_verifications"
    ADD CONSTRAINT "appointment_action_verifications_pkey" PRIMARY KEY
("id");



ALTER TABLE ONLY "public"."appointment_action_verifications"
    ADD CONSTRAINT "appointment_action_verifications_token_hash_key"
UNIQUE ("token_hash");



ALTER TABLE ONLY "public"."appointment_completion_notifications"
```

```
    ADD CONSTRAINT
"appointment_completion_notifications_appointment_id_channel_key" UNIQUE
("appointment_id", "channel");


ALTER TABLE ONLY "public"."appointment_completion_notifications"
    ADD CONSTRAINT "appointment_completion_notifications_pkey" PRIMARY
KEY ("id");


ALTER TABLE ONLY "public"."appointment_confirmation_emails"
    ADD CONSTRAINT "appointment_confirmation_emails_appointment_id_key"
UNIQUE ("appointment_id");


ALTER TABLE ONLY "public"."appointment_confirmation_emails"
    ADD CONSTRAINT "appointment_confirmation_emails_pkey" PRIMARY KEY
("id");


ALTER TABLE ONLY "public"."appointment_confirmation_sms"
    ADD CONSTRAINT "appointment_confirmation_sms_appointment_id_key"
UNIQUE ("appointment_id");


ALTER TABLE ONLY "public"."appointment_confirmation_sms"
    ADD CONSTRAINT "appointment_confirmation_sms_pkey" PRIMARY KEY
("id");


ALTER TABLE ONLY "public"."appointment_continuations"
    ADD CONSTRAINT
"appointment_continuations_appointment_id_segment_number_key" UNIQUE
("appointment_id", "segment_number");


ALTER TABLE ONLY "public"."appointment_continuations"
    ADD CONSTRAINT "appointment_continuations_pkey" PRIMARY KEY ("id");
```

```
ALTER TABLE ONLY "public"."appointment_products"
    ADD CONSTRAINT "appointment_products_appointment_id_product_id_key"
UNIQUE ("appointment_id", "product_id");



ALTER TABLE ONLY "public"."appointment_products"
    ADD CONSTRAINT "appointment_products_pkey" PRIMARY KEY ("id");



ALTER TABLE ONLY "public"."appointment_reference_recovery_challenges"
    ADD CONSTRAINT "appointment_reference_recovery_challenges_pkey"
PRIMARY KEY ("id");



ALTER TABLE ONLY "public"."appointment_reschedule_history"
    ADD CONSTRAINT "appointment_reschedule_history_pkey" PRIMARY KEY
("id");



ALTER TABLE ONLY "public"."appointment_services"
    ADD CONSTRAINT "appointment_services_pkey" PRIMARY KEY ("id");



ALTER TABLE ONLY "public"."appointments"
    ADD CONSTRAINT "appointments_crew_time_no_overlap" EXCLUDE
USING "gist" ("assigned_crew_id" WITH =, "tsrange"(("appointment_date" +
"start_time"), (("appointment_date" + "start_time") + "make_interval"("mins" =>
"booking_duration_minutes")), '[)'::"text") WITH &&) WHERE ((("is_archived" =
false) AND ("rescheduled_to_appointment_id" IS NULL) AND ("status" = ANY
(ARRAY['pending'::"text", 'confirmed'::"text", 'in_progress'::"text",
'rescheduled'::"text"])))));



ALTER TABLE "public"."appointments"
    ADD CONSTRAINT "appointments_phone_local_check" CHECK (("phone" ~
'^09[0-9]{9}$'::"text")) NOT VALID;
```

```
ALTER TABLE ONLY "public"."appointments"
    ADD CONSTRAINT "appointments_pkey" PRIMARY KEY ("id");


ALTER TABLE ONLY "public"."appointments"
    ADD CONSTRAINT "appointments_reference_code_key" UNIQUE
("reference_code");


ALTER TABLE "public"."blocked_numbers"
    ADD CONSTRAINT "blocked_numbers_phone_local_check" CHECK
(("phone" ~ '^09[0-9]{9}$'::"text")) NOT VALID;


ALTER TABLE ONLY "public"."blocked_numbers"
    ADD CONSTRAINT "blocked_numbers_pkey" PRIMARY KEY ("id");


ALTER TABLE ONLY "public"."crew_availability_exceptions"
    ADD CONSTRAINT "crew_availability_exceptions_pkey" PRIMARY KEY
("id");


ALTER TABLE "public"."crew_members"
    ADD CONSTRAINT "crew_members_phone_local_check" CHECK ((("phone"
IS NULL) OR ("phone" ~ '^09[0-9]{9}$'::"text"))) NOT VALID;


ALTER TABLE ONLY "public"."crew_members"
    ADD CONSTRAINT "crew_members_pkey" PRIMARY KEY ("id");


ALTER TABLE ONLY "public"."crew_schedules"
    ADD CONSTRAINT "crew_schedules_pkey" PRIMARY KEY ("id");
```

```
ALTER TABLE ONLY "public"."customer_service_booking_alerts"
    ADD CONSTRAINT
"customer_service_booking_aler_customer_phone_service_key_al_key"
UNIQUE ("customer_phone", "service_key", "alert_cycle",
"active_appointment_ids");



ALTER TABLE ONLY "public"."customer_service_booking_alert_state"
    ADD CONSTRAINT "customer_service_booking_alert_state_pkey" PRIMARY
KEY ("customer_phone", "service_key");



ALTER TABLE ONLY "public"."customer_service_booking_alerts"
    ADD CONSTRAINT "customer_service_booking_alerts_pkey" PRIMARY KEY
("id");



ALTER TABLE ONLY "public"."motorcycle_catalog"
    ADD CONSTRAINT "motorcycle_catalog_pkey" PRIMARY KEY ("id");



ALTER TABLE ONLY "public"."notifications"
    ADD CONSTRAINT "notifications_pkey" PRIMARY KEY ("id");



ALTER TABLE ONLY "public"."price_history"
    ADD CONSTRAINT "price_history_pkey" PRIMARY KEY ("id");



ALTER TABLE ONLY "public"."products"
    ADD CONSTRAINT "products_pkey" PRIMARY KEY ("id");



ALTER TABLE ONLY "public"."public_endpoint_rate_limits"
    ADD CONSTRAINT "public_endpoint_rate_limits_pkey" PRIMARY KEY
("scope", "subject");
```

```
ALTER TABLE ONLY "public"."schedule_blocks"
    ADD CONSTRAINT "schedule_blocks_pkey" PRIMARY KEY ("id");


ALTER TABLE ONLY "public"."services"
    ADD CONSTRAINT "services_pkey" PRIMARY KEY ("id");


ALTER TABLE ONLY "public"."shop_settings"
    ADD CONSTRAINT "shop_settings_pkey" PRIMARY KEY ("id");


ALTER TABLE ONLY "public"."time_slots"
    ADD CONSTRAINT "time_slots_pkey" PRIMARY KEY ("id");


ALTER TABLE ONLY "public"."time_slots"
    ADD CONSTRAINT "time_slots_start_time_key" UNIQUE ("start_time");


ALTER TABLE ONLY "public"."user_roles"
    ADD CONSTRAINT "user_roles_pkey" PRIMARY KEY ("id");


ALTER TABLE ONLY "public"."user_roles"
    ADD CONSTRAINT "user_roles_user_id_role_key" UNIQUE ("user_id",
"role");


CREATE INDEX "admin_activity_logs_created_at_idx" ON
"public"."admin_activity_logs" USING "btree" ("created_at" DESC);


CREATE INDEX "admin_activity_logs_filter_idx" ON
```

"public"."admin_activity_logs" USING "btree" ("activity_date" DESC, "activity_time" DESC, "action", "resource_type");

CREATE INDEX "admin_activity_logs_record_id_idx" ON "public"."admin_activity_logs" USING "btree" ("record_id");

CREATE INDEX "admin_session_handover_tokens_expiry_idx" ON "public"."admin_session_handover_tokens" USING "btree" ("expires_at");

CREATE INDEX "appointment_action_verifications_lookup_idx" ON "public"."appointment_action_verifications" USING "btree" ("appointment_id", "action", "created_at" DESC);

CREATE INDEX "appointment_continuations_crew_schedule_idx" ON "public"."appointment_continuations" USING "btree" ("assigned_crew_id", "appointment_date", "start_time");

CREATE INDEX "appointment_continuations_schedule_idx" ON "public"."appointment_continuations" USING "btree" ("appointment_date", "start_time");

CREATE INDEX "appointment_products_appointment_id_idx" ON "public"."appointment_products" USING "btree" ("appointment_id");

CREATE INDEX "appointment_reference_recovery_challenges_email_created_idx" ON "public"."appointment_reference_recovery_challenges" USING "btree" ("email", "created_at" DESC);

```
CREATE INDEX "appointment_reschedule_history_appointment_id_idx" ON
"public"."appointment_reschedule_history" USING "btree" ("appointment_id");


CREATE INDEX "appointment_reschedule_history_new_appointment_id_idx"
ON "public"."appointment_reschedule_history" USING "btree"
("new_appointment_id");


CREATE INDEX "appointment_reschedule_history_reschedule_number_idx" ON
"public"."appointment_reschedule_history" USING "btree" ("appointment_id",
"reschedule_number");


CREATE INDEX "appointment_services_appointment_id_idx" ON
"public"."appointment_services" USING "btree" ("appointment_id");


CREATE INDEX "appointment_services_service_appointment_idx" ON
"public"."appointment_services" USING "btree" ("service_id", "appointment_id");


CREATE INDEX "appointments_active_customer_date_idx" ON
"public"."appointments" USING "btree" ("phone", "appointment_date") WHERE
(("is_archived" = false) AND ("rescheduled_to_appointment_id" IS NULL) AND
("status" <> ALL (ARRAY['completed'::"text", 'cancelled'::"text",
'rejected'::"text"])));


CREATE INDEX "appointments_active_email_created_idx" ON
"public"."appointments" USING "btree" ("email", "created_at" DESC) WHERE
(("is_archived" = false) AND ("email" IS NOT NULL));


CREATE INDEX "appointments_admin_list_idx" ON "public"."appointments"
USING "btree" ("is_archived", "appointment_date" DESC, "start_time");
```

```
CREATE INDEX "appointments_archived_retention_idx" ON
"public"."appointments" USING "btree" ("archived_at") WHERE ("is_archived" =
true);


CREATE UNIQUE INDEX "appointments_booking_request_id_unique" ON
"public"."appointments" USING "btree" ("booking_request_id") WHERE
("booking_request_id" IS NOT NULL);


CREATE INDEX "appointments_cancelled_phone_month_idx" ON
"public"."appointments" USING "btree" ("phone", "cancelled_at") WHERE
(("status" = 'cancelled'::"text") AND ("cancelled_at" IS NOT NULL));


CREATE INDEX "appointments_customer_history_idx" ON
"public"."appointments" USING "btree" ("phone", "appointment_date" DESC,
"start_time");


CREATE INDEX "appointments_date_idx" ON "public"."appointments" USING
"btree" ("appointment_date", "start_time");


CREATE UNIQUE INDEX
"appointments_pending_reschedule_reference_code_unique" ON
"public"."appointments" USING "btree" ("pending_reschedule_reference_code")
WHERE ("pending_reschedule_reference_code" IS NOT NULL);


CREATE UNIQUE INDEX
"appointments_pending_reschedule_request_id_unique" ON
"public"."appointments" USING "btree" ("pending_reschedule_request_id")
WHERE ("pending_reschedule_request_id" IS NOT NULL);


CREATE INDEX "appointments_rescheduled_from_idx" ON
```

"public"."appointments" USING "btree" ("rescheduled_from_appointment_id");

CREATE INDEX "appointments_rescheduled_to_idx" ON "public"."appointments" USING "btree" ("rescheduled_to_appointment_id");

CREATE INDEX "appointments_service_progress_arrival_idx" ON "public"."appointments" USING "btree" ("appointment_date", "start_time") WHERE (("is_archived" = false) AND ("status" = 'confirmed'::"text") AND ("service_started_at" IS NULL));

CREATE INDEX "appointments_service_progress_completion_idx" ON "public"."appointments" USING "btree" ("service_started_at") WHERE (("is_archived" = false) AND ("status" = 'in_progress'::"text") AND ("service_started_at" IS NOT NULL) AND ("service_ended_at" IS NULL));

CREATE INDEX "blocked_numbers_archive_list_idx" ON "public"."blocked_numbers" USING "btree" ("is_archived", "created_at" DESC);

CREATE INDEX "blocked_numbers_archived_retention_idx" ON "public"."blocked_numbers" USING "btree" ("archived_at") WHERE ("is_archived" = true);

CREATE INDEX "blocked_numbers_is_archived_idx" ON "public"."blocked_numbers" USING "btree" ("is_archived");

CREATE UNIQUE INDEX "blocked_numbers_phone_idx" ON "public"."blocked_numbers" USING "btree" ("lower"("phone"));

CREATE INDEX "crew_members_archive_list_idx" ON "public"."crew_members"

```
USING "btree" ("is_archived", "name");


CREATE INDEX "crew_members_archived_retention_idx" ON
"public"."crew_members" USING "btree" ("archived_at") WHERE ("is_archived" =
true);


CREATE INDEX "crew_members_is_archived_idx" ON "public"."crew_members"
USING "btree" ("is_archived");


CREATE UNIQUE INDEX "crew_members_name_idx" ON
"public"."crew_members" USING "btree" ("name");


CREATE INDEX "crew_schedules_date_idx" ON "public"."crew_schedules"
USING "btree" ("schedule_date");


CREATE UNIQUE INDEX "crew_schedules_date_override_idx" ON
"public"."crew_schedules" USING "btree" ("crew_id", "schedule_date");


CREATE INDEX "crew_schedules_effective_idx" ON "public"."crew_schedules"
USING "btree" ("effective_from", "effective_to");


CREATE UNIQUE INDEX "crew_schedules_weekly_unique" ON
"public"."crew_schedules" USING "btree" ("crew_id", "day_of_week") WHERE
("schedule_date" IS NULL);


CREATE INDEX "motorcycle_catalog_active_list_idx" ON
"public"."motorcycle_catalog" USING "btree" ("brand", "model") WHERE
(("is_active" = true) AND ("is_archived" = false));
```

```
CREATE INDEX "motorcycle_catalog_archive_list_idx" ON
"public"."motorcycle_catalog" USING "btree" ("is_archived", "brand", "model");


CREATE INDEX "motorcycle_catalog_archived_retention_idx" ON
"public"."motorcycle_catalog" USING "btree" ("archived_at") WHERE
("is_archived" = true);


CREATE UNIQUE INDEX "motorcycle_catalog_brand_model_unique" ON
"public"."motorcycle_catalog" USING "btree" ("brand", "model") NULLS NOT
DISTINCT;


CREATE UNIQUE INDEX "notifications_active_service_progress_unique" ON
"public"."notifications" USING "btree" ("appointment_id", "type") WHERE
(("is_read" = false) AND ("type" = ANY (ARRAY['service_arrival'::"text",
'service_completion'::"text"])));


CREATE INDEX "price_history_configuration_idx" ON "public"."price_history"
USING "btree" ("table_name", "item_id", "configuration_id", "created_at" DESC);


CREATE INDEX "price_history_table_item_idx" ON "public"."price_history"
USING "btree" ("table_name", "item_id");


CREATE INDEX "products_archive_list_idx" ON "public"."products" USING
"btree" ("is_archived", "category", "sort_order");


CREATE INDEX "products_archived_retention_idx" ON "public"."products"
USING "btree" ("archived_at") WHERE ("is_archived" = true);
```

```
CREATE INDEX "products_is_archived_idx" ON "public"."products" USING "btree" ("is_archived");


CREATE UNIQUE INDEX "products_name_idx" ON "public"."products" USING "btree" ("name");


CREATE INDEX "schedule_blocks_archive_list_idx" ON "public"."schedule_blocks" USING "btree" ("is_active", "block_date" DESC);


CREATE INDEX "schedule_blocks_archived_retention_idx" ON "public"."schedule_blocks" USING "btree" ("archived_at") WHERE ("is_active" = false);


CREATE INDEX "services_archived_retention_idx" ON "public"."services" USING "btree" ("archived_at") WHERE ("is_archived" = true);


CREATE INDEX "services_category_idx" ON "public"."services" USING "btree" ("category");


CREATE INDEX "services_is_archived_idx" ON "public"."services" USING "btree" ("is_archived");


CREATE UNIQUE INDEX "services_name_idx" ON "public"."services" USING "btree" ("name");


CREATE OR REPLACE TRIGGER "admin_activity_appointment_services" AFTER INSERT OR DELETE OR UPDATE ON "public"."appointment_services" FOR EACH ROW EXECUTE FUNCTION "public"."record_admin_activity"();
```

```
CREATE OR REPLACE TRIGGER "admin_activity_appointments" AFTER
INSERT OR DELETE OR UPDATE ON "public"."appointments" FOR EACH
ROW EXECUTE FUNCTION "public"."record_admin_activity"();


CREATE OR REPLACE TRIGGER "admin_activity_blocked_numbers" AFTER
INSERT OR DELETE OR UPDATE ON "public"."blocked_numbers" FOR EACH
ROW EXECUTE FUNCTION "public"."record_admin_activity"();


CREATE OR REPLACE TRIGGER
"admin_activity_crew_availability_exceptions" AFTER INSERT OR DELETE OR
UPDATE ON "public"."crew_availability_exceptions" FOR EACH ROW
EXECUTE FUNCTION "public"."record_admin_activity"();


CREATE OR REPLACE TRIGGER "admin_activity_crew_members" AFTER
INSERT OR DELETE OR UPDATE ON "public"."crew_members" FOR EACH
ROW EXECUTE FUNCTION "public"."record_admin_activity"();


CREATE OR REPLACE TRIGGER "admin_activity_crew_schedules" AFTER
INSERT OR DELETE OR UPDATE ON "public"."crew_schedules" FOR EACH
ROW EXECUTE FUNCTION "public"."record_admin_activity"();


CREATE OR REPLACE TRIGGER "admin_activity_motorcycle_catalog" AFTER
INSERT OR DELETE OR UPDATE ON "public"."motorcycle_catalog" FOR
EACH ROW EXECUTE FUNCTION "public"."record_admin_activity"();


CREATE OR REPLACE TRIGGER "admin_activity_notifications" AFTER
INSERT OR UPDATE ON "public"."notifications" FOR EACH ROW WHEN
(("new"."type" <> ALL (ARRAY['service_arrival'::"text",
'service_completion'::"text"]))) EXECUTE FUNCTION
"public"."record_admin_activity"();
```

```
CREATE OR REPLACE TRIGGER "admin_activity_notifications_delete" AFTER DELETE ON "public"."notifications" FOR EACH ROW WHEN (("old"."type" <> ALL (ARRAY['service_arrival'::"text", 'service_completion'::"text"]))) EXECUTE FUNCTION "public"."record_admin_activity"();


CREATE OR REPLACE TRIGGER "admin_activity_price_history" AFTER INSERT OR DELETE OR UPDATE ON "public"."price_history" FOR EACH ROW EXECUTE FUNCTION "public"."record_admin_activity"();


CREATE OR REPLACE TRIGGER "admin_activity_products" AFTER INSERT OR DELETE OR UPDATE ON "public"."products" FOR EACH ROW EXECUTE FUNCTION "public"."record_admin_activity"();


CREATE OR REPLACE TRIGGER "admin_activity_schedule_blocks" AFTER INSERT OR DELETE OR UPDATE ON "public"."schedule_blocks" FOR EACH ROW EXECUTE FUNCTION "public"."record_admin_activity"();


CREATE OR REPLACE TRIGGER "admin_activity_services" AFTER INSERT OR DELETE OR UPDATE ON "public"."services" FOR EACH ROW EXECUTE FUNCTION "public"."record_admin_activity"();


CREATE OR REPLACE TRIGGER "admin_activity_time_slots" AFTER INSERT OR DELETE OR UPDATE ON "public"."time_slots" FOR EACH ROW EXECUTE FUNCTION "public"."record_admin_activity"();


CREATE OR REPLACE TRIGGER "admin_activity_user_roles" AFTER INSERT OR DELETE OR UPDATE ON "public"."user_roles" FOR EACH ROW EXECUTE FUNCTION "public"."record_admin_activity"();
```

```
CREATE OR REPLACE TRIGGER
"appointment_continuations_enforce_active_capacity" BEFORE INSERT OR
UPDATE OF "appointment_date", "start_time", "booking_duration_minutes",
"assigned_crew_id" ON "public"."appointment_continuations" FOR EACH ROW
EXECUTE FUNCTION
"public"."enforce_active_appointment_continuation_capacity"();
```

```
CREATE OR REPLACE TRIGGER "appointment_products_updated" BEFORE
UPDATE ON "public"."appointment_products" FOR EACH ROW EXECUTE
FUNCTION "public"."update_updated_at_column"();
```

```
CREATE CONSTRAINT TRIGGER
"appointment_services_prevent_duplicate_active_customer_services" AFTER
INSERT OR UPDATE ON "public"."appointment_services" DEFERRABLE
INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION
"public"."prevent_duplicate_active_customer_service_from_service"();
```

```
CREATE CONSTRAINT TRIGGER
"appointment_services_sync_customer_service_booking_alert" AFTER INSERT
OR DELETE OR UPDATE ON "public"."appointment_services" DEFERRABLE
INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION
"public"."sync_customer_service_booking_alert"();
```

```
CREATE OR REPLACE TRIGGER "appointments_archive_timestamp" BEFORE
UPDATE OF "is_archived" ON "public"."appointments" FOR EACH ROW
EXECUTE FUNCTION "public"."set_catalog_archive_timestamp"();
```

```
CREATE OR REPLACE TRIGGER
"appointments_capture_customer_no_show_count" BEFORE INSERT ON
"public"."appointments" FOR EACH ROW EXECUTE FUNCTION
"public"."capture_customer_no_show_count_at_booking"();
```

```
CREATE OR REPLACE TRIGGER "appointments_enforce_active_capacity"
BEFORE INSERT OR UPDATE OF "appointment_date", "start_time",
"booking_duration_minutes", "assigned_crew_id", "status", "is_archived",
"rescheduled_to_appointment_id" ON "public"."appointments" FOR EACH ROW
EXECUTE FUNCTION "public"."enforce_active_appointment_capacity"();


CREATE OR REPLACE TRIGGER
"appointments_enforce_active_crew_conflict" BEFORE INSERT OR UPDATE
OF "appointment_date", "start_time", "booking_duration_minutes",
"assigned_crew_id", "status", "is_archived", "rescheduled_to_appointment_id"
ON "public"."appointments" FOR EACH ROW EXECUTE FUNCTION
"public"."enforce_active_appointment_crew_conflict"();


CREATE OR REPLACE TRIGGER
"appointments_enforce_customer_upcoming_limit" BEFORE INSERT ON
"public"."appointments" FOR EACH ROW EXECUTE FUNCTION
"public"."enforce_customer_upcoming_appointment_limit"();


CREATE OR REPLACE TRIGGER
"appointments_enforce_reschedule_request_capacity" BEFORE UPDATE OF
"pending_reschedule_request_id", "pending_reschedule_date",
"pending_reschedule_start_time" ON "public"."appointments" FOR EACH ROW
EXECUTE FUNCTION "public"."enforce_reschedule_request_capacity"();


CREATE OR REPLACE TRIGGER "appointments_inherit_rescheduled_email"
BEFORE INSERT ON "public"."appointments" FOR EACH ROW EXECUTE
FUNCTION "public"."inherit_rescheduled_appointment_email"();


CREATE OR REPLACE TRIGGER
"appointments_notify_customer_no_show_threshold" AFTER INSERT OR
UPDATE OF "status" ON "public"."appointments" FOR EACH ROW EXECUTE
FUNCTION "public"."notify_customer_no_show_threshold"();
```

```
CREATE CONSTRAINT TRIGGER
"appointments_prevent_duplicate_active_customer_services" AFTER INSERT
OR UPDATE ON "public"."appointments" DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW EXECUTE FUNCTION
"public"."prevent_duplicate_active_customer_service_from_appointment"();
```

```
CREATE OR REPLACE TRIGGER "appointments_protect_completed" BEFORE
DELETE OR UPDATE OF "is_archived", "status" ON "public"."appointments"
FOR EACH ROW EXECUTE FUNCTION
"public"."protect_completed_appointments"();
```

```
CREATE OR REPLACE TRIGGER
"appointments_queue_completion_notifications" AFTER UPDATE OF "status"
ON "public"."appointments" FOR EACH ROW EXECUTE FUNCTION
"public"."queue_appointment_completion_notifications"();
```

```
CREATE OR REPLACE TRIGGER "appointments_queue_confirmation_email"
AFTER UPDATE OF "status" ON "public"."appointments" FOR EACH ROW
EXECUTE FUNCTION "public"."queue_appointment_confirmation_email"();
```

```
CREATE OR REPLACE TRIGGER
"appointments_queue_confirmation_email_on_insert" AFTER INSERT ON
"public"."appointments" FOR EACH ROW EXECUTE FUNCTION
"public"."queue_appointment_confirmation_email"();
```

```
CREATE OR REPLACE TRIGGER "appointments_queue_confirmation_sms"
AFTER UPDATE OF "status" ON "public"."appointments" FOR EACH ROW
EXECUTE FUNCTION "public"."queue_appointment_confirmation_sms"();
```

```
CREATE OR REPLACE TRIGGER
"appointments_queue_confirmation_sms_on_insert" AFTER INSERT ON
"public"."appointments" FOR EACH ROW EXECUTE FUNCTION
"public"."queue_appointment_confirmation_sms"();
```

CREATE OR REPLACE TRIGGER "appointments_release_pre_service_product_reservations" BEFORE UPDATE OF "status" ON "public"."appointments" FOR EACH ROW EXECUTE FUNCTION "public"."release_pre_service_product_reservations"();

CREATE CONSTRAINT TRIGGER "appointments_sync_customer_service_booking_alert" AFTER INSERT OR DELETE OR UPDATE ON "public"."appointments" DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION "public"."sync_customer_service_booking_alert"();

CREATE OR REPLACE TRIGGER "appointments_track_reschedule" BEFORE UPDATE OF "appointment_date", "start_time", "pending_reschedule_request_id" ON "public"."appointments" FOR EACH ROW EXECUTE FUNCTION "public"."track_appointment_reschedule"();

CREATE OR REPLACE TRIGGER "appointments_track_service_progress" BEFORE UPDATE OF "status" ON "public"."appointments" FOR EACH ROW EXECUTE FUNCTION "public"."track_appointment_service_progress"();

CREATE OR REPLACE TRIGGER "appointments_updated" BEFORE UPDATE ON "public"."appointments" FOR EACH ROW EXECUTE FUNCTION "public"."update_updated_at_column"();

CREATE OR REPLACE TRIGGER "appointments_validate_completion_transition" BEFORE UPDATE OF "status" ON "public"."appointments" FOR EACH ROW EXECUTE FUNCTION "public"."validate_appointment_completion_transition"();

CREATE OR REPLACE TRIGGER "blocked_numbers_archive_timestamp"

```
BEFORE UPDATE OF "is_archived" ON "public"."blocked_numbers" FOR EACH
ROW EXECUTE FUNCTION "public"."set_catalog_archive_timestamp"();


CREATE OR REPLACE TRIGGER "crew_members_archive_timestamp"
BEFORE UPDATE OF "is_archived" ON "public"."crew_members" FOR EACH
ROW EXECUTE FUNCTION "public"."set_catalog_archive_timestamp"();


CREATE OR REPLACE TRIGGER "crew_schedules_updated" BEFORE
UPDATE ON "public"."crew_schedules" FOR EACH ROW EXECUTE
FUNCTION "public"."update_updated_at_column"();


CREATE OR REPLACE TRIGGER "motorcycle_catalog_archive_timestamp"
BEFORE UPDATE ON "public"."motorcycle_catalog" FOR EACH ROW
EXECUTE FUNCTION "public"."set_catalog_archive_timestamp"();


CREATE OR REPLACE TRIGGER "motorcycle_catalog_updated" BEFORE
UPDATE ON "public"."motorcycle_catalog" FOR EACH ROW EXECUTE
FUNCTION "public"."update_updated_at_column"();


CREATE OR REPLACE TRIGGER
"normalize_appointment_phone_before_write" BEFORE INSERT OR UPDATE
OF "phone" ON "public"."appointments" FOR EACH ROW EXECUTE
FUNCTION "public"."normalize_philippine_mobile_before_write"();


CREATE OR REPLACE TRIGGER
"normalize_blocked_number_phone_before_write" BEFORE INSERT OR
UPDATE OF "phone" ON "public"."blocked_numbers" FOR EACH ROW
EXECUTE FUNCTION "public"."normalize_philippine_mobile_before_write"();


CREATE OR REPLACE TRIGGER
"normalize_crew_member_phone_before_write" BEFORE INSERT OR UPDATE
```

OF "phone" ON "public"."crew_members" FOR EACH ROW EXECUTE FUNCTION "public"."normalize_philippine_mobile_before_write"();

CREATE OR REPLACE TRIGGER "price_history_actor" BEFORE INSERT ON "public"."price_history" FOR EACH ROW EXECUTE FUNCTION "public"."set_price_history_actor"();

CREATE OR REPLACE TRIGGER "products_archive_timestamp" BEFORE UPDATE ON "public"."products" FOR EACH ROW EXECUTE FUNCTION "public"."set_catalog_archive_timestamp"();

CREATE OR REPLACE TRIGGER "products_sync_stock_status" BEFORE INSERT OR UPDATE ON "public"."products" FOR EACH ROW EXECUTE FUNCTION "public"."sync_product_stock_status"();

CREATE OR REPLACE TRIGGER "products_updated" BEFORE UPDATE ON "public"."products" FOR EACH ROW EXECUTE FUNCTION "public"."update_updated_at_column"();

CREATE OR REPLACE TRIGGER "schedule_blocks_archive_timestamp" BEFORE UPDATE OF "is_active" ON "public"."schedule_blocks" FOR EACH ROW EXECUTE FUNCTION "public"."set_schedule_block_archive_timestamp"();

CREATE OR REPLACE TRIGGER "services_archive_timestamp" BEFORE UPDATE ON "public"."services" FOR EACH ROW EXECUTE FUNCTION "public"."set_catalog_archive_timestamp"();

CREATE OR REPLACE TRIGGER "services_updated" BEFORE UPDATE ON "public"."services" FOR EACH ROW EXECUTE FUNCTION "public"."update_updated_at_column"();

```
CREATE OR REPLACE TRIGGER "shop_settings_updated" BEFORE UPDATE
ON "public"."shop_settings" FOR EACH ROW EXECUTE FUNCTION
"public"."update_updated_at_column"();



CREATE OR REPLACE TRIGGER "update_crew_members_updated_at"
BEFORE UPDATE ON "public"."crew_members" FOR EACH ROW EXECUTE
FUNCTION "public"."update_updated_at_column"();



ALTER TABLE ONLY "public"."admin_activity_logs"
    ADD CONSTRAINT "admin_activity_logs_actor_id_fkey" FOREIGN KEY
("actor_id") REFERENCES "auth"."users"("id") ON DELETE SET NULL;



ALTER TABLE ONLY "public"."admin_session_handover_tokens"
    ADD CONSTRAINT "admin_session_handover_tokens_admin_id_fkey"
FOREIGN KEY ("admin_id") REFERENCES "auth"."users"("id") ON DELETE
CASCADE;



ALTER TABLE ONLY "public"."admin_session_leases"
    ADD CONSTRAINT "admin_session_leases_admin_id_fkey" FOREIGN KEY
("admin_id") REFERENCES "auth"."users"("id") ON DELETE CASCADE;



ALTER TABLE ONLY "public"."appointment_action_verifications"
    ADD CONSTRAINT "appointment_action_verifications_appointment_id_fkey"
FOREIGN KEY ("appointment_id") REFERENCES "public"."appointments"("id")
ON DELETE CASCADE;



ALTER TABLE ONLY "public"."appointment_completion_notifications"
    ADD CONSTRAINT
"appointment_completion_notifications_appointment_id_fkey" FOREIGN KEY
("appointment_id") REFERENCES "public"."appointments"("id") ON DELETE
```

```
CASCADE;


ALTER TABLE ONLY "public"."appointment_confirmation_emails"
    ADD CONSTRAINT "appointment_confirmation_emails_appointment_id_fkey"
FOREIGN KEY ("appointment_id") REFERENCES "public"."appointments"("id")
ON DELETE CASCADE;


ALTER TABLE ONLY "public"."appointment_confirmation_sms"
    ADD CONSTRAINT "appointment_confirmation_sms_appointment_id_fkey"
FOREIGN KEY ("appointment_id") REFERENCES "public"."appointments"("id")
ON DELETE CASCADE;


ALTER TABLE ONLY "public"."appointment_continuations"
    ADD CONSTRAINT "appointment_continuations_appointment_id_fkey"
FOREIGN KEY ("appointment_id") REFERENCES "public"."appointments"("id")
ON DELETE CASCADE;


ALTER TABLE ONLY "public"."appointment_continuations"
    ADD CONSTRAINT "appointment_continuations_assigned_crew_id_fkey"
FOREIGN KEY ("assigned_crew_id") REFERENCES
"public"."crew_members"("id");


ALTER TABLE ONLY "public"."appointment_products"
    ADD CONSTRAINT "appointment_products_appointment_id_fkey" FOREIGN
KEY ("appointment_id") REFERENCES "public"."appointments"("id") ON
DELETE CASCADE;


ALTER TABLE ONLY "public"."appointment_products"
    ADD CONSTRAINT "appointment_products_product_id_fkey" FOREIGN KEY
("product_id") REFERENCES "public"."products"("id");
```

```
ALTER TABLE ONLY "public"."appointment_reschedule_history"
    ADD CONSTRAINT "appointment_reschedule_history_appointment_id_fkey"
FOREIGN KEY ("appointment_id") REFERENCES "public"."appointments"("id")
ON DELETE CASCADE;


ALTER TABLE ONLY "public"."appointment_reschedule_history"
    ADD CONSTRAINT "appointment_reschedule_history_approved_by_fkey"
FOREIGN KEY ("approved_by") REFERENCES "auth"."users"("id") ON DELETE
SET NULL;


ALTER TABLE ONLY "public"."appointment_reschedule_history"
    ADD CONSTRAINT
"appointment_reschedule_history_new_appointment_id_fkey" FOREIGN KEY
("new_appointment_id") REFERENCES "public"."appointments"("id") ON
DELETE SET NULL;


ALTER TABLE ONLY "public"."appointment_services"
    ADD CONSTRAINT "appointment_services_appointment_id_fkey" FOREIGN
KEY ("appointment_id") REFERENCES "public"."appointments"("id") ON
DELETE CASCADE;


ALTER TABLE ONLY "public"."appointment_services"
    ADD CONSTRAINT "appointment_services_service_id_fkey" FOREIGN KEY
("service_id") REFERENCES "public"."services"("id") ON DELETE SET NULL;


ALTER TABLE ONLY "public"."appointments"
    ADD CONSTRAINT "appointments_assigned_crew_id_fkey" FOREIGN KEY
("assigned_crew_id") REFERENCES "public"."crew_members"("id") ON
DELETE SET NULL;


ALTER TABLE ONLY "public"."appointments"
    ADD CONSTRAINT "appointments_rescheduled_from_appointment_id_fkey"
FOREIGN KEY ("rescheduled_from_appointment_id") REFERENCES
```

```
"public"."appointments"("id") ON DELETE SET NULL;


ALTER TABLE ONLY "public"."appointments"
    ADD CONSTRAINT "appointments_rescheduled_to_appointment_id_fkey"
FOREIGN KEY ("rescheduled_to_appointment_id") REFERENCES
"public"."appointments"("id") ON DELETE SET NULL;


ALTER TABLE ONLY "public"."blocked_numbers"
    ADD CONSTRAINT "blocked_numbers_created_by_fkey" FOREIGN KEY
("created_by") REFERENCES "auth"."users"("id") ON DELETE SET NULL;


ALTER TABLE ONLY "public"."crew_availability_exceptions"
    ADD CONSTRAINT "crew_availability_exceptions_crew_id_fkey" FOREIGN
KEY ("crew_id") REFERENCES "public"."crew_members"("id") ON DELETE
CASCADE;


ALTER TABLE ONLY "public"."crew_schedules"
    ADD CONSTRAINT "crew_schedules_crew_id_fkey" FOREIGN KEY
("crew_id") REFERENCES "public"."crew_members"("id") ON DELETE
CASCADE;


ALTER TABLE ONLY "public"."customer_service_booking_alerts"
    ADD CONSTRAINT "customer_service_booking_alerts_notification_id_fkey"
FOREIGN KEY ("notification_id") REFERENCES "public"."notifications"("id") ON
DELETE SET NULL;


ALTER TABLE ONLY "public"."notifications"
    ADD CONSTRAINT "notifications_appointment_id_fkey" FOREIGN KEY
("appointment_id") REFERENCES "public"."appointments"("id") ON DELETE
CASCADE;
```

```
ALTER TABLE ONLY "public"."price_history"
    ADD CONSTRAINT "price_history_changed_by_fkey" FOREIGN KEY
("changed_by") REFERENCES "auth"."users"("id");


ALTER TABLE ONLY "public"."user_roles"
    ADD CONSTRAINT "user_roles_user_id_fkey" FOREIGN KEY ("user_id")
REFERENCES "auth"."users"("id") ON DELETE CASCADE;


CREATE POLICY "admin activity logs read" ON "public"."admin_activity_logs"
FOR SELECT TO "authenticated" USING
("public"."has_role_legacy"("auth"."uid"(), 'admin'::"public"."app_role_legacy"));


ALTER TABLE "public"."admin_activity_logs" ENABLE ROW LEVEL SECURITY;


ALTER TABLE "public"."admin_session_handover_tokens" ENABLE ROW
LEVEL SECURITY;


ALTER TABLE "public"."admin_session_leases" ENABLE ROW LEVEL
SECURITY;


CREATE POLICY "appointment continuations admin all" ON
"public"."appointment_continuations" TO "authenticated" USING
("public"."has_role"("auth"."uid"(), 'admin'::"public"."app_role")) WITH CHECK
("public"."has_role"("auth"."uid"(), 'admin'::"public"."app_role"));


CREATE POLICY "appointment products admin all" ON
"public"."appointment_products" TO "authenticated" USING
("public"."has_role"("auth"."uid"(), 'admin'::"public"."app_role")) WITH CHECK
("public"."has_role"("auth"."uid"(), 'admin'::"public"."app_role"));


ALTER TABLE "public"."appointment_action_verifications" ENABLE ROW
LEVEL SECURITY;
```

```
ALTER TABLE "public"."appointment_completion_notifications" ENABLE ROW
LEVEL SECURITY;


ALTER TABLE "public"."appointment_confirmation_emails" ENABLE ROW
LEVEL SECURITY;


ALTER TABLE "public"."appointment_confirmation_sms" ENABLE ROW LEVEL
SECURITY;


ALTER TABLE "public"."appointment_continuations" ENABLE ROW LEVEL
SECURITY;


ALTER TABLE "public"."appointment_products" ENABLE ROW LEVEL
SECURITY;


ALTER TABLE "public"."appointment_reference_recovery_challenges" ENABLE
ROW LEVEL SECURITY;


ALTER TABLE "public"."appointment_reschedule_history" ENABLE ROW
LEVEL SECURITY;


ALTER TABLE "public"."appointment_services" ENABLE ROW LEVEL
SECURITY;


ALTER TABLE "public"."appointments" ENABLE ROW LEVEL SECURITY;


CREATE POLICY "appointments admin all" ON "public"."appointments" TO
"authenticated" USING ("public"."has_role_legacy"("auth"."uid"(),
'admin'::"public"."app_role_legacy")) WITH CHECK
("public"."has_role_legacy"("auth"."uid"(), 'admin'::"public"."app_role_legacy"));



CREATE POLICY "appt services admin all" ON "public"."appointment_services"
```

```
TO "authenticated" USING ("public"."has_role_legacy"("auth"."uid"(),
'admin'::"public"."app_role_legacy")) WITH CHECK
("public"."has_role_legacy"("auth"."uid"(), 'admin'::"public"."app_role_legacy"));


ALTER TABLE "public"."blocked_numbers" ENABLE ROW LEVEL SECURITY;


CREATE POLICY "blocked_numbers admin all" ON "public"."blocked_numbers"
TO "authenticated" USING ("public"."has_role_legacy"("auth"."uid"(),
'admin'::"public"."app_role_legacy")) WITH CHECK
("public"."has_role_legacy"("auth"."uid"(), 'admin'::"public"."app_role_legacy"));


CREATE POLICY "blocks admin write" ON "public"."schedule_blocks" TO
"authenticated" USING ("public"."has_role_legacy"("auth"."uid"(),
'admin'::"public"."app_role_legacy")) WITH CHECK
("public"."has_role_legacy"("auth"."uid"(), 'admin'::"public"."app_role_legacy"));


CREATE POLICY "crew admin all" ON "public"."crew_members" TO
"authenticated" USING ("public"."has_role_legacy"("auth"."uid"(),
'admin'::"public"."app_role_legacy")) WITH CHECK
("public"."has_role_legacy"("auth"."uid"(), 'admin'::"public"."app_role_legacy"));


ALTER TABLE "public"."crew_availability_exceptions" ENABLE ROW LEVEL
SECURITY;


CREATE POLICY "crew_exceptions admin all" ON
"public"."crew_availability_exceptions" TO "authenticated" USING
("public"."has_role_legacy"("auth"."uid"(), 'admin'::"public"."app_role_legacy"))
WITH CHECK ("public"."has_role_legacy"("auth"."uid"(),
'admin'::"public"."app_role_legacy"));


ALTER TABLE "public"."crew_members" ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE "public"."crew_schedules" ENABLE ROW LEVEL SECURITY;


CREATE POLICY "crew_schedules admin all" ON "public"."crew_schedules" TO
"authenticated" USING ("public"."has_role_legacy"("auth"."uid"(),
'admin'::"public"."app_role_legacy")) WITH CHECK
("public"."has_role_legacy"("auth"."uid"(), 'admin'::"public"."app_role_legacy"));



ALTER TABLE "public"."customer_service_booking_alert_state" ENABLE ROW
LEVEL SECURITY;


ALTER TABLE "public"."customer_service_booking_alerts" ENABLE ROW
LEVEL SECURITY;


CREATE POLICY "motorcycle catalog admin write" ON
"public"."motorcycle_catalog" TO "authenticated" USING
("public"."has_role"("auth"."uid"(), 'admin'::"public"."app_role")) WITH CHECK
("public"."has_role"("auth"."uid"(), 'admin'::"public"."app_role"));



CREATE POLICY "motorcycle catalog public read" ON
"public"."motorcycle_catalog" FOR SELECT TO "anon" USING ((("is_active" =
true) AND ("is_archived" = false)));



ALTER TABLE "public"."motorcycle_catalog" ENABLE ROW LEVEL SECURITY;


ALTER TABLE "public"."notifications" ENABLE ROW LEVEL SECURITY;


CREATE POLICY "notifications admin all" ON "public"."notifications" TO
"authenticated" USING ("public"."has_role_legacy"("auth"."uid"(),
'admin'::"public"."app_role_legacy")) WITH CHECK
("public"."has_role_legacy"("auth"."uid"(), 'admin'::"public"."app_role_legacy"));



CREATE POLICY "price history admin all" ON "public"."price_history" TO
```

"authenticated" USING ("public"."has_role_legacy"("auth"."uid"(), 'admin'::"public"."app_role_legacy")) WITH CHECK ("public"."has_role_legacy"("auth"."uid"(), 'admin'::"public"."app_role_legacy"));

ALTER TABLE "public"."price_history" ENABLE ROW LEVEL SECURITY;

ALTER TABLE "public"."products" ENABLE ROW LEVEL SECURITY;

CREATE POLICY "products admin write" ON "public"."products" TO "authenticated" USING ("public"."has_role_legacy"("auth"."uid"(), 'admin'::"public"."app_role_legacy")) WITH CHECK ("public"."has_role_legacy"("auth"."uid"(), 'admin'::"public"."app_role_legacy"));

CREATE POLICY "products public read" ON "public"."products" FOR SELECT TO "anon" USING ((("is_active" = true) AND ("is_archived" = false)));

ALTER TABLE "public"."public_endpoint_rate_limits" ENABLE ROW LEVEL SECURITY;

CREATE POLICY "reschedule history admin all" ON "public"."appointment_reschedule_history" TO "authenticated" USING ("public"."has_role"("auth"."uid"(), 'admin'::"public"."app_role")) WITH CHECK ("public"."has_role"("auth"."uid"(), 'admin'::"public"."app_role"));

ALTER TABLE "public"."schedule_blocks" ENABLE ROW LEVEL SECURITY;

ALTER TABLE "public"."services" ENABLE ROW LEVEL SECURITY;

CREATE POLICY "services admin write" ON "public"."services" TO "authenticated" USING ("public"."has_role_legacy"("auth"."uid"(), 'admin'::"public"."app_role_legacy")) WITH CHECK ("public"."has_role_legacy"("auth"."uid"(), 'admin'::"public"."app_role_legacy"));

```
CREATE POLICY "services public read" ON "public"."services" FOR SELECT
TO "anon" USING ((("is_active" = true) AND ("is_archived" = false)));


CREATE POLICY "shop settings admin insert" ON "public"."shop_settings" FOR
INSERT TO "authenticated" WITH CHECK ("public"."has_role"("auth"."uid"(),
'admin'::"public"."app_role"));


CREATE POLICY "shop settings admin update" ON "public"."shop_settings"
FOR UPDATE TO "authenticated" USING ("public"."has_role"("auth"."uid"(),
'admin'::"public"."app_role")) WITH CHECK ("public"."has_role"("auth"."uid"(),
'admin'::"public"."app_role"));


CREATE POLICY "shop settings public read" ON "public"."shop_settings" FOR
SELECT TO "authenticated", "anon" USING (true);


ALTER TABLE "public"."shop_settings" ENABLE ROW LEVEL SECURITY;


CREATE POLICY "slots admin write" ON "public"."time_slots" TO "authenticated"
USING ("public"."has_role_legacy"("auth"."uid"(),
'admin'::"public"."app_role_legacy")) WITH CHECK
("public"."has_role_legacy"("auth"."uid"(), 'admin'::"public"."app_role_legacy"));


ALTER TABLE "public"."time_slots" ENABLE ROW LEVEL SECURITY;


ALTER TABLE "public"."user_roles" ENABLE ROW LEVEL SECURITY;


CREATE POLICY "users read own roles" ON "public"."user_roles" FOR SELECT
TO "authenticated" USING (("auth"."uid"() = "user_id"));
```

```
ALTER PUBLICATION "supabase_realtime" OWNER TO "postgres";


ALTER PUBLICATION "supabase_realtime" ADD TABLE ONLY
"public"."appointments";


ALTER PUBLICATION "supabase_realtime" ADD TABLE ONLY
"public"."notifications";


GRANT USAGE ON SCHEMA "public" TO "postgres";
GRANT USAGE ON SCHEMA "public" TO "anon";
GRANT USAGE ON SCHEMA "public" TO "authenticated";
GRANT USAGE ON SCHEMA "public" TO "service_role";


GRANT ALL ON FUNCTION "public"."gbtreekey16_in"("cstring") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbtreekey16_in"("cstring") TO "anon";
GRANT ALL ON FUNCTION "public"."gbtreekey16_in"("cstring") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbtreekey16_in"("cstring") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbtreekey16_out"("public"."gbtreekey16")
TO "postgres";
GRANT ALL ON FUNCTION "public"."gbtreekey16_out"("public"."gbtreekey16")
TO "anon";
GRANT ALL ON FUNCTION "public"."gbtreekey16_out"("public"."gbtreekey16")
```

```
TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbtreekey16_out"("public"."gbtreekey16")
TO "service_role";


GRANT ALL ON FUNCTION "public"."gbtreekey2_in"("cstring") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbtreekey2_in"("cstring") TO "anon";
GRANT ALL ON FUNCTION "public"."gbtreekey2_in"("cstring") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbtreekey2_in"("cstring") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbtreekey2_out"("public"."gbtreekey2") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbtreekey2_out"("public"."gbtreekey2") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbtreekey2_out"("public"."gbtreekey2") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbtreekey2_out"("public"."gbtreekey2") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbtreekey32_in"("cstring") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbtreekey32_in"("cstring") TO "anon";
GRANT ALL ON FUNCTION "public"."gbtreekey32_in"("cstring") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbtreekey32_in"("cstring") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbtreekey32_out"("public"."gbtreekey32")
TO "postgres";
GRANT ALL ON FUNCTION "public"."gbtreekey32_out"("public"."gbtreekey32")
TO "anon";
GRANT ALL ON FUNCTION "public"."gbtreekey32_out"("public"."gbtreekey32")
TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbtreekey32_out"("public"."gbtreekey32")
TO "service_role";
```

```
GRANT ALL ON FUNCTION "public"."gbtreekey4_in"("cstring") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbtreekey4_in"("cstring") TO "anon";
GRANT ALL ON FUNCTION "public"."gbtreekey4_in"("cstring") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbtreekey4_in"("cstring") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbtreekey4_out"("public"."gbtreekey4") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbtreekey4_out"("public"."gbtreekey4") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbtreekey4_out"("public"."gbtreekey4") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbtreekey4_out"("public"."gbtreekey4") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbtreekey8_in"("cstring") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbtreekey8_in"("cstring") TO "anon";
GRANT ALL ON FUNCTION "public"."gbtreekey8_in"("cstring") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbtreekey8_in"("cstring") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbtreekey8_out"("public"."gbtreekey8") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbtreekey8_out"("public"."gbtreekey8") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbtreekey8_out"("public"."gbtreekey8") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbtreekey8_out"("public"."gbtreekey8") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbtreekey_var_in"("cstring") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbtreekey_var_in"("cstring") TO "anon";
GRANT ALL ON FUNCTION "public"."gbtreekey_var_in"("cstring") TO
```

"authenticated";
GRANT ALL ON FUNCTION "public"."gbtreekey_var_in"("cstring") TO
"service_role";

GRANT ALL ON FUNCTION
"public"."gbtreekey_var_out"("public"."gbtreekey_var") TO "postgres";
GRANT ALL ON FUNCTION
"public"."gbtreekey_var_out"("public"."gbtreekey_var") TO "anon";
GRANT ALL ON FUNCTION
"public"."gbtreekey_var_out"("public"."gbtreekey_var") TO "authenticated";
GRANT ALL ON FUNCTION
"public"."gbtreekey_var_out"("public"."gbtreekey_var") TO "service_role";

```
REVOKE ALL ON FUNCTION "public"."admin_activity_request_ip"() FROM
PUBLIC;
GRANT ALL ON FUNCTION "public"."admin_activity_request_ip"() TO "anon";
GRANT ALL ON FUNCTION "public"."admin_activity_request_ip"() TO
"authenticated";
GRANT ALL ON FUNCTION "public"."admin_activity_request_ip"() TO
"service_role";



GRANT ALL ON FUNCTION "public"."assert_no_duplicate_active_customer_ser
vice_appointment"("p_appointment_id" "uuid") TO "anon";
GRANT ALL ON FUNCTION "public"."assert_no_duplicate_active_customer_ser
vice_appointment"("p_appointment_id" "uuid") TO "authenticated";
GRANT ALL ON FUNCTION "public"."assert_no_duplicate_active_customer_ser
vice_appointment"("p_appointment_id" "uuid") TO "service_role";



REVOKE ALL ON FUNCTION "public"."booking_slot_capacity"("p_start_time"
time without time zone) FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."booking_slot_capacity"("p_start_time" time
without time zone) TO "anon";
GRANT ALL ON FUNCTION "public"."booking_slot_capacity"("p_start_time" time
without time zone) TO "authenticated";
GRANT ALL ON FUNCTION "public"."booking_slot_capacity"("p_start_time" time
without time zone) TO "service_role";



REVOKE ALL ON FUNCTION
"public"."cancel_public_appointment"("p_appointment_id" "uuid",
"p_cancellation_notice_hours" integer) FROM PUBLIC;
GRANT ALL ON FUNCTION
"public"."cancel_public_appointment"("p_appointment_id" "uuid",
"p_cancellation_notice_hours" integer) TO "anon";
GRANT ALL ON FUNCTION
"public"."cancel_public_appointment"("p_appointment_id" "uuid",
"p_cancellation_notice_hours" integer) TO "authenticated";
GRANT ALL ON FUNCTION
```

"public"."cancel_public_appointment"("p_appointment_id" "uuid",
"p_cancellation_notice_hours" integer) TO "service_role";

REVOKE ALL ON FUNCTION
"public"."capture_customer_no_show_count_at_booking"() FROM PUBLIC;
GRANT ALL ON FUNCTION
"public"."capture_customer_no_show_count_at_booking"() TO "anon";
GRANT ALL ON FUNCTION
"public"."capture_customer_no_show_count_at_booking"() TO "authenticated";
GRANT ALL ON FUNCTION
"public"."capture_customer_no_show_count_at_booking"() TO "service_role";

GRANT ALL ON FUNCTION "public"."cash_dist"("money", "money") TO
"postgres";
GRANT ALL ON FUNCTION "public"."cash_dist"("money", "money") TO "anon";
GRANT ALL ON FUNCTION "public"."cash_dist"("money", "money") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."cash_dist"("money", "money") TO
"service_role";

REVOKE ALL ON FUNCTION
"public"."change_crew_schedule"("p_schedule_id" "uuid", "p_remove" boolean,
"p_start_time" time without time zone, "p_end_time" time without time zone)
FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."change_crew_schedule"("p_schedule_id"
"uuid", "p_remove" boolean, "p_start_time" time without time zone, "p_end_time"
time without time zone) TO "authenticated";
GRANT ALL ON FUNCTION "public"."change_crew_schedule"("p_schedule_id"
"uuid", "p_remove" boolean, "p_start_time" time without time zone, "p_end_time"
time without time zone) TO "service_role";

REVOKE ALL ON FUNCTION
"public"."claim_admin_session"("p_handover_token" "uuid") FROM PUBLIC;
GRANT ALL ON FUNCTION
"public"."claim_admin_session"("p_handover_token" "uuid") TO "authenticated";
GRANT ALL ON FUNCTION
"public"."claim_admin_session"("p_handover_token" "uuid") TO "service_role";

REVOKE ALL ON FUNCTION
"public"."consume_reference_recovery_challenge"("p_email" "text",
"p_code_hash" "text") FROM PUBLIC;
GRANT ALL ON FUNCTION
"public"."consume_reference_recovery_challenge"("p_email" "text",
"p_code_hash" "text") TO "anon";
GRANT ALL ON FUNCTION
"public"."consume_reference_recovery_challenge"("p_email" "text",
"p_code_hash" "text") TO "authenticated";
GRANT ALL ON FUNCTION
"public"."consume_reference_recovery_challenge"("p_email" "text",
"p_code_hash" "text") TO "service_role";

REVOKE ALL ON FUNCTION "public"."create_admin_session_handover"()
FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."create_admin_session_handover"() TO
"authenticated";
GRANT ALL ON FUNCTION "public"."create_admin_session_handover"() TO
"service_role";

REVOKE ALL ON FUNCTION
"public"."create_booking_atomic"("p_reference_code" "text",
"p_booking_request_id" "uuid", "p_customer_name" "text", "p_phone" "text",
"p_moto_brand" "text", "p_moto_model" "text", "p_moto_variant" "text",
"p_moto_year" integer, "p_plate_number" "text", "p_appointment_date" "date",
"p_start_time" time without time zone, "p_notes" "text", "p_total_estimate"
numeric, "p_booking_duration_minutes" integer, "p_assigned_crew_id" "uuid",
"p_services" "jsonb", "p_notification_title" "text", "p_notification_message" "text",
"p_first_name" "text", "p_middle_name" "text", "p_last_name" "text") FROM
PUBLIC;
GRANT ALL ON FUNCTION
"public"."create_booking_atomic"("p_reference_code" "text",
"p_booking_request_id" "uuid", "p_customer_name" "text", "p_phone" "text",
"p_moto_brand" "text", "p_moto_model" "text", "p_moto_variant" "text",
"p_moto_year" integer, "p_plate_number" "text", "p_appointment_date" "date",
"p_start_time" time without time zone, "p_notes" "text", "p_total_estimate"
numeric, "p_booking_duration_minutes" integer, "p_assigned_crew_id" "uuid",
"p_services" "jsonb", "p_notification_title" "text", "p_notification_message" "text",

"p_first_name" "text", "p_middle_name" "text", "p_last_name" "text") TO "anon";
GRANT ALL ON FUNCTION
"public"."create_booking_atomic"("p_reference_code" "text",
"p_booking_request_id" "uuid", "p_customer_name" "text", "p_phone" "text",
"p_moto_brand" "text", "p_moto_model" "text", "p_moto_variant" "text",
"p_moto_year" integer, "p_plate_number" "text", "p_appointment_date" "date",
"p_start_time" time without time zone, "p_notes" "text", "p_total_estimate"
numeric, "p_booking_duration_minutes" integer, "p_assigned_crew_id" "uuid",
"p_services" "jsonb", "p_notification_title" "text", "p_notification_message" "text",
"p_first_name" "text", "p_middle_name" "text", "p_last_name" "text") TO
"authenticated";
GRANT ALL ON FUNCTION
"public"."create_booking_atomic"("p_reference_code" "text",
"p_booking_request_id" "uuid", "p_customer_name" "text", "p_phone" "text",
"p_moto_brand" "text", "p_moto_model" "text", "p_moto_variant" "text",
"p_moto_year" integer, "p_plate_number" "text", "p_appointment_date" "date",
"p_start_time" time without time zone, "p_notes" "text", "p_total_estimate"
numeric, "p_booking_duration_minutes" integer, "p_assigned_crew_id" "uuid",
"p_services" "jsonb", "p_notification_title" "text", "p_notification_message" "text",
"p_first_name" "text", "p_middle_name" "text", "p_last_name" "text") TO
"service_role";

REVOKE ALL ON FUNCTION
"public"."create_booking_atomic"("p_reference_code" "text",
"p_booking_request_id" "uuid", "p_customer_name" "text", "p_phone" "text",
"p_moto_brand" "text", "p_moto_model" "text", "p_moto_variant" "text",
"p_moto_year" integer, "p_plate_number" "text", "p_appointment_date" "date",
"p_start_time" time without time zone, "p_notes" "text", "p_total_estimate"
numeric, "p_booking_duration_minutes" integer, "p_assigned_crew_id" "uuid",
"p_services" "jsonb", "p_continuation_segments" "jsonb", "p_notification_title"
"text", "p_notification_message" "text", "p_first_name" "text", "p_middle_name"
"text", "p_last_name" "text") FROM PUBLIC;
GRANT ALL ON FUNCTION
"public"."create_booking_atomic"("p_reference_code" "text",
"p_booking_request_id" "uuid", "p_customer_name" "text", "p_phone" "text",
"p_moto_brand" "text", "p_moto_model" "text", "p_moto_variant" "text",
"p_moto_year" integer, "p_plate_number" "text", "p_appointment_date" "date",
"p_start_time" time without time zone, "p_notes" "text", "p_total_estimate"
numeric, "p_booking_duration_minutes" integer, "p_assigned_crew_id" "uuid",
"p_services" "jsonb", "p_continuation_segments" "jsonb", "p_notification_title"
"text", "p_notification_message" "text", "p_first_name" "text", "p_middle_name"
"text", "p_last_name" "text") TO "anon";
GRANT ALL ON FUNCTION

"public"."create_booking_atomic"("p_reference_code" "text",
"p_booking_request_id" "uuid", "p_customer_name" "text", "p_phone" "text",
"p_moto_brand" "text", "p_moto_model" "text", "p_moto_variant" "text",
"p_moto_year" integer, "p_plate_number" "text", "p_appointment_date" "date",
"p_start_time" time without time zone, "p_notes" "text", "p_total_estimate"
numeric, "p_booking_duration_minutes" integer, "p_assigned_crew_id" "uuid",
"p_services" "jsonb", "p_continuation_segments" "jsonb", "p_notification_title"
"text", "p_notification_message" "text", "p_first_name" "text", "p_middle_name"
"text", "p_last_name" "text") TO "authenticated";
GRANT ALL ON FUNCTION
"public"."create_booking_atomic"("p_reference_code" "text",
"p_booking_request_id" "uuid", "p_customer_name" "text", "p_phone" "text",
"p_moto_brand" "text", "p_moto_model" "text", "p_moto_variant" "text",
"p_moto_year" integer, "p_plate_number" "text", "p_appointment_date" "date",
"p_start_time" time without time zone, "p_notes" "text", "p_total_estimate"
numeric, "p_booking_duration_minutes" integer, "p_assigned_crew_id" "uuid",
"p_services" "jsonb", "p_continuation_segments" "jsonb", "p_notification_title"
"text", "p_notification_message" "text", "p_first_name" "text", "p_middle_name"
"text", "p_last_name" "text") TO "service_role";

REVOKE ALL ON FUNCTION
"public"."create_booking_atomic"("p_reference_code" "text",
"p_booking_request_id" "uuid", "p_customer_name" "text", "p_phone" "text",
"p_email" "text", "p_moto_brand" "text", "p_moto_model" "text", "p_moto_variant"
"text", "p_moto_year" integer, "p_plate_number" "text", "p_appointment_date"
"date", "p_start_time" time without time zone, "p_notes" "text", "p_total_estimate"
numeric, "p_booking_duration_minutes" integer, "p_assigned_crew_id" "uuid",
"p_services" "jsonb", "p_continuation_segments" "jsonb", "p_notification_title"
"text", "p_notification_message" "text", "p_first_name" "text", "p_middle_name"
"text", "p_last_name" "text") FROM PUBLIC;
GRANT ALL ON FUNCTION
"public"."create_booking_atomic"("p_reference_code" "text",
"p_booking_request_id" "uuid", "p_customer_name" "text", "p_phone" "text",
"p_email" "text", "p_moto_brand" "text", "p_moto_model" "text", "p_moto_variant"
"text", "p_moto_year" integer, "p_plate_number" "text", "p_appointment_date"
"date", "p_start_time" time without time zone, "p_notes" "text", "p_total_estimate"
numeric, "p_booking_duration_minutes" integer, "p_assigned_crew_id" "uuid",
"p_services" "jsonb", "p_continuation_segments" "jsonb", "p_notification_title"
"text", "p_notification_message" "text", "p_first_name" "text", "p_middle_name"
"text", "p_last_name" "text") TO "anon";
GRANT ALL ON FUNCTION
"public"."create_booking_atomic"("p_reference_code" "text",
"p_booking_request_id" "uuid", "p_customer_name" "text", "p_phone" "text",

"p_email" "text", "p_moto_brand" "text", "p_moto_model" "text", "p_moto_variant" "text", "p_moto_year" integer, "p_plate_number" "text", "p_appointment_date" "date", "p_start_time" time without time zone, "p_notes" "text", "p_total_estimate" numeric, "p_booking_duration_minutes" integer, "p_assigned_crew_id" "uuid", "p_services" "jsonb", "p_continuation_segments" "jsonb", "p_notification_title" "text", "p_notification_message" "text", "p_first_name" "text", "p_middle_name" "text", "p_last_name" "text") TO "authenticated";
GRANT ALL ON FUNCTION
"public"."create_booking_atomic"("p_reference_code" "text",
"p_booking_request_id" "uuid", "p_customer_name" "text", "p_phone" "text",
"p_email" "text", "p_moto_brand" "text", "p_moto_model" "text", "p_moto_variant" "text", "p_moto_year" integer, "p_plate_number" "text", "p_appointment_date" "date", "p_start_time" time without time zone, "p_notes" "text", "p_total_estimate" numeric, "p_booking_duration_minutes" integer, "p_assigned_crew_id" "uuid", "p_services" "jsonb", "p_continuation_segments" "jsonb", "p_notification_title" "text", "p_notification_message" "text", "p_first_name" "text", "p_middle_name" "text", "p_last_name" "text") TO "service_role";

REVOKE ALL ON FUNCTION
"public"."create_booking_atomic"("p_reference_code" "text",
"p_booking_request_id" "uuid", "p_customer_name" "text", "p_phone" "text",
"p_email" "text", "p_moto_brand" "text", "p_moto_model" "text", "p_moto_variant" "text", "p_moto_year" integer, "p_plate_number" "text", "p_appointment_date" "date", "p_start_time" time without time zone, "p_notes" "text", "p_total_estimate" numeric, "p_booking_duration_minutes" integer, "p_assigned_crew_id" "uuid", "p_services" "jsonb", "p_continuation_segments" "jsonb", "p_notification_title" "text", "p_notification_message" "text", "p_first_name" "text", "p_middle_name" "text", "p_last_name" "text", "p_wants_products" boolean) FROM PUBLIC;
GRANT ALL ON FUNCTION
"public"."create_booking_atomic"("p_reference_code" "text",
"p_booking_request_id" "uuid", "p_customer_name" "text", "p_phone" "text",
"p_email" "text", "p_moto_brand" "text", "p_moto_model" "text", "p_moto_variant" "text", "p_moto_year" integer, "p_plate_number" "text", "p_appointment_date" "date", "p_start_time" time without time zone, "p_notes" "text", "p_total_estimate" numeric, "p_booking_duration_minutes" integer, "p_assigned_crew_id" "uuid", "p_services" "jsonb", "p_continuation_segments" "jsonb", "p_notification_title" "text", "p_notification_message" "text", "p_first_name" "text", "p_middle_name" "text", "p_last_name" "text", "p_wants_products" boolean) TO "anon";
GRANT ALL ON FUNCTION
"public"."create_booking_atomic"("p_reference_code" "text",
"p_booking_request_id" "uuid", "p_customer_name" "text", "p_phone" "text",
"p_email" "text", "p_moto_brand" "text", "p_moto_model" "text", "p_moto_variant" "text", "p_moto_year" integer, "p_plate_number" "text", "p_appointment_date"

"date", "p_start_time" time without time zone, "p_notes" "text", "p_total_estimate" numeric, "p_booking_duration_minutes" integer, "p_assigned_crew_id" "uuid", "p_services" "jsonb", "p_continuation_segments" "jsonb", "p_notification_title" "text", "p_notification_message" "text", "p_first_name" "text", "p_middle_name" "text", "p_last_name" "text", "p_wants_products" boolean) TO "authenticated";
GRANT ALL ON FUNCTION "public"."create_booking_atomic"("p_reference_code" "text", "p_booking_request_id" "uuid", "p_customer_name" "text", "p_phone" "text", "p_email" "text", "p_moto_brand" "text", "p_moto_model" "text", "p_moto_variant" "text", "p_moto_year" integer, "p_plate_number" "text", "p_appointment_date" "date", "p_start_time" time without time zone, "p_notes" "text", "p_total_estimate" numeric, "p_booking_duration_minutes" integer, "p_assigned_crew_id" "uuid", "p_services" "jsonb", "p_continuation_segments" "jsonb", "p_notification_title" "text", "p_notification_message" "text", "p_first_name" "text", "p_middle_name" "text", "p_last_name" "text", "p_wants_products" boolean) TO "service_role";

REVOKE ALL ON FUNCTION "public"."crew_daily_workload"("p_crew_id" "uuid", "p_date" "date", "p_excluded_appointment_id" "uuid") FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."crew_daily_workload"("p_crew_id" "uuid", "p_date" "date", "p_excluded_appointment_id" "uuid") TO "authenticated";
GRANT ALL ON FUNCTION "public"."crew_daily_workload"("p_crew_id" "uuid", "p_date" "date", "p_excluded_appointment_id" "uuid") TO "service_role";

REVOKE ALL ON FUNCTION "public"."current_admin_auth_session_id"() FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."current_admin_auth_session_id"() TO "authenticated";
GRANT ALL ON FUNCTION "public"."current_admin_auth_session_id"() TO "service_role";

GRANT ALL ON FUNCTION "public"."date_dist"("date", "date") TO "postgres";
GRANT ALL ON FUNCTION "public"."date_dist"("date", "date") TO "anon";
GRANT ALL ON FUNCTION "public"."date_dist"("date", "date") TO "authenticated";
GRANT ALL ON FUNCTION "public"."date_dist"("date", "date") TO "service_role";

```
GRANT ALL ON FUNCTION "public"."enforce_active_appointment_capacity"()
TO "anon";
GRANT ALL ON FUNCTION "public"."enforce_active_appointment_capacity"()
TO "authenticated";
GRANT ALL ON FUNCTION "public"."enforce_active_appointment_capacity"()
TO "service_role";



GRANT ALL ON FUNCTION
"public"."enforce_active_appointment_continuation_capacity"() TO "anon";
GRANT ALL ON FUNCTION
"public"."enforce_active_appointment_continuation_capacity"() TO
"authenticated";
GRANT ALL ON FUNCTION
"public"."enforce_active_appointment_continuation_capacity"() TO
"service_role";



GRANT ALL ON FUNCTION
"public"."enforce_active_appointment_crew_conflict"() TO "anon";
GRANT ALL ON FUNCTION
"public"."enforce_active_appointment_crew_conflict"() TO "authenticated";
GRANT ALL ON FUNCTION
"public"."enforce_active_appointment_crew_conflict"() TO "service_role";



GRANT ALL ON FUNCTION
"public"."enforce_customer_upcoming_appointment_limit"() TO "anon";
GRANT ALL ON FUNCTION
"public"."enforce_customer_upcoming_appointment_limit"() TO "authenticated";
GRANT ALL ON FUNCTION
"public"."enforce_customer_upcoming_appointment_limit"() TO "service_role";



REVOKE ALL ON FUNCTION "public"."enforce_public_rate_limit"("p_scope"
"text", "p_subject" "text", "p_limit" integer, "p_window_seconds" integer) FROM
PUBLIC;
GRANT ALL ON FUNCTION "public"."enforce_public_rate_limit"("p_scope"
"text", "p_subject" "text", "p_limit" integer, "p_window_seconds" integer) TO
"anon";
GRANT ALL ON FUNCTION "public"."enforce_public_rate_limit"("p_scope"
```

"text", "p_subject" "text", "p_limit" integer, "p_window_seconds" integer) TO "authenticated";
GRANT ALL ON FUNCTION "public"."enforce_public_rate_limit"("p_scope" "text", "p_subject" "text", "p_limit" integer, "p_window_seconds" integer) TO "service_role";

GRANT ALL ON FUNCTION "public"."enforce_reschedule_request_capacity"() TO "anon";
GRANT ALL ON FUNCTION "public"."enforce_reschedule_request_capacity"() TO "authenticated";
GRANT ALL ON FUNCTION "public"."enforce_reschedule_request_capacity"() TO "service_role";

REVOKE ALL ON FUNCTION "public"."enqueue_due_service_progress_notifications"() FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."enqueue_due_service_progress_notifications"() TO "service_role";

GRANT ALL ON FUNCTION "public"."float4_dist"(real, real) TO "postgres";
GRANT ALL ON FUNCTION "public"."float4_dist"(real, real) TO "anon";
GRANT ALL ON FUNCTION "public"."float4_dist"(real, real) TO "authenticated";
GRANT ALL ON FUNCTION "public"."float4_dist"(real, real) TO "service_role";

GRANT ALL ON FUNCTION "public"."float8_dist"(double precision, double precision) TO "postgres";
GRANT ALL ON FUNCTION "public"."float8_dist"(double precision, double precision) TO "anon";
GRANT ALL ON FUNCTION "public"."float8_dist"(double precision, double precision) TO "authenticated";
GRANT ALL ON FUNCTION "public"."float8_dist"(double precision, double precision) TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_bit_compress"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_bit_compress"("internal") TO "anon";

```
GRANT ALL ON FUNCTION "public"."gbt_bit_compress"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_bit_compress"("internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_bit_consistent"("internal", bit, smallint,
"oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_bit_consistent"("internal", bit, smallint,
"oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_bit_consistent"("internal", bit, smallint,
"oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_bit_consistent"("internal", bit, smallint,
"oid", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_bit_penalty"("internal", "internal",
"internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_bit_penalty"("internal", "internal",
"internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_bit_penalty"("internal", "internal",
"internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_bit_penalty"("internal", "internal",
"internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_bit_picksplit"("internal", "internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_bit_picksplit"("internal", "internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_bit_picksplit"("internal", "internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_bit_picksplit"("internal", "internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_bit_same"("public"."gbtreekey_var",
"public"."gbtreekey_var", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_bit_same"("public"."gbtreekey_var",
"public"."gbtreekey_var", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_bit_same"("public"."gbtreekey_var",
```

"public"."gbtreekey_var", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_bit_same"("public"."gbtreekey_var", "public"."gbtreekey_var", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_bit_union"("internal", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_bit_union"("internal", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_bit_union"("internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_bit_union"("internal", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_bool_compress"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_bool_compress"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_bool_compress"("internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_bool_compress"("internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_bool_consistent"("internal", boolean, smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_bool_consistent"("internal", boolean, smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_bool_consistent"("internal", boolean, smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_bool_consistent"("internal", boolean, smallint, "oid", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_bool_fetch"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_bool_fetch"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_bool_fetch"("internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_bool_fetch"("internal") TO "service_role";

```
GRANT ALL ON FUNCTION "public"."gbt_bool_penalty"("internal", "internal",
"internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_bool_penalty"("internal", "internal",
"internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_bool_penalty"("internal", "internal",
"internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_bool_penalty"("internal", "internal",
"internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_bool_picksplit"("internal", "internal")
TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_bool_picksplit"("internal", "internal")
TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_bool_picksplit"("internal", "internal")
TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_bool_picksplit"("internal", "internal")
TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_bool_same"("public"."gbtreekey2",
"public"."gbtreekey2", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_bool_same"("public"."gbtreekey2",
"public"."gbtreekey2", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_bool_same"("public"."gbtreekey2",
"public"."gbtreekey2", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_bool_same"("public"."gbtreekey2",
"public"."gbtreekey2", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_bool_union"("internal", "internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_bool_union"("internal", "internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_bool_union"("internal", "internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_bool_union"("internal", "internal") TO
"service_role";
```

```
GRANT ALL ON FUNCTION "public"."gbt_bpchar_compress"("internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_bpchar_compress"("internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_bpchar_compress"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_bpchar_compress"("internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_bpchar_consistent"("internal",
character, smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_bpchar_consistent"("internal",
character, smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_bpchar_consistent"("internal",
character, smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_bpchar_consistent"("internal",
character, smallint, "oid", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_bytea_compress"("internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_bytea_compress"("internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_bytea_compress"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_bytea_compress"("internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_bytea_consistent"("internal", "bytea",
smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_bytea_consistent"("internal", "bytea",
smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_bytea_consistent"("internal", "bytea",
smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_bytea_consistent"("internal", "bytea",
smallint, "oid", "internal") TO "service_role";
```

```
GRANT ALL ON FUNCTION "public"."gbt_bytea_penalty"("internal", "internal",
"internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_bytea_penalty"("internal", "internal",
"internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_bytea_penalty"("internal", "internal",
"internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_bytea_penalty"("internal", "internal",
"internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_bytea_picksplit"("internal", "internal")
TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_bytea_picksplit"("internal", "internal")
TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_bytea_picksplit"("internal", "internal")
TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_bytea_picksplit"("internal", "internal")
TO "service_role";


GRANT ALL ON FUNCTION
"public"."gbt_bytea_same"("public"."gbtreekey_var", "public"."gbtreekey_var",
"internal") TO "postgres";
GRANT ALL ON FUNCTION
"public"."gbt_bytea_same"("public"."gbtreekey_var", "public"."gbtreekey_var",
"internal") TO "anon";
GRANT ALL ON FUNCTION
"public"."gbt_bytea_same"("public"."gbtreekey_var", "public"."gbtreekey_var",
"internal") TO "authenticated";
GRANT ALL ON FUNCTION
"public"."gbt_bytea_same"("public"."gbtreekey_var", "public"."gbtreekey_var",
"internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_bytea_union"("internal", "internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_bytea_union"("internal", "internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_bytea_union"("internal", "internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_bytea_union"("internal", "internal") TO
"service_role";
```

```
GRANT ALL ON FUNCTION "public"."gbt_cash_compress"("internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_cash_compress"("internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_cash_compress"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_cash_compress"("internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_cash_consistent"("internal", "money",
smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_cash_consistent"("internal", "money",
smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_cash_consistent"("internal", "money",
smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_cash_consistent"("internal", "money",
smallint, "oid", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_cash_distance"("internal", "money",
smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_cash_distance"("internal", "money",
smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_cash_distance"("internal", "money",
smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_cash_distance"("internal", "money",
smallint, "oid", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_cash_fetch"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_cash_fetch"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_cash_fetch"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_cash_fetch"("internal") TO
"service_role";
```

GRANT ALL ON FUNCTION "public"."gbt_cash_penalty"("internal", "internal", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_cash_penalty"("internal", "internal", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_cash_penalty"("internal", "internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_cash_penalty"("internal", "internal", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_cash_picksplit"("internal", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_cash_picksplit"("internal", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_cash_picksplit"("internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_cash_picksplit"("internal", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_cash_same"("public"."gbtreekey16", "public"."gbtreekey16", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_cash_same"("public"."gbtreekey16", "public"."gbtreekey16", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_cash_same"("public"."gbtreekey16", "public"."gbtreekey16", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_cash_same"("public"."gbtreekey16", "public"."gbtreekey16", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_cash_union"("internal", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_cash_union"("internal", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_cash_union"("internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_cash_union"("internal", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_date_compress"("internal") TO

"postgres";
GRANT ALL ON FUNCTION "public"."gbt_date_compress"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_date_compress"("internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_date_compress"("internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_date_consistent"("internal", "date", smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_date_consistent"("internal", "date", smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_date_consistent"("internal", "date", smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_date_consistent"("internal", "date", smallint, "oid", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_date_distance"("internal", "date", smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_date_distance"("internal", "date", smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_date_distance"("internal", "date", smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_date_distance"("internal", "date", smallint, "oid", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_date_fetch"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_date_fetch"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_date_fetch"("internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_date_fetch"("internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_date_penalty"("internal", "internal", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_date_penalty"("internal", "internal", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_date_penalty"("internal", "internal",

```
"internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_date_penalty"("internal", "internal",
"internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_date_picksplit"("internal", "internal")
TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_date_picksplit"("internal", "internal")
TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_date_picksplit"("internal", "internal")
TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_date_picksplit"("internal", "internal")
TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_date_same"("public"."gbtreekey8",
"public"."gbtreekey8", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_date_same"("public"."gbtreekey8",
"public"."gbtreekey8", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_date_same"("public"."gbtreekey8",
"public"."gbtreekey8", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_date_same"("public"."gbtreekey8",
"public"."gbtreekey8", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_date_union"("internal", "internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_date_union"("internal", "internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_date_union"("internal", "internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_date_union"("internal", "internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_decompress"("internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_decompress"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_decompress"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_decompress"("internal") TO
```

```
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_enum_compress"("internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_enum_compress"("internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_enum_compress"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_enum_compress"("internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_enum_consistent"("internal",
"anyenum", smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_enum_consistent"("internal",
"anyenum", smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_enum_consistent"("internal",
"anyenum", smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_enum_consistent"("internal",
"anyenum", smallint, "oid", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_enum_fetch"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_enum_fetch"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_enum_fetch"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_enum_fetch"("internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_enum_penalty"("internal", "internal",
"internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_enum_penalty"("internal", "internal",
"internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_enum_penalty"("internal", "internal",
"internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_enum_penalty"("internal", "internal",
"internal") TO "service_role";
```

```
GRANT ALL ON FUNCTION "public"."gbt_enum_picksplit"("internal", "internal")
TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_enum_picksplit"("internal", "internal")
TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_enum_picksplit"("internal", "internal")
TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_enum_picksplit"("internal", "internal")
TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_enum_same"("public"."gbtreekey8",
"public"."gbtreekey8", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_enum_same"("public"."gbtreekey8",
"public"."gbtreekey8", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_enum_same"("public"."gbtreekey8",
"public"."gbtreekey8", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_enum_same"("public"."gbtreekey8",
"public"."gbtreekey8", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_enum_union"("internal", "internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_enum_union"("internal", "internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_enum_union"("internal", "internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_enum_union"("internal", "internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_float4_compress"("internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_float4_compress"("internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_float4_compress"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_float4_compress"("internal") TO
"service_role";
```

```
GRANT ALL ON FUNCTION "public"."gbt_float4_consistent"("internal", real,
smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_float4_consistent"("internal", real,
smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_float4_consistent"("internal", real,
smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_float4_consistent"("internal", real,
smallint, "oid", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_float4_distance"("internal", real,
smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_float4_distance"("internal", real,
smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_float4_distance"("internal", real,
smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_float4_distance"("internal", real,
smallint, "oid", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_float4_fetch"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_float4_fetch"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_float4_fetch"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_float4_fetch"("internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_float4_penalty"("internal", "internal",
"internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_float4_penalty"("internal", "internal",
"internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_float4_penalty"("internal", "internal",
"internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_float4_penalty"("internal", "internal",
"internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_float4_picksplit"("internal", "internal")
TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_float4_picksplit"("internal", "internal")
```

```
TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_float4_picksplit"("internal", "internal")
TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_float4_picksplit"("internal", "internal")
TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_float4_same"("public"."gbtreekey8",
"public"."gbtreekey8", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_float4_same"("public"."gbtreekey8",
"public"."gbtreekey8", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_float4_same"("public"."gbtreekey8",
"public"."gbtreekey8", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_float4_same"("public"."gbtreekey8",
"public"."gbtreekey8", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_float4_union"("internal", "internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_float4_union"("internal", "internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_float4_union"("internal", "internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_float4_union"("internal", "internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_float8_compress"("internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_float8_compress"("internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_float8_compress"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_float8_compress"("internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_float8_consistent"("internal", double
precision, smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_float8_consistent"("internal", double
precision, smallint, "oid", "internal") TO "anon";
```

GRANT ALL ON FUNCTION "public"."gbt_float8_consistent"("internal", double precision, smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_float8_consistent"("internal", double precision, smallint, "oid", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_float8_distance"("internal", double precision, smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_float8_distance"("internal", double precision, smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_float8_distance"("internal", double precision, smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_float8_distance"("internal", double precision, smallint, "oid", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_float8_fetch"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_float8_fetch"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_float8_fetch"("internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_float8_fetch"("internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_float8_penalty"("internal", "internal", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_float8_penalty"("internal", "internal", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_float8_penalty"("internal", "internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_float8_penalty"("internal", "internal", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_float8_picksplit"("internal", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_float8_picksplit"("internal", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_float8_picksplit"("internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_float8_picksplit"("internal", "internal")

TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_float8_same"("public"."gbtreekey16", "public"."gbtreekey16", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_float8_same"("public"."gbtreekey16", "public"."gbtreekey16", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_float8_same"("public"."gbtreekey16", "public"."gbtreekey16", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_float8_same"("public"."gbtreekey16", "public"."gbtreekey16", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_float8_union"("internal", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_float8_union"("internal", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_float8_union"("internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_float8_union"("internal", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_inet_compress"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_inet_compress"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_inet_compress"("internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_inet_compress"("internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_inet_consistent"("internal", "inet", smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_inet_consistent"("internal", "inet", smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_inet_consistent"("internal", "inet", smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_inet_consistent"("internal", "inet", smallint, "oid", "internal") TO "service_role";

```
GRANT ALL ON FUNCTION "public"."gbt_inet_penalty"("internal", "internal",
"internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_inet_penalty"("internal", "internal",
"internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_inet_penalty"("internal", "internal",
"internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_inet_penalty"("internal", "internal",
"internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_inet_picksplit"("internal", "internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_inet_picksplit"("internal", "internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_inet_picksplit"("internal", "internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_inet_picksplit"("internal", "internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_inet_same"("public"."gbtreekey16",
"public"."gbtreekey16", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_inet_same"("public"."gbtreekey16",
"public"."gbtreekey16", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_inet_same"("public"."gbtreekey16",
"public"."gbtreekey16", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_inet_same"("public"."gbtreekey16",
"public"."gbtreekey16", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_inet_union"("internal", "internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_inet_union"("internal", "internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_inet_union"("internal", "internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_inet_union"("internal", "internal") TO
"service_role";
```

```
GRANT ALL ON FUNCTION "public"."gbt_int2_compress"("internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_int2_compress"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_int2_compress"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_int2_compress"("internal") TO
"service_role";



GRANT ALL ON FUNCTION "public"."gbt_int2_consistent"("internal", smallint,
smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_int2_consistent"("internal", smallint,
smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_int2_consistent"("internal", smallint,
smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_int2_consistent"("internal", smallint,
smallint, "oid", "internal") TO "service_role";



GRANT ALL ON FUNCTION "public"."gbt_int2_distance"("internal", smallint,
smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_int2_distance"("internal", smallint,
smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_int2_distance"("internal", smallint,
smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_int2_distance"("internal", smallint,
smallint, "oid", "internal") TO "service_role";



GRANT ALL ON FUNCTION "public"."gbt_int2_fetch"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_int2_fetch"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_int2_fetch"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_int2_fetch"("internal") TO
"service_role";



GRANT ALL ON FUNCTION "public"."gbt_int2_penalty"("internal", "internal",
"internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_int2_penalty"("internal", "internal",
```

"internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_int2_penalty"("internal", "internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_int2_penalty"("internal", "internal", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_int2_picksplit"("internal", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_int2_picksplit"("internal", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_int2_picksplit"("internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_int2_picksplit"("internal", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_int2_same"("public"."gbtreekey4", "public"."gbtreekey4", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_int2_same"("public"."gbtreekey4", "public"."gbtreekey4", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_int2_same"("public"."gbtreekey4", "public"."gbtreekey4", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_int2_same"("public"."gbtreekey4", "public"."gbtreekey4", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_int2_union"("internal", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_int2_union"("internal", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_int2_union"("internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_int2_union"("internal", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_int4_compress"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_int4_compress"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_int4_compress"("internal") TO

```
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_int4_compress"("internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_int4_consistent"("internal", integer,
smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_int4_consistent"("internal", integer,
smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_int4_consistent"("internal", integer,
smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_int4_consistent"("internal", integer,
smallint, "oid", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_int4_distance"("internal", integer,
smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_int4_distance"("internal", integer,
smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_int4_distance"("internal", integer,
smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_int4_distance"("internal", integer,
smallint, "oid", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_int4_fetch"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_int4_fetch"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_int4_fetch"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_int4_fetch"("internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_int4_penalty"("internal", "internal",
"internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_int4_penalty"("internal", "internal",
"internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_int4_penalty"("internal", "internal",
"internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_int4_penalty"("internal", "internal",
"internal") TO "service_role";
```

```
GRANT ALL ON FUNCTION "public"."gbt_int4_picksplit"("internal", "internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_int4_picksplit"("internal", "internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_int4_picksplit"("internal", "internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_int4_picksplit"("internal", "internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_int4_same"("public"."gbtreekey8",
"public"."gbtreekey8", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_int4_same"("public"."gbtreekey8",
"public"."gbtreekey8", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_int4_same"("public"."gbtreekey8",
"public"."gbtreekey8", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_int4_same"("public"."gbtreekey8",
"public"."gbtreekey8", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_int4_union"("internal", "internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_int4_union"("internal", "internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_int4_union"("internal", "internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_int4_union"("internal", "internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_int8_compress"("internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_int8_compress"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_int8_compress"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_int8_compress"("internal") TO
"service_role";
```

GRANT ALL ON FUNCTION "public"."gbt_int8_consistent"("internal", bigint, smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_int8_consistent"("internal", bigint, smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_int8_consistent"("internal", bigint, smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_int8_consistent"("internal", bigint, smallint, "oid", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_int8_distance"("internal", bigint, smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_int8_distance"("internal", bigint, smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_int8_distance"("internal", bigint, smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_int8_distance"("internal", bigint, smallint, "oid", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_int8_fetch"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_int8_fetch"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_int8_fetch"("internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_int8_fetch"("internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_int8_penalty"("internal", "internal", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_int8_penalty"("internal", "internal", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_int8_penalty"("internal", "internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_int8_penalty"("internal", "internal", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_int8_picksplit"("internal", "internal") TO "postgres";

GRANT ALL ON FUNCTION "public"."gbt_int8_picksplit"("internal", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_int8_picksplit"("internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_int8_picksplit"("internal", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_int8_same"("public"."gbtreekey16", "public"."gbtreekey16", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_int8_same"("public"."gbtreekey16", "public"."gbtreekey16", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_int8_same"("public"."gbtreekey16", "public"."gbtreekey16", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_int8_same"("public"."gbtreekey16", "public"."gbtreekey16", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_int8_union"("internal", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_int8_union"("internal", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_int8_union"("internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_int8_union"("internal", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_intv_compress"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_intv_compress"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_intv_compress"("internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_intv_compress"("internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_intv_consistent"("internal", interval, smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_intv_consistent"("internal", interval, smallint, "oid", "internal") TO "anon";

```
GRANT ALL ON FUNCTION "public"."gbt_intv_consistent"("internal", interval,
smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_intv_consistent"("internal", interval,
smallint, "oid", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_intv_decompress"("internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_intv_decompress"("internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_intv_decompress"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_intv_decompress"("internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_intv_distance"("internal", interval,
smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_intv_distance"("internal", interval,
smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_intv_distance"("internal", interval,
smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_intv_distance"("internal", interval,
smallint, "oid", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_intv_fetch"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_intv_fetch"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_intv_fetch"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_intv_fetch"("internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_intv_penalty"("internal", "internal",
"internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_intv_penalty"("internal", "internal",
"internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_intv_penalty"("internal", "internal",
"internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_intv_penalty"("internal", "internal",
```

"internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_intv_picksplit"("internal", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_intv_picksplit"("internal", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_intv_picksplit"("internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_intv_picksplit"("internal", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_intv_same"("public"."gbtreekey32", "public"."gbtreekey32", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_intv_same"("public"."gbtreekey32", "public"."gbtreekey32", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_intv_same"("public"."gbtreekey32", "public"."gbtreekey32", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_intv_same"("public"."gbtreekey32", "public"."gbtreekey32", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_intv_union"("internal", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_intv_union"("internal", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_intv_union"("internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_intv_union"("internal", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_macad8_compress"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_macad8_compress"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_macad8_compress"("internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_macad8_compress"("internal") TO "service_role";

```
GRANT ALL ON FUNCTION "public"."gbt_macad8_consistent"("internal",
"macaddr8", smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_macad8_consistent"("internal",
"macaddr8", smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_macad8_consistent"("internal",
"macaddr8", smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_macad8_consistent"("internal",
"macaddr8", smallint, "oid", "internal") TO "service_role";



GRANT ALL ON FUNCTION "public"."gbt_macad8_fetch"("internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_macad8_fetch"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_macad8_fetch"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_macad8_fetch"("internal") TO
"service_role";



GRANT ALL ON FUNCTION "public"."gbt_macad8_penalty"("internal", "internal",
"internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_macad8_penalty"("internal", "internal",
"internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_macad8_penalty"("internal", "internal",
"internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_macad8_penalty"("internal", "internal",
"internal") TO "service_role";



GRANT ALL ON FUNCTION "public"."gbt_macad8_picksplit"("internal",
"internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_macad8_picksplit"("internal",
"internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_macad8_picksplit"("internal",
"internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_macad8_picksplit"("internal",
"internal") TO "service_role";
```

```
GRANT ALL ON FUNCTION
"public"."gbt_macad8_same"("public"."gbtreekey16", "public"."gbtreekey16",
"internal") TO "postgres";
GRANT ALL ON FUNCTION
"public"."gbt_macad8_same"("public"."gbtreekey16", "public"."gbtreekey16",
"internal") TO "anon";
GRANT ALL ON FUNCTION
"public"."gbt_macad8_same"("public"."gbtreekey16", "public"."gbtreekey16",
"internal") TO "authenticated";
GRANT ALL ON FUNCTION
"public"."gbt_macad8_same"("public"."gbtreekey16", "public"."gbtreekey16",
"internal") TO "service_role";



GRANT ALL ON FUNCTION "public"."gbt_macad8_union"("internal", "internal")
TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_macad8_union"("internal", "internal")
TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_macad8_union"("internal", "internal")
TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_macad8_union"("internal", "internal")
TO "service_role";



GRANT ALL ON FUNCTION "public"."gbt_macad_compress"("internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_macad_compress"("internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_macad_compress"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_macad_compress"("internal") TO
"service_role";



GRANT ALL ON FUNCTION "public"."gbt_macad_consistent"("internal",
"macaddr", smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_macad_consistent"("internal",
"macaddr", smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_macad_consistent"("internal",
"macaddr", smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_macad_consistent"("internal",
```

"macaddr", smallint, "oid", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_macad_fetch"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_macad_fetch"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_macad_fetch"("internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_macad_fetch"("internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_macad_penalty"("internal", "internal", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_macad_penalty"("internal", "internal", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_macad_penalty"("internal", "internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_macad_penalty"("internal", "internal", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_macad_picksplit"("internal", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_macad_picksplit"("internal", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_macad_picksplit"("internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_macad_picksplit"("internal", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_macad_same"("public"."gbtreekey16", "public"."gbtreekey16", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_macad_same"("public"."gbtreekey16", "public"."gbtreekey16", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_macad_same"("public"."gbtreekey16", "public"."gbtreekey16", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_macad_same"("public"."gbtreekey16", "public"."gbtreekey16", "internal") TO "service_role";

```
GRANT ALL ON FUNCTION "public"."gbt_macad_union"("internal", "internal")
TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_macad_union"("internal", "internal")
TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_macad_union"("internal", "internal")
TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_macad_union"("internal", "internal")
TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_numeric_compress"("internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_numeric_compress"("internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_numeric_compress"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_numeric_compress"("internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_numeric_consistent"("internal",
numeric, smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_numeric_consistent"("internal",
numeric, smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_numeric_consistent"("internal",
numeric, smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_numeric_consistent"("internal",
numeric, smallint, "oid", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_numeric_penalty"("internal", "internal",
"internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_numeric_penalty"("internal", "internal",
"internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_numeric_penalty"("internal", "internal",
"internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_numeric_penalty"("internal", "internal",
"internal") TO "service_role";
```

```
GRANT ALL ON FUNCTION "public"."gbt_numeric_picksplit"("internal",
"internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_numeric_picksplit"("internal",
"internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_numeric_picksplit"("internal",
"internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_numeric_picksplit"("internal",
"internal") TO "service_role";


GRANT ALL ON FUNCTION
"public"."gbt_numeric_same"("public"."gbtreekey_var", "public"."gbtreekey_var",
"internal") TO "postgres";
GRANT ALL ON FUNCTION
"public"."gbt_numeric_same"("public"."gbtreekey_var", "public"."gbtreekey_var",
"internal") TO "anon";
GRANT ALL ON FUNCTION
"public"."gbt_numeric_same"("public"."gbtreekey_var", "public"."gbtreekey_var",
"internal") TO "authenticated";
GRANT ALL ON FUNCTION
"public"."gbt_numeric_same"("public"."gbtreekey_var", "public"."gbtreekey_var",
"internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_numeric_union"("internal", "internal")
TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_numeric_union"("internal", "internal")
TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_numeric_union"("internal", "internal")
TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_numeric_union"("internal", "internal")
TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_oid_compress"("internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_oid_compress"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_oid_compress"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_oid_compress"("internal") TO
"service_role";
```

```
GRANT ALL ON FUNCTION "public"."gbt_oid_consistent"("internal", "oid",
smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_oid_consistent"("internal", "oid",
smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_oid_consistent"("internal", "oid",
smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_oid_consistent"("internal", "oid",
smallint, "oid", "internal") TO "service_role";
```

```
GRANT ALL ON FUNCTION "public"."gbt_oid_distance"("internal", "oid",
smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_oid_distance"("internal", "oid",
smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_oid_distance"("internal", "oid",
smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_oid_distance"("internal", "oid",
smallint, "oid", "internal") TO "service_role";
```

```
GRANT ALL ON FUNCTION "public"."gbt_oid_fetch"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_oid_fetch"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_oid_fetch"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_oid_fetch"("internal") TO
"service_role";
```

```
GRANT ALL ON FUNCTION "public"."gbt_oid_penalty"("internal", "internal",
"internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_oid_penalty"("internal", "internal",
"internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_oid_penalty"("internal", "internal",
"internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_oid_penalty"("internal", "internal",
"internal") TO "service_role";
```

```
GRANT ALL ON FUNCTION "public"."gbt_oid_picksplit"("internal", "internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_oid_picksplit"("internal", "internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_oid_picksplit"("internal", "internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_oid_picksplit"("internal", "internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_oid_same"("public"."gbtreekey8",
"public"."gbtreekey8", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_oid_same"("public"."gbtreekey8",
"public"."gbtreekey8", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_oid_same"("public"."gbtreekey8",
"public"."gbtreekey8", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_oid_same"("public"."gbtreekey8",
"public"."gbtreekey8", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_oid_union"("internal", "internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_oid_union"("internal", "internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_oid_union"("internal", "internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_oid_union"("internal", "internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_text_compress"("internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_text_compress"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_text_compress"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_text_compress"("internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_text_consistent"("internal", "text",
smallint, "oid", "internal") TO "postgres";
```

```
GRANT ALL ON FUNCTION "public"."gbt_text_consistent"("internal", "text",
smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_text_consistent"("internal", "text",
smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_text_consistent"("internal", "text",
smallint, "oid", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_text_penalty"("internal", "internal",
"internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_text_penalty"("internal", "internal",
"internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_text_penalty"("internal", "internal",
"internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_text_penalty"("internal", "internal",
"internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_text_picksplit"("internal", "internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_text_picksplit"("internal", "internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_text_picksplit"("internal", "internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_text_picksplit"("internal", "internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_text_same"("public"."gbtreekey_var",
"public"."gbtreekey_var", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_text_same"("public"."gbtreekey_var",
"public"."gbtreekey_var", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_text_same"("public"."gbtreekey_var",
"public"."gbtreekey_var", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_text_same"("public"."gbtreekey_var",
"public"."gbtreekey_var", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_text_union"("internal", "internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_text_union"("internal", "internal") TO
```

"anon";
GRANT ALL ON FUNCTION "public"."gbt_text_union"("internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_text_union"("internal", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_time_compress"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_time_compress"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_time_compress"("internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_time_compress"("internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_time_consistent"("internal", time without time zone, smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_time_consistent"("internal", time without time zone, smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_time_consistent"("internal", time without time zone, smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_time_consistent"("internal", time without time zone, smallint, "oid", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_time_distance"("internal", time without time zone, smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_time_distance"("internal", time without time zone, smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_time_distance"("internal", time without time zone, smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_time_distance"("internal", time without time zone, smallint, "oid", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_time_fetch"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_time_fetch"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_time_fetch"("internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_time_fetch"("internal") TO

"service_role";

GRANT ALL ON FUNCTION "public"."gbt_time_penalty"("internal", "internal", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_time_penalty"("internal", "internal", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_time_penalty"("internal", "internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_time_penalty"("internal", "internal", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_time_picksplit"("internal", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_time_picksplit"("internal", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_time_picksplit"("internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_time_picksplit"("internal", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_time_same"("public"."gbtreekey16", "public"."gbtreekey16", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_time_same"("public"."gbtreekey16", "public"."gbtreekey16", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_time_same"("public"."gbtreekey16", "public"."gbtreekey16", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_time_same"("public"."gbtreekey16", "public"."gbtreekey16", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_time_union"("internal", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_time_union"("internal", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_time_union"("internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_time_union"("internal", "internal") TO "service_role";

```
GRANT ALL ON FUNCTION "public"."gbt_timetz_compress"("internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_timetz_compress"("internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_timetz_compress"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_timetz_compress"("internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_timetz_consistent"("internal", time with
time zone, smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_timetz_consistent"("internal", time with
time zone, smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_timetz_consistent"("internal", time with
time zone, smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_timetz_consistent"("internal", time with
time zone, smallint, "oid", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_ts_compress"("internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_ts_compress"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_ts_compress"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_ts_compress"("internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_ts_consistent"("internal", timestamp
without time zone, smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_ts_consistent"("internal", timestamp
without time zone, smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_ts_consistent"("internal", timestamp
without time zone, smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_ts_consistent"("internal", timestamp
without time zone, smallint, "oid", "internal") TO "service_role";
```

```
GRANT ALL ON FUNCTION "public"."gbt_ts_distance"("internal", timestamp without time zone, smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_ts_distance"("internal", timestamp without time zone, smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_ts_distance"("internal", timestamp without time zone, smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_ts_distance"("internal", timestamp without time zone, smallint, "oid", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_ts_fetch"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_ts_fetch"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_ts_fetch"("internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_ts_fetch"("internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_ts_penalty"("internal", "internal", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_ts_penalty"("internal", "internal", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_ts_penalty"("internal", "internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_ts_penalty"("internal", "internal", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_ts_picksplit"("internal", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_ts_picksplit"("internal", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_ts_picksplit"("internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_ts_picksplit"("internal", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_ts_same"("public"."gbtreekey16", "public"."gbtreekey16", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_ts_same"("public"."gbtreekey16",
```

"public"."gbtreekey16", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_ts_same"("public"."gbtreekey16", "public"."gbtreekey16", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_ts_same"("public"."gbtreekey16", "public"."gbtreekey16", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_ts_union"("internal", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_ts_union"("internal", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_ts_union"("internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_ts_union"("internal", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_tstz_compress"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_tstz_compress"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_tstz_compress"("internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_tstz_compress"("internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_tstz_consistent"("internal", timestamp with time zone, smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_tstz_consistent"("internal", timestamp with time zone, smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_tstz_consistent"("internal", timestamp with time zone, smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_tstz_consistent"("internal", timestamp with time zone, smallint, "oid", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_tstz_distance"("internal", timestamp with time zone, smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_tstz_distance"("internal", timestamp with time zone, smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_tstz_distance"("internal", timestamp

with time zone, smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_tstz_distance"("internal", timestamp with time zone, smallint, "oid", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_uuid_compress"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_uuid_compress"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_uuid_compress"("internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_uuid_compress"("internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_uuid_consistent"("internal", "uuid", smallint, "oid", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_uuid_consistent"("internal", "uuid", smallint, "oid", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_uuid_consistent"("internal", "uuid", smallint, "oid", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_uuid_consistent"("internal", "uuid", smallint, "oid", "internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_uuid_fetch"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_uuid_fetch"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_uuid_fetch"("internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_uuid_fetch"("internal") TO "service_role";

GRANT ALL ON FUNCTION "public"."gbt_uuid_penalty"("internal", "internal", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_uuid_penalty"("internal", "internal", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_uuid_penalty"("internal", "internal", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_uuid_penalty"("internal", "internal", "internal") TO "service_role";

```
GRANT ALL ON FUNCTION "public"."gbt_uuid_picksplit"("internal", "internal")
TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_uuid_picksplit"("internal", "internal")
TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_uuid_picksplit"("internal", "internal")
TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_uuid_picksplit"("internal", "internal")
TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_uuid_same"("public"."gbtreekey32",
"public"."gbtreekey32", "internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_uuid_same"("public"."gbtreekey32",
"public"."gbtreekey32", "internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_uuid_same"("public"."gbtreekey32",
"public"."gbtreekey32", "internal") TO "authenticated";
GRANT ALL ON FUNCTION "public"."gbt_uuid_same"("public"."gbtreekey32",
"public"."gbtreekey32", "internal") TO "service_role";


GRANT ALL ON FUNCTION "public"."gbt_uuid_union"("internal", "internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_uuid_union"("internal", "internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_uuid_union"("internal", "internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_uuid_union"("internal", "internal") TO
"service_role";


GRANT ALL ON FUNCTION "public"."gbt_var_decompress"("internal") TO
"postgres";
GRANT ALL ON FUNCTION "public"."gbt_var_decompress"("internal") TO
"anon";
GRANT ALL ON FUNCTION "public"."gbt_var_decompress"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_var_decompress"("internal") TO
"service_role";
```

```
GRANT ALL ON FUNCTION "public"."gbt_var_fetch"("internal") TO "postgres";
GRANT ALL ON FUNCTION "public"."gbt_var_fetch"("internal") TO "anon";
GRANT ALL ON FUNCTION "public"."gbt_var_fetch"("internal") TO
"authenticated";
GRANT ALL ON FUNCTION "public"."gbt_var_fetch"("internal") TO
"service_role";


REVOKE ALL ON FUNCTION "public"."get_admin_customers_page"("p_search"
"text", "p_limit" integer, "p_offset" integer) FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."get_admin_customers_page"("p_search"
"text", "p_limit" integer, "p_offset" integer) TO "anon";
GRANT ALL ON FUNCTION "public"."get_admin_customers_page"("p_search"
"text", "p_limit" integer, "p_offset" integer) TO "authenticated";
GRANT ALL ON FUNCTION "public"."get_admin_customers_page"("p_search"
"text", "p_limit" integer, "p_offset" integer) TO "service_role";


REVOKE ALL ON FUNCTION "public"."get_admin_dashboard_chart"("p_from"
"date", "p_to" "date", "p_by_month" boolean) FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."get_admin_dashboard_chart"("p_from"
"date", "p_to" "date", "p_by_month" boolean) TO "authenticated";
GRANT ALL ON FUNCTION "public"."get_admin_dashboard_chart"("p_from"
"date", "p_to" "date", "p_by_month" boolean) TO "service_role";


REVOKE ALL ON FUNCTION
"public"."get_admin_dashboard_summary"("p_from" "date", "p_to" "date",
"p_previous_from" "date", "p_previous_to" "date") FROM PUBLIC;
GRANT ALL ON FUNCTION
"public"."get_admin_dashboard_summary"("p_from" "date", "p_to" "date",
"p_previous_from" "date", "p_previous_to" "date") TO "authenticated";
GRANT ALL ON FUNCTION
"public"."get_admin_dashboard_summary"("p_from" "date", "p_to" "date",
"p_previous_from" "date", "p_previous_to" "date") TO "service_role";


REVOKE ALL ON FUNCTION "public"."get_admin_report_page"("p_report_kind"
"text", "p_from" "date", "p_to" "date", "p_status" "text", "p_category" "text",
"p_service_name" "text", "p_limit" integer, "p_offset" integer) FROM PUBLIC;
```

GRANT ALL ON FUNCTION "public"."get_admin_report_page"("p_report_kind" "text", "p_from" "date", "p_to" "date", "p_status" "text", "p_category" "text", "p_service_name" "text", "p_limit" integer, "p_offset" integer) TO "anon";
GRANT ALL ON FUNCTION "public"."get_admin_report_page"("p_report_kind" "text", "p_from" "date", "p_to" "date", "p_status" "text", "p_category" "text", "p_service_name" "text", "p_limit" integer, "p_offset" integer) TO "authenticated";
GRANT ALL ON FUNCTION "public"."get_admin_report_page"("p_report_kind" "text", "p_from" "date", "p_to" "date", "p_status" "text", "p_category" "text", "p_service_name" "text", "p_limit" integer, "p_offset" integer) TO "service_role";

REVOKE ALL ON FUNCTION "public"."get_completed_financial_report_page"("p_from" "date", "p_to" "date", "p_category" "text", "p_service_names" "text"[], "p_limit" integer, "p_offset" integer) FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."get_completed_financial_report_page"("p_from" "date", "p_to" "date", "p_category" "text", "p_service_names" "text"[], "p_limit" integer, "p_offset" integer) TO "authenticated";
GRANT ALL ON FUNCTION "public"."get_completed_financial_report_page"("p_from" "date", "p_to" "date", "p_category" "text", "p_service_names" "text"[], "p_limit" integer, "p_offset" integer) TO "service_role";

REVOKE ALL ON FUNCTION "public"."has_role"("_user_id" "uuid", "_role" "public"."app_role") FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."has_role"("_user_id" "uuid", "_role" "public"."app_role") TO "authenticated";
GRANT ALL ON FUNCTION "public"."has_role"("_user_id" "uuid", "_role" "public"."app_role") TO "service_role";

REVOKE ALL ON FUNCTION "public"."has_role_legacy"("_user_id" "uuid", "_role" "public"."app_role_legacy") FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."has_role_legacy"("_user_id" "uuid", "_role" "public"."app_role_legacy") TO "authenticated";
GRANT ALL ON FUNCTION "public"."has_role_legacy"("_user_id" "uuid", "_role" "public"."app_role_legacy") TO "service_role";

```
GRANT ALL ON FUNCTION "public"."inherit_rescheduled_appointment_email"()
TO "anon";
GRANT ALL ON FUNCTION "public"."inherit_rescheduled_appointment_email"()
TO "authenticated";
GRANT ALL ON FUNCTION "public"."inherit_rescheduled_appointment_email"()
TO "service_role";


GRANT ALL ON FUNCTION "public"."int2_dist"(smallint, smallint) TO "postgres";
GRANT ALL ON FUNCTION "public"."int2_dist"(smallint, smallint) TO "anon";
GRANT ALL ON FUNCTION "public"."int2_dist"(smallint, smallint) TO
"authenticated";
GRANT ALL ON FUNCTION "public"."int2_dist"(smallint, smallint) TO
"service_role";


GRANT ALL ON FUNCTION "public"."int4_dist"(integer, integer) TO "postgres";
GRANT ALL ON FUNCTION "public"."int4_dist"(integer, integer) TO "anon";
GRANT ALL ON FUNCTION "public"."int4_dist"(integer, integer) TO
"authenticated";
GRANT ALL ON FUNCTION "public"."int4_dist"(integer, integer) TO
"service_role";


GRANT ALL ON FUNCTION "public"."int8_dist"(bigint, bigint) TO "postgres";
GRANT ALL ON FUNCTION "public"."int8_dist"(bigint, bigint) TO "anon";
GRANT ALL ON FUNCTION "public"."int8_dist"(bigint, bigint) TO
"authenticated";
GRANT ALL ON FUNCTION "public"."int8_dist"(bigint, bigint) TO "service_role";


GRANT ALL ON FUNCTION "public"."interval_dist"(interval, interval) TO
"postgres";
GRANT ALL ON FUNCTION "public"."interval_dist"(interval, interval) TO "anon";
GRANT ALL ON FUNCTION "public"."interval_dist"(interval, interval) TO
"authenticated";
GRANT ALL ON FUNCTION "public"."interval_dist"(interval, interval) TO
"service_role";
```

```
GRANT ALL ON FUNCTION
"public"."is_crew_available_for_appointment"("p_crew_id" "uuid",
"p_appointment_date" "date", "p_start_time" time without time zone,
"p_duration_minutes" integer, "p_excluded_appointment_id" "uuid",
"p_excluded_continuation_id" "uuid") TO "anon";
GRANT ALL ON FUNCTION
"public"."is_crew_available_for_appointment"("p_crew_id" "uuid",
"p_appointment_date" "date", "p_start_time" time without time zone,
"p_duration_minutes" integer, "p_excluded_appointment_id" "uuid",
"p_excluded_continuation_id" "uuid") TO "authenticated";
GRANT ALL ON FUNCTION
"public"."is_crew_available_for_appointment"("p_crew_id" "uuid",
"p_appointment_date" "date", "p_start_time" time without time zone,
"p_duration_minutes" integer, "p_excluded_appointment_id" "uuid",
"p_excluded_continuation_id" "uuid") TO "service_role";



REVOKE ALL ON FUNCTION "public"."is_current_admin_session"("p_admin_id"
"uuid") FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."is_current_admin_session"("p_admin_id"
"uuid") TO "authenticated";
GRANT ALL ON FUNCTION "public"."is_current_admin_session"("p_admin_id"
"uuid") TO "service_role";



GRANT ALL ON FUNCTION "public"."is_valid_customer_name_part"("p_name"
"text") TO "anon";
GRANT ALL ON FUNCTION "public"."is_valid_customer_name_part"("p_name"
"text") TO "authenticated";
GRANT ALL ON FUNCTION "public"."is_valid_customer_name_part"("p_name"
"text") TO "service_role";



REVOKE ALL ON FUNCTION
"public"."manage_appointment_service_progress"("p_appointment_id" "uuid",
"p_action" "text") FROM PUBLIC;
GRANT ALL ON FUNCTION
"public"."manage_appointment_service_progress"("p_appointment_id" "uuid",
"p_action" "text") TO "anon";
GRANT ALL ON FUNCTION
"public"."manage_appointment_service_progress"("p_appointment_id" "uuid",
"p_action" "text") TO "authenticated";
```

```
GRANT ALL ON FUNCTION
"public"."manage_appointment_service_progress"("p_appointment_id" "uuid",
"p_action" "text") TO "service_role";


GRANT ALL ON FUNCTION "public"."normalize_philippine_mobile"("p_phone"
"text") TO "anon";
GRANT ALL ON FUNCTION "public"."normalize_philippine_mobile"("p_phone"
"text") TO "authenticated";
GRANT ALL ON FUNCTION "public"."normalize_philippine_mobile"("p_phone"
"text") TO "service_role";


GRANT ALL ON FUNCTION
"public"."normalize_philippine_mobile_before_write"() TO "anon";
GRANT ALL ON FUNCTION
"public"."normalize_philippine_mobile_before_write"() TO "authenticated";
GRANT ALL ON FUNCTION
"public"."normalize_philippine_mobile_before_write"() TO "service_role";


REVOKE ALL ON FUNCTION "public"."normalize_plate_number"("p_plate"
"text") FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."normalize_plate_number"("p_plate" "text")
TO "authenticated";
GRANT ALL ON FUNCTION "public"."normalize_plate_number"("p_plate" "text")
TO "service_role";


REVOKE ALL ON FUNCTION "public"."notify_customer_no_show_threshold"()
FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."notify_customer_no_show_threshold"() TO
"anon";
GRANT ALL ON FUNCTION "public"."notify_customer_no_show_threshold"() TO
"authenticated";
GRANT ALL ON FUNCTION "public"."notify_customer_no_show_threshold"() TO
"service_role";


GRANT ALL ON FUNCTION "public"."oid_dist"("oid", "oid") TO "postgres";
```

```
GRANT ALL ON FUNCTION "public"."oid_dist"("oid", "oid") TO "anon";
GRANT ALL ON FUNCTION "public"."oid_dist"("oid", "oid") TO "authenticated";
GRANT ALL ON FUNCTION "public"."oid_dist"("oid", "oid") TO "service_role";


GRANT ALL ON FUNCTION
"public"."prevent_duplicate_active_customer_service_appointments"() TO
"anon";
GRANT ALL ON FUNCTION
"public"."prevent_duplicate_active_customer_service_appointments"() TO
"authenticated";
GRANT ALL ON FUNCTION
"public"."prevent_duplicate_active_customer_service_appointments"() TO
"service_role";


GRANT ALL ON FUNCTION
"public"."prevent_duplicate_active_customer_service_from_appointment"() TO
"anon";
GRANT ALL ON FUNCTION
"public"."prevent_duplicate_active_customer_service_from_appointment"() TO
"authenticated";
GRANT ALL ON FUNCTION
"public"."prevent_duplicate_active_customer_service_from_appointment"() TO
"service_role";


GRANT ALL ON FUNCTION
"public"."prevent_duplicate_active_customer_service_from_service"() TO "anon";
GRANT ALL ON FUNCTION
"public"."prevent_duplicate_active_customer_service_from_service"() TO
"authenticated";
GRANT ALL ON FUNCTION
"public"."prevent_duplicate_active_customer_service_from_service"() TO
"service_role";


GRANT ALL ON FUNCTION "public"."protect_completed_appointments"() TO
"anon";
GRANT ALL ON FUNCTION "public"."protect_completed_appointments"() TO
"authenticated";
```

```
GRANT ALL ON FUNCTION "public"."protect_completed_appointments"() TO
"service_role";


REVOKE ALL ON FUNCTION "public"."purge_expired_admin_activity_logs"()
FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."purge_expired_admin_activity_logs"() TO
"authenticated";
GRANT ALL ON FUNCTION "public"."purge_expired_admin_activity_logs"() TO
"service_role";


REVOKE ALL ON FUNCTION "public"."purge_expired_archived_records"()
FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."purge_expired_archived_records"() TO
"service_role";


GRANT ALL ON FUNCTION
"public"."queue_appointment_completion_notifications"() TO "anon";
GRANT ALL ON FUNCTION
"public"."queue_appointment_completion_notifications"() TO "authenticated";
GRANT ALL ON FUNCTION
"public"."queue_appointment_completion_notifications"() TO "service_role";


GRANT ALL ON FUNCTION "public"."queue_appointment_confirmation_email"()
TO "anon";
GRANT ALL ON FUNCTION "public"."queue_appointment_confirmation_email"()
TO "authenticated";
GRANT ALL ON FUNCTION "public"."queue_appointment_confirmation_email"()
TO "service_role";


GRANT ALL ON FUNCTION "public"."queue_appointment_confirmation_sms"()
TO "anon";
GRANT ALL ON FUNCTION "public"."queue_appointment_confirmation_sms"()
TO "authenticated";
GRANT ALL ON FUNCTION "public"."queue_appointment_confirmation_sms"()
TO "service_role";
```

REVOKE ALL ON FUNCTION "public"."record_admin_activity"() FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."record_admin_activity"() TO "anon";
GRANT ALL ON FUNCTION "public"."record_admin_activity"() TO
"authenticated";
GRANT ALL ON FUNCTION "public"."record_admin_activity"() TO
"service_role";

REVOKE ALL ON FUNCTION "public"."record_admin_activity_event"("p_action"
"text", "p_resource_type" "text", "p_target_label" "text", "p_summary" "text",
"p_changed_fields" "text"[]) FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."record_admin_activity_event"("p_action"
"text", "p_resource_type" "text", "p_target_label" "text", "p_summary" "text",
"p_changed_fields" "text"[]) TO "authenticated";
GRANT ALL ON FUNCTION "public"."record_admin_activity_event"("p_action"
"text", "p_resource_type" "text", "p_target_label" "text", "p_summary" "text",
"p_changed_fields" "text"[]) TO "service_role";

REVOKE ALL ON FUNCTION
"public"."refresh_customer_service_booking_alert"("p_customer_phone" "text",
"p_service_id" "uuid", "p_service_name" "text", "p_preferred_appointment_id"
"uuid", "p_customer_name" "text") FROM PUBLIC;
GRANT ALL ON FUNCTION
"public"."refresh_customer_service_booking_alert"("p_customer_phone" "text",
"p_service_id" "uuid", "p_service_name" "text", "p_preferred_appointment_id"
"uuid", "p_customer_name" "text") TO "service_role";

REVOKE ALL ON FUNCTION "public"."release_admin_session"() FROM
PUBLIC;
GRANT ALL ON FUNCTION "public"."release_admin_session"() TO
"authenticated";
GRANT ALL ON FUNCTION "public"."release_admin_session"() TO
"service_role";

GRANT ALL ON FUNCTION

```
"public"."release_pre_service_product_reservations"() TO "anon";
GRANT ALL ON FUNCTION
"public"."release_pre_service_product_reservations"() TO "authenticated";
GRANT ALL ON FUNCTION
"public"."release_pre_service_product_reservations"() TO "service_role";


REVOKE ALL ON FUNCTION "public"."renew_admin_session"() FROM
PUBLIC;
GRANT ALL ON FUNCTION "public"."renew_admin_session"() TO
"authenticated";
GRANT ALL ON FUNCTION "public"."renew_admin_session"() TO
"service_role";


GRANT ALL ON FUNCTION
"public"."resolve_appointment_service_progress_notifications"() TO "anon";
GRANT ALL ON FUNCTION
"public"."resolve_appointment_service_progress_notifications"() TO
"authenticated";
GRANT ALL ON FUNCTION
"public"."resolve_appointment_service_progress_notifications"() TO
"service_role";


REVOKE ALL ON FUNCTION
"public"."review_pending_appointment"("p_appointment_id" "uuid", "p_decision"
"text") FROM PUBLIC;
GRANT ALL ON FUNCTION
"public"."review_pending_appointment"("p_appointment_id" "uuid", "p_decision"
"text") TO "anon";
GRANT ALL ON FUNCTION
"public"."review_pending_appointment"("p_appointment_id" "uuid", "p_decision"
"text") TO "authenticated";
GRANT ALL ON FUNCTION
"public"."review_pending_appointment"("p_appointment_id" "uuid", "p_decision"
"text") TO "service_role";


REVOKE ALL ON FUNCTION
"public"."review_reschedule_request"("p_appointment_id" "uuid", "p_decision"
```

"text") FROM PUBLIC;
GRANT ALL ON FUNCTION
"public"."review_reschedule_request"("p_appointment_id" "uuid", "p_decision"
"text") TO "anon";
GRANT ALL ON FUNCTION
"public"."review_reschedule_request"("p_appointment_id" "uuid", "p_decision"
"text") TO "authenticated";
GRANT ALL ON FUNCTION
"public"."review_reschedule_request"("p_appointment_id" "uuid", "p_decision"
"text") TO "service_role";

REVOKE ALL ON FUNCTION "public"."set_catalog_archive_timestamp"() FROM
PUBLIC;
GRANT ALL ON FUNCTION "public"."set_catalog_archive_timestamp"() TO
"anon";
GRANT ALL ON FUNCTION "public"."set_catalog_archive_timestamp"() TO
"authenticated";
GRANT ALL ON FUNCTION "public"."set_catalog_archive_timestamp"() TO
"service_role";

REVOKE ALL ON FUNCTION "public"."set_price_history_actor"() FROM
PUBLIC;
GRANT ALL ON FUNCTION "public"."set_price_history_actor"() TO "anon";
GRANT ALL ON FUNCTION "public"."set_price_history_actor"() TO
"authenticated";
GRANT ALL ON FUNCTION "public"."set_price_history_actor"() TO
"service_role";

GRANT ALL ON FUNCTION "public"."set_schedule_block_archive_timestamp"()
TO "anon";
GRANT ALL ON FUNCTION "public"."set_schedule_block_archive_timestamp"()
TO "authenticated";
GRANT ALL ON FUNCTION "public"."set_schedule_block_archive_timestamp"()
TO "service_role";

REVOKE ALL ON FUNCTION
"public"."submit_reschedule_request"("p_appointment_id" "uuid", "p_request_id"

"uuid", "p_appointment_date" "date", "p_start_time" time without time zone,
"p_reason" "text") FROM PUBLIC;
GRANT ALL ON FUNCTION
"public"."submit_reschedule_request"("p_appointment_id" "uuid", "p_request_id"
"uuid", "p_appointment_date" "date", "p_start_time" time without time zone,
"p_reason" "text") TO "anon";
GRANT ALL ON FUNCTION
"public"."submit_reschedule_request"("p_appointment_id" "uuid", "p_request_id"
"uuid", "p_appointment_date" "date", "p_start_time" time without time zone,
"p_reason" "text") TO "authenticated";
GRANT ALL ON FUNCTION
"public"."submit_reschedule_request"("p_appointment_id" "uuid", "p_request_id"
"uuid", "p_appointment_date" "date", "p_start_time" time without time zone,
"p_reason" "text") TO "service_role";

REVOKE ALL ON FUNCTION "public"."sync_customer_service_booking_alert"()
FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."sync_customer_service_booking_alert"()
TO "anon";
GRANT ALL ON FUNCTION "public"."sync_customer_service_booking_alert"()
TO "authenticated";
GRANT ALL ON FUNCTION "public"."sync_customer_service_booking_alert"()
TO "service_role";

GRANT ALL ON FUNCTION "public"."sync_product_stock_status"() TO "anon";
GRANT ALL ON FUNCTION "public"."sync_product_stock_status"() TO
"authenticated";
GRANT ALL ON FUNCTION "public"."sync_product_stock_status"() TO
"service_role";

REVOKE ALL ON FUNCTION "public"."sync_service_progress_notifications"()
FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."sync_service_progress_notifications"() TO
"anon";
GRANT ALL ON FUNCTION "public"."sync_service_progress_notifications"() TO
"authenticated";
GRANT ALL ON FUNCTION "public"."sync_service_progress_notifications"() TO
"service_role";

```
GRANT ALL ON FUNCTION "public"."time_dist"(time without time zone, time without time zone) TO "postgres";
GRANT ALL ON FUNCTION "public"."time_dist"(time without time zone, time without time zone) TO "anon";
GRANT ALL ON FUNCTION "public"."time_dist"(time without time zone, time without time zone) TO "authenticated";
GRANT ALL ON FUNCTION "public"."time_dist"(time without time zone, time without time zone) TO "service_role";


GRANT ALL ON FUNCTION "public"."track_appointment_reschedule"() TO "anon";
GRANT ALL ON FUNCTION "public"."track_appointment_reschedule"() TO "authenticated";
GRANT ALL ON FUNCTION "public"."track_appointment_reschedule"() TO "service_role";


GRANT ALL ON FUNCTION "public"."track_appointment_service_progress"() TO "anon";
GRANT ALL ON FUNCTION "public"."track_appointment_service_progress"() TO "authenticated";
GRANT ALL ON FUNCTION "public"."track_appointment_service_progress"() TO "service_role";


GRANT ALL ON FUNCTION "public"."ts_dist"(timestamp without time zone, timestamp without time zone) TO "postgres";
GRANT ALL ON FUNCTION "public"."ts_dist"(timestamp without time zone, timestamp without time zone) TO "anon";
GRANT ALL ON FUNCTION "public"."ts_dist"(timestamp without time zone, timestamp without time zone) TO "authenticated";
GRANT ALL ON FUNCTION "public"."ts_dist"(timestamp without time zone, timestamp without time zone) TO "service_role";


GRANT ALL ON FUNCTION "public"."tstz_dist"(timestamp with time zone, timestamp with time zone) TO "postgres";
GRANT ALL ON FUNCTION "public"."tstz_dist"(timestamp with time zone,
```

timestamp with time zone) TO "anon";
GRANT ALL ON FUNCTION "public"."tstz_dist"(timestamp with time zone, timestamp with time zone) TO "authenticated";
GRANT ALL ON FUNCTION "public"."tstz_dist"(timestamp with time zone, timestamp with time zone) TO "service_role";

REVOKE ALL ON FUNCTION "public"."update_appointment_details_atomic"("p_appointment_id" "uuid", "p_service_ids" "uuid"[], "p_appointment_date" "date", "p_start_time" time without time zone, "p_assigned_crew_id" "uuid", "p_crew_assignment_manual" boolean, "p_status" "text", "p_admin_notes" "text") FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."update_appointment_details_atomic"("p_appointment_id" "uuid", "p_service_ids" "uuid"[], "p_appointment_date" "date", "p_start_time" time without time zone, "p_assigned_crew_id" "uuid", "p_crew_assignment_manual" boolean, "p_status" "text", "p_admin_notes" "text") TO "anon";
GRANT ALL ON FUNCTION "public"."update_appointment_details_atomic"("p_appointment_id" "uuid", "p_service_ids" "uuid"[], "p_appointment_date" "date", "p_start_time" time without time zone, "p_assigned_crew_id" "uuid", "p_crew_assignment_manual" boolean, "p_status" "text", "p_admin_notes" "text") TO "authenticated";
GRANT ALL ON FUNCTION "public"."update_appointment_details_atomic"("p_appointment_id" "uuid", "p_service_ids" "uuid"[], "p_appointment_date" "date", "p_start_time" time without time zone, "p_assigned_crew_id" "uuid", "p_crew_assignment_manual" boolean, "p_status" "text", "p_admin_notes" "text") TO "service_role";

REVOKE ALL ON FUNCTION "public"."update_appointment_details_atomic"("p_appointment_id" "uuid", "p_service_ids" "uuid"[], "p_appointment_date" "date", "p_start_time" time without time zone, "p_assigned_crew_id" "uuid", "p_crew_assignment_manual" boolean, "p_status" "text", "p_admin_notes" "text", "p_first_name" "text", "p_middle_name" "text", "p_last_name" "text", "p_phone" "text") FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."update_appointment_details_atomic"("p_appointment_id" "uuid", "p_service_ids" "uuid"[], "p_appointment_date" "date", "p_start_time" time without time zone, "p_assigned_crew_id" "uuid", "p_crew_assignment_manual" boolean, "p_status" "text", "p_admin_notes" "text", "p_first_name" "text", "p_middle_name" "text", "p_last_name" "text", "p_phone" "text") TO "anon";
GRANT ALL ON FUNCTION "public"."update_appointment_details_atomic"("p_appointment_id" "uuid",

"p_service_ids" "uuid"[], "p_appointment_date" "date", "p_start_time" time without time zone, "p_assigned_crew_id" "uuid", "p_crew_assignment_manual" boolean, "p_status" "text", "p_admin_notes" "text", "p_first_name" "text", "p_middle_name" "text", "p_last_name" "text", "p_phone" "text") TO "authenticated";
GRANT ALL ON FUNCTION "public"."update_appointment_details_atomic"("p_appointment_id" "uuid", "p_service_ids" "uuid"[], "p_appointment_date" "date", "p_start_time" time without time zone, "p_assigned_crew_id" "uuid", "p_crew_assignment_manual" boolean, "p_status" "text", "p_admin_notes" "text", "p_first_name" "text", "p_middle_name" "text", "p_last_name" "text", "p_phone" "text") TO "service_role";

REVOKE ALL ON FUNCTION "public"."update_appointment_financial_details_atomic"("p_appointment_id" "uuid", "p_service_ids" "uuid"[], "p_appointment_date" "date", "p_start_time" time without time zone, "p_assigned_crew_id" "uuid", "p_crew_assignment_manual" boolean, "p_status" "text", "p_admin_notes" "text", "p_first_name" "text", "p_middle_name" "text", "p_last_name" "text", "p_phone" "text", "p_products" "jsonb") FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."update_appointment_financial_details_atomic"("p_appointment_id" "uuid", "p_service_ids" "uuid"[], "p_appointment_date" "date", "p_start_time" time without time zone, "p_assigned_crew_id" "uuid", "p_crew_assignment_manual" boolean, "p_status" "text", "p_admin_notes" "text", "p_first_name" "text", "p_middle_name" "text", "p_last_name" "text", "p_phone" "text", "p_products" "jsonb") TO "anon";
GRANT ALL ON FUNCTION "public"."update_appointment_financial_details_atomic"("p_appointment_id" "uuid", "p_service_ids" "uuid"[], "p_appointment_date" "date", "p_start_time" time without time zone, "p_assigned_crew_id" "uuid", "p_crew_assignment_manual" boolean, "p_status" "text", "p_admin_notes" "text", "p_first_name" "text", "p_middle_name" "text", "p_last_name" "text", "p_phone" "text", "p_products" "jsonb") TO "authenticated";
GRANT ALL ON FUNCTION "public"."update_appointment_financial_details_atomic"("p_appointment_id" "uuid", "p_service_ids" "uuid"[], "p_appointment_date" "date", "p_start_time" time without time zone, "p_assigned_crew_id" "uuid", "p_crew_assignment_manual" boolean, "p_status" "text", "p_admin_notes" "text", "p_first_name" "text", "p_middle_name" "text", "p_last_name" "text", "p_phone" "text", "p_products" "jsonb") TO "service_role";

REVOKE ALL ON FUNCTION "public"."update_updated_at_column"() FROM

PUBLIC;
GRANT ALL ON FUNCTION "public"."update_updated_at_column"() TO "service_role";
GRANT ALL ON FUNCTION "public"."update_updated_at_column"() TO "authenticated";

REVOKE ALL ON FUNCTION "public"."validate_admin_session"() FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."validate_admin_session"() TO "authenticated";
GRANT ALL ON FUNCTION "public"."validate_admin_session"() TO "service_role";

GRANT ALL ON FUNCTION "public"."validate_appointment_completion_transition"() TO "anon";
GRANT ALL ON FUNCTION "public"."validate_appointment_completion_transition"() TO "authenticated";
GRANT ALL ON FUNCTION "public"."validate_appointment_completion_transition"() TO "service_role";

REVOKE ALL ON FUNCTION "public"."verify_appointment_action_code"("p_appointment_id" "uuid", "p_action" "text", "p_code_hash" "text", "p_token_hash" "text", "p_token_ttl_seconds" integer) FROM PUBLIC;
GRANT ALL ON FUNCTION "public"."verify_appointment_action_code"("p_appointment_id" "uuid", "p_action" "text", "p_code_hash" "text", "p_token_hash" "text", "p_token_ttl_seconds" integer) TO "service_role";

```
GRANT ALL ON TABLE "public"."admin_activity_logs" TO "anon";
GRANT ALL ON TABLE "public"."admin_activity_logs" TO "authenticated";
GRANT ALL ON TABLE "public"."admin_activity_logs" TO "service_role";


GRANT ALL ON TABLE "public"."admin_session_handover_tokens" TO
"service_role";


GRANT ALL ON TABLE "public"."admin_session_leases" TO "service_role";


GRANT ALL ON TABLE "public"."appointment_action_verifications" TO
"service_role";


GRANT ALL ON TABLE "public"."appointment_completion_notifications" TO
"service_role";


GRANT ALL ON TABLE "public"."appointment_confirmation_emails" TO
"service_role";


GRANT ALL ON TABLE "public"."appointment_confirmation_sms" TO
"service_role";
```

```
GRANT ALL ON TABLE "public"."appointment_continuations" TO "anon";
GRANT ALL ON TABLE "public"."appointment_continuations" TO
"authenticated";
GRANT ALL ON TABLE "public"."appointment_continuations" TO "service_role";


GRANT ALL ON TABLE "public"."appointment_products" TO "authenticated";
GRANT ALL ON TABLE "public"."appointment_products" TO "service_role";


GRANT ALL ON TABLE "public"."appointment_reference_recovery_challenges"
TO "service_role";


GRANT ALL ON TABLE "public"."appointment_reschedule_history" TO "anon";
GRANT ALL ON TABLE "public"."appointment_reschedule_history" TO
"authenticated";
GRANT ALL ON TABLE "public"."appointment_reschedule_history" TO
"service_role";


GRANT ALL ON TABLE "public"."appointment_services" TO "anon";
GRANT ALL ON TABLE "public"."appointment_services" TO "authenticated";
GRANT ALL ON TABLE "public"."appointment_services" TO "service_role";


GRANT ALL ON TABLE "public"."appointments" TO "anon";
GRANT ALL ON TABLE "public"."appointments" TO "authenticated";
GRANT ALL ON TABLE "public"."appointments" TO "service_role";


GRANT ALL ON TABLE "public"."blocked_numbers" TO "anon";
GRANT ALL ON TABLE "public"."blocked_numbers" TO "authenticated";
GRANT ALL ON TABLE "public"."blocked_numbers" TO "service_role";
```

```
GRANT ALL ON TABLE "public"."crew_availability_exceptions" TO "anon";
GRANT ALL ON TABLE "public"."crew_availability_exceptions" TO
"authenticated";
GRANT ALL ON TABLE "public"."crew_availability_exceptions" TO
"service_role";


GRANT ALL ON TABLE "public"."crew_members" TO "anon";
GRANT ALL ON TABLE "public"."crew_members" TO "authenticated";
GRANT ALL ON TABLE "public"."crew_members" TO "service_role";


GRANT ALL ON TABLE "public"."crew_schedules" TO "anon";
GRANT ALL ON TABLE "public"."crew_schedules" TO "authenticated";
GRANT ALL ON TABLE "public"."crew_schedules" TO "service_role";


GRANT ALL ON TABLE "public"."customer_service_booking_alert_state" TO
"service_role";


GRANT ALL ON TABLE "public"."customer_service_booking_alerts" TO
"service_role";


GRANT ALL ON TABLE "public"."motorcycle_catalog" TO "anon";
GRANT ALL ON TABLE "public"."motorcycle_catalog" TO "authenticated";
GRANT ALL ON TABLE "public"."motorcycle_catalog" TO "service_role";


GRANT ALL ON TABLE "public"."notifications" TO "anon";
GRANT ALL ON TABLE "public"."notifications" TO "authenticated";
GRANT ALL ON TABLE "public"."notifications" TO "service_role";


GRANT ALL ON TABLE "public"."price_history" TO "anon";
GRANT ALL ON TABLE "public"."price_history" TO "authenticated";
```

```
GRANT ALL ON TABLE "public"."price_history" TO "service_role";


GRANT ALL ON TABLE "public"."products" TO "anon";
GRANT ALL ON TABLE "public"."products" TO "authenticated";
GRANT ALL ON TABLE "public"."products" TO "service_role";


GRANT ALL ON TABLE "public"."public_endpoint_rate_limits" TO "anon";
GRANT ALL ON TABLE "public"."public_endpoint_rate_limits" TO
"authenticated";
GRANT ALL ON TABLE "public"."public_endpoint_rate_limits" TO "service_role";


GRANT
INSERT,REFERENCES,DELETE,TRIGGER,TRUNCATE,MAINTAIN,UPDATE
ON TABLE "public"."schedule_blocks" TO "anon";
GRANT ALL ON TABLE "public"."schedule_blocks" TO "authenticated";
GRANT ALL ON TABLE "public"."schedule_blocks" TO "service_role";


GRANT ALL ON TABLE "public"."services" TO "anon";
GRANT ALL ON TABLE "public"."services" TO "authenticated";
GRANT ALL ON TABLE "public"."services" TO "service_role";


GRANT ALL ON TABLE "public"."shop_settings" TO "anon";
GRANT ALL ON TABLE "public"."shop_settings" TO "authenticated";
GRANT ALL ON TABLE "public"."shop_settings" TO "service_role";


GRANT
INSERT,REFERENCES,DELETE,TRIGGER,TRUNCATE,MAINTAIN,UPDATE
ON TABLE "public"."time_slots" TO "anon";
GRANT ALL ON TABLE "public"."time_slots" TO "authenticated";
GRANT ALL ON TABLE "public"."time_slots" TO "service_role";
```

```
GRANT ALL ON TABLE "public"."user_roles" TO "anon";
GRANT ALL ON TABLE "public"."user_roles" TO "authenticated";
GRANT ALL ON TABLE "public"."user_roles" TO "service_role";
```

```
ALTER DEFAULT PRIVILEGES FOR ROLE "postgres" IN SCHEMA "public"
GRANT ALL ON SEQUENCES TO "postgres";
ALTER DEFAULT PRIVILEGES FOR ROLE "postgres" IN SCHEMA "public"
GRANT ALL ON SEQUENCES TO "anon";
ALTER DEFAULT PRIVILEGES FOR ROLE "postgres" IN SCHEMA "public"
GRANT ALL ON SEQUENCES TO "authenticated";
ALTER DEFAULT PRIVILEGES FOR ROLE "postgres" IN SCHEMA "public"
GRANT ALL ON SEQUENCES TO "service_role";
```

```
ALTER DEFAULT PRIVILEGES FOR ROLE "postgres" IN SCHEMA "public"
GRANT ALL ON FUNCTIONS TO "postgres";
ALTER DEFAULT PRIVILEGES FOR ROLE "postgres" IN SCHEMA "public"
GRANT ALL ON FUNCTIONS TO "anon";
ALTER DEFAULT PRIVILEGES FOR ROLE "postgres" IN SCHEMA "public"
GRANT ALL ON FUNCTIONS TO "authenticated";
ALTER DEFAULT PRIVILEGES FOR ROLE "postgres" IN SCHEMA "public"
GRANT ALL ON FUNCTIONS TO "service_role";
```

```
ALTER DEFAULT PRIVILEGES FOR ROLE "postgres" IN SCHEMA "public"
GRANT ALL ON TABLES TO "postgres";
ALTER DEFAULT PRIVILEGES FOR ROLE "postgres" IN SCHEMA "public"
GRANT ALL ON TABLES TO "anon";
ALTER DEFAULT PRIVILEGES FOR ROLE "postgres" IN SCHEMA "public"
```

```
GRANT ALL ON TABLES TO "authenticated";
ALTER DEFAULT PRIVILEGES FOR ROLE "postgres" IN SCHEMA "public"
GRANT ALL ON TABLES TO "service_role";
```

offline/setup.sql

```
-- Run after loading the live data into the offline copy. Adds what a data copy
-- leaves out: the image buckets with the live rules (migrations 20260821161200
-- and 20260906120000), the scheduled jobs (20260927100000,
20261015100000 and
-- 20261026110000), where the scheduled notifications run calls, and the
-- offline-only tables and job that send offline bookings to the live site.
-- Safe to run again.

INSERT INTO storage.buckets (id, name, public)
VALUES ('products', 'products', true)
ON CONFLICT (id) DO NOTHING;

INSERT INTO storage.buckets (id, name, public, file_size_limit,
allowed_mime_types)
VALUES ('shop-assets', 'shop-assets', true, 5242880, ARRAY['image/jpeg',
'image/png', 'image/webp'])
ON CONFLICT (id) DO UPDATE
  SET public = EXCLUDED.public,
      file_size_limit = EXCLUDED.file_size_limit,
      allowed_mime_types = EXCLUDED.allowed_mime_types;

DROP POLICY IF EXISTS "products admin upload" ON storage.objects;
CREATE POLICY "products admin upload" ON storage.objects
FOR INSERT TO authenticated WITH CHECK (bucket_id = 'products' AND
public.has_role(auth.uid(), 'admin'));

DROP POLICY IF EXISTS "products admin update" ON storage.objects;
CREATE POLICY "products admin update" ON storage.objects
FOR UPDATE TO authenticated USING (bucket_id = 'products' AND
public.has_role(auth.uid(), 'admin'));

DROP POLICY IF EXISTS "products admin delete" ON storage.objects;
CREATE POLICY "products admin delete" ON storage.objects
FOR DELETE TO authenticated USING (bucket_id = 'products' AND
public.has_role(auth.uid(), 'admin'));

DROP POLICY IF EXISTS "products public read" ON storage.objects;
CREATE POLICY "products public read" ON storage.objects
FOR SELECT TO anon, authenticated USING (bucket_id = 'products');

DROP POLICY IF EXISTS "shop assets public read" ON storage.objects;
CREATE POLICY "shop assets public read" ON storage.objects
  FOR SELECT TO public
```

```
  USING (bucket_id = 'shop-assets');

DROP POLICY IF EXISTS "shop assets admin insert" ON storage.objects;
CREATE POLICY "shop assets admin insert" ON storage.objects
  FOR INSERT TO authenticated
  WITH CHECK (bucket_id = 'shop-assets' AND public.has_role(auth.uid(),
'admin'));

DROP POLICY IF EXISTS "shop assets admin update" ON storage.objects;
CREATE POLICY "shop assets admin update" ON storage.objects
  FOR UPDATE TO authenticated
  USING (bucket_id = 'shop-assets' AND public.has_role(auth.uid(), 'admin'))
  WITH CHECK (bucket_id = 'shop-assets' AND public.has_role(auth.uid(),
'admin'));

DROP POLICY IF EXISTS "shop assets admin delete" ON storage.objects;
CREATE POLICY "shop assets admin delete" ON storage.objects
  FOR DELETE TO authenticated
  USING (bucket_id = 'shop-assets' AND public.has_role(auth.uid(), 'admin'));

-- Scheduled jobs. pg_cron runs in UTC: 16:15 and 16:30 UTC are 12:15 AM and
-- 12:30 AM Philippine time.
CREATE EXTENSION IF NOT EXISTS pg_cron;

DO $$
BEGIN
  PERFORM cron.unschedule(jobid)
  FROM cron.job
  WHERE jobname IN (
    'enqueue-due-service-progress-notifications',
    'purge-admin-activity-logs',
    'purge-expired-archives',
    'request-scheduled-notifications',
    'alert-stopped-scheduled-notifications'
  );

  PERFORM cron.schedule(
    'enqueue-due-service-progress-notifications',
    '* * * * *',
    'SELECT public.enqueue_due_service_progress_notifications();'
  );
  PERFORM cron.schedule(
    'purge-admin-activity-logs',
    '15 16 * * *',
    'SELECT public.purge_expired_admin_activity_logs();'
```

```
  );
  PERFORM cron.schedule(
    'purge-expired-archives',
    '30 16 * * *',
    'SELECT public.purge_expired_archived_records();'
  );
  PERFORM cron.schedule(
    'request-scheduled-notifications',
    '*/5 * * * *',
    'SELECT public.request_scheduled_notifications();'
  );
  PERFORM cron.schedule(
    'alert-stopped-scheduled-notifications',
    '*/15 * * * *',
    'SELECT public.alert_if_scheduled_notifications_stopped();'
  );
END;
$$;

-- Where the scheduled notifications run calls: the offline app on this laptop,
-- which Docker reaches as host.docker.internal. The secret must match
-- SCHEDULED_NOTIFICATIONS_SECRET in .env.offline. It only guards the
offline
-- copy, so a fixed value is fine here; the live site uses its own secret.
CREATE EXTENSION IF NOT EXISTS pg_net WITH SCHEMA extensions;

DO $$
DECLARE
  v_secret record;
  v_id uuid;
BEGIN
  FOR v_secret IN
    SELECT *
    FROM (
      VALUES
        ('scheduled_notifications_url',
'http://host.docker.internal:4000/api/scheduled-notifications'),
        ('scheduled_notifications_secret', 'offline-copy-scheduled-notifications')
    ) AS secrets (name, value)
  LOOP
    SELECT id INTO v_id FROM vault.secrets WHERE name = v_secret.name;
    IF v_id IS NULL THEN
      PERFORM vault.create_secret(v_secret.value, v_secret.name);
    ELSE
      PERFORM vault.update_secret(v_id, v_secret.value);
```

```
    END IF;
  END LOOP;
END;
$$;

-- Sending offline bookings to the live site (src/lib/offline-sync.server.ts).
-- These tables exist only in the offline copy, never on the live site.

-- Which offline bookings were handled: added on the live site, or reported to
-- the admin because they no longer fit there.
CREATE TABLE IF NOT EXISTS public.offline_booking_uploads (
  appointment_id uuid PRIMARY KEY REFERENCES public.appointments(id)
ON DELETE CASCADE,
  status text NOT NULL CHECK (status IN ('uploaded', 'conflict')),
  live_appointment_id uuid,
  detail text,
  created_at timestamptz NOT NULL DEFAULT now()
);

-- The latest internet check, which every page shows.
CREATE TABLE IF NOT EXISTS public.offline_sync_status (
  id boolean PRIMARY KEY DEFAULT true CHECK (id),
  online boolean NOT NULL DEFAULT false,
  checked_at timestamptz,
  online_since timestamptz,
  waiting integer NOT NULL DEFAULT 0,
  last_error text
);

ALTER TABLE public.offline_booking_uploads ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.offline_sync_status ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON TABLE public.offline_booking_uploads FROM anon,
authenticated;
REVOKE ALL ON TABLE public.offline_sync_status FROM anon, authenticated;
INSERT INTO public.offline_sync_status (id) VALUES (true) ON CONFLICT (id)
DO NOTHING;

-- Every minute, ask the offline app to check the internet and send offline
-- bookings. The app ignores it unless OFFLINE_SYNC_TO_LIVE=1.
CREATE OR REPLACE FUNCTION public.request_offline_sync()
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
```

```
AS $$
DECLARE
  v_url text;
  v_secret text;
BEGIN
  SELECT replace(decrypted_secret, '/api/scheduled-notifications',
'/api/offline-sync')
    INTO v_url
  FROM vault.decrypted_secrets
  WHERE name = 'scheduled_notifications_url';
  SELECT decrypted_secret INTO v_secret
  FROM vault.decrypted_secrets
  WHERE name = 'scheduled_notifications_secret';
  IF NULLIF(v_url, '') IS NULL OR NULLIF(v_secret, '') IS NULL THEN
    RETURN;
  END IF;

  PERFORM net.http_post(
    url := v_url,
    headers := jsonb_build_object(
      'Content-Type', 'application/json',
      'Authorization', 'Bearer ' || v_secret
    ),
    body := '{}'::jsonb,
    timeout_milliseconds := 55000
  );
END;
$$;

REVOKE ALL ON FUNCTION public.request_offline_sync() FROM PUBLIC,
anon, authenticated;

DO $$
BEGIN
  PERFORM cron.unschedule(jobid) FROM cron.job WHERE jobname =
'request-offline-sync';
  PERFORM cron.schedule('request-offline-sync', '* * * * *', 'SELECT
public.request_offline_sync();');
END;
$$;
```

offline/supabase/config.toml

```
# For detailed configuration reference documentation, visit:
# https://supabase.com/docs/guides/local-development/cli/config
# A string used to distinguish different Supabase projects on the same host.
Defaults to the
# working directory name when running `supabase init`.
project_id = "fake-rider-offline"

[api]
enabled = true
# Port to use for the API URL.
port = 54321
# Schemas to expose in your API. Tables, views and stored procedures in this
schema will get API
# endpoints. `public` and `graphql_public` schemas are included by default.
schemas = ["public", "graphql_public"]
# Extra schemas to add to the search_path of every request.
extra_search_path = ["public", "extensions"]
# The maximum number of rows returns from a view, table, or stored procedure.
Limits payload size
# for accidental or malicious requests.
max_rows = 1000
# Controls whether new tables, views, sequences and functions created in the
`public` schema by
# `postgres` are reachable through the Data API roles (`anon`, `authenticated`,
`service_role`)
# without explicit GRANTs. When unset, new entities are NOT auto-exposed,
matching the new cloud
# default. Set to `true` to keep the legacy behaviour of auto-exposing new
entities; this is
# deprecated and the field is removed on 2026-10-30 once the always-revoked
behaviour is permanent.
# auto_expose_new_tables = true

[api.tls]
# Enable HTTPS endpoints locally using a self-signed certificate.
enabled = false
# Paths to self-signed certificate pair.
# cert_path = "../certs/my-cert.pem"
# key_path = "../certs/my-key.pem"

[db]
# Port to use for the local database URL.
port = 54322
```

```
# Port used by db diff command to initialize the shadow database.
shadow_port = 54320
# Maximum amount of time to wait for health check when starting the local
database.
health_timeout = "2m"
# The database major version to use. This has to be the same as your remote
database's. Run `SHOW
# server_version;` on the remote database to check.
major_version = 17

[db.pooler]
enabled = false
# Port to use for the local connection pooler.
port = 54329
# Specifies when a server connection can be reused by other clients.
# Configure one of the supported pooler modes: `transaction`, `session`.
pool_mode = "transaction"
# How many server connections to allow per user/database pair.
default_pool_size = 20
# Maximum number of client connections allowed.
max_client_conn = 100

# [db.vault]
# secret_key = "env(SECRET_VALUE)"

[db.migrations]
# If disabled, migrations will be skipped during a db push or reset.
enabled = true
# Specifies an ordered list of schema files, directories, or glob patterns that
describe your database.
# Supports paths relative to supabase directory: "./schemas/*.sql", "./database".
schema_paths = []

[db.seed]
# If enabled, seeds the database after migrations during a db reset.
enabled = true
# Specifies an ordered list of seed files to load during db reset.
# Supports glob patterns relative to supabase directory: "./seeds/*.sql"
sql_paths = ["./seed.sql"]

[db.network_restrictions]
# Enable management of network restrictions.
enabled = false
# List of IPv4 CIDR blocks allowed to connect to the database.
# Defaults to allow all IPv4 connections. Set empty array to block all IPs.
```

```
allowed_cidrs = ["0.0.0.0/0"]
# List of IPv6 CIDR blocks allowed to connect to the database.
# Defaults to allow all IPv6 connections. Set empty array to block all IPs.
allowed_cidrs_v6 = ["::/0"]

# Uncomment to reject non-secure connections to the database.
# [db.ssl_enforcement]
# enabled = true

[realtime]
enabled = true
# Bind realtime via either IPv4 or IPv6. (default: IPv4)
# ip_version = "IPv6"
# The maximum length in bytes of HTTP request headers. (default: 4096)
# max_header_length = 4096

[studio]
enabled = true
# Port to use for Supabase Studio.
port = 54323
# External URL of the API server that frontend connects to.
api_url = "http://127.0.0.1"
# OpenAI API Key to use for Supabase AI in the Supabase Studio.
openai_api_key = "env(OPENAI_API_KEY)"

# Email testing server. Emails sent with the local dev setup are not actually sent - rather, they
# are monitored, and you can view the emails that would have been sent from the web interface.
[local_smtp]
enabled = true
# Port to use for the email testing server web interface.
port = 54324
# Uncomment to expose additional ports for testing user applications that send emails.
smtp_port = 54325
# pop3_port = 54326
# admin_email = "admin@email.com"
# sender_name = "Admin"

[storage]
enabled = true
# The maximum file size allowed (e.g. "5MB", "500KB").
file_size_limit = "50MiB"
```

```
# Uncomment to configure local storage buckets
# [storage.buckets.images]
# public = false
# file_size_limit = "50MiB"
# allowed_mime_types = ["image/png", "image/jpeg"]
# objects_path = "./images"

# Allow connections via S3 compatible clients
[storage.s3_protocol]
enabled = true

# Image transformation API is available to Supabase Pro plan.
# [storage.image_transformation]
# enabled = true

# Store analytical data in S3 for running ETL jobs over Iceberg Catalog
# This feature is only available on the hosted platform.
[storage.analytics]
enabled = false
max_namespaces = 5
max_tables = 10
max_catalogs = 2

# Analytics Buckets is available to Supabase Pro plan.
# [storage.analytics.buckets.my-warehouse]

# Store vector embeddings in S3 for large and durable datasets
[storage.vector]
enabled = true
max_buckets = 10
max_indexes = 5

# Vector Buckets is available to Supabase Pro plan.
# [storage.vector.buckets.documents-openai]

[auth]
enabled = true
# The base URL of your website. Used as an allow-list for redirects and for
constructing URLs used
# in emails.
site_url = "http://localhost:4000"
# The public URL that Auth serves on. Defaults to the API external URL with
`/auth/v1` appended.
# external_url = ""
# A list of *exact* URLs that auth providers are permitted to redirect to post
```

```
authentication.
additional_redirect_urls = ["http://localhost:4000/reset-password",
"http://127.0.0.1:4000/reset-password"]
# How long tokens are valid for, in seconds. Defaults to 3600 (1 hour), maximum
604,800 (1 week).
jwt_expiry = 3600
# JWT issuer URL. If not set, defaults to auth.external_url.
# jwt_issuer = ""
# Path to JWT signing key. DO NOT commit your signing keys file to git.
# signing_keys_path = "./signing_keys.json"
# If disabled, the refresh token will never expire.
enable_refresh_token_rotation = true
# Allows refresh tokens to be reused after expiry, up to the specified interval in
seconds.
# Requires enable_refresh_token_rotation = true.
refresh_token_reuse_interval = 10
# Allow/disallow new user signups to your project.
enable_signup = false
# Allow/disallow anonymous sign-ins to your project.
enable_anonymous_sign_ins = false
# Allow/disallow testing manual linking of accounts
enable_manual_linking = false
# Passwords shorter than this value will be rejected as weak. Minimum 6,
recommended 8 or more.
minimum_password_length = 6
# Passwords that do not meet the following requirements will be rejected as
weak. Supported values
# are: `letters_digits`, `lower_upper_letters_digits`,
`lower_upper_letters_digits_symbols`
password_requirements = ""

# Configure passkey sign-ins.
# [auth.passkey]
# enabled = false

# Configure WebAuthn relying party settings (required when passkey is enabled).
# [auth.webauthn]
# rp_display_name = "Supabase"
# rp_id = "localhost"
# rp_origins = ["http://127.0.0.1:4000"]

[auth.rate_limit]
# Number of emails that can be sent per hour. Requires auth.email.smtp to be
enabled.
email_sent = 2
```

```
# Number of SMS messages that can be sent per hour. Requires auth.sms to be
enabled.
sms_sent = 30
# Number of anonymous sign-ins that can be made per hour per IP address.
Requires enable_anonymous_sign_ins = true.
anonymous_users = 30
# Number of sessions that can be refreshed in a 5 minute interval per IP address.
token_refresh = 150
# Number of sign up and sign-in requests that can be made in a 5 minute interval
per IP address (excludes anonymous users).
sign_in_sign_ups = 30
# Number of OTP / Magic link verifications that can be made in a 5 minute
interval per IP address.
token_verifications = 30
# Number of Web3 logins that can be made in a 5 minute interval per IP address.
web3 = 30

# Configure one of the supported captcha providers: `hcaptcha`, `turnstile`.
# [auth.captcha]
# enabled = true
# provider = "hcaptcha"
# secret = ""

[auth.email]
# Allow/disallow new user signups via email to your project.
enable_signup = true
# If enabled, a user will be required to confirm any email change on both the old,
and new email
# addresses. If disabled, only the new email is required to confirm.
double_confirm_changes = true
# If enabled, users need to confirm their email address before signing in.
enable_confirmations = false
# If enabled, users will need to reauthenticate or have logged in recently to
change their password.
secure_password_change = false
# Controls the minimum amount of time that must pass before sending another
signup confirmation or password reset email.
max_frequency = "1s"
# Number of characters used in the email OTP.
otp_length = 6
# Number of seconds before the email OTP expires (defaults to 1 hour).
otp_expiry = 3600

# Use a production-ready SMTP server
# [auth.email.smtp]
```

```
# enabled = true
# host = "smtp.sendgrid.net"
# port = 587
# user = "apikey"
# pass = "env(SENDGRID_API_KEY)"
# admin_email = "admin@email.com"
# sender_name = "Admin"

# Uncomment to customize email template
# [auth.email.template.invite]
# subject = "You have been invited"
# content_path = "./supabase/templates/invite.html"

# Uncomment to customize notification email template
# [auth.email.notification.password_changed]
# enabled = true
# subject = "Your password has been changed"
# content_path = "./supabase/templates/password_changed_notification.html"

[auth.sms]
# Allow/disallow new user signups via SMS to your project.
enable_signup = false
# If enabled, users need to confirm their phone number before signing in.
enable_confirmations = false
# Template for sending OTP to users
template = "Your code is {{ .Code }}"
# Controls the minimum amount of time that must pass before sending another
sms otp.
max_frequency = "5s"

# Use pre-defined map of phone number to OTP for testing.
# [auth.sms.test_otp]
# 4152127777 = "123456"

# Configure logged in session timeouts.
# [auth.sessions]
# Force log out after the specified duration.
# timebox = "24h"
# Force log out if the user has been inactive longer than the specified duration.
# inactivity_timeout = "8h"

# This hook runs before a new user is created and allows developers to reject the
request based on the incoming user object.
# [auth.hook.before_user_created]
# enabled = true
```

```
# uri = "pg-functions://postgres/auth/before-user-created-hook"

# This hook runs before a token is issued and allows you to add additional claims based on the authentication method used.
# [auth.hook.custom_access_token]
# enabled = true
# uri = "pg-functions://<database>/<schema>/<hook_name>"

# Configure one of the supported SMS providers: `twilio`, `twilio_verify`, `messagebird`, `textlocal`, `vonage`.
[auth.sms.twilio]
enabled = false
account_sid = ""
message_service_sid = ""
# DO NOT commit your Twilio auth token to git. Use environment variable substitution instead:
auth_token = "env(SUPABASE_AUTH_SMS_TWILIO_AUTH_TOKEN)"

# Multi-factor-authentication is available to Supabase Pro plan.
[auth.mfa]
# Control how many MFA factors can be enrolled at once per user.
max_enrolled_factors = 10

# Control MFA via App Authenticator (TOTP)
[auth.mfa.totp]
enroll_enabled = false
verify_enabled = false

# Configure MFA via Phone Messaging
[auth.mfa.phone]
enroll_enabled = false
verify_enabled = false
otp_length = 6
template = "Your code is {{ .Code }}"
max_frequency = "5s"

# Configure MFA via WebAuthn
# [auth.mfa.web_authn]
# enroll_enabled = true
# verify_enabled = true

# Use an external OAuth provider. The full list of providers are: `apple`, `azure`, `bitbucket`,
# `discord`, `facebook`, `github`, `gitlab`, `google`, `keycloak`, `linkedin_oidc`, `notion`, `twitch`,
```

```
# `twitter`, `x`, `slack`, `spotify`, `workos`, `zoom`.
[auth.external.apple]
enabled = false
client_id = ""
# DO NOT commit your OAuth provider secret to git. Use environment variable substitution instead:
secret = "env(SUPABASE_AUTH_EXTERNAL_APPLE_SECRET)"
# Overrides the default auth callback URL derived from auth.external_url.
redirect_uri = ""
# Overrides the default auth provider URL. Used to support self-hosted gitlab, single-tenant Azure,
# or any other third-party OIDC providers.
url = ""
# If enabled, the nonce check will be skipped. Required for local sign in with Google auth.
skip_nonce_check = false
# If enabled, it will allow the user to successfully authenticate when the provider does not return an email address.
email_optional = false

# Allow Solana wallet holders to sign in to your project via the Sign in with Solana (SIWS, EIP-4361) standard.
# You can configure "web3" rate limit in the [auth.rate_limit] section and set up [auth.captcha] if self-hosting.
[auth.web3.solana]
enabled = false

# Use Firebase Auth as a third-party provider alongside Supabase Auth.
[auth.third_party.firebase]
enabled = false
# project_id = "my-firebase-project"

# Use Auth0 as a third-party provider alongside Supabase Auth.
[auth.third_party.auth0]
enabled = false
# tenant = "my-auth0-tenant"
# tenant_region = "us"

# Use AWS Cognito (Amplify) as a third-party provider alongside Supabase Auth.
[auth.third_party.aws_cognito]
enabled = false
# user_pool_id = "my-user-pool-id"
# user_pool_region = "us-east-1"

# Use Clerk as a third-party provider alongside Supabase Auth.
```

```
[auth.third_party.clerk]
enabled = false
# Obtain from https://clerk.com/setup/supabase
# domain = "example.clerk.accounts.dev"

# OAuth server configuration
[auth.oauth_server]
# Enable OAuth server functionality
enabled = false
# Path for OAuth consent flow UI
authorization_url_path = "/oauth/consent"
# Allow dynamic client registration
allow_dynamic_registration = false

[edge_runtime]
enabled = false
# Supported request policies: `oneshot`, `per_worker`.
# `per_worker` (default) — enables hot reload during local development.
# `oneshot` — fallback mode if hot reload causes issues (e.g. in large repos or
with symlinks).
policy = "per_worker"
# Port to attach the Chrome inspector for debugging edge functions.
inspector_port = 8083
# The Deno major version to use.
deno_version = 2

# [edge_runtime.secrets]
# secret_key = "env(SECRET_VALUE)"

[analytics]
enabled = false
port = 54327
# Configure one of the supported backends: `postgres`, `bigquery`.
backend = "postgres"

# Experimental features may be deprecated any time
[experimental]
# Configures Postgres storage engine to use OrioleDB (S3)
orioledb_version = ""
# Configures S3 bucket URL, eg. <bucket_name>.s3-<region>.amazonaws.com
s3_host = "env(S3_HOST)"
# Configures S3 bucket region, eg. us-east-1
s3_region = "env(S3_REGION)"
# Configures AWS_ACCESS_KEY_ID for S3 bucket
s3_access_key = "env(S3_ACCESS_KEY)"
```

```
# Configures AWS_SECRET_ACCESS_KEY for S3 bucket
s3_secret_key = "env(S3_SECRET_KEY)"

# pg-delta is the schema diff engine for db diff / db pull / db remote commit.
# Set enabled = false to fall back to the legacy migra engine.
[experimental.pgdelta]
enabled = true
# Directory under `supabase/` where declarative files are written.
# declarative_schema_path = "./schemas"
# JSON string passed through to pg-delta SQL formatting.
# format_options = "{\"keywordCase\":\"upper\",\"indent\":2,\"maxWidth\":80,\"com
maStyle\":\"trailing\"}"
```

package.json

```
{
  "name": "tanstack_start_ts",
  "private": true,
  "sideEffects": false,
  "type": "module",
  "scripts": {
    "dev": "vite dev",
    "build": "vite build",
    "build:dev": "vite build --mode development",
    "preview": "vite preview",
    "lint": "eslint .",
    "format": "prettier --write .",
    "format:check": "prettier --check .",
    "typecheck": "tsc --noEmit",
    "ci": "npm run format:check && npm run lint && npm run typecheck && npm
run build",
    "offline": "vite dev --mode offline",
    "offline:db": "supabase start --workdir offline -x
edge-runtime,logflare,vector,imgproxy,supavisor",
    "offline:db:stop": "supabase stop --workdir offline"
  },
  "dependencies": {
    "@fontsource/barlow": "^5.3.0",
    "@fontsource/barlow-condensed": "^5.3.0",
    "@radix-ui/react-alert-dialog": "^1.1.15",
    "@radix-ui/react-checkbox": "^1.3.3",
    "@radix-ui/react-dialog": "^1.1.15",
    "@radix-ui/react-dropdown-menu": "^2.1.16",
    "@radix-ui/react-label": "^2.1.8",
    "@radix-ui/react-popover": "^1.1.15",
    "@radix-ui/react-select": "^2.2.6",
    "@radix-ui/react-separator": "^1.1.8",
    "@radix-ui/react-slot": "^1.2.4",
    "@radix-ui/react-switch": "^1.2.6",
    "@radix-ui/react-tabs": "^1.1.13",
    "@radix-ui/react-toggle": "^1.1.10",
    "@radix-ui/react-toggle-group": "^1.1.11",
    "@radix-ui/react-tooltip": "^1.2.8",
    "@supabase/supabase-js": "^2.112.3",
    "@tailwindcss/vite": "^4.2.1",
    "@tanstack/react-query": "^5.101.1",
    "@tanstack/react-router": "1.170.45",
    "@tanstack/react-start": "1.169.3",
```

```
    "class-variance-authority": "^0.7.1",
    "clsx": "^2.1.1",
    "date-fns": "^4.1.0",
    "docx": "^9.7.1",
    "jspdf": "^4.2.1",
    "jspdf-autotable": "^5.0.8",
    "lucide-react": "^0.575.0",
    "nodemailer": "^10.0.12",
    "react": "^19.2.0",
    "react-day-picker": "^9.14.0",
    "react-dom": "^19.2.0",
    "recharts": "^2.15.4",
    "tailwind-merge": "^3.5.0",
    "tailwindcss": "^4.2.1",
    "tw-animate-css": "^1.3.4",
    "zod": "^3.24.2"
  },
  "devDependencies": {
    "@eslint/js": "^9.32.0",
    "@types/node": "^22.16.5",
    "@types/nodemailer": "^8.0.2",
    "@types/react": "^19.2.0",
    "@types/react-dom": "^19.2.0",
    "@vitejs/plugin-react": "^5.2.0",
    "eslint": "^9.32.0",
    "eslint-config-prettier": "^10.1.1",
    "eslint-plugin-prettier": "^5.2.6",
    "eslint-plugin-react-hooks": "^5.2.0",
    "eslint-plugin-react-refresh": "^0.4.20",
    "globals": "^15.15.0",
    "nitro": "3.0.260603-beta",
    "prettier": "^3.7.3",
    "typescript": "^5.8.3",
    "typescript-eslint": "^8.56.1",
    "vite": "^8.2.0"
  }
}
```

src/assets/logo-shp.png.asset.json

```
{
  "version": 1,
  "asset_id": "dde1eb34-55d2-4772-8329-28b6ba5ade80",
  "project_id": "d32aea67-c1f8-4aa5-ae1c-84370df11c38",
  "url": "/logo-shp.png",
  "r2_key": "a/v1/d32aea67-c1f8-4aa5-ae1c-84370df11c38/dde1eb34-55d2-4772-
8329-28b6ba5ade80/logo-shp.png",
  "original_filename": "logo-shp.png",
  "size": 12437,
  "content_type": "image/png",
  "created_at": "2026-08-18T22:28:32Z"
}
```

src/components/admin/archive-confirmation-dialog.tsx

```
import {
  AlertDialog,
  AlertDialogAction,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
} from "@/components/ui/alert-dialog";

type ArchiveConfirmationDialogProps = {
  open: boolean;
  recordLabel: string;
  pending?: boolean;
  onOpenChange: (open: boolean) => void;
  onConfirm: () => void;
};

export function ArchiveConfirmationDialog({
  open,
  recordLabel,
  pending = false,
  onOpenChange,
  onConfirm,
}: ArchiveConfirmationDialogProps) {
  return (
    <AlertDialog open={open} onOpenChange={onOpenChange}>
      <AlertDialogContent>
        <AlertDialogHeader>
          <AlertDialogTitle>Archive this {recordLabel}?</AlertDialogTitle>
          <AlertDialogDescription>
            This record will be moved to the Archives page and removed from the
active list.
          </AlertDialogDescription>
        </AlertDialogHeader>
        <AlertDialogFooter>
          <AlertDialogCancel>Cancel</AlertDialogCancel>
          <AlertDialogAction disabled={pending} onClick={onConfirm}>
            Confirm
          </AlertDialogAction>
        </AlertDialogFooter>
      </AlertDialogContent>
```

```
    </AlertDialog>
  );
}
```

src/components/admin/data-list.tsx

```
import {
  Archive,
  CircleAlert,
  EllipsisVertical,
  RotateCcw,
  Search,
  type LucideIcon,
} from "lucide-react";
import type { ReactNode } from "react";

import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Card, CardContent } from "@/components/ui/card";
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuTrigger,
} from "@/components/ui/dropdown-menu";
import { Input } from "@/components/ui/input";
import { Skeleton } from "@/components/ui/skeleton";
import {
  Table,
  TableBody,
  TableCell,
  TableHead,
  TableHeader,
  TableRow,
} from "@/components/ui/table";
import { Tooltip, TooltipContent, TooltipTrigger } from "@/components/ui/tooltip";
import { activeStatusTone } from "@/lib/shop";
import { cn } from "@/lib/utils";

/*
 * Shared building blocks for admin list pages, so every page has the same
 * width, toolbar, list card, and loading/error/empty states.
 *
 * Lists show one card per record below `lg` and a compact table from `lg` up.
 * The admin sidebar leaves too little room for a table on tablets, and cards
 * never need to scroll sideways.
 */

export function AdminPage({ children, className }: { children: ReactNode;
```

```
className?: string }) {
  return <div className={cn("mx-auto w-full max-w-7xl",
className)}>{children}</div>;
}

/** Search, filters, and a result summary above a list. */
export function ListToolbar({ children, summary }: { children: ReactNode;
summary?: ReactNode }) {
  return (
    <div className="mb-4 flex flex-col gap-3 lg:flex-row lg:flex-wrap
lg:items-center">
      {children}
      {summary && <p className="text-sm text-muted-foreground
lg:ml-auto">{summary}</p>}
    </div>
  );
}

export function ToolbarSearch({
  value,
  onChange,
  placeholder,
  label,
  id,
  onEnter,
  className,
}: {
  value: string;
  onChange: (value: string) => void;
  placeholder: string;
  /** Accessible name, e.g. "Search mechanics by name". */
  label: string;
  id?: string;
  /** For toolbars with an explicit apply step. */
  onEnter?: () => void;
  className?: string;
}) {
  return (
    <div className={cn("relative min-w-0 lg:w-72", className)}>
      <Search
        className="pointer-events-none absolute top-1/2 left-3 size-4
-translate-y-1/2 text-muted-foreground"
        aria-hidden="true"
      />
      <Input
```

```
        id={id}
        type="search"
        value={value}
        onChange={(event) => onChange(event.target.value)}
        onKeyDown={
          onEnter
            ? (event) => {
                if (event.key === "Enter") onEnter();
              }
            : undefined
        }
        placeholder={placeholder}
        aria-label={label}
        className="pl-9"
      />
    </div>
  );
}

/** Filter controls: two per row on phones, one line from `sm` up. */
export function ToolbarFilters({ children }: { children: ReactNode }) {
  return (
    <div className="grid grid-cols-2 gap-3 sm:flex sm:flex-wrap
sm:items-center">{children}</div>
  );
}

export function ResetFiltersButton({ onClick }: { onClick: () => void }) {
  return (
    <Button type="button" variant="outline" onClick={onClick}
className="col-span-2 sm:col-span-1">
      <RotateCcw /> Reset filters
    </Button>
  );
}

/** The card that holds a list, with an optional bar above it and footer below. */
export function ListCard({
  children,
  header,
  footer,
  className,
}: {
  children: ReactNode;
  header?: ReactNode;
```

```
  footer?: ReactNode;
  className?: string;
}) {
  return (
    <Card className={cn("overflow-hidden border-border/70 bg-card/60",
className)}>
      {header && <div className="border-b border-border/70 px-4 py-3
sm:px-5">{header}</div>}
      <CardContent className="p-0">{children}</CardContent>
      {footer}
    </Card>
  );
}

/** A section title and description for the bar at the top of a ListCard. */
export function ListCardTitle({
  title,
  description,
  action,
}: {
  title: string;
  description?: ReactNode;
  action?: ReactNode;
}) {
  return (
    <div className="flex flex-wrap items-center justify-between gap-3">
      <div className="min-w-0">
        <h2 className="font-display text-base tracking-wide
uppercase">{title}</h2>
        {description && <p className="mt-0.5 text-xs
text-muted-foreground">{description}</p>}
      </div>
      {action}
    </div>
  );
}

export type DataListColumn<T> = {
  key: string;
  /** Table header, and the field label in the card layout. */
  header: string;
  cell: (row: T) => ReactNode;
  /**
   * Where the value goes on a card: "title" stacks at the top left, "badge" at
   * the top right, "field" in the labelled grid, and "wide" spans that grid.
```

```
   * `false` leaves it off the card (for example, when a title already shows it).
   */
  card?: "title" | "badge" | "field" | "wide" | false;
  /** Leave the column out of the table and show it on cards only. */
  table?: false;
  /** Show the table column from `xl` up only, where there is room for it. */
  xlOnly?: boolean;
  /** Width and alignment for the table column, e.g. "w-32 text-right". */
  className?: string;
};

export function DataList<T>({
  rows,
  columns,
  rowKey,
  label,
  actions,
  actionsClassName = "w-40",
  rowClassName,
  loading,
  error,
  empty,
  loadingRows = 5,
}: {
  rows: readonly T[];
  columns: DataListColumn<T>[];
  rowKey: (row: T) => string;
  /** Accessible name for the list, e.g. "Mechanics". */
  label: string;
  /** Row actions: the last table column and the end of each card. */
  actions?: (row: T) => ReactNode;
  /** Width of the table's actions column. */
  actionsClassName?: string;
  rowClassName?: (row: T) => string | undefined;
  loading?: boolean;
  error?: { message: string; onRetry?: () => void } | null;
  /** Shown when there are no rows and nothing is loading or failing. */
  empty?: ReactNode;
  loadingRows?: number;
}) {
  if (loading)
    return <ListLoading label={`Loading ${label.toLocaleLowerCase()}`}
rows={loadingRows} />;
  if (error) return <ListError message={error.message} onRetry={error.onRetry}
/>;
```

```
  if (rows.length === 0) return <>{empty}</>;

  const titles = columns.filter((column) => column.card === "title");
  const badges = columns.filter((column) => column.card === "badge");
  const fields = columns.filter(
    (column) => column.card === undefined || column.card === "field" ||
column.card === "wide",
  );
  const tableColumns = columns.filter((column) => column.table !== false);
  const lastColumn = tableColumns.length - 1;

  return (
    <>
      <ul aria-label={label} className="divide-y divide-border/70 lg:hidden">
        {rows.map((row) => (
          <li
            key={rowKey(row)}
            className={cn(
              "grid gap-3 p-4 sm:grid-cols-[minmax(0,1fr)_auto] sm:gap-x-6",
              rowClassName?.(row),
            )}
          >
            <div className="min-w-0 space-y-3">
              {(titles.length > 0 || badges.length > 0) && (
                <div className="flex items-start justify-between gap-3">
                  <div className="min-w-0 space-y-0.5 break-words">
                    {titles.map((column) => (
                      <div key={column.key}>{column.cell(row)}</div>
                    ))}
                  </div>
                  {badges.length > 0 && (
                    <div className="flex shrink-0 flex-wrap justify-end gap-1.5">
                      {badges.map((column) => (
                        <span key={column.key}>{column.cell(row)}</span>
                      ))}
                    </div>
                  )}
                </div>
              )}
              {fields.length > 0 && (
                <dl className="grid grid-cols-2 gap-x-4 gap-y-2.5 text-sm
sm:grid-cols-3">
                  {fields.map((column) => (
                    <div
                      key={column.key}
```

```
                  className={cn(
                    "min-w-0",
                    column.card === "wide" && "col-span-2 sm:col-span-3",
                  )}
                >
                  <dt className="text-[11px] tracking-wide text-muted-foreground
uppercase">
                    {column.header}
                  </dt>
                  <dd className="mt-0.5 break-words">{column.cell(row)}</dd>
                </div>
              ))}
            </dl>
          )}
        </div>
        {actions && (
          <div className="flex flex-wrap items-center justify-end gap-2
sm:flex-col sm:items-end sm:justify-start">
            {actions(row)}
          </div>
        )}
      </li>
    ))}
  </ul>

  <Table aria-label={label} className="hidden w-full table-fixed lg:table">
    <TableHeader>
      <TableRow>
        {tableColumns.map((column, index) => (
          <TableHead
            key={column.key}
            className={cn(
              index === 0 && "pl-5",
              index === lastColumn && !actions && "pr-5",
              column.xlOnly && "hidden xl:table-cell",
              column.className,
            )}
          >
            {column.header}
          </TableHead>
        ))}
        {actions && (
          <TableHead className={cn("pr-5 text-right", actionsClassName)}>
            <span className="sr-only">Actions</span>
          </TableHead>
```

```
          )}
        </TableRow>
      </TableHeader>
      <TableBody>
        {rows.map((row) => (
          <TableRow key={rowKey(row)} className={cn("align-top",
rowClassName?.(row))}>
            {tableColumns.map((column, index) => (
              <TableCell
                key={column.key}
                className={cn(
                  "py-3 break-words",
                  index === 0 && "pl-5",
                  index === lastColumn && !actions && "pr-5",
                  column.xlOnly && "hidden xl:table-cell",
                  column.className,
                )}
              >
                {column.cell(row)}
              </TableCell>
            ))}
            {actions && (
              <TableCell className="py-3 pr-5">
                <div className="flex flex-wrap justify-end
gap-2">{actions(row)}</div>
              </TableCell>
            )}
          </TableRow>
        ))}
      </TableBody>
    </Table>
  </>
 );
}

export function ListLoading({ label, rows = 5 }: { label: string; rows?: number }) {
 return (
  <div className="divide-y divide-border/70" role="status" aria-label={label}>
    {Array.from({ length: rows }, (_, index) => (
      <div key={index} className="space-y-2 p-4">
        <Skeleton className="h-4 w-48 max-w-full" />
        <Skeleton className="h-3 w-72 max-w-full" />
      </div>
    ))}
  </div>
```

```
  );
}

export function ListError({
  message,
  onRetry,
}: {
  message: string;
  onRetry?: (() => void) | undefined;
}) {
  return (
    <div className="px-4 py-10 text-center">
      <p className="inline-flex items-center gap-2 text-sm text-destructive">
        <CircleAlert className="size-4 shrink-0" aria-hidden="true" /> {message}
      </p>
      {onRetry && (
        <div className="mt-3">
          <Button size="sm" variant="outline" onClick={onRetry}>
            Try again
          </Button>
        </div>
      )}
    </div>
  );
}

export function ListEmpty({
  icon: Icon,
  title,
  description,
  action,
}: {
  icon: LucideIcon;
  title: string;
  description?: ReactNode;
  action?: ReactNode;
}) {
  return (
    <div className="px-4 py-12 text-center">
      <Icon className="mx-auto size-8 text-muted-foreground/60"
aria-hidden="true" />
      <p className="mt-3 text-sm font-medium">{title}</p>
      {description && <p className="mt-1 text-sm
text-muted-foreground">{description}</p>}
      {action && <div className="mt-4">{action}</div>}
```

```
    </div>
  );
}

/**
 * The "" menu next to a row's main action, holding secondary actions such as
 * View details and Archive. Pass DropdownMenuItem elements as children.
 */
export function MoreActionsMenu({
  itemLabel,
  children,
}: {
  /** Names the record for screen readers, e.g. "More actions for Juan Cruz". */
  itemLabel: string;
  children: ReactNode;
}) {
  return (
    <DropdownMenu>
      <Tooltip>
        <TooltipTrigger asChild>
          <DropdownMenuTrigger asChild>
            <Button
              type="button"
              size="icon"
              variant="ghost"
              aria-label={`More actions for ${itemLabel}`}
            >
              <EllipsisVertical className="size-4" />
            </Button>
          </DropdownMenuTrigger>
        </TooltipTrigger>
        <TooltipContent>More actions</TooltipContent>
      </Tooltip>
      <DropdownMenuContent align="end">{children}</DropdownMenuContent>
    </DropdownMenu>
  );
}

/**
 * The red Archive entry for a MoreActionsMenu. With `unavailableReason`, the
 * entry is disabled and says why.
 */
export function ArchiveMenuItem({
  onSelect,
  disabled,
```

```
  unavailableReason,
}: {
  onSelect: () => void;
  disabled?: boolean;
  unavailableReason?: string | undefined;
}) {
  if (unavailableReason) {
    return (
      <DropdownMenuItem disabled className="max-w-60 flex-col items-start
gap-0.5">
        <span className="flex items-center gap-2">
          <Archive /> Archive
        </span>
        <span className="text-xs
text-muted-foreground">{unavailableReason}</span>
      </DropdownMenuItem>
    );
  }
  return (
    <DropdownMenuItem
      onSelect={onSelect}
      disabled={disabled ?? false}
      className="text-destructive focus:text-destructive"
    >
      <Archive /> Archive
    </DropdownMenuItem>
  );
}

/** A failed action or warning shown above a list. */
export function InlineAlert({ children, className }: { children: ReactNode;
className?: string }) {
  return (
    <p
      role="alert"
      className={cn(
        "mb-4 flex items-start gap-2 rounded-lg border border-destructive/40
bg-destructive/5 p-3 text-sm text-destructive",
        className,
      )}
    >
      <CircleAlert className="mt-0.5 size-4 shrink-0" aria-hidden="true" />
      <span>{children}</span>
    </p>
  );
```

```
}

export function ActiveStatusBadge({
  active,
  activeLabel = "Active",
  inactiveLabel = "Inactive",
}: {
  active: boolean;
  activeLabel?: string;
  inactiveLabel?: string;
}) {
  return (
    <Badge variant="outline" className={cn("uppercase",
activeStatusTone(active))}>
      {active ? activeLabel : inactiveLabel}
    </Badge>
  );
}
```

src/components/admin/motorcycle-catalog-manager.tsx

```
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import { Bike, Eye, Pencil, Plus } from "lucide-react";
import { forwardRef, useImperativeHandle, useMemo, useState } from "react";

import { ArchiveConfirmationDialog } from
"@/components/admin/archive-confirmation-dialog";
import {
  ActiveStatusBadge,
  ArchiveMenuItem,
  DataList,
  InlineAlert,
  ListCard,
  ListEmpty,
  ListToolbar,
  MoreActionsMenu,
  ResetFiltersButton,
  ToolbarFilters,
  ToolbarSearch,
} from "@/components/admin/data-list";
import { PaginationControls } from "@/components/admin/pagination-controls";
import {
  AlertDialog,
  AlertDialogAction,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
} from "@/components/ui/alert-dialog";
import { Button } from "@/components/ui/button";
import { DropdownMenuItem, DropdownMenuSeparator } from
"@/components/ui/dropdown-menu";
import { FieldError } from "@/components/ui/field-error";
import {
  Dialog,
  DialogContent,
  DialogFooter,
  DialogHeader,
  DialogTitle,
} from "@/components/ui/dialog";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
```

```
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@/components/ui/select";
import { Switch } from "@/components/ui/switch";
import { supabase } from "@/integrations/supabase/client";

type MotorcycleCatalogItem = {
  id: string;
  brand: string | null;
  cc_category: string | null;
  model: string;
  engine_cc: number | null;
  is_active: boolean;
  is_archived: boolean;
  created_at: string;
};

const blank = {
  brand: "",
  model: "",
  engineCc: "",
  is_active: true,
};

const ALL_BRANDS = "__all_brands__";
const PAGE_SIZE = 25;

type CategoryFilter = "all" | "small_bike" | "big_bike" | "needs_cc";
type StatusFilter = "all" | "active" | "inactive";

export interface MotorcycleCatalogManagerHandle {
  openNew: () => void;
}

export const MotorcycleCatalogManager =
forwardRef<MotorcycleCatalogManagerHandle>(
  function MotorcycleCatalogManager(_, ref) {
    const queryClient = useQueryClient();
    const [editing, setEditing] = useState<MotorcycleCatalogItem | null>(null);
    const [viewing, setViewing] = useState<MotorcycleCatalogItem | null>(null);
    const [open, setOpen] = useState(false);
```

```
  const [form, setForm] = useState({ ...blank });
  const [formErrors, setFormErrors] = useState<
    Partial<Record<"brand" | "model" | "engineCc", string>>
  >({});
  const [filterBrand, setFilterBrand] = useState(ALL_BRANDS);
  const [modelSearch, setModelSearch] = useState("");
  const [filterCategory, setFilterCategory] = useState<CategoryFilter>("all");
  const [filterStatus, setFilterStatus] = useState<StatusFilter>("all");
  const [page, setPage] = useState(0);
  const [archiveTarget, setArchiveTarget] = useState<string | null>(null);
  const [saveConfirmationOpen, setSaveConfirmationOpen] = useState(false);
  const [actionError, setActionError] = useState<string | null>(null);

  const catalog = useQuery({
    queryKey: ["admin-motorcycle-catalog"],
    queryFn: async (): Promise<MotorcycleCatalogItem[]> => {
      const { data, error } = await supabase
        .from("motorcycle_catalog")
        .select("*")
        .eq("is_archived", false)
        .order("brand")
        .order("model");
      if (error) throw error;
      return data as MotorcycleCatalogItem[];
    },
  });

  const brands = useMemo(
    () =>
      Array.from(
        new Set(
          (catalog.data ?? []).flatMap((item) => (item.brand?.trim() ?
[item.brand.trim()] : [])),
        ),
      ).sort((left, right) => left.localeCompare(right, undefined, { sensitivity: "base"
})),
    [catalog.data],
  );

  const modelSearchTerms = useMemo(
    () => modelSearch.trim().toLocaleLowerCase().split(/ /).filter(Boolean),
    [modelSearch],
  );

  const filteredItems = useMemo(() => {
```

```
      const filtered = (catalog.data ?? []).filter(
        (item) =>
          (filterBrand === ALL_BRANDS || item.brand === filterBrand) &&
          (filterCategory === "all" ||
            (filterCategory === "needs_cc"
              ? item.cc_category !== "small_bike" && item.cc_category !== "big_bike"
              : item.cc_category === filterCategory)) &&
          (filterStatus === "all" || (filterStatus === "active") === item.is_active) &&
          modelSearchTerms.every((term) =>
item.model.toLocaleLowerCase().includes(term)),
      );
      return filtered.sort(
        (left, right) =>
          (left.brand ?? "").localeCompare(right.brand ?? "", undefined, {
            sensitivity: "base",
          }) || left.model.localeCompare(right.model, undefined, { sensitivity: "base"
}),
      );
    }, [catalog.data, filterBrand, filterCategory, filterStatus, modelSearchTerms]);

    const hasFilters =
      filterBrand !== ALL_BRANDS ||
      filterCategory !== "all" ||
      filterStatus !== "all" ||
      modelSearchTerms.length > 0;
    // Hundreds of models: show them a page at a time.
    const lastPage = Math.max(0, Math.ceil(filteredItems.length / PAGE_SIZE) -
1);
    const currentPage = Math.min(page, lastPage);
    const pageItems = filteredItems.slice(currentPage * PAGE_SIZE, (currentPage
+ 1) * PAGE_SIZE);

    function resetFilters() {
      setFilterBrand(ALL_BRANDS);
      setFilterCategory("all");
      setFilterStatus("all");
      setModelSearch("");
      setPage(0);
    }

    const save = useMutation({
      mutationFn: async (): Promise<MotorcycleCatalogItem> => {
        const payload = {
          brand: form.brand.trim(),
          model: form.model.trim(),
```

```
        engine_cc: form.engineCc.trim() ? Number(form.engineCc) : null,
        is_active: form.is_active,
      };
      const result = editing
        ? await supabase
            .from("motorcycle_catalog")
            .update(payload)
            .eq("id", editing.id)
            .select()
            .single()
        : await
supabase.from("motorcycle_catalog").insert(payload).select().single();
      if (result.error) throw result.error;
      return result.data as MotorcycleCatalogItem;
    },
    onSuccess: () => {
      setSaveConfirmationOpen(false);
      closeEditor();
      invalidateCatalogQueries(queryClient);
    },
    onError: (error: Error) => {
      setSaveConfirmationOpen(false);
      const duplicate = /duplicate key|unique/i.test(error.message);
      setFormErrors({
        model: duplicate
          ? "This brand and model are already in the catalog."
          : "Could not save this catalog item. Please try again.",
      });
    },
  });

  const archive = useMutation({
    mutationFn: async (id: string) => {
      const { error } = await supabase
        .from("motorcycle_catalog")
        .update({ is_archived: true })
        .eq("id", id);
      if (error) throw error;
    },
    onSuccess: () => {
      setActionError(null);
      invalidateCatalogQueries(queryClient);
    },
    onError: (error: Error) => {
      console.error("Could not archive motorcycle catalog item:", error);
```

```
        setActionError("Could not archive this motorcycle catalog item. Please try
again.");
      },
    });

    useImperativeHandle(ref, () => ({ openNew }));

    function closeEditor() {
      setSaveConfirmationOpen(false);
      setOpen(false);
      setEditing(null);
      setForm({ ...blank });
      setFormErrors({});
    }

    function openNew() {
      setEditing(null);
      setForm({ ...blank });
      setFormErrors({});
      setOpen(true);
    }

    function openEdit(item: MotorcycleCatalogItem) {
      setEditing(item);
      setForm({
        brand: item.brand ?? "",
        model: item.model,
        engineCc: item.engine_cc === null ? "" : String(item.engine_cc),
        is_active: item.is_active,
      });
      setFormErrors({});
      setOpen(true);
    }

    function validateForm() {
      const errors: typeof formErrors = {};
      const brand = form.brand.trim();
      const model = form.model.trim();
      const hasEngineCc = Boolean(form.engineCc.trim());
      const engineCc = Number(form.engineCc);

      if (!brand) errors.brand = "Enter the motorcycle brand.";
      if (model.length < 2) errors.model = "Enter a model name with at least 2
characters.";
      if (hasEngineCc && (!Number.isFinite(engineCc) || engineCc <= 0 || engineCc
```

```
> 99999.9)) {
        errors.engineCc = "Enter a valid engine displacement in CC.";
      }
      if (!hasEngineCc && !editing) {
        errors.engineCc = "Enter the motorcycle engine displacement in CC.";
      }
      if (!hasEngineCc && form.is_active) {
        errors.engineCc = "A motorcycle without a combustion CC must remain
inactive.";
      }
      if (
        brand &&
        model &&
        (catalog.data ?? []).some(
          (item) =>
            item.id !== editing?.id &&
            item.brand?.trim().toLocaleLowerCase() === brand.toLocaleLowerCase()
&&
            item.model.trim().toLocaleLowerCase() === model.toLocaleLowerCase(),
        )
      ) {
        errors.model = "This brand and model are already in the catalog.";
      }

      setFormErrors(errors);
      return Object.keys(errors).length === 0;
    }

    function requestSave() {
      if (!validateForm()) return;
      if (editing) {
        setSaveConfirmationOpen(true);
        return;
      }
      save.mutate();
    }

    function clearFormError(field: keyof typeof formErrors) {
      setFormErrors((current) => {
        const next = { ...current };
        delete next[field];
        return next;
      });
    }
```

```
    return (
      <>
        {actionError && <InlineAlert>{actionError}</InlineAlert>}

        <ListToolbar
          summary={
            catalog.data && `Showing ${filteredItems.length} of ${catalog.data.length}
motorcycles`
          }
        >
          <ToolbarSearch
            id="catalog-model-search"
            value={modelSearch}
            onChange={(value) => {
              setModelSearch(value);
              setPage(0);
            }}
            placeholder="Search models"
            label="Search motorcycles by model"
          />
          <ToolbarFilters>
            <Select
              value={filterBrand}
              onValueChange={(value) => {
                setFilterBrand(value);
                setPage(0);
              }}
            >
              <SelectTrigger aria-label="Filter by brand" className="w-full sm:w-40">
                <SelectValue placeholder="All brands" />
              </SelectTrigger>
              <SelectContent>
                <SelectItem value={ALL_BRANDS}>All brands</SelectItem>
                {brands.map((brand) => (
                  <SelectItem key={brand} value={brand}>
                    {brand}
                  </SelectItem>
                ))}
              </SelectContent>
            </Select>
            <Select
              value={filterCategory}
              onValueChange={(value) => {
                setFilterCategory(value as CategoryFilter);
                setPage(0);
```

```
          }}
        >
          <SelectTrigger aria-label="Filter by bike size" className="w-full
sm:w-40">
            <SelectValue />
          </SelectTrigger>
          <SelectContent>
            <SelectItem value="all">All bike sizes</SelectItem>
            <SelectItem value="small_bike">Small Bike</SelectItem>
            <SelectItem value="big_bike">Large Bike</SelectItem>
            <SelectItem value="needs_cc">Needs CC</SelectItem>
          </SelectContent>
        </Select>
        <Select
          value={filterStatus}
          onValueChange={(value) => {
            setFilterStatus(value as StatusFilter);
            setPage(0);
          }}
        >
          <SelectTrigger aria-label="Filter by status" className="w-full
sm:w-36">
            <SelectValue />
          </SelectTrigger>
          <SelectContent>
            <SelectItem value="all">All statuses</SelectItem>
            <SelectItem value="active">Active</SelectItem>
            <SelectItem value="inactive">Inactive</SelectItem>
          </SelectContent>
        </Select>
        {hasFilters && <ResetFiltersButton onClick={resetFilters} />}
      </ToolbarFilters>
    </ListToolbar>

    <Dialog open={open} onOpenChange={(nextOpen) => !nextOpen &&
closeEditor()}>
      <DialogContent>
        <DialogHeader>
          <DialogTitle className="font-display uppercase">
            {editing ? "Edit motorcycle" : "New motorcycle"}
          </DialogTitle>
        </DialogHeader>
        <div className="space-y-3">
          <div className="space-y-1.5">
            <Label htmlFor="motorcycle-brand">Brand</Label>
```

```
              <Input
                id="motorcycle-brand"
                list="motorcycle-brand-options"
                value={form.brand}
                onChange={(event) => {
                  setForm({ ...form, brand: event.target.value });
                  clearFormError("brand");
                }}
                placeholder="Type or select a brand"
                aria-invalid={Boolean(formErrors.brand)}
              />
              <datalist id="motorcycle-brand-options">
                {brands.map((brand) => (
                  <option key={brand} value={brand} />
                ))}
              </datalist>
              <FieldError message={formErrors.brand} />
            </div>
            <div className="space-y-1.5">
              <Label htmlFor="motorcycle-model">Model name</Label>
              <Input
                id="motorcycle-model"
                value={form.model}
                onChange={(event) => {
                  setForm({ ...form, model: event.target.value });
                  clearFormError("model");
                }}
                aria-invalid={Boolean(formErrors.model)}
              />
              <FieldError message={formErrors.model} />
            </div>
            <div className="space-y-1.5">
              <Label htmlFor="motorcycle-engine-cc">Engine displacement
(CC)</Label>
              <Input
                id="motorcycle-engine-cc"
                type="number"
                min="0.1"
                max="99999.9"
                step="0.1"
                inputMode="decimal"
                value={form.engineCc}
                onChange={(event) => {
                  setForm({ ...form, engineCc: event.target.value });
                  clearFormError("engineCc");
```

```
              }}
              placeholder="e.g. 125"
              aria-invalid={Boolean(formErrors.engineCc)}
            />
            <p className="text-xs text-muted-foreground">
              125CC and below are Small Bike; 126CC and above are Large Bike.
            </p>
            {form.engineCc.trim() && (
              <p className="text-xs text-muted-foreground">
                Category:
{ccCategoryLabelFromEngineCc(Number(form.engineCc))}
              </p>
            )}
            {!form.engineCc.trim() && editing && (
              <p className="text-xs text-muted-foreground">
                This record has no combustion CC and must stay inactive.
              </p>
            )}
            <FieldError message={formErrors.engineCc} />
          </div>
          <div className="space-y-1.5">
            <Label>Status</Label>
            <label className="flex items-center gap-2 text-sm">
              <Switch
                id="motorcycle-active"
                checked={form.is_active}
                onCheckedChange={(is_active) => setForm({ ...form, is_active })}
              />
              <span
                className={
                  form.is_active
                    ? "font-medium text-emerald-600 dark:text-emerald-300"
                    : "font-medium text-muted-foreground"
                }
              >
                {form.is_active ? "Active" : "Inactive"}
              </span>
            </label>
            <p className="text-xs text-muted-foreground">
              Inactive motorcycles are hidden from customer booking.
            </p>
          </div>
        </div>

        <DialogFooter>
```

```
            <Button type="button" variant="outline" onClick={closeEditor}>
              Close
            </Button>
            <Button type="button" onClick={requestSave}
disabled={save.isPending}>
              {save.isPending ? "Saving..." : editing ? "Update" : "Save motorcycle"}
            </Button>
          </DialogFooter>
        </DialogContent>
      </Dialog>

      <Dialog open={Boolean(viewing)} onOpenChange={(nextOpen) =>
!nextOpen && setViewing(null)}>
        <DialogContent>
          <DialogHeader>
            <DialogTitle className="font-display uppercase">Motorcycle
details</DialogTitle>
          </DialogHeader>
          {viewing && (
            <dl className="grid grid-cols-[auto_1fr] gap-x-6 gap-y-3 text-sm">
              <dt className="text-muted-foreground">Brand</dt>
              <dd>{viewing.brand ?? "-"}</dd>
              <dt className="text-muted-foreground">Model</dt>
              <dd>{modelLabel(viewing.brand, viewing.model)}</dd>
              <dt className="text-muted-foreground">Engine displacement</dt>
              <dd>{viewing.engine_cc === null ? "Not applicable" :
`${viewing.engine_cc}cc`}</dd>
              <dt className="text-muted-foreground">Category</dt>
              <dd>{ccCategoryLabel(viewing.cc_category)}</dd>
              <dt className="text-muted-foreground">Status</dt>
              <dd>{viewing.is_active ? "Active" : "Inactive"}</dd>
            </dl>
          )}
          <DialogFooter>
            <Button type="button" onClick={() => setViewing(null)}>
              Close
            </Button>
          </DialogFooter>
        </DialogContent>
      </Dialog>

      <ListCard
        footer={
          <PaginationControls
            page={currentPage}
```

```
            pageSize={PAGE_SIZE}
            total={filteredItems.length}
            onPageChange={setPage}
          />
        }
      >
        <DataList
          label="Motorcycle catalog"
          rows={pageItems}
          rowKey={(item) => item.id}
          loading={catalog.isLoading}
          error={
            catalog.isError
              ? {
                  message: "Could not load the motorcycle catalog.",
                  onRetry: () => void catalog.refetch(),
                }
              : null
          }
          empty={
            <ListEmpty
              icon={Bike}
              title={hasFilters ? "No motorcycles match your filters." : "No
motorcycles yet."}
              description={
                hasFilters
                  ? "Try a different model, brand, bike size, or status."
                  : "Add the models customers can choose when they book."
              }
              action={
                hasFilters ? (
                  <ResetFiltersButton onClick={resetFilters} />
                ) : (
                  <Button size="sm" onClick={openNew}>
                    <Plus /> Add motorcycle
                  </Button>
                )
              }
            />
          }
          columns={[
            {
              key: "model",
              header: "Model",
              card: "title",
```

```
            cell: (item) => (
              <>
                <p className="font-medium">{modelLabel(item.brand,
item.model)}</p>
                <p className="mt-0.5 text-xs text-muted-foreground">
                  {item.brand ?? "No brand"}
                </p>
              </>
            ),
          },
          {
            key: "engine",
            header: "Engine",
            className: "w-28 text-right",
            cell: (item) =>
              item.engine_cc === null ? (
                <span className="text-sm text-muted-foreground">Not set</span>
              ) : (
                <span className="text-sm">{item.engine_cc}cc</span>
              ),
          },
          {
            key: "size",
            header: "Bike size",
            className: "w-32",
            cell: (item) => (
              <span
className="text-sm">{ccCategoryLabel(item.cc_category)}</span>
            ),
          },
          {
            key: "created",
            header: "Added",
            xlOnly: true,
            className: "w-32",
            cell: (item) => (
              <span className="text-sm text-muted-foreground">
                {formatCreatedOn(item.created_at)}
              </span>
            ),
          },
          {
            key: "status",
            header: "Status",
            card: "badge",
```

```
            className: "w-28 text-center",
            cell: (item) => <ActiveStatusBadge active={item.is_active} />,
          },
        ]}
        actionsClassName="w-36"
        actions={(item) => {
          const name = `${item.brand ?? ""} ${modelLabel(item.brand,
item.model)}`.trim();
          return (
            <>
              <Button size="sm" variant="outline" onClick={() => openEdit(item)}>
                <Pencil /> Edit
              </Button>
              <MoreActionsMenu itemLabel={name}>
                <DropdownMenuItem onSelect={() => setViewing(item)}>
                  <Eye /> View details
                </DropdownMenuItem>
                <DropdownMenuSeparator />
                <ArchiveMenuItem onSelect={() => setArchiveTarget(item.id)} />
              </MoreActionsMenu>
            </>
          );
        }}
      />
    </ListCard>
    <AlertDialog open={saveConfirmationOpen}
onOpenChange={setSaveConfirmationOpen}>
      <AlertDialogContent>
        <AlertDialogHeader>
          <AlertDialogTitle>Save motorcycle changes?</AlertDialogTitle>
          <AlertDialogDescription>
            This will update the selected motorcycle record and its customer
booking visibility.
          </AlertDialogDescription>
        </AlertDialogHeader>
        <AlertDialogFooter>
          <AlertDialogCancel
disabled={save.isPending}>Cancel</AlertDialogCancel>
          <AlertDialogAction disabled={save.isPending} onClick={() =>
save.mutate()}>
            Save Changes
          </AlertDialogAction>
        </AlertDialogFooter>
      </AlertDialogContent>
    </AlertDialog>
```

```
        <ArchiveConfirmationDialog
          open={Boolean(archiveTarget)}
          recordLabel="motorcycle catalog item"
          pending={archive.isPending}
          onOpenChange={(nextOpen) => !nextOpen && setArchiveTarget(null)}
          onConfirm={() => {
            if (archiveTarget) archive.mutate(archiveTarget);
            setArchiveTarget(null);
          }}
        />
      </>
    );
  },
);

function invalidateCatalogQueries(queryClient: ReturnType<typeof
useQueryClient>) {
  queryClient.invalidateQueries({ queryKey: ["admin-motorcycle-catalog"], exact:
false });
  queryClient.invalidateQueries({ queryKey: ["motorcycle-catalog"], exact: false });
  queryClient.invalidateQueries({ queryKey: ["admin-service-model-catalog"],
exact: false });
  queryClient.invalidateQueries({ queryKey: ["archived-motorcycles"], exact: false
});
}

function formatCreatedOn(createdAt: string) {
  return new Intl.DateTimeFormat("en-PH", {
    timeZone: "Asia/Manila",
    month: "short",
    day: "numeric",
    year: "numeric",
  }).format(new Date(createdAt));
}

function modelLabel(brand: string | null, model: string) {
  const prefix = brand ? `${brand} ` : "";
  return model.startsWith(prefix) ? model.slice(prefix.length) : model;
}

function ccCategoryLabel(category: string | null) {
  if (category === "small_bike") return "Small Bike";
  if (category === "big_bike") return "Large Bike";
  return "Needs CC";
}
```

```
function ccCategoryLabelFromEngineCc(engineCc: number) {
  if (!Number.isFinite(engineCc) || engineCc <= 0) return "Needs CC";
  return engineCc <= 125 ? "Small Bike" : "Large Bike";
}
```

src/components/admin/page-header.tsx

```
export function PageHeader({
  title,
  description,
  action,
}: {
  title: string;
  description?: string;
  action?: React.ReactNode;
}) {
  return (
    <div className="mb-6 flex flex-wrap items-end justify-between gap-3">
      <div>
        <h1 className="font-display text-2xl tracking-wide uppercase
md:text-3xl">{title}</h1>
        {description && <p className="mt-1 text-sm
text-muted-foreground">{description}</p>}
      </div>
      {action}
    </div>
  );
}
```

src/components/admin/pagination-controls.tsx

```
import { ChevronLeft, ChevronRight } from "lucide-react";

import { Button } from "@/components/ui/button";

type PaginationControlsProps = {
  page: number;
  pageSize: number;
  total: number;
  onPageChange: (page: number) => void;
};

export function PaginationControls({
  page,
  pageSize,
  total,
  onPageChange,
}: PaginationControlsProps) {
  if (total === 0) return null;

  const pageCount = Math.max(1, Math.ceil(total / pageSize));
  const from = page * pageSize + 1;
  const to = Math.min(total, (page + 1) * pageSize);

  return (
    <div className="flex flex-wrap items-center justify-between gap-3 border-t
border-border px-5 py-3 text-sm text-muted-foreground">
      <span>
        Showing {from}–{to} of {total}
      </span>
      <div className="flex items-center gap-2">
        <Button
          type="button"
          size="sm"
          variant="outline"
          disabled={page === 0}
          onClick={() => onPageChange(page - 1)}
        >
          <ChevronLeft /> Previous
        </Button>
        <span className="text-xs">
          Page {page + 1} of {pageCount}
        </span>
        <Button
```

```
          type="button"
          size="sm"
          variant="outline"
          disabled={page >= pageCount - 1}
          onClick={() => onPageChange(page + 1)}
        >
          Next <ChevronRight />
        </Button>
      </div>
    </div>
  );
}
```

src/components/admin/print-document.tsx

```
import type { ReactNode } from "react";
import { createPortal } from "react-dom";

import exportLogoUrl from "@/assets/export-logo.png";
import { SHOP_EXPORT_NAME, SHOP_OWNER_NAME } from
"@/lib/export-branding";
import { PRINT_DOCUMENT_STYLES } from "@/lib/print-document";
import { SHOP } from "@/lib/shop";

export type PrintDocumentInfo = {
  title: string;
  /** An optional small line under the title, e.g. "Activity record". */
  subtitle?: string;
  generatedAt: string;
  coverage: string;
  scope: string;
};

/**
 * A printable document: the shop header, coverage and scope, the given
 * sections, and the Prepared by / Approved by sign-off. It is hidden on screen
 * and printed with usePrintWhenReady (lib/print-document.ts).
 */
export function PrintDocument({
  info,
  children,
}: {
  info: PrintDocumentInfo | null;
  children?: ReactNode;
}) {
  if (!info || typeof document === "undefined") return null;

  return createPortal(
    <div className="report-print-root">
      <style>{PRINT_DOCUMENT_STYLES}</style>
      <div className="report-print-sheets" />
      {/* Printed only if the document could not be laid out on sheets. */}
      <table className="report-print-page report-print-flow">
        <thead>
          <tr>
            <td />
          </tr>
        </thead>
```

```
<tfoot>
  <tr>
    <td />
  </tr>
</tfoot>
<tbody>
  <tr>
    <td>
      <article className="report-print-document">
        <div className="report-print-content">
          <header className="report-print-header">
            <img src={exportLogoUrl} alt="" className="report-print-logo" />
            <div className="report-print-heading">
              <p>{SHOP_EXPORT_NAME}</p>
              <p>{SHOP.tagline}</p>
              <h1>{info.title}</h1>
              {info.subtitle && <p>{info.subtitle}</p>}
            </div>
            <p className="report-print-generated">
              Generated
              <br />
              {info.generatedAt}
            </p>
          </header>
          <div className="report-print-rule" />
          <section className="report-print-meta">
            <div>
              <span>Coverage</span>
              <strong>{info.coverage}</strong>
            </div>
            <div>
              <span>Report scope</span>
              <strong>{info.scope}</strong>
            </div>
          </section>
          {children}
        </div>
        <div className="report-print-signoff">
          <footer className="report-print-footer">
            <div>
              <span>Prepared by</span>
              <div />
            </div>
            <div>
              <span>Approved by</span>
```

```
                  <strong>{SHOP_OWNER_NAME}</strong>
                  <small>Shop Owner</small>
                  <div />
                </div>
              </footer>
              <p className="report-print-timestamp">Generated on
{info.generatedAt}</p>
            </div>
          </article>
        </td>
      </tr>
    </tbody>
  </table>
 </div>,
 document.body,
 );
}

/** A titled table section of a PrintDocument. Long tables continue across pages.
*/
export function PrintTable({
 title,
 headers,
 columnWidths,
 rows,
 footerRow,
 emptyMessage,
 lastColumn = "amount",
}: {
 title: string;
 headers: string[];
 columnWidths?: string[];
 rows: string[][];
 footerRow?: string[];
 emptyMessage: string;
 /** An amount column is right-aligned; a text column stays left-aligned. */
 lastColumn?: "amount" | "text";
}) {
 return (
  <section className="report-print-section">
   <h2>{title}</h2>
   <table
    className={
     lastColumn === "text" ? "report-print-table report-print-text-end" :
"report-print-table"
```

```
        }
      >
        {columnWidths && (
          <colgroup>
            {columnWidths.map((width, index) => (
              <col key={`${width}-${index}`} style={{ width }} />
            ))}
          </colgroup>
        )}
        <thead>
          <tr>
            {headers.map((header) => (
              <th key={header}>{header}</th>
            ))}
          </tr>
        </thead>
        <tbody>
          {rows.map((row, index) => (
            <tr key={`${row.join("-")}-${index}`}>
              {row.map((cell, cellIndex) => (
                <td key={`${cell}-${cellIndex}`}>{cell}</td>
              ))}
            </tr>
          ))}
          {rows.length === 0 && (
            <tr>
              <td colSpan={headers.length}>{emptyMessage}</td>
            </tr>
          )}
        </tbody>
        {footerRow && rows.length > 0 && (
          <tfoot>
            <tr>
              {footerRow.map((cell, index) => (
                <td key={`${cell}-${index}`}>{cell}</td>
              ))}
            </tr>
          </tfoot>
        )}
      </table>
    </section>
  );
}
```

src/components/admin/product-manager.tsx

```
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import { Package, Pencil, Plus } from "lucide-react";
import { forwardRef, useImperativeHandle, useMemo, useState } from "react";

import { ArchiveConfirmationDialog } from
"@/components/admin/archive-confirmation-dialog";
import {
  ActiveStatusBadge,
  ArchiveMenuItem,
  DataList,
  InlineAlert,
  ListCard,
  ListEmpty,
  ListToolbar,
  MoreActionsMenu,
  ResetFiltersButton,
  ToolbarFilters,
  ToolbarSearch,
} from "@/components/admin/data-list";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { FieldError } from "@/components/ui/field-error";
import {
  Dialog,
  DialogContent,
  DialogFooter,
  DialogHeader,
  DialogTitle,
} from "@/components/ui/dialog";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@/components/ui/select";
import { Switch } from "@/components/ui/switch";
import { Textarea } from "@/components/ui/textarea";
import { supabase } from "@/integrations/supabase/client";
import { formatPHP } from "@/lib/shop";
import { randomId } from "@/lib/utils";
```

```
type Product = {
  id: string;
  name: string;
  category: "part" | "accessory";
  brand: string | null;
  description: string | null;
  price: number;
  image_url: string | null;
  in_stock: boolean;
  stock_quantity: number;
  is_featured: boolean;
  is_active: boolean;
};

const blank = {
  name: "",
  brand: "",
  description: "",
  price: "",
  image_url: "",
  stock_quantity: "",
  is_featured: false,
  is_active: true,
};

const ALL_BRANDS = "__all_brands__";
const NO_BRAND = "__no_brand__";
const NEW_BRAND = "__new_brand__";

export interface ProductManagerHandle {
  openNew: () => void;
}

/** Shop inventory only. Motorcycle records use MotorcycleCatalogManager. */
export const ProductManager = forwardRef<ProductManagerHandle, { category:
"part" | "accessory" }>(
  function ProductManager({ category }, ref) {
    const queryClient = useQueryClient();
    const [editing, setEditing] = useState<Product | null>(null);
    const [open, setOpen] = useState(false);
    const [form, setForm] = useState({ ...blank });
    const [formErrors, setFormErrors] = useState<
      Partial<Record<"name" | "price" | "stockQuantity" | "imageUrl" | "imageFile",
string>>
```

```
  >({});
  const [isCustomBrand, setIsCustomBrand] = useState(false);
  const [filterBrand, setFilterBrand] = useState(ALL_BRANDS);
  const [filterStock, setFilterStock] = useState<"all" | "in_stock" | "out">("all");
  const [search, setSearch] = useState("");
  const [imageSource, setImageSource] = useState<"url" | "local">("url");
  const [uploadedFile, setUploadedFile] = useState<File | null>(null);
  const [uploadPreview, setUploadPreview] = useState<string | null>(null);
  const [uploading, setUploading] = useState(false);
  const [archiveTarget, setArchiveTarget] = useState<string | null>(null);
  const [actionError, setActionError] = useState<string | null>(null);

  useImperativeHandle(ref, () => ({ openNew }));

  const items = useQuery({
    queryKey: ["admin-products", category],
    queryFn: async (): Promise<Product[]> => {
      const { data, error } = await supabase
        .from("products")
        .select("*")
        .in("category", category === "part" ? ["part", "accessory"] : [category])
        .eq("is_archived", false)
        .order("sort_order");
      if (error) throw error;
      return (data ?? []) as Product[];
    },
  });

  const brands = useMemo(
    () =>
      Array.from(
        new Set(
          (items.data ?? []).flatMap((item) => (item.brand?.trim() ?
[item.brand.trim()] : [])),
        ),
      ).sort((left, right) => left.localeCompare(right, undefined, { sensitivity: "base"
})),
    [items.data],
  );

  const searchTerm = search.trim().toLocaleLowerCase();
  const filteredItems = useMemo(
    () =>
      (items.data ?? []).filter(
        (item) =>
```

```
          (!searchTerm ||
            `${item.name} ${item.brand ?? ""} ${item.description ?? ""}`
              .toLocaleLowerCase()
              .includes(searchTerm)) &&
          (filterBrand === ALL_BRANDS || item.brand === filterBrand) &&
          (filterStock === "all" ||
            (filterStock === "in_stock" ? item.in_stock : !item.in_stock)),
        ),
      [filterBrand, filterStock, items.data, searchTerm],
    );
    const hasFilters = Boolean(searchTerm) || filterBrand !== ALL_BRANDS ||
filterStock !== "all";
    function resetFilters() {
      setSearch("");
      setFilterBrand(ALL_BRANDS);
      setFilterStock("all");
    }

    const save = useMutation({
      mutationFn: async (): Promise<Product> => {
        let imageUrl = form.image_url.trim() || null;
        if (imageSource === "local" && uploadedFile) {
          setUploading(true);
          try {
            imageUrl = await uploadImage(uploadedFile);
          } finally {
            setUploading(false);
          }
        }

        const stockQuantity = Number(form.stock_quantity);
        const payload = {
          name: form.name.trim(),
          brand: form.brand.trim() || null,
          description: form.description.trim() || null,
          image_url: imageUrl,
          stock_quantity: stockQuantity,
          in_stock: stockQuantity > 0,
          is_featured: form.is_featured,
          is_active: form.is_active,
          category: editing?.category ?? category,
        };
        const result = editing
          ? await supabase.from("products").update(payload).eq("id",
editing.id).select().single()
```

```
      : await supabase
          .from("products")
          .insert({ ...payload, price: Number(form.price) })
          .select()
          .single();
    if (result.error) throw result.error;
    return result.data as Product;
  },
  onSuccess: () => {
    closeEditor();
    invalidateProductQueries(queryClient);
  },
  onError: (error: Error) => {
    console.error("Could not save product:", error);
    setFormErrors({ name: "Could not save this product. Please try again." });
  },
});

const archive = useMutation({
  mutationFn: async (id: string) => {
    const { error } = await supabase
      .from("products")
      .update({ is_archived: true })
      .eq("id", id);
    if (error) throw error;
  },
  onSuccess: () => {
    setActionError(null);
    invalidateProductQueries(queryClient);
  },
  onError: (error: Error) => {
    console.error("Could not archive product:", error);
    setActionError("Could not archive this product. Please try again.");
  },
});

function openNew() {
  setEditing(null);
  setForm({ ...blank });
  setFormErrors({});
  setIsCustomBrand(false);
  setImageSource("url");
  setUploadedFile(null);
  setUploadPreview(null);
  setOpen(true);
```

```
  }

  function closeEditor() {
    setOpen(false);
    setEditing(null);
    setForm({ ...blank });
    setFormErrors({});
    setUploadedFile(null);
    setUploadPreview(null);
  }

  function openEdit(item: Product) {
    setEditing(item);
    setForm({
      name: item.name,
      brand: item.brand ?? "",
      description: item.description ?? "",
      price: String(item.price),
      image_url: item.image_url ?? "",
      stock_quantity: String(item.stock_quantity),
      is_featured: item.is_featured,
      is_active: item.is_active,
    });
    setIsCustomBrand(Boolean(item.brand) && !brands.includes(item.brand ??
""));
    setImageSource("url");
    setUploadedFile(null);
    setUploadPreview(null);
    setFormErrors({});
    setOpen(true);
  }

  function validateForm() {
    const errors: typeof formErrors = {};
    if (form.name.trim().length < 2) {
      errors.name = "Enter an item name with at least 2 characters.";
    }
    if (
      !editing &&
      (!form.price.trim() || !Number.isFinite(Number(form.price)) ||
Number(form.price) < 0)
    ) {
      errors.price = "Enter a valid price of PHP 0 or more.";
    }
    if (
```

```
      !form.stock_quantity.trim() ||
      !Number.isInteger(Number(form.stock_quantity)) ||
      Number(form.stock_quantity) < 0
    ) {
      errors.stockQuantity = "Enter a whole stock quantity of 0 or more.";
    }
    if (imageSource === "url" && form.image_url.trim()) {
      try {
        new URL(form.image_url.trim());
      } catch {
        errors.imageUrl = "Enter a valid image URL, including https://, or choose
Local Storage.";
      }
    }
    if (imageSource === "local" && uploadedFile &&
!uploadedFile.type.startsWith("image/")) {
      errors.imageFile = "Choose a valid image file.";
    }
    setFormErrors(errors);
    return Object.keys(errors).length === 0;
  }

  function clearFormError(field: keyof typeof formErrors) {
    setFormErrors((current) => {
      const next = { ...current };
      delete next[field];
      return next;
    });
  }

  return (
    <>
      {actionError && <InlineAlert>{actionError}</InlineAlert>}

      <ListToolbar
        summary={items.data && `Showing ${filteredItems.length} of
${items.data.length} items`}
      >
        <ToolbarSearch
          value={search}
          onChange={setSearch}
          placeholder="Search items, brands, or details"
          label="Search shop items by name, brand, or details"
        />
        <ToolbarFilters>
```

```
          <Select value={filterBrand} onValueChange={setFilterBrand}>
            <SelectTrigger aria-label="Filter by brand" className="w-full sm:w-44">
              <SelectValue placeholder="All brands" />
            </SelectTrigger>
            <SelectContent>
              <SelectItem value={ALL_BRANDS}>All brands</SelectItem>
              {brands.map((brand) => (
                <SelectItem key={brand} value={brand}>
                  {brand}
                </SelectItem>
              ))}
            </SelectContent>
          </Select>
          <Select
            value={filterStock}
            onValueChange={(value) => setFilterStock(value as "all" | "in_stock" |
"out")}
          >
            <SelectTrigger aria-label="Filter by stock" className="w-full sm:w-40">
              <SelectValue />
            </SelectTrigger>
            <SelectContent>
              <SelectItem value="all">All stock</SelectItem>
              <SelectItem value="in_stock">In stock</SelectItem>
              <SelectItem value="out">Out of stock</SelectItem>
            </SelectContent>
          </Select>
          {hasFilters && <ResetFiltersButton onClick={resetFilters} />}
        </ToolbarFilters>
      </ListToolbar>

      <Dialog open={open} onOpenChange={(nextOpen) => !nextOpen &&
closeEditor()}>
        <DialogContent>
          <DialogHeader>
            <DialogTitle className="font-display uppercase">
              {editing ? "Edit product" : "New product"}
            </DialogTitle>
          </DialogHeader>
          <div className="space-y-3">
            <div className="space-y-1.5">
              <Label htmlFor={`product-name-${category}`}>Item name</Label>
              <Input
                id={`product-name-${category}`}
                value={form.name}
```

```
                onChange={(event) => {
                  setForm((current) => ({ ...current, name: event.target.value }));
                  clearFormError("name");
                }}
                aria-invalid={Boolean(formErrors.name)}
              />
              <FieldError message={formErrors.name} />
            </div>
            <div className="space-y-1.5">
              <Label htmlFor={`product-brand-${category}`}>Brand</Label>
              <Select
                value={isCustomBrand ? NEW_BRAND : form.brand || NO_BRAND}
                onValueChange={(value) => {
                  if (value === NEW_BRAND) {
                    setIsCustomBrand(true);
                    setForm((current) => ({ ...current, brand: "" }));
                  } else {
                    setIsCustomBrand(false);
                    setForm((current) => ({
                      ...current,
                      brand: value === NO_BRAND ? "" : value,
                    }));
                  }
                }}
              >
                <SelectTrigger id={`product-brand-${category}`}>
                  <SelectValue placeholder="Select a brand" />
                </SelectTrigger>
                <SelectContent>
                  <SelectItem value={NO_BRAND}>No brand</SelectItem>
                  {brands.map((brand) => (
                    <SelectItem key={brand} value={brand}>
                      {brand}
                    </SelectItem>
                  ))}
                  <SelectItem value={NEW_BRAND}>Add a new
brand…</SelectItem>
                </SelectContent>
              </Select>
              {isCustomBrand && (
                <Input
                  value={form.brand}
                  onChange={(event) =>
                    setForm((current) => ({ ...current, brand: event.target.value }))
                  }
```

```
                placeholder="Enter a new brand"
                aria-label="New brand"
              />
            )}
          </div>
          <div className="space-y-3">
            <div className="space-y-1.5">
              <Label>Image source</Label>
              <Select
                value={imageSource}
                onValueChange={(value: "url" | "local") => {
                  setImageSource(value);
                  if (value === "url") {
                    setUploadedFile(null);
                    setUploadPreview(null);
                  } else {
                    setForm((current) => ({ ...current, image_url: "" }));
                  }
                }}
              >
                <SelectTrigger>
                  <SelectValue />
                </SelectTrigger>
                <SelectContent>
                  <SelectItem value="url">URL</SelectItem>
                  <SelectItem value="local">Local Storage</SelectItem>
                </SelectContent>
              </Select>
            </div>
            {imageSource === "url" ? (
              <div className="space-y-1.5">
                <Label>Image URL</Label>
                <Input
                  value={form.image_url}
                  onChange={(event) => {
                    setForm((current) => ({ ...current, image_url: event.target.value
}));
                    clearFormError("imageUrl");
                  }}
                  placeholder="https://"
                  aria-invalid={Boolean(formErrors.imageUrl)}
                />
                <FieldError message={formErrors.imageUrl} />
              </div>
            ) : (
```

```
              <div className="space-y-1.5">
                <Label>Upload image</Label>
                <Input
                  type="file"
                  accept="image/*"
                  onChange={(event) => {
                    const file = event.target.files?.[0] ?? null;
                    setUploadedFile(file);
                    clearFormError("imageFile");
                    if (!file) {
                      setUploadPreview(null);
                      return;
                    }
                    const reader = new FileReader();
                    reader.onloadend = () => setUploadPreview(reader.result as
string);
                    reader.readAsDataURL(file);
                  }}
                  aria-invalid={Boolean(formErrors.imageFile)}
                />
                <FieldError message={formErrors.imageFile} />
                {uploadPreview && (
                  <img
                    src={uploadPreview}
                    alt="Upload preview"
                    className="h-24 w-24 rounded-md object-cover"
                  />
                )}
              </div>
            )}
          </div>
          {editing ? (
            <p className="text-xs text-muted-foreground">
              Current price: {formatPHP(editing.price)}. Change prices in Price
Management.
            </p>
          ) : (
            <div className="space-y-1.5">
              <Label>Price (PHP)</Label>
              <Input
                type="number"
                min="0"
                step="0.01"
                inputMode="decimal"
                value={form.price}
```

```
                onChange={(event) => {
                  setForm((current) => ({ ...current, price: event.target.value }));
                  clearFormError("price");
                }}
                placeholder="0.00"
                aria-invalid={Boolean(formErrors.price)}
              />
              <FieldError message={formErrors.price} />
            </div>
          )}
          <div className="space-y-1.5">
            <Label>Stock quantity</Label>
            <Input
              type="number"
              min="0"
              step="1"
              inputMode="numeric"
              value={form.stock_quantity}
              onChange={(event) => {
                setForm((current) => ({ ...current, stock_quantity: event.target.value
}));
                clearFormError("stockQuantity");
              }}
              placeholder="0"
              aria-invalid={Boolean(formErrors.stockQuantity)}
            />
            <FieldError message={formErrors.stockQuantity} />
          </div>
          <div className="space-y-1.5">
            <Label>Description</Label>
            <Textarea
              value={form.description}
              onChange={(event) =>
                setForm((current) => ({ ...current, description: event.target.value }))
              }
            />
          </div>
          <div className="flex flex-wrap gap-6 pt-1">
            <div className="space-y-1.5">
              <Label htmlFor="product-active">Status</Label>
              <label className="flex items-center gap-2 text-sm">
                <Switch
                  id="product-active"
                  checked={form.is_active}
                  onCheckedChange={(is_active) =>
```

```
            setForm((current) => ({ ...current, is_active }))
          }
        />
        <span
          className={
            form.is_active
              ? "font-medium text-emerald-600 dark:text-emerald-300"
              : "font-medium text-muted-foreground"
          }
        >
          {form.is_active ? "Active" : "Inactive"}
        </span>
      </label>
    </div>
    <div className="space-y-1.5">
      <Label htmlFor="product-featured">Featured product</Label>
      <label className="flex items-center gap-2 text-sm">
        <Switch
          id="product-featured"
          checked={form.is_featured}
          onCheckedChange={(is_featured) =>
            setForm((current) => ({ ...current, is_featured }))
          }
        />
        <span
          className={
            form.is_featured
              ? "font-medium text-emerald-600 dark:text-emerald-300"
              : "font-medium text-muted-foreground"
          }
        >
          {form.is_featured ? "Featured" : "Unfeatured"}
        </span>
      </label>
    </div>
  </div>
</div>
<DialogFooter>
  <Button
    type="button"
    variant="outline"
    onClick={closeEditor}
    disabled={save.isPending || uploading}
  >
    Close
```

```
            </Button>
            <Button
              type="button"
              disabled={save.isPending || uploading}
              onClick={() => validateForm() && save.mutate()}
            >
              {save.isPending || uploading ? "Saving…" : "Save product"}
            </Button>
          </DialogFooter>
        </DialogContent>
      </Dialog>

      <ListCard>
        <DataList
          label="Shop items"
          rows={filteredItems}
          rowKey={(item) => item.id}
          loading={items.isLoading}
          error={
            items.isError
              ? { message: "Could not load shop items.", onRetry: () => void
items.refetch() }
              : null
          }
          empty={
            <ListEmpty
              icon={Package}
              title={hasFilters ? "No items match your filters." : "No items yet."}
              description={
                hasFilters
                  ? "Try a different search, brand, or stock filter."
                  : "Add parts and accessories to show them in the public shop."
              }
              action={
                hasFilters ? (
                  <ResetFiltersButton onClick={resetFilters} />
                ) : (
                  <Button size="sm" onClick={openNew}>
                    <Plus /> Add item
                  </Button>
                )
              }
            />
          }
          columns={[
```

```
            {
              key: "item",
              header: "Item",
              card: "title",
              cell: (item) => (
                <>
                  <p className="font-medium">{item.name}</p>
                  <p className="mt-0.5 text-xs text-muted-foreground">
                    <span className="capitalize">{item.category}</span>
                    {item.brand ? ` · ${item.brand}` : ""}
                  </p>
                </>
              ),
            },
            {
              key: "price",
              header: "Price",
              className: "w-32 text-right",
              cell: (item) => (
                <span className="text-sm
text-primary">{formatPHP(item.price)}</span>
              ),
            },
            {
              key: "stock",
              header: "Stock",
              className: "w-32 text-right",
              cell: (item) => (
                <span className="text-sm">
                  <span
className="font-mono">{item.stock_quantity.toLocaleString()}</span>
                  {!item.in_stock && (
                    <span className="block text-xs text-muted-foreground">Out of
stock</span>
                  )}
                </span>
              ),
            },
            {
              key: "status",
              header: "Status",
              card: "badge",
              className: "w-32 text-center",
              cell: (item) => (
                <span className="inline-flex flex-wrap justify-center gap-1.5">
```

```
                <ActiveStatusBadge active={item.is_active} />
                {item.is_featured && (
                  <Badge className="bg-primary text-primary-foreground
uppercase">
                    Featured
                  </Badge>
                )}
              </span>
            ),
          },
        ]}
        actionsClassName="w-36"
        actions={(item) => (
          <>
            <Button size="sm" variant="outline" onClick={() => openEdit(item)}>
              <Pencil /> Edit
            </Button>
            <MoreActionsMenu itemLabel={item.name}>
              <ArchiveMenuItem onSelect={() => setArchiveTarget(item.id)} />
            </MoreActionsMenu>
          </>
        )}
      />
    </ListCard>

    <ArchiveConfirmationDialog
      open={Boolean(archiveTarget)}
      recordLabel="shop product"
      pending={archive.isPending}
      onOpenChange={(nextOpen) => !nextOpen && setArchiveTarget(null)}
      onConfirm={() => {
        if (archiveTarget) archive.mutate(archiveTarget);
        setArchiveTarget(null);
      }}
    />
  </>
 );
},
);

function invalidateProductQueries(queryClient: ReturnType<typeof
useQueryClient>) {
  queryClient.invalidateQueries({ queryKey: ["admin-products"], exact: false });
  queryClient.invalidateQueries({ queryKey: ["archived-products"], exact: false });
  queryClient.invalidateQueries({ queryKey: ["products"], exact: false });
```

```
  queryClient.invalidateQueries({ queryKey: ["featured-products"], exact: false });
}

async function uploadImage(file: File): Promise<string> {
  const extension = file.name.split(".").pop()?.toLowerCase() ?? "jpg";
  const path = `${randomId()}.${extension}`;
  const { error } = await supabase.storage.from("products").upload(path, file, {
    cacheControl: "3600",
    upsert: false,
  });
  if (error) throw error;
  return supabase.storage.from("products").getPublicUrl(path).data.publicUrl;
}
```

src/components/admin/settings-page.tsx

```
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import type { User } from "@supabase/supabase-js";
import {
  BellRing,
  Building2,
  Eye,
  EyeOff,
  ImageUp,
  Loader2,
  LockKeyhole,
  Save,
  Store,
  UserRound,
} from "lucide-react";
import { useEffect, useRef, useState } from "react";

import { PageHeader } from "@/components/admin/page-header";
import {
  AlertDialog,
  AlertDialogAction,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
} from "@/components/ui/alert-dialog";
import { Button } from "@/components/ui/button";
import { Card, CardContent } from "@/components/ui/card";
import { FieldError } from "@/components/ui/field-error";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@/components/ui/select";
import { Switch } from "@/components/ui/switch";
import { Tabs, TabsContent, TabsList, TabsTrigger } from
"@/components/ui/tabs";
import { Textarea } from "@/components/ui/textarea";
```

```
import { supabase } from "@/integrations/supabase/client";
import { recordAdminActivityEvent } from "@/lib/admin-activity";
import { DEFAULT_BOOKING_HOURS, DEFAULT_BOOKING_TERMS, SHOP
} from "@/lib/shop";
import { cn, randomId } from "@/lib/utils";

type SettingsTab = "shop" | "account";

type ShopSettingsForm = {
  address: string;
  contactNumber: string;
  contactEmail: string;
  bookingTerms: string;
  openingTime: string;
  closingTime: string;
  minimumBookingLeadHours: number;
  maxAdvanceBookingDays: number;
  defaultAppointmentDurationMinutes: number;
  allowSameDayAppointments: boolean;
  cancellationNoticeHours: number;
  reschedulingNoticeHours: number;
  sendReminders: boolean;
  reminderHoursBefore: number;
};

type AccountForm = {
  name: string;
  email: string;
  currentPassword: string;
  newPassword: string;
  confirmNewPassword: string;
  avatarUrl: string | null;
};

type SettingsErrorField = "contactEmail";

type AccountErrorField =
  "name" | "email" | "currentPassword" | "newPassword" | "confirmNewPassword"
| "profilePicture";

const defaultSettings: ShopSettingsForm = {
  address: SHOP.address,
  contactNumber: SHOP.phone,
  contactEmail: SHOP.email,
  bookingTerms: DEFAULT_BOOKING_TERMS,
```

```
  openingTime: DEFAULT_BOOKING_HOURS.openingTime,
  closingTime: DEFAULT_BOOKING_HOURS.closingTime,
  minimumBookingLeadHours: SHOP.noticeHours,
  maxAdvanceBookingDays: 30,
  defaultAppointmentDurationMinutes: 90,
  allowSameDayAppointments: false,
  cancellationNoticeHours: SHOP.noticeHours,
  reschedulingNoticeHours: SHOP.noticeHours,
  sendReminders: true,
  reminderHoursBefore: 24,
};

/** Reminder timings offered in Settings, plus the saved one if it differs. */
function reminderHourOptions(saved: number) {
  return [...new Set([2, 6, 12, 24, 48, saved])].sort((a, b) => a - b);
}

function reminderTimingLabel(hours: number) {
  if (hours % 24 === 0) return hours === 24 ? "1 day before" : `${hours / 24} days
before`;
  return hours === 1 ? "1 hour before" : `${hours} hours before`;
}

const settingFieldLabels: Array<[keyof ShopSettingsForm, string]> = [
  ["address", "Address"],
  ["contactNumber", "Contact number"],
  ["contactEmail", "Contact email"],
  ["bookingTerms", "Booking terms & conditions"],
  ["sendReminders", "Appointment reminders"],
  ["reminderHoursBefore", "Reminder timing"],
];

const tabLabels: Record<Exclude<SettingsTab, "account">, string> = {
  shop: "Shop information",
};

const settingKeysByTab: Record<Exclude<SettingsTab, "account">, Array<keyof
ShopSettingsForm>> = {
  shop: [
    "address",
    "contactNumber",
    "contactEmail",
    "bookingTerms",
    "sendReminders",
    "reminderHoursBefore",
```

```
  ],
};

function fromDatabase(row: {
  address: string;
  contact_number: string;
  contact_email: string;
  booking_terms: string;
  opening_time: string;
  closing_time: string;
  minimum_booking_lead_hours: number;
  max_advance_booking_days: number;
  default_appointment_duration_minutes: number;
  allow_same_day_appointments: boolean;
  cancellation_notice_hours: number;
  rescheduling_notice_hours: number;
  notify_reminder: boolean;
  reminder_hours_before: number;
}): ShopSettingsForm {
  return {
    address: row.address,
    contactNumber: row.contact_number,
    contactEmail: row.contact_email,
    bookingTerms: row.booking_terms,
    openingTime: row.opening_time.slice(0, 5),
    closingTime: row.closing_time.slice(0, 5),
    minimumBookingLeadHours: row.minimum_booking_lead_hours,
    maxAdvanceBookingDays: row.max_advance_booking_days,
    defaultAppointmentDurationMinutes:
row.default_appointment_duration_minutes,
    allowSameDayAppointments: row.allow_same_day_appointments,
    cancellationNoticeHours: row.cancellation_notice_hours,
    reschedulingNoticeHours: row.rescheduling_notice_hours,
    sendReminders: row.notify_reminder,
    reminderHoursBefore: row.reminder_hours_before,
  };
}

function changedSettingFields(
  previous: ShopSettingsForm,
  next: ShopSettingsForm,
  section: Exclude<SettingsTab, "account">,
) {
  return settingFieldLabels
    .filter(([key]) => settingKeysByTab[section].includes(key) && previous[key] !==
```

```
next[key])
    .map(([, label]) => label);
}

async function uploadAsset(file: File, folder: string) {
  if (!file.type.match(/^image\/(jpeg|png|webp)$/)) {
    throw new Error("Choose a JPG, PNG, or WebP image.");
  }
  if (file.size > 5 * 1024 * 1024) throw new Error("Images must be 5 MB or
smaller.");

  const extension = file.type.split("/")[1] ?? "png";
  const path = `${folder}/${randomId()}.${extension}`;
  const { error } = await supabase.storage.from("shop-assets").upload(path, file, {
    cacheControl: "3600",
    contentType: file.type,
    upsert: false,
  });
  if (error) throw error;
  return supabase.storage.from("shop-assets").getPublicUrl(path).data.publicUrl;
}

export function SettingsPage() {
  const queryClient = useQueryClient();
  const [tab, setTab] = useState<SettingsTab>("shop");
  const [settings, setSettings] = useState<ShopSettingsForm>(defaultSettings);
  const bookingTermsRef = useRef<HTMLTextAreaElement>(null);
  const [user, setUser] = useState<User | null>(null);
  const [account, setAccount] = useState<AccountForm>({
    name: "",
    email: "",
    currentPassword: "",
    newPassword: "",
    confirmNewPassword: "",
    avatarUrl: null,
  });
  const [avatarFile, setAvatarFile] = useState<File | null>(null);
  const [avatarPreview, setAvatarPreview] = useState<string | null>(null);
  const [showCurrentPassword, setShowCurrentPassword] = useState(false);
  const [showNewPassword, setShowNewPassword] = useState(false);
  const [showConfirmNewPassword, setShowConfirmNewPassword] =
useState(false);
  const [settingsErrors, setSettingsErrors] = useState<
    Partial<Record<SettingsErrorField, string | undefined>>
  >({});
```

```
const [accountErrors, setAccountErrors] = useState<
  Partial<Record<AccountErrorField, string | undefined>>
>({});
const [pendingSave, setPendingSave] = useState<SettingsTab | null>(null);

const savedSettings = useQuery({
  queryKey: ["shop-settings"],
  queryFn: async (): Promise<ShopSettingsForm> => {
    const { data, error } = await supabase
      .from("shop_settings")
      .select("*")
      .eq("id", true)
      .maybeSingle();
    if (error) throw error;
    return data ? fromDatabase(data) : defaultSettings;
  },
});

useEffect(() => {
  if (savedSettings.data) setSettings(savedSettings.data);
}, [savedSettings.data]);

useEffect(() => {
  const textarea = bookingTermsRef.current;
  if (!textarea) return;

  textarea.style.height = "auto";
  textarea.style.height = `${textarea.scrollHeight}px`;
}, [settings.bookingTerms]);

useEffect(() => {
  void supabase.auth.getUser().then(({ data }) => {
    const currentUser = data.user;
    setUser(currentUser ?? null);
    if (!currentUser) return;
    setAccount({
      name:
        (typeof currentUser.user_metadata["full_name"] === "string" &&
          currentUser.user_metadata["full_name"]) ||
        currentUser.email?.split("@")[0] ||
        "Administrator",
      email: currentUser.email ?? "",
      currentPassword: "",
      newPassword: "",
      confirmNewPassword: "",
```

```
        avatarUrl:
          typeof currentUser.user_metadata["avatar_url"] === "string"
            ? currentUser.user_metadata["avatar_url"]
            : null,
      });
    });
  }, []);

  useEffect(
    () => () => {
      if (avatarPreview?.startsWith("blob:")) URL.revokeObjectURL(avatarPreview);
    },
    [avatarPreview],
  );

  const saveSettings = useMutation({
    mutationFn: async (section: Exclude<SettingsTab, "account">) => {
      const previous = savedSettings.data ?? defaultSettings;
      const next = settings;
      const updates = {
        address: next.address.trim(),
        contact_number: next.contactNumber.trim(),
        contact_email: next.contactEmail.trim().toLowerCase(),
        booking_terms: next.bookingTerms.trim(),
        notify_reminder: next.sendReminders,
        reminder_hours_before: next.reminderHoursBefore,
      };
      const { error } = await supabase
        .from("shop_settings")
        .upsert({ id: true, ...updates }, { onConflict: "id" });
      if (error) throw error;
      const persisted = {
        ...previous,
        ...Object.fromEntries(settingKeysByTab[section].map((key) => [key,
next[key]])),
      } as ShopSettingsForm;
      return {
        persisted,
        section,
        changedFields: changedSettingFields(previous, persisted, section),
      };
    },
    onSuccess: async ({ persisted, section, changedFields }) => {
      setSettings((current) => ({
        ...current,
```

```
        ...Object.fromEntries(settingKeysByTab[section].map((key) => [key,
persisted[key]])),
      }));
      queryClient.setQueryData(["shop-settings"], persisted);
      queryClient.invalidateQueries({ queryKey: ["public-shop-settings"] });
      if (changedFields.length) {
        try {
          await recordAdminActivityEvent({
            action: "updated",
            resourceType: "Settings",
            targetLabel: tabLabels[section],
            summary: `${tabLabels[section]} updated.`,
            changedFields,
          });
          queryClient.invalidateQueries({ queryKey: ["admin-activity-logs"] });
        } catch (error) {
          console.error("Settings activity log failed", error);
        }
      }
    },
    onError: (error: Error) => {
      const message = error.message || "Could not save settings.";
      if (message.includes("contact email")) setSettingsErrors({ contactEmail:
message });
      else setSettingsErrors({ contactEmail: message });
    },
  });

  const saveAccount = useMutation({
    mutationFn: async () => {
      if (!user) throw new Error("Your account could not be loaded. Please refresh
the page.");
      if (!account.name.trim()) throw new Error("Enter the administrator name.");
      if (!/^\S+@\S+\.\S+$/.test(account.email.trim())) {
        throw new Error("Enter a valid email address.");
      }
      if (account.newPassword || account.confirmNewPassword ||
account.currentPassword) {
        if (!account.currentPassword) throw new Error("Enter your current password
first.");
        if (account.newPassword.length < 8)
          throw new Error("Your new password needs at least 8 characters.");
        if (account.newPassword !== account.confirmNewPassword) {
          throw new Error("The new passwords do not match.");
        }
```

```
        // Password verification creates a fresh Supabase Auth session. Create a
        // short-lived handover from the active, server-verified session first so
        // this same browser can rotate its session without weakening the
        // one-device restriction.
        const { data: handoverToken, error: handoverError } = await supabase.rpc(
          "create_admin_session_handover",
        );
        if (handoverError || !handoverToken) {
          throw new Error("Your administrator session has expired. Please sign in
again.");
        }

        const { error } = await supabase.auth.signInWithPassword({
          email: user.email ?? "",
          password: account.currentPassword,
        });
        if (error) throw new Error("Your current password is incorrect.");

        const { error: claimError } = await supabase.rpc("claim_admin_session", {
          p_handover_token: handoverToken,
        });
        if (claimError) {
          await supabase.auth.signOut({ scope: "local" });
          throw new Error("Your administrator session could not be renewed. Please
sign in again.");
        }
      }

      const oldName =
        (typeof user.user_metadata["full_name"] === "string" &&
user.user_metadata["full_name"]) ||
        user.email?.split("@")[0] ||
        "Administrator";
      const changedFields = [
        ...(oldName !== account.name.trim() ? ["Administrator name"] : []),
        ...(user.email !== account.email.trim().toLowerCase() ? ["Email address"] :
[]),
        ...(avatarFile ? ["Profile picture"] : []),
        ...(account.newPassword ? ["Password"] : []),
      ];
      const avatarUrl = avatarFile
        ? await uploadAsset(avatarFile, `admin-profiles/${user.id}`)
        : account.avatarUrl;
      const { data, error } = await supabase.auth.updateUser({
        email: account.email.trim().toLowerCase(),
```

```
        ...(account.newPassword ? { password: account.newPassword } : {}),
        data: { full_name: account.name.trim(), avatar_url: avatarUrl },
      });
      if (error) throw error;
      return { user: data.user, avatarUrl, changedFields };
    },
    onSuccess: async ({ user: updatedUser, avatarUrl, changedFields }) => {
      setUser(updatedUser);
      setAvatarFile(null);
      setAvatarPreview(null);
      setAccount((current) => ({
        ...current,
        email: updatedUser.email ?? current.email,
        avatarUrl,
        currentPassword: "",
        newPassword: "",
        confirmNewPassword: "",
      }));
      if (changedFields.length) {
        try {
          await recordAdminActivityEvent({
            action: "updated",
            resourceType: "Account Settings",
            targetLabel: "Administrator profile",
            summary: "Administrator account settings updated.",
            changedFields,
          });
          queryClient.invalidateQueries({ queryKey: ["admin-activity-logs"] });
        } catch (error) {
          console.error("Account settings activity log failed", error);
        }
      }
    },
    onError: (error: Error) => {
      const message = error.message || "Could not save account changes.";
      if (message.includes("JPG, PNG") || message.includes("Images must be")) {
        setAccountErrors({ profilePicture: message });
      } else if (message.includes("administrator name")) setAccountErrors({ name:
message });
      else if (message.includes("email")) setAccountErrors({ email: message });
      else if (message.includes("current password")) setAccountErrors({
currentPassword: message });
      else if (message.includes("new password needs")) setAccountErrors({
newPassword: message });
      else if (message.includes("passwords do not match")) {
```

```
        setAccountErrors({ confirmNewPassword: message });
      } else setAccountErrors({ currentPassword: message });
    },
  });

  function setSetting<K extends keyof ShopSettingsForm>(key: K, value:
ShopSettingsForm[K]) {
    setSettings((current) => ({ ...current, [key]: value }));
  }

  function validateSettings() {
    const nextErrors: Partial<Record<SettingsErrorField, string | undefined>> = {};

    if (!/^\S+@\S+\.\S+$/.test(settings.contactEmail.trim())) {
      nextErrors.contactEmail = "Enter a valid shop contact email.";
    }

    setSettingsErrors(nextErrors);
    return Object.keys(nextErrors).length === 0;
  }

  function requestSave(section: SettingsTab) {
    if (section === "account") {
      if (validateAccount()) setPendingSave(section);
      return;
    }
    if (validateSettings()) setPendingSave(section);
  }

  function confirmSave() {
    if (!pendingSave) return;
    if (pendingSave === "account") saveAccount.mutate();
    else saveSettings.mutate(pendingSave);
    setPendingSave(null);
  }

  function validateAccount() {
    const nextErrors: Partial<Record<AccountErrorField, string | undefined>> = {};
    if (account.name.trim().length < 2) {
      nextErrors.name = "Enter an administrator name with at least 2 characters.";
    }
    if (!/^\S+@\S+\.\S+$/.test(account.email.trim())) {
      nextErrors.email = "Enter a valid email address.";
    }
```

```
    const changingPassword = Boolean(
      account.currentPassword || account.newPassword ||
account.confirmNewPassword,
    );
    if (changingPassword) {
      if (!account.currentPassword) nextErrors.currentPassword = "Enter your
current password.";
      if (account.newPassword.length < 8) {
        nextErrors.newPassword = "Your new password needs at least 8
characters.";
      }
      if (account.newPassword !== account.confirmNewPassword) {
        nextErrors.confirmNewPassword = "The new passwords do not match.";
      }
    }

    setAccountErrors(nextErrors);
    return Object.keys(nextErrors).length === 0;
  }

  function chooseImage(
    event: React.ChangeEvent<HTMLInputElement>,
    setFile: (file: File | null) => void,
    setPreview: (preview: string | null) => void,
  ) {
    const file = event.target.files?.[0] ?? null;
    setFile(file);
    setPreview(file ? URL.createObjectURL(file) : null);
  }

  const avatar = avatarPreview ?? account.avatarUrl;
  const pendingSaveLabel =
    pendingSave === "account"
      ? "account changes"
      : pendingSave
        ? tabLabels[pendingSave]
        : "settings";

  return (
    <div className="mx-auto w-full max-w-5xl">
      <PageHeader
        title="Settings"
        description="Manage customer-facing shop details and your administrator
account."
      />
```

```
    <Tabs value={tab} onValueChange={(value) => setTab(value as
SettingsTab)}>
      <TabsList className="grid w-full grid-cols-2 md:inline-flex md:w-auto">
        <TabsTrigger value="shop" className="w-full gap-2 md:w-auto">
          <Store className="h-4 w-4" /> Shop information
        </TabsTrigger>
        <TabsTrigger value="account" className="w-full gap-2 md:w-auto">
          <UserRound className="h-4 w-4" /> Account
        </TabsTrigger>
      </TabsList>

      {savedSettings.isError ? (
        <Card className="mt-6 border-destructive/40 bg-card/60">
          <CardContent className="flex flex-wrap items-center justify-between
gap-3 p-5 text-sm">
            <p className="text-destructive">
              Could not load the saved settings. Please try again.
            </p>
            <Button type="button" variant="outline" onClick={() => void
savedSettings.refetch()}>
              Retry
            </Button>
          </CardContent>
        </Card>
      ) : savedSettings.isLoading ? (
        <Card className="mt-6 border-border/70 bg-card/60">
          <CardContent className="flex items-center gap-2 p-6 text-sm
text-muted-foreground">
            <Loader2 className="h-4 w-4 animate-spin" /> Loading saved
settings…
          </CardContent>
        </Card>
      ) : (
        <>
          <TabsContent value="shop" className="mt-6 space-y-5">
            <Card className="border-border/70 bg-card/60">
              <CardContent className="p-5 sm:p-6">
                <div className="grid gap-5 md:grid-cols-2">
                  <Field label="Contact email">
                    <Input
                      type="email"
                      value={settings.contactEmail}
                      onChange={(e) => {
                        setSetting("contactEmail", e.target.value);
```

```
                    setSettingsErrors((current) => ({ ...current, contactEmail:
undefined }));
                  }}
                  aria-invalid={!!settingsErrors.contactEmail}
                />
                <FieldError message={settingsErrors.contactEmail} />
              </Field>
              <Field label="Contact number">
                <Input
                  value={settings.contactNumber}
                  onChange={(e) => setSetting("contactNumber", e.target.value)}
                />
              </Field>
              <div className="rounded-lg border border-border bg-secondary/30
p-4 text-xs leading-relaxed text-muted-foreground">
                <Building2 className="mb-2 h-4 w-4 text-primary" />
                This information can appear on customer-facing appointment
pages and receipts.
              </div>
              <Field label="Shop address" className="md:col-span-2">
                <Textarea
                  value={settings.address}
                  onChange={(e) => setSetting("address", e.target.value)}
                />
              </Field>
              <Field label="Booking terms & conditions"
className="md:col-span-2">
                <Textarea
                  ref={bookingTermsRef}
                  value={settings.bookingTerms}
                  onChange={(e) => setSetting("bookingTerms", e.target.value)}
                  className="min-h-36 resize-y overflow-y-hidden"
                />
              </Field>
            </div>
          </CardContent>
        </Card>
        <Card className="border-border/70 bg-card/60">
          <CardContent className="space-y-4 p-5 sm:p-6">
            <div className="flex items-start gap-3">
              <BellRing className="mt-0.5 h-4 w-4 shrink-0 text-primary"
aria-hidden="true" />
              <div>
                <p className="font-medium">Appointment reminders</p>
                <p className="mt-1 text-sm text-muted-foreground">
```

```
                Customers with a confirmed appointment get a reminder by email
and text. It
                is skipped when the booking was confirmed in the last 12 hours,
since the
                confirmation has the same details.
              </p>
            </div>
          </div>
          <label className="flex items-center gap-3 text-sm">
            <Switch
              checked={settings.sendReminders}
              onCheckedChange={(checked) => setSetting("sendReminders",
checked)}
            />
            Send appointment reminders
          </label>
          <Field label="When to send">
            <Select
              value={String(settings.reminderHoursBefore)}
              onValueChange={(value) => setSetting("reminderHoursBefore",
Number(value))}
              disabled={!settings.sendReminders}
            >
              <SelectTrigger aria-label="When to send reminders"
className="w-full sm:w-64">
                <SelectValue />
              </SelectTrigger>
              <SelectContent>

{reminderHourOptions(settings.reminderHoursBefore).map((hours) => (
                  <SelectItem key={hours} value={String(hours)}>
                    {reminderTimingLabel(hours)} the appointment
                  </SelectItem>
                ))}
              </SelectContent>
            </Select>
          </Field>
        </CardContent>
      </Card>
      <SaveButton
        loading={saveSettings.isPending}
        onClick={() => requestSave("shop")}
        label="Save shop information"
      />
    </TabsContent>
```

```
          <TabsContent value="account" className="mt-6 space-y-5">
            <Card className="border-border/70 bg-card/60">
              <CardContent className="p-5 sm:p-6">
                <div className="mb-6 flex flex-wrap items-center gap-4 border-b
border-border/70 pb-5">
                  <div className="grid h-16 w-16 overflow-hidden rounded-full
border border-border bg-secondary/50">
                    {avatar ? (
                      <img
                        src={avatar}
                        alt="Profile preview"
                        className="h-full w-full object-cover"
                      />
                    ) : (
                      <UserRound className="m-auto h-7 w-7 text-primary" />
                    )}
                  </div>
                  <div className="min-w-0 flex-1">
                    <p className="font-medium">Profile picture</p>
                    <p className="mt-1 text-xs text-muted-foreground">
                      Use a square JPG, PNG, or WebP image.
                    </p>
                  </div>
                  <UploadButton
                    label="Change photo"
                    invalid={!!accountErrors.profilePicture}
                    onChange={(event) => {
                      chooseImage(event, setAvatarFile, setAvatarPreview);
                      setAccountErrors((current) => ({ ...current, profilePicture:
undefined }));
                    }}
                  />
                  <FieldError
                    message={accountErrors.profilePicture}
                    className="w-full basis-full"
                  />
                </div>
                <div className="grid gap-5 md:grid-cols-2">
                  <Field label="Administrator name">
                    <Input
                      value={account.name}
                      onChange={(e) => {
                        setAccount((current) => ({ ...current, name: e.target.value }));
                        setAccountErrors((current) => ({ ...current, name: undefined }));
```

```
          }}
          aria-invalid={!!accountErrors.name}
        />
        <FieldError message={accountErrors.name} />
      </Field>
      <Field label="Email address">
        <Input
          type="email"
          value={account.email}
          onChange={(e) => {
            setAccount((current) => ({ ...current, email: e.target.value }));
            setAccountErrors((current) => ({ ...current, email: undefined }));
          }}
          aria-invalid={!!accountErrors.email}
        />
        <FieldError message={accountErrors.email} />
      </Field>
    </div>
  </CardContent>
</Card>
<SettingsCard
  title="Change password"
  icon={<LockKeyhole className="h-5 w-5 text-primary" />}
>
  <p className="-mt-2 text-xs text-muted-foreground">
    Leave these fields blank to keep your current password.
  </p>
  <div className="grid gap-5 md:grid-cols-3">
    <Field label="Current password">
      <div className="relative">
        <Input
          type={showCurrentPassword ? "text" : "password"}
          autoComplete="current-password"
          value={account.currentPassword}
          onChange={(e) => {
            setAccount((current) => ({
              ...current,
              currentPassword: e.target.value,
            }));
            setAccountErrors((current) => ({
              ...current,
              currentPassword: undefined,
            }));
          }}
          aria-invalid={!!accountErrors.currentPassword}
```

```
                  className="pr-11"
                />
                <Button
                  type="button"
                  variant="ghost"
                  size="icon"
                  className="absolute top-0 right-0 text-muted-foreground
hover:text-foreground"
                  onClick={() => setShowCurrentPassword((visible) => !visible)}
                  aria-label={
                    showCurrentPassword ? "Hide current password" : "Show
current password"
                  }
                  aria-pressed={showCurrentPassword}
                >
                  {showCurrentPassword ? <EyeOff /> : <Eye />}
                </Button>
              </div>
              <FieldError message={accountErrors.currentPassword} />
            </Field>
            <Field label="New password">
              <div className="relative">
                <Input
                  type={showNewPassword ? "text" : "password"}
                  autoComplete="new-password"
                  value={account.newPassword}
                  onChange={(e) => {
                    setAccount((current) => ({ ...current, newPassword:
e.target.value }));
                    setAccountErrors((current) => ({ ...current, newPassword:
undefined }));
                  }}
                  aria-invalid={!!accountErrors.newPassword}
                  className="pr-11"
                />
                <Button
                  type="button"
                  variant="ghost"
                  size="icon"
                  className="absolute top-0 right-0 text-muted-foreground
hover:text-foreground"
                  onClick={() => setShowNewPassword((visible) => !visible)}
                  aria-label={showNewPassword ? "Hide new password" : "Show
new password"}
                  aria-pressed={showNewPassword}
```

```
              >
                {showNewPassword ? <EyeOff /> : <Eye />}
              </Button>
            </div>
            <FieldError message={accountErrors.newPassword} />
          </Field>
          <Field label="Confirm new password">
            <div className="relative">
              <Input
                type={showConfirmNewPassword ? "text" : "password"}
                autoComplete="new-password"
                value={account.confirmNewPassword}
                onChange={(e) => {
                  setAccount((current) => ({
                    ...current,
                    confirmNewPassword: e.target.value,
                  }));
                  setAccountErrors((current) => ({
                    ...current,
                    confirmNewPassword: undefined,
                  }));
                }}
                aria-invalid={!!accountErrors.confirmNewPassword}
                className="pr-11"
              />
              <Button
                type="button"
                variant="ghost"
                size="icon"
                className="absolute top-0 right-0 text-muted-foreground
hover:text-foreground"
                onClick={() => setShowConfirmNewPassword((visible) =>
!visible)}
                aria-label={
                  showConfirmNewPassword
                    ? "Hide confirmed password"
                    : "Show confirmed password"
                }
                aria-pressed={showConfirmNewPassword}
              >
                {showConfirmNewPassword ? <EyeOff /> : <Eye />}
              </Button>
            </div>
            <FieldError message={accountErrors.confirmNewPassword} />
          </Field>
```

```
            </div>
          </SettingsCard>
          <SaveButton
            loading={saveAccount.isPending}
            onClick={() => requestSave("account")}
            label="Save account changes"
          />
        </TabsContent>
      </>
    )}
  </Tabs>
  <AlertDialog
    open={pendingSave !== null}
    onOpenChange={(open) => !open && setPendingSave(null)}
  >
    <AlertDialogContent>
      <AlertDialogHeader>
        <AlertDialogTitle>Save {pendingSaveLabel}?</AlertDialogTitle>
        <AlertDialogDescription>
          Confirm that you want to apply these changes. The updated settings will
take effect
          immediately.
        </AlertDialogDescription>
      </AlertDialogHeader>
      <AlertDialogFooter>
        <AlertDialogCancel>Cancel</AlertDialogCancel>
        <AlertDialogAction onClick={confirmSave}>Save
changes</AlertDialogAction>
      </AlertDialogFooter>
    </AlertDialogContent>
  </AlertDialog>
 </div>
 );
}

function Field({
 label,
 className,
 children,
}: {
 label: string;
 className?: string;
 children: React.ReactNode;
}) {
 return (
```

```
    <div className={cn("space-y-2", className)}>
      <Label>{label}</Label>
      {children}
    </div>
  );
}

function SettingsCard({
  title,
  icon,
  children,
}: {
  title: string;
  icon: React.ReactNode;
  children: React.ReactNode;
}) {
  return (
    <Card className="border-border/70 bg-card/60">
      <CardContent className="p-5 sm:p-6">
        <div className="mb-5 flex items-center gap-3 border-b border-border/70
pb-4">
          {icon}
          <h2 className="font-display text-lg tracking-wide uppercase">{title}</h2>
        </div>
        <div className="space-y-5">{children}</div>
      </CardContent>
    </Card>
  );
}

function UploadButton({
  label,
  invalid,
  onChange,
}: {
  label: string;
  invalid?: boolean;
  onChange: (event: React.ChangeEvent<HTMLInputElement>) => void;
}) {
  return (
    <Label
      className={cn(
        "inline-flex h-9 cursor-pointer items-center gap-2 rounded-md border
border-input bg-background px-3 text-sm font-medium transition-colors
hover:bg-accent",
```

```
        invalid && "border-destructive ring-1 ring-destructive/30",
      )}
    >
      <ImageUp className="h-4 w-4" /> {label}
      <Input
        type="file"
        accept="image/jpeg,image/png,image/webp"
        className="sr-only"
        aria-invalid={invalid}
        onChange={onChange}
      />
    </Label>
  );
}

function SaveButton({
  loading,
  onClick,
  label,
}: {
  loading: boolean;
  onClick: () => void;
  label: string;
}) {
  return (
    <div className="flex justify-stretch sm:justify-end">
      <Button
        type="button"
        disabled={loading}
        onClick={onClick}
        className="w-full sm:min-w-52 sm:w-auto"
      >
        {loading ? <Loader2 className="animate-spin" /> : <Save />} {label}
      </Button>
    </div>
  );
}
```

src/components/site/appointment-verification-dialog.tsx

```
import { useMutation } from "@tanstack/react-query";
import { useServerFn } from "@tanstack/react-start";
import { CircleAlert, Info, Loader2, Mail, ShieldCheck } from "lucide-react";
import { useEffect, useState } from "react";

import { Alert, AlertDescription } from "@/components/ui/alert";
import { Button } from "@/components/ui/button";
import {
  Dialog,
  DialogClose,
  DialogContent,
  DialogDescription,
  DialogFooter,
  DialogHeader,
  DialogTitle,
} from "@/components/ui/dialog";
import { FieldError } from "@/components/ui/field-error";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { requestAppointmentActionCode, verifyAppointmentActionCode } from
"@/lib/booking.functions";

export type AppointmentAction = "cancel" | "reschedule";

export type AppointmentVerification = {
  token: string;
  expiresAt: string;
  /** The booking's full mobile number, returned for reschedules only. */
  phone?: string;
};

/**
 * Confirms the customer can read the booking's email before an online
 * cancellation or reschedule: sends a 6-digit code, then exchanges it for a
 * short-lived verification token that the action request must include.
 */
export function AppointmentVerificationDialog({
  open,
  action,
  reference,
  emailHint,
  schedule,
  notice,
```

```
  onOpenChange,
  onVerified,
}: {
  open: boolean;
  action: AppointmentAction;
  reference: string;
  emailHint: string;
  /** For example "Saturday, October 24, 2026 at 9:00 AM". */
  schedule: string;
  /** Why verification is being asked for again, such as an expired verification. */
  notice?: string | null;
  onOpenChange: (open: boolean) => void;
  onVerified: (verification: AppointmentVerification) => void;
}) {
  const requestCodeFn = useServerFn(requestAppointmentActionCode);
  const verifyCodeFn = useServerFn(verifyAppointmentActionCode);
  const [step, setStep] = useState<"send" | "code">("send");
  const [code, setCode] = useState("");
  const [codeError, setCodeError] = useState<string | null>(null);
  const [error, setError] = useState<string | null>(null);
  const [sentTo, setSentTo] = useState(emailHint);
  const [resent, setResent] = useState(false);

  useEffect(() => {
    if (!open) return;
    setStep("send");
    setCode("");
    setCodeError(null);
    setError(null);
    setSentTo(emailHint);
    setResent(false);
  }, [open, action, emailHint]);

  const requestCode = useMutation({
    mutationFn: () => requestCodeFn({ data: { reference, action } }),
    onSuccess: (result) => {
      if (!result.ok) {
        setError(result.error);
        return;
      }
      setError(null);
      setCodeError(null);
      setCode("");
      setSentTo(result.emailHint);
      setResent(step === "code");
```

```
      setStep("code");
    },
    onError: () => setError("We could not send the verification email. Please try
again shortly."),
  });

  const verifyCode = useMutation({
    mutationFn: () => verifyCodeFn({ data: { reference, action, code: code.trim() }
}),
    onSuccess: (result) => {
      if (!result.ok) {
        setError(result.error);
        return;
      }
      onVerified({
        token: result.token,
        expiresAt: result.expiresAt,
        ...("phone" in result && result.phone ? { phone: result.phone } : {}),
      });
    },
    onError: () => setError("We could not check the code right now. Please try
again shortly."),
  });

  const isCancel = action === "cancel";
  const busy = requestCode.isPending || verifyCode.isPending;
  const dismissLabel = isCancel ? "Keep appointment" : "Not now";

  return (
    <Dialog open={open} onOpenChange={(next) => !busy &&
onOpenChange(next)}>
      {/* minmax(0,1fr) keeps long, non-wrapping text from widening the dialog. */}
      <DialogContent className="grid-cols-[minmax(0,1fr)] sm:max-w-md">
        <DialogHeader>
          <DialogTitle>
            {step === "code"
              ? "Enter your verification code"
              : isCancel
                ? "Cancel this appointment?"
                : "Verify it's you"}
          </DialogTitle>
          <DialogDescription>
            {step === "code" ? (
              <>
                We sent a 6-digit code to <strong
```

```
className="text-foreground">{sentTo}</strong>. It
              expires in 10 minutes.
            </>
          ) : isCancel ? (
            <>
              This cannot be undone. Your {schedule} slot will be released, and you
will need to
              book a new schedule, subject to availability.
            </>
          ) : (
            <>
              Before you choose a new schedule for {reference}, confirm that this
booking is
              yours.
            </>
          )}
        </DialogDescription>
      </DialogHeader>

      <form
        noValidate
        className="space-y-4"
        onSubmit={(event) => {
          event.preventDefault();
          setError(null);
          if (step === "send") {
            requestCode.mutate();
            return;
          }
          if (!/^\d{6}$/.test(code.trim())) {
            setCodeError("Enter the 6-digit code from the email.");
            return;
          }
          verifyCode.mutate();
        }}
      >
        {notice && step === "send" && (
          <Alert className="border-primary/30 bg-primary/5">
            <Info className="size-4" aria-hidden="true" />
            <AlertDescription>{notice}</AlertDescription>
          </Alert>
        )}

        {step === "send" ? (
          <div className="flex items-start gap-3 rounded-lg border
```

```
border-border/70 bg-muted/30 p-3 text-sm">
            <ShieldCheck className="mt-0.5 size-5 shrink-0 text-primary"
aria-hidden="true" />
            <p>
              To protect your booking, we&apos;ll email a 6-digit code to{" "}
              <strong className="break-words">{emailHint}</strong>.
            </p>
          </div>
        ) : (
          <div className="space-y-1.5">
            <Label htmlFor="appointment-verification-code">Verification
code</Label>
            <Input
              id="appointment-verification-code"
              type="text"
              value={code}
              maxLength={6}
              inputMode="numeric"
              autoComplete="one-time-code"
              autoFocus
              onChange={(event) => {
                setCode(event.target.value.replace(/\D/g, ""));
                setCodeError(null);
                setError(null);
              }}
              placeholder="123456"
              aria-invalid={Boolean(codeError)}
              className="h-11 font-display text-xl tracking-[0.4em] md:h-11
md:text-xl"
            />
            <FieldError message={codeError ?? undefined} />
            <div className="flex flex-wrap items-center gap-x-1 gap-y-1 text-xs
text-muted-foreground">
              <span>Didn&apos;t get it? Check your spam folder or</span>
              <Button
                type="button"
                variant="link"
                className="h-auto px-0 text-xs md:h-auto"
                disabled={busy}
                onClick={() => {
                  setError(null);
                  requestCode.mutate();
                }}
              >
                {requestCode.isPending && <Loader2 className="animate-spin" />}
```

```
              send a new code
            </Button>
          </div>
          {resent && !requestCode.isPending && (
            <p role="status" className="flex items-center gap-1.5 text-xs
text-foreground">
              <Mail className="size-3.5 text-primary" aria-hidden="true" /> A new
code is on its
              way. Only the newest code works.
            </p>
          )}
        </div>
      )}

      {error && (
        <Alert variant="destructive" className="border-destructive/40
bg-destructive/5">
          <CircleAlert className="size-4" aria-hidden="true" />
          <AlertDescription>{error}</AlertDescription>
        </Alert>
      )}

      <DialogFooter className="sm:flex-wrap sm:space-x-0">
        <DialogClose asChild>
          <Button type="button" variant="outline" disabled={busy}>
            {dismissLabel}
          </Button>
        </DialogClose>
        <Button
          type="submit"
          variant={step === "code" && isCancel ? "destructive" : "default"}
          disabled={busy}
        >
          {busy ? (
            <Loader2 className="animate-spin" aria-hidden="true" />
          ) : step === "send" ? (
            <Mail aria-hidden="true" />
          ) : (
            <ShieldCheck aria-hidden="true" />
          )}
          {step === "send"
            ? requestCode.isPending
              ? "Sending code…"
              : "Email me a code"
            : verifyCode.isPending
```

```
                ? "Checking code…"
                : isCancel
                  ? "Verify and cancel"
                  : "Verify and continue"}
            </Button>
          </DialogFooter>
        </form>
      </DialogContent>
    </Dialog>
  );
}
```

src/components/site/booking-schedule.tsx

```
import { format, parseISO } from "date-fns";

import { Calendar } from "@/components/ui/calendar";
import type { ComputedSlot } from "@/lib/availability";
import { formatTime, timeToMinutes } from "@/lib/shop";
import { cn } from "@/lib/utils";

/**
 * Calendar shared by the booking and reschedule flows. Only dates in
 * `availableDates` can be chosen; fully booked dates are struck through.
 */
export function BookingCalendar({
  selected,
  availableDates,
  fullyBookedDates,
  onSelect,
  className,
}: {
  selected: string;
  availableDates: Set<string>;
  fullyBookedDates: string[];
  onSelect: (date: string) => void;
  className?: string;
}) {
  const sortedDates = Array.from(availableDates).sort();
  const firstDate = sortedDates[0];
  const lastDate = sortedDates.at(-1);
  const initialMonth = selected || firstDate;

  return (
    <Calendar
      mode="single"
      selected={selected ? parseISO(selected) : undefined}
      onSelect={(day) => {
        if (!day) return;
        const iso = format(day, "yyyy-MM-dd");
        if (!availableDates.has(iso)) return;
        onSelect(iso);
      }}
      // Open on the month that has the first bookable date and keep navigation
      // within the months that actually contain available dates.
      {...(initialMonth ? { defaultMonth: parseISO(initialMonth) } : {})}
      {...(firstDate ? { startMonth: parseISO(firstDate) } : {})}
```

```
      {...(lastDate ? { endMonth: parseISO(lastDate) } : {})}
      disabled={(day) => !availableDates.has(format(day, "yyyy-MM-dd"))}
      modifiers={{ fullyBooked: fullyBookedDates.map((value) => parseISO(value))
}}
      modifiersClassNames={{
        fullyBooked: "bg-destructive/15 text-destructive line-through opacity-100",
      }}
      className={cn("p-0 min-[360px]:p-3", className)}
      classNames={{
        nav: "justify-between gap-1",
        month_caption: "flex h-(--cell-size) w-full items-center justify-center
px-(--cell-size)",
      }}
    />
  );
}

export function FullyBookedLegend({ className }: { className?: string }) {
  return (
    <p className={cn("flex items-center gap-2 text-xs text-muted-foreground",
className)}>
      <span
        aria-hidden="true"
        className="h-3 w-3 rounded-sm bg-destructive/15 ring-1
ring-destructive/40"
      />
      Fully booked dates cannot be selected.
    </p>
  );
}

const SLOT_GROUPS = [
  { label: "Morning", from: 0, to: 12 * 60 },
  { label: "Afternoon", from: 12 * 60, to: 17 * 60 },
  { label: "Evening", from: 17 * 60, to: 24 * 60 },
] as const;

/**
 * Start times for one date, grouped by part of the day. Times that are taken
 * or blocked are shown greyed out; times the shop never offers for this
 * service (lunch, too late to finish) are left out.
 */
export function TimeSlotPicker({
  slots,
  value,
```

```
  onChange,
  invalid,
  className,
  gridClassName,
}: {
  slots: ComputedSlot[];
  value: string;
  onChange: (startTime: string) => void;
  invalid?: boolean;
  className?: string;
  gridClassName?: string;
}) {
  const shown = slots.filter((slot) => slot.offered);
  const hasUnavailable = shown.some((slot) => slot.disabled);

  if (!shown.some((slot) => !slot.disabled)) {
    return (
      <p className="rounded-lg border border-dashed border-border p-4
text-center text-sm text-muted-foreground">
        No times remain available for this date. Please choose another date.
      </p>
    );
  }

  const groups = SLOT_GROUPS.map((group) => ({
    label: group.label,
    slots: shown.filter((slot) => {
      const minutes = timeToMinutes(slot.startTime);
      return minutes >= group.from && minutes < group.to;
    }),
  })).filter((group) => group.slots.length > 0);

  return (
    <div
      className={cn("space-y-4", invalid && "rounded-lg p-2 ring-1
ring-destructive", className)}
    >
      {groups.map((group) => (
        <div key={group.label}>
          <p className="mb-2 text-xs font-medium tracking-widest
text-muted-foreground uppercase">
            {group.label}
          </p>
          <div className={cn("grid grid-cols-2 gap-2 sm:grid-cols-3",
gridClassName)}>
```

```
          {group.slots.map((slot) => {
            const selected = value === slot.startTime && !slot.disabled;
            return (
              <button
                type="button"
                key={slot.id}
                disabled={slot.disabled}
                aria-pressed={selected}
                aria-label={
                  slot.disabled ? `${formatTime(slot.startTime)}, unavailable` :
undefined
                }
                onClick={() => onChange(slot.startTime)}
                className={cn(
                  "h-11 rounded-lg border px-3 font-display text-base tracking-wide
transition-colors focus-visible:ring-2 focus-visible:ring-ring
focus-visible:outline-none",
                  slot.disabled
                    ? "cursor-not-allowed border-dashed border-border bg-muted/40
text-muted-foreground/70 line-through"
                    : selected
                      ? "border-primary bg-primary text-primary-foreground shadow-sm"
                      : "border-border bg-background/60 hover:border-primary/60
hover:bg-primary/5",
                )}
              >
                {formatTime(slot.startTime)}
              </button>
            );
          })}
        </div>
      </div>
    ))}
    {hasUnavailable && (
      <p className="flex items-center gap-2 text-xs text-muted-foreground">
        <span
          aria-hidden="true"
          className="h-3 w-3 rounded-sm border border-dashed border-border
bg-muted/40"
        />
        Crossed-out times are already booked or unavailable.
      </p>
    )}
  </div>
);
```

```
}
```

src/components/site/copy-reference-button.tsx

```
import { Check, Copy } from "lucide-react";
import { useEffect, useState } from "react";

import { Button } from "@/components/ui/button";
import { cn } from "@/lib/utils";

/** Copies a booking reference code and briefly confirms the copy. */
export function CopyReferenceButton({
  value,
  className,
  size = "sm",
}: {
  value: string;
  className?: string;
  size?: "sm" | "default";
}) {
  const [status, setStatus] = useState<"idle" | "copied" | "error">("idle");

  useEffect(() => {
    if (status !== "copied") return;
    const timer = window.setTimeout(() => setStatus("idle"), 2500);
    return () => window.clearTimeout(timer);
  }, [status]);

  return (
    <span className={cn("inline-flex flex-col items-center gap-1.5", className)}>
      <Button
        type="button"
        variant="outline"
        size={size}
        onClick={async () => {
          try {
            await navigator.clipboard.writeText(value);
            setStatus("copied");
          } catch {
            setStatus("error");
          }
        }}
      >
        {status === "copied" ? <Check aria-hidden="true" /> : <Copy
aria-hidden="true" />}
        {status === "copied" ? "Copied" : "Copy code"}
      </Button>
```

```
      <span className="sr-only" aria-live="polite">
        {status === "copied" ? "Reference code copied." : ""}
      </span>
      {status === "error" && (
        <span className="text-xs text-destructive" role="alert">
          Could not copy the code automatically. Please copy it manually.
        </span>
      )}
    </span>
  );
}
```

src/components/site/offline-connection.tsx

```
import { useQuery } from "@tanstack/react-query";
import { useRouter } from "@tanstack/react-router";
import { Loader2, Wifi, WifiOff } from "lucide-react";
import { useEffect, useRef, useState } from "react";

type OfflineStatus = {
  enabled: true;
  online: boolean;
  /** Offline bookings not yet sent to the live site. */
  waiting: number;
  liveSiteUrl: string | null;
};

/**
 * Offline copy only (src/lib/offline-sync.server.ts): shows whether the shop is
 * offline. Once the internet is back and every offline booking has been sent,
 * people move to the live site on their next page change, never in the middle
 * of a form. Shows nothing while sending to the live site is switched off.
 */
export function OfflineConnection() {
  const router = useRouter();
  const status = useQuery({
    queryKey: ["offline-status"],
    queryFn: async (): Promise<OfflineStatus | null> => {
      const response = await fetch("/api/offline-status", { cache: "no-store" });
      return response.ok ? ((await response.json()) as OfflineStatus) : null;
    },
    refetchInterval: 15_000,
    retry: false,
  });
  const data = status.data;
  const liveSiteUrl = data?.online && data.waiting === 0 ? data.liveSiteUrl : null;

  // The laptop can have internet while a phone on its hotspot does not, so each
  // browser checks for itself that it can open the live site.
  const [reachableUrl, setReachableUrl] = useState<string | null>(null);
  useEffect(() => {
    if (!liveSiteUrl || reachableUrl === liveSiteUrl) return;
    const controller = new AbortController();
    const timer = setTimeout(() => controller.abort(), 5_000);
    fetch(liveSiteUrl, { mode: "no-cors", cache: "no-store", signal: controller.signal
})
      .then(() => setReachableUrl(liveSiteUrl))
```

```
      .catch(() => setReachableUrl(null))
      .finally(() => clearTimeout(timer));
    return () => {
      controller.abort();
      clearTimeout(timer);
    };
  }, [liveSiteUrl, reachableUrl, status.dataUpdatedAt]);

  const moveTo = liveSiteUrl && reachableUrl === liveSiteUrl ? liveSiteUrl : null;
  const moveToRef = useRef(moveTo);
  useEffect(() => {
    moveToRef.current = moveTo;
  }, [moveTo]);
  useEffect(
    () =>
      router.subscribe("onBeforeNavigate", ({ toLocation }) => {
        if (moveToRef.current)
window.location.assign(`${moveToRef.current}${toLocation.href}`);
      }),
    [router],
  );

  if (!data?.enabled) return null;
  const plural = data.waiting === 1 ? "" : "s";
  const message = !data.online
    ? data.waiting
      ? `Offline mode: ${data.waiting} booking${plural} will be sent to the live site
when the internet is back.`
      : "Offline mode: bookings are saved here and sent to the live site when the
internet is back."
    : data.waiting
      ? `Internet is back. Sending ${data.waiting} offline booking${plural} to the live
site…`
      : moveTo
        ? "Internet is back. The live site takes over from your next page."
        : "Internet is back. Offline bookings have been sent to the live site.";

  return (
    <div
      role="status"
      className="fixed right-3 bottom-3 z-50 flex max-w-[calc(100vw-1.5rem)]
items-center gap-2 rounded-2xl border border-border/70 bg-card/95 px-3 py-2
text-xs text-foreground shadow-md backdrop-blur-sm sm:max-w-sm"
    >
      {!data.online ? (
```

```
        <WifiOff className="size-4 shrink-0 text-muted-foreground"
aria-hidden="true" />
      ) : data.waiting ? (
        <Loader2 className="size-4 shrink-0 animate-spin text-primary"
aria-hidden="true" />
      ) : (
        <Wifi className="size-4 shrink-0 text-primary" aria-hidden="true" />
      )}
      <span>{message}</span>
      {moveTo && (
        <a
          href={`${moveTo}${router.state.location.href}`}
          className="shrink-0 font-medium text-primary underline
underline-offset-2"
        >
          Go now
        </a>
      )}
    </div>
  );
}
```

src/components/site/product-card.tsx

```
import { ImageIcon } from "lucide-react";

import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Card, CardContent } from "@/components/ui/card";
import { formatPHP, SHOP } from "@/lib/shop";
import { cn } from "@/lib/utils";

export type ProductRow = {
  id: string;
  name: string;
  category: string;
  brand: string | null;
  description: string | null;
  price: number | string;
  image_url: string | null;
  in_stock: boolean;
  is_featured: boolean;
};

/** Set to "1" only in the offline copy (.env.offline). */
const OFFLINE = import.meta.env["VITE_OFFLINE"] === "1";

/**
 * The offline copy keeps the live database's image addresses and serves the
 * copied files through the app's /supabase proxy instead.
 */
function imageSrc(url: string) {
  const path = url.indexOf("/storage/v1/object/public/");
  return OFFLINE && path !== -1 ? `/supabase${url.slice(path)}` : url;
}

/**
 * Opens the shop's Messenger chat with a message naming the item already
typed
 * in; the customer still presses Send.
 */
function messengerReserveLink(product: ProductRow) {
  const item = [product.brand, product.name].filter(Boolean).join(" ");
  const message = `Hi! I would like to reserve ${item}
(${formatPHP(product.price)}). Is it still available?`;
  return `${SHOP.messenger}?text=${encodeURIComponent(message)}`;
}
```

```
/** compact: a smaller card for two-per-row grids on phones; unchanged from sm
up. */
export function ProductCard({
  product,
  compact = false,
}: {
  product: ProductRow;
  compact?: boolean;
}) {
  return (
    <Card className="group flex h-full flex-col overflow-hidden border-border/70
bg-card/70 py-0 transition-colors hover:border-primary/60 lg:min-w-0">
      {/* The whole product is shown on white, the way most product photos are
shot. */}
      <div
        className={cn(
          "relative flex items-center justify-center overflow-hidden bg-white",
          compact ? "h-32 sm:h-44" : "h-44",
        )}
      >
        {product.image_url ? (
          <img
            src={imageSrc(product.image_url)}
            alt={product.name}
            loading="lazy"
            className="h-full w-full object-contain p-3 transition-transform
duration-300 group-hover:scale-105"
          />
        ) : (
          <ImageIcon className="h-12 w-12 text-muted-foreground/50" />
        )}
        {product.is_featured && (
          <Badge className="absolute top-2 left-2 bg-primary
text-primary-foreground uppercase">
            Featured
          </Badge>
        )}
        {!product.in_stock && (
          <Badge variant="outline" className="absolute top-2 right-2
bg-background/90 uppercase">
            Out of stock
          </Badge>
        )}
      </div>
```

```
    <CardContent
      className={cn("flex flex-1 flex-col", compact ? "p-3 sm:p-4 sm:pb-5" : "p-4
pb-5")}
    >
      <p className="truncate text-xs tracking-widest text-orange-700 uppercase
dark:text-accent">
        {product.brand ?? product.category}
      </p>
      <div
        className={cn(
          "mt-1 flex items-start justify-between gap-3",
          compact && "max-sm:flex-col max-sm:gap-1",
        )}
      >
        <h3
          className={cn(
            "min-w-0 flex-1 font-display leading-tight tracking-wide uppercase
lg:break-words",
            compact ? "text-base sm:text-lg" : "text-lg",
          )}
        >
          {product.name}
        </h3>
        <p className="shrink-0 font-display text-xl leading-tight font-bold
tracking-wide text-primary">
          {formatPHP(product.price)}
        </p>
      </div>
      {product.description && (
        <p
          className={cn(
            "mt-2 line-clamp-2 text-sm text-muted-foreground lg:break-words",
            compact && "max-sm:hidden",
          )}
        >
          {product.description}
        </p>
      )}
      {product.in_stock ? (
        <div className={cn("mt-auto", compact ? "pt-3 sm:pt-4" : "pt-4")}>
          <Button asChild variant="outline" size="sm" className="w-full
uppercase">
            <a
              href={messengerReserveLink(product)}
              target="_blank"
```

```
              rel="noopener noreferrer"
              aria-label={`Reserve ${product.name} via Messenger`}
            >
              {compact ? (
                <>
                  <span className="sm:hidden">Reserve</span>
                  <span className="max-sm:hidden">Reserve via
Messenger</span>
                </>
              ) : (
                "Reserve via Messenger"
              )}
            </a>
          </Button>
        </div>
      ) : (
        <div className="mt-auto pt-4">
          <p className="text-xs text-muted-foreground">
            Message us to check future availability.
          </p>
        </div>
      )}
    </CardContent>
  </Card>
 );
}
```

src/components/site/service-booking-row.tsx

```
import { Link } from "@tanstack/react-router";
import { ArrowRight } from "lucide-react";

import { Button } from "@/components/ui/button";

/**
 * The bottom of a service card on the homepage and the Services page: a Book
 * button that opens booking with this service already selected. Prices are not
 * shown here because they depend on the motorcycle; the booking form shows
the
 * right price once the motorcycle is chosen.
 */
export function ServiceBookingRow({ service }: { service: { id: string; name: string
} }) {
  return (
    <div className="mt-auto flex justify-end pt-4">
      <Button asChild size="sm" variant="outline" className="font-display
tracking-wide uppercase">
        <Link to="/book" search={{ serviceId: service.id }} aria-label={`Book
${service.name}`}>
          Book <ArrowRight />
        </Link>
      </Button>
    </div>
  );
}
```

src/components/site/site-footer.tsx

```
import { Link } from "@tanstack/react-router";
import { Clock, Facebook, Mail, MapPin, MessageCircle, Phone } from
"lucide-react";
import type { ReactNode } from "react";

import { SHOP } from "@/lib/shop";
import { cn } from "@/lib/utils";

/** Set to "1" only in the offline copy (.env.offline). */
const OFFLINE = import.meta.env["VITE_OFFLINE"] === "1";

const QUICK_LINKS = [
  { to: "/book", label: "Book an appointment" },
  { to: "/my-appointment", label: "View or cancel a booking" },
  { to: "/services", label: "Our services" },
  { to: "/shop", label: "Parts & Accessories" },
] as const;

/** "Monday to Saturday, 8:00 AM – 5:00 PM", with the times kept on one line. */
export function ShopHours() {
  return (
    <>
      {SHOP.openDays}, <span
className="whitespace-nowrap">{SHOP.openTimes}</span>
    </>
  );
}

export function SiteFooter({ className }: { className?: string }) {
  return (
    <footer className={cn("border-t border-border/70 bg-card/40", className)}>
      <div className="site-container grid gap-7 py-8 md:grid-cols-3 md:gap-10
md:py-12">
        {/* On phones the contact details come first, then a small map beside the
description. */}
        <div className="order-last flex items-center gap-4 md:order-none
md:block">
          <a
            href={SHOP.mapsUrl}
            target="_blank"
            rel="noopener noreferrer"
            aria-label="Open Fake Rider Motorparts location in Google Maps"
            className="group relative block w-40 shrink-0 overflow-hidden
```

```
rounded-xl border border-border/70 bg-muted/30 transition-colors
hover:border-primary/60 md:w-auto"
        >
          {OFFLINE ? (
            // The offline copy cannot load Google's map, so it shows the address
instead.
            <div className="flex h-28 w-full flex-col items-center justify-center
gap-1 px-3 pb-7 text-center md:h-40">
              <MapPin className="h-6 w-6 text-primary" aria-hidden="true" />
              <span className="text-xs text-muted-foreground">Baclayon,
Bohol</span>
            </div>
          ) : (
            <iframe
              title="Map showing the Fake Rider Motorparts location"
              src={SHOP.mapEmbedUrl}
              loading="lazy"
              tabIndex={-1}
              referrerPolicy="strict-origin-when-cross-origin"
              className="pointer-events-none block h-28 w-full border-0 md:h-40"
            />
          )}
          <span className="absolute inset-x-0 bottom-0 flex items-center gap-1.5
bg-background/85 px-2.5 py-1.5 text-xs font-medium text-foreground
backdrop-blur-sm md:gap-2 md:px-3 md:py-2">
            <MapPin className="h-3.5 w-3.5 text-primary" aria-hidden="true" />
Open in Google Maps
          </span>
        </a>
        <p className="max-w-xs text-sm text-muted-foreground md:mt-3">
          Motorcycle service, parts and accessories in Baclayon, Bohol. Book
online or drop by
          during shop hours.
        </p>
      </div>

      <div className="text-sm text-muted-foreground">
        <h3 className="font-display text-base tracking-wide text-foreground
uppercase">
          Visit the Shop
        </h3>
        <ul className="mt-2">
          <FooterItem icon={MapPin} href={SHOP.mapsUrl} external>
            {SHOP.address}
          </FooterItem>
```

```
            <FooterItem icon={Clock}>
              <ShopHours />
            </FooterItem>
            <FooterItem icon={Phone} href={`tel:${SHOP.phone.replace(/\s/g, "")}`}>
              {SHOP.phone}
            </FooterItem>
            <FooterItem icon={Mail} href={`mailto:${SHOP.email}`}>
              {SHOP.email}
            </FooterItem>
            <FooterItem icon={MessageCircle} href={SHOP.messenger} external>
              Message us on Messenger
            </FooterItem>
            <FooterItem icon={Facebook} href={SHOP.facebook} external>
              Facebook page
            </FooterItem>
          </ul>
        </div>

        <div className="text-sm">
          <h3 className="font-display text-base tracking-wide text-foreground
uppercase">
            Quick Links
          </h3>
          {/* Two per row on phones keeps the footer short. */}
          <ul className="mt-2 grid grid-cols-2 gap-x-4 text-muted-foreground
md:grid-cols-1">
            {QUICK_LINKS.map((link) => (
              <li key={link.to}>
                <Link to={link.to} className="block py-1.5 hover:text-foreground">
                  {link.label}
                </Link>
              </li>
            ))}
          </ul>
        </div>
      </div>
      <div className="border-t border-border/70 py-4 text-center text-xs
text-muted-foreground">
        &copy; {new Date().getFullYear()} {SHOP.name}. All rights reserved.
      </div>
    </footer>
  );
}

/** A contact line; a link when href is given, with a tap area of about 32px. */
```

```
function FooterItem({
  icon: Icon,
  href,
  external = false,
  children,
}: {
  icon: typeof MapPin;
  href?: string;
  external?: boolean;
  children: ReactNode;
}) {
  const content = (
    <>
      <Icon className="mt-0.5 h-4 w-4 shrink-0 text-primary" aria-hidden="true" />
      <span className="min-w-0 break-words">{children}</span>
    </>
  );
  return (
    <li>
      {href ? (
        <a
          href={href}
          {...(external ? { target: "_blank", rel: "noopener noreferrer" } : {})}
          className="flex items-start gap-2 py-1.5 hover:text-foreground"
        >
          {content}
        </a>
      ) : (
        <div className="flex items-start gap-2 py-1.5">{content}</div>
      )}
    </li>
  );
}
```

src/components/site/site-header.tsx

```
import { Link } from "@tanstack/react-router";
import { Menu, X } from "lucide-react";
import { useState } from "react";

import logo from "@/assets/logo-shp.png.asset.json";
import { ThemeToggle } from "@/components/site/theme-toggle";
import { Button } from "@/components/ui/button";
import { cn } from "@/lib/utils";

const links = [
  { to: "/", label: "Home" },
  { to: "/services", label: "Services" },
  { to: "/shop", label: "Shop" },
  { to: "/my-appointment", label: "My Appointment" },
] as const;

export function SiteHeader() {
  const [open, setOpen] = useState(false);

  return (
    <header className="sticky top-0 z-50 border-b border-border/70
bg-background/85 backdrop-blur">
      <div className="site-container grid h-20
grid-cols-[minmax(0,1fr)_auto_auto] items-center gap-2 sm:gap-4
md:grid-cols-[auto_minmax(0,1fr)_auto]">
        <Link to="/" className="flex items-center">
          <img
            src={logo.url}
            alt="Fake Rider Motorparts logo"
            className="-ml-8 h-16 w-auto object-contain -mr-8"
          />
          <span className="font-display text-lg leading-none font-bold
tracking-wide uppercase">
            Fake Rider
            <span className="block text-[0.65rem] tracking-[0.28em]
text-muted-foreground">
              Motorparts
            </span>
          </span>
        </Link>

        <nav
          aria-label="Primary navigation"
```

```
          className="hidden items-center justify-self-center gap-1 md:flex"
        >
          {links.map((l) => (
            <Link
              key={l.to}
              to={l.to}
              className="rounded-md px-3 py-2 text-sm font-medium
text-muted-foreground transition-colors hover:bg-secondary
hover:text-foreground"
              activeProps={{ className: "text-primary" }}
              activeOptions={{ exact: l.to === "/" }}
            >
              {l.label}
            </Link>
          ))}
          <Button asChild className="ml-2 font-display tracking-wide uppercase">
            <Link to="/book">Book Now</Link>
          </Button>
        </nav>

        <div className="hidden justify-self-end md:block">
          <ThemeToggle />
        </div>

        {/* On phones the booking button stays in view next to the menu. */}
        <Button
          asChild
          size="sm"
          className="justify-self-end font-display tracking-wide uppercase
md:hidden"
        >
          <Link to="/book" onClick={() => setOpen(false)}>
            Book now
          </Link>
        </Button>
        <Button
          variant="ghost"
          size="icon"
          className="justify-self-end md:hidden"
          aria-label={open ? "Close menu" : "Open menu"}
          aria-expanded={open}
          onClick={() => setOpen((v) => !v)}
        >
          {open ? <X /> : <Menu />}
        </Button>
```

```
      </div>

      <div className={cn("border-t border-border/70 md:hidden", open ? "block" :
"hidden")}>
        <nav className="site-container flex flex-col py-3">
          {links.map((l) => (
            <Link
              key={l.to}
              to={l.to}
              onClick={() => setOpen(false)}
              className="rounded-md px-3 py-2 text-sm font-medium
text-muted-foreground hover:bg-secondary hover:text-foreground"
              activeProps={{ className: "text-primary" }}
              activeOptions={{ exact: l.to === "/" }}
            >
              {l.label}
            </Link>
          ))}
          <ThemeToggle showLabel className="mt-2 w-full justify-start" />
        </nav>
      </div>
      <div className="checker-strip h-[6px] w-full opacity-20" />
    </header>
  );
}
```

src/components/site/terms-agreement.tsx

```
import { Button } from "@/components/ui/button";
import { Checkbox } from "@/components/ui/checkbox";
import {
  Dialog,
  DialogClose,
  DialogContent,
  DialogDescription,
  DialogFooter,
  DialogHeader,
  DialogTitle,
  DialogTrigger,
} from "@/components/ui/dialog";
import { cn } from "@/lib/utils";

/** Terms checkbox with a readable Terms and Conditions dialog. */
export function TermsAgreement({
  id,
  checked,
  onCheckedChange,
  terms,
  invalid,
}: {
  id: string;
  checked: boolean;
  onCheckedChange: (checked: boolean) => void;
  terms: string;
  invalid?: boolean;
}) {
  return (
    <div
      className={cn(
        "flex items-start gap-3 rounded-lg border p-3 text-sm transition-colors",
        invalid ? "border-destructive bg-destructive/5" : "border-border/70
bg-muted/20",
      )}
    >
      <Checkbox
        id={id}
        checked={checked}
        onCheckedChange={(value) => onCheckedChange(value === true)}
        aria-invalid={invalid}
        className="mt-1"
      />
```

```
    <div className="leading-6">
      <label htmlFor={id}>I have read and agree to the </label>
      <Dialog>
        <DialogTrigger asChild>
          <button
            type="button"
            className="font-bold text-primary underline decoration-primary/70
underline-offset-4 transition-colors hover:text-primary/80 focus-visible:ring-2
focus-visible:ring-ring focus-visible:ring-offset-2 focus-visible:outline-none"
          >
            Terms and Conditions
          </button>
        </DialogTrigger>
        <DialogContent className="sm:max-w-2xl">
          <DialogHeader>
            <DialogTitle className="font-display text-2xl uppercase">
              Terms and Conditions
            </DialogTitle>
            <DialogDescription>
              Please read the complete booking terms before submitting your
request.
            </DialogDescription>
          </DialogHeader>
          <div className="max-h-[50vh] overflow-y-auto rounded-lg border
border-border/70 bg-card/50 p-4 text-sm leading-6 whitespace-pre-line
text-muted-foreground">
            {terms}
          </div>
          <DialogFooter className="gap-2">
            <DialogClose asChild>
              <Button type="button" variant="outline">
                Close
              </Button>
            </DialogClose>
            <DialogClose asChild>
              <Button type="button" onClick={() => onCheckedChange(true)}>
                I agree to the terms
              </Button>
            </DialogClose>
          </DialogFooter>
        </DialogContent>
      </Dialog>
      <span>.</span>
    </div>
  </div>
```

```
  );
}
```

src/components/site/theme-toggle.tsx

```
import { Moon, Sun } from "lucide-react";
import { useEffect, useState } from "react";

import { Button } from "@/components/ui/button";
import { cn } from "@/lib/utils";

type Theme = "light" | "dark";

const THEME_STORAGE_KEY = "fake-rider-theme";

function applyTheme(theme: Theme) {
  document.documentElement.classList.toggle("dark", theme === "dark");
  document.documentElement.style.colorScheme = theme;
}

function preferredTheme(): Theme {
  const saved = window.localStorage.getItem(THEME_STORAGE_KEY);
  if (saved === "light" || saved === "dark") return saved;
  return window.matchMedia("(prefers-color-scheme: dark)").matches ? "dark" :
"light";
}

type ThemeToggleProps = {
  className?: string;
  showLabel?: boolean;
};

/** A shared, persisted appearance switch for public pages and the admin
console. */
export function ThemeToggle({ className, showLabel = false }:
ThemeToggleProps) {
  const [theme, setTheme] = useState<Theme>("dark");

  useEffect(() => {
    const initialTheme = preferredTheme();
    setTheme(initialTheme);
    applyTheme(initialTheme);
  }, []);

  const nextTheme: Theme = theme === "dark" ? "light" : "dark";

  return (
    <Button
```

```
      type="button"
      variant="outline"
      size={showLabel ? "sm" : "icon"}
      className={cn(showLabel && "gap-2", className)}
      onClick={() => {
        window.localStorage.setItem(THEME_STORAGE_KEY, nextTheme);
        setTheme(nextTheme);
        applyTheme(nextTheme);
      }}
      aria-label={`Switch to ${nextTheme} mode`}
      title={`Switch to ${nextTheme} mode`}
    >
      {theme === "dark" ? <Sun /> : <Moon />}
      {showLabel && <span>{theme === "dark" ? "Light mode" : "Dark
mode"}</span>}
    </Button>
  );
}
```

src/components/site/turnstile-challenge.tsx

```
import { CircleAlert, Loader2 } from "lucide-react";
import { useEffect, useRef, useState } from "react";

import { cn } from "@/lib/utils";

const TURNSTILE_SCRIPT_ID = "cloudflare-turnstile";
/** Cloudflare's full-width widget needs at least this much room; below it, the
compact one is used. */
const TURNSTILE_MIN_WIDTH = 300;

type TurnstileTheme = "light" | "dark";

type TurnstileWidget = {
  render: (
    container: HTMLElement,
    options: {
      sitekey: string;
      action: string;
      theme: TurnstileTheme;
      size: "flexible" | "compact";
      callback: (token: string) => void;
      "expired-callback": () => void;
      "error-callback": () => void;
    },
  ) => string;
  remove: (widgetId: string) => void;
};

declare global {
  interface Window {
    turnstile?: TurnstileWidget;
  }
}

type TurnstileChallengeProps = {
  resetKey: string;
  onToken: (token: string) => void;
  /** What the check unlocks, shown above the widget. */
  label?: string;
};

type ChallengeStatus = "loading" | "shown" | "failed";
```

```
/** The site's own light/dark choice, which can differ from the device setting. */
function siteTheme(): TurnstileTheme {
  return document.documentElement.classList.contains("dark") ? "dark" : "light";
}

/** Cloudflare's explicitly rendered widget, used only when a site key is
configured. */
export function TurnstileChallenge({
  resetKey,
  onToken,
  label = "Complete the security check to submit your booking.",
}: TurnstileChallengeProps) {
  const containerRef = useRef<HTMLDivElement>(null);
  const [status, setStatus] = useState<ChallengeStatus>("loading");
  const enabled = Boolean(import.meta.env["VITE_TURNSTILE_SITE_KEY"]);

  useEffect(() => {
    const sitekey = import.meta.env["VITE_TURNSTILE_SITE_KEY"];
    const container = containerRef.current;
    if (!sitekey || !container) return;

    let active = true;
    let widgetId: string | undefined;
    let renderedTheme = siteTheme();
    // The booking review step is mounted while hidden. Waiting until the
    // container has a width lets the widget size fit the space it is shown in.
    const resizeObserver =
      typeof ResizeObserver === "undefined" ? null : new ResizeObserver(() =>
render());
    // The widget cannot change theme once shown, so it is recreated (and
    // verifies again) when the site switches between light and dark.
    const themeObserver = new MutationObserver(() => {
      if (!widgetId || !window.turnstile || siteTheme() === renderedTheme) return;
      window.turnstile.remove(widgetId);
      widgetId = undefined;
      onToken("");
      render();
    });

    // Runs once api.js has loaded. turnstile.ready() is only for pages that
    // load the script synchronously; Cloudflare warns when it is used here.
    function render() {
      if (!active || !container || widgetId || !window.turnstile) return;
      if (resizeObserver && container.clientWidth === 0) return;
      resizeObserver?.disconnect();
```

```
      renderedTheme = siteTheme();
      widgetId = window.turnstile.render(container, {
        sitekey,
        action: "booking",
        theme: renderedTheme,
        size: container.clientWidth < TURNSTILE_MIN_WIDTH ? "compact" :
"flexible",
        callback: (token) => {
          setStatus("shown");
          onToken(token);
        },
        "expired-callback": () => onToken(""),
        "error-callback": () => {
          setStatus("failed");
          onToken("");
        },
      });
      setStatus("shown");
    }

    const scriptFailed = () => {
      if (active) setStatus("failed");
    };
    let script = document.getElementById(TURNSTILE_SCRIPT_ID) as
HTMLScriptElement | null;
    if (!script) {
      script = document.createElement("script");
      script.id = TURNSTILE_SCRIPT_ID;
      script.src =
"https://challenges.cloudflare.com/turnstile/v0/api.js?render=explicit";
      script.async = true;
      document.head.append(script);
    }

    resizeObserver?.observe(container);
    themeObserver.observe(document.documentElement, {
      attributes: true,
      attributeFilter: ["class"],
    });
    if (window.turnstile) render();
    else {
      script.addEventListener("load", render);
      script.addEventListener("error", scriptFailed);
    }
```

```
    return () => {
      active = false;
      resizeObserver?.disconnect();
      themeObserver.disconnect();
      script?.removeEventListener("load", render);
      script?.removeEventListener("error", scriptFailed);
      if (widgetId && window.turnstile) window.turnstile.remove(widgetId);
      setStatus("loading");
    };
  }, [onToken, resetKey]);

  if (!enabled) return null;

  return (
    <div className="space-y-2">
      <p className="text-sm text-muted-foreground">{label}</p>
      {/* Space for the widget (65px tall when full width) so the page does not
          jump; not kept when the widget could not load at all. */}
      <div className={cn("relative w-full sm:max-w-sm", status !== "failed" &&
"min-h-[65px]")}>
        {status === "loading" && (
          <div
            className="absolute inset-0 flex items-center gap-2 rounded-md border
border-dashed border-border px-3 text-sm text-muted-foreground"
            aria-live="polite"
          >
            <Loader2 className="size-4 animate-spin" aria-hidden="true" />
            Loading security check…
          </div>
        )}
        <div ref={containerRef} className="w-full" />
      </div>
      {status === "failed" && (
        <p role="alert" className="flex items-start gap-2 text-sm text-destructive">
          <CircleAlert className="mt-0.5 size-4 shrink-0" aria-hidden="true" />
          The security check couldn’t load. Check your internet connection or turn off
ad blockers
          for this site, then reload the page.
        </p>
      )}
    </div>
  );
}
```

src/components/ui/alert-dialog.tsx

```
import * as React from "react";
import * as AlertDialogPrimitive from "@radix-ui/react-alert-dialog";

import { cn } from "@/lib/utils";
import { buttonVariants } from "@/components/ui/button";

const AlertDialog = AlertDialogPrimitive.Root;

const AlertDialogTrigger = AlertDialogPrimitive.Trigger;

const AlertDialogPortal = AlertDialogPrimitive.Portal;

const AlertDialogOverlay = React.forwardRef<
  React.ElementRef<typeof AlertDialogPrimitive.Overlay>,
  React.ComponentPropsWithoutRef<typeof AlertDialogPrimitive.Overlay>
>(({ className, ...props }, ref) => (
  <AlertDialogPrimitive.Overlay
    className={cn(
      "fixed inset-0 z-50 bg-black/80 data-[state=open]:animate-in
data-[state=closed]:animate-out data-[state=closed]:fade-out-0
data-[state=open]:fade-in-0",
      className,
    )}
    {...props}
    ref={ref}
  />
));
AlertDialogOverlay.displayName = AlertDialogPrimitive.Overlay.displayName;

const AlertDialogContent = React.forwardRef<
  React.ElementRef<typeof AlertDialogPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof AlertDialogPrimitive.Content>
>(({ className, ...props }, ref) => (
  <AlertDialogPortal>
    <AlertDialogOverlay />
    <AlertDialogPrimitive.Content
      ref={ref}
      className={cn(
        "fixed left-[50%] top-[50%] z-50 grid max-h-[calc(100dvh-2rem)]
w-[calc(100%-2rem)] max-w-lg translate-x-[-50%] translate-y-[-50%] gap-4
overflow-y-auto overscroll-contain rounded-lg border bg-background p-4
shadow-lg duration-200 data-[state=open]:animate-in
data-[state=closed]:animate-out data-[state=closed]:fade-out-0
```

```
data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
data-[state=open]:zoom-in-95 sm:p-6",
        className,
      )}
      {...props}
    />
  </AlertDialogPortal>
));
AlertDialogContent.displayName = AlertDialogPrimitive.Content.displayName;

const AlertDialogHeader = ({ className, ...props }:
React.HTMLAttributes<HTMLDivElement>) => (
  <div className={cn("flex flex-col space-y-2 text-center sm:text-left",
className)} {...props} />
);
AlertDialogHeader.displayName = "AlertDialogHeader";

const AlertDialogFooter = ({ className, ...props }:
React.HTMLAttributes<HTMLDivElement>) => (
  <div
    className={cn(
      "flex flex-col-reverse gap-2 sm:flex-row sm:justify-end sm:space-x-2
[&>button]:w-full sm:[&>button]:w-auto",
      className,
    )}
    {...props}
  />
);
AlertDialogFooter.displayName = "AlertDialogFooter";

const AlertDialogTitle = React.forwardRef<
  React.ElementRef<typeof AlertDialogPrimitive.Title>,
  React.ComponentPropsWithoutRef<typeof AlertDialogPrimitive.Title>
>(({ className, ...props }, ref) => (
  <AlertDialogPrimitive.Title
    ref={ref}
    className={cn("text-lg font-semibold", className)}
    {...props}
  />
));
AlertDialogTitle.displayName = AlertDialogPrimitive.Title.displayName;

const AlertDialogDescription = React.forwardRef<
  React.ElementRef<typeof AlertDialogPrimitive.Description>,
  React.ComponentPropsWithoutRef<typeof AlertDialogPrimitive.Description>
```

```
>(({ className, ...props }, ref) => (
  <AlertDialogPrimitive.Description
    ref={ref}
    className={cn("text-sm text-muted-foreground", className)}
    {...props}
  />
));
AlertDialogDescription.displayName =
AlertDialogPrimitive.Description.displayName;

const AlertDialogAction = React.forwardRef<
  React.ElementRef<typeof AlertDialogPrimitive.Action>,
  React.ComponentPropsWithoutRef<typeof AlertDialogPrimitive.Action>
>(({ className, ...props }, ref) => (
  <AlertDialogPrimitive.Action ref={ref} className={cn(buttonVariants(),
className)} {...props} />
));
AlertDialogAction.displayName = AlertDialogPrimitive.Action.displayName;

const AlertDialogCancel = React.forwardRef<
  React.ElementRef<typeof AlertDialogPrimitive.Cancel>,
  React.ComponentPropsWithoutRef<typeof AlertDialogPrimitive.Cancel>
>(({ className, ...props }, ref) => (
  <AlertDialogPrimitive.Cancel
    ref={ref}
    className={cn(buttonVariants({ variant: "outline" }), "mt-2 sm:mt-0",
className)}
    {...props}
  />
));
AlertDialogCancel.displayName = AlertDialogPrimitive.Cancel.displayName;

export {
  AlertDialog,
  AlertDialogPortal,
  AlertDialogOverlay,
  AlertDialogTrigger,
  AlertDialogContent,
  AlertDialogHeader,
  AlertDialogFooter,
  AlertDialogTitle,
  AlertDialogDescription,
  AlertDialogAction,
  AlertDialogCancel,
};
```

src/components/ui/alert.tsx

```
import * as React from "react";
import { cva, type VariantProps } from "class-variance-authority";

import { cn } from "@/lib/utils";

const alertVariants = cva(
  "relative w-full rounded-lg border px-4 py-3 text-sm
[&>svg+div]:translate-y-[-3px] [&>svg]:absolute [&>svg]:left-4 [&>svg]:top-4
[&>svg]:text-foreground [&>svg~*]:pl-7",
  {
    variants: {
      variant: {
        default: "bg-background text-foreground",
        destructive:
          "border-destructive/50 text-destructive dark:border-destructive
[&>svg]:text-destructive",
      },
    },
    defaultVariants: {
      variant: "default",
    },
  },
);

const Alert = React.forwardRef<
  HTMLDivElement,
  React.HTMLAttributes<HTMLDivElement> & VariantProps<typeof
alertVariants>
>(({ className, variant, ...props }, ref) => (
  <div ref={ref} role="alert" className={cn(alertVariants({ variant }), className)}
{...props} />
));
Alert.displayName = "Alert";

const AlertTitle = React.forwardRef<HTMLParagraphElement,
React.HTMLAttributes<HTMLHeadingElement>>(
  ({ className, ...props }, ref) => (
    <h5
      ref={ref}
      className={cn("mb-1 font-medium leading-none tracking-tight", className)}
      {...props}
    />
  ),
```

```
);
AlertTitle.displayName = "AlertTitle";

const AlertDescription = React.forwardRef<
  HTMLParagraphElement,
  React.HTMLAttributes<HTMLParagraphElement>
>(({ className, ...props }, ref) => (
  <div ref={ref} className={cn("text-sm [&_p]:leading-relaxed", className)}
{...props} />
));
AlertDescription.displayName = "AlertDescription";

export { Alert, AlertTitle, AlertDescription };
```

src/components/ui/badge.tsx

```
import * as React from "react";
import { cva, type VariantProps } from "class-variance-authority";

import { cn } from "@/lib/utils";

const badgeVariants = cva(
  "inline-flex items-center rounded-md border px-2.5 py-0.5 text-xs font-semibold
transition-colors focus:outline-none focus:ring-2 focus:ring-ring
focus:ring-offset-2",
  {
    variants: {
      variant: {
        default: "border-transparent bg-primary text-primary-foreground shadow
hover:bg-primary/80",
        secondary:
          "border-transparent bg-secondary text-secondary-foreground
hover:bg-secondary/80",
        destructive:
          "border-transparent bg-destructive text-destructive-foreground shadow
hover:bg-destructive/80",
        outline: "text-foreground",
      },
    },
    defaultVariants: {
      variant: "default",
    },
  },
);

export interface BadgeProps
  extends React.HTMLAttributes<HTMLDivElement>, VariantProps<typeof
badgeVariants> {}

function Badge({ className, variant, ...props }: BadgeProps) {
  return <div className={cn(badgeVariants({ variant }), className)} {...props}
/>;
}

export { Badge, badgeVariants };
```

src/components/ui/button.tsx

```
import * as React from "react";
import { Slot } from "@radix-ui/react-slot";
import { cva, type VariantProps } from "class-variance-authority";

import { cn } from "@/lib/utils";

const buttonVariants = cva(
  "inline-flex items-center justify-center gap-2 whitespace-nowrap rounded-md
text-sm font-medium cursor-pointer transition-colors focus-visible:outline-none
focus-visible:ring-1 focus-visible:ring-ring disabled:pointer-events-none
disabled:opacity-50 disabled:cursor-not-allowed [&_svg]:pointer-events-none
[&_svg]:size-4 [&_svg]:shrink-0",
  {
    variants: {
      variant: {
        default: "bg-primary text-primary-foreground shadow hover:bg-primary/90",
        destructive: "bg-destructive text-destructive-foreground shadow-sm
hover:bg-destructive/90",
        outline:
          "border border-input bg-background shadow-sm hover:bg-accent
hover:text-accent-foreground",
        secondary: "bg-secondary text-secondary-foreground shadow-sm
hover:bg-secondary/80",
        ghost: "hover:bg-accent hover:text-accent-foreground",
        link: "text-primary underline-offset-4 hover:underline",
      },
      size: {
        default: "h-11 px-4 py-2 md:h-9",
        sm: "h-10 rounded-md px-3 text-xs md:h-8",
        lg: "h-11 rounded-md px-6 md:h-10 md:px-8",
        icon: "h-11 w-11 md:h-9 md:w-9",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  },
);

export interface ButtonProps
  extends React.ButtonHTMLAttributes<HTMLButtonElement>,
VariantProps<typeof buttonVariants> {
```

```
  asChild?: boolean;
}

const Button = React.forwardRef<HTMLButtonElement, ButtonProps>(
  ({ className, variant, size, asChild = false, ...props }, ref) => {
    const Comp = asChild ? Slot : "button";
    return (
      <Comp className={cn(buttonVariants({ variant, size, className }))}
ref={ref} {...props} />
    );
  },
);
Button.displayName = "Button";

export { Button, buttonVariants };
```

src/components/ui/calendar.tsx

```
"use client";

import * as React from "react";
import { ChevronDownIcon, ChevronLeftIcon, ChevronRightIcon } from
"lucide-react";
import { DayButton, DayPicker, getDefaultClassNames } from "react-day-picker";

import { cn } from "@/lib/utils";
import { Button, buttonVariants } from "@/components/ui/button";

function Calendar({
  className,
  classNames,
  showOutsideDays = true,
  captionLayout = "label",
  buttonVariant = "ghost",
  formatters,
  components,
  ...props
}: React.ComponentProps<typeof DayPicker> & {
  buttonVariant?: React.ComponentProps<typeof Button>["variant"];
}) {
  const defaultClassNames = getDefaultClassNames();

  const baseClassNames = {
    root: cn("w-fit", defaultClassNames.root),
    months: cn("relative flex flex-col gap-4 md:flex-row",
defaultClassNames.months),
    month: cn("flex w-full flex-col gap-4", defaultClassNames.month),
    nav: cn(
      "absolute inset-x-0 top-0 z-10 flex w-full items-center justify-center gap-2",
      defaultClassNames.nav,
    ),
    button_previous: cn(
      buttonVariants({ variant: buttonVariant }),
      "z-20 h-(--cell-size) w-(--cell-size) select-none p-0 aria-disabled:opacity-50",
      defaultClassNames.button_previous,
    ),
    button_next: cn(
      buttonVariants({ variant: buttonVariant }),
      "z-20 h-(--cell-size) w-(--cell-size) select-none p-0 aria-disabled:opacity-50",
      defaultClassNames.button_next,
    ),
```

```
    month_caption: cn(
      "flex h-(--cell-size) items-center justify-center",
      defaultClassNames.month_caption,
    ),
    dropdowns: cn(
      "flex h-(--cell-size) w-full items-center justify-center gap-1.5 text-sm
font-medium",
      defaultClassNames.dropdowns,
    ),
    dropdown_root: cn(
      "has-focus:border-ring border-input shadow-xs has-focus:ring-ring/50
has-focus:ring-[3px] relative rounded-md border",
      defaultClassNames.dropdown_root,
    ),
    dropdown: cn("bg-popover absolute inset-0 opacity-0",
defaultClassNames.dropdown),
    caption_label: cn(
      "select-none font-medium",
      captionLayout === "label"
        ? "text-sm"
        : "[&>svg]:text-muted-foreground flex h-8 items-center gap-1 rounded-md
pl-2 pr-1 text-sm [&>svg]:size-3.5",
      defaultClassNames.caption_label,
    ),
    table: "w-full border-collapse",
    weekdays: cn("flex", defaultClassNames.weekdays),
    weekday: cn(
      "text-muted-foreground flex-1 select-none rounded-md text-[0.8rem]
font-normal",
      defaultClassNames.weekday,
    ),
    week: cn("mt-2 flex w-full", defaultClassNames.week),
    week_number_header: cn("w-(--cell-size) select-none",
defaultClassNames.week_number_header),
    week_number: cn(
      "text-muted-foreground select-none text-[0.8rem]",
      defaultClassNames.week_number,
    ),
    day: cn(
      "group/day relative aspect-square h-full w-full select-none p-0 text-center
[&:first-child[data-selected=true]_button]:rounded-l-md
[&:last-child[data-selected=true]_button]:rounded-r-md",
      defaultClassNames.day,
    ),
    range_start: cn("bg-accent rounded-l-md", defaultClassNames.range_start),
```

```
    range_middle: cn("rounded-none", defaultClassNames.range_middle),
    range_end: cn("bg-accent rounded-r-md", defaultClassNames.range_end),
    today: cn(
      "bg-accent text-accent-foreground rounded-md
data-[selected=true]:rounded-none",
      defaultClassNames.today,
    ),
    outside: cn(
      "text-muted-foreground aria-selected:text-muted-foreground",
      defaultClassNames.outside,
    ),
    disabled: cn("text-muted-foreground opacity-50",
defaultClassNames.disabled),
    hidden: cn("invisible", defaultClassNames.hidden),
  };

  // Merge custom classNames with base, preserving base classes instead of
replacing
  const mergedClassNames = classNames
    ? {
        ...baseClassNames,
        ...Object.fromEntries(
          Object.entries(classNames).map(([key, value]) => [
            key,
            cn(baseClassNames[key as keyof typeof baseClassNames], value),
          ]),
        ),
      }
    : baseClassNames;

  return (
    <DayPicker
      showOutsideDays={showOutsideDays}
      className={cn(
        "bg-background group/calendar p-3 [--cell-size:2rem]
[[data-slot=card-content]_&]:bg-transparent
[[data-slot=popover-content]_&]:bg-transparent",
        String.raw`rtl:**:[.rdp-button\_next>svg]:rotate-180`,
        String.raw`rtl:**:[.rdp-button\_previous>svg]:rotate-180`,
        className,
      )}
      captionLayout={captionLayout}
      formatters={{
        formatMonthDropdown: (date) => date.toLocaleString("default", { month:
"short" }),
```

```
        ...formatters,
      }}
      classNames={mergedClassNames}
      components={{
        Root: ({ className, rootRef, ...props }) => {
          return <div data-slot="calendar" ref={rootRef} className={cn(className)}
{...props} />;
        },
        Chevron: ({ className, orientation, ...props }) => {
          if (orientation === "left") {
            return <ChevronLeftIcon className={cn("size-4", className)} {...props}
/>;
          }

          if (orientation === "right") {
            return <ChevronRightIcon className={cn("size-4", className)}
{...props} />;
          }

          return <ChevronDownIcon className={cn("size-4", className)} {...props}
/>;
        },
        DayButton: CalendarDayButton,
        WeekNumber: ({ children, ...props }) => {
          return (
            <td {...props}>
              <div className="flex size-(--cell-size) items-center justify-center
text-center">
                {children}
              </div>
            </td>
          );
        },
        ...components,
      }}
      {...props}
    />
  );
}

function CalendarDayButton({
  className,
  day,
  modifiers,
  ...props
```

```
}: React.ComponentProps<typeof DayButton>) {
  const defaultClassNames = getDefaultClassNames();

  const ref = React.useRef<HTMLButtonElement>(null);
  React.useEffect(() => {
    if (modifiers["focused"]) ref.current?.focus();
  }, [modifiers]);

  return (
    <Button
      ref={ref}
      variant="ghost"
      size="icon"
      data-day={day.date.toLocaleDateString()}
      data-selected-single={
        modifiers["selected"] &&
        !modifiers["range_start"] &&
        !modifiers["range_end"] &&
        !modifiers["range_middle"]
      }
      data-range-start={modifiers["range_start"]}
      data-range-end={modifiers["range_end"]}
      data-range-middle={modifiers["range_middle"]}
      className={cn(
        "data-[selected-single=true]:bg-primary
data-[selected-single=true]:text-primary-foreground
data-[range-middle=true]:bg-accent
data-[range-middle=true]:text-accent-foreground
data-[range-start=true]:bg-primary
data-[range-start=true]:text-primary-foreground data-[range-end=true]:bg-primary
data-[range-end=true]:text-primary-foreground
group-data-[focused=true]/day:border-ring
group-data-[focused=true]/day:ring-ring/50 flex aspect-square h-auto w-full
min-w-(--cell-size) flex-col gap-1 font-normal leading-none
data-[range-end=true]:rounded-md data-[range-middle=true]:rounded-none
data-[range-start=true]:rounded-md group-data-[focused=true]/day:relative
group-data-[focused=true]/day:z-10 group-data-[focused=true]/day:ring-[3px]
[&>span]:text-xs [&>span]:opacity-70",
        defaultClassNames.day,
        className,
      )}
      {...props}
    />
  );
}
```

```
export { Calendar, CalendarDayButton };
```

src/components/ui/card.tsx

```
import * as React from "react";

import { cn } from "@/lib/utils";

const Card = React.forwardRef<HTMLDivElement,
React.HTMLAttributes<HTMLDivElement>>(
  ({ className, ...props }, ref) => (
    <div
      ref={ref}
      className={cn("rounded-xl border bg-card text-card-foreground shadow",
classNam e)}
      {...props}
    />
  ),
);
Card.displayName = "Card";

const CardHeader = React.forwardRef<HTMLDivElement,
React.HTMLAttributes<HTMLDivElement>>(
  ({ className, ...props }, ref) => (
    <div ref={ref} className={cn("flex flex-col space-y-1.5 p-6", className)}
{...props} />
  ),
);
CardHeader.displayName = "CardHeader";

const CardTitle = React.forwardRef<HTMLDivElement,
React.HTMLAttributes<HTMLDivElement>>(
  ({ className, ...props }, ref) => (
    <div
      ref={ref}
      className={cn("font-semibold leading-none tracking-tight", className)}
      {...props}
    />
  ),
);
CardTitle.displayName = "CardTitle";

const CardDescription = React.forwardRef<HTMLDivElement,
React.HTMLAttributes<HTMLDivElement>>(
  ({ className, ...props }, ref) => (
    <div ref={ref} className={cn("text-sm text-muted-foreground", className)}
{...props} />
```

```
  ),
);
CardDescription.displayName = "CardDescription";

const CardContent = React.forwardRef<HTMLDivElement,
React.HTMLAttributes<HTMLDivElement>>(
  ({ className, ...props }, ref) => (
    <div ref={ref} className={cn("p-6 pt-0", className)} {...props} />
  ),
);
CardContent.displayName = "CardContent";

const CardFooter = React.forwardRef<HTMLDivElement,
React.HTMLAttributes<HTMLDivElement>>(
  ({ className, ...props }, ref) => (
    <div ref={ref} className={cn("flex items-center p-6 pt-0", className)}
{...props} />
  ),
);
CardFooter.displayName = "CardFooter";

export { Card, CardHeader, CardFooter, CardTitle, CardDescription,
CardContent };
```

src/components/ui/chart.tsx

```
import * as React from "react";
import * as RechartsPrimitive from "recharts";

import { cn } from "@/lib/utils";

// Format: { THEME_NAME: CSS_SELECTOR }
const THEMES = { light: "", dark: ".dark" } as const;

export type ChartConfig = {
  [k in string]: {
    label?: React.ReactNode;
    icon?: React.ComponentType;
  } & (
    | { color?: string; theme?: never }
    | { color?: never; theme: Record<keyof typeof THEMES, string> }
  );
};

type ChartContextProps = {
  config: ChartConfig;
};

const ChartContext = React.createContext<ChartContextProps | null>(null);

function useChart() {
  const context = React.useContext(ChartContext);

  if (!context) {
    throw new Error("useChart must be used within a <ChartContainer />");
  }

  return context;
}

const ChartContainer = React.forwardRef<
  HTMLDivElement,
  React.ComponentProps<"div"> & {
    config: ChartConfig;
    children: React.ComponentProps<typeof
RechartsPrimitive.ResponsiveContainer>["children"];
  }
>(({ id, className, children, config, ...props }, ref) => {
  const uniqueId = React.useId();
```

```
  const chartId = `chart-${id || uniqueId.replace(/:/g, "")}`;

  return (
    <ChartContext.Provider value={{ config }}>
      <div
        data-chart={chartId}
        ref={ref}
        className={cn(
          "flex aspect-video justify-center text-xs
[&_.recharts-cartesian-axis-tick_text]:fill-muted-foreground
[&_.recharts-cartesian-grid_line[stroke='#ccc']]:stroke-border/50
[&_.recharts-curve.recharts-tooltip-cursor]:stroke-border
[&_.recharts-dot[stroke='#fff']]:stroke-transparent [&_.recharts-layer]:outline-none
[&_.recharts-polar-grid_[stroke='#ccc']]:stroke-border
[&_.recharts-radial-bar-background-sector]:fill-muted
[&_.recharts-rectangle.recharts-tooltip-cursor]:fill-muted
[&_.recharts-reference-line_[stroke='#ccc']]:stroke-border
[&_.recharts-sector[stroke='#fff']]:stroke-transparent
[&_.recharts-sector]:outline-none [&_.recharts-surface]:outline-none",
          className,
        )}
        {...props}
      >
        <ChartStyle id={chartId} config={config} />
        <RechartsPrimitive.ResponsiveContainer>{children}</RechartsPrimitive.Res
ponsiveContainer>
      </div>
    </ChartContext.Provider>
  );
});
ChartContainer.displayName = "Chart";

const ChartStyle = ({ id, config }: { id: string; config: ChartConfig }) => {
  const colorConfig = Object.entries(config).filter(([, config]) => config.theme ||
config.color);

  if (!colorConfig.length) {
    return null;
  }

  return (
    <style
      dangerouslySetInnerHTML={{
        __html: Object.entries(THEMES)
          .map(
```

```
          ([theme, prefix]) => `
${prefix} [data-chart=${id}] {
${colorConfig
  .map(([key, itemConfig]) => {
    const color = itemConfig.theme?.[theme as keyof typeof itemConfig.theme] ||
itemConfig.color;
    return color ? `  --color-${key}: ${color};` : null;
  })
  .join("\n")}
}
`,
          )
          .join("\n"),
      }}
    />
  );
};

const ChartTooltip = RechartsPrimitive.Tooltip;

const ChartTooltipContent = React.forwardRef<
  HTMLDivElement,
  React.ComponentProps<typeof RechartsPrimitive.Tooltip> &
    React.ComponentProps<"div"> & {
      hideLabel?: boolean;
      hideIndicator?: boolean;
      indicator?: "line" | "dot" | "dashed";
      nameKey?: string;
      labelKey?: string;
    }
>(
  (
    {
      active,
      payload,
      className,
      indicator = "dot",
      hideLabel = false,
      hideIndicator = false,
      label,
      labelFormatter,
      labelClassName,
      formatter,
      color,
      nameKey,
```

```
      labelKey,
    },
    ref,
  ) => {
    const { config } = useChart();

    const tooltipLabel = React.useMemo(() => {
      if (hideLabel || !payload?.length) {
        return null;
      }

      const [item] = payload;
      const key = `${labelKey || item?.dataKey || item?.name || "value"}`;
      const itemConfig = getPayloadConfigFromPayload(config, item, key);
      const value =
        !labelKey && typeof label === "string"
          ? config[label as keyof typeof config]?.label || label
          : itemConfig?.label;

      if (labelFormatter) {
        return (
          <div className={cn("font-medium",
labelClassName)}>{labelFormatter(value, payload)}</div>
        );
      }

      if (!value) {
        return null;
      }

      return <div className={cn("font-medium", labelClassName)}>{value}</div>;
    }, [label, labelFormatter, payload, hideLabel, labelClassName, config,
labelKey]);

    if (!active || !payload?.length) {
      return null;
    }

    const nestLabel = payload.length === 1 && indicator !== "dot";

    return (
      <div
        ref={ref}
        className={cn(
          "grid min-w-[8rem] items-start gap-1.5 rounded-lg border border-border/50
```

```
bg-background px-2.5 py-1.5 text-xs shadow-xl",
        className,
      )}
    >
      {!nestLabel ? tooltipLabel : null}
      <div className="grid gap-1.5">
        {payload
          .filter((item) => item.type !== "none")
          .map((item, index) => {
            const key = `${nameKey || item.name || item.dataKey || "value"}`;
            const itemConfig = getPayloadConfigFromPayload(config, item, key);
            const indicatorColor = color || item.payload.fill || item.color;

            return (
              <div
                key={item.dataKey}
                className={cn(
                  "flex w-full flex-wrap items-stretch gap-2 [&>svg]:h-2.5 [&>svg]:w-2.5
[&>svg]:text-muted-foreground",
                  indicator === "dot" && "items-center",
                )}
              >
                {formatter && item?.value !== undefined && item.name ? (
                  formatter(item.value, item.name, item, index, item.payload)
                ) : (
                  <>
                    {itemConfig?.icon ? (
                      <itemConfig.icon />
                    ) : (
                      !hideIndicator && (
                        <div
                          className={cn(
                            "shrink-0 rounded-[2px] border-(--color-border)
bg-(--color-bg)",
                            {
                              "h-2.5 w-2.5": indicator === "dot",
                              "w-1": indicator === "line",
                              "w-0 border-[1.5px] border-dashed bg-transparent":
                                indicator === "dashed",
                              "my-0.5": nestLabel && indicator === "dashed",
                            },
                          )}
                          style={{
                            "--color-bg": indicatorColor,
```

```
                        "--color-border": indicatorColor,
                      } as React.CSSProperties
                    }
                  />
                )
              )}
              <div
                className={cn(
                  "flex flex-1 justify-between leading-none",
                  nestLabel ? "items-end" : "items-center",
                )}
              >
                <div className="grid gap-1.5">
                  {nestLabel ? tooltipLabel : null}
                  <span className="text-muted-foreground">
                    {itemConfig?.label || item.name}
                  </span>
                </div>
                {item.value && (
                  <span className="font-mono font-medium tabular-nums
text-foreground">
                    {item.value.toLocaleString()}
                  </span>
                )}
              </div>
            </>
          )}
        </div>
      );
    })}
  </div>
 </div>
 );
},
);
ChartTooltipContent.displayName = "ChartTooltip";

const ChartLegend = RechartsPrimitive.Legend;

const ChartLegendContent = React.forwardRef<
  HTMLDivElement,
  React.ComponentProps<"div"> &
    Pick<RechartsPrimitive.LegendProps, "payload" | "verticalAlign"> & {
      hideIcon?: boolean;
      nameKey?: string;
```

```
  }
>(({ className, hideIcon = false, payload, verticalAlign = "bottom", nameKey },
ref) => {
  const { config } = useChart();

  if (!payload?.length) {
    return null;
  }

  return (
    <div
      ref={ref}
      className={cn(
        "flex items-center justify-center gap-4",
        verticalAlign === "top" ? "pb-3" : "pt-3",
        className,
      )}
    >
      {payload
        .filter((item) => item.type !== "none")
        .map((item) => {
          const key = `${nameKey || item.dataKey || "value"}`;
          const itemConfig = getPayloadConfigFromPayload(config, item, key);

          return (
            <div
              key={item.value}
              className={cn(
                "flex items-center gap-1.5 [&>svg]:h-3 [&>svg]:w-3
[&>svg]:text-muted-foreground",
              )}
            >
              {itemConfig?.icon && !hideIcon ? (
                <itemConfig.icon />
              ) : (
                <div
                  className="h-2 w-2 shrink-0 rounded-[2px]"
                  style={{
                    backgroundColor: item.color,
                  }}
                />
              )}
              {itemConfig?.label}
            </div>
          );
```

```
        })}
    </div>
  );
});
ChartLegendContent.displayName = "ChartLegend";

// Helper to extract item config from a payload.
function getPayloadConfigFromPayload(config: ChartConfig, payload: unknown,
key: string) {
  if (typeof payload !== "object" || payload === null) {
    return undefined;
  }

  const payloadPayload =
    "payload" in payload && typeof payload.payload === "object" &&
payload.payload !== null
      ? payload.payload
      : undefined;

  let configLabelKey: string = key;

  if (key in payload && typeof payload[key as keyof typeof payload] === "string") {
    configLabelKey = payload[key as keyof typeof payload] as string;
  } else if (
    payloadPayload &&
    key in payloadPayload &&
    typeof payloadPayload[key as keyof typeof payloadPayload] === "string"
  ) {
    configLabelKey = payloadPayload[key as keyof typeof payloadPayload] as
string;
  }

  return configLabelKey in config ? config[configLabelKey] : config[key as keyof
typeof config];
}

export {
  ChartContainer,
  ChartTooltip,
  ChartTooltipContent,
  ChartLegend,
  ChartLegendContent,
  ChartStyle,
};
```

src/components/ui/checkbox.tsx

```
import * as React from "react";
import * as CheckboxPrimitive from "@radix-ui/react-checkbox";
import { Check } from "lucide-react";

import { cn } from "@/lib/utils";

const Checkbox = React.forwardRef<
  React.ElementRef<typeof CheckboxPrimitive.Root>,
  React.ComponentPropsWithoutRef<typeof CheckboxPrimitive.Root>
>(({ className, ...props }, ref) => (
  <CheckboxPrimitive.Root
    ref={ref}
    className={cn(
      "relative grid h-5 w-5 shrink-0 place-content-center rounded-sm border
border-primary shadow cursor-pointer focus-visible:outline-none
focus-visible:ring-1 focus-visible:ring-ring after:absolute after:-inset-2
after:md:hidden disabled:cursor-not-allowed disabled:opacity-50
data-[state=checked]:bg-primary data-[state=checked]:text-primary-foreground
md:h-4 md:w-4",
      className,
    )}
    {...props}
  >
    <CheckboxPrimitive.Indicator className={cn("grid place-content-center
text-current")}>
      <Check className="h-4 w-4" />
    </CheckboxPrimitive.Indicator>
  </CheckboxPrimitive.Root>
));
Checkbox.displayName = CheckboxPrimitive.Root.displayName;

export { Checkbox };
```

src/components/ui/dialog.tsx

```
"use client";

import * as React from "react";
import * as DialogPrimitive from "@radix-ui/react-dialog";

import { cn } from "@/lib/utils";

const Dialog = DialogPrimitive.Root;

const DialogTrigger = DialogPrimitive.Trigger;

const DialogPortal = DialogPrimitive.Portal;

const DialogClose = DialogPrimitive.Close;

const DialogOverlay = React.forwardRef<
  React.ElementRef<typeof DialogPrimitive.Overlay>,
  React.ComponentPropsWithoutRef<typeof DialogPrimitive.Overlay>
>(({ className, ...props }, ref) => (
  <DialogPrimitive.Overlay
    ref={ref}
    className={cn(
      "fixed inset-0 z-50 bg-black/80  data-[state=open]:animate-in
data-[state=closed]:animate-out data-[state=closed]:fade-out-0
data-[state=open]:fade-in-0",
      className,
    )}
    {...props}
  />
));
DialogOverlay.displayName = DialogPrimitive.Overlay.displayName;

const DialogContent = React.forwardRef<
  React.ElementRef<typeof DialogPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof DialogPrimitive.Content>
>(({ className, children, ...props }, ref) => (
  <DialogPortal>
    <DialogOverlay />
    <DialogPrimitive.Content
      ref={ref}
      className={cn(
        "fixed left-[50%] top-[50%] z-50 grid max-h-[calc(100dvh-2rem)]
w-[calc(100%-2rem)] max-w-lg translate-x-[-50%] translate-y-[-50%] gap-4
```

```
overflow-y-auto overscroll-contain rounded-lg border bg-background p-4
shadow-lg duration-200 data-[state=open]:animate-in
data-[state=closed]:animate-out data-[state=closed]:fade-out-0
data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
data-[state=open]:zoom-in-95 sm:p-6",
        className,
      )}
      {...props}
    >
      {children}
    </DialogPrimitive.Content>
  </DialogPortal>
));
DialogContent.displayName = DialogPrimitive.Content.displayName;

const DialogHeader = ({ className, ...props }:
React.HTMLAttributes<HTMLDivElement>) => (
  <div className={cn("flex flex-col space-y-1.5 text-center sm:text-left",
className)} {...props} />
);
DialogHeader.displayName = "DialogHeader";

const DialogFooter = ({ className, ...props }:
React.HTMLAttributes<HTMLDivElement>) => (
  <div
    className={cn(
      "flex flex-col-reverse gap-2 sm:flex-row sm:justify-end sm:space-x-2
[&>button]:w-full sm:[&>button]:w-auto",
      className,
    )}
    {...props}
  />
);
DialogFooter.displayName = "DialogFooter";

const DialogTitle = React.forwardRef<
  React.ElementRef<typeof DialogPrimitive.Title>,
  React.ComponentPropsWithoutRef<typeof DialogPrimitive.Title>
>(({ className, ...props }, ref) => (
  <DialogPrimitive.Title
    ref={ref}
    className={cn("text-lg font-semibold leading-none tracking-tight",
className)}
    {...props}
  />
```

```
));
DialogTitle.displayName = DialogPrimitive.Title.displayName;

const DialogDescription = React.forwardRef<
  React.ElementRef<typeof DialogPrimitive.Description>,
  React.ComponentPropsWithoutRef<typeof DialogPrimitive.Description>
>(({ className, ...props }, ref) => (
  <DialogPrimitive.Description
    ref={ref}
    className={cn("text-sm text-muted-foreground", className)}
    {...props}
  />
));
DialogDescription.displayName = DialogPrimitive.Description.displayName;

export {
  Dialog,
  DialogPortal,
  DialogOverlay,
  DialogTrigger,
  DialogClose,
  DialogContent,
  DialogHeader,
  DialogFooter,
  DialogTitle,
  DialogDescription,
};
```

src/components/ui/dropdown-menu.tsx

```
"use client";

import * as React from "react";
import * as DropdownMenuPrimitive from "@radix-ui/react-dropdown-menu";
import { Check, ChevronRight, Circle } from "lucide-react";

import { cn } from "@/lib/utils";

const DropdownMenu = DropdownMenuPrimitive.Root;

const DropdownMenuTrigger = DropdownMenuPrimitive.Trigger;

const DropdownMenuGroup = DropdownMenuPrimitive.Group;

const DropdownMenuPortal = DropdownMenuPrimitive.Portal;

const DropdownMenuSub = DropdownMenuPrimitive.Sub;

const DropdownMenuRadioGroup = DropdownMenuPrimitive.RadioGroup;

const DropdownMenuSubTrigger = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.SubTrigger>,
  React.ComponentPropsWithoutRef<typeof
DropdownMenuPrimitive.SubTrigger> & {
    inset?: boolean;
  }
>(({ className, inset, children, ...props }, ref) => (
  <DropdownMenuPrimitive.SubTrigger
    ref={ref}
    className={cn(
      "flex cursor-default select-none items-center gap-2 rounded-sm px-2 py-1.5
text-sm outline-none focus:bg-accent data-[state=open]:bg-accent
[&_svg]:pointer-events-none [&_svg]:size-4 [&_svg]:shrink-0",
      inset && "pl-8",
      className,
    )}
    {...props}
  >
    {children}
    <ChevronRight className="ml-auto" />
  </DropdownMenuPrimitive.SubTrigger>
));
DropdownMenuSubTrigger.displayName =
```

```
DropdownMenuPrimitive.SubTrigger.displayName;

const DropdownMenuSubContent = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.SubContent>,
  React.ComponentPropsWithoutRef<typeof
DropdownMenuPrimitive.SubContent>
>(({ className, ...props }, ref) => (
  <DropdownMenuPrimitive.SubContent
    ref={ref}
    className={cn(
      "z-50 max-w-[calc(100vw-2rem)] min-w-[8rem] overflow-hidden rounded-md
border bg-popover p-1 text-popover-foreground shadow-lg
data-[state=open]:animate-in data-[state=closed]:animate-out
data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0
data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95
data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2
data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2
origin-(--radix-dropdown-menu-content-transform-origin)",
      className,
    )}
    {...props}
  />
));
DropdownMenuSubContent.displayName =
DropdownMenuPrimitive.SubContent.displayName;

const DropdownMenuContent = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.Content>
>(({ className, sideOffset = 4, ...props }, ref) => (
  <DropdownMenuPrimitive.Portal>
    <DropdownMenuPrimitive.Content
      ref={ref}
      sideOffset={sideOffset}
      className={cn(
        "z-50 max-h-[var(--radix-dropdown-menu-content-available-height)]
max-w-[calc(100vw-2rem)] min-w-[8rem] overflow-y-auto overflow-x-hidden
rounded-md border bg-popover p-1 text-popover-foreground shadow-md",
        "data-[state=open]:animate-in data-[state=closed]:animate-out
data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0
data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95
data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2
data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2
origin-(--radix-dropdown-menu-content-transform-origin)",
        className,
```

```
      )}
      {...props}
    />
  </DropdownMenuPrimitive.Portal>
));
DropdownMenuContent.displayName =
DropdownMenuPrimitive.Content.displayName;

const DropdownMenuItem = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.Item>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.Item> & {
    inset?: boolean;
  }
>(({ className, inset, ...props }, ref) => (
  <DropdownMenuPrimitive.Item
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center gap-2 rounded-sm px-2
py-1.5 text-sm outline-none transition-colors focus:bg-accent
focus:text-accent-foreground data-[disabled]:pointer-events-none
data-[disabled]:opacity-50 [&>svg]:size-4 [&>svg]:shrink-0",
      inset && "pl-8",
      className,
    )}
    {...props}
  />
));
DropdownMenuItem.displayName = DropdownMenuPrimitive.Item.displayName;

const DropdownMenuCheckboxItem = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.CheckboxItem>,
  React.ComponentPropsWithoutRef<typeof
DropdownMenuPrimitive.CheckboxItem>
>(({ className, children, ...props }, ref) => (
  <DropdownMenuPrimitive.CheckboxItem
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8
pr-2 text-sm outline-none transition-colors focus:bg-accent
focus:text-accent-foreground data-[disabled]:pointer-events-none
data-[disabled]:opacity-50",
      className,
    )}
    {...props}
  >
```

```
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center
justify-center">
      <DropdownMenuPrimitive.ItemIndicator>
        <Check className="h-4 w-4" />
      </DropdownMenuPrimitive.ItemIndicator>
    </span>
    {children}
  </DropdownMenuPrimitive.CheckboxItem>
));
DropdownMenuCheckboxItem.displayName =
DropdownMenuPrimitive.CheckboxItem.displayName;

const DropdownMenuRadioItem = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.RadioItem>,
  React.ComponentPropsWithoutRef<typeof
DropdownMenuPrimitive.RadioItem>
>(({ className, children, ...props }, ref) => (
  <DropdownMenuPrimitive.RadioItem
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8
pr-2 text-sm outline-none transition-colors focus:bg-accent
focus:text-accent-foreground data-[disabled]:pointer-events-none
data-[disabled]:opacity-50",
      className,
    )}
    {...props}
  >
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center
justify-center">
      <DropdownMenuPrimitive.ItemIndicator>
        <Circle className="h-2 w-2 fill-current" />
      </DropdownMenuPrimitive.ItemIndicator>
    </span>
    {children}
  </DropdownMenuPrimitive.RadioItem>
));
DropdownMenuRadioItem.displayName =
DropdownMenuPrimitive.RadioItem.displayName;

const DropdownMenuLabel = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.Label>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.Label> & {
    inset?: boolean;
  }
```

```
>(({ className, inset, ...props }, ref) => (
  <DropdownMenuPrimitive.Label
    ref={ref}
    className={cn("px-2 py-1.5 text-sm font-semibold", inset && "pl-8",
className)}
    {...props}
  />
));
DropdownMenuLabel.displayName =
DropdownMenuPrimitive.Label.displayName;

const DropdownMenuSeparator = React.forwardRef<
  React.ElementRef<typeof DropdownMenuPrimitive.Separator>,
  React.ComponentPropsWithoutRef<typeof DropdownMenuPrimitive.Separator>
>(({ className, ...props }, ref) => (
  <DropdownMenuPrimitive.Separator
    ref={ref}
    className={cn("-mx-1 my-1 h-px bg-muted", className)}
    {...props}
  />
));
DropdownMenuSeparator.displayName =
DropdownMenuPrimitive.Separator.displayName;

const DropdownMenuShortcut = ({ className, ...props }:
React.HTMLAttributes<HTMLSpanElement>) => {
  return (
    <span className={cn("ml-auto text-xs tracking-widest opacity-60",
className)} {...props} />
  );
};
DropdownMenuShortcut.displayName = "DropdownMenuShortcut";

export {
  DropdownMenu,
  DropdownMenuTrigger,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuCheckboxItem,
  DropdownMenuRadioItem,
  DropdownMenuLabel,
  DropdownMenuSeparator,
  DropdownMenuShortcut,
  DropdownMenuGroup,
  DropdownMenuPortal,
```

```
  DropdownMenuSub,
  DropdownMenuSubContent,
  DropdownMenuSubTrigger,
  DropdownMenuRadioGroup,
};
```

src/components/ui/field-error.tsx

```
import { CircleAlert } from "lucide-react";

import { cn } from "@/lib/utils";

/** Consistent inline validation feedback for every form field. */
export function FieldError({
  message,
  className,
}: {
  message?: string | undefined;
  className?: string | undefined;
}) {
  if (!message) return null;

  return (
    <p role="alert" className={cn("flex items-center gap-1 text-xs
text-destructive", className)}>
      <CircleAlert className="h-3.5 w-3.5 shrink-0" aria-hidden="true" />
      <span>{message}</span>
    </p>
  );
}
```

src/components/ui/input.tsx

```
import * as React from "react";

import { cn } from "@/lib/utils";

const Input = React.forwardRef<HTMLInputElement,
React.ComponentProps<"input">>(
  ({ className, type, ...props }, ref) => {
    return (
      <input
        type={type}
        className={cn(
          "flex h-11 w-full rounded-md border border-input bg-transparent px-3 py-1
text-base shadow-sm transition-colors file:border-0 file:bg-transparent file:text-sm
file:font-medium file:text-foreground placeholder:text-muted-foreground
focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring
aria-invalid:border-destructive aria-invalid:ring-1 aria-invalid:ring-destructive/30
disabled:cursor-not-allowed disabled:opacity-50 md:h-9 md:text-sm",
          className,
        )}
        ref={ref}
        {...props}
      />
    );
  },
);
Input.displayName = "Input";

export { Input };
```

src/components/ui/label.tsx

```
"use client";

import * as React from "react";
import * as LabelPrimitive from "@radix-ui/react-label";
import { cva, type VariantProps } from "class-variance-authority";

import { cn } from "@/lib/utils";

const labelVariants = cva(
  "text-sm font-medium leading-none peer-disabled:cursor-not-allowed
peer-disabled:opacity-70",
);

const Label = React.forwardRef<
  React.ElementRef<typeof LabelPrimitive.Root>,
  React.ComponentPropsWithoutRef<typeof LabelPrimitive.Root> &
VariantProps<typeof labelVariants>
>(({ className, ...props }, ref) => (
  <LabelPrimitive.Root ref={ref} className={cn(labelVariants(), className)}
{...props} />
));
Label.displayName = LabelPrimitive.Root.displayName;

export { Label };
```

src/components/ui/popover.tsx

```
import * as React from "react";
import * as PopoverPrimitive from "@radix-ui/react-popover";

import { cn } from "@/lib/utils";

const Popover = PopoverPrimitive.Root;

const PopoverTrigger = PopoverPrimitive.Trigger;

const PopoverAnchor = PopoverPrimitive.Anchor;

const PopoverContent = React.forwardRef<
  React.ElementRef<typeof PopoverPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof PopoverPrimitive.Content>
>(({ className, align = "center", sideOffset = 4, ...props }, ref) => (
  <PopoverPrimitive.Portal>
    <PopoverPrimitive.Content
      ref={ref}
      align={align}
      sideOffset={sideOffset}
      className={cn(
        "z-50 w-72 max-w-[calc(100vw-2rem)] rounded-md border bg-popover p-4
text-popover-foreground shadow-md outline-none data-[state=open]:animate-in
data-[state=closed]:animate-out data-[state=closed]:fade-out-0
data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
data-[side=top]:slide-in-from-bottom-2
origin-(--radix-popover-content-transform-origin)",
        className,
      )}
      {...props}
    />
  </PopoverPrimitive.Portal>
));
PopoverContent.displayName = PopoverPrimitive.Content.displayName;

export { Popover, PopoverTrigger, PopoverContent, PopoverAnchor };
```

src/components/ui/select.tsx

```
"use client";

import * as React from "react";
import * as SelectPrimitive from "@radix-ui/react-select";
import { Check, ChevronDown, ChevronUp } from "lucide-react";

import { cn } from "@/lib/utils";

const Select = SelectPrimitive.Root;

const SelectGroup = SelectPrimitive.Group;

const SelectValue = SelectPrimitive.Value;

const SelectTrigger = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.Trigger>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.Trigger>
>(({ className, children, ...props }, ref) => (
  <SelectPrimitive.Trigger
    ref={ref}
    className={cn(
      "flex h-11 w-full items-center justify-between whitespace-nowrap rounded-md
border border-input bg-transparent px-3 py-2 text-base shadow-sm
ring-offset-background cursor-pointer data-[placeholder]:text-muted-foreground
focus:outline-none focus:ring-1 focus:ring-ring aria-invalid:border-destructive
aria-invalid:ring-1 aria-invalid:ring-destructive/30 disabled:cursor-not-allowed
disabled:opacity-50 [&>span]:line-clamp-1 md:h-9 md:text-sm",
      className,
    )}
    {...props}
  >
    {children}
    <SelectPrimitive.Icon asChild>
      <ChevronDown className="h-4 w-4 opacity-50" />
    </SelectPrimitive.Icon>
  </SelectPrimitive.Trigger>
));
SelectTrigger.displayName = SelectPrimitive.Trigger.displayName;

const SelectScrollUpButton = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.ScrollUpButton>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.ScrollUpButton>
>(({ className, ...props }, ref) => (
```

```
  <SelectPrimitive.ScrollUpButton
    ref={ref}
    className={cn("flex cursor-default items-center justify-center py-1",
className)}
    {...props}
  >
    <ChevronUp className="h-4 w-4" />
  </SelectPrimitive.ScrollUpButton>
));
SelectScrollUpButton.displayName =
SelectPrimitive.ScrollUpButton.displayName;

const SelectScrollDownButton = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.ScrollDownButton>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.ScrollDownButton>
>(({ className, ...props }, ref) => (
  <SelectPrimitive.ScrollDownButton
    ref={ref}
    className={cn("flex cursor-default items-center justify-center py-1",
className)}
    {...props}
  >
    <ChevronDown className="h-4 w-4" />
  </SelectPrimitive.ScrollDownButton>
));
SelectScrollDownButton.displayName =
SelectPrimitive.ScrollDownButton.displayName;

const SelectContent = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.Content>
>(({ className, children, position = "popper", ...props }, ref) => (
  <SelectPrimitive.Portal>
    <SelectPrimitive.Content
      ref={ref}
      className={cn(
        "relative z-50 max-h-(--radix-select-content-available-height)
max-w-[calc(100vw-2rem)] min-w-[8rem] overflow-y-auto overflow-x-hidden
rounded-md border bg-popover text-popover-foreground shadow-md
data-[state=open]:animate-in data-[state=closed]:animate-out
data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0
data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95
data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2
data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2
origin-(--radix-select-content-transform-origin)",
```

```
        position === "popper" &&
          "data-[side=bottom]:translate-y-1 data-[side=left]:-translate-x-1
data-[side=right]:translate-x-1 data-[side=top]:-translate-y-1",
        className,
      )}
      position={position}
      {...props}
    >
      <SelectScrollUpButton />
      <SelectPrimitive.Viewport
        className={cn(
          "p-1",
          position === "popper" &&
            "h-[var(--radix-select-trigger-height)] w-full
min-w-[var(--radix-select-trigger-width)]",
        )}
      >
        {children}
      </SelectPrimitive.Viewport>
      <SelectScrollDownButton />
    </SelectPrimitive.Content>
  </SelectPrimitive.Portal>
));
SelectContent.displayName = SelectPrimitive.Content.displayName;

const SelectLabel = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.Label>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.Label>
>(({ className, ...props }, ref) => (
  <SelectPrimitive.Label
    ref={ref}
    className={cn("px-2 py-1.5 text-sm font-semibold", className)}
    {...props}
  />
));
SelectLabel.displayName = SelectPrimitive.Label.displayName;

const SelectItem = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.Item>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.Item>
>(({ className, children, ...props }, ref) => (
  <SelectPrimitive.Item
    ref={ref}
    className={cn(
      "relative flex w-full cursor-default select-none items-center rounded-sm py-1.5
```

```
pl-2 pr-8 text-sm outline-none focus:bg-accent focus:text-accent-foreground
data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
      className,
    )}
    {...props}
  >
    <span className="absolute right-2 flex h-3.5 w-3.5 items-center
justify-center">
      <SelectPrimitive.ItemIndicator>
        <Check className="h-4 w-4" />
      </SelectPrimitive.ItemIndicator>
    </span>
    <SelectPrimitive.ItemText>{children}</SelectPrimitive.ItemText>
  </SelectPrimitive.Item>
));
SelectItem.displayName = SelectPrimitive.Item.displayName;

const SelectSeparator = React.forwardRef<
  React.ElementRef<typeof SelectPrimitive.Separator>,
  React.ComponentPropsWithoutRef<typeof SelectPrimitive.Separator>
>(({ className, ...props }, ref) => (
  <SelectPrimitive.Separator
    ref={ref}
    className={cn("-mx-1 my-1 h-px bg-muted", className)}
    {...props}
  />
));
SelectSeparator.displayName = SelectPrimitive.Separator.displayName;

export {
  Select,
  SelectGroup,
  SelectValue,
  SelectTrigger,
  SelectContent,
  SelectLabel,
  SelectItem,
  SelectSeparator,
  SelectScrollUpButton,
  SelectScrollDownButton,
};
```

src/components/ui/separator.tsx

```
import * as React from "react";
import * as SeparatorPrimitive from "@radix-ui/react-separator";

import { cn } from "@/lib/utils";

const Separator = React.forwardRef<
  React.ElementRef<typeof SeparatorPrimitive.Root>,
  React.ComponentPropsWithoutRef<typeof SeparatorPrimitive.Root>
>(({ className, orientation = "horizontal", decorative = true, ...props }, ref) => (
  <SeparatorPrimitive.Root
    ref={ref}
    decorative={decorative}
    orientation={orientation}
    className={cn(
      "shrink-0 bg-border",
      orientation === "horizontal" ? "h-[1px] w-full" : "h-full w-[1px]",
      className,
    )}
    {...props}
  />
));
Separator.displayName = SeparatorPrimitive.Root.displayName;

export { Separator };
```

src/components/ui/sheet.tsx

```
"use client";

import * as React from "react";
import * as SheetPrimitive from "@radix-ui/react-dialog";
import { cva, type VariantProps } from "class-variance-authority";
import { X } from "lucide-react";

import { cn } from "@/lib/utils";

const Sheet = SheetPrimitive.Root;

const SheetTrigger = SheetPrimitive.Trigger;

const SheetClose = SheetPrimitive.Close;

const SheetPortal = SheetPrimitive.Portal;

const SheetOverlay = React.forwardRef<
  React.ElementRef<typeof SheetPrimitive.Overlay>,
  React.ComponentPropsWithoutRef<typeof SheetPrimitive.Overlay>
>(({ className, ...props }, ref) => (
  <SheetPrimitive.Overlay
    className={cn(
      "fixed inset-0 z-50 bg-black/80  data-[state=open]:animate-in
data-[state=closed]:animate-out data-[state=closed]:fade-out-0
data-[state=open]:fade-in-0",
      className,
    )}
    {...props}
    ref={ref}
  />
));
SheetOverlay.displayName = SheetPrimitive.Overlay.displayName;

const sheetVariants = cva(
  "fixed z-50 max-h-[100dvh] overflow-y-auto overscroll-contain bg-background
p-4 shadow-lg transition ease-in-out data-[state=closed]:duration-300
data-[state=open]:duration-500 data-[state=open]:animate-in
data-[state=closed]:animate-out sm:p-6",
  {
    variants: {
      side: {
        top: "inset-x-0 top-0 border-b data-[state=closed]:slide-out-to-top
```

```
data-[state=open]:slide-in-from-top",
        bottom:
          "inset-x-0 bottom-0 border-t data-[state=closed]:slide-out-to-bottom
data-[state=open]:slide-in-from-bottom",
        left: "inset-y-0 left-0 h-full w-3/4 border-r data-[state=closed]:slide-out-to-left
data-[state=open]:slide-in-from-left sm:max-w-sm",
        right:
          "inset-y-0 right-0 h-full w-3/4 border-l data-[state=closed]:slide-out-to-right
data-[state=open]:slide-in-from-right sm:max-w-sm",
      },
    },
    defaultVariants: {
      side: "right",
    },
  },
);

interface SheetContentProps
  extends
    React.ComponentPropsWithoutRef<typeof SheetPrimitive.Content>,
    VariantProps<typeof sheetVariants> {}

const SheetContent = React.forwardRef<
  React.ElementRef<typeof SheetPrimitive.Content>,
  SheetContentProps
>(({ side = "right", className, children, ...props }, ref) => (
  <SheetPortal>
    <SheetOverlay />
    <SheetPrimitive.Content ref={ref} className={cn(sheetVariants({ side }),
className)} {...props}>
      <SheetPrimitive.Close className="absolute top-2 right-2 inline-flex h-11
w-11 items-center justify-center rounded-sm opacity-70 ring-offset-background
cursor-pointer transition-opacity hover:opacity-100 focus:outline-none
focus:ring-2 focus:ring-ring focus:ring-offset-2 disabled:pointer-events-none
data-[state=open]:bg-secondary sm:top-4 sm:right-4 sm:h-9 sm:w-9">
        <X className="h-4 w-4" />
        <span className="sr-only">Close</span>
      </SheetPrimitive.Close>
      {children}
    </SheetPrimitive.Content>
  </SheetPortal>
));
SheetContent.displayName = SheetPrimitive.Content.displayName;

const SheetHeader = ({ className, ...props }:
```

```
React.HTMLAttributes<HTMLDivElement>) => (
  <div className={cn("flex flex-col space-y-2 text-center sm:text-left",
className)} {...props} />
);
SheetHeader.displayName = "SheetHeader";

const SheetFooter = ({ className, ...props }:
React.HTMLAttributes<HTMLDivElement>) => (
  <div
    className={cn("flex flex-col-reverse sm:flex-row sm:justify-end
sm:space-x-2", className)}
    {...props}
  />
);
SheetFooter.displayName = "SheetFooter";

const SheetTitle = React.forwardRef<
  React.ElementRef<typeof SheetPrimitive.Title>,
  React.ComponentPropsWithoutRef<typeof SheetPrimitive.Title>
>(({ className, ...props }, ref) => (
  <SheetPrimitive.Title
    ref={ref}
    className={cn("text-lg font-semibold text-foreground", className)}
    {...props}
  />
));
SheetTitle.displayName = SheetPrimitive.Title.displayName;

const SheetDescription = React.forwardRef<
  React.ElementRef<typeof SheetPrimitive.Description>,
  React.ComponentPropsWithoutRef<typeof SheetPrimitive.Description>
>(({ className, ...props }, ref) => (
  <SheetPrimitive.Description
    ref={ref}
    className={cn("text-sm text-muted-foreground", className)}
    {...props}
  />
));
SheetDescription.displayName = SheetPrimitive.Description.displayName;

export {
  Sheet,
  SheetPortal,
  SheetOverlay,
  SheetTrigger,
```

```
  SheetClose,
  SheetContent,
  SheetHeader,
  SheetFooter,
  SheetTitle,
  SheetDescription,
};
```

src/components/ui/sidebar.tsx

```
import * as React from "react";
import { Slot } from "@radix-ui/react-slot";
import { cva, type VariantProps } from "class-variance-authority";
import { PanelLeft } from "lucide-react";

import { useIsMobile } from "@/hooks/use-mobile";
import { cn } from "@/lib/utils";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Separator } from "@/components/ui/separator";
import {
  Sheet,
  SheetContent,
  SheetDescription,
  SheetHeader,
  SheetTitle,
} from "@/components/ui/sheet";
import { Skeleton } from "@/components/ui/skeleton";
import { Tooltip, TooltipContent, TooltipProvider, TooltipTrigger } from
"@/components/ui/tooltip";

const SIDEBAR_COOKIE_NAME = "sidebar_state";
const SIDEBAR_COOKIE_MAX_AGE = 60 * 60 * 24 * 7;
const SIDEBAR_WIDTH = "16rem";
const SIDEBAR_WIDTH_MOBILE = "18rem";
const SIDEBAR_WIDTH_ICON = "3rem";
const SIDEBAR_KEYBOARD_SHORTCUT = "b";

type SidebarContextProps = {
  state: "expanded" | "collapsed";
  open: boolean;
  setOpen: (open: boolean) => void;
  openMobile: boolean;
  setOpenMobile: (open: boolean) => void;
  isMobile: boolean;
  toggleSidebar: () => void;
};

const SidebarContext = React.createContext<SidebarContextProps | null>(null);

function useSidebar() {
  const context = React.useContext(SidebarContext);
  if (!context) {
```

```
    throw new Error("useSidebar must be used within a SidebarProvider.");
  }

  return context;
}

const SidebarProvider = React.forwardRef<
  HTMLDivElement,
  React.ComponentProps<"div"> & {
    defaultOpen?: boolean;
    open?: boolean;
    onOpenChange?: (open: boolean) => void;
  }
>(
  (
    {
      defaultOpen = true,
      open: openProp,
      onOpenChange: setOpenProp,
      className,
      style,
      children,
      ...props
    },
    ref,
  ) => {
    const isMobile = useIsMobile();
    const [openMobile, setOpenMobile] = React.useState(false);

    // This is the internal state of the sidebar.
    // We use openProp and setOpenProp for control from outside the component.
    const [_open, _setOpen] = React.useState(defaultOpen);
    const open = openProp ?? _open;
    const setOpen = React.useCallback(
      (value: boolean | ((value: boolean) => boolean)) => {
        const openState = typeof value === "function" ? value(open) : value;
        if (setOpenProp) {
          setOpenProp(openState);
        } else {
          _setOpen(openState);
        }

        // This sets the cookie to keep the sidebar state.
        document.cookie = `${SIDEBAR_COOKIE_NAME}=${openState}; path=/;
max-age=${SIDEBAR_COOKIE_MAX_AGE}`;
```

```
    },
    [setOpenProp, open],
  );

  // Helper to toggle the sidebar.
  const toggleSidebar = React.useCallback(() => {
    return isMobile ? setOpenMobile((open) => !open) : setOpen((open) =>
!open);
  }, [isMobile, setOpen, setOpenMobile]);

  // Adds a keyboard shortcut to toggle the sidebar.
  React.useEffect(() => {
    const handleKeyDown = (event: KeyboardEvent) => {
      if (event.key === SIDEBAR_KEYBOARD_SHORTCUT && (event.metaKey
|| event.ctrlKey)) {
        event.preventDefault();
        toggleSidebar();
      }
    };

    window.addEventListener("keydown", handleKeyDown);
    return () => window.removeEventListener("keydown", handleKeyDown);
  }, [toggleSidebar]);

  // We add a state so that we can do data-state="expanded" or "collapsed".
  // This makes it easier to style the sidebar with Tailwind classes.
  const state = open ? "expanded" : "collapsed";

  const contextValue = React.useMemo<SidebarContextProps>(
    () => ({
      state,
      open,
      setOpen,
      isMobile,
      openMobile,
      setOpenMobile,
      toggleSidebar,
    }),
    [state, open, setOpen, isMobile, openMobile, setOpenMobile, toggleSidebar],
  );

  return (
    <SidebarContext.Provider value={contextValue}>
      <TooltipProvider delayDuration={0}>
        <div
```

```
            style={
              {
                "--sidebar-width": SIDEBAR_WIDTH,
                "--sidebar-width-icon": SIDEBAR_WIDTH_ICON,
                ...style,
              } as React.CSSProperties
            }
            className={cn(
              "group/sidebar-wrapper flex min-h-svh w-full
has-[[data-variant=inset]]:bg-sidebar",
              className,
            )}
            ref={ref}
            {...props}
          >
            {children}
          </div>
        </TooltipProvider>
      </SidebarContext.Provider>
    );
  },
);
SidebarProvider.displayName = "SidebarProvider";

const Sidebar = React.forwardRef<
  HTMLDivElement,
  React.ComponentProps<"div"> & {
    side?: "left" | "right";
    variant?: "sidebar" | "floating" | "inset";
    collapsible?: "offcanvas" | "icon" | "none";
  }
>(
  (
    {
      side = "left",
      variant = "sidebar",
      collapsible = "offcanvas",
      className,
      children,
      ...props
    },
    ref,
  ) => {
    const { isMobile, state, openMobile, setOpenMobile } = useSidebar();
```

```
    if (collapsible === "none") {
      return (
        <div
          className={cn(
            "flex h-full w-(--sidebar-width) flex-col bg-sidebar
text-sidebar-foreground",
            className,
          )}
          ref={ref}
          {...props}
        >
          {children}
        </div>
      );
    }

    if (isMobile) {
      return (
        <Sheet open={openMobile} onOpenChange={setOpenMobile} {...props}>
          <SheetContent
            data-sidebar="sidebar"
            data-mobile="true"
            className="w-(--sidebar-width) max-w-[calc(100vw-1rem)] bg-sidebar
p-0 text-sidebar-foreground"
            style={
              {
                "--sidebar-width": SIDEBAR_WIDTH_MOBILE,
              } as React.CSSProperties
            }
            side={side}
          >
            <SheetHeader className="sr-only">
              <SheetTitle>Sidebar</SheetTitle>
              <SheetDescription>Displays the mobile sidebar.</SheetDescription>
            </SheetHeader>
            <div className="flex h-full w-full flex-col">{children}</div>
          </SheetContent>
        </Sheet>
      );
    }

    return (
      <div
        ref={ref}
        className="group peer hidden text-sidebar-foreground md:block"
```

```
      data-state={state}
      data-collapsible={state === "collapsed" ? collapsible : ""}
      data-variant={variant}
      data-side={side}
    >
      {/* This is what handles the sidebar gap on desktop */}
      <div
        className={cn(
          "relative w-(--sidebar-width) bg-transparent transition-[width] duration-200
ease-linear",
          "group-data-[collapsible=offcanvas]:w-0",
          "group-data-[side=right]:rotate-180",
          variant === "floating" || variant === "inset"
            ? "group-data-[collapsible=icon]:w-[calc(var(--sidebar-width-icon)_+_the
me(spacing.4))]"
            : "group-data-[collapsible=icon]:w-(--sidebar-width-icon)",
        )}
      />
      <div
        data-slot="sidebar-container"
        data-side={side}
        data-variant={variant}
        data-collapsible={state === "collapsed" ? collapsible : ""}
        data-state={state}
        data-collapsible={state === "collapsed" ? collapsible : ""}
        data-variant={variant}
        data-side={side}
```

```
          </div>
        </div>
      </div>
    );
  },
);
Sidebar.displayName = "Sidebar";

const SidebarTrigger = React.forwardRef<
  React.ElementRef<typeof Button>,
  React.ComponentProps<typeof Button>
>(({ className, onClick, ...props }, ref) => {
  const { toggleSidebar } = useSidebar();

  return (
    <Button
      ref={ref}
      data-sidebar="trigger"
      variant="ghost"
      size="icon"
      className={cn("h-11 w-11 md:h-7 md:w-7", className)}
      onClick={(event) => {
        onClick?.(event);
        toggleSidebar();
      }}
      {...props}
    >
      <PanelLeft />
      <span className="sr-only">Toggle Sidebar</span>
    </Button>
  );
});
SidebarTrigger.displayName = "SidebarTrigger";

const SidebarRail = React.forwardRef<HTMLButtonElement,
React.ComponentProps<"button">>(
  ({ className, ...props }, ref) => {
    const { toggleSidebar } = useSidebar();

    return (
      <button
        ref={ref}
        data-sidebar="rail"
        aria-label="Toggle Sidebar"
        tabIndex={-1}
```

```
        onClick={toggleSidebar}
        title="Toggle Sidebar"
        className={cn(
          "absolute inset-y-0 z-20 hidden w-4 -translate-x-1/2 transition-all
ease-linear after:absolute after:inset-y-0 after:left-1/2 after:w-[2px]
hover:after:bg-sidebar-border group-data-[side=left]:-right-4
group-data-[side=right]:left-0 sm:flex",
          "[[data-side=left]_&]:cursor-w-resize [[data-side=right]_&]:cursor-e-resize",
          "[[data-side=left][data-state=collapsed]_&]:cursor-e-resize
[[data-side=right][data-state=collapsed]_&]:cursor-w-resize",
          "group-data-[collapsible=offcanvas]:translate-x-0
group-data-[collapsible=offcanvas]:after:left-full
group-data-[collapsible=offcanvas]:hover:bg-sidebar",
          "[[data-side=left][data-collapsible=offcanvas]_&]:-right-2",
          "[[data-side=right][data-collapsible=offcanvas]_&]:-left-2",
          className,
        )}
        {...props}
      />
    );
  },
);
SidebarRail.displayName = "SidebarRail";

const SidebarInset = React.forwardRef<HTMLDivElement,
React.ComponentProps<"main">>(
  ({ className, ...props }, ref) => {
    return (
      <main
        ref={ref}
        className={cn(
          "relative flex w-full flex-1 flex-col bg-background",
          "md:peer-data-[variant=inset]:m-2
md:peer-data-[state=collapsed]:peer-data-[variant=inset]:ml-2
md:peer-data-[variant=inset]:ml-0 md:peer-data-[variant=inset]:rounded-xl
md:peer-data-[variant=inset]:shadow",
          className,
        )}
        {...props}
      />
    );
  },
);
SidebarInset.displayName = "SidebarInset";
```

```
const SidebarInput = React.forwardRef<
  React.ElementRef<typeof Input>,
  React.ComponentProps<typeof Input>
>(({ className, ...props }, ref) => {
  return (
    <Input
      ref={ref}
      data-sidebar="input"
      className={cn(
        "h-8 w-full bg-background shadow-none focus-visible:ring-2
focus-visible:ring-sidebar-ring",
        className,
      )}
      {...props}
    />
  );
});
SidebarInput.displayName = "SidebarInput";

const SidebarHeader = React.forwardRef<HTMLDivElement,
React.ComponentProps<"div">>(
  ({ className, ...props }, ref) => {
    return (
      <div
        ref={ref}
        data-sidebar="header"
        className={cn("flex flex-col gap-2 p-2", className)}
        {...props}
      />
    );
  },
);
SidebarHeader.displayName = "SidebarHeader";

const SidebarFooter = React.forwardRef<HTMLDivElement,
React.ComponentProps<"div">>(
  ({ className, ...props }, ref) => {
    return (
      <div
        ref={ref}
        data-sidebar="footer"
        className={cn("flex flex-col gap-2 p-2", className)}
        {...props}
      />
    );
```

```
  },
);
SidebarFooter.displayName = "SidebarFooter";

const SidebarSeparator = React.forwardRef<
  React.ElementRef<typeof Separator>,
  React.ComponentProps<typeof Separator>
>(({ className, ...props }, ref) => {
  return (
    <Separator
      ref={ref}
      data-sidebar="separator"
      className={cn("mx-2 w-auto bg-sidebar-border", className)}
      {...props}
    />
  );
});
SidebarSeparator.displayName = "SidebarSeparator";

const SidebarContent = React.forwardRef<HTMLDivElement,
React.ComponentProps<"div">>(
  ({ className, ...props }, ref) => {
    return (
      <div
        ref={ref}
        data-sidebar="content"
        className={cn(
          "flex min-h-0 flex-1 flex-col gap-2 overflow-auto
group-data-[collapsible=icon]:overflow-hidden",
          className,
        )}
        {...props}
      />
    );
  },
);
SidebarContent.displayName = "SidebarContent";

const SidebarGroup = React.forwardRef<HTMLDivElement,
React.ComponentProps<"div">>(
  ({ className, ...props }, ref) => {
    return (
      <div
        ref={ref}
        data-sidebar="group"
```

```
        className={cn("relative flex w-full min-w-0 flex-col p-2", className)}
        {...props}
      />
    );
  },
);
SidebarGroup.displayName = "SidebarGroup";

const SidebarGroupLabel = React.forwardRef<
  HTMLDivElement,
  React.ComponentProps<"div"> & { asChild?: boolean }
>(({ className, asChild = false, ...props }, ref) => {
  const Comp = asChild ? Slot : "div";

  return (
    <Comp
      ref={ref}
      data-sidebar="group-label"
      className={cn(
        "flex h-8 shrink-0 items-center rounded-md px-2 text-xs font-medium
text-sidebar-foreground/70 outline-none ring-sidebar-ring
transition-[margin,opacity] duration-200 ease-linear focus-visible:ring-2
[&>svg]:size-4 [&>svg]:shrink-0",
        "group-data-[collapsible=icon]:-mt-8
group-data-[collapsible=icon]:opacity-0",
        className,
      )}
      {...props}
    />
  );
});
SidebarGroupLabel.displayName = "SidebarGroupLabel";

const SidebarGroupAction = React.forwardRef<
  HTMLButtonElement,
  React.ComponentProps<"button"> & { asChild?: boolean }
>(({ className, asChild = false, ...props }, ref) => {
  const Comp = asChild ? Slot : "button";

  return (
    <Comp
      ref={ref}
      data-sidebar="group-action"
      className={cn(
        "absolute right-3 top-3.5 flex aspect-square w-5 items-center justify-center
```

```
rounded-md p-0 text-sidebar-foreground outline-none ring-sidebar-ring
cursor-pointer transition-transform hover:bg-sidebar-accent
hover:text-sidebar-accent-foreground focus-visible:ring-2 [&>svg]:size-4
[&>svg]:shrink-0",
        // Increases the hit area of the button on mobile.
        "after:absolute after:-inset-2 after:md:hidden",
        "group-data-[collapsible=icon]:hidden",
        className,
      )}
      {...props}
    />
  );
});
SidebarGroupAction.displayName = "SidebarGroupAction";

const SidebarGroupContent = React.forwardRef<HTMLDivElement,
React.ComponentProps<"div">>(
  ({ className, ...props }, ref) => (
    <div
      ref={ref}
      data-sidebar="group-content"
      className={cn("w-full text-sm", className)}
      {...props}
    />
  ),
);
SidebarGroupContent.displayName = "SidebarGroupContent";

const SidebarMenu = React.forwardRef<HTMLUListElement,
React.ComponentProps<"ul">>(
  ({ className, ...props }, ref) => (
    <ul
      ref={ref}
      data-sidebar="menu"
      className={cn("flex w-full min-w-0 flex-col gap-1", className)}
      {...props}
    />
  ),
);
SidebarMenu.displayName = "SidebarMenu";

const SidebarMenuItem = React.forwardRef<HTMLLIElement,
React.ComponentProps<"li">>(
  ({ className, ...props }, ref) => (
    <li
```

```
      ref={ref}
      data-sidebar="menu-item"
      className={cn("group/menu-item relative", className)}
      {...props}
    />
  ),
);
SidebarMenuItem.displayName = "SidebarMenuItem";

const sidebarMenuButtonVariants = cva(
  "peer/menu-button flex w-full items-center gap-2 overflow-hidden rounded-md
p-2 text-left text-sm outline-none ring-sidebar-ring cursor-pointer
transition-[width,height,padding] hover:bg-sidebar-accent
hover:text-sidebar-accent-foreground focus-visible:ring-2
active:bg-sidebar-accent active:text-sidebar-accent-foreground
disabled:pointer-events-none disabled:opacity-50 disabled:cursor-not-allowed
group-has-[[data-sidebar=menu-action]]/menu-item:pr-8
aria-disabled:pointer-events-none aria-disabled:opacity-50
data-[active=true]:bg-sidebar-accent data-[active=true]:font-medium
data-[active=true]:text-sidebar-accent-foreground
data-[state=open]:hover:bg-sidebar-accent
data-[state=open]:hover:text-sidebar-accent-foreground
group-data-[collapsible=icon]:!size-8 group-data-[collapsible=icon]:!p-2
[&>span:last-child]:truncate [&>svg]:size-4 [&>svg]:shrink-0",
  {
    variants: {
      variant: {
        default: "hover:bg-sidebar-accent hover:text-sidebar-accent-foreground",
        outline:
          "bg-background shadow-[0_0_0_1px_var(--sidebar-border)]
hover:bg-sidebar-accent hover:text-sidebar-accent-foreground
hover:shadow-[0_0_0_1px_var(--sidebar-accent)]",
      },
      size: {
        default: "h-11 text-sm md:h-8",
        sm: "h-10 text-xs md:h-7",
        lg: "h-12 text-sm group-data-[collapsible=icon]:!p-0",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  },
);
```

```
const SidebarMenuButton = React.forwardRef<
  HTMLButtonElement,
  React.ComponentProps<"button"> & {
    asChild?: boolean;
    isActive?: boolean;
    tooltip?: string | React.ComponentProps<typeof TooltipContent>;
  } & VariantProps<typeof sidebarMenuButtonVariants>
>(
  (
    {
      asChild = false,
      isActive = false,
      variant = "default",
      size = "default",
      tooltip,
      className,
      onClick,
      ...props
    },
    ref,
  ) => {
    const Comp = asChild ? Slot : "button";
    const { isMobile, state, setOpenMobile } = useSidebar();

    const button = (
      <Comp
        ref={ref}
        data-sidebar="menu-button"
        data-size={size}
        data-active={isActive}
        className={cn(sidebarMenuButtonVariants({ variant, size }), className)}
        onClick={(event) => {
          onClick?.(event);
          if (isMobile) setOpenMobile(false);
        }}
        {...props}
      />
    );

    if (!tooltip) {
      return button;
    }

    if (typeof tooltip === "string") {
```

```
      tooltip = {
        children: tooltip,
      };
    }

    return (
      <Tooltip>
        <TooltipTrigger asChild>{button}</TooltipTrigger>
        <TooltipContent
          side="right"
          align="center"
          hidden={state !== "collapsed" || isMobile}
          {...tooltip}
        />
      </Tooltip>
    );
  },
);
SidebarMenuButton.displayName = "SidebarMenuButton";

const SidebarMenuAction = React.forwardRef<
  HTMLButtonElement,
  React.ComponentProps<"button"> & {
    asChild?: boolean;
    showOnHover?: boolean;
  }
>(({ className, asChild = false, showOnHover = false, ...props }, ref) => {
  const Comp = asChild ? Slot : "button";

  return (
    <Comp
      ref={ref}
      data-sidebar="menu-action"
      className={cn(
        "absolute right-1 top-1.5 flex aspect-square w-5 items-center justify-center
rounded-md p-0 text-sidebar-foreground outline-none ring-sidebar-ring
cursor-pointer transition-transform hover:bg-sidebar-accent
hover:text-sidebar-accent-foreground focus-visible:ring-2
peer-hover/menu-button:text-sidebar-accent-foreground [&>svg]:size-4
[&>svg]:shrink-0",
        // Increases the hit area of the button on mobile.
        "after:absolute after:-inset-2 after:md:hidden",
        "peer-data-[size=sm]/menu-button:top-1",
        "peer-data-[size=default]/menu-button:top-1.5",
        "peer-data-[size=lg]/menu-button:top-2.5",
```

```
        "group-data-[collapsible=icon]:hidden",
        showOnHover &&
          "group-focus-within/menu-item:opacity-100
group-hover/menu-item:opacity-100 data-[state=open]:opacity-100
peer-data-[active=true]/menu-button:text-sidebar-accent-foreground
md:opacity-0",
        className,
      )}
      {...props}
    />
  );
});
SidebarMenuAction.displayName = "SidebarMenuAction";

const SidebarMenuBadge = React.forwardRef<HTMLDivElement,
React.ComponentProps<"div">>(
  ({ className, ...props }, ref) => (
    <div
      ref={ref}
      data-sidebar="menu-badge"
      className={cn(
        "pointer-events-none absolute right-1 flex h-5 min-w-5 select-none
items-center justify-center rounded-md px-1 text-xs font-medium tabular-nums
text-sidebar-foreground",
        "peer-hover/menu-button:text-sidebar-accent-foreground
peer-data-[active=true]/menu-button:text-sidebar-accent-foreground",
        "peer-data-[size=sm]/menu-button:top-1",
        "peer-data-[size=default]/menu-button:top-1.5",
        "peer-data-[size=lg]/menu-button:top-2.5",
        "group-data-[collapsible=icon]:hidden",
        className,
      )}
      {...props}
    />
  ),
);
SidebarMenuBadge.displayName = "SidebarMenuBadge";

const SidebarMenuSkeleton = React.forwardRef<
  HTMLDivElement,
  React.ComponentProps<"div"> & {
    showIcon?: boolean;
  }
>(({ className, showIcon = false, ...props }, ref) => {
  // Random width between 50 to 90%.
```

```
  const width = React.useMemo(() => {
    return `${Math.floor(Math.random() * 40) + 50}%`;
  }, []);

  return (
    <div
      ref={ref}
      data-sidebar="menu-skeleton"
      className={cn("flex h-8 items-center gap-2 rounded-md px-2", className)}
      {...props}
    >
      {showIcon && <Skeleton className="size-4 rounded-md"
data-sidebar="menu-skeleton-icon" />}
      <Skeleton
        className="h-4 max-w-(--skeleton-width) flex-1"
        data-sidebar="menu-skeleton-text"
        style={
          {
            "--skeleton-width": width,
          } as React.CSSProperties
        }
      />
    </div>
  );
});
SidebarMenuSkeleton.displayName = "SidebarMenuSkeleton";

const SidebarMenuSub = React.forwardRef<HTMLUListElement,
React.ComponentProps<"ul">>(
  ({ className, ...props }, ref) => (
    <ul
      ref={ref}
      data-sidebar="menu-sub"
      className={cn(
        "mx-3.5 flex min-w-0 translate-x-px flex-col gap-1 border-l
border-sidebar-border px-2.5 py-0.5",
        "group-data-[collapsible=icon]:hidden",
        className,
      )}
      {...props}
    />
  ),
);
SidebarMenuSub.displayName = "SidebarMenuSub";
```

```
const SidebarMenuSubItem = React.forwardRef<HTMLLIElement,
React.ComponentProps<"li">>(
  ({ ...props }, ref) => <li ref={ref} {...props} />,
);
SidebarMenuSubItem.displayName = "SidebarMenuSubItem";

const SidebarMenuSubButton = React.forwardRef<
  HTMLAnchorElement,
  React.ComponentProps<"a"> & {
    asChild?: boolean;
    size?: "sm" | "md";
    isActive?: boolean;
  }
>(({ asChild = false, size = "md", isActive, className, ...props }, ref) => {
  const Comp = asChild ? Slot : "a";

  return (
    <Comp
      ref={ref}
      data-sidebar="menu-sub-button"
      data-size={size}
      data-active={isActive}
      className={cn(
        "flex h-7 min-w-0 -translate-x-px items-center gap-2 overflow-hidden
rounded-md px-2 text-sidebar-foreground outline-none ring-sidebar-ring
cursor-pointer hover:bg-sidebar-accent hover:text-sidebar-accent-foreground
focus-visible:ring-2 active:bg-sidebar-accent
active:text-sidebar-accent-foreground disabled:pointer-events-none
disabled:opacity-50 disabled:cursor-not-allowed
aria-disabled:pointer-events-none aria-disabled:opacity-50
[&>span:last-child]:truncate [&>svg]:size-4 [&>svg]:shrink-0
[&>svg]:text-sidebar-accent-foreground",
        "data-[active=true]:bg-sidebar-accent
data-[active=true]:text-sidebar-accent-foreground",
        size === "sm" && "text-xs",
        size === "md" && "text-sm",
        "group-data-[collapsible=icon]:hidden",
        className,
      )}
      {...props}
    />
  );
});
SidebarMenuSubButton.displayName = "SidebarMenuSubButton";
```

```
export {
  Sidebar,
  SidebarContent,
  SidebarFooter,
  SidebarGroup,
  SidebarGroupAction,
  SidebarGroupContent,
  SidebarGroupLabel,
  SidebarHeader,
  SidebarInput,
  SidebarInset,
  SidebarMenu,
  SidebarMenuAction,
  SidebarMenuBadge,
  SidebarMenuButton,
  SidebarMenuItem,
  SidebarMenuSkeleton,
  SidebarMenuSub,
  SidebarMenuSubButton,
  SidebarMenuSubItem,
  SidebarProvider,
  SidebarRail,
  SidebarSeparator,
  SidebarTrigger,
  useSidebar,
};
```

src/components/ui/skeleton.tsx

```
import { cn } from "@/lib/utils";

function Skeleton({ className, ...props }:
React.HTMLAttributes<HTMLDivElement>) {
  return <div className={cn("animate-pulse rounded-md bg-primary/10",
className)} {...props} />;
}

export { Skeleton };
```

src/components/ui/switch.tsx

```
import * as React from "react";
import * as SwitchPrimitives from "@radix-ui/react-switch";

import { cn } from "@/lib/utils";

const Switch = React.forwardRef<
  React.ElementRef<typeof SwitchPrimitives.Root>,
  React.ComponentPropsWithoutRef<typeof SwitchPrimitives.Root>
>(({ className, ...props }, ref) => (
  <SwitchPrimitives.Root
    className={cn(
      "peer inline-flex h-6 w-11 shrink-0 cursor-pointer items-center rounded-full
border-2 border-transparent shadow-sm transition-colors
focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2 focus-visible:ring-offset-background
disabled:cursor-not-allowed disabled:opacity-50 data-[state=checked]:bg-primary
data-[state=unchecked]:bg-input md:h-5 md:w-9",
      className,
    )}
    {...props}
    ref={ref}
  >
    <SwitchPrimitives.Thumb
      className={cn(
        "pointer-events-none block h-5 w-5 rounded-full bg-background shadow-lg
ring-0 transition-transform data-[state=checked]:translate-x-5
data-[state=unchecked]:translate-x-0 md:h-4 md:w-4
md:data-[state=checked]:translate-x-4",
      )}
    />
  </SwitchPrimitives.Root>
));
Switch.displayName = SwitchPrimitives.Root.displayName;

export { Switch };
```

src/components/ui/table.tsx

```
import * as React from "react";

import { cn } from "@/lib/utils";

const Table = React.forwardRef<HTMLTableElement,
React.HTMLAttributes<HTMLTableElement>>(
  ({ className, ...props }, ref) => (
    <div className="relative w-full overflow-auto">
      <table ref={ref} className={cn("w-full caption-bottom text-sm", className)}
{...props} />
    </div>
  ),
);
Table.displayName = "Table";

const TableHeader = React.forwardRef<
  HTMLTableSectionElement,
  React.HTMLAttributes<HTMLTableSectionElement>
>(({ className, ...props }, ref) => (
  <thead ref={ref} className={cn("[&_tr]:border-b", className)} {...props} />
));
TableHeader.displayName = "TableHeader";

const TableBody = React.forwardRef<
  HTMLTableSectionElement,
  React.HTMLAttributes<HTMLTableSectionElement>
>(({ className, ...props }, ref) => (
  <tbody ref={ref} className={cn("[&_tr:last-child]:border-0", className)}
{...props} />
));
TableBody.displayName = "TableBody";

const TableFooter = React.forwardRef<
  HTMLTableSectionElement,
  React.HTMLAttributes<HTMLTableSectionElement>
>(({ className, ...props }, ref) => (
  <tfoot
    ref={ref}
    className={cn("border-t bg-muted/50 font-medium [&>tr]:last:border-b-0",
className)}
    {...props}
  />
));
```

```
TableFooter.displayName = "TableFooter";

const TableRow = React.forwardRef<HTMLTableRowElement,
React.HTMLAttributes<HTMLTableRowElement>>(
  ({ className, ...props }, ref) => (
    <tr
      ref={ref}
      className={cn(
        "border-b transition-colors hover:bg-muted/50
data-[state=selected]:bg-muted",
        className,
      )}
      {...props}
    />
  ),
);
TableRow.displayName = "TableRow";

const TableHead = React.forwardRef<
  HTMLTableCellElement,
  React.ThHTMLAttributes<HTMLTableCellElement>
>(({ className, ...props }, ref) => (
  <th
    ref={ref}
    className={cn(
      "h-10 px-2 text-left align-middle font-medium text-muted-foreground
[&:has([role=checkbox])]:pr-0 [&>[role=checkbox]]:translate-y-[2px]",
      className,
    )}
    {...props}
  />
));
TableHead.displayName = "TableHead";

const TableCell = React.forwardRef<
  HTMLTableCellElement,
  React.TdHTMLAttributes<HTMLTableCellElement>
>(({ className, ...props }, ref) => (
  <td
    ref={ref}
    className={cn(
      "p-2 align-middle [&:has([role=checkbox])]:pr-0
[&>[role=checkbox]]:translate-y-[2px]",
      className,
    )}
```

```
    {...props}
  />
));
TableCell.displayName = "TableCell";

const TableCaption = React.forwardRef<
  HTMLTableCaptionElement,
  React.HTMLAttributes<HTMLTableCaptionElement>
>(({ className, ...props }, ref) => (
  <caption ref={ref} className={cn("mt-4 text-sm text-muted-foreground",
className)} {...props} />
));
TableCaption.displayName = "TableCaption";

export { Table, TableHeader, TableBody, TableFooter, TableHead, TableRow,
TableCell, TableCaption };
```

src/components/ui/tabs.tsx

```
import * as React from "react";
import * as TabsPrimitive from "@radix-ui/react-tabs";

import { cn } from "@/lib/utils";

const Tabs = TabsPrimitive.Root;

const TabsList = React.forwardRef<
  React.ElementRef<typeof TabsPrimitive.List>,
  React.ComponentPropsWithoutRef<typeof TabsPrimitive.List>
>(({ className, ...props }, ref) => (
  <TabsPrimitive.List
    ref={ref}
    // The list grows with its triggers (each at least 36px tall for touch) and
    // wraps onto another row when narrow. A fixed height shorter than the
    // triggers, combined with overflow, used to show a scrollbar inside the tabs.
    className={cn(
      "inline-flex h-auto max-w-full flex-wrap items-center justify-center gap-y-1
rounded-lg bg-muted p-1 text-muted-foreground",
      className,
    )}
    {...props}
  />
));
TabsList.displayName = TabsPrimitive.List.displayName;

const TabsTrigger = React.forwardRef<
  React.ElementRef<typeof TabsPrimitive.Trigger>,
  React.ComponentPropsWithoutRef<typeof TabsPrimitive.Trigger>
>(({ className, ...props }, ref) => (
  <TabsPrimitive.Trigger
    ref={ref}
    className={cn(
      "inline-flex min-h-9 items-center justify-center whitespace-nowrap
rounded-md px-3 py-1 text-sm font-medium ring-offset-background cursor-pointer
transition-all focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2 disabled:pointer-events-none disabled:opacity-50
disabled:cursor-not-allowed data-[state=active]:bg-background
data-[state=active]:text-foreground data-[state=active]:shadow",
      className,
    )}
    {...props}
  />
```

```
));
TabsTrigger.displayName = TabsPrimitive.Trigger.displayName;

const TabsContent = React.forwardRef<
  React.ElementRef<typeof TabsPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof TabsPrimitive.Content>
>(({ className, ...props }, ref) => (
  <TabsPrimitive.Content
    ref={ref}
    className={cn(
      "mt-2 ring-offset-background focus-visible:outline-none focus-visible:ring-2
focus-visible:ring-ring focus-visible:ring-offset-2",
      className,
    )}
    {...props}
  />
));
TabsContent.displayName = TabsPrimitive.Content.displayName;

export { Tabs, TabsList, TabsTrigger, TabsContent };
```

src/components/ui/textarea.tsx

```
import * as React from "react";

import { cn } from "@/lib/utils";

const Textarea = React.forwardRef<HTMLTextAreaElement,
React.ComponentProps<"textarea">>(
  ({ className, ...props }, ref) => {
    return (
      <textarea
        className={cn(
          "flex min-h-[88px] w-full rounded-md border border-input bg-transparent
px-3 py-2 text-base shadow-sm placeholder:text-muted-foreground
focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring
aria-invalid:border-destructive aria-invalid:ring-1 aria-invalid:ring-destructive/30
disabled:cursor-not-allowed disabled:opacity-50 md:min-h-[60px] md:text-sm",
          className,
        )}
        ref={ref}
        {...props}
      />
    );
  },
);
Textarea.displayName = "Textarea";

export { Textarea };
```

src/components/ui/toggle-group.tsx

```
"use client";

import * as React from "react";
import * as ToggleGroupPrimitive from "@radix-ui/react-toggle-group";
import { type VariantProps } from "class-variance-authority";

import { cn } from "@/lib/utils";
import { toggleVariants } from "@/components/ui/toggle";

const ToggleGroupContext = React.createContext<VariantProps<typeof
toggleVariants>>({
  size: "default",
  variant: "default",
});

const ToggleGroup = React.forwardRef<
  React.ElementRef<typeof ToggleGroupPrimitive.Root>,
  React.ComponentPropsWithoutRef<typeof ToggleGroupPrimitive.Root> &
    VariantProps<typeof toggleVariants>
>(({ className, variant, size, children, ...props }, ref) => (
  <ToggleGroupPrimitive.Root
    ref={ref}
    className={cn("flex items-center justify-center gap-1", className)}
    {...props}
  >
    <ToggleGroupContext.Provider value={{ variant, size
}}>{children}</ToggleGroupContext.Provider>
  </ToggleGroupPrimitive.Root>
));

ToggleGroup.displayName = ToggleGroupPrimitive.Root.displayName;

const ToggleGroupItem = React.forwardRef<
  React.ElementRef<typeof ToggleGroupPrimitive.Item>,
  React.ComponentPropsWithoutRef<typeof ToggleGroupPrimitive.Item> &
    VariantProps<typeof toggleVariants>
>(({ className, children, variant, size, ...props }, ref) => {
  const context = React.useContext(ToggleGroupContext);

  return (
    <ToggleGroupPrimitive.Item
      ref={ref}
      className={cn(
```

```
        toggleVariants({
          variant: context.variant || variant,
          size: context.size || size,
        }),
        className,
      )}
      {...props}
    >
      {children}
    </ToggleGroupPrimitive.Item>
  );
});

ToggleGroupItem.displayName = ToggleGroupPrimitive.Item.displayName;

export { ToggleGroup, ToggleGroupItem };
```

src/components/ui/toggle.tsx

```
import * as React from "react";
import * as TogglePrimitive from "@radix-ui/react-toggle";
import { cva, type VariantProps } from "class-variance-authority";

import { cn } from "@/lib/utils";

const toggleVariants = cva(
  "inline-flex items-center justify-center gap-2 rounded-md text-sm font-medium
cursor-pointer transition-colors hover:bg-muted hover:text-muted-foreground
focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring
disabled:pointer-events-none disabled:opacity-50 disabled:cursor-not-allowed
data-[state=on]:bg-accent data-[state=on]:text-accent-foreground
[&_svg]:pointer-events-none [&_svg]:size-4 [&_svg]:shrink-0",
  {
    variants: {
      variant: {
        default: "bg-transparent",
        outline:
          "border border-input bg-transparent shadow-sm hover:bg-accent
hover:text-accent-foreground",
      },
      size: {
        default: "h-9 px-2 min-w-9",
        sm: "h-8 px-1.5 min-w-8",
        lg: "h-10 px-2.5 min-w-10",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  },
);

const Toggle = React.forwardRef<
  React.ElementRef<typeof TogglePrimitive.Root>,
  React.ComponentPropsWithoutRef<typeof TogglePrimitive.Root> &
VariantProps<typeof toggleVariants>
>(({ className, variant, size, ...props }, ref) => (
  <TogglePrimitive.Root
    ref={ref}
    className={cn(toggleVariants({ variant, size, className }))}
    {...props}
```

```
  />
));

Toggle.displayName = TogglePrimitive.Root.displayName;

export { Toggle, toggleVariants };
```

src/components/ui/tooltip.tsx

```
"use client";

import * as React from "react";
import * as TooltipPrimitive from "@radix-ui/react-tooltip";

import { cn } from "@/lib/utils";

const TooltipProvider = TooltipPrimitive.Provider;

const Tooltip = TooltipPrimitive.Root;

const TooltipTrigger = TooltipPrimitive.Trigger;

const TooltipContent = React.forwardRef<
  React.ElementRef<typeof TooltipPrimitive.Content>,
  React.ComponentPropsWithoutRef<typeof TooltipPrimitive.Content>
>(({ className, sideOffset = 4, ...props }, ref) => (
  <TooltipPrimitive.Portal>
    <TooltipPrimitive.Content
      ref={ref}
      sideOffset={sideOffset}
      className={cn(
        "z-50 overflow-hidden rounded-md bg-primary px-3 py-1.5 text-xs
text-primary-foreground animate-in fade-in-0 zoom-in-95
data-[state=closed]:animate-out data-[state=closed]:fade-out-0
data-[state=closed]:zoom-out-95 data-[side=bottom]:slide-in-from-top-2
data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
data-[side=top]:slide-in-from-bottom-2
origin-(--radix-tooltip-content-transform-origin)",
        className,
      )}
      {...props}
    />
  </TooltipPrimitive.Portal>
));
TooltipContent.displayName = TooltipPrimitive.Content.displayName;

export { Tooltip, TooltipTrigger, TooltipContent, TooltipProvider };
```

src/hooks/use-mobile.tsx

```
import * as React from "react";

const MOBILE_BREAKPOINT = 768;

export function useIsMobile() {
  const [isMobile, setIsMobile] = React.useState(
    typeof window !== "undefined" ? window.innerWidth < MOBILE_BREAKPOINT
: false,
  );

  React.useEffect(() => {
    const mql = window.matchMedia(`(max-width: ${MOBILE_BREAKPOINT -
1}px)`);
    const onChange = () => {
      setIsMobile(window.innerWidth < MOBILE_BREAKPOINT);
    };
    mql.addEventListener("change", onChange);
    setIsMobile(window.innerWidth < MOBILE_BREAKPOINT);
    return () => mql.removeEventListener("change", onChange);
  }, []);

  return !!isMobile;
}
```

src/integrations/supabase/auth-attacher.ts

```
// This file is automatically generated. Do not edit it directly.
import { createMiddleware } from "@tanstack/react-start";
import { supabase } from "./client";

// Must be registered as a global `functionMiddleware` in `src/start.ts`; otherwise
// the browser never attaches the bearer token to serverFn RPCs.
export const attachSupabaseAuth = createMiddleware({ type: "function" }).client(
  async ({ next }) => {
    const { data } = await supabase.auth.getSession();
    const token = data.session?.access_token;
    return next({
      headers: token ? { Authorization: `Bearer ${token}` } : {},
    });
  },
);
```

src/integrations/supabase/client.server.ts

```
// This file is automatically generated. Do not edit it directly.
// Server-side Supabase client with service role key - bypasses RLS.
// Use this for admin operations in server functions and server routes only.
// For user-authenticated queries (with RLS), use the auth middleware instead.
import { createClient } from "@supabase/supabase-js";
import type { Database } from "./types";

function isNewSupabaseApiKey(value: string): boolean {
  return value.startsWith("sb_publishable_") || value.startsWith("sb_secret_");
}

function createSupabaseFetch(supabaseKey: string): typeof fetch {
  return (input, init) => {
    const headers = new Headers(
      typeof Request !== "undefined" && input instanceof Request ? input.headers
: undefined,
    );

    if (init?.headers) {
      new Headers(init.headers).forEach((value, key) => headers.set(key, value));
    }

    // New Supabase API keys are opaque strings, not bearer JWTs.
    if (
      isNewSupabaseApiKey(supabaseKey) &&
      headers.get("Authorization") === `Bearer ${supabaseKey}`
    ) {
      headers.delete("Authorization");
    }

    headers.set("apikey", supabaseKey);
    return fetch(input, { ...init, headers });
  };
}

function projectHost(url: string | undefined) {
  if (!url) return "not set";
  try {
    return new URL(url.trim()).host;
  } catch {
    return "not a valid URL";
  }
}
```

```
/** The Supabase project server code talks to, for logs. Never includes a key. */
export function supabaseServerProject() {
  return projectHost(process.env["SUPABASE_URL"]);
}

function createSupabaseAdminClient() {
  const SUPABASE_URL = process.env["SUPABASE_URL"];
  const SUPABASE_SERVICE_ROLE_KEY =
process.env["SUPABASE_SERVICE_ROLE_KEY"];

  // The browser uses VITE_SUPABASE_URL and server code uses
SUPABASE_URL. If
  // they point to different projects, public pages still load while server
  // features (availability, bookings, lookups) query the other project and fail.
  const browserUrl = process.env["VITE_SUPABASE_URL"];
  if (SUPABASE_URL && browserUrl && projectHost(browserUrl) !==
projectHost(SUPABASE_URL)) {
    console.error(
      `[Supabase] SUPABASE_URL (${projectHost(SUPABASE_URL)}) and
VITE_SUPABASE_URL (${projectHost(browserUrl)}) point to different projects.
Set both to the same project.`,
    );
  }

  if (!SUPABASE_URL || !SUPABASE_SERVICE_ROLE_KEY) {
    const missing = [
      ...(!SUPABASE_URL ? ["SUPABASE_URL"] : []),
      ...(!SUPABASE_SERVICE_ROLE_KEY ?
["SUPABASE_SERVICE_ROLE_KEY"] : []),
    ];
    const message = `Missing Supabase environment variable(s): ${missing.join(",
")}. Ensure SUPABASE_URL and SUPABASE_SERVICE_ROLE_KEY are set in
your .env file.`;
    console.error(`[Supabase] ${message}`);
    throw new Error(message);
  }

  return createClient<Database>(SUPABASE_URL,
SUPABASE_SERVICE_ROLE_KEY, {
    global: {
      fetch: createSupabaseFetch(SUPABASE_SERVICE_ROLE_KEY),
    },
    auth: {
      storage: undefined,
```

```
      persistSession: false,
      autoRefreshToken: false,
    },
  });
}

let _supabaseAdmin: ReturnType<typeof createSupabaseAdminClient> |
undefined;

// Server-side Supabase client with service role - bypasses RLS
// SECURITY: Only use this for trusted server-side operations, never expose to
client code
// Load inside server handlers: const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
// Top-level import is safe only in other .server.ts modules - route files and
*.functions.ts ship to the client bundle.
export const supabaseAdmin = new Proxy({} as ReturnType<typeof
createSupabaseAdminClient>, {
  get(_, prop, receiver) {
    if (!_supabaseAdmin) _supabaseAdmin = createSupabaseAdminClient();
    return Reflect.get(_supabaseAdmin, prop, receiver);
  },
});
```

src/integrations/supabase/client.ts

```
// This file is automatically generated. Do not edit it directly.
import { createClient } from "@supabase/supabase-js";
import type { Database } from "./types";

function isNewSupabaseApiKey(value: string): boolean {
  return value.startsWith("sb_publishable_") || value.startsWith("sb_secret_");
}

function createSupabaseFetch(supabaseKey: string): typeof fetch {
  return (input, init) => {
    const headers = new Headers(
      typeof Request !== "undefined" && input instanceof Request ? input.headers
: undefined,
    );

    if (init?.headers) {
      new Headers(init.headers).forEach((value, key) => headers.set(key, value));
    }

    // New Supabase API keys are opaque strings, not bearer JWTs.
    if (
      isNewSupabaseApiKey(supabaseKey) &&
      headers.get("Authorization") === `Bearer ${supabaseKey}`
    ) {
      headers.delete("Authorization");
    }

    headers.set("apikey", supabaseKey);
    return fetch(input, { ...init, headers });
  };
}

function createSupabaseClient() {
  // Use import.meta.env for client-side (Vite build-time replacement)
  // Fall back to process.env for SSR (server-side rendering)
  // The offline copy's browser goes through the app server's /supabase proxy
  // (vite.config.ts), so phones on the laptop's hotspot reach the local database.
  const SUPABASE_URL =
    import.meta.env["VITE_OFFLINE"] === "1" && typeof window !== "undefined"
      ? `${window.location.origin}/supabase`
      : import.meta.env["VITE_SUPABASE_URL"] ||
process.env["SUPABASE_URL"];
  const SUPABASE_PUBLISHABLE_KEY =
```

```
    import.meta.env["VITE_SUPABASE_PUBLISHABLE_KEY"] ||
process.env["SUPABASE_PUBLISHABLE_KEY"];

  if (!SUPABASE_URL || !SUPABASE_PUBLISHABLE_KEY) {
    const missing = [
      ...(!SUPABASE_URL ? ["SUPABASE_URL"] : []),
      ...(!SUPABASE_PUBLISHABLE_KEY ?
["SUPABASE_PUBLISHABLE_KEY"] : []),
    ];
    const message = `Missing Supabase environment variable(s): ${missing.join(",
")}. Ensure SUPABASE_URL and SUPABASE_PUBLISHABLE_KEY are set in
your .env file.`;
    console.error(`[Supabase] ${message}`);
    throw new Error(message);
  }

  return createClient<Database>(SUPABASE_URL,
SUPABASE_PUBLISHABLE_KEY, {
    global: {
      fetch: createSupabaseFetch(SUPABASE_PUBLISHABLE_KEY),
    },
    auth: {
      // Admin authentication is intentionally scoped to one browser tab.
      // sessionStorage survives normal reloads and same-tab navigation, but is
      // cleared by the browser when that tab/window closes. This prevents a
      // closed admin tab's token from authorizing the admin area when reopened.
      storage: typeof window !== "undefined" ? sessionStorage : undefined,
      persistSession: true,
      autoRefreshToken: true,
    },
  });
}

let _supabase: ReturnType<typeof createSupabaseClient> | undefined;

// Import the supabase client like this:
// import { supabase } from "@/integrations/supabase/client";
export const supabase = new Proxy({} as ReturnType<typeof
createSupabaseClient>, {
  get(_, prop, receiver) {
    if (!_supabase) _supabase = createSupabaseClient();
    return Reflect.get(_supabase, prop, receiver);
  },
});
```

src/integrations/supabase/types.ts

```
export type Json = string | number | boolean | null | { [key: string]: Json |
undefined } | Json[];

export type Database = {
  // Allows to automatically instantiate createClient with right options
  // instead of createClient<Database, { PostgrestVersion: 'XX' }>(URL, KEY)
  __InternalSupabase: {
    PostgrestVersion: "14.15";
  };
  graphql_public: {
    Tables: {
      [_ in never]: never;
    };
    Views: {
      [_ in never]: never;
    };
    Functions: {
      graphql: {
        Args: {
          extensions?: Json;
          operationName?: string;
          query?: string;
          variables?: Json;
        };
        Returns: Json;
      };
    };
    Enums: {
      [_ in never]: never;
    };
    CompositeTypes: {
      [_ in never]: never;
    };
  };
  public: {
    Tables: {
      admin_activity_logs: {
        Row: {
          action: string;
          activity_date: string;
          activity_time: string;
          actor_email: string | null;
          actor_id: string | null;
```

```
    changed_fields: string[];
    created_at: string;
    id: string;
    ip_address: string | null;
    record_id: string | null;
    resource_type: string;
    summary: string;
    target_label: string;
  };
  Insert: {
    action: string;
    activity_date?: string;
    activity_time?: string;
    actor_email?: string | null;
    actor_id?: string | null;
    changed_fields?: string[];
    created_at?: string;
    id?: string;
    ip_address?: string | null;
    record_id?: string | null;
    resource_type: string;
    summary: string;
    target_label: string;
  };
  Update: {
    action?: string;
    activity_date?: string;
    activity_time?: string;
    actor_email?: string | null;
    actor_id?: string | null;
    changed_fields?: string[];
    created_at?: string;
    id?: string;
    ip_address?: string | null;
    record_id?: string | null;
    resource_type?: string;
    summary?: string;
    target_label?: string;
  };
  Relationships: [];
};
appointment_services: {
  Row: {
    appointment_id: string;
    duration_minutes: number;
```

```
      id: string;
      price: number;
      service_id: string | null;
      service_name: string;
    };
    Insert: {
      appointment_id: string;
      duration_minutes?: number;
      id?: string;
      price?: number;
      service_id?: string | null;
      service_name: string;
    };
    Update: {
      appointment_id?: string;
      duration_minutes?: number;
      id?: string;
      price?: number;
      service_id?: string | null;
      service_name?: string;
    };
    Relationships: [
      {
        foreignKeyName: "appointment_services_appointment_id_fkey";
        columns: ["appointment_id"];
        isOneToOne: false;
        referencedRelation: "appointments";
        referencedColumns: ["id"];
      },
      {
        foreignKeyName: "appointment_services_service_id_fkey";
        columns: ["service_id"];
        isOneToOne: false;
        referencedRelation: "services";
        referencedColumns: ["id"];
      },
    ];
  };
  appointment_products: {
    Row: {
      appointment_id: string;
      created_at: string;
      id: string;
      product_id: string;
      product_name: string;
```

```
      quantity: number;
      unit_price: number;
      updated_at: string;
    };
    Insert: {
      appointment_id: string;
      created_at?: string;
      id?: string;
      product_id: string;
      product_name: string;
      quantity?: number;
      unit_price?: number;
      updated_at?: string;
    };
    Update: {
      appointment_id?: string;
      created_at?: string;
      id?: string;
      product_id?: string;
      product_name?: string;
      quantity?: number;
      unit_price?: number;
      updated_at?: string;
    };
    Relationships: [
      {
        foreignKeyName: "appointment_products_appointment_id_fkey";
        columns: ["appointment_id"];
        isOneToOne: false;
        referencedRelation: "appointments";
        referencedColumns: ["id"];
      },
      {
        foreignKeyName: "appointment_products_product_id_fkey";
        columns: ["product_id"];
        isOneToOne: false;
        referencedRelation: "products";
        referencedColumns: ["id"];
      },
    ];
  };
  appointment_continuations: {
    Row: {
      appointment_date: string;
      appointment_id: string;
```

```
    assigned_crew_id: string;
    booking_duration_minutes: number;
    created_at: string;
    id: string;
    segment_number: number;
    start_time: string;
  };
  Insert: {
    appointment_date: string;
    appointment_id: string;
    assigned_crew_id: string;
    booking_duration_minutes: number;
    created_at?: string;
    id?: string;
    segment_number: number;
    start_time: string;
  };
  Update: {
    appointment_date?: string;
    appointment_id?: string;
    assigned_crew_id?: string;
    booking_duration_minutes?: number;
    created_at?: string;
    id?: string;
    segment_number?: number;
    start_time?: string;
  };
  Relationships: [
    {
      foreignKeyName: "appointment_continuations_appointment_id_fkey";
      columns: ["appointment_id"];
      isOneToOne: false;
      referencedRelation: "appointments";
      referencedColumns: ["id"];
    },
    {
      foreignKeyName: "appointment_continuations_assigned_crew_id_fkey";
      columns: ["assigned_crew_id"];
      isOneToOne: false;
      referencedRelation: "crew_members";
      referencedColumns: ["id"];
    },
  ];
};
appointments: {
```

```
Row: {
  admin_notes: string | null;
  appointment_date: string;
  assigned_crew_id: string | null;
  booking_request_id: string | null;
  cancelled_at: string | null;
  archived_at: string | null;
  arrival_notification_snooze_count: number;
  arrival_notification_snoozed_until: string | null;
  created_at: string;
  booking_duration_minutes: number;
  completion_notification_snoozed_until: string | null;
  customer_name: string;
  email: string | null;
  first_name: string | null;
  id: string;
  is_archived: boolean;
  last_reschedule_rejected_at: string | null;
  last_reschedule_rejection_message: string | null;
  last_name: string | null;
  middle_name: string | null;
  moto_brand: string;
  moto_model: string;
  moto_variant: string | null;
  moto_year: number | null;
  notes: string | null;
  no_show_count_at_booking: number;
  phone: string;
  plate_number: string;
  pending_reschedule_date: string | null;
  pending_reschedule_reference_code: string | null;
  pending_reschedule_reason: string | null;
  pending_reschedule_request_id: string | null;
  pending_reschedule_start_time: string | null;
  reference_code: string;
  reschedule_count: number;
  rescheduled_from_appointment_id: string | null;
  rescheduled_to_appointment_id: string | null;
  service_ended_at: string | null;
  service_started_at: string | null;
  start_time: string;
  status: string;
  terms_accepted: boolean;
  total_estimate: number;
  updated_at: string;
```

```
  wants_products: boolean;
};
Insert: {
  admin_notes?: string | null;
  appointment_date: string;
  assigned_crew_id?: string | null;
  booking_request_id?: string | null;
  cancelled_at?: string | null;
  archived_at?: string | null;
  arrival_notification_snooze_count?: number;
  arrival_notification_snoozed_until?: string | null;
  created_at?: string;
  booking_duration_minutes?: number;
  completion_notification_snoozed_until?: string | null;
  customer_name: string;
  email?: string | null;
  first_name?: string | null;
  id?: string;
  is_archived?: boolean;
  last_reschedule_rejected_at?: string | null;
  last_reschedule_rejection_message?: string | null;
  last_name?: string | null;
  middle_name?: string | null;
  moto_brand: string;
  moto_model: string;
  moto_variant?: string | null;
  moto_year?: number | null;
  notes?: string | null;
  no_show_count_at_booking?: number;
  phone: string;
  plate_number: string;
  pending_reschedule_date?: string | null;
  pending_reschedule_reference_code?: string | null;
  pending_reschedule_reason?: string | null;
  pending_reschedule_request_id?: string | null;
  pending_reschedule_start_time?: string | null;
  reference_code: string;
  reschedule_count?: number;
  rescheduled_from_appointment_id?: string | null;
  rescheduled_to_appointment_id?: string | null;
  service_ended_at?: string | null;
  service_started_at?: string | null;
  start_time: string;
  status?: string;
  terms_accepted?: boolean;
```

```
  total_estimate?: number;
  updated_at?: string;
  wants_products?: boolean;
};
Update: {
  admin_notes?: string | null;
  appointment_date?: string;
  assigned_crew_id?: string | null;
  booking_request_id?: string | null;
  cancelled_at?: string | null;
  archived_at?: string | null;
  arrival_notification_snooze_count?: number;
  arrival_notification_snoozed_until?: string | null;
  created_at?: string;
  booking_duration_minutes?: number;
  completion_notification_snoozed_until?: string | null;
  customer_name?: string;
  email?: string | null;
  first_name?: string | null;
  id?: string;
  is_archived?: boolean;
  last_reschedule_rejected_at?: string | null;
  last_reschedule_rejection_message?: string | null;
  last_name?: string | null;
  middle_name?: string | null;
  moto_brand?: string;
  moto_model?: string;
  moto_variant?: string | null;
  moto_year?: number | null;
  notes?: string | null;
  no_show_count_at_booking?: number;
  phone?: string;
  plate_number?: string;
  pending_reschedule_date?: string | null;
  pending_reschedule_reference_code?: string | null;
  pending_reschedule_reason?: string | null;
  pending_reschedule_request_id?: string | null;
  pending_reschedule_start_time?: string | null;
  reference_code?: string;
  reschedule_count?: number;
  rescheduled_from_appointment_id?: string | null;
  rescheduled_to_appointment_id?: string | null;
  service_ended_at?: string | null;
  service_started_at?: string | null;
  start_time?: string;
```

```
      status?: string;
      terms_accepted?: boolean;
      total_estimate?: number;
      updated_at?: string;
      wants_products?: boolean;
    };
    Relationships: [
      {
        foreignKeyName: "appointments_assigned_crew_id_fkey";
        columns: ["assigned_crew_id"];
        isOneToOne: false;
        referencedRelation: "crew_members";
        referencedColumns: ["id"];
      },
    ];
  };
  appointment_confirmation_emails: {
    Row: {
      appointment_id: string;
      attempt_count: number;
      created_at: string;
      id: string;
      last_error: string | null;
      recipient_email: string;
      sent_at: string | null;
      status: string;
      updated_at: string;
    };
    Insert: {
      appointment_id: string;
      attempt_count?: number;
      created_at?: string;
      id?: string;
      last_error?: string | null;
      recipient_email: string;
      sent_at?: string | null;
      status?: string;
      updated_at?: string;
    };
    Update: {
      appointment_id?: string;
      attempt_count?: number;
      created_at?: string;
      id?: string;
      last_error?: string | null;
```

```
          recipient_email?: string;
          sent_at?: string | null;
          status?: string;
          updated_at?: string;
        };
        Relationships: [
          {
            foreignKeyName:
"appointment_confirmation_emails_appointment_id_fkey";
            columns: ["appointment_id"];
            isOneToOne: false;
            referencedRelation: "appointments";
            referencedColumns: ["id"];
          },
        ];
      };
      appointment_confirmation_sms: {
        Row: {
          appointment_id: string;
          attempt_count: number;
          created_at: string;
          id: string;
          last_error: string | null;
          recipient_phone: string;
          sent_at: string | null;
          status: string;
          updated_at: string;
        };
        Insert: {
          appointment_id: string;
          attempt_count?: number;
          created_at?: string;
          id?: string;
          last_error?: string | null;
          recipient_phone: string;
          sent_at?: string | null;
          status?: string;
          updated_at?: string;
        };
        Update: {
          appointment_id?: string;
          attempt_count?: number;
          created_at?: string;
          id?: string;
          last_error?: string | null;
```

```
          recipient_phone?: string;
          sent_at?: string | null;
          status?: string;
          updated_at?: string;
        };
        Relationships: [
          {
            foreignKeyName:
"appointment_confirmation_sms_appointment_id_fkey";
            columns: ["appointment_id"];
            isOneToOne: false;
            referencedRelation: "appointments";
            referencedColumns: ["id"];
          },
        ];
      };
      appointment_completion_notifications: {
        Row: {
          appointment_id: string;
          attempt_count: number;
          channel: string;
          created_at: string;
          id: string;
          last_error: string | null;
          recipient: string;
          sent_at: string | null;
          status: string;
          updated_at: string;
        };
        Insert: {
          appointment_id: string;
          attempt_count?: number;
          channel: string;
          created_at?: string;
          id?: string;
          last_error?: string | null;
          recipient: string;
          sent_at?: string | null;
          status?: string;
          updated_at?: string;
        };
        Update: {
          appointment_id?: string;
          attempt_count?: number;
          channel?: string;
```

```
          created_at?: string;
          id?: string;
          last_error?: string | null;
          recipient?: string;
          sent_at?: string | null;
          status?: string;
          updated_at?: string;
        };
        Relationships: [
          {
            foreignKeyName:
"appointment_completion_notifications_appointment_id_fkey";
            columns: ["appointment_id"];
            isOneToOne: false;
            referencedRelation: "appointments";
            referencedColumns: ["id"];
          },
        ];
      };
      appointment_reminders: {
        Row: {
          appointment_id: string;
          attempt_count: number;
          channel: string;
          created_at: string;
          id: string;
          last_error: string | null;
          recipient: string;
          sent_at: string | null;
          status: string;
          updated_at: string;
        };
        Insert: {
          appointment_id: string;
          attempt_count?: number;
          channel: string;
          created_at?: string;
          id?: string;
          last_error?: string | null;
          recipient: string;
          sent_at?: string | null;
          status?: string;
          updated_at?: string;
        };
        Update: {
```

```
      appointment_id?: string;
      attempt_count?: number;
      channel?: string;
      created_at?: string;
      id?: string;
      last_error?: string | null;
      recipient?: string;
      sent_at?: string | null;
      status?: string;
      updated_at?: string;
    };
    Relationships: [
      {
        foreignKeyName: "appointment_reminders_appointment_id_fkey";
        columns: ["appointment_id"];
        isOneToOne: false;
        referencedRelation: "appointments";
        referencedColumns: ["id"];
      },
    ];
  };
  appointment_action_verifications: {
    Row: {
      action: string;
      appointment_id: string;
      attempt_count: number;
      code_hash: string;
      created_at: string;
      expires_at: string;
      id: string;
      token_expires_at: string | null;
      token_hash: string | null;
      verified_at: string | null;
    };
    Insert: {
      action: string;
      appointment_id: string;
      attempt_count?: number;
      code_hash: string;
      created_at?: string;
      expires_at: string;
      id?: string;
      token_expires_at?: string | null;
      token_hash?: string | null;
      verified_at?: string | null;
```

```
        };
        Update: {
          action?: string;
          appointment_id?: string;
          attempt_count?: number;
          code_hash?: string;
          created_at?: string;
          expires_at?: string;
          id?: string;
          token_expires_at?: string | null;
          token_hash?: string | null;
          verified_at?: string | null;
        };
        Relationships: [
          {
            foreignKeyName:
"appointment_action_verifications_appointment_id_fkey";
            columns: ["appointment_id"];
            isOneToOne: false;
            referencedRelation: "appointments";
            referencedColumns: ["id"];
          },
        ];
      };
      appointment_reference_recovery_challenges: {
        Row: {
          attempt_count: number;
          code_hash: string;
          created_at: string;
          email: string;
          expires_at: string;
          id: string;
          verified_at: string | null;
        };
        Insert: {
          attempt_count?: number;
          code_hash: string;
          created_at?: string;
          email: string;
          expires_at: string;
          id?: string;
          verified_at?: string | null;
        };
        Update: {
          attempt_count?: number;
```

```
      code_hash?: string;
      created_at?: string;
      email?: string;
      expires_at?: string;
      id?: string;
      verified_at?: string | null;
    };
    Relationships: [];
  };
  appointment_reschedule_history: {
    Row: {
      id: string;
      appointment_id: string;
      new_appointment_id: string | null;
      reschedule_number: number;
      from_date: string;
      from_start_time: string;
      to_date: string | null;
      to_start_time: string | null;
      reason: string;
      decision: string;
      approved_at: string;
      approved_by: string | null;
    };
    Insert: {
      id?: string;
      appointment_id: string;
      new_appointment_id?: string | null;
      reschedule_number: number;
      from_date: string;
      from_start_time: string;
      to_date?: string | null;
      to_start_time?: string | null;
      reason: string;
      decision: string;
      approved_at?: string;
      approved_by?: string | null;
    };
    Update: {
      id?: string;
      appointment_id?: string;
      new_appointment_id?: string | null;
      reschedule_number?: number;
      from_date?: string;
      from_start_time?: string;
```

```
          to_date?: string | null;
          to_start_time?: string | null;
          reason?: string;
          decision?: string;
          approved_at?: string;
          approved_by?: string | null;
        };
        Relationships: [
          {
            foreignKeyName:
"appointment_reschedule_history_appointment_id_fkey";
            columns: ["appointment_id"];
            isOneToOne: false;
            referencedRelation: "appointments";
            referencedColumns: ["id"];
          },
          {
            foreignKeyName:
"appointment_reschedule_history_new_appointment_id_fkey";
            columns: ["new_appointment_id"];
            isOneToOne: false;
            referencedRelation: "appointments";
            referencedColumns: ["id"];
          },
        ];
      };
      blocked_numbers: {
        Row: {
          archived_at: string | null;
          id: string;
          phone: string;
          reason: string | null;
          created_at: string;
          created_by: string | null;
          is_archived: boolean;
        };
        Insert: {
          archived_at?: string | null;
          id?: string;
          phone: string;
          reason?: string | null;
          created_at?: string;
          created_by?: string | null;
          is_archived?: boolean;
        };
```

```
      Update: {
        archived_at?: string | null;
        id?: string;
        phone?: string;
        reason?: string | null;
        created_at?: string;
        created_by?: string | null;
        is_archived?: boolean;
      };
      Relationships: [];
    };
    crew_availability_exceptions: {
      Row: {
        created_at: string;
        crew_id: string;
        end_date: string;
        end_time: string | null;
        id: string;
        is_all_day: boolean;
        note: string | null;
        reason: string;
        start_date: string;
        start_time: string | null;
      };
      Insert: {
        created_at?: string;
        crew_id: string;
        end_date: string;
        end_time?: string | null;
        id?: string;
        is_all_day?: boolean;
        note?: string | null;
        reason: string;
        start_date: string;
        start_time?: string | null;
      };
      Update: {
        created_at?: string;
        crew_id?: string;
        end_date?: string;
        end_time?: string | null;
        id?: string;
        is_all_day?: boolean;
        note?: string | null;
        reason?: string;
```

```
      start_date?: string;
      start_time?: string | null;
    };
    Relationships: [
      {
        foreignKeyName: "crew_availability_exceptions_crew_id_fkey";
        columns: ["crew_id"];
        isOneToOne: false;
        referencedRelation: "crew_members";
        referencedColumns: ["id"];
      },
    ];
  };
  crew_members: {
    Row: {
      archived_at: string | null;
      created_at: string;
      id: string;
      is_active: boolean;
      is_archived: boolean;
      name: string;
      phone: string | null;
      role: string;
    };
    Insert: {
      archived_at?: string | null;
      created_at?: string;
      id?: string;
      is_active?: boolean;
      is_archived?: boolean;
      name: string;
      phone?: string | null;
      role?: string;
    };
    Update: {
      archived_at?: string | null;
      created_at?: string;
      id?: string;
      is_active?: boolean;
      is_archived?: boolean;
      name?: string;
      phone?: string | null;
      role?: string;
    };
    Relationships: [];
```

```
};
crew_schedules: {
  Row: {
    created_at: string;
    crew_id: string;
    day_of_week: number;
    effective_from: string | null;
    effective_to: string | null;
    end_time: string | null;
    id: string;
    is_working: boolean;
    note: string | null;
    schedule_date: string | null;
    start_time: string | null;
  };
  Insert: {
    created_at?: string;
    crew_id: string;
    day_of_week: number;
    effective_from?: string | null;
    effective_to?: string | null;
    end_time?: string | null;
    id?: string;
    is_working?: boolean;
    note?: string | null;
    schedule_date?: string | null;
    start_time?: string | null;
  };
  Update: {
    created_at?: string;
    crew_id?: string;
    day_of_week?: number;
    effective_from?: string | null;
    effective_to?: string | null;
    end_time?: string | null;
    id?: string;
    is_working?: boolean;
    note?: string | null;
    schedule_date?: string | null;
    start_time?: string | null;
  };
  Relationships: [
    {
      foreignKeyName: "crew_schedules_crew_id_fkey";
      columns: ["crew_id"];
```

```
      isOneToOne: false;
      referencedRelation: "crew_members";
      referencedColumns: ["id"];
    },
  ];
};
notifications: {
  Row: {
    appointment_id: string | null;
    created_at: string;
    id: string;
    is_read: boolean;
    message: string | null;
    title: string;
    type: string;
  };
  Insert: {
    appointment_id?: string | null;
    created_at?: string;
    id?: string;
    is_read?: boolean;
    message?: string | null;
    title: string;
    type?: string;
  };
  Update: {
    appointment_id?: string | null;
    created_at?: string;
    id?: string;
    is_read?: boolean;
    message?: string | null;
    title?: string;
    type?: string;
  };
  Relationships: [
    {
      foreignKeyName: "notifications_appointment_id_fkey";
      columns: ["appointment_id"];
      isOneToOne: false;
      referencedRelation: "appointments";
      referencedColumns: ["id"];
    },
  ];
};
price_history: {
```

```
      Row: {
        changed_by: string | null;
        changed_by_email: string | null;
        configuration_id: string | null;
        configuration_label: string | null;
        created_at: string;
        id: string;
        item_id: string;
        item_name: string;
        new_price: number;
        old_price: number;
        reason: string | null;
        table_name: string;
      };
      Insert: {
        changed_by?: string | null;
        changed_by_email?: string | null;
        configuration_id?: string | null;
        configuration_label?: string | null;
        created_at?: string;
        id?: string;
        item_id: string;
        item_name: string;
        new_price: number;
        old_price: number;
        reason?: string | null;
        table_name: string;
      };
      Update: {
        changed_by?: string | null;
        changed_by_email?: string | null;
        configuration_id?: string | null;
        configuration_label?: string | null;
        created_at?: string;
        id?: string;
        item_id?: string;
        item_name?: string;
        new_price?: number;
        old_price?: number;
        reason?: string | null;
        table_name?: string;
      };
      Relationships: [];
    };
    products: {
```

```
      Row: {
        archived_at: string | null;
        brand: string | null;
        category: string;
        created_at: string;
        description: string | null;
        id: string;
        image_url: string | null;
        in_stock: boolean;
        is_active: boolean;
        is_archived: boolean;
        is_featured: boolean;
        name: string;
        price: number;
        sort_order: number;
        stock_quantity: number;
        updated_at: string;
      };
      Insert: {
        archived_at?: string | null;
        brand?: string | null;
        category?: string;
        created_at?: string;
        description?: string | null;
        id?: string;
        image_url?: string | null;
        in_stock?: boolean;
        is_active?: boolean;
        is_archived?: boolean;
        is_featured?: boolean;
        name: string;
        price?: number;
        sort_order?: number;
        stock_quantity?: number;
        updated_at?: string;
      };
      Update: {
        archived_at?: string | null;
        brand?: string | null;
        category?: string;
        created_at?: string;
        description?: string | null;
        id?: string;
        image_url?: string | null;
        in_stock?: boolean;
```

```
      is_active?: boolean;
      is_archived?: boolean;
      is_featured?: boolean;
      name?: string;
      price?: number;
      sort_order?: number;
      stock_quantity?: number;
      updated_at?: string;
    };
    Relationships: [];
  };
  motorcycle_catalog: {
    Row: {
      archived_at: string | null;
      brand: string | null;
      cc_category: string | null;
      created_at: string;
      engine_cc: number | null;
      id: string;
      is_active: boolean;
      is_archived: boolean;
      model: string;
      sort_order: number;
      updated_at: string;
    };
    Insert: {
      archived_at?: string | null;
      brand?: string | null;
      created_at?: string;
      engine_cc?: number | null;
      id?: string;
      is_active?: boolean;
      is_archived?: boolean;
      model: string;
      sort_order?: number;
      updated_at?: string;
    };
    Update: {
      archived_at?: string | null;
      brand?: string | null;
      created_at?: string;
      engine_cc?: number | null;
      id?: string;
      is_active?: boolean;
      is_archived?: boolean;
```

```
      model?: string;
      sort_order?: number;
      updated_at?: string;
    };
    Relationships: [];
  };
  scheduled_notification_runs: {
    Row: {
      id: boolean;
      last_error: string | null;
      last_finished_at: string | null;
      last_started_at: string | null;
    };
    Insert: {
      id?: boolean;
      last_error?: string | null;
      last_finished_at?: string | null;
      last_started_at?: string | null;
    };
    Update: {
      id?: boolean;
      last_error?: string | null;
      last_finished_at?: string | null;
      last_started_at?: string | null;
    };
    Relationships: [];
  };
  schedule_blocks: {
    Row: {
      archived_at: string | null;
      block_date: string;
      created_at: string;
      id: string;
      is_active: boolean;
      reason: string | null;
      start_time: string | null;
    };
    Insert: {
      archived_at?: string | null;
      block_date: string;
      created_at?: string;
      id?: string;
      is_active?: boolean;
      reason?: string | null;
      start_time?: string | null;
```

```
    };
    Update: {
      archived_at?: string | null;
      block_date?: string;
      created_at?: string;
      id?: string;
      is_active?: boolean;
      reason?: string | null;
      start_time?: string | null;
    };
    Relationships: [];
  };
  shop_settings: {
    Row: {
      address: string;
      allow_same_day_appointments: boolean;
      booking_terms: string;
      cancellation_notice_hours: number;
      contact_email: string;
      contact_number: string;
      default_appointment_duration_minutes: number;
      id: boolean;
      logo_url: string | null;
      max_advance_booking_days: number;
      minimum_booking_lead_hours: number;
      opening_time: string;
      closing_time: string;
      notify_admin: boolean;
      notify_cancellation: boolean;
      notify_confirmation: boolean;
      notify_reminder: boolean;
      reminder_hours_before: number;
      rescheduling_notice_hours: number;
      shop_name: string;
      updated_at: string;
    };
    Insert: {
      address?: string;
      allow_same_day_appointments?: boolean;
      booking_terms?: string;
      cancellation_notice_hours?: number;
      contact_email?: string;
      contact_number?: string;
      default_appointment_duration_minutes?: number;
      id?: boolean;
```

```
      logo_url?: string | null;
      max_advance_booking_days?: number;
      minimum_booking_lead_hours?: number;
      opening_time?: string;
      closing_time?: string;
      notify_admin?: boolean;
      notify_cancellation?: boolean;
      notify_confirmation?: boolean;
      notify_reminder?: boolean;
      reminder_hours_before?: number;
      rescheduling_notice_hours?: number;
      shop_name?: string;
      updated_at?: string;
    };
    Update: {
      address?: string;
      allow_same_day_appointments?: boolean;
      booking_terms?: string;
      cancellation_notice_hours?: number;
      contact_email?: string;
      contact_number?: string;
      default_appointment_duration_minutes?: number;
      id?: boolean;
      logo_url?: string | null;
      max_advance_booking_days?: number;
      minimum_booking_lead_hours?: number;
      opening_time?: string;
      closing_time?: string;
      notify_admin?: boolean;
      notify_cancellation?: boolean;
      notify_confirmation?: boolean;
      notify_reminder?: boolean;
      reminder_hours_before?: number;
      rescheduling_notice_hours?: number;
      shop_name?: string;
      updated_at?: string;
    };
    Relationships: [];
  };
  services: {
    Row: {
      archived_at: string | null;
      category: string;
      created_at: string;
      description: string | null;
```

```
      duration_minutes: number;
      id: string;
      image_url: string | null;
      is_active: boolean;
      is_archived: boolean;
      large_bike_duration_minutes: number | null;
      large_bike_price: number | null;
      name: string;
      price: number;
      sort_order: number;
      updated_at: string;
    };
    Insert: {
      archived_at?: string | null;
      category?: string;
      created_at?: string;
      description?: string | null;
      duration_minutes?: number;
      id?: string;
      image_url?: string | null;
      is_active?: boolean;
      is_archived?: boolean;
      large_bike_duration_minutes?: number | null;
      large_bike_price?: number | null;
      name: string;
      price?: number;
      sort_order?: number;
      updated_at?: string;
    };
    Update: {
      archived_at?: string | null;
      category?: string;
      created_at?: string;
      description?: string | null;
      duration_minutes?: number;
      id?: string;
      image_url?: string | null;
      is_active?: boolean;
      is_archived?: boolean;
      large_bike_duration_minutes?: number | null;
      large_bike_price?: number | null;
      name?: string;
      price?: number;
      sort_order?: number;
      updated_at?: string;
```

```
      };
      Relationships: [];
    };
    time_slots: {
      Row: {
        capacity: number;
        created_at: string;
        end_time: string;
        id: string;
        is_active: boolean;
        start_time: string;
      };
      Insert: {
        capacity?: number;
        created_at?: string;
        end_time: string;
        id?: string;
        is_active?: boolean;
        start_time: string;
      };
      Update: {
        capacity?: number;
        created_at?: string;
        end_time?: string;
        id?: string;
        is_active?: boolean;
        start_time?: string;
      };
      Relationships: [];
    };
    user_roles: {
      Row: {
        created_at: string;
        id: string;
        role: Database["public"]["Enums"]["app_role"];
        user_id: string;
      };
      Insert: {
        created_at?: string;
        id?: string;
        role: Database["public"]["Enums"]["app_role"];
        user_id: string;
      };
      Update: {
        created_at?: string;
```

```
      id?: string;
      role?: Database["public"]["Enums"]["app_role"];
      user_id?: string;
    };
    Relationships: [];
  };
};
Views: {
  [_ in never]: never;
};
Functions: {
  cancel_public_appointment: {
    Args: {
      p_appointment_id: string;
      p_cancellation_notice_hours: number;
    };
    Returns: undefined;
  };
  claim_admin_session: {
    Args: {
      p_handover_token?: string | null;
    };
    Returns: boolean;
  };
  create_admin_session_handover: {
    Args: Record<PropertyKey, never>;
    Returns: string;
  };
  create_booking_atomic: {
    Args: {
      p_appointment_date: string;
      p_assigned_crew_id: string | null;
      p_booking_duration_minutes: number;
      p_booking_request_id: string;
      p_continuation_segments: Json;
      p_customer_name: string;
      p_email: string;
      p_first_name: string;
      p_last_name: string;
      p_middle_name: string;
      p_moto_brand: string;
      p_moto_model: string;
      p_moto_variant: string | null;
      p_moto_year: number;
      p_notification_message: string;
```

```
      p_notification_title: string;
      p_notes: string | null;
      p_phone: string;
      p_plate_number: string;
      p_reference_code: string;
      p_services: Json;
      p_start_time: string;
      p_total_estimate: number;
      p_wants_products: boolean;
    };
    Returns: {
      appointment_id: string;
      reference_code: string;
    }[];
  };
  queue_due_appointment_reminders: {
    Args: Record<PropertyKey, never>;
    Returns: number;
  };
  review_pending_appointment: {
    Args: {
      p_appointment_id: string;
      p_decision: string;
    };
    Returns: undefined;
  };
  review_reschedule_request: {
    Args: {
      p_appointment_id: string;
      p_decision: string;
    };
    Returns: undefined;
  };
  release_admin_session: {
    Args: Record<PropertyKey, never>;
    Returns: boolean;
  };
  renew_admin_session: {
    Args: Record<PropertyKey, never>;
    Returns: boolean;
  };
  submit_reschedule_request: {
    Args: {
      p_appointment_date: string;
      p_appointment_id: string;
```

```
      p_reason: string;
      p_request_id: string;
      p_start_time: string;
    };
    Returns: undefined;
  };
  validate_admin_session: {
    Args: Record<PropertyKey, never>;
    Returns: boolean;
  };
  update_appointment_details_atomic: {
    Args: {
      p_admin_notes: string | null;
      p_appointment_date: string;
      p_appointment_id: string;
      p_assigned_crew_id: string | null;
      p_crew_assignment_manual: boolean;
      p_first_name: string;
      p_last_name: string;
      p_middle_name: string;
      p_phone: string;
      p_service_ids: string[];
      p_start_time: string;
      p_status: string;
    };
    Returns: undefined;
  };
  update_appointment_financial_details_atomic: {
    Args: {
      p_admin_notes: string | null;
      p_appointment_date: string;
      p_appointment_id: string;
      p_assigned_crew_id: string | null;
      p_crew_assignment_manual: boolean;
      p_first_name: string;
      p_last_name: string;
      p_middle_name: string;
      p_phone: string;
      p_products: Json;
      p_service_ids: string[];
      p_start_time: string;
      p_status: string;
    };
    Returns: undefined;
  };
```

```
enforce_public_rate_limit: {
  Args: {
    p_limit: number;
    p_scope: string;
    p_subject: string;
    p_window_seconds: number;
  };
  Returns: boolean;
};
consume_reference_recovery_challenge: {
  Args: {
    p_code_hash: string;
    p_email: string;
  };
  Returns: string;
};
verify_appointment_action_code: {
  Args: {
    p_action: string;
    p_appointment_id: string;
    p_code_hash: string;
    p_token_hash: string;
    p_token_ttl_seconds: number;
  };
  Returns: string;
};
has_role: {
  Args: {
    _role: Database["public"]["Enums"]["app_role"];
    _user_id: string;
  };
  Returns: boolean;
};
manage_appointment_service_progress: {
  Args: {
    p_action: string;
    p_appointment_id: string;
  };
  Returns: undefined;
};
get_admin_customers_page: {
  Args: {
    p_limit?: number;
    p_offset?: number;
    p_search?: string | null;
```

```
  };
  Returns: Json;
};
get_admin_report_page: {
  Args: {
    p_category?: string | null;
    p_from: string;
    p_limit?: number;
    p_offset?: number;
    p_report_kind: string;
    p_service_name?: string | null;
    p_status?: string | null;
    p_to: string;
  };
  Returns: Json;
};
get_completed_financial_report_page: {
  Args: {
    p_category?: string | null;
    p_from: string;
    p_limit?: number;
    p_offset?: number;
    p_service_names?: string[] | null;
    p_to: string;
  };
  Returns: Json;
};
get_admin_dashboard_summary: {
  Args: {
    p_from: string;
    p_to: string;
    p_previous_from: string;
    p_previous_to: string;
  };
  Returns: Json;
};
get_admin_dashboard_chart: {
  Args: {
    p_from: string;
    p_to: string;
    p_by_month?: boolean;
  };
  Returns: Json;
};
change_crew_schedule: {
```

```
        Args: {
          p_schedule_id: string;
          p_remove: boolean;
          p_start_time?: string | null;
          p_end_time?: string | null;
        };
        Returns: Json;
      };
      purge_expired_admin_activity_logs: {
        Args: Record<PropertyKey, never>;
        Returns: undefined;
      };
      record_admin_activity_event: {
        Args: {
          p_action: string;
          p_changed_fields?: string[];
          p_resource_type: string;
          p_summary: string;
          p_target_label: string;
        };
        Returns: string;
      };
      sync_service_progress_notifications: {
        Args: Record<PropertyKey, never>;
        Returns: number;
      };
    };
    Enums: {
      app_role: "admin";
    };
    CompositeTypes: {
      [_ in never]: never;
    };
  };
};

type DatabaseWithoutInternals = Omit<Database, "__InternalSupabase">;

type DefaultSchema = DatabaseWithoutInternals[Extract<keyof Database,
"public">];

export type Tables<
  DefaultSchemaTableNameOrOptions extends
    | keyof (DefaultSchema["Tables"] & DefaultSchema["Views"])
    | { schema: keyof DatabaseWithoutInternals },
```

```
  TableName extends (DefaultSchemaTableNameOrOptions extends {
    schema: keyof DatabaseWithoutInternals;
  }
    ? keyof (DatabaseWithoutInternals[DefaultSchemaTableNameOrOptions["sch
ema"]]["Tables"] &
        DatabaseWithoutInternals[DefaultSchemaTableNameOrOptions["schema"]][
"Views"])
    : never) = never,
> = DefaultSchemaTableNameOrOptions extends {
  schema: keyof DatabaseWithoutInternals;
}
  ? (DatabaseWithoutInternals[DefaultSchemaTableNameOrOptions["schema"]]["
Tables"] &
      DatabaseWithoutInternals[DefaultSchemaTableNameOrOptions["schema"]]["
Views"])[TableName] extends {
      Row: infer R;
    }
    ? R
    : never
  : DefaultSchemaTableNameOrOptions extends keyof (DefaultSchema["Tables"]
& DefaultSchema["Views"])
    ? (DefaultSchema["Tables"] &
DefaultSchema["Views"])[DefaultSchemaTableNameOrOptions] extends {
        Row: infer R;
      }
      ? R
      : never
    : never;

export type TablesInsert<
  DefaultSchemaTableNameOrOptions extends
    keyof DefaultSchema["Tables"] | { schema: keyof DatabaseWithoutInternals },
  TableName extends (DefaultSchemaTableNameOrOptions extends {
    schema: keyof DatabaseWithoutInternals;
  }
    ? keyof DatabaseWithoutInternals[DefaultSchemaTableNameOrOptions["sche
ma"]]["Tables"]
    : never) = never,
> = DefaultSchemaTableNameOrOptions extends {
  schema: keyof DatabaseWithoutInternals;
}
  ? DatabaseWithoutInternals[DefaultSchemaTableNameOrOptions["schema"]]["
Tables"][TableName] extends {
      Insert: infer I;
    }
```

```
    ? I
    : never
  : DefaultSchemaTableNameOrOptions extends keyof DefaultSchema["Tables"]
    ? DefaultSchema["Tables"][DefaultSchemaTableNameOrOptions] extends {
        Insert: infer I;
      }
      ? I
      : never
    : never;

export type TablesUpdate<
  DefaultSchemaTableNameOrOptions extends
    keyof DefaultSchema["Tables"] | { schema: keyof DatabaseWithoutInternals },
  TableName extends (DefaultSchemaTableNameOrOptions extends {
    schema: keyof DatabaseWithoutInternals;
  }
    ? keyof DatabaseWithoutInternals[DefaultSchemaTableNameOrOptions["sche
ma"]]["Tables"]
    : never) = never,
> = DefaultSchemaTableNameOrOptions extends {
  schema: keyof DatabaseWithoutInternals;
}
  ? DatabaseWithoutInternals[DefaultSchemaTableNameOrOptions["schema"]]["
Tables"][TableName] extends {
      Update: infer U;
    }
    ? U
    : never
  : DefaultSchemaTableNameOrOptions extends keyof DefaultSchema["Tables"]
    ? DefaultSchema["Tables"][DefaultSchemaTableNameOrOptions] extends {
        Update: infer U;
      }
      ? U
      : never
    : never;

export type Enums<
  DefaultSchemaEnumNameOrOptions extends
    keyof DefaultSchema["Enums"] | { schema: keyof DatabaseWithoutInternals },
  EnumName extends (DefaultSchemaEnumNameOrOptions extends {
    schema: keyof DatabaseWithoutInternals;
  }
    ? keyof DatabaseWithoutInternals[DefaultSchemaEnumNameOrOptions["sche
ma"]]["Enums"]
    : never) = never,
```

```
> = DefaultSchemaEnumNameOrOptions extends {
  schema: keyof DatabaseWithoutInternals;
}
  ? DatabaseWithoutInternals[DefaultSchemaEnumNameOrOptions["schema"]]["
Enums"][EnumName]
  : DefaultSchemaEnumNameOrOptions extends keyof DefaultSchema["Enums"]
    ? DefaultSchema["Enums"][DefaultSchemaEnumNameOrOptions]
    : never;

export type CompositeTypes<
  PublicCompositeTypeNameOrOptions extends
    keyof DefaultSchema["CompositeTypes"] | { schema: keyof
DatabaseWithoutInternals },
  CompositeTypeName extends (PublicCompositeTypeNameOrOptions extends {
    schema: keyof DatabaseWithoutInternals;
  }
    ? keyof DatabaseWithoutInternals[PublicCompositeTypeNameOrOptions["sch
ema"]]["CompositeTypes"]
    : never) = never,
> = PublicCompositeTypeNameOrOptions extends {
  schema: keyof DatabaseWithoutInternals;
}
  ? DatabaseWithoutInternals[PublicCompositeTypeNameOrOptions["schema"]]["
CompositeTypes"][CompositeTypeName]
  : PublicCompositeTypeNameOrOptions extends keyof
DefaultSchema["CompositeTypes"]
    ? DefaultSchema["CompositeTypes"][PublicCompositeTypeNameOrOptions]
    : never;

export const Constants = {
  graphql_public: {
    Enums: {},
  },
  public: {
    Enums: {
      app_role: ["admin"],
    },
  },
} as const;
```

src/lib/admin-activity.ts

```
import { supabase } from "@/integrations/supabase/client";

type AdminActivityEvent = {
  action: string;
  resourceType: string;
  targetLabel: string;
  summary: string;
  changedFields?: string[];
};

/**
 * Records non-mutating, security-relevant admin actions. Database triggers
 * record create/update/archive/delete actions separately.
 */
export async function recordAdminActivityEvent(event: AdminActivityEvent) {
  const { error } = await supabase.rpc("record_admin_activity_event", {
    p_action: event.action,
    p_resource_type: event.resourceType,
    p_target_label: event.targetLabel,
    p_summary: event.summary,
    p_changed_fields: event.changedFields ?? [],
  });

  if (error) throw error;
}
```

src/lib/admin-alerts.server.ts

```
/**
 * Alerts for problems a person needs to act on, shown in the admin's
 * notifications like new bookings are.
 */

export type AdminAlert = {
  /** The notification type, e.g. "customer_notice_failed". */
  type: string;
  title: string;
  message: string;
  appointmentId?: string | null;
  /** Skip the alert while an unread one of the same type is this recent. */
  onceWithinMinutes?: number;
};

export async function alertAdmin(alert: AdminAlert) {
  try {
    const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
    if (alert.onceWithinMinutes) {
      const since = new Date(Date.now() - alert.onceWithinMinutes *
60_000).toISOString();
      const { data: recent, error } = await supabaseAdmin
        .from("notifications")
        .select("id")
        .eq("type", alert.type)
        .eq("is_read", false)
        .gte("created_at", since)
        .limit(1);
      if (error) throw error;
      if (recent.length) return;
    }
    const { error } = await supabaseAdmin.from("notifications").insert({
      type: alert.type,
      title: alert.title.slice(0, 200),
      message: alert.message.slice(0, 1000),
      appointment_id: alert.appointmentId ?? null,
    });
    if (error) throw error;
  } catch (error) {
    console.error("[Admin alert] could not be saved", { type: alert.type, error });
  }
}
```

```
/** Remove email addresses and phone numbers before an error is shown or
stored. */
export function redactPersonalDetails(value: string) {
  return value
    .replace(/[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}/gi, "[email]")
    .replace(/\+?\d[\d\s()-]{7,}\d/g, "[phone]")
    .replace(/ /g, " ")
    .trim();
}
```

src/lib/admin-date-range.ts

```
import type { DateRange } from "react-day-picker";

import { addDays, manilaNow } from "@/lib/shop";

export type IsoDateRange = { from: string; to: string };

/**
 * Calendar selections are calendar days, not instants. Read their local date
 * fields directly so an administrator's browser timezone cannot shift a
 * selected day when it is sent to the database.
 */
export function calendarDateToIso(date: Date) {
  const year = date.getFullYear();
  const month = String(date.getMonth() + 1).padStart(2, "0");
  const day = String(date.getDate()).padStart(2, "0");
  return `${year}-${month}-${day}`;
}

/** Create a local-noon calendar value for an ISO date, safe for date pickers. */
export function isoDateToCalendarDate(value: string) {
  const [year, month, day] = value.split("-").map(Number);
  return new Date(year ?? 1970, (month ?? 1) - 1, day ?? 1, 12);
}

export function normalizeIsoDateRange(from: string, to: string): IsoDateRange {
  return from <= to ? { from, to } : { from: to, to: from };
}

export function calendarRangeToIso(range: DateRange | undefined):
IsoDateRange | undefined {
  if (!range?.from || !range.to) return undefined;
  return normalizeIsoDateRange(calendarDateToIso(range.from),
calendarDateToIso(range.to));
}

export function manilaWeekDateRange(today = manilaNow().date):
IsoDateRange {
  const date = isoDateToCalendarDate(today);
  const mondayOffset = (date.getDay() + 6) % 7;
  const from = addDays(today, -mondayOffset);
  return { from, to: addDays(from, 6) };
}
```

```
export function manilaMonthDateRange(today = manilaNow().date):
IsoDateRange {
  const date = isoDateToCalendarDate(today);
  const from = `${today.slice(0, 7)}-01`;
  const to = calendarDateToIso(new Date(date.getFullYear(), date.getMonth() +
1, 0, 12));
  return { from, to };
}

export function manilaYearDateRange(today = manilaNow().date): IsoDateRange
{
  const year = today.slice(0, 4);
  return { from: `${year}-01-01`, to: `${year}-12-31` };
}

/** Inclusive date filters on timestamptz columns use this exclusive upper bound.
*/
export function startOfNextManilaDay(date: string) {
  return `${addDays(date, 1)}T00:00:00+08:00`;
}

export function startOfManilaDay(date: string) {
  return `${date}T00:00:00+08:00`;
}

/** Short Manila date, e.g. "Oct 14, 2026". */
export function formatShortDate(value: string) {
  return new Intl.DateTimeFormat("en-PH", {
    timeZone: "Asia/Manila",
    month: "short",
    day: "numeric",
    year: "numeric",
  }).format(new Date(value));
}
```

src/lib/appointment-confirmation.functions.ts

```
import { createClient } from "@supabase/supabase-js";
import { createServerFn } from "@tanstack/react-start";
import { getRequest } from "@tanstack/react-start/server";
import { z } from "zod";

import type { Database } from "@/integrations/supabase/types";
import {
  appointmentReminderEmail,
  appointmentReminderSms,
  bookingNotAcceptedEmail,
  bookingNotAcceptedSms,
  escapeHtml,
  formatAppointmentDate,
  formatAppointmentTime,
  formatSmsDate,
  rescheduleNotApprovedEmail,
  rescheduleNotApprovedSms,
  type AppointmentSummary,
  type EmailContent,
  type ShopContact,
} from "./customer-emails";
import { SHOP, normalizePhilippineMobile, phoneSchema } from "./shop";

const appointmentStatuses = [
  "pending",
  "confirmed",
  "in_progress",
  "completed",
  "rescheduled",
  "cancelled",
  "rejected",
  "no_show",
] as const;

const reviewSchema = z.object({
  appointmentId: z.string().uuid(),
  decision: z.enum(["confirmed", "rejected"]),
});

const serviceProgressSchema = z.object({
  appointmentId: z.string().uuid(),
  action: z.enum(["start", "snooze_arrival", "no_show", "complete",
"snooze_completion"]),
```

```
});

const appointmentUpdateSchema = z.object({
  appointmentId: z.string().uuid(),
  serviceIds: z.array(z.string().uuid()).min(1),
  appointmentDate: z.string().regex(/^\d{4}-\d{2}-\d{2}$/),
  startTime: z.string().regex(/^\d{2}:\d{2}$/),
  assignedCrewId: z.string().uuid().nullable(),
  crewAssignmentManual: z.boolean(),
  status: z.enum(appointmentStatuses),
  adminNotes: z.string().max(5_000).nullable(),
  firstName: z.string().trim().min(1).max(40),
  middleName: z.string().trim().max(40),
  lastName: z.string().trim().min(1).max(40),
  phone: phoneSchema,
  products: z
    .array(
      z.object({
        productId: z.string().uuid(),
        quantity: z.number().int().min(1).max(999),
        unitPrice: z.number().finite().min(0).max(9_999_999.99),
      }),
    )
    .max(50)
    .refine(
      (products) => new Set(products.map((product) => product.productId)).size
=== products.length,
      "Products must be unique.",
    ),
});

type ConfirmationEmailResult = { sent: true } | { sent: false; error: string | null };

type TextBeeConfig = {
  apiKey: string;
  deviceId?: string;
  simSubscriptionId?: number;
};

const DELIVERY_LEASE_MS = 5 * 60 * 1000;

type ConfirmationSmsResult = {
  sent: boolean;
  skipped: boolean;
  error: string | null;
```

```
};

/** An Android phone running SMS Gateway for Android in Local Server mode. */
type PhoneGatewayConfig = {
  url: string;
  username: string;
  password: string;
  /** Bookings created before this were copied from the live site and are never
texted. */
  bookingsFrom: Date;
};

type SmsConfig =
  ({ provider: "phone" } & PhoneGatewayConfig) | ({ provider: "textbee" } &
TextBeeConfig);

const COPIED_BOOKING_NOT_TEXTED =
  "No text was sent: this booking was copied from the live site, so the offline copy
does not text its customer.";

/**
 * SMS_GATEWAY_URL (the offline copy) sends through a phone on the local
network,
 * which needs no internet. Otherwise TextBee is used.
 */
function smsConfig(): { config: SmsConfig } | { error: string } {
  const gatewayUrl = process.env["SMS_GATEWAY_URL"]?.trim().replace(/\/+$/,
"");
  if (!gatewayUrl) {
    const textBee = textBeeConfig();
    return "error" in textBee ? textBee : { config: { provider: "textbee",
...textBee.config } };
  }

  const username = process.env["SMS_GATEWAY_USERNAME"]?.trim() ?? "";
  const password = process.env["SMS_GATEWAY_PASSWORD"]?.trim() ?? "";
  const bookingsFrom = new
Date(process.env["OFFLINE_COPY_TAKEN_AT"]?.trim() ?? "");
  let validUrl = false;
  try {
    validUrl = ["http:", "https:"].includes(new URL(gatewayUrl).protocol);
  } catch {
    validUrl = false;
  }
  if (!validUrl || !username || !password ||
```

```
Number.isNaN(bookingsFrom.getTime())) {
    return {
      error:
        "SMS is not configured correctly. Set SMS_GATEWAY_URL,
SMS_GATEWAY_USERNAME, SMS_GATEWAY_PASSWORD and
OFFLINE_COPY_TAKEN_AT.",
    };
  }
  return {
    config: { provider: "phone", url: gatewayUrl, username, password,
bookingsFrom },
  };
}

function textBeeConfig(): { config: TextBeeConfig } | { error: string } {
  const apiKey = process.env["TEXTBEE_API_KEY"]?.trim();
  if (!apiKey) {
    return {
      error: "SMS is not configured. Set TEXTBEE_API_KEY before sending
customer notifications.",
    };
  }

  const deviceId = process.env["TEXTBEE_DEVICE_ID"]?.trim() || undefined;
  const rawSimSubscriptionId =
process.env["TEXTBEE_SIM_SUBSCRIPTION_ID"]?.trim();
  if (!rawSimSubscriptionId) {
    return { config: deviceId ? { apiKey, deviceId } : { apiKey } };
  }

  const simSubscriptionId = Number(rawSimSubscriptionId);
  if (!Number.isInteger(simSubscriptionId) || simSubscriptionId < 0) {
    return {
      error:
        "SMS is not configured correctly. TEXTBEE_SIM_SUBSCRIPTION_ID must
be a non-negative integer.",
    };
  }
  return {
    config: deviceId ? { apiKey, deviceId, simSubscriptionId } : { apiKey,
simSubscriptionId },
  };
}

function formatDuration(durationMinutes: number) {
```

```
  const hours = Math.floor(durationMinutes / 60);
  const minutes = durationMinutes % 60;
  return [
    hours ? `${hours} hour${hours === 1 ? "" : "s"}` : "",
    minutes ? `${minutes} minutes` : "",
  ]
    .filter(Boolean)
    .join(" ");
}

function formatPhilippineMobileE164(phone: string) {
  const localNumber = normalizePhilippineMobile(phone);
  return localNumber ? `+63${localNumber.slice(1)}` : null;
}

function deliveryLeaseExpired(updatedAt: string | null | undefined) {
  if (!updatedAt) return true;
  const updatedAtMs = new Date(updatedAt).getTime();
  return !Number.isFinite(updatedAtMs) || updatedAtMs <= Date.now() -
DELIVERY_LEASE_MS;
}

function redactProviderDetail(value: string) {
  return value
    .replace(/\b(?:re|txb)_[A-Za-z0-9_-]+\b/g, "[redacted-api-key]")
    .replace(/[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}/gi, "[redacted-email]")
    .replace(/\+?\d[\d\s()-]{7,}\d/g, "[redacted-phone]")
    .replace(/ /g, " ")
    .trim()
    .slice(0, 400);
}

/** Sends one text message through TextBee; throws when TextBee does not
accept it. */
async function sendTextBeeSms(config: TextBeeConfig, recipient: string,
message: string) {
  const body: {
    recipients: string[];
    message: string;
    deviceId?: string;
    simSubscriptionId?: number;
  } = {
    recipients: [recipient],
    message,
  };
```

```
  if (config.deviceId) body.deviceId = config.deviceId;
  if (config.simSubscriptionId !== undefined) body.simSubscriptionId =
config.simSubscriptionId;

  const response = await fetch("https://api.textbee.dev/api/v1/gateway/send-sms",
{
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "x-api-key": config.apiKey,
    },
    body: JSON.stringify(body),
  });
  await assertTextBeeAccepted(response);
}

async function assertTextBeeAccepted(response: Response) {
  const body = await response.text();
  if (!response.ok) {
    const detail = redactProviderDetail(body);
    console.error("[TextBee] provider rejected notification", {
      status: response.status,
      detail: detail || undefined,
    });
    throw new Error(`TextBee returned ${response.status}${detail ? `: ${detail}` :
"."}`);
  }

  if (!body) return;
  try {
    const payload = JSON.parse(body) as {
      data?: { success?: boolean; successCount?: number; failureCount?: number
};
    };
    const delivery = payload.data;
    if (
      delivery?.success === false ||
      (typeof delivery?.failureCount === "number" && delivery.failureCount > 0) ||
      (typeof delivery?.successCount === "number" && delivery.successCount < 1)
    ) {
      console.error("[TextBee] notification was not accepted", {
        success: delivery?.success,
        successCount: delivery?.successCount,
        failureCount: delivery?.failureCount,
      });
```

```
      throw new Error("TextBee did not accept the SMS for delivery.");
    }
  } catch (error) {
    if (error instanceof SyntaxError) {
      console.warn("[TextBee] provider returned a non-JSON success response");
      return;
    }
    throw error;
  }
}

/** Sends one text message through the phone gateway; throws when the phone
does not accept it. */
async function sendPhoneGatewaySms(config: PhoneGatewayConfig, recipient:
string, message: string) {
  let response: Response;
  try {
    response = await fetch(`${config.url}/message`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        Authorization: `Basic ${btoa(`${config.username}:${config.password}`)}`,
      },
      body: JSON.stringify({ textMessage: { text: message }, phoneNumbers:
[recipient] }),
      signal: AbortSignal.timeout(15_000),
    });
  } catch (error) {
    console.error("[SMS phone] the phone could not be reached", error);
    throw new Error(
      "The SMS phone could not be reached. Check that it is on the same hotspot
and its Local Server is online.",
    );
  }

  const body = await response.text();
  if (!response.ok) {
    const detail = redactProviderDetail(body);
    console.error("[SMS phone] the phone rejected the message", {
      status: response.status,
      detail: detail || undefined,
    });
    throw new Error(
      response.status === 401
        ? "The SMS phone rejected the username or password."
```

```
        : `The SMS phone returned ${response.status}${detail ? `: ${detail}` : "."}`,
    );
  }
  try {
    const payload = JSON.parse(body) as { state?: string };
    if (payload.state === "Failed") throw new Error("The SMS phone could not
send the text.");
  } catch (error) {
    if (!(error instanceof SyntaxError)) throw error;
  }
}

function sendSms(config: SmsConfig, recipient: string, message: string) {
  return config.provider === "phone"
    ? sendPhoneGatewaySms(config, recipient, message)
    : sendTextBeeSms(config, recipient, message);
}

/**
 * The offline copy texts only bookings made in it. Customers copied from the
 * live site are real people, so their bookings are never texted from it.
 */
async function isCopiedBooking(config: SmsConfig, appointmentId: string) {
  if (config.provider !== "phone") return false;
  const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
  const { data, error } = await supabaseAdmin
    .from("appointments")
    .select("created_at")
    .eq("id", appointmentId)
    .maybeSingle();
  if (error || !data) throw new Error("The booking could not be checked before
texting.");
  return new Date(data.created_at) < config.bookingsFrom;
}

function authenticatedSupabaseClient() {
  const authorization = getRequest()?.headers.get("authorization");
  const url = process.env["SUPABASE_URL"];
  const key =
    process.env["SUPABASE_PUBLISHABLE_KEY"] ??
process.env["VITE_SUPABASE_PUBLISHABLE_KEY"];
  if (!authorization?.startsWith("Bearer ") || !url || !key) {
    throw new Error("Your administrator session has expired. Please sign in
again.");
```

```
  }

  return createClient<Database>(url, key, {
    global: { headers: { Authorization: authorization } },
    auth: { persistSession: false, autoRefreshToken: false },
  });
}

async function sendQueuedConfirmationEmail(
  appointmentId: string,
): Promise<ConfirmationEmailResult> {
  const { emailTransport, sendEmail } = await import("./email.server");
  const configured = emailTransport();
  const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
  if ("error" in configured) {
    console.error("[Confirmation email] configuration unavailable", { reason:
configured.error });
    const { error } = await supabaseAdmin
      .from("appointment_confirmation_emails")
      .update({
        last_error: configured.error.slice(0, 500),
        updated_at: new Date().toISOString(),
      })
      .eq("appointment_id", appointmentId)
      .eq("status", "pending");
    if (error) console.error("[Confirmation email] configuration-state update failed",
error);
    return {
      sent: false,
      error: configured.error,
    };
  }

  const now = new Date().toISOString();
  const { data: queuedDelivery, error: pendingError } = await supabaseAdmin
    .from("appointment_confirmation_emails")
    .select("id,attempt_count,recipient_email,status,updated_at")
    .eq("appointment_id", appointmentId)
    .maybeSingle();
  if (pendingError) {
    console.error("[Confirmation email] queue lookup failed", pendingError);
    return { sent: false, error: "The confirmation email could not be queued." };
  }
  if (!queuedDelivery) {
```

```
    return { sent: false, error: "The confirmation email could not be queued." };
  }
  if (queuedDelivery.status === "sent") return { sent: true };
  const retryingStaleClaim =
    queuedDelivery.status === "sending" &&
deliveryLeaseExpired(queuedDelivery.updated_at);
  if (queuedDelivery.status !== "pending" && !retryingStaleClaim) {
    console.info("[Confirmation email] delivery is already in progress", {
appointmentId });
    return { sent: false, error: null };
  }
  if (retryingStaleClaim)
    console.warn("[Confirmation email] retrying stale delivery lease", {
appointmentId });

  let claimQuery = supabaseAdmin
    .from("appointment_confirmation_emails")
    .update({
      status: "sending",
      attempt_count: queuedDelivery.attempt_count + 1,
      last_error: null,
      updated_at: now,
    })
    .eq("id", queuedDelivery.id);
  claimQuery = retryingStaleClaim
    ? claimQuery
        .eq("status", "sending")
        .lte("updated_at", new Date(Date.now() -
DELIVERY_LEASE_MS).toISOString())
    : claimQuery.eq("status", "pending");
  const { data: delivery, error: claimError } = await claimQuery
    .select("id,recipient_email")
    .maybeSingle();
  if (claimError) {
    console.error("[Confirmation email] queue claim failed", claimError);
    return { sent: false, error: "The confirmation email could not be queued." };
  }
  if (!delivery) {
    console.info("[Confirmation email] delivery claim lost to another request", {
appointmentId });
    return { sent: false, error: null };
  }

  try {
    const { data: appointment, error: appointmentError } = await supabaseAdmin
```

```
      .from("appointments")
      .select(
        "reference_code,customer_name,appointment_date,start_time,booking_dur
ation_minutes,total_estimate,moto_brand,moto_model,moto_variant,moto_year,
plate_number,notes,appointment_services(service_name,duration_minutes),app
ointment_continuations(segment_number,appointment_date,start_time,booking_
duration_minutes)",
      )
      .eq("id", appointmentId)
      .in("status", ["confirmed", "in_progress", "completed"])
      .maybeSingle();
    if (appointmentError || !appointment) {
      throw new Error("The approved appointment details could not be loaded.");
    }

    const primarySchedule =
`${formatAppointmentDate(appointment.appointment_date)} at
${formatAppointmentTime(appointment.start_time)}
(${formatDuration(appointment.booking_duration_minutes)})`;
    const continuationSchedules = (appointment.appointment_continuations ?? [])
      .sort((left, right) => left.segment_number - right.segment_number)
      .map(
        (segment) =>
          `${formatAppointmentDate(segment.appointment_date)} at
${formatAppointmentTime(segment.start_time)}
(${formatDuration(segment.booking_duration_minutes)})`,
      );
    const services = (appointment.appointment_services ?? []).map(
      (service) => service.service_name,
    );
    const motorcycle = [
      appointment.moto_brand,
      appointment.moto_model,
      appointment.moto_variant,
      appointment.moto_year,
    ]
      .filter(Boolean)
      .join(" ");
    const scheduleRows = [primarySchedule, ...continuationSchedules]
      .map((schedule) => `<li>${escapeHtml(schedule)}</li>`)
      .join("");
    const serviceList = services.map((service) =>
`<li>${escapeHtml(service)}</li>`).join("");
    const text = [
      `Hello ${appointment.customer_name},`,
```

```
      "",
      "Your Fake Rider Motorparts appointment has been approved.",
      `Reference code: ${appointment.reference_code}`,
      `Schedule: ${[primarySchedule, ...continuationSchedules].join("; ")}`,
      `Services: ${services.join(", ")}`,
      `Motorcycle: ${motorcycle}`,
      `Plate number: ${appointment.plate_number}`,
      `Estimated total: PHP ${Number(appointment.total_estimate).toFixed(2)}`,
      appointment.notes ? `Notes: ${appointment.notes}` : "",
      "",
      "Please keep your reference code for appointment tracking.",
    ]
      .filter(Boolean)
      .join("\n");
    const html = `<p>Hello
${escapeHtml(appointment.customer_name)},</p><p>Your Fake Rider
Motorparts appointment has been
<strong>approved</strong>.</p><p><strong>Reference code:</strong>
${escapeHtml(appointment.reference_code)}</p><p><strong>Confirmed schedul
e:</strong></p><ul>${scheduleRows}</ul><p><strong>Services:</strong></p><
ul>${serviceList}</ul><p><strong>Motorcycle:</strong>
${escapeHtml(motorcycle)}<br /><strong>Plate number:</strong>
${escapeHtml(appointment.plate_number)}<br /><strong>Estimated
total:</strong> PHP ${escapeHtml(Number(appointment.total_estimate).toFixed(
2))}</p>${appointment.notes ? `<p><strong>Notes:</strong>
${escapeHtml(appointment.notes)}</p>` : ""}<p>Please keep your reference
code for appointment tracking.</p>`;

    await sendEmail(configured.transport, {
      to: delivery.recipient_email,
      subject: `Appointment confirmed - ${appointment.reference_code}`,
      html,
      text,
      idempotencyKey: `appointment-confirmation-${delivery.id}`,
    });

    const { error: sentError } = await supabaseAdmin
      .from("appointment_confirmation_emails")
      .update({
        status: "sent",
        sent_at: new Date().toISOString(),
        updated_at: new Date().toISOString(),
      })
      .eq("id", delivery.id);
    if (sentError) console.error("[Confirmation email] sent-state update failed",
```

```
sentError);
    return { sent: true };
  } catch (error) {
    const message =
      error instanceof Error ? error.message : "The email provider could not be
reached.";
    console.error("[Confirmation email] send failed", error);
    await supabaseAdmin
      .from("appointment_confirmation_emails")
      .update({
        status: "pending",
        last_error: message.slice(0, 500),
        updated_at: new Date().toISOString(),
      })
      .eq("id", delivery.id);
    return {
      sent: false,
      error: "The appointment was confirmed, but the confirmation email could not
be sent.",
    };
  }
}

async function sendQueuedConfirmationSms(appointmentId: string):
Promise<ConfirmationSmsResult> {
  const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
  const now = new Date().toISOString();
  const { data: queuedDelivery, error: pendingError } = await supabaseAdmin
    .from("appointment_confirmation_sms")
    .select("id,attempt_count,recipient_phone,status,updated_at")
    .eq("appointment_id", appointmentId)
    .maybeSingle();
  if (pendingError) {
    console.error("[Confirmation SMS] queue lookup failed", pendingError);
    return { sent: false, skipped: false, error: "The confirmation SMS could not be
queued." };
  }
  if (!queuedDelivery) {
    return { sent: false, skipped: false, error: "The confirmation SMS could not be
queued." };
  }
  if (queuedDelivery.status === "sent") return { sent: true, skipped: false, error:
null };
  if (queuedDelivery.status === "skipped") return { sent: false, skipped: true, error:
```

```
null };
  const retryingStaleClaim =
    queuedDelivery.status === "sending" &&
deliveryLeaseExpired(queuedDelivery.updated_at);
  if (queuedDelivery.status !== "pending" && !retryingStaleClaim) {
    console.info("[Confirmation SMS] delivery is already in progress", {
appointmentId });
    return { sent: false, skipped: false, error: null };
  }
  if (retryingStaleClaim)
    console.warn("[Confirmation SMS] retrying stale delivery lease", {
appointmentId });

  let claimQuery = supabaseAdmin
    .from("appointment_confirmation_sms")
    .update({
      status: "sending",
      attempt_count: queuedDelivery.attempt_count + 1,
      last_error: null,
      updated_at: now,
    })
    .eq("id", queuedDelivery.id);
  claimQuery = retryingStaleClaim
    ? claimQuery
        .eq("status", "sending")
        .lte("updated_at", new Date(Date.now() -
DELIVERY_LEASE_MS).toISOString())
    : claimQuery.eq("status", "pending");
  const { data: delivery, error: claimError } = await claimQuery
    .select("id,recipient_phone")
    .maybeSingle();
  if (claimError) {
    console.error("[Confirmation SMS] queue claim failed", claimError);
    return { sent: false, skipped: false, error: "The confirmation SMS could not be
queued." };
  }
  if (!delivery) {
    console.info("[Confirmation SMS] delivery claim lost to another request", {
appointmentId });
    return { sent: false, skipped: false, error: null };
  }

  const configured = smsConfig();
  if ("error" in configured) {
    await supabaseAdmin
```

```
      .from("appointment_confirmation_sms")
      .update({
        status: "pending",
        last_error: configured.error.slice(0, 500),
        updated_at: new Date().toISOString(),
      })
      .eq("id", delivery.id);
    return { sent: false, skipped: false, error: configured.error };
  }

  try {
    if (await isCopiedBooking(configured.config, appointmentId)) {
      await supabaseAdmin
        .from("appointment_confirmation_sms")
        .update({
          status: "skipped",
          last_error: COPIED_BOOKING_NOT_TEXTED,
          updated_at: new Date().toISOString(),
        })
        .eq("id", delivery.id);
      return { sent: false, skipped: true, error: COPIED_BOOKING_NOT_TEXTED
};
    }
    const recipient = formatPhilippineMobileE164(delivery.recipient_phone);
    if (!recipient) throw new Error("The customer's mobile number is invalid.");

    const { data: appointment, error: appointmentError } = await supabaseAdmin
      .from("appointments")
      .select(
        "reference_code,appointment_date,start_time,booking_duration_minutes,tot
al_estimate,rescheduled_from_appointment_id,appointment_services(service_na
me)",
      )
      .eq("id", appointmentId)
      .in("status", ["confirmed", "in_progress", "completed"])
      .maybeSingle();
    if (appointmentError || !appointment) {
      throw new Error("The approved appointment details could not be loaded.");
    }

    const services = (appointment.appointment_services ?? [])
      .map((service) => service.service_name)
      .filter(Boolean)
      .join(", ");
    const appointmentKind = appointment.rescheduled_from_appointment_id
```

```
      ? "rescheduled appointment"
      : "appointment";
    const message = [
      `Fake Rider: Your ${appointmentKind} is CONFIRMED.`,
      `Ref: ${appointment.reference_code}.`,
      `Date: ${formatSmsDate(appointment.appointment_date)} at
${formatAppointmentTime(appointment.start_time)}
(${formatDuration(appointment.booking_duration_minutes)}).`,
      services ? `Services: ${services}.` : "",
      `Estimated total: PHP ${Number(appointment.total_estimate).toFixed(2)}.`,
      "Please keep your reference code for tracking.",
    ]
      .filter(Boolean)
      .join(" ");
    await sendSms(configured.config, recipient, message);

    const { error: sentError } = await supabaseAdmin
      .from("appointment_confirmation_sms")
      .update({
        status: "sent",
        sent_at: new Date().toISOString(),
        updated_at: new Date().toISOString(),
      })
      .eq("id", delivery.id);
    if (sentError) console.error("[Confirmation SMS] sent-state update failed",
sentError);
    return { sent: true, skipped: false, error: null };
  } catch (error) {
    const message =
      error instanceof Error ? error.message : "The SMS provider could not be
reached.";
    console.error("[Confirmation SMS] send failed", error);
    await supabaseAdmin
      .from("appointment_confirmation_sms")
      .update({
        status: "pending",
        last_error: message.slice(0, 500),
        updated_at: new Date().toISOString(),
      })
      .eq("id", delivery.id);
    return {
      sent: false,
      skipped: false,
      error: "The appointment was confirmed, but the confirmation SMS could not
be sent.",
```

```
    };
  }
}

async function sendConfirmationNotifications(appointmentId: string) {
  const [emailResult, smsResult] = await Promise.allSettled([
    sendQueuedConfirmationEmail(appointmentId),
    sendQueuedConfirmationSms(appointmentId),
  ]);
  const emailDelivery: ConfirmationEmailResult =
    emailResult.status === "fulfilled"
      ? emailResult.value
      : (() => {
          console.error("[Confirmation email] unhandled delivery failure",
emailResult.reason);
          return {
            sent: false,
            error: "The appointment was confirmed, but the confirmation email could
not be sent.",
          };
        })();
  const smsDelivery: ConfirmationSmsResult =
    smsResult.status === "fulfilled"
      ? smsResult.value
      : (() => {
          console.error("[Confirmation SMS] unhandled delivery failure",
smsResult.reason);
          return {
            sent: false,
            skipped: false,
            error: "The appointment was confirmed, but the confirmation SMS could
not be sent.",
          };
        })();
  return { emailDelivery, smsDelivery };
}

type CompletionDeliveryResult = { sent: true } | { sent: false; error: string | null };
type DeliveryChannel = "email" | "sms";
/** Queues with one row per appointment and channel. */
type ChannelQueue = "appointment_completion_notifications" |
"appointment_reminders";
type ChannelClaim =
  | { kind: "claimed"; id: string; recipient: string }
  | { kind: "sent" }
```

```
  | { kind: "skipped" }
  | { kind: "in_progress" }
  | { kind: "error"; error: string };

/** "completion email", "reminder SMS" and so on, for messages and logs. */
function channelDeliveryLabel(queue: ChannelQueue, channel:
DeliveryChannel) {
  const kind = queue === "appointment_reminders" ? "reminder" : "completion";
  return `${kind} ${channel === "email" ? "email" : "SMS"}`;
}

function channelLogPrefix(queue: ChannelQueue, channel: DeliveryChannel) {
  const label = channelDeliveryLabel(queue, channel);
  return `[${label.charAt(0).toUpperCase()}${label.slice(1)}]`;
}

async function claimChannelDelivery(
  queue: ChannelQueue,
  appointmentId: string,
  channel: DeliveryChannel,
): Promise<ChannelClaim> {
  const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
  const label = channelDeliveryLabel(queue, channel);
  const log = channelLogPrefix(queue, channel);
  const { data: queuedDelivery, error: pendingError } = await supabaseAdmin
    .from(queue)
    .select("id,attempt_count,recipient,status,updated_at")
    .eq("appointment_id", appointmentId)
    .eq("channel", channel)
    .maybeSingle();
  if (pendingError || !queuedDelivery) {
    if (pendingError) console.error(`${log} queue lookup failed`, pendingError);
    return { kind: "error", error: `The ${label} could not be queued.` };
  }
  if (queuedDelivery.status === "sent") return { kind: "sent" };
  if (queuedDelivery.status === "skipped") return { kind: "skipped" };
  const retryingStaleClaim =
    queuedDelivery.status === "sending" &&
deliveryLeaseExpired(queuedDelivery.updated_at);
  if (queuedDelivery.status !== "pending" && !retryingStaleClaim) {
    console.info(`${log} delivery is already in progress`, { appointmentId });
    return { kind: "in_progress" };
  }
  if (retryingStaleClaim) {
```

```
    console.warn(`${log} retrying stale delivery lease`, { appointmentId });
  }

  let claimQuery = supabaseAdmin
    .from(queue)
    .update({
      status: "sending",
      attempt_count: queuedDelivery.attempt_count + 1,
      last_error: null,
      updated_at: new Date().toISOString(),
    })
    .eq("id", queuedDelivery.id);
  claimQuery = retryingStaleClaim
    ? claimQuery
        .eq("status", "sending")
        .lte("updated_at", new Date(Date.now() -
DELIVERY_LEASE_MS).toISOString())
    : claimQuery.eq("status", "pending");
  const { data: delivery, error: claimError } = await claimQuery
    .select("id,recipient")
    .maybeSingle();
  if (claimError) {
    console.error(`${log} queue claim failed`, claimError);
    return { kind: "error", error: `The ${label} could not be queued.` };
  }
  if (!delivery) {
    console.info(`${log} delivery claim lost to another request`, { appointmentId });
    return { kind: "in_progress" };
  }
  return { kind: "claimed", ...delivery };
}

async function resetChannelDelivery(
  queue: ChannelQueue,
  deliveryId: string,
  error: unknown,
  channel: DeliveryChannel,
) {
  const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
  const message =
    error instanceof Error ? error.message : "The notification provider could not be
reached.";
  console.error(`${channelLogPrefix(queue, channel)} send failed`, error);
  await supabaseAdmin
```

```
    .from(queue)
    .update({
      status: "pending",
      last_error: message.slice(0, 500),
      updated_at: new Date().toISOString(),
    })
    .eq("id", deliveryId);
}

async function markChannelDeliverySent(
  queue: ChannelQueue,
  deliveryId: string,
  channel: DeliveryChannel,
) {
  const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
  const { error } = await supabaseAdmin
    .from(queue)
    .update({
      status: "sent",
      sent_at: new Date().toISOString(),
      updated_at: new Date().toISOString(),
    })
    .eq("id", deliveryId);
  if (error) console.error(`${channelLogPrefix(queue, channel)} sent-state update
failed`, error);
}

async function markChannelDeliverySkipped(
  queue: ChannelQueue,
  deliveryId: string,
  channel: DeliveryChannel,
  reason: string,
) {
  const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
  const { error } = await supabaseAdmin
    .from(queue)
    .update({ status: "skipped", last_error: reason, updated_at: new
Date().toISOString() })
    .eq("id", deliveryId);
  if (error)
    console.error(`${channelLogPrefix(queue, channel)} skipped-state update
failed`, error);
}
```

```
async function loadCompletedAppointment(appointmentId: string) {
  const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
  const { data: appointment, error } = await supabaseAdmin
    .from("appointments")
    .select(
      "reference_code,customer_name,appointment_date,service_ended_at,total_
estimate,appointment_services(service_name),appointment_products(product_n
ame,quantity,unit_price)",
    )
    .eq("id", appointmentId)
    .eq("status", "completed")
    .maybeSingle();
  if (error || !appointment) {
    throw new Error("The completed appointment details could not be loaded.");
  }

  const services = (appointment.appointment_services ?? [])
    .map((service) => service.service_name)
    .filter(Boolean);
  const productTotal = (appointment.appointment_products ?? []).reduce(
    (total, product) => total + product.quantity * Number(product.unit_price),
    0,
  );
  return {
    ...appointment,
    services,
    finalAmount: Number(appointment.total_estimate) + productTotal,
  };
}

function formatCompletionDate(value: string | null, appointmentDate: string) {
  if (!value) return formatAppointmentDate(appointmentDate);
  return new Intl.DateTimeFormat("en-PH", {
    dateStyle: "full",
    timeZone: "Asia/Manila",
  }).format(new Date(value));
}

function formatCompletionSmsDate(value: string | null, appointmentDate: string) {
  if (!value) return formatSmsDate(appointmentDate);
  return new Intl.DateTimeFormat("en-PH", {
    month: "short",
    day: "numeric",
```

```
    year: "numeric",
    timeZone: "Asia/Manila",
  }).format(new Date(value));
}

async function sendQueuedCompletionEmail(appointmentId: string):
Promise<CompletionDeliveryResult> {
  const { emailTransport, sendEmail } = await import("./email.server");
  const configured = emailTransport();
  const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
  if ("error" in configured) {
    console.error("[Completion email] configuration unavailable", { reason:
configured.error });
    const { error } = await supabaseAdmin
      .from("appointment_completion_notifications")
      .update({
        last_error: configured.error.slice(0, 500),
        updated_at: new Date().toISOString(),
      })
      .eq("appointment_id", appointmentId)
      .eq("channel", "email")
      .eq("status", "pending");
    if (error) console.error("[Completion email] configuration-state update failed",
error);
    return {
      sent: false,
      error: configured.error,
    };
  }
  const claim = await claimChannelDelivery(
    "appointment_completion_notifications",
    appointmentId,
    "email",
  );
  if (claim.kind === "sent") return { sent: true };
  if (claim.kind === "in_progress") return { sent: false, error: null };
  if (claim.kind === "skipped") {
    return { sent: false, error: "The completion email was skipped." };
  }
  if (claim.kind === "error") return { sent: false, error: claim.error };

  try {
    const appointment = await loadCompletedAppointment(appointmentId);
    const completedOn = formatCompletionDate(
```

```
      appointment.service_ended_at,
      appointment.appointment_date,
    );
    const serviceList = appointment.services
      .map((service) => `<li>${escapeHtml(service)}</li>`)
      .join("");
    const text = [
      `Hello ${appointment.customer_name},`,
      "",
      "Your service at Fake Rider Motorparts has been completed.",
      `Reference code: ${appointment.reference_code}`,
      `Completed on: ${completedOn}`,
      `Services: ${appointment.services.join(", ")}`,
      `Final amount: PHP ${appointment.finalAmount.toFixed(2)}`,
      "Thank you for choosing Fake Rider Motorparts.",
    ].join("\n");
    const html = `<p>Hello
${escapeHtml(appointment.customer_name)},</p><p>Your service at Fake Rider
Motorparts has been <strong>completed</strong>.</p><p><strong>Reference
code:</strong> ${escapeHtml(appointment.reference_code)}<br
/><strong>Completed on:</strong> ${escapeHtml(completedOn)}<br
/><strong>Final amount:</strong> PHP ${escapeHtml(appointment.finalAmount.t
oFixed(2))}</p><p><strong>Services:</strong></p><ul>${serviceList}</ul><p>Th
ank you for choosing Fake Rider Motorparts.</p>`;
    await sendEmail(configured.transport, {
      to: claim.recipient,
      subject: `Service completed - ${appointment.reference_code}`,
      html,
      text,
      idempotencyKey: `appointment-completion-${claim.id}`,
    });
    await markChannelDeliverySent("appointment_completion_notifications",
claim.id, "email");
    return { sent: true };
  } catch (error) {
    await resetChannelDelivery("appointment_completion_notifications", claim.id,
error, "email");
    return {
      sent: false,
      error: "The service was completed, but the completion email could not be
sent.",
    };
  }
}
```

```
async function sendQueuedCompletionSms(appointmentId: string):
Promise<ConfirmationSmsResult> {
  const claim = await claimChannelDelivery(
    "appointment_completion_notifications",
    appointmentId,
    "sms",
  );
  if (claim.kind === "sent") return { sent: true, skipped: false, error: null };
  if (claim.kind === "skipped") return { sent: false, skipped: true, error: null };
  if (claim.kind === "in_progress") return { sent: false, skipped: false, error: null };
  if (claim.kind === "error") return { sent: false, skipped: false, error: claim.error };
  if (claim.kind !== "claimed") {
    return { sent: false, skipped: false, error: "The completion SMS could not be
queued." };
  }

  const configured = smsConfig();
  if ("error" in configured) {
    await resetChannelDelivery(
      "appointment_completion_notifications",
      claim.id,
      new Error(configured.error),
      "sms",
    );
    return { sent: false, skipped: false, error: configured.error };
  }

  try {
    if (await isCopiedBooking(configured.config, appointmentId)) {
      await markChannelDeliverySkipped(
        "appointment_completion_notifications",
        claim.id,
        "sms",
        COPIED_BOOKING_NOT_TEXTED,
      );
      return { sent: false, skipped: true, error: COPIED_BOOKING_NOT_TEXTED
};
    }
    const recipient = formatPhilippineMobileE164(claim.recipient);
    if (!recipient) throw new Error("The customer's mobile number is invalid.");
    const appointment = await loadCompletedAppointment(appointmentId);
    const message = [
      "Fake Rider: Your service is COMPLETED.",
      `Ref: ${appointment.reference_code}.`,
      `Completed: ${formatCompletionSmsDate(appointment.service_ended_at,
```

```
appointment.appointment_date)}.`,
      appointment.services.length ? `Services: ${appointment.services.join(", ")}.` :
"",
      `Final amount: PHP ${appointment.finalAmount.toFixed(2)}.`,
      "Thank you for choosing Fake Rider.",
    ]
      .filter(Boolean)
      .join(" ");
    await sendSms(configured.config, recipient, message);
    await markChannelDeliverySent("appointment_completion_notifications",
claim.id, "sms");
    return { sent: true, skipped: false, error: null };
  } catch (error) {
    await resetChannelDelivery("appointment_completion_notifications", claim.id,
error, "sms");
    return {
      sent: false,
      skipped: false,
      error: "The service was completed, but the completion SMS could not be
sent.",
    };
  }
}

async function sendCompletionNotifications(appointmentId: string) {
  const [emailResult, smsResult] = await Promise.allSettled([
    sendQueuedCompletionEmail(appointmentId),
    sendQueuedCompletionSms(appointmentId),
  ]);
  const emailDelivery: CompletionDeliveryResult =
    emailResult.status === "fulfilled"
      ? emailResult.value
      : {
          sent: false,
          error: "The service was completed, but the completion email could not be
sent.",
        };
  const smsDelivery: ConfirmationSmsResult =
    smsResult.status === "fulfilled"
      ? smsResult.value
      : {
          sent: false,
          skipped: false,
          error: "The service was completed, but the completion SMS could not be
sent.",
```

```
      };
  if (emailResult.status === "rejected") {
    console.error("[Completion email] unhandled delivery failure",
emailResult.reason);
  }
  if (smsResult.status === "rejected") {
    console.error("[Completion SMS] unhandled delivery failure",
smsResult.reason);
  }
  return { emailDelivery, smsDelivery };
}

type CustomerEmailResult = { sent: true } | { sent: false; error: string };

/** The shop's contact details as set in Settings, for customer emails and texts. */
async function shopContact(): Promise<ShopContact> {
  const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
  const { data, error } = await supabaseAdmin
    .from("shop_settings")
    .select("shop_name,contact_number,contact_email")
    .eq("id", true)
    .maybeSingle();
  if (error) console.warn("[Customer email] shop contact lookup failed; using
defaults", error);
  return {
    name: data?.shop_name?.trim() || SHOP.name,
    phone: data?.contact_number?.trim() || SHOP.phone,
    email: data?.contact_email?.trim() || SHOP.email,
  };
}

/**
 * A public page on this site, from the admin's own request. Links are left
 * out when the console runs on a local development address.
 */
function publicPageUrl(path: string) {
  const origin = getRequest()?.headers.get("origin");
  return origin?.startsWith("https://") ? `${origin}${path}` : null;
}

/** One customer email. Failures are logged and reported to the admin, not
retried. */
async function deliverCustomerEmail(
  label: string,
```

```
  to: string,
  content: EmailContent,
  failure: string,
): Promise<CustomerEmailResult> {
  const { emailTransport, sendEmail } = await import("./email.server");
  const configured = emailTransport();
  if ("error" in configured) {
    console.error(`[${label}] configuration unavailable`, { reason: configured.error
});
    return { sent: false, error: `${failure} ${configured.error}` };
  }
  try {
    await sendEmail(configured.transport, { to, ...content });
    return { sent: true };
  } catch (error) {
    console.error(`[${label}] send failed`, error);
    return { sent: false, error: failure };
  }
}

type NoticeDelivery = { sent: boolean; error: string | null };

/** One text message to a customer. Failures are logged and reported to the
admin, not retried. */
async function deliverCustomerSms(
  label: string,
  appointmentId: string,
  phone: string,
  message: string,
  failure: string,
): Promise<NoticeDelivery> {
  const recipient = formatPhilippineMobileE164(phone);
  if (!recipient) return { sent: false, error: `${failure} The mobile number is invalid.`
};
  const configured = smsConfig();
  if ("error" in configured) {
    console.error(`[${label}] configuration unavailable`, { reason: configured.error
});
    return { sent: false, error: `${failure} ${configured.error}` };
  }
  try {
    if (await isCopiedBooking(configured.config, appointmentId)) {
      return { sent: false, error: COPIED_BOOKING_NOT_TEXTED };
    }
    await sendSms(configured.config, recipient, message);
```

```
    return { sent: true, error: null };
  } catch (error) {
    console.error(`[${label}] send failed`, error);
    return { sent: false, error: failure };
  }
}

/** The appointment details a customer notice needs, or null if it cannot be
loaded. */
async function loadNoticeAppointment(appointmentId: string, status?: "rejected" |
"confirmed") {
  const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
  let query = supabaseAdmin
    .from("appointments")
    .select(
      "reference_code,customer_name,email,phone,appointment_date,start_time,
moto_brand,moto_model,plate_number,appointment_services(service_name)",
    )
    .eq("id", appointmentId);
  if (status) query = query.eq("status", status);
  const { data: appointment, error } = await query.maybeSingle();
  if (error || !appointment) {
    if (error) console.error("[Customer notice] appointment lookup failed", error);
    return null;
  }
  const summary: AppointmentSummary = {
    referenceCode: appointment.reference_code,
    customerName: appointment.customer_name,
    appointmentDate: appointment.appointment_date,
    startTime: appointment.start_time,
    services: (appointment.appointment_services ?? []).map((service) =>
service.service_name),
    motorcycle: [appointment.moto_brand,
appointment.moto_model].filter(Boolean).join(" "),
    plateNumber: appointment.plate_number,
  };
  return {
    summary,
    email: appointment.email?.trim().toLowerCase() || null,
    phone: appointment.phone,
  };
}

/**
```

```
 * Emails and texts the customer at the same time. Without an email on file the
 * text alone is sent; the admin is asked to call only when neither went out.
 */
async function deliverNotice({
  label,
  outcome,
  appointmentId,
  appointment,
  email,
  sms,
}: {
  label: string;
  /** What happened, e.g. "The booking was rejected". */
  outcome: string;
  appointmentId: string;
  appointment: { email: string | null; phone: string };
  email: EmailContent;
  sms: string;
}): Promise<{ emailDelivery: NoticeDelivery; smsDelivery: NoticeDelivery }> {
  const [emailDelivery, smsDelivery] = await Promise.all([
    appointment.email
      ? deliverCustomerEmail(
          `${label} email`,
          appointment.email,
          email,
          `${outcome}, but the email to the customer could not be sent.`,
        ).then((result) => (result.sent ? { sent: true, error: null } : result))
      : Promise.resolve({ sent: false, error: null }),
    deliverCustomerSms(
      `${label} SMS`,
      appointmentId,
      appointment.phone,
      sms,
      `${outcome}, but the text message to the customer could not be sent.`,
    ),
  ]);
  if (!appointment.email && !smsDelivery.sent) {
    smsDelivery.error = `${smsDelivery.error} This customer has no email on file,
so please let them know by phone.`;
  }
  return { emailDelivery, smsDelivery };
}

/** Tells the customer their pending booking request was not accepted. */
async function notifyBookingNotAccepted(appointmentId: string) {
```

```
  const appointment = await loadNoticeAppointment(appointmentId, "rejected");
  if (!appointment) {
    return {
      emailDelivery: {
        sent: false,
        error: "The booking was rejected, but the customer could not be notified.",
      },
      smsDelivery: { sent: false, error: null },
    };
  }
  const contact = await shopContact();
  return deliverNotice({
    label: "Booking not accepted",
    outcome: "The booking was rejected",
    appointmentId,
    appointment,
    email: bookingNotAcceptedEmail({
      appointment: appointment.summary,
      contact,
      bookingUrl: publicPageUrl("/book"),
    }),
    sms: bookingNotAcceptedSms({ appointment: appointment.summary, contact
}),
  });
}

/** Tells the customer their reschedule request was not approved. */
async function notifyRescheduleNotApproved(appointmentId: string) {
  const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
  const [appointment, requestRes] = await Promise.all([
    loadNoticeAppointment(appointmentId),
    // Rejecting clears the request from the appointment; its history keeps the
requested time.
    supabaseAdmin
      .from("appointment_reschedule_history")
      .select("to_date,to_start_time")
      .eq("appointment_id", appointmentId)
      .eq("decision", "rejected")
      .order("approved_at", { ascending: false })
      .limit(1)
      .maybeSingle(),
  ]);
  if (!appointment) {
    return {
```

```
      emailDelivery: {
        sent: false,
        error: "The reschedule request was rejected, but the customer could not be
notified.",
      },
      smsDelivery: { sent: false, error: null },
    };
  }
  if (requestRes.error) {
    console.warn("[Reschedule not approved] request lookup failed",
requestRes.error);
  }
  const contact = await shopContact();
  const requested = {
    requestedDate: requestRes.data?.to_date ?? null,
    requestedStartTime: requestRes.data?.to_start_time ?? null,
  };
  return deliverNotice({
    label: "Reschedule not approved",
    outcome: "The reschedule request was rejected",
    appointmentId,
    appointment,
    email: rescheduleNotApprovedEmail({
      appointment: appointment.summary,
      ...requested,
      contact,
      appointmentUrl: publicPageUrl("/my-appointment"),
    }),
    sms: rescheduleNotApprovedSms({ appointment: appointment.summary,
...requested, contact }),
  });
}

/** Tries per customer message, the first included, before the admin is asked to
step in. */
const MAX_DELIVERY_ATTEMPTS = 5;

/** Most messages one scheduled run sends, so a backlog never outlasts the
request. */
const SCHEDULED_RUN_SEND_LIMIT = 20;

/** Waits 5, 10, 20, then 40 minutes after each failed try. */
function retryIsDue(attemptCount: number, updatedAt: string) {
  const waitMinutes = 5 * 2 ** Math.max(0, attemptCount - 1);
  return Date.parse(updatedAt) <= Date.now() - waitMinutes * 60_000;
```

```
}

/** When an appointment starts, from its Philippine date and time. */
function appointmentStartMs(date: string, time: string) {
  return Date.parse(`${date}T${time.slice(0, 5)}:00+08:00`);
}

/** Sends one queued reminder. A failure stays queued for the next scheduled
run. */
async function sendQueuedReminder(
  appointmentId: string,
  channel: DeliveryChannel,
  siteOrigin: string | null,
): Promise<NoticeDelivery> {
  const queue = "appointment_reminders";
  const claim = await claimChannelDelivery(queue, appointmentId, channel);
  if (claim.kind !== "claimed") {
    return { sent: claim.kind === "sent", error: claim.kind === "error" ? claim.error :
null };
  }

  try {
    const appointment = await loadNoticeAppointment(appointmentId,
"confirmed");
    if (
      !appointment ||
      appointmentStartMs(appointment.summary.appointmentDate,
appointment.summary.startTime) <=
        Date.now()
    ) {
      await markChannelDeliverySkipped(
        queue,
        claim.id,
        channel,
        "No reminder was sent: the appointment is no longer confirmed or has
already started.",
      );
      return { sent: false, error: null };
    }
    const contact = await shopContact();

    if (channel === "email") {
      const { emailTransport, sendEmail } = await import("./email.server");
      const configured = emailTransport();
      if ("error" in configured) throw new Error(configured.error);
```

```
      await sendEmail(configured.transport, {
        to: claim.recipient,
        ...appointmentReminderEmail({
          appointment: appointment.summary,
          contact,
          appointmentUrl: siteOrigin ? `${siteOrigin}/my-appointment` : null,
        }),
        idempotencyKey: `appointment-reminder-${claim.id}`,
      });
    } else {
      const configured = smsConfig();
      if ("error" in configured) throw new Error(configured.error);
      if (await isCopiedBooking(configured.config, appointmentId)) {
        await markChannelDeliverySkipped(queue, claim.id, channel,
COPIED_BOOKING_NOT_TEXTED);
        return { sent: false, error: null };
      }
      const recipient = formatPhilippineMobileE164(claim.recipient);
      if (!recipient) throw new Error("The customer's mobile number is invalid.");
      await sendSms(
        configured.config,
        recipient,
        appointmentReminderSms({ appointment: appointment.summary, contact }),
      );
    }
    await markChannelDeliverySent(queue, claim.id, channel);
    return { sent: true, error: null };
  } catch (error) {
    await resetChannelDelivery(queue, claim.id, error, channel);
    return {
      sent: false,
      error: `The ${channelDeliveryLabel(queue, channel)} could not be sent.`,
    };
  }
}

/** A customer message waiting in one of the delivery queues. */
type QueuedDelivery = {
  kind: "confirmation" | "completion" | "reminder";
  queue: "appointment_confirmation_emails" | "appointment_confirmation_sms" |
ChannelQueue;
  id: string;
  appointmentId: string;
  channel: DeliveryChannel;
  attemptCount: number;
```

```
  createdAt: string;
  updatedAt: string;
  lastError: string | null;
};

type DeliveryAppointment = {
  id: string;
  status: string;
  appointment_date: string;
  start_time: string;
  reference_code: string;
  customer_name: string;
  phone: string;
};

async function loadQueuedDeliveries(): Promise<QueuedDelivery[]> {
  const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
  const [emails, texts, completions, reminders] = await Promise.all([
    supabaseAdmin
      .from("appointment_confirmation_emails")
      .select("id,appointment_id,attempt_count,created_at,updated_at,last_error")
      .eq("status", "pending")
      .order("created_at")
      .limit(50),
    supabaseAdmin
      .from("appointment_confirmation_sms")
      .select("id,appointment_id,attempt_count,created_at,updated_at,last_error")
      .eq("status", "pending")
      .order("created_at")
      .limit(50),
    supabaseAdmin
      .from("appointment_completion_notifications")
      .select("id,appointment_id,channel,attempt_count,created_at,updated_at,last
_error")
      .eq("status", "pending")
      .order("created_at")
      .limit(50),
    supabaseAdmin
      .from("appointment_reminders")
      .select("id,appointment_id,channel,attempt_count,created_at,updated_at,last
_error")
      .eq("status", "pending")
      .order("created_at")
      .limit(50),
```

```
  ]);
  const error = emails.error ?? texts.error ?? completions.error ??
reminders.error;
  if (error) throw new Error(`The delivery queues could not be read:
${error.message}`);

  const toDelivery =
    (kind: QueuedDelivery["kind"], queue: QueuedDelivery["queue"], channel?:
DeliveryChannel) =>
    (row: {
      id: string;
      appointment_id: string;
      channel?: string;
      attempt_count: number;
      created_at: string;
      updated_at: string;
      last_error: string | null;
    }): QueuedDelivery => ({
      kind,
      queue,
      id: row.id,
      appointmentId: row.appointment_id,
      channel: channel ?? (row.channel === "email" ? "email" : "sms"),
      attemptCount: row.attempt_count,
      createdAt: row.created_at,
      updatedAt: row.updated_at,
      lastError: row.last_error,
    });
  return [
    ...(emails.data ?? []).map(
      toDelivery("confirmation", "appointment_confirmation_emails", "email"),
    ),
    ...(texts.data ?? []).map(toDelivery("confirmation",
"appointment_confirmation_sms", "sms")),
    ...(completions.data ?? []).map(
      toDelivery("completion", "appointment_completion_notifications"),
    ),
    ...(reminders.data ?? []).map(toDelivery("reminder",
"appointment_reminders")),
  ];
}

/** "confirmation SMS", "reminder email" and so on, for the admin. */
function queuedDeliveryLabel(delivery: QueuedDelivery) {
  return `${delivery.kind} ${delivery.channel === "email" ? "email" : "SMS"}`;
```

```
}

/** Why a queued message is no longer worth sending, or null while it still is. */
function deliveryNoLongerNeeded(
  delivery: QueuedDelivery,
  appointment: DeliveryAppointment | undefined,
) {
  if (!appointment) return "Not sent: the appointment no longer exists.";
  if (delivery.kind === "completion") {
    if (appointment.status !== "completed") return "Not sent: the service is no
longer completed.";
    return Date.parse(delivery.createdAt) < Date.now() - 3 * 24 * 60 * 60_000
      ? "Not sent: the service was completed more than 3 days ago."
      : null;
  }
  if (appointment.status !== "confirmed") {
    return `Not sent: the appointment is now ${appointment.status.replace("_", "
")}.`;
  }
  return appointmentStartMs(appointment.appointment_date,
appointment.start_time) <= Date.now()
    ? "Not sent: the appointment time has passed."
    : null;
}

async function markQueuedDeliverySkipped(delivery: QueuedDelivery, reason:
string) {
  const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
  const { error } = await supabaseAdmin
    .from(delivery.queue)
    .update({
      status: "skipped",
      last_error: reason.slice(0, 500),
      updated_at: new Date().toISOString(),
    })
    .eq("id", delivery.id)
    .eq("status", "pending");
  if (error) console.error("[Scheduled notifications] skipped-state update failed",
error);
}

/** Stops retrying a message and asks the admin to contact the customer
instead. */
async function giveUpDelivery(delivery: QueuedDelivery, appointment:
```

```
DeliveryAppointment) {
  const label = queuedDeliveryLabel(delivery);
  const reason = delivery.lastError
    ? ` The last error was: ${delivery.lastError.slice(0, 200)}`
    : "";
  await markQueuedDeliverySkipped(
    delivery,
    `Gave up after ${delivery.attemptCount} tries.${reason}`,
  );
  const { alertAdmin } = await import("./admin-alerts.server");
  const phone = normalizePhilippineMobile(appointment.phone) ??
appointment.phone;
  await alertAdmin({
    type: "customer_notice_failed",
    appointmentId: delivery.appointmentId,
    title: `${delivery.channel === "sms" ? "Couldn't text" : "Couldn't email"}
${appointment.customer_name}`,
    message: `The ${label} for ${appointment.reference_code} could not be sent
after ${delivery.attemptCount} tries.${reason} Please contact the customer at
${phone}.`,
  });
}

/**
 * The scheduled run: queues reminders that are due, sends them, retries
 * customer messages that failed, and alerts the admin about any that still
 * fail after MAX_DELIVERY_ATTEMPTS tries. siteOrigin is the site's https
 * address, for links in reminder emails.
 */
export async function runScheduledNotifications(siteOrigin: string | null) {
  const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
  await supabaseAdmin
    .from("scheduled_notification_runs")
    .upsert({ id: true, last_started_at: new Date().toISOString() });

  try {
    const queued = await
supabaseAdmin.rpc("queue_due_appointment_reminders");
    if (queued.error) throw new Error(`Reminders could not be queued:
${queued.error.message}`);

    const deliveries = await loadQueuedDeliveries();
    const appointmentIds = [...new Set(deliveries.map((delivery) =>
delivery.appointmentId))];
```

```
  const appointments = new Map<string, DeliveryAppointment>();
  if (appointmentIds.length) {
    const { data, error } = await supabaseAdmin
      .from("appointments")
      .select("id,status,appointment_date,start_time,reference_code,customer_na
me,phone")
      .in("id", appointmentIds);
    if (error) throw new Error(`Appointments could not be read:
${error.message}`);
    for (const appointment of data) appointments.set(appointment.id,
appointment);
  }

  let skipped = 0;
  let gaveUp = 0;
  // Confirmations and completions send both channels together, so one send
per appointment.
  const sends = new Map<string, () => Promise<unknown>>();
  for (const delivery of deliveries) {
    const appointment = appointments.get(delivery.appointmentId);
    const notNeeded = deliveryNoLongerNeeded(delivery, appointment);
    if (notNeeded || !appointment) {
      await markQueuedDeliverySkipped(delivery, notNeeded ?? "Not sent.");
      skipped += 1;
      continue;
    }
    if (delivery.attemptCount >= MAX_DELIVERY_ATTEMPTS) {
      await giveUpDelivery(delivery, appointment);
      gaveUp += 1;
      continue;
    }
    // A new reminder goes out at once. Anything else waits, so a first send
    // still being made by an admin's own request is not repeated.
    const due =
      (delivery.kind === "reminder" && delivery.attemptCount === 0) ||
      retryIsDue(delivery.attemptCount, delivery.updatedAt);
    if (!due) continue;
    if (delivery.kind === "reminder") {
      sends.set(`reminder:${delivery.id}`, () =>
        sendQueuedReminder(delivery.appointmentId, delivery.channel,
siteOrigin),
      );
    } else if (delivery.kind === "confirmation") {
      sends.set(`confirmation:${delivery.appointmentId}`, () =>
        sendConfirmationNotifications(delivery.appointmentId),
```

```
        );
      } else {
        sends.set(`completion:${delivery.appointmentId}`, () =>
          sendCompletionNotifications(delivery.appointmentId),
        );
      }
    }

    const batch = [...sends.values()].slice(0, SCHEDULED_RUN_SEND_LIMIT);
    for (const send of batch) {
      try {
        await send();
      } catch (error) {
        console.error("[Scheduled notifications] send failed", error);
      }
    }

    await supabaseAdmin
      .from("scheduled_notification_runs")
      .update({ last_finished_at: new Date().toISOString(), last_error: null })
      .eq("id", true);
    return { remindersQueued: queued.data ?? 0, sent: batch.length, skipped,
gaveUp };
  } catch (error) {
    const message = error instanceof Error ? error.message : "The scheduled run
failed.";
    console.error("[Scheduled notifications] run failed", error);
    await supabaseAdmin
      .from("scheduled_notification_runs")
      .update({ last_error: message.slice(0, 500) })
      .eq("id", true);
    throw error;
  }
}

async function requireActiveAdminSession() {
  const client = authenticatedSupabaseClient();
  const { data: isActive, error } = await client.rpc("validate_admin_session");
  if (error || !isActive) {
    throw new Error("Your administrator session has expired. Please sign in
again.");
  }
}

/**
```

```
 * A provider outage or a deployment during delivery can leave an
already-approved
 * notification pending. Retrying the durable queue from an active admin session
 * recovers those messages without changing an appointment or exposing the
queue
 * to the browser.
 */
export const retryPendingCustomerNotifications = createServerFn({ method:
"POST" }).handler(
  async () => {
    await requireActiveAdminSession();
    const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
    const [confirmationEmailRows, confirmationSmsRows, completionRows] =
await Promise.all([
      supabaseAdmin
        .from("appointment_confirmation_emails")
        .select("appointment_id")
        .in("status", ["pending", "sending"])
        .limit(25),
      supabaseAdmin
        .from("appointment_confirmation_sms")
        .select("appointment_id")
        .in("status", ["pending", "sending"])
        .limit(25),
      supabaseAdmin
        .from("appointment_completion_notifications")
        .select("appointment_id")
        .in("status", ["pending", "sending"])
        .limit(25),
    ]);
    const queueError =
      confirmationEmailRows.error ?? confirmationSmsRows.error ??
completionRows.error;
    if (queueError) {
      console.error("[Customer notifications] retry queue lookup failed",
queueError);
      return { ok: false as const };
    }

    const confirmationAppointmentIds = new Set([
      ...(confirmationEmailRows.data ?? []).map((delivery) =>
delivery.appointment_id),
      ...(confirmationSmsRows.data ?? []).map((delivery) =>
delivery.appointment_id),
```

```
    ]);
    const completionAppointmentIds = new Set(
      (completionRows.data ?? []).map((delivery) => delivery.appointment_id),
    );
    await Promise.allSettled([
      ...Array.from(confirmationAppointmentIds, sendConfirmationNotifications),
      ...Array.from(completionAppointmentIds, sendCompletionNotifications),
    ]);
    const attempted = confirmationAppointmentIds.size +
completionAppointmentIds.size;
    if (attempted) {
      console.info("[Customer notifications] retried durable deliveries", {
        confirmationAppointments: confirmationAppointmentIds.size,
        completionAppointments: completionAppointmentIds.size,
      });
    }
    return { ok: true as const, attempted };
  },
);

export const updateAppointmentServiceProgressWithNotifications =
createServerFn({ method: "POST" })
  .validator((input: unknown) => serviceProgressSchema.parse(input))
  .handler(async ({ data }) => {
    const client = authenticatedSupabaseClient();
    const { error } = await client.rpc("manage_appointment_service_progress", {
      p_appointment_id: data.appointmentId,
      p_action: data.action,
    });
    if (error) return { ok: false as const, error: error.message };
    if (data.action !== "complete") return { ok: true as const };

    const { emailDelivery, smsDelivery } = await
sendCompletionNotifications(data.appointmentId);
    return {
      ok: true as const,
      emailSent: emailDelivery.sent,
      emailError: emailDelivery.sent ? null : emailDelivery.error,
      smsSent: smsDelivery.sent,
      smsError: smsDelivery.sent ? null : smsDelivery.error,
    };
  });

export const reviewPendingAppointmentWithConfirmation = createServerFn({
method: "POST" })
```

```
  .validator((input: unknown) => reviewSchema.parse(input))
  .handler(async ({ data }) => {
    const client = authenticatedSupabaseClient();

    const { error } = await client.rpc("review_pending_appointment", {
      p_appointment_id: data.appointmentId,
      p_decision: data.decision,
    });
    if (error) return { ok: false as const, error: error.message };
    if (data.decision === "rejected") {
      const { emailDelivery, smsDelivery } = await
notifyBookingNotAccepted(data.appointmentId);
      return {
        ok: true as const,
        emailSent: emailDelivery.sent,
        emailError: emailDelivery.error,
        smsSent: smsDelivery.sent,
        smsError: smsDelivery.error,
      };
    }

    const { emailDelivery, smsDelivery } = await
sendConfirmationNotifications(data.appointmentId);
    return {
      ok: true as const,
      emailSent: emailDelivery.sent,
      emailError: emailDelivery.sent ? null : emailDelivery.error,
      smsSent: smsDelivery.sent,
      smsError: smsDelivery.sent ? null : smsDelivery.error,
    };
  });

export const reviewRescheduleRequestWithConfirmation = createServerFn({
method: "POST" })
  .validator((input: unknown) => reviewSchema.parse(input))
  .handler(async ({ data }) => {
    const client = authenticatedSupabaseClient();

    const { error } = await client.rpc("review_reschedule_request", {
      p_appointment_id: data.appointmentId,
      p_decision: data.decision,
    });
    if (error) return { ok: false as const, error: error.message };
    if (data.decision === "rejected") {
      const { emailDelivery, smsDelivery } = await
```

```
notifyRescheduleNotApproved(data.appointmentId);
      return {
        ok: true as const,
        emailSent: emailDelivery.sent,
        emailError: emailDelivery.error,
        smsSent: smsDelivery.sent,
        smsError: smsDelivery.error,
      };
    }

    const { data: replacement, error: replacementError } = await client
      .from("appointments")
      .select("id")
      .eq("rescheduled_from_appointment_id", data.appointmentId)
      .eq("status", "confirmed")
      .maybeSingle();
    if (replacementError || !replacement) {
      return {
        ok: true as const,
        emailSent: false,
        emailError:
          "The reschedule was approved, but its confirmation email could not be
prepared.",
        smsSent: false,
        smsError: "The reschedule was approved, but its confirmation SMS could
not be prepared.",
      };
    }

    const { emailDelivery, smsDelivery } = await
sendConfirmationNotifications(replacement.id);
    return {
      ok: true as const,
      emailSent: emailDelivery.sent,
      emailError: emailDelivery.sent ? null : emailDelivery.error,
      smsSent: smsDelivery.sent,
      smsError: smsDelivery.sent ? null : smsDelivery.error,
    };
  });

export const updateAppointmentWithConfirmation = createServerFn({ method:
"POST" })
  .validator((input: unknown) => appointmentUpdateSchema.parse(input))
  .handler(async ({ data }) => {
    const client = authenticatedSupabaseClient();
```

```
    const { data: existingAppointment, error: existingAppointmentError } = await
client
      .from("appointments")
      .select("status")
      .eq("id", data.appointmentId)
      .maybeSingle();
    if (existingAppointmentError || !existingAppointment) {
      return { ok: false as const, error: "This appointment is no longer available." };
    }

    const isBeingConfirmed =
      existingAppointment.status !== "confirmed" && data.status === "confirmed";
    const isBeingCompleted =
      existingAppointment.status !== "completed" && data.status === "completed";
    const isBeingRejected = existingAppointment.status === "pending" &&
data.status === "rejected";
    const { error } = await
client.rpc("update_appointment_financial_details_atomic", {
      p_appointment_id: data.appointmentId,
      p_service_ids: data.serviceIds,
      p_appointment_date: data.appointmentDate,
      p_start_time: data.startTime,
      p_assigned_crew_id: data.assignedCrewId,
      p_crew_assignment_manual: data.crewAssignmentManual,
      p_status: data.status,
      p_admin_notes: data.adminNotes,
      p_first_name: data.firstName,
      p_middle_name: data.middleName,
      p_last_name: data.lastName,
      p_phone: data.phone,
      p_products: data.products.map((product) => ({
        product_id: product.productId,
        quantity: product.quantity,
        unit_price: product.unitPrice,
      })),
    });
    if (error) return { ok: false as const, error: error.message };
    if (isBeingRejected) {
      const { emailDelivery, smsDelivery } = await
notifyBookingNotAccepted(data.appointmentId);
      return {
        ok: true as const,
        emailSent: emailDelivery.sent,
        emailError: emailDelivery.error,
        smsSent: smsDelivery.sent,
```

```
        smsError: smsDelivery.error,
      };
    }
    if (!isBeingConfirmed && !isBeingCompleted) return { ok: true as const,
emailSent: false };

    const { emailDelivery, smsDelivery } = isBeingCompleted
      ? await sendCompletionNotifications(data.appointmentId)
      : await sendConfirmationNotifications(data.appointmentId);
    return {
      ok: true as const,
      emailSent: emailDelivery.sent,
      emailError: emailDelivery.sent ? null : emailDelivery.error,
      smsSent: smsDelivery.sent,
      smsError: smsDelivery.sent ? null : smsDelivery.error,
    };
  });
```

src/lib/appointment-reference-recovery.functions.ts

```
import { createServerFn } from "@tanstack/react-start";
import { getRequest } from "@tanstack/react-start/server";
import { z } from "zod";

import type { EmailTransport } from "./email.server";

const recoveryEmailSchema = z
  .string()
  .trim()
  .email("Enter a valid email address.")
  .max(254)
  .transform((email) => email.toLocaleLowerCase());

const requestRecoverySchema = z.object({ email: recoveryEmailSchema });
const verifyRecoverySchema = z.object({
  email: recoveryEmailSchema,
  code: z
    .string()
    .trim()
    .regex(/^\d{6}$/, "Enter the 6-digit verification code."),
});

type RateLimitResult = "allowed" | "limited" | "unavailable";

/** The configured mail transport, or null (with the reason logged) when email is
not set up. */
async function recoveryEmailTransport(): Promise<EmailTransport | null> {
  const { emailTransport } = await import("./email.server");
  const configured = emailTransport();
  if ("error" in configured) {
    console.error("[Reference recovery] email configuration unavailable", {
      reason: configured.error,
    });
    return null;
  }
  return configured.transport;
}

function clientIp() {
  const request = getRequest();
  const platformIp =
    request?.headers.get("x-vercel-forwarded-for")?.trim() ||
    request?.headers.get("cf-connecting-ip")?.trim();
```

```
  return (
    platformIp ||
    request?.headers.get("x-forwarded-for")?.split(",")[0]?.trim() ||
    request?.headers.get("x-real-ip")?.trim() ||
    "unknown"
  );
}

async function sha256(value: string) {
  const digest = await crypto.subtle.digest("SHA-256", new
TextEncoder().encode(value));
  return Array.from(new Uint8Array(digest), (byte) =>
byte.toString(16).padStart(2, "0")).join("");
}

async function checkRateLimit(
  scope: "reference_recovery_email_request" |
"reference_recovery_email_verify",
  email: string,
  limit: number,
): Promise<RateLimitResult> {
  try {
    const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
    const subjects = [await sha256(`ip:${clientIp()}`), await
sha256(`email:${email}`)];
    for (const subject of subjects) {
      const { data, error } = await supabaseAdmin.rpc("enforce_public_rate_limit", {
        p_scope: scope,
        p_subject: subject,
        p_limit: limit,
        p_window_seconds: 15 * 60,
      });
      if (error) {
        console.error(`[Reference recovery] ${scope} rate limit failed`, error);
        return "unavailable";
      }
      if (data !== true) return "limited";
    }
    return "allowed";
  } catch (error) {
    console.error(`[Reference recovery] ${scope} rate limit failed`, error);
    return "unavailable";
  }
}
```

```
function makeVerificationCode() {
  const random = new Uint32Array(1);
  const range = 1_000_000;
  const unbiasedUpperBound = Math.floor(0x1_0000_0000 / range) * range;
  let value: number;
  do {
    crypto.getRandomValues(random);
    value = random[0] ?? 0;
  } while (value >= unbiasedUpperBound);

  const code = value % range;
  return String(code).padStart(6, "0");
}

async function sendEmail(
  transport: EmailTransport,
  recipient: string,
  subject: string,
  text: string,
) {
  const { sendEmail: sendWithTransport } = await import("./email.server");
  await sendWithTransport(transport, { to: recipient, subject, text });
}

const sentCodeMessage =
  "If an active booking uses that email address, we sent a verification code.";

export const requestReferenceRecoveryCode = createServerFn({ method:
"POST" })
  .validator((input: unknown) => requestRecoverySchema.parse(input))
  .handler(async ({ data }) => {
    const rateLimit = await checkRateLimit("reference_recovery_email_request",
data.email, 3);
    if (rateLimit !== "allowed") {
      return {
        ok: false as const,
        error:
          rateLimit === "limited"
            ? "Too many verification requests. Please wait a few minutes before
trying again."
            : "Reference recovery is temporarily unavailable. Please try again
shortly.",
      };
    }
```

```
    const config = await recoveryEmailTransport();
    if (!config) {
      return {
        ok: false as const,
        error: "Reference recovery email is not configured. Please contact the
shop.",
      };
    }

    const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
    const { data: appointment, error: appointmentError } = await supabaseAdmin
      .from("appointments")
      .select("id")
      .eq("email", data.email)
      .eq("is_archived", false)
      .limit(1)
      .maybeSingle();
    if (appointmentError) {
      console.error("[Reference recovery] email lookup failed", appointmentError);
      return {
        ok: false as const,
        error: "Reference recovery is temporarily unavailable. Please try again
shortly.",
      };
    }
    if (!appointment) return { ok: true as const, message: sentCodeMessage };

    const verificationCode = makeVerificationCode();
    const { data: challenge, error: challengeError } = await supabaseAdmin
      .from("appointment_reference_recovery_challenges")
      .insert({
        email: data.email,
        code_hash: await sha256(verificationCode),
        expires_at: new Date(Date.now() + 10 * 60 * 1000).toISOString(),
      })
      .select("id")
      .single();
    if (challengeError || !challenge) {
      console.error("[Reference recovery] challenge creation failed",
challengeError);
      return {
        ok: false as const,
        error: "Reference recovery is temporarily unavailable. Please try again
```

```
shortly.",
      };
    }

    try {
      await sendEmail(
        config,
        data.email,
        "Your reference recovery verification code",
        `Your Fake Rider Motorparts verification code is ${verificationCode}. It
expires in 10 minutes. If you did not request this, you can ignore this email.`,
      );
    } catch (error) {
      console.error("[Reference recovery] verification email failed", error);
      await supabaseAdmin
        .from("appointment_reference_recovery_challenges")
        .delete()
        .eq("id", challenge.id);
      return {
        ok: false as const,
        error: "We could not send the verification email. Please try again shortly.",
      };
    }

    return { ok: true as const, message: sentCodeMessage };
  });

export const verifyReferenceRecoveryCode = createServerFn({ method: "POST"
})
  .validator((input: unknown) => verifyRecoverySchema.parse(input))
  .handler(async ({ data }) => {
    const rateLimit = await checkRateLimit("reference_recovery_email_verify",
data.email, 8);
    if (rateLimit !== "allowed") {
      return {
        ok: false as const,
        error:
          rateLimit === "limited"
            ? "Too many verification attempts. Please request a new code in a few
minutes."
            : "Reference recovery is temporarily unavailable. Please try again
shortly.",
      };
    }
```

```
    const config = await recoveryEmailTransport();
    if (!config) {
      return {
        ok: false as const,
        error: "Reference recovery email is not configured. Please contact the
shop.",
      };
    }

    const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
    const { data: outcome, error: verificationError } = await supabaseAdmin.rpc(
      "consume_reference_recovery_challenge",
      { p_email: data.email, p_code_hash: await sha256(data.code) },
    );
    if (verificationError) {
      console.error("[Reference recovery] code verification failed",
verificationError);
      return {
        ok: false as const,
        error: "Reference recovery is temporarily unavailable. Please try again
shortly.",
      };
    }
    if (outcome !== "verified") {
      return {
        ok: false as const,
        error:
          outcome === "invalid"
            ? "That verification code is incorrect. Please try again."
            : "That verification code has expired or is no longer valid. Request a new
code.",
      };
    }

    const { data: appointments, error: appointmentsError } = await supabaseAdmin
      .from("appointments")
      .select("reference_code")
      .eq("email", data.email)
      .eq("is_archived", false)
      .order("created_at", { ascending: false })
      .limit(20);
    if (appointmentsError) {
      console.error("[Reference recovery] reference lookup failed",
appointmentsError);
```

```
      return {
        ok: false as const,
        error: "We could not retrieve your reference code. Please request a new
verification code.",
      };
    }
    const references = Array.from(
      new Set((appointments ?? []).map((appointment) =>
appointment.reference_code)),
    );
    if (references.length === 0) {
      return {
        ok: false as const,
        error: "No active appointments were found for that verified email address.",
      };
    }

    try {
      await sendEmail(
        config,
        data.email,
        "Your Fake Rider Motorparts reference code",
        `Your appointment reference code${references.length === 1 ? " is" : "s
are"}:\n\n${references.map((reference) => `- ${reference}`).join("\n")}\n\nKeep
${references.length === 1 ? "this code" : "these codes"} for appointment
tracking.`,
      );
    } catch (error) {
      console.error("[Reference recovery] reference email failed", error);
      return {
        ok: false as const,
        error: "We could not email your reference code. Please request a new
verification code.",
      };
    }

    return {
      ok: true as const,
      message: "Your reference code has been sent to your verified email
address.",
    };
  });
```

src/lib/appointment-verification.server.ts

```
// Email one-time codes that protect online cancellation and rescheduling.
// A correct code issues a short-lived token bound to one appointment and one
// action. Codes and tokens are only ever stored as SHA-256 hashes.
import { supabaseAdmin } from "@/integrations/supabase/client.server";

import { emailTransport, sendEmail } from "./email.server";
import { formatDateLong, formatTime } from "./shop";

export type AppointmentAction = "cancel" | "reschedule";

export const VERIFICATION_CODE_TTL_MINUTES = 10;
/** Long enough to choose a new schedule after verifying, short enough to limit
reuse. */
export const VERIFICATION_TOKEN_TTL_MINUTES = 30;

export const VERIFICATION_TOKEN_PATTERN = /^[A-Za-z0-9_-]{43}$/;

async function sha256(value: string) {
  const digest = await crypto.subtle.digest("SHA-256", new
TextEncoder().encode(value));
  return Array.from(new Uint8Array(digest), (byte) =>
byte.toString(16).padStart(2, "0")).join("");
}

function makeVerificationCode() {
  const random = new Uint32Array(1);
  const range = 1_000_000;
  const unbiasedUpperBound = Math.floor(0x1_0000_0000 / range) * range;
  let value: number;
  do {
    crypto.getRandomValues(random);
    value = random[0] ?? 0;
  } while (value >= unbiasedUpperBound);

  return String(value % range).padStart(6, "0");
}

/** 32 random bytes as unpadded base64url (43 characters). */
function makeVerificationToken() {
  const bytes = crypto.getRandomValues(new Uint8Array(32));
  return Buffer.from(bytes).toString("base64url");
}
```

```
export async function issueActionCode(
  appointmentId: string,
  action: AppointmentAction,
): Promise<{ code: string; challengeId: string } | null> {
  const now = new Date().toISOString();
  // Keep the table small: expired codes and expired tokens are never used again.
  const cleanup = await supabaseAdmin
    .from("appointment_action_verifications")
    .delete()
    .eq("appointment_id", appointmentId)
    .eq("action", action)
    .lt("expires_at", now)
    .or(`token_expires_at.is.null,token_expires_at.lt."${now}"`);
  if (cleanup.error) {
    console.error("[Appointment verification] cleanup failed", cleanup.error);
  }

  const code = makeVerificationCode();
  const { data, error } = await supabaseAdmin
    .from("appointment_action_verifications")
    .insert({
      appointment_id: appointmentId,
      action,
      code_hash: await sha256(code),
      expires_at: new Date(Date.now() + VERIFICATION_CODE_TTL_MINUTES *
60_000).toISOString(),
    })
    .select("id")
    .single();
  if (error || !data) {
    console.error("[Appointment verification] code creation failed", error);
    return null;
  }
  return { code, challengeId: data.id };
}

export async function discardActionCode(challengeId: string) {
  const { error } = await supabaseAdmin
    .from("appointment_action_verifications")
    .delete()
    .eq("id", challengeId);
  if (error) console.error("[Appointment verification] code cleanup failed", error);
}

export type VerifyActionCodeResult =
```

```
  | { outcome: "verified"; token: string; expiresAt: string }
  | { outcome: "invalid" | "expired" | "unavailable" };

export async function verifyActionCode(
  appointmentId: string,
  action: AppointmentAction,
  code: string,
): Promise<VerifyActionCodeResult> {
  const token = makeVerificationToken();
  const { data, error } = await
supabaseAdmin.rpc("verify_appointment_action_code", {
    p_appointment_id: appointmentId,
    p_action: action,
    p_code_hash: await sha256(code),
    p_token_hash: await sha256(token),
    p_token_ttl_seconds: VERIFICATION_TOKEN_TTL_MINUTES * 60,
  });
  if (error) {
    console.error("[Appointment verification] code verification failed", error);
    return { outcome: "unavailable" };
  }
  if (data === "verified") {
    return {
      outcome: "verified",
      token,
      // The database sets the authoritative expiry; this copy only lets the
      // browser ask for a fresh code before sending a request it would reject.
      expiresAt: new Date(Date.now() + VERIFICATION_TOKEN_TTL_MINUTES
* 60_000).toISOString(),
    };
  }
  return { outcome: data === "invalid" ? "invalid" : "expired" };
}

export async function checkActionToken(
  appointmentId: string,
  action: AppointmentAction,
  token: string | undefined,
): Promise<"valid" | "invalid" | "unavailable"> {
  if (!token || !VERIFICATION_TOKEN_PATTERN.test(token)) return "invalid";
  const { data, error } = await supabaseAdmin
    .from("appointment_action_verifications")
    .select("id")
    .eq("token_hash", await sha256(token))
    .eq("appointment_id", appointmentId)
```

```
    .eq("action", action)
    .gt("token_expires_at", new Date().toISOString())
    .maybeSingle();
  if (error) {
    console.error("[Appointment verification] token check failed", error);
    return "unavailable";
  }
  return data ? "valid" : "invalid";
}

export async function sendActionCodeEmail(input: {
  to: string;
  code: string;
  action: AppointmentAction;
  reference: string;
  date: string;
  startTime: string;
}) {
  const configured = emailTransport();
  if ("error" in configured) {
    console.error("[Appointment verification] email configuration unavailable", {
      reason: configured.error,
    });
    throw new Error("Email is not configured.");
  }

  const actionText = input.action === "cancel" ? "cancel" : "reschedule";
  const schedule = `${formatDateLong(input.date)} at
${formatTime(input.startTime)}`;
  await sendEmail(configured.transport, {
    to: input.to,
    subject: `Your code to ${actionText} appointment ${input.reference}`,
    text: [
      `Your Fake Rider Motorparts verification code is ${input.code}.`,
      "",
      `Use it to ${actionText} appointment ${input.reference} on ${schedule}. It
expires in ${VERIFICATION_CODE_TTL_MINUTES} minutes.`,
      "",
      "If you did not request this, you can ignore this email. Your appointment has
not been changed.",
    ].join("\n"),
  });
}
```

src/lib/availability.ts

```
import {
  addDays,
  intervalsOverlap,
  isBookingTimeRangeWithinHours,
  isShopOpenDate,
  isSlotBookable,
  timeToMinutes,
  type BookingHours,
} from "./shop";

export const ACTIVE_RESERVATION_STATUSES = [
  "pending",
  "confirmed",
  "in_progress",
  "rescheduled",
] as const;

const activeReservationStatusSet = new
Set<string>(ACTIVE_RESERVATION_STATUSES);

export function isActiveReservationStatus(status: string | null | undefined):
boolean {
  return Boolean(status && activeReservationStatusSet.has(status));
}

export type TimeSlot = {
  id: string;
  startTime: string;
  endTime: string;
  capacity: number;
};

export type DateBlock = {
  date: string;
  startTime: string | null;
  endTime: string | null;
  reason: string | null;
};

export type CrewAvailabilitySchedule = {
  crew_id: string;
  start_time: string | null;
  end_time: string | null;
```

```
  is_working: boolean;
  schedule_date: string | null;
};

export type CrewException = {
  crew_id: string;
  start_date: string;
  end_date: string;
  start_time: string | null;
  end_time: string | null;
  is_all_day: boolean;
};

export type Assignment = {
  date: string;
  startTime: string;
  durationMinutes: number;
  crewId: string | null;
};

export type CrewAvailabilityResult = {
  availableCrewIds: string[];
  unassignedReservationCount: number;
  remainingCapacity: number;
};

/**
 * Finds the crew who can perform one reservation and how many more
 * reservations fit. Two limits apply, as in the database's final check: the
 * time slot's capacity counts every overlapping reservation, and each
 * reservation needs its own free mechanic (unassigned reservations claim one
 * each). All callers must pass only active crew schedules; this keeps public
 * availability, final booking, and continuations on the same staffing rules.
 */
export function evaluateCrewAvailability({
  date,
  startTime,
  durationMinutes,
  capacity,
  schedules,
  exceptions,
  assignments,
}: {
  date: string;
  startTime: string;
```

```
  durationMinutes: number;
  capacity: number;
  schedules: CrewAvailabilitySchedule[];
  exceptions: CrewException[];
  assignments: Assignment[];
}): CrewAvailabilityResult {
  const startMinutes = timeToMinutes(startTime);
  const endMinutes = startMinutes + durationMinutes;
  if (
    !Number.isFinite(startMinutes) ||
    !Number.isFinite(durationMinutes) ||
    durationMinutes <= 0 ||
    !Number.isFinite(endMinutes)
  ) {
    return { availableCrewIds: [], unassignedReservationCount: 0,
remainingCapacity: 0 };
  }

  const schedulesByCrew = new Map<string, CrewAvailabilitySchedule[]>();
  for (const schedule of schedules) {
    if (
      schedule.schedule_date !== date ||
      !schedule.is_working ||
      !schedule.start_time ||
      !schedule.end_time
    ) {
      continue;
    }
    const crewSchedules = schedulesByCrew.get(schedule.crew_id) ?? [];
    crewSchedules.push(schedule);
    schedulesByCrew.set(schedule.crew_id, crewSchedules);
  }

  const overlappingAssignments = assignments.filter((assignment) => {
    if (assignment.date !== date) return false;
    const assignmentStart = timeToMinutes(assignment.startTime);
    return (
      Number.isFinite(assignmentStart) &&
      Number.isFinite(assignment.durationMinutes) &&
      assignment.durationMinutes > 0 &&
      intervalsOverlap(
        startMinutes,
        endMinutes,
        assignmentStart,
        assignmentStart + assignment.durationMinutes,
```

```
      )
    );
  });
  const occupiedCrewIds = new Set(
    overlappingAssignments.flatMap((assignment) => (assignment.crewId ?
[assignment.crewId] : [])),
  );
  const unassignedReservationCount = overlappingAssignments.filter(
    (assignment) => !assignment.crewId,
  ).length;

  const availableCrewIds = [...schedulesByCrew.entries()]
    .sort(([left], [right]) => left.localeCompare(right))
    .flatMap(([crewId, crewSchedules]) => {
      if (occupiedCrewIds.has(crewId)) return [];

      const shiftCoversReservation = crewSchedules.some((schedule) => {
        const shiftStart = timeToMinutes(String(schedule.start_time).slice(0, 5));
        const shiftEnd = timeToMinutes(String(schedule.end_time).slice(0, 5));
        return (
          Number.isFinite(shiftStart) &&
          Number.isFinite(shiftEnd) &&
          startMinutes >= shiftStart &&
          endMinutes <= shiftEnd
        );
      });
      if (!shiftCoversReservation) return [];

      const hasException = exceptions.some((exception) => {
        if (
          exception.crew_id !== crewId ||
          exception.start_date > date ||
          exception.end_date < date
        ) {
          return false;
        }
        if (exception.is_all_day) return true;
        if (!exception.start_time || !exception.end_time) return false;
        return intervalsOverlap(
          startMinutes,
          endMinutes,
          timeToMinutes(String(exception.start_time).slice(0, 5)),
          timeToMinutes(String(exception.end_time).slice(0, 5)),
        );
      });
```

```
      return hasException ? [] : [crewId];
    });

  const normalizedCapacity = Number.isInteger(capacity) && capacity > 0 ?
capacity : 0;
  return {
    availableCrewIds,
    unassignedReservationCount,
    remainingCapacity: Math.max(
      0,
      Math.min(
        normalizedCapacity - overlappingAssignments.length,
        availableCrewIds.length - unassignedReservationCount,
      ),
    ),
  };
}

/**
 * The free mechanic with the fewest bookings that day; ties go to the one with
 * fewer booked minutes. Any free mechanic may be chosen: the capacity check
 * already leaves enough free mechanics for reservations not yet assigned one.
 */
export function pickLeastBusyCrew(
  crewIds: string[],
  assignments: Assignment[],
  date: string,
): string | undefined {
  const workload = new Map(crewIds.map((crewId) => [crewId, { bookings: 0,
minutes: 0 }]));
  for (const assignment of assignments) {
    if (assignment.date !== date || !assignment.crewId) continue;
    const load = workload.get(assignment.crewId);
    if (!load) continue;
    load.bookings += 1;
    load.minutes += assignment.durationMinutes;
  }
  return [...crewIds].sort((left, right) => {
    const a = workload.get(left)!;
    const b = workload.get(right)!;
    return a.bookings - b.bookings || a.minutes - b.minutes ||
left.localeCompare(right);
  })[0];
}
```

```ts
/** Internal scheduling data. It must stay on the server. */
export type AvailabilitySource = {
  from: string;
  to: string;
  operatingHours?: BookingHours;
  minimumBookingLeadHours?: number;
  totalDurationMinutes?: number;
  allowMultiDayContinuation?: boolean;
  slots: TimeSlot[];
  blocks: DateBlock[];
  assignments: Assignment[];
  schedules: CrewAvailabilitySchedule[];
  exceptions: CrewException[];
};

/** A slot shape that is safe to send to a public booking page. */
export type ComputedSlot = {
  id: string;
  startTime: string;
  endTime: string;
  remaining: number;
  disabled: boolean;
  notBookable: boolean;
  recommended: boolean;
  /**
   * Whether the shop normally takes this service at this time. False for hours
   * without capacity (such as lunch) and for times too soon, or too late for
   * the service to finish. Pages hide these and grey out the other disabled
   * times, which are taken or blocked.
   */
  offered: boolean;
};

type InternalComputedSlot = ComputedSlot & { capacity: number };

export type ContinuationSegment = {
  appointmentDate: string;
  startTime: string;
  durationMinutes: number;
  crewId: string;
};

/** The minimal availability payload exposed to public booking pages. */
export type Availability = {
  from: string;
```

```
  to: string;
  error?: string;
  totalDurationMinutes?: number;
  operatingHours?: BookingHours;
  dates: string[];
  fullyBookedDates: string[];
  slotsByDate: Record<string, ComputedSlot[]>;
  serviceEstimates?: Array<{
    id: string;
    name: string;
    price: number;
    durationMinutes: number;
    pricingSource: "small_bike" | "large_bike";
  }>;
};

function isNonSundayUnblockedDate(source: AvailabilitySource, date: string) {
  const [y, m, d] = date.split("-").map(Number);
  const dayOfWeek = new Date(Date.UTC(y ?? 1970, (m ?? 1) - 1, d ??
1)).getUTCDay();
  const fullDayBlocked = source.blocks.some((block) => block.date === date &&
!block.startTime);
  return dayOfWeek !== 0 && !fullDayBlocked;
}

function hasOverlappingBlock(
  blocks: DateBlock[],
  date: string,
  startTime: string,
  durationMinutes: number,
) {
  const startMinutes = timeToMinutes(startTime);
  const endMinutes = startMinutes + durationMinutes;
  return blocks.some((block) => {
    if (block.date !== date) return false;
    if (!block.startTime) return true;
    if (block.startTime === startTime) return true;
    if (!block.endTime) return false;
    return intervalsOverlap(
      startMinutes,
      endMinutes,
      timeToMinutes(block.startTime),
      timeToMinutes(block.endTime),
    );
  });
```

```
}

/**
 * Reserves the remaining work on later days using the same staffing and
 * capacity calculation as the public availability grid. A null result means
 * the full multi-day reservation cannot be completed in the displayed window.
 */
export function planContinuationSegments({
  source,
  firstDate,
  remainingMinutes,
}: {
  source: AvailabilitySource;
  firstDate: string;
  remainingMinutes: number;
}): ContinuationSegment[] | null {
  if (remainingMinutes <= 0) return [];

  const closingMinutes = timeToMinutes(source.operatingHours?.closingTime ??
"17:00");
  if (!Number.isFinite(closingMinutes)) return null;

  const assignments = [...source.assignments];
  const segments: ContinuationSegment[] = [];
  let remaining = remainingMinutes;
  let cursor = addDays(firstDate, 1);

  while (remaining > 0 && cursor <= source.to) {
    if (!isShopOpenDate(cursor)) {
      cursor = addDays(cursor, 1);
      continue;
    }

    for (const slot of source.slots) {
      const slotStart = timeToMinutes(slot.startTime);
      const duration = Math.min(remaining, closingMinutes - slotStart);
      if (
        duration <= 0 ||
        slot.capacity <= 0 ||
        !isBookingTimeRangeWithinHours(slot.startTime, duration,
source.operatingHours) ||
        hasOverlappingBlock(source.blocks, cursor, slot.startTime, duration)
      ) {
        continue;
      }
```

```
      const { availableCrewIds, remainingCapacity } = evaluateCrewAvailability({
        date: cursor,
        startTime: slot.startTime,
        durationMinutes: duration,
        capacity: slot.capacity,
        schedules: source.schedules,
        exceptions: source.exceptions,
        assignments,
      });
      const crewId = pickLeastBusyCrew(availableCrewIds, assignments, cursor);
      if (remainingCapacity <= 0 || !crewId) continue;

      const segment: ContinuationSegment = {
        appointmentDate: cursor,
        startTime: slot.startTime,
        durationMinutes: duration,
        crewId,
      };
      segments.push(segment);
      assignments.push({
        date: segment.appointmentDate,
        startTime: segment.startTime,
        durationMinutes: segment.durationMinutes,
        crewId: segment.crewId,
      });
      remaining -= duration;
      break;
    }
    cursor = addDays(cursor, 1);
  }

  return remaining === 0 ? segments : null;
}

/** Compute availability without exposing its staffing and booking inputs. */
export function buildPublicAvailability(source: AvailabilitySource): Availability {
  const out: string[] = [];
  const fullyBookedDates: string[] = [];
  const slotsByDate: Record<string, ComputedSlot[]> = {};
  let cursor = source.from;

  while (cursor <= source.to) {
    const slots = computeAvailableSlotsFromSource(source, cursor);
    slotsByDate[cursor] = slots.map(({ capacity: _capacity, ...slot }) => slot);
```

```
    if (isNonSundayUnblockedDate(source, cursor)) {
      const hasOpenSlot = slots.some((slot) => !slot.disabled);
      if (hasOpenSlot) out.push(cursor);

      const hasCapacityFreeFromBlocks = slots.some(
        (slot) => !slot.notBookable && slot.remaining > 0,
      );
      if (!hasOpenSlot && !hasCapacityFreeFromBlocks && slots.length > 0) {
        fullyBookedDates.push(cursor);
      }
    }
    cursor = addDays(cursor, 1);
  }

  return {
    from: source.from,
    to: source.to,
    ...(source.totalDurationMinutes === undefined
      ? {}
      : { totalDurationMinutes: source.totalDurationMinutes }),
    ...(source.operatingHours === undefined ? {} : { operatingHours:
source.operatingHours }),
    dates: out,
    fullyBookedDates,
    slotsByDate,
  };
}

/** Compute the per-slot availability for one date from private scheduling data. */
function computeAvailableSlotsFromSource(
  availability: AvailabilitySource,
  date: string,
): InternalComputedSlot[] {
  if (!date || !availability.slots?.length) return [];

  const totalDuration = availability.totalDurationMinutes ?? 60;
  const computedSlots = availability.slots.map((slot) => {
    const slotStartMin =
      parseInt(slot.startTime.slice(0, 2)) * 60 + parseInt(slot.startTime.slice(3, 5));
    const configuredClosingMinutes = timeToMinutes(
      availability.operatingHours?.closingTime ?? "17:00",
    );
    const continuationEligible =
      availability.allowMultiDayContinuation &&
```

```
      slotStartMin + totalDuration > configuredClosingMinutes;
    const slotDuration = continuationEligible
      ? Math.min(totalDuration, Math.max(0, configuredClosingMinutes -
slotStartMin))
      : totalDuration;

    const notBookable =
      !isSlotBookable(date, slot.startTime, availability.minimumBookingLeadHours)
||
      (!continuationEligible &&
        !isBookingTimeRangeWithinHours(slot.startTime, totalDuration,
availability.operatingHours));

    const blocked = hasOverlappingBlock(availability.blocks, date, slot.startTime,
slotDuration);

    const { remainingCapacity: remaining } = evaluateCrewAvailability({
      date,
      startTime: slot.startTime,
      durationMinutes: slotDuration,
      capacity: slot.capacity,
      schedules: availability.schedules,
      exceptions: availability.exceptions,
      assignments: availability.assignments,
    });
    const continuationPlan = continuationEligible
      ? planContinuationSegments({
          source: availability,
          firstDate: date,
          remainingMinutes: totalDuration - slotDuration,
        })
      : [];
    const disabled = notBookable || blocked || remaining === 0 || continuationPlan
=== null;

    return {
      ...slot,
      remaining,
      disabled,
      notBookable,
      recommended: false,
      offered: slot.capacity > 0 && !notBookable,
    };
  });
```

```
  const bestRemaining = Math.max(
    0,
    ...computedSlots.filter((slot) => !slot.disabled).map((slot) => slot.remaining),
  );
  let recommendedAssigned = false;
  return computedSlots.map((slot) => {
    const recommended =
      !recommendedAssigned &&
      !slot.disabled &&
      slot.remaining === bestRemaining &&
      bestRemaining > 0;
    if (recommended) recommendedAssigned = true;
    return { ...slot, recommended };
  });
}

export function computeAvailableDates(availability: Availability): string[] {
  return availability.dates;
}

export function computeAvailableSlots(availability: Availability, date: string):
ComputedSlot[] {
  return availability.slotsByDate[date] ?? [];
}

export function computeFullyBookedDates(availability: Availability): string[] {
  return availability.fullyBookedDates;
}
```

src/lib/booking-draft.ts

```
// An unfinished booking form, kept in this browser tab so a customer who looks
// at another page can come back to it. sessionStorage is cleared when the tab
// closes, and a draft is dropped 10 minutes after its last change.

const DRAFT_KEY = "fake-rider-booking-draft";
export const BOOKING_DRAFT_LIFETIME_MINUTES = 10;

export function loadBookingDraft<T>(parse: (value: unknown) => T | null): T | null
{
  try {
    const raw = sessionStorage.getItem(DRAFT_KEY);
    if (!raw) return null;
    const saved = JSON.parse(raw) as { savedAt?: unknown; draft?: unknown };
    const age = Date.now() - Number(saved.savedAt);
    if (!(age >= 0 && age <= BOOKING_DRAFT_LIFETIME_MINUTES * 60_000))
{
      sessionStorage.removeItem(DRAFT_KEY);
      return null;
    }
    return parse(saved.draft);
  } catch {
    return null;
  }
}

export function saveBookingDraft(draft: unknown) {
  try {
    sessionStorage.setItem(DRAFT_KEY, JSON.stringify({ savedAt: Date.now(),
draft }));
  } catch {
    // Storage can be full or blocked; the form still works without a draft.
  }
}

export function clearBookingDraft() {
  try {
    sessionStorage.removeItem(DRAFT_KEY);
  } catch {
    // Nothing to clear when storage is blocked.
  }
}
```

src/lib/booking.functions.ts

```
import { createServerFn } from "@tanstack/react-start";
import { getRequest } from "@tanstack/react-start/server";
import type { SupabaseClient } from "@supabase/supabase-js";
import { z } from "zod";

import type { Database } from "@/integrations/supabase/types";
import {
  ACTIVE_RESERVATION_STATUSES,
  buildPublicAvailability,
  evaluateCrewAvailability,
  isActiveReservationStatus,
  pickLeastBusyCrew,
  planContinuationSegments as planContinuationSegmentsFromAvailability,
  type Assignment,
} from "./availability";
import {
  addDays,
  buildBookingTimeSlots,
  DEFAULT_BOOKING_HOURS,
  decodeBlockReason,
  earliestBookableDate,
  formatTime,
  NAME_PART_PATTERN,
  NAME_PART_VALIDATION_MESSAGE,
  bookingDurationForFirstDay,
  bookingDurationOverflowMinutes,
  isBookingStartTime,
  isBookingTimeRangeWithinHours,
  isValidCalendarDate,
  isShopOpenDate,
  isSlotBookable,
  manilaNow,
  MAX_BOOKING_SERVICE_SELECTIONS,
  normalizePhilippineMobile,
  phoneSchema,
  REFERENCE_CODE_PATTERN,
  normalizeReferenceCode,
  SHOP,
  timeToMinutes,
} from "./shop";
import type { AppointmentAction } from "./appointment-verification.server";

type BookingRules = {
```

```
  operatingHours: {
    openingTime: string;
    closingTime: string;
  };
  minimumBookingLeadHours: number;
  maxAdvanceBookingDays: number;
  allowSameDayAppointments: boolean;
  cancellationNoticeHours: number;
  reschedulingNoticeHours: number;
};

const defaultBookingRules: BookingRules = {
  operatingHours: DEFAULT_BOOKING_HOURS,
  minimumBookingLeadHours: 48,
  maxAdvanceBookingDays: 30,
  allowSameDayAppointments: false,
  cancellationNoticeHours: 48,
  reschedulingNoticeHours: 48,
};

async function getBookingRules(): Promise<BookingRules> {
  try {
    const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
    const { data, error } = await supabaseAdmin
      .from("shop_settings")
      .select(
        "opening_time,closing_time,minimum_booking_lead_hours,max_advance_b
ooking_days,allow_same_day_appointments,cancellation_notice_hours,resched
uling_notice_hours",
      )
      .eq("id", true)
      .maybeSingle();
    if (error || !data) {
      if (error) console.warn("[Booking settings] using defaults", error.message);
      return defaultBookingRules;
    }
    return {
      operatingHours: {
        openingTime: String(data.opening_time).slice(0, 5),
        closingTime: String(data.closing_time).slice(0, 5),
      },
      minimumBookingLeadHours: data.minimum_booking_lead_hours,
      maxAdvanceBookingDays: data.max_advance_booking_days,
      allowSameDayAppointments: data.allow_same_day_appointments,
```

```
      cancellationNoticeHours: data.cancellation_notice_hours,
      reschedulingNoticeHours: data.rescheduling_notice_hours,
    };
  } catch (error) {
    console.warn("[Booking settings] using defaults", error);
    return defaultBookingRules;
  }
}

function firstBookableDate(rules: BookingRules, minimumLeadHours =
rules.minimumBookingLeadHours) {
  let date = earliestBookableDate(minimumLeadHours);
  if (!rules.allowSameDayAppointments && date === manilaNow().date) date =
addDays(date, 1);
  return date;
}

const currentYear = new Date().getFullYear();

const motorcycleSelectionSchema = z.object({
  brand: z.string().trim().min(1).max(50),
  model: z.string().trim().min(1).max(50),
});

type ResolvedService = {
  id: string;
  name: string;
  price: number;
  durationMinutes: number;
  pricingSource: "small_bike" | "large_bike";
};

type MotorcycleCcCategory = "small_bike" | "big_bike";

function resolveServicePricing(
  services: Array<{
    id: string;
    name: string;
    price: number;
    duration_minutes: number | null;
    large_bike_price: number | null;
    large_bike_duration_minutes: number | null;
  }>,
  motorcycleCategory?: MotorcycleCcCategory,
): { services: ResolvedService[] } | { error: string } {
```

```
  if (motorcycleCategory === "big_bike") {
    const missingConfiguration = services.find(
      (service) =>
        service.large_bike_price === null || service.large_bike_duration_minutes
=== null,
    );
    if (missingConfiguration) {
      return {
        error: `${missingConfiguration.name} does not have a Large Bike price and
duration configured yet.`,
      };
    }
    return {
      services: services.map((service) => ({
        id: service.id,
        name: service.name,
        price: Number(service.large_bike_price),
        durationMinutes: service.large_bike_duration_minutes ?? 60,
        pricingSource: "large_bike",
      })),
    };
  }

  return {
    services: services.map((service) => ({
      id: service.id,
      name: service.name,
      price: Number(service.price),
      durationMinutes: service.duration_minutes ?? 60,
      pricingSource: "small_bike",
    })),
  };
}

const availabilitySchema = z
  .object({
    days: z.number().int().min(7).max(90).default(45),
    serviceIds: z
      .array(z.string().uuid())
      .max(MAX_BOOKING_SERVICE_SELECTIONS)
      .refine((ids) => new Set(ids).size === ids.length, "Services must be unique")
      .default([]),
    // Administrators editing an appointment need to see the current slot as
    // available while still applying the exact same booking rules to every
    // other appointment.
```

```
    excludeAppointmentId: z.string().uuid().optional(),
    // Admin edits may refer to a motorcycle that has since been deactivated.
    // Its catalog classification still determines the saved service snapshot.
    allowInactiveMotorcycle: z.boolean().default(false),
    allowMultiDayContinuation: z.boolean().default(false),
    rescheduling: z.boolean().default(false),
    motorcycle: motorcycleSelectionSchema.optional(),
    rescheduleReference: z
      .string()
      .trim()
      .regex(REFERENCE_CODE_PATTERN, "Enter a valid original appointment
reference.")
      .transform(normalizeReferenceCode)
      .optional(),
    reschedulePhone: phoneSchema.optional(),
  })
  .superRefine((data, context) => {
    if (Boolean(data.rescheduleReference) !== Boolean(data.reschedulePhone)) {
      context.addIssue({
        code: z.ZodIssueCode.custom,
        message: "A reschedule reference and mobile number must be provided
together.",
      });
    }
  });

const namePartSchema = z
  .string()
  .trim()
  .max(40)
  .regex(NAME_PART_PATTERN, NAME_PART_VALIDATION_MESSAGE);

const emailSchema = z
  .string()
  .trim()
  .email("Enter a valid email address.")
  .max(254)
  .transform((email) => email.toLocaleLowerCase());

const bookingSchema = z
  .object({
    firstName: namePartSchema.default(""),
    middleName: namePartSchema.default(""),
    lastName: namePartSchema.default(""),
    phone: phoneSchema,
```

```
    email: emailSchema.optional(),
    motoBrand: z.string().trim().min(1).max(50),
    motoModel: z.string().trim().min(1).max(50),
    motoVariant: z.string().trim().max(50).optional().or(z.literal("")),
    motoYear: z.number().int().min(1970).max(currentYear),
    plateNumber: z.string().trim().min(2).max(20),
    serviceIds: z
      .array(z.string().uuid())
      .min(1)
      .max(MAX_BOOKING_SERVICE_SELECTIONS)
      .refine((ids) => new Set(ids).size === ids.length, "Services must be unique"),
    date: z.string().refine(isValidCalendarDate, "Enter a valid appointment date."),
    startTime: z
      .string()
      .regex(/^\d{2}:\d{2}$/, "Enter a valid appointment start time.")
      .refine(
        (value) => Number.isFinite(timeToMinutes(value)),
        "Enter a valid appointment start time.",
      ),
    notes: z.string().trim().max(500).optional().or(z.literal("")),
    wantsProducts: z.boolean(),
    turnstileToken: z.string().trim().max(2048).default(""),
    idempotencyKey: z.string().uuid(),
    termsAccepted: z.literal(true),
    multiDayContinuationAccepted: z.boolean().default(false),
    rescheduleReference: z
      .string()
      .trim()
      .regex(REFERENCE_CODE_PATTERN, "Enter a valid original appointment
reference.")
      .optional(),
    rescheduleReason: z.string().trim().max(500).optional().or(z.literal("")),
    // Issued by verifyAppointmentActionCode; required whenever
rescheduleReference is set.
    verificationToken: z.string().trim().max(100).optional(),
  })
  .superRefine((data, context) => {
    if (!data.rescheduleReference && !data.email) {
      context.addIssue({
        code: z.ZodIssueCode.custom,
        path: ["email"],
        message: "Enter a valid email address.",
      });
    }
    if (data.rescheduleReference && !data.rescheduleReason?.trim()) {
```

```
      context.addIssue({
        code: z.ZodIssueCode.custom,
        path: ["rescheduleReason"],
        message: "A reason for rescheduling is required.",
      });
    }
  });

function fullName(firstName: string, middleName: string, lastName: string) {
  return [firstName.trim(), middleName.trim(),
lastName.trim()].filter(Boolean).join(" ");
}

function unavailableAvailability(from: string, to: string, error: string) {
  return {
    from,
    to,
    error,
    totalDurationMinutes: 60,
    dates: [],
    fullyBookedDates: [],
    slotsByDate: {},
  };
}

type ContinuationSegment = {
  segmentNumber: number;
  appointmentDate: string;
  startTime: string;
  bookingDurationMinutes: number;
  assignedCrewId: string;
};

/**
 * A pending reschedule request holds its requested time until the shop reviews
 * it, so other customers cannot take it in the meantime.
 */
function rescheduleHoldsQuery(supabaseAdmin: SupabaseClient<Database>,
from: string, to: string) {
  return supabaseAdmin
    .from("appointments")
    .select("id,pending_reschedule_date,pending_reschedule_start_time,booking_
duration_minutes")
    .not("pending_reschedule_request_id", "is", null)
    .eq("is_archived", false)
```

```
    .is("rescheduled_to_appointment_id", null)
    .in("status", ["pending", "confirmed"])
    .gte("pending_reschedule_date", from)
    .lte("pending_reschedule_date", to);
}

/** Holds count as reservations that do not have a mechanic yet. */
function rescheduleHoldAssignments(
  rows: Array<{
    id: string;
    pending_reschedule_date: string | null;
    pending_reschedule_start_time: string | null;
    booking_duration_minutes: number | null;
  }>,
  excludedAppointmentId?: string | null,
): Assignment[] {
  return rows.flatMap((row) =>
    row.id !== excludedAppointmentId &&
    row.pending_reschedule_date &&
    row.pending_reschedule_start_time
      ? [
          {
            date: row.pending_reschedule_date,
            startTime: String(row.pending_reschedule_start_time).slice(0, 5),
            // The same default the database uses for a hold.
            durationMinutes: row.booking_duration_minutes ?? 75,
            crewId: null,
          },
        ]
      : [],
  );
}

/** Most appointments one mobile number or email address can have coming up
at once. */
const MAX_UPCOMING_APPOINTMENTS = 3;

/**
 * The reason a new booking is refused for having too many upcoming
 * appointments, or null. The database enforces the same limit.
 */
async function upcomingAppointmentLimitError(
  supabaseAdmin: SupabaseClient<Database>,
  phone: string,
  email: string | undefined,
```

```
): Promise<string | null> {
  const upcoming = () =>
    supabaseAdmin
      .from("appointments")
      .select("id", { count: "exact", head: true })
      .eq("is_archived", false)
      .is("rescheduled_to_appointment_id", null)
      .in("status", ["pending", "confirmed", "in_progress"])
      .gte("appointment_date", manilaNow().date);
  const normalizedEmail = email?.trim().toLowerCase();
  const [byPhone, byEmail] = await Promise.all([
    upcoming().eq("phone", phone),
    normalizedEmail ? upcoming().eq("email", normalizedEmail) :
Promise.resolve(null),
  ]);
  if (byPhone.error || byEmail?.error) {
    console.error("[Booking] upcoming appointment count failed", byPhone.error
?? byEmail?.error);
    return null; // The database limit still applies when the booking is saved.
  }
  if ((byPhone.count ?? 0) >= MAX_UPCOMING_APPOINTMENTS) {
    return upcomingLimitMessage("mobile number");
  }
  if ((byEmail?.count ?? 0) >= MAX_UPCOMING_APPOINTMENTS) {
    return upcomingLimitMessage("email address");
  }
  return null;
}

function upcomingLimitMessage(contact: "mobile number" | "email address") {
  return `This ${contact} already has ${MAX_UPCOMING_APPOINTMENTS}
upcoming appointments, the most allowed at one time. Please cancel one or wait
until one is done before booking another.`;
}

async function planContinuationSegments({
  supabaseAdmin,
  firstDate,
  remainingMinutes,
  lastAvailableDate,
  operatingHours,
  slotConfig,
}: {
  supabaseAdmin: SupabaseClient<Database>;
  firstDate: string;
```

```
  remainingMinutes: number;
  lastAvailableDate: string;
  operatingHours: BookingRules["operatingHours"];
  slotConfig: Array<{ startTime: string; capacity: number }>;
}): Promise<{ ok: true; segments: ContinuationSegment[] } | { ok: false; error:
string }> {
  const from = addDays(firstDate, 1);
  if (from > lastAvailableDate) {
    return {
      ok: false,
      error: "There is not enough future shop time available to complete these
services.",
    };
  }

  const [
    blocksRes,
    schedulesRes,
    crewRes,
    exceptionsRes,
    appointmentsRes,
    continuationsRes,
    holdsRes,
  ] = await Promise.all([
    supabaseAdmin
      .from("schedule_blocks")
      .select("block_date,start_time,reason")
      .eq("is_active", true)
      .gte("block_date", from)
      .lte("block_date", lastAvailableDate),
    supabaseAdmin
      .from("crew_schedules")
      .select("crew_id,schedule_date,start_time,end_time,is_working")
      .gte("schedule_date", from)
      .lte("schedule_date", lastAvailableDate),
    supabaseAdmin.from("crew_members").select("id").eq("is_active",
true).eq("is_archived", false),
    supabaseAdmin
      .from("crew_availability_exceptions")
      .select("crew_id,start_date,end_date,start_time,end_time,is_all_day")
      .lte("start_date", lastAvailableDate)
      .gte("end_date", from),
    supabaseAdmin
      .from("appointments")
      .select("appointment_date,start_time,assigned_crew_id,booking_duration_mi
```

```
nutes")
      .eq("is_archived", false)
      .is("rescheduled_to_appointment_id", null)
      .in("status", ACTIVE_RESERVATION_STATUSES)
      .gte("appointment_date", from)
      .lte("appointment_date", lastAvailableDate),
    supabaseAdmin
      .from("appointment_continuations")
      .select(
        "appointment_id,appointment_date,start_time,assigned_crew_id,booking_d
uration_minutes",
      )
      .gte("appointment_date", from)
      .lte("appointment_date", lastAvailableDate),
    rescheduleHoldsQuery(supabaseAdmin, from, lastAvailableDate),
  ]);

  if (
    blocksRes.error ||
    schedulesRes.error ||
    crewRes.error ||
    exceptionsRes.error ||
    appointmentsRes.error ||
    continuationsRes.error ||
    holdsRes.error
  ) {
    console.error("[Booking continuation] availability lookup failed", {
      blocks: blocksRes.error,
      schedules: schedulesRes.error,
      crew: crewRes.error,
      exceptions: exceptionsRes.error,
      appointments: appointmentsRes.error,
      continuations: continuationsRes.error,
      holds: holdsRes.error,
    });
    return {
      ok: false,
      error: "We could not reserve the next available shop day. Please try again.",
    };
  }

  const continuationParentIds = Array.from(
    new Set((continuationsRes.data ?? []).map((continuation) =>
continuation.appointment_id)),
  );
```

```
  const continuationParents = continuationParentIds.length
    ? await supabaseAdmin
        .from("appointments")
        .select("id,is_archived,status,rescheduled_to_appointment_id")
        .in("id", continuationParentIds)
    : { data: [], error: null };
  if (continuationParents.error) {
    return {
      ok: false,
      error: "We could not reserve the next available shop day. Please try again.",
    };
  }

  const activeContinuationParentIds = new Set(
    (continuationParents.data ?? [])
      .filter(
        (appointment) =>
          !appointment.is_archived &&
          !appointment.rescheduled_to_appointment_id &&
          isActiveReservationStatus(appointment.status),
      )
      .map((appointment) => appointment.id),
  );
  const assignments = [
    ...(appointmentsRes.data ?? []).map((appointment) => ({
      date: appointment.appointment_date,
      startTime: String(appointment.start_time).slice(0, 5),
      durationMinutes: appointment.booking_duration_minutes ?? 60,
      crewId: appointment.assigned_crew_id,
    })),
    ...(continuationsRes.data ?? [])
      .filter((continuation) =>
activeContinuationParentIds.has(continuation.appointment_id))
      .map((continuation) => ({
        date: continuation.appointment_date,
        startTime: String(continuation.start_time).slice(0, 5),
        durationMinutes: continuation.booking_duration_minutes,
        crewId: continuation.assigned_crew_id,
      })),
    ...rescheduleHoldAssignments(holdsRes.data ?? []),
  ];
  const activeCrewIds = new Set((crewRes.data ?? []).map((crew) => crew.id));
  const segments = planContinuationSegmentsFromAvailability({
    source: {
      from,
```

```
      to: lastAvailableDate,
      operatingHours,
      slots: buildBookingTimeSlots(slotConfig, operatingHours),
      blocks: (blocksRes.data ?? []).map((block) => {
        const { endTime } = decodeBlockReason(block.reason);
        return {
          date: block.block_date,
          startTime: block.start_time ? String(block.start_time).slice(0, 5) : null,
          endTime,
          reason: null,
        };
      }),
      assignments,
      schedules: (schedulesRes.data ?? []).filter((schedule) =>
        activeCrewIds.has(schedule.crew_id),
      ),
      exceptions: exceptionsRes.data ?? [],
    },
    firstDate,
    remainingMinutes,
  });
  if (!segments) {
    return {
      ok: false,
      error:
        "There is not enough mechanic availability in the booking window to
complete these services.",
    };
  }

  return {
    ok: true,
    segments: segments.map((segment, index) => ({
      segmentNumber: index + 1,
      appointmentDate: segment.appointmentDate,
      startTime: segment.startTime,
      bookingDurationMinutes: segment.durationMinutes,
      assignedCrewId: segment.crewId,
    })),
  };
}

function availabilityErrorMessage(errors: Array<{ message: string } | null>) {
  const message = errors
    .filter((error): error is { message: string } => !!error)
```

```
    .map((error) => error.message)
    .join(" ");

  if (
    /booking_duration_minutes|appointment_services.*duration_minutes|schedule
_date/i.test(message)
  ) {
    return "Booking capacity setup is incomplete. The shop needs to apply its
scheduling database update.";
  }

  return "We could not load booking availability. Please try again shortly.";
}

function makeReference() {
  const alphabet = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789";
  let out = "";
  const bytes = crypto.getRandomValues(new Uint8Array(6));
  for (const b of bytes) out += alphabet[b % alphabet.length];
  return `FRM-${out}`;
}

function getClientIp() {
  const request = getRequest();
  const platformIp =
    request?.headers.get("x-vercel-forwarded-for")?.trim() ||
    request?.headers.get("cf-connecting-ip")?.trim();
  const forwarded = request?.headers.get("x-forwarded-for")?.split(",")[0]?.trim();
  return platformIp || forwarded || request?.headers.get("x-real-ip")?.trim() ||
"unknown";
}

async function hashRateLimitSubject(subject: string) {
  const digest = await crypto.subtle.digest("SHA-256", new
TextEncoder().encode(subject));
  return Array.from(new Uint8Array(digest), (byte) =>
byte.toString(16).padStart(2, "0")).join("");
}

/**
 * Persistent, per-endpoint limits. The database function serializes increments,
 * so this also works when the app runs across multiple serverless instances.
 */
type RateLimitResult = "allowed" | "limited" | "unavailable";
```

```
async function checkPublicRequestRateLimit(
  scope:
    | "availability"
    | "booking"
    | "lookup"
    | "cancellation"
    | "rescheduling"
    | "verification_request"
    | "verification_verify",
  maxRequests: number,
  windowSeconds: number,
  subject?: string,
): Promise<RateLimitResult> {
  try {
    const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
    const subjects = [await hashRateLimitSubject(`ip:${getClientIp()}`)];
    if (subject) subjects.push(await hashRateLimitSubject(`subject:${subject}`));

    for (const opaqueSubject of subjects) {
      const { data, error } = await supabaseAdmin.rpc("enforce_public_rate_limit", {
        p_scope: scope,
        p_subject: opaqueSubject,
        p_limit: maxRequests,
        p_window_seconds: windowSeconds,
      });
      if (error) {
        console.error(`[Rate limit] ${scope} check failed`, error);
        return "unavailable";
      }
      if (data !== true) return "limited";
    }
    return "allowed";
  } catch (error) {
    console.error(`[Rate limit] ${scope} check failed`, error);
    return "unavailable";
  }
}

async function isTurnstileVerificationValid(token: string, idempotencyKey: string)
{
  const secret = process.env["TURNSTILE_SECRET_KEY"];
  if (!secret) return true;
  if (!token) return false;
```

```
  try {
    const body = new FormData();
    body.set("secret", secret);
    body.set("response", token);
    body.set("idempotency_key", idempotencyKey);
    const ip = getClientIp();
    if (ip !== "unknown") body.set("remoteip", ip);

    const response = await
fetch("https://challenges.cloudflare.com/turnstile/v0/siteverify", {
      method: "POST",
      body,
    });
    const result = (await response.json()) as { success?: unknown; action?:
unknown };
    return response.ok && result.success === true && result.action === "booking";
  } catch (error) {
    console.error("[Turnstile] booking verification failed", error);
    return false;
  }
}

/** Slot availability for a date range (Manila dates). */
export const getAvailability = createServerFn({ method: "GET" })
  .validator((input: unknown) => availabilitySchema.parse(input ?? {}))
  .handler(async ({ data }) => {
    const from = earliestBookableDate();
    const to = addDays(from, data.days);
    const supabaseModule = await
import("@/integrations/supabase/client.server").catch((error) => {
      console.error("[Booking availability] Supabase configuration failed", error);
      return null;
    });
    if (!supabaseModule) {
      return unavailableAvailability(
        from,
        to,
        "Booking availability is temporarily unavailable. Please try again shortly.",
      );
    }
    const { supabaseAdmin } = supabaseModule;
    const bookingRules = await getBookingRules();
    const configuredFrom = firstBookableDate(
      bookingRules,
      data.rescheduling ? bookingRules.reschedulingNoticeHours : undefined,
```

```
    );
    const configuredTo = addDays(
      manilaNow().date,
      Math.max(0, Math.min(data.days, bookingRules.maxAdvanceBookingDays)),
    );

    const availabilityRateLimit = await checkPublicRequestRateLimit("availability",
60, 60);
    if (availabilityRateLimit === "limited") {
      return unavailableAvailability(
        configuredFrom,
        configuredTo,
        "Too many availability checks. Please try again shortly.",
      );
    }
    // Availability is read-only. A rate-limit infrastructure issue must not
    // prevent a customer from selecting a service and seeing schedules.
    if (availabilityRateLimit === "unavailable") {
      console.warn(
        "[Rate limit] availability protection unavailable; continuing with read-only
lookup",
      );
    }

    // Builds fresh queries on each call so a failed attempt can be retried.
    const loadSchedulingData = () => {
      let appointmentsQuery = supabaseAdmin
        .from("appointments")
        .select("appointment_date,start_time,assigned_crew_id,booking_duration_
minutes")
        .eq("is_archived", false)
        .gte("appointment_date", configuredFrom)
        .lte("appointment_date", configuredTo)
        .is("rescheduled_to_appointment_id", null)
        .in("status", ACTIVE_RESERVATION_STATUSES);
      if (data.excludeAppointmentId) {
        appointmentsQuery = appointmentsQuery.neq("id",
data.excludeAppointmentId);
      }
      return Promise.all([
        supabaseAdmin
          .from("time_slots")
          .select("id,start_time,end_time,capacity")
          .eq("is_active", true)
          .order("start_time"),
```

```
        supabaseAdmin
          .from("schedule_blocks")
          .select("block_date,start_time,reason")
          .eq("is_active", true)
          .gte("block_date", configuredFrom)
          .lte("block_date", configuredTo),
        appointmentsQuery,
        data.serviceIds.length > 0
          ? supabaseAdmin
              .from("services")
              .select("id,name,price,duration_minutes,large_bike_price,large_bike_dur
ation_minutes")
              .in("id", data.serviceIds)
              .eq("is_active", true)
              .eq("is_archived", false)
          : {
              data: [] as {
                id: string;
                name: string;
                price: number;
                duration_minutes: number | null;
                large_bike_price: number | null;
                large_bike_duration_minutes: number | null;
              }[],
              error: null,
            },
        supabaseAdmin
          .from("crew_schedules")
          .select("*")
          .gte("schedule_date", configuredFrom)
          .lte("schedule_date", configuredTo),
        supabaseAdmin
          .from("crew_members")
          .select("id")
          .eq("is_active", true)
          .eq("is_archived", false),
        supabaseAdmin
          .from("crew_availability_exceptions")
          .select("*")
          .gte("end_date", configuredFrom)
          .lte("start_date", configuredTo),
        supabaseAdmin
          .from("appointment_continuations")
          .select(
            "appointment_id,appointment_date,start_time,assigned_crew_id,booking
```

```
_duration_minutes",
          )
          .gte("appointment_date", configuredFrom)
          .lte("appointment_date", configuredTo),
        rescheduleHoldsQuery(supabaseAdmin, configuredFrom, configuredTo),
      ]);
    };

    let results = await loadSchedulingData();
    // One retry covers a dropped connection or a brief database restart.
    if (results.some((result) => result.error)) {
      await new Promise((resolve) => setTimeout(resolve, 500));
      results = await loadSchedulingData();
    }
    const [
      slotsRes,
      blocksRes,
      apptsRes,
      servicesRes,
      schedulesRes,
      activeCrewRes,
      exceptionsRes,
      continuationsRes,
      holdsRes,
    ] = results;

    const errors = results.map((result) => result.error);
    if (errors.some(Boolean)) {
      const tables = [
        "time_slots",
        "schedule_blocks",
        "appointments",
        "services",
        "crew_schedules",
        "crew_members",
        "crew_availability_exceptions",
        "appointment_continuations",
        "appointments (reschedule requests)",
      ];
      // Names the project and each failed read, so the hosting logs show the
cause.
      console.error("[Booking availability] database query failed", {
        supabaseProject: supabaseModule.supabaseServerProject(),
        failures: errors.flatMap((error, index) =>
          error
```

```
          ? [{ table: tables[index], code: error.code, message: error.message, hint:
error.hint }]
          : [],
      ),
    });
    return unavailableAvailability(
      configuredFrom,
      configuredTo,
      availabilityErrorMessage(errors),
    );
  }

  if (
    data.serviceIds.length > 0 &&
    (servicesRes.data?.length ?? 0) !== new Set(data.serviceIds).size
  ) {
    return unavailableAvailability(
      configuredFrom,
      configuredTo,
      "One or more selected services are no longer available. Please choose a
different service.",
    );
  }

  let motorcycle: { brand: string; model: string } | undefined;
  if (data.rescheduleReference && data.reschedulePhone) {
    const original = await findAppointment(data.rescheduleReference,
data.reschedulePhone);
    if (!original) {
      return unavailableAvailability(
        configuredFrom,
        configuredTo,
        "We could not verify the original appointment. Please return to the
Appointment Tracker and try again.",
      );
    }

    motorcycle = { brand: original.moto_brand, model: original.moto_model };
  }

  if (!motorcycle && data.motorcycle) {
    motorcycle = { ...data.motorcycle };
  }

  let motorcycleCategory: MotorcycleCcCategory | undefined;
```

```
    if (motorcycle) {
      let motorcycleQuery = supabaseAdmin
        .from("motorcycle_catalog")
        .select("cc_category")
        .eq("brand", motorcycle.brand)
        .eq("model", motorcycle.model)
        .eq("is_archived", false);
      if (!data.rescheduleReference && !data.allowInactiveMotorcycle) {
        motorcycleQuery = motorcycleQuery.eq("is_active", true);
      }
      const { data: motorcycleRecord, error: motorcycleError } =
        await motorcycleQuery.maybeSingle();
      if (motorcycleError || !motorcycleRecord) {
        return unavailableAvailability(
          configuredFrom,
          configuredTo,
          "Select a valid motorcycle model from the Motorcycle Catalog.",
        );
      }
      if (
        motorcycleRecord.cc_category !== "small_bike" &&
        motorcycleRecord.cc_category !== "big_bike"
      ) {
        return unavailableAvailability(
          configuredFrom,
          configuredTo,
          "The selected motorcycle needs an engine CC classification before its
service estimate can be calculated.",
        );
      }
      motorcycleCategory = motorcycleRecord.cc_category;
    }

    // A reservation lasts exactly as long as the selected services.
    const pricing = resolveServicePricing(servicesRes.data ?? [],
motorcycleCategory);
    if ("error" in pricing) {
      return unavailableAvailability(configuredFrom, configuredTo, pricing.error);
    }
    const resolvedServices = pricing.services;
    const totalDuration = resolvedServices.reduce(
      (sum, service) => sum + service.durationMinutes,
      0,
    );
```

```
    const assignments: {
      date: string;
      startTime: string;
      durationMinutes: number;
      crewId: string | null;
    }[] = [];
    for (const a of apptsRes.data ?? []) {
      assignments.push({
        date: a.appointment_date,
        startTime: String(a.start_time).slice(0, 5),
        durationMinutes: a.booking_duration_minutes ?? 60,
        crewId: a.assigned_crew_id,
      });
    }
    const continuationParentIds = Array.from(
      new Set((continuationsRes.data ?? []).map((continuation) =>
continuation.appointment_id)),
    );
    if (continuationParentIds.length > 0) {
      const continuationParents = await supabaseAdmin
        .from("appointments")
        .select("id,is_archived,status,rescheduled_to_appointment_id")
        .in("id", continuationParentIds);
      if (continuationParents.error) {
        return unavailableAvailability(
          configuredFrom,
          configuredTo,
          "Booking availability is temporarily unavailable. Please try again shortly.",
        );
      }
      const activeContinuationParents = new Set(
        (continuationParents.data ?? [])
          .filter(
            (appointment) =>
              !appointment.is_archived &&
              !appointment.rescheduled_to_appointment_id &&
              isActiveReservationStatus(appointment.status),
          )
          .map((appointment) => appointment.id),
      );
      for (const continuation of continuationsRes.data ?? []) {
        if (!activeContinuationParents.has(continuation.appointment_id)) continue;
        assignments.push({
          date: continuation.appointment_date,
          startTime: String(continuation.start_time).slice(0, 5),
```

```
          durationMinutes: continuation.booking_duration_minutes,
          crewId: continuation.assigned_crew_id,
        });
      }
    }

    // An admin editing an appointment does not compete with its own request.
    assignments.push(...rescheduleHoldAssignments(holdsRes.data ?? [],
data.excludeAppointmentId));

    const capacityConfig = (slotsRes.data ?? []).map((slot) => ({
      startTime: String(slot.start_time).slice(0, 5),
      capacity: slot.capacity,
    }));

    const publicAvailability = buildPublicAvailability({
      from: configuredFrom,
      to: configuredTo,
      minimumBookingLeadHours: data.rescheduling
        ? bookingRules.reschedulingNoticeHours
        : bookingRules.minimumBookingLeadHours,
      totalDurationMinutes: totalDuration || 60,
      allowMultiDayContinuation: data.allowMultiDayContinuation &&
!data.rescheduling,
      slots: buildBookingTimeSlots(capacityConfig, bookingRules.operatingHours),
      operatingHours: bookingRules.operatingHours,
      blocks: (blocksRes.data ?? []).map((b) => {
        const { endTime, userReason } = decodeBlockReason(b.reason);
        return {
          date: b.block_date,
          startTime: b.start_time ? String(b.start_time).slice(0, 5) : null,
          endTime,
          reason: userReason || null,
        };
      }),
      assignments,
      schedules: (schedulesRes.data ?? []).filter((schedule) =>
        (activeCrewRes.data ?? []).some((crew) => crew.id === schedule.crew_id),
      ),
      exceptions: exceptionsRes.data ?? [],
    });
    return {
      ...publicAvailability,
      operatingHours: bookingRules.operatingHours,
      serviceEstimates: resolvedServices,
```

```
    };
  });

export const createBooking = createServerFn({ method: "POST" })
  .validator((input: unknown) => bookingSchema.parse(input))
  .handler(async ({ data }) => {
    const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
    const bookingRules = await getBookingRules();
    const customerName = fullName(data.firstName, data.middleName,
data.lastName);

    if (!data.rescheduleReference && !data.firstName.trim()) {
      return { ok: false as const, error: "Enter the customer's first name." };
    }
    if (!data.rescheduleReference && !data.lastName.trim()) {
      return { ok: false as const, error: "Enter the customer's last name." };
    }

    // A retry after a lost response must return the original reservation instead
    // of consuming another slot or requiring a previously used challenge token.
    const existingRequest = await supabaseAdmin
      .from("appointments")
      .select("reference_code,total_estimate")
      .eq("booking_request_id", data.idempotencyKey)
      .maybeSingle();
    if (existingRequest.error) {
      return {
        ok: false as const,
        error: "We could not verify this booking request. Please try again.",
      };
    }
    if (existingRequest.data) {
      const reference =
normalizeReferenceCode(existingRequest.data.reference_code);
      if (!REFERENCE_CODE_PATTERN.test(reference)) {
        console.error("[Booking] existing booking has an invalid reference", {
          bookingRequestId: data.idempotencyKey,
        });
        return {
          ok: false as const,
          error:
            "We saved your booking, but could not prepare its reference code. Please
contact the shop.",
        };
```

```
      }
      return {
        ok: true as const,
        reference,
        total: Number(existingRequest.data.total_estimate),
      };
    }

    let rescheduledFrom: {
      id: string;
      reference_code: string;
      total_estimate: number;
    } | null = null;
    if (data.rescheduleReference) {
      const original = await findAppointment(data.rescheduleReference,
data.phone);
      if (!original) {
        return {
          ok: false as const,
          error: "We could not verify the original appointment. Please look it up
again.",
        };
      }
      const verification = await requireActionVerification(
        original.id,
        "reschedule",
        data.verificationToken,
      );
      if (verification) return verification;
      const continuations = await supabaseAdmin
        .from("appointment_continuations")
        .select("id", { count: "exact", head: true })
        .eq("appointment_id", original.id);
      if (continuations.error) {
        console.error("[Reschedule] continuation lookup failed", continuations.error);
        return {
          ok: false as const,
          error: "We could not verify the original appointment schedule. Please try
again.",
        };
      }
      // The approval RPC creates a linked replacement for a one-day booking.
      // Allowing a multi-day parent through would release its continuation
      // reservations without recreating them on the replacement appointment.
      if ((continuations.count ?? 0) > 0) {
```

```
        return {
          ok: false as const,
          error:
            "This appointment includes a multi-day service. Please call the shop to
reschedule all reserved service days together.",
        };
      }
      if (original.rescheduled_to_appointment_id) {
        return {
          ok: false as const,
          error: "This appointment has already been rescheduled.",
        };
      }
      if (original.reschedule_count >= 3) {
        return {
          ok: false as const,
          error: "This appointment has reached the maximum of 3 reschedules.",
        };
      }
      if (!["pending", "confirmed"].includes(original.status)) {
        return {
          ok: false as const,
          error: "This appointment can no longer be rescheduled online. Please call
the shop.",
        };
      }
      if (original.pending_reschedule_request_id) {
        // The first request may have reached the database but lost its response.
        // Always return its reserved code, even if a browser retry has a new key.
        if (original.pending_reschedule_reference_code) {
          return {
            ok: true as const,
            reference: original.pending_reschedule_reference_code,
            total: Number(original.total_estimate),
            rescheduleRequested: true as const,
          };
        }

        // Recover a legacy request created before references were reserved at
        // submission. Reuse the server-stored request and schedule details so
        // the new RPC can reserve its missing linked-booking reference safely.
        if (
          !original.pending_reschedule_date ||
          !original.pending_reschedule_start_time ||
          !original.pending_reschedule_reason
```

```
      ) {
        return {
          ok: false as const,
          error:
            "A reschedule request is awaiting review. Please contact the shop to
complete it.",
        };
      }
      data = {
        ...data,
        idempotencyKey: original.pending_reschedule_request_id,
        date: original.pending_reschedule_date,
        startTime: String(original.pending_reschedule_start_time).slice(0, 5),
        rescheduleReason: original.pending_reschedule_reason,
      };
    }
    if (
      !isSlotBookable(
        original.appointment_date,
        String(original.start_time).slice(0, 5),
        bookingRules.reschedulingNoticeHours,
      )
    ) {
      return {
        ok: false as const,
        error: `Rescheduling needs ${bookingRules.reschedulingNoticeHours}
hours notice. Please call the shop instead.`,
      };
    }
    if (data.date === original.appointment_date) {
      return {
        ok: false as const,
        error: "Choose a date different from your current appointment date.",
      };
    }

    const originalServices = (original.appointment_services ??
[]).flatMap((service) =>
      service.service_id ? [service.service_id] : [],
    );
    if (originalServices.length === 0) {
      return {
        ok: false as const,
        error:
          "The original services are unavailable for online rescheduling. Please call
```

```
the shop.",
        };
      }

      // Personal, motorcycle, and service data always comes from the original
      // server-side record. Client-supplied copies cannot be used to alter it.
      data = {
        ...data,
        phone: original.phone,
        email: original.email ?? data.email,
        motoBrand: original.moto_brand,
        motoModel: original.moto_model,
        motoVariant: original.moto_variant ?? "",
        motoYear: original.moto_year ?? new Date().getFullYear(),
        plateNumber: original.plate_number,
        serviceIds: originalServices,
        notes: original.notes ?? "",
      };
      rescheduledFrom = {
        id: original.id,
        reference_code: original.reference_code,
        total_estimate: Number(original.total_estimate),
      };
    }

    const bookingRateLimit = await checkPublicRequestRateLimit("booking", 5, 15
* 60, data.phone);
    if (bookingRateLimit !== "allowed") {
      return {
        ok: false as const,
        error:
          bookingRateLimit === "limited"
            ? "Too many booking attempts. Please wait a few minutes before trying
again."
            : "Booking protection is temporarily unavailable. Please try again shortly.",
      };
    }

    if (!(await isTurnstileVerificationValid(data.turnstileToken,
data.idempotencyKey))) {
      return {
        ok: false as const,
        error: "Security verification failed. Please complete the challenge and try
again.",
      };
```

```
    }

    const startTime = data.startTime.slice(0, 5);
    const slotStartMin = timeToMinutes(startTime);

    const isBlocked = await supabaseAdmin
      .from("blocked_numbers")
      .select("id")
      .ilike("phone", data.phone)
      .eq("is_archived", false)
      .maybeSingle();
    // Without an answer the number might be blocked, so the booking stops here.
    if (isBlocked.error) {
      console.error("[Booking] blocked-number check failed", isBlocked.error);
      return {
        ok: false as const,
        error: "We could not verify your booking right now. Please try again shortly.",
      };
    }
    if (isBlocked.data) {
      return {
        ok: false as const,
        error:
          "This number has been blocked from booking online due to previous
violations. Please contact the shop directly.",
      };
    }

    // A reschedule replaces an existing appointment, so it is not limited.
    if (!rescheduledFrom) {
      const limitError = await upcomingAppointmentLimitError(supabaseAdmin,
data.phone, data.email);
      if (limitError) return { ok: false as const, error: limitError };
    }

    if (!isBookingStartTime(startTime, bookingRules.operatingHours)) {
      return {
        ok: false as const,
        error: `Choose a booking start time from
${formatTime(bookingRules.operatingHours.openingTime)} to
${formatTime(bookingRules.operatingHours.closingTime)} in 30-minute
intervals.`,
      };
    }
```

```
    if (!isShopOpenDate(data.date)) {
      return {
        ok: false as const,
        error: "Bookings are available Monday through Saturday only.",
      };
    }

    const firstAvailableDate = firstBookableDate(
      bookingRules,
      rescheduledFrom ? bookingRules.reschedulingNoticeHours : undefined,
    );
    const lastAvailableDate = addDays(manilaNow().date,
bookingRules.maxAdvanceBookingDays);
    if (
      data.date < firstAvailableDate ||
      !isSlotBookable(
        data.date,
        startTime,
        rescheduledFrom
          ? bookingRules.reschedulingNoticeHours
          : bookingRules.minimumBookingLeadHours,
      )
    ) {
      return {
        ok: false as const,
        error: "This slot is no longer bookable. Please select a later time.",
      };
    }
    if (data.date > lastAvailableDate) {
      return {
        ok: false as const,
        error: `Bookings can be made up to
${bookingRules.maxAdvanceBookingDays} days in advance.`,
      };
    }

    const slotConfigRes = await supabaseAdmin
      .from("time_slots")
      .select("start_time,capacity")
      .eq("is_active", true);
    if (slotConfigRes.error) {
      return { ok: false as const, error: "We could not verify that time slot. Please try
again." };
    }
    const slotConfig = (slotConfigRes.data ?? []).map((configuredSlot) => ({
```

```
      startTime: String(configuredSlot.start_time).slice(0, 5),
      capacity: configuredSlot.capacity,
    }));
    const slot = buildBookingTimeSlots(slotConfig,
bookingRules.operatingHours).find(
      (configuredSlot) => configuredSlot.startTime === startTime,
    );
    if (!slot || slot.capacity <= 0)
      return { ok: false as const, error: "That time slot is not available." };

    // Service defaults are for Small Bikes. Large Bikes use the optional
    // category-level price and duration configured on each service.
    const services = await supabaseAdmin
      .from("services")
      .select("id,name,price,duration_minutes,large_bike_price,large_bike_duration
_minutes")
      .in("id", data.serviceIds)
      .eq("is_active", true)
      .eq("is_archived", false);
    if (
      services.error ||
      !services.data?.length ||
      services.data.length !== data.serviceIds.length
    ) {
      return { ok: false as const, error: "Please select available services and try
again." };
    }

    // The catalog's CC category is the server-side source of truth. A
    // reschedule may retain an inactive model, but it must still resolve to a
    // non-archived catalog record so that its pricing is never guessed.
    const motorcycleRecord = await supabaseAdmin
      .from("motorcycle_catalog")
      .select("cc_category,is_active")
      .eq("brand", data.motoBrand)
      .eq("model", data.motoModel)
      .eq("is_archived", false)
      .maybeSingle();
    if (motorcycleRecord.error || !motorcycleRecord.data) {
      return {
        ok: false as const,
        error: "Please select an available motorcycle from the Motorcycle Catalog.",
      };
    }
    if (!rescheduledFrom && !motorcycleRecord.data.is_active) {
```

```
      return {
        ok: false as const,
        error: "Please select an available motorcycle from the Motorcycle Catalog.",
      };
    }
    if (
      motorcycleRecord.data.cc_category !== "small_bike" &&
      motorcycleRecord.data.cc_category !== "big_bike"
    ) {
      return {
        ok: false as const,
        error: "The selected motorcycle needs an engine CC classification before it
can be booked.",
      };
    }

    const pricing = resolveServicePricing(services.data,
motorcycleRecord.data.cc_category);
    if ("error" in pricing) return { ok: false as const, error: pricing.error };
    const resolvedServices = pricing.services;
    const totalDuration = resolvedServices.reduce(
      (sum, service) => sum + service.durationMinutes,
      0,
    );

    const overflowMinutes = bookingDurationOverflowMinutes(
      startTime,
      totalDuration,
      bookingRules.operatingHours,
    );
    const needsContinuation = !rescheduledFrom && data.serviceIds.length > 1
&& overflowMinutes > 0;
    if (overflowMinutes > 0 && !needsContinuation) {
      return {
        ok: false as const,
        error:
          "The selected services do not fit within the shop's operating hours. Please
choose an earlier time.",
      };
    }
    if (needsContinuation && !data.multiDayContinuationAccepted) {
      return {
        ok: false as const,
        error: "Please confirm the multi-day service before continuing.",
      };
```

```
  }
  const firstDayDuration = needsContinuation
    ? bookingDurationForFirstDay(startTime, totalDuration,
bookingRules.operatingHours)
    : totalDuration;
  if (
    firstDayDuration <= 0 ||
    !isBookingTimeRangeWithinHours(startTime, firstDayDuration,
bookingRules.operatingHours)
  ) {
    return {
      ok: false as const,
      error:
        "The selected services do not fit within the shop's operating hours. Please
choose an earlier time.",
    };
  }

  const blocked = await supabaseAdmin
    .from("schedule_blocks")
    .select("id,start_time,reason")
    .eq("block_date", data.date)
    .eq("is_active", true);
  if (blocked.error) {
    return {
      ok: false as const,
      error: "We could not verify the shop schedule. Please try again.",
    };
  }
  if (
    (blocked.data ?? []).some((b) => {
      if (!b.start_time) return true; // whole-day block
      const { endTime } = decodeBlockReason(b.reason);
      const bs = String(b.start_time).slice(0, 5);
      // Older single-slot blocks have no RANGE marker. They block their
      // exact start time; treating them as a zero-length range lets a final
      // booking bypass a slot that public availability correctly hides.
      if (!endTime) return bs === startTime;
      const bsMin = parseInt(bs.slice(0, 2)) * 60 + parseInt(bs.slice(3, 5));
      const beMin = parseInt(endTime.slice(0, 2)) * 60 +
parseInt(endTime.slice(3, 5));
      // Overlap: the first-day reservation overlaps a blocked time range.
      return slotStartMin < beMin && slotStartMin + firstDayDuration > bsMin;
    })
  ) {
```

```
      return {
        ok: false as const,
        error: "The shop is closed for that schedule. Please pick another one.",
      };
    }

    // 2. Only active mechanics explicitly assigned to this date can be booked.
    //    There is intentionally no fallback to every active crew member.
    const [schedulesRes, activeCrewRes] = await Promise.all([
      supabaseAdmin
        .from("crew_schedules")
        .select("*")
        .eq("schedule_date", data.date)
        .order("crew_id"),
      supabaseAdmin
        .from("crew_members")
        .select("id")
        .eq("is_active", true)
        .eq("is_archived", false),
    ]);
    if (schedulesRes.error || activeCrewRes.error) {
      return {
        ok: false as const,
        error: "We could not verify mechanic availability. Please try again.",
      };
    }

    const activeCrewIds = new Set((activeCrewRes.data ?? []).map((crew) =>
crew.id));
    const effectiveSchedules = (schedulesRes.data ?? []).filter(
      (schedule) => schedule.is_working && activeCrewIds.has(schedule.crew_id),
    );

    if (!effectiveSchedules.length) {
      return { ok: false as const, error: "No mechanics are scheduled to work on
that day." };
    }

    // 3. Get availability exceptions for that date
    const exceptionsRes = await supabaseAdmin
      .from("crew_availability_exceptions")
      .select("*")
      .lte("start_date", data.date)
      .gte("end_date", data.date);
    if (exceptionsRes.error) {
```

```
      return {
        ok: false as const,
        error: "We could not verify mechanic availability. Please try again.",
      };
    }

    // 4. Evaluate staffing and capacity with the same rule used by the public
availability grid.
    const [existingApptsRes, existingContinuationsRes, existingHoldsRes] = await
Promise.all([
      supabaseAdmin
        .from("appointments")
        .select("assigned_crew_id,start_time,booking_duration_minutes")
        .eq("appointment_date", data.date)
        .eq("is_archived", false)
        .is("rescheduled_to_appointment_id", null)
        .in("status", ACTIVE_RESERVATION_STATUSES),
      supabaseAdmin
        .from("appointment_continuations")
        .select("appointment_id,assigned_crew_id,start_time,booking_duration_min
utes")
        .eq("appointment_date", data.date),
      rescheduleHoldsQuery(supabaseAdmin, data.date, data.date),
    ]);
    if (existingApptsRes.error || existingContinuationsRes.error ||
existingHoldsRes.error) {
      return {
        ok: false as const,
        error: "We could not verify mechanic availability. Please try again.",
      };
    }

    const continuationParentIds = Array.from(
      new Set(
        (existingContinuationsRes.data ?? []).map((continuation) =>
continuation.appointment_id),
      ),
    );
    const continuationParents = continuationParentIds.length
      ? await supabaseAdmin
          .from("appointments")
          .select("id,is_archived,status,rescheduled_to_appointment_id")
          .in("id", continuationParentIds)
      : { data: [], error: null };
    if (continuationParents.error) {
```

```
      return {
        ok: false as const,
        error: "We could not verify mechanic availability. Please try again.",
      };
    }
    const activeContinuationParentIds = new Set(
      (continuationParents.data ?? [])
        .filter(
          (appointment) =>
            !appointment.is_archived &&
            !appointment.rescheduled_to_appointment_id &&
            isActiveReservationStatus(appointment.status),
        )
        .map((appointment) => appointment.id),
    );
    const existingAssignments = [
      ...(existingApptsRes.data ?? []).map((appointment) => ({
        date: data.date,
        startTime: String(appointment.start_time).slice(0, 5),
        durationMinutes: appointment.booking_duration_minutes ?? 60,
        crewId: appointment.assigned_crew_id,
      })),
      ...(existingContinuationsRes.data ?? [])
        .filter((continuation) =>
activeContinuationParentIds.has(continuation.appointment_id))
        .map((continuation) => ({
          date: data.date,
          startTime: String(continuation.start_time).slice(0, 5),
          durationMinutes: continuation.booking_duration_minutes,
          crewId: continuation.assigned_crew_id,
        })),
      // A customer's own pending request never blocks their new requested time.
      ...rescheduleHoldAssignments(existingHoldsRes.data ?? [],
rescheduledFrom?.id),
    ];
    const { availableCrewIds, remainingCapacity } = evaluateCrewAvailability({
      date: data.date,
      startTime,
      durationMinutes: firstDayDuration,
      capacity: slot.capacity,
      schedules: effectiveSchedules,
      exceptions: exceptionsRes.data ?? [],
      assignments: existingAssignments,
    });
    if (remainingCapacity <= 0) {
```

```
      return {
        ok: false as const,
        error: "No mechanic is available at that time. Please pick another slot.",
      };
    }

    // 5. Assign the free mechanic with the fewest bookings that day.
    const assignedMechanicId =
      pickLeastBusyCrew(availableCrewIds, existingAssignments, data.date) ??
null;

    const total = resolvedServices.reduce((sum, service) => sum + service.price,
0);

    const continuationPlan = needsContinuation
      ? await planContinuationSegments({
          supabaseAdmin,
          firstDate: data.date,
          remainingMinutes: totalDuration - firstDayDuration,
          lastAvailableDate,
          operatingHours: bookingRules.operatingHours,
          slotConfig,
        })
      : { ok: true as const, segments: [] as ContinuationSegment[] };
    if (!continuationPlan.ok) {
      return { ok: false as const, error: continuationPlan.error };
    }

    if (rescheduledFrom) {
      const request = await supabaseAdmin.rpc("submit_reschedule_request", {
        p_appointment_id: rescheduledFrom.id,
        p_request_id: data.idempotencyKey,
        p_appointment_date: data.date,
        p_start_time: `${startTime}:00`,
        p_reason:
          data.rescheduleReason?.trim() || "Customer requested a new appointment
date and time.",
      });

      if (request.error) {
        console.error("[Rescheduling] request write failed", {
          code: request.error.code,
          message: request.error.message,
          details: request.error.details,
        });
```

```
        if (
          /PGRST202|submit_reschedule_request|function .* does not exist/i.test(
            request.error.message,
          )
        ) {
          return {
            ok: false as const,
            error: "Rescheduling setup is incomplete. The shop needs to apply its
database update.",
          };
        }
        return {
          ok: false as const,
          error:
            request.error.message ||
            "We could not submit your reschedule request. Please try again.",
        };
      }

      const reservedReference = await supabaseAdmin
        .from("appointments")
        .select("pending_reschedule_reference_code")
        .eq("id", rescheduledFrom.id)
        .maybeSingle();
      if (reservedReference.error ||
!reservedReference.data?.pending_reschedule_reference_code) {
        return {
          ok: false as const,
          error:
            "Your reschedule request was saved, but we could not prepare its new
reference. Please try again.",
        };
      }

      return {
        ok: true as const,
        reference: reservedReference.data.pending_reschedule_reference_code,
        total: rescheduledFrom.total_estimate,
        rescheduleRequested: true as const,
      };
    }

    let reference = makeReference();
    for (let attempt = 0; attempt < 4; attempt++) {
      const existing = await supabaseAdmin
```

```
        .from("appointments")
        .select("id")
        .or(`reference_code.eq.${reference},pending_reschedule_reference_code.e
q.${reference}`)
        .maybeSingle();
      if (!existing.data) break;
      reference = makeReference();
    }

    const atomicInput = {
      p_reference_code: reference,
      p_booking_request_id: data.idempotencyKey,
      p_customer_name: customerName,
      p_phone: data.phone,
      p_email: data.email ?? "",
      p_moto_brand: data.motoBrand,
      p_moto_model: data.motoModel,
      p_moto_variant: data.motoVariant || null,
      p_moto_year: data.motoYear,
      p_plate_number: data.plateNumber.toUpperCase(),
      p_appointment_date: data.date,
      p_start_time: `${startTime}:00`,
      p_notes: data.notes || null,
      p_wants_products: data.wantsProducts,
      p_total_estimate: total,
      p_booking_duration_minutes: firstDayDuration,
      p_assigned_crew_id: assignedMechanicId,
      p_services: resolvedServices.map((service) => ({
        service_id: service.id,
        service_name: service.name,
        price: service.price,
        duration_minutes: service.durationMinutes,
      })),
      p_continuation_segments: continuationPlan.segments.map((segment) => ({
        segment_number: segment.segmentNumber,
        appointment_date: segment.appointmentDate,
        start_time: `${segment.startTime}:00`,
        booking_duration_minutes: segment.bookingDurationMinutes,
        assigned_crew_id: segment.assignedCrewId,
      })),
      p_notification_title: rescheduledFrom
        ? `Rescheduled appointment ${reference}`
        : `New booking ${reference}`,
      p_notification_message: `${customerName} ${rescheduledFrom ?
"rescheduled" : "booked"} ${resolvedServices.map((service) =>
```

```
service.name).join(", ")} on ${data.date}.${continuationPlan.segments.length ? `
Continued across ${continuationPlan.segments.length + 1} shop days.` : ""}`,
      p_first_name: data.firstName.trim(),
      p_middle_name: data.middleName.trim(),
      p_last_name: data.lastName.trim(),
    };
    const inserted = await supabaseAdmin.rpc("create_booking_atomic",
atomicInput);

    if (inserted.error?.code === "23P01") {
      return {
        ok: false as const,
        error: "That time was just booked. Please choose another available slot.",
      };
    }

    if (
      (inserted.error?.code === "23505" &&
        /already (?:have|has) an appointment for/i.test(inserted.error.message)) ||
      (inserted.error?.code === "23514" && /upcoming
appointments/i.test(inserted.error.message))
    ) {
      return {
        ok: false as const,
        error: inserted.error.message,
      };
    }

    if (inserted.error) {
      console.error("[Booking] atomic database write failed", {
        code: inserted.error.code,
        message: inserted.error.message,
        details: inserted.error.details,
        hint: inserted.error.hint,
      });
      if (
        inserted.error.code === "PGRST202" ||
        /could not find the function
public\.create_booking_atomic/i.test(inserted.error.message)
      ) {
        return {
          ok: false as const,
          error:
            "Booking setup is incomplete. The shop needs to apply its booking
database update.",
```

```
        };
      }
      return { ok: false as const, error: "We could not save your booking. Please try
again." };
    }

    // `create_booking_atomic` is idempotent, but a proxy/RPC response can be
    // empty even after PostgreSQL has committed the booking. Resolve the
saved
    // appointment from its request key instead of withholding a real reference
    // code because of that transport-level response shape.
    const savedBooking = await supabaseAdmin
      .from("appointments")
      .select("reference_code,total_estimate")
      .eq("booking_request_id", data.idempotencyKey)
      .maybeSingle();
    if (savedBooking.error || !savedBooking.data) {
      console.error("[Booking] could not resolve saved atomic booking", {
        bookingRequestId: data.idempotencyKey,
        code: savedBooking.error?.code,
        message: savedBooking.error?.message,
      });
      return {
        ok: false as const,
        error: "Your booking could not be confirmed. Please try again.",
      };
    }

    const savedReference =
normalizeReferenceCode(savedBooking.data.reference_code);
    if (!REFERENCE_CODE_PATTERN.test(savedReference)) {
      console.error("[Booking] saved booking has an invalid reference", {
        bookingRequestId: data.idempotencyKey,
      });
      return {
        ok: false as const,
        error:
          "We saved your booking, but could not prepare its reference code. Please
contact the shop.",
      };
    }

    return {
      ok: true as const,
      reference: savedReference,
```

```
      total: Number(savedBooking.data.total_estimate),
      continuationSegments: continuationPlan.segments.map((segment) => ({
        date: segment.appointmentDate,
        startTime: segment.startTime,
        durationMinutes: segment.bookingDurationMinutes,
      })),
    };
  });

const appointmentReferenceSchema = z.object({
  reference: z
    .string()
    .trim()
    .regex(REFERENCE_CODE_PATTERN, "Enter a valid reference code in the
format FRM-XXXXXX.")
    .transform(normalizeReferenceCode),
});

const appointmentIdentitySchema = appointmentReferenceSchema.extend({
  phone: phoneSchema,
});

// Issued by verifyAppointmentActionCode. Its format is checked with the token
itself,
// so a missing or malformed value gets the "verify again" response, not a parse
error.
const verificationTokenSchema = z.string().trim().max(100).optional();

const appointmentActionCodeSchema = appointmentReferenceSchema.extend({
  action: z.enum(["cancel", "reschedule"]),
});

const appointmentActionVerifySchema =
appointmentActionCodeSchema.extend({
  code: z
    .string()
    .trim()
    .regex(/^\d{6}$/, "Enter the 6-digit verification code."),
});

const MANAGED_APPOINTMENT_FIELDS =
  "id,reference_code,customer_name,first_name,middle_name,last_name,phone,
email,moto_brand,moto_model,moto_variant,moto_year,plate_number,appointm
ent_date,start_time,status,notes,total_estimate,wants_products,created_at,resch
eduled_from_appointment_id,rescheduled_to_appointment_id,reschedule_count,
```

```
last_reschedule_rejected_at,last_reschedule_rejection_message,pending_resch
edule_request_id,pending_reschedule_date,pending_reschedule_start_time,pen
ding_reschedule_reason,pending_reschedule_reference_code,appointment_serv
ices(service_id,service_name,price)";

async function findAppointmentByReference(reference: string) {
  const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
  const primaryLookup = await supabaseAdmin
    .from("appointments")
    .select(MANAGED_APPOINTMENT_FIELDS)
    // Legacy records may have been written with lower-case reference codes.
    // Input is validated before this query, so it cannot introduce LIKE wildcards.
    .ilike("reference_code", normalizeReferenceCode(reference))
    .maybeSingle();

  // Keep the appointment tracker usable during a rolling deployment: the
  // linked-booking column is introduced by the reschedule migration and may
  // not be present in the database while the application code is already live.
  const requiresLegacyLookup =
    primaryLookup.error &&
    ["42703", "PGRST204"].includes(primaryLookup.error.code) &&

primaryLookup.error.message.includes("pending_reschedule_reference_code");
  const legacyLookup = requiresLegacyLookup
    ? await supabaseAdmin
        .from("appointments")
        .select(
          "id,reference_code,customer_name,first_name,middle_name,last_name,p
hone,email,moto_brand,moto_model,moto_variant,moto_year,plate_number,app
ointment_date,start_time,status,notes,total_estimate,wants_products,created_at,
rescheduled_from_appointment_id,rescheduled_to_appointment_id,reschedule_
count,last_reschedule_rejected_at,last_reschedule_rejection_message,pending_
reschedule_request_id,pending_reschedule_date,pending_reschedule_start_tim
e,pending_reschedule_reason,appointment_services(service_id,service_name,pr
ice)",
        )
        .ilike("reference_code", normalizeReferenceCode(reference))
        .maybeSingle()
    : null;
  const appointment = legacyLookup?.data
    ? { ...legacyLookup.data, pending_reschedule_reference_code: null }
    : primaryLookup.data;
  if (!appointment) return null;
```

```
  // Normalize legacy phone records when possible so downstream rescheduling
  // continues to use the canonical local mobile format.
  return {
    ...appointment,
    phone: normalizePhilippineMobile(appointment.phone) ?? appointment.phone,
  };
}

/**
 * A reschedule request reserves its new reference on the original booking; the
 * new booking only exists once the shop approves it. Until the shop reviews
 * the request, that reference finds the original booking.
 */
async function findAppointmentByPendingRescheduleReference(reference:
string) {
  const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
  const { data: appointment, error } = await supabaseAdmin
    .from("appointments")
    .select(MANAGED_APPOINTMENT_FIELDS)
    .ilike("pending_reschedule_reference_code",
normalizeReferenceCode(reference))
    .not("pending_reschedule_request_id", "is", null)
    .maybeSingle();
  if (error) console.error("[Appointment lookup] reschedule request lookup failed",
error);
  if (!appointment) return null;
  return {
    ...appointment,
    phone: normalizePhilippineMobile(appointment.phone) ?? appointment.phone,
  };
}

async function findAppointment(reference: string, phone: string) {
  const appointment = await findAppointmentByReference(reference);
  if (!appointment || normalizePhilippineMobile(appointment.phone) !== phone)
return null;
  return appointment;
}

type ManagedAppointment = NonNullable<Awaited<ReturnType<typeof
findAppointmentByReference>>>;

/** Enough of an address for customers to recognise it, e.g. "ju•••@gmail.com". */
/** The network prefix and last 2 digits, e.g. "0917 ••• ••67". */
```

```
function maskPhone(phone: string) {
  const local = normalizePhilippineMobile(phone);
  return local ? `${local.slice(0, 4)} ••• ••${local.slice(-2)}` : "On file";
}

function maskEmail(email: string) {
  const at = email.lastIndexOf("@");
  if (at < 1) return "your booking email";
  const local = email.slice(0, at);
  return `${local.slice(0, local.length > 2 ? 2 : 1)}•••${email.slice(at)}`;
}

function cancellationBlockedReason(appt: ManagedAppointment, rules:
BookingRules) {
  if (appt.rescheduled_to_appointment_id) {
    return "This appointment has already been rescheduled. Please use the new
reference.";
  }
  if (appt.pending_reschedule_request_id) {
    return "A reschedule request is already awaiting the shop's review.";
  }
  if (!["pending", "confirmed", "rescheduled"].includes(appt.status)) {
    return "This appointment can no longer be cancelled online. Please call the
shop.";
  }
  if (
    !isSlotBookable(
      appt.appointment_date,
      String(appt.start_time).slice(0, 5),
      rules.cancellationNoticeHours,
    )
  ) {
    return `Cancellations need ${rules.cancellationNoticeHours} hours notice.
Please call the shop instead.`;
  }
  return null;
}

function rescheduleBlockedReason(appt: ManagedAppointment, rules:
BookingRules) {
  if (appt.rescheduled_to_appointment_id) {
    return "This appointment has already been rescheduled. Please use the new
reference.";
  }
  if (appt.pending_reschedule_request_id) {
```

```
    return "A reschedule request for this appointment is already awaiting the
shop's review.";
  }
  if (appt.reschedule_count >= 3) {
    return "This appointment has reached the maximum of 3 reschedules.";
  }
  if (!["pending", "confirmed"].includes(appt.status)) {
    return "This appointment can no longer be rescheduled online. Please call the
shop.";
  }
  if (
    !isSlotBookable(
      appt.appointment_date,
      String(appt.start_time).slice(0, 5),
      rules.reschedulingNoticeHours,
    )
  ) {
    return `Rescheduling needs ${rules.reschedulingNoticeHours} hours notice.
Please call the shop instead.`;
  }
  return null;
}

/**
 * Online cancellation and rescheduling need a token from
verifyAppointmentActionCode,
 * which proves access to the booking's email. Returns the response to send
when the
 * token is missing, expired, or cannot be checked; null when the action may
proceed.
 */
async function requireActionVerification(
  appointmentId: string,
  action: AppointmentAction,
  token: string | undefined,
) {
  const { checkActionToken } = await import("./appointment-verification.server");
  const result = await checkActionToken(appointmentId, action, token);
  if (result === "valid") return null;
  if (result === "unavailable") {
    return {
      ok: false as const,
      error: "Email verification is temporarily unavailable. Please try again shortly.",
    };
  }
```

```
  return {
    ok: false as const,
    verificationRequired: true as const,
    error:
      action === "cancel"
        ? "Please verify your email again to cancel this appointment."
        : "Please verify your email again to submit your reschedule request.",
  };
}

export const getRescheduleDetails = createServerFn({ method: "POST" })
  .validator((input: unknown) =>
    appointmentIdentitySchema.extend({ verificationToken:
verificationTokenSchema }).parse(input),
  )
  .handler(async ({ data }) => {
    const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
    const rateLimit = await checkPublicRequestRateLimit("rescheduling", 5, 15 *
60, data.phone);
    if (rateLimit !== "allowed") {
      return {
        ok: false as const,
        error:
          rateLimit === "limited"
            ? "Too many reschedule attempts. Please wait a few minutes before
trying again."
            : "Appointment rescheduling is temporarily unavailable. Please try again
shortly.",
      };
    }

    const appt = await findAppointment(data.reference, data.phone);
    if (!appt) {
      return {
        ok: false as const,
        error: "No appointment found for that reference code and mobile number.",
      };
    }
    // These details include the customer's email, so they need the same email
    // verification as the reschedule itself.
    const verification = await requireActionVerification(
      appt.id,
      "reschedule",
      data.verificationToken,
```

```
    );
    if (verification) {
      return {
        ok: false as const,
        error:
          "verificationRequired" in verification
            ? "For your security, reschedule from the My Appointment page. We'll
email a verification code to the address on your booking."
            : verification.error,
      };
    }
    if (appt.rescheduled_to_appointment_id) {
      return {
        ok: false as const,
        error: "This appointment has already been rescheduled. Please use the new
reference.",
      };
    }
    if (appt.pending_reschedule_request_id) {
      return {
        ok: false as const,
        error: "A reschedule request for this appointment is already awaiting the
shop's review.",
      };
    }
    // A completed reschedule retains the original appointment for history,
    // but only pending and confirmed appointments are valid reschedule
    // sources for the database RPC.
    if (!["pending", "confirmed"].includes(appt.status)) {
      return {
        ok: false as const,
        error: "This appointment can no longer be rescheduled online. Please call
the shop.",
      };
    }

    const bookingRules = await getBookingRules();
    if (
      !isSlotBookable(
        appt.appointment_date,
        String(appt.start_time).slice(0, 5),
        bookingRules.reschedulingNoticeHours,
      )
    ) {
      return {
```

```
        ok: false as const,
        error: `Rescheduling needs ${bookingRules.reschedulingNoticeHours}
hours notice. Please call the shop instead.`,
      };
    }

    const continuations = await supabaseAdmin
      .from("appointment_continuations")
      .select("id", { count: "exact", head: true })
      .eq("appointment_id", appt.id);
    if (continuations.error) {
      console.error("[Reschedule] continuation lookup failed", continuations.error);
      return {
        ok: false as const,
        error: "We could not verify the original appointment schedule. Please try
again.",
      };
    }
    if ((continuations.count ?? 0) > 0) {
      return {
        ok: false as const,
        error:
          "This appointment includes a multi-day service. Please call the shop to
reschedule all reserved service days together.",
      };
    }

    const serviceIds = (appt.appointment_services ?? []).flatMap((service) =>
      service.service_id ? [service.service_id] : [],
    );
    if (serviceIds.length !== (appt.appointment_services ?? []).length ||
serviceIds.length === 0) {
      return {
        ok: false as const,
        error:
          "The original services are no longer available for online rescheduling.
Please call the shop.",
      };
    }
    const activeServices = await supabaseAdmin
      .from("services")
      .select("id")
      .in("id", serviceIds)
      .eq("is_active", true)
      .eq("is_archived", false);
```

```
    if (activeServices.error || activeServices.data.length !== serviceIds.length) {
      return {
        ok: false as const,
        error:
          "One or more original services are no longer available for online
rescheduling. Please call the shop.",
      };
    }

    return {
      ok: true as const,
      appointment: {
        customerName: appt.customer_name,
        firstName: appt.first_name ?? "",
        middleName: appt.middle_name ?? "",
        lastName: appt.last_name ?? "",
        phone: appt.phone,
        email: appt.email ?? "",
        motoBrand: appt.moto_brand,
        motoModel: appt.moto_model,
        motoVariant: appt.moto_variant ?? "",
        motoYear: String(appt.moto_year ?? new Date().getFullYear()),
        plateNumber: appt.plate_number,
        notes: appt.notes ?? "",
        wantsProducts: appt.wants_products,
        serviceIds,
      },
    };
  });

export const lookupAppointment = createServerFn({ method: "POST" })
  .validator((input: unknown) => appointmentReferenceSchema.parse(input))
  .handler(async ({ data }) => {
    const lookupRateLimit = await checkPublicRequestRateLimit(
      "lookup",
      12,
      10 * 60,
      data.reference,
    );
    if (lookupRateLimit !== "allowed") {
      return {
        ok: false as const,
        error:
          lookupRateLimit === "limited"
            ? "Too many lookup attempts. Please wait a few minutes before trying
```

```
again."
          : "Appointment lookup is temporarily unavailable. Please try again
shortly.",
    };
  }

  const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
  const booking = await findAppointmentByReference(data.reference);
  const appt = booking ?? (await
findAppointmentByPendingRescheduleReference(data.reference));
  if (!appt)
    return {
      ok: false as const,
      error: "No appointment found for that reference code.",
    };
  const linkedIds = [
    appt.rescheduled_from_appointment_id,
    appt.rescheduled_to_appointment_id,
  ].filter((id): id is string => Boolean(id));
  const linkedReferences = new Map<string, string>();
  if (linkedIds.length > 0) {
    const linked = await supabaseAdmin
      .from("appointments")
      .select("id,reference_code")
      .in("id", linkedIds);
    if (!linked.error) {
      for (const row of linked.data ?? []) linkedReferences.set(row.id,
row.reference_code);
    }
  }
  return {
    ok: true as const,
    appointment: {
      reference: appt.reference_code,
      customerName: appt.customer_name,
      firstName: appt.first_name ?? "",
      middleName: appt.middle_name ?? "",
      lastName: appt.last_name ?? "",
      // Anyone with the reference code can look a booking up, so contact details
      // are masked. Rescheduling gets the full number after email verification.
      phoneHint: maskPhone(appt.phone),
      // Where cancel/reschedule verification codes go; null means no email on
file.
      emailHint: appt.email ? maskEmail(appt.email) : null,
```

```
        motorcycle: [appt.moto_brand, appt.moto_model, appt.moto_variant,
appt.moto_year]
          .filter(Boolean)
          .join(" "),
        plateNumber: appt.plate_number,
        date: appt.appointment_date,
        startTime: String(appt.start_time).slice(0, 5),
        status: appt.status,
        notes: appt.notes,
        total: Number(appt.total_estimate),
        hasReplacement: Boolean(appt.rescheduled_to_appointment_id),
        rescheduleCount: appt.reschedule_count,
        rescheduleRequestPending:
Boolean(appt.pending_reschedule_request_id),
        rescheduleRequestRejected: Boolean(appt.last_reschedule_rejected_at),
        rescheduleRequestRejectionMessage:
appt.last_reschedule_rejection_message,
        requestedRescheduleDate: appt.pending_reschedule_date,
        requestedRescheduleStartTime: appt.pending_reschedule_start_time
          ? String(appt.pending_reschedule_start_time).slice(0, 5)
          : null,
        requestedRescheduleReference:
appt.pending_reschedule_reference_code,
        // The customer entered the reference reserved by a reschedule request
        // the shop has not reviewed yet; the details are the original booking's.
        foundByRescheduleRequest: !booking,
        requestedRescheduleReason: appt.pending_reschedule_reason,
        rescheduledFromReference: appt.rescheduled_from_appointment_id
          ? (linkedReferences.get(appt.rescheduled_from_appointment_id) ?? null)
          : null,
        rescheduledToReference: appt.rescheduled_to_appointment_id
          ? (linkedReferences.get(appt.rescheduled_to_appointment_id) ?? null)
          : null,
        services: (appt.appointment_services ?? []).map((s) => ({
          serviceId: s.service_id,
          name: s.service_name,
          price: Number(s.price),
        })),
      },
    };
  });

export const cancelAppointment = createServerFn({ method: "POST" })
  .validator((input: unknown) =>
    appointmentReferenceSchema.extend({ verificationToken:
```

```
verificationTokenSchema }).parse(input),
  )
  .handler(async ({ data }) => {
    const cancellationRateLimit = await checkPublicRequestRateLimit(
      "cancellation",
      3,
      15 * 60,
      data.reference,
    );
    if (cancellationRateLimit !== "allowed") {
      return {
        ok: false as const,
        error:
          cancellationRateLimit === "limited"
            ? "Too many cancellation attempts. Please wait a few minutes before
trying again."
            : "Appointment cancellation is temporarily unavailable. Please try again
shortly.",
      };
    }

    const appt = await findAppointmentByReference(data.reference);
    if (!appt)
      return {
        ok: false as const,
        error: "No appointment found for that reference code.",
      };
    const bookingRules = await getBookingRules();
    const blockedReason = cancellationBlockedReason(appt, bookingRules);
    if (blockedReason) return { ok: false as const, error: blockedReason };

    const verification = await requireActionVerification(appt.id, "cancel",
data.verificationToken);
    if (verification) return verification;

    const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
    const cancellation = await supabaseAdmin.rpc("cancel_public_appointment", {
      p_appointment_id: appt.id,
      p_cancellation_notice_hours: bookingRules.cancellationNoticeHours,
    });
    if (cancellation.error) {
      console.error("[Cancellation] atomic cancellation failed", {
        code: cancellation.error.code,
        message: cancellation.error.message,
```

```
    });
    if (
      /PGRST202|cancel_public_appointment|function .* does not exist/i.test(
        cancellation.error.message,
      )
    ) {
      return {
        ok: false as const,
        error: "Cancellation setup is incomplete. The shop needs to apply its
database update.",
      };
    }
    return {
      ok: false as const,
      error:
        cancellation.error.message || "We could not cancel the appointment.
Please try again.",
    };
  }
  return { ok: true as const };
 });

/** Email a one-time code that unlocks cancelling or rescheduling one
appointment. */
export const requestAppointmentActionCode = createServerFn({ method:
"POST" })
  .validator((input: unknown) => appointmentActionCodeSchema.parse(input))
  .handler(async ({ data }) => {
    const rateLimit = await checkPublicRequestRateLimit(
      "verification_request",
      5,
      15 * 60,
      data.reference,
    );
    if (rateLimit !== "allowed") {
      return {
        ok: false as const,
        error:
          rateLimit === "limited"
            ? "Too many verification codes requested. Please wait a few minutes
before trying again."
            : "Email verification is temporarily unavailable. Please try again shortly.",
      };
    }
```

```
    const appt = await findAppointmentByReference(data.reference);
    if (!appt) {
      return { ok: false as const, error: "No appointment found for that reference
code." };
    }
    if (!appt.email) {
      return {
        ok: false as const,
        error: `This booking has no email address for verification, so it can't be
changed online. Please call the shop at ${SHOP.phone}.`,
      };
    }

    // Check eligibility first so customers aren't sent a code they can't use.
    const bookingRules = await getBookingRules();
    const blockedReason =
      data.action === "cancel"
        ? cancellationBlockedReason(appt, bookingRules)
        : rescheduleBlockedReason(appt, bookingRules);
    if (blockedReason) return { ok: false as const, error: blockedReason };
    if (data.action === "reschedule") {
      const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
      const continuations = await supabaseAdmin
        .from("appointment_continuations")
        .select("id", { count: "exact", head: true })
        .eq("appointment_id", appt.id);
      if (continuations.error) {
        console.error("[Appointment verification] continuation lookup failed",
continuations.error);
        return {
          ok: false as const,
          error: "Email verification is temporarily unavailable. Please try again
shortly.",
        };
      }
      if ((continuations.count ?? 0) > 0) {
        return {
          ok: false as const,
          error:
            "This appointment includes a multi-day service. Please call the shop to
reschedule all reserved service days together.",
        };
      }
    }
```

```
    const verification = await import("./appointment-verification.server");
    const issued = await verification.issueActionCode(appt.id, data.action);
    if (!issued) {
      return {
        ok: false as const,
        error: "Email verification is temporarily unavailable. Please try again
shortly.",
      };
    }
    try {
      await verification.sendActionCodeEmail({
        to: appt.email,
        code: issued.code,
        action: data.action,
        reference: normalizeReferenceCode(appt.reference_code),
        date: appt.appointment_date,
        startTime: String(appt.start_time).slice(0, 5),
      });
    } catch (error) {
      console.error("[Appointment verification] code email failed", error);
      await verification.discardActionCode(issued.challengeId);
      return {
        ok: false as const,
        error: "We could not send the verification email. Please try again shortly.",
      };
    }

    return { ok: true as const, emailHint: maskEmail(appt.email) };
  });

/** Check an emailed code and issue a short-lived token for that one action. */
export const verifyAppointmentActionCode = createServerFn({ method: "POST"
})
  .validator((input: unknown) => appointmentActionVerifySchema.parse(input))
  .handler(async ({ data }) => {
    const rateLimit = await checkPublicRequestRateLimit(
      "verification_verify",
      10,
      15 * 60,
      data.reference,
    );
    if (rateLimit !== "allowed") {
      return {
        ok: false as const,
```

```
        error:
          rateLimit === "limited"
            ? "Too many verification attempts. Please wait a few minutes, then
request a new code."
            : "Email verification is temporarily unavailable. Please try again shortly.",
      };
    }

    const appt = await findAppointmentByReference(data.reference);
    if (!appt) {
      return { ok: false as const, error: "No appointment found for that reference
code." };
    }

    const { verifyActionCode } = await import("./appointment-verification.server");
    const result = await verifyActionCode(appt.id, data.action, data.code);
    switch (result.outcome) {
      case "verified":
        return {
          ok: true as const,
          token: result.token,
          expiresAt: result.expiresAt,
          // Rescheduling needs the full mobile number, which the lookup masks.
          ...(data.action === "reschedule"
            ? { phone: normalizePhilippineMobile(appt.phone) ?? appt.phone }
            : {}),
        };
      case "invalid":
        return {
          ok: false as const,
          error: "That code is incorrect. Check the latest email from us and try
again.",
        };
      case "expired":
        return {
          ok: false as const,
          error: "That code has expired or is no longer valid. Request a new code.",
        };
      default:
        return {
          ok: false as const,
          error: "Email verification is temporarily unavailable. Please try again
shortly.",
        };
    }
```

```
});
```

src/lib/customer-emails.ts

```
/**
 * Wording and formatting for customer emails and text messages. These are
 * plain functions with no secrets or network access, so the content can be
 * checked without sending.
 */

export function escapeHtml(value: string | number | null | undefined) {
  return String(value ?? "")
    .replaceAll("&", "&amp;")
    .replaceAll("<", "&lt;")
    .replaceAll(">", "&gt;")
    .replaceAll("", "&quot;")
    .replaceAll("'", "&#039;");
}

export function formatAppointmentDate(date: string) {
  return new Intl.DateTimeFormat("en-PH", {
    dateStyle: "full",
    timeZone: "Asia/Manila",
  }).format(new Date(`${date}T12:00:00+08:00`));
}

export function formatAppointmentTime(time: string) {
  const [hour = "0", minute = "0"] = time.slice(0, 5).split(":");
  const date = new Date(`2000-01-01T${hour.padStart(2,
"0")}:${minute}:00+08:00`);
  return new Intl.DateTimeFormat("en-PH", {
    hour: "numeric",
    minute: "2-digit",
    hour12: true,
    timeZone: "Asia/Manila",
  }).format(date);
}

/** A short date for text messages, e.g. "Oct 20, 2026". */
export function formatSmsDate(date: string) {
  return new Intl.DateTimeFormat("en-PH", {
    month: "short",
    day: "numeric",
    year: "numeric",
    timeZone: "Asia/Manila",
  }).format(new Date(`${date}T12:00:00+08:00`));
}
```

```
export type ShopContact = { name: string; phone: string; email: string };

export type EmailContent = { subject: string; text: string; html: string };

export type AppointmentSummary = {
  referenceCode: string;
  customerName: string;
  appointmentDate: string;
  startTime: string;
  services: string[];
  motorcycle: string;
  plateNumber: string;
};

function schedule(date: string, time: string) {
  return `${formatAppointmentDate(date)} at ${formatAppointmentTime(time)}`;
}

function contactText(contact: ShopContact) {
  return contact.email ? `${contact.phone} or ${contact.email}` : contact.phone;
}

function contactHtml(contact: ShopContact) {
  return contact.email
    ? `${escapeHtml(contact.phone)} or <a
href="mailto:${escapeHtml(contact.email)}">${escapeHtml(contact.email)}</a>`
    : escapeHtml(contact.phone);
}

function link(url: string, label: string) {
  return `<a href="${escapeHtml(url)}">${escapeHtml(label)}</a>`;
}

/** Sent when the shop rejects a pending booking request. */
export function bookingNotAcceptedEmail({
  appointment,
  contact,
  bookingUrl,
}: {
  appointment: AppointmentSummary;
  contact: ShopContact;
  /** The public booking page, when the site address is known. */
  bookingUrl?: string | null;
}): EmailContent {
```

```
  const requested = schedule(appointment.appointmentDate,
appointment.startTime);
  const services = appointment.services.join(", ") || "Not listed";
  const text = [
    `Hello ${appointment.customerName},`,
    "",
    `We're sorry, but we could not accept your booking request at
${contact.name}.`,
    "",
    `Reference code: ${appointment.referenceCode}`,
    `Requested schedule: ${requested}`,
    `Services: ${services}`,
    `Motorcycle: ${appointment.motorcycle} (plate ${appointment.plateNumber})`,
    "",
    "This time is no longer reserved for you. You are welcome to book a different
date and time" +
      (bookingUrl ? `: ${bookingUrl}` : " on our website."),
    `If you have questions, contact us at ${contactText(contact)}.`,
  ].join("\n");
  const html = [
    `<p>Hello ${escapeHtml(appointment.customerName)},</p>`,
    `<p>We're sorry, but we could not accept your booking request at
${escapeHtml(contact.name)}.</p>`,
    `<p><strong>Reference code:</strong>
${escapeHtml(appointment.referenceCode)}<br />`,
    `<strong>Requested schedule:</strong> ${escapeHtml(requested)}<br />`,
    `<strong>Services:</strong> ${escapeHtml(services)}<br />`,
    `<strong>Motorcycle:</strong> ${escapeHtml(appointment.motorcycle)} (plate
${escapeHtml(appointment.plateNumber)})</p>`,
    `<p>This time is no longer reserved for you. You are welcome to ${bookingUrl
? link(bookingUrl, "book a different date and time") : "book a different date and
time on our website"}.</p>`,
    `<p>If you have questions, contact us at ${contactHtml(contact)}.</p>`,
  ].join("");
  return {
    subject: `Booking request not accepted - ${appointment.referenceCode}`,
    text,
    html,
  };
}

/** Sent when the shop rejects a customer's reschedule request. */
export function rescheduleNotApprovedEmail({
  appointment,
  requestedDate,
```

```
  requestedStartTime,
  contact,
  appointmentUrl,
}: {
  /** The customer's current appointment, which stays as it is. */
  appointment: AppointmentSummary;
  requestedDate: string | null;
  requestedStartTime: string | null;
  contact: ShopContact;
  /** The public My Appointment page, when the site address is known. */
  appointmentUrl?: string | null;
}): EmailContent {
  const requested =
    requestedDate && requestedStartTime ? schedule(requestedDate,
requestedStartTime) : null;
  const current = schedule(appointment.appointmentDate,
appointment.startTime);
  const services = appointment.services.join(", ") || "Not listed";
  const request = requested
    ? `your request to move your appointment to ${requested}`
    : "your request to reschedule your appointment";
  const text = [
    `Hello ${appointment.customerName},`,
    "",
    `We could not approve ${request}.`,
    "",
    "Your current appointment has not changed:",
    `Reference code: ${appointment.referenceCode}`,
    `Schedule: ${current}`,
    `Services: ${services}`,
    "",
    "If this schedule no longer works for you, you can request a different date" +
      (appointmentUrl ? ` on the My Appointment page: ${appointmentUrl}` : " on
our website."),
    `If you have questions, contact us at ${contactText(contact)}.`,
  ].join("\n");
  const html = [
    `<p>Hello ${escapeHtml(appointment.customerName)},</p>`,
    `<p>We could not approve ${escapeHtml(request)}.</p>`,
    `<p><strong>Your current appointment has not changed:</strong><br />`,
    `<strong>Reference code:</strong>
${escapeHtml(appointment.referenceCode)}<br />`,
    `<strong>Schedule:</strong> ${escapeHtml(current)}<br />`,
    `<strong>Services:</strong> ${escapeHtml(services)}</p>`,
    `<p>If this schedule no longer works for you, you can request a different date
```

```
${appointmentUrl ? `on the ${link(appointmentUrl, "My Appointment page")}` : "on
our website"}.</p>`,
    `<p>If you have questions, contact us at ${contactHtml(contact)}.</p>`,
  ].join("");
  return {
    subject: `Reschedule request not approved - ${appointment.referenceCode}`,
    text,
    html,
  };
}

/** Sent before a confirmed appointment, as set in Settings. */
export function appointmentReminderEmail({
  appointment,
  contact,
  appointmentUrl,
}: {
  appointment: AppointmentSummary;
  contact: ShopContact;
  /** The public My Appointment page, when the site address is known. */
  appointmentUrl?: string | null;
}): EmailContent {
  const when = schedule(appointment.appointmentDate, appointment.startTime);
  const services = appointment.services.join(", ") || "Not listed";
  const text = [
    `Hello ${appointment.customerName},`,
    "",
    `This is a reminder of your appointment at ${contact.name}.`,
    "",
    `Reference code: ${appointment.referenceCode}`,
    `Schedule: ${when}`,
    `Services: ${services}`,
    `Motorcycle: ${appointment.motorcycle} (plate ${appointment.plateNumber})`,
    "",
    "Please arrive 10-15 minutes early for check-in.",
    "If you need to cancel or reschedule, you can do it" +
      (appointmentUrl ? ` on the My Appointment page: ${appointmentUrl}` : " on
our website.") +
      ` You can also contact us at ${contactText(contact)}.`,
  ].join("\n");
  const html = [
    `<p>Hello ${escapeHtml(appointment.customerName)},</p>`,
    `<p>This is a reminder of your appointment at
${escapeHtml(contact.name)}.</p>`,
    `<p><strong>Reference code:</strong>
```

```
${escapeHtml(appointment.referenceCode)}<br />`,
    `<strong>Schedule:</strong> ${escapeHtml(when)}<br />`,
    `<strong>Services:</strong> ${escapeHtml(services)}<br />`,
    `<strong>Motorcycle:</strong> ${escapeHtml(appointment.motorcycle)} (plate
${escapeHtml(appointment.plateNumber)})</p>`,
    "<p>Please arrive 10-15 minutes early for check-in.</p>",
    `<p>If you need to cancel or reschedule, you can do it ${appointmentUrl ? `on
the ${link(appointmentUrl, "My Appointment page")}` : "on our website"}. You can
also contact us at ${contactHtml(contact)}.</p>`,
  ].join("");
  return {
    subject: `Appointment reminder - ${appointment.referenceCode}`,
    text,
    html,
  };
}

function smsSchedule(date: string, time: string) {
  return `${formatSmsDate(date)} at ${formatAppointmentTime(time)}`;
}

/** Text message for a pending booking request the shop rejected (fits one SMS).
*/
export function bookingNotAcceptedSms({
  appointment,
  contact,
}: {
  appointment: AppointmentSummary;
  contact: ShopContact;
}) {
  return `Fake Rider: Sorry, we could not accept your booking request (Ref:
${appointment.referenceCode}) for
${smsSchedule(appointment.appointmentDate, appointment.startTime)}. Please
book another time or call ${contact.phone}.`;
}

/** Text message sent before a confirmed appointment. */
export function appointmentReminderSms({
  appointment,
  contact,
}: {
  appointment: AppointmentSummary;
  contact: ShopContact;
}) {
  return `Fake Rider: Reminder of your appointment on
```

```
${smsSchedule(appointment.appointmentDate, appointment.startTime)} (Ref:
${appointment.referenceCode}). Please arrive 10-15 min early. To cancel or
reschedule, call ${contact.phone}.`;
}

/** Text message for a reschedule request the shop rejected (fits one SMS). */
export function rescheduleNotApprovedSms({
  appointment,
  requestedDate,
  requestedStartTime,
  contact,
}: {
  /** The customer's current appointment, which stays as it is. */
  appointment: AppointmentSummary;
  requestedDate: string | null;
  requestedStartTime: string | null;
  contact: ShopContact;
}) {
  const request =
    requestedDate && requestedStartTime
      ? `Your request to move ${appointment.referenceCode} to
${smsSchedule(requestedDate, requestedStartTime)} was not approved.`
      : `Your reschedule request for ${appointment.referenceCode} was not
approved.`;
  return `Fake Rider: ${request} Your booking stays on
${smsSchedule(appointment.appointmentDate, appointment.startTime)}. Call
${contact.phone}.`;
}
```

src/lib/email.server.ts

```
/**
 * Server-only outgoing email for customer notifications.
 *
 * Gmail SMTP is used when GMAIL_USER and GMAIL_APP_PASSWORD are
set. The
 * password must be a Google App Password (Google Account > Security > App
 * passwords, which requires 2-Step Verification); Gmail rejects the normal
 * account password for SMTP. Without Gmail settings, Resend is used through
 * RESEND_API_KEY and RESEND_FROM_EMAIL.
 *
 * SMTP_HOST (with SMTP_PORT and SMTP_FROM) takes priority over both:
a plain,
 * unauthenticated mail server. The offline copy uses it to deliver to the local
 * Mailpit mailbox; the live site does not set it.
 */

const EMAIL_ADDRESS_PATTERN = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
const SENDER_NAME = "Fake Rider Motorparts";

export type EmailTransport =
  | { provider: "smtp"; host: string; port: number; from: string }
  | { provider: "gmail"; user: string; appPassword: string; from: string }
  | { provider: "resend"; apiKey: string; from: string };

export type OutgoingEmail = {
  to: string;
  subject: string;
  text: string;
  html?: string;
  /** Resend only: lets the provider ignore a repeated send of the same message.
*/
  idempotencyKey?: string;
};

function senderAddress(value: string) {
  const trimmed = value.trim();
  const namedAddress = trimmed.match(/^[^<>]*<\s*([^<>\s]+)\s*>$/);
  const address = (namedAddress?.[1] ?? trimmed).toLowerCase();
  return EMAIL_ADDRESS_PATTERN.test(address) ? address : null;
}

/** Remove credentials and personal details before provider errors are logged or
stored. */
```

```
function redactProviderDetail(value: string) {
  return value
    .replace(/\bre_[A-Za-z0-9_-]+\b/g, "[redacted-api-key]")
    .replace(/[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}/gi, "[redacted-email]")
    .replace(/ /g, " ")
    .trim()
    .slice(0, 400);
}

export function emailTransport(): { transport: EmailTransport } | { error: string } {
  const smtpHost = process.env["SMTP_HOST"]?.trim();
  if (smtpHost) {
    const port = Number(process.env["SMTP_PORT"] ?? 25);
    const from = process.env["SMTP_FROM"]?.trim() ?? "";
    if (!Number.isInteger(port) || port <= 0 || !senderAddress(from)) {
      return {
        error: "Local email is not configured. Set SMTP_PORT and a valid
SMTP_FROM address.",
      };
    }
    return {
      transport: {
        provider: "smtp",
        host: smtpHost,
        port,
        from: `"${SENDER_NAME}" <${senderAddress(from)}>`,
      },
    };
  }
  const gmailUser = process.env["GMAIL_USER"]?.trim();
  // Google shows App Passwords in groups of four characters; spaces are not
part of it.
  const gmailAppPassword =
process.env["GMAIL_APP_PASSWORD"]?.replace(/ /g, "");
  if (gmailUser || gmailAppPassword) {
    if (!gmailUser || !gmailAppPassword) {
      return {
        error:
          "Gmail email is partly configured. Set both GMAIL_USER and
GMAIL_APP_PASSWORD in the server environment.",
      };
    }
    if (!EMAIL_ADDRESS_PATTERN.test(gmailUser)) {
      return { error: "Email is not configured correctly. GMAIL_USER must be an
email address." };
```

```
    }
    return {
      transport: {
        provider: "gmail",
        user: gmailUser,
        appPassword: gmailAppPassword,
        from: `"${SENDER_NAME}" <${gmailUser}>`,
      },
    };
  }

  const apiKey = process.env["RESEND_API_KEY"]?.trim();
  const from = process.env["RESEND_FROM_EMAIL"]?.trim();
  if (!apiKey || !from) {
    return {
      error:
        "Email is not configured. Set GMAIL_USER and GMAIL_APP_PASSWORD
(or RESEND_API_KEY and RESEND_FROM_EMAIL) in the server
environment.",
    };
  }
  if (!senderAddress(from)) {
    return {
      error:
        "Email is not configured correctly. RESEND_FROM_EMAIL must be a valid
email address or display-name address.",
    };
  }
  return { transport: { provider: "resend", apiKey, from } };
}

async function sendWithSmtp(
  transport: Extract<EmailTransport, { provider: "smtp" }>,
  email: OutgoingEmail,
) {
  const { createTransport } = await import("nodemailer");
  const mailer = createTransport({ host: transport.host, port: transport.port,
secure: false });
  try {
    await mailer.sendMail({
      from: transport.from,
      to: email.to,
      subject: email.subject,
      text: email.text,
      ...(email.html ? { html: email.html } : {}),
```

```
    });
  } catch (error) {
    const detail = redactProviderDetail(error instanceof Error ? error.message :
String(error));
    throw new Error(`The local mail server rejected the email: ${detail}.`);
  } finally {
    mailer.close();
  }
}

async function sendWithGmail(
  transport: Extract<EmailTransport, { provider: "gmail" }>,
  email: OutgoingEmail,
) {
  const { createTransport } = await import("nodemailer");
  const mailer = createTransport({
    host: "smtp.gmail.com",
    port: 465,
    secure: true,
    auth: { user: transport.user, pass: transport.appPassword },
  });
  try {
    await mailer.sendMail({
      from: transport.from,
      to: email.to,
      subject: email.subject,
      text: email.text,
      ...(email.html ? { html: email.html } : {}),
    });
  } catch (error) {
    const detail = redactProviderDetail(error instanceof Error ? error.message :
String(error));
    const responseCode = (error as { responseCode?: number } |
null)?.responseCode;
    const hint =
      responseCode === 535 || /username and password not accepted|invalid
login/i.test(detail)
        ? " GMAIL_APP_PASSWORD must be a Google App Password, not the
account password."
        : "";
    throw new Error(`Gmail SMTP rejected the email: ${detail}.${hint}`);
  } finally {
    mailer.close();
  }
}
```

```
async function sendWithResend(
  transport: Extract<EmailTransport, { provider: "resend" }>,
  email: OutgoingEmail,
) {
  const response = await fetch("https://api.resend.com/emails", {
    method: "POST",
    headers: {
      Authorization: `Bearer ${transport.apiKey}`,
      "Content-Type": "application/json",
      ...(email.idempotencyKey ? { "Idempotency-Key": email.idempotencyKey } :
{}),
    },
    body: JSON.stringify({
      from: transport.from,
      to: [email.to],
      subject: email.subject,
      text: email.text,
      ...(email.html ? { html: email.html } : {}),
    }),
  });
  if (response.ok) return;

  const body = await response.text();
  const detail = redactProviderDetail(body);
  // Resend's shared testing sender only delivers to the Resend account owner.
  const testingSender = /@resend\.dev>?$/i.test(transport.from)
    ? " RESEND_FROM_EMAIL uses Resend's testing sender, which only
delivers to the Resend account owner's address."
    : "";
  throw new Error(
    `Resend returned ${response.status}${detail ? `: ${detail}` :
"."}${testingSender}`,
  );
}

/** Send one email; throws with a redacted provider reason when delivery is
refused. */
export async function sendEmail(transport: EmailTransport, email:
OutgoingEmail) {
  if (transport.provider === "smtp") await sendWithSmtp(transport, email);
  else if (transport.provider === "gmail") await sendWithGmail(transport, email);
  else await sendWithResend(transport, email);
}
```

src/lib/error-capture.ts

```
// Captures the original Error out-of-band so server.ts can recover the stack
// when h3 has already swallowed the throw into a generic 500 Response.

let lastCapturedError: { error: unknown; at: number } | undefined;
const TTL_MS = 5_000;

function record(error: unknown) {
  lastCapturedError = { error, at: Date.now() };
}

// h3's HTTPError serializes to
{"status":500,"unhandled":true,"message":"HTTPError"} —
// no stack, no cause — so a plain console.error(error) reaches the log pipeline
with
// the failure detail stripped. Expand Error-like args into a string that keeps the
// message, stack, and the full cause chain.
const CAUSE_DEPTH_LIMIT = 5;
const DESCRIPTION_LENGTH_LIMIT = 8_000;

export function describeError(error: unknown): string {
  const parts: string[] = [];
  let current: unknown = error;
  for (let depth = 0; depth < CAUSE_DEPTH_LIMIT && current != null; depth++) {
    if (!(current instanceof Error)) {
      parts.push(typeof current === "string" ? current : safeStringify(current));
      break;
    }
    const label = depth === 0 ? "" : "caused by: ";
    const status = describeStatus(current);
    parts.push(`${label}${current.stack ?? `${current.name}:
${current.message}`}${status}`);
    current = current.cause;
  }
  return parts.join("\n").slice(0, DESCRIPTION_LENGTH_LIMIT);
}

function describeStatus(error: Error): string {
  const { status, statusCode } = error as { status?: unknown; statusCode?:
unknown };
  const value = status ?? statusCode;
  return typeof value === "number" ? ` (status ${value})` : "";
}
```

```
function safeStringify(value: unknown): string {
  try {
    return JSON.stringify(value) ?? String(value);
  } catch {
    return String(value);
  }
}

function isErrorLike(value: unknown): value is Error {
  return value instanceof Error;
}

// Wrap console.error so errors logged by any layer — including h3's internal
// unhandled-error logging, which this file cannot hook directly — are both
// recorded for consumeLastCapturedError and expanded before serialization.
const originalConsoleError = console.error.bind(console);
console.error = (...args: unknown[]) => {
  const expanded = args.map((arg) => {
    if (!isErrorLike(arg)) return arg;
    record(arg);
    return describeError(arg);
  });
  originalConsoleError(...expanded);
};

if (typeof globalThis.addEventListener === "function") {
  globalThis.addEventListener("error", (event) => record((event as
ErrorEvent).error ?? event));
  globalThis.addEventListener("unhandledrejection", (event) =>
    record((event as PromiseRejectionEvent).reason),
  );
}

export function consumeLastCapturedError(): unknown {
  if (!lastCapturedError) return undefined;
  if (Date.now() - lastCapturedError.at > TTL_MS) {
    lastCapturedError = undefined;
    return undefined;
  }
  const { error } = lastCapturedError;
  lastCapturedError = undefined;
  return error;
}
```

src/lib/error-page.ts

```
export function renderErrorPage(): string {
  return `<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <title>This page didn't load</title>
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <style>
      body { font: 15px/1.5 system-ui, -apple-system, sans-serif; background:
#fafafa; color: #111; display: grid; place-items: center; min-height: 100vh; margin:
0; padding: 1.5rem; }
      .card { max-width: 28rem; width: 100%; text-align: center; padding: 2rem; }
      h1 { font-size: 1.25rem; margin: 0 0 0.5rem; }
      p { color: #4b5563; margin: 0 0 1.5rem; }
      .actions { display: flex; gap: 0.5rem; justify-content: center; flex-wrap: wrap; }
      a, button { padding: 0.5rem 1rem; border-radius: 0.375rem; font: inherit;
cursor: pointer; text-decoration: none; border: 1px solid transparent; }
      .primary { background: #111; color: #fff; }
      .secondary { background: #fff; color: #111; border-color: #d1d5db; }
    </style>
  </head>
  <body>
    <div class="card">
      <h1>This page didn't load</h1>
      <p>Something went wrong on our end. You can try refreshing or head back
home.</p>
      <div class="actions">
        <button class="primary" onclick="location.reload()">Try again</button>
        <a class="secondary" href="/">Go home</a>
      </div>
    </div>
  </body>
</html>`;
}
```

src/lib/export-branding.ts

```
export const SHOP_EXPORT_NAME = "FAKE RIDER MOTORPARTS";
export const SHOP_OWNER_NAME = "JOEMAR TATO";

export function formatBusinessTimestamp() {
  return new Date().toLocaleString("en-PH", {
    timeZone: "Asia/Manila",
    day: "2-digit",
    month: "short",
    year: "numeric",
    hour: "numeric",
    minute: "2-digit",
  });
}
```

src/lib/offline-sync.server.ts

```
/**
 * Offline copy only: sends bookings made offline to the live site once the
 * internet is back.
 *
 * Every minute the local database (offline/setup.sql) calls /api/offline-sync,
 * which checks whether the live database can be reached. If it can, each
 * booking made in the offline copy is created on the live site with the same
 * reference code, oldest first. A booking that no longer fits there (the time
 * was taken online meanwhile, for example) is reported to the admin to call the
 * customer. Off unless OFFLINE_SYNC_TO_LIVE=1 in .env.offline.
 */
import { createClient } from "@supabase/supabase-js";

import type { Database } from "@/integrations/supabase/types";

import { formatAppointmentTime, formatSmsDate } from "./customer-emails";

type SyncConfig = {
  liveUrl: string;
  liveKey: string;
  /** When the live data was copied: bookings created after it were made offline.
*/
  copiedAt: Date;
  /** The live website, where people are sent once the internet is back. */
  liveSiteUrl: string | null;
};

export function offlineSyncConfig(): SyncConfig | null {
  if (process.env["OFFLINE_SYNC_TO_LIVE"]?.trim() !== "1") return null;
  const liveUrl = process.env["LIVE_SUPABASE_URL"]?.trim().replace(/\/+$/, "");
  const liveKey =
process.env["LIVE_SUPABASE_SERVICE_ROLE_KEY"]?.trim();
  const copiedAt = new Date(process.env["OFFLINE_COPY_TAKEN_AT"]?.trim()
?? "");
  const localUrl = process.env["SUPABASE_URL"]?.trim().replace(/\/+$/, "");
  // Uploading to the offline copy's own database would duplicate every booking.
  if (!liveUrl || !liveKey || Number.isNaN(copiedAt.getTime()) || liveUrl === localUrl)
{
    return null;
  }
  const site = process.env["LIVE_SITE_URL"]?.trim().replace(/\/+$/, "");
  return {
    liveUrl,
```

```
    liveKey,
    copiedAt,
    liveSiteUrl: site && /^https?:\/\/.test(site) ? site : null,
  };
}

/** The offline copy's own tables for this, which the live site does not have. */
function localSyncTables() {
  return createClient(
    process.env["SUPABASE_URL"] ?? "",
    process.env["SUPABASE_SERVICE_ROLE_KEY"] ?? "",
    { auth: { persistSession: false, autoRefreshToken: false } },
  );
}

function liveClient(config: SyncConfig) {
  return createClient<Database>(config.liveUrl, config.liveKey, {
    auth: { persistSession: false, autoRefreshToken: false },
  });
}

async function liveIsReachable(config: SyncConfig) {
  try {
    const response = await fetch(`${config.liveUrl}/auth/v1/health`, {
      headers: { apikey: config.liveKey },
      signal: AbortSignal.timeout(5_000),
    });
    return response.ok;
  } catch {
    return false;
  }
}

const OFFLINE_BOOKING_FIELDS =
  "id,reference_code,booking_request_id,customer_name,first_name,middle_na
me,last_name,phone,email,moto_brand,moto_model,moto_variant,moto_year,pl
ate_number,appointment_date,start_time,notes,wants_products,total_estimate,b
ooking_duration_minutes,assigned_crew_id,status,rescheduled_from_appointme
nt_id,appointment_services(service_id,service_name,price,duration_minutes),ap
pointment_continuations(segment_number,appointment_date,start_time,booking
_duration_minutes,assigned_crew_id)";

async function loadOfflineBookings(config: SyncConfig) {
  const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
```

```
  const [bookings, handled] = await Promise.all([
    supabaseAdmin
      .from("appointments")
      .select(OFFLINE_BOOKING_FIELDS)
      .gte("created_at", config.copiedAt.toISOString())
      .in("status", ["pending", "confirmed"])
      .eq("is_archived", false)
      .order("created_at"),
    localSyncTables().from("offline_booking_uploads").select("appointment_id"),
  ]);
  if (bookings.error)
    throw new Error(`Offline bookings could not be read:
${bookings.error.message}`);
  if (handled.error) throw new Error(`Upload records could not be read:
${handled.error.message}`);
  const done = new Set(
    ((handled.data ?? []) as { appointment_id: string }[]).map((row) =>
row.appointment_id),
  );
  // Bookings whose time has passed are left alone: there is nothing to keep for
them.
  return (bookings.data ?? []).filter(
    (booking) =>
      !done.has(booking.id) &&
      Date.parse(`${booking.appointment_date}T${booking.start_time.slice(0,
5)}:00+08:00`) >
        Date.now(),
  );
}

type OfflineBooking = Awaited<ReturnType<typeof
loadOfflineBookings>>[number];

/** Why the live site refused a booking, in words for the admin. */
function refusalReason(error: { code?: string; message: string }) {
  if (error.code === "23P01") return "That time is no longer free on the live
schedule.";
  if (error.code === "23503") return "Its later days no longer fit on the live
schedule.";
  if (error.code === "23514" && /upcoming/i.test(error.message)) {
    return "The customer already has the most upcoming bookings allowed on the
live site.";
  }
  if (error.code === "23505" && /already (?:have|has) an
appointment/i.test(error.message)) {
```

```
    return "The customer already has an active booking for the same service on
the live site.";
  }
  if (error.code === "23505") return "Its reference code is already used on the live
site.";
  return `The live site refused it: ${error.message.slice(0, 200)}`;
}

type UploadResult = { ok: true; liveAppointmentId: string } | { ok: false; reason:
string };

async function uploadBooking(
  live: ReturnType<typeof liveClient>,
  booking: OfflineBooking,
  crewIds: string[],
): Promise<UploadResult> {
  if (booking.rescheduled_from_appointment_id) {
    return {
      ok: false,
      reason:
        "It replaced another booking through a reschedule made offline, so it has to
be updated on the live site by hand.",
    };
  }
  const services = booking.appointment_services ?? [];
  const requestId = booking.booking_request_id ?? booking.id;
  const input = {
    p_reference_code: booking.reference_code,
    p_booking_request_id: requestId,
    p_customer_name: booking.customer_name,
    p_phone: booking.phone,
    p_email: booking.email ?? "",
    p_moto_brand: booking.moto_brand,
    p_moto_model: booking.moto_model,
    p_moto_variant: booking.moto_variant,
    p_moto_year: booking.moto_year ?? new Date().getFullYear(),
    p_plate_number: booking.plate_number,
    p_appointment_date: booking.appointment_date,
    p_start_time: booking.start_time,
    p_notes: booking.notes,
    p_wants_products: booking.wants_products,
    p_total_estimate: Number(booking.total_estimate),
    p_booking_duration_minutes: booking.booking_duration_minutes,
    p_services: services.map((service) => ({
      service_id: service.service_id,
```

```
      service_name: service.service_name,
      price: service.price,
      duration_minutes: service.duration_minutes,
    })),
    p_continuation_segments: [...(booking.appointment_continuations ?? [])]
      .sort((a, b) => a.segment_number - b.segment_number)
      .map((segment) => ({
        segment_number: segment.segment_number,
        appointment_date: segment.appointment_date,
        start_time: segment.start_time,
        booking_duration_minutes: segment.booking_duration_minutes,
        assigned_crew_id: segment.assigned_crew_id,
      })),
    p_notification_title: `New booking ${booking.reference_code} (made offline)`,
    p_notification_message: `${booking.customer_name} booked
${services.map((service) => service.service_name).join(", ")} on
${booking.appointment_date} while the shop was offline. It was added when the
internet came back.`,
    p_first_name: booking.first_name ?? "",
    p_middle_name: booking.middle_name ?? "",
    p_last_name: booking.last_name ?? "",
  };

  // The mechanic chosen offline may be busy on the live schedule; another one
  // who is free at that time can take the booking instead.
  const crewOrder: (string | null)[] = [
    ...new Set([booking.assigned_crew_id, ...crewIds].filter((id): id is string =>
Boolean(id))),
  ];
  if (!crewOrder.length) crewOrder.push(null);
  let refusal: { code?: string; message: string } | null = null;
  for (const crewId of crewOrder) {
    const { error } = await live.rpc("create_booking_atomic", {
      ...input,
      p_assigned_crew_id: crewId,
    });
    if (!error) {
      refusal = null;
      break;
    }
    refusal = error;
    if (error.code !== "23P01") break;
  }
  if (refusal) return { ok: false, reason: refusalReason(refusal) };
```

```
  const saved = await live
    .from("appointments")
    .select("id")
    .eq("booking_request_id", requestId)
    .maybeSingle();
  if (saved.error || !saved.data) {
    throw new Error("The booking was sent, but the live site did not return it.");
  }
  return { ok: true, liveAppointmentId: saved.data.id };
}

/**
 * Keeps a booking confirmed offline confirmed on the live site. The live site
 * then emails the confirmation (offline emails only reached the test inbox),
 * but does not repeat a confirmation text the phone already sent.
 */
async function carryOverConfirmation(
  live: ReturnType<typeof liveClient>,
  booking: OfflineBooking,
  liveAppointmentId: string,
) {
  if (booking.status !== "confirmed") return;
  const { error } = await live
    .from("appointments")
    .update({ status: "confirmed" })
    .eq("id", liveAppointmentId)
    .eq("status", "pending");
  if (error)
    throw new Error(`The booking could not be confirmed on the live site:
${error.message}`);

  const { supabaseAdmin } = await
import("@/integrations/supabase/client.server");
  const text = await supabaseAdmin
    .from("appointment_confirmation_sms")
    .select("status")
    .eq("appointment_id", booking.id)
    .maybeSingle();
  if (text.data?.status === "sent") {
    await live
      .from("appointment_confirmation_sms")
      .update({
        status: "skipped",
        last_error: "Not sent: the customer was texted when it was confirmed
offline.",
```

```
        updated_at: new Date().toISOString(),
      })
      .eq("appointment_id", liveAppointmentId)
      .eq("status", "pending");
  }
}

async function reportConflict(
  live: ReturnType<typeof liveClient>,
  booking: OfflineBooking,
  reason: string,
) {
  const services = (booking.appointment_services ?? []).map((service) =>
service.service_name);
  const alert = {
    type: "offline_booking_conflict",
    title: `Call ${booking.customer_name}: offline booking
${booking.reference_code} was not added`,
    message: `${booking.customer_name} booked ${services.join(", ") || "a
service"} for ${formatSmsDate(booking.appointment_date)} at
${formatAppointmentTime(booking.start_time)} while the shop was offline
(reference ${booking.reference_code}). ${reason} Please call them at
${booking.phone} to arrange another time.`,
  };
  // The live admin sees it once people move there; the offline admin sees it too.
  const { error } = await live.from("notifications").insert(alert);
  if (error) throw new Error(`The live admin could not be told: ${error.message}`);
  const { alertAdmin } = await import("./admin-alerts.server");
  await alertAdmin({ ...alert, appointmentId: booking.id });
}

/** Checks the connection and, when the live site can be reached, uploads offline
bookings. */
export async function runOfflineSync() {
  const config = offlineSyncConfig();
  if (!config) return { enabled: false as const };
  const tables = localSyncTables();
  const previous = await tables
    .from("offline_sync_status")
    .select("online,online_since")
    .eq("id", true)
    .maybeSingle();

  const online = await liveIsReachable(config);
  let uploaded = 0;
```

```
  let conflicts = 0;
  let lastError: string | null = null;
  let waiting: OfflineBooking[] = [];
  try {
    waiting = await loadOfflineBookings(config);
    if (online && waiting.length) {
      const live = liveClient(config);
      const crew = await live
        .from("crew_members")
        .select("id")
        .eq("is_active", true)
        .eq("is_archived", false);
      if (crew.error) throw new Error(`The live crew could not be read:
${crew.error.message}`);
      const crewIds = crew.data.map((member) => member.id);

      for (const booking of waiting.slice(0, 20)) {
        const result = await uploadBooking(live, booking, crewIds);
        if (result.ok) {
          await carryOverConfirmation(live, booking, result.liveAppointmentId);
          uploaded += 1;
        } else {
          await reportConflict(live, booking, result.reason);
          conflicts += 1;
        }
        const { error } = await tables.from("offline_booking_uploads").insert({
          appointment_id: booking.id,
          status: result.ok ? "uploaded" : "conflict",
          live_appointment_id: result.ok ? result.liveAppointmentId : null,
          detail: result.ok ? null : result.reason,
        });
        if (error) throw new Error(`The upload could not be recorded:
${error.message}`);
      }
      waiting = await loadOfflineBookings(config);
    }
  } catch (error) {
    lastError = error instanceof Error ? error.message : "The offline sync failed.";
    console.error("[Offline sync] failed", error);
  }

  const now = new Date().toISOString();
  const wasOnline = previous.data?.online === true;
  await tables.from("offline_sync_status").upsert({
    id: true,
```

```
    online,
    checked_at: now,
    online_since: online ? (wasOnline ? (previous.data?.online_since ?? now) :
now) : null,
    waiting: waiting.length,
    last_error: lastError?.slice(0, 500) ?? null,
  });
  return {
    enabled: true as const,
    online,
    waiting: waiting.length,
    uploaded,
    conflicts,
    lastError,
  };
}

/** What browsers need to know, without any booking details. */
export async function offlineSyncStatus() {
  const config = offlineSyncConfig();
  if (!config) return { enabled: false as const };
  const { data } = await localSyncTables()
    .from("offline_sync_status")
    .select("online,waiting,checked_at")
    .eq("id", true)
    .maybeSingle();
  const status = data as { online: boolean; waiting: number; checked_at: string |
null } | null;
  // A check older than 3 minutes means the sync is not running; treat it as offline.
  const fresh =
    Boolean(status?.checked_at) && Date.parse(status!.checked_at!) >
Date.now() - 3 * 60_000;
  return {
    enabled: true as const,
    online: Boolean(fresh && status?.online),
    waiting: status?.waiting ?? 0,
    liveSiteUrl: config.liveSiteUrl,
  };
}
```

src/lib/print-document.ts

```
/**
 * Printing through the browser's print dialog ("Save as PDF" included): A4
 * landscape pages without the browser's own header and footer, laid out here
 * so tables break cleanly and the sign-off sits at the bottom of the last page.
 * The printable markup is PrintDocument in
components/admin/print-document.tsx.
 */
import { useEffect, useRef } from "react";

/** A4 landscape, and the margins on every printed page (millimetres). */
const PRINT_PAGE_MM = { width: 297, height: 210, top: 12, bottom: 15, side: 11
};
/** Sheets are a little shorter than the paper so rounding never spills onto an
extra page. */
const PRINT_SHEET_HEIGHT_MM = PRINT_PAGE_MM.height - 1;
/** Content is never filled to the very edge of a sheet. */
const PRINT_SHEET_SAFETY_MM = 2;
const MM_TO_PX = 96 / 25.4;
/** Minimum space between the last table and the signatures. */
const SIGN_OFF_GAP_MM = 10;

/** The part of a table that is on one sheet. */
type PrintTablePart = {
  section: HTMLElement;
  table: HTMLElement;
  body: HTMLTableSectionElement;
  footer: HTMLElement | null;
  rows: number;
};

/**
 * Lays a printable document out on fixed A4 sheets, one per printed page, so the
page
 * breaks are decided here rather than guessed. Rows and blocks are never split,
 * a section heading stays with its table, and a table's header row repeats on
 * each sheet it continues on. The signatures go at the bottom of the last sheet.
 * When there is not enough room below the content, they go on a new sheet,
 * together with the last row and total of the final table.
 *
 * Fills .report-print-sheets; if anything fails, the sheets stay empty and the
 * document prints as one flowing page instead.
 */
export function paginatePrintDocument(root: HTMLElement) {
```

```
  const sheets = root.querySelector<HTMLElement>(".report-print-sheets");
  const content = root.querySelector<HTMLElement>(".report-print-flow
.report-print-content");
  const signOff = root.querySelector<HTMLElement>(".report-print-flow
.report-print-signoff");
  if (!sheets || !content || !signOff) return;
  sheets.replaceChildren();

  const pageWidth = (PRINT_PAGE_MM.width - 2 * PRINT_PAGE_MM.side) *
MM_TO_PX;
  const pageHeight =
    (PRINT_SHEET_HEIGHT_MM - PRINT_PAGE_MM.top -
PRINT_PAGE_MM.bottom - PRINT_SHEET_SAFETY_MM) *
    MM_TO_PX;

  // Measure everything once at the printed width, off screen.
  const probe = document.createElement("article");
  probe.className = "report-print-document";
  Object.assign(probe.style, {
    position: "absolute",
    top: "0",
    left: "-100000px",
    width: `${pageWidth}px`,
    visibility: "hidden",
  });
  const flow = content.cloneNode(true) as HTMLElement;
  const signatures = signOff.cloneNode(true) as HTMLElement;
  probe.append(flow, signatures);
  document.body.append(probe);

  try {
    const height = (element: Element) => element.getBoundingClientRect().height;
    const margin = (element: Element, side: "marginTop" | "marginBottom") =>
      parseFloat(getComputedStyle(element)[side]) || 0;

    let sheet = document.createElement("section");
    let used = 0;
    const newSheet = () => {
      sheet = document.createElement("section");
      sheet.className = "report-print-document report-print-sheet";
      sheets.append(sheet);
      used = 0;
    };
    const fits = (extra: number) => used + extra <= pageHeight;
    newSheet();
```

```
    // The part of the final table on the current sheet, if the document ends in one.
    let lastTable: { part: PrintTablePart; continueOnNewSheet: () => PrintTablePart
} | null = null;

    for (const block of Array.from(flow.children)) {
      const table = block.querySelector(".report-print-table");
      const rows = table
        ? [
            ...Array.from(table.querySelectorAll(":scope > tbody > tr"), (row) => ({
              row,
              inFooter: false,
            })),
            ...Array.from(table.querySelectorAll(":scope > tfoot > tr"), (row) => ({
              row,
              inFooter: true,
            })),
          ]
        : [];
      const firstRow = rows[0];
      const marginTop = margin(block, "marginTop");

      if (!table || !firstRow) {
        const blockHeight = height(block) + margin(block, "marginBottom");
        if (used > 0 && !fits(marginTop + blockHeight)) newSheet();
        const clone = block.cloneNode(true) as HTMLElement;
        // Space above a block is dropped at the top of a sheet.
        if (used === 0) clone.style.marginTop = "0";
        sheet.append(clone);
        used += (used === 0 ? 0 : marginTop) + blockHeight;
        lastTable = null;
        continue;
      }

      const heading = block.querySelector(":scope > h2");
      const colgroup = table.querySelector(":scope > colgroup");
      const thead = table.querySelector(":scope > thead");
      const headingHeight = heading
        ? height(heading) + margin(heading, "marginTop") + margin(heading,
"marginBottom")
        : 0;
      const headerHeight = thead ? height(thead) : 0;

      // The part of this section on one sheet: the heading on the first sheet
      // only, and the table's column widths and header row on every sheet.
      const startPart = (withHeading: boolean) => {
```

```
      const section = block.cloneNode(false) as HTMLElement;
      if (used === 0) section.style.marginTop = "0";
      if (withHeading && heading) section.append(heading.cloneNode(true));
      const shell = table.cloneNode(false) as HTMLElement;
      if (colgroup) shell.append(colgroup.cloneNode(true));
      if (thead) shell.append(thead.cloneNode(true));
      const body = shell.appendChild(document.createElement("tbody"));
      section.append(shell);
      sheet.append(section);
      return { section, table: shell, body, footer: null as HTMLElement | null, rows:
0 };
    };

    // The heading, header row, and first row stay together.
    if (used > 0 && !fits(marginTop + headingHeight + headerHeight +
height(firstRow.row))) {
      newSheet();
    }
    const sectionStart = (used === 0 ? 0 : marginTop) + headingHeight +
headerHeight;
    let part = startPart(true);
    used += sectionStart;

    for (const { row, inFooter } of rows) {
      const rowHeight = height(row);
      if (part.rows > 0 && !fits(rowHeight)) {
        newSheet();
        part = startPart(false);
        used += headerHeight;
      }
      if (inFooter) {
        part.footer ??= part.table.appendChild(document.createElement("tfoot"));
        part.footer.append(row.cloneNode(true));
      } else {
        part.body.append(row.cloneNode(true));
      }
      part.rows += 1;
      used += rowHeight;
    }
    lastTable = { part, continueOnNewSheet: () => startPart(false) };
  }

  if (!fits(SIGN_OFF_GAP_MM * MM_TO_PX + height(signatures))) {
    newSheet();
    // The signatures never sit alone on a page: the final table's last row
```

```
      // and total move down with them.
      if (lastTable) {
        const { part } = lastTable;
        const bodyRows = Array.from(part.body.rows);
        const lastRow = bodyRows.at(-1);
        if (bodyRows.length > 1 && lastRow) {
          const moved = lastTable.continueOnNewSheet();
          moved.body.append(lastRow);
          if (part.footer) moved.table.append(part.footer);
        } else {
          part.section.style.marginTop = "0";
          sheet.append(part.section);
        }
      }
    }
    sheet.append(signOff.cloneNode(true));
  } catch (error) {
    console.error("Could not lay out the printed pages:", error);
    sheets.replaceChildren();
  } finally {
    probe.remove();
  }
}

export const PRINT_DOCUMENT_STYLES = `
  .report-print-root { display: none; }

  /*
   * A zero page margin stops the browser from printing its own header (page
   * title) and footer (URL, page numbers). The .report-print-page table puts
   * the margins back: its header and footer rows repeat on every page.
   */
  @page { size: A4 landscape; margin: 0; }

  @media print {
    html,
    body {
      width: auto !important;
      min-width: 0 !important;
      height: auto !important;
      margin: 0 !important;
      padding: 0 !important;
      background: #fff !important;
    }
    body > :not(.report-print-root) { display: none !important; }
```

```
    .report-print-root {
      position: static !important;
      display: block !important;
      width: auto !important;
      min-width: 0 !important;
      min-height: 0 !important;
      margin: 0 !important;
      padding: 0 !important;
      overflow: visible !important;
      background: #fff;
    }
  }

  /* Layout and typography. These apply outside print too, so the document can
     be measured before printing (see paginatePrintDocument). */
  .report-print-page { width: 100%; border-collapse: collapse; border: 0; }
  .report-print-page > thead > tr > td { height: ${PRINT_PAGE_MM.top}mm;
padding: 0; border: 0; }
  .report-print-page > tfoot > tr > td { height: ${PRINT_PAGE_MM.bottom}mm;
padding: 0; border: 0; }
  .report-print-page > tbody > tr > td { padding: 0 ${PRINT_PAGE_MM.side}mm;
border: 0; vertical-align: top; }
  .report-print-document {
    width: 100%;
    min-width: 0;
    color: #1f2937;
    font-family: Arial, Helvetica, sans-serif;
  }

  /* One sheet per printed page, laid out by paginatePrintDocument. Once the
     sheets are there, the flowing copy of the document is not printed. */
  .report-print-sheets:not(:empty) + .report-print-flow { display: none !important; }
  .report-print-sheet {
    box-sizing: border-box;
    display: flex;
    flex-direction: column;
    width: ${PRINT_PAGE_MM.width}mm;
    height: ${PRINT_SHEET_HEIGHT_MM}mm;
    padding: ${PRINT_PAGE_MM.top}mm ${PRINT_PAGE_MM.side}mm
${PRINT_PAGE_MM.bottom}mm;
    break-after: page;
    page-break-after: always;
  }
  .report-print-sheet:last-child { break-after: auto; page-break-after: auto; }
  .report-print-sheet > * { flex: none; }
```

```
  .report-print-sheet > .report-print-signoff { margin-top: auto; padding-top: ${SIGN_OFF_GAP_MM}mm; }
  .report-print-flow .report-print-signoff { margin-top: ${SIGN_OFF_GAP_MM}mm; }
  .report-print-header,
  .report-print-meta,
  .report-print-summary,
  .report-print-signoff {
    break-inside: avoid-page;
    page-break-inside: avoid;
  }
  .report-print-header {
    display: grid;
    grid-template-columns: 26mm minmax(0, 1fr) 36mm;
    align-items: center;
    gap: 6mm;
  }
  .report-print-logo { display: block; width: 24mm; max-width: 100%; height: auto; aspect-ratio: 360 / 202; object-fit: contain; }
  .report-print-heading { min-width: 0; text-align: center; }
  .report-print-heading p { margin: 0; font-size: 8pt; letter-spacing: .08em; text-transform: uppercase; }
  .report-print-heading p:nth-child(2) { margin-top: 1mm; font-size: 7pt; letter-spacing: .03em; text-transform: none; }
  .report-print-heading h1 { margin: 3mm 0 2mm; font-size: 18pt; font-weight: 700; letter-spacing: .04em; overflow-wrap: anywhere; text-transform: uppercase; }
  .report-print-heading p:last-child { font-size: 7pt; font-weight: 700; }
  .report-print-generated { min-width: 0; margin: 0; overflow-wrap: anywhere; text-align: right; font-size: 7pt; line-height: 1.45; color: #4b5563; }
  .report-print-rule { height: 1.25pt; margin: 5mm 0 4mm; background: #0f5b49; }
  .report-print-meta { display: grid; grid-template-columns: minmax(0, 1fr) minmax(0, 1fr); gap: 8mm; font-size: 8pt; }
  .report-print-meta div:last-child { text-align: right; }
  .report-print-meta span,
  .report-print-summary span { display: block; color: #4b5563; font-size: 7pt; text-transform: uppercase; letter-spacing: .04em; }
  .report-print-meta strong { display: block; margin-top: 1mm; overflow-wrap: anywhere; font-size: 9pt; font-weight: 600; }
  .report-print-section { min-width: 0; margin-top: 6mm; break-inside: auto; page-break-inside: auto; }
  .report-print-section h2 { margin: 0 0 3mm; break-after: avoid-page; page-break-after: avoid; font-size: 11pt; font-weight: 700; }
  .report-print-summary-table { width: 100%; border-collapse: collapse; font-size: 8pt; }
  .report-print-summary-table th,
```

```
  .report-print-summary-table td { border: .75pt solid #6b7280; padding: 2.25mm 2mm; }
  .report-print-summary-table th { background: #e5e7eb; font-size: 7pt; text-align: left; text-transform: uppercase; }
  .report-print-summary-table th:last-child,
  .report-print-summary-table td:last-child { text-align: right; }
  .report-print-summary-table tbody tr:last-child td { font-weight: 700; background: #f3f4f6; }
  .report-print-table { width: 100%; max-width: 100%; border-collapse: collapse; table-layout: fixed; font-size: 7.5pt; }
  .report-print-table thead { display: table-header-group; break-inside: avoid-page; background: #e5e7eb; }
  .report-print-table th,
  .report-print-table td { box-sizing: border-box; border: .75pt solid #6b7280; padding: 2.25mm 2mm; vertical-align: top; overflow-wrap: anywhere; word-break: break-word; }
  .report-print-table th { font-size: 7pt; text-align: left; font-weight: 700; }
  .report-print-table th:last-child,
  .report-print-table td:last-child { text-align: right; }
  .report-print-table.report-print-text-end th:last-child,
  .report-print-table.report-print-text-end td:last-child { text-align: left; }
  .report-print-table tr { break-inside: avoid-page; page-break-inside: avoid; }
  .report-print-table tbody tr:nth-child(even) { background: #f9fafb; }
  .report-print-table tfoot { display: table-row-group; break-inside: avoid-page; font-weight: 700; background: #f3f4f6; }
  .report-print-table tfoot td:first-child { text-transform: uppercase; }
  .report-print-footer { display: grid; grid-template-columns: 1fr 1fr; gap: 30mm; font-size: 8pt; }
  .report-print-footer > div:last-child { text-align: right; }
  .report-print-footer span,
  .report-print-footer small { display: block; color: #4b5563; }
  .report-print-footer strong { display: block; margin-top: 6mm; }
  .report-print-footer div > div { margin-top: 2mm; border-bottom: .75pt solid #374151; }
  .report-print-timestamp { margin: 6mm 0 0; text-align: center; color: #6b7280; font-size: 7pt; }
`;

/**
 * Prints the rendered PrintDocument: waits for the logo, lays out the pages,
 * names the print (the browser uses it as the "Save as PDF" file name), opens
 * the print dialog, then calls onDone. A null title means nothing to print.
 */
export function usePrintWhenReady(documentTitle: string | null, onDone: () => void) {
```

```
  const onDoneRef = useRef(onDone);
  useEffect(() => {
    onDoneRef.current = onDone;
  });

  useEffect(() => {
    if (!documentTitle) return;
    let cancelled = false;
    const timer = window.setTimeout(async () => {
      const root = document.querySelector<HTMLElement>(".report-print-root");
      if (root) {
        // The logo must be decoded before the pages are measured.
        await Promise.all(
          Array.from(root.querySelectorAll("img"), (image) =>
            image.decode().catch(() => undefined),
          ),
        );
        if (cancelled) return;
        paginatePrintDocument(root);
      }
      const previousTitle = document.title;
      document.title = documentTitle;
      window.addEventListener("afterprint", () => (document.title = previousTitle), {
        once: true,
      });
      window.print();
      onDoneRef.current();
    }, 0);
    return () => {
      cancelled = true;
      window.clearTimeout(timer);
    };
  }, [documentTitle]);
}
```

src/lib/scheduled-request.server.ts

```
/**
 * Requests the database sends on a schedule (pg_cron) carry
 * SCHEDULED_NOTIFICATIONS_SECRET as a bearer token. Returns the
response to
 * send when the request is not allowed, or null when it is.
 */
export function refuseUnscheduledRequest(request: Request): Response | null {
  const secret =
process.env["SCHEDULED_NOTIFICATIONS_SECRET"]?.trim();
  if (!secret) {
    console.error("[Scheduled request] SCHEDULED_NOTIFICATIONS_SECRET
is not set");
    return Response.json({ ok: false, error: "Not configured." }, { status: 503 });
  }
  if (!sameSecret(request.headers.get("authorization") ?? "", `Bearer ${secret}`)) {
    return Response.json({ ok: false, error: "Unauthorized." }, { status: 401 });
  }
  return null;
}

/** Compares in constant time, so response timing does not reveal the secret. */
function sameSecret(received: string, expected: string) {
  const a = new TextEncoder().encode(received);
  const b = new TextEncoder().encode(expected);
  let difference = a.length ^ b.length;
  for (let i = 0; i < b.length; i += 1) difference |= (a[i] ?? 0) ^ (b[i] ?? 0);
  return difference === 0;
}
```

src/lib/shop.ts

```
import { z } from "zod";

const OPEN_DAYS = "Monday to Saturday";
const OPEN_TIMES = "8:00 AM – 5:00 PM";
// The shop's Google Maps listing, "Fake Rider Motoparts and Accessories", and
its pin.
const MAPS_LISTING_URL =
"https://maps.google.com/?cid=7858024619263222524";
const COORDINATES = "9.6281673,123.8865123";

export const SHOP = {
  name: "Fake Rider Motorparts",
  tagline: "Motorparts, Accessories & Race-Grade Service",
  address: "Purok Bangkal Sta. Cruz, Baclayon, Bohol, Philippines",
  hours: `${OPEN_DAYS}, ${OPEN_TIMES}`,
  openDays: OPEN_DAYS,
  openTimes: OPEN_TIMES,
  // Every map and directions link points at the same spot.
  mapsUrl: MAPS_LISTING_URL,
  mapEmbedUrl:
`https://maps.google.com/maps?q=${COORDINATES}&z=16&output=embed`,
  phone: "0916 126 3317",
  email: "joemartato4@gmail.com",
  facebook: "https://www.facebook.com/profile.php?id=100082988659961",
  // m.me opens the Messenger app on phones and messenger.com on
computers.
  messenger: "https://m.me/101547759281909",
  noticeHours: 48,
  timeZone: "Asia/Manila",
};

/** Default copy shown on the customer booking form; administrators can replace
it in Settings. */
export const DEFAULT_BOOKING_TERMS = `Bookings are subject to shop
confirmation. Please arrive 15 minutes before your slot. Late arrivals beyond 30
minutes may be rescheduled. Quoted prices are starting rates; parts and
additional labor are billed separately. The shop is not liable for personal items left
on the unit.

Cancellations must be made at least ${SHOP.noticeHours} hours before the
schedule.

We use your name, contact details, motorcycle details, selected services, and
```

```
notes only to manage this booking, contact you about it, and provide shop
services. We do not sell your information.`;

export const APPOINTMENT_STATUSES = [
  "pending",
  "confirmed",
  "in_progress",
  "completed",
  "rescheduled",
  "cancelled",
  "rejected",
  "no_show",
] as const;

/** Shown where archiving or deleting a completed appointment is refused (the
database enforces it). */
export const COMPLETED_APPOINTMENT_KEPT = "Completed appointments
are kept for financial records.";

export function statusLabel(status: string) {
  return status.replace(/_/g, " ").replace(/\b\w/g, (c) => c.toUpperCase());
}

export function statusTone(status: string) {
  switch (status) {
    case "confirmed":
      return "border-emerald-500/30 bg-emerald-500/15 text-emerald-400";
    case "in_progress":
      return "bg-accent/15 text-accent border-accent/30";
    case "completed":
      return "border-sky-500/30 bg-sky-500/15 text-sky-400";
    case "rescheduled":
      return "bg-accent/15 text-accent border-accent/30";
    case "cancelled":
    case "rejected":
    case "no_show":
      return "bg-destructive/15 text-destructive border-destructive/30";
    default:
      return "bg-muted text-muted-foreground border-border";
  }
}

/** Shared display treatment for records that can be enabled or deactivated. */
export function activeStatusTone(isActive: boolean) {
  return isActive
```

```
    ? "border-emerald-500/45 bg-emerald-500/20 text-emerald-300"
    : "border-border bg-muted text-muted-foreground";
}

export function formatPHP(value: number | string | null | undefined) {
  const n = Number(value ?? 0);
  return new Intl.NumberFormat("en-PH", {
    style: "currency",
    currency: "PHP",
    maximumFractionDigits: 2,
  }).format(n);
}

export function formatTime(value: string) {
  const [h, m] = value.split(":");
  const hour = Number(h);
  const suffix = hour >= 12 ? "PM" : "AM";
  const display = hour % 12 === 0 ? 12 : hour % 12;
  return `${display}:${m} ${suffix}`;
}

export const DEFAULT_BOOKING_HOURS = {
  openingTime: "08:00",
  closingTime: "17:00",
} as const;

export type BookingHours = {
  openingTime: string;
  closingTime: string;
};

const BOOKING_INTERVAL_MINUTES = 30;

// Keep the public booking request bounded while allowing a customer to select
// the full current service catalog in one appointment.
export const MAX_BOOKING_SERVICE_SELECTIONS = 20;

export type BookingCapacityConfig = {
  startTime: string;
  capacity: number;
};

export type BookingTimeSlot = {
  id: string;
  startTime: string;
```

```
  endTime: string;
  capacity: number;
};

export function timeToMinutes(time: string) {
  const [hours = Number.NaN, minutes = Number.NaN] = time.slice(0,
5).split(":").map(Number);
  if (
    !Number.isInteger(hours) ||
    !Number.isInteger(minutes) ||
    hours < 0 ||
    hours > 23 ||
    minutes < 0 ||
    minutes > 59
  ) {
    return Number.NaN;
  }
  return hours * 60 + minutes;
}

export function minutesToTime(minutes: number) {
  return `${String(Math.floor(minutes / 60)).padStart(2, "0")}:${String(minutes %
60).padStart(2, "0")}`;
}

/** Operating hours must use the public booking grid's 30-minute increments. */
export function isValidBookingHours(openingTime: string, closingTime: string) {
  const openingMinutes = timeToMinutes(openingTime);
  const closingMinutes = timeToMinutes(closingTime);
  return (
    Number.isFinite(openingMinutes) &&
    Number.isFinite(closingMinutes) &&
    openingMinutes < closingMinutes &&
    openingMinutes % BOOKING_INTERVAL_MINUTES === 0 &&
    closingMinutes % BOOKING_INTERVAL_MINUTES === 0
  );
}

function resolveBookingHours(hours: BookingHours =
DEFAULT_BOOKING_HOURS): BookingHours {
  return isValidBookingHours(hours.openingTime, hours.closingTime)
    ? { openingTime: hours.openingTime.slice(0, 5), closingTime:
hours.closingTime.slice(0, 5) }
    : DEFAULT_BOOKING_HOURS;
}
```

```
export function shopTimeOptions(hours: BookingHours =
DEFAULT_BOOKING_HOURS) {
  const { openingTime, closingTime } = resolveBookingHours(hours);
  const openingMinutes = timeToMinutes(openingTime);
  const closingMinutes = timeToMinutes(closingTime);
  const options: { value: string; label: string }[] = [];
  for (
    let minutes = openingMinutes;
    minutes <= closingMinutes;
    minutes += BOOKING_INTERVAL_MINUTES
  ) {
    const time24 = minutesToTime(minutes);
    options.push({ value: time24, label: formatTime(time24) });
  }
  return options;
}

/** True for a 30-minute booking start within the 8 AM–5 PM operating window. */
export function isBookingStartTime(time: string, hours: BookingHours =
DEFAULT_BOOKING_HOURS) {
  const { openingTime, closingTime } = resolveBookingHours(hours);
  const minutes = timeToMinutes(time);
  const openingMinutes = timeToMinutes(openingTime);
  const closingMinutes = timeToMinutes(closingTime);
  return (
    Number.isFinite(minutes) &&
    minutes >= openingMinutes &&
    minutes < closingMinutes &&
    (minutes - openingMinutes) % BOOKING_INTERVAL_MINUTES === 0
  );
}

/** True when the entire appointment fits inside the shop's operating hours. */
export function isBookingTimeRangeWithinHours(
  time: string,
  durationMinutes: number,
  hours: BookingHours = DEFAULT_BOOKING_HOURS,
) {
  const { closingTime } = resolveBookingHours(hours);
  const startMinutes = timeToMinutes(time);
  return (
    isBookingStartTime(time, hours) &&
    Number.isFinite(durationMinutes) &&
    durationMinutes > 0 &&
```

```
    startMinutes + durationMinutes <= timeToMinutes(closingTime)
  );
}

/** Minutes of a booking that would fall after the configured closing time. */
export function bookingDurationOverflowMinutes(
  time: string,
  durationMinutes: number,
  hours: BookingHours = DEFAULT_BOOKING_HOURS,
) {
  const { closingTime } = resolveBookingHours(hours);
  const startMinutes = timeToMinutes(time);
  const closingMinutes = timeToMinutes(closingTime);
  if (!Number.isFinite(startMinutes) || !Number.isFinite(durationMinutes)) return 0;
  return Math.max(0, startMinutes + durationMinutes - closingMinutes);
}

/** The portion of a booking that can be completed on its selected first day. */
export function bookingDurationForFirstDay(
  time: string,
  durationMinutes: number,
  hours: BookingHours = DEFAULT_BOOKING_HOURS,
) {
  const { closingTime } = resolveBookingHours(hours);
  const startMinutes = timeToMinutes(time);
  const closingMinutes = timeToMinutes(closingTime);
  if (!Number.isFinite(startMinutes) || !Number.isFinite(durationMinutes)) return 0;
  return Math.max(0, Math.min(durationMinutes, closingMinutes - startMinutes));
}

/** True for a valid Monday-Saturday Manila calendar date. */
export function isShopOpenDate(dateIso: string) {
  if (!isValidCalendarDate(dateIso)) return false;
  const [year = Number.NaN, month = Number.NaN, day = Number.NaN] =
dateIso.split("-").map(Number);
  const date = new Date(Date.UTC(year, month - 1, day));
  return date.getUTCDay() !== 0;
}

/** True only for a real Gregorian calendar date in the YYYY-MM-DD form. */
export function isValidCalendarDate(dateIso: string) {
  if (!/^\d{4}-\d{2}-\d{2}$/.test(dateIso)) return false;
  const [year = Number.NaN, month = Number.NaN, day = Number.NaN] =
dateIso.split("-").map(Number);
  const date = new Date(Date.UTC(year, month - 1, day));
```

```
  return (
    date.getUTCFullYear() === year && date.getUTCMonth() === month - 1 &&
date.getUTCDate() === day
  );
}

/**
 * Build the public booking grid. The configured hourly slot capacity applies
 * to both half-hour starts within that hour.
 */
export function buildBookingTimeSlots(
  config: BookingCapacityConfig[],
  hours: BookingHours = DEFAULT_BOOKING_HOURS,
): BookingTimeSlot[] {
  const { openingTime, closingTime } = resolveBookingHours(hours);
  const openingMinutes = timeToMinutes(openingTime);
  const closingMinutes = timeToMinutes(closingTime);
  return Array.from(
    { length: (closingMinutes - openingMinutes) /
BOOKING_INTERVAL_MINUTES },
    (_, index) => {
      const startMinutes = openingMinutes + index *
BOOKING_INTERVAL_MINUTES;
      const startTime = minutesToTime(startMinutes);
      const hourStart = minutesToTime(Math.floor(startMinutes / 60) * 60);
      const configured = config.find((slot) => slot.startTime.slice(0, 5) ===
startTime);
      const hourlyConfigured = config.find((slot) => slot.startTime.slice(0, 5) ===
hourStart);

      return {
        id: `booking-${startTime}`,
        startTime,
        endTime: minutesToTime(startMinutes +
BOOKING_INTERVAL_MINUTES),
        // A configured hour covers its two half-hour starts. Do not infer a
        // capacity for hours absent from the configuration: doing so can
        // inadvertently reopen an intentionally omitted time such as lunch.
        capacity: Math.max(0, Number(configured?.capacity ??
hourlyConfigured?.capacity ?? 0)),
      };
    },
  );
}
```

```
export function intervalsOverlap(startA: number, endA: number, startB: number,
endB: number) {
  return startA < endB && endA > startB;
}

export function formatDateLong(iso: string) {
  const [y, m, d] = iso.split("-").map(Number);
  return new Date(Date.UTC(y ?? 1970, (m ?? 1) - 1, d ??
1)).toLocaleDateString("en-PH", {
    weekday: "long",
    month: "long",
    day: "numeric",
    year: "numeric",
    timeZone: "UTC",
  });
}

/** Current date/time parts in the shop's configured local timezone. */
export function manilaNow(now = new Date()) {
  const parts = new Intl.DateTimeFormat("en-CA", {
    timeZone: SHOP.timeZone,
    year: "numeric",
    month: "2-digit",
    day: "2-digit",
    hour: "2-digit",
    minute: "2-digit",
    hour12: false,
  }).formatToParts(now);
  const get = (t: string) => parts.find((p) => p.type === t)?.value ?? "00";
  return {
    date: `${get("year")}-${get("month")}-${get("day")}`,
    time: `${get("hour")}:${get("minute")}`,
  };
}

/** Epoch ms for a Manila local date + time string. */
export function manilaTimestamp(dateIso: string, time: string) {
  return Date.parse(`${dateIso}T${time.slice(0, 5)}:00+08:00`);
}

export function addDays(dateIso: string, days: number) {
  const [y, m, d] = dateIso.split("-").map(Number);
  const dt = new Date(Date.UTC(y ?? 1970, (m ?? 1) - 1, d ?? 1));
  dt.setUTCDate(dt.getUTCDate() + days);
  return dt.toISOString().slice(0, 10);
```

```
}

const RANGE_PREFIX = "RANGE:";

export function encodeBlockReason(endTime: string | null, userReason: string):
string | null {
  if (!endTime) return userReason || null;
  const base = `${RANGE_PREFIX}${endTime.slice(0, 5)}`;
  return userReason ? `${base}|${userReason}` : base;
}

export function decodeBlockReason(reason: string | null): {
  endTime: string | null;
  userReason: string;
} {
  if (!reason || !reason.startsWith(RANGE_PREFIX))
    return { endTime: null, userReason: reason ?? "" };
  const rest = reason.slice(RANGE_PREFIX.length);
  const pipeIdx = rest.indexOf("|");
  if (pipeIdx === -1) return { endTime: rest, userReason: "" };
  return { endTime: rest.slice(0, 5), userReason: rest.slice(pipeIdx + 1) };
}

export const PHONE_VALIDATION_MESSAGE = "Enter a valid mobile
number.";

export const NAME_PART_VALIDATION_MESSAGE =
  "Names may contain letters, spaces, hyphens, and apostrophes only.";

/** Matches one name part without changing the customer's spelling or spacing.
*/
export const NAME_PART_PATTERN = /^(?:\p{L}+(?:(?: |[-'’])\p{L}+)*)?$/u;

/**
 * Capitalize only the first character of a name segment. The rest of each
 * segment remains exactly as entered, so intentional capitalization such as
 * McDonald and MacArthur is never guessed or rewritten.
 */
export function formatNamePartInput(value: string) {
  return value
    .replace(/[^\p{L}\s'’-]/gu, "")
    .replace(/(^|[\s'’-])(\p{Ll})/gu, (_, separator: string, initial: string) => {
      return `${separator}${initial.toLocaleUpperCase()}`;
    });
}
```

```
/** Keep the mobile-number field in its local 11-digit format while the user types.
*/
export function sanitizePhilippineMobileInput(value: string) {
  const normalized = normalizePhilippineMobile(value);
  return normalized ?? value.replace(/\D/g, "").slice(0, 11);
}

/** Convert a stored or pasted Philippine mobile number to its local
09XXXXXXXXX form. */
export function toLocalPhilippineMobile(value: string) {
  return normalizePhilippineMobile(value) ?? value.replace(/\D/g, "").slice(0, 11);
}

/**
 * Convert local or E.164 Philippine mobile input to the application's canonical
 * local 09XXXXXXXXX format. Supabase Auth phone APIs, if introduced, must
make
 * their E.164 conversion only at that authentication boundary.
 */
export function normalizePhilippineMobile(value: string): string | null {
  const digits = value.replace(/\D/g, "");

  if (/^639\d{9}$/.test(digits)) return `0${digits.slice(2)}`;

  if (/^09\d{9}$/.test(digits)) return digits;

  return null;
}

export const phoneSchema = z
  .string()
  .trim()
  .refine((value) => normalizePhilippineMobile(value) !== null,
PHONE_VALIDATION_MESSAGE)
  .transform((value) => normalizePhilippineMobile(value)!);

export const REFERENCE_CODE_PATTERN = /^FRM-[A-Z0-9]{6}$/i;

/** Reference codes are case-insensitive; store and query them in one canonical
form. */
export function normalizeReferenceCode(value: string) {
  return value.trim().toUpperCase();
}
```

```
export function isReferenceCode(value: string) {
  return REFERENCE_CODE_PATTERN.test(normalizeReferenceCode(value));
}

/** Booking rule: the slot must start after the configured lead time (Manila). */
export function isSlotBookable(dateIso: string, time: string, minimumLeadHours =
SHOP.noticeHours) {
  return manilaTimestamp(dateIso, time) - Date.now() >= minimumLeadHours *
3600 * 1000;
}

export function earliestBookableDate(minimumLeadHours = SHOP.noticeHours)
{
  const now = manilaNow();
  const ts = manilaTimestamp(now.date, `${now.time}:00`);
  const cutoff = ts + minimumLeadHours * 3600 * 1000;
  return new Date(cutoff).toLocaleDateString("en-CA", {
    timeZone: "Asia/Manila",
    year: "numeric",
    month: "2-digit",
    day: "2-digit",
  });
}
```

src/lib/utils.ts

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

/**
 * A random UUID. Browsers only provide crypto.randomUUID on https or
localhost,
 * so phones opening the offline copy over the laptop's hotspot address get one
 * built from crypto.getRandomValues instead.
 */
export function randomId(): string {
  if (typeof crypto.randomUUID === "function") return crypto.randomUUID();
  const bytes = crypto.getRandomValues(new Uint8Array(16));
  bytes[6] = (bytes[6]! & 0x0f) | 0x40;
  bytes[8] = (bytes[8]! & 0x3f) | 0x80;
  const hex = [...bytes].map((byte) => byte.toString(16).padStart(2, "0")).join("");
  return `${hex.slice(0, 8)}-${hex.slice(8, 12)}-${hex.slice(12, 16)}-${hex.slice(16,
20)}-${hex.slice(20)}`;
}
```

src/router.tsx

```
import { QueryClient } from "@tanstack/react-query";
import { createRouter } from "@tanstack/react-router";
import { routeTree } from "./routeTree.gen";

export const getRouter = () => {
  const queryClient = new QueryClient();

  const router = createRouter({
    routeTree,
    context: { queryClient },
    scrollRestoration: true,
    defaultPreloadStaleTime: 0,
  });

  return router;
};
```

src/routes/__root.tsx

```
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import {
  Outlet,
  Link,
  createRootRouteWithContext,
  useRouter,
  HeadContent,
  Scripts,
  type ErrorComponentProps,
} from "@tanstack/react-router";
import type { ReactNode } from "react";

import { OfflineConnection } from "@/components/site/offline-connection";

import appCss from "../styles.css?url";

/** Set to "1" only in the offline copy (.env.offline). */
const OFFLINE = import.meta.env["VITE_OFFLINE"] === "1";

const themeInitScript = `(() => {
  try {
    const saved = localStorage.getItem("fake-rider-theme");
    const theme = saved === "light" || saved === "dark"
      ? saved
      : matchMedia("(prefers-color-scheme: dark)").matches ? "dark" : "light";
    document.documentElement.classList.toggle("dark", theme === "dark");
    document.documentElement.style.colorScheme = theme;
  } catch {}
})();`;

function NotFoundComponent() {
  return (
    <div className="flex min-h-screen items-center justify-center bg-background
px-4">
      <div className="max-w-md text-center">
        <h1 className="text-7xl font-bold text-foreground">404</h1>
        <h2 className="mt-4 text-xl font-semibold text-foreground">Page not
found</h2>
        <p className="mt-2 text-sm text-muted-foreground">
          The page you're looking for doesn't exist or has been moved.
        </p>
        <div className="mt-6">
          <Link
```

```
          to="/"
          className="inline-flex items-center justify-center rounded-md bg-primary
px-4 py-2 text-sm font-medium text-primary-foreground transition-colors
hover:bg-primary/90"
        >
          Go home
        </Link>
      </div>
    </div>
  </div>
 );
}

function ErrorComponent({ error, reset }: ErrorComponentProps) {
 console.error(error);
 const router = useRouter();

 return (
  <div className="flex min-h-screen items-center justify-center bg-background
px-4">
    <div className="max-w-md text-center">
      <h1 className="text-xl font-semibold tracking-tight text-foreground">
        This page didn't load
      </h1>
      <p className="mt-2 text-sm text-muted-foreground">
        Something went wrong on our end. You can try refreshing or head back
home.
      </p>
      <div className="mt-6 flex flex-wrap justify-center gap-2">
        <button
          onClick={() => {
            router.invalidate();
            reset();
          }}
          className="inline-flex items-center justify-center rounded-md bg-primary
px-4 py-2 text-sm font-medium text-primary-foreground transition-colors
hover:bg-primary/90"
        >
          Try again
        </button>
        <a
          href="/"
          className="inline-flex items-center justify-center rounded-md border
border-input bg-background px-4 py-2 text-sm font-medium text-foreground
transition-colors hover:bg-accent"
```

```
          >
            Go home
          </a>
        </div>
      </div>
    </div>
  );
}

export const Route = createRootRouteWithContext<{ queryClient: QueryClient
}>()({
  head: () => ({
    meta: [
      { charSet: "utf-8" },
      { name: "viewport", content: "width=device-width, initial-scale=1" },
      { title: "Fake Rider Motorparts | Book Motorcycle Service" },
      {
        name: "description",
        content:
          "Premium motorparts, accessories and race-grade motorcycle service.
Book your slot online and get a reference code.",
      },
      { name: "author", content: "Fake Rider Motorparts" },
      { property: "og:title", content: "Fake Rider Motorparts" },
      {
        property: "og:description",
        content: "Book motorcycle service online with Fake Rider Motorparts.",
      },
      { property: "og:type", content: "website" },
      { name: "twitter:card", content: "summary_large_image" },
    ],
    links: [
      {
        rel: "stylesheet",
        href: appCss,
      },
      { rel: "icon", href: "/favicon.png", type: "image/png" },
    ],
  }),
  shellComponent: RootShell,
  component: RootComponent,
  notFoundComponent: NotFoundComponent,
  errorComponent: ErrorComponent,
});
```

```
function RootShell({ children }: { children: ReactNode }) {
  return (
    // themeInitScript sets the theme class before React loads, so the server's
    // <html> never matches it exactly.
    <html lang="en" suppressHydrationWarning>
      <head>
        <HeadContent />
        <script dangerouslySetInnerHTML={{ __html: themeInitScript }} />
      </head>
      <body>
        {children}
        <Scripts />
      </body>
    </html>
  );
}

function RootComponent() {
  const { queryClient } = Route.useRouteContext();

  return (
    <QueryClientProvider client={queryClient}>
      {/* Required: nested routes render here. Removing <Outlet /> breaks all child
routes. */}
      <Outlet />
      {OFFLINE && <OfflineConnection />}
    </QueryClientProvider>
  );
}
```

src/routes/_authenticated/admin.tsx

```
import { useQuery, useQueryClient } from "@tanstack/react-query";
import { createFileRoute, Link, Outlet, useNavigate, useRouterState } from
"@tanstack/react-router";
import {
  Archive,
  Ban,
  Bell,
  Bike,
  CalendarDays,
  CalendarX,
  ClipboardList,
  Clock,
  History,
  Home,
  LayoutDashboard,
  LogOut,
  Package,
  Settings,
  Tags,
  UserCheck,
  Users,
  Wrench,
} from "lucide-react";
import { useCallback, useEffect, useRef, useState } from "react";

import logo from "@/assets/logo-shp.png.asset.json";
import { ThemeToggle } from "@/components/site/theme-toggle";
import {
  AlertDialog,
  AlertDialogAction,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
} from "@/components/ui/alert-dialog";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import {
  Sidebar,
  SidebarContent,
  SidebarGroup,
  SidebarGroupContent,
```

```
  SidebarGroupLabel,
  SidebarMenu,
  SidebarMenuButton,
  SidebarMenuItem,
  SidebarProvider,
  SidebarTrigger,
} from "@/components/ui/sidebar";
import { supabase } from "@/integrations/supabase/client";
import { recordAdminActivityEvent } from "@/lib/admin-activity";

export const Route = createFileRoute("/_authenticated/admin")({
  component: AdminLayout,
});

const groups = [
  {
    label: "Overview",
    items: [
      { title: "Dashboard", url: "/admin", icon: LayoutDashboard },
      { title: "Appointments Management", url: "/admin/appointments", icon:
CalendarDays },
    ],
  },
  {
    label: "Shop",
    items: [
      { title: "Services", url: "/admin/services", icon: Wrench },
      { title: "Parts & Accessories", url: "/admin/parts", icon: Package },
      { title: "Motorcycle Catalog", url: "/admin/motorcycles", icon: Bike },
    ],
  },
  {
    label: "Business",
    items: [
      { title: "Prices Management", url: "/admin/prices", icon: Tags },
      { title: "Customer List", url: "/admin/customers", icon: Users },
    ],
  },
  {
    label: "Staff",
    items: [
      { title: "Mechanics", url: "/admin/mechanics", icon: UserCheck },
      { title: "Availability", url: "/admin/availability", icon: CalendarDays },
      { title: "Blocked Customers", url: "/admin/blocked-numbers", icon: Ban },
    ],
```

```
  },
  {
    label: "Shop Schedule",
    items: [{ title: "Schedule Blocks", url: "/admin/schedule-blocks", icon:
CalendarX }],
  },
  {
    label: "Records",
    items: [
      { title: "Reports", url: "/admin/reports", icon: ClipboardList },
      { title: "Archive", url: "/admin/archive", icon: Archive },
      { title: "Activity Log", url: "/admin/activity-log", icon: History },
    ],
  },
  {
    label: "Settings",
    items: [{ title: "Settings", url: "/admin/settings", icon: Settings }],
  },
];

const ADMIN_INACTIVITY_TIMEOUT_MINUTES = 15;
const ADMIN_INACTIVITY_TIMEOUT_MS =
ADMIN_INACTIVITY_TIMEOUT_MINUTES * 60_000;
const ADMIN_INACTIVITY_WARNING_MS = 60_000;
const ADMIN_INACTIVITY_CHECK_INTERVAL_MS = 1_000;
const NOTIFICATION_POLL_INTERVAL_MS = 30_000;
const ADMIN_SESSION_HEARTBEAT_INTERVAL_MS = 30_000;

function AdminLayout() {
  const navigate = useNavigate();
  const queryClient = useQueryClient();
  const path = useRouterState({ select: (s) => s.location.pathname });
  const lastActivityAtRef = useRef(Date.now());
  // Mirrors inactivityDeadline for the window listeners, which must not
  // re-subscribe every time the warning opens or closes.
  const inactivityWarningOpenRef = useRef(false);
  const inactivitySignOutStartedRef = useRef(false);
  const signingOutRef = useRef(false);
  const [isSigningOut, setIsSigningOut] = useState(false);
  const [signOutError, setSignOutError] = useState<string | null>(null);
  const [inactivityDeadline, setInactivityDeadline] = useState<number | null>(null);

  const role = useQuery({
    queryKey: ["my-role"],
    queryFn: async () => {
```

```
      const { data } = await supabase.from("user_roles").select("role");
      return (data ?? []).map((r) => r.role);
    },
  });

  const unread = useQuery({
    queryKey: ["unread-notifications"],
    queryFn: async () => {
      const { count } = await supabase
        .from("notifications")
        .select("id", { count: "exact", head: true })
        .eq("is_read", false);
      return count ?? 0;
    },
    refetchInterval: NOTIFICATION_POLL_INTERVAL_MS,
  });

  useEffect(() => {
    const channel = supabase
      .channel("admin-notifications")
      .on("postgres_changes", { event: "*", schema: "public", table: "notifications" },
() => {
        queryClient.invalidateQueries({ queryKey: ["unread-notifications"] });
        queryClient.invalidateQueries({ queryKey: ["notifications"] });
        queryClient.invalidateQueries({ queryKey: ["admin-appointments"] });
      })
      .subscribe();
    return () => {
      supabase.removeChannel(channel);
    };
  }, [queryClient]);

  const signOut = useCallback(
    async (reason: "manual" | "inactivity" | "public") => {
      if (signingOutRef.current) return;
      signingOutRef.current = true;
      setIsSigningOut(true);
      setSignOutError(null);

      void recordAdminActivityEvent({
        action: "signed out",
        resourceType: "Authentication",
        targetLabel: "Admin console",
        summary:
          reason === "inactivity"
```

```
          ? `Administrator was automatically signed out after
${ADMIN_INACTIVITY_TIMEOUT_MINUTES} minutes of inactivity.`
          : reason === "public"
            ? "Administrator signed out before visiting the public site."
            : "Administrator signed out of the admin console.",
      }).catch(() => {
        // Sign-out should always complete, even if the audit service is unavailable.
      });

      try {
        // Release the database lease before clearing the local Supabase session.
        // If the request cannot complete (for example, the browser is offline),
        // the short server-side lease expires automatically.
        const { error: releaseError } = await
supabase.rpc("release_admin_session");
        if (releaseError)
          console.warn("Could not release the administrator session:", releaseError);

        const { error } = await supabase.auth.signOut({ scope: "local" });
        if (error) throw error;

        const { data, error: sessionError } = await supabase.auth.getSession();
        if (sessionError || data.session) {
          throw new Error("The administrator session could not be cleared.");
        }

        await queryClient.cancelQueries();
        queryClient.clear();
        await navigate({ to: reason === "inactivity" ? "/auth" : "/", replace: true });
      } catch (error) {
        console.error("Could not sign out of the admin console:", error);
        setSignOutError("Could not sign out. Please try again.");
      } finally {
        signingOutRef.current = false;
        setIsSigningOut(false);
      }
    },
    [navigate, queryClient],
  );

  const staySignedIn = useCallback(() => {
    if (signingOutRef.current) return;
    lastActivityAtRef.current = Date.now();
    inactivityWarningOpenRef.current = false;
    inactivitySignOutStartedRef.current = false;
```

```
    setInactivityDeadline(null);
    setSignOutError(null);
  }, []);

  const isAdmin = (role.data ?? []).includes("admin");

  useEffect(() => {
    if (!isAdmin) return;

    let isMounted = true;
    let isRenewing = false;
    const renewSession = async () => {
      if (isRenewing || signingOutRef.current) return;
      isRenewing = true;
      try {
        const { data: renewed, error } = await
supabase.rpc("renew_admin_session");
        if (!isMounted || (!error && renewed)) return;

        // A lease can be lost after it expires, after a server-side logout, or
        // when another approved session handover completes.
        await supabase.auth.signOut({ scope: "local" });
        await queryClient.cancelQueries();
        queryClient.clear();
        await navigate({ to: "/auth", replace: true });
      } finally {
        isRenewing = false;
      }
    };

    void renewSession();
    const intervalId = window.setInterval(
      () => void renewSession(),
      ADMIN_SESSION_HEARTBEAT_INTERVAL_MS,
    );
    return () => {
      isMounted = false;
      window.clearInterval(intervalId);
    };
  }, [isAdmin, navigate, queryClient]);

  useEffect(() => {
    if (!isAdmin) return;

    let isActive = true;
```

```
    let isSyncing = false;
    const syncServiceProgressNotifications = async () => {
      if (isSyncing) return;
      isSyncing = true;
      try {
        const { data, error } = await
supabase.rpc("sync_service_progress_notifications");
        if (!isActive || error || !data) return;
        queryClient.invalidateQueries({ queryKey: ["unread-notifications"] });
        queryClient.invalidateQueries({ queryKey: ["notifications"] });
        queryClient.invalidateQueries({ queryKey: ["admin-appointments"] });
      } finally {
        isSyncing = false;
      }
    };

    void syncServiceProgressNotifications();
    const intervalId = window.setInterval(
      () => void syncServiceProgressNotifications(),
      NOTIFICATION_POLL_INTERVAL_MS,
    );

    return () => {
      isActive = false;
      window.clearInterval(intervalId);
    };
  }, [isAdmin, queryClient]);

  useEffect(() => {
    if (!isAdmin) return;

    lastActivityAtRef.current = Date.now();
    inactivityWarningOpenRef.current = false;
    inactivitySignOutStartedRef.current = false;

    const recordActivity = () => {
      // Once the warning is showing, only an explicit choice keeps the session,
      // so a stray mouse movement cannot dismiss it unseen.
      if (inactivityWarningOpenRef.current || signingOutRef.current) return;
      lastActivityAtRef.current = Date.now();
    };

    // Compare timestamps instead of relying on one long timer: browsers delay
    // timers in background tabs, and the countdown must stay accurate when the
    // administrator returns to the tab.
```

```
    const checkInactivity = () => {
      if (signingOutRef.current) return;
      const deadline = lastActivityAtRef.current +
ADMIN_INACTIVITY_TIMEOUT_MS;
      const remaining = deadline - Date.now();
      if (remaining <= 0) {
        // Try once per timeout. If sign-out fails, the warning stays open with
        // the error so the administrator can retry or keep working.
        if (inactivitySignOutStartedRef.current) return;
        inactivitySignOutStartedRef.current = true;
        inactivityWarningOpenRef.current = true;
        setInactivityDeadline(deadline);
        void signOut("inactivity");
      } else if (remaining <= ADMIN_INACTIVITY_WARNING_MS &&
!inactivityWarningOpenRef.current) {
        inactivityWarningOpenRef.current = true;
        setInactivityDeadline(deadline);
      }
    };

    const activityEvents = ["mousemove", "mousedown", "keydown", "scroll",
"touchstart"] as const;
    for (const eventName of activityEvents) {
      // Capture also sees scrolling inside dialogs and panels, which does not
bubble.
      window.addEventListener(eventName, recordActivity, { capture: true,
passive: true });
    }
    document.addEventListener("visibilitychange", checkInactivity);
    const intervalId = window.setInterval(checkInactivity,
ADMIN_INACTIVITY_CHECK_INTERVAL_MS);

    return () => {
      window.clearInterval(intervalId);
      document.removeEventListener("visibilitychange", checkInactivity);
      for (const eventName of activityEvents) {
        window.removeEventListener(eventName, recordActivity, { capture: true });
      }
    };
  }, [isAdmin, signOut]);

  return (
    <SidebarProvider>
      <div className="flex min-h-screen w-full bg-background">
        <Sidebar collapsible="icon">
```

```
        <SidebarContent>
          <div className="flex-shrink-0">
            <div className="flex items-center gap-2 px-3 py-4">
              <img src={logo.url} alt="Fake Rider logo" className="h-16 w-auto
object-contain" />
              <span className="-ml-4 font-display text-sm leading-tight
tracking-wide uppercase">
                Fake Rider
                <span className="block text-[10px] text-muted-foreground">Admin
console</span>
              </span>
            </div>
          </div>
          {groups.map((group) => (
            <SidebarGroup key={group.label}>
              <SidebarGroupLabel className="text-[10px] tracking-[0.2em]
uppercase">
                {group.label}
              </SidebarGroupLabel>
              <SidebarGroupContent>
                <SidebarMenu>
                  {group.items.map((item) => (
                    <SidebarMenuItem key={item.url}>
                      <SidebarMenuButton
                        asChild
                        isActive={path === item.url}
                        tooltip={item.title}
                      >
                        <Link to={item.url} className="flex items-center gap-2">
                          <item.icon className="h-4 w-4" />
                          <span className="truncate text-xs
uppercase">{item.title}</span>
                          {item.url === "/admin/notifications" && (unread.data ?? 0) > 0
&& (
                            <Badge className="ml-auto h-5 min-w-5 justify-center
bg-primary px-1 text-[10px] text-primary-foreground">
                              {unread.data}
                            </Badge>
                          )}
                        </Link>
                      </SidebarMenuButton>
                    </SidebarMenuItem>
                  ))}
                </SidebarMenu>
              </SidebarGroupContent>
```

```
            </SidebarGroup>
          ))}
        </SidebarContent>
      </Sidebar>

      <div className="flex min-h-0 flex-1 flex-col">
        <header className="sticky top-0 z-10 flex min-h-14 items-center gap-2
border-b border-border bg-background/90 px-3 py-1.5 backdrop-blur md:h-14
md:px-4 md:py-0">
          <SidebarTrigger className="shrink-0" />
          <button
            type="button"
            onClick={() => void signOut("public")}
            className="inline-flex h-11 w-11 shrink-0 items-center justify-center
rounded-md text-muted-foreground transition-colors hover:bg-secondary
hover:text-foreground focus-visible:outline-none focus-visible:ring-1
focus-visible:ring-ring md:w-auto md:gap-2 md:px-2"
            disabled={isSigningOut}
          >
            <Home className="h-4 w-4" />
            <span className="hidden text-xs uppercase md:inline">Public
site</span>
            <span className="sr-only md:hidden">View public site</span>
          </button>
          <ThemeToggle className="shrink-0" />
          <div className="ml-auto flex shrink-0 items-center gap-1.5 sm:gap-3">
            <Link
              to="/admin/notifications"
              className="relative inline-flex h-11 w-11 items-center justify-center
rounded-md text-muted-foreground transition-colors hover:bg-secondary
hover:text-foreground focus-visible:outline-none focus-visible:ring-1
focus-visible:ring-ring md:h-9 md:w-9"
              aria-label="View notifications"
            >
              <Bell className="h-5 w-5 text-muted-foreground" />
              {(unread.data ?? 0) > 0 && (
                <span className="absolute -top-1 -right-1 flex h-4 min-w-4
items-center justify-center rounded-full bg-primary px-1 text-[10px] font-bold
text-primary-foreground">
                  {unread.data}
                </span>
              )}
            </Link>
            <Button
              variant="outline"
```

```
              size="sm"
              onClick={() => void signOut("manual")}
              className="font-display uppercase"
              disabled={isSigningOut}
            >
              <LogOut /> <span className="hidden sm:inline">Exit</span>
            </Button>
          </div>
        </header>

        <main className="flex-1 p-4 md:p-6">
          {signOutError && (
            <p role="alert" className="mb-4 text-sm text-destructive">
              {signOutError}
            </p>
          )}
          {role.isLoading ? (
            <p className="text-sm text-muted-foreground">Checking your
access...</p>
          ) : isAdmin ? (
            <Outlet />
          ) : (
            <div className="mx-auto max-w-md rounded-xl border
border-destructive/40 bg-destructive/5 p-8 text-center">
              <h1 className="font-display text-xl uppercase">No admin
access</h1>
              <p className="mt-2 text-sm text-muted-foreground">
                This account is signed in but has no administrator role for the shop
console.
              </p>
              <Button
                className="mt-5 font-display uppercase"
                variant="outline"
                onClick={() => void signOut("manual")}
                disabled={isSigningOut}
              >
                Exit
              </Button>
            </div>
          )}
        </main>
      </div>
    </div>
    <InactivityWarningDialog
      deadline={inactivityDeadline}
```

```
        isSigningOut={isSigningOut}
        error={signOutError}
        onStay={staySignedIn}
        onSignOut={() => void signOut("manual")}
      />
    </SidebarProvider>
  );
}

function InactivityWarningDialog({
  deadline,
  isSigningOut,
  error,
  onStay,
  onSignOut,
}: {
  deadline: number | null;
  isSigningOut: boolean;
  error: string | null;
  onStay: () => void;
  onSignOut: () => void;
}) {
  const stayButtonRef = useRef<HTMLButtonElement>(null);
  const [now, setNow] = useState(() => Date.now());

  // The countdown ticks here so the admin page behind the dialog does not
  // re-render every second.
  useEffect(() => {
    if (deadline === null) return;
    setNow(Date.now());
    const intervalId = window.setInterval(() => setNow(Date.now()), 1_000);
    return () => window.clearInterval(intervalId);
  }, [deadline]);

  const secondsLeft = deadline === null ? 0 : Math.max(0, Math.ceil((deadline -
now) / 1_000));

  return (
    <AlertDialog
      open={deadline !== null}
      onOpenChange={(open) => {
        if (!open && !isSigningOut) onStay();
      }}
    >
      <AlertDialogContent
```

```
        className="sm:max-w-md"
        onOpenAutoFocus={(event) => {
          event.preventDefault();
          stayButtonRef.current?.focus();
        }}
      >
        <AlertDialogHeader>
          <AlertDialogTitle className="flex items-center justify-center gap-2
font-display uppercase sm:justify-start">
            <Clock className="h-5 w-5 text-primary" aria-hidden="true" /> Stay
signed in?
          </AlertDialogTitle>
          <AlertDialogDescription>
            {isSigningOut
              ? "Signing you out…"
              : secondsLeft > 0
                ? `You haven't used the admin console for a while. For your security,
you'll be signed out in ${secondsLeft} ${secondsLeft === 1 ? "second" :
"seconds"}, and any unsaved changes will be lost.`
                : "Your session timed out because of inactivity."}
          </AlertDialogDescription>
        </AlertDialogHeader>
        {error && (
          <p role="alert" className="text-sm text-destructive">
            {error}
          </p>
        )}
        <AlertDialogFooter>
          <Button variant="outline" onClick={onSignOut} disabled={isSigningOut}>
            <LogOut /> Sign out now
          </Button>
          <AlertDialogAction ref={stayButtonRef} onClick={onStay}
disabled={isSigningOut}>
            Stay signed in
          </AlertDialogAction>
        </AlertDialogFooter>
      </AlertDialogContent>
    </AlertDialog>
  );
}
```

src/routes/_authenticated/admin/activity-log.tsx

```
import { useQuery, useQueryClient } from "@tanstack/react-query";
import { createFileRoute } from "@tanstack/react-router";
import { format } from "date-fns";
import { CalendarRange, ChevronDown, History, Printer, RefreshCw } from
"lucide-react";
import type { DateRange } from "react-day-picker";
import { useDeferredValue, useEffect, useMemo, useState } from "react";

import {
  AdminPage,
  DataList,
  InlineAlert,
  ListCard,
  ListEmpty,
  ListToolbar,
  ResetFiltersButton,
  ToolbarFilters,
  ToolbarSearch,
} from "@/components/admin/data-list";
import { PageHeader } from "@/components/admin/page-header";
import { PaginationControls } from "@/components/admin/pagination-controls";
import { PrintDocument, PrintTable } from
"@/components/admin/print-document";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Calendar } from "@/components/ui/calendar";
import { Popover, PopoverContent, PopoverTrigger } from
"@/components/ui/popover";
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@/components/ui/select";
import { supabase } from "@/integrations/supabase/client";
import {
  calendarRangeToIso,
  isoDateToCalendarDate,
  manilaMonthDateRange,
  manilaYearDateRange,
} from "@/lib/admin-date-range";
import { formatBusinessTimestamp } from "@/lib/export-branding";
```

```
import { recordAdminActivityEvent } from "@/lib/admin-activity";
import { usePrintWhenReady } from "@/lib/print-document";
import { addDays, manilaNow } from "@/lib/shop";
import { cn } from "@/lib/utils";

export const Route = createFileRoute("/_authenticated/admin/activity-log")({
  component: ActivityLogPage,
});

const actionOptions = [
  "created",
  "updated",
  "status changed",
  "activated",
  "deactivated",
  "archived",
  "restored",
  "deleted",
  "exported",
  "printed",
  "signed in",
  "signed out",
];

const resourceOptions = [
  "Appointment Services",
  "Appointments",
  "Services",
  "Products",
  "Crew Members",
  "Schedule Blocks",
  "Blocked Numbers",
  "Time Slots",
  "Crew Schedules",
  "Crew Availability Exceptions",
  "Price History",
  "Notifications",
  "User Roles",
  "Activity Log",
  "Authentication",
  "Reports",
  "Settings",
  "Account Settings",
];
```

```
type ActivityLog = {
  id: string;
  actor_email: string | null;
  action: string;
  resource_type: string;
  target_label: string;
  summary: string;
  changed_fields: string[];
  activity_date: string;
  activity_time: string;
  created_at: string;
  record_id: string | null;
  ip_address: string | null;
};

type ActivityPrint = {
  records: ActivityLog[];
  generatedAt: string;
  coverage: string;
  scope: string;
};
type ActivityPeriod = "all" | "today" | "7d" | "month" | "year" | "custom";
type ActivityDateRange = { from: string; to: string };

function ActivityLogPage() {
  const pageSize = 10;
  const queryClient = useQueryClient();
  const [search, setSearch] = useState("");
  const [action, setAction] = useState("all");
  const [resource, setResource] = useState("all");
  const [activityPeriod, setActivityPeriod] = useState<ActivityPeriod>("all");
  const [activityCustomDateRange, setActivityCustomDateRange] =
useState<DateRange>();
  const [page, setPage] = useState(0);
  const [printJob, setPrintJob] = useState<ActivityPrint | null>(null);
  const [isPrinting, setIsPrinting] = useState(false);
  const [printError, setPrintError] = useState<string | null>(null);
  const deferredSearch = useDeferredValue(search);
  const filtersAreUpdating = search !== deferredSearch;
  const today = manilaNow().date;
  const activityDateRange = useMemo(
    () => activityRangeFromPeriod(activityPeriod, today,
activityCustomDateRange),
    [activityCustomDateRange, activityPeriod, today],
  );
```

```
  const fromDate = activityDateRange?.from ?? "";
  const toDate = activityDateRange?.to ?? "";

  const hasFilters = Boolean(
    search.trim() || action !== "all" || resource !== "all" || activityPeriod !== "all",
  );

  const logs = useQuery({
    queryKey: [
      "admin-activity-logs",
      {
        search: deferredSearch,
        action,
        resource,
        fromDate,
        toDate,
        page,
      },
    ],
    queryFn: async () => {
      const { error: cleanupError } = await
supabase.rpc("purge_expired_admin_activity_logs");
      if (cleanupError) throw cleanupError;
      let query = supabase
        .from("admin_activity_logs")
        .select("*", { count: "exact" })
        .order("created_at", { ascending: false })
        .range(page * pageSize, page * pageSize + pageSize - 1);

      if (fromDate) query = query.gte("activity_date", fromDate);
      if (toDate) query = query.lte("activity_date", toDate);

      if (action !== "all") query = query.eq("action", action);
      if (resource !== "all") query = query.eq("resource_type", resource);
      if (deferredSearch.trim()) {
        const term = cleanSearchTerm(deferredSearch);
        query = query.or(
          `summary.ilike.%${term}%,actor_email.ilike.%${term}%,target_label.ilike.%
${term}%,ip_address.ilike.%${term}%`,
        );
      }

      const { data, error, count } = await query;
      if (error) throw error;
      return { rows: (data ?? []) as ActivityLog[], total: count ?? 0 };
```

```
    },
  });

  const rows = logs.data?.rows ?? [];
  const filterDescription = useMemo(() => {
    const total = logs.data?.total ?? 0;
    const noun = total === 1 ? "record" : "records";
    return hasFilters ? `${total} matching ${noun}` : `${total} ${noun} from the last
50 days`;
  }, [hasFilters, logs.data?.total]);

  useEffect(() => {
    setPage(0);
  }, [search, action, resource, activityPeriod, activityCustomDateRange]);

  function refresh() {
    queryClient.invalidateQueries({ queryKey: ["admin-activity-logs"] });
  }

  function clearFilters() {
    setSearch("");
    setAction("all");
    setResource("all");
    setActivityPeriod("all");
    setActivityCustomDateRange(undefined);
    setPage(0);
  }

  async function fetchAllFilteredRows() {
    const { error: cleanupError } = await
supabase.rpc("purge_expired_admin_activity_logs");
    if (cleanupError) throw cleanupError;

    const exportPageSize = 1_000;
    const allRows: ActivityLog[] = [];

    for (let offset = 0; ; offset += exportPageSize) {
      let query = supabase
        .from("admin_activity_logs")
        .select("*")
        .order("created_at", { ascending: false })
        .order("id", { ascending: false });

      if (fromDate) query = query.gte("activity_date", fromDate);
      if (toDate) query = query.lte("activity_date", toDate);
```

```
    if (action !== "all") query = query.eq("action", action);
    if (resource !== "all") query = query.eq("resource_type", resource);
    if (deferredSearch.trim()) {
      const term = cleanSearchTerm(deferredSearch);
      query = query.or(
        `summary.ilike.%${term}%,actor_email.ilike.%${term}%,target_label.ilike.%
${term}%,ip_address.ilike.%${term}%`,
      );
    }

    const { data, error } = await query.range(offset, offset + exportPageSize - 1);
    if (error) throw error;

    const batch = (data ?? []) as ActivityLog[];
    allRows.push(...batch);
    if (batch.length < exportPageSize) return allRows;
  }
 }

 usePrintWhenReady(printJob && `Admin Activity Log - ${printJob.coverage}`, ()
=>
  setPrintJob(null),
 );

 /** The dates the printout covers, as shown on its first page. */
 function printCoverage() {
  if (!fromDate || !toDate) return "All retained activity (last 50 days)";
  const label = activityPeriodLabel(activityPeriod, activityCustomDateRange);
  const dates =
    fromDate === toDate
      ? formatActivityDate(fromDate)
      : `${formatActivityDate(fromDate)} to ${formatActivityDate(toDate)}`;
  return activityPeriod === "custom" ? dates : `${label} (${dates})`;
 }

 /** The filters behind the printout, as shown on its first page. */
 function printScope(recordCount: number) {
  return [
    `${recordCount} ${recordCount === 1 ? "activity record" : "activity records"}`,
    `Action: ${action === "all" ? "All actions" : action}`,
    `Category: ${resource === "all" ? "All categories" : resource}`,
    deferredSearch.trim() ? `Search: ${deferredSearch.trim()}` : "",
  ]
    .filter(Boolean)
    .join(" · ");
```

```
  }

  async function printActivityLog() {
    if (isPrinting) return;
    setIsPrinting(true);
    try {
      setPrintError(null);
      // The printout lists every matching record, not only the current page.
      const records = await fetchAllFilteredRows();
      setPrintJob({
        records,
        generatedAt: formatBusinessTimestamp(),
        coverage: printCoverage(),
        scope: printScope(records.length),
      });
      try {
        await recordAdminActivityEvent({
          action: "printed",
          resourceType: "Activity Log",
          targetLabel: "Admin Activity Log",
          summary: `Opened the print dialog for ${records.length} activity
${records.length === 1 ? "record" : "records"}.`,
          changedFields: ["date_range", "print_format"],
        });
        queryClient.invalidateQueries({ queryKey: ["admin-activity-logs"] });
      } catch (error) {
        // An audit-log failure must not stop a valid printout.
        console.error("Could not record activity log print:", error);
      }
    } catch (error) {
      console.error("Could not prepare the activity log for printing:", error);
      setPrintError("Could not prepare the activity log for printing. Please try
again.");
    } finally {
      setIsPrinting(false);
    }
  }

  return (
    <AdminPage>
      <PageHeader
        title="Activity Log"
        description="Admin changes and sign-in events are retained for 50 days
with record IDs and request IP addresses."
        action={
```

```
        <div className="flex gap-2">
          <Button variant="outline" onClick={refresh} disabled={logs.isFetching}>
            <RefreshCw className={logs.isFetching ? "animate-spin" : ""} />
Refresh
          </Button>
          <Button
            variant="outline"
            className="font-display uppercase"
            onClick={() => void printActivityLog()}
            disabled={
              (logs.data?.total ?? 0) === 0 || logs.isFetching || filtersAreUpdating ||
isPrinting
            }
          >
            <Printer /> {isPrinting ? "Preparing print..." : "Print log"}
          </Button>
        </div>
      }
    />

    {printError && <InlineAlert>{printError}</InlineAlert>}

    <ListToolbar summary={logs.data && filterDescription}>
      <ToolbarSearch
        id="activity-search"
        value={search}
        onChange={setSearch}
        placeholder="Search activity, user, record, or IP"
        label="Search activity by summary, user, record, or IP address"
      />
      <ToolbarFilters>
        <Select value={action} onValueChange={setAction}>
          <SelectTrigger aria-label="Filter by action" className="w-full sm:w-40">
            <SelectValue />
          </SelectTrigger>
          <SelectContent>
            <SelectItem value="all">All actions</SelectItem>
            {actionOptions.map((option) => (
              <SelectItem key={option} value={option} className="capitalize">
                {option}
              </SelectItem>
            ))}
          </SelectContent>
        </Select>
        <Select value={resource} onValueChange={setResource}>
```

```
          <SelectTrigger aria-label="Filter by category" className="w-full
sm:w-48">
            <SelectValue />
          </SelectTrigger>
          <SelectContent>
            <SelectItem value="all">All categories</SelectItem>
            {resourceOptions.map((option) => (
              <SelectItem key={option} value={option}>
                {option}
              </SelectItem>
            ))}
          </SelectContent>
        </Select>
        <ActivityPeriodFilter
          period={activityPeriod}
          customDateRange={activityCustomDateRange}
          onPeriodChange={setActivityPeriod}
          onCustomDateRangeChange={setActivityCustomDateRange}
          className="col-span-2 w-full justify-between sm:w-auto"
        />
        {hasFilters && <ResetFiltersButton onClick={clearFilters} />}
      </ToolbarFilters>
    </ListToolbar>

    <ListCard
      footer={
        <PaginationControls
          page={page}
          pageSize={pageSize}
          total={logs.data?.total ?? 0}
          onPageChange={setPage}
        />
      }
    >
      <DataList
        label="Activity records"
        rows={rows}
        rowKey={(row) => row.id}
        loading={logs.isLoading}
        error={
          logs.isError
            ? { message: "Could not load the activity log.", onRetry: () => void
logs.refetch() }
            : null
        }
```

```
        empty={
          <ListEmpty
            icon={History}
            title={
              hasFilters
                ? "No activity matches your filters."
                : "No activity in the last 50 days."
            }
            description={
              hasFilters
                ? "Try a different search, action, category, or date."
                : "Admin changes and sign-ins will appear here."
            }
            action={hasFilters ? <ResetFiltersButton onClick={clearFilters} /> :
undefined}
          />
        }
        columns={[
          {
            key: "when",
            header: "When",
            className: "w-36",
            cell: (row) => (
              <span className="text-sm">
                {formatActivityDate(row.activity_date)}
                <span className="block text-xs text-muted-foreground">
                  {formatActivityTime(row.activity_time)}
                </span>
              </span>
            ),
          },
          {
            key: "activity",
            header: "Activity",
            card: "title",
            cell: (row) => (
              <>
                <p className="text-sm font-medium">{row.summary}</p>
                <p className="mt-0.5 text-xs text-muted-foreground">
                  {row.target_label}
                  <span className="block">by {row.actor_email ?? "Unknown
admin"}</span>
                </p>
              </>
            ),
```

```
          },
          {
            key: "category",
            header: "Category",
            card: "badge",
            className: "w-40",
            cell: (row) => (
              <Badge variant="outline" className="max-w-full text-left
whitespace-normal">
                {row.resource_type}
              </Badge>
            ),
          },
          {
            key: "fields",
            header: "Changed fields",
            card: "wide",
            xlOnly: true,
            className: "w-56",
            cell: (row) =>
              row.changed_fields.length > 0 ? (
                <span className="flex flex-wrap gap-1">
                  {row.changed_fields.map((field) => (
                    <Badge key={field} variant="secondary" className="font-normal">
                      {field.replace(/_/g, " ")}
                    </Badge>
                  ))}
                </span>
              ) : (
                <span className="text-sm text-muted-foreground">None</span>
              ),
          },
          {
            key: "record",
            header: "Record ID",
            table: false,
            cell: (row) => (
              <span className="font-mono text-xs break-all">{row.record_id ??
"None"}</span>
            ),
          },
          {
            key: "ip",
            header: "IP address",
            table: false,
```

```
            cell: (row) => (
              <span className="font-mono text-xs">{row.ip_address ??
"Unavailable"}</span>
            ),
          },
          {
            key: "source",
            header: "Record & IP",
            card: false,
            xlOnly: true,
            className: "w-52",
            cell: (row) => (
              <span className="font-mono text-xs break-all">
                {row.record_id ?? "No record ID"}
                <span className="block text-muted-foreground">
                  {row.ip_address ?? "IP unavailable"}
                </span>
              </span>
            ),
          },
        ]}
      />
    </ListCard>

    <PrintActivityLog job={printJob} />
  </AdminPage>
 );
}

function ActivityPeriodFilter({
 period,
 customDateRange,
 onPeriodChange,
 onCustomDateRangeChange,
 className,
}: {
 period: ActivityPeriod;
 customDateRange: DateRange | undefined;
 onPeriodChange: (period: ActivityPeriod) => void;
 onCustomDateRangeChange: (range: DateRange | undefined) => void;
 className?: string;
}) {
 const [open, setOpen] = useState(false);
 const [isPickingCustomRange, setIsPickingCustomRange] = useState(false);
 const [draftCustomDateRange, setDraftCustomDateRange] =
```

```
useState<DateRange>();

  const selectPeriod = (nextPeriod: Exclude<ActivityPeriod, "custom">) => {
    onPeriodChange(nextPeriod);
    setIsPickingCustomRange(false);
    setOpen(false);
  };

  const cancelCustomRange = () => {
    setDraftCustomDateRange(customDateRange);
    setIsPickingCustomRange(false);
    setOpen(false);
  };

  const applyCustomRange = () => {
    if (!draftCustomDateRange?.from || !draftCustomDateRange.to) return;
    onCustomDateRangeChange(draftCustomDateRange);
    onPeriodChange("custom");
    setIsPickingCustomRange(false);
    setOpen(false);
  };

  return (
    <Popover
      open={open}
      onOpenChange={(nextOpen) => {
        setOpen(nextOpen);
        if (!nextOpen) setIsPickingCustomRange(false);
      }}
    >
      <PopoverTrigger asChild>
        <Button type="button" variant="outline" className={cn("gap-1.5",
className)}>
          <CalendarRange className="size-3.5" aria-hidden="true" />
          {activityPeriodLabel(period, customDateRange)}
          <ChevronDown className="size-3.5" aria-hidden="true" />
        </Button>
      </PopoverTrigger>
      <PopoverContent align="end" className="w-72 p-2">
        {isPickingCustomRange ? (
          <div>
            <div className="px-2 pb-2">
              <p className="text-sm font-medium">Custom activity date range</p>
              <p className="text-xs text-muted-foreground">Select a start date and
an end date.</p>
```

```
          </div>
          <Calendar
            mode="range"
            selected={draftCustomDateRange}
            onSelect={setDraftCustomDateRange}
            className="mx-auto p-0"
            classNames={{
              nav: "inset-x-auto left-1/2 w-48 -translate-x-1/2 justify-between",
            }}
          />
          <div className="mt-2 grid grid-cols-2 gap-2 px-2 text-xs">
            <ActivityDateRangeValue label="Start date"
value={draftCustomDateRange?.from} />
            <ActivityDateRangeValue label="End date"
value={draftCustomDateRange?.to} />
          </div>
          <div className="mt-3 flex justify-end gap-2 border-t border-border pt-3">
            <Button type="button" variant="outline" size="sm"
onClick={cancelCustomRange}>
              Cancel
            </Button>
            <Button
              type="button"
              size="sm"
              onClick={applyCustomRange}
              disabled={!draftCustomDateRange?.from ||
!draftCustomDateRange.to}
            >
              Apply
            </Button>
          </div>
        </div>
      ) : (
        <div className="space-y-1">
          <p className="px-2 pb-1 text-xs font-medium tracking-wide
text-muted-foreground uppercase">
            Activity date
          </p>
          <ActivityPeriodOption
            active={period === "all"}
            label="All activity dates"
            onClick={() => selectPeriod("all")}
          />
          <ActivityPeriodOption
            active={period === "today"}
```

```
            label="Today"
            onClick={() => selectPeriod("today")}
          />
          <ActivityPeriodOption
            active={period === "7d"}
            label="Last 7 days"
            onClick={() => selectPeriod("7d")}
          />
          <ActivityPeriodOption
            active={period === "month"}
            label="This month"
            onClick={() => selectPeriod("month")}
          />
          <ActivityPeriodOption
            active={period === "year"}
            label="This year"
            onClick={() => selectPeriod("year")}
          />
          <ActivityPeriodOption
            active={period === "custom"}
            label="Custom date range"
            onClick={() => {
              setDraftCustomDateRange(customDateRange);
              setIsPickingCustomRange(true);
            }}
          />
        </div>
      )}
    </PopoverContent>
  </Popover>
 );
}

function ActivityPeriodOption({
 active,
 label,
 onClick,
}: {
 active: boolean;
 label: string;
 onClick: () => void;
}) {
 return (
  <Button
    type="button"
```

```
      variant="ghost"
      className={cn("h-9 w-full justify-start text-sm", active && "bg-muted")}
      onClick={onClick}
    >
      {label}
    </Button>
  );
}

function ActivityDateRangeValue({ label, value }: { label: string; value: Date |
undefined }) {
  return (
    <div className="rounded-md border border-border bg-muted/30 px-2
py-1.5">
      <p className="text-[10px] tracking-wide text-muted-foreground
uppercase">{label}</p>
      <p className="mt-0.5 truncate font-medium text-foreground">
        {value ? format(value, "MMM d, yyyy") : "Not selected"}
      </p>
    </div>
  );
}

function activityPeriodLabel(period: ActivityPeriod, customRange?: DateRange) {
  if (period === "all") return "All activity dates";
  if (period === "today") return "Today";
  if (period === "7d") return "Last 7 days";
  if (period === "month") return "This month";
  if (period === "year") return "This year";
  if (!customRange?.from || !customRange.to) return "Custom date range";
  return `${format(customRange.from, "MMM d")} – ${format(customRange.to,
"MMM d, yyyy")}`;
}

function activityRangeFromPeriod(
  period: ActivityPeriod,
  today: string,
  customRange?: DateRange,
): ActivityDateRange | undefined {
  if (period === "all") return undefined;
  if (period === "today") return { from: today, to: today };
  if (period === "7d") return { from: addDays(today, -6), to: today };

  if (period === "month") {
    return manilaMonthDateRange(today);
```

```
  }
  if (period === "year") {
    return manilaYearDateRange(today);
  }
  if (customRange?.from && customRange.to) {
    return calendarRangeToIso(customRange);
  }
  return undefined;
}

function cleanSearchTerm(value: string) {
  return value
    .trim()
    .replace(/[,%_()]/g, " ")
    .replace(/ /g, " ");
}

function formatActivityDate(date: string) {
  return isoDateToCalendarDate(date).toLocaleDateString("en-PH", {
    day: "numeric",
    month: "short",
    year: "numeric",
  });
}

function formatActivityTime(time: string) {
  const [hours = "0", minutes = "0"] = time.split(":");
  const hour = Number(hours);
  return `${hour % 12 || 12}:${minutes} ${hour >= 12 ? "PM" : "AM"}`;
}

function printRows(rows: ActivityLog[]) {
  return rows.map((row) => [
    formatActivityDate(row.activity_date),
    formatActivityTime(row.activity_time),
    row.actor_email ?? "Unknown admin",
    row.action,
    row.resource_type,
    row.target_label,
    row.record_id ?? "—",
    row.ip_address ?? "Unavailable",
    row.changed_fields.map((field) => field.replace(/_/g, " ")).join(", ") || "—",
  ]);
}
```

```
/** The printable activity log: every matching record in one table. */
function PrintActivityLog({ job }: { job: ActivityPrint | null }) {
  if (!job) return null;
  return (
    <PrintDocument
      info={{
        title: "Admin Activity Log",
        subtitle: "Activity record",
        generatedAt: job.generatedAt,
        coverage: job.coverage,
        scope: job.scope,
      }}
    >
      <PrintTable
        title="Activity records"
        headers={[
          "Date",
          "Time",
          "User",
          "Action",
          "Category",
          "Record",
          "Record ID",
          "IP address",
          "Changed fields",
        ]}
        columnWidths={["8%", "6%", "14%", "8%", "10%", "14%", "16%", "9%",
"15%"]}
        rows={printRows(job.records)}
        emptyMessage="No activity matches these filters."
        lastColumn="text"
      />
    </PrintDocument>
  );
}
```

src/routes/_authenticated/admin/appointments.tsx

```
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import { createFileRoute, useNavigate } from "@tanstack/react-router";
import { useServerFn } from "@tanstack/react-start";
import {
  CalendarClock,
  CalendarDays,
  CalendarRange,
  CheckCircle2,
  ChevronDown,
  CircleAlert,
  CircleX,
  ClipboardCheck,
  Eye,
  Loader2,
  Pencil,
  Play,
  type LucideIcon,
} from "lucide-react";
import { type ReactNode, useDeferredValue, useEffect, useMemo, useRef,
useState } from "react";
import type { DateRange } from "react-day-picker";
import { z } from "zod";
import { format, parseISO } from "date-fns";

import { ArchiveConfirmationDialog } from
"@/components/admin/archive-confirmation-dialog";
import {
  AdminPage,
  ArchiveMenuItem,
  DataList,
  InlineAlert,
  ListCard,
  ListEmpty,
  ListToolbar,
  MoreActionsMenu,
  ResetFiltersButton,
  ToolbarFilters,
  ToolbarSearch,
} from "@/components/admin/data-list";
import { PageHeader } from "@/components/admin/page-header";
import { PaginationControls } from "@/components/admin/pagination-controls";
import {
  AlertDialog,
```

```
  AlertDialogAction,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
} from "@/components/ui/alert-dialog";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Calendar } from "@/components/ui/calendar";
import { Checkbox } from "@/components/ui/checkbox";
import {
  Dialog,
  DialogContent,
  DialogDescription,
  DialogFooter,
  DialogHeader,
  DialogTitle,
} from "@/components/ui/dialog";
import { DropdownMenuItem, DropdownMenuSeparator } from
"@/components/ui/dropdown-menu";
import { FieldError } from "@/components/ui/field-error";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Popover, PopoverContent, PopoverTrigger } from
"@/components/ui/popover";
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@/components/ui/select";
import { Textarea } from "@/components/ui/textarea";
import { supabase } from "@/integrations/supabase/client";
import type { Database } from "@/integrations/supabase/types";
import {
  calendarDateToIso,
  calendarRangeToIso,
  isoDateToCalendarDate,
  manilaMonthDateRange,
  manilaYearDateRange,
  type IsoDateRange,
} from "@/lib/admin-date-range";
```

```
import {
  ACTIVE_RESERVATION_STATUSES,
  type Availability,
  computeAvailableDates,
  computeAvailableSlots,
  computeFullyBookedDates,
  evaluateCrewAvailability,
  isActiveReservationStatus,
} from "@/lib/availability";
import {
  reviewPendingAppointmentWithConfirmation,
  reviewRescheduleRequestWithConfirmation,
  updateAppointmentServiceProgressWithNotifications,
  updateAppointmentWithConfirmation,
} from "@/lib/appointment-confirmation.functions";
import { getAvailability } from "@/lib/booking.functions";
import {
  APPOINTMENT_STATUSES,
  COMPLETED_APPOINTMENT_KEPT,
  addDays,
  formatDateLong,
  formatPHP,
  formatTime,
  formatNamePartInput,
  manilaNow,
  normalizePhilippineMobile,
  PHONE_VALIDATION_MESSAGE,
  sanitizePhilippineMobileInput,
  statusLabel,
  statusTone,
  toLocalPhilippineMobile,
} from "@/lib/shop";
import { cn } from "@/lib/utils";

export const Route = createFileRoute("/_authenticated/admin/appointments")({
  validateSearch: z.object({
    appointmentId: z.string().uuid().optional(),
    // The dashboard's Needs action card opens the list with this filter.
    status: z.literal("needs_action").optional(),
  }),
  component: AppointmentsPage,
});

type AppointmentDatePeriod = "all" | "today" | "7d" | "month" | "year" | "custom";
const EMPTY_SERVICE_IDS: string[] = [];
```

```
type AppointmentService = Pick<
  Database["public"]["Tables"]["appointment_services"]["Row"],
  "service_id" | "service_name" | "price" | "duration_minutes"
>;
type AppointmentProduct = Pick<
  Database["public"]["Tables"]["appointment_products"]["Row"],
  "id" | "product_id" | "product_name" | "quantity" | "unit_price"
>;
type RescheduleHistoryEntry = {
  id: string;
  appointment_id: string;
  new_appointment_id: string | null;
  reschedule_number: number;
  from_date: string;
  from_start_time: string;
  to_date: string | null;
  to_start_time: string | null;
  reason: string;
  decision: string;
  approved_at: string;
};
type AppointmentDetails = Database["public"]["Tables"]["appointments"]["Row"] &
{
  appointment_services: AppointmentService[];
  appointment_products: AppointmentProduct[];
  crew_members: { name: string } | null;
};
type EditableService = Pick<
  Database["public"]["Tables"]["services"]["Row"],
  "id" | "name" | "price" | "duration_minutes"
>;
type EditableProduct = Pick<
  Database["public"]["Tables"]["products"]["Row"],
  "id" | "name" | "brand" | "price" | "stock_quantity" | "is_active" | "in_stock"
>;
type AppointmentProductInput = {
  productId: string;
  quantity: number;
  unitPrice: number;
};
type AppointmentEditForm = {
  firstName: string;
  middleName: string;
  lastName: string;
  phone: string;
```

```
  serviceIds: string[];
  appointmentDate: string;
  startTime: string;
  assignedCrewId: string;
  crewAssignmentManual: boolean;
  status: string;
  adminNotes: string;
  products: AppointmentProductInput[];
};
type AppointmentEditErrors = Partial<
  Record<
    | "firstName"
    | "middleName"
    | "lastName"
    | "phone"
    | "services"
    | "products"
    | "appointmentDate"
    | "startTime"
    | "schedule"
    | "assignedCrew"
    | "status"
    | "form",
    string
  >
>;

const PRODUCT_RECORDABLE_STATUSES = new Set(["confirmed",
"in_progress", "completed"]);

function canRecordAppointmentProducts(status: string) {
  return PRODUCT_RECORDABLE_STATUSES.has(status);
}

function AppointmentServicesList({
  appointmentId,
  services,
}: {
  appointmentId: string;
  services: AppointmentService[];
}) {
  const [isExpanded, setIsExpanded] = useState(false);
  const [isTruncated, setIsTruncated] = useState(false);
  const serviceListRef = useRef<HTMLDivElement>(null);
  const serviceListId = `appointment-services-${appointmentId}`;
```

```
  const canCollapse = services.length > 1;

  useEffect(() => {
    if (!canCollapse) {
      setIsTruncated(false);
      return;
    }

    if (isExpanded) return;

    const list = serviceListRef.current;
    if (!list) return;

    const updateTruncation = () => {
      setIsTruncated(list.scrollHeight > list.clientHeight + 1);
    };

    updateTruncation();
    const resizeObserver = new ResizeObserver(updateTruncation);
    resizeObserver.observe(list);

    return () => resizeObserver.disconnect();
  }, [canCollapse, isExpanded, services]);

  return (
    <div className="min-w-0 md:max-w-72">
      <div
        ref={serviceListRef}
        id={serviceListId}
        className={cn(
          "space-y-0.5 leading-4",
          canCollapse && !isExpanded && "max-h-[2.25rem] overflow-hidden",
        )}
      >
        {services.map((service) => (
          <span key={service.service_id ?? service.service_name}
classNameName="block">
            {service.service_name}
          </span>
        ))}
      </div>
      {canCollapse && isTruncated && (
        <Button
          type="button"
          variant="link"
```

```
          size="sm"
          className="mt-0.5 h-auto min-h-0 px-0 py-0 text-xs"
          aria-controls={serviceListId}
          aria-expanded={isExpanded}
          onClick={() => setIsExpanded((expanded) => !expanded)}
        >
          {isExpanded ? "See Less" : "See More"}
        </Button>
      )}
    </div>
  );
}

function appointmentEditHasChanges(
  appointment: AppointmentDetails,
  form: AppointmentEditForm,
): boolean {
  const originalServiceIds = appointment.appointment_services
    .map((service) => service.service_id)
    .filter((serviceId): serviceId is string => Boolean(serviceId));
  const productSignature = (products: AppointmentProductInput[]) =>
    products
      .map((product) =>
`${product.productId}:${product.quantity}:${product.unitPrice.toFixed(2)}`)
      .sort()
      .join(",");
  const originalProducts: AppointmentProductInput[] =
appointment.appointment_products.map(
    (product) => ({
      productId: product.product_id,
      quantity: product.quantity,
      unitPrice: Number(product.unit_price),
    }),
  );

  return (
    form.firstName.trim() !== (appointment.first_name ?? "").trim() ||
    form.middleName.trim() !== (appointment.middle_name ?? "").trim() ||
    form.lastName.trim() !== (appointment.last_name ?? "").trim() ||
    normalizePhilippineMobile(form.phone) !==
normalizePhilippineMobile(appointment.phone) ||
    [...form.serviceIds].sort().join(",") !== [...originalServiceIds].sort().join(",") ||
    form.appointmentDate !== appointment.appointment_date ||
    form.startTime !== String(appointment.start_time).slice(0, 5) ||
    form.assignedCrewId !== (appointment.assigned_crew_id ?? "none") ||
```

```
    form.status !== appointment.status ||
    form.adminNotes.trim() !== (appointment.admin_notes ?? "").trim() ||
    productSignature(form.products) !== productSignature(originalProducts)
  );
}

function appointmentScheduleHasChanged(
  appointment: AppointmentDetails,
  form: AppointmentEditForm,
): boolean {
  return (
    form.appointmentDate !== appointment.appointment_date ||
    form.startTime !== String(appointment.start_time).slice(0, 5)
  );
}

function AppointmentsPage() {
  const pageSize = 10;
  const qc = useQueryClient();
  const navigate = useNavigate();
  const availabilityFn = useServerFn(getAvailability);
  const updateWithConfirmation =
useServerFn(updateAppointmentWithConfirmation);
  const reviewBookingFn =
useServerFn(reviewPendingAppointmentWithConfirmation);
  const reviewRescheduleFn =
useServerFn(reviewRescheduleRequestWithConfirmation);
  const serviceProgressFn =
useServerFn(updateAppointmentServiceProgressWithNotifications);
  const search = Route.useSearch();
  const [status, setStatus] = useState<string>(search.status ?? "all");
  const [term, setTerm] = useState("");
  const [openId, setOpenId] = useState<string | null>(null);
  const [isEditing, setIsEditing] = useState(false);
  const [confirmationNotificationWarning, setConfirmationNotificationWarning] =
useState<
    string | null
  >(null);
  const [editForm, setEditForm] = useState<AppointmentEditForm | null>(null);
  const [editErrors, setEditErrors] = useState<AppointmentEditErrors>({});
  const [pendingSaveForm, setPendingSaveForm] =
useState<AppointmentEditForm | null>(null);
  const [datePeriod, setDatePeriod] = useState<AppointmentDatePeriod>("all");
  const [customDateRange, setCustomDateRange] = useState<DateRange>();
  const [page, setPage] = useState(0);
```

```
  const [archiveTarget, setArchiveTarget] = useState<string | null>(null);
  const [archiveError, setArchiveError] = useState<string | null>(null);
  const [quickAction, setQuickAction] = useState<{
    appointment: AppointmentDetails;
    step: NextStep;
  } | null>(null);
  const [quickActionError, setQuickActionError] = useState<string | null>(null);
  const focusedAppointmentId = search.appointmentId;
  const today = manilaNow().date;
  const deferredTerm = useDeferredValue(term);
  const dateRange = useMemo(
    () => buildAppointmentDateRange(datePeriod, customDateRange),
    [customDateRange, datePeriod],
  );
  const dateFilterKey = dateRange ? `${dateRange.from}:${dateRange.to}` : "all";

  const appointments = useQuery({
    queryKey: [
      "admin-appointments",
      { focusedAppointmentId, status, term: deferredTerm, dateFilterKey, page },
    ],
    queryFn: async () => {
      let query = supabase
        .from("appointments")
        .select(
          "*, appointment_services(service_id, service_name, price,
duration_minutes), appointment_products(id, product_id, product_name,
quantity, unit_price), crew_members(name)",
          {
            count: "exact",
          },
        )
        .eq("is_archived", false);

      if (focusedAppointmentId) {
        query = query.eq("id", focusedAppointmentId);
      } else {
        if (status === "needs_action") {
          // New bookings to approve and reschedule requests to review, as
          // counted on the dashboard.
          query = query
            .is("rescheduled_to_appointment_id", null)
            .or(
              "status.eq.pending,and(pending_reschedule_request_id.not.is.null,statu
s.in.(pending,confirmed))",
```

```
          );
        } else if (status !== "all") {
          query = query.eq("status", status);
        }
        if (dateRange) {
          query = query
            .gte("appointment_date", dateRange.from)
            .lte("appointment_date", dateRange.to);
        }
        if (deferredTerm.trim()) {
          const cleanedTerm = cleanSearchTerm(deferredTerm);
          const value = normalizePhilippineMobile(cleanedTerm) ?? cleanedTerm;
          query = query.or(
            `reference_code.ilike.%${value}%,customer_name.ilike.%${value}%,phon
e.ilike.%${value}%,plate_number.ilike.%${value}%`,
          );
        }
        query = query.range(page * pageSize, page * pageSize + pageSize - 1);
      }
      // Waiting requests are listed soonest first; everything else newest first.
      query = query
        .order("appointment_date", { ascending: status === "needs_action" })
        .order("start_time");

      const { data, error, count } = await query;
      if (error) throw error;
      return { rows: data ?? [], total: count ?? 0 };
    },
  });

  const selectedAppointment =
    ((appointments.data?.rows ?? []) as AppointmentDetails[]).find(
      (appointment) => appointment.id === openId,
    ) ?? null;

  const crew = useQuery({
    queryKey: ["crew"],
    queryFn: async () => {
      const { data } = await supabase
        .from("crew_members")
        .select("*")
        .eq("is_active", true)
        .eq("is_archived", false)
        .order("name");
      return Array.from(new Map((data ?? []).map((c) => [c.name.trim(),
```

```
c])).values());
    },
  });

  const services = useQuery({
    queryKey: ["admin-appointment-edit-services"],
    queryFn: async (): Promise<EditableService[]> => {
      const { data, error } = await supabase
        .from("services")
        .select("id, name, price, duration_minutes")
        .eq("is_active", true)
        .eq("is_archived", false)
        .order("sort_order");
      if (error) throw error;
      return data;
    },
  });

  const products = useQuery({
    queryKey: ["admin-appointment-products"],
    queryFn: async (): Promise<EditableProduct[]> => {
      const { data, error } = await supabase
        .from("products")
        .select("id,name,brand,price,stock_quantity,is_active,in_stock")
        .eq("is_archived", false)
        .order("name");
      if (error) throw error;
      return data;
    },
    enabled: isEditing,
  });

  const editServiceIds = editForm?.serviceIds ?? EMPTY_SERVICE_IDS;
  const appointmentAvailability = useQuery<Availability>({
    queryKey: [
      "admin-appointment-availability",
      {
        appointmentId: openId,
        serviceIds: editServiceIds,
        motorcycle: selectedAppointment
          ?
`${selectedAppointment.moto_brand}:${selectedAppointment.moto_model}`
          : null,
      },
    ],
```

```
    queryFn: () =>
      availabilityFn({
        data: {
          days: 45,
          serviceIds: editServiceIds,
          excludeAppointmentId: openId ?? undefined,
          allowInactiveMotorcycle: true,
          motorcycle: selectedAppointment
            ? { brand: selectedAppointment.moto_brand, model:
selectedAppointment.moto_model }
            : undefined,
        },
      }),
    enabled: isEditing && Boolean(openId) && editServiceIds.length > 0,
    staleTime: 5_000,
  });

  const availableAppointmentDates = useMemo(
    () => (appointmentAvailability.data ?
computeAvailableDates(appointmentAvailability.data) : []),
    [appointmentAvailability.data],
  );
  const availableAppointmentDateSet = useMemo(
    () => new Set(availableAppointmentDates),
    [availableAppointmentDates],
  );
  const editableAppointmentDate = editForm?.appointmentDate ?? "";
  const availableAppointmentSlots = useMemo(
    () =>
      appointmentAvailability.data && editableAppointmentDate
        ? computeAvailableSlots(appointmentAvailability.data,
editableAppointmentDate)
        : [],
    [appointmentAvailability.data, editableAppointmentDate],
  );
  const fullyBookedAppointmentDates = useMemo(
    () =>
      appointmentAvailability.data ?
computeFullyBookedDates(appointmentAvailability.data) : [],
    [appointmentAvailability.data],
  );
  const editableAppointmentDuration = useMemo(
    () =>
      appointmentAvailability.data?.totalDurationMinutes ??
      (editServiceIds.length > 0
```

```
        ? editServiceIds.reduce(
            (total, serviceId) =>
              total +
              (services.data?.find((service) => service.id ===
serviceId)?.duration_minutes ?? 60),
            0,
          )
        : 0),
    [appointmentAvailability.data?.totalDurationMinutes, editServiceIds,
services.data],
  );
  const assignableCrew = useQuery({
    queryKey: [
      "admin-appointment-assignable-crew",
      {
        appointmentId: openId,
        date: editableAppointmentDate,
        startTime: editForm?.startTime ?? "",
        duration: editableAppointmentDuration,
      },
    ],
    queryFn: async () => {
      let appointmentsQuery = supabase
        .from("appointments")
        .select("id, assigned_crew_id, start_time, booking_duration_minutes")
        .eq("appointment_date", editableAppointmentDate)
        .eq("is_archived", false)
        .is("rescheduled_to_appointment_id", null)
        .in("status", ACTIVE_RESERVATION_STATUSES);
      if (openId) appointmentsQuery = appointmentsQuery.neq("id", openId);
      const continuationsQuery = supabase
        .from("appointment_continuations")
        .select("appointment_id, assigned_crew_id, start_time,
booking_duration_minutes")
        .eq("appointment_date", editableAppointmentDate);

      const [schedules, exceptions, appointments, continuations] = await
Promise.all([
        supabase
          .from("crew_schedules")
          .select("crew_id, day_of_week, schedule_date, start_time, end_time,
is_working")
          .eq("schedule_date", editableAppointmentDate)
          .eq("is_working", true),
        supabase
```

```
        .from("crew_availability_exceptions")
        .select("crew_id, start_date, end_date, start_time, end_time, is_all_day")
        .lte("start_date", editableAppointmentDate)
        .gte("end_date", editableAppointmentDate),
      appointmentsQuery,
      continuationsQuery,
    ]);

    if (schedules.error || exceptions.error || appointments.error ||
continuations.error) {
      throw schedules.error ?? exceptions.error ?? appointments.error ??
continuations.error;
    }

    const continuationParentIds = Array.from(
      new Set((continuations.data ?? []).map((continuation) =>
continuation.appointment_id)),
    );
    const continuationParents = continuationParentIds.length
      ? await supabase
          .from("appointments")
          .select("id, is_archived, status, rescheduled_to_appointment_id")
          .in("id", continuationParentIds)
      : { data: [], error: null };
    if (continuationParents.error) throw continuationParents.error;

    const activeContinuationParentIds = new Set(
      (continuationParents.data ?? [])
        .filter(
          (appointment) =>
            !appointment.is_archived &&
            !appointment.rescheduled_to_appointment_id &&
            isActiveReservationStatus(appointment.status),
        )
        .map((appointment) => appointment.id),
    );
    const activeCrewIds = new Set((crew.data ?? []).map((member) =>
member.id));
    const { availableCrewIds } = evaluateCrewAvailability({
      date: editableAppointmentDate,
      startTime: editForm?.startTime ?? "",
      durationMinutes: editableAppointmentDuration,
      // The picker only determines whether an individual crew member can
      // work this range; the atomic database write remains the capacity gate.
      capacity: Number.MAX_SAFE_INTEGER,
```

```
        schedules: (schedules.data ?? []).filter((schedule) =>
activeCrewIds.has(schedule.crew_id)),
        exceptions: exceptions.data ?? [],
        assignments: [
          ...(appointments.data ?? []).map((appointment) => ({
            date: editableAppointmentDate,
            startTime: String(appointment.start_time).slice(0, 5),
            durationMinutes: appointment.booking_duration_minutes ?? 60,
            crewId: appointment.assigned_crew_id,
          })),
          ...(continuations.data ?? [])
            .filter((continuation) =>
activeContinuationParentIds.has(continuation.appointment_id))
            .map((continuation) => ({
              date: editableAppointmentDate,
              startTime: String(continuation.start_time).slice(0, 5),
              durationMinutes: continuation.booking_duration_minutes,
              crewId: continuation.assigned_crew_id,
            })),
        ],
      });
      const availableCrewIdSet = new Set(availableCrewIds);
      return (crew.data ?? []).filter((member) =>
availableCrewIdSet.has(member.id));
    },
    enabled:
      isEditing &&
      Boolean(openId) &&
      Boolean(editableAppointmentDate) &&
      Boolean(editForm?.startTime) &&
      editableAppointmentDuration > 0 &&
      !services.isLoading &&
      !crew.isLoading,
  });
  const assignableCrewIds = useMemo(
    () => new Set((assignableCrew.data ?? []).map((member) => member.id)),
    [assignableCrew.data],
  );

  const saveAppointment = useMutation({
    mutationFn: async ({ id, form }: { id: string; form: AppointmentEditForm }) => {
      const result = await updateWithConfirmation({
        data: {
          appointmentId: id,
          serviceIds: form.serviceIds,
```

```
          appointmentDate: form.appointmentDate,
          startTime: form.startTime,
          assignedCrewId: form.assignedCrewId === "none" ? null :
form.assignedCrewId,
          crewAssignmentManual: form.crewAssignmentManual,
          status: form.status,
          adminNotes: form.adminNotes.trim() || null,
          firstName: form.firstName.trim(),
          middleName: form.middleName.trim(),
          lastName: form.lastName.trim(),
          phone: normalizePhilippineMobile(form.phone)!,
          products: form.products.map((product) => ({
            productId: product.productId,
            quantity: product.quantity,
            unitPrice: product.unitPrice,
          })),
        },
      });
      if (!result.ok) throw new Error(result.error);
      return result;
    },
    onSuccess: (result) => {
      setIsEditing(false);
      setEditErrors({});
      setPendingSaveForm(null);
      const warnings = [
        "emailError" in result ? result.emailError : null,
        "smsError" in result ? result.smsError : null,
      ].filter((warning): warning is string => Boolean(warning));
      setConfirmationNotificationWarning(warnings.join(" ") || null);
      qc.invalidateQueries({ queryKey: ["admin-appointments"], exact: false });
      qc.invalidateQueries({ queryKey: ["admin-dashboard"], exact: false });
      qc.invalidateQueries({ queryKey: ["archived-appointments"], exact: false });
      qc.invalidateQueries({ queryKey: ["notifications"] });
      qc.invalidateQueries({ queryKey: ["unread-notifications"] });
    },
    onError: (err: Error) => {
      setEditErrors((current) => ({
        ...current,
        ...errorsForAppointmentUpdate(err),
      }));
    },
  });

  const archive = useMutation({
```

```
    mutationFn: async (id: string) => {
      const { error } = await supabase
        .from("appointments")
        .update({ is_archived: true } as never)
        .eq("id", id);
      if (error) throw error;
    },
    onSuccess: () => {
      setArchiveError(null);
      qc.invalidateQueries({ queryKey: ["admin-appointments"], exact: false });
      qc.invalidateQueries({ queryKey: ["admin-dashboard"], exact: false });
      qc.invalidateQueries({ queryKey: ["archived-appointments"], exact: false });
    },
    onError: (err: Error) => {
      console.error("Archive failed:", err);
      setArchiveError(
        // The database refuses completed appointments, e.g. one completed in
another tab.
        err.message.includes("Completed appointments")
          ? COMPLETED_APPOINTMENT_KEPT
          : "Could not archive this appointment. Please try again.",
      );
    },
  });

  // The same server actions the Notifications page uses, so customers get the
  // same confirmation and completion messages.
  const runNextStep = useMutation({
    mutationFn: async ({
      appointmentId,
      step,
      decision,
    }: {
      appointmentId: string;
      step: NextStep;
      decision: "confirmed" | "rejected";
    }) => {
      const result =
        step === "review-booking"
          ? await reviewBookingFn({ data: { appointmentId, decision } })
          : step === "review-reschedule"
            ? await reviewRescheduleFn({ data: { appointmentId, decision } })
            : await serviceProgressFn({
                data: { appointmentId, action: step === "start" ? "start" : "complete" },
              });
```

```
      if (!result.ok) throw new Error(result.error);
      return result;
    },
    onSuccess: (result) => {
      const warnings = [
        "emailError" in result ? result.emailError : null,
        "smsError" in result ? result.smsError : null,
      ].filter((warning): warning is string => Boolean(warning));
      setConfirmationNotificationWarning(warnings.join(" ") || null);
      setQuickAction(null);
      setQuickActionError(null);
      qc.invalidateQueries({ queryKey: ["admin-appointments"], exact: false });
      qc.invalidateQueries({ queryKey: ["admin-dashboard"], exact: false });
      qc.invalidateQueries({ queryKey: ["reports"], exact: false });
      qc.invalidateQueries({ queryKey: ["notifications"] });
      qc.invalidateQueries({ queryKey: ["unread-notifications"] });
    },
    onError: (err: Error) => {
      setQuickActionError(err.message || "Could not update this appointment.
Please try again.");
    },
  });

  function openDetails(appointment: AppointmentDetails, edit = false) {
    setOpenId(appointment.id);
    setEditErrors({});
    if (edit) {
      startEditing(appointment);
    } else {
      setIsEditing(false);
      setEditForm(null);
    }
  }

  function openNextStep(appointment: AppointmentDetails, step: NextStep) {
    setQuickActionError(null);
    setQuickAction({ appointment, step });
  }

  useEffect(() => {
    setOpenId(focusedAppointmentId ?? null);
    setPage(0);
  }, [focusedAppointmentId]);

  useEffect(() => {
```

```
    setPage(0);
  }, [dateFilterKey, status, term]);

  const rows = appointments.data?.rows ?? [];
  const typedRows = rows as AppointmentDetails[];

  const rescheduleHistory = useQuery({
    queryKey: ["appointment-reschedule-history", openId],
    queryFn: async () => {
      if (!openId) return [];
      const { data, error } = await supabase
        .from("appointment_reschedule_history")
        .select(
          "id, from_date, from_start_time, to_date, to_start_time, reason, decision,
reschedule_number, approved_at",
        )
        .eq("appointment_id", openId)
        .order("reschedule_number", { ascending: true });
      if (error) throw error;
      return (data ?? []) as RescheduleHistoryEntry[];
    },
    enabled: Boolean(openId),
    staleTime: 5_000,
  });
  const hasEditChanges = Boolean(
    selectedAppointment && editForm &&
appointmentEditHasChanges(selectedAppointment, editForm),
  );
  const scheduleHasChanged = Boolean(
    selectedAppointment && editForm &&
appointmentScheduleHasChanged(selectedAppointment, editForm),
  );
  const activeFilters = [
    ...(term.trim() ? [{ label: "Search", value: term.trim(), onClear: () => setTerm("")
}] : []),
    ...(status !== "all"
      ? [{ label: "Status", value: statusLabel(status), onClear: () => setStatus("all") }]
      : []),
    ...(dateRange
      ? [
          {
            label: "Date",
            value: appointmentDatePeriodLabel(datePeriod, dateRange),
            onClear: () => {
              setDatePeriod("all");
```

```
            setCustomDateRange(undefined);
          },
        },
      ]
    : []),
];

const appointmentTotal = appointments.data?.total ?? 0;
const appointmentCountLabel = `${appointmentTotal} ${
  activeFilters.length > 0 ? "matching " : ""
}${appointmentTotal === 1 ? "appointment" : "appointments"}`;

function resetFilters() {
  setTerm("");
  setStatus("all");
  setDatePeriod("all");
  setCustomDateRange(undefined);
  setPage(0);
}

function closeAppointmentDialog() {
  setOpenId(null);
  setIsEditing(false);
  setEditForm(null);
  setEditErrors({});
  setPendingSaveForm(null);
}

function startEditing(appointment: AppointmentDetails) {
  setConfirmationNotificationWarning(null);
  setEditForm({
    firstName: appointment.first_name ?? "",
    middleName: appointment.middle_name ?? "",
    lastName: appointment.last_name ?? "",
    phone: toLocalPhilippineMobile(appointment.phone),
    serviceIds: appointment.appointment_services
      .map((service) => service.service_id)
      .filter((serviceId): serviceId is string => Boolean(serviceId)),
    appointmentDate: appointment.appointment_date,
    startTime: String(appointment.start_time).slice(0, 5),
    assignedCrewId: appointment.assigned_crew_id ?? "none",
    crewAssignmentManual: false,
    status: appointment.status,
    adminNotes: appointment.admin_notes ?? "",
    products: appointment.appointment_products.map((product) => ({
```

```
      productId: product.product_id,
      quantity: product.quantity,
      unitPrice: Number(product.unit_price),
    })),
  });
  setEditErrors({});
  setPendingSaveForm(null);
  setIsEditing(true);
}

function toggleService(serviceId: string, selected: boolean) {
  setEditForm((current) =>
    current
      ? {
          ...current,
          serviceIds: selected
            ? [...current.serviceIds, serviceId]
            : current.serviceIds.filter((id) => id !== serviceId),
          assignedCrewId: "none",
          crewAssignmentManual: false,
        }
      : current,
  );
  clearAppointmentEditErrors("services", "schedule", "form");
}

function addProduct() {
  setEditForm((current) =>
    current && current.products.length < 50
      ? {
          ...current,
          products: [...current.products, { productId: "", quantity: 1, unitPrice: 0 }],
        }
      : current,
  );
  clearAppointmentEditErrors("products", "form");
}

function updateProduct(index: number, patch:
Partial<AppointmentProductInput>) {
  setEditForm((current) =>
    current
      ? {
          ...current,
          products: current.products.map((product, productIndex) =>
```

```
            productIndex === index ? { ...product, ...patch } : product,
          ),
        }
      : current,
  );
  clearAppointmentEditErrors("products", "form");
}

function removeProduct(index: number) {
  setEditForm((current) =>
    current
      ? {
          ...current,
          products: current.products.filter((_, productIndex) => productIndex !==
index),
        }
      : current,
  );
  clearAppointmentEditErrors("products", "form");
}

function clearAppointmentEditErrors(...keys: Array<keyof
AppointmentEditErrors>) {
  setEditErrors((current) => {
    const next = { ...current };
    keys.forEach((key) => delete next[key]);
    return next;
  });
}

function handleSaveAppointment() {
  if (!selectedAppointment || !editForm) return;

  if (!hasEditChanges) {
    setEditErrors({ form: "No changes detected. Update at least one appointment
detail first." });
    return;
  }

  const nextErrors: AppointmentEditErrors = {};
  if (editForm.firstName.trim().length < 2) {
    nextErrors.firstName = "Enter a valid first name.";
  }
  if (editForm.lastName.trim().length < 2) {
    nextErrors.lastName = "Enter a valid last name.";
```

```
    }
    if (!normalizePhilippineMobile(editForm.phone)) {
      nextErrors.phone = PHONE_VALIDATION_MESSAGE;
    }
    if (editForm.serviceIds.length === 0) {
      nextErrors.services = "Select at least one service.";
    }
    if (editForm.products.length > 50) {
      nextErrors.products = "Add up to 50 products to an appointment.";
    } else if (
      editForm.products.some(
        (product) =>
          !product.productId ||
          !Number.isInteger(product.quantity) ||
          product.quantity < 1 ||
          product.quantity > 999 ||
          !Number.isFinite(product.unitPrice) ||
          product.unitPrice < 0,
      )
    ) {
      nextErrors.products = "Choose each product and enter a valid quantity and
price.";
    } else if (
      new Set(editForm.products.map((product) => product.productId)).size !==
      editForm.products.length
    ) {
      nextErrors.products = "Each product can only be added once.";
    } else if (editForm.products.length > 0 &&
!selectedAppointment.wants_products) {
      nextErrors.products = "This customer selected No for shop products.";
    } else if (editForm.products.length > 0 &&
!canRecordAppointmentProducts(editForm.status)) {
      nextErrors.products =
        "Products can be recorded after the appointment is confirmed, before or
during service.";
    }
    if (!editForm.appointmentDate) {
      nextErrors.appointmentDate = "Select an appointment date.";
    }
    if (!editForm.startTime) {
      nextErrors.startTime = "Select an appointment time.";
    }
    const statusChanged = editForm.status !== selectedAppointment.status;
    if (
      statusChanged &&
```

```
      editForm.status === "completed" &&
      !selectedAppointment.service_started_at
    ) {
      nextErrors.status = "Start the service before marking the appointment
completed.";
    }
    if (
      statusChanged &&
      editForm.status === "no_show" &&
      (selectedAppointment.service_started_at ||
        selectedAppointment.arrival_notification_snooze_count < 3)
    ) {
      nextErrors.status = "Mark an appointment as no-show only after three arrival
snoozes.";
    }

    if (scheduleHasChanged) {
      if (appointmentAvailability.isLoading) {
        nextErrors.schedule = "Available schedules are still loading. Please wait a
moment.";
      } else if (appointmentAvailability.isError || appointmentAvailability.data?.error)
{
        nextErrors.schedule =
          appointmentAvailability.data?.error ??
          "We could not verify available schedules. Please try again.";
      } else if (!availableAppointmentDateSet.has(editForm.appointmentDate)) {
        nextErrors.appointmentDate = "Choose an available appointment date.";
      } else if (
        !availableAppointmentSlots.some(
          (slot) => slot.startTime === editForm.startTime && !slot.disabled,
        )
      ) {
        nextErrors.startTime = "Choose an available appointment time.";
      }
    }

    const crewSelectionChanged =
      editForm.assignedCrewId !== (selectedAppointment.assigned_crew_id ??
"none");
    const confirmingAppointment =
      editForm.status === "confirmed" && editForm.status !==
selectedAppointment.status;
    if (scheduleHasChanged || crewSelectionChanged || confirmingAppointment) {
      if (editForm.assignedCrewId !== "none") {
        if (assignableCrew.isLoading) {
```

```
        nextErrors.assignedCrew = "Checking crew availability. Please wait a
moment.";
      } else if (assignableCrew.isError ||
!assignableCrewIds.has(editForm.assignedCrewId)) {
        nextErrors.assignedCrew =
          "Choose a crew member who is free for the full appointment period.";
      }
    } else if (editForm.status === "confirmed" &&
!editForm.crewAssignmentManual) {
      if (assignableCrew.isLoading) {
        nextErrors.assignedCrew = "Checking crew availability. Please wait a
moment.";
      } else if (assignableCrew.isError || assignableCrewIds.size === 0) {
        nextErrors.assignedCrew = "No crew member is available for this
appointment period.";
      }
    }
  }

  if (Object.keys(nextErrors).length > 0) {
    setEditErrors(nextErrors);
    return;
  }

  setPendingSaveForm({ ...editForm, serviceIds: [...editForm.serviceIds] });
 }

 function confirmSaveAppointment() {
  if (!selectedAppointment || !pendingSaveForm) return;
  const form = pendingSaveForm;
  setPendingSaveForm(null);
  saveAppointment.mutate({ id: selectedAppointment.id, form });
 }

 return (
  <AdminPage>
    <PageHeader
      title="Appointments Management"
      description={
        focusedAppointmentId
          ? "Showing the appointment selected from Notifications."
          : "Confirm, reschedule, assign crew and archive bookings."
      }
    />
```

```
      {archiveError && <InlineAlert>{archiveError}</InlineAlert>}
      {confirmationNotificationWarning && (
        <InlineAlert>{confirmationNotificationWarning}</InlineAlert>
      )}

      {focusedAppointmentId ? (
        <div className="mb-4 flex flex-wrap items-center justify-between gap-3
rounded-lg border border-primary/30 bg-primary/5 px-4 py-3 text-sm">
          <span>Showing one appointment opened from Notifications.</span>
          <Button
            size="sm"
            variant="outline"
            onClick={() => navigate({ to: "/admin/appointments", search: {} })}
          >
            Show all appointments
          </Button>
        </div>
      ) : (
        <ListToolbar summary={appointments.data && appointmentCountLabel}>
          <ToolbarSearch
            value={term}
            onChange={(value) => {
              setTerm(value);
              setPage(0);
            }}
            placeholder="Search reference, name, phone, or plate"
            label="Search appointments by reference, customer name, phone, or
plate number"
          />
          <ToolbarFilters>
            <Select
              value={status}
              onValueChange={(value) => {
                setStatus(value);
                setPage(0);
              }}
            >
              <SelectTrigger aria-label="Filter by status" className="w-full
sm:w-44">
                <SelectValue />
              </SelectTrigger>
              <SelectContent>
                <SelectItem value="all">All statuses</SelectItem>
                <SelectItem value="needs_action">Needs action</SelectItem>
                {APPOINTMENT_STATUSES.map((s) => (
```

```
            <SelectItem key={s} value={s}>
              {statusLabel(s)}
            </SelectItem>
          ))}
        </SelectContent>
      </Select>
      <AppointmentDateFilter
        period={datePeriod}
        customDateRange={customDateRange}
        onPeriodChange={(period) => {
          setDatePeriod(period);
          if (period === "all") setCustomDateRange(undefined);
        }}
        onCustomDateRangeChange={setCustomDateRange}
      />
      {activeFilters.length > 0 && <ResetFiltersButton onClick={resetFilters} />}
    </ToolbarFilters>
  </ListToolbar>
)}

<ListCard
  footer={
    !focusedAppointmentId && (
      <PaginationControls
        page={page}
        pageSize={pageSize}
        total={appointments.data?.total ?? 0}
        onPageChange={setPage}
      />
    )
  }
>
  <DataList
    label="Appointments"
    rows={typedRows}
    rowKey={(a) => a.id}
    loading={appointments.isLoading}
    error={
      appointments.isError
        ? {
            message: "Could not load appointments.",
            onRetry: () => void appointments.refetch(),
          }
        : null
    }
```

```
        empty={
          <ListEmpty
            icon={CalendarDays}
            title={
              focusedAppointmentId
                ? "This appointment is no longer available."
                : activeFilters.length > 0
                  ? "No appointments match your filters."
                  : "No appointments yet."
            }
            description={
              focusedAppointmentId
                ? "It may have been archived or deleted."
                : activeFilters.length > 0
                  ? "Try a different search, status, or date."
                  : "Bookings from the website will appear here."
            }
            action={
              activeFilters.length > 0 && !focusedAppointmentId ? (
                <ResetFiltersButton onClick={resetFilters} />
              ) : undefined
            }
          />
        }
        columns={[
          {
            key: "customer",
            header: "Customer",
            card: "title",
            cell: (a) => (
              <>
                <button
                  type="button"
                  onClick={() => openDetails(a)}
                  className="text-left font-medium underline-offset-4
hover:text-primary hover:underline focus-visible:rounded-sm focus-visible:ring-2
focus-visible:ring-ring focus-visible:outline-none"
                >
                  {a.customer_name}
                </button>
                <p className="mt-0.5 text-xs text-muted-foreground">
                  <span className="font-mono">{a.reference_code}</span> ·{" "}
                  {toLocalPhilippineMobile(a.phone)}
                </p>
                {/* Services get their own table column from xl. */}
```

```
              <p className="mt-1 hidden text-xs text-muted-foreground lg:block
xl:hidden">
                {a.appointment_services.map((service) =>
service.service_name).join(", ")}
              </p>
            </>
          ),
        },
        {
          key: "schedule",
          header: "Schedule",
          className: "w-40",
          cell: (a) => (
            <span className="text-sm">
              {formatDateLong(a.appointment_date)}
              <span className="block text-xs text-muted-foreground">
                {formatTime(String(a.start_time).slice(0, 5))}
              </span>
            </span>
          ),
        },
        {
          key: "services",
          header: "Services",
          card: "wide",
          xlOnly: true,
          className: "w-52",
          cell: (a) => (
            <span className="text-sm">
              <AppointmentServicesList appointmentId={a.id}
services={a.appointment_services} />
            </span>
          ),
        },
        {
          key: "booked",
          header: "Booked on",
          // Least important column: the table shows it only on very wide screens.
          className: "hidden w-32 2xl:table-cell",
          cell: (a) => {
            const [day, time] = formatBookedOn(a.created_at).split(" - ");
            return (
              <span className="text-sm text-muted-foreground">
                {day}
                <span className="block text-xs">{time}</span>
```

```
              </span>
            );
          },
        },
        {
          key: "status",
          header: "Status",
          card: "badge",
          className: "w-28 text-center",
          cell: (a) => (
            <span className="inline-flex flex-col items-center gap-1">
              <Badge variant="outline" className={cn("uppercase",
statusTone(a.status))}>
                {statusLabel(a.status)}
              </Badge>
              {a.pending_reschedule_request_id && (
                <span className="text-[11px] leading-tight font-medium
text-accent">
                  Reschedule requested
                </span>
              )}
            </span>
          ),
        },
      ]}
      actionsClassName="w-52"
      actions={(a) => {
        const step = nextStepFor(a, today);
        const StepIcon = step ? NEXT_STEPS[step].icon : Eye;
        return (
          <>
            {step ? (
              <Button
                size="sm"
                onClick={() => openNextStep(a, step)}
                aria-label={`${NEXT_STEPS[step].label} for ${a.reference_code}`}
              >
                <StepIcon /> {NEXT_STEPS[step].label}
              </Button>
            ) : (
              <Button
                size="sm"
                variant="outline"
                onClick={() => openDetails(a)}
                aria-label={`View appointment ${a.reference_code}`}
```

```
              >
                <StepIcon /> View
              </Button>
            )}
            <MoreActionsMenu itemLabel={`appointment ${a.reference_code}`}>
              <DropdownMenuItem onSelect={() => openDetails(a)}>
                <Eye /> View details
              </DropdownMenuItem>
              <DropdownMenuItem onSelect={() => openDetails(a, true)}>
                <Pencil /> Edit appointment
              </DropdownMenuItem>
              <DropdownMenuSeparator />
              <ArchiveMenuItem
                onSelect={() => setArchiveTarget(a.id)}
                unavailableReason={
                  a.status === "completed" ? COMPLETED_APPOINTMENT_KEPT
: undefined
                }
              />
            </MoreActionsMenu>
          </>
        );
      }}
    />
  </ListCard>

  <Dialog
    open={Boolean(selectedAppointment)}
    onOpenChange={(open) => {
      if (!open) closeAppointmentDialog();
    }}
  >
    {selectedAppointment && (
      <DialogContent className="max-w-4xl">
        <DialogHeader>
          <DialogTitle className="font-display uppercase">Appointment
details</DialogTitle>
          <DialogDescription>
            Reference{" "}
            <span className="font-mono text-foreground">
              {selectedAppointment.reference_code}
            </span>
          </DialogDescription>
        </DialogHeader>
```

```
          {selectedAppointment.reschedule_count >= 3 && (
            <p className="rounded-lg border border-destructive/40
bg-destructive/5 p-3 text-sm text-muted-foreground">
              <strong className="text-destructive">Maximum reschedule limit
reached.</strong> This
              appointment has already been rescheduled 3 times. Its schedule can
no longer be
              changed.
            </p>
          )}

          {selectedAppointment.no_show_count_at_booking > 2 && (
            <p
              className="rounded-lg border border-amber-500/40 bg-amber-500/5
p-3 text-sm text-muted-foreground"
              role="alert"
            >
              <strong className="text-amber-700 dark:text-amber-400">
                Customer no-show warning.
              </strong>{" "}
              This customer had {selectedAppointment.no_show_count_at_booking}
recorded no-show
              {selectedAppointment.no_show_count_at_booking === 1 ? "" : "s"}
when this booking
              was made. Review the customer&amp;apos;s booking history before
confirming service.
            </p>
          )}

          {isEditing && editForm ? (
            <div className="space-y-5">
              <AppointmentReadOnlyDetails appointment={selectedAppointment}
hideName />

              <section className="space-y-3">
                <div>
                  <h3 className="font-medium">Edit appointment</h3>
                  <p className="text-sm text-muted-foreground">
                    Update the service, schedule, crew assignment, status, and
internal shop
                    notes.
                  </p>
                </div>
                <div className="grid gap-4 sm:grid-cols-2 xl:grid-cols-4">
                  {(
```

```
  [
    ["First Name", "firstName"],
    ["Middle Name (Optional)", "middleName"],
    ["Last Name", "lastName"],
  ] as const
).map(([label, field]) => (
  <div key={field} className="space-y-1.5">
    <Label htmlFor={`appointment-${field}`}>{label}</Label>
    <Input
      id={`appointment-${field}`}
      value={editForm[field]}
      maxLength={40}
      onChange={(event) => {
        setEditForm({
          ...editForm,
          [field]: formatNamePartInput(event.target.value),
        });
        clearAppointmentEditErrors(field, "form");
      }}
      aria-invalid={Boolean(editErrors[field])}
    />
    <FieldError message={editErrors[field]} />
  </div>
))}
<div className="space-y-1.5">
  <Label htmlFor="appointment-phone">Mobile number</Label>
  <Input
    id="appointment-phone"
    type="tel"
    value={editForm.phone}
    maxLength={11}
    inputMode="tel"
    autoComplete="tel"
    onChange={(event) => {
      setEditForm({
        ...editForm,
        phone: sanitizePhilippineMobileInput(event.target.value),
      });
      clearAppointmentEditErrors("phone", "form");
    }}
    placeholder="09171234567"
    aria-invalid={Boolean(editErrors.phone)}
  />
  <FieldError message={editErrors.phone} />
</div>
```

```
              </div>
              <div className="space-y-2">
                <Label>Services</Label>
                <div className="grid gap-2 sm:grid-cols-2">
                  {services.data?.map((service) => {
                    const selected = editForm.serviceIds.includes(service.id);
                    return (
                      <label
                        key={service.id}
                        className={cn(
                          "flex cursor-pointer items-start gap-3 rounded-md border p-3
text-sm transition-colors",
                          selected && "border-primary bg-primary/5",
                        )}
                      >
                        <Checkbox
                          checked={selected}
                          onCheckedChange={(checked) =>
                            toggleService(service.id, checked === true)
                          }
                          aria-label={`Select ${service.name}`}
                        />
                        <span className="min-w-0 flex-1">
                          <span className="block
font-medium">{service.name}</span>
                          <span className="text-xs text-muted-foreground">
                            {service.duration_minutes} mins · {formatPHP(service.price)}
                          </span>
                        </span>
                      </label>
                    );
                  })}
                </div>
                {services.isLoading && (
                  <p className="text-sm text-muted-foreground">Loading available
services…</p>
                )}
                <FieldError message={editErrors.services} />
              </div>

              <div className="space-y-3 rounded-lg border border-border/70
p-4">
                <div className="flex flex-wrap items-start justify-between gap-3">
                  <div>
                    <h4 className="font-medium">Products for this
```

```
appointment</h4>
                  <p className="mt-1 text-sm text-muted-foreground">
                    Customer preference: {selectedAppointment.wants_products ?
"Yes" : "No"}.
                    Product quantities and selling prices are saved as appointment
records.
                  </p>
                </div>
                <Button
                  type="button"
                  variant="outline"
                  size="sm"
                  disabled={
                    !selectedAppointment.wants_products ||
                    !canRecordAppointmentProducts(editForm.status) ||
                    editForm.products.length >= 50
                  }
                  onClick={addProduct}
                >
                  Add product
                </Button>
              </div>
              {!selectedAppointment.wants_products && (
                <p className="text-sm text-muted-foreground">
                  Products cannot be added because the customer selected No
during booking.
                </p>
              )}
              {selectedAppointment.wants_products &&
                !canRecordAppointmentProducts(editForm.status) && (
                  <p className="text-sm text-muted-foreground">
                    Confirm the appointment to add products before service begins.
Products
                    can also be adjusted while service is in progress or completed.
                  </p>
                )}
              <div className="space-y-3">
                {editForm.products.map((product, index) => {
                  const selectedProduct = (products.data ?? []).find(
                    (item) => item.id === product.productId,
                  );
                  return (
                    <div
                      key={`${product.productId || "new"}-${index}`}
                      className="grid gap-3 rounded-md border border-border/70
```

```
p-3 md:grid-cols-[minmax(0,1fr)_7rem_9rem_auto] md:items-end"
                        >
                          <div className="space-y-1.5">
                            <Label>Product</Label>
                            <Select
                              {...(product.productId ? { value: product.productId } : {})}
                              onValueChange={(productId) => {
                                const item = (products.data ?? []).find(
                                  (entry) => entry.id === productId,
                                );
                                updateProduct(index, {
                                  productId,
                                  unitPrice: item ? Number(item.price) : product.unitPrice,
                                });
                              }}
                            >
                              <SelectTrigger>
                                <SelectValue placeholder="Choose product" />
                              </SelectTrigger>
                              <SelectContent>
                                {(products.data ?? []).map((item) => (
                                  <SelectItem
                                    key={item.id}
                                    value={item.id}
                                    disabled={
                                      (!item.is_active || !item.in_stock) &&
                                      item.id !== product.productId
                                    }
                                  >
                                    {item.brand ? `${item.brand} — ` : ""}
                                    {item.name} ({item.stock_quantity} in stock)
                                  </SelectItem>
                                ))}
                              </SelectContent>
                            </Select>
                          </div>
                          <div className="space-y-1.5">
                            <Label>Quantity</Label>
                            <Input
                              type="number"
                              min={1}
                              max={999}
                              value={product.quantity}
                              onChange={(event) =>
                                updateProduct(index, {
```

```
                        quantity: Number.parseInt(event.target.value, 10) || 0,
                      })
                    }
                  />
                </div>
                <div className="space-y-1.5">
                  <Label>Unit price</Label>
                  <Input
                    type="number"
                    min={0}
                    step="0.01"
                    value={product.unitPrice}
                    onChange={(event) =>
                      updateProduct(index, { unitPrice:
Number(event.target.value) })
                    }
                  />
                </div>
                <Button
                  type="button"
                  variant="outline"
                  size="sm"
                  onClick={() => removeProduct(index)}
                >
                  Remove
                </Button>
                {selectedProduct && !selectedProduct.is_active && (
                  <p className="md:col-span-4 text-xs
text-muted-foreground">
                    This product is inactive and can only remain as an existing
                    appointment record.
                  </p>
                )}
              </div>
            );
          })}
        </div>
        {products.isLoading && (
          <p className="text-sm text-muted-foreground">Loading shop
products…</p>
        )}
        {products.isError && (
          <p className="text-sm text-destructive">
            Could not load shop products. Please try again.
          </p>
```

```
                )}
                {editForm.products.length === 0 &&
selectedAppointment.wants_products && (
                  <p className="text-sm text-muted-foreground">
                    No products have been recorded.
                  </p>
                )}
                <FieldError message={editErrors.products} />
              </div>

              <div className="grid gap-4 lg:grid-cols-[auto_minmax(0,1fr)]">
                <div className="space-y-1.5">
                  <Label>Appointment date</Label>
                  {appointmentAvailability.isLoading ? (
                    <div className="rounded-lg border border-border p-4 text-sm
text-muted-foreground">
                      Loading available dates…
                    </div>
                  ) : appointmentAvailability.isError ||
appointmentAvailability.data?.error ? (
                    <div className="rounded-lg border border-destructive/60 p-4
text-sm text-destructive">
                      {appointmentAvailability.data?.error ??
                        "We could not load available schedules. Please try again."}
                    </div>
                  ) : (
                    <div
                      className={cn(
                        "rounded-lg border border-border bg-card/50 p-2",
                        editErrors.appointmentDate && "border-destructive",
                      )}
                    >
                      <Calendar
                        mode="single"
                        selected={
                          editForm.appointmentDate
                            ? parseISO(editForm.appointmentDate)
                            : undefined
                        }
                        onSelect={(selectedDate) => {
                          if (!selectedDate) return;
                          const nextDate = calendarDateToIso(selectedDate);
                          if (nextDate === editForm.appointmentDate) return;
                          setEditForm({
                            ...editForm,
```

```
                      appointmentDate: nextDate,
                      startTime: "",
                      assignedCrewId: "none",
                      crewAssignmentManual: false,
                    });
                    clearAppointmentEditErrors(
                      "appointmentDate",
                      "startTime",
                      "schedule",
                      "form",
                    );
                  }}
                  disabled={(date) =>
                    !availableAppointmentDateSet.has(calendarDateToIso(date))
                  }
                  modifiers={{
                    fullyBooked: fullyBookedAppointmentDates.map((value) =>
                      parseISO(value),
                    ),
                  }}
                  modifiersClassNames={{
                    fullyBooked:
                      "bg-destructive/15 text-destructive line-through opacity-100",
                  }}
                  classNames={{
                    nav: "justify-between gap-1",
                    month_caption:
                      "flex h-(--cell-size) w-full items-center justify-center
px-(--cell-size)",
                  }}
                />
              </div>
            )}
            <FieldError message={editErrors.appointmentDate} />
          </div>
          <div className="space-y-1.5">
            <Label>Appointment time</Label>
            {editableAppointmentDate ? (
              <div
                className={cn(
                  "grid grid-cols-2 gap-2 sm:grid-cols-3",
                  editErrors.startTime && "rounded-lg ring-1 ring-destructive",
                )}
              >
                {availableAppointmentSlots.map((slot) => (
```

```
                    <button
                      type="button"
                      key={slot.id}
                      disabled={slot.disabled}
                      onClick={() => {
                        if (slot.startTime === editForm.startTime) return;
                        setEditForm({
                          ...editForm,
                          startTime: slot.startTime,
                          assignedCrewId: "none",
                          crewAssignmentManual: false,
                        });
                        clearAppointmentEditErrors(
                          "startTime",
                          "schedule",
                          "assignedCrew",
                          "form",
                        );
                      }}
                      className={cn(
                        "rounded-lg border p-3 text-center transition-colors",
                        slot.disabled && "cursor-not-allowed opacity-40",
                        editForm.startTime === slot.startTime
                          ? "border-primary bg-primary/15"
                          : "border-border bg-card/50 hover:border-primary/50",
                      )}
                    >
                      <span className="font-display block">
                        {formatTime(slot.startTime)}
                      </span>
                      <span className="block text-[11px] text-muted-foreground">
                        {slot.disabled ? "Unavailable" : "Available"}
                      </span>
                    </button>
                  ))}
                  {availableAppointmentSlots.length === 0 &&
                    !appointmentAvailability.isLoading && (
                      <p className="col-span-full rounded-lg border
border-dashed border-border p-4 text-sm text-muted-foreground">
                        No available times for this date. Please choose another date.
                      </p>
                    )}
                </div>
              ) : (
                <div className="rounded-lg border border-dashed border-border
```

```
p-4 text-sm text-muted-foreground">
                    Choose an available date to view available time slots.
                  </div>
                )}
                <FieldError message={editErrors.startTime} />
              </div>
            </div>
            {fullyBookedAppointmentDates.length > 0 &&
              !appointmentAvailability.isLoading &&
              !appointmentAvailability.isError && (
                <p className="flex items-center gap-2 text-xs
text-muted-foreground">
                  <span className="h-3 w-3 rounded-sm bg-destructive/15 ring-1
ring-destructive/40" />
                  Fully booked dates cannot be selected.
                </p>
              )}
            <FieldError message={editErrors.schedule} />

            <div className="grid gap-4 sm:grid-cols-2">
              <div className="space-y-1.5">
                <Label>Assigned crew</Label>
                <Select
                  value={editForm.assignedCrewId}
                  disabled={assignableCrew.isLoading || assignableCrew.isError}
                  onValueChange={(assignedCrewId) => {
                    setEditForm({
                      ...editForm,
                      assignedCrewId,
                      crewAssignmentManual: true,
                    });
                    clearAppointmentEditErrors("assignedCrew", "form");
                  }}
                >
                  <SelectTrigger aria-invalid={Boolean(editErrors.assignedCrew)}>
                    <SelectValue
                      placeholder={
                        assignableCrew.isLoading ? "Checking availability…" :
"Unassigned"
                      }
                    />
                  </SelectTrigger>
                  <SelectContent>
                    <SelectItem value="none">Unassigned</SelectItem>
                    {assignableCrew.data?.map((member) => (
```

```
                      <SelectItem key={member.id} value={member.id}>
                        {member.name}
                      </SelectItem>
                    ))}
                  </SelectContent>
                </Select>
                {editForm.status === "confirmed" &&
                  editForm.assignedCrewId === "none" &&
                  !editForm.crewAssignmentManual && (
                    <p className="text-xs text-muted-foreground">
                      A free crew member will be assigned automatically when you
save.
                    </p>
                  )}
                  <FieldError message={editErrors.assignedCrew} />
                </div>
                <div className="space-y-1.5">
                  <Label>Status</Label>
                  <Select
                    value={editForm.status}
                    onValueChange={(status) => {
                      setEditForm({ ...editForm, status });
                      clearAppointmentEditErrors("status", "form");
                    }}
                  >
                    <SelectTrigger aria-invalid={Boolean(editErrors.status)}>
                      <SelectValue />
                    </SelectTrigger>
                    <SelectContent>
                      {APPOINTMENT_STATUSES.map((appointmentStatus) => (
                        <SelectItem key={appointmentStatus}
value={appointmentStatus}>
                          {statusLabel(appointmentStatus)}
                        </SelectItem>
                      ))}
                    </SelectContent>
                  </Select>
                  <FieldError message={editErrors.status} />
                </div>
              </div>
              <div className="space-y-1.5">
                <Label htmlFor="shop-notes">Shop notes</Label>
                <Textarea
                  id="shop-notes"
                  value={editForm.adminNotes}
```

```
                onChange={(event) =>
                  setEditForm({ ...editForm, adminNotes: event.target.value })
                }
                placeholder="Internal notes visible to shop staff only"
                rows={4}
              />
            </div>
            {!hasEditChanges && (
              <p className="text-sm text-muted-foreground">
                No changes to save. Update at least one appointment detail first.
              </p>
            )}
            <FieldError message={editErrors.form} />
          </section>
        </div>
      ) : (
        <div className="space-y-5">
          <AppointmentReadOnlyDetails appointment={selectedAppointment} />
          <section className="rounded-lg border border-border/70 p-4">
            <div className="flex flex-wrap items-center justify-between gap-2">
              <h3 className="font-medium">Appointment</h3>
              <Badge
                variant="outline"
                className={cn("uppercase",
statusTone(selectedAppointment.status))}
              >
                {statusLabel(selectedAppointment.status)}
              </Badge>
            </div>
            <div className="mt-3 grid gap-4 text-sm sm:grid-cols-2
lg:grid-cols-4">
              <AppointmentDetail
                label="Date"
                value={formatDateLong(selectedAppointment.appointment_date)}
              />
              <AppointmentDetail
                label="Time"
                value={formatTime(String(selectedAppointment.start_time).slice(0,
5))}
              />
              <AppointmentDetail
                label="Assigned crew"
                value={selectedAppointment.crew_members?.name ??
"Unassigned"}
              />
```

```
              <AppointmentDetail
                label="Booked on"
                value={formatBookedOn(selectedAppointment.created_at)}
              />
              <AppointmentDetail
                label="Reschedules"
                value={`${selectedAppointment.reschedule_count} / 3`}
              />
            </div>
          </section>
          <ServiceProgress appointment={selectedAppointment} />
          {rescheduleHistory.data && rescheduleHistory.data.length > 0 && (
            <section className="rounded-lg border border-border/70 p-4">
              <h3 className="font-medium">Reschedule history</h3>
              <div className="mt-3 divide-y divide-border/70 text-sm">
                {rescheduleHistory.data.map((entry) => (
                  <div
                    key={entry.id}
                    className="flex flex-col gap-1 py-3 first:pt-0 last:pb-0"
                  >
                    <div className="flex items-center justify-between gap-2">
                      <span className="font-medium">
                        {entry.reschedule_number === 1
                          ? "1st reschedule"
                          : entry.reschedule_number === 2
                            ? "2nd reschedule"
                            : entry.reschedule_number === 3
                              ? "3rd reschedule"
                              : `${entry.reschedule_number}th reschedule`}
                      </span>
                      <Badge
                        variant="outline"
                        className={
                          entry.decision === "confirmed"
                            ? "border-emerald-500/30 text-emerald-600"
                            : "border-destructive/40 text-destructive"
                        }
                      >
                        {entry.decision === "confirmed" ? "Approved" : "Rejected"}
                      </Badge>
                    </div>
                    <div className="grid gap-1 text-xs text-muted-foreground
sm:grid-cols-2">
                      <div>
                        <span className="font-medium
```

```
text-foreground">From:</span>{" "}
                        {formatDateLong(entry.from_date)} at{" "}
                        {formatTime(String(entry.from_start_time).slice(0, 5))}
                      </div>
                      {entry.to_date && entry.to_start_time && (
                        <div>
                          <span className="font-medium
text-foreground">To:</span>{" "}
                          {formatDateLong(entry.to_date)} at{" "}
                          {formatTime(String(entry.to_start_time).slice(0, 5))}
                        </div>
                      )}
                    </div>
                    <p className="text-xs text-muted-foreground">Reason:
{entry.reason}</p>
                    <p className="text-[10px] text-muted-foreground/70">
                      {formatBookedOn(entry.approved_at)}
                    </p>
                  </div>
                ))}
              </div>
            </section>
          )}
          <section className="rounded-lg border border-border/70 p-4">
            <h3 className="font-medium">Services</h3>
            <div className="mt-3 divide-y divide-border/70 text-sm">
              {selectedAppointment.appointment_services.map((service) => (
                <div
                  key={service.service_name}
                  className="flex items-center justify-between gap-4 py-2"
                >
                  <span>
                    {service.service_name}
                    <span className="ml-2 text-xs text-muted-foreground">
                      {service.duration_minutes} mins
                    </span>
                  </span>
                  <span
className="text-primary">{formatPHP(service.price)}</span>
                </div>
              ))}
              <div className="flex items-center justify-between gap-4 pt-3
font-medium">
                <span>Service subtotal</span>
                <span className="text-primary">
```

```
                {formatPHP(selectedAppointment.total_estimate)}
              </span>
            </div>
          </div>
        </section>
        <section className="rounded-lg border border-border/70 p-4">
          <div className="flex flex-wrap items-center justify-between gap-3">
            <div>
              <h3 className="font-medium">Products</h3>
              <p className="mt-1 text-sm text-muted-foreground">
                Customer selected: {selectedAppointment.wants_products ?
"Yes" : "No"}
              </p>
            </div>
            <span className="text-sm font-medium text-primary">
              Product subtotal{" "}
              {formatPHP(
                selectedAppointment.appointment_products.reduce(
                  (total, product) => total + product.quantity *
Number(product.unit_price),
                  0,
                ),
              )}
            </span>
          </div>
          {selectedAppointment.appointment_products.length > 0 ? (
            <div className="mt-3 divide-y divide-border/70 text-sm">
              {selectedAppointment.appointment_products.map((product) => (
                <div
                  key={product.id}
                  className="flex items-center justify-between gap-4 py-2"
                >
                  <span>
                    {product.product_name}
                    <span className="ml-2 text-xs text-muted-foreground">
                      {product.quantity} × {formatPHP(product.unit_price)}
                    </span>
                  </span>
                  <span className="text-primary">
                    {formatPHP(product.quantity * Number(product.unit_price))}
                  </span>
                </div>
              ))}
            </div>
          ) : (
```

```
              <p className="mt-3 text-sm text-muted-foreground">No products
recorded.</p>
            )}
            <div className="mt-3 flex items-center justify-between gap-4
border-t border-border/70 pt-3 font-medium">
              <span>Financial total</span>
              <span className="text-primary">
                {formatPHP(
                  Number(selectedAppointment.total_estimate) +
                    selectedAppointment.appointment_products.reduce(
                      (total, product) =>
                        total + product.quantity * Number(product.unit_price),
                      0,
                    ),
                )}
              </span>
            </div>
          </section>
          <section className="grid gap-4 sm:grid-cols-2">
            <AppointmentDetail
              className="rounded-lg border border-border/70 p-4"
              label="Customer notes"
              value={selectedAppointment.notes || "No customer notes provided."}
            />
            <AppointmentDetail
              className="rounded-lg border border-border/70 p-4"
              label="Shop notes"
              value={selectedAppointment.admin_notes || "No shop notes
added."}
            />
          </section>
        </div>
      )}

      <DialogFooter>
        <Button type="button" variant="outline"
onClick={closeAppointmentDialog}>
          Close
        </Button>
        {isEditing ? (
          <Button
            type="button"
            onClick={handleSaveAppointment}
            disabled={saveAppointment.isPending || !hasEditChanges}
          >
```

```
              {saveAppointment.isPending ? "Saving…" : "Save"}
            </Button>
          ) : (
            <Button type="button" onClick={() =>
startEditing(selectedAppointment)}>
              Edit
            </Button>
          )}
        </DialogFooter>
      </DialogContent>
    )}
  </Dialog>
  <AlertDialog
    open={Boolean(pendingSaveForm)}
    onOpenChange={(open) => {
      if (!open && !saveAppointment.isPending) setPendingSaveForm(null);
    }}
  >
    <AlertDialogContent>
      <AlertDialogHeader>
        <AlertDialogTitle>Save appointment changes?</AlertDialogTitle>
        <AlertDialogDescription>
          Confirm that the updated appointment details are correct. The
appointment will only be
          changed after you confirm.
        </AlertDialogDescription>
      </AlertDialogHeader>
      <AlertDialogFooter>
        <AlertDialogCancel
disabled={saveAppointment.isPending}>Cancel</AlertDialogCancel>
        <AlertDialogAction
          disabled={saveAppointment.isPending}
          onClick={confirmSaveAppointment}
        >
          {saveAppointment.isPending ? "Saving…" : "Confirm changes"}
        </AlertDialogAction>
      </AlertDialogFooter>
    </AlertDialogContent>
  </AlertDialog>
  <ArchiveConfirmationDialog
    open={Boolean(archiveTarget)}
    recordLabel="appointment"
    pending={archive.isPending}
    onOpenChange={(nextOpen) => !nextOpen && setArchiveTarget(null)}
    onConfirm={() => {
```

```
          if (archiveTarget) archive.mutate(archiveTarget);
          setArchiveTarget(null);
        }}
      />
      <NextStepDialog
        action={quickAction}
        pending={runNextStep.isPending}
        error={quickActionError}
        onCancel={() => {
          setQuickAction(null);
          setQuickActionError(null);
        }}
        onDecide={(decision) => {
          if (!quickAction) return;
          setQuickActionError(null);
          runNextStep.mutate({
            appointmentId: quickAction.appointment.id,
            step: quickAction.step,
            decision,
          });
        }}
      />
    </AdminPage>
  );
}

type NextStep = "review-booking" | "review-reschedule" | "start" | "complete";

const NEXT_STEPS: Record<
  NextStep,
  {
    label: string;
    icon: LucideIcon;
    title: string;
    description: string;
    confirm: string;
    reject?: string;
  }
> = {
  "review-booking": {
    label: "Review booking",
    icon: ClipboardCheck,
    title: "Review booking request",
    description:
      "Confirming assigns an available mechanic and notifies the customer.
```

```
Rejecting frees the time slot and emails and texts the customer that the booking
was not accepted.",
    confirm: "Confirm booking",
    reject: "Reject booking",
  },
  "review-reschedule": {
    label: "Review request",
    icon: CalendarClock,
    title: "Review reschedule request",
    description:
      "Approving books the requested schedule under the new reference and
notifies the customer. Rejecting keeps the current appointment and emails and
texts the customer that the request was not approved.",
    confirm: "Approve new schedule",
    reject: "Reject request",
  },
  start: {
    label: "Start service",
    icon: Play,
    title: "Start service?",
    description: "Records the start time and moves this appointment to In
progress.",
    confirm: "Start service",
  },
  complete: {
    label: "Mark completed",
    icon: CheckCircle2,
    title: "Mark service completed?",
    description:
      "Records the end time, counts the appointment in revenue, and lets the
customer know the service is done.",
    confirm: "Mark completed",
  },
};

/** What an appointment is waiting for, if anything. */
function nextStepFor(appointment: AppointmentDetails, today: string): NextStep |
null {
  if (appointment.pending_reschedule_request_id) return "review-reschedule";
  if (appointment.status === "pending") return "review-booking";
  // Not offered ahead of the appointment date, so a future booking is not started
by mistake.
  if (
    appointment.status === "confirmed" &&
    !appointment.service_started_at &&
```

```
    appointment.appointment_date <= today
  ) {
    return "start";
  }
  if (appointment.status === "in_progress") return "complete";
  return null;
}

/** Confirms a next step from the table, showing the booking it applies to. */
function NextStepDialog({
  action,
  pending,
  error,
  onCancel,
  onDecide,
}: {
  action: { appointment: AppointmentDetails; step: NextStep } | null;
  pending: boolean;
  error: string | null;
  onCancel: () => void;
  onDecide: (decision: "confirmed" | "rejected") => void;
}) {
  const appointment = action?.appointment;
  const step = action ? NEXT_STEPS[action.step] : null;
  const isReschedule = action?.step === "review-reschedule";
  const atRescheduleLimit = isReschedule && (appointment?.reschedule_count
?? 0) >= 3;

  return (
    <AlertDialog open={Boolean(action)} onOpenChange={(open) => !open &&
!pending && onCancel()}>
      <AlertDialogContent className="sm:max-w-lg">
        {appointment && step && (
          <>
            <AlertDialogHeader>
              <AlertDialogTitle>{step.title}</AlertDialogTitle>
              <AlertDialogDescription>{step.description}</AlertDialogDescription>
            </AlertDialogHeader>

            <dl className="grid gap-x-4 gap-y-3 rounded-lg border border-border/70
bg-muted/30 p-3 text-sm sm:grid-cols-2">
              <NextStepFact label="Customer">
                {appointment.customer_name}
                <span className="block text-xs text-muted-foreground">
                  <span
```

```
className="font-mono">{appointment.reference_code}</span> ·{" "}
                {toLocalPhilippineMobile(appointment.phone)}
              </span>
            </NextStepFact>
            <NextStepFact label="Motorcycle">
              {appointment.moto_brand} {appointment.moto_model}
              <span className="block text-xs text-muted-foreground">
                Plate {appointment.plate_number}
              </span>
            </NextStepFact>
            <NextStepFact label={isReschedule ? "Current schedule" : "Schedule"}>
              {formatDateLong(appointment.appointment_date)}
              <span className="block text-xs text-muted-foreground">
                {formatTime(String(appointment.start_time).slice(0, 5))}
              </span>
            </NextStepFact>
            {isReschedule ? (
              <NextStepFact label="Requested schedule">
                {appointment.pending_reschedule_date
                  ? formatDateLong(appointment.pending_reschedule_date)
                  : "Not provided"}
                <span className="block text-xs text-muted-foreground">
                  {appointment.pending_reschedule_start_time
                    ?
formatTime(String(appointment.pending_reschedule_start_time).slice(0, 5))
                    : ""}
                  {appointment.pending_reschedule_reference_code
                    ? ` · New reference
${appointment.pending_reschedule_reference_code}`
                    : ""}
                </span>
              </NextStepFact>
            ) : (
              <NextStepFact label="Services">
                {appointment.appointment_services
                  .map((service) => service.service_name)
                  .join(", ") || "None"}
              </NextStepFact>
            )}
            {isReschedule && appointment.pending_reschedule_reason && (
              <NextStepFact label="Reason" wide>
                {appointment.pending_reschedule_reason}
              </NextStepFact>
            )}
          </dl>
```

```
          {atRescheduleLimit && (
            <p className="text-sm text-destructive">
              This appointment has reached the maximum of 3 reschedules, so this
request can only
              be rejected.
            </p>
          )}
          {error && (
            <p
              role="alert"
              className="flex items-start gap-2 rounded-lg border
border-destructive/40 bg-destructive/5 p-3 text-sm text-destructive"
            >
              <CircleAlert className="mt-0.5 size-4 shrink-0" aria-hidden="true" />
              {error}
            </p>
          )}

          <AlertDialogFooter>
            <AlertDialogCancel disabled={pending}>Cancel</AlertDialogCancel>
            {step.reject && (
              <Button
                type="button"
                variant="outline"
                disabled={pending}
                onClick={() => onDecide("rejected")}
                className="border-destructive/40 text-destructive
hover:bg-destructive/10 hover:text-destructive"
              >
                <CircleX /> {step.reject}
              </Button>
            )}
            <Button
              type="button"
              disabled={pending || atRescheduleLimit}
              onClick={() => onDecide("confirmed")}
            >
              {pending ? (
                <Loader2 className="animate-spin" aria-hidden="true" />
              ) : (
                <step.icon aria-hidden="true" />
              )}
              {step.confirm}
            </Button>
```

```
          </AlertDialogFooter>
        </>
      )}
    </AlertDialogContent>
  </AlertDialog>
 );
}

function NextStepFact({
 label,
 wide = false,
 children,
}: {
 label: string;
 wide?: boolean;
 children: ReactNode;
}) {
 return (
  <div className={cn("min-w-0", wide && "sm:col-span-2")}>
   <dt className="text-[11px] tracking-wide text-muted-foreground
uppercase">{label}</dt>
   <dd className="mt-0.5 break-words">{children}</dd>
  </div>
 );
}

function AppointmentReadOnlyDetails({
 appointment,
 hideName = false,
}: {
 appointment: AppointmentDetails;
 hideName?: boolean;
}) {
 return (
  <section className="grid gap-4 rounded-lg border border-border/70 p-4
sm:grid-cols-2">
   <div>
    <h3 className="font-medium">Customer details</h3>
    <div className="mt-3 grid gap-3 text-sm">
     {!hideName && <AppointmentDetail label="Name"
value={appointment.customer_name} />}
     <AppointmentDetail
      label="Contact number"
      value={toLocalPhilippineMobile(appointment.phone)}
     />
```

```
        <AppointmentDetail label="Email" value={appointment.email || "Not
provided"} />
      </div>
    </div>
    <div>
      <h3 className="font-medium">Motorcycle details</h3>
      <div className="mt-3 grid gap-3 text-sm">
        <AppointmentDetail label="Brand" value={appointment.moto_brand} />
        <AppointmentDetail label="Model" value={appointment.moto_model} />
        <AppointmentDetail
          label="Variant / year"
          value={
            [appointment.moto_variant,
appointment.moto_year].filter(Boolean).join(" · ") ||
            "Not provided"
          }
        />
        <AppointmentDetail label="Plate number"
value={appointment.plate_number} />
      </div>
    </div>
  </section>
 );
}

function ServiceProgress({ appointment }: { appointment: AppointmentDetails }) {
 const serviceDurationMinutes = appointment.appointment_services.reduce(
  (total, service) => total + service.duration_minutes,
  0,
 );

 return (
  <section className="rounded-lg border border-border/70 p-4">
    <h3 className="font-medium">Service progress</h3>
    <div className="mt-3 grid gap-4 text-sm sm:grid-cols-3">
      <AppointmentDetail
        label="Service Start"
        value={
          appointment.service_started_at
            ? formatBookedOn(appointment.service_started_at)
            : "Not started"
        }
      />
      <AppointmentDetail
        label="Service End"
```

```
          value={
            appointment.service_ended_at
              ? formatBookedOn(appointment.service_ended_at)
              : "Not completed"
          }
        />
        <AppointmentDetail
          label="Service Duration"
          value={
            serviceDurationMinutes > 0
              ? formatServiceDuration(serviceDurationMinutes)
              : "Not available"
          }
        />
      </div>
    </section>
  );
}

function AppointmentDetail({
  label,
  value,
  className,
}: {
  label: string;
  value: string;
  className?: string;
}) {
  return (
    <div className={className}>
      <p className="text-xs tracking-wide text-muted-foreground
uppercase">{label}</p>
      <p className="mt-1 text-sm whitespace-pre-wrap text-foreground
[overflow-wrap:anywhere]">
        {value}
      </p>
    </div>
  );
}

function errorsForAppointmentUpdate(error: Error): AppointmentEditErrors {
  const message = error.message || "Could not save the appointment.";
  if (message.includes("appointments_crew_time_no_overlap")) {
    return {
      startTime:
```

```
      "This crew member already has an appointment that overlaps with the
selected time.",
    };
  }
  if (message.includes("selected crew member")) {
    return { assignedCrew: "The selected crew member is not available for this
appointment." };
  }
  if (
    message.includes("Start the service") ||
    message.includes("no-show") ||
    message.includes("completed service") ||
    message.includes("pre-service appointment")
  ) {
    return { status: message };
  }
  if (message.includes("service")) {
    return { services: "One or more selected services are not available." };
  }
  if (message.includes("status")) return { status: "Select a valid appointment
status." };
  return { form: "Could not save this appointment. Please try again." };
}

function cleanSearchTerm(value: string) {
  return value
    .trim()
    .replace(/[,%_()]/g, " ")
    .replace(/ /g, " ");
}

function AppointmentDateFilter({
  period,
  customDateRange,
  onPeriodChange,
  onCustomDateRangeChange,
}: {
  period: AppointmentDatePeriod;
  customDateRange: DateRange | undefined;
  onPeriodChange: (period: AppointmentDatePeriod) => void;
  onCustomDateRangeChange: (range: DateRange | undefined) => void;
}) {
  const [open, setOpen] = useState(false);
  const [isPickingCustomRange, setIsPickingCustomRange] = useState(false);
  const [draftCustomDateRange, setDraftCustomDateRange] =
```

```
useState<DateRange>();

  const selectPeriod = (nextPeriod: Exclude<AppointmentDatePeriod, "custom">)
=> {
    onPeriodChange(nextPeriod);
    setIsPickingCustomRange(false);
    setOpen(false);
  };

  const cancelCustomRange = () => {
    setDraftCustomDateRange(customDateRange);
    setIsPickingCustomRange(false);
    setOpen(false);
  };

  const applyCustomRange = () => {
    if (!draftCustomDateRange?.from || !draftCustomDateRange.to) return;
    onCustomDateRangeChange(draftCustomDateRange);
    onPeriodChange("custom");
    setIsPickingCustomRange(false);
    setOpen(false);
  };

  return (
    <Popover
      open={open}
      onOpenChange={(nextOpen) => {
        setOpen(nextOpen);
        if (!nextOpen) setIsPickingCustomRange(false);
      }}
    >
      <PopoverTrigger asChild>
        <Button
          type="button"
          variant="outline"
          aria-label="Filter by appointment date"
          className="w-full min-w-0 justify-between gap-2 sm:w-auto sm:min-w-40"
        >
          <span className="flex min-w-0 items-center gap-2 truncate">
            <CalendarRange className="size-4 shrink-0" aria-hidden="true" />
            {appointmentDatePeriodLabel(period,
buildAppointmentDateRange(period, customDateRange))}
          </span>
          <ChevronDown className="size-4 shrink-0" aria-hidden="true" />
        </Button>
```

```
    </PopoverTrigger>
    <PopoverContent align="start" className="w-72 p-2">
     {isPickingCustomRange ? (
      <div>
       <div className="px-2 pb-2">
        <p className="text-sm font-medium">Custom date range</p>
        <p className="text-xs text-muted-foreground">Select a start date and
an end date.</p>
       </div>
       <Calendar
        mode="range"
        selected={draftCustomDateRange}
        onSelect={setDraftCustomDateRange}
        className="mx-auto p-0"
        classNames={{
         nav: "inset-x-auto left-1/2 w-48 -translate-x-1/2 justify-between",
        }}
       />
       <div className="mt-2 grid grid-cols-2 gap-2 px-2 text-xs">
        <DateRangeValue label="Start date"
value={draftCustomDateRange?.from} />
        <DateRangeValue label="End date" value={draftCustomDateRange?.to}
/>
       </div>
       <div className="mt-3 flex justify-end gap-2 border-t border-border pt-3">
        <Button type="button" variant="outline" size="sm"
onClick={cancelCustomRange}>
         Cancel
        </Button>
        <Button
         type="button"
         size="sm"
         onClick={applyCustomRange}
         disabled={!draftCustomDateRange?.from ||
!draftCustomDateRange.to}
        >
         Apply
        </Button>
       </div>
      </div>
     ) : (
      <div className="space-y-1">
       <p className="px-2 pb-1 text-xs font-medium tracking-wide
text-muted-foreground uppercase">
        Appointment date
```

```
            </p>
            <AppointmentDateOption
              active={period === "all"}
              label="All dates"
              onClick={() => selectPeriod("all")}
            />
            <AppointmentDateOption
              active={period === "today"}
              label="Today"
              onClick={() => selectPeriod("today")}
            />
            <AppointmentDateOption
              active={period === "7d"}
              label="Last 7 days"
              onClick={() => selectPeriod("7d")}
            />
            <AppointmentDateOption
              active={period === "month"}
              label="This month"
              onClick={() => selectPeriod("month")}
            />
            <AppointmentDateOption
              active={period === "year"}
              label="This year"
              onClick={() => selectPeriod("year")}
            />
            <AppointmentDateOption
              active={period === "custom"}
              label="Custom date range"
              onClick={() => {
                setDraftCustomDateRange(customDateRange);
                setIsPickingCustomRange(true);
              }}
            />
          </div>
        )}
      </PopoverContent>
    </Popover>
  );
}

function AppointmentDateOption({
  active,
  label,
  onClick,
```

```
}: {
  active: boolean;
  label: string;
  onClick: () => void;
}) {
  return (
    <Button
      type="button"
      variant="ghost"
      className={cn("h-9 w-full justify-start text-sm", active && "bg-muted")}
      onClick={onClick}
    >
      {label}
    </Button>
  );
}

function DateRangeValue({ label, value }: { label: string; value: Date | undefined
}) {
  return (
    <div className="rounded-md border border-border bg-muted/30 px-2
py-1.5">
      <p className="text-[10px] tracking-wide text-muted-foreground
uppercase">{label}</p>
      <p className="mt-0.5 truncate font-medium text-foreground">
        {value ? format(value, "MMM d, yyyy") : "Not selected"}
      </p>
    </div>
  );
}

function formatBookedOn(createdAt: string) {
  const date = new Date(createdAt);

  const datePart = new Intl.DateTimeFormat("en-PH", {
    timeZone: "Asia/Manila",
    month: "short",
    day: "numeric",
    year: "numeric",
  }).format(date);

  const timePart = new Intl.DateTimeFormat("en-PH", {
    timeZone: "Asia/Manila",
    hour: "numeric",
    minute: "2-digit",
```

```
    hour12: true,
  }).format(date);

  return `${datePart.replace(/^([A-Za-z]{3})/, "$1.")} - ${timePart}`;
}

function formatServiceDuration(minutes: number) {
  const hours = Math.floor(minutes / 60);
  const remainingMinutes = minutes % 60;

  if (hours === 0) return `${remainingMinutes} min`;
  if (remainingMinutes === 0) return `${hours} hr`;
  return `${hours} hr ${remainingMinutes} min`;
}

function buildAppointmentDateRange(
  period: AppointmentDatePeriod,
  customRange?: DateRange,
): IsoDateRange | undefined {
  const today = manilaNow().date;

  if (period === "today") return { from: today, to: today };
  if (period === "7d") return { from: addDays(today, -6), to: today };
  if (period === "month") return manilaMonthDateRange(today);
  if (period === "year") return manilaYearDateRange(today);
  if (period === "custom" && customRange?.from && customRange.to) {
    return calendarRangeToIso(customRange);
  }
  return undefined;
}

function appointmentDatePeriodLabel(period: AppointmentDatePeriod, range?:
IsoDateRange) {
  if (period === "all") return "All dates";
  if (period === "today") return "Today";
  if (period === "7d") return "Last 7 days";
  if (period === "month") return "This month";
  if (period === "year") return "This year";
  if (!range) return "Custom date range";
  return `${format(isoDateToCalendarDate(range.from), "MMM d")} –
${format(isoDateToCalendarDate(range.to), "MMM d, yyyy")}`;
}
```

src/routes/_authenticated/admin/archive.tsx

```
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import { createFileRoute } from "@tanstack/react-router";
import { format } from "date-fns";
import {
  Archive,
  CalendarRange,
  ChevronDown,
  RotateCcw,
  SlidersHorizontal,
  Trash2,
} from "lucide-react";
import { useDeferredValue, useEffect, useRef, useState } from "react";
import type { DateRange } from "react-day-picker";

import {
  AdminPage,
  DataList,
  InlineAlert,
  ListCard,
  ListEmpty,
  ListToolbar,
  ResetFiltersButton,
  ToolbarFilters,
  ToolbarSearch,
} from "@/components/admin/data-list";
import { PageHeader } from "@/components/admin/page-header";
import { PaginationControls } from "@/components/admin/pagination-controls";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Calendar } from "@/components/ui/calendar";
import {
  AlertDialog,
  AlertDialogAction,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
} from "@/components/ui/alert-dialog";
import { Popover, PopoverContent, PopoverTrigger } from
"@/components/ui/popover";
import {
```

```
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@/components/ui/select";
import { Tabs, TabsContent, TabsList, TabsTrigger } from
"@/components/ui/tabs";
import { supabase } from "@/integrations/supabase/client";
import {
  calendarRangeToIso,
  formatShortDate,
  isoDateToCalendarDate,
  manilaMonthDateRange,
  manilaWeekDateRange,
  startOfManilaDay,
  startOfNextManilaDay,
} from "@/lib/admin-date-range";
import {
  COMPLETED_APPOINTMENT_KEPT,
  decodeBlockReason,
  formatDateLong,
  formatPHP,
  formatTime,
  manilaNow,
  normalizePhilippineMobile,
  statusLabel,
  statusTone,
  toLocalPhilippineMobile,
} from "@/lib/shop";
import { cn } from "@/lib/utils";

export const Route = createFileRoute("/_authenticated/admin/archive")({
  component: ArchivePage,
});

const ARCHIVE_TABS = [
  { value: "appointments", label: "Appointments" },
  { value: "services", label: "Services" },
  { value: "products", label: "Products" },
  { value: "motorcycles", label: "Motorcycles" },
  { value: "crew", label: "Pit Crew" },
  { value: "blocks", label: "Schedule Blocks" },
  { value: "blocked-numbers", label: "Blocked Numbers" },
] as const;
```

```
const APPOINTMENT_STATUSES = [
  "pending",
  "confirmed",
  "completed",
  "cancelled",
  "rejected",
  "no_show",
];

type ArchiveFilters = {
  term: string;
  datePeriod: ArchiveDatePeriod;
  dateFrom: string;
  dateTo: string;
  customDateFrom: string;
  customDateTo: string;
  status: string;
};

type ArchiveDatePeriod = "all" | "today" | "week" | "month" | "custom";

type ArchiveItemType =
  "appointment" | "service" | "product" | "motorcycle" | "crew" | "block" |
"blockedNumber";

type ArchiveActionTarget = {
  id: string;
  type: ArchiveItemType;
  label: string;
};

const ARCHIVE_ITEM_LABELS: Record<ArchiveItemType, string> = {
  appointment: "appointment",
  service: "service",
  product: "product",
  motorcycle: "motorcycle catalog item",
  crew: "pit crew member",
  block: "schedule block",
  blockedNumber: "blocked number",
};

const EMPTY_FILTERS: ArchiveFilters = {
  term: "",
  datePeriod: "all",
```

```
  dateFrom: "",
  dateTo: "",
  customDateFrom: "",
  customDateTo: "",
  status: "all",
};

function ArchivePage() {
  const pageSize = 10;
  const queryClient = useQueryClient();
  const [draftFilters, setDraftFilters] =
useState<ArchiveFilters>(EMPTY_FILTERS);
  const [filters, setFilters] = useState<ArchiveFilters>(EMPTY_FILTERS);
  const [activeTab, setActiveTab] = useState("appointments");
  const [page, setPage] = useState(0);
  const [restoreTarget, setRestoreTarget] = useState<ArchiveActionTarget |
null>(null);
  const [deleteTarget, setDeleteTarget] = useState<ArchiveActionTarget |
null>(null);
  const [actionError, setActionError] = useState<string | null>(null);
  const deferredTerm = useDeferredValue(filters.term);
  const cleanedSearchTerm = cleanSearchTerm(deferredTerm);
  const searchTerm = normalizePhilippineMobile(cleanedSearchTerm) ??
cleanedSearchTerm;

  const archived = useQuery({
    queryKey: ["archived-appointments", { searchTerm, filters, page }],
    queryFn: async () => {
      let query = supabase
        .from("appointments")
        .select("*", { count: "exact" })
        .eq("is_archived", true)
        .order("appointment_date", { ascending: false });
      if (searchTerm)
        query = query.or(
          `reference_code.ilike.%${searchTerm}%,customer_name.ilike.%${searchT
erm}%,phone.ilike.%${searchTerm}%,plate_number.ilike.%${searchTerm}%`,
        );
      if (filters.status !== "all") query = query.eq("status", filters.status);
      if (filters.dateFrom) query = query.gte("appointment_date", filters.dateFrom);
      if (filters.dateTo) query = query.lte("appointment_date", filters.dateTo);
      const { data, error, count } = await query.range(
        page * pageSize,
        page * pageSize + pageSize - 1,
      );
```

```
      if (error) throw error;
      return { rows: data ?? [], total: count ?? 0 };
    },
  });

  const archivedServices = useQuery({
    queryKey: ["archived-services", { searchTerm, filters, page }],
    queryFn: async () => {
      let query = supabase
        .from("services")
        .select("*", { count: "exact" })
        .eq("is_archived", true)
        .order("sort_order");
      if (searchTerm) query = query.ilike("name", `%${searchTerm}%`);
      if (filters.dateFrom) query = query.gte("created_at",
startOfManilaDay(filters.dateFrom));
      if (filters.dateTo) query = query.lt("created_at",
startOfNextManilaDay(filters.dateTo));
      const { data, error, count } = await query.range(
        page * pageSize,
        page * pageSize + pageSize - 1,
      );
      if (error) throw error;
      return { rows: data ?? [], total: count ?? 0 };
    },
  });

  const archivedProducts = useQuery({
    queryKey: ["archived-products", { searchTerm, filters, page }],
    queryFn: async () => {
      let query = supabase
        .from("products")
        .select("*", { count: "exact" })
        .in("category", ["part", "accessory"])
        .eq("is_archived", true)
        .order("sort_order");
      if (searchTerm) query =
query.or(`name.ilike.%${searchTerm}%,brand.ilike.%${searchTerm}%`);
      if (filters.dateFrom) query = query.gte("created_at",
startOfManilaDay(filters.dateFrom));
      if (filters.dateTo) query = query.lt("created_at",
startOfNextManilaDay(filters.dateTo));
      const { data, error, count } = await query.range(
        page * pageSize,
        page * pageSize + pageSize - 1,
```

```
      );
      if (error) throw error;
      return { rows: data ?? [], total: count ?? 0 };
    },
  });

  const archivedMotorcycles = useQuery({
    queryKey: ["archived-motorcycles", { searchTerm, filters, page }],
    queryFn: async () => {
      let query = supabase
        .from("motorcycle_catalog")
        .select("*", { count: "exact" })
        .eq("is_archived", true)
        .order("brand")
        .order("model");
      if (searchTerm) query =
query.or(`model.ilike.%${searchTerm}%,brand.ilike.%${searchTerm}%`);
      if (filters.dateFrom) query = query.gte("created_at",
startOfManilaDay(filters.dateFrom));
      if (filters.dateTo) query = query.lt("created_at",
startOfNextManilaDay(filters.dateTo));
      const { data, error, count } = await query.range(
        page * pageSize,
        page * pageSize + pageSize - 1,
      );
      if (error) throw error;
      return { rows: data ?? [], total: count ?? 0 };
    },
  });

  const archivedCrew = useQuery({
    queryKey: ["archived-crew", { searchTerm, filters, page }],
    queryFn: async () => {
      let query = supabase
        .from("crew_members")
        .select("*", { count: "exact" })
        .eq("is_archived", true)
        .order("name");
      if (searchTerm)
        query = query.or(
          `name.ilike.%${searchTerm}%,role.ilike.%${searchTerm}%,phone.ilike.%${
searchTerm}%`,
        );
      if (filters.dateFrom) query = query.gte("created_at",
startOfManilaDay(filters.dateFrom));
```

```
      if (filters.dateTo) query = query.lt("created_at",
startOfNextManilaDay(filters.dateTo));
      const { data, error, count } = await query.range(
        page * pageSize,
        page * pageSize + pageSize - 1,
      );
      if (error) throw error;
      return { rows: data ?? [], total: count ?? 0 };
    },
  });

  const archivedBlocks = useQuery({
    queryKey: ["archived-blocks", { searchTerm, filters, page }],
    queryFn: async () => {
      let query = supabase
        .from("schedule_blocks")
        .select("*", { count: "exact" })
        .eq("is_active", false)
        .order("block_date", { ascending: false });
      if (searchTerm) query = query.ilike("reason", `%${searchTerm}%`);
      if (filters.dateFrom) query = query.gte("block_date", filters.dateFrom);
      if (filters.dateTo) query = query.lte("block_date", filters.dateTo);
      const { data, error, count } = await query.range(
        page * pageSize,
        page * pageSize + pageSize - 1,
      );
      if (error) throw error;
      return { rows: data ?? [], total: count ?? 0 };
    },
  });

  const archivedBlockedNumbers = useQuery({
    queryKey: ["archived-blocked-numbers", { searchTerm, filters, page }],
    queryFn: async () => {
      let query = supabase
        .from("blocked_numbers")
        .select("*", { count: "exact" })
        .eq("is_archived", true)
        .order("created_at", { ascending: false });
      if (searchTerm) query =
query.or(`phone.ilike.%${searchTerm}%,reason.ilike.%${searchTerm}%`);
      if (filters.dateFrom) query = query.gte("created_at",
startOfManilaDay(filters.dateFrom));
      if (filters.dateTo) query = query.lt("created_at",
startOfNextManilaDay(filters.dateTo));
```

```
      const { data, error, count } = await query.range(
        page * pageSize,
        page * pageSize + pageSize - 1,
      );
      if (error) throw error;
      return { rows: data ?? [], total: count ?? 0 };
    },
  });

  const restoreAppointment = useMutation({
    mutationFn: async (id: string) => {
      const { error } = await supabase
        .from("appointments")
        .update({ is_archived: false })
        .eq("id", id);
      if (error) throw error;
    },
    onSuccess: () => {
      setActionError(null);
      queryClient.invalidateQueries({ queryKey: ["archived-appointments"], exact:
false });
      queryClient.invalidateQueries({ queryKey: ["admin-appointments"], exact:
false });
      queryClient.invalidateQueries({ queryKey: ["admin-dashboard"], exact: false
});
    },
    onError: (err: Error) => {
      console.error("Restore failed:", err);
      setActionError("Could not restore this appointment. Please try again.");
    },
  });

  const restoreService = useMutation({
    mutationFn: async (id: string) => {
      const { error } = await supabase.from("services").update({ is_archived: false
}).eq("id", id);
      if (error) throw error;
    },
    onSuccess: () => {
      setActionError(null);
      queryClient.invalidateQueries({ queryKey: ["archived-services"], exact: false
});
      queryClient.invalidateQueries({ queryKey: ["admin-services"], exact: false });
    },
    onError: (err: Error) => {
```

```
      console.error("Restore failed:", err);
      setActionError("Could not restore this service. Please try again.");
    },
  });

  const restoreProduct = useMutation({
    mutationFn: async (id: string) => {
      const { error } = await supabase.from("products").update({ is_archived: false
}).eq("id", id);
      if (error) throw error;
    },
    onSuccess: () => {
      setActionError(null);
      queryClient.invalidateQueries({ queryKey: ["archived-products"], exact: false
});
      queryClient.invalidateQueries({ queryKey: ["admin-products"], exact: false });
    },
    onError: (err: Error) => {
      console.error("Restore failed:", err);
      setActionError("Could not restore this product. Please try again.");
    },
  });

  const restoreMotorcycle = useMutation({
    mutationFn: async (id: string) => {
      const { error } = await supabase
        .from("motorcycle_catalog")
        .update({ is_archived: false })
        .eq("id", id);
      if (error) throw error;
    },
    onSuccess: () => {
      setActionError(null);
      queryClient.invalidateQueries({ queryKey: ["archived-motorcycles"], exact:
false });
      queryClient.invalidateQueries({ queryKey: ["admin-motorcycle-catalog"],
exact: false });
      queryClient.invalidateQueries({ queryKey: ["motorcycle-catalog"], exact: false
});
      queryClient.invalidateQueries({ queryKey: ["admin-service-model-catalog"],
exact: false });
    },
    onError: (err: Error) => {
      console.error("Restore failed:", err);
      setActionError("Could not restore this motorcycle catalog item. Please try
```

```
again.");
    },
  });

  const restoreCrew = useMutation({
    mutationFn: async (id: string) => {
      const { error } = await supabase
        .from("crew_members")
        .update({ is_archived: false })
        .eq("id", id);
      if (error) throw error;
    },
    onSuccess: () => {
      setActionError(null);
      queryClient.invalidateQueries({ queryKey: ["archived-crew"], exact: false });
      queryClient.invalidateQueries({ queryKey: ["crew-all"], exact: false });
    },
    onError: (err: Error) => {
      console.error("Restore failed:", err);
      setActionError("Could not restore this crew member. Please try again.");
    },
  });

  const restoreBlock = useMutation({
    mutationFn: async (id: string) => {
      const { error } = await supabase
        .from("schedule_blocks")
        .update({ is_active: true })
        .eq("id", id);
      if (error) throw error;
    },
    onSuccess: () => {
      setActionError(null);
      queryClient.invalidateQueries({ queryKey: ["archived-blocks"], exact: false });
      queryClient.invalidateQueries({ queryKey: ["schedule-blocks"], exact: false });
    },
    onError: (err: Error) => {
      console.error("Restore failed:", err);
      setActionError("Could not restore this schedule block. Please try again.");
    },
  });

  const restoreBlockedNumber = useMutation({
    mutationFn: async (id: string) => {
      const { error } = await supabase
```

```
        .from("blocked_numbers")
        .update({ is_archived: false })
        .eq("id", id);
      if (error) throw error;
    },
    onSuccess: () => {
      setActionError(null);
      queryClient.invalidateQueries({ queryKey: ["archived-blocked-numbers"],
exact: false });
      queryClient.invalidateQueries({ queryKey: ["blocked-numbers"], exact: false
});
    },
    onError: (err: Error) => {
      console.error("Restore failed:", err);
      setActionError("Could not restore this blocked number. Please try again.");
    },
  });

  const deleteItem = useMutation({
    mutationFn: async ({ id, type }: Pick<ArchiveActionTarget, "id" | "type">) => {
      const tables: Record<ArchiveItemType, string> = {
        appointment: "appointments",
        service: "services",
        product: "products",
        motorcycle: "motorcycle_catalog",
        crew: "crew_members",
        block: "schedule_blocks",
        blockedNumber: "blocked_numbers",
      };
      const table = tables[type] as
        | "appointments"
        | "services"
        | "products"
        | "motorcycle_catalog"
        | "crew_members"
        | "schedule_blocks"
        | "blocked_numbers";
      if (!table) throw new Error(`Unknown type: ${type}`);
      const { error } = await supabase.from(table).delete().eq("id", id);
      if (error) throw error;
    },
    onSuccess: () => {
      setDeleteTarget(null);
      setActionError(null);
      queryClient.invalidateQueries({ queryKey: ["archived-appointments"], exact:
```

```
false });
      queryClient.invalidateQueries({ queryKey: ["archived-services"], exact: false
});
      queryClient.invalidateQueries({ queryKey: ["archived-products"], exact: false
});
      queryClient.invalidateQueries({ queryKey: ["archived-motorcycles"], exact:
false });
      queryClient.invalidateQueries({ queryKey: ["archived-crew"], exact: false });
      queryClient.invalidateQueries({ queryKey: ["archived-blocks"], exact: false });
      queryClient.invalidateQueries({ queryKey: ["archived-blocked-numbers"],
exact: false });
      queryClient.invalidateQueries({ queryKey: ["admin-appointments"], exact:
false });
      queryClient.invalidateQueries({ queryKey: ["admin-dashboard"], exact: false
});
      // The customer list is derived from appointment history. Refresh it after
      // a permanent appointment deletion so a customer with no appointments
      // is removed immediately.
      queryClient.invalidateQueries({ queryKey: ["customers"], exact: false });
      queryClient.invalidateQueries({ queryKey: ["customer-history"], exact: false });
      queryClient.invalidateQueries({ queryKey: ["admin-services"], exact: false });
      queryClient.invalidateQueries({ queryKey: ["admin-products"], exact: false });
      queryClient.invalidateQueries({ queryKey: ["crew-all"], exact: false });
      queryClient.invalidateQueries({ queryKey: ["schedule-blocks"], exact: false });
      queryClient.invalidateQueries({ queryKey: ["blocked-numbers"], exact: false
});
      queryClient.invalidateQueries({ queryKey: ["motorcycle-catalog"], exact: false
});
    },
    onError: (err: Error) => {
      console.error("Delete failed:", err);
      setActionError(
        err.message.includes("Completed appointments")
          ? COMPLETED_APPOINTMENT_KEPT
          : "Could not permanently delete this item. Please try again.",
      );
    },
  });

  const appointmentRows = archived.data?.rows ?? [];
  const serviceRows = archivedServices.data?.rows ?? [];
  const productRows = archivedProducts.data?.rows ?? [];
  const motorcycleRows = archivedMotorcycles.data?.rows ?? [];
  const crewRows = archivedCrew.data?.rows ?? [];
  const blockRows = archivedBlocks.data?.rows ?? [];
```

```
const blockedNumberRows = archivedBlockedNumbers.data?.rows ?? [];
const archiveCounts: Record<string, number> = {
  appointments: archived.data?.total ?? 0,
  services: archivedServices.data?.total ?? 0,
  products: archivedProducts.data?.total ?? 0,
  motorcycles: archivedMotorcycles.data?.total ?? 0,
  crew: archivedCrew.data?.total ?? 0,
  blocks: archivedBlocks.data?.total ?? 0,
  "blocked-numbers": archivedBlockedNumbers.data?.total ?? 0,
};
const activeArchive = {
  appointments: archived.data,
  services: archivedServices.data,
  products: archivedProducts.data,
  motorcycles: archivedMotorcycles.data,
  crew: archivedCrew.data,
  blocks: archivedBlocks.data,
  "blocked-numbers": archivedBlockedNumbers.data,
}[activeTab];

useEffect(() => {
  const total = activeArchive?.total ?? 0;
  const lastPage = Math.max(0, Math.ceil(total / pageSize) - 1);
  if (page > lastPage) setPage(lastPage);
}, [activeArchive?.total, page, pageSize]);

const confirmRestore = (target: ArchiveActionTarget) => {
  setRestoreTarget(target);
};

const confirmDelete = (target: ArchiveActionTarget) => {
  setDeleteTarget(target);
};

const handleRestore = () => {
  const target = restoreTarget;
  if (!target) return;

  setRestoreTarget(null);
  switch (target.type) {
    case "appointment":
      restoreAppointment.mutate(target.id);
      break;
    case "service":
      restoreService.mutate(target.id);
```

```
      break;
    case "product":
      restoreProduct.mutate(target.id);
      break;
    case "motorcycle":
      restoreMotorcycle.mutate(target.id);
      break;
    case "crew":
      restoreCrew.mutate(target.id);
      break;
    case "block":
      restoreBlock.mutate(target.id);
      break;
    case "blockedNumber":
      restoreBlockedNumber.mutate(target.id);
      break;
  }
};

const handleDelete = () => {
  if (deleteTarget) {
    deleteItem.mutate(deleteTarget);
  }
};

const applyFilters = () => {
  setFilters({ ...draftFilters });
  setPage(0);
};

const resetFilters = () => {
  setDraftFilters(EMPTY_FILTERS);
  setFilters(EMPTY_FILTERS);
  setPage(0);
};

const setArchiveType = (value: string) => {
  if (!ARCHIVE_TABS.some((tab) => tab.value === value)) return;
  setActiveTab(value);
  setPage(0);
};

const hasAppliedFilters =
  Boolean(filters.term.trim()) || filters.datePeriod !== "all" || filters.status !== "all";
```

```
  const rowActions = (target: ArchiveActionTarget, kept = false) => (
    <ArchiveRowActions
      label={target.label}
      onRestore={() => confirmRestore(target)}
      onDelete={() => confirmDelete(target)}
      kept={kept}
    />
  );
  const loadError = (
    query: { isError: boolean; refetch: () => unknown },
    label: string,
  ): { message: string; onRetry: () => void } | null =>
    query.isError
      ? { message: `Could not load archived ${label}.`, onRetry: () => void
query.refetch() }
      : null;
  const emptyState = (label: string) => (
    <ListEmpty
      icon={Archive}
      title={
        hasAppliedFilters ? `No archived ${label} match your filters.` : `No archived
${label}.`
      }
      description={
        hasAppliedFilters
          ? "Try a different search or date range."
          : "Archived records stay here for 30 days before they are deleted
automatically."
      }
      action={hasAppliedFilters ? <ResetFiltersButton onClick={resetFilters} /> :
undefined}
    />
  );

  return (
    <AdminPage>
      <PageHeader
        title="Archive"
        description="Restore archived bookings, services, products, crew, schedule
blocks, and blocked numbers, or delete them permanently. Archived records are
deleted automatically after 30 days, except completed appointments, which are
kept for financial records."
      />
      {actionError && <InlineAlert>{actionError}</InlineAlert>}
```

```
    <Tabs value={activeTab} onValueChange={setArchiveType}>
      <TabsList className="mb-4">
        {ARCHIVE_TABS.map((tab) => (
          <TabsTrigger key={tab.value} value={tab.value} className="gap-1.5">
            {tab.label}
            <span className="rounded-full bg-foreground/10 px-1.5 text-xs
tabular-nums">
              {archiveCounts[tab.value]}
            </span>
          </TabsTrigger>
        ))}
      </TabsList>

      <ListToolbar
        summary={
          activeArchive &&
          `${activeArchive.total} archived ${activeArchive.total === 1 ? "record" :
"records"}${hasAppliedFilters ? " match your filters" : ""}`
        }
      >
        <ToolbarSearch
          id="archive-search"
          value={draftFilters.term}
          onChange={(term) => setDraftFilters((current) => ({ ...current, term }))}
          onEnter={applyFilters}
          placeholder="Search name, customer, or reference"
          label="Search archived records"
        />
        <ToolbarFilters>
          <div className="col-span-2 sm:w-56">
            <ArchiveDateRangeFilter
              period={draftFilters.datePeriod}
              dateFrom={draftFilters.dateFrom}
              dateTo={draftFilters.dateTo}
              customDateFrom={draftFilters.customDateFrom}
              customDateTo={draftFilters.customDateTo}
              onPeriodChange={(datePeriod) =>
                setDraftFilters((current) => ({ ...current, datePeriod }))
              }
              onDateRangeChange={(dateFrom, dateTo) =>
                setDraftFilters((current) => ({ ...current, dateFrom, dateTo }))
              }
              onCustomDateRangeChange={(customDateFrom, customDateTo) =>
                setDraftFilters((current) => ({ ...current, customDateFrom,
customDateTo }))
```

```
        }
      />
    </div>
    {activeTab === "appointments" && (
      <Select
        value={draftFilters.status}
        onValueChange={(value) =>
          setDraftFilters((current) => ({ ...current, status: value }))
        }
      >
        <SelectTrigger
          aria-label="Filter by appointment status"
          className="col-span-2 w-full sm:w-40"
        >
          <SelectValue />
        </SelectTrigger>
        <SelectContent>
          <SelectItem value="all">All statuses</SelectItem>
          {APPOINTMENT_STATUSES.map((status) => (
            <SelectItem key={status} value={status}>
              {statusLabel(status)}
            </SelectItem>
          ))}
        </SelectContent>
      </Select>
    )}
    <Button type="button" onClick={applyFilters}>
      <SlidersHorizontal /> Apply filters
    </Button>
    <Button type="button" variant="outline" onClick={resetFilters}>
      <RotateCcw /> Reset
    </Button>
  </ToolbarFilters>
</ListToolbar>

<ListCard
  footer={
    <PaginationControls
      page={page}
      pageSize={pageSize}
      total={activeArchive?.total ?? 0}
      onPageChange={setPage}
    />
  }
>
```

```
        <TabsContent value="appointments" className="mt-0">
          <DataList
            label="Archived appointments"
            rows={appointmentRows}
            rowKey={(a) => a.id}
            loading={archived.isLoading}
            error={loadError(archived, "appointments")}
            empty={emptyState("appointments")}
            columns={[
              {
                key: "customer",
                header: "Customer",
                card: "title",
                cell: (a) => (
                  <>
                    <p className="font-medium">{a.customer_name}</p>
                    <p className="mt-0.5 text-xs text-muted-foreground">
                      <span className="font-mono">{a.reference_code}</span> ·
{a.moto_brand}{" "}
                      {a.moto_model} · {a.plate_number}
                    </p>
                  </>
                ),
              },
              {
                key: "schedule",
                header: "Schedule",
                className: "w-40",
                cell: (a) => (
                  <span className="text-sm">
                    {formatDateLong(a.appointment_date)}
                    <span className="block text-xs text-muted-foreground">
                      {formatTime(String(a.start_time).slice(0, 5))}
                    </span>
                  </span>
                ),
              },
              {
                key: "estimate",
                header: "Estimate",
                xlOnly: true,
                className: "w-28 text-right",
                cell: (a) => <span
className="text-sm">{formatPHP(a.total_estimate)}</span>,
              },
```

```
            {
              key: "status",
              header: "Status",
              card: "badge",
              className: "w-28 text-center",
              cell: (a) => (
                <Badge variant="outline" className={cn("uppercase",
statusTone(a.status))}>
                  {statusLabel(a.status)}
                </Badge>
              ),
            },
          ]}
          actionsClassName="w-56"
          actions={(a) =>
            rowActions(
              { id: a.id, type: "appointment", label: a.reference_code },
              a.status === "completed",
            )
          }
        />
      </TabsContent>

      <TabsContent value="services" className="mt-0">
        <DataList
          label="Archived services"
          rows={serviceRows}
          rowKey={(s) => s.id}
          loading={archivedServices.isLoading}
          error={loadError(archivedServices, "services")}
          empty={emptyState("services")}
          columns={[
            {
              key: "service",
              header: "Service",
              card: "title",
              cell: (s) => (
                <>
                  <p className="font-medium">{s.name}</p>
                  <div className="mt-1 text-xs text-muted-foreground">
                    <ArchiveServiceDetails
                      serviceId={s.id}
                      serviceName={s.name}
                      description={s.description}
                    />
```

```
                </div>
              </>
            ),
          },
          {
            key: "price",
            header: "Price",
            className: "w-28 text-right",
            cell: (s) => <span className="text-sm
text-primary">{formatPHP(s.price)}</span>,
          },
          {
            key: "duration",
            header: "Duration",
            className: "w-28",
            cell: (s) => <span className="text-sm">{s.duration_minutes}
min</span>,
          },
        ]}
        actionsClassName="w-56"
        actions={(s) => rowActions({ id: s.id, type: "service", label: s.name })}
      />
    </TabsContent>

    <TabsContent value="products" className="mt-0">
      <DataList
        label="Archived products"
        rows={productRows}
        rowKey={(p) => p.id}
        loading={archivedProducts.isLoading}
        error={loadError(archivedProducts, "products")}
        empty={emptyState("products")}
        columns={[
          {
            key: "item",
            header: "Item",
            card: "title",
            cell: (p) => (
              <>
                <p className="font-medium">{p.name}</p>
                <p className="mt-0.5 text-xs text-muted-foreground">
                  <span className="capitalize">{p.category}</span>
                  {p.brand ? ` · ${p.brand}` : ""}
                </p>
              </>
```

```
              ),
            },
            {
              key: "price",
              header: "Price",
              className: "w-28 text-right",
              cell: (p) => <span className="text-sm
text-primary">{formatPHP(p.price)}</span>,
            },
          ]}
          actionsClassName="w-56"
          actions={(p) => rowActions({ id: p.id, type: "product", label: p.name })}
        />
      </TabsContent>

      <TabsContent value="motorcycles" className="mt-0">
        <DataList
          label="Archived motorcycles"
          rows={motorcycleRows}
          rowKey={(m) => m.id}
          loading={archivedMotorcycles.isLoading}
          error={loadError(archivedMotorcycles, "motorcycles")}
          empty={emptyState("motorcycles")}
          columns={[
            {
              key: "model",
              header: "Model",
              card: "title",
              cell: (m) => (
                <>
                  <p className="font-medium">{m.model}</p>
                  <p className="mt-0.5 text-xs text-muted-foreground">
                    {m.brand ?? "No brand"}
                  </p>
                </>
              ),
            },
            {
              key: "engine",
              header: "Engine",
              className: "w-28 text-right",
              cell: (m) =>
                m.engine_cc === null ? (
                  <span className="text-sm text-muted-foreground">Not
set</span>
```

```
              ) : (
                <span className="text-sm">{m.engine_cc}cc</span>
              ),
            },
          ]}
          actionsClassName="w-56"
          actions={(m) =>
            rowActions({
              id: m.id,
              type: "motorcycle",
              label: `${m.brand ?? ""} ${m.model}`.trim(),
            })
          }
        />
      </TabsContent>

      <TabsContent value="crew" className="mt-0">
        <DataList
          label="Archived crew members"
          rows={crewRows}
          rowKey={(c) => c.id}
          loading={archivedCrew.isLoading}
          error={loadError(archivedCrew, "crew members")}
          empty={emptyState("crew members")}
          columns={[
            {
              key: "name",
              header: "Name",
              card: "title",
              cell: (c) => (
                <>
                  <p className="font-medium">{c.name}</p>
                  <p className="mt-0.5 text-xs text-muted-foreground">{c.role}</p>
                </>
              ),
            },
            {
              key: "phone",
              header: "Phone",
              className: "w-40",
              cell: (c) =>
                c.phone ? (
                  <span
                    className="text-sm">{toLocalPhilippineMobile(c.phone)}</span>
                ) : (
```

```
                <span className="text-sm text-muted-foreground">Not
set</span>
              ),
            },
          ]}
          actionsClassName="w-56"
          actions={(c) => rowActions({ id: c.id, type: "crew", label: c.name })}
        />
      </TabsContent>

      <TabsContent value="blocks" className="mt-0">
        <DataList
          label="Archived schedule blocks"
          rows={blockRows}
          rowKey={(b) => b.id}
          loading={archivedBlocks.isLoading}
          error={loadError(archivedBlocks, "schedule blocks")}
          empty={emptyState("schedule blocks")}
          columns={[
            {
              key: "date",
              header: "Date",
              card: "title",
              className: "w-48",
              cell: (b) => <span
className="font-medium">{formatDateLong(b.block_date)}</span>,
            },
            {
              key: "slot",
              header: "Time",
              className: "w-40",
              cell: (b) => <span
className="text-sm">{archivedBlockSlotLabel(b)}</span>,
            },
            {
              key: "reason",
              header: "Reason",
              card: "wide",
              cell: (b) => (
                <span className="text-sm text-muted-foreground">
                  {decodeBlockReason(b.reason).userReason || "No reason given"}
                </span>
              ),
            },
          ]}
```

```
            actionsClassName="w-56"
            actions={(b) =>
              rowActions({
                id: b.id,
                type: "block",
                label:
                  decodeBlockReason(b.reason).userReason.trim() ||
formatDateLong(b.block_date),
              })
            }
          />
        </TabsContent>

        <TabsContent value="blocked-numbers" className="mt-0">
          <DataList
            label="Archived blocked numbers"
            rows={blockedNumberRows}
            rowKey={(n) => n.id}
            loading={archivedBlockedNumbers.isLoading}
            error={loadError(archivedBlockedNumbers, "blocked numbers")}
            empty={emptyState("blocked numbers")}
            columns={[
              {
                key: "phone",
                header: "Phone",
                card: "title",
                className: "w-44",
                cell: (n) => (
                  <span className="font-mono text-sm font-medium">
                    {toLocalPhilippineMobile(n.phone)}
                  </span>
                ),
              },
              {
                key: "reason",
                header: "Reason",
                card: "wide",
                cell: (n) => (
                  <span className="text-sm text-muted-foreground">
                    {n.reason || "No reason given"}
                  </span>
                ),
              },
              {
                key: "created",
```

```
              header: "Blocked on",
              className: "w-36",
              cell: (n) => <span
className="text-sm">{formatShortDate(n.created_at)}</span>,
            },
          ]}
          actionsClassName="w-56"
          actions={(n) =>
            rowActions({
              id: n.id,
              type: "blockedNumber",
              label: toLocalPhilippineMobile(n.phone),
            })
          }
        />
      </TabsContent>
    </ListCard>
  </Tabs>

  <AlertDialog open={!!restoreTarget} onOpenChange={(open) => !open &&
setRestoreTarget(null)}>
    <AlertDialogContent>
      <AlertDialogHeader>
        <AlertDialogTitle>
          Restore {restoreTarget &&
ARCHIVE_ITEM_LABELS[restoreTarget.type]}?
        </AlertDialogTitle>
        <AlertDialogDescription>
          Restore <span className="font-medium
text-foreground">{restoreTarget?.label}</span> to
          the active {restoreTarget &&
ARCHIVE_ITEM_LABELS[restoreTarget.type]} list? It will be
          available in the corresponding admin module again.
        </AlertDialogDescription>
      </AlertDialogHeader>
      <AlertDialogFooter>
        <AlertDialogCancel>Cancel</AlertDialogCancel>
        <AlertDialogAction
onClick={handleRestore}>Restore</AlertDialogAction>
      </AlertDialogFooter>
    </AlertDialogContent>
  </AlertDialog>

  <AlertDialog open={!!deleteTarget} onOpenChange={(open) => !open &&
setDeleteTarget(null)}>
```

```
        <AlertDialogContent>
          <AlertDialogHeader>
            <AlertDialogTitle>
              Permanently delete {deleteTarget &&
ARCHIVE_ITEM_LABELS[deleteTarget.type]}?
            </AlertDialogTitle>
            <AlertDialogDescription>
              <span className="font-medium
text-foreground">{deleteTarget?.label}</span> will be
              removed permanently from the database. This action cannot be undone.
            </AlertDialogDescription>
          </AlertDialogHeader>
          <AlertDialogFooter>
            <AlertDialogCancel>Cancel</AlertDialogCancel>
            <AlertDialogAction
              className="bg-destructive text-destructive-foreground
hover:bg-destructive/90"
              onClick={handleDelete}
            >
              Permanently Delete
            </AlertDialogAction>
          </AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialog>
    </AdminPage>
  );
}

function cleanSearchTerm(value: string) {
  return value
    .trim()
    .replace(/[,%_()]/g, " ")
    .replace(/ /g, " ");
}

function ArchiveDateRangeFilter({
  period,
  dateFrom,
  dateTo,
  customDateFrom,
  customDateTo,
  onPeriodChange,
  onDateRangeChange,
  onCustomDateRangeChange,
}: {
```

```
  period: ArchiveDatePeriod;
  dateFrom: string;
  dateTo: string;
  customDateFrom: string;
  customDateTo: string;
  onPeriodChange: (period: ArchiveDatePeriod) => void;
  onDateRangeChange: (dateFrom: string, dateTo: string) => void;
  onCustomDateRangeChange: (dateFrom: string, dateTo: string) => void;
}) {
  const [open, setOpen] = useState(false);
  const [isPickingCustomRange, setIsPickingCustomRange] = useState(false);
  const [draftRange, setDraftRange] = useState<DateRange>();
  const selectedRange = customDateFrom
    ? {
        from: isoDateToCalendarDate(customDateFrom),
        to: customDateTo ? isoDateToCalendarDate(customDateTo) : undefined,
      }
    : undefined;

  const selectPeriod = (nextPeriod: Exclude<ArchiveDatePeriod, "all" | "custom">)
=> {
    const range = buildArchiveDateRange(nextPeriod);
    onDateRangeChange(range.from, range.to);
    onPeriodChange(nextPeriod);
    setIsPickingCustomRange(false);
    setOpen(false);
  };

  const cancelCustomRange = () => {
    setDraftRange(selectedRange);
    setIsPickingCustomRange(false);
    setOpen(false);
  };

  const applyCustomRange = () => {
    if (!draftRange?.from || !draftRange.to) return;
    const range = calendarRangeToIso(draftRange);
    if (!range) return;
    onDateRangeChange(range.from, range.to);
    onCustomDateRangeChange(range.from, range.to);
    onPeriodChange("custom");
    setIsPickingCustomRange(false);
    setOpen(false);
  };
```

```
  return (
    <Popover
      open={open}
      onOpenChange={(nextOpen) => {
        setOpen(nextOpen);
        if (!nextOpen) setIsPickingCustomRange(false);
      }}
    >
      <PopoverTrigger asChild>
        <Button
          type="button"
          variant="outline"
          aria-label="Select archive date range"
          title={archiveDatePeriodLabel(period, dateFrom, dateTo)}
          className="w-full justify-between gap-2 px-3 text-left font-normal"
        >
          <span className="flex min-w-0 items-center gap-2 truncate">
            <CalendarRange className="size-4 shrink-0" aria-hidden="true" />
            <span className={cn(period === "all" && "text-muted-foreground")}>
              {archiveDatePeriodLabel(period, dateFrom, dateTo)}
            </span>
          </span>
          <ChevronDown className="size-4 shrink-0" aria-hidden="true" />
        </Button>
      </PopoverTrigger>
      <PopoverContent align="start" className="w-72 p-2">
        {isPickingCustomRange ? (
          <div>
            <div className="px-2 pb-2">
              <p className="text-sm font-medium">Custom date range</p>
              <p className="text-xs text-muted-foreground">Select a start date and
an end date.</p>
            </div>
            <Calendar
              mode="range"
              selected={draftRange}
              onSelect={setDraftRange}
              className="mx-auto p-0"
              classNames={{
                nav: "inset-x-auto left-1/2 w-48 -translate-x-1/2 justify-between",
              }}
            />
            <div className="mt-2 grid grid-cols-2 gap-2 px-2 text-xs">
              <ArchiveDateRangeValue label="Start date" value={draftRange?.from}
/>
```

```
            <ArchiveDateRangeValue label="End date" value={draftRange?.to} />
          </div>
          <div className="mt-3 flex justify-end gap-2 border-t border-border pt-3">
            <Button type="button" variant="outline" size="sm"
onClick={cancelCustomRange}>
              Cancel
            </Button>
            <Button
              type="button"
              size="sm"
              onClick={applyCustomRange}
              disabled={!draftRange?.from || !draftRange.to}
            >
              Apply
            </Button>
          </div>
        </div>
      ) : (
        <div className="space-y-1">
          <p className="px-2 pb-1 text-xs font-medium tracking-wide
text-muted-foreground uppercase">
            Archive date
          </p>
          <ArchiveDateOption
            active={period === "today"}
            label="Today"
            onClick={() => selectPeriod("today")}
          />
          <ArchiveDateOption
            active={period === "week"}
            label="This Week"
            onClick={() => selectPeriod("week")}
          />
          <ArchiveDateOption
            active={period === "month"}
            label="This Month"
            onClick={() => selectPeriod("month")}
          />
          <ArchiveDateOption
            active={period === "custom"}
            label="Custom Date Range"
            onClick={() => {
              setDraftRange(selectedRange);
              setIsPickingCustomRange(true);
            }}
```

```
            />
          </div>
        )}
      </PopoverContent>
    </Popover>
  );
}

function ArchiveDateOption({
  active,
  label,
  onClick,
}: {
  active: boolean;
  label: string;
  onClick: () => void;
}) {
  return (
    <Button
      type="button"
      variant="ghost"
      className={cn("h-9 w-full justify-start text-sm", active && "bg-muted")}
      onClick={onClick}
    >
      {label}
    </Button>
  );
}

function ArchiveDateRangeValue({ label, value }: { label: string; value: Date |
undefined }) {
  return (
    <div className="rounded-md border border-border bg-muted/30 px-2
py-1.5">
      <p className="text-[10px] tracking-wide text-muted-foreground
uppercase">{label}</p>
      <p className="mt-0.5 truncate font-medium text-foreground">
        {value ? format(value, "MMM d, yyyy") : "Not selected"}
      </p>
    </div>
  );
}

function buildArchiveDateRange(period: Exclude<ArchiveDatePeriod, "all" |
"custom">): {
```

```
  from: string;
  to: string;
} {
  const today = manilaNow().date;

  if (period === "today") return { from: today, to: today };
  if (period === "week") {
    return manilaWeekDateRange(today);
  }

  return manilaMonthDateRange(today);
}

function archiveDatePeriodLabel(period: ArchiveDatePeriod, dateFrom: string,
dateTo: string) {
  if (period === "all") return "Date range";
  if (period === "today") return "Today";
  if (period === "week") return "This Week";
  if (period === "month") return "This Month";
  return dateFrom ? formatDateRange(dateFrom, dateTo) : "Custom Date
Range";
}

function formatDateRange(from: string, to: string) {
  if (!from) return "Select date range";

  const fromDate = isoDateToCalendarDate(from);
  const fromLabel = format(fromDate, "MMM d, yyyy");
  if (!to) return `${fromLabel} – Select end date`;

  const toDate = isoDateToCalendarDate(to);
  return fromDate.getFullYear() === toDate.getFullYear()
    ? `${format(fromDate, "MMM d")} – ${format(toDate, "MMM d, yyyy")}`
    : `${fromLabel} – ${format(toDate, "MMM d, yyyy")}`;
}

/** Restore and Delete. Records that are kept (completed appointments) have no
Delete. */
function ArchiveRowActions({
  label,
  onRestore,
  onDelete,
  kept,
}: {
  label: string;
```

```
  onRestore: () => void;
  onDelete: () => void;
  kept: boolean;
}) {
  return (
    <>
      <Button size="sm" variant="outline" onClick={onRestore} aria-label={`Restore
${label}`}>
        <RotateCcw /> Restore
      </Button>
      {kept ? (
        <span className="w-20 text-left text-[11px] leading-tight
text-muted-foreground">
          Kept for financial records
        </span>
      ) : (
        <Button
          size="sm"
          variant="destructive"
          onClick={onDelete}
          aria-label={`Permanently delete ${label}`}
        >
          <Trash2 /> Delete
        </Button>
      )}
    </>
  );
}

/** "Whole day", or the archived block's time range. */
function archivedBlockSlotLabel(block: { start_time: string | null; reason: string |
null }) {
  if (!block.start_time) return "Whole day";
  const start = formatTime(String(block.start_time).slice(0, 5));
  const { endTime } = decodeBlockReason(block.reason);
  return endTime ? `${start} – ${formatTime(endTime.slice(0, 5))}` : start;
}

function ArchiveServiceDetails({
  serviceId,
  serviceName,
  description,
}: {
  serviceId: string;
  serviceName: string;
```

```
  description: string | null;
}) {
  const detailsRef = useRef<HTMLParagraphElement>(null);
  const [expanded, setExpanded] = useState(false);
  const [canExpand, setCanExpand] = useState(false);
  const details = description?.trim() ?? "";
  const detailsId = `archive-service-details-${serviceId}`;

  useEffect(() => {
    const element = detailsRef.current;
    if (!element || !details) return;

    const checkOverflow = () => {
      if (!expanded) setCanExpand(element.scrollHeight > element.clientHeight +
1);
    };

    checkOverflow();
    if (typeof ResizeObserver === "undefined") return;

    const observer = new ResizeObserver(checkOverflow);
    observer.observe(element);
    return () => observer.disconnect();
  }, [details, expanded]);

  if (!details) return "-";

  return (
    <div className="w-full max-w-[22rem] text-left">
      <p
        id={detailsId}
        ref={detailsRef}
        className={cn("break-words whitespace-pre-wrap", !expanded &&
"line-clamp-3")}
      >
        {details}
      </p>
      {canExpand && (
        <Button
          type="button"
          variant="link"
          size="sm"
          className="mt-1 h-auto p-0 text-xs"
          onClick={() => setExpanded((current) => !current)}
          aria-controls={detailsId}
```

```
          aria-expanded={expanded}
          aria-label={`${expanded ? "Collapse" : "Expand"} details for
${serviceName}`}
        >
          {expanded ? "See Less" : "See More"}
        </Button>
      )}
    </div>
  );
}
```

src/routes/_authenticated/admin/availability.tsx

```
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import { createFileRoute } from "@tanstack/react-router";
import { format } from "date-fns";
import {
  CalendarRange,
  CalendarSearch,
  Check,
  ChevronDown,
  Filter,
  Loader2,
  Pencil,
  Plus,
  RotateCcw,
  Trash2,
} from "lucide-react";
import { useMemo, useState } from "react";
import type { DateRange } from "react-day-picker";

import {
  AdminPage,
  DataList,
  InlineAlert,
  ListCard,
  ListCardTitle,
  ListEmpty,
  ListToolbar,
  ToolbarFilters,
} from "@/components/admin/data-list";
import { PageHeader } from "@/components/admin/page-header";
import {
  AlertDialog,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
} from "@/components/ui/alert-dialog";
import { Button } from "@/components/ui/button";
import { Calendar } from "@/components/ui/calendar";
import { FieldError } from "@/components/ui/field-error";
import {
  Dialog,
```

```
  DialogContent,
  DialogDescription,
  DialogFooter,
  DialogHeader,
  DialogTitle,
} from "@/components/ui/dialog";
import { Label } from "@/components/ui/label";
import { Popover, PopoverContent, PopoverTrigger } from
"@/components/ui/popover";
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@/components/ui/select";
import { supabase } from "@/integrations/supabase/client";
import {
  calendarDateToIso,
  calendarRangeToIso,
  isoDateToCalendarDate,
  manilaMonthDateRange,
  manilaWeekDateRange,
  normalizeIsoDateRange,
} from "@/lib/admin-date-range";
import { formatDateLong, formatTime, manilaNow, shopTimeOptions } from
"@/lib/shop";
import { cn } from "@/lib/utils";

export const Route = createFileRoute("/_authenticated/admin/availability")({
  component: AvailabilityPage,
});

const SHIFT_PRESETS = {
  "whole-day": { start: "08:00", end: "17:00", label: "Whole Day (8:00 AM – 5:00
PM)" },
  morning: { start: "08:00", end: "12:00", label: "Morning (8:00 AM – 12:00 PM)" },
  afternoon: { start: "13:00", end: "17:00", label: "Afternoon (1:00 PM – 5:00 PM)"
},
} as const;

const PRESET_LABELS: Record<string, string> = {};
for (const preset of Object.values(SHIFT_PRESETS)) {
  PRESET_LABELS[`${preset.start}|${preset.end}`] = preset.label;
}
```

```
type ScheduleWithCrew = {
  id: string;
  crew_id: string;
  day_of_week: number;
  start_time: string | null;
  end_time: string | null;
  is_working: boolean;
  schedule_date: string | null;
  note: string | null;
  crew_members?: { name: string; role: string } | null;
};

type WorkingMechanic = {
  id: string;
  crew_id: string;
  schedule_date: string;
  name: string;
  role: string;
  start_time: string | null;
  end_time: string | null;
  is_date_assignment: boolean;
};

type AssignmentFilter = {
  crewId: string;
  role: string;
  period: "weekly" | "monthly" | "custom";
  customStart: string;
  customEnd: string;
};

type CrewMember = {
  id: string;
  name: string;
  role: string;
};

/** A booking, or a later day of a multi-day booking, assigned to a mechanic. */
type ShiftWork = {
  key: string;
  referenceCode: string;
  customerName: string;
  startTime: string;
  durationMinutes: number;
```

```
};

/** What change_crew_schedule returns: the bookings it gave to other
mechanics. */
type ShiftChangeResult = {
  moved: { reference_code: string; start_time: string; mechanic: string }[];
};

const ACTIVE_WORK_STATUSES = ["pending", "confirmed", "in_progress",
"rescheduled"] as const;

async function fetchShiftWork(crewId: string, date: string): Promise<ShiftWork[]>
{
  const [appointments, continuations] = await Promise.all([
    supabase
      .from("appointments")
      .select("id,reference_code,customer_name,start_time,booking_duration_min
utes")
      .eq("assigned_crew_id", crewId)
      .eq("appointment_date", date)
      .eq("is_archived", false)
      .is("rescheduled_to_appointment_id", null)
      .in("status", [...ACTIVE_WORK_STATUSES]),
    supabase
      .from("appointment_continuations")
      .select(
        "id,start_time,booking_duration_minutes,appointments!inner(reference_cod
e,customer_name)",
      )
      .eq("assigned_crew_id", crewId)
      .eq("appointment_date", date)
      .eq("appointments.is_archived", false)
      .is("appointments.rescheduled_to_appointment_id", null)
      .in("appointments.status", [...ACTIVE_WORK_STATUSES]),
  ]);
  if (appointments.error) throw appointments.error;
  if (continuations.error) throw continuations.error;
  return [
    ...(appointments.data ?? []).map((appointment) => ({
      key: appointment.id,
      referenceCode: appointment.reference_code,
      customerName: appointment.customer_name,
      startTime: String(appointment.start_time).slice(0, 5),
      durationMinutes: appointment.booking_duration_minutes ?? 0,
    })),
```

```
    ...(continuations.data ?? []).map((continuation) => ({
      key: continuation.id,
      referenceCode: continuation.appointments.reference_code,
      customerName: continuation.appointments.customer_name,
      startTime: String(continuation.start_time).slice(0, 5),
      durationMinutes: continuation.booking_duration_minutes,
    })),
  ].sort((first, second) => first.startTime.localeCompare(second.startTime));
}

function minutesOf(time: string) {
  const [hours, minutes] = time.split(":").map(Number);
  return (hours ?? 0) * 60 + (minutes ?? 0);
}

/** Work that no longer fits between the new start and end. */
function workOutsideHours(work: ShiftWork[], start: string, end: string) {
  return work.filter(
    (item) =>
      minutesOf(item.startTime) < minutesOf(start) ||
      minutesOf(item.startTime) + item.durationMinutes > minutesOf(end),
  );
}

function shiftLabel(start: string | null, end: string | null) {
  if (!start || !end) return "Not set";
  return PRESET_LABELS[`${start}|${end}`] ?? `${formatTime(start)} –
${formatTime(end)}`;
}

function movedSummary(result: ShiftChangeResult) {
  return result.moved
    .map(
      (booking) =>
        ` ${booking.reference_code} (${formatTime(booking.start_time.slice(0, 5))})
now goes to ${booking.mechanic}.`,
    )
    .join("");
}

function shiftChangeError(error: { message?: string; hint?: string }, action:
"remove" | "edit") {
  if (error.hint === "crew_shift_conflict") {
    return `${error.message} Reschedule or cancel that booking first, or ${
      action === "remove" ? "keep this shift" : "choose hours that still cover it"
```

```
    }.`;
  }
  if (/no longer exists|must end after it starts/i.test(error.message ?? "")) {
    return error.message ?? "";
  }
  console.error("Shift change failed:", error);
  return "Could not change this shift. Please try again.";
}

function listDays(dates: string[]) {
  const labels = dates.map((date) => format(dateFromKey(date), "MMM d"));
  return labels.length > 1
    ? `${labels.slice(0, -1).join(", ")} and ${labels.at(-1)}`
    : (labels[0] ?? "");
}

function dateToKey(date: Date) {
  return calendarDateToIso(date);
}

function dateFromKey(value: string) {
  return isoDateToCalendarDate(value);
}

function assignmentPeriodBounds(filter: AssignmentFilter) {
  if (filter.period === "custom") {
    return normalizeIsoDateRange(filter.customStart, filter.customEnd);
  }

  const today = manilaNow().date;
  return filter.period === "monthly" ? manilaMonthDateRange(today) :
manilaWeekDateRange(today);
}

function AvailabilityDateRangeFilter({
  period,
  dateFrom,
  dateTo,
  onPeriodChange,
  onCustomDateRangeChange,
}: {
  period: AssignmentFilter["period"];
  dateFrom: string;
  dateTo: string;
  onPeriodChange: (period: AssignmentFilter["period"]) => void;
```

```
  onCustomDateRangeChange: (dateFrom: string, dateTo: string) => void;
}) {
  const [open, setOpen] = useState(false);
  const [isPickingCustomRange, setIsPickingCustomRange] = useState(false);
  const [draftRange, setDraftRange] = useState<DateRange>();
  const selectedRange = dateFrom
    ? { from: dateFromKey(dateFrom), to: dateTo ? dateFromKey(dateTo) :
undefined }
    : undefined;

  const selectPeriod = (nextPeriod: Exclude<AssignmentFilter["period"],
"custom">) => {
    onPeriodChange(nextPeriod);
    setIsPickingCustomRange(false);
    setOpen(false);
  };

  const cancelCustomRange = () => {
    setDraftRange(selectedRange);
    setIsPickingCustomRange(false);
    setOpen(false);
  };

  const applyCustomRange = () => {
    if (!draftRange?.from || !draftRange.to) return;
    const range = calendarRangeToIso(draftRange);
    if (!range) return;
    onCustomDateRangeChange(range.from, range.to);
    onPeriodChange("custom");
    setIsPickingCustomRange(false);
    setOpen(false);
  };

  return (
    <Popover
      open={open}
      onOpenChange={(nextOpen) => {
        setOpen(nextOpen);
        if (!nextOpen) setIsPickingCustomRange(false);
      }}
    >
      <PopoverTrigger asChild>
        <Button
          type="button"
          variant="outline"
```

```
          aria-label="Select availability date range"
          title={availabilityDatePeriodLabel(period, dateFrom, dateTo)}
          className="w-full justify-between gap-2 px-3 text-left font-normal"
        >
          <span className="flex min-w-0 items-center gap-2 truncate">
            <CalendarRange className="size-4 shrink-0" aria-hidden="true" />
            {availabilityDatePeriodLabel(period, dateFrom, dateTo)}
          </span>
          <ChevronDown className="size-4 shrink-0" aria-hidden="true" />
        </Button>
      </PopoverTrigger>
      <PopoverContent align="start" className="w-72 p-2">
        {isPickingCustomRange ? (
          <div>
            <div className="px-2 pb-2">
              <p className="text-sm font-medium">Custom date range</p>
              <p className="text-xs text-muted-foreground">Select a start date and
an end date.</p>
            </div>
            <Calendar
              mode="range"
              selected={draftRange}
              onSelect={setDraftRange}
              className="mx-auto p-0"
              classNames={{
                nav: "inset-x-auto left-1/2 w-48 -translate-x-1/2 justify-between",
              }}
            />
            <div className="mt-2 grid grid-cols-2 gap-2 px-2 text-xs">
              <AvailabilityDateRangeValue label="Start date"
value={draftRange?.from} />
              <AvailabilityDateRangeValue label="End date" value={draftRange?.to}
/>
            </div>
            <div className="mt-3 flex justify-end gap-2 border-t border-border pt-3">
              <Button type="button" variant="outline" size="sm"
onClick={cancelCustomRange}>
                Cancel
              </Button>
              <Button
                type="button"
                size="sm"
                onClick={applyCustomRange}
                disabled={!draftRange?.from || !draftRange.to}
              >
```

```
              Apply
            </Button>
          </div>
        </div>
      ) : (
        <div className="space-y-1">
          <p className="px-2 pb-1 text-xs font-medium tracking-wide
text-muted-foreground uppercase">
            Date range
          </p>
          <AvailabilityDateOption
            active={period === "weekly"}
            label="This Week"
            onClick={() => selectPeriod("weekly")}
          />
          <AvailabilityDateOption
            active={period === "monthly"}
            label="This Month"
            onClick={() => selectPeriod("monthly")}
          />
          <AvailabilityDateOption
            active={period === "custom"}
            label="Custom Date Range"
            onClick={() => {
              setDraftRange(selectedRange);
              setIsPickingCustomRange(true);
            }}
          />
        </div>
      )}
    </PopoverContent>
  </Popover>
 );
}

function AvailabilityDateOption({
 active,
 label,
 onClick,
}: {
 active: boolean;
 label: string;
 onClick: () => void;
}) {
 return (
```

```
    <Button
      type="button"
      variant="ghost"
      className={cn("h-9 w-full justify-start text-sm", active && "bg-muted")}
      onClick={onClick}
    >
      {label}
    </Button>
  );
}

function AvailabilityDateRangeValue({ label, value }: { label: string; value: Date |
undefined }) {
  return (
    <div className="rounded-md border border-border bg-muted/30 px-2
py-1.5">
      <p className="text-[10px] tracking-wide text-muted-foreground
uppercase">{label}</p>
      <p className="mt-0.5 truncate font-medium text-foreground">
        {value ? format(value, "MMM d, yyyy") : "Not selected"}
      </p>
    </div>
  );
}

/** "This week · All crew members · All roles", for the results header. */
function assignmentFilterSummary(filter: AssignmentFilter, crew: CrewMember[])
{
  const period =
    filter.period === "custom"
      ? availabilityDatePeriodLabel(filter.period, filter.customStart, filter.customEnd)
      : filter.period === "monthly"
        ? "This month"
        : "This week";
  const member =
    filter.crewId === "all"
      ? "All crew members"
      : (crew.find((item) => item.id === filter.crewId)?.name ?? "Selected crew
member");
  const role = filter.role === "all" ? "All roles" : filter.role;
  return [period, member, role].join(" · ");
}

function availabilityDatePeriodLabel(
  period: AssignmentFilter["period"],
```

```
  dateFrom: string,
  dateTo: string,
) {
  if (period === "weekly") return "This Week";
  if (period === "monthly") return "This Month";
  if (!dateFrom) return "Custom Date Range";

  const fromDate = dateFromKey(dateFrom);
  const fromLabel = format(fromDate, "MMM d, yyyy");
  if (!dateTo) return `${fromLabel} – Select end date`;

  const toDate = dateFromKey(dateTo);
  return fromDate.getFullYear() === toDate.getFullYear()
    ? `${format(fromDate, "MMM d")} – ${format(toDate, "MMM d, yyyy")}`
    : `${fromLabel} – ${format(toDate, "MMM d, yyyy")}`;
}

function AvailabilityPage() {
  const qc = useQueryClient();
  const [filterCrewId, setFilterCrewId] = useState("all");
  const [filterRole, setFilterRole] = useState("all");
  const [filterPeriod, setFilterPeriod] =
useState<AssignmentFilter["period"]>("weekly");
  const [customPeriodStart, setCustomPeriodStart] = useState("");
  const [customPeriodEnd, setCustomPeriodEnd] = useState("");
  const [appliedFilter, setAppliedFilter] = useState<AssignmentFilter | null>(null);
  const [dialogOpen, setDialogOpen] = useState(false);
  const [assignDates, setAssignDates] = useState<Date[]>([]);
  const [assignMechanic, setAssignMechanic] = useState<string>("");
  const [assignShift, setAssignShift] = useState("whole-day");
  const [customStart, setCustomStart] = useState("");
  const [customEnd, setCustomEnd] = useState("");
  const [filterErrors, setFilterErrors] = useState<
    Partial<Record<"customStart" | "customEnd", string | undefined>>
  >({});
  const [assignmentErrors, setAssignmentErrors] = useState<
    Partial<Record<"dates" | "mechanic" | "customStart" | "customEnd", string |
undefined>>
  >({});
  const [removeTarget, setRemoveTarget] = useState<WorkingMechanic |
null>(null);
  const [editTarget, setEditTarget] = useState<WorkingMechanic | null>(null);
  const [editShift, setEditShift] = useState("whole-day");
  const [editStart, setEditStart] = useState("");
  const [editEnd, setEditEnd] = useState("");
```

```
const [shiftError, setShiftError] = useState<string | null>(null);
const [shiftNotice, setShiftNotice] = useState<string | null>(null);

const crew = useQuery({
  queryKey: ["crew-active"],
  queryFn: async () => {
    const { data, error } = await supabase
      .from("crew_members")
      .select("*")
      .eq("is_active", true)
      .eq("is_archived", false)
      .order("name");
    if (error) throw error;
    return Array.from(
      new Map((data ?? []).map((c) => [c.name.trim(), c])).values(),
    ) as CrewMember[];
  },
});

const schedules = useQuery({
  queryKey: ["crew-schedules-all"],
  queryFn: async () => {
    const { data, error } = await supabase
      .from("crew_schedules")
      .select("*,crew_members!inner(name,role)")
      .order("crew_id")
      .order("schedule_date", { ascending: true, nullsFirst: false })
      .order("day_of_week");
    if (error) throw error;
    return (data ?? []) as ScheduleWithCrew[];
  },
});

const saveSchedule = useMutation({
  mutationFn: async (payload: {
    crew_id: string;
    schedule_dates: string[];
    start_time: string;
    end_time: string;
    is_working: boolean;
    note?: string;
  }) => {
    const { error } = await supabase.from("crew_schedules").upsert(
      payload.schedule_dates.map((scheduleDate) => {
        const [y, m, d] = scheduleDate.split("-").map(Number);
```

```
          return {
            crew_id: payload.crew_id,
            schedule_date: scheduleDate,
            day_of_week: new Date(Date.UTC(y ?? 1970, (m ?? 1) - 1, d ??
1)).getUTCDay(),
            start_time: payload.is_working ? `${payload.start_time}:00` : null,
            end_time: payload.is_working ? `${payload.end_time}:00` : null,
            is_working: payload.is_working,
            note: payload.note || null,
          };
        }),
        // A day marked as not working is replaced by the new shift.
        { onConflict: "crew_id,schedule_date" },
      );
      if (error) throw error;
    },
    onSuccess: () => {
      qc.invalidateQueries({ queryKey: ["crew-schedules-all"] });
    },
    onError: (err: Error) => {
      const msg = err.message.toLowerCase();
      if (msg.includes("unique") || msg.includes("duplicate")) {
        setAssignmentErrors({
          dates: "A schedule for this crew on this date already exists. Remove or edit
it first.",
        });
      } else if (msg.includes("foreign")) {
        setAssignmentErrors({
          mechanic: "Referenced crew member no longer exists. Refresh the page
and try again.",
        });
      } else {
        console.error("Schedule save failed:", err);
        setAssignmentErrors({
          dates: "Could not save the schedule. Please try again.",
        });
      }
    },
  });

  const shiftTarget = removeTarget ?? editTarget;
  const shiftWork = useQuery({
    queryKey: ["crew-shift-work", shiftTarget?.crew_id,
shiftTarget?.schedule_date],
    enabled: Boolean(shiftTarget),
```

```
    queryFn: () => fetchShiftWork(shiftTarget!.crew_id,
shiftTarget!.schedule_date),
  });

  const changeShift = useMutation({
    mutationFn: async (input: {
      scheduleId: string;
      remove: boolean;
      start?: string;
      end?: string;
    }) => {
      const { data, error } = await supabase.rpc("change_crew_schedule", {
        p_schedule_id: input.scheduleId,
        p_remove: input.remove,
        p_start_time: input.start ? `${input.start}:00` : null,
        p_end_time: input.end ? `${input.end}:00` : null,
      });
      if (error) throw error;
      return data as unknown as ShiftChangeResult;
    },
    onSuccess: () => {
      qc.invalidateQueries({ queryKey: ["crew-schedules-all"] });
      qc.invalidateQueries({ queryKey: ["crew-shift-work"] });
      qc.invalidateQueries({ queryKey: ["admin-appointments"] });
      qc.invalidateQueries({ queryKey: ["admin-dashboard"], exact: false });
    },
  });

  function openRemove(row: WorkingMechanic) {
    setShiftError(null);
    setShiftNotice(null);
    setRemoveTarget(row);
  }

  function openEdit(row: WorkingMechanic) {
    const preset = Object.entries(SHIFT_PRESETS).find(
      ([, hours]) => hours.start === row.start_time && hours.end === row.end_time,
    )?.[0];
    setEditShift(preset ?? "custom");
    setEditStart(preset ? "" : (row.start_time ?? ""));
    setEditEnd(preset ? "" : (row.end_time ?? ""));
    setShiftError(null);
    setShiftNotice(null);
    setEditTarget(row);
  }
```

```
  /** The hours chosen in the edit dialog, or null if they are incomplete. */
  function chosenEditHours() {
    if (editShift !== "custom") {
      const preset = SHIFT_PRESETS[editShift as keyof typeof
SHIFT_PRESETS];
      return { start: preset.start, end: preset.end };
    }
    return editStart && editEnd && editStart < editEnd ? { start: editStart, end:
editEnd } : null;
  }

  async function confirmRemove() {
    if (!removeTarget) return;
    setShiftError(null);
    try {
      const result = await changeShift.mutateAsync({ scheduleId: removeTarget.id,
remove: true });
      setShiftNotice(
        `Removed ${removeTarget.name}'s shift on
${formatDateLong(removeTarget.schedule_date)}.${movedSummary(result)}`,
      );
      setRemoveTarget(null);
    } catch (error) {
      setShiftError(shiftChangeError(error as { message?: string; hint?: string },
"remove"));
    }
  }

  async function saveEdit() {
    if (!editTarget) return;
    const hours = chosenEditHours();
    if (!hours) {
      setShiftError(
        !editStart || !editEnd
          ? "Select a start and an end time."
          : "End time must be after start time.",
      );
      return;
    }
    if (hours.start === editTarget.start_time && hours.end ===
editTarget.end_time) {
      setEditTarget(null);
      return;
    }
```

```
    setShiftError(null);
    try {
      const result = await changeShift.mutateAsync({
        scheduleId: editTarget.id,
        remove: false,
        ...hours,
      });
      setShiftNotice(
        `Changed ${editTarget.name}'s shift on
${formatDateLong(editTarget.schedule_date)} to ${shiftLabel(hours.start,
hours.end)}.${movedSummary(result)}`,
      );
      setEditTarget(null);
    } catch (error) {
      setShiftError(shiftChangeError(error as { message?: string; hint?: string },
"edit"));
    }
  }

  const editHours = chosenEditHours();
  const editAffected = editHours
    ? workOutsideHours(shiftWork.data ?? [], editHours.start, editHours.end)
    : [];

  const assignDateStrs = useMemo(() => assignDates.map(dateToKey),
[assignDates]);
  const roleOptions = useMemo(
    () =>
      Array.from(
        new Set(
          (schedules.data ?? [])
            .map((schedule) => schedule.crew_members?.role)
            .filter((role): role is string => Boolean(role)),
        ),
      ).sort(),
    [schedules.data],
  );

  const filteredAssignments = useMemo(() => {
    if (!appliedFilter) return null;
    const { from, to } = assignmentPeriodBounds(appliedFilter);
    const crewMap = new Map((crew.data ?? []).map((member) => [member.id,
member]));

    return (schedules.data ?? [])
```

```
      .filter(
        (schedule) =>
          schedule.is_working &&
          schedule.schedule_date &&
          schedule.schedule_date >= from &&
          schedule.schedule_date <= to &&
          (appliedFilter.crewId === "all" || schedule.crew_id === appliedFilter.crewId)
&&
          (appliedFilter.role === "all" || schedule.crew_members?.role ===
appliedFilter.role),
      )
      .map((schedule) => ({
        id: schedule.id,
        crew_id: schedule.crew_id,
        schedule_date: schedule.schedule_date as string,
        name: schedule.crew_members?.name ??
crewMap.get(schedule.crew_id)?.name ?? "Unknown",
        role: schedule.crew_members?.role ?? "Mechanic",
        start_time: schedule.start_time ? String(schedule.start_time).slice(0, 5) :
null,
        end_time: schedule.end_time ? String(schedule.end_time).slice(0, 5) : null,
        is_date_assignment: true,
      }))
      .sort(
        (first, second) =>
          first.schedule_date.localeCompare(second.schedule_date) ||
          first.name.localeCompare(second.name),
      ) as WorkingMechanic[];
  }, [appliedFilter, crew.data, schedules.data]);

  function applyAssignmentFilter() {
    if (filterPeriod === "custom") {
      const nextErrors: Partial<Record<"customStart" | "customEnd", string>> = {};
      if (!customPeriodStart || !customPeriodEnd) {
        if (!customPeriodStart) nextErrors.customStart = "Choose a custom period
start date.";
        if (!customPeriodEnd) nextErrors.customEnd = "Choose a custom period
end date.";
      }
      if (customPeriodStart && customPeriodEnd && customPeriodStart >
customPeriodEnd) {
        nextErrors.customEnd = "The custom period end must be after the start
date.";
      }
      setFilterErrors(nextErrors);
```

```
      if (Object.keys(nextErrors).length) {
        return;
      }
    }
    setAppliedFilter({
      crewId: filterCrewId,
      role: filterRole,
      period: filterPeriod,
      customStart: customPeriodStart,
      customEnd: customPeriodEnd,
    });
  }

  function clearAssignmentFilter() {
    setFilterCrewId("all");
    setFilterRole("all");
    setFilterPeriod("weekly");
    setCustomPeriodStart("");
    setCustomPeriodEnd("");
    setAppliedFilter(null);
  }

  const handleSave = async () => {
    const nextErrors: Partial<Record<"dates" | "mechanic" | "customStart" |
"customEnd", string>> =
      {};
    if (assignDateStrs.length === 0) nextErrors.dates = "Select at least one
working day.";
    if (!assignMechanic) nextErrors.mechanic = "Select a mechanic.";
    if (Object.keys(nextErrors).length) {
      setAssignmentErrors(nextErrors);
      return;
    }

    const workingDates = new Set(
      (schedules.data ?? [])
        .filter(
          (schedule) =>
            schedule.crew_id === assignMechanic && schedule.is_working &&
schedule.schedule_date,
        )
        .map((schedule) => schedule.schedule_date as string),
    );
    const alreadyWorking = assignDateStrs.filter((date) =>
workingDates.has(date)).sort();
```

```
    if (alreadyWorking.length > 0) {
      const mechanicName =
        (crew.data ?? []).find((member) => member.id === assignMechanic)?.name
?? "This mechanic";
      setAssignmentErrors({
        dates: `${mechanicName} already has a shift on
${listDays(alreadyWorking)}. Unselect ${
          alreadyWorking.length === 1 ? "that day" : "those days"
        }, or use Edit on the existing shift.`,
      });
      return;
    }

    let startTime: string;
    let endTime: string;

    if (assignShift === "custom") {
      if (!customStart || !customEnd) {
        setAssignmentErrors({
          ...(customStart ? {} : { customStart: "Select a start time." }),
          ...(customEnd ? {} : { customEnd: "Select an end time." }),
        });
        return;
      }
      if (customStart >= customEnd) {
        setAssignmentErrors({ customEnd: "End time must be after start time." });
        return;
      }
      startTime = customStart;
      endTime = customEnd;
    } else {
      const preset = SHIFT_PRESETS[assignShift as keyof typeof
SHIFT_PRESETS];
      startTime = preset.start;
      endTime = preset.end;
    }

    try {
      setAssignmentErrors({});
      await saveSchedule.mutateAsync({
        crew_id: assignMechanic,
        schedule_dates: assignDateStrs,
        start_time: startTime,
        end_time: endTime,
        is_working: true,
```

```
      });
      setAssignMechanic("");
      setAssignDates([]);
      setAssignShift("whole-day");
      setCustomStart("");
      setCustomEnd("");
      setDialogOpen(false);
    } catch {
      // Error is shown inline by the mutation callback.
    }
  };

  const openAssignDialog = () => {
    setAssignMechanic("");
    setAssignShift("whole-day");
    setCustomStart("");
    setCustomEnd("");
    setAssignDates([]);
    setDialogOpen(true);
  };

  return (
    <AdminPage>
      <PageHeader
        title="Crew Scheduling"
        description="Find and manage crew assignments by crew member, role,
and period."
        action={
          <Button className="font-display uppercase"
onClick={openAssignDialog}>
            <Plus /> Assign crew
          </Button>
        }
      />
      {shiftNotice && (
        <p
          role="status"
          className="mb-4 flex items-start gap-2 rounded-lg border
border-emerald-500/40 bg-emerald-500/5 p-3 text-sm text-emerald-700
dark:text-emerald-400"
        >
          <Check className="mt-0.5 size-4 shrink-0" aria-hidden="true" />
          <span>{shiftNotice}</span>
        </p>
      )}
```

```
      <ListToolbar
        summary={
          filteredAssignments &&
          `${filteredAssignments.length} ${filteredAssignments.length === 1 ?
"assignment" : "assignments"}`
        }
      >
        <ToolbarFilters>
          <Select value={filterCrewId} onValueChange={setFilterCrewId}>
            <SelectTrigger aria-label="Filter by crew member" className="w-full
sm:w-48">
              <SelectValue placeholder="All crew members" />
            </SelectTrigger>
            <SelectContent>
              <SelectItem value="all">All crew members</SelectItem>
              {(crew.data ?? []).map((member) => (
                <SelectItem key={member.id} value={member.id}>
                  {member.name}
                </SelectItem>
              ))}
            </SelectContent>
          </Select>
          <Select value={filterRole} onValueChange={setFilterRole}>
            <SelectTrigger aria-label="Filter by role" className="w-full sm:w-40">
              <SelectValue placeholder="All roles" />
            </SelectTrigger>
            <SelectContent>
              <SelectItem value="all">All roles</SelectItem>
              {roleOptions.map((role) => (
                <SelectItem key={role} value={role}>
                  {role}
                </SelectItem>
              ))}
            </SelectContent>
          </Select>
          <div className="col-span-2 sm:w-56">
            <AvailabilityDateRangeFilter
              period={filterPeriod}
              dateFrom={customPeriodStart}
              dateTo={customPeriodEnd}
              onPeriodChange={setFilterPeriod}
              onCustomDateRangeChange={(dateFrom, dateTo) => {
                setCustomPeriodStart(dateFrom);
                setCustomPeriodEnd(dateTo);
```

```
              setFilterErrors({});
            }}
          />
        </div>
        <Button type="button" onClick={applyAssignmentFilter}>
          <Filter /> Apply filters
        </Button>
        <Button type="button" variant="outline" onClick={clearAssignmentFilter}>
          <RotateCcw /> Reset
        </Button>
      </ToolbarFilters>
    </ListToolbar>
    {filterPeriod === "custom" && (
      <FieldError
        message={filterErrors.customStart ?? filterErrors.customEnd}
        className="-mt-2 mb-4"
      />
    )}

    <ListCard
      header={
        <ListCardTitle
          title="Crew assignments"
          description={
            appliedFilter
              ? assignmentFilterSummary(appliedFilter, crew.data ?? [])
              : "Choose your crew, role, and period, then apply the filters."
          }
        />
      }
    >
      {filteredAssignments === null ? (
        <ListEmpty
          icon={CalendarSearch}
          title="No search yet"
          description="Choose your crew, role, and period, then apply the filters to
see assignments."
        />
      ) : (
        <DataList
          label="Crew assignments"
          rows={filteredAssignments}
          rowKey={(w) => w.id}
          loading={schedules.isLoading}
          error={
```

```
            schedules.isError
              ? {
                  message: "Could not load crew schedules.",
                  onRetry: () => void schedules.refetch(),
                }
              : null
          }
          empty={
            <ListEmpty
              icon={CalendarSearch}
              title="No crew assignments match these filters."
              description="Try another crew member, role, or period, or assign crew
to these days."
              action={
                <Button size="sm" onClick={openAssignDialog}>
                  <Plus /> Assign crew
                </Button>
              }
            />
          }
          columns={[
            {
              key: "date",
              header: "Date",
              card: "title",
              className: "w-52",
              cell: (w) => <span
className="font-medium">{formatDateLong(w.schedule_date)}</span>,
            },
            {
              key: "name",
              header: "Mechanic",
              cell: (w) => (
                <span className="text-sm">
                  {w.name}
                  <span className="block text-xs
text-muted-foreground">{w.role}</span>
                </span>
              ),
            },
            {
              key: "shift",
              header: "Shift",
              card: "wide",
              className: "w-56",
```

```
            cell: (w) => (
              <span className="text-sm">{shiftLabel(w.start_time,
w.end_time)}</span>
            ),
          },
        ]}
        // Icons only on small laptops, where the labels would squeeze the table.
        actionsClassName="w-32 xl:w-52"
        actions={(w) =>
          w.is_date_assignment ? (
            <>
              <Button
                size="sm"
                variant="outline"
                onClick={() => openEdit(w)}
                aria-label={`Edit ${w.name}'s shift on
${formatDateLong(w.schedule_date)}`}
              >
                <Pencil /> <span className="lg:max-xl:sr-only">Edit</span>
              </Button>
              <Button
                size="sm"
                variant="outline"
                className="border-destructive/40 text-destructive
hover:bg-destructive/10 hover:text-destructive"
                onClick={() => openRemove(w)}
                aria-label={`Remove ${w.name}'s shift on
${formatDateLong(w.schedule_date)}`}
              >
                <Trash2 /> <span className="lg:max-xl:sr-only">Remove</span>
              </Button>
            </>
          ) : null
        }
      />
    )}
  </ListCard>

  <AlertDialog
    open={Boolean(removeTarget)}
    onOpenChange={(nextOpen) => {
      if (!nextOpen && !changeShift.isPending) setRemoveTarget(null);
    }}
  >
    <AlertDialogContent>
```

```
        <AlertDialogHeader>
          <AlertDialogTitle>Remove this shift?</AlertDialogTitle>
          <AlertDialogDescription>
            {removeTarget &&
              `${removeTarget.name} ·
${formatDateLong(removeTarget.schedule_date)} ·
${shiftLabel(removeTarget.start_time, removeTarget.end_time)}`}
          </AlertDialogDescription>
        </AlertDialogHeader>
        <ShiftWorkNotice
          loading={shiftWork.isLoading}
          failed={shiftWork.isError}
          affected={shiftWork.data ?? []}
          emptyMessage={`No bookings are assigned to ${removeTarget?.name
?? "this mechanic"} on this day.`}
        />
        {shiftError && <InlineAlert className="mb-0">{shiftError}</InlineAlert>}
        <AlertDialogFooter>
          <AlertDialogCancel
disabled={changeShift.isPending}>Cancel</AlertDialogCancel>
          <Button
            variant="destructive"
            onClick={() => void confirmRemove()}
            disabled={changeShift.isPending || shiftWork.isLoading ||
shiftWork.isError}
          >
            {changeShift.isPending && <Loader2 className="animate-spin" />}
            {(shiftWork.data?.length ?? 0) > 0 ? "Remove and move bookings" :
"Remove shift"}
          </Button>
        </AlertDialogFooter>
      </AlertDialogContent>
    </AlertDialog>

    <Dialog
      open={Boolean(editTarget)}
      onOpenChange={(nextOpen) => {
        if (!nextOpen && !changeShift.isPending) setEditTarget(null);
      }}
    >
      <DialogContent>
        <DialogHeader>
          <DialogTitle className="font-display uppercase">Edit shift</DialogTitle>
          <DialogDescription>
            {editTarget && `${editTarget.name} ·
```

```
${formatDateLong(editTarget.schedule_date)}`}
          </DialogDescription>
        </DialogHeader>
        <div className="space-y-4">
          <ShiftHoursFields
            shift={editShift}
            start={editStart}
            end={editEnd}
            onShiftChange={(value) => {
              setEditShift(value);
              setEditStart(value === "custom" ? (editTarget?.start_time ?? "") : "");
              setEditEnd(value === "custom" ? (editTarget?.end_time ?? "") : "");
              setShiftError(null);
            }}
            onStartChange={(value) => {
              setEditStart(value);
              setShiftError(null);
            }}
            onEndChange={(value) => {
              setEditEnd(value);
              setShiftError(null);
            }}
          />
          <ShiftWorkNotice
            loading={shiftWork.isLoading}
            failed={shiftWork.isError}
            affected={editAffected}
            emptyMessage={`Every booking assigned to ${editTarget?.name ??
"this mechanic"} on this day fits these hours.`}
          />
          {shiftError && <InlineAlert className="mb-0">{shiftError}</InlineAlert>}
        </div>
        <DialogFooter>
          <Button
            type="button"
            variant="outline"
            onClick={() => setEditTarget(null)}
            disabled={changeShift.isPending}
          >
            Cancel
          </Button>
          <Button
            onClick={() => void saveEdit()}
            disabled={changeShift.isPending || shiftWork.isLoading ||
shiftWork.isError}
```

```
          >
            {changeShift.isPending && <Loader2 className="animate-spin" />}
            Save shift
          </Button>
        </DialogFooter>
      </DialogContent>
    </Dialog>

    {/* Assign crew dialog */}
    <Dialog open={dialogOpen} onOpenChange={setDialogOpen}>
      <DialogContent>
        <DialogHeader>
          <DialogTitle className="font-display uppercase">Assign
crew</DialogTitle>
        </DialogHeader>
        <div className="space-y-4">
          <div className="space-y-1.5">
            <Label>Working days</Label>
            <Calendar
              mode="multiple"
              selected={assignDates}
              onSelect={(dates) => {
                setAssignDates(dates ?? []);
                setAssignmentErrors((current) => ({ ...current, dates: undefined }));
              }}
              className={cn("border-border/70", assignmentErrors.dates &&
"border-destructive")}
              classNames={{
                months: "w-full",
                month: "w-full",
                table: "w-full",
                nav: "justify-between gap-1",
                month_caption:
                  "flex h-(--cell-size) w-full items-center justify-center px-(--cell-size)",
              }}
            />
            <p className="text-xs text-muted-foreground">
              {assignDateStrs.length === 0
                ? "Choose one or more days for this crew member."
                : `${assignDateStrs.length} ${assignDateStrs.length === 1 ? "day" :
"days"} selected.`}
            </p>
            <FieldError message={assignmentErrors.dates} />
          </div>
```

```
          <div className="space-y-1.5">
            <Label>Mechanic</Label>
            <Select
              value={assignMechanic}
              onValueChange={(value) => {
                setAssignMechanic(value);
                setAssignmentErrors((current) => ({ ...current, mechanic: undefined
}));
              }}
              disabled={crew.isLoading}
            >
              <SelectTrigger className="w-full"
aria-invalid={!!assignmentErrors.mechanic}>
                <SelectValue placeholder="Select a mechanic" />
              </SelectTrigger>
              <SelectContent>
                {(crew.data ?? []).map((c) => (
                  <SelectItem key={c.id} value={c.id}>
                    {c.name} ({c.role})
                  </SelectItem>
                ))}
              </SelectContent>
            </Select>
            <FieldError message={assignmentErrors.mechanic} />
          </div>

          <ShiftHoursFields
            shift={assignShift}
            start={customStart}
            end={customEnd}
            errors={assignmentErrors}
            onShiftChange={(value) => {
              setAssignShift(value);
              setCustomStart("");
              setCustomEnd("");
            }}
            onStartChange={(value) => {
              setCustomStart(value);
              setAssignmentErrors((current) => ({ ...current, customStart: undefined
}));
            }}
            onEndChange={(value) => {
              setCustomEnd(value);
              setAssignmentErrors((current) => ({ ...current, customEnd: undefined
}));
```

```
            }}
          />
        </div>
        <DialogFooter>
          <Button type="button" variant="outline" onClick={() =>
setDialogOpen(false)}>
            Close
          </Button>
          <Button onClick={handleSave} disabled={saveSchedule.isPending}>
            {saveSchedule.isPending && <Loader2 className="animate-spin" />}
            Save working days
          </Button>
        </DialogFooter>
      </DialogContent>
    </Dialog>
  </AdminPage>
 );
}

/** The shift preset picker, with start and end times for a custom shift. */
function ShiftHoursFields({
 shift,
 start,
 end,
 errors = {},
 onShiftChange,
 onStartChange,
 onEndChange,
}: {
 shift: string;
 start: string;
 end: string;
 errors?: { customStart?: string | undefined; customEnd?: string | undefined };
 onShiftChange: (value: string) => void;
 onStartChange: (value: string) => void;
 onEndChange: (value: string) => void;
}) {
 return (
  <>
   <div className="space-y-1.5">
    <Label>Shift</Label>
    <Select value={shift} onValueChange={onShiftChange}>
     <SelectTrigger className="w-full">
      <SelectValue />
     </SelectTrigger>
```

```
        <SelectContent>
          <SelectItem value="whole-day">Whole Day (8:00 AM – 5:00
PM)</SelectItem>
          <SelectItem value="morning">Morning (8:00 AM – 12:00
PM)</SelectItem>
          <SelectItem value="afternoon">Afternoon (1:00 PM – 5:00
PM)</SelectItem>
          <SelectItem value="custom">Custom Time Range</SelectItem>
        </SelectContent>
      </Select>
    </div>

    {shift === "custom" && (
      <div className="grid grid-cols-2 gap-3">
        <div className="space-y-1">
          <Label>Start</Label>
          <Select value={start} onValueChange={onStartChange}>
            <SelectTrigger className="w-full" aria-invalid={!!errors.customStart}>
              <SelectValue placeholder="Start time" />
            </SelectTrigger>
            <SelectContent>
              {shopTimeOptions().map((t) => (
                <SelectItem key={t.value} value={t.value}>
                  {t.label}
                </SelectItem>
              ))}
            </SelectContent>
          </Select>
          <FieldError message={errors.customStart} />
        </div>
        <div className="space-y-1">
          <Label>End</Label>
          <Select value={end} onValueChange={onEndChange}>
            <SelectTrigger className="w-full" aria-invalid={!!errors.customEnd}>
              <SelectValue placeholder="End time" />
            </SelectTrigger>
            <SelectContent>
              {shopTimeOptions().map((t) => (
                <SelectItem key={t.value} value={t.value}>
                  {t.label}
                </SelectItem>
              ))}
            </SelectContent>
          </Select>
          <FieldError message={errors.customEnd} />
```

```
        </div>
      </div>
    )}
  </>
 );
}

/** The bookings a shift change moves to another mechanic. */
function ShiftWorkNotice({
 loading,
 failed,
 affected,
 emptyMessage,
}: {
 loading: boolean;
 failed: boolean;
 affected: ShiftWork[];
 emptyMessage: string;
}) {
 if (loading) {
  return (
   <p className="flex items-center gap-2 text-sm text-muted-foreground">
    <Loader2 className="size-4 animate-spin" aria-hidden="true" /> Checking
this mechanic's
    bookings...
   </p>
  );
 }
 if (failed) {
  return (
   <InlineAlert className="mb-0">
    Could not check this mechanic's bookings. Close this and try again.
   </InlineAlert>
  );
 }
 if (affected.length === 0) {
  return <p className="text-sm text-muted-foreground">{emptyMessage}</p>;
 }
 return (
  <div className="rounded-lg border border-amber-500/40 bg-amber-500/5 p-3
text-sm">
   <p className="font-medium text-foreground">
    {affected.length === 1
     ? "This booking moves to another free mechanic:"
     : `These ${affected.length} bookings move to other free mechanics:`}
```

```
      </p>
      <ul className="mt-2 space-y-1 text-muted-foreground">
        {affected.map((item) => (
          <li key={item.key}>
            {formatTime(item.startTime)} · {item.customerName} ·
{item.referenceCode}
          </li>
        ))}
      </ul>
      <p className="mt-2 text-xs text-muted-foreground">
        If no one else is free for one of them, nothing changes.
      </p>
    </div>
  );
}
```

src/routes/_authenticated/admin/blocked-numbers.tsx

```
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import { createFileRoute } from "@tanstack/react-router";
import { Archive, Ban, Plus } from "lucide-react";
import { useState } from "react";

import { ArchiveConfirmationDialog } from
"@/components/admin/archive-confirmation-dialog";
import {
  AdminPage,
  DataList,
  InlineAlert,
  ListCard,
  ListEmpty,
  ListToolbar,
  ResetFiltersButton,
  ToolbarSearch,
} from "@/components/admin/data-list";
import { PageHeader } from "@/components/admin/page-header";
import { Button } from "@/components/ui/button";
import { FieldError } from "@/components/ui/field-error";
import {
  Dialog,
  DialogContent,
  DialogDescription,
  DialogFooter,
  DialogHeader,
  DialogTitle,
} from "@/components/ui/dialog";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Textarea } from "@/components/ui/textarea";
import { supabase } from "@/integrations/supabase/client";
import { formatShortDate } from "@/lib/admin-date-range";
import {
  PHONE_VALIDATION_MESSAGE,
  phoneSchema,
  sanitizePhilippineMobileInput,
  toLocalPhilippineMobile,
} from "@/lib/shop";

type BlockedNumber = {
  id: string;
  phone: string;
```

```
  reason: string | null;
  created_at: string;
  created_by: string | null;
  is_archived: boolean;
};

export const Route =
createFileRoute("/_authenticated/admin/blocked-numbers")({
  component: BlockedNumbersPage,
});

function BlockedNumbersPage() {
  const qc = useQueryClient();
  const [open, setOpen] = useState(false);
  const [phone, setPhone] = useState("");
  const [reason, setReason] = useState("");
  const [archiveTarget, setArchiveTarget] = useState<string | null>(null);
  const [phoneError, setPhoneError] = useState("");
  const [archiveError, setArchiveError] = useState<string | null>(null);
  const [search, setSearch] = useState("");

  const blocked = useQuery({
    queryKey: ["blocked-numbers"],
    queryFn: async () => {
      const { data, error } = await supabase
        .from("blocked_numbers")
        .select("*")
        .eq("is_archived", false)
        .order("created_at", { ascending: false });
      if (error) throw error;
      return (data ?? []) as BlockedNumber[];
    },
  });

  const add = useMutation({
    mutationFn: async (normalizedPhone: string) => {
      const existing = await supabase
        .from("blocked_numbers")
        .select("id,is_archived")
        .ilike("phone", normalizedPhone)
        .maybeSingle();
      if (existing.error) throw existing.error;

      if (existing.data) {
        if (!existing.data.is_archived) {
```

```
          throw new Error("This phone number is already blocked.");
        }

        const { error } = await supabase
          .from("blocked_numbers")
          .update({ is_archived: false, reason: reason.trim() || null })
          .eq("id", existing.data.id);
        if (error) throw error;
        return;
      }

      const { error } = await supabase.from("blocked_numbers").insert({
        phone: normalizedPhone,
        reason: reason.trim() || null,
      });
      if (error) throw error;
    },
    onSuccess: () => {
      setPhone("");
      setReason("");
      setOpen(false);
      qc.invalidateQueries({ queryKey: ["blocked-numbers"], exact: false });
      qc.invalidateQueries({ queryKey: ["archived-blocked-numbers"], exact: false
});
    },
    onError: (err: Error) => {
      const msg = err.message.toLowerCase();
      if (msg.includes("unique") || msg.includes("duplicate") ||
msg.includes("already blocked")) {
        setPhoneError("This phone number is already blocked.");
      } else {
        console.error("Block failed:", err);
        setPhoneError("Could not block this number. Please try again.");
      }
    },
  });

  const archive = useMutation({
    mutationFn: async (id: string) => {
      const { error } = await supabase
        .from("blocked_numbers")
        .update({ is_archived: true })
        .eq("id", id);
      if (error) throw error;
    },
```

```
    onSuccess: (_, id) => {
      qc.setQueryData<BlockedNumber[]>(["blocked-numbers"], (numbers) =>
        numbers?.filter((number) => number.id !== id),
      );
      setArchiveError(null);
      qc.invalidateQueries({ queryKey: ["blocked-numbers"], exact: false });
      qc.invalidateQueries({ queryKey: ["archived-blocked-numbers"], exact: false
});
    },
    onError: (err: Error) => {
      console.error("Archive failed:", err);
      setArchiveError("Could not archive this blocked customer. Please try again.");
    },
  });

  const handleAdd = async () => {
    let normalizedPhone: string;
    try {
      normalizedPhone = phoneSchema.parse(phone);
    } catch {
      setPhoneError(PHONE_VALIDATION_MESSAGE);
      return;
    }
    add.mutate(normalizedPhone);
  };

  const searchTerm = search.trim().toLocaleLowerCase();
  const searchDigits = searchTerm.replace(/\D/g, "");
  const filtered = (blocked.data ?? []).filter(
    (b) =>
      !searchTerm ||
      (searchDigits && toLocalPhilippineMobile(b.phone).includes(searchDigits)) ||
      (b.reason ?? "").toLocaleLowerCase().includes(searchTerm),
  );

  const openBlockDialog = () => {
    setPhone("");
    setReason("");
    setPhoneError("");
    setOpen(true);
  };

  return (
    <AdminPage>
      <PageHeader
```

```
      title="Blocked Customers"
      description="Phone numbers blocked from booking online."
      action={
        <Button className="font-display uppercase" onClick={openBlockDialog}>
          <Plus /> Block number
        </Button>
      }
    />
    {archiveError && <InlineAlert>{archiveError}</InlineAlert>}

    <ListToolbar
      summary={
        blocked.data && `Showing ${filtered.length} of ${blocked.data.length}
blocked numbers`
      }
    >
      <ToolbarSearch
        value={search}
        onChange={setSearch}
        placeholder="Search number or reason"
        label="Search blocked numbers by phone number or reason"
      />
    </ListToolbar>

    <ListCard>
      <DataList
        label="Blocked numbers"
        rows={filtered}
        rowKey={(b) => b.id}
        loading={blocked.isLoading}
        error={
          blocked.isError
            ? {
                message: "Could not load blocked numbers.",
                onRetry: () => void blocked.refetch(),
              }
            : null
        }
        empty={
          <ListEmpty
            icon={Ban}
            title={search ? "No blocked numbers match your search." : "No blocked
numbers."}
            description={
              search
```

```
              ? "Try a different number or reason."
              : "Numbers you block can't book online and will need to call the shop."
            }
            action={
              search ? (
                <ResetFiltersButton onClick={() => setSearch("")} />
              ) : (
                <Button size="sm" onClick={openBlockDialog}>
                  <Plus /> Block number
                </Button>
              )
            }
          />
        }
        // Fixed-width columns first and the free-text reason last, so the
        // space between columns stays even however long a reason is.
        columns={[
          {
            key: "phone",
            header: "Phone",
            card: "title",
            className: "w-40",
            cell: (b) => (
              <span className="font-mono text-sm font-medium">
                {toLocalPhilippineMobile(b.phone)}
              </span>
            ),
          },
          {
            key: "created",
            header: "Blocked on",
            className: "w-36",
            cell: (b) => <span
className="text-sm">{formatShortDate(b.created_at)}</span>,
          },
          {
            key: "reason",
            header: "Reason",
            card: "wide",
            className: "pr-8",
            cell: (b) => (
              <span className="text-sm text-muted-foreground">
                {b.reason || "No reason given"}
              </span>
            ),
```

```
          },
        ]}
        actionsClassName="w-32"
        actions={(b) => (
          <Button
            size="sm"
            variant="outline"
            onClick={() => setArchiveTarget(b.id)}
            disabled={archive.isPending}
            aria-label={`Archive blocked number
${toLocalPhilippineMobile(b.phone)}`}
          >
            <Archive /> Archive
          </Button>
        )}
      />
    </ListCard>

    <Dialog open={open} onOpenChange={setOpen}>
      <DialogContent>
        <DialogHeader>
          <DialogTitle className="font-display uppercase">Block a
number</DialogTitle>
          <DialogDescription>
            Blocked numbers cannot book online. They'll need to call the shop.
          </DialogDescription>
        </DialogHeader>
        <div className="space-y-3">
          <div className="space-y-1.5">
            <Label>Phone number</Label>
            <Input
              type="tel"
              value={phone}
              maxLength={11}
              inputMode="numeric"
              autoComplete="tel"
              onChange={(e) => {
                setPhone(sanitizePhilippineMobileInput(e.target.value));
                setPhoneError("");
              }}
              placeholder="09171234567"
              aria-invalid={!!phoneError}
            />
            <FieldError message={phoneError} />
          </div>
```

```
            <div className="space-y-1.5">
              <Label>Reason (optional)</Label>
              <Textarea
                value={reason}
                onChange={(e) => setReason(e.target.value)}
                placeholder="No-shows, abusive behaviour, etc."
                rows={3}
              />
            </div>
          </div>
          <DialogFooter>
            <Button variant="outline" size="sm" onClick={() => setOpen(false)}>
              Cancel
            </Button>
            <Button onClick={handleAdd} disabled={add.isPending || !phone.trim()}>
              {add.isPending ? "Blocking…" : "Block"}
            </Button>
          </DialogFooter>
        </DialogContent>
      </Dialog>
      <ArchiveConfirmationDialog
        open={Boolean(archiveTarget)}
        recordLabel="blocked customer"
        pending={archive.isPending}
        onOpenChange={(nextOpen) => !nextOpen && setArchiveTarget(null)}
        onConfirm={() => {
          if (archiveTarget) archive.mutate(archiveTarget);
          setArchiveTarget(null);
        }}
      />
    </AdminPage>
  );
}
```

src/routes/_authenticated/admin/customers.tsx

```
import { useQuery } from "@tanstack/react-query";
import { createFileRoute, Link } from "@tanstack/react-router";
import {
  CalendarDays,
  ChevronRight,
  CircleAlert,
  History,
  Loader2,
  Phone,
  Search,
  Users,
  X,
} from "lucide-react";
import { useDeferredValue, useEffect, useState } from "react";
import { z } from "zod";

import { PageHeader } from "@/components/admin/page-header";
import { PaginationControls } from "@/components/admin/pagination-controls";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Card, CardContent } from "@/components/ui/card";
import {
  Dialog,
  DialogClose,
  DialogContent,
  DialogDescription,
  DialogFooter,
  DialogHeader,
  DialogTitle,
} from "@/components/ui/dialog";
import { Input } from "@/components/ui/input";
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@/components/ui/select";
import { Skeleton } from "@/components/ui/skeleton";
import {
  Table,
  TableBody,
  TableCell,
```

```
  TableHead,
  TableHeader,
  TableRow,
} from "@/components/ui/table";
import { supabase } from "@/integrations/supabase/client";
import {
  formatPHP,
  formatTime,
  manilaNow,
  normalizePhilippineMobile,
  statusLabel,
  statusTone,
  toLocalPhilippineMobile,
} from "@/lib/shop";
import { cn } from "@/lib/utils";

export const Route = createFileRoute("/_authenticated/admin/customers")({
  validateSearch: z.object({
    phone: z.string().min(1).optional(),
  }),
  component: CustomersPage,
});

type Customer = {
  customer_name: string;
  phone: string;
  visits: number;
  completed_spend: number | string;
  last_booking: string;
  units: string[];
};

type CustomerPage = { rows: Customer[]; total: number };

const PAGE_SIZE_OPTIONS = [10, 25, 50] as const;
const VISIBLE_UNIT_COUNT = 2;

function CustomersPage() {
  const search = Route.useSearch();
  const historyPageSize = 10;
  const [pageSize, setPageSize] =
useState<number>(PAGE_SIZE_OPTIONS[0]);
  const [term, setTerm] = useState("");
  const [page, setPage] = useState(0);
  const [historyPage, setHistoryPage] = useState(0);
```

```
  const [historyCustomer, setHistoryCustomer] = useState<string | null>(null);
  const deferredTerm = useDeferredValue(term);
  const cleanedSearchTerm = cleanSearchTerm(deferredTerm);
  const customerSearchTerm = normalizePhilippineMobile(cleanedSearchTerm)
?? cleanedSearchTerm;

  const customers = useQuery({
    queryKey: ["customers", { search: customerSearchTerm, page, pageSize }],
    queryFn: async () => {
      const { data, error } = await supabase.rpc("get_admin_customers_page", {
        p_search: customerSearchTerm || null,
        p_limit: pageSize,
        p_offset: page * pageSize,
      });
      if (error) throw error;
      return data as unknown as CustomerPage;
    },
  });

  const customerHistory = useQuery({
    queryKey: ["customer-history", historyCustomer, historyPage],
    queryFn: async () => {
      if (!historyCustomer) return { rows: [], total: 0 };
      const { data, error, count } = await supabase
        .from("appointments")
        .select(
          "id,reference_code,customer_name,appointment_date,start_time,status,tot
al_estimate,moto_brand,moto_model,plate_number,is_archived",
          { count: "exact" },
        )
        .eq("phone", normalizePhilippineMobile(historyCustomer) ??
historyCustomer)
        .order("appointment_date", { ascending: false })
        .order("start_time", { ascending: false })
        .range(historyPage * historyPageSize, historyPage * historyPageSize +
historyPageSize - 1);
      if (error) throw error;
      return { rows: data ?? [], total: count ?? 0 };
    },
    enabled: !!historyCustomer,
  });

  useEffect(() => {
    setPage(0);
  }, [term]);
```

```
  useEffect(() => {
    if (!search.phone) return;
    setTerm(normalizePhilippineMobile(search.phone) ?? search.phone);
    setPage(0);
    setHistoryCustomer(normalizePhilippineMobile(search.phone) ??
search.phone);
    setHistoryPage(0);
  }, [search.phone]);

  const rows = customers.data?.rows ?? [];
  const total = customers.data?.total ?? 0;
  const today = manilaNow().date;
  const searching = Boolean(term.trim());

  function openHistory(customer: Customer) {
    setHistoryCustomer(customer.phone);
    setHistoryPage(0);
  }

  const historyButton = (customer: Customer) => (
    <Button
      size="sm"
      variant="outline"
      aria-label={`View ${customer.customer_name}'s appointment history`}
      onClick={() => openHistory(customer)}
    >
      <History /> View history
    </Button>
  );

  return (
    <div className="mx-auto w-full max-w-7xl">
      <PageHeader
        title="Customer List"
        description="Everyone who has booked with the shop. Search by name or
mobile number to find a customer and review their bookings."
      />

      <div className="mb-4 flex flex-col gap-3 lg:flex-row lg:items-center
lg:justify-between">
        <div className="relative min-w-0 lg:w-80">
          <Search
            className="pointer-events-none absolute top-1/2 left-3 size-4
-translate-y-1/2 text-muted-foreground"
```

```
            aria-hidden="true"
          />
          <Input
            type="search"
            value={term}
            onChange={(event) => setTerm(event.target.value)}
            placeholder="Search by name or mobile number"
            aria-label="Search customers by name or mobile number"
            className="pr-9 pl-9 [&::-webkit-search-cancel-button]:hidden"
          />
          {searching && (
            <span className="absolute top-1/2 right-2 -translate-y-1/2">
              {customers.isFetching ? (
                <Loader2
                  className="mr-1 size-4 animate-spin text-muted-foreground"
                  aria-label="Searching"
                />
              ) : (
                <button
                  type="button"
                  onClick={() => setTerm("")}
                  aria-label="Clear search"
                  className="grid size-7 place-items-center rounded-md
text-muted-foreground transition-colors hover:bg-muted hover:text-foreground
focus-visible:ring-1 focus-visible:ring-ring focus-visible:outline-none"
                >
                  <X className="size-4" aria-hidden="true" />
                </button>
              )}
            </span>
          )}
        </div>
        <div className="flex flex-wrap items-center justify-between gap-3 text-sm
text-muted-foreground lg:justify-end">
          <span className="inline-flex items-center gap-1.5">
            <Users className="size-4" aria-hidden="true" />
            {customers.data
              ? `${total} ${total === 1 ? "customer" : "customers"}${searching ? "
found" : ""}`
              : "Loading customers…"}
          </span>
          <label className="inline-flex items-center gap-2">
            <span>Rows per page</span>
            <Select
              value={String(pageSize)}
```

```
            onValueChange={(value) => {
              setPageSize(Number(value));
              setPage(0);
            }}
          >
            <SelectTrigger aria-label="Rows per page" className="h-9 w-20
md:h-8">
              <SelectValue />
            </SelectTrigger>
            <SelectContent>
              {PAGE_SIZE_OPTIONS.map((size) => (
                <SelectItem key={size} value={String(size)}>
                  {size}
                </SelectItem>
              ))}
            </SelectContent>
          </Select>
        </label>
      </div>
    </div>

    <Card className="overflow-hidden border-border/70 bg-card/60">
      <CardContent className="p-0">
        {rows.length > 0 && (
          <>
            {/* Phones and tablets: one card per customer, so nothing scrolls
                sideways next to the admin sidebar. */}
            <ul className="divide-y divide-border/70 lg:hidden">
              {rows.map((customer) => (
                <li key={customer.phone} className="space-y-3 p-4">
                  <div className="flex items-start justify-between gap-3">
                    <CustomerIdentity customer={customer} />
                    <div className="shrink-0 text-right">
                      <p
                        className={cn(
                          "font-medium tabular-nums",
                          Number(customer.completed_spend) > 0
                            ? "text-primary"
                            : "text-muted-foreground",
                        )}
                      >
                        {formatPHP(customer.completed_spend)}
                      </p>
                      <p className="text-xs text-muted-foreground">completed
spend</p>
```

```
                  </div>
                </div>
                <UnitList units={customer.units} />
                <div className="flex flex-wrap items-center justify-between gap-2">
                  <p className="text-xs text-muted-foreground">
                    {customer.visits} {customer.visits === 1 ? "booking" : "bookings"} ·
Last
                    booking <LastBooking date={customer.last_booking}
today={today} />
                  </p>
                  {historyButton(customer)}
                </div>
              </li>
            ))}
          </ul>

          {/* Laptops and desktops: a compact table. */}
          <Table className="hidden w-full table-fixed lg:table">
            <TableHeader>
              <TableRow>
                <TableHead className="pl-5">Customer</TableHead>
                <TableHead className="w-52">Motorcycles</TableHead>
                <TableHead className="hidden w-24 text-center xl:table-cell">
                  Bookings
                </TableHead>
                <TableHead className="w-32 text-right">Completed
spend</TableHead>
                <TableHead className="hidden w-36 xl:table-cell">Last
booking</TableHead>
                <TableHead className="w-40 pr-5 text-right">
                  <span className="sr-only">Actions</span>
                </TableHead>
              </TableRow>
            </TableHeader>
            <TableBody>
              {rows.map((customer) => (
                <TableRow key={customer.phone}>
                  <TableCell className="pl-5">
                    <CustomerIdentity customer={customer} />
                    {/* Bookings and last booking get their own columns from xl. */}
                    <p className="mt-1 pl-12 text-xs text-muted-foreground
xl:hidden">
                      {customer.visits} {customer.visits === 1 ? "booking" : "bookings"}
· Last{" "}
                      <LastBooking date={customer.last_booking} today={today} />
```

```
                  </p>
                </TableCell>
                <TableCell>
                  <UnitList units={customer.units} limit={VISIBLE_UNIT_COUNT} />
                </TableCell>
                <TableCell className="hidden text-center xl:table-cell">
                  <span className="inline-flex min-w-8 justify-center rounded-full
bg-muted px-2 py-0.5 text-sm font-medium tabular-nums">
                    {customer.visits}
                  </span>
                </TableCell>
                <TableCell
                  className={cn(
                    "text-right font-medium tabular-nums",
                    Number(customer.completed_spend) > 0
                      ? "text-primary"
                      : "text-muted-foreground",
                  )}
                >
                  {formatPHP(customer.completed_spend)}
                </TableCell>
                <TableCell className="hidden text-sm xl:table-cell">
                  <LastBooking date={customer.last_booking} today={today}
stacked />
                </TableCell>
                <TableCell className="pr-5
text-right">{historyButton(customer)}</TableCell>
              </TableRow>
            ))}
          </TableBody>
        </Table>
      </>
    )}

    {customers.isLoading && <CustomerListSkeleton />}
    {customers.isError && (
      <div className="px-4 py-10 text-center">
        <p className="inline-flex items-center gap-2 text-sm text-destructive">
          <CircleAlert className="size-4" aria-hidden="true" /> Could not load
customers.
          Please try again.
        </p>
        <div className="mt-3">
          <Button size="sm" variant="outline" onClick={() => void
customers.refetch()}>
```

```
              Try again
            </Button>
          </div>
        </div>
      )}
      {customers.isSuccess && rows.length === 0 && (
        <div className="px-4 py-12 text-center">
          <Users className="mx-auto size-8 text-muted-foreground/60"
aria-hidden="true" />
          <p className="mt-3 text-sm font-medium">
            {searching ? "No customers match your search." : "No customers
found."}
          </p>
          <p className="mt-1 text-sm text-muted-foreground">
            {searching
              ? "Check the spelling or try a mobile number such as 09171234567."
              : "Customers appear here after their first booking."}
          </p>
          {searching && (
            <Button size="sm" variant="outline" className="mt-4" onClick={() =>
setTerm("")}>
              Clear search
            </Button>
          )}
        </div>
      )}
      <PaginationControls
        page={page}
        pageSize={pageSize}
        total={total}
        onPageChange={setPage}
      />
    </CardContent>
  </Card>

  <CustomerHistoryDialog
    phone={historyCustomer}
    customer={rows.find((customer) => customer.phone === historyCustomer)
?? null}
    history={customerHistory.data}
    loading={customerHistory.isLoading}
    error={customerHistory.isError}
    onRetry={() => void customerHistory.refetch()}
    page={historyPage}
    pageSize={historyPageSize}
```

```
        onPageChange={setHistoryPage}
        onClose={() => setHistoryCustomer(null)}
      />
    </div>
  );
}

type HistoryRow = {
  id: string;
  reference_code: string;
  customer_name: string;
  appointment_date: string;
  start_time: string | null;
  status: string;
  total_estimate: number;
  moto_brand: string | null;
  moto_model: string | null;
  plate_number: string | null;
  is_archived: boolean;
};

function CustomerHistoryDialog({
  phone,
  customer,
  history,
  loading,
  error,
  onRetry,
  page,
  pageSize,
  onPageChange,
  onClose,
}: {
  phone: string | null;
  customer: Customer | null;
  history: { rows: HistoryRow[]; total: number } | undefined;
  loading: boolean;
  error: boolean;
  onRetry: () => void;
  page: number;
  pageSize: number;
  onPageChange: (page: number) => void;
  onClose: () => void;
}) {
  const name = customer?.customer_name ?? history?.rows[0]?.customer_name
```

```
?? "Customer";
  const total = history?.total ?? customer?.visits ?? 0;

  return (
    <Dialog open={!!phone} onOpenChange={(open) => !open && onClose()}>
      {/* No bottom padding: the pinned footer below supplies it. */}
      <DialogContent className="grid-cols-[minmax(0,1fr)] pb-0 sm:max-w-3xl
sm:pb-0">
        <DialogHeader>
          <p className="text-xs tracking-widest text-muted-foreground
uppercase">
            Appointment history
          </p>
          <DialogTitle className="font-display text-2xl break-words
uppercase">{name}</DialogTitle>
          <DialogDescription asChild>
            <div>
              {phone && (
                <a
                  href={`tel:${toLocalPhilippineMobile(phone)}`}
                  className="inline-flex items-center gap-1 text-foreground
hover:text-primary"
                >
                  <Phone size={12} aria-hidden="true" />
                  {toLocalPhilippineMobile(phone)}
                </a>
              )}
              <span>
                {total} {total === 1 ? "booking" : "bookings"}
              </span>
              {customer && (
                <span>
                  Completed spend{" "}
                  <span>{formatPHP(customer.completed_spend)}</span>
                </span>
              )}
            </div>
          </DialogDescription>
        </DialogHeader>

        {loading ? (
          <div aria-label="Loading appointment history">
            {Array.from({ length: 3 }, (_, index) => (
              <Skeleton key={index} />
```

```
          ))}
        </div>
      ) : error ? (
        <div className="py-8 text-center">
          <p className="inline-flex items-center gap-2 text-sm text-destructive">
            <CircleAlert className="size-4" aria-hidden="true" /> Could not load
the appointment
            history.
          </p>
          <div className="mt-3">
            <Button size="sm" variant="outline" onClick={onRetry}>
              Try again
            </Button>
          </div>
        </div>
      ) : history && history.rows.length > 0 ? (
        <ul className="space-y-2">
          {history.rows.map((appointment) => (
            <li
              key={appointment.id}
              className="flex flex-col gap-3 rounded-lg border border-border/70
bg-card/40 p-3 sm:flex-row sm:items-center sm:justify-between"
            >
              <div className="flex min-w-0 items-start gap-3">
                <span className="grid size-9 shrink-0 place-items-center
rounded-md bg-muted text-muted-foreground">
                  <CalendarDays className="size-4" aria-hidden="true" />
                </span>
                <div className="min-w-0 text-sm">
                  <p className="font-medium">
                    {formatShortDate(appointment.appointment_date, { weekday: true
})}
                    {appointment.start_time && (
                      <span className="ml-2 font-normal text-muted-foreground">
                        {formatTime(String(appointment.start_time).slice(0, 5))}
                      </span>
                    )}
                  </p>
                  <p className="text-xs break-words text-muted-foreground">
                    <span className="font-medium text-foreground/80">
                      {appointment.reference_code}
                    </span>
                    {" · "}
                    {[appointment.moto_brand,
appointment.moto_model].filter(Boolean).join(" ")}
```

```
                {appointment.plate_number ? ` (${appointment.plate_number})` :
""}
                </p>
              </div>
            </div>
            <div className="flex flex-wrap items-center gap-2 pl-12 sm:justify-end
sm:pl-0">
              {appointment.is_archived && (
                <Badge variant="outline" className="text-[10px] uppercase">
                  Archived
                </Badge>
              )}
              <Badge
                variant="outline"
                className={cn("text-[10px] uppercase",
statusTone(appointment.status))}
              >
                {statusLabel(appointment.status)}
              </Badge>
              <span className="text-sm font-medium text-primary tabular-nums">
                {formatPHP(appointment.total_estimate)}
              </span>
              {!appointment.is_archived && (
                <Button asChild size="sm" variant="ghost" className="h-8 px-2
md:h-8">
                  <Link
                    to="/admin/appointments"
                    search={{ appointmentId: appointment.id }}
                    aria-label={`Open appointment ${appointment.reference_code}`}
                  >
                    Open <ChevronRight aria-hidden="true" />
                  </Link>
                </Button>
              )}
            </div>
          </li>
        ))}
      </ul>
    ) : (
      <p className="py-8 text-center text-sm text-muted-foreground">No
appointments found.</p>
    )}

    <PaginationControls
      page={page}
```

```
          pageSize={pageSize}
          total={history?.total ?? 0}
          onPageChange={onPageChange}
        />

        {/* Pinned to the bottom so it stays visible while a long history scrolls. */}
        <DialogFooter className="sticky bottom-0 -mx-4 gap-2 border-t
border-border/70 bg-background px-4 py-3 sm:-mx-6 sm:px-6">
          <DialogClose asChild>
            <Button type="button" variant="outline">
              Close
            </Button>
          </DialogClose>
        </DialogFooter>
      </DialogContent>
    </Dialog>
  );
}

function CustomerIdentity({ customer }: { customer: Customer }) {
  const phone = toLocalPhilippineMobile(customer.phone);
  return (
    <div className="flex min-w-0 items-center gap-3">
      <span
        aria-hidden="true"
        className="grid size-9 shrink-0 place-items-center rounded-full
bg-primary/15 font-display text-sm text-primary"
      >
        {initials(customer.customer_name)}
      </span>
      <div className="min-w-0">
        <p className="font-medium
break-words">{customer.customer_name}</p>
        <a
          href={`tel:${phone}`}
          className="inline-flex items-center gap-1 text-xs text-muted-foreground
tabular-nums hover:text-primary"
          aria-label={`Call ${customer.customer_name} at ${phone}`}
        >
          <Phone className="size-3" aria-hidden="true" />
          {phone}
        </a>
      </div>
    </div>
  );
```

```
}

function UnitList({ units, limit }: { units: string[]; limit?: number }) {
  const visible = limit ? units.slice(0, limit) : units;
  const hidden = units.length - visible.length;
  if (units.length === 0) return <span className="text-xs
text-muted-foreground">—</span>;
  return (
    <ul className="flex flex-wrap gap-1.5" aria-label="Motorcycles">
      {visible.map((unit) => (
        <li
          key={unit}
          className="max-w-full rounded-md border border-border/70 bg-muted/40
px-2 py-0.5 text-xs break-words"
        >
          {unit}
        </li>
      ))}
      {hidden > 0 && (
        <li
          className="rounded-md px-1 py-0.5 text-xs text-muted-foreground"
          title={units.slice(visible.length).join(", ")}
        >
          +{hidden} more
        </li>
      )}
    </ul>
  );
}

function LastBooking({ date, today, stacked }: { date: string; today: string;
stacked?: boolean }) {
  const upcoming = date > today;
  return (
    <span className={cn(stacked && "flex flex-col items-start gap-1")}>
      <span className="text-foreground">{formatShortDate(date)}</span>
      {upcoming && (
        <span
          className={cn(
            "rounded-full border border-sky-500/30 bg-sky-500/15 px-1.5 py-px
text-[10px] font-medium text-sky-600 uppercase dark:text-sky-400",
            !stacked && "ml-1.5",
          )}
        >
          Upcoming
```

```
        </span>
      )}
    </span>
  );
}

function CustomerListSkeleton() {
  return (
    <div className="divide-y divide-border/70" aria-label="Loading customers">
      {Array.from({ length: 5 }, (_, index) => (
        <div key={index} className="flex items-center gap-3 p-4">
          <Skeleton className="size-9 rounded-full" />
          <div className="flex-1 space-y-2">
            <Skeleton className="h-4 w-40 max-w-full" />
            <Skeleton className="h-3 w-28 max-w-full" />
          </div>
          <Skeleton className="hidden h-8 w-28 sm:block" />
        </div>
      ))}
    </div>
  );
}

function initials(name: string) {
  const parts = name.trim().split(/ /).filter(Boolean);
  const first = parts[0]?.[0] ?? "";
  const last = parts.length > 1 ? (parts.at(-1)?.[0] ?? "") : "";
  return `${first}${last}`.toUpperCase() || "?";
}

/** Formats a YYYY-MM-DD date without shifting it across time zones. */
function formatShortDate(iso: string, options: { weekday?: boolean } = {}) {
  const [year, month, day] = iso.split("-").map(Number);
  return new Date(Date.UTC(year ?? 1970, (month ?? 1) - 1, day ??
1)).toLocaleDateString("en-PH", {
    ...(options.weekday ? { weekday: "short" } : {}),
    month: "short",
    day: "numeric",
    year: "numeric",
    timeZone: "UTC",
  });
}

function cleanSearchTerm(value: string) {
  return value
```

```
    .trim()
    .replace(/[,%_()]/g, " ")
    .replace(/ /g, " ");
}
```

src/routes/_authenticated/admin/index.tsx

```
import { useQuery } from "@tanstack/react-query";
import { createFileRoute, Link } from "@tanstack/react-router";
import { format } from "date-fns";
import {
  BarChart3,
  CalendarCheck,
  CalendarClock,
  CalendarRange,
  ChevronDown,
  ChevronRight,
  PhilippinePeso,
  TrendingDown,
  TrendingUp,
  Users,
} from "lucide-react";
import type { LucideIcon } from "lucide-react";
import type { DateRange } from "react-day-picker";
import { useMemo, useState } from "react";
import { Bar, BarChart, CartesianGrid, Cell, Pie, PieChart, XAxis, YAxis } from
"recharts";

import { AdminPage, InlineAlert } from "@/components/admin/data-list";
import { PageHeader } from "@/components/admin/page-header";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Calendar } from "@/components/ui/calendar";
import { Card, CardContent } from "@/components/ui/card";
import {
  ChartContainer,
  ChartTooltip,
  ChartTooltipContent,
  type ChartConfig,
} from "@/components/ui/chart";
import { Popover, PopoverContent, PopoverTrigger } from
"@/components/ui/popover";
import { supabase } from "@/integrations/supabase/client";
import {
  calendarDateToIso,
  calendarRangeToIso,
  isoDateToCalendarDate,
  type IsoDateRange,
} from "@/lib/admin-date-range";
import { addDays, formatPHP, formatTime, manilaNow, statusLabel, statusTone }
```

```
from "@/lib/shop";
import { cn } from "@/lib/utils";

export const Route = createFileRoute("/_authenticated/admin/")({
  component: Dashboard,
});

const appointmentChartConfig = {
  completed: { label: "Complete", color: "var(--chart-3)" },
  confirmed: { label: "Confirmed", color: "var(--primary)" },
  cancelled: { label: "Canceled", color: "var(--destructive)" },
  in_progress: { label: "In Progress", color: "var(--chart-4)" },
} satisfies ChartConfig;

const statusChartConfig = {
  pending: { label: "Pending", color: "var(--primary)" },
  confirmed: { label: "Confirmed", color: "var(--chart-3)" },
  in_progress: { label: "In Progress", color: "var(--chart-4)" },
  completed: { label: "Complete", color: "var(--chart-2)" },
  rescheduled: { label: "Rescheduled", color: "var(--chart-5)" },
  cancelled: { label: "Canceled", color: "var(--destructive)" },
  no_show: { label: "No Show", color: "var(--muted-foreground)" },
} satisfies ChartConfig;

type DashboardAppointment = {
  id: string;
  reference_code: string;
  customer_name: string;
  appointment_date: string;
  start_time: string;
  status: string;
  serviceName: string;
};

type StatusCounts = Record<
  "pending" | "confirmed" | "in_progress" | "completed" | "rescheduled" |
"cancelled" | "no_show",
  number
>;

/** Counted in the database (get_admin_dashboard_summary). */
type DashboardSummary = {
  current: { appointments: number; customers: number; revenue: number;
statuses: StatusCounts };
  previous: { appointments: number; customers: number; revenue: number };
```

```
  needs_action: { new_bookings: number; reschedule_requests: number };
};

type ChartRow = {
  period: string;
  completed: number;
  confirmed: number;
  cancelled: number;
  in_progress: number;
};

const LIST_COLUMNS =
  "id,reference_code,customer_name,appointment_date,start_time,status,appoint
ment_services(service_name)";

/** Appointments the dashboard lists, as counted by
get_admin_dashboard_summary. */
function activeAppointmentsQuery() {
  return (
    supabase
      .from("appointments")
      .select(LIST_COLUMNS)
      // Completed appointments always count, as in the financial report.
      .or("is_archived.eq.false,status.eq.completed")
      // A rescheduled booking's old record is replaced by its new appointment.
      .is("rescheduled_to_appointment_id", null)
      .not("status", "in", "(cancelled,rejected,no_show)")
      .order("appointment_date")
      .order("start_time")
      .limit(5)
  );
}

function toDashboardAppointments(
  rows: {
    id: string;
    reference_code: string;
    customer_name: string;
    appointment_date: string;
    start_time: string;
    status: string;
    appointment_services: { service_name: string }[] | null;
  }[],
): DashboardAppointment[] {
  return rows.map(({ appointment_services, ...appointment }) => ({
```

```
    ...appointment,
    serviceName:
      (appointment_services ?? []).map((service) => service.service_name).join(",
") ||
      "Service appointment",
  }));
}

type Trend = {
  direction: "up" | "down" | "flat";
  label: string;
  description: string;
};

type OverviewPeriod = "today" | "7d" | "month" | "year" | "custom";

function Dashboard() {
  const today = manilaNow().date;
  const selectedDate = useMemo(() => isoDateToCalendarDate(today), [today]);
  const [dashboardPeriod, setDashboardPeriod] =
useState<OverviewPeriod>("today");
  const [dashboardCustomDateRange, setDashboardCustomDateRange] =
useState<DateRange>();
  const [chartPeriod, setChartPeriod] = useState<OverviewPeriod>("7d");
  const [chartCustomDateRange, setChartCustomDateRange] =
useState<DateRange>();
  const selectedDateIso = calendarDateToIso(selectedDate);

  const dashboardBuckets = useMemo(
    () => buildOverviewBuckets(selectedDate, dashboardPeriod,
dashboardCustomDateRange),
    [dashboardCustomDateRange, dashboardPeriod, selectedDate],
  );
  const dashboardRange = rangeFromBuckets(dashboardBuckets,
selectedDateIso);
  const previousDashboardRange = previousRange(dashboardRange);

  const summary = useQuery({
    queryKey: ["admin-dashboard", "summary", dashboardRange.from,
dashboardRange.to],
    queryFn: async (): Promise<DashboardSummary> => {
      const { data, error } = await supabase.rpc("get_admin_dashboard_summary",
{
        p_from: dashboardRange.from,
        p_to: dashboardRange.to,
```

```
        p_previous_from: previousDashboardRange.from,
        p_previous_to: previousDashboardRange.to,
      });
      if (error) throw error;
      return data as unknown as DashboardSummary;
    },
  });

  const schedule = useQuery({
    queryKey: ["admin-dashboard", "schedule", dashboardRange.from,
dashboardRange.to],
    queryFn: async () => {
      const { data, error } = await activeAppointmentsQuery()
        .gte("appointment_date", dashboardRange.from)
        .lte("appointment_date", dashboardRange.to);
      if (error) throw error;
      return toDashboardAppointments(data ?? []);
    },
  });

  const upcoming = useQuery({
    queryKey: ["admin-dashboard", "upcoming", dashboardRange.to],
    queryFn: async () => {
      const { data, error } = await activeAppointmentsQuery()
        .gt("appointment_date", dashboardRange.to)
        .neq("status", "rescheduled");
      if (error) throw error;
      return toDashboardAppointments(data ?? []);
    },
  });

  const chartBuckets = useMemo(
    () => buildOverviewBuckets(selectedDate, chartPeriod,
chartCustomDateRange),
    [chartCustomDateRange, chartPeriod, selectedDate],
  );
  const chartRange = rangeFromBuckets(chartBuckets, selectedDateIso);
  // A year is shown by month; every other period by day.
  const chartByMonth = chartPeriod === "year";
  const chart = useQuery({
    queryKey: ["admin-dashboard", "chart", chartRange.from, chartRange.to,
chartByMonth],
    queryFn: async () => {
      const { data, error } = await supabase.rpc("get_admin_dashboard_chart", {
        p_from: chartRange.from,
```

```
        p_to: chartRange.to,
        p_by_month: chartByMonth,
      });
      if (error) throw error;
      return new Map(((data ?? []) as unknown as ChartRow[]).map((row) =>
[row.period, row]));
    },
  });
  const appointmentOverview = chartBuckets.map((bucket) => {
    const row = chart.data?.get(bucket.from);
    return {
      day: bucket.label,
      completed: row?.completed ?? 0,
      confirmed: row?.confirmed ?? 0,
      cancelled: row?.cancelled ?? 0,
      in_progress: row?.in_progress ?? 0,
    };
  });

  const current = summary.data?.current;
  const previous = summary.data?.previous;
  const statusCounts = current?.statuses;
  const loadFailed = summary.isError || schedule.isError || upcoming.isError ||
chart.isError;
  const bookingStatuses = [
    {
      key: "pending",
      label: "Pending",
      value: statusCounts?.pending ?? 0,
      color: "var(--primary)",
      dotClassName: "bg-primary",
    },
    {
      key: "confirmed",
      label: "Confirmed",
      value: statusCounts?.confirmed ?? 0,
      color: "var(--chart-3)",
      dotClassName: "bg-chart-3",
    },
    {
      key: "in_progress",
      label: "In Progress",
      value: statusCounts?.in_progress ?? 0,
      color: "var(--chart-4)",
      dotClassName: "bg-chart-4",
```

```
    },
    {
      key: "completed",
      label: "Complete",
      value: statusCounts?.completed ?? 0,
      color: "var(--chart-2)",
      dotClassName: "bg-chart-2",
    },
    {
      key: "rescheduled",
      label: "Rescheduled",
      value: statusCounts?.rescheduled ?? 0,
      color: "var(--chart-5)",
      dotClassName: "bg-chart-5",
    },
    {
      key: "cancelled",
      label: "Canceled",
      value: statusCounts?.cancelled ?? 0,
      color: "var(--destructive)",
      dotClassName: "bg-destructive",
    },
    {
      key: "no_show",
      label: "No Show",
      value: statusCounts?.no_show ?? 0,
      color: "var(--muted-foreground)",
      dotClassName: "bg-muted-foreground",
    },
  ];
  const bookingStatusTotal = bookingStatuses.reduce((sum, status) => sum +
status.value, 0);

  return (
    <AdminPage>
      <PageHeader
        title="Overview"
        description={`Here's what's happening with your shop ${
          dashboardPeriod === "today" ? "today." : "for the selected period."
        }`}
        action={
          <OverviewPeriodFilter
            period={dashboardPeriod}
            customDateRange={dashboardCustomDateRange}
            onPeriodChange={setDashboardPeriod}
```

```
            onCustomDateRangeChange={setDashboardCustomDateRange}
            className="w-full justify-start sm:w-auto"
          />
        }
      />

      {loadFailed && (
        <InlineAlert>
          Could not load the dashboard data. Please refresh the page and try again.
        </InlineAlert>
      )}

      <div className="grid gap-4 sm:grid-cols-2 xl:grid-cols-4">
        <DashboardStat
          icon={CalendarCheck}
          iconClassName="bg-chart-4/15 text-chart-4"
          label={dashboardPeriod === "today" ? "Today's appointments" :
"Appointments"}
          value={current ? String(current.appointments) : "—"}
          trend={makeTrend(
            current?.appointments ?? 0,
            previous?.appointments ?? 0,
            "vs. previous period",
          )}
        />
        <NeedsActionStat
          newBookings={summary.data?.needs_action.new_bookings}
          rescheduleRequests={summary.data?.needs_action.reschedule_requests}
        />
        <DashboardStat
          icon={Users}
          iconClassName="bg-primary/15 text-primary"
          label="Unique customers"
          value={current ? String(current.customers) : "—"}
          trend={makeTrend(
            current?.customers ?? 0,
            previous?.customers ?? 0,
            "vs. previous period",
          )}
        />
        <DashboardStat
          icon={PhilippinePeso}
          iconClassName="bg-accent/15 text-accent"
          label="Completed revenue"
          value={current ? formatPHP(Number(current.revenue)) : "—"}
```

```
        trend={makeTrend(
          Number(current?.revenue ?? 0),
          Number(previous?.revenue ?? 0),
          "vs. previous period",
        )}
      />
    </div>

    <div className="mt-5 grid gap-5
xl:grid-cols-[minmax(0,1.7fr)_minmax(19rem,1fr)]">
      <Card className="border-border/70 bg-card/60">
        <CardContent className="p-4 sm:p-5">
          <div className="mb-4 flex flex-wrap items-center justify-between
gap-3">
            <div className="flex items-center gap-2">
              <BarChart3 className="size-4 text-primary" aria-hidden="true" />
              <h2 className="font-display text-sm tracking-wide uppercase">
                Appointment overview
              </h2>
            </div>
            <OverviewPeriodFilter
              period={chartPeriod}
              customDateRange={chartCustomDateRange}
              onPeriodChange={setChartPeriod}
              onCustomDateRangeChange={setChartCustomDateRange}
              className="h-8 text-xs"
            />
          </div>
          <ChartContainer config={appointmentChartConfig} className="h-64
w-full aspect-auto">
            <BarChart data={appointmentOverview} margin={{ top: 8, right: 4, left: 0
}}>
              <CartesianGrid vertical={false} />
              <XAxis
                dataKey="day"
                tickLine={false}
                axisLine={false}
                tickMargin={8}
                minTickGap={chartPeriod === "today" || chartPeriod === "7d" ? 0 :
16}
              />
              <YAxis
                allowDecimals={false}
                tickLine={false}
                axisLine={false}
```

```
              width={32}
              tickCount={6}
              tick={{ fill: "hsl(var(--muted-foreground))", fontSize: 11 }}
            />
            <ChartTooltip cursor={false} content={<ChartTooltipContent />} />
            <Bar
              dataKey="completed"
              fill="var(--color-completed)"
              radius={[3, 3, 0, 0]}
              minPointSize={2}
            />
            <Bar
              dataKey="confirmed"
              fill="var(--color-confirmed)"
              radius={[3, 3, 0, 0]}
              minPointSize={2}
            />
            <Bar
              dataKey="cancelled"
              fill="var(--color-cancelled)"
              radius={[3, 3, 0, 0]}
              minPointSize={2}
            />
            <Bar
              dataKey="in_progress"
              fill="var(--color-in_progress)"
              radius={[3, 3, 0, 0]}
              minPointSize={2}
            />
          </BarChart>
        </ChartContainer>
        <ChartKey items={appointmentChartConfig} />
      </CardContent>
    </Card>

    <Card className="border-border/70 bg-card/60">
      <CardContent className="p-4 sm:p-5">
        <div className="flex items-center gap-2">
          <PhilippinePeso className="size-4 text-primary" aria-hidden="true" />
          <h2 className="font-display text-sm tracking-wide
uppercase">Appointment status</h2>
        </div>
        <div className="mt-4 grid items-center gap-5
sm:grid-cols-[minmax(10rem,.9fr)_minmax(0,1fr)]
xl:grid-cols-[minmax(10rem,.9fr)_minmax(0,1fr)]">
```

```
            <div className="relative mx-auto aspect-square w-full max-w-52">
              <ChartContainer
                config={statusChartConfig}
                className="absolute inset-0 h-full w-full aspect-auto"
              >
                <PieChart>
                  <ChartTooltip cursor={false} content={<ChartTooltipContent
hideLabel />} />
                  <Pie
                    data={bookingStatuses}
                    dataKey="value"
                    nameKey="label"
                    innerRadius="58%"
                    outerRadius="84%"
                    paddingAngle={2}
                    strokeWidth={0}
                  >
                    {bookingStatuses.map((status) => (
                      <Cell key={status.key} fill={status.color} />
                    ))}
                  </Pie>
                </PieChart>
              </ChartContainer>
              <div className="pointer-events-none absolute inset-0 grid
place-items-center text-center">
                <div>
                  <p className="font-display text-3xl
text-foreground">{bookingStatusTotal}</p>
                  <p className="text-xs text-muted-foreground
uppercase">Total</p>
                </div>
              </div>
            </div>
            <ul className="space-y-3 text-sm">
              {bookingStatuses.map((status) => (
                <li key={status.key} className="flex items-center justify-between
gap-3">
                  <span className="flex min-w-0 items-center gap-2">
                    <span className={cn("size-2.5 shrink-0 rounded-full",
status.dotClassName)} />
                    <span className="truncate">{status.label}</span>
                  </span>
                  <span className="flex shrink-0 items-center gap-2
text-muted-foreground">
                    <strong className="font-medium
```

```
text-foreground">{status.value}</strong>
                    <span className="w-10 text-right text-xs">
                     {bookingStatusTotal
                       ? `${((status.value / bookingStatusTotal) * 100).toFixed(1)}%`
                       : "0.0%"}
                    </span>
                   </span>
                  </li>
                ))}
               </ul>
              </div>
             </CardContent>
            </Card>
           </div>

           <div className="mt-5 grid gap-5
xl:grid-cols-[minmax(0,1.08fr)_minmax(0,1fr)]">
            <SchedulePanel
             title={dashboardPeriod === "today" ? "Today's schedule" : "Appointments"}
             empty="No appointments found for this period."
             rows={schedule.data ?? []}
             showDate={dashboardPeriod !== "today"}
            />
            <SchedulePanel
             title="Upcoming appointments"
             empty="No upcoming appointments yet."
             rows={upcoming.data ?? []}
             upcoming
            />
           </div>
          </AdminPage>
         );
        }

        function DashboardStat({
         icon: Icon,
         iconClassName,
         label,
         value,
         trend,
        }: {
         icon: LucideIcon;
         iconClassName: string;
         label: string;
         value: string;
```

```
  trend: Trend;
}) {
  return (
    <Card className="h-full border-border/70 bg-card/60">
      <CardContent className="p-4 sm:p-5">
        <div className="flex items-center gap-3">
          <span
            className={cn("grid size-10 shrink-0 place-items-center rounded-lg",
iconClassName)}
          >
            <Icon className="size-4" aria-hidden="true" />
          </span>
          <p className="text-xs tracking-wider text-muted-foreground
uppercase">{label}</p>
        </div>
        <div className="mt-3 flex items-end justify-between gap-3">
          <div>
            <p className="font-display text-2xl text-foreground">{value}</p>
            <p
              className={cn(
                "mt-2 flex items-center gap-1 text-xs font-medium",
                trend.direction === "up" && "text-emerald-500",
                trend.direction === "down" && "text-destructive",
                trend.direction === "flat" && "text-muted-foreground",
              )}
            >
              {trend.direction === "up" && <TrendingUp className="size-3.5"
aria-hidden="true" />}
              {trend.direction === "down" && (
                <TrendingDown className="size-3.5" aria-hidden="true" />
              )}
              {trend.label}
            </p>
            <p className="mt-0.5 text-[11px]
text-muted-foreground">{trend.description}</p>
          </div>
          <TrendLine direction={trend.direction} />
        </div>
      </CardContent>
    </Card>
  );
}

/** Bookings and reschedule requests waiting for the shop, whatever their date. */
function NeedsActionStat({
```

```
  newBookings,
  rescheduleRequests,
}: {
  newBookings: number | undefined;
  rescheduleRequests: number | undefined;
}) {
  const loaded = newBookings !== undefined && rescheduleRequests !==
undefined;
  const total = (newBookings ?? 0) + (rescheduleRequests ?? 0);
  return (
    <Link
      to="/admin/appointments"
      search={{ status: "needs_action" }}
      aria-label={`Needs action: ${total} waiting. Open the list.`}
      className="group rounded-xl focus-visible:ring-2 focus-visible:ring-ring
focus-visible:outline-none"
    >
      <Card className="h-full border-border/70 bg-card/60 transition-colors
group-hover:border-primary/50">
        <CardContent className="p-4 sm:p-5">
          <div className="flex items-center gap-3">
            <span className="grid size-10 shrink-0 place-items-center rounded-lg
bg-emerald-500/15 text-emerald-400">
              <CalendarClock className="size-4" aria-hidden="true" />
            </span>
            <p className="text-xs tracking-wider text-muted-foreground
uppercase">Needs action</p>
          </div>
          <div className="mt-3 flex items-end justify-between gap-3">
            <div>
              <p
                className={cn(
                  "font-display text-2xl",
                  total > 0 ? "text-primary" : "text-foreground",
                )}
              >
                {loaded ? total : "—"}
              </p>
              <p className="mt-2 text-xs font-medium text-muted-foreground">
                {newBookings ?? 0} new {newBookings === 1 ? "booking" :
"bookings"}
              </p>
              <p className="mt-0.5 text-[11px] text-muted-foreground">
                {rescheduleRequests ?? 0} reschedule{" "}
                {rescheduleRequests === 1 ? "request" : "requests"}
```

```
            </p>
          </div>
          <ChevronRight
            className="size-4 shrink-0 text-muted-foreground transition-colors
group-hover:text-primary"
            aria-hidden="true"
          />
        </div>
      </CardContent>
    </Card>
  </Link>
 );
}

function ChartKey({ items }: { items: ChartConfig }) {
 return (
  <div className="mt-3 flex flex-wrap justify-center gap-x-4 gap-y-2 text-xs
text-muted-foreground">
    {Object.entries(items).map(([key, item]) => (
      <span key={key} className="flex items-center gap-1.5">
        <span className="size-2.5 rounded-full" style={{ backgroundColor:
item.color }} />
        {item.label}
      </span>
    ))}
  </div>
 );
}

function OverviewPeriodFilter({
 period,
 customDateRange,
 onPeriodChange,
 onCustomDateRangeChange,
 className,
}: {
 period: OverviewPeriod;
 customDateRange: DateRange | undefined;
 onPeriodChange: (period: OverviewPeriod) => void;
 onCustomDateRangeChange: (range: DateRange | undefined) => void;
 className?: string;
}) {
 const [open, setOpen] = useState(false);
 const [isPickingCustomRange, setIsPickingCustomRange] = useState(false);
 const [draftCustomDateRange, setDraftCustomDateRange] =
```

```
useState<DateRange>();

  const selectPeriod = (nextPeriod: Exclude<OverviewPeriod, "custom">) => {
    onPeriodChange(nextPeriod);
    setIsPickingCustomRange(false);
    setOpen(false);
  };

  const cancelCustomRange = () => {
    setDraftCustomDateRange(customDateRange);
    setIsPickingCustomRange(false);
    setOpen(false);
  };

  const applyCustomRange = () => {
    if (!draftCustomDateRange?.from || !draftCustomDateRange.to) return;
    onCustomDateRangeChange(draftCustomDateRange);
    onPeriodChange("custom");
    setIsPickingCustomRange(false);
    setOpen(false);
  };

  return (
    <Popover
      open={open}
      onOpenChange={(nextOpen) => {
        setOpen(nextOpen);
        if (!nextOpen) setIsPickingCustomRange(false);
      }}
    >
      <PopoverTrigger asChild>
        <Button type="button" variant="outline" className={cn("gap-1.5",
className)}>
          <CalendarRange className="size-3.5" aria-hidden="true" />
          {overviewPeriodLabel(period, customDateRange)}
          <ChevronDown className="size-3.5" aria-hidden="true" />
        </Button>
      </PopoverTrigger>
      <PopoverContent align="end" className="w-72 p-2">
        {isPickingCustomRange ? (
          <div>
            <div className="px-2 pb-2">
              <p className="text-sm font-medium">Custom date range</p>
              <p className="text-xs text-muted-foreground">Select a start date and
an end date.</p>
```

```
          </div>
          <Calendar
            mode="range"
            selected={draftCustomDateRange}
            onSelect={setDraftCustomDateRange}
            className="mx-auto p-0"
            classNames={{
              nav: "inset-x-auto left-1/2 w-48 -translate-x-1/2 justify-between",
            }}
          />
          <div className="mt-2 grid grid-cols-2 gap-2 px-2 text-xs">
            <DateRangeValue label="Start date"
value={draftCustomDateRange?.from} />
            <DateRangeValue label="End date" value={draftCustomDateRange?.to}
/>
          </div>
          <div className="mt-3 flex justify-end gap-2 border-t border-border pt-3">
            <Button type="button" variant="outline" size="sm"
onClick={cancelCustomRange}>
              Cancel
            </Button>
            <Button
              type="button"
              size="sm"
              onClick={applyCustomRange}
              disabled={!draftCustomDateRange?.from ||
!draftCustomDateRange.to}
            >
              Apply
            </Button>
          </div>
        </div>
      ) : (
        <div className="space-y-1">
          <p className="px-2 pb-1 text-xs font-medium tracking-wide
text-muted-foreground uppercase">
            Overview period
          </p>
          <OverviewPeriodOption
            active={period === "today"}
            label="Today"
            onClick={() => selectPeriod("today")}
          />
          <OverviewPeriodOption
            active={period === "7d"}
```

```
            label="Last 7 days"
            onClick={() => selectPeriod("7d")}
          />
          <OverviewPeriodOption
            active={period === "month"}
            label="This month"
            onClick={() => selectPeriod("month")}
          />
          <OverviewPeriodOption
            active={period === "year"}
            label="This year"
            onClick={() => selectPeriod("year")}
          />
          <OverviewPeriodOption
            active={period === "custom"}
            label="Custom date range"
            onClick={() => {
              setDraftCustomDateRange(customDateRange);
              setIsPickingCustomRange(true);
            }}
          />
        </div>
      )}
    </PopoverContent>
  </Popover>
 );
}

function OverviewPeriodOption({
 active,
 label,
 onClick,
}: {
 active: boolean;
 label: string;
 onClick: () => void;
}) {
 return (
  <Button
   type="button"
   variant="ghost"
   className={cn("h-9 w-full justify-start text-sm", active && "bg-muted")}
   onClick={onClick}
  >
   {label}
```

```
    </Button>
  );
}

function DateRangeValue({ label, value }: { label: string; value: Date | undefined
}) {
  return (
    <div className="rounded-md border border-border bg-muted/30 px-2
py-1.5">
      <p className="text-[10px] tracking-wide text-muted-foreground
uppercase">{label}</p>
      <p className="mt-0.5 truncate font-medium text-foreground">
        {value ? format(value, "MMM d, yyyy") : "Not selected"}
      </p>
    </div>
  );
}

function SchedulePanel({
  title,
  rows,
  empty,
  upcoming = false,
  showDate = false,
}: {
  title: string;
  rows: DashboardAppointment[];
  empty: string;
  upcoming?: boolean;
  showDate?: boolean;
}) {
  return (
    <Card className="border-border/70 bg-card/60">
      <CardContent className="p-4 sm:p-5">
        <div className="mb-4 flex items-center justify-between gap-3">
          <div className="flex items-center gap-2">
            <CalendarCheck className="size-4 text-primary" aria-hidden="true" />
            <h2 className="font-display text-sm tracking-wide
uppercase">{title}</h2>
          </div>
          <Link
            to="/admin/appointments"
            className="text-xs font-medium text-primary hover:underline"
          >
            View all
```

```
        </Link>
      </div>
      {rows.length === 0 ? (
        <p className="py-7 text-center text-sm
text-muted-foreground">{empty}</p>
      ) : (
        <ul className="divide-y divide-border/60">
          {rows.map((appointment) => (
            <li
              key={appointment.id}
              className={cn(
                "grid items-center gap-x-2 gap-y-3 py-3 first:pt-0 last:pb-0",
                upcoming
                  ? "grid-cols-[4px_minmax(0,1fr)_auto_auto]
sm:grid-cols-[4px_minmax(9rem,.9fr)_minmax(0,1fr)_auto_auto]"
                  : showDate
                    ? "grid-cols-[4px_minmax(6.5rem,.75fr)_minmax(0,1fr)_auto] sm:gri
d-cols-[4px_minmax(8rem,.85fr)_minmax(5.5rem,.8fr)_minmax(6rem,1fr)_auto]"
                    : "grid-cols-[4px_minmax(3.5rem,.55fr)_minmax(0,1fr)_auto]
sm:grid-cols-[4px_4.5rem_minmax(5.5rem,.8fr)_minmax(6rem,1fr)_auto]",
              )}
            >
              <span
                className={cn(
                  "h-8 w-1 shrink-0 rounded-full",
                  scheduleStripe(appointment.status),
                )}
                aria-hidden="true"
              />
              {upcoming ? (
                <>
                  <div className="min-w-0">
                    <p className="text-sm font-medium">
                      {format(isoDateToCalendarDate(appointment.appointment_date),
"MMM d, yyyy")}{" "}
                      &middot; {formatTime(String(appointment.start_time).slice(0, 5))}
                    </p>
                    <p className="mt-0.5 truncate text-xs text-muted-foreground
sm:hidden">
                      {appointment.customer_name} &middot;
{appointment.reference_code}
                    </p>
                  </div>
                  <p className="hidden min-w-0 truncate text-xs
text-muted-foreground sm:block">
```

```
                {appointment.customer_name} &middot;
{appointment.reference_code}
              </p>
            </>
          ) : (
            <>
              <p className="text-sm font-medium">
                {showDate && (
                  <>

{format(isoDateToCalendarDate(appointment.appointment_date), "MMM d")}{" "}
                    &middot;{" "}
                  </>
                )}
                {formatTime(String(appointment.start_time).slice(0, 5))}
              </p>
              <div className="min-w-0">
                <p className="truncate text-sm
font-medium">{appointment.customer_name}</p>
                <p className="mt-0.5 truncate text-xs text-muted-foreground
sm:hidden">
                  {appointment.serviceName}
                </p>
              </div>
              <div className="hidden min-w-0 sm:block">
                <p className="text-[10px] tracking-wide text-muted-foreground
uppercase">
                  Service
                </p>
                <p className="mt-0.5 truncate text-xs text-foreground">
                  {appointment.serviceName}
                </p>
              </div>
            </>
          )}
          <Badge
            variant="outline"
            className={cn("shrink-0 text-[10px] uppercase",
statusTone(appointment.status))}
          >
            {statusLabel(appointment.status)}
          </Badge>
          {upcoming && (
            <Link
              to="/admin/appointments"
```

```
                search={{ appointmentId: appointment.id }}
                className="grid size-7 shrink-0 place-items-center rounded-md
text-muted-foreground transition-colors hover:bg-muted hover:text-foreground
focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring"
                aria-label={`View appointment ${appointment.reference_code}`}
              >
                <ChevronRight className="size-4" aria-hidden="true" />
              </Link>
            )}
          </li>
        ))}
      </ul>
    )}
  </CardContent>
 </Card>
 );
}

function rangeFromBuckets(buckets: { from: string; to: string }[], fallback: string):
IsoDateRange {
 return {
  from: buckets[0]?.from ?? fallback,
  to: buckets.at(-1)?.to ?? fallback,
 };
}

function previousRange(range: IsoDateRange): IsoDateRange {
 const from = Date.parse(`${range.from}T00:00:00Z`);
 const to = Date.parse(`${range.to}T00:00:00Z`);
 const days = Math.max(1, Math.round((to - from) / 86_400_000) + 1);
 return {
  from: addDays(range.from, -days),
  to: addDays(range.to, -days),
 };
}

function overviewPeriodLabel(period: OverviewPeriod, customRange?:
DateRange) {
 if (period === "today") return "Today";
 if (period === "7d") return "Last 7 days";
 if (period === "month") return "This month";
 if (period === "year") return "This year";
 if (!customRange?.from || !customRange.to) return "Custom date range";

 return `${format(customRange.from, "MMM d")} – ${format(customRange.to,
```

```
"MMM d, yyyy")}`;
}

function buildOverviewBuckets(date: Date, period: OverviewPeriod,
customRange?: DateRange) {
  const year = date.getFullYear();
  const month = date.getMonth();

  if (period === "year") {
    return Array.from({ length: 12 }, (_, index) => {
      const monthStart = new Date(year, index, 1, 12);
      const monthEnd = new Date(year, index + 1, 0, 12);
      return {
        label: format(monthStart, "MMM"),
        from: format(monthStart, "yyyy-MM-dd"),
        to: format(monthEnd, "yyyy-MM-dd"),
      };
    });
  }

  if (period === "month") {
    const daysInMonth = new Date(year, month + 1, 0).getDate();
    return Array.from({ length: daysInMonth }, (_, index) => {
      const day = new Date(year, month, index + 1, 12);
      const value = format(day, "yyyy-MM-dd");
      return { label: format(day, "d"), from: value, to: value };
    });
  }

  const today = format(date, "yyyy-MM-dd");
  if (period === "today") {
    return [{ label: format(date, "MMM d"), from: today, to: today }];
  }

  if (period === "custom" && customRange?.from && customRange.to) {
    const range = calendarRangeToIso(customRange);
    if (range) return buildDailyBuckets(range.from, range.to);
  }

  return Array.from({ length: 7 }, (_, index) => {
    const day = addDays(today, index - 6);
    return { label: format(isoDateToCalendarDate(day), "MMM d"), from: day, to:
day };
  });
}
```

```
function buildDailyBuckets(from: string, to: string) {
  const buckets = [] as { label: string; from: string; to: string }[];
  for (let day = from; day <= to; day = addDays(day, 1)) {
    buckets.push({ label: format(isoDateToCalendarDate(day), "MMM d"), from:
day, to: day });
  }
  return buckets;
}

function makeTrend(current: number, previous: number, description = "vs.
yesterday"): Trend {
  if (previous === 0) {
    if (current === 0) return { direction: "flat", label: "0%", description };
    return { direction: "up", label: "New", description };
  }
  const change = ((current - previous) / Math.abs(previous)) * 100;
  return {
    direction: change > 0 ? "up" : change < 0 ? "down" : "flat",
    label: `${Math.abs(change).toFixed(change >= 10 ? 0 : 1)}%`,
    description,
  };
}

function TrendLine({ direction }: Pick<Trend, "direction">) {
  const path =
    direction === "up"
      ? "M2 25 L13 16 L23 19 L34 8 L46 12 L58 2"
      : direction === "down"
        ? "M2 4 L13 13 L23 10 L34 21 L46 17 L58 28"
        : "M2 16 L13 16 L23 15 L34 16 L46 15 L58 16";
  return (
    <svg
      viewBox="0 0 60 30"
      className={cn(
        "h-8 w-14 shrink-0",
        direction === "up" && "text-emerald-500",
        direction === "down" && "text-destructive",
        direction === "flat" && "text-muted-foreground",
      )}
      aria-hidden="true"
    >
      <path d={path} fill="none" stroke="currentColor" strokeWidth="2"
strokeLinecap="round" />
    </svg>
```

```
  );
}

function scheduleStripe(status: string) {
  if (status === "completed") return "bg-chart-4";
  if (status === "pending") return "bg-primary";
  if (status === "in_progress") return "bg-accent";
  return "bg-emerald-500";
}
```

src/routes/_authenticated/admin/mechanics.tsx

```
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import { createFileRoute } from "@tanstack/react-router";
import { Pencil, Plus, UserCheck } from "lucide-react";
import { useState } from "react";

import { ArchiveConfirmationDialog } from
"@/components/admin/archive-confirmation-dialog";
import {
  ActiveStatusBadge,
  AdminPage,
  ArchiveMenuItem,
  DataList,
  InlineAlert,
  ListCard,
  ListEmpty,
  ListToolbar,
  MoreActionsMenu,
  ResetFiltersButton,
  ToolbarFilters,
  ToolbarSearch,
} from "@/components/admin/data-list";
import { PageHeader } from "@/components/admin/page-header";
import { Button } from "@/components/ui/button";
import { FieldError } from "@/components/ui/field-error";
import {
  Dialog,
  DialogContent,
  DialogFooter,
  DialogHeader,
  DialogTitle,
} from "@/components/ui/dialog";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@/components/ui/select";
import { Switch } from "@/components/ui/switch";
import { supabase } from "@/integrations/supabase/client";
import {
```

```
  PHONE_VALIDATION_MESSAGE,
  normalizePhilippineMobile,
  sanitizePhilippineMobileInput,
  toLocalPhilippineMobile,
} from "@/lib/shop";

export const Route = createFileRoute("/_authenticated/admin/mechanics")({
  component: MechanicsPage,
});

type CrewMember = {
  id: string;
  name: string;
  role: string;
  phone: string | null;
  is_active: boolean;
  is_archived: boolean;
};

const blank = { name: "", role: "Mechanic", phone: "", is_active: true };

function MechanicsPage() {
  const qc = useQueryClient();
  const [open, setOpen] = useState(false);
  const [editing, setEditing] = useState<CrewMember | null>(null);
  const [form, setForm] = useState({ ...blank });
  const [archiveTarget, setArchiveTarget] = useState<string | null>(null);
  const [formErrors, setFormErrors] = useState<
    Partial<Record<"name" | "phone", string | undefined>>
  >({});
  const [filterStatus, setFilterStatus] = useState<"all" | "active" | "inactive">("all");
  const [filterName, setFilterName] = useState("");
  const [archiveError, setArchiveError] = useState<string | null>(null);
  const [isSaving, setIsSaving] = useState(false);

  const crew = useQuery({
    queryKey: ["crew-all"],
    queryFn: async () => {
      const { data, error } = await supabase
        .from("crew_members")
        .select("*")
        .eq("is_archived", false)
        .order("name");
      if (error) throw error;
      const seen = new Set<string>();
```

```
    const deduped: CrewMember[] = [];
    for (const c of data as CrewMember[]) {
      if (seen.has(c.name.trim())) continue;
      seen.add(c.name.trim());
      deduped.push(c);
    }
    return deduped;
  },
});

const add = useMutation({
  mutationFn: async () => {
    const { error } = await supabase.from("crew_members").insert({
      name: form.name.trim(),
      role: form.role.trim() || "Mechanic",
      phone: form.phone ? normalizePhilippineMobile(form.phone) : null,
    });
    if (error) throw error;
  },
  onSuccess: () => {
    setForm({ ...blank });
    setOpen(false);
    qc.invalidateQueries({ queryKey: ["crew-all"] });
  },
  onError: (err: Error) => {
    console.error("Add failed:", err);
    setFormErrors({ name: "Could not add this mechanic. Please try again." });
  },
});

const archive = useMutation({
  mutationFn: async (id: string) => {
    const { error } = await supabase
      .from("crew_members")
      .update({ is_archived: true })
      .eq("id", id);
    if (error) throw error;
  },
  onSuccess: (_, id) => {
    qc.setQueryData<CrewMember[]>(["crew-all"], (items) =>
      items?.filter((member) => member.id !== id),
    );
    setArchiveError(null);
    qc.invalidateQueries({ queryKey: ["crew-all"], exact: false });
    qc.invalidateQueries({ queryKey: ["archived-crew"], exact: false });
```

```
    },
    onError: (err: Error) => {
      console.error("Archive failed:", err);
      setArchiveError("Could not archive this mechanic. Please try again.");
    },
  });

  const handleSave = async () => {
    if (form.name.trim().length < 2) {
      setFormErrors({ name: "Enter a mechanic name with at least 2 characters."
});
      return;
    }
    if (form.phone && !normalizePhilippineMobile(form.phone)) {
      setFormErrors({ phone: PHONE_VALIDATION_MESSAGE });
      return;
    }
    setFormErrors({});
    if (editing) {
      setIsSaving(true);
      try {
        const { error } = await supabase
          .from("crew_members")
          .update({
            name: form.name.trim(),
            role: form.role.trim() || "Mechanic",
            phone: form.phone ? normalizePhilippineMobile(form.phone) : null,
            is_active: form.is_active,
          })
          .eq("id", editing.id);
        if (error) {
          setFormErrors({ name: "Could not update this mechanic. Please try again."
});
          return;
        }
      } finally {
        setIsSaving(false);
      }
    } else {
      add.mutate();
      return;
    }
    setEditing(null);
    setForm({ ...blank });
    setOpen(false);
```

```
  qc.invalidateQueries({ queryKey: ["crew-all"] });
};

const searchTerm = filterName.trim().toLocaleLowerCase();
const filteredCrew = (crew.data ?? []).filter(
  (c) =>
    (!searchTerm ||
      `${c.name} ${c.role} ${c.phone ? toLocalPhilippineMobile(c.phone) : ""}`
        .toLocaleLowerCase()
        .includes(searchTerm)) &&
    (filterStatus === "all" || (filterStatus === "active" ? c.is_active : !c.is_active)),
);
const activeCount = (crew.data ?? []).filter((c) => c.is_active).length;
const hasFilters = Boolean(searchTerm) || filterStatus !== "all";
const resetFilters = () => {
  setFilterName("");
  setFilterStatus("all");
};
const openNew = () => {
  setEditing(null);
  setForm({ ...blank });
  setFormErrors({});
  setOpen(true);
};
const openEdit = (c: CrewMember) => {
  setEditing(c);
  setForm({
    name: c.name,
    role: c.role,
    phone: c.phone ? toLocalPhilippineMobile(c.phone) : "",
    is_active: c.is_active,
  });
  setFormErrors({});
  setOpen(true);
};

return (
  <AdminPage>
    <PageHeader
      title="Mechanics"
      description="Manage mechanics and view their current work status."
      action={
        <Button className="font-display uppercase" onClick={openNew}>
          <Plus /> Add mechanic
        </Button>
```

```
      }
    />
    {archiveError && <InlineAlert>{archiveError}</InlineAlert>}

    <ListToolbar
      summary={crew.data && `Showing ${filteredCrew.length} of
${crew.data.length} mechanics`}
    >
      <ToolbarSearch
        value={filterName}
        onChange={setFilterName}
        placeholder="Search name, role, or phone"
        label="Search mechanics by name, role, or phone"
      />
      <ToolbarFilters>
        <Select
          value={filterStatus}
          onValueChange={(value) => setFilterStatus(value as "all" | "active" |
"inactive")}
        >
          <SelectTrigger aria-label="Filter by status" className="w-full sm:w-40">
            <SelectValue />
          </SelectTrigger>
          <SelectContent>
            <SelectItem value="all">All statuses</SelectItem>
            <SelectItem value="active">Active ({activeCount})</SelectItem>
            <SelectItem value="inactive">
              Inactive ({(crew.data?.length ?? 0) - activeCount})
            </SelectItem>
          </SelectContent>
        </Select>
        {hasFilters && <ResetFiltersButton onClick={resetFilters} />}
      </ToolbarFilters>
    </ListToolbar>

    <ListCard>
      <DataList
        label="Mechanics"
        rows={filteredCrew}
        rowKey={(c) => c.id}
        loading={crew.isLoading}
        error={
          crew.isError
            ? { message: "Could not load mechanics.", onRetry: () => void
crew.refetch() }
```

```
            : null
        }
        empty={
          <ListEmpty
            icon={UserCheck}
            title={hasFilters ? "No mechanics match your filters." : "No mechanics
yet."}
            description={
              hasFilters
                ? "Try a different search or status."
                : "Add your crew so you can assign them to appointments."
            }
            action={
              hasFilters ? (
                <ResetFiltersButton onClick={resetFilters} />
              ) : (
                <Button size="sm" onClick={openNew}>
                  <Plus /> Add mechanic
                </Button>
              )
            }
          />
        }
        columns={[
          {
            key: "name",
            header: "Name",
            card: "title",
            cell: (c) => <span className="font-medium">{c.name}</span>,
          },
          {
            key: "role",
            header: "Role",
            className: "w-32",
            cell: (c) => <span className="text-sm">{c.role}</span>,
          },
          {
            key: "phone",
            header: "Phone",
            className: "w-36",
            cell: (c) => (
              <span className="text-sm">
                {c.phone ? (
                  toLocalPhilippineMobile(c.phone)
                ) : (
```

```
            <span className="text-muted-foreground">Not set</span>
          )}
        </span>
      ),
    },
    {
      key: "status",
      header: "Status",
      card: "badge",
      className: "w-28 text-center",
      cell: (c) => <ActiveStatusBadge active={c.is_active} />,
    },
  ]}
  actionsClassName="w-36"
  actions={(c) => (
    <>
      <Button size="sm" variant="outline" onClick={() => openEdit(c)}>
        <Pencil /> Edit
      </Button>
      <MoreActionsMenu itemLabel={c.name}>
        <ArchiveMenuItem
          disabled={archive.isPending}
          onSelect={() => setArchiveTarget(c.id)}
        />
      </MoreActionsMenu>
    </>
  )}
/>
</ListCard>

<Dialog open={open} onOpenChange={setOpen}>
  <DialogContent>
    <DialogHeader>
      <DialogTitle className="font-display uppercase">
        {editing ? "Edit mechanic" : "New mechanic"}
      </DialogTitle>
    </DialogHeader>
    <div className="space-y-3">
      <div className="space-y-1.5">
        <Label>Name</Label>
        <Input
          value={form.name}
          onChange={(e) => {
            setForm({ ...form, name: e.target.value });
            setFormErrors((current) => ({ ...current, name: undefined }));
```

```
            }}
            aria-invalid={!!formErrors.name}
          />
          <FieldError message={formErrors.name} />
        </div>
        <div className="space-y-1.5">
          <Label>Role</Label>
          <Input
            value={form.role}
            onChange={(e) => {
              setForm({ ...form, role: e.target.value });
            }}
          />
        </div>
        <div className="space-y-1.5">
          <Label>Phone</Label>
          <Input
            type="tel"
            value={form.phone}
            maxLength={11}
            inputMode="numeric"
            onChange={(e) => {
              setForm({ ...form, phone: sanitizePhilippineMobileInput(e.target.value)
});
              setFormErrors((current) => ({ ...current, phone: undefined }));
            }}
            placeholder="09171234567"
            aria-invalid={!!formErrors.phone}
          />
          <FieldError message={formErrors.phone} />
        </div>
        {editing && (
          <div className="space-y-1.5 rounded-lg border border-border/70
bg-muted/20 p-3">
            <Label htmlFor="mechanic-active">Status</Label>
            <label className="flex items-center gap-2 text-sm"
htmlFor="mechanic-active">
              <Switch
                id="mechanic-active"
                checked={form.is_active}
                onCheckedChange={(is_active) =>
                  setForm((current) => ({ ...current, is_active }))
                }
              />
              <span
```

```
                  className={
                    form.is_active
                      ? "font-medium text-emerald-600 dark:text-emerald-300"
                      : "font-medium text-muted-foreground"
                  }
                >
                  {form.is_active ? "Active" : "Inactive"}
                </span>
              </label>
            </div>
          )}
        </div>
        <DialogFooter>
          <Button type="button" variant="outline" onClick={() => setOpen(false)}>
            Close
          </Button>
          <Button onClick={handleSave} disabled={add.isPending || isSaving ||
!form.name.trim()}>
            {add.isPending || isSaving ? "Saving..." : "Save"}
          </Button>
        </DialogFooter>
      </DialogContent>
    </Dialog>
    <ArchiveConfirmationDialog
      open={Boolean(archiveTarget)}
      recordLabel="mechanic"
      pending={archive.isPending}
      onOpenChange={(nextOpen) => !nextOpen && setArchiveTarget(null)}
      onConfirm={() => {
        if (archiveTarget) archive.mutate(archiveTarget);
        setArchiveTarget(null);
      }}
    />
  </AdminPage>
 );
}
```

src/routes/_authenticated/admin/motorcycles.tsx

```
import { createFileRoute } from "@tanstack/react-router";
import { Plus } from "lucide-react";
import { useRef } from "react";

import { AdminPage } from "@/components/admin/data-list";
import { PageHeader } from "@/components/admin/page-header";
import {
  MotorcycleCatalogManager,
  MotorcycleCatalogManagerHandle,
} from "@/components/admin/motorcycle-catalog-manager";
import { Button } from "@/components/ui/button";

export const Route = createFileRoute("/_authenticated/admin/motorcycles")({
  component: MotorcyclesAdmin,
});

function MotorcyclesAdmin() {
  const motorcycleCatalogRef =
useRef<MotorcycleCatalogManagerHandle>(null);

  return (
    <AdminPage>
      <PageHeader
        title="Motorcycle Catalog"
        description="Manage motorcycle brands and models shown in the booking
form."
        action={
          <Button
            className="font-display uppercase"
            onClick={() => motorcycleCatalogRef.current?.openNew()}
          >
            <Plus /> Add motorcycle
          </Button>
        }
      />
      <MotorcycleCatalogManager ref={motorcycleCatalogRef} />
    </AdminPage>
  );
}
```

src/routes/_authenticated/admin/notifications.tsx

```
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import { createFileRoute, useNavigate } from "@tanstack/react-router";
import { useServerFn } from "@tanstack/react-start";
import { BellRing, CheckCheck, CheckCircle2, CircleX, Clock3, Play, Trash2 }
from "lucide-react";
import { useEffect, useRef, useState } from "react";

import {
  AdminPage,
  InlineAlert,
  ListCard,
  ListCardTitle,
  ListEmpty,
  ListError,
  ListLoading,
} from "@/components/admin/data-list";
import { PageHeader } from "@/components/admin/page-header";
import {
  AlertDialog,
  AlertDialogAction,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
  AlertDialogTrigger,
} from "@/components/ui/alert-dialog";
import { Button } from "@/components/ui/button";
import { supabase } from "@/integrations/supabase/client";
import {
  reviewPendingAppointmentWithConfirmation,
  reviewRescheduleRequestWithConfirmation,
  retryPendingCustomerNotifications,
  updateAppointmentServiceProgressWithNotifications,
} from "@/lib/appointment-confirmation.functions";
import { cn } from "@/lib/utils";

export const Route = createFileRoute("/_authenticated/admin/notifications")({
  component: NotificationsPage,
});

function isServiceProgressNotification(type: string) {
```

```
  return type === "service_arrival" || type === "service_completion";
}

function NotificationsPage() {
  const queryClient = useQueryClient();
  const navigate = useNavigate();
  const [actionError, setActionError] = useState<string | null>(null);
  const [deleteTarget, setDeleteTarget] = useState<string | null>(null);
  const retryAttempted = useRef(false);
  const [noShowTarget, setNoShowTarget] = useState<{
    appointmentId: string;
    referenceCode: string;
  } | null>(null);
  const reviewPendingWithConfirmation =
useServerFn(reviewPendingAppointmentWithConfirmation);
  const reviewRescheduleWithConfirmation =
useServerFn(reviewRescheduleRequestWithConfirmation);
  const retryPendingNotifications =
useServerFn(retryPendingCustomerNotifications);
  const updateProgressWithNotifications = useServerFn(
    updateAppointmentServiceProgressWithNotifications,
  );

  useEffect(() => {
    if (retryAttempted.current) return;
    retryAttempted.current = true;
    void retryPendingNotifications().then((result) => {
      if (!result.ok) console.error("Could not retry pending customer notifications.");
    });
  }, [retryPendingNotifications]);

  const list = useQuery({
    queryKey: ["notifications"],
    queryFn: async () => {
      const { data, error } = await supabase
        .from("notifications")
        .select(
          "*, appointments(status, reference_code, customer_name, phone,
reschedule_count, pending_reschedule_request_id, pending_reschedule_date,
pending_reschedule_start_time, pending_reschedule_reason,
arrival_notification_snooze_count, service_started_at, service_ended_at)",
        )
        .order("created_at", { ascending: false });
      if (error) throw error;
      return data;
```

```
    },
  });

  function refresh() {
    queryClient.invalidateQueries({ queryKey: ["notifications"] });
    queryClient.invalidateQueries({ queryKey: ["unread-notifications"] });
  }

  const markOne = useMutation({
    mutationFn: async (id: string) => {
      const { error } = await supabase.from("notifications").update({ is_read: true
}).eq("id", id);
      if (error) throw error;
    },
    onSuccess: () => {
      setActionError(null);
      refresh();
    },
    onError: () => setActionError("Could not mark this notification as read. Please
try again."),
  });

  const markAll = useMutation({
    mutationFn: async (ids: string[]) => {
      if (ids.length === 0) return;
      const { error } = await supabase
        .from("notifications")
        .update({ is_read: true })
        .in("id", ids);
      if (error) throw error;
    },
    onSuccess: () => {
      setActionError(null);
      refresh();
    },
    onError: () => setActionError("Could not update notifications. Please try
again."),
  });

  const viewAppointment = useMutation({
    mutationFn: async ({ id, isRead }: { id: string; isRead: boolean; appointmentId:
string }) => {
      if (isRead) return;
      const { error } = await supabase.from("notifications").update({ is_read: true
}).eq("id", id);
```

```
      if (error) throw error;
    },
    onSuccess: async (_, notification) => {
      setActionError(null);
      refresh();
      await navigate({
        to: "/admin/appointments",
        search: { appointmentId: notification.appointmentId },
      });
    },
    onError: () => setActionError("Could not open this appointment. Please try
again."),
  });

  const deleteNotification = useMutation({
    mutationFn: async (id: string) => {
      const { error } = await supabase.from("notifications").delete().eq("id", id);
      if (error) throw error;
    },
    onSuccess: () => {
      setActionError(null);
      refresh();
    },
    onError: (err: Error) => {
      console.error("Delete notification failed:", err);
      setActionError("Could not remove this notification. Please try again.");
    },
  });

  const removeAllRead = useMutation({
    mutationFn: async () => {
      // Resolved/snoozed service-progress records are retained for their
      // workflow history. Normal read notifications remain removable.
      const { error } = await supabase
        .from("notifications")
        .delete()
        .eq("is_read", true)
        .not("type", "in", "(service_arrival,service_completion)");
      if (error) throw error;
    },
    onSuccess: () => {
      setActionError(null);
      refresh();
    },
    onError: (err: Error) => {
```

```
      console.error("Remove read notifications failed:", err);
      setActionError("Could not remove the read notifications. Please try again.");
    },
  });

  const reviewPendingAppointment = useMutation({
    mutationFn: async ({
      appointmentId,
      decision,
    }: {
      appointmentId: string;
      decision: "confirmed" | "rejected";
    }) => {
      const result = await reviewPendingWithConfirmation({
        data: { appointmentId, decision },
      });
      if (!result.ok) throw new Error(result.error);
      return result;
    },
    onSuccess: (result) => {
      const warnings = [
        "emailError" in result ? result.emailError : null,
        "smsError" in result ? result.smsError : null,
      ].filter((warning): warning is string => Boolean(warning));
      setActionError(warnings.join(" ") || null);
      refresh();
      queryClient.invalidateQueries({ queryKey: ["admin-appointments"], exact:
false });
      queryClient.invalidateQueries({ queryKey: ["admin-dashboard"], exact: false
});
      queryClient.invalidateQueries({ queryKey: ["reports"], exact: false });
      queryClient.invalidateQueries({ queryKey: ["archived-appointments"], exact:
false });
    },
    onError: (err: Error) => {
      console.error("Appointment review failed:", err);
      setActionError("Could not review this appointment. Please try again.");
    },
  });

  const reviewRescheduleRequest = useMutation({
    mutationFn: async ({
      appointmentId,
      decision,
    }: {
```

```
      appointmentId: string;
      decision: "confirmed" | "rejected";
    }) => {
      const result = await reviewRescheduleWithConfirmation({
        data: { appointmentId, decision },
      });
      if (!result.ok) throw new Error(result.error);
      return result;
    },
    onSuccess: (result) => {
      const warnings = [
        "emailError" in result ? result.emailError : null,
        "smsError" in result ? result.smsError : null,
      ].filter((warning): warning is string => Boolean(warning));
      setActionError(warnings.join(" ") || null);
      refresh();
      queryClient.invalidateQueries({ queryKey: ["admin-appointments"], exact:
false });
      queryClient.invalidateQueries({ queryKey: ["admin-dashboard"], exact: false
});
      queryClient.invalidateQueries({ queryKey: ["reports"], exact: false });
    },
    onError: (err: Error) => {
      console.error("Reschedule review failed:", err);
      setActionError("Could not review this reschedule request. Please try again.");
    },
  });

  const updateServiceProgress = useMutation({
    mutationFn: async ({
      appointmentId,
      action,
    }: {
      appointmentId: string;
      action: "start" | "snooze_arrival" | "no_show" | "complete" |
"snooze_completion";
    }) => {
      const result = await updateProgressWithNotifications({
        data: { appointmentId, action },
      });
      if (!result.ok) throw new Error(result.error);
      return result;
    },
    onSuccess: (result) => {
      const warnings = [
```

```
        "emailError" in result ? result.emailError : null,
        "smsError" in result ? result.smsError : null,
      ].filter((warning): warning is string => Boolean(warning));
      setActionError(warnings.join(" ") || null);
      refresh();
      queryClient.invalidateQueries({ queryKey: ["admin-appointments"], exact:
false });
      queryClient.invalidateQueries({ queryKey: ["admin-dashboard"], exact: false
});
      queryClient.invalidateQueries({ queryKey: ["reports"], exact: false });
    },
    onError: (err: Error) => {
      console.error("Service progress update failed:", err);
      setActionError("Could not update service progress. Refresh the page and try
again.");
    },
  });

  // Read progress entries are resolved or awaiting a snooze interval; they do
  // not belong in the inbox. Active progress alerts remain unread until an
  // explicit workflow action handles them.
  const rows = (list.data ?? []).filter(
    (notification) => !isServiceProgressNotification(notification.type) ||
!notification.is_read,
  );
  const unread = rows.filter((n) => !n.is_read).length;
  const read = rows.length - unread;
  const unreadIds = rows
    .filter((n) => !n.is_read && !isServiceProgressNotification(n.type))
    .map((n) => n.id);

  return (
    <AdminPage>
      <PageHeader
        title="Notifications"
        description="Review appointments, booking updates, and active service
progress alerts."
        action={
          <div className="flex flex-wrap gap-2">
            <Button
              variant="outline"
              className="uppercase"
              disabled={unreadIds.length === 0 || markAll.isPending}
              onClick={() => markAll.mutate(unreadIds)}
            >
```

```
            <CheckCheck /> Mark all read
          </Button>
          <AlertDialog>
            <AlertDialogTrigger asChild>
              <Button
                variant="outline"
                className="uppercase"
                disabled={read === 0 || removeAllRead.isPending}
              >
                <Trash2 /> Remove all read
              </Button>
            </AlertDialogTrigger>
            <AlertDialogContent>
              <AlertDialogHeader>
                <AlertDialogTitle>Remove all read notifications?</AlertDialogTitle>
                <AlertDialogDescription>
                  This removes {read} notification{read === 1 ? "" : "s"} that have
already been
                  read. Unread notifications will remain unchanged.
                </AlertDialogDescription>
              </AlertDialogHeader>
              <AlertDialogFooter>
                <AlertDialogCancel>Cancel</AlertDialogCancel>
                <AlertDialogAction
                  disabled={removeAllRead.isPending}
                  onClick={() => removeAllRead.mutate()}
                >
                  {removeAllRead.isPending && <Trash2 className="animate-pulse"
/>} Remove all
                  read
                </AlertDialogAction>
              </AlertDialogFooter>
            </AlertDialogContent>
          </AlertDialog>
        </div>
      }
    />

    {actionError && <InlineAlert>{actionError}</InlineAlert>}

    <AlertDialog
      open={Boolean(deleteTarget)}
      onOpenChange={(nextOpen) => !nextOpen && setDeleteTarget(null)}
    >
      <AlertDialogContent>
```

```
        <AlertDialogHeader>
          <AlertDialogTitle>Remove this notification?</AlertDialogTitle>
          <AlertDialogDescription>
            This notification will be permanently removed from the admin inbox.
          </AlertDialogDescription>
        </AlertDialogHeader>
        <AlertDialogFooter>
          <AlertDialogCancel
disabled={deleteNotification.isPending}>Cancel</AlertDialogCancel>
          <AlertDialogAction
            disabled={deleteNotification.isPending}
            onClick={() => {
              if (deleteTarget) deleteNotification.mutate(deleteTarget);
              setDeleteTarget(null);
            }}
          >
            Remove notification
          </AlertDialogAction>
        </AlertDialogFooter>
      </AlertDialogContent>
    </AlertDialog>

    <AlertDialog
      open={Boolean(noShowTarget)}
      onOpenChange={(nextOpen) => !nextOpen && setNoShowTarget(null)}
    >
      <AlertDialogContent>
        <AlertDialogHeader>
          <AlertDialogTitle>Mark this appointment as no-show?</AlertDialogTitle>
          <AlertDialogDescription>
            {noShowTarget
              ? `This will mark ${noShowTarget.referenceCode} as no-show. This
cannot be changed back to a pre-service status.`
              : "This will mark the appointment as no-show."}
          </AlertDialogDescription>
        </AlertDialogHeader>
        <AlertDialogFooter>
          <AlertDialogCancel
disabled={updateServiceProgress.isPending}>Cancel</AlertDialogCancel>
          <AlertDialogAction
            disabled={updateServiceProgress.isPending}
            onClick={() => {
              if (noShowTarget) {
                updateServiceProgress.mutate({
                  appointmentId: noShowTarget.appointmentId,
```

```
                action: "no_show",
              });
            }
            setNoShowTarget(null);
          }}
        >
          Mark no-show
        </AlertDialogAction>
      </AlertDialogFooter>
    </AlertDialogContent>
  </AlertDialog>

  <ListCard
    header={
      <ListCardTitle
        title="Inbox"
        description={
          list.data
            ? `${unread} unread · ${rows.length} ${rows.length === 1 ?
"notification" : "notifications"}`
            : undefined
        }
      />
    }
  >
    {list.isLoading ? (
      <ListLoading label="Loading notifications" />
    ) : list.isError ? (
      <ListError message="Could not load notifications." onRetry={() => void
list.refetch()} />
    ) : rows.length === 0 ? (
      <ListEmpty
        icon={BellRing}
        title="You're all caught up."
        description="New bookings, reschedule requests, and service alerts will
appear here."
      />
    ) : (
      <ul aria-label="Notifications" className="divide-y divide-border/70">
        {rows.map((n) => {
          const isServiceProgress = isServiceProgressNotification(n.type);
          const canStartService =
            n.type === "service_arrival" &&
            n.appointments?.status === "confirmed" &&
            !n.appointments.service_started_at;
```

```
            const canCompleteService =
              n.type === "service_completion" &&
              n.appointments?.status === "in_progress" &&
              Boolean(n.appointments.service_started_at) &&
              !n.appointments.service_ended_at;
            const arrivalSnoozes =
n.appointments?.arrival_notification_snooze_count ?? 0;

            return (
              <li
                key={n.id}
                className={cn(
                  "flex flex-wrap items-start gap-4 border-l-2 border-transparent p-4",
                  !n.is_read && "border-l-primary bg-primary/5",
                )}
              >
                <BellRing
                  className={cn(
                    "mt-0.5 h-5 w-5",
                    n.is_read ? "text-muted-foreground" : "text-primary",
                  )}
                />
                <div className="min-w-0 flex-1">
                  <p className="text-sm font-medium">
                    {n.title}
                    {!n.is_read && <span className="sr-only"> (unread)</span>}
                  </p>
                  {n.message && <p className="mt-1 text-sm
text-muted-foreground">{n.message}</p>}
                  <p className="mt-1 text-[11px] tracking-wider
text-muted-foreground uppercase">
                    {new Date(n.created_at).toLocaleString("en-PH", { timeZone:
"Asia/Manila" })}
                  </p>
                </div>
                <div className="flex flex-wrap gap-2">
                  {n.appointment_id && canStartService && (
                    <>
                      <Button
                        size="sm"
                        className="bg-emerald-600 text-white hover:bg-emerald-700"
                        disabled={updateServiceProgress.isPending}
                        onClick={() =>
                          updateServiceProgress.mutate({
                            appointmentId: n.appointment_id!,
```

```
                        action: "start",
                      })
                    }
                  >
                    <Play className="h-4 w-4" /> In Progress
                  </Button>
                  {arrivalSnoozes >= 3 ? (
                    <Button
                      size="sm"
                      variant="destructive"
                      disabled={updateServiceProgress.isPending}
                      onClick={() =>
                        setNoShowTarget({
                          appointmentId: n.appointment_id!,
                          referenceCode: n.appointments?.reference_code ?? "this
appointment",
                        })
                      }
                    >
                      <CircleX className="h-4 w-4" /> No-Show
                    </Button>
                  ) : (
                    <Button
                      size="sm"
                      variant="outline"
                      disabled={updateServiceProgress.isPending}
                      onClick={() =>
                        updateServiceProgress.mutate({
                          appointmentId: n.appointment_id!,
                          action: "snooze_arrival",
                        })
                      }
                    >
                      <Clock3 className="h-4 w-4" /> Snooze 5 min
                    </Button>
                  )}
                </>
              )}
              {n.appointment_id && canCompleteService && (
                <>
                  <Button
                    size="sm"
                    className="bg-emerald-600 text-white hover:bg-emerald-700"
                    disabled={updateServiceProgress.isPending}
                    onClick={() =>
```

```
            updateServiceProgress.mutate({
              appointmentId: n.appointment_id!,
              action: "complete",
            })
          }
        >
          <CheckCircle2 className="h-4 w-4" /> Completed
        </Button>
        <Button
          size="sm"
          variant="outline"
          disabled={updateServiceProgress.isPending}
          onClick={() =>
            updateServiceProgress.mutate({
              appointmentId: n.appointment_id!,
              action: "snooze_completion",
            })
          }
        >
          <Clock3 className="h-4 w-4" /> Snooze 5 min
        </Button>
      </>
    )}
    {n.appointment_id &&
      n.type !== "reschedule_request" &&
      n.appointments?.status === "pending" &&
      !n.appointments.pending_reschedule_request_id && (
        <>
          <Button
            size="sm"
            className="bg-emerald-600 text-white hover:bg-emerald-700"
            disabled={reviewPendingAppointment.isPending}
            onClick={() =>
              reviewPendingAppointment.mutate({
                appointmentId: n.appointment_id!,
                decision: "confirmed",
              })
            }
          >
            <CheckCircle2 className="h-4 w-4" /> Confirmed
          </Button>
          <Button
            size="sm"
            variant="destructive"
            disabled={reviewPendingAppointment.isPending}
```

```
                    onClick={() =>
                      reviewPendingAppointment.mutate({
                        appointmentId: n.appointment_id!,
                        decision: "rejected",
                      })
                    }
                  >
                    <CircleX className="h-4 w-4" /> Rejected
                  </Button>
                </>
              )}
            {n.appointment_id &&
              n.type === "reschedule_request" &&
              n.appointments?.pending_reschedule_request_id && (
                <>
                  <Button
                    size="sm"
                    className="bg-emerald-600 text-white hover:bg-emerald-700"
                    disabled={
                      reviewRescheduleRequest.isPending ||
                      (n.appointments.reschedule_count ?? 0) >= 3
                    }
                    onClick={() =>
                      reviewRescheduleRequest.mutate({
                        appointmentId: n.appointment_id!,
                        decision: "confirmed",
                      })
                    }
                  >
                    <CheckCircle2 className="h-4 w-4" /> Confirm
                  </Button>
                  {(n.appointments.reschedule_count ?? 0) >= 3 && (
                    <p className="basis-full text-xs text-destructive">
                      Maximum of 3 reschedules reached. This request can only be
rejected.
                    </p>
                  )}
                  <Button
                    size="sm"
                    variant="destructive"
                    disabled={reviewRescheduleRequest.isPending}
                    onClick={() =>
                      reviewRescheduleRequest.mutate({
                        appointmentId: n.appointment_id!,
                        decision: "rejected",
```

```
                      })
                    }
                    >
                      <CircleX className="h-4 w-4" /> Reject
                    </Button>
                  </>
                )}
              {n.type === "customer_cancellation_threshold" &&
n.appointments?.phone && (
                <Button
                  size="sm"
                  variant="outline"
                  onClick={() => {
                    const customerPhone = n.appointments?.phone;
                    if (!customerPhone) return;
                    if (!n.is_read) markOne.mutate(n.id);
                    void navigate({
                      to: "/admin/customers",
                      search: { phone: customerPhone },
                    });
                  }}
                >
                  Review customer history
                </Button>
              )}
              <Button
                size="sm"
                variant="outline"
                disabled={!n.appointment_id || viewAppointment.isPending}
                onClick={() => {
                  if (!n.appointment_id) return;
                  viewAppointment.mutate({
                    id: n.id,
                    // Progress alerts remain actionable until the related
                    // start, no-show, completion, or snooze action occurs.
                    isRead: n.is_read || isServiceProgress,
                    appointmentId: n.appointment_id,
                  });
                }}
              >
                View appointment
              </Button>
              {!n.is_read && !isServiceProgress && (
                <Button size="sm" variant="ghost" onClick={() =>
markOne.mutate(n.id)}>
```

```
                    Mark read
                  </Button>
                )}
                {!isServiceProgress && (
                  <Button
                    size="sm"
                    variant="ghost"
                    disabled={deleteNotification.isPending}
                    onClick={() => setDeleteTarget(n.id)}
                    title="Remove notification"
                  >
                    <Trash2 className="h-4 w-4 text-destructive" />
                  </Button>
                )}
              </div>
            </li>
          );
        })}
      </ul>
    )}
  </ListCard>
 </AdminPage>
 );
}
```

src/routes/_authenticated/admin/parts.tsx

```
import { createFileRoute } from "@tanstack/react-router";
import { Plus } from "lucide-react";
import { useRef } from "react";

import { AdminPage } from "@/components/admin/data-list";
import { PageHeader } from "@/components/admin/page-header";
import { ProductManager, ProductManagerHandle } from
"@/components/admin/product-manager";
import { Button } from "@/components/ui/button";

export const Route = createFileRoute("/_authenticated/admin/parts")({
  component: PartsAdmin,
});

function PartsAdmin() {
  const productManagerRef = useRef<ProductManagerHandle>(null);

  return (
    <AdminPage>
      <PageHeader
        title="Parts & Accessories"
        description="Items shown in the public shop showcase."
        action={
          <Button
            className="font-display uppercase"
            onClick={() => productManagerRef.current?.openNew()}
          >
            <Plus /> Add item
          </Button>
        }
      />
      <ProductManager ref={productManagerRef} category="part" />
    </AdminPage>
  );
}
```

src/routes/_authenticated/admin/prices.tsx

```
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import { createFileRoute } from "@tanstack/react-router";
import {
  ArrowLeft,
  Check,
  Clock,
  Eye,
  History,
  Info,
  Loader2,
  Package,
  Pencil,
  RotateCcw,
  Tags,
  Wrench,
} from "lucide-react";
import { useEffect, useMemo, useState } from "react";

import {
  AdminPage,
  ListCard,
  ListEmpty,
  ListError,
  ListLoading,
  ListToolbar,
  ResetFiltersButton,
  ToolbarFilters,
  ToolbarSearch,
} from "@/components/admin/data-list";
import { PageHeader } from "@/components/admin/page-header";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Card, CardContent } from "@/components/ui/card";
import { FieldError } from "@/components/ui/field-error";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@/components/ui/select";
```

```
import {
  Table,
  TableBody,
  TableCell,
  TableHead,
  TableHeader,
  TableRow,
} from "@/components/ui/table";
import { Tabs, TabsContent, TabsList, TabsTrigger } from
"@/components/ui/tabs";
import { supabase } from "@/integrations/supabase/client";
import { activeStatusTone, formatPHP } from "@/lib/shop";
import { cn } from "@/lib/utils";

export const Route = createFileRoute("/_authenticated/admin/prices")({
component: PricesPage });

type PriceTable = "services" | "products";
type PriceItem = {
  id: string;
  name: string;
  category: string;
  price: number;
  duration_minutes: number | null;
  large_bike_price: number | null;
  large_bike_duration_minutes: number | null;
  is_active: boolean;
  is_archived: boolean;
  created_at: string;
  updated_at: string;
  archived_at: string | null;
};
type PriceConfiguration = {
  id: string;
  historyId: string | null;
  kind: "small_bike" | "large_bike" | "product";
  label: string;
  price: number;
  durationMinutes: number | null;
};
type PriceHistory = {
  id: string;
  item_id: string;
  configuration_id: string | null;
  configuration_label: string | null;
```

```
  old_price: number;
  new_price: number;
  reason: string | null;
  changed_by_email: string | null;
  created_at: string;
};
type PriceDraft = { price: string; reason: string };

const ALL_CATEGORIES = "__all__";
const HISTORY_PREVIEW_COUNT = 5;

function PricesPage() {
  const [showArchives, setShowArchives] = useState(false);
  return (
    <AdminPage>
      <PageHeader
        title="Prices Management"
        description={
          showArchives
            ? "Review archived service and parts prices and their change history."
            : "Review and update Small Bike, Large Bike, and parts prices. Every
change is recorded with a reason."
        }
        action={<PriceViewToggle archived={showArchives}
onChange={setShowArchives} />}
      />
      <Tabs defaultValue="services">
        {/* Two equal tabs on phones, where a long label may wrap. */}
        <TabsList className="mb-4 grid w-full grid-cols-2 sm:inline-flex
sm:w-auto">
          <TabsTrigger
            value="services"
            className="min-w-0 gap-2 self-stretch text-center leading-tight
whitespace-normal"
          >
            <Wrench className="hidden size-4 shrink-0 sm:block"
aria-hidden="true" /> Services
          </TabsTrigger>
          <TabsTrigger
            value="products"
            className="min-w-0 gap-2 self-stretch text-center leading-tight
whitespace-normal"
          >
            <Package className="hidden size-4 shrink-0 sm:block"
aria-hidden="true" /> Parts &
```

```
            Accessories
          </TabsTrigger>
        </TabsList>
        <TabsContent value="services">
          <PriceCatalog table="services" archived={showArchives} />
        </TabsContent>
        <TabsContent value="products">
          <PriceCatalog table="products" archived={showArchives} />
        </TabsContent>
      </Tabs>
    </AdminPage>
  );
}

function PriceViewToggle({
  archived,
  onChange,
}: {
  archived: boolean;
  onChange: (archived: boolean) => void;
}) {
  return (
    <div
      role="group"
      aria-label="Price list"
      className="inline-flex rounded-md border border-border bg-card/60 p-1"
    >
      {[
        { value: false, label: "Active prices" },
        { value: true, label: "Archived" },
      ].map((option) => (
        <button
          key={option.label}
          type="button"
          aria-pressed={archived === option.value}
          onClick={() => onChange(option.value)}
          className={cn(
            "h-9 rounded px-3 text-sm font-medium transition-colors
focus-visible:ring-1 focus-visible:ring-ring focus-visible:outline-none md:h-8",
            archived === option.value
              ? "bg-primary text-primary-foreground shadow-sm"
              : "text-muted-foreground hover:bg-secondary hover:text-foreground",
          )}
        >
          {option.label}
```

```
        </button>
      ))}
    </div>
  );
}

function PriceCatalog({ table, archived }: { table: PriceTable; archived: boolean })
{
  const queryClient = useQueryClient();
  const [selectedId, setSelectedId] = useState<string | null>(null);
  const [drafts, setDrafts] = useState<Record<string, PriceDraft>>({});
  const [errors, setErrors] = useState<Record<string, string>>({});
  const [savedMessages, setSavedMessages] = useState<Record<string,
string>>({});
  const [search, setSearch] = useState("");
  const [category, setCategory] = useState(ALL_CATEGORIES);

  const items = useQuery({
    queryKey: ["prices", table, archived],
    queryFn: async (): Promise<PriceItem[]> => {
      if (table === "services") {
        const { data, error } = await supabase
          .from("services")
          .select(
            "id,name,category,price,duration_minutes,large_bike_price,large_bike_du
ration_minutes,is_active,is_archived,created_at,updated_at,archived_at",
          )
          .eq("is_archived", archived)
          .order("name");
        if (error) throw error;
        return (data ?? []).map((item) => ({
          ...item,
          price: Number(item.price),
          large_bike_price: item.large_bike_price === null ? null :
Number(item.large_bike_price),
        }));
      }
      const { data, error } = await supabase
        .from("products")
        .select("id,name,category,price,is_active,is_archived,created_at,updated_at,
archived_at")
        .in("category", ["part", "accessory"])
        .eq("is_archived", archived)
        .order("name");
      if (error) throw error;
```

```
      return (data ?? []).map((item) => ({
        ...item,
        price: Number(item.price),
        duration_minutes: null,
        large_bike_price: null,
        large_bike_duration_minutes: null,
      }));
    },
  });

  const history = useQuery({
    queryKey: ["price-history-index", table],
    queryFn: async (): Promise<PriceHistory[]> => {
      const { data, error } = await supabase
        .from("price_history")
        .select(
          "id,item_id,configuration_id,configuration_label,old_price,new_price,reason
,changed_by_email,created_at",
        )
        .eq("table_name", table)
        .order("created_at", { ascending: false });
      if (error) throw error;
      return (data ?? []) as PriceHistory[];
    },
  });

  useEffect(() => {
    setSelectedId(null);
    setDrafts({});
    setErrors({});
    setSavedMessages({});
    setSearch("");
    setCategory(ALL_CATEGORIES);
  }, [archived, table]);

  // Read the selected item from the latest query data so the editor shows the
  // saved price immediately after a change, not the value it was opened with.
  const selected = (items.data ?? []).find((item) => item.id === selectedId) ?? null;

  const categories = useMemo(
    () => Array.from(new Set((items.data ?? []).map((item) =>
item.category))).sort(),
    [items.data],
  );
  const normalizedSearch = search.trim().toLowerCase();
```

```
  const filteredItems = (items.data ?? []).filter(
    (item) =>
      (category === ALL_CATEGORIES || item.category === category) &&
      (!normalizedSearch ||
item.name.toLowerCase().includes(normalizedSearch)),
  );
  const latestChangeByItem = useMemo(() => {
    const latest = new Map<string, string>();
    for (const record of history.data ?? []) {
      if (!latest.has(record.item_id)) latest.set(record.item_id, record.created_at);
    }
    return latest;
  }, [history.data]);

  function configurationsFor(item: PriceItem): PriceConfiguration[] {
    if (table === "products")
      return [
        {
          id: item.id,
          historyId: item.id,
          kind: "product",
          label: "Price",
          price: item.price,
          durationMinutes: null,
        },
      ];
    const configurations: PriceConfiguration[] = [
      {
        id: `small-${item.id}`,
        historyId: null,
        kind: "small_bike",
        label: "Small Bike (125cc and below)",
        price: item.price,
        durationMinutes: item.duration_minutes,
      },
    ];
    if (item.large_bike_price !== null && item.large_bike_duration_minutes !==
null)
      configurations.push({
        id: `large-${item.id}`,
        historyId: item.id,
        kind: "large_bike",
        label: "Large Bike (126cc and above)",
        price: item.large_bike_price,
        durationMinutes: item.large_bike_duration_minutes,
```

```
    });
  return configurations;
 }

 const save = useMutation({
  mutationFn: async ({
   item,
   configuration,
   price,
   reason,
  }: {
   item: PriceItem;
   configuration: PriceConfiguration;
   price: number;
   reason: string;
  }) => {
   if (table === "services") {
    const update =
     configuration.kind === "large_bike" ? { large_bike_price: price } : { price };
    const { error } = await supabase.from("services").update(update).eq("id",
item.id);
    if (error) throw error;
   } else {
    const { error } = await supabase.from("products").update({ price }).eq("id",
item.id);
    if (error) throw error;
   }
   const { data: user } = await supabase.auth.getUser();
   const { error } = await supabase.from("price_history").insert({
    table_name: table,
    item_id: item.id,
    item_name: item.name,
    configuration_id: configuration.historyId,
    configuration_label: configuration.label,
    old_price: configuration.price,
    new_price: price,
    reason,
    changed_by: user.user?.id ?? null,
    changed_by_email: user.user?.email ?? null,
   });
   if (error) throw error;
  },
  onSuccess: (_data, variables) => {
   const configurationId = variables.configuration.id;
   setDrafts(({ [configurationId]: _saved, ...current }) => current);
```

```
      setSavedMessages((current) => ({
        ...current,
        [configurationId]: `Saved. The price changed from
${formatPHP(variables.configuration.price)} to ${formatPHP(variables.price)}.`,
      }));
      queryClient.invalidateQueries({ queryKey: ["prices"], exact: false });
      queryClient.invalidateQueries({ queryKey: ["price-history-index", table] });
      queryClient.invalidateQueries({ queryKey: ["services"], exact: false });
      queryClient.invalidateQueries({ queryKey: ["products"], exact: false });
    },
    onError: (_error, variables) => {
      setErrors((current) => ({
        ...current,
        [variables.configuration.id]:
          "Could not save this price. Please review the value and try again.",
      }));
    },
  });

  function draftFor(configuration: PriceConfiguration): PriceDraft {
    return drafts[configuration.id] ?? { price: String(configuration.price), reason: ""
};
  }
  function setDraft(configuration: PriceConfiguration, changes:
Partial<PriceDraft>) {
    setDrafts((current) => ({
      ...current,
      [configuration.id]: { ...draftFor(configuration), ...changes },
    }));
    setErrors((current) => ({ ...current, [configuration.id]: "" }));
    setSavedMessages((current) => ({ ...current, [configuration.id]: "" }));
  }
  function resetDraft(configuration: PriceConfiguration) {
    setDrafts(({ [configuration.id]: _discarded, ...current }) => current);
    setErrors((current) => ({ ...current, [configuration.id]: "" }));
  }
  function requestSave(item: PriceItem, configuration: PriceConfiguration) {
    const draft = draftFor(configuration);
    const price = Number(draft.price);
    if (!draft.price.trim() || !Number.isFinite(price) || price < 0)
      return setErrors((current) => ({ ...current, [configuration.id]: "Enter a valid
price." }));
    if (!draft.reason.trim())
      return setErrors((current) => ({
        ...current,
```

```
        [configuration.id]: "Enter a reason for this change.",
      }));
    if (price === configuration.price)
      return setErrors((current) => ({
        ...current,
        [configuration.id]: "Enter a different price.",
      }));
    if (
      configuration.kind === "small_bike" &&
      item.large_bike_price !== null &&
      price >= item.large_bike_price
    )
      return setErrors((current) => ({
        ...current,
        [configuration.id]: "Small Bike pricing must remain lower than the Large Bike
price.",
      }));
    if (configuration.kind === "large_bike" && price <= item.price)
      return setErrors((current) => ({
        ...current,
        [configuration.id]: "Large Bike pricing must be higher than the Small Bike
price.",
      }));
    save.mutate({ item, configuration, price, reason: draft.reason.trim() });
  }
  function configurationHistory(item: PriceItem, configuration: PriceConfiguration)
{
    return (history.data ?? []).filter(
      (record) => record.item_id === item.id && record.configuration_id ===
configuration.historyId,
    );
  }

  if (selected)
    return (
      <PriceEditor
        item={selected}
        configurations={configurationsFor(selected)}
        historyFor={configurationHistory}
        historyLoading={history.isLoading}
        draftFor={draftFor}
        setDraft={setDraft}
        resetDraft={resetDraft}
        errors={errors}
        savedMessages={savedMessages}
```

```
        onSave={requestSave}
        saving={save.isPending}
        savingConfigurationId={save.isPending ? save.variables?.configuration.id :
undefined}
        readOnly={archived}
        table={table}
        onBack={() => {
          setSelectedId(null);
          setSavedMessages({});
        }}
      />
    );

  const itemNoun = table === "services" ? "services" : "parts and accessories";
  const hasFilters = Boolean(normalizedSearch) || category !==
ALL_CATEGORIES;
  const resetFilters = () => {
    setSearch("");
    setCategory(ALL_CATEGORIES);
  };
  const lastChangeLabel = (itemId: string) => {
    if (history.isLoading) return "…";
    const lastChange = latestChangeByItem.get(itemId);
    return lastChange ? formatDate(lastChange) : "No changes yet";
  };
  const editButton = (item: PriceItem) => (
    <Button
      size="sm"
      variant="outline"
      onClick={() => setSelectedId(item.id)}
      aria-label={`${archived ? "View prices for" : "Edit prices for"} ${item.name}`}
    >
      {archived ? <Eye /> : <Pencil />}
      {archived ? "View" : table === "services" ? "Edit prices" : "Edit price"}
    </Button>
  );

  return (
    <div>
      <ListToolbar
        summary={
          items.data && `Showing ${filteredItems.length} of ${items.data.length}
${itemNoun}`
        }
      >
```

```
        <ToolbarSearch
          value={search}
          onChange={setSearch}
          placeholder={table === "services" ? "Search services" : "Search parts and
accessories"}
          label={table === "services" ? "Search services" : "Search parts and
accessories"}
        />
        <ToolbarFilters>
          <Select value={category} onValueChange={setCategory}>
            <SelectTrigger
              aria-label="Filter by category"
              className="col-span-2 w-full capitalize sm:w-52"
            >
              <SelectValue placeholder="All categories" />
            </SelectTrigger>
            <SelectContent>
              <SelectItem value={ALL_CATEGORIES}>All categories</SelectItem>
              {categories.map((value) => (
                <SelectItem key={value} value={value} className="capitalize">
                  {value}
                </SelectItem>
              ))}
            </SelectContent>
          </Select>
          {hasFilters && <ResetFiltersButton onClick={resetFilters} />}
        </ToolbarFilters>
      </ListToolbar>

      <ListCard>
        {filteredItems.length > 0 && (
          <>
            {/* Phones and tablets: one card per item, so nothing scrolls sideways
                next to the admin sidebar. */}
            <ul className="divide-y divide-border/70 lg:hidden">
              {filteredItems.map((item) => (
                <li
                  key={item.id}
                  className="grid gap-3 p-4 sm:grid-cols-[minmax(0,1fr)_auto]
sm:items-center sm:gap-x-6"
                >
                  <div className="min-w-0 space-y-2.5">
                    <div className="flex items-start justify-between gap-3">
                      <ItemName item={item} archived={archived} />
                      <span className="sm:hidden">
```

```
                  <StatusBadge archived={archived} active={item.is_active} />
                </span>
              </div>
              <dl className="grid grid-cols-2 gap-3 text-sm sm:flex sm:flex-wrap
sm:gap-x-8">
                {table === "services" ? (
                  <>
                    <PriceFact
                      label="Small Bike"
                      price={item.price}
                      durationMinutes={item.duration_minutes}
                    />
                    <PriceFact
                      label="Large Bike"
                      price={item.large_bike_price}
                      durationMinutes={item.large_bike_duration_minutes}
                    />
                  </>
                ) : (
                  <PriceFact label="Price" price={item.price} durationMinutes={null}
/>
                )}
              </dl>
            </div>
            <div className="flex flex-wrap items-center justify-between gap-2
sm:flex-col sm:items-end">
              <span className="hidden sm:block">
                <StatusBadge archived={archived} active={item.is_active} />
              </span>
              <p className="text-xs text-muted-foreground sm:text-right">
                Last price change: {lastChangeLabel(item.id)}
              </p>
              {editButton(item)}
            </div>
          </li>
        ))}
      </ul>

      {/* Laptops and desktops: a compact table. */}
      <Table className="hidden w-full table-fixed lg:table">
        <TableHeader>
          <TableRow>
            <TableHead className="pl-5">
              {table === "services" ? "Service" : "Part / Accessory"}
            </TableHead>
```

```
              {table === "services" ? (
                <>
                  <TableHead className="w-32 text-right">Small
Bike</TableHead>
                  <TableHead className="w-32 text-right">Large
Bike</TableHead>
                </>
              ) : (
                <TableHead className="w-36 text-right">Price</TableHead>
              )}
              <TableHead className="hidden w-44 xl:table-cell">Last price
change</TableHead>
              <TableHead className="w-28 text-center">Status</TableHead>
              <TableHead className="w-40 pr-5 text-right">
                <span className="sr-only">Actions</span>
              </TableHead>
            </TableRow>
          </TableHeader>
          <TableBody>
            {filteredItems.map((item) => (
              <TableRow key={item.id} className="align-top [&>td]:py-3">
                <TableCell className="pl-5">
                  <ItemName item={item} archived={archived} />
                </TableCell>
                {table === "services" ? (
                  <>
                    <TableCell className="text-right">
                      <PriceSummary
                        price={item.price}
                        durationMinutes={item.duration_minutes}
                      />
                    </TableCell>
                    <TableCell className="text-right">
                      <PriceSummary
                        price={item.large_bike_price}
                        durationMinutes={item.large_bike_duration_minutes}
                      />
                    </TableCell>
                  </>
                ) : (
                  <TableCell className="text-right">
                    <PriceSummary price={item.price} durationMinutes={null} />
                  </TableCell>
                )}
                <TableCell className="hidden text-xs text-muted-foreground
```

```
xl:table-cell">
                    {lastChangeLabel(item.id)}
                  </TableCell>
                  <TableCell className="text-center">
                    <StatusBadge archived={archived} active={item.is_active} />
                  </TableCell>
                  <TableCell className="pr-5
text-right">{editButton(item)}</TableCell>
                </TableRow>
              ))}
            </TableBody>
          </Table>
        </>
      )}

      {items.isLoading && <ListLoading label="Loading prices" />}
      {items.isError && (
        <ListError message="Prices could not be loaded." onRetry={() => void
items.refetch()} />
      )}
      {items.isSuccess && filteredItems.length === 0 && (
        <ListEmpty
          icon={Tags}
          title={
            hasFilters
              ? `No ${itemNoun} match your search.`
              : archived
                ? `There are no archived ${itemNoun}.`
                : `No ${itemNoun} yet.`
          }
          description={hasFilters ? "Try a different search or category." : undefined}
          action={hasFilters ? <ResetFiltersButton onClick={resetFilters} /> :
undefined}
        />
      )}
    </ListCard>
  </div>
 );
}

function ItemName({ item, archived }: { item: PriceItem; archived: boolean }) {
 return (
  <div className="min-w-0">
    <p className="font-medium break-words">{item.name}</p>
    <p className="mt-0.5 text-xs text-muted-foreground">
```

```
        <span className="capitalize">{item.category}</span> · {archived ?
"Archived" : "Added"}{" "}
        {formatShortDate(item.archived_at ?? item.created_at)}
      </p>
    </div>
  );
}

function StatusBadge({ archived, active }: { archived: boolean; active: boolean }) {
  return (
    <Badge
      variant="outline"
      className={
        archived ? "uppercase text-muted-foreground" : `uppercase
${activeStatusTone(active)}`
      }
    >
      {archived ? "Archived" : active ? "Active" : "Inactive"}
    </Badge>
  );
}

function PriceFact({
  label,
  price,
  durationMinutes,
}: {
  label: string;
  price: number | null;
  durationMinutes: number | null;
}) {
  return (
    <div className="min-w-0">
      <dt className="text-xs tracking-wide text-muted-foreground
uppercase">{label}</dt>
      <dd className="mt-0.5">
        {price === null ? (
          <span className="text-muted-foreground">Not set</span>
        ) : (
          <>
            <span className="font-medium
tabular-nums">{formatPHP(price)}</span>
            {durationMinutes !== null && (
              <span className="text-xs text-muted-foreground"> · {durationMinutes}
min</span>
```

```
            )}
          </>
        )}
      </dd>
    </div>
  );
}

function PriceSummary({
  price,
  durationMinutes,
}: {
  price: number | null;
  durationMinutes: number | null;
}) {
  if (price === null) return <span className="text-xs text-muted-foreground">Not
set</span>;
  return (
    <span className="inline-flex flex-col items-end">
      <span className="font-medium text-foreground
tabular-nums">{formatPHP(price)}</span>
      {durationMinutes !== null && (
        <span className="text-xs text-muted-foreground">{durationMinutes}
min</span>
      )}
    </span>
  );
}

function PriceEditor({
  item,
  configurations,
  historyFor,
  historyLoading,
  draftFor,
  setDraft,
  resetDraft,
  errors,
  savedMessages,
  onSave,
  saving,
  savingConfigurationId,
  readOnly,
  table,
  onBack,
```

```
}: {
  item: PriceItem;
  configurations: PriceConfiguration[];
  historyFor: (item: PriceItem, configuration: PriceConfiguration) => PriceHistory[];
  historyLoading: boolean;
  draftFor: (configuration: PriceConfiguration) => PriceDraft;
  setDraft: (configuration: PriceConfiguration, changes: Partial<PriceDraft>) =>
void;
  resetDraft: (configuration: PriceConfiguration) => void;
  errors: Record<string, string>;
  savedMessages: Record<string, string>;
  onSave: (item: PriceItem, configuration: PriceConfiguration) => void;
  saving: boolean;
  savingConfigurationId: string | undefined;
  readOnly: boolean;
  table: PriceTable;
  onBack: () => void;
}) {
  return (
    <div className="space-y-5">
      <Button variant="ghost" size="sm" onClick={onBack} disabled={saving}
className="-ml-2">
        <ArrowLeft /> Back to {table === "services" ? "services" : "parts &
accessories"}
      </Button>

      <Card className="border-border/70 bg-card/60">
        <CardContent className="flex flex-wrap items-start justify-between gap-4
p-4 sm:p-6">
          <div className="min-w-0">
            <p className="text-xs tracking-widest text-muted-foreground
uppercase">
              {table === "services" ? "Service" : "Part / Accessory"} ·{" "}
              <span className="capitalize">{item.category}</span>
            </p>
            <h2 className="mt-1 font-display text-2xl break-words
uppercase">{item.name}</h2>
            <p className="mt-1 text-sm text-muted-foreground">
              {readOnly
                ? "Archived pricing details. Prices can't be changed here."
                : table === "services"
                  ? "Each bike size is priced separately. Every change is saved with its
reason in the price history."
                  : "Every price change is saved with its reason in the price history."}
            </p>
```

```
          </div>
          <Badge
            variant="outline"
            className={
              readOnly
                ? "uppercase text-muted-foreground"
                : `uppercase ${activeStatusTone(item.is_active)}`
            }
          >
            {readOnly ? "Archived" : item.is_active ? "Active" : "Inactive"}
          </Badge>
        </CardContent>
      </Card>

      {table === "services" && !readOnly && (
        <p className="flex items-start gap-2 text-sm text-muted-foreground">
          <Info className="mt-0.5 size-4 shrink-0 text-primary" aria-hidden="true"
/>
          {configurations.length > 1
            ? "The Small Bike price must stay lower than the Large Bike price."
            : "This service has no Large Bike price. You can add one by editing the
service on the Services page."}
        </p>
      )}

      <div className={cn("grid gap-5", configurations.length > 1 &&
"xl:grid-cols-2")}>
        {configurations.map((configuration) => (
          <PriceConfigurationCard
            key={configuration.id}
            configuration={configuration}
            history={historyFor(item, configuration)}
            historyLoading={historyLoading}
            draft={draftFor(configuration)}
            onDraftChange={(changes) => setDraft(configuration, changes)}
            onReset={() => resetDraft(configuration)}
            error={errors[configuration.id]}
            savedMessage={savedMessages[configuration.id]}
            onSave={() => onSave(item, configuration)}
            saving={saving}
            isSaving={savingConfigurationId === configuration.id}
            readOnly={readOnly}
          />
        ))}
      </div>
```

```
    </div>
  );
}

function PriceConfigurationCard({
  configuration,
  history,
  historyLoading,
  draft,
  onDraftChange,
  onReset,
  error,
  savedMessage,
  onSave,
  saving,
  isSaving,
  readOnly,
}: {
  configuration: PriceConfiguration;
  history: PriceHistory[];
  historyLoading: boolean;
  draft: PriceDraft;
  onDraftChange: (changes: Partial<PriceDraft>) => void;
  onReset: () => void;
  error: string | undefined;
  savedMessage: string | undefined;
  onSave: () => void;
  saving: boolean;
  isSaving: boolean;
  readOnly: boolean;
}) {
  const [showAllHistory, setShowAllHistory] = useState(false);
  const priceInputId = `${configuration.id}-new-price`;
  const reasonInputId = `${configuration.id}-reason`;
  const latest = history[0];
  const draftPrice = Number(draft.price);
  const dirty = draft.price !== String(configuration.price) || draft.reason !== "";
  const difference =
    draft.price.trim() && Number.isFinite(draftPrice) && draftPrice >= 0
      ? draftPrice - configuration.price
      : 0;
  const visibleHistory = showAllHistory ? history : history.slice(0,
HISTORY_PREVIEW_COUNT);

  return (
```

```
   <Card className="border-border/70 bg-card/60">
     <CardContent className="space-y-5 p-4 sm:p-6">
       <div className="flex flex-wrap items-start justify-between gap-x-6
gap-y-3">
         <div className="min-w-0">
           <h3 className="font-display text-lg tracking-wide
uppercase">{configuration.label}</h3>
           {configuration.durationMinutes !== null && (
             <p className="mt-0.5 flex items-center gap-1.5 text-sm
text-muted-foreground">
               <Clock className="size-3.5" aria-hidden="true" />
{configuration.durationMinutes}{" "}
               min service time
             </p>
           )}
           <p className="mt-1 text-xs text-muted-foreground">
             Last change: {latest ? formatDate(latest.created_at) : "No changes
recorded yet"}
           </p>
         </div>
         <div className="text-left sm:text-right">
           <p className="text-xs tracking-widest text-muted-foreground
uppercase">Current price</p>
           <p className="font-display text-3xl text-primary tabular-nums">
             {formatPHP(configuration.price)}
           </p>
         </div>
       </div>

       {!readOnly && (
         <form
           noValidate
           className="space-y-4 rounded-lg border border-border/70
bg-background/40 p-4"
           onSubmit={(event) => {
             event.preventDefault();
             onSave();
           }}
         >
           <div className="grid gap-4
sm:grid-cols-[minmax(0,13rem)_minmax(0,1fr)]">
             <div className="space-y-1.5">
               <Label htmlFor={priceInputId}>New price</Label>
               <div className="relative">
                 <span
```

```
                aria-hidden="true"
                className="pointer-events-none absolute top-1/2 left-3
-translate-y-1/2 text-sm text-muted-foreground"
              >
                ₱
              </span>
              <Input
                id={priceInputId}
                type="number"
                inputMode="decimal"
                min="0"
                step="0.01"
                value={draft.price}
                onChange={(event) => onDraftChange({ price: event.target.value })}
                aria-invalid={Boolean(error)}
                className="pl-7 tabular-nums"
              />
            </div>
            {difference !== 0 && (
              <p
                className={cn(
                  "text-xs tabular-nums",
                  difference > 0
                    ? "text-emerald-600 dark:text-emerald-400"
                    : "text-destructive",
                )}
              >
                {difference > 0 ? "+" : "−"}
                {formatPHP(Math.abs(difference))}
                {configuration.price > 0 &&
                  ` (${difference > 0 ? "+" : "−"}${Math.abs((difference /
configuration.price) * 100).toFixed(1)}%)`}{" "}
                from current price
              </p>
            )}
          </div>
          <div className="space-y-1.5">
            <Label htmlFor={reasonInputId}>Reason for change</Label>
            <Input
              id={reasonInputId}
              value={draft.reason}
              onChange={(event) => onDraftChange({ reason: event.target.value })}
              placeholder="e.g. Updated supplier cost"
              aria-invalid={Boolean(error)}
            />
```

```
            </div>
          </div>
          <FieldError message={error} />
          {savedMessage && (
            <p
              role="status"
              className="flex items-center gap-1.5 text-xs text-emerald-600
dark:text-emerald-400"
            >
              <Check className="size-3.5 shrink-0" aria-hidden="true" />
{savedMessage}
            </p>
          )}
          <div className="flex flex-col-reverse gap-2 sm:flex-row sm:items-center
sm:justify-end">
            {dirty && (
              <Button
                type="button"
                variant="ghost"
                onClick={onReset}
                disabled={saving}
                className="w-full sm:w-auto"
              >
                <RotateCcw /> Discard changes
              </Button>
            )}
            <Button type="submit" disabled={saving} className="w-full
sm:w-auto">
              {isSaving ? <Loader2 className="animate-spin" /> : <Check />}
              {isSaving ? "Saving…" : "Save price"}
            </Button>
          </div>
        </form>
      )}

      <div>
        <h4 className="flex items-center gap-2 font-sans text-xs font-semibold
tracking-widest text-muted-foreground uppercase">
          <History className="size-3.5" aria-hidden="true" /> Price history
        </h4>
        {historyLoading ? (
          <p className="mt-2 text-sm text-muted-foreground">Loading
history…</p>
        ) : history.length === 0 ? (
          <p className="mt-2 text-sm text-muted-foreground">No price changes
```

```
recorded yet.</p>
        ) : (
          <>
            <ol className="mt-2 divide-y divide-border/60 overflow-hidden
rounded-lg border border-border/70">
              {visibleHistory.map((record) => (
                <li key={record.id} className="space-y-0.5 px-3 py-2.5 text-sm">
                  <div className="flex flex-wrap items-baseline justify-between
gap-x-3 gap-y-0.5">
                    <span className="tabular-nums">
                      <span className="text-muted-foreground line-through">
                        {formatPHP(record.old_price)}
                      </span>{" "}
                      <span aria-label="changed to">→</span>{" "}
                      <span className="font-medium text-foreground">
                        {formatPHP(record.new_price)}
                      </span>
                    </span>
                    <span className="text-xs text-muted-foreground">
                      {formatDate(record.created_at)}
                    </span>
                  </div>
                  {record.reason && (
                    <p className="break-words
text-muted-foreground">{record.reason}</p>
                  )}
                  {record.changed_by_email && (
                    <p className="text-xs break-all text-muted-foreground">
                      by {record.changed_by_email}
                    </p>
                  )}
                </li>
              ))}
            </ol>
            {history.length > HISTORY_PREVIEW_COUNT && (
              <Button
                type="button"
                variant="link"
                size="sm"
                className="mt-1 h-auto px-0 md:h-auto"
                onClick={() => setShowAllHistory((current) => !current)}
              >
                {showAllHistory ? "Show fewer changes" : `Show all ${history.length}
changes`}
              </Button>
```

```
            )}
          </>
        )}
      </div>
    </CardContent>
  </Card>
 );
}

function formatDate(value: string | null) {
 return value
  ? new Date(value).toLocaleString("en-PH", { dateStyle: "medium", timeStyle:
"short" })
  : "—";
}

function formatShortDate(value: string | null) {
 return value
  ? new Date(value).toLocaleDateString("en-PH", {
     day: "2-digit",
     month: "short",
     year: "numeric",
    })
  : "—";
}
```

src/routes/_authenticated/admin/reports.tsx

```
import { useQuery } from "@tanstack/react-query";
import { createFileRoute } from "@tanstack/react-router";
import { format } from "date-fns";
import {
  BarChart3,
  CalendarDays,
  CalendarRange,
  CheckCircle2,
  ChevronDown,
  CircleDollarSign,
  Filter,
  Info,
  ListFilter,
  Printer,
  RotateCcw,
} from "lucide-react";
import type { LucideIcon } from "lucide-react";
import type { DateRange } from "react-day-picker";
import { useEffect, useMemo, useState } from "react";

import {
  AdminPage,
  DataList,
  type DataListColumn,
  InlineAlert,
  ListCard,
  ListCardTitle,
  ListEmpty,
  ListToolbar,
  ToolbarFilters,
} from "@/components/admin/data-list";
import { PageHeader } from "@/components/admin/page-header";
import { PrintDocument, PrintTable } from
"@/components/admin/print-document";
import { PaginationControls } from "@/components/admin/pagination-controls";
import { Button } from "@/components/ui/button";
import { Calendar } from "@/components/ui/calendar";
import { Card, CardContent } from "@/components/ui/card";
import { Checkbox } from "@/components/ui/checkbox";
import {
  Dialog,
  DialogContent,
  DialogDescription,
```

```
  DialogFooter,
  DialogHeader,
  DialogTitle,
} from "@/components/ui/dialog";
import { FieldError } from "@/components/ui/field-error";
import { Label } from "@/components/ui/label";
import { Input } from "@/components/ui/input";
import { Popover, PopoverContent, PopoverTrigger } from
"@/components/ui/popover";
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@/components/ui/select";
import { supabase } from "@/integrations/supabase/client";
import {
  calendarDateToIso,
  calendarRangeToIso,
  isoDateToCalendarDate,
  manilaMonthDateRange,
  manilaWeekDateRange,
  manilaYearDateRange,
} from "@/lib/admin-date-range";
import { recordAdminActivityEvent } from "@/lib/admin-activity";
import { formatBusinessTimestamp } from "@/lib/export-branding";
import { usePrintWhenReady } from "@/lib/print-document";
import { formatDateLong, formatPHP, manilaNow } from "@/lib/shop";
import { cn } from "@/lib/utils";

export const Route = createFileRoute("/_authenticated/admin/reports")({
  component: ReportsPage,
});

const periodOptions = [
  { value: "today", label: "Today" },
  { value: "this_week", label: "This week" },
  { value: "this_month", label: "This month" },
  { value: "this_year", label: "This year" },
  { value: "custom", label: "Custom dates" },
] as const;

type PeriodPreset = (typeof periodOptions)[number]["value"];
type ReportFilters = {
```

```
  periodPreset: PeriodPreset;
  from: string;
  to: string;
  category: string;
  /** Empty means all services. */
  serviceNames: string[];
};

type CatalogService = { id: string; name: string; category: string };
type FinancialTransactionRow = {
  id: string;
  reference_code: string;
  transaction_date: string;
  services: string;
  products: string;
  service_amount: number | string;
  product_amount: number | string;
  total_amount: number | string;
};
type FinancialMetrics = {
  total_revenue: number | string;
  service_revenue: number | string;
  product_revenue: number | string;
  completed_transactions: number;
  total_completed_services: number;
  total_products_sold: number;
};
type ServicePerformanceRow = {
  service_name: string;
  completed_count: number;
  revenue: number | string;
};
type ProductSalesRow = {
  product_name: string;
  quantity_sold: number;
  unit_price: number | string;
  total_sales: number | string;
};
type RevenuePeriodRow = {
  period: string;
  completed_transactions: number;
  service_revenue: number | string;
  product_revenue: number | string;
  total_revenue: number | string;
};
```

```
type FinancialReportPage = {
  rows: FinancialTransactionRow[];
  total: number;
  metrics: FinancialMetrics;
  service_performance: ServicePerformanceRow[];
  product_sales: ProductSalesRow[];
  daily_revenue: RevenuePeriodRow[];
  monthly_revenue: RevenuePeriodRow[];
};
type ReportSummaryCard = {
  label: string;
  value: string;
  detail: string;
  tone: "primary" | "chart" | "success" | "accent";
};
type ReportData = {
  headers: string[];
  rows: string[][];
  cards: ReportSummaryCard[];
  totalAmount: number;
};
type PrintPayload = {
  data: ReportData;
  generatedAt: string;
  period: string;
  scope: string;
  title: string;
  financial: {
    servicePerformance: ServicePerformanceRow[];
    productSales: ProductSalesRow[];
    dailyRevenue: RevenuePeriodRow[];
  };
};
function ReportsPage() {
  const pageSize = 10;
  const today = manilaNow().date;
  const initialFilters = getInitialFilters(today);
  const [draftFilters, setDraftFilters] = useState<ReportFilters>(initialFilters);
  const [filters, setFilters] = useState<ReportFilters>(initialFilters);
  const [page, setPage] = useState(0);
  const [isPrinting, setIsPrinting] = useState(false);
  const [printError, setPrintError] = useState<string | null>(null);
  const [printPayload, setPrintPayload] = useState<PrintPayload | null>(null);
  const [revenuePeriod, setRevenuePeriod] = useState<"daily" |
"monthly">("daily");
```

```
  const [serviceListDialog, setServiceListDialog] = useState<{
    referenceCode: string;
    services: string[];
  } | null>(null);
  const { from, to, category, serviceNames } = filters;
  const customDateError =
    draftFilters.periodPreset === "custom" && (!draftFilters.from || !draftFilters.to)
      ? "Select both a start and end date."
      : draftFilters.from > draftFilters.to
        ? "The end date must be on or after the start date."
        : undefined;

  const catalog = useQuery({
    queryKey: ["report-service-catalog"],
    queryFn: async () => {
      const { data, error } = await supabase
        .from("services")
        .select("id,name,category")
        .order("category")
        .order("name");
      if (error) throw error;
      return (data ?? []) as CatalogService[];
    },
  });

  const data = useQuery({
    queryKey: ["reports", { from, to, category, serviceNames, page }],
    queryFn: () =>
      getReportPage({
        from,
        to,
        category,
        serviceNames,
        limit: pageSize,
        offset: page * pageSize,
      }),
  });

  const categories = useMemo(
    () =>
      Array.from(new Set((catalog.data ?? []).map((item) =>
item.category).filter(Boolean))).sort(),
    [catalog.data],
  );
  const serviceOptions = useMemo(
```

```
    () =>
      (catalog.data ?? [])
        .filter(
          (item) => draftFilters.category === "all" || item.category ===
draftFilters.category,
        )
        .sort((first, second) => first.name.localeCompare(second.name)),
    [catalog.data, draftFilters.category],
  );
  const financialReport = data.data;
  const metrics = financialReport?.metrics;
  const reportData = makeReportData(financialReport?.rows ?? [], metrics);
  const reportTitle = "Financial Report";
  const revenueRows =
    revenuePeriod === "daily"
      ? (financialReport?.daily_revenue ?? [])
      : (financialReport?.monthly_revenue ?? []);
  const appliedFilters = [
    { label: "Completed on", value: formatDateRange(from, to) },
    { label: "Service category", value: category === "all" ? "All categories" :
category },
    {
      label: serviceNames.length > 1 ? "Services" : "Specific service",
      value: serviceNames.length ? serviceNames.join(", ") : "All services",
    },
    { label: "Transactions", value: "Completed only" },
  ];
  const reportScope = appliedFilters
    .slice(1)
    .map((filter) => `${filter.label}: ${filter.value}`)
    .join(" · ");

  function updatePeriod(value: PeriodPreset) {
    setDraftFilters((current) => {
      if (value === "custom") return { ...current, periodPreset: value };
      const range = periodRange(value, today);
      return {
        ...current,
        periodPreset: value,
        ...range,
      };
    });
  }

  function applyFilters() {
```

```
    if (customDateError) return;
    setFilters({ ...draftFilters });
    setPage(0);
  }

  function resetFilters() {
    const nextFilters = getInitialFilters(today);
    setDraftFilters(nextFilters);
    setFilters(nextFilters);
    setPage(0);
  }

  usePrintWhenReady(printPayload && `${printPayload.title} -
${printPayload.period}`, () =>
    setPrintPayload(null),
  );

  async function printReport() {
    if (isPrinting) return;
    setIsPrinting(true);
    try {
      setPrintError(null);
      const allRows = await getAllReportRows({ from, to, category, serviceNames
});
      // Print totals and completed metrics are derived from the complete
      // filtered result set, not the currently visible paginated page.
      const completeReportData = makeReportData(allRows.rows,
allRows.metrics, {
        hasCompleteRows: true,
      });
      setPrintPayload({
        data: completeReportData,
        generatedAt: formatBusinessTimestamp(),
        period: formatDateRange(from, to),
        scope: reportScope,
        title: reportTitle,
        financial: {
          servicePerformance: allRows.servicePerformance,
          productSales: allRows.productSales,
          dailyRevenue: allRows.dailyRevenue,
        },
      });
      try {
        await recordAdminActivityEvent({
          action: "printed",
```

```
          resourceType: "Reports",
          targetLabel: reportTitle,
          summary: `Opened the print dialog for ${allRows.total} completed
transactions.`,
          changedFields: ["report_type", "date_range", "print_format"],
        });
      } catch (error) {
        // An audit-log failure must not prevent a valid report from printing.
        console.error("Could not record report print activity:", error);
      }
    } catch (error) {
      console.error("Could not prepare report for printing:", error);
      setPrintError("Could not prepare this report for printing. Please try again.");
    } finally {
      setIsPrinting(false);
    }
  }

  const rowLabel = "completed transactions";
  const transactionRows: TransactionRow[] = reportData.rows.map((cells, index)
=> ({
    key: `${cells[0]}-${index}`,
    cells,
  }));

  return (
    <AdminPage>
      <PageHeader
        title="Financial Report"
        description="Track completed revenue, service performance, and product
sales for the selected period."
        action={
          <Button
            type="button"
            variant="outline"
            disabled={!data.data?.total || isPrinting}
            className="font-display uppercase"
            onClick={() => void printReport()}
          >
            <Printer /> {isPrinting ? "Preparing print..." : "Print report"}
          </Button>
        }
      />
      {printError && <InlineAlert>{printError}</InlineAlert>}
```

```
    <ListToolbar>
      <ToolbarFilters>
        <Select
          value={draftFilters.category}
          onValueChange={(value) =>
            setDraftFilters((current) => ({ ...current, category: value, serviceNames:
[] }))
          }
        >
          <SelectTrigger aria-label="Filter by service category" className="w-full
sm:w-44">
            <SelectValue />
          </SelectTrigger>
          <SelectContent>
            <SelectItem value="all">All categories</SelectItem>
            {categories.map((value) => (
              <SelectItem key={value} value={value} className="capitalize">
                {value}
              </SelectItem>
            ))}
          </SelectContent>
        </Select>
        <ServiceMultiSelect
          options={serviceOptions}
          value={draftFilters.serviceNames}
          onChange={(serviceNames) =>
            setDraftFilters((current) => ({ ...current, serviceNames }))
          }
        />
        <div className="col-span-2 sm:w-52">
          <PeriodPicker
            value={draftFilters.periodPreset}
            from={draftFilters.from}
            to={draftFilters.to}
            error={customDateError}
            onValueChange={updatePeriod}
            onRangeChange={(range) =>
              setDraftFilters((current) => ({
                ...current,
                periodPreset: "custom",
                ...range,
              }))
            }
          />
        </div>
```

```
        <Button type="button" onClick={applyFilters}
disabled={Boolean(customDateError)}>
          <Filter /> Apply filters
        </Button>
        <Button type="button" variant="outline" onClick={resetFilters}>
          <RotateCcw /> Reset
        </Button>
      </ToolbarFilters>
    </ListToolbar>
    <p className="mb-6 flex items-start gap-2 text-sm text-muted-foreground">
      <Info className="mt-0.5 size-4 shrink-0 text-primary" aria-hidden="true" />
      Only completed appointments count toward revenue. Pending, rejected,
cancelled, and
      rescheduled appointments are left out of every total.
    </p>

    <section aria-labelledby="report-summary-heading" className="mb-6">
      <div className="mb-3 flex flex-wrap items-end justify-between gap-3">
        <div>
          <div className="flex items-center gap-2">
            <BarChart3 className="size-4 text-primary" aria-hidden="true" />
            <h2
              id="report-summary-heading"
              className="font-display text-base tracking-wide uppercase"
            >
              Report summary
            </h2>
          </div>
          <p className="mt-1 text-xs text-muted-foreground" aria-live="polite">
            {reportScope}
          </p>
        </div>
        <div className="rounded-md border border-primary/20 bg-primary/5 px-3
py-2 text-right">
          <p className="text-[10px] font-semibold tracking-wider text-primary
uppercase">
            Reporting period
          </p>
          <p className="font-display text-sm tracking-wide uppercase">
            {formatDateRange(from, to)}
          </p>
        </div>
      </div>
      <div className="grid gap-3 sm:grid-cols-2 xl:grid-cols-3 2xl:grid-cols-6">
        <Stat
```

```
        label="Total Revenue"
        value={formatPHP(metrics?.total_revenue ?? 0)}
        detail="Completed service and product revenue"
        icon={CircleDollarSign}
        iconClassName="bg-emerald-500/15 text-emerald-400"
      />
      <Stat
        label="Service Revenue"
        value={formatPHP(metrics?.service_revenue ?? 0)}
        detail="From completed services"
        icon={CircleDollarSign}
        iconClassName="bg-primary/15 text-primary"
      />
      <Stat
        label="Product Revenue"
        value={formatPHP(metrics?.product_revenue ?? 0)}
        detail="From products sold during service"
        icon={CircleDollarSign}
        iconClassName="bg-accent/15 text-accent"
      />
      <Stat
        label="Completed Transactions"
        value={String(metrics?.completed_transactions ?? 0)}
        detail="Revenue-recognized appointments"
        icon={CheckCircle2}
        iconClassName="bg-chart-4/15 text-chart-4"
      />
      <Stat
        label="Total Completed Services"
        value={String(metrics?.total_completed_services ?? 0)}
        detail="Service entries performed"
        icon={CalendarDays}
        iconClassName="bg-primary/15 text-primary"
      />
      <Stat
        label="Total Products Sold"
        value={String(metrics?.total_products_sold ?? 0)}
        detail="Units recorded during completed service"
        icon={CircleDollarSign}
        iconClassName="bg-accent/15 text-accent"
      />
    </div>
  </section>

  <div className="mb-6 grid gap-4 xl:grid-cols-2">
```

```
        <FinancialTableCard
          title="Service Performance"
          description="Completed services and revenue recognized from their saved
service prices."
          headers={["Service", "Completed", "Revenue"]}
          rows={(financialReport?.service_performance ?? []).map((row) => [
            row.service_name,
            String(row.completed_count),
            formatPHP(row.revenue),
          ])}
          loading={data.isLoading}
          emptyMessage="No completed services match this report selection."
          totalLabel="Total service revenue"
          totalValue={formatPHP(metrics?.service_revenue ?? 0)}
        />
        <FinancialTableCard
          title="Product Sales"
          description="Products recorded during completed appointments, grouped
by their saved selling price."
          headers={["Product", "Qty Sold", "Unit Price", "Total Sales"]}
          rows={(financialReport?.product_sales ?? []).map((row) => [
            row.product_name,
            String(row.quantity_sold),
            formatPHP(row.unit_price),
            formatPHP(row.total_sales),
          ])}
          loading={data.isLoading}
          emptyMessage="No products were sold in completed transactions for this
period."
          totalLabel="Total product revenue"
          totalValue={formatPHP(metrics?.product_revenue ?? 0)}
        />
      </div>

      <ListCard
        className="mb-6"
        header={
          <ListCardTitle
            title="Revenue over time"
            description={`Completed transaction revenue by ${revenuePeriod ===
"daily" ? "day" : "month"}.`}
            action={
              <div className="flex gap-2" role="group" aria-label="Revenue period">
                <Button
                  type="button"
```

```
              size="sm"
              variant={revenuePeriod === "daily" ? "default" : "outline"}
              aria-pressed={revenuePeriod === "daily"}
              onClick={() => setRevenuePeriod("daily")}
            >
              Daily
            </Button>
            <Button
              type="button"
              size="sm"
              variant={revenuePeriod === "monthly" ? "default" : "outline"}
              aria-pressed={revenuePeriod === "monthly"}
              onClick={() => setRevenuePeriod("monthly")}
            >
              Monthly
            </Button>
          </div>
        }
      />
    }
  >
    <DataList
      label="Revenue over time"
      rows={revenueRows}
      rowKey={(row) => row.period}
      loading={data.isLoading}
      loadingRows={3}
      error={
        data.isError
          ? { message: "Could not load revenue.", onRetry: () => void
data.refetch() }
          : null
      }
      empty={
        <ListEmpty
          icon={BarChart3}
          title="No completed revenue for this period."
          description="Revenue appears once appointments in this period are
completed."
        />
      }
      columns={[
        {
          key: "period",
          header: revenuePeriod === "daily" ? "Date" : "Month",
```

```
            card: "title",
            cell: (row) => (
              <span className="font-medium">
                {formatRevenuePeriod(row.period, revenuePeriod)}
              </span>
            ),
          },
          {
            key: "transactions",
            header: "Transactions",
            className: "w-32 text-right",
            cell: (row) => <span
className="text-sm">{row.completed_transactions}</span>,
          },
          {
            key: "service",
            header: "Service revenue",
            className: "w-36 text-right",
            cell: (row) => <span
className="text-sm">{formatPHP(row.service_revenue)}</span>,
          },
          {
            key: "product",
            header: "Product revenue",
            className: "w-36 text-right",
            cell: (row) => <span
className="text-sm">{formatPHP(row.product_revenue)}</span>,
          },
          {
            key: "total",
            header: "Total revenue",
            className: "w-36 text-right",
            cell: (row) => (
              <span className="text-sm font-medium text-primary">
                {formatPHP(row.total_revenue)}
              </span>
            ),
          },
        ]}
      />
    </ListCard>

    <ListCard
      header={<ListCardTitle title="Completed Transactions"
description={reportScope} />}
```

```
        footer={
          <PaginationControls
            page={page}
            pageSize={pageSize}
            total={data.data?.total ?? 0}
            onPageChange={setPage}
          />
        }
      >
        <DataList
          label="Completed transactions"
          rows={transactionRows}
          rowKey={(row) => row.key}
          loading={data.isLoading}
          error={
            data.isError
              ? { message: "Could not load this report.", onRetry: () => void
data.refetch() }
              : null
          }
          empty={
            <ListEmpty
              icon={ListFilter}
              title={`No ${rowLabel} match this report selection.`}
              description="Try a different period, category, or service."
            />
          }
          columns={transactionColumns(reportData.headers, (services,
referenceCode) =>
            setServiceListDialog({ referenceCode, services }),
          )}
        />
      </ListCard>

      <Dialog
        open={serviceListDialog !== null}
        onOpenChange={(open) => !open && setServiceListDialog(null)}
      >
        <DialogContent className="max-w-lg">
          <DialogHeader>
            <DialogTitle className="font-display uppercase">Services</DialogTitle>
            <DialogDescription>
              Complete service list for booking {serviceListDialog?.referenceCode}.
            </DialogDescription>
          </DialogHeader>
```

```
        <ul className="max-h-[50vh] space-y-2 overflow-y-auto pr-1">
          {serviceListDialog?.services.map((service, index) => (
            <li
              key={`${service}-${index}`}
              className="rounded-md border border-border/70 bg-muted/30 px-3
py-2 text-sm"
            >
              {service}
            </li>
          ))}
        </ul>
        <DialogFooter>
          <Button type="button" variant="outline" onClick={() =>
setServiceListDialog(null)}>
            Close
          </Button>
        </DialogFooter>
      </DialogContent>
    </Dialog>

    <PrintReport payload={printPayload} />
  </AdminPage>
 );
}

function ReportServicesCell({
 value,
 referenceCode,
 onSeeMore,
}: {
 value: string;
 referenceCode: string;
 onSeeMore: (services: string[]) => void;
}) {
 const services = splitServiceNames(value);
 const needsSeeMore = services.length > 1 || value.length > 42;

 if (!needsSeeMore) return <span className="break-words">{value}</span>;

 const summary =
   services.length > 1 ? `${services[0]} +${services.length - 1} more` : (services[0]
?? value);

 return (
   <div className="grid w-full min-w-0 grid-cols-[minmax(0,1fr)_auto]
```

```
items-center gap-2 text-left">
      <span className="min-w-0 truncate">{summary}</span>
      <Button
        type="button"
        variant="link"
        size="sm"
        className="h-auto shrink-0 px-0 py-0 text-xs leading-5"
        aria-label={`See all services for booking ${referenceCode}`}
        onClick={() => onSeeMore(services)}
      >
        See More
      </Button>
    </div>
  );
}

function FinancialTableCard({
  title,
  description,
  headers,
  rows,
  loading,
  emptyMessage,
  totalLabel,
  totalValue,
}: {
  title: string;
  description: string;
  headers: string[];
  rows: string[][];
  loading: boolean;
  emptyMessage: string;
  totalLabel: string;
  totalValue: string;
}) {
  const items = rows.map((cells, index) => ({ key: `${cells[0]}-${index}`, cells }));
  return (
    <ListCard
      header={<ListCardTitle title={title} description={description} />}
      footer={
        <div className="flex items-center justify-between gap-3 border-t
border-border/70 bg-muted/40 px-4 py-3 text-sm font-medium sm:px-5">
          <span>{totalLabel}</span>
          <span className="text-primary">{totalValue}</span>
        </div>
```

```
    }
   >
    <DataList
     label={title}
     rows={items}
     rowKey={(item) => item.key}
     loading={loading}
     loadingRows={3}
     empty={<ListEmpty icon={CircleDollarSign} title={emptyMessage} />}
     columns={headers.map((header, index): DataListColumn<(typeof
items)[number]> =>
       index === 0
        ? {
           key: header,
           header,
           card: "title",
           cell: (item) => <span
className="font-medium">{item.cells[0]}</span>,
          }
        : {
           key: header,
           header,
           className: "w-28 text-right",
           cell: (item) => <span className="text-sm">{item.cells[index]}</span>,
          },
     )}
    />
   </ListCard>
  );
}

type TransactionRow = { key: string; cells: string[] };

/**
 * Columns for the paginated report rows. Cards show every value; the table
 * keeps the reference, date, services, and total, and adds the service and
 * product amounts from `xl` up.
 */
function transactionColumns(
  headers: string[],
  onSeeMore: (services: string[], referenceCode: string) => void,
): DataListColumn<TransactionRow>[] {
  const cellAt = (row: TransactionRow, header: string) =>
row.cells[headers.indexOf(header)] ?? "";
  const referenceOf = (row: TransactionRow) => row.cells[0] || "this booking";
```

```
  const servicesCell = (row: TransactionRow) => (
    <ReportServicesCell
      value={cellAt(row, "Services")}
      referenceCode={referenceOf(row)}
      onSeeMore={(services) => onSeeMore(services, referenceOf(row))}
    />
  );

  return headers.flatMap((header, index): DataListColumn<TransactionRow>[] =>
{
    const value = (row: TransactionRow) => row.cells[index] ?? "";
    switch (header) {
      case "Reference":
        return [
          {
            key: header,
            header,
            card: "title",
            className: "w-32",
            cell: (row) => <span className="font-mono text-sm
font-medium">{value(row)}</span>,
          },
        ];
      case "Date":
        return [
          {
            key: header,
            header,
            className: "w-36",
            cell: (row) => <span className="text-sm">{value(row)}</span>,
          },
        ];
      case "Services":
        return [
          {
            key: header,
            header,
            card: "wide",
            table: false,
            cell: (row) => <span className="text-sm">{servicesCell(row)}</span>,
          },
          {
            key: "services-and-products",
            header: headers.includes("Products") ? "Services & products" :
"Services",
```

```
          card: false,
          cell: (row) => {
            const products = cellAt(row, "Products");
            return (
              <span className="text-sm">
                {servicesCell(row)}
                {products && products !== "-" && (
                  <span className="mt-0.5 block text-xs
text-muted-foreground">{products}</span>
                )}
              </span>
            );
          },
        },
      ];
    case "Products":
      return [
        {
          key: header,
          header,
          card: "wide",
          table: false,
          cell: (row) => <span className="text-sm">{value(row)}</span>,
        },
      ];
    default: {
      const isTotal = /^total/i.test(header);
      return [
        {
          key: header,
          header,
          xlOnly: !isTotal && /amount/i.test(header),
          className: "w-32 text-right",
          cell: (row) => (
            <span className={cn("text-sm", isTotal && "font-medium
text-primary")}>
              {value(row)}
            </span>
          ),
        },
      ];
    }
  }
 });
}
```

```
function splitServiceNames(value: string) {
  return value
    .split(",")
    .map((service) => service.trim())
    .filter(Boolean);
}

function PeriodPicker({
  value,
  from,
  to,
  error,
  onValueChange,
  onRangeChange,
}: {
  value: PeriodPreset;
  from: string;
  to: string;
  error?: string | undefined;
  onValueChange: (value: PeriodPreset) => void;
  onRangeChange: (range: { from: string; to: string }) => void;
}) {
  const [customDialogOpen, setCustomDialogOpen] = useState(false);
  const [isSelectingCustomRange, setIsSelectingCustomRange] =
useState(false);
  const selectedRange = from
    ? { from: isoDateToCalendarDate(from), to: to ? isoDateToCalendarDate(to) :
undefined }
    : undefined;
  const [draftRange, setDraftRange] = useState<DateRange |
undefined>(selectedRange);
  const [calendarMonth, setCalendarMonth] = useState<Date>(
    () => selectedRange?.from ?? isoDateToCalendarDate(manilaNow().date),
  );
  const draftFrom = draftRange?.from ? calendarDateToIso(draftRange.from) : "";
  const draftTo = draftRange?.to ? calendarDateToIso(draftRange.to) : "";
  const draftRangeError =
    !draftFrom || !draftTo
      ? "Select both a start and end date."
      : draftFrom > draftTo
        ? "The end date must be on or after the start date."
        : undefined;

  useEffect(() => {
```

```
    if (!isSelectingCustomRange || customDialogOpen) return;
    const timeout = window.setTimeout(() => setCustomDialogOpen(true), 0);
    return () => window.clearTimeout(timeout);
  }, [customDialogOpen, isSelectingCustomRange]);

  function startCustomRangeSelection() {
    setIsSelectingCustomRange(true);
    setDraftRange(selectedRange);
    setCalendarMonth(selectedRange?.from ??
isoDateToCalendarDate(manilaNow().date));
  }

  function handlePeriodValueChange(nextValue: string) {
    const nextPeriod = nextValue as PeriodPreset;
    if (nextPeriod !== "custom") {
      setIsSelectingCustomRange(false);
      onValueChange(nextPeriod);
      setCustomDialogOpen(false);
      return;
    }

    startCustomRangeSelection();
  }

  function cancelCustomRange() {
    setDraftRange(selectedRange);
    setIsSelectingCustomRange(false);
    setCustomDialogOpen(false);
  }

  function updateDraftDate(boundary: "from" | "to", value: string) {
    const date = value ? isoDateToCalendarDate(value) : undefined;
    setDraftRange((current) => {
      const next =
        boundary === "from" ? { from: date, to: current?.to } : { from: current?.from,
to: date };
      return next.from || next.to ? next : undefined;
    });
    if (date) setCalendarMonth(date);
  }

  return (
    <div className="space-y-1.5">
      <Select
        value={isSelectingCustomRange ? "custom" : value}
```

```
      onValueChange={handlePeriodValueChange}
    >
      <SelectTrigger aria-label="Select report period"
aria-invalid={Boolean(error)}>
        <CalendarRange className="mr-2 size-4 shrink-0 text-muted-foreground"
/>
        <SelectValue />
      </SelectTrigger>
      <SelectContent>
        {periodOptions.map((option) => (
          <SelectItem key={option.value} value={option.value}>
            {option.label}
          </SelectItem>
        ))}
      </SelectContent>
    </Select>
    <Dialog
      open={customDialogOpen}
      onOpenChange={(nextOpen) => {
        if (nextOpen) {
          setCustomDialogOpen(true);
          return;
        }
        cancelCustomRange();
      }}
    >
      <DialogContent className="w-auto max-w-[calc(100vw-2rem)] p-4
sm:max-w-md">
        <DialogHeader>
          <DialogTitle className="font-display uppercase">Custom date
range</DialogTitle>
          <DialogDescription>Select a start date and an end
date.</DialogDescription>
        </DialogHeader>
        <Calendar
          mode="range"
          selected={draftRange}
          month={calendarMonth}
          onMonthChange={setCalendarMonth}
          onSelect={setDraftRange}
          classNames={{
            nav: "inset-x-auto left-1/2 w-48 -translate-x-1/2 justify-between",
          }}
        />
        <div className="grid gap-3 sm:grid-cols-2">
```

```
      <div className="space-y-1.5">
        <Label htmlFor="report-custom-start-date">Start date</Label>
        <Input
          id="report-custom-start-date"
          type="date"
          value={draftFrom}
          max={draftTo || undefined}
          aria-invalid={Boolean(draftRangeError)}
          onChange={(event) => updateDraftDate("from", event.target.value)}
        />
      </div>
      <div className="space-y-1.5">
        <Label htmlFor="report-custom-end-date">End date</Label>
        <Input
          id="report-custom-end-date"
          type="date"
          value={draftTo}
          min={draftFrom || undefined}
          aria-invalid={Boolean(draftRangeError)}
          onChange={(event) => updateDraftDate("to", event.target.value)}
        />
      </div>
    </div>
    <FieldError message={draftRangeError} />
    <DialogFooter className="mt-1 border-t border-border pt-3">
      <Button type="button" variant="outline" onClick={cancelCustomRange}>
        Cancel
      </Button>
      <Button
        type="button"
        disabled={Boolean(draftRangeError)}
        onClick={() => {
          if (draftRangeError) return;
          const range = calendarRangeToIso(draftRange);
          if (!range) return;
          setIsSelectingCustomRange(false);
          onRangeChange(range);
          setCustomDialogOpen(false);
        }}
      >
        Apply
      </Button>
    </DialogFooter>
  </DialogContent>
</Dialog>
```

```
      <FieldError message={error} />
    </div>
  );
}

async function getReportPage(input: {
  from: string;
  to: string;
  category: string;
  serviceNames: string[];
  limit: number;
  offset: number;
}) {
  const { data, error } = await
supabase.rpc("get_completed_financial_report_page", {
    p_from: input.from,
    p_to: input.to,
    p_category: input.category === "all" ? null : input.category,
    p_service_names: input.serviceNames.length ? input.serviceNames : null,
    p_limit: input.limit,
    p_offset: input.offset,
  });
  if (error) throw error;
  if (!data || typeof data !== "object" || !Array.isArray((data as
FinancialReportPage).rows)) {
    throw new Error("The financial report response was incomplete.");
  }
  return data as unknown as FinancialReportPage;
}

/** Every row of the report, for printing, with the breakdowns from the first page. */
async function getAllReportRows(
  input: Omit<Parameters<typeof getReportPage>[0], "limit" | "offset">,
) {
  const limit = 250;
  let offset = 0;
  let first: FinancialReportPage | null = null;
  let rows: FinancialTransactionRow[] = [];
  do {
    const current = await getReportPage({ ...input, limit, offset });
    first ??= current;
    if (current.rows.length === 0 && offset < current.total) {
      throw new Error("The report printout could not retrieve all matching rows.");
    }
    rows = rows.concat(current.rows);
```

```
    offset += current.rows.length;
  } while (first && offset < first.total);
  return {
    rows,
    total: first?.total ?? 0,
    metrics: first?.metrics,
    servicePerformance: first?.service_performance ?? [],
    productSales: first?.product_sales ?? [],
    dailyRevenue: first?.daily_revenue ?? [],
  };
}

function makeReportData(
  rows: FinancialTransactionRow[],
  metrics?: FinancialMetrics,
  options: { hasCompleteRows?: boolean } = {},
): ReportData {
  const serviceAmount = sumMoney(rows.map((row) => row.service_amount));
  const productAmount = sumMoney(rows.map((row) => row.product_amount));
  const totalAmount = sumMoney(rows.map((row) => row.total_amount));
  return {
    headers: [
      "Reference",
      "Date",
      "Services",
      "Products",
      "Service Amount",
      "Product Amount",
      "Total Amount",
    ],
    rows: rows.map((row) => [
      row.reference_code,
      formatDateLong(row.transaction_date),
      row.services || "-",
      row.products || "-",
      formatPHP(row.service_amount),
      formatPHP(row.product_amount),
      formatPHP(row.total_amount),
    ]),
    cards: [
      {
        label: "Total revenue",
        value: formatPHP(
          options.hasCompleteRows ? totalAmount : (metrics?.total_revenue ??
totalAmount),
```

```
        ),
        detail: "Completed transactions only",
        tone: "success",
      },
      {
        label: "Service revenue",
        value: formatPHP(
          options.hasCompleteRows ? serviceAmount : (metrics?.service_revenue
?? serviceAmount),
        ),
        detail: "Completed service charges",
        tone: "primary",
      },
      {
        label: "Product revenue",
        value: formatPHP(
          options.hasCompleteRows ? productAmount : (metrics?.product_revenue
?? productAmount),
        ),
        detail: "Products sold during service",
        tone: "accent",
      },
      {
        label: "Completed transactions",
        value: String(
          options.hasCompleteRows ? rows.length :
(metrics?.completed_transactions ?? rows.length),
        ),
        detail: "Revenue-recognized appointments",
        tone: "chart",
      },
    ],
    totalAmount,
  };
}

function moneyValue(value: number | string | null | undefined) {
  const parsed = Number(value);
  return Number.isFinite(parsed) ? parsed : 0;
}

function sumMoney(values: Array<number | string | null | undefined>) {
  return values.reduce<number>((total, value) => total + moneyValue(value), 0);
}
```

```
function formatRevenuePeriod(period: string, kind: "daily" | "monthly") {
  const date = isoDateToCalendarDate(period);
  return format(date, kind === "daily" ? "MMM d, yyyy" : "MMMM yyyy");
}

function getInitialFilters(today: string): ReportFilters {
  const range = periodRange("this_month", today);
  return {
    periodPreset: "this_month",
    ...range,
    category: "all",
    serviceNames: [],
  };
}

function formatDateRange(from: string, to: string) {
  if (!from) return "Select custom dates";

  const fromDate = isoDateToCalendarDate(from);
  const fromLabel = format(fromDate, "MMM d, yyyy");
  if (!to) return `${fromLabel} to Select end date`;

  const toDate = isoDateToCalendarDate(to);
  return fromDate.getFullYear() === toDate.getFullYear()
    ? `${format(fromDate, "MMM d")} to ${format(toDate, "MMM d, yyyy")}`
    : `${fromLabel} to ${format(toDate, "MMM d, yyyy")}`;
}

function periodRange(preset: Exclude<PeriodPreset, "custom">, today: string) {
  if (preset === "today") return { from: today, to: today };
  if (preset === "this_week") {
    return manilaWeekDateRange(today);
  }
  if (preset === "this_year") return manilaYearDateRange(today);
  return manilaMonthDateRange(today);
}

function PrintReport({ payload }: { payload: PrintPayload | null }) {
  if (!payload) return null;

  const { financial } = payload;
  const serviceCompleted = financial.servicePerformance.reduce(
    (total, row) => total + row.completed_count,
    0,
  );
```

```
  const serviceRevenue = sumMoney(financial.servicePerformance.map((row) =>
row.revenue));
  const productsSold = financial.productSales.reduce((total, row) => total +
row.quantity_sold, 0);
  const productRevenue = sumMoney(financial.productSales.map((row) =>
row.total_sales));
  const dailyRevenue = financial.dailyRevenue;
  const dailyRevenueTotals = revenueTotals(dailyRevenue);

  return (
    <PrintDocument
      info={{
        title: payload.title,
        generatedAt: payload.generatedAt,
        coverage: payload.period,
        scope: payload.scope,
      }}
    >
      <section className="report-print-section report-print-summary">
        <h2>A. Financial summary</h2>
        <table className="report-print-summary-table">
          <thead>
            <tr>
              <th>Description</th>
              <th>Amount</th>
            </tr>
          </thead>
          <tbody>
            {payload.data.cards.map((card) => (
              <tr key={card.label}>
                <td>{card.label}</td>
                <td>{card.value}</td>
              </tr>
            ))}
          </tbody>
        </table>
      </section>
      <PrintTable
        title="B. Service revenue breakdown"
        headers={["Service", "Completed", "Revenue"]}
        columnWidths={["58%", "18%", "24%"]}
        rows={financial.servicePerformance.map((row) => [
          row.service_name,
          String(row.completed_count),
          formatPHP(row.revenue),
```

```
        ])}
        footerRow={["Total service revenue", String(serviceCompleted),
formatPHP(serviceRevenue)]}
        emptyMessage="No completed services match this report selection."
      />
      <PrintTable
        title="C. Product sales breakdown"
        headers={["Product", "Qty Sold", "Unit Price", "Total Sales"]}
        columnWidths={["42%", "18%", "20%", "20%"]}
        rows={financial.productSales.map((row) => [
          row.product_name,
          String(row.quantity_sold),
          formatPHP(row.unit_price),
          formatPHP(row.total_sales),
        ])}
        footerRow={["Total product revenue", String(productsSold), "",
formatPHP(productRevenue)]}
        emptyMessage="No products were sold in completed transactions for this
period."
      />
      <PrintTable
        title="D. Daily revenue"
        headers={["Date", "Transactions", "Service Revenue", "Product Revenue",
"Total Revenue"]}
        columnWidths={["26%", "14%", "20%", "20%", "20%"]}
        rows={dailyRevenue.map((row) => [
          formatRevenuePeriod(row.period, "daily"),
          String(row.completed_transactions),
          formatPHP(row.service_revenue),
          formatPHP(row.product_revenue),
          formatPHP(row.total_revenue),
        ])}
        footerRow={[
          "Total daily revenue",
          String(dailyRevenueTotals.transactions),
          formatPHP(dailyRevenueTotals.serviceRevenue),
          formatPHP(dailyRevenueTotals.productRevenue),
          formatPHP(dailyRevenueTotals.totalRevenue),
        ]}
        emptyMessage="No completed revenue is available for this period."
      />
      <section className="report-print-section">
        <h2>E. Completed transactions</h2>
        <table className="report-print-table">
          <colgroup>
```

```
          {["10%", "13%", "23%", "16%", "12%", "13%", "13%"].map((width, index)
=> (
            <col key={`${width}-${index}`} style={{ width }} />
          ))}
        </colgroup>
        <thead>
          <tr>
            {payload.data.headers.map((header) => (
              <th key={header}>{header}</th>
            ))}
          </tr>
        </thead>
        <tbody>
          {payload.data.rows.map((row, index) => (
            <tr key={`${row.join("-")}-${index}`}>
              {row.map((cell, cellIndex) => (
                <td key={`${cell}-${cellIndex}`}>{cell}</td>
              ))}
            </tr>
          ))}
        </tbody>
        <tfoot>
          <tr>
            <td colSpan={payload.data.headers.length - 1}>Total amount</td>
            <td>{formatPHP(payload.data.totalAmount)}</td>
          </tr>
        </tfoot>
      </table>
    </section>
  </PrintDocument>
 );
}

function revenueTotals(rows: RevenuePeriodRow[]) {
 return {
  transactions: rows.reduce((total, row) => total + row.completed_transactions,
0),
  serviceRevenue: sumMoney(rows.map((row) => row.service_revenue)),
  productRevenue: sumMoney(rows.map((row) => row.product_revenue)),
  totalRevenue: sumMoney(rows.map((row) => row.total_revenue)),
 };
}

function Stat({
 label,
```

```
  value,
  detail,
  icon: Icon,
  iconClassName,
}: {
  label: string;
  value: string;
  detail: string;
  icon: LucideIcon;
  iconClassName?: string;
}) {
  return (
    <Card className="border-border/70 bg-card/70 shadow-sm">
      <CardContent className="p-3.5 sm:p-4">
        <div className="flex items-center gap-2.5">
          <span
            className={cn(
              "grid size-8 shrink-0 place-items-center rounded-full bg-primary/10
text-primary",
              iconClassName,
            )}
          >
            <Icon className="size-3.5" aria-hidden="true" />
          </span>
          <div className="min-w-0">
            <p className="truncate text-[10px] font-semibold tracking-wider
text-muted-foreground uppercase">
              {label}
            </p>
            <p className="mt-0.5 font-display text-2xl leading-none
text-primary">{value}</p>
          </div>
        </div>
        <p className="mt-2 truncate text-[11px]
text-muted-foreground">{detail}</p>
      </CardContent>
    </Card>
  );
}

/** Pick any number of services; none picked means all services. */
function ServiceMultiSelect({
  options,
  value,
  onChange,
```

```
}: {
  options: CatalogService[];
  value: string[];
  onChange: (serviceNames: string[]) => void;
}) {
  const label =
    value.length === 0
      ? "All services"
      : value.length === 1
        ? value[0]
        : `${value.length} services`;
  const toggle = (name: string, checked: boolean) =>
    onChange(checked ? [...value, name] : value.filter((item) => item !== name));

  return (
    <Popover>
      <PopoverTrigger asChild>
        <Button
          type="button"
          variant="outline"
          aria-label={`Filter by service: ${value.length ? value.join(", ") : "all
services"}`}
          className="w-full justify-between font-normal sm:w-52"
        >
          <span className="truncate">{label}</span>
          <ChevronDown className="size-4 shrink-0 opacity-50" aria-hidden="true"
/>
        </Button>
      </PopoverTrigger>
      <PopoverContent align="start" className="w-72 p-0">
        <div className="flex items-center justify-between gap-2 border-b
border-border px-3 py-2">
          <p className="text-sm font-medium">
            {value.length ? `${value.length} selected` : "All services"}
          </p>
          <Button
            type="button"
            variant="ghost"
            size="sm"
            className="h-7 px-2 text-xs"
            disabled={value.length === 0}
            onClick={() => onChange([])}
          >
            Clear
          </Button>
```

```
        </div>
        <ul className="max-h-72 overflow-y-auto p-1">
          {options.map((item) => {
            const id = `report-service-${item.id}`;
            const checked = value.includes(item.name);
            return (
              <li key={item.id}>
                <label
                  htmlFor={id}
                  className="flex cursor-pointer items-center gap-2.5 rounded-md
px-2 py-2 text-sm hover:bg-muted"
                >
                  <Checkbox
                    id={id}
                    checked={checked}
                    onCheckedChange={(next) => toggle(item.name, next === true)}
                  />
                  <span className="min-w-0 flex-1 truncate" title={item.name}>
                    {item.name}
                  </span>
                </label>
              </li>
            );
          })}
          {options.length === 0 && (
            <li className="px-2 py-3 text-sm text-muted-foreground">
              No services in this category.
            </li>
          )}
        </ul>
      </PopoverContent>
    </Popover>
  );
}
```

src/routes/_authenticated/admin/schedule-blocks.tsx

```
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import { createFileRoute } from "@tanstack/react-router";
import { CalendarX, Pencil, Plus } from "lucide-react";
import { useState } from "react";

import { ArchiveConfirmationDialog } from
"@/components/admin/archive-confirmation-dialog";
import {
  AdminPage,
  ArchiveMenuItem,
  DataList,
  InlineAlert,
  ListCard,
  ListEmpty,
  ListToolbar,
  MoreActionsMenu,
  ResetFiltersButton,
  ToolbarSearch,
} from "@/components/admin/data-list";
import { PageHeader } from "@/components/admin/page-header";
import {
  AlertDialog,
  AlertDialogAction,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
} from "@/components/ui/alert-dialog";
import { Button } from "@/components/ui/button";
import { Card, CardContent } from "@/components/ui/card";
import {
  Dialog,
  DialogContent,
  DialogFooter,
  DialogHeader,
  DialogTitle,
} from "@/components/ui/dialog";
import { FieldError } from "@/components/ui/field-error";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import {
```

```
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@/components/ui/select";
import { supabase } from "@/integrations/supabase/client";
import {
  decodeBlockReason,
  encodeBlockReason,
  formatDateLong,
  formatTime,
  shopTimeOptions,
} from "@/lib/shop";

export const Route = createFileRoute("/_authenticated/admin/schedule-blocks")({
  component: ScheduleBlocks,
});

type ScheduleBlock = {
  id: string;
  block_date: string;
  start_time: string | null;
  reason: string | null;
};

function ScheduleBlocks() {
  const qc = useQueryClient();
  const [editingId, setEditingId] = useState<string | null>(null);
  const [date, setDate] = useState("");
  const [slot, setSlot] = useState("all");
  const [customStart, setCustomStart] = useState("");
  const [customEnd, setCustomEnd] = useState("");
  const [reason, setReason] = useState("");
  const [createConfirmationOpen, setCreateConfirmationOpen] = useState(false);
  const [archiveTarget, setArchiveTarget] = useState<string | null>(null);
  const [formErrors, setFormErrors] = useState<
    Partial<Record<"date" | "start" | "end", string | undefined>>
  >({});
  const [archiveError, setArchiveError] = useState<string | null>(null);
  const [search, setSearch] = useState("");

  const blocks = useQuery({
    queryKey: ["schedule-blocks"],
    queryFn: async () => {
```

```
    const { data, error } = await supabase
      .from("schedule_blocks")
      .select("*")
      .eq("is_active", true)
      .order("block_date");
    if (error) throw error;
    return data;
  },
});

function resetForm() {
  setEditingId(null);
  setDate("");
  setReason("");
  setSlot("all");
  setCustomStart("");
  setCustomEnd("");
  setFormErrors({});
}

const save = useMutation({
  mutationFn: async () => {
    const payload =
      slot === "all"
        ? {
            block_date: date,
            start_time: null,
            reason: encodeBlockReason(null, reason.trim()),
          }
        : {
            block_date: date,
            start_time: `${customStart}:00`,
            reason: encodeBlockReason(`${customEnd}:00`, reason.trim()),
          };

    if (editingId) {
      const { error } = await supabase
        .from("schedule_blocks")
        .update(payload)
        .eq("id", editingId);
      if (error) throw error;
      return;
    }

    const { error } = await supabase.from("schedule_blocks").insert({ ...payload
```

```
});
      if (error) throw error;
    },
    onSuccess: () => {
      resetForm();
      qc.invalidateQueries({ queryKey: ["schedule-blocks"] });
    },
    onError: (err: Error) => {
      const msg = err.message.toLowerCase();
      if (msg.includes("unique") || msg.includes("duplicate")) {
        setFormErrors({
          date: "A block for this date already exists. Remove the existing block first.",
        });
      } else if (msg.includes("foreign")) {
        setFormErrors({
          date: "Referenced data no longer exists. Refresh the page and try again.",
        });
      } else {
        setFormErrors({
          date: "Could not block that schedule. Please check your inputs and try
again.",
        });
      }
    },
  });

  const archive = useMutation({
    mutationFn: async (id: string) => {
      const { error } = await supabase
        .from("schedule_blocks")
        .update({ is_active: false })
        .eq("id", id);
      if (error) throw error;
    },
    onSuccess: (_, id) => {
      qc.setQueryData<Array<{ id: string }>>(["schedule-blocks"], (blocks) =>
        blocks?.filter((block) => block.id !== id),
      );
      setArchiveError(null);
      qc.invalidateQueries({ queryKey: ["schedule-blocks"], exact: false });
      qc.invalidateQueries({ queryKey: ["archived-blocks"], exact: false });
    },
    onError: (err: Error) => {
      const msg = err.message.toLowerCase();
      if (msg.includes("not found") || msg.includes("no rows")) {
```

```
        setArchiveError("This schedule block no longer exists. Refresh the list and
try again.");
      } else if (msg.includes("permission") || msg.includes("forbidden")) {
        setArchiveError("You do not have permission to archive schedule blocks.");
      } else {
        console.error("Could not archive schedule block:", err);
        setArchiveError("Could not archive this schedule block. Please try again.");
      }
    },
  });

  function validateBlockForm() {
    const nextErrors: Partial<Record<"date" | "start" | "end", string>> = {};
    if (!date) nextErrors.date = "Choose the date to block.";
    if (slot === "custom") {
      if (!customStart) nextErrors.start = "Please set a start time.";
      if (!customEnd) nextErrors.end = "Please set an end time.";
      if (customStart && customStart < "08:00") {
        nextErrors.start = "Start time cannot be earlier than 8:00 AM (shop
opening).";
      }
      if (customEnd && customEnd > "17:00") {
        nextErrors.end = "End time cannot be later than 5:00 PM (shop closing).";
      }
      if (customStart && customEnd && customStart >= customEnd) {
        nextErrors.end = "End time must be after start time.";
      }
    }
    setFormErrors(nextErrors);
    return Object.keys(nextErrors).length === 0;
  }

  function handleSave() {
    if (validateBlockForm()) save.mutate();
  }

  function requestScheduleBlock() {
    if (validateBlockForm()) setCreateConfirmationOpen(true);
  }

  function startEditing(block: ScheduleBlock) {
    const { endTime, userReason } = decodeBlockReason(block.reason);
    setEditingId(block.id);
    setDate(block.block_date);
    setSlot(block.start_time ? "custom" : "all");
```

```
    setCustomStart(block.start_time ? String(block.start_time).slice(0, 5) : "");
    setCustomEnd(endTime?.slice(0, 5) ?? "");
    setReason(userReason);
    setFormErrors({});
  }

  const searchTerm = search.trim().toLocaleLowerCase();
  const filteredBlocks = (blocks.data ?? []).filter(
    (b) =>
      !searchTerm ||
      `${formatDateLong(b.block_date)} ${b.block_date}
${decodeBlockReason(b.reason).userReason}`
        .toLocaleLowerCase()
        .includes(searchTerm),
  );

  return (
    <AdminPage>
      <PageHeader
        title="Schedule Blocks"
        description="Close whole days or custom time ranges (holidays,
out-of-town, maintenance)."
      />
      {archiveError && <InlineAlert>{archiveError}</InlineAlert>}

      <Card className="mb-6 border-border/70 bg-card/60">
        <CardContent className="grid gap-4 p-5 sm:grid-cols-4 sm:items-end">
          <div className="space-y-1.5">
            <Label>Date</Label>
            <Input
              type="date"
              value={date}
              onChange={(e) => {
                setDate(e.target.value);
                setFormErrors((current) => ({ ...current, date: undefined }));
              }}
              aria-invalid={!!formErrors.date}
            />
            <ReservedFieldError message={formErrors.date} />
          </div>
          <div className="space-y-1.5">
            <Label>Slot</Label>
            <Select
              value={slot}
              onValueChange={(v) => {
```

```
      setSlot(v);
      setCustomStart("");
      setCustomEnd("");
    }}
  >
    <SelectTrigger>
      <SelectValue />
    </SelectTrigger>
    <SelectContent>
      <SelectItem value="all">Whole day</SelectItem>
      <SelectItem value="custom">Custom Time Range</SelectItem>
    </SelectContent>
  </Select>
  <ValidationSpace className="hidden sm:block" />
</div>
{slot === "custom" && (
  <>
    <div className="grid grid-cols-2 gap-2 space-y-0 space-x-2">
      <div className="space-y-1.5">
        <Label>Start</Label>
        <Select
          value={customStart}
          onValueChange={(value) => {
            setCustomStart(value);
            setFormErrors((current) => ({ ...current, start: undefined }));
          }}
        >
          <SelectTrigger className="w-full" aria-invalid={!!formErrors.start}>
            <SelectValue placeholder="Start time" />
          </SelectTrigger>
          <SelectContent>
            {shopTimeOptions().map((t) => (
              <SelectItem key={t.value} value={t.value}>
                {t.label}
              </SelectItem>
            ))}
          </SelectContent>
        </Select>
        <ReservedFieldError message={formErrors.start} />
      </div>
      <div className="space-y-1.5">
        <Label>End</Label>
        <Select
          value={customEnd}
          onValueChange={(value) => {
```

```
                setCustomEnd(value);
                setFormErrors((current) => ({ ...current, end: undefined }));
              }}
            >
              <SelectTrigger className="w-full" aria-invalid={!!formErrors.end}>
                <SelectValue placeholder="End time" />
              </SelectTrigger>
              <SelectContent>
                {shopTimeOptions().map((t) => (
                  <SelectItem key={t.value} value={t.value}>
                    {t.label}
                  </SelectItem>
                ))}
              </SelectContent>
            </Select>
            <ReservedFieldError message={formErrors.end} />
          </div>
        </div>
        <p className="text-xs text-muted-foreground">
          Note: Time slots are available in 30-minute intervals only, from 8:00
AM to 5:00 PM.
          The End Time must be later than the Start Time.
        </p>
      </>
    )}
    <div className="space-y-1.5">
      <Label>Reason</Label>
      <Input
        value={reason}
        onChange={(e) => setReason(e.target.value)}
        placeholder="Holiday, team event..."
      />
      <ValidationSpace className="hidden sm:block" />
    </div>
    <div className="flex flex-col gap-1.5">
      <div className="flex flex-wrap gap-2">
        <Button
          onClick={requestScheduleBlock}
          disabled={save.isPending || !!editingId}
          className="font-display uppercase"
        >
          <Plus /> Block
        </Button>
      </div>
      <ValidationSpace className="hidden sm:block" />
```

```
        </div>
      </CardContent>
    </Card>

    <AlertDialog open={createConfirmationOpen}
onOpenChange={setCreateConfirmationOpen}>
      <AlertDialogContent>
        <AlertDialogHeader>
          <AlertDialogTitle>Block this schedule?</AlertDialogTitle>
          <AlertDialogDescription>
            This will make the selected date or time range unavailable for new
appointments.
            Review the details before confirming.
          </AlertDialogDescription>
        </AlertDialogHeader>
        <AlertDialogFooter>
          <AlertDialogCancel>Go back</AlertDialogCancel>
          <AlertDialogAction disabled={save.isPending} onClick={handleSave}>
            Confirm block
          </AlertDialogAction>
        </AlertDialogFooter>
      </AlertDialogContent>
    </AlertDialog>

    <Dialog
      open={Boolean(editingId)}
      onOpenChange={(open) => {
        if (!open) resetForm();
      }}
    >
      <DialogContent className="sm:max-w-lg">
        <DialogHeader>
          <DialogTitle className="font-display uppercase">Edit schedule
block</DialogTitle>
        </DialogHeader>

        <div className="space-y-4">
          <div className="space-y-1.5">
            <Label>Date</Label>
            <Input
              type="date"
              value={date}
              onChange={(e) => {
                setDate(e.target.value);
                setFormErrors((current) => ({ ...current, date: undefined }));
```

```
    }}
    aria-invalid={!!formErrors.date}
   />
   <FieldError message={formErrors.date} />
  </div>

  <div className="space-y-1.5">
   <Label>Slot</Label>
   <Select
    value={slot}
    onValueChange={(value) => {
     setSlot(value);
     setCustomStart("");
     setCustomEnd("");
    }}
   >
    <SelectTrigger>
     <SelectValue />
    </SelectTrigger>
    <SelectContent>
     <SelectItem value="all">Whole day</SelectItem>
     <SelectItem value="custom">Custom Time Range</SelectItem>
    </SelectContent>
   </Select>
  </div>

  {slot === "custom" && (
   <div className="grid gap-4 sm:grid-cols-2">
    <div className="space-y-1.5">
     <Label>Start</Label>
     <Select
      value={customStart}
      onValueChange={(value) => {
       setCustomStart(value);
       setFormErrors((current) => ({ ...current, start: undefined }));
      }}
     >
      <SelectTrigger aria-invalid={!!formErrors.start}>
       <SelectValue placeholder="Start time" />
      </SelectTrigger>
      <SelectContent>
       {shopTimeOptions().map((time) => (
        <SelectItem key={time.value} value={time.value}>
         {time.label}
        </SelectItem>
```

```
                ))}
              </SelectContent>
            </Select>
            <FieldError message={formErrors.start} />
          </div>

          <div className="space-y-1.5">
            <Label>End</Label>
            <Select
              value={customEnd}
              onValueChange={(value) => {
                setCustomEnd(value);
                setFormErrors((current) => ({ ...current, end: undefined }));
              }}
            >
              <SelectTrigger aria-invalid={!!formErrors.end}>
                <SelectValue placeholder="End time" />
              </SelectTrigger>
              <SelectContent>
                {shopTimeOptions().map((time) => (
                  <SelectItem key={time.value} value={time.value}>
                    {time.label}
                  </SelectItem>
                ))}
              </SelectContent>
            </Select>
            <FieldError message={formErrors.end} />
          </div>
        </div>
      )}

      <div className="space-y-1.5">
        <Label>Reason</Label>
        <Input
          value={reason}
          onChange={(e) => setReason(e.target.value)}
          placeholder="Holiday, team event..."
        />
      </div>
    </div>

    <DialogFooter>
      <Button variant="outline" onClick={resetForm}
disabled={save.isPending}>
        Cancel
```

```
          </Button>
          <Button
            onClick={handleSave}
            disabled={save.isPending}
            className="font-display uppercase"
          >
            <Pencil /> Save changes
          </Button>
        </DialogFooter>
      </DialogContent>
    </Dialog>

    <ListToolbar
      summary={
        blocks.data && `Showing ${filteredBlocks.length} of ${blocks.data.length}
schedule blocks`
      }
    >
      <ToolbarSearch
        value={search}
        onChange={setSearch}
        placeholder="Search date or reason"
        label="Search schedule blocks by date or reason"
      />
    </ListToolbar>

    <ListCard>
      <DataList
        label="Schedule blocks"
        rows={filteredBlocks}
        rowKey={(b) => b.id}
        loading={blocks.isLoading}
        error={
          blocks.isError
            ? {
                message: "Could not load schedule blocks.",
                onRetry: () => void blocks.refetch(),
              }
            : null
        }
        empty={
          <ListEmpty
            icon={CalendarX}
            title={search ? "No schedule blocks match your search." : "No blocked
schedules."}
```

```
            description={
              search
                ? "Try a different date or reason."
                : "Use the form above to close a day or a time range."
            }
            action={search ? <ResetFiltersButton onClick={() => setSearch("")} /> :
undefined}
          />
        }
        columns={[
          {
            key: "date",
            header: "Date",
            card: "title",
            className: "w-56",
            cell: (b) => <span
className="font-medium">{formatDateLong(b.block_date)}</span>,
          },
          {
            key: "slot",
            header: "Time",
            className: "w-44",
            cell: (b) => <span className="text-sm">{blockSlotLabel(b)}</span>,
          },
          {
            key: "reason",
            header: "Reason",
            card: "wide",
            cell: (b) => (
              <span className="text-sm text-muted-foreground">
                {decodeBlockReason(b.reason).userReason || "No reason given"}
              </span>
            ),
          },
        ]}
        actionsClassName="w-36"
        actions={(b) => (
          <>
            <Button size="sm" variant="outline" onClick={() => startEditing(b)}>
              <Pencil /> Edit
            </Button>
            <MoreActionsMenu itemLabel={`the block on
${formatDateLong(b.block_date)}`}>
              <ArchiveMenuItem
                disabled={archive.isPending}
```

```
                onSelect={() => setArchiveTarget(b.id)}
              />
            </MoreActionsMenu>
          </>
        )}
      />
    </ListCard>
    <ArchiveConfirmationDialog
      open={Boolean(archiveTarget)}
      recordLabel="schedule block"
      pending={archive.isPending}
      onOpenChange={(nextOpen) => !nextOpen && setArchiveTarget(null)}
      onConfirm={() => {
        if (archiveTarget) archive.mutate(archiveTarget);
        setArchiveTarget(null);
      }}
    />
  </AdminPage>
 );
}

/** "Whole day", or the blocked time range. */
function blockSlotLabel(block: ScheduleBlock) {
 if (!block.start_time) return "Whole day";
 const start = formatTime(String(block.start_time).slice(0, 5));
 const { endTime } = decodeBlockReason(block.reason);
 return endTime ? `${start} – ${formatTime(endTime.slice(0, 5))}` : start;
}

function ReservedFieldError({ message }: { message?: string | undefined }) {
 return (
  <div className="min-h-8">
   <FieldError message={message} />
  </div>
 );
}

function ValidationSpace({ className = "" }: { className?: string }) {
 return <div aria-hidden="true" className={`min-h-8 ${className}`} />;
}
```

src/routes/_authenticated/admin/services.tsx

```
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import { createFileRoute, Link } from "@tanstack/react-router";
import {
  Archive,
  CircleAlert,
  Clock,
  EllipsisVertical,
  Eye,
  Loader2,
  Pencil,
  Plus,
  RotateCcw,
  Search,
  Tags,
  Wrench,
} from "lucide-react";
import { useEffect, useMemo, useRef, useState, type ReactNode } from "react";

import { ArchiveConfirmationDialog } from
"@/components/admin/archive-confirmation-dialog";
import { PageHeader } from "@/components/admin/page-header";
import {
  AlertDialog,
  AlertDialogAction,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
} from "@/components/ui/alert-dialog";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Card, CardContent } from "@/components/ui/card";
import {
  Dialog,
  DialogContent,
  DialogDescription,
  DialogFooter,
  DialogHeader,
  DialogTitle,
} from "@/components/ui/dialog";
import {
```

```
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuSeparator,
  DropdownMenuTrigger,
} from "@/components/ui/dropdown-menu";
import { FieldError } from "@/components/ui/field-error";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@/components/ui/select";
import { Skeleton } from "@/components/ui/skeleton";
import { Switch } from "@/components/ui/switch";
import {
  Table,
  TableBody,
  TableCell,
  TableHead,
  TableHeader,
  TableRow,
} from "@/components/ui/table";
import { Textarea } from "@/components/ui/textarea";
import { Tooltip, TooltipContent, TooltipTrigger } from "@/components/ui/tooltip";
import { supabase } from "@/integrations/supabase/client";
import { activeStatusTone, formatPHP } from "@/lib/shop";
import { cn } from "@/lib/utils";

export const Route = createFileRoute("/_authenticated/admin/services")({
  component: ServicesAdmin,
});

type Service = {
  id: string;
  name: string;
  description: string | null;
  price: number | string;
  duration_minutes: number;
  large_bike_price: number | string | null;
  large_bike_duration_minutes: number | null;
  is_active: boolean;
```

```
  is_archived: boolean;
  category: string;
  created_at: string;
};

type ServiceForm = {
  name: string;
  category: string;
  description: string;
  defaultPrice: number;
  defaultDuration: number;
  largeBikeEnabled: boolean;
  largeBikePrice: number;
  largeBikeDuration: number;
  isActive: boolean;
};

type FormErrors = Partial<
  Record<
    "name" | "category" | "description" | "defaultPrice" | "defaultDuration" |
"largeBike",
    string
  >
>;

type StatusFilter = "all" | "active" | "inactive";

const blank: ServiceForm = {
  name: "",
  category: "",
  description: "",
  defaultPrice: Number.NaN,
  defaultDuration: Number.NaN,
  largeBikeEnabled: false,
  largeBikePrice: Number.NaN,
  largeBikeDuration: Number.NaN,
  isActive: true,
};
const ALL_CATEGORIES = "__all_categories__";

function ServicesAdmin() {
  const queryClient = useQueryClient();
  const [open, setOpen] = useState(false);
  const [editing, setEditing] = useState<Service | null>(null);
  const [viewing, setViewing] = useState<Service | null>(null);
```

```
  const [form, setForm] = useState<ServiceForm>({ ...blank });
  const [formErrors, setFormErrors] = useState<FormErrors>({});
  const [saveError, setSaveError] = useState<string | null>(null);
  const [search, setSearch] = useState("");
  const [filterCategory, setFilterCategory] = useState(ALL_CATEGORIES);
  const [filterStatus, setFilterStatus] = useState<StatusFilter>("all");
  const [archiveTarget, setArchiveTarget] = useState<string | null>(null);
  const [archiveError, setArchiveError] = useState<string | null>(null);
  const [saveConfirmationOpen, setSaveConfirmationOpen] = useState(false);

  const services = useQuery({
    queryKey: ["admin-services"],
    queryFn: async () => {
      const { data, error } = await supabase
        .from("services")
        .select("*")
        .eq("is_archived", false)
        .order("sort_order");
      if (error) throw error;
      return (data ?? []) as Service[];
    },
  });

  const categories = useMemo(
    () =>
      Array.from(
        new Set((services.data ?? []).map((service) =>
service.category).filter(Boolean)),
      ).sort((left, right) => left.localeCompare(right)),
    [services.data],
  );

  const save = useMutation({
    mutationFn: async () => {
      const largeBikeFields = form.largeBikeEnabled
        ? {
            large_bike_price: form.largeBikePrice,
            large_bike_duration_minutes: form.largeBikeDuration,
          }
        : { large_bike_price: null, large_bike_duration_minutes: null };
      const payload = {
        name: form.name.trim(),
        category: form.category.trim().toLowerCase(),
        description: form.description.trim(),
        duration_minutes: form.defaultDuration,
```

```
        is_active: form.isActive,
        ...largeBikeFields,
      };
      const { error } = editing
        ? await supabase.from("services").update(payload).eq("id", editing.id)
        : await supabase.from("services").insert({ ...payload, price: form.defaultPrice
});
      if (error) throw error;
    },
    onSuccess: () => {
      setSaveConfirmationOpen(false);
      closeEditor();
      queryClient.invalidateQueries({ queryKey: ["admin-services"], exact: false });
      queryClient.invalidateQueries({ queryKey: ["services"], exact: false });
      queryClient.invalidateQueries({ queryKey: ["prices"], exact: false });
    },
    onError: (error: Error) => {
      console.error("Could not save service:", error);
      setSaveConfirmationOpen(false);
      setSaveError("Could not save this service. Please review the details and try
again.");
    },
  });

  const archive = useMutation({
    mutationFn: async (id: string) => {
      const { error } = await supabase.from("services").update({ is_archived: true
}).eq("id", id);
      if (error) throw error;
    },
    onSuccess: () => {
      setArchiveError(null);
      queryClient.invalidateQueries({ queryKey: ["admin-services"], exact: false });
      queryClient.invalidateQueries({ queryKey: ["archived-services"], exact: false
});
    },
    onError: () => setArchiveError("Could not archive this service. Please try
again."),
  });

  function clearError(field: keyof FormErrors) {
    setFormErrors((current) => {
      const next = { ...current };
      delete next[field];
      return next;
```

```
    });
    setSaveError(null);
  }

  function closeEditor() {
    setOpen(false);
    setEditing(null);
    setForm({ ...blank });
    setFormErrors({});
    setSaveError(null);
  }

  function openEditor(service?: Service) {
    setEditing(service ?? null);
    setForm(
      service
        ? {
            name: service.name,
            category: service.category,
            description: service.description ?? "",
            defaultPrice: Number(service.price),
            defaultDuration: service.duration_minutes,
            largeBikeEnabled:
              service.large_bike_price !== null &&
service.large_bike_duration_minutes !== null,
            largeBikePrice: Number(service.large_bike_price ?? Number.NaN),
            largeBikeDuration: service.large_bike_duration_minutes ?? Number.NaN,
            isActive: service.is_active,
          }
        : { ...blank },
    );
    setFormErrors({});
    setSaveError(null);
    setOpen(true);
  }

  function validateForm() {
    const errors: FormErrors = {};
    if (form.name.trim().length < 2)
      errors.name = "Enter a service name with at least 2 characters.";
    if (form.category.trim().length < 2)
      errors.category = "Enter a service category with at least 2 characters.";
    if (!form.description.trim()) errors.description = "Service details are required.";
    if (!Number.isFinite(form.defaultPrice) || form.defaultPrice < 0)
      errors.defaultPrice = "Enter a valid Small Bike price of PHP 0 or more.";
```

```
    if (
      !Number.isInteger(form.defaultDuration) ||
      form.defaultDuration < 15 ||
      form.defaultDuration > 480
    )
      errors.defaultDuration = "Enter a whole duration from 15 to 480 minutes.";
    if (
      form.largeBikeEnabled &&
      (!Number.isFinite(form.largeBikePrice) ||
        form.largeBikePrice <= form.defaultPrice ||
        !Number.isInteger(form.largeBikeDuration) ||
        form.largeBikeDuration < form.defaultDuration ||
        form.largeBikeDuration > 480)
    ) {
      errors.largeBike =
        "Large Bike price must be higher than the Small Bike price, and duration
must be from the Small Bike duration to 480 minutes.";
    }
    setFormErrors(errors);
    return Object.keys(errors).length === 0;
  }

  const normalizedSearch = search.trim().toLowerCase();
  const filteredServices = (services.data ?? []).filter(
    (service) =>
      (filterCategory === ALL_CATEGORIES || service.category ===
filterCategory) &&
      (filterStatus === "all" || service.is_active === (filterStatus === "active")) &&
      (!normalizedSearch ||
        service.name.toLowerCase().includes(normalizedSearch) ||
        (service.description ?? "").toLowerCase().includes(normalizedSearch)),
  );
  const hasFilters =
    Boolean(normalizedSearch) || filterCategory !== ALL_CATEGORIES ||
filterStatus !== "all";
  const activeCount = (services.data ?? []).filter((service) =>
service.is_active).length;
  const inactiveCount = (services.data ?? []).length - activeCount;

  function resetFilters() {
    setSearch("");
    setFilterCategory(ALL_CATEGORIES);
    setFilterStatus("all");
  }
```

```
const actions = (service: Service) => (
  <div className="flex items-center justify-end gap-1">
    <Button
      type="button"
      size="sm"
      variant="outline"
      onClick={() => openEditor(service)}
      aria-label={`Edit ${service.name}`}
    >
      <Pencil /> Edit
    </Button>
    <DropdownMenu>
      <Tooltip>
        <TooltipTrigger asChild>
          <DropdownMenuTrigger asChild>
            <Button
              type="button"
              size="icon"
              variant="ghost"
              aria-label={`More actions for ${service.name}`}
            >
              <EllipsisVertical className="size-4" />
            </Button>
          </DropdownMenuTrigger>
        </TooltipTrigger>
        <TooltipContent>More actions</TooltipContent>
      </Tooltip>
      <DropdownMenuContent align="end">
        <DropdownMenuItem onSelect={() => setViewing(service)}>
          <Eye /> View details
        </DropdownMenuItem>
        <DropdownMenuItem asChild>
          <Link to="/admin/prices">
            <Tags /> Manage prices
          </Link>
        </DropdownMenuItem>
        <DropdownMenuSeparator />
        <DropdownMenuItem
          onSelect={() => setArchiveTarget(service.id)}
          className="text-destructive focus:text-destructive"
        >
          <Archive /> Archive
        </DropdownMenuItem>
      </DropdownMenuContent>
    </DropdownMenu>
```

```
  </div>
 );

 return (
  <div className="mx-auto w-full max-w-7xl">
   <PageHeader
    title="Services"
    description="Manage the service menu, Small Bike defaults, and optional
Large Bike pricing and duration."
    action={
     <Button className="font-display uppercase" onClick={() =>
openEditor()}>
      <Plus /> Add service
     </Button>
    }
   />
   {archiveError && (
    <p
     role="alert"
     className="mb-4 flex items-center gap-2 rounded-lg border
border-destructive/40 bg-destructive/5 p-3 text-sm text-destructive"
    >
     <CircleAlert className="size-4 shrink-0" aria-hidden="true" />
{archiveError}
    </p>
   )}

   <div className="mb-4 flex flex-col gap-3 lg:flex-row lg:flex-wrap
lg:items-center">
    <div className="relative min-w-0 lg:w-72">
     <Search
      className="pointer-events-none absolute top-1/2 left-3 size-4
-translate-y-1/2 text-muted-foreground"
      aria-hidden="true"
     />
     <Input
      type="search"
      value={search}
      onChange={(event) => setSearch(event.target.value)}
      placeholder="Search services or details"
      aria-label="Search services by name or details"
      className="pl-9"
     />
    </div>
    <div className="grid grid-cols-2 gap-3 sm:flex sm:flex-wrap
```

```
sm:items-center">
        <Select value={filterCategory} onValueChange={setFilterCategory}>
          <SelectTrigger aria-label="Filter by category" className="w-full
capitalize sm:w-48">
            <SelectValue placeholder="All categories" />
          </SelectTrigger>
          <SelectContent>
            <SelectItem value={ALL_CATEGORIES}>All categories</SelectItem>
            {categories.map((category) => (
              <SelectItem key={category} value={category} className="capitalize">
                {category}
              </SelectItem>
            ))}
          </SelectContent>
        </Select>
        <Select
          value={filterStatus}
          onValueChange={(value) => setFilterStatus(value as StatusFilter)}
        >
          <SelectTrigger aria-label="Filter by status" className="w-full sm:w-40">
            <SelectValue />
          </SelectTrigger>
          <SelectContent>
            <SelectItem value="all">All statuses</SelectItem>
            <SelectItem value="active">Active ({activeCount})</SelectItem>
            <SelectItem value="inactive">Inactive ({inactiveCount})</SelectItem>
          </SelectContent>
        </Select>
        {hasFilters && (
          <Button variant="outline" onClick={resetFilters} className="col-span-2
sm:col-span-1">
            <RotateCcw /> Reset filters
          </Button>
        )}
      </div>
      {services.data && (
        <p className="text-sm text-muted-foreground lg:ml-auto">
          Showing {filteredServices.length} of {services.data.length} services
        </p>
      )}
    </div>

    <Card className="overflow-hidden border-border/70 bg-card/60">
      <CardContent className="p-0">
        {filteredServices.length > 0 && (
```

```
        <>
          {/* Phones and tablets: one card per service, so nothing scrolls
              sideways next to the admin sidebar. */}
          <ul className="divide-y divide-border/70 lg:hidden">
            {filteredServices.map((service) => (
              <li
                key={service.id}
                className="grid gap-3 p-4 sm:grid-cols-[minmax(0,1fr)_auto]
sm:gap-x-6"
              >
                <div className="min-w-0 space-y-2">
                  <div className="flex items-start justify-between gap-3">
                    <ServiceIdentity service={service} />
                    <span className="sm:hidden">
                      <StatusBadge active={service.is_active} />
                    </span>
                  </div>
                  <ExpandableServiceDescription
                    description={service.description}
                    serviceName={service.name}
                  />
                  <ServicePricing service={service} layout="inline" />
                </div>
                <div className="flex items-center justify-between gap-2 sm:flex-col
sm:items-end sm:justify-start">
                  <span className="hidden sm:block">
                    <StatusBadge active={service.is_active} />
                  </span>
                  {actions(service)}
                </div>
              </li>
            ))}
          </ul>

          {/* Laptops and desktops: a compact table. */}
          <Table className="hidden w-full table-fixed lg:table">
            <TableHeader>
              <TableRow>
                <TableHead className="pl-5">Service</TableHead>
                <TableHead className="w-52">Pricing</TableHead>
                <TableHead className="w-28 text-center">Status</TableHead>
                <TableHead className="w-40 pr-5 text-right">
                  <span className="sr-only">Actions</span>
                </TableHead>
              </TableRow>
```

```
            </TableHeader>
            <TableBody>
              {filteredServices.map((service) => (
                <TableRow key={service.id} className="align-top">
                  <TableCell className="py-3 pl-5">
                    <ServiceIdentity service={service} />
                    <ExpandableServiceDescription
                      description={service.description}
                      serviceName={service.name}
                    />
                  </TableCell>
                  <TableCell className="py-3">
                    <ServicePricing service={service} layout="stacked" />
                  </TableCell>
                  <TableCell className="py-3 text-center">
                    <StatusBadge active={service.is_active} />
                  </TableCell>
                  <TableCell className="py-3 pr-5">{actions(service)}</TableCell>
                </TableRow>
              ))}
            </TableBody>
          </Table>
        </>
      )}

      {services.isLoading && (
        <div className="divide-y divide-border/70" aria-label="Loading
services">
          {Array.from({ length: 5 }, (_, index) => (
            <div key={index} className="space-y-2 p-4">
              <Skeleton className="h-4 w-48 max-w-full" />
              <Skeleton className="h-3 w-72 max-w-full" />
            </div>
          ))}
        </div>
      )}
      {services.isError && (
        <div className="px-4 py-10 text-center">
          <p className="inline-flex items-center gap-2 text-sm text-destructive">
            <CircleAlert className="size-4" aria-hidden="true" /> Could not load
services.
          </p>
          <div className="mt-3">
            <Button size="sm" variant="outline" onClick={() => void
services.refetch()}>
```

```
              Try again
            </Button>
          </div>
        </div>
      )}
      {services.isSuccess && filteredServices.length === 0 && (
        <div className="px-4 py-12 text-center">
          <Wrench className="mx-auto size-8 text-muted-foreground/60"
aria-hidden="true" />
          <p className="mt-3 text-sm font-medium">
            {hasFilters ? "No services match your filters." : "No services yet."}
          </p>
          <p className="mt-1 text-sm text-muted-foreground">
            {hasFilters
              ? "Try a different search, category, or status."
              : "Add your first service to show it on the booking page."}
          </p>
          <div className="mt-4">
            {hasFilters ? (
              <Button size="sm" variant="outline" onClick={resetFilters}>
                <RotateCcw /> Reset filters
              </Button>
            ) : (
              <Button size="sm" onClick={() => openEditor()}>
                <Plus /> Add service
              </Button>
            )}
          </div>
        </div>
      )}
    </CardContent>
  </Card>

  <ServiceViewDialog
    service={viewing}
    onOpenChange={(isOpen) => !isOpen && setViewing(null)}
    onEdit={(service) => {
      setViewing(null);
      openEditor(service);
    }}
  />

  <Dialog open={open} onOpenChange={(isOpen) => !isOpen &&
!save.isPending && closeEditor()}>
    <DialogContent className="grid-cols-[minmax(0,1fr)] sm:max-w-2xl">
```

```
        <DialogHeader>
          <DialogTitle className="font-display text-2xl uppercase">
            {editing ? "Edit service" : "Add service"}
          </DialogTitle>
          <DialogDescription>
            {editing
              ? "Update the service details, Small Bike duration, status, and Large
Bike configuration."
              : "Fill in the details customers see when booking, then set the Small
Bike default price and duration."}
          </DialogDescription>
        </DialogHeader>

        <form
          noValidate
          className="space-y-6"
          onSubmit={(event) => {
            event.preventDefault();
            if (validateForm()) setSaveConfirmationOpen(true);
          }}
        >
          <FormSection title="Service details">
            <div className="grid gap-4 sm:grid-cols-2">
              <FormField id="service-name" label="Service name"
error={formErrors.name}>
                <Input
                  id="service-name"
                  value={form.name}
                  placeholder="e.g. Change Oil"
                  onChange={(event) => {
                    setForm({ ...form, name: event.target.value });
                    clearError("name");
                  }}
                  aria-invalid={Boolean(formErrors.name)}
                />
              </FormField>
              <FormField
                id="service-category"
                label="Category"
                hint="Pick an existing category or type a new one."
                error={formErrors.category}
              >
                <Input
                  id="service-category"
                  value={form.category}
```

```
                list="service-category-options"
                placeholder="e.g. maintenance"
                onChange={(event) => {
                  setForm({ ...form, category: event.target.value });
                  clearError("category");
                }}
                aria-invalid={Boolean(formErrors.category)}
              />
              <datalist id="service-category-options">
                {categories.map((category) => (
                  <option key={category} value={category} />
                ))}
              </datalist>
            </FormField>
          </div>
          <FormField
            id="service-description"
            label="Details"
            hint="Shown to customers on the services page. Describe what the
service includes."
            error={formErrors.description}
          >
            <Textarea
              id="service-description"
              rows={4}
              value={form.description}
              onChange={(event) => {
                setForm({ ...form, description: event.target.value });
                clearError("description");
              }}
              aria-invalid={Boolean(formErrors.description)}
            />
          </FormField>
        </FormSection>

        <FormSection
          title="Small Bike (125cc and below)"
          description="The default price and service time used for most
motorcycles."
        >
          <div className="grid gap-4 sm:grid-cols-2">
            <FormField
              id="service-small-duration"
              label="Duration (minutes)"
              hint="15 to 480 minutes."
```

```
            error={formErrors.defaultDuration}
          >
            <Input
              id="service-small-duration"
              type="number"
              inputMode="numeric"
              min="15"
              max="480"
              step="15"
              value={Number.isFinite(form.defaultDuration) ? form.defaultDuration
: ""}
              onChange={(event) => {
                setForm({
                  ...form,
                  defaultDuration:
                    event.target.value === "" ? Number.NaN :
Number(event.target.value),
                });
                clearError("defaultDuration");
              }}
              aria-invalid={Boolean(formErrors.defaultDuration)}
            />
          </FormField>
          {editing ? (
            <div className="rounded-lg border border-border/70 bg-muted/30
p-3">
              <p className="text-xs tracking-wide text-muted-foreground
uppercase">
                Current price
              </p>
              <p className="font-display text-2xl text-primary tabular-nums">
                {formatPHP(form.defaultPrice)}
              </p>
              <p className="mt-1 text-xs text-muted-foreground">
                Change prices in{" "}
                <Link
                  to="/admin/prices"
                  className="font-medium text-primary underline
underline-offset-2"
                >
                  Prices Management
                </Link>{" "}
                so each change is recorded with a reason.
              </p>
            </div>
```

```
            ) : (
              <FormField id="service-small-price" label="Price"
error={formErrors.defaultPrice}>
                <PesoInput
                  id="service-small-price"
                  value={form.defaultPrice}
                  invalid={Boolean(formErrors.defaultPrice)}
                  onChange={(defaultPrice) => {
                    setForm({ ...form, defaultPrice });
                    clearError("defaultPrice");
                  }}
                />
              </FormField>
            )}
          </div>
        </FormSection>

        <FormSection
          title="Large Bike (126cc and above)"
          description="Offer a separate price and service time for larger
motorcycles."
          action={
            <div className="flex items-center gap-2">
              <Label htmlFor="service-large-enabled" className="text-sm
font-normal">
                {form.largeBikeEnabled ? "Offered" : "Not offered"}
              </Label>
              <Switch
                id="service-large-enabled"
                checked={form.largeBikeEnabled}
                onCheckedChange={(enabled) => {
                  setForm((current) =>
                    enabled
                      ? {
                          ...current,
                          largeBikeEnabled: true,
                          largeBikePrice: Number.NaN,
                          largeBikeDuration: current.defaultDuration,
                        }
                      : {
                          ...current,
                          largeBikeEnabled: false,
                          largeBikePrice: Number.NaN,
                          largeBikeDuration: Number.NaN,
                        },
```

```
                );
                clearError("largeBike");
              }}
            />
          </div>
        }
      >
        {form.largeBikeEnabled ? (
          <div className="space-y-2">
            <div className="grid gap-4 sm:grid-cols-2">
              <FormField id="service-large-price" label="Price">
                <PesoInput
                  id="service-large-price"
                  value={form.largeBikePrice}
                  invalid={Boolean(formErrors.largeBike)}
                  onChange={(largeBikePrice) => {
                    setForm({ ...form, largeBikePrice });
                    clearError("largeBike");
                  }}
                />
              </FormField>
              <FormField id="service-large-duration" label="Duration (minutes)">
                <Input
                  id="service-large-duration"
                  type="number"
                  inputMode="numeric"
                  min="15"
                  max="480"
                  step="15"
                  value={
                    Number.isFinite(form.largeBikeDuration) ?
form.largeBikeDuration : ""
                  }
                  onChange={(event) => {
                    setForm({
                      ...form,
                      largeBikeDuration:
                        event.target.value === "" ? Number.NaN :
Number(event.target.value),
                    });
                    clearError("largeBike");
                  }}
                  aria-invalid={Boolean(formErrors.largeBike)}
                />
              </FormField>
```

```
              </div>
              {formErrors.largeBike ? (
                <FieldError message={formErrors.largeBike} />
              ) : (
                <p className="text-xs text-muted-foreground">
                  Must cost more than the Small Bike price and take at least as long
as the
                  Small Bike duration (up to 480 minutes).
                </p>
              )}
            </div>
          ) : (
            <p className="rounded-lg border border-dashed border-border/70 p-3
text-sm text-muted-foreground">
              Large Bike bookings for this service are not available until you turn
this on.
            </p>
          )}
        </FormSection>

        {editing && (
          <FormSection title="Status">
            <div className="flex items-start justify-between gap-4 rounded-lg
border border-border/70 p-3">
              <div>
                <Label htmlFor="service-active">{form.isActive ? "Active" :
"Inactive"}</Label>
                <p className="mt-0.5 text-xs text-muted-foreground">
                  Inactive services are hidden from the customer booking page.
                </p>
              </div>
              <Switch
                id="service-active"
                checked={form.isActive}
                onCheckedChange={(isActive) => {
                  setForm((current) => ({ ...current, isActive }));
                  setSaveError(null);
                }}
              />
            </div>
          </FormSection>
        )}

        {saveError && (
          <p
```

```
            role="alert"
            className="flex items-start gap-2 rounded-lg border
border-destructive/40 bg-destructive/5 p-3 text-sm text-destructive"
          >
            <CircleAlert className="mt-0.5 size-4 shrink-0" aria-hidden="true" />
            {saveError}
          </p>
        )}

        <DialogFooter className="gap-2 border-t border-border/70 pt-4">
          <Button
            type="button"
            variant="outline"
            onClick={closeEditor}
            disabled={save.isPending}
          >
            Cancel
          </Button>
          <Button type="submit" disabled={save.isPending}>
            {save.isPending && <Loader2 className="animate-spin" />}
            {editing ? "Save changes" : "Add service"}
          </Button>
        </DialogFooter>
      </form>
    </DialogContent>
  </Dialog>

  <AlertDialog open={saveConfirmationOpen}
onOpenChange={setSaveConfirmationOpen}>
    <AlertDialogContent>
      <AlertDialogHeader>
        <AlertDialogTitle>
          {editing ? "Save changes to this service?" : "Add this service?"}
        </AlertDialogTitle>
        <AlertDialogDescription>
          {editing
            ? "This will update the service details, the Small Bike duration, its
status, and any Large Bike configuration."
            : "The service will be added with these details and prices. While it is
active, customers can select it when booking."}
        </AlertDialogDescription>
      </AlertDialogHeader>
      <AlertDialogFooter>
        <AlertDialogCancel disabled={save.isPending}>Go
back</AlertDialogCancel>
```

```
            <AlertDialogAction disabled={save.isPending} onClick={() =>
save.mutate()}>
              {editing ? "Save changes" : "Add service"}
            </AlertDialogAction>
          </AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialog>
      <ArchiveConfirmationDialog
        open={Boolean(archiveTarget)}
        recordLabel="service"
        pending={archive.isPending}
        onOpenChange={(isOpen) => !isOpen && setArchiveTarget(null)}
        onConfirm={() => {
          if (archiveTarget) archive.mutate(archiveTarget);
          setArchiveTarget(null);
        }}
      />
    </div>
  );
}

function ServiceIdentity({ service }: { service: Service }) {
  return (
    <div className="min-w-0">
      <p className="font-medium break-words">{service.name}</p>
      <p className="mt-0.5 text-xs text-muted-foreground">
        <span className="capitalize">{service.category}</span> · Added{" "}
        {new Date(service.created_at).toLocaleDateString("en-PH", {
          day: "2-digit",
          month: "short",
          year: "numeric",
        })}
      </p>
    </div>
  );
}

function StatusBadge({ active }: { active: boolean }) {
  return (
    <Badge variant="outline" className={`uppercase
${activeStatusTone(active)}`}>
      {active ? "Active" : "Inactive"}
    </Badge>
  );
}
```

```
function hasLargeBikePricing(service: Service) {
  return service.large_bike_price !== null && service.large_bike_duration_minutes
!== null;
}

/** Small and Large Bike price and duration for one service. */
function ServicePricing({ service, layout }: { service: Service; layout: "inline" |
"stacked" }) {
  const rows = [
    {
      label: "Small Bike",
      price: service.price,
      duration: service.duration_minutes,
    },
    {
      label: "Large Bike",
      price: hasLargeBikePricing(service) ? service.large_bike_price : null,
      duration: service.large_bike_duration_minutes,
    },
  ];
  return (
    <dl
      className={cn(
        "text-sm",
        layout === "inline"
          ? "grid grid-cols-2 gap-3 sm:flex sm:flex-wrap sm:gap-x-8"
          : "space-y-1.5",
      )}
    >
      {rows.map((row) => (
        <div
          key={row.label}
          className={cn(
            "min-w-0",
            layout === "stacked" && "flex items-baseline justify-between gap-2",
          )}
        >
          <dt className="text-xs tracking-wide whitespace-nowrap
text-muted-foreground uppercase">
            {row.label}
          </dt>
          <dd className={cn(layout === "inline" ? "mt-0.5" :
"whitespace-nowrap")}>
            {row.price === null ? (
```

```
            <span className="text-xs text-muted-foreground">Not set</span>
          ) : (
            <>
              <span className="font-medium
tabular-nums">{formatPHP(Number(row.price))}</span>
              <span className="text-xs text-muted-foreground"> · {row.duration}
min</span>
            </>
          )}
        </dd>
      </div>
    ))}
  </dl>
 );
}

function ExpandableServiceDescription({
 description,
 serviceName,
}: {
 description: string | null;
 serviceName: string;
}) {
 const descriptionRef = useRef<HTMLParagraphElement>(null);
 const [expanded, setExpanded] = useState(false);
 const [canExpand, setCanExpand] = useState(false);
 const details = description?.trim() || "No details provided.";

 useEffect(() => {
  const element = descriptionRef.current;
  if (!element) return;

  const checkOverflow = () => {
   if (!expanded) setCanExpand(element.scrollHeight > element.clientHeight +
1);
  };

  checkOverflow();
  if (typeof ResizeObserver === "undefined") return;

  const observer = new ResizeObserver(checkOverflow);
  observer.observe(element);
  return () => observer.disconnect();
 }, [details, expanded]);
```

```
  return (
    <div className="mt-1 max-w-prose text-xs leading-relaxed
text-muted-foreground">
      <p
        ref={descriptionRef}
        className={cn(
          expanded ? "break-words" : "line-clamp-2 break-words",
          !description?.trim() && "italic",
        )}
      >
        {details}
      </p>
      {canExpand && (
        <Button
          type="button"
          variant="link"
          size="sm"
          className="mt-0.5 h-auto p-0 text-xs md:h-auto"
          onClick={() => setExpanded((current) => !current)}
          aria-expanded={expanded}
          aria-label={`${expanded ? "Collapse" : "Expand"} details for
${serviceName}`}
        >
          {expanded ? "See less" : "See more"}
        </Button>
      )}
    </div>
  );
}

function FormSection({
  title,
  description,
  action,
  children,
}: {
  title: string;
  description?: string;
  action?: ReactNode;
  children: ReactNode;
}) {
  return (
    <section className="space-y-4">
      <div className="flex flex-wrap items-start justify-between gap-3">
        <div className="min-w-0">
```

```
          <h3 className="font-display text-base tracking-wide
uppercase">{title}</h3>
          {description && <p className="mt-0.5 text-sm
text-muted-foreground">{description}</p>}
        </div>
        {action}
      </div>
      {children}
    </section>
  );
}

function FormField({
  id,
  label,
  hint,
  error,
  children,
}: {
  id: string;
  label: string;
  hint?: string;
  error?: string | undefined;
  children: ReactNode;
}) {
  return (
    <div className="space-y-1.5">
      <Label htmlFor={id}>{label}</Label>
      {children}
      {error ? (
        <FieldError message={error} />
      ) : hint ? (
        <p className="text-xs text-muted-foreground">{hint}</p>
      ) : null}
    </div>
  );
}

function PesoInput({
  id,
  value,
  invalid,
  onChange,
}: {
  id: string;
```

```
  value: number;
  invalid: boolean;
  onChange: (value: number) => void;
}) {
  return (
    <div className="relative">
      <span
        aria-hidden="true"
        className="pointer-events-none absolute top-1/2 left-3 -translate-y-1/2
text-sm text-muted-foreground"
      >
        ₱
      </span>
      <Input
        id={id}
        type="number"
        inputMode="decimal"
        min="0"
        step="0.01"
        value={Number.isFinite(value) ? value : ""}
        onChange={(event) =>
          onChange(event.target.value === "" ? Number.NaN :
Number(event.target.value))
        }
        aria-invalid={invalid}
        className="pl-7 tabular-nums"
      />
    </div>
  );
}

function ServiceViewDialog({
  service,
  onOpenChange,
  onEdit,
}: {
  service: Service | null;
  onOpenChange: (open: boolean) => void;
  onEdit: (service: Service) => void;
}) {
  return (
    <Dialog open={Boolean(service)} onOpenChange={onOpenChange}>
      <DialogContent className="grid-cols-[minmax(0,1fr)] sm:max-w-2xl">
        <DialogHeader>
          <p className="text-xs tracking-widest text-muted-foreground
```

```
uppercase">Service details</p>
        <DialogTitle className="font-display text-2xl break-words uppercase">
          {service?.name}
        </DialogTitle>
        <DialogDescription asChild>
          <div className="flex flex-wrap items-center justify-center gap-2
sm:justify-start">
            <span className="capitalize">{service?.category}</span>
            {service && <StatusBadge active={service.is_active} />}
          </div>
        </DialogDescription>
      </DialogHeader>
      <div className="space-y-5">
        <section className="rounded-lg border border-border/70 bg-muted/20
p-4">
          <p className="text-xs tracking-wide text-muted-foreground
uppercase">Details</p>
          <p className="mt-1 text-sm leading-relaxed break-words
whitespace-pre-line">
            {service?.description || "No details provided."}
          </p>
        </section>
        {service && (
          <section className="grid gap-3 sm:grid-cols-2">
            <PricingCard
              title="Small Bike (125cc and below)"
              price={service.price}
              duration={service.duration_minutes}
            />
            {hasLargeBikePricing(service) ? (
              <PricingCard
                title="Large Bike (126cc and above)"
                price={service.large_bike_price}
                duration={service.large_bike_duration_minutes}
              />
            ) : (
              <div className="rounded-lg border border-dashed border-border/70
p-4 text-sm text-muted-foreground">
                <p className="text-xs tracking-wide uppercase">Large Bike (126cc
and above)</p>
                <p className="mt-3">Not offered for this service.</p>
              </div>
            )}
          </section>
        )}
```

```
        </div>
        <DialogFooter className="gap-2">
          <Button variant="outline" onClick={() => onOpenChange(false)}>
            Close
          </Button>
          {service && (
            <Button onClick={() => onEdit(service)}>
              <Pencil /> Edit service
            </Button>
          )}
        </DialogFooter>
      </DialogContent>
    </Dialog>
  );
}

function PricingCard({
  title,
  price,
  duration,
}: {
  title: string;
  price: number | string | null | undefined;
  duration: number | null | undefined;
}) {
  return (
    <div className="rounded-lg border border-border/70 p-4">
      <p className="text-xs tracking-wide text-muted-foreground
uppercase">{title}</p>
      <p className="mt-2 font-display text-2xl text-primary tabular-nums">
        {formatPHP(Number(price ?? 0))}
      </p>
      <p className="mt-1 flex items-center gap-1.5 text-sm
text-muted-foreground">
        <Clock className="size-3.5" aria-hidden="true" /> {duration ?? 0} minutes
      </p>
    </div>
  );
}
```

src/routes/_authenticated/admin/settings.tsx

```
import { createFileRoute } from "@tanstack/react-router";

import { SettingsPage } from "@/components/admin/settings-page";

export const Route = createFileRoute("/_authenticated/admin/settings")({
  component: SettingsPage,
});
```

src/routes/_authenticated/route.tsx

```
import { createFileRoute, Outlet, redirect } from "@tanstack/react-router";

import { supabase } from "@/integrations/supabase/client";

export const Route = createFileRoute("/_authenticated")({
  ssr: false,
  beforeLoad: async () => {
    const { data, error } = await supabase.auth.getUser();
    if (error || !data.user) throw redirect({ to: "/auth" });

    const { data: hasActiveAdminSession, error: sessionError } =
      await supabase.rpc("validate_admin_session");
    if (sessionError || !hasActiveAdminSession) {
      await supabase.auth.signOut({ scope: "local" });
      throw redirect({ to: "/auth" });
    }

    return { user: data.user };
  },
  component: () => <Outlet />,
});
```

src/routes/api/offline-status.ts

```
import { createFileRoute } from "@tanstack/react-router";

/**
 * Offline copy only: whether the internet is back and how many offline
 * bookings are still waiting to be sent, for the status shown on every page.
 * Not found anywhere else.
 */
export const Route = createFileRoute("/api/offline-status")({
  server: {
    handlers: {
      GET: async () => {
        const { offlineSyncStatus } = await import("@/lib/offline-sync.server");
        const status = await offlineSyncStatus();
        if (!status.enabled) return new Response("Not found", { status: 404 });
        return Response.json(status, { headers: { "cache-control": "no-store" } });
      },
    },
  },
});
```

src/routes/api/offline-sync.ts

```
import { createFileRoute } from "@tanstack/react-router";

/**
 * Offline copy only. Called every minute by the local database
 * (offline/setup.sql) to check the internet and send offline bookings to the
 * live site (src/lib/offline-sync.server.ts). Not found anywhere else.
 */
export const Route = createFileRoute("/api/offline-sync")({
  server: {
    handlers: {
      POST: async ({ request }) => {
        const { offlineSyncConfig, runOfflineSync } = await
import("@/lib/offline-sync.server");
        if (!offlineSyncConfig()) return new Response("Not found", { status: 404 });
        const { refuseUnscheduledRequest } = await
import("@/lib/scheduled-request.server");
        const refusal = refuseUnscheduledRequest(request);
        if (refusal) return refusal;
        return Response.json(await runOfflineSync());
      },
    },
  },
});
```

src/routes/api/scheduled-notifications.ts

```
import { createFileRoute } from "@tanstack/react-router";

/**
 * Called every 5 minutes by the database (pg_cron, see migration
 * 20261026110000) to send appointment reminders and retry failed customer
 * messages. Only requests carrying SCHEDULED_NOTIFICATIONS_SECRET
are accepted.
 */
export const Route = createFileRoute("/api/scheduled-notifications")({
  server: {
    handlers: {
      POST: async ({ request }) => {
        const { refuseUnscheduledRequest } = await
import("@/lib/scheduled-request.server");
        const refusal = refuseUnscheduledRequest(request);
        if (refusal) return refusal;

        // Reminder emails link to My Appointment only on the real (https) site.
        const origin = new URL(request.url).origin;
        const { runScheduledNotifications } =
          await import("@/lib/appointment-confirmation.functions");
        try {
          const result = await runScheduledNotifications(
            origin.startsWith("https://") ? origin : null,
          );
          return Response.json({ ok: true, ...result });
        } catch {
          return Response.json({ ok: false, error: "The scheduled run failed." }, {
status: 500 });
        }
      },
    },
  },
});
```

src/routes/auth.tsx

```
import { useMutation } from "@tanstack/react-query";
import { createFileRoute, Link, useNavigate } from "@tanstack/react-router";
import { ArrowLeft, Check, Eye, EyeOff, Loader2, Lock, Mail, MailCheck } from
"lucide-react";
import { useEffect, useState } from "react";
import { z } from "zod";

import logo from "@/assets/logo-shp.png.asset.json";
import { Button } from "@/components/ui/button";
import { Card, CardContent } from "@/components/ui/card";
import { FieldError } from "@/components/ui/field-error";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { supabase } from "@/integrations/supabase/client";
import { recordAdminActivityEvent } from "@/lib/admin-activity";

export const Route = createFileRoute("/auth")({
  // reset=done: the password was just changed on /reset-password.
  validateSearch: z.object({ reset: z.literal("done").optional() }),
  head: () => ({
    meta: [
      { title: "Admin Login | Fake Rider Motorparts" },
      {
        name: "description",
        content:
          "Administrator login for the Fake Rider Motorparts appointment
management console.",
      },
      { name: "robots", content: "noindex" },
      { property: "og:title", content: "Admin Login | Fake Rider Motorparts" },
      {
        property: "og:description",
        content: "Administrator access to the shop management console.",
      },
    ],
  }),
  component: AuthPage,
});

function AuthPage() {
  const navigate = useNavigate();
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
```

```
  const [showPassword, setShowPassword] = useState(false);
  const [errors, setErrors] = useState<{ email?: string; password?: string }>({});
  const { reset } = Route.useSearch();
  const [mode, setMode] = useState<"sign-in" | "forgot">("sign-in");
  const [resetSentTo, setResetSentTo] = useState<string | null>(null);

  useEffect(() => {
    let isMounted = true;
    void supabase.auth.getSession().then(async ({ data }) => {
      if (!data.session) return;
      const { data: isActive, error } = await
supabase.rpc("validate_admin_session");
      if (!isMounted) return;
      if (!error && isActive) {
        navigate({ to: "/admin", replace: true });
        return;
      }
      await supabase.auth.signOut({ scope: "local" });
    });
    return () => {
      isMounted = false;
    };
  }, [navigate]);

  const signIn = useMutation({
    mutationFn: async () => {
      const { error } = await supabase.auth.signInWithPassword({
        email: email.trim().toLowerCase(),
        password,
      });
      if (error) throw new Error(`AUTHENTICATION_FAILED: ${error.message}`);

      const { error: claimError } = await supabase.rpc("claim_admin_session", {
        p_handover_token: null,
      });
      if (claimError) {
        await supabase.auth.signOut({ scope: "local" });
        throw new Error(`ADMIN_SESSION_FAILED: ${claimError.message}`);
      }

      // Do not navigate until the same server-side check used by protected
      // admin routes confirms this JWT owns the lease.
      const { data: isActive, error: validationError } =
        await supabase.rpc("validate_admin_session");
      if (validationError || !isActive) {
```

```
        await supabase.auth.signOut({ scope: "local" });
        throw new Error(
          `ADMIN_SESSION_FAILED: ${validationError?.message ?? "The new
administrator session could not be verified."}`,
        );
      }
    },
    onSuccess: async () => {
      try {
        await recordAdminActivityEvent({
          action: "signed in",
          resourceType: "Authentication",
          targetLabel: "Admin console",
          summary: "Administrator signed in to the admin console.",
        });
      } catch {
        // Access is verified by the protected admin route; a log failure must
        // not leave an authenticated administrator stuck on the login screen.
      }
      navigate({ to: "/admin", replace: true });
    },
    onError: (error: Error) => {
      const message = error.message.replace(
        /^(AUTHENTICATION_FAILED|ADMIN_SESSION_FAILED):\s*/,
        "",
      );
      setErrors((current) => ({
        ...current,
        password: /^AUTHENTICATION_FAILED:/.test(error.message)
          ? "Invalid email or password."
          : message || "We could not start the administrator session. Please try
again.",
      }));
    },
  });

  const sendReset = useMutation({
    mutationFn: async () => {
      const address = email.trim().toLowerCase();
      const { error } = await supabase.auth.resetPasswordForEmail(address, {
        redirectTo: `${window.location.origin}/reset-password`,
      });
      if (error) throw error;
      return address;
    },
```

```
    onSuccess: (address) => setResetSentTo(address),
    onError: (error: Error & { status?: number }) => {
      setErrors({
        email:
          error.status === 429 || /rate limit|too many/i.test(error.message)
            ? "Too many reset emails were requested. Wait a few minutes and try
again."
            : "We could not send the reset email. Please try again shortly.",
      });
    },
  });

  function showSignIn() {
    setMode("sign-in");
    setResetSentTo(null);
    setErrors({});
  }

  function validate() {
    const nextErrors: { email?: string; password?: string } = {};
    if (!/^\S+@\S+\.\S+$/.test(email.trim())) nextErrors.email = "Enter a valid email
address.";
    if (!password) nextErrors.password = "Enter your password.";
    setErrors(nextErrors);
    return Object.keys(nextErrors).length === 0;
  }

  return (
    <div className="flex min-h-screen items-center justify-center bg-background
px-4 py-12">
      <Card className="w-full max-w-md border-border/70 bg-card/70">
        <CardContent className="p-8">
          <img src={logo.url} alt="Fake Rider Motorparts logo" className="mx-auto
h-24 w-auto" />
          <h1 className="mt-4 text-center font-display text-2xl uppercase">
            {mode === "forgot" ? "Reset password" : "Admin login"}
          </h1>
          <p className="mt-1 text-center text-sm text-muted-foreground">
            {mode === "forgot"
              ? "Enter your admin email and we'll send a link to set a new password."
              : "Administrator access only."}
          </p>
          {reset === "done" && mode === "sign-in" && (
            <p
              role="status"
```

```
            className="mt-4 flex items-start gap-2 rounded-lg border
border-emerald-500/40 bg-emerald-500/5 p-3 text-sm text-emerald-700
dark:text-emerald-400"
          >
            <Check className="mt-0.5 size-4 shrink-0" aria-hidden="true" />
            Your password was changed. Sign in with your new password.
          </p>
        )}

        {mode === "forgot" ? (
          resetSentTo ? (
            <div className="mt-6 space-y-4">
              <p
                role="status"
                className="flex items-start gap-2 rounded-lg border
border-emerald-500/40 bg-emerald-500/5 p-3 text-sm text-emerald-700
dark:text-emerald-400"
              >
                <MailCheck className="mt-0.5 size-4 shrink-0" aria-hidden="true" />
                If {resetSentTo} belongs to an admin account, a reset link is on its
way. The link
                works once.
              </p>
              <Button
                type="button"
                className="w-full font-display uppercase"
                onClick={showSignIn}
              >
                Back to sign in
              </Button>
            </div>
          ) : (
            <form
              className="mt-6 space-y-4"
              onSubmit={(e) => {
                e.preventDefault();
                if (!/^\S+@\S+\.\S+$/.test(email.trim())) {
                  setErrors({ email: "Enter a valid email address." });
                  return;
                }
                sendReset.mutate();
              }}
            >
              <div className="space-y-1.5">
                <Label htmlFor="reset-email">Email</Label>
```

```
              <Input
                id="reset-email"
                type="email"
                autoComplete="email"
                value={email}
                onChange={(e) => {
                  setEmail(e.target.value.toLowerCase());
                  setErrors({});
                }}
                placeholder="admin@fakerider.ph"
                aria-invalid={!!errors.email}
              />
              <FieldError message={errors.email} />
            </div>
            <Button
              type="submit"
              className="w-full font-display uppercase"
              disabled={sendReset.isPending}
            >
              {sendReset.isPending ? <Loader2 className="animate-spin" /> :
<Mail />} Send reset
              link
            </Button>
            <Button type="button" variant="ghost" className="w-full"
onClick={showSignIn}>
              Back to sign in
            </Button>
          </form>
        )
      ) : (
        <form
          className="mt-6 space-y-4"
          onSubmit={(e) => {
            e.preventDefault();
            if (validate()) signIn.mutate();
          }}
        >
          <div className="space-y-1.5">
            <Label>Email</Label>
            <Input
              type="email"
              value={email}
              onChange={(e) => {
                setEmail(e.target.value.toLowerCase());
                setErrors((current) => {
```

```
                const { email: _, ...rest } = current;
                return rest;
              });
            }}
            placeholder="admin@fakerider.ph"
            required
            aria-invalid={!!errors.email}
          />
          <FieldError message={errors.email} />
        </div>
        <div className="space-y-1.5">
          <Label htmlFor="admin-password">Password</Label>
          <div className="relative">
            <Input
              id="admin-password"
              type={showPassword ? "text" : "password"}
              value={password}
              onChange={(e) => {
                setPassword(e.target.value);
                setErrors((current) => {
                  const { password: _, ...rest } = current;
                  return rest;
                });
              }}
              required
              aria-invalid={!!errors.password}
              className="pr-11"
            />
            <Button
              type="button"
              variant="ghost"
              size="icon"
              className="absolute top-0 right-0 text-muted-foreground
hover:text-foreground"
              onClick={() => setShowPassword((visible) => !visible)}
              aria-label={showPassword ? "Hide password" : "Show password"}
              aria-pressed={showPassword}
            >
              {showPassword ? <EyeOff /> : <Eye />}
            </Button>
          </div>
          <FieldError message={errors.password} />
          <div className="flex justify-end">
            <button
              type="button"
```

```
                  className="text-xs font-medium text-primary hover:underline
focus-visible:ring-2 focus-visible:ring-ring focus-visible:outline-none"
                  onClick={() => {
                    setMode("forgot");
                    setErrors({});
                  }}
                >
                  Forgot password?
                </button>
              </div>
            </div>
            <Button
              type="submit"
              className="w-full font-display uppercase"
              disabled={signIn.isPending}
            >
              {signIn.isPending ? <Lock className="animate-pulse" /> : <Lock />}
Sign in
            </Button>
          </form>
        )}

        <Button asChild variant="ghost" className="mt-4 w-full justify-center
text-sm">
          <Link to="/">
            <ArrowLeft /> Back to home
          </Link>
        </Button>
      </CardContent>
    </Card>
  </div>
 );
}
```

src/routes/book.tsx

```
import { useMutation, useQuery } from "@tanstack/react-query";
import { createFileRoute, Link, useSearch } from "@tanstack/react-router";
import { useServerFn } from "@tanstack/react-start";
import {
  ArrowLeft,
  ArrowRight,
  CalendarDays,
  Check,
  CheckCircle2,
  CircleAlert,
  Clock,
  History,
  Info,
  Loader2,
  Pencil,
  Search,
  X,
} from "lucide-react";
import { useEffect, useId, useMemo, useState } from "react";
import { z } from "zod";

import {
  BookingCalendar,
  FullyBookedLegend,
  TimeSlotPicker,
} from "@/components/site/booking-schedule";
import { CopyReferenceButton } from
"@/components/site/copy-reference-button";
import { SiteFooter } from "@/components/site/site-footer";
import { SiteHeader } from "@/components/site/site-header";
import { TermsAgreement } from "@/components/site/terms-agreement";
import { TurnstileChallenge } from "@/components/site/turnstile-challenge";
import {
  AlertDialog,
  AlertDialogAction,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
} from "@/components/ui/alert-dialog";
import { Button } from "@/components/ui/button";
```

```
import { Card, CardContent } from "@/components/ui/card";
import {
  Dialog,
  DialogContent,
  DialogDescription,
  DialogFooter,
  DialogHeader,
  DialogTitle,
} from "@/components/ui/dialog";
import { FieldError } from "@/components/ui/field-error";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@/components/ui/select";
import { Textarea } from "@/components/ui/textarea";
import { supabase } from "@/integrations/supabase/client";
import {
  type Availability,
  computeAvailableDates,
  computeAvailableSlots,
  computeFullyBookedDates,
} from "@/lib/availability";
import {
  BOOKING_DRAFT_LIFETIME_MINUTES,
  clearBookingDraft,
  loadBookingDraft,
  saveBookingDraft,
} from "@/lib/booking-draft";
import { createBooking, getAvailability, getRescheduleDetails } from
"@/lib/booking.functions";
import {
  DEFAULT_BOOKING_TERMS,
  DEFAULT_BOOKING_HOURS,
  PHONE_VALIDATION_MESSAGE,
  MAX_BOOKING_SERVICE_SELECTIONS,
  bookingDurationOverflowMinutes,
  formatDateLong,
  formatPHP,
  formatTime,
  formatNamePartInput,
```

```
  isValidCalendarDate,
  normalizePhilippineMobile,
  sanitizePhilippineMobileInput,
} from "@/lib/shop";
import { cn, randomId } from "@/lib/utils";

const searchSchema = z.object({
  date: z.string().refine(isValidCalendarDate, "Enter a valid appointment
date.").optional(),
  startTime: z
    .string()
    .regex(/^\d{2}:\d{2}$/, "Enter a valid appointment start time.")
    .optional(),
  serviceId: z.string().uuid().optional(),
  reschedule: z
    .string()
    .regex(/^FRM-[A-Z0-9]{6}$/)
    .optional(),
  phone: z
    .string()
    .trim()
    .refine(
      (value) => normalizePhilippineMobile(value) !== null,
      "Enter a valid Philippine mobile number.",
    )
    .transform((value) => normalizePhilippineMobile(value)!)
    .optional(),
});

const turnstileEnabled =
Boolean(import.meta.env["VITE_TURNSTILE_SITE_KEY"]);

function emptyBookingForm() {
  return {
    firstName: "",
    middleName: "",
    lastName: "",
    phone: "",
    email: "",
    motoBrand: "",
    motoModel: "",
    motoVariant: "",
    motoYear: String(new Date().getFullYear()),
    plateNumber: "",
    notes: "",
```

```
    wantsProducts: null as boolean | null,
  };
}

const bookingDraftSchema = z.object({
  form: z.object({
    firstName: z.string(),
    middleName: z.string(),
    lastName: z.string(),
    phone: z.string(),
    email: z.string(),
    motoBrand: z.string(),
    motoModel: z.string(),
    motoVariant: z.string(),
    motoYear: z.string(),
    plateNumber: z.string(),
    notes: z.string(),
    wantsProducts: z.boolean().nullable(),
  }),
  serviceIds: z.array(z.string()),
  date: z.string(),
  startTime: z.string(),
});
type BookingDraft = z.infer<typeof bookingDraftSchema>;

function parseBookingDraft(value: unknown) {
  const parsed = bookingDraftSchema.safeParse(value);
  return parsed.success ? parsed.data : null;
}
const BOOKING_PAGES = ["Booking details", "Review & submit"] as const;

export const Route = createFileRoute("/book")({
  validateSearch: searchSchema,
  head: () => ({
    meta: [
      { title: "Book a Service Appointment | Fake Rider Motorparts" },
      {
        name: "description",
        content:
          "Reserve your motorcycle service slot online. Choose your services and
schedule a convenient time, then save your reference code.",
      },
      { property: "og:title", content: "Book a Service Appointment | Fake Rider" },
      {
        property: "og:description",
```

```
        content: "Reserve your motorcycle service slot online in a few minutes.",
      },
    ],
  }),
  component: BookPage,
});

type Errors = Partial<{
  firstName: string;
  middleName: string;
  lastName: string;
  phone: string;
  email: string;
  motoBrand: string;
  motoModel: string;
  plateNumber: string;
  services: string;
  wantsProducts: string;
  date: string;
  startTime: string;
  schedule: string;
  terms: string;
  rescheduleReason: string;
}>;

const validationFieldOrder: (keyof Errors)[] = [
  "lastName",
  "firstName",
  "middleName",
  "phone",
  "email",
  "motoBrand",
  "motoModel",
  "plateNumber",
  "services",
  "wantsProducts",
  "rescheduleReason",
  "date",
  "startTime",
  "schedule",
  "terms",
];

const validationFocusTargets: Partial<Record<keyof Errors, string>> = {
  firstName: "#booking-first-name",
```

```
  middleName: "#booking-middle-name",
  lastName: "#booking-last-name",
  phone: "#booking-phone",
  email: "#booking-email",
  motoBrand: "#booking-moto-brand",
  motoModel: "#booking-moto-model",
  plateNumber: "#booking-plate-number",
  services: "#booking-services",
  wantsProducts: "#booking-products",
  rescheduleReason: "#reschedule-reason",
  date: "#booking-date button[data-day]:not([disabled])",
  startTime: "#booking-start-time button:not([disabled])",
  schedule: "#booking-schedule",
  terms: "#booking-terms",
};

function isDuplicateBookingMessage(message: string) {
  return /already (?:have|has) an appointment for/i.test(message);
}

function BookPage() {
  const book = useServerFn(createBooking);
  const availabilityFn = useServerFn(getAvailability);
  const rescheduleDetailsFn = useServerFn(getRescheduleDetails);
  const search = useSearch({ from: "/book" });

  const [form, setForm] = useState(emptyBookingForm);
  const [serviceIds, setServiceIds] = useState<string[]>(
    search.serviceId ? [search.serviceId] : [],
  );
  const [serviceCategory, setServiceCategory] = useState("all");
  const [serviceSearch, setServiceSearch] = useState("");
  const [date, setDate] = useState<string>(search.date ?? "");
  const [startTime, setStartTime] = useState<string>(search.startTime ?? "");
  const [terms, setTerms] = useState(false);
  const [bookingPage, setBookingPage] = useState<"details" |
"review">("details");
  const [errors, setErrors] = useState<Errors>({});
  const [focusRequest, setFocusRequest] = useState<keyof Errors | null>(null);
  const [multiDayContinuationOpen, setMultiDayContinuationOpen] =
useState(false);
  const [multiDayContinuationAccepted, setMultiDayContinuationAccepted] =
useState(false);
  const [duplicateBookingMessage, setDuplicateBookingMessage] =
useState<string | null>(null);
```

```
  const [result, setResult] = useState<{
    reference: string;
    total: number;
    date: string;
    startTime: string;
    continuationSegments?: Array<{ date: string; startTime: string;
durationMinutes: number }>;
  } | null>(null);
  const [turnstileToken, setTurnstileToken] = useState("");
  const [bookingRequestId, setBookingRequestId] = useState(() => randomId());
  const isReschedule = Boolean(search.reschedule && search.phone);
  const [rescheduleReason, setRescheduleReason] = useState("");
  const [reschedulePrefillReady, setReschedulePrefillReady] =
useState(!isReschedule);

  const rescheduleDetails = useQuery({
    queryKey: ["reschedule-details", search.reschedule, search.phone],
    queryFn: async () => {
      if (!search.reschedule || !search.phone) {
        throw new Error("Missing reschedule appointment details.");
      }
      return rescheduleDetailsFn({
        data: {
          reference: search.reschedule,
          phone: normalizePhilippineMobile(search.phone)!,
        },
      });
    },
    enabled: isReschedule,
    retry: false,
  });

  const rescheduleError =
    rescheduleDetails.data && !rescheduleDetails.data.ok
      ? rescheduleDetails.data.error
      : rescheduleDetails.isError
        ? "We could not load your original appointment. Please return to the
Appointment Tracker and try again."
        : null;
  const rescheduleReady =
    !isReschedule || (rescheduleDetails.data?.ok === true &&
reschedulePrefillReady);

  const services = useQuery({
    queryKey: ["services"],
```

```
    queryFn: async () => {
      const { data, error } = await supabase
        .from("services")
        .select("*")
        .eq("is_active", true)
        .eq("is_archived", false)
        .order("sort_order");
      if (error) throw error;
      return Array.from(new Map((data ?? []).map((s) => [s.name.trim(),
s])).values());
    },
  });

  const shopSettings = useQuery({
    queryKey: ["public-shop-settings"],
    queryFn: async () => {
      const { data, error } = await supabase
        .from("shop_settings")
        .select("booking_terms,opening_time,closing_time")
        .eq("id", true)
        .maybeSingle();
      if (error) throw error;
      return data;
    },
  });

  useEffect(() => {
    if (!isReschedule || !rescheduleDetails.data?.ok) return;
    const appointment = rescheduleDetails.data.appointment;
    setForm({
      firstName: appointment.firstName,
      middleName: appointment.middleName,
      lastName: appointment.lastName,
      phone: appointment.phone,
      email: appointment.email,
      motoBrand: appointment.motoBrand,
      motoModel: appointment.motoModel,
      motoVariant: appointment.motoVariant,
      motoYear: appointment.motoYear,
      plateNumber: appointment.plateNumber,
      notes: appointment.notes,
      wantsProducts: appointment.wantsProducts,
    });
    setServiceIds(appointment.serviceIds);
    setServiceCategory("all");
```

```
    setDate("");
    setStartTime("");
    setBookingPage("details");
    setReschedulePrefillReady(true);
  }, [isReschedule, rescheduleDetails.data]);

  const motorcycleCatalog = useQuery({
    queryKey: ["motorcycle-catalog"],
    queryFn: async () => {
      const { data, error } = await supabase
        .from("motorcycle_catalog")
        .select("brand,model")
        .eq("is_active", true)
        .eq("is_archived", false)
        .order("brand")
        .order("model");
      if (error) throw error;
      return Array.from(
        new Map((data ?? []).map((m) => [`${m.brand ?? ""}:${m.model.trim()}`,
m])).values(),
      );
    },
  });

  const bookableMotorcycles = useMemo(
    () =>
      (motorcycleCatalog.data ?? []).filter((motorcycle) => {
        return motorcycle.brand && motorcycle.model;
      }),
    [motorcycleCatalog.data],
  );

  const brands = useMemo(() => {
    const set = new Set<string>();
    bookableMotorcycles.forEach((p) => p.brand && set.add(p.brand));
    return Array.from(set).sort();
  }, [bookableMotorcycles]);

  const catalogUnavailable =
    motorcycleCatalog.isError || (!motorcycleCatalog.isLoading && brands.length
=== 0);

  const modelsForBrand = useMemo(() => {
    const seen = new Set<string>();
    const list = bookableMotorcycles.filter((p) => p.brand === form.motoBrand);
```

```
    return list
      .filter((p) => {
        if (seen.has(p.model)) return false;
        seen.add(p.model);
        return true;
      })
      .map((p) => ({
        value: p.model,
        label: p.model.replace(new RegExp(`^${p.brand} `), ""),
      }))
      .sort((a, b) => a.label.localeCompare(b.label, undefined, { sensitivity: "base"
}));
  }, [bookableMotorcycles, form.motoBrand]);

  const motorcycleSelectionReady = Boolean(form.motoBrand &&
form.motoModel);

  const availability = useQuery<Availability>({
    queryKey: [
      "availability",
      serviceIds,
      form.motoBrand,
      form.motoModel,
      isReschedule,
      serviceIds.length > 1,
    ],
    queryFn: () =>
      availabilityFn({
        data: {
          days: 45,
          serviceIds,
          rescheduling: isReschedule,
          allowMultiDayContinuation: !isReschedule && serviceIds.length > 1,
          motorcycle: { brand: form.motoBrand, model: form.motoModel },
          ...(search.reschedule && search.phone
            ? {
                rescheduleReference: search.reschedule,
                reschedulePhone: normalizePhilippineMobile(search.phone)!,
              }
            : {}),
        },
      }),
    enabled: serviceIds.length > 0 && motorcycleSelectionReady,
  });
```

```
  const dates = useMemo(
    () => (availability.data ? computeAvailableDates(availability.data) : []),
    [availability.data],
  );

  const slots = useMemo(
    () => (availability.data && date ? computeAvailableSlots(availability.data, date)
: []),
    [availability.data, date],
  );

  const availableDateSet = useMemo(() => new Set(dates), [dates]);
  const fullyBookedDates = useMemo(
    () => (availability.data ? computeFullyBookedDates(availability.data) : []),
    [availability.data],
  );

  const availabilityEnabled = serviceIds.length > 0 && motorcycleSelectionReady;
  const availabilityLoading =
    availabilityEnabled && (availability.isPending || availability.isFetching);
  const availabilityError = availability.data?.error;

  const selectedServices = (services.data ?? []).filter((s) =>
serviceIds.includes(s.id));
  const serviceCategories = useMemo(
    () =>
      Array.from(
        new Set((services.data ?? []).map((service) =>
service.category).filter(Boolean)),
      ).sort(),
    [services.data],
  );
  const normalizedServiceSearch = serviceSearch.trim().toLowerCase();
  const filteredServices = (services.data ?? []).filter(
    (service) =>
      (serviceCategory === "all" || service.category === serviceCategory) &&
      (!normalizedServiceSearch ||
service.name.toLowerCase().includes(normalizedServiceSearch)),
  );
  const serviceEstimates =
    availability.data?.serviceEstimates ??
    selectedServices.map((service) => ({
      id: service.id,
      name: service.name,
      price: Number(service.price),
```

```
      durationMinutes: service.duration_minutes ?? 60,
      pricingSource: "small_bike" as const,
    }));
  const estimatedServices = serviceIds
    .map((serviceId) => serviceEstimates.find((service) => service.id ===
serviceId))
    .filter((service): service is NonNullable<typeof service> => Boolean(service));
  const total = estimatedServices.reduce((sum, service) => sum + service.price,
0);
  const customerFullName = [form.firstName, form.middleName, form.lastName]
    .map((part) => part.trim())
    .filter(Boolean)
    .join(" ");
  const totalDuration = estimatedServices.reduce(
    (sum, service) => sum + service.durationMinutes,
    0,
  );
  const operatingHours =
    availability.data?.operatingHours ??
    (shopSettings.data
      ? {
          openingTime: shopSettings.data.opening_time
            ? String(shopSettings.data.opening_time).slice(0, 5)
            : DEFAULT_BOOKING_HOURS.openingTime,
          closingTime: shopSettings.data.closing_time
            ? String(shopSettings.data.closing_time).slice(0, 5)
            : DEFAULT_BOOKING_HOURS.closingTime,
        }
      : DEFAULT_BOOKING_HOURS);
  const serviceStepOverflowMinutes =
    !isReschedule && serviceIds.length > 1
      ? bookingDurationOverflowMinutes(operatingHours.openingTime,
totalDuration, operatingHours)
      : 0;
  const needsServiceStepMultiDayConfirmation = serviceStepOverflowMinutes >
0;
  const multiDayOverflowMinutes =
    !isReschedule && date && startTime && serviceIds.length > 1
      ? bookingDurationOverflowMinutes(startTime, totalDuration, operatingHours)
      : 0;
  const needsMultiDayContinuationConfirmation = multiDayOverflowMinutes > 0;
  const bookingTerms = shopSettings.data?.booking_terms ||
DEFAULT_BOOKING_TERMS;
  // Prices depend on the motorcycle's size category, so only show a total once
  // the server has priced the selected services for the chosen motorcycle.
```

```
  const pricingReady = Boolean(availability.data?.serviceEstimates) &&
!availabilityError;
  const selectedModelLabel =
    modelsForBrand.find((model) => model.value === form.motoModel)?.label ??
form.motoModel;
  const sectionComplete = {
    personal:
      isReschedule ||
      (form.lastName.trim().length >= 2 &&
        form.firstName.trim().length >= 2 &&
        normalizePhilippineMobile(form.phone) !== null &&
        /^\S+@\S+\.\S+$/.test(form.email.trim())),
    motorcycle: motorcycleSelectionReady && form.plateNumber.trim().length >=
2,
    services: serviceIds.length > 0 && form.wantsProducts !== null,
    schedule: Boolean(date && startTime) && (!isReschedule ||
Boolean(rescheduleReason.trim())),
  };

  function clearErrors(...fields: (keyof Errors)[]) {
    setErrors((current) => {
      if (!fields.some((field) => current[field])) return current;
      const next = { ...current };
      for (const field of fields) delete next[field];
      return next;
    });
  }

  function updateForm<K extends keyof typeof form>(field: K, value: (typeof
form)[K]) {
    setForm((current) => ({ ...current, [field]: value }));
    if (field in validationFocusTargets) clearErrors(field as keyof Errors);
  }

  // Restore an unfinished booking once the page is open in the browser. A
  // service, date or time in the link (from "Book this service") wins.
  const [draftLoaded, setDraftLoaded] = useState(false);
  const [draftRestored, setDraftRestored] = useState(false);
  useEffect(() => {
    if (draftLoaded) return;
    const draft = isReschedule ? null : loadBookingDraft(parseBookingDraft);
    if (draft) {
      setForm(draft.form);
      if (!search.serviceId) setServiceIds(draft.serviceIds);
      if (!search.date) {
```

```
      setDate(draft.date);
      setStartTime(draft.startTime);
    }
    setDraftRestored(true);
  }
  setDraftLoaded(true);
}, [draftLoaded, isReschedule, search.serviceId, search.date]);

// Keep the draft up to date while the customer fills in the form.
useEffect(() => {
  if (!draftLoaded || isReschedule || result) return;
  const draft: BookingDraft = { form, serviceIds, date, startTime };
  const hasInput =
    Object.entries(form).some(
      ([field, value]) => field !== "motoYear" && value !== "" && value !== null,
    ) ||
    serviceIds.length > 0 ||
    date !== "";
  if (hasInput) saveBookingDraft(draft);
  else clearBookingDraft();
}, [draftLoaded, isReschedule, result, form, serviceIds, date, startTime]);

function startOver() {
  clearBookingDraft();
  setDraftRestored(false);
  setForm(emptyBookingForm());
  setServiceIds([]);
  setServiceCategory("all");
  setServiceSearch("");
  resetSchedule();
  setTerms(false);
  setErrors({});
  setDuplicateBookingMessage(null);
  setBookingPage("details");
}

function resetSchedule() {
  setDate("");
  setStartTime("");
  setMultiDayContinuationAccepted(false);
}

function toggleService(serviceId: string) {
  const checked = serviceIds.includes(serviceId);
  if (!checked && serviceIds.length >=
```

```
MAX_BOOKING_SERVICE_SELECTIONS) {
      setErrors((current) => ({
        ...current,
        services: `You can select up to
${MAX_BOOKING_SERVICE_SELECTIONS} services per appointment.`,
      }));
      return;
    }
    setServiceIds((prev) =>
      checked ? prev.filter((id) => id !== serviceId) : [...prev, serviceId],
    );
    clearErrors("services");
    resetSchedule();
  }

  function clearServices() {
    setServiceIds([]);
    resetSchedule();
  }

  useEffect(() => {
    if (!focusRequest) return;

    const target = validationFocusTargets[focusRequest];
    if (!target) return;

    // Let React show the appropriate booking page before trying to focus its input.
    const timer = window.setTimeout(() => {
      const element = document.querySelector<HTMLElement>(target);
      element?.scrollIntoView({ behavior: "smooth", block: "center" });
      element?.focus({ preventScroll: true });
      setFocusRequest(null);
    }, 0);

    return () => window.clearTimeout(timer);
  }, [bookingPage, focusRequest]);

  useEffect(() => {
    if (!availability.data || !date) return;

    if (!availableDateSet.has(date)) {
      setDate("");
      setStartTime("");
      return;
    }
```

```
  if (startTime) {
    const selectedSlot = computeAvailableSlots(availability.data, date).find(
      (slot) => slot.startTime === startTime,
    );
    if (!selectedSlot || selectedSlot.disabled) setStartTime("");
  }
}, [availability.data, availableDateSet, date, startTime]);

const mutation = useMutation({
  mutationFn: async () => {
    if (form.wantsProducts === null) {
      throw new Error("Choose whether you would like to avail shop products.");
    }
    const submittedSchedule = { date, startTime };
    const res = await book({
      data: {
        firstName: form.firstName.trim(),
        middleName: form.middleName.trim(),
        lastName: form.lastName.trim(),
        phone: normalizePhilippineMobile(form.phone)!,
        email: form.email.trim().toLowerCase(),
        motoBrand: form.motoBrand.trim(),
        motoModel: form.motoModel.trim(),
        motoVariant: form.motoVariant.trim(),
        motoYear: Number(form.motoYear),
        plateNumber: form.plateNumber.trim(),
        serviceIds,
        date,
        startTime,
        notes: form.notes.trim(),
        wantsProducts: form.wantsProducts,
        turnstileToken,
        idempotencyKey: bookingRequestId,
        termsAccepted: true as const,
        multiDayContinuationAccepted,
        ...(isReschedule ? { rescheduleReference: search.reschedule } : {}),
        ...(isReschedule ? { rescheduleReason: rescheduleReason.trim() } : {}),
      },
    });
    return { res, submittedSchedule };
  },
  onSuccess: ({ res, submittedSchedule }) => {
    if (!res.ok) {
      if (!isReschedule && isDuplicateBookingMessage(res.error)) {
```

```
      setDuplicateBookingMessage(res.error);
      setErrors(({ schedule: _, ...current }) => current);
      setDate("");
      setStartTime("");
      setTurnstileToken("");
      setBookingRequestId(randomId());
      availability.refetch();
      return;
    }
    setErrors((current) => ({ ...current, schedule: res.error }));
    setTurnstileToken("");
    setBookingRequestId(randomId());
    availability.refetch();
    return;
  }
  clearBookingDraft();
  setResult(
    "continuationSegments" in res
      ? {
          reference: res.reference,
          total: res.total,
          date: submittedSchedule.date,
          startTime: submittedSchedule.startTime,
          continuationSegments: res.continuationSegments,
        }
      : {
          reference: res.reference,
          total: res.total,
          date: submittedSchedule.date,
          startTime: submittedSchedule.startTime,
        },
  );
  window.scrollTo({ top: 0, behavior: "smooth" });
},
onError: (err: Error) => {
  if (!isReschedule && isDuplicateBookingMessage(err.message)) {
    setDuplicateBookingMessage(err.message);
    setErrors(({ schedule: _, ...current }) => current);
    setDate("");
    setStartTime("");
    return;
  }
  const msg = err.message.toLowerCase();
  let scheduleError: string;
  if (msg.includes("network") || msg.includes("fetch") ||
```

```
msg.includes("connect")) {
        scheduleError = "Network error. Please check your connection and try
again.";
      } else if (msg.includes("timeout")) {
        scheduleError = "The request timed out. Please try again.";
      } else if (msg.includes("validation") || msg.includes("invalid")) {
        scheduleError = "Some fields have invalid values. Please review the form.";
      } else {
        scheduleError =
          "We could not submit your booking. Please review your details and try
again.";
      }
      setErrors((current) => ({ ...current, schedule: scheduleError }));
      availability.refetch();
    },
  });

  function validationErrors(steps: number[]) {
    const e: Errors = {};
    if (isReschedule && !rescheduleReady) {
      e.schedule = rescheduleError ?? "Loading the original appointment details.";
      return e;
    }
    if (steps.includes(1)) {
      if (form.lastName.trim().length < 2) e.lastName = "Enter your last name.";
      if (form.firstName.trim().length < 2) e.firstName = "Enter your first name.";
      if (!normalizePhilippineMobile(form.phone)) e.phone =
PHONE_VALIDATION_MESSAGE;
      if (!isReschedule && !/^\S+@\S+\.\S+$/.test(form.email.trim())) {
        e.email = "Enter a valid email address.";
      }
    }
    if (steps.includes(2)) {
      if (!form.motoBrand.trim()) e.motoBrand = "Required";
      if (!form.motoModel.trim()) e.motoModel = "Required";
      if (!isReschedule && catalogUnavailable) {
        e.motoBrand = "The motorcycle catalog is unavailable. Please try again
shortly.";
      }
      if (
        !isReschedule &&
        !catalogUnavailable &&
        form.motoBrand &&
        !brands.includes(form.motoBrand)
      ) {
```

```
      e.motoBrand = "Select a brand from the motorcycle catalog.";
    }
    if (
      !isReschedule &&
      !catalogUnavailable &&
      form.motoModel &&
      !modelsForBrand.some((model) => model.value === form.motoModel)
    ) {
      e.motoModel = "Select a model from the motorcycle catalog.";
    }
    if (form.plateNumber.trim().length < 2) e.plateNumber = "Required";
  }
  if (steps.includes(3) && serviceIds.length === 0) e.services = "Select at least
one service.";
  if (steps.includes(3) && serviceIds.length >
MAX_BOOKING_SERVICE_SELECTIONS) {
    e.services = `Select up to ${MAX_BOOKING_SERVICE_SELECTIONS}
services per appointment.`;
  }
  if (steps.includes(4) && !date) e.date = "Select your preferred date.";
  if (steps.includes(4) && date && !startTime) e.startTime = "Select your
preferred time.";
  if (steps.includes(3) && form.wantsProducts === null) {
    e.wantsProducts = "Choose Yes or No for shop products.";
  }
  if (steps.includes(4) && availabilityLoading) {
    e.schedule = "Availability is still loading. Please wait a moment.";
  }
  if (steps.includes(4) && (availability.isError || availabilityError)) {
    e.schedule = availabilityError ?? "We could not verify availability. Please try
again.";
  }
  if (steps.includes(4) && isReschedule && !rescheduleReason.trim()) {
    e.rescheduleReason = "Tell us why you need to reschedule.";
  }
  if (steps.includes(5) && !terms) e.terms = "You must accept the terms and
conditions.";
  return e;
}

function validateBooking() {
  const e = validationErrors([1, 2, 3, 4, 5]);
  return showValidationErrors(e);
}
```

```
  function showValidationErrors(e: Errors) {
    setErrors(e);
    const firstInvalidField = validationFieldOrder.find((field) => e[field]);
    if (!firstInvalidField) return true;

    setBookingPage(firstInvalidField === "terms" ? "review" : "details");
    setFocusRequest(firstInvalidField);
    return false;
  }

  function continueToReview() {
    const e = validationErrors([1, 2, 3, 4]);
    if (!showValidationErrors(e)) return;
    const requiresMultiDayConfirmation =
      needsServiceStepMultiDayConfirmation ||
needsMultiDayContinuationConfirmation;
    if (requiresMultiDayConfirmation && !multiDayContinuationAccepted) {
      setMultiDayContinuationOpen(true);
      return;
    }
    setBookingPage("review");
    window.scrollTo({ top: 0, behavior: "smooth" });
  }

  function continueWithMultiDayBooking() {
    setMultiDayContinuationAccepted(true);
    setMultiDayContinuationOpen(false);
    setBookingPage("review");
    window.scrollTo({ top: 0, behavior: "smooth" });
  }

  function returnToBookingDetails() {
    setErrors({});
    setBookingPage("details");
    window.scrollTo({ top: 0, behavior: "smooth" });
  }

  function submitBooking() {
    if (!rescheduleReady || !validateBooking()) return;
    mutation.mutate();
  }

  /** Return from the review page to a specific part of the booking form. */
  function editBookingSection(field: keyof Errors) {
    setErrors({});
```

```
  setBookingPage("details");
  setFocusRequest(field);
 }

 function dismissDuplicateBookingDialog() {
  setDuplicateBookingMessage(null);
 }

 function chooseAnotherDate() {
  dismissDuplicateBookingDialog();
  editBookingSection("schedule");
 }

 if (result) {
  const nextSteps = isReschedule
   ? [
      "The shop reviews your requested new date and time.",
      "Your original appointment stays reserved until the request is reviewed.",
      "Check the status of your request anytime on the My Appointment page.",
     ]
   : [
      "The shop reviews your booking request.",
      "Once it is approved, we will email your confirmed appointment details.",
      "Use your reference code on the My Appointment page to check the
status, reschedule, or cancel.",
      "On the day, please arrive 10–15 minutes early for check-in.",
     ];

  return (
   <div className="min-h-screen">
    <SiteHeader />
    <main className="mx-auto w-full max-w-2xl px-4 py-10 sm:py-16">
     <Card className="overflow-hidden border-primary/40 bg-card/70">
      <CardContent className="p-0">
       <div className="px-5 pt-8 pb-6 text-center sm:px-8">
        <CheckCircle2 className="mx-auto h-14 w-14 text-primary"
aria-hidden="true" />
        <h1 className="mt-4 font-display text-3xl font-bold uppercase
sm:text-4xl">
         {isReschedule ? "Reschedule request sent" : "Appointment
requested"}
        </h1>
        <p className="mx-auto mt-2 max-w-md text-sm
text-muted-foreground">
         {isReschedule
```

```
              ? "Your original appointment remains reserved while the shop
reviews your requested new date and time."
              : "Save your reference code. Your request remains pending until the
shop approves it, then we will email your confirmed appointment details."}
            </p>
            <div className="mt-6 rounded-xl border border-dashed
border-primary/50 bg-primary/5 p-5">
              <p className="text-xs tracking-[0.3em] text-muted-foreground
uppercase">
                Reference code
              </p>
              <p className="mt-1 font-display text-4xl font-bold tracking-widest
text-primary">
                {result.reference}
              </p>
              <CopyReferenceButton value={result.reference} className="mt-3" />
            </div>
          </div>

          <dl className="grid gap-px border-y border-border/70 bg-border/70
sm:grid-cols-3">
            <SuccessDetail label="Schedule">
              {formatDateLong(result.date)}
              <span className="block text-muted-foreground">
                {formatTime(result.startTime)}
              </span>
            </SuccessDetail>
            <SuccessDetail label="Estimated
total">{formatPHP(result.total)}</SuccessDetail>
            <SuccessDetail label="Status">Pending shop
confirmation</SuccessDetail>
          </dl>

          <div className="space-y-6 px-5 py-6 sm:px-8">
            {(result.continuationSegments?.length ?? 0) > 0 && (
              <div className="rounded-lg border border-primary/30 bg-primary/5
p-4 text-sm">
                <p className="font-medium text-foreground">Continued service
schedule</p>
                <ul className="mt-2 space-y-1 text-muted-foreground">
                  {result.continuationSegments?.map((segment) => (
                    <li key={`${segment.date}-${segment.startTime}`}>
                      {formatDateLong(segment.date)} at
{formatTime(segment.startTime)} (
                      {segment.durationMinutes} min)
```

```
                  </li>
                ))}
              </ul>
            </div>
          )}

          <div>
            <h2 className="font-display text-lg tracking-wide uppercase">
              What happens next
            </h2>
            <ol className="mt-3 space-y-3 text-sm">
              {nextSteps.map((step, index) => (
                <li key={step} className="flex gap-3">
                  <span
                    aria-hidden="true"
                    className="flex size-6 shrink-0 items-center justify-center
rounded-full bg-primary/15 font-display text-sm text-primary"
                  >
                    {index + 1}
                  </span>
                  <span className="pt-0.5 text-muted-foreground">{step}</span>
                </li>
              ))}
            </ol>
          </div>

          <div className="flex flex-col gap-3 sm:flex-row sm:justify-center">
            <Button asChild size="lg" className="font-display tracking-wide
uppercase">
              <Link to="/my-appointment">Track my appointment</Link>
            </Button>
            <Button
              asChild
              size="lg"
              variant="outline"
              className="font-display tracking-wide uppercase"
            >
              <Link to="/">Back to homepage</Link>
            </Button>
          </div>
        </div>
      </CardContent>
    </Card>
  </main>
  <SiteFooter />
```

```
    </div>
  );
 }

 if (isReschedule && (rescheduleDetails.isLoading || rescheduleError ||
!reschedulePrefillReady)) {
  return (
   <div className="min-h-screen">
    <SiteHeader />
    <main className="mx-auto w-full max-w-2xl px-4 py-16">
     <Card className="border-border/70 bg-card/60">
      <CardContent className="p-8 text-center">
       {rescheduleDetails.isLoading ? (
        <>
         <Loader2 className="mx-auto size-8 animate-spin text-primary" />
         <h1 className="mt-4 font-display text-2xl uppercase">Loading
appointment</h1>
         <p className="mt-2 text-sm text-muted-foreground">
          Preparing your saved details and selected services for rescheduling.
         </p>
        </>
       ) : (
        <>
         <CircleAlert className="mx-auto size-8 text-destructive"
aria-hidden="true" />
         <h1 className="mt-4 font-display text-2xl uppercase">
          Unable to reschedule online
         </h1>
         <p className="mt-2 text-sm text-destructive">{rescheduleError}</p>
         <Button asChild variant="outline" className="mt-6">
          <Link to="/my-appointment">
           <ArrowLeft /> Back to My Appointment
          </Link>
         </Button>
        </>
       )}
      </CardContent>
     </Card>
    </main>
    <SiteFooter />
   </div>
  );
 }

 const productChoiceLabel =
```

```
    form.wantsProducts === null
      ? "Not selected"
      : form.wantsProducts
        ? "Yes, if needed"
        : "No, service only";
  const showMultiDayNote =
    !isReschedule &&
    (needsServiceStepMultiDayConfirmation ||
needsMultiDayContinuationConfirmation);

  return (
    <div className="min-h-screen">
      <SiteHeader />
      <main className="mx-auto w-full max-w-6xl px-4 py-8 sm:px-6 sm:py-12">
        <div className="flex flex-col gap-6 md:flex-row md:items-end
md:justify-between">
          <div className="max-w-2xl">
            <p className="text-xs tracking-[0.3em] text-accent uppercase">
              {isReschedule ? "Reschedule appointment" : "Book a service"}
            </p>
            <h1 className="font-display text-3xl font-bold uppercase sm:text-4xl
md:text-5xl">
              {isReschedule ? "Choose a new service slot" : "Reserve your service
slot"}
            </h1>
            <p className="mt-2 text-muted-foreground">
              {isReschedule
                ? "Your saved appointment details are retained. Enter a reason and
choose a new available date and time; your original slot stays reserved until the
shop confirms."
                : "Tell us about you and your motorcycle, pick your services, and
choose an available time. You'll review everything before submitting."}
            </p>
          </div>
          <BookingSteps
            current={bookingPage === "details" ? 0 : 1}
            onSelectDetails={returnToBookingDetails}
          />
        </div>

        {draftRestored && (
          <div
            role="status"
            className="mt-6 flex flex-col gap-3 rounded-lg border border-primary/40
bg-primary/5 p-3 text-sm sm:flex-row sm:items-center sm:justify-between"
```

```
        >
          <p className="flex items-start gap-2">
            <History className="mt-0.5 size-4 shrink-0 text-primary"
aria-hidden="true" />
            <span>
              We kept the details you entered earlier. An unfinished booking is
cleared after{" "}
              {BOOKING_DRAFT_LIFETIME_MINUTES} minutes without changes.
            </span>
          </p>
          <Button type="button" variant="outline" size="sm" onClick={startOver}>
            Start over
          </Button>
        </div>
      )}

      <form
        className="mt-8"
        noValidate
        onSubmit={(e) => {
          e.preventDefault();
          if (bookingPage === "review") submitBooking();
          else continueToReview();
        }}
      >
        <div
          className={cn(
            "grid gap-6 lg:grid-cols-[minmax(0,1fr)_22rem] lg:items-start xl:gap-8",
            bookingPage !== "details" && "hidden",
          )}
        >
          <div className="min-w-0 space-y-6">
            <Section
              step={1}
              title="Personal information"
              description={
                isReschedule
                  ? "These details come from your original appointment and are
verified when you submit."
                  : "We use these details to confirm your booking and contact you
about it."
              }
              complete={sectionComplete.personal}
            >
              <div className="grid gap-4 sm:grid-cols-2 lg:grid-cols-3">
```

```
              <Field id="booking-last-name" label="Last name"
error={errors.lastName}>
                <Input
                  id="booking-last-name"
                  value={form.lastName}
                  maxLength={40}
                  autoComplete="family-name"
                  disabled={isReschedule}
                  aria-invalid={Boolean(errors.lastName)}
                  onChange={(e) => updateForm("lastName",
formatNamePartInput(e.target.value))}
                  placeholder="Dela Cruz"
                />
              </Field>
              <Field id="booking-first-name" label="First name"
error={errors.firstName}>
                <Input
                  id="booking-first-name"
                  value={form.firstName}
                  maxLength={40}
                  autoComplete="given-name"
                  disabled={isReschedule}
                  aria-invalid={Boolean(errors.firstName)}
                  onChange={(e) => updateForm("firstName",
formatNamePartInput(e.target.value))}
                  placeholder="Juan"
                />
              </Field>
              <Field
                id="booking-middle-name"
                label="Middle name (optional)"
                error={errors.middleName}
              >
                <Input
                  id="booking-middle-name"
                  value={form.middleName}
                  maxLength={40}
                  autoComplete="additional-name"
                  disabled={isReschedule}
                  aria-invalid={Boolean(errors.middleName)}
                  onChange={(e) =>
                    updateForm("middleName", formatNamePartInput(e.target.value))
                  }
                  placeholder="Santos"
                />
```

```
              </Field>
              <Field
                id="booking-phone"
                label="Mobile number"
                hint="11 digits, starting with 09."
                error={errors.phone}
              >
                <Input
                  id="booking-phone"
                  type="tel"
                  value={form.phone}
                  maxLength={11}
                  inputMode="tel"
                  autoComplete="tel"
                  disabled={isReschedule}
                  aria-invalid={Boolean(errors.phone)}
                  aria-describedby="booking-phone-hint"
                  onChange={(e) =>
                    updateForm("phone",
sanitizePhilippineMobileInput(e.target.value))
                  }
                  placeholder="09171234567"
                />
              </Field>
              <Field
                id="booking-email"
                label="Email address"
                hint="We'll email your confirmed appointment details here."
                error={errors.email}
              >
                <Input
                  id="booking-email"
                  type="email"
                  value={form.email}
                  maxLength={254}
                  autoComplete="email"
                  disabled={isReschedule}
                  aria-invalid={Boolean(errors.email)}
                  aria-describedby="booking-email-hint"
                  onChange={(e) => updateForm("email",
e.target.value.toLowerCase())}
                  placeholder="juan@example.com"
                />
              </Field>
            </div>
```

```
          </Section>

          <Section
            step={2}
            title="Motorcycle details"
            description="Choose your motorcycle from our catalog. Service rates
and available times depend on your model."
            complete={sectionComplete.motorcycle}
          >
            {catalogUnavailable && (
              <p className="mb-4 flex items-start gap-2 rounded-lg border
border-destructive/40 bg-destructive/5 p-3 text-sm text-destructive">
                <CircleAlert className="mt-0.5 size-4 shrink-0" aria-hidden="true"
/>
                The motorcycle catalog is unavailable. Please try again shortly.
              </p>
            )}
            <div className="grid gap-4 sm:grid-cols-2 lg:grid-cols-3">
              <Field id="booking-moto-brand" label="Brand"
error={errors.motoBrand}>
                <Select
                  value={form.motoBrand}
                  onValueChange={(v) => {
                    setForm((current) => ({ ...current, motoBrand: v, motoModel: "" }));
                    clearErrors("motoBrand", "motoModel");
                    resetSchedule();
                  }}
                  disabled={isReschedule || motorcycleCatalog.isLoading ||
catalogUnavailable}
                >
                  <SelectTrigger id="booking-moto-brand">
                    <SelectValue
                      placeholder={
                        motorcycleCatalog.isLoading ? "Loading brands…" : "Select a
brand"
                      }
                    />
                  </SelectTrigger>
                  <SelectContent>
                    {brands.map((b) => (
                      <SelectItem key={b} value={b}>
                        {b}
                      </SelectItem>
                    ))}
                  </SelectContent>
```

```
              </Select>
            </Field>
            <Field id="booking-moto-model" label="Model"
error={errors.motoModel}>
              <Select
                value={form.motoModel}
                onValueChange={(v) => {
                  updateForm("motoModel", v);
                  resetSchedule();
                }}
                disabled={
                  isReschedule ||
                  catalogUnavailable ||
                  !form.motoBrand ||
                  modelsForBrand.length === 0
                }
              >
                <SelectTrigger id="booking-moto-model">
                  <SelectValue
                    placeholder={form.motoBrand ? "Select a model" : "Select a
brand first"}
                  />
                </SelectTrigger>
                <SelectContent>
                  {modelsForBrand.map((m) => (
                    <SelectItem key={m.value} value={m.value}>
                      {m.label}
                    </SelectItem>
                  ))}
                </SelectContent>
              </Select>
            </Field>
            <Field id="booking-plate-number" label="Plate number"
error={errors.plateNumber}>
              <Input
                id="booking-plate-number"
                value={form.plateNumber}
                maxLength={20}
                disabled={isReschedule}
                aria-invalid={Boolean(errors.plateNumber)}
                onChange={(e) => updateForm("plateNumber",
e.target.value.toUpperCase())}
                placeholder="ABC 1234"
              />
            </Field>
```

```
            </div>
          </Section>

          <Section
            step={3}
            title="Services"
            description={
              isReschedule
                ? "Your original services are kept for this reschedule request."
                : `Select the services you need. You can choose up to
${MAX_BOOKING_SERVICE_SELECTIONS} per appointment.`
            }
            complete={sectionComplete.services}
          >
            {!isReschedule && (
              <div className="flex flex-col gap-3 sm:flex-row">
                <div className="relative min-w-0 flex-1">
                  <Search
                    className="pointer-events-none absolute top-1/2 left-3 size-4
-translate-y-1/2 text-muted-foreground"
                    aria-hidden="true"
                  />
                  <Input
                    id="booking-service-search"
                    type="search"
                    value={serviceSearch}
                    onChange={(e) => setServiceSearch(e.target.value)}
                    // Enter should filter, not submit the whole booking form.
                    onKeyDown={(e) => {
                      if (e.key === "Enter") e.preventDefault();
                    }}
                    placeholder="Search services"
                    aria-label="Search services"
                    className="pl-9"
                  />
                </div>
                <Select value={serviceCategory}
onValueChange={setServiceCategory}>
                  <SelectTrigger
                    id="booking-service-category"
                    aria-label="Service category"
                    className="w-full sm:w-52"
                  >
                    <SelectValue placeholder="All categories" />
                  </SelectTrigger>
```

```
                <SelectContent position="item-aligned" className="max-h-64">
                  <SelectItem value="all">All categories</SelectItem>
                  {serviceCategories.map((category) => (
                    <SelectItem key={category} value={category}
className="capitalize">
                      {category.replace(/[-_]/g, " ")}
                    </SelectItem>
                  ))}
                </SelectContent>
              </Select>
            </div>
          )}

          <div id="booking-services" tabIndex={-1}
className="focus:outline-none">
            {selectedServices.length > 0 && (
              <div
                className={cn("flex flex-wrap items-center gap-2", !isReschedule
&& "mt-4")}
              >
                <span className="mr-1 text-xs font-medium tracking-widest
text-muted-foreground uppercase">
                  Selected ({selectedServices.length})
                </span>
                {selectedServices.map((service) =>
                  isReschedule ? (
                    <span
                      key={service.id}
                      className="rounded-full border border-primary/40
bg-primary/10 px-3 py-1 text-xs font-medium"
                    >
                      {service.name}
                    </span>
                  ) : (
                    <button
                      type="button"
                      key={service.id}
                      onClick={() => toggleService(service.id)}
                      aria-label={`Remove ${service.name}`}
                      className="inline-flex items-center gap-1 rounded-full border
border-primary/40 bg-primary/10 py-1 pr-2 pl-3 text-xs font-medium
transition-colors hover:bg-primary/20 focus-visible:ring-2 focus-visible:ring-ring
focus-visible:outline-none"
                    >
                      {service.name}
```

```
                      <X className="size-3.5" aria-hidden="true" />
                    </button>
                  ),
                )}
                {!isReschedule && selectedServices.length > 1 && (
                  <Button
                    type="button"
                    variant="link"
                    size="sm"
                    onClick={clearServices}
                    className="h-auto px-1 text-xs md:h-auto"
                  >
                    Clear all
                  </Button>
                )}
              </div>
            )}

            {!isReschedule && (
              <div
                className={cn(
                  "mt-4 max-h-[26rem] overflow-y-auto rounded-lg border
border-border/70 bg-background/40 p-2 sm:max-h-[30rem]",
                  errors.services && "border-destructive",
                )}
              >
                {services.isLoading ? (
                  <p className="flex items-center gap-2 p-3 text-sm
text-muted-foreground">
                    <Loader2 className="size-4 animate-spin" aria-hidden="true"
/> Loading
                    services…
                  </p>
                ) : filteredServices.length === 0 ? (
                  <div className="p-4 text-center text-sm text-muted-foreground">
                    <p>
                      {normalizedServiceSearch
                        ? `No services match “${serviceSearch.trim()}”.`
                        : "No services are available in this category."}
                    </p>
                    {(normalizedServiceSearch || serviceCategory !== "all") && (
                      <Button
                        type="button"
                        variant="link"
                        size="sm"
```

```
                    onClick={() => {
                      setServiceSearch("");
                      setServiceCategory("all");
                    }}
                  >
                    Show all services
                  </Button>
                )}
              </div>
            ) : (
              <div className="grid gap-2 sm:grid-cols-2 xl:grid-cols-3">
                {filteredServices.map((s) => {
                  const checked = serviceIds.includes(s.id);
                  return (
                    <button
                      type="button"
                      key={s.id}
                      aria-pressed={checked}
                      onClick={() => toggleService(s.id)}
                      className={cn(
                        "flex min-h-12 items-center gap-3 rounded-lg border px-3
py-2.5 text-left transition-colors focus-visible:ring-2 focus-visible:ring-ring
focus-visible:outline-none",
                        checked
                          ? "border-primary bg-primary/10"
                          : "border-border/70 bg-card/60 hover:border-primary/50",
                      )}
                    >
                      <span
                        aria-hidden="true"
                        className={cn(
                          "flex size-5 shrink-0 items-center justify-center rounded
border transition-colors",
                          checked
                            ? "border-primary bg-primary text-primary-foreground"
                            : "border-input bg-background",
                        )}
                      >
                        {checked && <Check className="size-3.5" />}
                      </span>
                      <span className="font-display text-sm leading-tight
tracking-wide uppercase">
                        {s.name}
                      </span>
                    </button>
```

```
                    );
                  })}
                </div>
              )}
            </div>
          )}
        </div>
        <FieldError message={errors.services} className="mt-2" />
        {!isReschedule && (
          <p className="mt-3 text-sm text-muted-foreground">
            Not sure which service you need?{" "}
            <Link
              to="/services"
              className="font-medium text-primary underline underline-offset-4"
            >
              Browse our services
            </Link>{" "}
            to see what each one includes.
          </p>
        )}

        <div className="mt-6 border-t border-border/70 pt-5">
          <p id="booking-products-label" className="text-sm font-medium">
            Shop products
          </p>
          <p className="mt-1 text-sm text-muted-foreground">
            Would you like to use shop-provided products for your service?
          </p>
          <div
            id="booking-products"
            tabIndex={-1}
            className="mt-3 grid gap-2 focus:outline-none sm:max-w-md
sm:grid-cols-2"
            role="radiogroup"
            aria-labelledby="booking-products-label"
            aria-invalid={Boolean(errors.wantsProducts)}
          >
            {[
              { value: true, label: "Yes," },
              { value: false, label: "No, service only" },
            ].map((option) => {
              const selected = form.wantsProducts === option.value;
              return (
                <button
                  type="button"
```

```
                    key={option.label}
                    role="radio"
                    aria-checked={selected}
                    disabled={isReschedule}
                    onClick={() => updateForm("wantsProducts", option.value)}
                    className={cn(
                      "flex items-center gap-3 rounded-lg border px-4 py-3 text-left
text-sm font-medium transition-colors focus-visible:ring-2 focus-visible:ring-ring
focus-visible:outline-none",
                      selected
                        ? "border-primary bg-primary/10 text-foreground"
                        : "border-border bg-card/50 hover:border-primary/50",
                      errors.wantsProducts && !selected && "border-destructive/60",
                      isReschedule && "cursor-not-allowed opacity-75",
                    )}
                  >
                    <span
                      aria-hidden="true"
                      className={cn(
                        "flex size-4 shrink-0 items-center justify-center rounded-full
border",
                        selected ? "border-primary" : "border-input",
                      )}
                    >
                      {selected && <span className="size-2 rounded-full
bg-primary" />}
                    </span>
                    {option.label}
                  </button>
                );
              })}
            </div>
            <FieldError message={errors.wantsProducts} className="mt-2" />
          </div>
        </Section>

        <Section
          id="booking-schedule"
          step={4}
          title="Date and time"
          description="Pick an available date, then choose a start time."
          complete={sectionComplete.schedule}
          error={errors.schedule}
        >
          {isReschedule && (
```

```
              <div className="mb-5">
                <Label htmlFor="reschedule-reason">Reason for
rescheduling</Label>
                <Textarea
                  id="reschedule-reason"
                  value={rescheduleReason}
                  maxLength={500}
                  onChange={(event) => {
                    setRescheduleReason(event.target.value);
                    clearErrors("rescheduleReason");
                  }}
                  placeholder="Tell the shop why you need a different schedule."
                  aria-invalid={Boolean(errors.rescheduleReason)}
                  className="mt-2"
                />
                <FieldError message={errors.rescheduleReason} className="mt-1"
/>
              </div>
            )}
            {serviceIds.length === 0 ? (
              <ScheduleHint>
                Select at least one service to see available dates and times.
              </ScheduleHint>
            ) : !motorcycleSelectionReady ? (
              <ScheduleHint>
                Select your motorcycle brand and model to see available dates and
times.
              </ScheduleHint>
            ) : availabilityLoading ? (
              <ScheduleHint loading>Checking available
schedules…</ScheduleHint>
            ) : availability.isError || availabilityError ? (
              <p className="flex items-start gap-2 rounded-lg border
border-destructive/40 bg-destructive/5 p-3 text-sm text-destructive">
                <CircleAlert className="mt-0.5 size-4 shrink-0" aria-hidden="true"
/>
                {availabilityError ??
                  "We could not load availability. Please refresh and try again."}
              </p>
            ) : (
              <>
                <p className="mb-4 text-sm text-muted-foreground">
                  Available dates have an assigned mechanic, an open shop
schedule, and enough
                  time for your selected services.
```

```
              </p>
              <div
                className={cn(
                  "w-full rounded-xl border border-border bg-card/50 p-3 sm:p-5",
                  (errors.date || errors.startTime) && "border-destructive",
                )}
              >
                <div className="grid gap-6 md:grid-cols-[auto_minmax(0,1fr)]
md:gap-8">
                  <div className="border-b border-border/70 pb-5 md:border-r
md:border-b-0 md:pr-8 md:pb-0">
                    <div
                      id="booking-date"
                      tabIndex={-1}
                      className="flex justify-center focus:outline-none
md:justify-start"
                    >
                      <BookingCalendar
                        selected={date}
                        availableDates={availableDateSet}
                        fullyBookedDates={fullyBookedDates}
                        onSelect={(iso) => {
                          setDate(iso);
                          setStartTime("");
                          clearErrors("date", "startTime", "schedule");
                        }}
                      />
                    </div>
                    {fullyBookedDates.length > 0 && (
                      <FullyBookedLegend className="mt-3 justify-center
md:justify-start" />
                    )}
                  </div>

                  <div
                    id="booking-start-time"
                    tabIndex={-1}
                    className="min-w-0 focus:outline-none"
                  >
                    <h3 className="font-display text-lg uppercase">Available
times</h3>
                    <p className="mt-1 mb-4 text-sm text-muted-foreground">
                      {date
                        ? formatDateLong(date)
                        : "Choose a date to see the available start times."}
```

```
                    </p>
                    {date ? (
                      <TimeSlotPicker
                        slots={slots}
                        value={startTime}
                        onChange={(value) => {
                          setStartTime(value);
                          clearErrors("startTime", "schedule");
                        }}
                        invalid={Boolean(errors.startTime)}
                        gridClassName="sm:grid-cols-3 md:grid-cols-2 xl:grid-cols-3"
                      />
                    ) : (
                      <div className="rounded-lg border border-dashed
border-border p-4 text-sm text-muted-foreground">
                        Time slots will appear here after you choose a date.
                      </div>
                    )}
                  </div>
                </div>
                <FieldError message={errors.date} className="mt-2" />
                <FieldError message={errors.startTime} className="mt-2" />
                {date && startTime && (
                  <div className="mt-4 flex items-center gap-3 rounded-lg border
border-primary/30 bg-primary/5 p-3 text-sm">
                    <CalendarDays className="size-5 shrink-0 text-primary"
aria-hidden="true" />
                    <div>
                      <p className="text-xs tracking-widest text-muted-foreground
uppercase">
                        Selected schedule
                      </p>
                      <p className="font-medium text-foreground">
                        {formatDateLong(date)} at {formatTime(startTime)}
                      </p>
                    </div>
                  </div>
                )}
                {showMultiDayNote && (
                  <p className="mt-3 flex items-start gap-2 text-xs
text-muted-foreground">
                    <Info
                      className="mt-0.5 size-3.5 shrink-0 text-primary"
                      aria-hidden="true"
```

```
                  />
                  Your selected services may need more than one day. The
remaining service
                  time will be reserved on the next available shop day.
                 </p>
                )}
               </>
              )}
             </Section>
            </div>

            <aside aria-label="Booking summary" className="lg:sticky lg:top-28">
             <div className="rounded-xl border border-border/70 bg-card/60 p-4
shadow-sm sm:p-5">
              <h2 className="font-display text-xl tracking-wide
uppercase">Booking summary</h2>
              <div className="mt-4 space-y-4 text-sm">
               <SummaryRow label="Your details"
done={sectionComplete.personal}>
                {customerFullName || (
                 <span className="text-muted-foreground">Not entered
yet</span>
                )}
               </SummaryRow>
               <SummaryRow label="Motorcycle"
done={sectionComplete.motorcycle}>
                {motorcycleSelectionReady ? (
                 `${form.motoBrand} ${selectedModelLabel}`
                ) : (
                 <span className="text-muted-foreground">Not selected
yet</span>
                )}
               </SummaryRow>
               <SummaryRow label="Services" done={sectionComplete.services}>
                {estimatedServices.length > 0 ? (
                 <ul className="max-h-40 space-y-1 overflow-y-auto pr-1">
                  {estimatedServices.map((service) => (
                   <li key={service.id} className="flex justify-between gap-3">
                    <span className="min-w-0">{service.name}</span>
                    {pricingReady && (
                     <span className="shrink-0 text-muted-foreground">
                      {formatPHP(service.price)}
                     </span>
                    )}
                   </li>
```

```
                  ))}
                </ul>
              ) : (
                <span className="text-muted-foreground">None selected
yet</span>
              )}
            </SummaryRow>
            <SummaryRow label="Date and time" done={Boolean(date &&
startTime)}>
              {date && startTime ? (
                `${formatDateLong(date)} at ${formatTime(startTime)}`
              ) : (
                <span className="text-muted-foreground">Not selected
yet</span>
              )}
            </SummaryRow>
          </div>

          <div className="mt-5 border-t border-border/70 pt-4">
            <div className="flex items-baseline justify-between gap-3">
              <span className="text-sm font-medium">Estimated total</span>
              <span className="font-display text-2xl text-primary">
                {pricingReady ? formatPHP(total) : "—"}
              </span>
            </div>
            <p className="mt-1 text-xs text-muted-foreground">
              {pricingReady
                ? "Labor/service charge only. Parts are quoted separately."
                : "Choose your motorcycle and services to see an estimate."}
            </p>
          </div>

          <Button
            type="submit"
            size="lg"
            disabled={!rescheduleReady}
            className="mt-5 w-full font-display tracking-wide uppercase"
          >
            {isReschedule ? "Review reschedule request" : "Review booking"}
            <ArrowRight aria-hidden="true" />
          </Button>
          <p className="mt-3 text-center text-xs text-muted-foreground">
            Nothing is submitted until you confirm on the next step.
          </p>
        </div>
```

```
          </aside>
        </div>

        <div
          className={cn(
            "grid gap-6 lg:grid-cols-[minmax(0,1fr)_22rem] lg:items-start xl:gap-8",
            bookingPage !== "review" && "hidden",
          )}
        >
          <div className="min-w-0 space-y-5">
            <div>
              <h2 className="font-display text-2xl tracking-wide uppercase
sm:text-3xl">
                {isReschedule ? "Review your reschedule request" : "Review your
booking"}
              </h2>
              <p className="mt-1 text-sm text-muted-foreground">
                Check your details below. You can edit any section before submitting.
              </p>
            </div>

            {errors.schedule && (
              <p
                role="alert"
                className="flex items-start gap-2 rounded-lg border
border-destructive/40 bg-destructive/5 p-3 text-sm text-destructive"
              >
                <CircleAlert className="mt-0.5 size-4 shrink-0" aria-hidden="true" />
                {errors.schedule}
              </p>
            )}

            <div className="flex flex-col gap-4 rounded-xl border
border-primary/40 bg-primary/5 p-4 sm:flex-row sm:items-center
sm:justify-between sm:p-5">
              <div className="flex items-start gap-3">
                <span className="flex size-11 shrink-0 items-center justify-center
rounded-lg bg-primary text-primary-foreground">
                  <CalendarDays className="size-5" aria-hidden="true" />
                </span>
                <div>
                  <p className="text-xs tracking-widest text-muted-foreground
uppercase">
                    {isReschedule ? "Requested new schedule" : "Appointment"}
                  </p>
```

```
                <p className="font-display text-xl uppercase sm:text-2xl">
                  {date ? formatDateLong(date) : "Date not selected"}
                </p>
                <p className="flex items-center gap-1.5 text-sm
text-muted-foreground">
                  <Clock className="size-4" aria-hidden="true" />
                  {startTime ? formatTime(startTime) : "Time not selected"}
                </p>
              </div>
            </div>
            <Button
              type="button"
              variant="outline"
              size="sm"
              onClick={() => editBookingSection("schedule")}
              className="self-start sm:self-center"
            >
              <Pencil aria-hidden="true" /> Change date and time
            </Button>
          </div>
          {multiDayContinuationAccepted &&
needsMultiDayContinuationConfirmation && (
            <p className="-mt-2 flex items-start gap-2 text-xs text-primary">
              <Info className="mt-0.5 size-3.5 shrink-0" aria-hidden="true" />
              The remaining service time will be reserved on the next available
shop day.
            </p>
          )}

          <div className="grid gap-5 md:grid-cols-2">
            <ReviewGroup
              title="Personal information"
              {...(isReschedule
                ? {}
                : {
                    onEdit: () => editBookingSection("lastName"),
                    editLabel: "Edit personal information",
                  })}
            >
              <dl className="space-y-3">
                <ReviewItem label="Name" value={customerFullName} />
                <ReviewItem label="Mobile number" value={form.phone} />
                {form.email && <ReviewItem label="Email address"
value={form.email} />}
              </dl>
```

```
            </ReviewGroup>
            <ReviewGroup
              title="Motorcycle"
              {...(isReschedule
                ? {}
                : {
                    onEdit: () => editBookingSection("motoBrand"),
                    editLabel: "Edit motorcycle details",
                  })}
            >
              <dl className="space-y-3">
                <ReviewItem
                  label="Brand and model"
                  value={[form.motoBrand, selectedModelLabel].filter(Boolean).join("
")}
                />
                <ReviewItem label="Plate number" value={form.plateNumber} />
              </dl>
            </ReviewGroup>
          </div>

          <ReviewGroup
            title="Services"
            {...(isReschedule
              ? {}
              : { onEdit: () => editBookingSection("services"), editLabel: "Edit
services" })}
          >
            <ul className="divide-y divide-border/70">
              {estimatedServices.map((service) => (
                <li
                  key={service.id}
                  className="flex flex-wrap items-center justify-between gap-x-4
gap-y-1 py-3 first:pt-0"
                >
                  <div>
                    <p className="font-medium
text-foreground">{service.name}</p>
                    <p className="text-xs text-muted-foreground">
                      {service.durationMinutes} min ·{" "}
                      {servicePricingLabel(service.pricingSource)}
                    </p>
                  </div>
                  <span className="font-medium
text-primary">{formatPHP(service.price)}</span>
```

```
                </li>
              ))}
            </ul>
            <div className="flex items-baseline justify-between gap-3 border-t
border-border/70 pt-3">
              <span className="font-medium">Estimated total</span>
              <span className="font-display text-2xl
text-primary">{formatPHP(total)}</span>
            </div>
            <dl className="mt-4 border-t border-border/70 pt-3">
              <ReviewItem label="Shop products" value={productChoiceLabel} />
            </dl>
          </ReviewGroup>

          {isReschedule && (
            <ReviewGroup
              title="Reason for rescheduling"
              onEdit={() => editBookingSection("rescheduleReason")}
              editLabel="Edit reason for rescheduling"
            >
              <p className="text-sm break-words
whitespace-pre-line">{rescheduleReason}</p>
            </ReviewGroup>
          )}

          <div className="rounded-xl border border-border/70 bg-muted/30 p-4
text-sm sm:p-5">
            <p className="flex items-center gap-2 font-medium text-foreground">
              <Info className="size-4 text-primary" aria-hidden="true" /> Before
you submit
            </p>
            <ul className="mt-2 list-disc space-y-1.5 pl-6 leading-5
text-muted-foreground">
              <li>
                Your estimated total covers the labor/service charge for the selected
services
                only. Parts &amp;amp; Accessories, replacement parts, tools, materials,
or other
                work required during service are not included. If anything additional
is needed,
                we’ll explain the cost and get your approval before applying any
extra charge.
              </li>
              <li>
                Please arrive 10–15 minutes early for check-in, motorcycle
```

```
verification, and
                  service preparation.
                </li>
              </ul>
            </div>
          </div>

          <aside aria-label="Confirm and submit" className="lg:sticky lg:top-28">
            <div className="space-y-4 rounded-xl border border-border/70
bg-card/60 p-4 shadow-sm sm:p-5">
              <h2 className="font-display text-xl tracking-wide uppercase">Confirm
and submit</h2>
              <div className="flex items-baseline justify-between gap-3 border-b
border-border/70 pb-4">
                <span className="text-sm font-medium">Estimated total</span>
                <span className="font-display text-2xl
text-primary">{formatPHP(total)}</span>
              </div>
              <div>
                <TermsAgreement
                  id="booking-terms"
                  checked={terms}
                  onCheckedChange={(value) => {
                    setTerms(value);
                    clearErrors("terms");
                  }}
                  terms={bookingTerms}
                  invalid={Boolean(errors.terms)}
                />
                <FieldError message={errors.terms} className="mt-2" />
              </div>
              <TurnstileChallenge
                resetKey={bookingRequestId}
                onToken={setTurnstileToken}
                label={
                  isReschedule
                    ? "Complete the security check to submit your reschedule request."
                    : "Complete the security check to submit your booking."
                }
              />
              <Button
                type="submit"
                size="lg"
                disabled={
                  !rescheduleReady || mutation.isPending || (turnstileEnabled &&
```

```
!turnstileToken)
                }
                className="w-full font-display tracking-wide uppercase"
              >
                {mutation.isPending ? (
                  <Loader2 className="animate-spin" aria-hidden="true" />
                ) : (
                  <Check aria-hidden="true" />
                )}
                {mutation.isPending
                  ? "Submitting request…"
                  : isReschedule
                    ? "Submit reschedule request"
                    : "Submit booking request"}
              </Button>
              <Button
                type="button"
                variant="outline"
                size="lg"
                onClick={returnToBookingDetails}
                className="w-full"
              >
                <ArrowLeft aria-hidden="true" /> Back to booking details
              </Button>
              <p className="text-center text-xs text-muted-foreground">
                Your request stays pending until the shop confirms it.
              </p>
            </div>
          </aside>
        </div>
      </form>
      {!isReschedule && (
        <>
          <AlertDialog
            open={multiDayContinuationOpen}
            onOpenChange={(open) => {
              setMultiDayContinuationOpen(open);
            }}
          >
            <AlertDialogContent className="sm:max-w-md">
              <AlertDialogHeader>
                <AlertDialogTitle className="font-display text-2xl uppercase">
                  Multi-day service
                </AlertDialogTitle>
                <AlertDialogDescription>
```

```
              The services you selected may require more than one day to
complete. If you
              continue, the remaining service time will be reserved on the next
available shop
              day.
            </AlertDialogDescription>
          </AlertDialogHeader>
          <AlertDialogFooter>
            <AlertDialogCancel>Adjust selection</AlertDialogCancel>
            <AlertDialogAction
              onClick={continueWithMultiDayBooking}
              className="bg-primary text-primary-foreground
hover:bg-primary/90"
            >
              Continue to review
            </AlertDialogAction>
          </AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialog>
      <Dialog
        open={Boolean(duplicateBookingMessage)}
        onOpenChange={(open) => !open && dismissDuplicateBookingDialog()}
      >
        <DialogContent className="sm:max-w-md">
          <DialogHeader>
            <DialogTitle className="font-display text-2xl uppercase">
              Appointment already exists
            </DialogTitle>
            <DialogDescription>{duplicateBookingMessage}</DialogDescription>
          </DialogHeader>
          <p className="text-sm text-muted-foreground">
            Choose a different available date for these services, or view your
existing
            appointment on the My Appointment page.
          </p>
          <DialogFooter className="gap-2">
            <Button asChild variant="outline">
              <Link to="/my-appointment">Go to My Appointment</Link>
            </Button>
            <Button type="button" onClick={chooseAnotherDate}>
              Choose another date
            </Button>
          </DialogFooter>
        </DialogContent>
      </Dialog>
```

```
        </>
       )}
      </main>
      <SiteFooter />
    </div>
  );
}

function BookingSteps({
  current,
  onSelectDetails,
}: {
  current: number;
  onSelectDetails: () => void;
}) {
  return (
    <ol aria-label="Booking progress" className="flex w-full items-center gap-3
md:w-auto">
      {BOOKING_PAGES.map((page, index) => {
        const complete = index < current;
        const isCurrent = index === current;
        const marker = (
          <>
            <span
              className={cn(
                "flex size-7 shrink-0 items-center justify-center rounded-full border
font-display text-sm",
                complete && "border-primary bg-primary text-primary-foreground",
                isCurrent && "border-primary bg-primary/10 text-primary",
                !complete && !isCurrent && "border-border text-muted-foreground",
              )}
            >
              {complete ? <Check className="size-4" aria-hidden="true" /> : index +
1}
            </span>
            <span
              className={cn(
                "text-sm whitespace-nowrap",
                isCurrent ? "font-medium text-foreground" : "text-muted-foreground",
              )}
            >
              {page}
              {complete && <span className="sr-only"> (completed)</span>}
            </span>
          </>
```

```
        );
        return (
          <li
            key={page}
            className={cn("flex items-center gap-3", index > 0 && "flex-1
md:flex-none")}
          >
            {index > 0 && (
              <span
                aria-hidden="true"
                className={cn(
                  "h-px min-w-6 flex-1 md:w-10 md:flex-none",
                  complete || isCurrent ? "bg-primary" : "bg-border",
                )}
              />
            )}
            {complete ? (
              <button
                type="button"
                onClick={onSelectDetails}
                className="flex items-center gap-2 rounded-md hover:underline
focus-visible:ring-2 focus-visible:ring-ring focus-visible:outline-none"
              >
                {marker}
              </button>
            ) : (
              <span
                aria-current={isCurrent ? "step" : undefined}
                className="flex items-center gap-2"
              >
                {marker}
              </span>
            )}
          </li>
        );
      })}
    </ol>
  );
}

function ScheduleHint({ children, loading }: { children: React.ReactNode;
loading?: boolean }) {
  return (
    <p className="flex items-center gap-2 rounded-lg border border-dashed
border-border p-4 text-sm text-muted-foreground">
```

```
      {loading ? (
        <Loader2 className="size-4 shrink-0 animate-spin" aria-hidden="true" />
      ) : (
        <Info className="size-4 shrink-0 text-primary" aria-hidden="true" />
      )}
      {children}
    </p>
  );
}

function SummaryRow({
  label,
  done,
  children,
}: {
  label: string;
  done: boolean;
  children: React.ReactNode;
}) {
  return (
    <div className="flex gap-3">
      <span
        aria-hidden="true"
        className={cn(
          "mt-0.5 flex size-5 shrink-0 items-center justify-center rounded-full",
          done ? "bg-primary text-primary-foreground" : "border border-border",
        )}
      >
        {done && <Check className="size-3" />}
      </span>
      <div className="min-w-0 flex-1">
        <p className="text-xs tracking-widest text-muted-foreground
uppercase">{label}</p>
        <div className="mt-0.5 break-words text-foreground">{children}</div>
      </div>
    </div>
  );
}

function SuccessDetail({ label, children }: { label: string; children:
React.ReactNode }) {
  return (
    <div className="bg-card px-5 py-4 text-center sm:text-left">
      <dt className="text-xs tracking-widest text-muted-foreground
uppercase">{label}</dt>
```

```
      <dd className="mt-1 text-sm font-medium text-foreground">{children}</dd>
    </div>
  );
}

function ReviewGroup({
  title,
  onEdit,
  editLabel,
  children,
}: {
  title: string;
  onEdit?: () => void;
  editLabel?: string;
  children: React.ReactNode;
}) {
  return (
    <section className="rounded-xl border border-border/70 bg-card/40 p-4
sm:p-5">
      <div className="mb-3 flex items-center justify-between gap-3">
        <h3 className="font-sans text-xs font-semibold tracking-widest
text-muted-foreground uppercase">
          {title}
        </h3>
        {onEdit && (
          <Button
            type="button"
            variant="ghost"
            size="sm"
            onClick={onEdit}
            aria-label={editLabel}
            className="-my-1 h-8 px-2 text-primary md:h-8"
          >
            <Pencil aria-hidden="true" /> Edit
          </Button>
        )}
      </div>
      {children}
    </section>
  );
}

function ReviewItem({ label, value }: { label: string; value: string }) {
  return (
    <div>
```

```
      <dt className="text-xs text-muted-foreground">{label}</dt>
      <dd className="mt-0.5 font-medium break-words text-foreground">{value ||
"—"}</dd>
    </div>
  );
}

function servicePricingLabel(source: "small_bike" | "large_bike") {
  if (source === "large_bike") return "Large Bike rate";
  return "Small Bike rate";
}

function Section({
  id,
  step,
  title,
  description,
  complete,
  error,
  className,
  children,
}: {
  id?: string;
  step?: number;
  title: string;
  description?: string;
  complete?: boolean;
  error?: string | undefined;
  className?: string;
  children: React.ReactNode;
}) {
  const titleId = useId();
  return (
    <section
      id={id}
      tabIndex={id ? -1 : undefined}
      aria-labelledby={titleId}
      className={cn(
        "rounded-xl border border-border/70 bg-card/40 p-4 focus:outline-none
sm:p-6",
        className,
      )}
    >
      <div className="mb-5 flex items-start gap-3">
        {step !== undefined && (
```

```
          <span
            aria-hidden="true"
            className={cn(
              "flex size-8 shrink-0 items-center justify-center rounded-full font-display
text-base transition-colors",
              complete
                ? "bg-primary text-primary-foreground"
                : "bg-primary/10 text-primary ring-1 ring-primary/30",
            )}
          >
            {complete ? <Check className="size-4" /> : step}
          </span>
        )}
        <div className="min-w-0">
          <h2 id={titleId} className="font-display text-xl tracking-wide uppercase">
            {title}
            {complete && <span className="sr-only"> (complete)</span>}
          </h2>
          {description && <p className="mt-0.5 text-sm
text-muted-foreground">{description}</p>}
        </div>
      </div>
      {children}
      <FieldError message={error} className="mt-3" />
    </section>
  );
}

function Field({
  id,
  label,
  hint,
  error,
  children,
}: {
  id: string;
  label: string;
  hint?: string;
  error?: string | undefined;
  children: React.ReactNode;
}) {
  return (
    <div
      data-invalid={!!error}
      className="space-y-1.5
```

```
data-[invalid=true]:[&_[role=combobox]]:border-destructive
data-[invalid=true]:[&_input]:border-destructive
data-[invalid=true]:[&_textarea]:border-destructive"
    >
      <Label htmlFor={id}>{label}</Label>
      {children}
      {error ? (
        <FieldError message={error} />
      ) : hint ? (
        <p id={`${id}-hint`} className="text-xs text-muted-foreground">
          {hint}
        </p>
      ) : null}
    </div>
  );
}
```

src/routes/index.tsx

```
import { useQuery } from "@tanstack/react-query";
import { createFileRoute, Link } from "@tanstack/react-router";
import {
  ArrowRight,
  BadgeCheck,
  CalendarCheck,
  CalendarDays,
  Clock,
  ExternalLink,
  Facebook,
  ListChecks,
  MapPin,
  Phone,
  ShieldCheck,
  Smartphone,
  Wrench,
} from "lucide-react";

import famImage from "@/assets/fam-image.jpg?url";
import { ProductCard } from "@/components/site/product-card";
import { ServiceBookingRow } from "@/components/site/service-booking-row";
import { ShopHours, SiteFooter } from "@/components/site/site-footer";
import { SiteHeader } from "@/components/site/site-header";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Card, CardContent } from "@/components/ui/card";
import { supabase } from "@/integrations/supabase/client";
import { SHOP } from "@/lib/shop";
import { cn } from "@/lib/utils";

export const Route = createFileRoute("/")({
  head: () => ({
    meta: [
      { title: "Fake Rider Motorparts | Motorcycle Service Booking" },
      {
        name: "description",
        content:
          "Book motorcycle service online with Fake Rider Motorparts. Premium
parts, accessories and race-grade mechanics with 48-hour advance scheduling.",
      },
      { property: "og:title", content: "Fake Rider Motorparts" },
      {
        property: "og:description",
```

```
        content:
          "Premium motorparts, accessories and race-grade motorcycle service.
Book your slot online.",
      },
    ],
  }),
  component: Home,
});

const BOOKING_STEPS = [
  {
    icon: ListChecks,
    title: "Choose your services",
    text: "Pick your motorcycle and the work you need. You see the estimated
price before you submit.",
  },
  {
    icon: CalendarDays,
    title: "Pick a date and time",
    text: "Choose any open slot at least 48 hours ahead, Monday to Saturday.",
  },
  {
    icon: BadgeCheck,
    title: "Get your reference code",
    text: "You get it right away. We confirm by email and text, and you can track or
reschedule online.",
  },
];

function Home() {
  const featured = useQuery({
    queryKey: ["featured-products"],
    queryFn: async () => {
      const { data, error } = await supabase
        .from("products")
        .select("*")
        .eq("is_active", true)
        .eq("is_archived", false)
        .order("is_featured", { ascending: false })
        .order("sort_order")
        .limit(8);
      if (error) throw error;
      return Array.from(new Map((data ?? []).map((p) => [p.name.trim(),
p])).values());
    },
```

```
  });

  const services = useQuery({
    queryKey: ["home-services"],
    queryFn: async () => {
      const { data, error, count } = await supabase
        .from("services")
        .select("id,name,description", { count: "exact" })
        .eq("is_active", true)
        .eq("is_archived", false)
        .order("sort_order")
        .limit(6);
      if (error) throw error;
      return {
        rows: Array.from(new Map((data ?? []).map((s) => [s.name.trim(),
s])).values()),
        total: count ?? 0,
      };
    },
  });

  return (
    <div className="min-h-screen">
      <SiteHeader />

      <main>
        <section className="relative isolate overflow-hidden border-b
border-border/50 bg-neutral-900">
          {/* Phones get the garage and the mechanic at work; wider screens get the
right
              two-thirds of the shopfront, so nothing busy sits behind the headline.
              License plates are blurred in both. */}
          <picture>
            <source media="(max-width: 767px)" srcSet="/hero-phone.webp" />
            <img
              src="/hero-wide.webp"
              alt=""
              aria-hidden="true"
              fetchPriority="high"
              className="absolute inset-0 -z-10 h-full w-full object-cover
md:object-[75%_65%]"
            />
          </picture>
          {/* Phones: an even shade, since the text spans the full width. Wider
screens:
```

```
            dark behind the text and light on the right, so the photo reads as
daytime. */}
          <div
            aria-hidden="true"
            className="absolute inset-0 -z-10 bg-black/60 md:bg-transparent
md:bg-gradient-to-r md:from-black/85 md:via-black/55 md:via-45%
md:to-black/5"
          />
          <div
            aria-hidden="true"
            className="absolute inset-0 -z-10 bg-gradient-to-t from-black/40
via-transparent to-transparent"
          />

          <div className="site-container py-16 md:py-28">
            <div className="max-w-2xl">
              <Badge
                className="mb-4 border-white/30 bg-black/40 text-orange-400
uppercase"
                variant="outline"
              >
                Local Pit Stop &middot; Philippines
              </Badge>
              <h1 className="font-display text-5xl leading-[0.95] font-extrabold
text-white uppercase md:text-7xl">
                Ride Hard.
                <span className="text-gradient-race block">Service Harder.</span>
              </h1>
              <p className="mt-5 max-w-lg text-base text-white/85">
                {SHOP.tagline}. Book your service online in a few minutes, get a
reference code
                right away, and let our pit crew take care of the rest.
              </p>
              <div className="mt-7 flex flex-wrap gap-3">
                <Button asChild size="lg" className="font-display tracking-wide
uppercase">
                  <Link to="/book">
                    <CalendarCheck /> Book an appointment
                  </Link>
                </Button>
                <Button
                  asChild
                  size="lg"
                  variant="outline"
                  className="font-display tracking-wide uppercase"
```

```
            >
              <Link to="/my-appointment">Track my booking</Link>
            </Button>
          </div>
          <ul className="mt-8 grid gap-3 text-sm text-white/85 sm:grid-cols-3">
            <li className="flex items-center gap-2">
              <ShieldCheck className="h-4 w-4 text-primary" aria-hidden="true"
/> Certified
              mechanics
            </li>
            <li className="flex items-center gap-2">
              <Wrench className="h-4 w-4 text-primary" aria-hidden="true" />
Genuine parts
            </li>
            <li className="flex items-center gap-2">
              <Smartphone className="h-4 w-4 text-primary" aria-hidden="true" />
Book and track
              online
            </li>
          </ul>
        </div>
      </div>
    </section>

    <section
      aria-label="Visit or contact the shop"
      className="border-b border-border/70 bg-card/40"
    >
      <div className="site-container grid gap-3 py-5 sm:grid-cols-3 sm:gap-4
md:py-6">
        <InfoTile
          icon={MapPin}
          title="Shop location"
          value={SHOP.address}
          href={SHOP.mapsUrl}
          external
        />
        <InfoTile icon={Clock} title="Open hours" value={<ShopHours />} />
        <InfoTile
          icon={Phone}
          title="Call or text"
          value={SHOP.phone}
          href={`tel:${SHOP.phone.replace(/\s/g, "")}`}
        />
      </div>
```

```
      </section>

      <section className="site-container py-7 md:py-9">
        <SectionHeading
          eyebrow="Service menu"
          title="What our pit crew can do"
          description="Pick a service to book it. You see the price for your
motorcycle on the booking form."
        />
        <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-3">
          {services.data?.rows.map((service, index) => (
            <Card
              key={service.id}
              // Four on phones; the link below leads to the rest.
              className={cn("border-border/70 bg-card/70", index >= 4 &&
"max-md:hidden")}
            >
              <CardContent className="flex h-full flex-col p-5">
                <h3 className="font-display text-lg tracking-wide
uppercase">{service.name}</h3>
                {service.description && (
                  <p className="mt-2 line-clamp-2 text-sm text-muted-foreground">
                    {service.description}
                  </p>
                )}
                <ServiceBookingRow service={service} />
              </CardContent>
            </Card>
          ))}
          {services.isLoading &&
            Array.from({ length: 6 }).map((_, i) => (
              <div
                key={i}
                className="h-40 animate-pulse rounded-xl border border-border/70
bg-card/60"
              />
            ))}
        </div>
        {services.isError && (
          <p className="text-sm text-muted-foreground">
            We couldn't load the service list right now.
          </p>
        )}
        <SectionLink to="/services">
          {services.data?.total ? `See all ${services.data.total} services` : "See all
```

```
services"}
        </SectionLink>
      </section>

      <section className="border-y border-border/70 bg-card/30">
        <div className="site-container py-7 md:py-9">
          <SectionHeading eyebrow="Booking" title="How booking works" />
          <ol className="grid gap-4 md:grid-cols-3 md:gap-6">
            {BOOKING_STEPS.map((step, index) => (
              <li key={step.title} className="flex gap-4">
                <span className="relative grid size-11 shrink-0 place-items-center
rounded-full bg-primary/15 text-primary">
                  <step.icon className="size-5" aria-hidden="true" />
                  <span className="absolute -top-1 -right-1 grid size-5
place-items-center rounded-full bg-primary text-[11px] font-bold
text-primary-foreground">
                    {index + 1}
                  </span>
                </span>
                <div>
                  <h3 className="font-display text-lg tracking-wide
uppercase">{step.title}</h3>
                  <p className="mt-1 text-sm text-muted-foreground">{step.text}</p>
                </div>
              </li>
            ))}
          </ol>
          <div className="mt-6 flex flex-col gap-3 sm:flex-row">
            <Button asChild size="lg" className="font-display tracking-wide
uppercase">
              <Link to="/book">
                <CalendarCheck /> Book an appointment
              </Link>
            </Button>
          </div>
        </div>
      </section>

      <section className="site-container py-7 md:py-9">
        <SectionHeading
          eyebrow="Shop showcase"
          title="Featured parts & accessories"
          description="Reserve an item through Messenger and pick it up at the
shop."
        />
```

```
        {/* Four items on phones and tablets, eight on wide screens. */}
        <div className="grid grid-cols-2 gap-3 sm:gap-5 lg:grid-cols-4">
          {featured.data?.map((product, index) => (
            <div key={product.id} className={cn(index >= 4 && "hidden lg:block")}>
              <ProductCard product={product} compact />
            </div>
          ))}
          {featured.isLoading &&
            Array.from({ length: 4 }).map((_, i) => (
              <div
                key={i}
                className="h-64 animate-pulse rounded-xl border border-border/70
bg-card/60 sm:h-72"
              />
            ))}
        </div>
        {featured.isError && (
          <p className="text-sm text-muted-foreground">
            We couldn't load the featured items right now.
          </p>
        )}
        <SectionLink to="/shop">View the full shop</SectionLink>
      </section>

      <section className="border-t border-border/70">
        <div className="site-container grid items-center gap-8 py-7
md:grid-cols-2 md:gap-10 md:py-9">
          <img
            src={famImage}
            alt="The Fake Rider Motorparts riding family at a local motocross event"
            loading="lazy"
            width={773}
            height={768}
            // Beside the text, a 3:2 crop from the bottom trims only sky and trees,
so
            // everyone in the photo stays whole.
            className="aspect-[773/768] h-auto w-full rounded-2xl border
border-border/70 object-cover shadow-xl lg:aspect-[3/2] lg:object-bottom"
          />
          <div>
            <SectionHeading
              eyebrow="Our riders family"
              title="More than a shop, a riding community"
            />
            <p className="text-muted-foreground">
```

```
            From weekend trail rides to local motocross events, Fake Rider
Motorparts grew with
            the riders it serves. Every bike that rolls into our garage gets the same
care we
            give our own race bikes.
           </p>
           <p className="mt-4 text-muted-foreground">
            Commuter, long-distance rider or weekend racer, our crew explains the
work in plain
            terms and helps you choose the right service or part.
           </p>
           <Button
            asChild
            variant="outline"
            className="mt-6 w-full font-display tracking-wide uppercase
sm:w-auto"
           >
            <a href={SHOP.facebook} target="_blank" rel="noopener noreferrer">
             <Facebook /> Follow us on Facebook
            </a>
           </Button>
          </div>
         </div>
        </section>
      </main>

      <SiteFooter />
     </div>
  );
}

function InfoTile({
  icon: Icon,
  title,
  value,
  href,
  external = false,
}: {
  icon: typeof MapPin;
  title: string;
  value: React.ReactNode;
  href?: string;
  external?: boolean;
}) {
  const content = (
```

```
    <>
      <Icon className="mt-0.5 h-5 w-5 shrink-0 text-primary" aria-hidden="true" />
      <div className="min-w-0">
        <p className="flex items-center gap-1 text-xs tracking-widest
text-muted-foreground uppercase">
          {title}
          {external && <ExternalLink className="h-3 w-3" aria-hidden="true" />}
        </p>
        <p className="text-sm text-foreground">{value}</p>
      </div>
    </>
  );
  const className =
    "flex items-start gap-3 rounded-lg border border-border/70 bg-background/50
p-3 sm:p-4";

  if (href) {
    return (
      <a
        href={href}
        {...(external ? { target: "_blank", rel: "noopener noreferrer" } : {})}
        className={cn(
          className,
          "text-left transition-colors hover:border-primary/50
hover:bg-background/80",
        )}
      >
        {content}
      </a>
    );
  }

  return <div className={className}>{content}</div>;
}

export function SectionHeading({
  eyebrow,
  title,
  description,
}: {
  eyebrow: string;
  title: string;
  description?: string;
}) {
  return (
```

```
    <div className="mb-5 md:mb-6">
      <p className="text-xs tracking-[0.3em] text-orange-700 uppercase
dark:text-accent">
        {eyebrow}
      </p>
      <h2 className="font-display text-3xl font-bold uppercase
md:text-4xl">{title}</h2>
      {description && <p className="mt-2 max-w-2xl
text-muted-foreground">{description}</p>}
    </div>
  );
}

/** The "see more" link under a section: full width on phones, a button elsewhere.
*/
function SectionLink({ to, children }: { to: "/services" | "/shop"; children: string }) {
  return (
    <Button
      asChild
      variant="outline"
      className="mt-5 w-full font-display tracking-wide uppercase sm:w-auto"
    >
      <Link to={to}>
        {children} <ArrowRight />
      </Link>
    </Button>
  );
}
```

src/routes/my-appointment.tsx

```
import { useMutation, useQuery } from "@tanstack/react-query";
import { createFileRoute, Link } from "@tanstack/react-router";
import { useServerFn } from "@tanstack/react-start";
import {
  ArrowLeft,
  ArrowRight,
  CalendarDays,
  CalendarX,
  CircleAlert,
  CircleCheck,
  CircleHelp,
  Clock,
  Info,
  Loader2,
  RefreshCw,
  Search,
  X,
} from "lucide-react";
import { useEffect, useMemo, useState } from "react";

import {
  type AppointmentAction,
  type AppointmentVerification,
  AppointmentVerificationDialog,
} from "@/components/site/appointment-verification-dialog";
import {
  BookingCalendar,
  FullyBookedLegend,
  TimeSlotPicker,
} from "@/components/site/booking-schedule";
import { CopyReferenceButton } from
"@/components/site/copy-reference-button";
import { SiteFooter } from "@/components/site/site-footer";
import { SiteHeader } from "@/components/site/site-header";
import { TermsAgreement } from "@/components/site/terms-agreement";
import { TurnstileChallenge } from "@/components/site/turnstile-challenge";
import { Alert, AlertDescription, AlertTitle } from "@/components/ui/alert";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Card, CardContent } from "@/components/ui/card";
import { FieldError } from "@/components/ui/field-error";
import {
  AlertDialog,
```

```
  AlertDialogAction,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogTitle,
  AlertDialogTrigger,
} from "@/components/ui/alert-dialog";
import {
  Dialog,
  DialogClose,
  DialogContent,
  DialogDescription,
  DialogFooter,
  DialogHeader,
  DialogTitle,
} from "@/components/ui/dialog";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Textarea } from "@/components/ui/textarea";
import { supabase } from "@/integrations/supabase/client";
import {
  createBooking,
  getAvailability,
  lookupAppointment,
  cancelAppointment,
} from "@/lib/booking.functions";
import {
  requestReferenceRecoveryCode,
  verifyReferenceRecoveryCode,
} from "@/lib/appointment-reference-recovery.functions";
import {
  type Availability,
  computeAvailableDates,
  computeAvailableSlots,
  computeFullyBookedDates,
} from "@/lib/availability";
import {
  DEFAULT_BOOKING_TERMS,
  formatDateLong,
  formatPHP,
  formatTime,
  isReferenceCode,
  normalizePhilippineMobile,
```

```
  normalizeReferenceCode,
  SHOP,
  statusLabel,
  statusTone,
} from "@/lib/shop";
import { cn, randomId } from "@/lib/utils";

export const Route = createFileRoute("/my-appointment")({
  head: () => ({
    meta: [
      { title: "View or Cancel My Appointment | Fake Rider" },
      {
        name: "description",
        content:
          "Enter your reference code to view your motorcycle service appointment or
cancel it online.",
      },
      { property: "og:title", content: "Track your appointment | Fake Rider
Motorparts" },
      {
        property: "og:description",
        content: "Check the status of your booking with your reference code.",
      },
    ],
  }),
  component: MyAppointment,
});

const turnstileEnabled =
Boolean(import.meta.env["VITE_TURNSTILE_SITE_KEY"]);
// Ask for a fresh code a little before the server-side expiry, so a request
// is never sent with a token that expires on the way.
const VERIFICATION_EXPIRY_MARGIN_MS = 60_000;

function MyAppointment() {
  const lookup = useServerFn(lookupAppointment);
  const cancel = useServerFn(cancelAppointment);
  const requestRecoveryCode = useServerFn(requestReferenceRecoveryCode);
  const verifyRecoveryCode = useServerFn(verifyReferenceRecoveryCode);
  const createRescheduleRequest = useServerFn(createBooking);
  const availabilityFn = useServerFn(getAvailability);
  const [reference, setReference] = useState("");
  const [errors, setErrors] = useState<{ reference?: string }>({});
  const [referenceRecoveryOpen, setReferenceRecoveryOpen] = useState(false);
  const [recoveryStep, setRecoveryStep] = useState<"email" | "code">("email");
```

```
  const [recoveryEmail, setRecoveryEmail] = useState("");
  const [recoveryCode, setRecoveryCode] = useState("");
  const [recoveryErrors, setRecoveryErrors] = useState<{ email?: string; code?:
string }>({});
  const [recoveryError, setRecoveryError] = useState<string | null>(null);
  const [recoveryMessage, setRecoveryMessage] = useState<string | null>(null);
  const [appointmentPreviewError, setAppointmentPreviewError] =
useState<string | null>(null);
  const [isRescheduling, setIsRescheduling] = useState(false);
  const [newDate, setNewDate] = useState("");
  const [newStartTime, setNewStartTime] = useState("");
  const [rescheduleReason, setRescheduleReason] = useState("");
  const [rescheduleRequestId, setRescheduleRequestId] = useState<string |
null>(null);
  const [rescheduleTurnstileToken, setRescheduleTurnstileToken] = useState("");
  const [rescheduleConfirmation, setRescheduleConfirmation] = useState<{
    reference: string;
    date: string;
    startTime: string;
  } | null>(null);
  const [statusNotice, setStatusNotice] = useState<string | null>(null);
  const [rescheduleTermsAccepted, setRescheduleTermsAccepted] =
useState(false);
  const [rescheduleErrors, setRescheduleErrors] = useState<{
    date?: string;
    time?: string;
    reason?: string;
    terms?: string;
    request?: string;
  }>({});
  const [rescheduleReviewOpen, setRescheduleReviewOpen] = useState(false);
  // Which action is waiting for email verification, and why it is being asked again.
  const [verificationRequest, setVerificationRequest] = useState<{
    action: AppointmentAction;
    notice: string | null;
  } | null>(null);
  // A reschedule spans several requests (schedules, then submit), so its token is
  // kept while the customer picks a new schedule. Cancelling uses its token at
once.
  const [rescheduleVerification, setRescheduleVerification] = useState<
    (AppointmentVerification & { reference: string }) | null
  >(null);
  const [supportsSlideTransition, setSupportsSlideTransition] = useState(true);
  const [appt, setAppt] = useState<
    null | Extract<Awaited<ReturnType<typeof lookupAppointment>>, { ok: true
```

```
}>["appointment"]
  >(null);
  // The lookup masks the mobile number. A reschedule verification returns it in
  // full, and it is kept for that appointment even if the verification expires.
  const [verifiedPhone, setVerifiedPhone] = useState<{ reference: string; phone:
string } | null>(
    null,
  );
  const reschedulePhone =
    appt && verifiedPhone?.reference === appt.reference ? verifiedPhone.phone :
null;

  useEffect(() => {
    setSupportsSlideTransition(
      typeof window !== "undefined" &&
        typeof window.CSS?.supports === "function" &&
        window.CSS.supports("transition", "opacity 1ms ease"),
    );
  }, []);

  const rescheduleServiceIds = useMemo(
    () =>
      (appt?.services ?? []).flatMap((service) => (service.serviceId ?
[service.serviceId] : [])),
    [appt?.services],
  );
  const rescheduleAvailability = useQuery<Availability>({
    queryKey: ["appointment-reschedule-availability", appt?.reference,
rescheduleServiceIds],
    queryFn: () =>
      availabilityFn({
        data: {
          days: 45,
          serviceIds: rescheduleServiceIds,
          rescheduling: true,
          rescheduleReference: appt?.reference,
          ...(reschedulePhone ? { reschedulePhone } : {}),
        },
      }),
    enabled:
      isRescheduling &&
      Boolean(appt) &&
      Boolean(reschedulePhone) &&
      rescheduleServiceIds.length > 0,
  });
```

```
  const bookingTerms = useQuery({
    queryKey: ["public-booking-terms"],
    queryFn: async () => {
      const { data, error } = await supabase
        .from("shop_settings")
        .select("booking_terms")
        .eq("id", true)
        .maybeSingle();
      if (error) throw error;
      return data?.booking_terms || DEFAULT_BOOKING_TERMS;
    },
  });
  const rescheduleDates = useMemo(
    () =>
      rescheduleAvailability.data
        ? computeAvailableDates(rescheduleAvailability.data).filter((date) => date
!== appt?.date)
        : [],
    [appt?.date, rescheduleAvailability.data],
  );
  const rescheduleDateSet = useMemo(() => new Set(rescheduleDates),
[rescheduleDates]);
  const rescheduleSlots = useMemo(
    () =>
      rescheduleAvailability.data && newDate
        ? computeAvailableSlots(rescheduleAvailability.data, newDate)
        : [],
    [newDate, rescheduleAvailability.data],
  );
  const rescheduleFullyBookedDates = useMemo(
    () =>
      rescheduleAvailability.data
        ? computeFullyBookedDates(rescheduleAvailability.data).filter((date) =>
date !== appt?.date)
        : [],
    [appt?.date, rescheduleAvailability.data],
  );

  const search = useMutation({
    mutationFn: (referenceCode: string) =>
      lookup({
        data: {
          reference: normalizeReferenceCode(referenceCode),
        },
      }),
```

```
    onSuccess: (res) => {
      if (!res.ok) {
        setAppt(null);
        setAppointmentPreviewError(res.error);
        return;
      }
      setAppointmentPreviewError(null);
      setAppt(res.appointment);
    },
    onError: () => {
      setAppt(null);
      setAppointmentPreviewError(
        "We could not retrieve your appointment details right now. Please try again
shortly.",
      );
    },
  });

  const requestRecoveryCodeMutation = useMutation({
    mutationFn: () => requestRecoveryCode({ data: { email: recoveryEmail.trim() }
}),
    onSuccess: (result) => {
      if (!result.ok) {
        setRecoveryError(result.error);
        return;
      }
      setRecoveryError(null);
      setRecoveryErrors({});
      setRecoveryCode("");
      setRecoveryMessage(result.message);
      setRecoveryStep("code");
    },
    onError: () => {
      setRecoveryError("Reference recovery is temporarily unavailable. Please try
again shortly.");
    },
  });

  const verifyRecoveryCodeMutation = useMutation({
    mutationFn: () =>
      verifyRecoveryCode({ data: { email: recoveryEmail.trim(), code:
recoveryCode.trim() } }),
    onSuccess: (result) => {
      if (!result.ok) {
        setRecoveryError(result.error);
```

```
        return;
      }
      setRecoveryError(null);
      setRecoveryErrors({});
      setRecoveryCode("");
      setRecoveryMessage(result.message);
      setRecoveryStep("email");
    },
    onError: () => {
      setRecoveryError("Reference recovery is temporarily unavailable. Please try
again shortly.");
    },
  });

  const cancelMutation = useMutation({
    mutationFn: ({ referenceCode, token }: { referenceCode: string; token: string })
=>
      cancel({
        data: {
          reference: normalizeReferenceCode(referenceCode),
          verificationToken: token,
        },
      }),
    onSuccess: (res, { referenceCode }) => {
      if (!res.ok) {
        if ("verificationRequired" in res) {
          setVerificationRequest({ action: "cancel", notice: res.error });
          return;
        }
        setAppointmentPreviewError(res.error);
        return;
      }
      setStatusNotice("Your appointment has been cancelled.");
      search.mutate(referenceCode);
    },
    onError: () =>
      setAppointmentPreviewError(
        "We could not cancel the appointment right now. Please try again or contact
the shop.",
      ),
  });

  const rescheduleMutation = useMutation({
    mutationFn: async () => {
      if (!appt) throw new Error("Appointment details are no longer available.");
```

```
      const phone = normalizePhilippineMobile(reschedulePhone ?? "");
      if (!phone) throw new Error("The verified mobile number is missing.");
      const result = await createRescheduleRequest({
        data: {
          // The server reloads and trusts the original appointment record. These
          // values only satisfy the public booking input shape and cannot alter it.
          firstName: appt.firstName,
          middleName: appt.middleName,
          lastName: appt.lastName,
          phone,
          motoBrand: "Original",
          motoModel: "Appointment",
          motoVariant: "",
          motoYear: 2000,
          plateNumber: appt.plateNumber,
          serviceIds: rescheduleServiceIds,
          date: newDate,
          startTime: newStartTime,
          notes: "",
          wantsProducts: false,
          turnstileToken: rescheduleTurnstileToken,
          // Keep this value stable for retries so a lost response returns the
          // originally reserved reference instead of creating a new request.
          idempotencyKey: rescheduleRequestId ?? randomId(),
          termsAccepted: true as const,
          rescheduleReference: appt.reference,
          rescheduleReason: rescheduleReason.trim(),
          ...(rescheduleVerification ? { verificationToken:
rescheduleVerification.token } : {}),
        },
      });
      return result;
    },
    onSuccess: (result) => {
      if (!result.ok) {
        if ("verificationRequired" in result) {
          setRescheduleVerification(null);
          setRescheduleReviewOpen(false);
          setVerificationRequest({ action: "reschedule", notice: result.error });
          return;
        }
        setRescheduleErrors((current) => ({ ...current, request: result.error }));
        return;
      }
      setRescheduleVerification(null);
```

```
      setRescheduleConfirmation({
        reference: result.reference,
        date: newDate,
        startTime: newStartTime,
      });
      setIsRescheduling(false);
      setRescheduleReviewOpen(false);
      if (appt) search.mutate(appt.reference);
    },
    onError: () => {
      setRescheduleErrors((current) => ({
        ...current,
        request: "We could not submit your reschedule request. Please try again.",
      }));
    },
  });

  function hasRescheduleVerification() {
    return Boolean(
      appt &&
      rescheduleVerification?.reference === appt.reference &&
      Date.parse(rescheduleVerification.expiresAt) - Date.now() >
VERIFICATION_EXPIRY_MARGIN_MS,
    );
  }

  function beginReschedule() {
    if (!appt || !["pending", "confirmed"].includes(appt.status)) {
      setAppointmentPreviewError(
        "This appointment can no longer be rescheduled online. Please call the
shop.",
      );
      return;
    }
    if ((appt?.rescheduleCount ?? 0) >= 3) {
      setAppointmentPreviewError("This appointment has reached the maximum of
3 reschedules.");
      return;
    }
    if (rescheduleServiceIds.length !== (appt?.services.length ?? 0)) {
      setAppointmentPreviewError(
        "The original services are no longer available for online rescheduling.",
      );
      return;
    }
```

```
    if (!hasRescheduleVerification()) {
      setVerificationRequest({ action: "reschedule", notice: null });
      return;
    }
    openRescheduleForm();
  }

  function openRescheduleForm() {
    setNewDate("");
    setNewStartTime("");
    setRescheduleReason("");
    setRescheduleRequestId(randomId());
    setRescheduleTurnstileToken("");
    setRescheduleConfirmation(null);
    setRescheduleTermsAccepted(false);
    setRescheduleErrors({});
    setIsRescheduling(true);
  }

  function validateReschedule() {
    const nextErrors: typeof rescheduleErrors = {};
    if (!newDate) nextErrors.date = "Choose a new appointment date.";
    else if (newDate === appt?.date)
      nextErrors.date = "Choose a date different from your current appointment.";
    if (!newStartTime) nextErrors.time = "Choose an available appointment time.";
    if (!rescheduleReason.trim()) nextErrors.reason = "Tell us why you need to
reschedule.";
    if (!rescheduleTermsAccepted) nextErrors.terms = "You must agree to the
terms and conditions.";
    setRescheduleErrors(nextErrors);
    return Object.keys(nextErrors).length === 0;
  }

  function openRescheduleReview() {
    if (validateReschedule()) setRescheduleReviewOpen(true);
  }

  function submitRescheduleRequest() {
    if (!validateReschedule()) {
      setRescheduleReviewOpen(false);
      return;
    }
    if (!hasRescheduleVerification()) {
      setRescheduleReviewOpen(false);
      setVerificationRequest({
```

```
        action: "reschedule",
        notice:
          "Your email verification has expired. Verify again to submit your request;
your chosen schedule is kept.",
      });
      return;
    }
    rescheduleMutation.mutate();
  }

  function handleVerified(action: AppointmentAction, verification:
AppointmentVerification) {
    if (!appt) return;
    setVerificationRequest(null);
    if (action === "cancel") {
      cancelMutation.mutate({ referenceCode: appt.reference, token:
verification.token });
      return;
    }
    setRescheduleVerification({ ...verification, reference: appt.reference });
    if (verification.phone)
      setVerifiedPhone({ reference: appt.reference, phone: verification.phone });
    if (isRescheduling) {
      // Verified again mid-way: keep the chosen schedule and return to the review
step.
      setRescheduleErrors(({ request: _, ...current }) => current);
      setRescheduleReviewOpen(true);
      return;
    }
    openRescheduleForm();
  }

  function closeAppointmentPreview() {
    setAppt(null);
    setVerificationRequest(null);
    setRescheduleVerification(null);
    setIsRescheduling(false);
    setNewDate("");
    setNewStartTime("");
    setRescheduleReason("");
    setRescheduleRequestId(null);
    setRescheduleTurnstileToken("");
    setRescheduleConfirmation(null);
    setRescheduleErrors({});
    setAppointmentPreviewError(null);
```

```
    setStatusNotice(null);
    setReference("");
    setErrors({});
  }

  /** Look up a linked appointment (for example, a rescheduled booking's new
reference). */
  function lookupReference(referenceCode: string) {
    setReference(referenceCode);
    setErrors({});
    setAppointmentPreviewError(null);
    setStatusNotice(null);
    setRescheduleConfirmation(null);
    setVerificationRequest(null);
    setRescheduleVerification(null);
    setIsRescheduling(false);
    search.mutate(referenceCode);
  }

  function validate() {
    const nextErrors: { reference?: string } = {};
    if (!isReferenceCode(reference)) {
      nextErrors.reference = "Enter a valid reference code.";
    }
    setErrors(nextErrors);
    setAppointmentPreviewError(null);
    return Object.keys(nextErrors).length === 0;
  }

  function resetReferenceRecovery() {
    setRecoveryStep("email");
    setRecoveryEmail("");
    setRecoveryCode("");
    setRecoveryErrors({});
    setRecoveryError(null);
    setRecoveryMessage(null);
  }

  function validateRecoveryEmail() {
    const nextErrors: typeof recoveryErrors = {};
    if (!/^\S+@\S+\.\S+$/.test(recoveryEmail.trim())) {
      nextErrors.email = "Enter a valid email address.";
    }
    setRecoveryErrors(nextErrors);
    setRecoveryError(null);
```

```
  setRecoveryMessage(null);
  return Object.keys(nextErrors).length === 0;
 }

 function validateRecoveryCode() {
  const nextErrors: typeof recoveryErrors = {};
  if (!/^\d{6}$/.test(recoveryCode.trim())) {
   nextErrors.code = "Enter the 6-digit verification code.";
  }
  setRecoveryErrors(nextErrors);
  setRecoveryError(null);
  setRecoveryMessage(null);
  return Object.keys(nextErrors).length === 0;
 }

 const canManageAppointment = Boolean(
  appt &&
  ["pending", "confirmed", "rescheduled"].includes(appt.status) &&
  !appt.hasReplacement &&
  !appt.rescheduleRequestPending,
 );
 const canRescheduleAppointment = Boolean(
  appt &&
  canManageAppointment &&
  ["pending", "confirmed"].includes(appt.status) &&
  appt.rescheduleCount < 3,
 );
 const statusHint = appt ? appointmentStatusHint(appt.status) : null;

 // The reschedule form is rendered in exactly one place: the sliding panel in
 // modern browsers, or inline below the appointment details as a fallback.
 const rescheduleForm = appt ? (
  rescheduleAvailability.isLoading ? (
   <p className="mt-6 flex items-center gap-2 rounded-lg border
border-dashed border-border p-4 text-sm text-muted-foreground">
    <Loader2 className="size-4 animate-spin" aria-hidden="true" /> Checking
available schedules…
   </p>
  ) : rescheduleAvailability.isError || rescheduleAvailability.data?.error ? (
   <p className="mt-6 flex items-start gap-2 rounded-lg border
border-destructive/40 bg-destructive/5 p-3 text-sm text-destructive">
    <CircleAlert className="mt-0.5 size-4 shrink-0" aria-hidden="true" />
    {rescheduleAvailability.data?.error ??
     "We could not load available schedules. Please try again."}
   </p>
```

```
    ) : (
      <div className="mt-6 space-y-6">
        <div
          className={cn(
            "rounded-xl border border-border bg-card/50 p-3 sm:p-5",
            (rescheduleErrors.date || rescheduleErrors.time) && "border-destructive",
          )}
        >
          <div className="grid gap-6 md:grid-cols-[auto_minmax(0,1fr)]
md:gap-8">
            <div className="border-b border-border/70 pb-5 md:border-r
md:border-b-0 md:pr-8 md:pb-0">
              <p className="text-sm font-medium">New date</p>
              <div className="mt-2 flex justify-center md:justify-start">
                <BookingCalendar
                  selected={newDate}
                  availableDates={rescheduleDateSet}
                  fullyBookedDates={rescheduleFullyBookedDates}
                  onSelect={(iso) => {
                    setNewDate(iso);
                    setNewStartTime("");
                    setRescheduleErrors(({ date: _, time: __, ...current }) => current);
                  }}
                />
              </div>
              {rescheduleFullyBookedDates.length > 0 && (
                <FullyBookedLegend className="mt-3 justify-center md:justify-start"
/>
              )}
              <FieldError message={rescheduleErrors.date} className="mt-2" />
            </div>
            <div className="min-w-0">
              <p className="text-sm font-medium">New time</p>
              <p className="mt-1 mb-4 text-sm text-muted-foreground">
                {newDate
                  ? formatDateLong(newDate)
                  : "Choose a date to see the available start times."}
              </p>
              {newDate ? (
                <TimeSlotPicker
                  slots={rescheduleSlots}
                  value={newStartTime}
                  onChange={(value) => {
                    setNewStartTime(value);
                    setRescheduleErrors(({ time: _, ...current }) => current);
```

```
              }}
              invalid={Boolean(rescheduleErrors.time)}
              gridClassName="sm:grid-cols-3 md:grid-cols-2 lg:grid-cols-3"
            />
          ) : (
            <p className="rounded-lg border border-dashed border-border p-4 text-sm text-muted-foreground">
              Time slots will appear here after you choose a date.
            </p>
          )}
          <FieldError message={rescheduleErrors.time} className="mt-2" />
        </div>
      </div>

      <div>
        <Label htmlFor="reschedule-reason">Reason for rescheduling</Label>
        <Textarea
          id="reschedule-reason"
          value={rescheduleReason}
          maxLength={500}
          onChange={(event) => {
            setRescheduleReason(event.target.value);
            setRescheduleErrors(({ reason: _, ...current }) => current);
          }}
          placeholder="Tell the shop why you need a different schedule."
          aria-invalid={Boolean(rescheduleErrors.reason)}
          className="mt-2"
        />
        <FieldError message={rescheduleErrors.reason} className="mt-2" />
      </div>

      <div>
        <TermsAgreement
          id="reschedule-terms"
          checked={rescheduleTermsAccepted}
          onCheckedChange={(value) => {
            setRescheduleTermsAccepted(value);
            setRescheduleErrors(({ terms: _, ...current }) => current);
          }}
          terms={bookingTerms.data || DEFAULT_BOOKING_TERMS}
          invalid={Boolean(rescheduleErrors.terms)}
        />
        <FieldError message={rescheduleErrors.terms} className="mt-2" />
      </div>
```

```
        {rescheduleErrors.request && (
          <p
            role="alert"
            className="flex items-start gap-2 rounded-lg border
border-destructive/40 bg-destructive/5 p-3 text-sm text-destructive"
          >
            <CircleAlert className="mt-0.5 size-4 shrink-0" aria-hidden="true" />
            {rescheduleErrors.request}
          </p>
        )}
        {isRescheduling && (
          <TurnstileChallenge
            resetKey={rescheduleRequestId ?? ""}
            onToken={setRescheduleTurnstileToken}
            label="Complete the security check to continue."
          />
        )}

        <div className="flex flex-col-reverse gap-3 border-t border-border/70 pt-5
sm:flex-row sm:justify-between">
          <Button
            type="button"
            variant="outline"
            onClick={() => {
              setIsRescheduling(false);
              setRescheduleErrors({});
            }}
            className="w-full sm:w-auto"
          >
            <ArrowLeft aria-hidden="true" /> Back to appointment
          </Button>
          <Button
            type="button"
            onClick={openRescheduleReview}
            disabled={
              rescheduleMutation.isPending || (turnstileEnabled &&
!rescheduleTurnstileToken)
            }
            className="w-full font-display tracking-wide uppercase sm:w-auto"
          >
            Review reschedule request <ArrowRight aria-hidden="true" />
          </Button>
        </div>
      </div>
    )
```

```
  ) : null;

  return (
    <div className="min-h-screen">
      <SiteHeader />
      <main className="mx-auto w-full max-w-4xl px-4 py-8 sm:px-6 sm:py-12">
        <p className="text-xs tracking-[0.3em] text-accent
uppercase">Appointment tracker</p>
        <h1 className="font-display text-3xl font-bold uppercase sm:text-4xl
md:text-5xl">
          My appointment
        </h1>
        <p className="mt-2 max-w-2xl text-muted-foreground">
          Enter your booking reference code to check your appointment status,
reschedule, or cancel.
          To protect your booking, changes are confirmed with a code sent to your
booking email.
        </p>

        <form
          noValidate
          className="mt-8 rounded-xl border border-border/70 bg-card/50 p-4
sm:p-6"
          onSubmit={(e) => {
            e.preventDefault();
            if (validate()) {
              setRescheduleConfirmation(null);
              setStatusNotice(null);
              setVerificationRequest(null);
              setRescheduleVerification(null);
              setIsRescheduling(false);
              search.mutate(reference);
            }
          }}
        >
          <Label htmlFor="appointment-reference">Reference code</Label>
          <div className="mt-1.5 flex flex-col gap-3 sm:flex-row">
            <Input
              id="appointment-reference"
              value={reference}
              onChange={(e) => {
                setReference(normalizeReferenceCode(e.target.value));
                setAppointmentPreviewError(null);
                setErrors((current) => {
                  const { reference: _, ...rest } = current;
```

```
              return rest;
            });
          }}
          placeholder="FRM-XXXXXX"
          maxLength={10}
          autoCapitalize="characters"
          autoComplete="off"
          spellCheck={false}
          aria-invalid={!!errors.reference}
          aria-describedby="appointment-reference-hint"
          className="h-11 font-display text-lg tracking-widest uppercase
md:h-10 md:text-lg"
        />
        <Button
          type="submit"
          size="lg"
          disabled={search.isPending}
          className="w-full shrink-0 font-display tracking-wide uppercase
sm:w-auto"
        >
          {search.isPending ? (
            <Loader2 className="animate-spin" aria-hidden="true" />
          ) : (
            <Search aria-hidden="true" />
          )}
          {search.isPending ? "Searching…" : "Find appointment"}
        </Button>
      </div>
      <FieldError message={errors.reference} className="mt-2" />
      <div className="mt-3 flex flex-col gap-1 text-xs text-muted-foreground
sm:flex-row sm:items-center sm:justify-between">
        <p id="appointment-reference-hint">
          Your code looks like FRM-ABC123 and was shown when you booked.
        </p>
        <Button
          type="button"
          variant="link"
          className="h-auto self-start px-0 text-xs md:h-auto"
          onClick={() => setReferenceRecoveryOpen(true)}
        >
          <CircleHelp className="size-3.5" aria-hidden="true" /> Forgot your
reference code?
        </Button>
      </div>
    </form>
```

```
      {/* Kept outside the lookup form so its own form submit cannot also trigger a
lookup. */}
      <Dialog
        open={referenceRecoveryOpen}
        onOpenChange={(open) => {
          setReferenceRecoveryOpen(open);
          if (!open) resetReferenceRecovery();
        }}
      >
        {/* minmax(0,1fr) keeps the dialog's single grid column from growing wider
            than the dialog when its content has long, non-wrapping text. */}
        <DialogContent className="grid-cols-[minmax(0,1fr)] sm:max-w-md">
          <DialogHeader>
            <DialogTitle>Recover your reference code</DialogTitle>
            <DialogDescription>
              {recoveryStep === "email"
                ? "Enter the email address used for your booking. We will send a
verification code before emailing your reference code."
                : "Enter the verification code we sent to your booking email address."}
            </DialogDescription>
          </DialogHeader>
          <form
            noValidate
            className="space-y-4"
            onSubmit={(event) => {
              event.preventDefault();
              if (recoveryStep === "email") {
                if (validateRecoveryEmail()) requestRecoveryCodeMutation.mutate();
                return;
              }
              if (validateRecoveryCode()) verifyRecoveryCodeMutation.mutate();
            }}
          >
            {recoveryStep === "email" ? (
              <div className="space-y-1.5">
                <Label htmlFor="recovery-email">Email address</Label>
                <Input
                  id="recovery-email"
                  type="email"
                  value={recoveryEmail}
                  maxLength={254}
                  autoComplete="email"
                  onChange={(event) => {
                    setRecoveryEmail(event.target.value);
```

```
        setRecoveryErrors(({ email: _, ...current }) => current);
        setRecoveryError(null);
      }}
      placeholder="you@example.com"
      aria-invalid={Boolean(recoveryErrors.email)}
    />
    <FieldError message={recoveryErrors.email} />
  </div>
) : (
  <div className="space-y-1.5">
    <Label htmlFor="recovery-code">Verification code</Label>
    <Input
      id="recovery-code"
      type="text"
      value={recoveryCode}
      maxLength={6}
      inputMode="numeric"
      autoComplete="one-time-code"
      onChange={(event) => {
        setRecoveryCode(event.target.value.replace(/\D/g, ""));
        setRecoveryErrors(({ code: _, ...current }) => current);
        setRecoveryError(null);
      }}
      placeholder="123456"
      aria-invalid={Boolean(recoveryErrors.code)}
    />
    <FieldError message={recoveryErrors.code} />
    <div className="flex flex-wrap items-center gap-x-4 gap-y-1">
      <Button
        type="button"
        variant="link"
        className="h-auto px-0 text-xs md:h-auto"
        disabled={requestRecoveryCodeMutation.isPending}
        onClick={() => requestRecoveryCodeMutation.mutate()}
      >
        {requestRecoveryCodeMutation.isPending && (
          <Loader2 className="animate-spin" />
        )}
        Resend code
      </Button>
      <Button
        type="button"
        variant="link"
        className="h-auto px-0 text-xs md:h-auto"
        onClick={() => {
```

```
                setRecoveryStep("email");
                setRecoveryCode("");
                setRecoveryErrors({});
                setRecoveryError(null);
                setRecoveryMessage(null);
              }}
            >
              Use a different email
            </Button>
          </div>
        </div>
      )}
      {recoveryError && (
        <Alert variant="destructive" className="border-destructive/40
bg-destructive/5">
          <CircleAlert className="size-4" aria-hidden="true" />
          <AlertDescription>{recoveryError}</AlertDescription>
        </Alert>
      )}
      {recoveryMessage && (
        <div role="status" className="rounded-lg border border-primary/30
bg-primary/5 p-4">
          <p className="text-sm font-medium">{recoveryMessage}</p>
        </div>
      )}
      {/* The footer's gap already spaces the buttons; drop the extra
          horizontal margin and let them wrap rather than overflow. */}
      <DialogFooter className="sm:flex-wrap sm:space-x-0">
        <DialogClose asChild>
          <Button type="button" variant="outline">
            Close
          </Button>
        </DialogClose>
        <Button
          type="submit"
          disabled={
            requestRecoveryCodeMutation.isPending ||
verifyRecoveryCodeMutation.isPending
          }
        >
          {(requestRecoveryCodeMutation.isPending ||
            verifyRecoveryCodeMutation.isPending) && <Loader2
className="animate-spin" />}
          {recoveryStep === "email" ? "Send verification code" : "Email my
reference code"}
```

```
            </Button>
          </DialogFooter>
        </form>
      </DialogContent>
    </Dialog>

    {appointmentPreviewError && (
      <Alert variant="destructive" className="mt-4 border-destructive/40
bg-destructive/5">
        <CircleAlert className="size-4" aria-hidden="true" />
        <div>
          <AlertTitle>Unable to show appointment details</AlertTitle>
          <AlertDescription>
            <p>{appointmentPreviewError}</p>
            <p className="mt-1 text-xs text-muted-foreground">
              No changes have been made to your appointment.
            </p>
          </AlertDescription>
        </div>
      </Alert>
    )}

    {statusNotice && (
      <p
        role="status"
        className="mt-4 flex items-center gap-2 rounded-lg border
border-emerald-500/30 bg-emerald-500/10 p-3 text-sm text-foreground"
      >
        <CircleCheck className="size-4 shrink-0 text-emerald-500"
aria-hidden="true" />
        {statusNotice}
      </p>
    )}

    {rescheduleConfirmation && (
      <section
        role="status"
        className="mt-8 rounded-xl border border-primary/40 bg-primary/5 p-5
text-center sm:p-6"
      >
        <CircleCheck className="mx-auto size-10 text-primary"
aria-hidden="true" />
        <h2 className="mt-3 font-display text-2xl uppercase">Reschedule
request sent</h2>
        <p className="mx-auto mt-1 max-w-md text-sm
```

```
text-muted-foreground">
            Requested for {formatDateLong(rescheduleConfirmation.date)} at{" "}
            {formatTime(rescheduleConfirmation.startTime)}. Your current
appointment stays
            reserved until the shop reviews your request.
          </p>
          <p className="mt-5 text-xs tracking-[0.24em] text-muted-foreground
uppercase">
            New reference code
          </p>
          <p className="mt-1 font-display text-3xl tracking-widest text-primary">
            {rescheduleConfirmation.reference}
          </p>
          <CopyReferenceButton value={rescheduleConfirmation.reference}
className="mt-3" />
          <p className="mt-3 text-xs text-muted-foreground">
            Save this code. The linked booking becomes active once the shop
approves your request.
          </p>
        </section>
      )}

      {!appt && !appointmentPreviewError && !rescheduleConfirmation &&
!search.isPending && (
        <>
          <ul aria-label="What you can do here" className="mt-8 grid gap-3
sm:grid-cols-3">
            {[
              {
                icon: Search,
                title: "Check status",
                text: "See whether your appointment is pending, confirmed, or
completed.",
              },
              {
                icon: RefreshCw,
                title: "Reschedule",
                text: "Request a new date and time for a pending or confirmed
appointment.",
              },
              {
                icon: CalendarX,
                title: "Cancel",
                text: "Cancel online if your plans change, subject to the shop's notice
period.",
```

```
            },
          ].map((item) => (
            <li
              key={item.title}
              className="flex items-start gap-3 rounded-lg border
border-border/70 bg-background/50 p-4"
            >
              <item.icon className="mt-0.5 size-5 shrink-0 text-primary"
aria-hidden="true" />
              <div>
                <p className="font-display text-base tracking-wide
uppercase">{item.title}</p>
                <p className="mt-0.5 text-sm
text-muted-foreground">{item.text}</p>
              </div>
            </li>
          ))}
        </ul>
        <p className="mt-6 text-center text-sm text-muted-foreground">
          Don&apos;t have a booking yet?{" "}
          <Link to="/book" className="font-medium text-primary underline
underline-offset-4">
            Book a service appointment
          </Link>
        </p>
      </>
    )}

    {appt && (
      <Card className="mt-8 overflow-hidden border-border/70 bg-card/60">
        <CardContent className="p-0">
          <div className="relative isolate overflow-hidden">
            <div
              aria-hidden={isRescheduling && supportsSlideTransition}
              inert={isRescheduling && supportsSlideTransition}
              className={cn(
                "transition-[opacity,transform] duration-500 ease-out
will-change-[opacity,transform] motion-reduce:transition-none",
                isRescheduling && supportsSlideTransition
                  ? "pointer-events-none absolute inset-x-0 top-0 -translate-x-8
opacity-0"
                  : "relative translate-x-0 opacity-100",
              )}
            >
              <div className="flex flex-wrap items-start justify-between gap-4
```

```
border-b border-border/70 bg-muted/30 px-5 py-4 sm:px-6">
                <div>
                  <p className="text-xs tracking-widest text-muted-foreground
uppercase">
                    Reference code
                  </p>
                  <div className="mt-1 flex flex-wrap items-center gap-3">
                    <p className="font-display text-2xl tracking-widest
text-primary">
                      {appt.reference}
                    </p>
                    <CopyReferenceButton value={appt.reference} />
                  </div>
                </div>
                <Badge variant="outline" className={cn("uppercase",
statusTone(appt.status))}>
                  {statusLabel(appt.status)}
                </Badge>
              </div>

              <div className="space-y-6 p-5 sm:p-6">
                <div className="flex items-start gap-4 rounded-xl border
border-primary/30 bg-primary/5 p-4">
                  <span className="flex size-11 shrink-0 items-center justify-center
rounded-lg bg-primary text-primary-foreground">
                    <CalendarDays className="size-5" aria-hidden="true" />
                  </span>
                  <div className="min-w-0">
                    <p className="text-xs tracking-widest text-muted-foreground
uppercase">
                      Scheduled for
                    </p>
                    <p className="font-display text-xl uppercase sm:text-2xl">
                      {formatDateLong(appt.date)}
                    </p>
                    <p className="flex items-center gap-1.5 text-sm
text-muted-foreground">
                      <Clock className="size-4" aria-hidden="true" />
                      {formatTime(appt.startTime)}
                    </p>
                    {statusHint && <p className="mt-2 text-sm">{statusHint}</p>}
                  </div>
                </div>

                {appt.rescheduledFromReference && (
```

```
                  <Notice tone="info">
                    This appointment was rescheduled from{" "}
                    <strong>{appt.rescheduledFromReference}</strong>.
                  </Notice>
                )}

                {appt.rescheduledToReference && (
                  <Notice tone="accent">
                    <p>
                      This appointment has been rescheduled. Your new reference is{"
"}
                      <strong>{appt.rescheduledToReference}</strong>.
                    </p>
                    <Button
                      type="button"
                      variant="link"
                      className="mt-1 h-auto px-0 md:h-auto"
                      onClick={() =>
lookupReference(appt.rescheduledToReference!)}
                    >
                      View new appointment <ArrowRight aria-hidden="true" />
                    </Button>
                  </Notice>
                )}

                {appt.rescheduleRequestPending &&
                  appt.requestedRescheduleDate &&
                  appt.requestedRescheduleStartTime && (
                    <Notice
                      tone="accent"
                      title={
                        appt.foundByRescheduleRequest &&
appt.requestedRescheduleReference
                          ? `Reschedule request
${appt.requestedRescheduleReference} is awaiting review`
                          : "Reschedule request awaiting review"
                      }
                    >
                      {appt.foundByRescheduleRequest &&
appt.requestedRescheduleReference ? (
                        <>
                          <strong>{appt.requestedRescheduleReference}</strong> is
the reference
                          for your request to move appointment
<strong>{appt.reference}</strong>{" "}
```

```
                    to {formatDateLong(appt.requestedRescheduleDate)} at{" "}
                    {formatTime(appt.requestedRescheduleStartTime)}. The shop
hasn&apos;t
                    reviewed it yet, so the details shown here are still for{" "}
                    {appt.reference}, which stays reserved until then. Once the
request is
                    approved, {appt.requestedRescheduleReference} becomes
your
                    appointment&apos;s reference.
                  </>
                ) : (
                  <>
                    Your request to move this appointment to{" "}
                    {formatDateLong(appt.requestedRescheduleDate)} at{" "}
                    {formatTime(appt.requestedRescheduleStartTime)} is awaiting
the
                    shop&apos;s review. Your current appointment remains
reserved until
                    then.
                  </>
                )}
                {!appt.foundByRescheduleRequest &&
appt.requestedRescheduleReference && (
                  <>
                    {" "}
                    Your reserved new reference is{" "}
                    <strong>{appt.requestedRescheduleReference}</strong>.
                  </>
                )}
                {appt.requestedRescheduleReason && (
                  <>
                    {" "}
                    Reason:
<strong>{appt.requestedRescheduleReason}</strong>
                  </>
                )}
              </Notice>
            )}

            {appt.rescheduleRequestRejected && (
              <Notice tone="destructive" title="Reschedule request not
approved">
                {appt.rescheduleRequestRejectionMessage ||
                  "Your requested reschedule was not approved. Your original
appointment remains unchanged and reserved."}
```

```
              </Notice>
            )}

            {!isRescheduling &&
              !appt.hasReplacement &&
              !appt.rescheduleRequestPending &&
              appt.rescheduleCount >= 3 && (
                <Notice tone="destructive" title="Maximum reschedule limit
reached">
                  This appointment has already been rescheduled 3 times and
cannot be
                  rescheduled again.
                </Notice>
              )}

            <dl className="grid gap-4 sm:grid-cols-2">
              <Row label="Customer" value={appt.customerName} />
              <Row label="Mobile number" value={appt.phoneHint} />
              <Row label="Motorcycle" value={appt.motorcycle} />
              <Row label="Plate number" value={appt.plateNumber} />
            </dl>

            <div>
              <p className="text-xs tracking-widest text-muted-foreground
uppercase">
                Services
              </p>
              <ul className="mt-2 divide-y divide-border/60 overflow-hidden
rounded-lg border border-border/70">
                {appt.services.map((s) => (
                  <li
                    key={s.name}
                    className="flex justify-between gap-3 px-3 py-2.5 text-sm"
                  >
                    <span>{s.name}</span>
                    <span className="shrink-0
text-primary">{formatPHP(s.price)}</span>
                  </li>
                ))}
                <li className="flex items-baseline justify-between gap-3
bg-primary/5 px-3 py-3">
                  <span className="text-sm font-medium">Estimated
total</span>
                  <span className="font-display text-xl text-primary">
                    {formatPHP(appt.total)}
```

```
                    </span>
                  </li>
                </ul>
              </div>

              {appt.notes && (
                <div>
                  <p className="text-xs tracking-widest text-muted-foreground
uppercase">
                    Notes
                  </p>
                  <p className="mt-1 text-sm
whitespace-pre-line">{appt.notes}</p>
                </div>
              )}

              {isRescheduling && !supportsSlideTransition && (
                <div className="border-t border-border/70 pt-6">
                  <RescheduleHeading reference={appt.reference} />
                  {rescheduleForm}
                </div>
              )}
            </div>

            {!isRescheduling && (
              <div className="flex flex-col-reverse gap-3 border-t
border-border/70 px-5 py-4 sm:flex-row sm:items-center sm:justify-between
sm:px-6">
                <AlertDialog>
                  <AlertDialogTrigger asChild>
                    <Button type="button" variant="ghost" className="w-full
sm:w-auto">
                      <X aria-hidden="true" /> Close details
                    </Button>
                  </AlertDialogTrigger>
                  <AlertDialogContent>
                    <AlertDialogHeader>
                      <AlertDialogTitle>Close appointment
details?</AlertDialogTitle>
                      <AlertDialogDescription>
                        Your appointment will not be changed. You can look it up
again anytime
                        with your reference code.
                      </AlertDialogDescription>
                    </AlertDialogHeader>
```

```
                    <AlertDialogFooter>
                      <AlertDialogCancel>Keep viewing</AlertDialogCancel>
                      <AlertDialogAction onClick={closeAppointmentPreview}>
                        Close details
                      </AlertDialogAction>
                    </AlertDialogFooter>
                  </AlertDialogContent>
                </AlertDialog>

                {canManageAppointment && !appt.emailHint && (
                  <p className="flex max-w-md items-start gap-2 text-sm
text-muted-foreground">
                    <Info
                      className="mt-0.5 size-4 shrink-0 text-primary"
                      aria-hidden="true"
                    />
                    <span>
                      To cancel or reschedule, please call the shop at{" "}
                      <a
                        href={`tel:${SHOP.phone.replace(/\s/g, "")}`}
                        className="font-medium whitespace-nowrap text-foreground
underline underline-offset-4"
                      >
                        {SHOP.phone}
                      </a>
                      . Online changes are confirmed with a code sent to the
booking&amp;apos;s
                      email, and this booking has none on file.
                    </span>
                  </p>
                )}

                {canManageAppointment && appt.emailHint && (
                  <div className="flex flex-col-reverse gap-3 sm:flex-row">
                    <Button
                      type="button"
                      variant="outline"
                      disabled={cancelMutation.isPending}
                      onClick={() =>
                        setVerificationRequest({ action: "cancel", notice: null })
                      }
                      className="w-full border-destructive/40 font-display
tracking-wide text-destructive uppercase hover:bg-destructive/10
hover:text-destructive sm:w-auto"
                    >
```

```
                    {cancelMutation.isPending ? (
                      <Loader2 className="animate-spin" aria-hidden="true" />
                    ) : (
                      <CalendarX aria-hidden="true" />
                    )}
                    {cancelMutation.isPending ? "Cancelling…" : "Cancel
appointment"}
                  </Button>
                  {canRescheduleAppointment && (
                    <Button
                      type="button"
                      onClick={beginReschedule}
                      className="w-full font-display tracking-wide uppercase
sm:w-auto"
                    >
                      <RefreshCw aria-hidden="true" /> Reschedule appointment
                    </Button>
                  )}
                </div>
              )}
            </div>
          )}
        </div>

        <div
          aria-hidden={!isRescheduling || !supportsSlideTransition}
          inert={!isRescheduling || !supportsSlideTransition}
          className={cn(
            "p-5 transition-[opacity,transform] duration-500 ease-out
will-change-[opacity,transform] motion-reduce:transition-none sm:p-6",
            !supportsSlideTransition && "hidden",
            isRescheduling && supportsSlideTransition
              ? "relative translate-x-0 opacity-100"
              : "pointer-events-none absolute inset-x-0 top-0 translate-x-8
opacity-0",
          )}
        >
          <RescheduleHeading reference={appt.reference} />

          <section className="mt-5 grid gap-4 rounded-xl border
border-border/70 bg-muted/30 p-4 text-sm sm:grid-cols-2">
            <SummaryBlock label="Current schedule">
              <p className="font-medium">{formatDateLong(appt.date)}</p>
              <p
className="text-muted-foreground">{formatTime(appt.startTime)}</p>
```

```
              </SummaryBlock>
              <SummaryBlock label="Customer">
                <p className="font-medium">{appt.customerName}</p>
                <p className="text-muted-foreground">{reschedulePhone ??
appt.phoneHint}</p>
              </SummaryBlock>
              <SummaryBlock label="Motorcycle">
                <p className="font-medium">{appt.motorcycle}</p>
                <p className="text-muted-foreground">Plate
{appt.plateNumber}</p>
              </SummaryBlock>
              <SummaryBlock label="Services">
                <ul className="space-y-0.5">
                  {appt.services.map((service) => (
                    <li key={service.name} className="flex justify-between gap-3">
                      <span>{service.name}</span>
                      <span className="shrink-0 text-muted-foreground">
                        {formatPHP(service.price)}
                      </span>
                    </li>
                  ))}
                </ul>
              </SummaryBlock>
            </section>
            <p className="mt-3 flex items-start gap-2 text-xs
text-muted-foreground">
              <Info className="mt-0.5 size-3.5 shrink-0 text-primary"
aria-hidden="true" />
              Your personal details, motorcycle, and services stay the same. Only
the date and
              time change.
            </p>

            {supportsSlideTransition && rescheduleForm}
          </div>
        </div>
      </CardContent>

      <AlertDialog open={rescheduleReviewOpen}
onOpenChange={setRescheduleReviewOpen}>
        <AlertDialogContent>
          <AlertDialogHeader>
            <AlertDialogTitle>Submit reschedule request?</AlertDialogTitle>
            <AlertDialogDescription>
              The shop will review your request before anything changes. Your
```

```
current
                appointment stays reserved until then, and a new reference code will
be reserved
                for the new schedule.
              </AlertDialogDescription>
            </AlertDialogHeader>
            <dl className="space-y-2 rounded-lg border border-border/70
bg-muted/30 p-3 text-sm">
              <div className="flex justify-between gap-4">
                <dt className="text-muted-foreground">Current</dt>
                <dd className="text-right">
                  {formatDateLong(appt.date)}, {formatTime(appt.startTime)}
                </dd>
              </div>
              <div className="flex justify-between gap-4">
                <dt className="text-muted-foreground">Requested</dt>
                <dd className="text-right font-medium">
                  {newDate ? formatDateLong(newDate) : "Date not selected"}
                  {newStartTime ? `, ${formatTime(newStartTime)}` : ""}
                </dd>
              </div>
              <div className="flex justify-between gap-4">
                <dt className="text-muted-foreground">Reason</dt>
                <dd className="text-right break-words">
                  {rescheduleReason.trim() || "Not provided"}
                </dd>
              </div>
            </dl>
            <AlertDialogFooter>
              <AlertDialogCancel>Go back and edit</AlertDialogCancel>
              <AlertDialogAction
                disabled={
                  rescheduleMutation.isPending ||
                  (turnstileEnabled && !rescheduleTurnstileToken)
                }
                onClick={submitRescheduleRequest}
              >
                {rescheduleMutation.isPending && <Loader2
className="animate-spin" />}
                Submit reschedule request
              </AlertDialogAction>
            </AlertDialogFooter>
          </AlertDialogContent>
        </AlertDialog>
```

```
          {appt.emailHint && (
            <AppointmentVerificationDialog
              open={verificationRequest !== null}
              action={verificationRequest?.action ?? "cancel"}
              reference={appt.reference}
              emailHint={appt.emailHint}
              schedule={`${formatDateLong(appt.date)} at
${formatTime(appt.startTime)}`}
              notice={verificationRequest?.notice ?? null}
              onOpenChange={(open) => {
                if (!open) setVerificationRequest(null);
              }}
              onVerified={(verification) =>
                handleVerified(verificationRequest?.action ?? "cancel", verification)
              }
            />
          )}
        </Card>
      )}
    </main>
    <SiteFooter />
  </div>
 );
}

function appointmentStatusHint(status: string) {
 switch (status) {
   case "pending":
     return "Waiting for the shop to confirm your request.";
   case "confirmed":
     return "Your appointment is confirmed. Please arrive 10–15 minutes early.";
   case "in_progress":
     return "Your motorcycle is currently being serviced.";
   case "completed":
     return "This service has been completed.";
   case "cancelled":
     return "This appointment has been cancelled.";
   default:
     return null;
 }
}

function RescheduleHeading({ reference }: { reference: string }) {
 return (
   <div className="flex flex-wrap items-start justify-between gap-3">
```

```
      <div>
        <p className="text-xs tracking-[0.3em] text-accent
uppercase">Reschedule appointment</p>
        <h2 className="mt-1 font-display text-2xl uppercase">Choose a new
schedule</h2>
        <p className="mt-1 text-sm text-muted-foreground">
          Your current appointment stays reserved until the shop approves this
request.
        </p>
      </div>
      <Badge variant="outline" className="border-primary/40 bg-primary/5
text-primary uppercase">
        From {reference}
      </Badge>
    </div>
  );
}

function SummaryBlock({ label, children }: { label: string; children:
React.ReactNode }) {
  return (
    <div className="min-w-0">
      <p className="text-[11px] tracking-wider text-muted-foreground
uppercase">{label}</p>
      <div className="mt-1">{children}</div>
    </div>
  );
}

function Notice({
  tone,
  title,
  children,
}: {
  tone: "info" | "accent" | "destructive";
  title?: string;
  children: React.ReactNode;
}) {
  return (
    <div
      role={tone === "destructive" ? "status" : undefined}
      className={cn(
        "rounded-lg border p-3 text-sm text-muted-foreground",
        tone === "info" && "border-primary/30 bg-primary/5",
        tone === "accent" && "border-accent/30 bg-accent/5",
```

```
        tone === "destructive" && "border-destructive/40 bg-destructive/5",
      )}
    >
      {title && (
        <p
          className={cn(
            "mb-1 font-medium",
            tone === "destructive" ? "text-destructive" : "text-foreground",
          )}
        >
          {title}
        </p>
      )}
      {children}
    </div>
  );
}

function Row({ label, value }: { label: string; value: string }) {
  return (
    <div>
      <dt className="text-xs tracking-widest text-muted-foreground
uppercase">{label}</dt>
      <dd className="mt-0.5 text-sm text-foreground">{value}</dd>
    </div>
  );
}
```

src/routes/reset-password.tsx

```
import { useMutation } from "@tanstack/react-query";
import { createFileRoute, Link, useNavigate } from "@tanstack/react-router";
import type { Session } from "@supabase/supabase-js";
import { ArrowLeft, Eye, EyeOff, KeyRound, Loader2 } from "lucide-react";
import { useEffect, useState } from "react";

import logo from "@/assets/logo-shp.png.asset.json";
import { Button } from "@/components/ui/button";
import { Card, CardContent } from "@/components/ui/card";
import { FieldError } from "@/components/ui/field-error";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { supabase } from "@/integrations/supabase/client";

export const Route = createFileRoute("/reset-password")({
  head: () => ({
    meta: [
      { title: "Set a New Password | Fake Rider Motorparts" },
      { name: "robots", content: "noindex" },
    ],
  }),
  ssr: false,
  component: ResetPasswordPage,
});

type Stage = "checking" | "ready" | "invalid";

/** The error Supabase puts in the link when it has expired or was used. */
function linkError() {
  const params = new URLSearchParams(window.location.hash.slice(1) ||
window.location.search);
  return params.get("error_description");
}

/** Whether the session was started by a password reset link. */
function isRecoverySession(session: Session | null) {
  if (!session) return false;
  try {
    const payload = session.access_token.split(".")[1] ?? "";
    const claims = JSON.parse(atob(payload.replace(/-/g, "+").replace(/_/g, "/")))
as {
      amr?: { method?: string }[];
    };
```

```
    return (claims.amr ?? []).some((entry) => entry.method === "recovery");
  } catch {
    return false;
  }
}

function ResetPasswordPage() {
  const navigate = useNavigate();
  const [stage, setStage] = useState<Stage>("checking");
  const [password, setPassword] = useState("");
  const [confirmPassword, setConfirmPassword] = useState("");
  const [showPassword, setShowPassword] = useState(false);
  const [errors, setErrors] = useState<{ password?: string; confirmPassword?:
string }>({});

  useEffect(() => {
    if (linkError()) {
      setStage("invalid");
      return;
    }
    // Opening the reset link signs this tab in and raises
PASSWORD_RECOVERY.
    const {
      data: { subscription },
    } = supabase.auth.onAuthStateChange((event, session) => {
      if (event === "PASSWORD_RECOVERY" && session) setStage("ready");
    });
    let timer: number | undefined;
    void supabase.auth.getSession().then(({ data }) => {
      // The event is sent just after the session is read from the link.
      timer = window.setTimeout(() => {
        setStage((current) =>
          current !== "checking" ? current : isRecoverySession(data.session) ?
"ready" : "invalid",
        );
      }, 0);
    });
    return () => {
      subscription.unsubscribe();
      window.clearTimeout(timer);
    };
  }, []);

  const save = useMutation({
    mutationFn: async () => {
```

```
      const { error } = await supabase.auth.updateUser({ password });
      if (error) throw error;
      // End every session, including any started with the old password.
      await supabase.auth.signOut({ scope: "global" });
    },
    onSuccess: () => {
      navigate({ to: "/auth", search: { reset: "done" }, replace: true });
    },
    onError: (error: Error) => {
      const message = error.message.toLowerCase();
      setErrors({
        password: message.includes("different from the old")
          ? "Choose a password you have not used for this account."
          : message.includes("session") || message.includes("expired")
            ? "This reset link has expired. Request a new one from the login page."
            : error.message || "We could not change the password. Please try
again.",
      });
    },
  });

  function validate() {
    const nextErrors: { password?: string; confirmPassword?: string } = {};
    if (password.length < 8) nextErrors.password = "Your new password needs at
least 8 characters.";
    if (confirmPassword !== password) nextErrors.confirmPassword = "The
passwords do not match.";
    setErrors(nextErrors);
    return Object.keys(nextErrors).length === 0;
  }

  return (
    <div className="flex min-h-screen items-center justify-center bg-background
px-4 py-12">
      <Card className="w-full max-w-md border-border/70 bg-card/70">
        <CardContent className="p-8">
          <img src={logo.url} alt="Fake Rider Motorparts logo" className="mx-auto
h-24 w-auto" />
          <h1 className="mt-4 text-center font-display text-2xl uppercase">Set a
new password</h1>

          {stage === "checking" && (
            <p
              role="status"
              className="mt-6 flex items-center justify-center gap-2 text-sm
```

```
text-muted-foreground"
          >
            <Loader2 className="size-4 animate-spin" aria-hidden="true" />
Checking your reset
            link...
          </p>
        )}

        {stage === "invalid" && (
          <div className="mt-4 space-y-4 text-center">
            <p className="text-sm text-muted-foreground">
              This reset link is invalid or has expired. Links work once; request a new
one from
              the login page.
            </p>
            <Button asChild className="w-full font-display uppercase">
              <Link to="/auth">Back to admin login</Link>
            </Button>
          </div>
        )}

        {stage === "ready" && (
          <form
            className="mt-6 space-y-4"
            onSubmit={(e) => {
              e.preventDefault();
              if (validate()) save.mutate();
            }}
          >
            <p className="text-center text-sm text-muted-foreground">
              Choose a new password. Every device signed in to this account will be
signed out.
            </p>
            <div className="space-y-1.5">
              <Label htmlFor="new-password">New password</Label>
              <div className="relative">
                <Input
                  id="new-password"
                  type={showPassword ? "text" : "password"}
                  autoComplete="new-password"
                  value={password}
                  onChange={(e) => {
                    setPassword(e.target.value);
                    setErrors({});
                  }}
```

```
                aria-invalid={!!errors.password}
                className="pr-11"
              />
              <Button
                type="button"
                variant="ghost"
                size="icon"
                className="absolute top-0 right-0 text-muted-foreground
hover:text-foreground"
                onClick={() => setShowPassword((visible) => !visible)}
                aria-label={showPassword ? "Hide passwords" : "Show passwords"}
                aria-pressed={showPassword}
              >
                {showPassword ? <EyeOff /> : <Eye />}
              </Button>
            </div>
            <FieldError message={errors.password} />
          </div>
          <div className="space-y-1.5">
            <Label htmlFor="confirm-new-password">Confirm new
password</Label>
            <Input
              id="confirm-new-password"
              type={showPassword ? "text" : "password"}
              autoComplete="new-password"
              value={confirmPassword}
              onChange={(e) => {
                setConfirmPassword(e.target.value);
                setErrors({});
              }}
              aria-invalid={!!errors.confirmPassword}
            />
            <FieldError message={errors.confirmPassword} />
          </div>
          <Button
            type="submit"
            className="w-full font-display uppercase"
            disabled={save.isPending}
          >
            {save.isPending ? <Loader2 className="animate-spin" /> :
<KeyRound />} Save new
            password
          </Button>
        </form>
      )}
```

```
          <Button asChild variant="ghost" className="mt-4 w-full justify-center
text-sm">
            <Link to="/">
              <ArrowLeft /> Back to home
            </Link>
          </Button>
        </CardContent>
      </Card>
    </div>
  );
}
```

src/routes/services.tsx

```
import { useQuery } from "@tanstack/react-query";
import { createFileRoute, Link } from "@tanstack/react-router";
import { useEffect, useRef, useState } from "react";

import { ServiceBookingRow } from "@/components/site/service-booking-row";
import { SiteFooter } from "@/components/site/site-footer";
import { SiteHeader } from "@/components/site/site-header";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Card, CardContent } from "@/components/ui/card";
import { ToggleGroup, ToggleGroupItem } from "@/components/ui/toggle-group";
import { supabase } from "@/integrations/supabase/client";

export const Route = createFileRoute("/services")({
  head: () => ({
    meta: [
      { title: "Motorcycle Services | Fake Rider" },
      {
        name: "description",
        content:
          "PMS, change oil, suspension tuning, brakes, tires and engine work at
Fake Rider Motorparts. Book online.",
      },
      { property: "og:title", content: "Services | Fake Rider Motorparts" },
      {
        property: "og:description",
        content: "Every motorcycle service we offer, ready to book online.",
      },
    ],
  }),
  component: ServicesPage,
});

function ServicesPage() {
  const [activeCategory, setActiveCategory] = useState("all");
  const services = useQuery({
    queryKey: ["services"],
    queryFn: async () => {
      const { data, error } = await supabase
        .from("services")
        .select("*")
        .eq("is_active", true)
        .eq("is_archived", false)
```

```
        .order("sort_order");
      if (error) throw error;
      return Array.from(new Map((data ?? []).map((s) => [s.name.trim(),
s])).values());
    },
  });

  const categories = ["all", ...new Set(services.data?.map((s) => s.category))];
  const filtered = services.data?.filter((s) => {
    if (activeCategory === "all") return true;
    return s.category === activeCategory;
  });

  return (
    <div className="min-h-screen">
      <SiteHeader />
      <main className="site-container py-12">
        <p className="text-xs tracking-[0.3em] text-orange-700 uppercase
dark:text-accent">
          Service menu
        </p>
        <h1 className="font-display text-4xl font-bold uppercase
md:text-5xl">Services</h1>
        <p className="mt-2 max-w-2xl text-muted-foreground">
          Browse our services and tap Book to reserve one. The booking form
shows the price for your
          motorcycle. Final details depend on the actual condition of your unit.
        </p>

        <div className="mt-8">
          <div className="mb-4">
            <ToggleGroup
              type="single"
              variant="outline"
              value={activeCategory}
              onValueChange={setActiveCategory}
              className="flex w-full flex-wrap justify-start gap-1.5"
            >
              {categories.map((cat) => {
                const count = services.data?.filter(
                  (s) => cat === "all" || s.category === cat,
                ).length;
                return (
                  <ToggleGroupItem key={cat} value={cat} className="capitalize">
                    {cat === "all" ? "All" : cat}
```

```
                <Badge
                  variant={activeCategory === cat ? "default" : "secondary"}
                  className="ml-1.5 h-5 px-1.5 text-xs font-medium"
                >
                  {count}
                </Badge>
              </ToggleGroupItem>
            );
          })}
        </ToggleGroup>
      </div>

      <div className="grid gap-4 sm:grid-cols-2 lg:grid-cols-3">
        {filtered?.map((s) => (
          <Card key={s.id} className="border-border/70 bg-card/60">
            <CardContent className="flex h-full flex-col p-5">
              <div className="flex items-center gap-2">
                <Badge
                  variant="outline"
                  className="font-display text-[0.55rem] font-medium uppercase"
                >
                  {s.category}
                </Badge>
                <h2 className="font-display text-lg tracking-wide
uppercase">{s.name}</h2>
              </div>
              <ExpandableServiceDescription description={s.description}
serviceName={s.name} />
              <ServiceBookingRow service={s} />
            </CardContent>
          </Card>
        ))}
      </div>
    </div>

    <div className="mt-12 rounded-2xl border border-primary/30 bg-primary/5
p-8 text-center">
      <h2 className="font-display text-2xl uppercase">Ready to book your
slot?</h2>
      <p className="mt-2 text-muted-foreground">
        Schedules open 48 hours ahead. Reserve now and get your reference
code.
      </p>
      <Button asChild size="lg" className="mt-5 font-display tracking-wide
uppercase">
```

```
            <Link to="/book">Book an appointment</Link>
          </Button>
        </div>
      </main>
      <SiteFooter />
    </div>
  );
}

function ExpandableServiceDescription({
  description,
  serviceName,
}: {
  description: string | null;
  serviceName: string;
}) {
  const descriptionRef = useRef<HTMLParagraphElement>(null);
  const [expanded, setExpanded] = useState(false);
  const [canExpand, setCanExpand] = useState(false);
  const details = description?.trim() ?? "";

  useEffect(() => {
    const element = descriptionRef.current;
    if (!element || !details) return;

    const checkOverflow = () => {
      if (!expanded) setCanExpand(element.scrollHeight > element.clientHeight +
1);
    };

    checkOverflow();
    if (typeof ResizeObserver === "undefined") return;

    const observer = new ResizeObserver(checkOverflow);
    observer.observe(element);
    return () => observer.disconnect();
  }, [details, expanded]);

  if (!details) return null;

  return (
    <div className="mt-2 text-sm text-muted-foreground">
      <p ref={descriptionRef} className={expanded ? "break-words" :
"line-clamp-3 break-words"}>
        {details}
```

```
      </p>
      {canExpand && (
        <Button
          type="button"
          variant="link"
          size="sm"
          className="mt-1 h-auto p-0 text-sm"
          onClick={() => setExpanded((current) => !current)}
          aria-expanded={expanded}
          aria-label={`${expanded ? "Collapse" : "Expand"} details for
${serviceName}`}
        >
          {expanded ? "See Less" : "See More"}
        </Button>
      )}
    </div>
  );
}
```

src/routes/shop.tsx

```
import { useQuery } from "@tanstack/react-query";
import { createFileRoute } from "@tanstack/react-router";
import { useMemo, useState } from "react";

import { ProductCard } from "@/components/site/product-card";
import { SiteFooter } from "@/components/site/site-footer";
import { SiteHeader } from "@/components/site/site-header";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Tabs, TabsList, TabsTrigger } from "@/components/ui/tabs";
import { supabase } from "@/integrations/supabase/client";

export const Route = createFileRoute("/shop")({
  head: () => ({
    meta: [
      { title: "Shop Parts & Accessories | Fake Rider" },
      {
        name: "description",
        content:
          "Browse genuine motorcycle parts and riding accessories available at Fake
Rider Motorparts.",
      },
      { property: "og:title", content: "Fake Rider Motorparts Shop" },
      {
        property: "og:description",
        content: "Parts and accessories available in store.",
      },
    ],
  }),
  component: ShopPage,
});

const tabs = [
  { value: "all", label: "All" },
  { value: "part", label: "Parts" },
  { value: "accessory", label: "Accessories" },
];

function ShopPage() {
  const [tab, setTab] = useState("all");
  const [term, setTerm] = useState("");
  const [searchTerm, setSearchTerm] = useState("");
```

```
  const products = useQuery({
    queryKey: ["products"],
    queryFn: async () => {
      const { data, error } = await supabase
        .from("products")
        .select("*")
        .eq("is_active", true)
        .eq("is_archived", false)
        .in("category", ["part", "accessory"])
        .order("sort_order");
      if (error) throw error;
      return data ?? [];
    },
  });

  const normalizedSearchTerm = searchTerm.trim().toLocaleLowerCase();
  const list = useMemo(
    () =>
      (products.data ?? []).filter(
        (product) =>
          (tab === "all" || product.category === tab) &&
          (normalizedSearchTerm === "" ||
            `${product.name} ${product.brand ?? ""} ${product.description ?? ""}`
              .toLocaleLowerCase()
              .includes(normalizedSearchTerm)),
      ),
    [normalizedSearchTerm, products.data, tab],
  );

  function clearSearch() {
    setTerm("");
    setSearchTerm("");
  }

  return (
    <div className="min-h-screen lg:flex lg:min-w-0 lg:flex-col lg:overflow-x-clip">
      <SiteHeader />
      <main className="site-container py-12 lg:min-w-0 lg:flex-1">
        <p className="text-xs tracking-[0.3em] text-accent uppercase">Shop
showcase</p>
        <h1 className="font-display text-4xl font-bold uppercase md:text-5xl">
          Parts & Accessories
        </h1>
        <p className="mt-2 max-w-2xl text-muted-foreground">
          Everything we stock in the garage. Prices are in Philippine peso and may
```

```
change without
        prior notice &mdash; use Reserve via Messenger on an item card to ask
the shop to hold it.
      </p>

      <div className="mt-8 flex flex-wrap items-center justify-between gap-4">
        <Tabs value={tab} onValueChange={setTab}>
          <TabsList>
            {tabs.map((t) => (
              <TabsTrigger key={t.value} value={t.value} className="font-display
uppercase">
                {t.label}
              </TabsTrigger>
            ))}
          </TabsList>
        </Tabs>
        <form
          className="flex w-full gap-2 sm:w-auto"
          onSubmit={(event) => {
            event.preventDefault();
            setSearchTerm(term);
          }}
        >
          <Input
            value={term}
            onChange={(event) => {
              const value = event.target.value;
              setTerm(value);
              if (!value.trim()) setSearchTerm("");
            }}
            placeholder="Search item or brand"
            aria-label="Search shop items"
            className="min-w-0 flex-1 sm:w-64"
          />
          <Button type="submit" className="font-display uppercase">
            Search
          </Button>
          {(term || searchTerm) && (
            <Button type="button" variant="outline" onClick={clearSearch}>
              Clear
            </Button>
          )}
        </form>
      </div>
```

```
        <div className="mt-8 grid gap-5 sm:grid-cols-2 lg:min-w-0 lg:grid-cols-4">
          {list.map((p) => (
            <ProductCard key={p.id} product={p} />
          ))}
        </div>
        {!products.isLoading && list.length === 0 && (
          <p className="py-16 text-center text-muted-foreground">No items match
your search.</p>
        )}
      </main>
      <SiteFooter className="lg:mt-auto" />
    </div>
  );
}
```

src/routeTree.gen.ts

```
/* eslint-disable */

// @ts-nocheck

// noinspection JSUnusedGlobalSymbols

// This file was automatically generated by TanStack Router.
// You should NOT make any changes in this file as it will be overwritten.
// Additionally, you should also exclude this file from your linter and/or formatter to prevent it from being checked or modified.

import { Route as rootRouteImport } from './routes/__root'
import { Route as IndexRouteImport } from './routes/index'
import { Route as AuthenticatedRouteRouteImport } from './routes/_authenticated/route'
import { Route as AuthRouteImport } from './routes/auth'
import { Route as BookRouteImport } from './routes/book'
import { Route as MyAppointmentRouteImport } from './routes/my-appointment'
import { Route as ResetPasswordRouteImport } from './routes/reset-password'
import { Route as ServicesRouteImport } from './routes/services'
import { Route as ShopRouteImport } from './routes/shop'
import { Route as AuthenticatedAdminRouteImport } from './routes/_authenticated/admin'
import { Route as ApiOfflineStatusRouteImport } from './routes/api/offline-status'
import { Route as ApiOfflineSyncRouteImport } from './routes/api/offline-sync'
import { Route as ApiScheduledNotificationsRouteImport } from './routes/api/scheduled-notifications'
import { Route as AuthenticatedAdminIndexRouteImport } from './routes/_authenticated/admin/index'
import { Route as AuthenticatedAdminActivityLogRouteImport } from './routes/_authenticated/admin/activity-log'
import { Route as AuthenticatedAdminAppointmentsRouteImport } from './routes/_authenticated/admin/appointments'
import { Route as AuthenticatedAdminArchiveRouteImport } from './routes/_authenticated/admin/archive'
import { Route as AuthenticatedAdminAvailabilityRouteImport } from './routes/_authenticated/admin/availability'
import { Route as AuthenticatedAdminBlockedNumbersRouteImport } from './routes/_authenticated/admin/blocked-numbers'
import { Route as AuthenticatedAdminCustomersRouteImport } from './routes/_authenticated/admin/customers'
import { Route as AuthenticatedAdminMechanicsRouteImport } from './routes/_authenticated/admin/mechanics'
```

```
import { Route as AuthenticatedAdminMotorcyclesRouteImport } from
'./routes/_authenticated/admin/motorcycles'
import { Route as AuthenticatedAdminNotificationsRouteImport } from
'./routes/_authenticated/admin/notifications'
import { Route as AuthenticatedAdminPartsRouteImport } from
'./routes/_authenticated/admin/parts'
import { Route as AuthenticatedAdminPricesRouteImport } from
'./routes/_authenticated/admin/prices'
import { Route as AuthenticatedAdminReportsRouteImport } from
'./routes/_authenticated/admin/reports'
import { Route as AuthenticatedAdminScheduleBlocksRouteImport } from
'./routes/_authenticated/admin/schedule-blocks'
import { Route as AuthenticatedAdminServicesRouteImport } from
'./routes/_authenticated/admin/services'
import { Route as AuthenticatedAdminSettingsRouteImport } from
'./routes/_authenticated/admin/settings'

const IndexRoute = IndexRouteImport.update({
  id: '/',
  path: '/',
  getParentRoute: () => rootRouteImport,
} as any)
const AuthenticatedRouteRoute = AuthenticatedRouteRouteImport.update({
  id: '/_authenticated',
  getParentRoute: () => rootRouteImport,
} as any)
const AuthRoute = AuthRouteImport.update({
  id: '/auth',
  path: '/auth',
  getParentRoute: () => rootRouteImport,
} as any)
const BookRoute = BookRouteImport.update({
  id: '/book',
  path: '/book',
  getParentRoute: () => rootRouteImport,
} as any)
const MyAppointmentRoute = MyAppointmentRouteImport.update({
  id: '/my-appointment',
  path: '/my-appointment',
  getParentRoute: () => rootRouteImport,
} as any)
const ResetPasswordRoute = ResetPasswordRouteImport.update({
  id: '/reset-password',
  path: '/reset-password',
  getParentRoute: () => rootRouteImport,
```

```
} as any)
const ServicesRoute = ServicesRouteImport.update({
  id: '/services',
  path: '/services',
  getParentRoute: () => rootRouteImport,
} as any)
const ShopRoute = ShopRouteImport.update({
  id: '/shop',
  path: '/shop',
  getParentRoute: () => rootRouteImport,
} as any)
const AuthenticatedAdminRoute = AuthenticatedAdminRouteImport.update({
  id: '/admin',
  path: '/admin',
  getParentRoute: () => AuthenticatedRouteRoute,
} as any)
const ApiOfflineStatusRoute = ApiOfflineStatusRouteImport.update({
  id: '/api/offline-status',
  path: '/api/offline-status',
  getParentRoute: () => rootRouteImport,
} as any)
const ApiOfflineSyncRoute = ApiOfflineSyncRouteImport.update({
  id: '/api/offline-sync',
  path: '/api/offline-sync',
  getParentRoute: () => rootRouteImport,
} as any)
const ApiScheduledNotificationsRoute =
  ApiScheduledNotificationsRouteImport.update({
    id: '/api/scheduled-notifications',
    path: '/api/scheduled-notifications',
    getParentRoute: () => rootRouteImport,
  } as any)
const AuthenticatedAdminIndexRoute =
AuthenticatedAdminIndexRouteImport.update({
  id: '/',
  path: '/',
  getParentRoute: () => AuthenticatedAdminRoute,
} as any)
const AuthenticatedAdminActivityLogRoute =
  AuthenticatedAdminActivityLogRouteImport.update({
    id: '/activity-log',
    path: '/activity-log',
    getParentRoute: () => AuthenticatedAdminRoute,
  } as any)
const AuthenticatedAdminAppointmentsRoute =
```

```
  AuthenticatedAdminAppointmentsRouteImport.update({
    id: '/appointments',
    path: '/appointments',
    getParentRoute: () => AuthenticatedAdminRoute,
  } as any)
const AuthenticatedAdminArchiveRoute =
  AuthenticatedAdminArchiveRouteImport.update({
    id: '/archive',
    path: '/archive',
    getParentRoute: () => AuthenticatedAdminRoute,
  } as any)
const AuthenticatedAdminAvailabilityRoute =
  AuthenticatedAdminAvailabilityRouteImport.update({
    id: '/availability',
    path: '/availability',
    getParentRoute: () => AuthenticatedAdminRoute,
  } as any)
const AuthenticatedAdminBlockedNumbersRoute =
  AuthenticatedAdminBlockedNumbersRouteImport.update({
    id: '/blocked-numbers',
    path: '/blocked-numbers',
    getParentRoute: () => AuthenticatedAdminRoute,
  } as any)
const AuthenticatedAdminCustomersRoute =
  AuthenticatedAdminCustomersRouteImport.update({
    id: '/customers',
    path: '/customers',
    getParentRoute: () => AuthenticatedAdminRoute,
  } as any)
const AuthenticatedAdminMechanicsRoute =
  AuthenticatedAdminMechanicsRouteImport.update({
    id: '/mechanics',
    path: '/mechanics',
    getParentRoute: () => AuthenticatedAdminRoute,
  } as any)
const AuthenticatedAdminMotorcyclesRoute =
  AuthenticatedAdminMotorcyclesRouteImport.update({
    id: '/motorcycles',
    path: '/motorcycles',
    getParentRoute: () => AuthenticatedAdminRoute,
  } as any)
const AuthenticatedAdminNotificationsRoute =
  AuthenticatedAdminNotificationsRouteImport.update({
    id: '/notifications',
    path: '/notifications',
```

```
    getParentRoute: () => AuthenticatedAdminRoute,
  } as any)
const AuthenticatedAdminPartsRoute =
AuthenticatedAdminPartsRouteImport.update({
  id: '/parts',
  path: '/parts',
  getParentRoute: () => AuthenticatedAdminRoute,
} as any)
const AuthenticatedAdminPricesRoute =
  AuthenticatedAdminPricesRouteImport.update({
    id: '/prices',
    path: '/prices',
    getParentRoute: () => AuthenticatedAdminRoute,
  } as any)
const AuthenticatedAdminReportsRoute =
  AuthenticatedAdminReportsRouteImport.update({
    id: '/reports',
    path: '/reports',
    getParentRoute: () => AuthenticatedAdminRoute,
  } as any)
const AuthenticatedAdminScheduleBlocksRoute =
  AuthenticatedAdminScheduleBlocksRouteImport.update({
    id: '/schedule-blocks',
    path: '/schedule-blocks',
    getParentRoute: () => AuthenticatedAdminRoute,
  } as any)
const AuthenticatedAdminServicesRoute =
  AuthenticatedAdminServicesRouteImport.update({
    id: '/services',
    path: '/services',
    getParentRoute: () => AuthenticatedAdminRoute,
  } as any)
const AuthenticatedAdminSettingsRoute =
  AuthenticatedAdminSettingsRouteImport.update({
    id: '/settings',
    path: '/settings',
    getParentRoute: () => AuthenticatedAdminRoute,
  } as any)

export interface FileRoutesByFullPath {
  '/': typeof IndexRoute
  '/auth': typeof AuthRoute
  '/book': typeof BookRoute
  '/my-appointment': typeof MyAppointmentRoute
  '/reset-password': typeof ResetPasswordRoute
```

```
  '/services': typeof ServicesRoute
  '/shop': typeof ShopRoute
  '/admin': typeof AuthenticatedAdminRouteWithChildren
  '/api/offline-status': typeof ApiOfflineStatusRoute
  '/api/offline-sync': typeof ApiOfflineSyncRoute
  '/api/scheduled-notifications': typeof ApiScheduledNotificationsRoute
  '/admin/activity-log': typeof AuthenticatedAdminActivityLogRoute
  '/admin/appointments': typeof AuthenticatedAdminAppointmentsRoute
  '/admin/archive': typeof AuthenticatedAdminArchiveRoute
  '/admin/availability': typeof AuthenticatedAdminAvailabilityRoute
  '/admin/blocked-numbers': typeof AuthenticatedAdminBlockedNumbersRoute
  '/admin/customers': typeof AuthenticatedAdminCustomersRoute
  '/admin/mechanics': typeof AuthenticatedAdminMechanicsRoute
  '/admin/motorcycles': typeof AuthenticatedAdminMotorcyclesRoute
  '/admin/notifications': typeof AuthenticatedAdminNotificationsRoute
  '/admin/parts': typeof AuthenticatedAdminPartsRoute
  '/admin/prices': typeof AuthenticatedAdminPricesRoute
  '/admin/reports': typeof AuthenticatedAdminReportsRoute
  '/admin/schedule-blocks': typeof AuthenticatedAdminScheduleBlocksRoute
  '/admin/services': typeof AuthenticatedAdminServicesRoute
  '/admin/settings': typeof AuthenticatedAdminSettingsRoute
  '/admin/': typeof AuthenticatedAdminIndexRoute
}
export interface FileRoutesByTo {
  '/': typeof IndexRoute
  '/auth': typeof AuthRoute
  '/book': typeof BookRoute
  '/my-appointment': typeof MyAppointmentRoute
  '/reset-password': typeof ResetPasswordRoute
  '/services': typeof ServicesRoute
  '/shop': typeof ShopRoute
  '/api/offline-status': typeof ApiOfflineStatusRoute
  '/api/offline-sync': typeof ApiOfflineSyncRoute
  '/api/scheduled-notifications': typeof ApiScheduledNotificationsRoute
  '/admin/activity-log': typeof AuthenticatedAdminActivityLogRoute
  '/admin/appointments': typeof AuthenticatedAdminAppointmentsRoute
  '/admin/archive': typeof AuthenticatedAdminArchiveRoute
  '/admin/availability': typeof AuthenticatedAdminAvailabilityRoute
  '/admin/blocked-numbers': typeof AuthenticatedAdminBlockedNumbersRoute
  '/admin/customers': typeof AuthenticatedAdminCustomersRoute
  '/admin/mechanics': typeof AuthenticatedAdminMechanicsRoute
  '/admin/motorcycles': typeof AuthenticatedAdminMotorcyclesRoute
  '/admin/notifications': typeof AuthenticatedAdminNotificationsRoute
  '/admin/parts': typeof AuthenticatedAdminPartsRoute
  '/admin/prices': typeof AuthenticatedAdminPricesRoute
```

```
  '/admin/reports': typeof AuthenticatedAdminReportsRoute
  '/admin/schedule-blocks': typeof AuthenticatedAdminScheduleBlocksRoute
  '/admin/services': typeof AuthenticatedAdminServicesRoute
  '/admin/settings': typeof AuthenticatedAdminSettingsRoute
  '/admin': typeof AuthenticatedAdminIndexRoute
}
export interface FileRoutesById {
  __root__: typeof rootRouteImport
  '/': typeof IndexRoute
  '/_authenticated': typeof AuthenticatedRouteRouteWithChildren
  '/auth': typeof AuthRoute
  '/book': typeof BookRoute
  '/my-appointment': typeof MyAppointmentRoute
  '/reset-password': typeof ResetPasswordRoute
  '/services': typeof ServicesRoute
  '/shop': typeof ShopRoute
  '/_authenticated/admin': typeof AuthenticatedAdminRouteWithChildren
  '/api/offline-status': typeof ApiOfflineStatusRoute
  '/api/offline-sync': typeof ApiOfflineSyncRoute
  '/api/scheduled-notifications': typeof ApiScheduledNotificationsRoute
  '/_authenticated/admin/activity-log': typeof AuthenticatedAdminActivityLogRoute
  '/_authenticated/admin/appointments': typeof
AuthenticatedAdminAppointmentsRoute
  '/_authenticated/admin/archive': typeof AuthenticatedAdminArchiveRoute
  '/_authenticated/admin/availability': typeof AuthenticatedAdminAvailabilityRoute
  '/_authenticated/admin/blocked-numbers': typeof
AuthenticatedAdminBlockedNumbersRoute
  '/_authenticated/admin/customers': typeof AuthenticatedAdminCustomersRoute
  '/_authenticated/admin/mechanics': typeof AuthenticatedAdminMechanicsRoute
  '/_authenticated/admin/motorcycles': typeof
AuthenticatedAdminMotorcyclesRoute
  '/_authenticated/admin/notifications': typeof
AuthenticatedAdminNotificationsRoute
  '/_authenticated/admin/parts': typeof AuthenticatedAdminPartsRoute
  '/_authenticated/admin/prices': typeof AuthenticatedAdminPricesRoute
  '/_authenticated/admin/reports': typeof AuthenticatedAdminReportsRoute
  '/_authenticated/admin/schedule-blocks': typeof
AuthenticatedAdminScheduleBlocksRoute
  '/_authenticated/admin/services': typeof AuthenticatedAdminServicesRoute
  '/_authenticated/admin/settings': typeof AuthenticatedAdminSettingsRoute
  '/_authenticated/admin/': typeof AuthenticatedAdminIndexRoute
}
export interface FileRouteTypes {
  fileRoutesByFullPath: FileRoutesByFullPath
  fullPaths:
```

```
  | '/'
  | '/auth'
  | '/book'
  | '/my-appointment'
  | '/reset-password'
  | '/services'
  | '/shop'
  | '/admin'
  | '/api/offline-status'
  | '/api/offline-sync'
  | '/api/scheduled-notifications'
  | '/admin/activity-log'
  | '/admin/appointments'
  | '/admin/archive'
  | '/admin/availability'
  | '/admin/blocked-numbers'
  | '/admin/customers'
  | '/admin/mechanics'
  | '/admin/motorcycles'
  | '/admin/notifications'
  | '/admin/parts'
  | '/admin/prices'
  | '/admin/reports'
  | '/admin/schedule-blocks'
  | '/admin/services'
  | '/admin/settings'
  | '/admin/'
fileRoutesByTo: FileRoutesByTo
to:
  | '/'
  | '/auth'
  | '/book'
  | '/my-appointment'
  | '/reset-password'
  | '/services'
  | '/shop'
  | '/api/offline-status'
  | '/api/offline-sync'
  | '/api/scheduled-notifications'
  | '/admin/activity-log'
  | '/admin/appointments'
  | '/admin/archive'
  | '/admin/availability'
  | '/admin/blocked-numbers'
  | '/admin/customers'
```

```
    | '/admin/mechanics'
    | '/admin/motorcycles'
    | '/admin/notifications'
    | '/admin/parts'
    | '/admin/prices'
    | '/admin/reports'
    | '/admin/schedule-blocks'
    | '/admin/services'
    | '/admin/settings'
    | '/admin'
  id:
    | '__root__'
    | '/'
    | '/_authenticated'
    | '/auth'
    | '/book'
    | '/my-appointment'
    | '/reset-password'
    | '/services'
    | '/shop'
    | '/_authenticated/admin'
    | '/api/offline-status'
    | '/api/offline-sync'
    | '/api/scheduled-notifications'
    | '/_authenticated/admin/activity-log'
    | '/_authenticated/admin/appointments'
    | '/_authenticated/admin/archive'
    | '/_authenticated/admin/availability'
    | '/_authenticated/admin/blocked-numbers'
    | '/_authenticated/admin/customers'
    | '/_authenticated/admin/mechanics'
    | '/_authenticated/admin/motorcycles'
    | '/_authenticated/admin/notifications'
    | '/_authenticated/admin/parts'
    | '/_authenticated/admin/prices'
    | '/_authenticated/admin/reports'
    | '/_authenticated/admin/schedule-blocks'
    | '/_authenticated/admin/services'
    | '/_authenticated/admin/settings'
    | '/_authenticated/admin/'
  fileRoutesById: FileRoutesById
}
export interface RootRouteChildren {
  IndexRoute: typeof IndexRoute
  AuthenticatedRouteRoute: typeof AuthenticatedRouteRouteWithChildren
```

```
  AuthRoute: typeof AuthRoute
  BookRoute: typeof BookRoute
  MyAppointmentRoute: typeof MyAppointmentRoute
  ResetPasswordRoute: typeof ResetPasswordRoute
  ServicesRoute: typeof ServicesRoute
  ShopRoute: typeof ShopRoute
  ApiOfflineStatusRoute: typeof ApiOfflineStatusRoute
  ApiOfflineSyncRoute: typeof ApiOfflineSyncRoute
  ApiScheduledNotificationsRoute: typeof ApiScheduledNotificationsRoute
}

declare module '@tanstack/react-router' {
  interface FileRoutesByPath {
    '/': {
      id: '/'
      path: '/'
      fullPath: '/'
      preLoaderRoute: typeof IndexRouteImport
      parentRoute: typeof rootRouteImport
    }
    '/_authenticated': {
      id: '/_authenticated'
      path: ''
      fullPath: '/'
      preLoaderRoute: typeof AuthenticatedRouteRouteImport
      parentRoute: typeof rootRouteImport
    }
    '/auth': {
      id: '/auth'
      path: '/auth'
      fullPath: '/auth'
      preLoaderRoute: typeof AuthRouteImport
      parentRoute: typeof rootRouteImport
    }
    '/book': {
      id: '/book'
      path: '/book'
      fullPath: '/book'
      preLoaderRoute: typeof BookRouteImport
      parentRoute: typeof rootRouteImport
    }
    '/my-appointment': {
      id: '/my-appointment'
      path: '/my-appointment'
      fullPath: '/my-appointment'
```

```
  preLoaderRoute: typeof MyAppointmentRouteImport
  parentRoute: typeof rootRouteImport
}
'/reset-password': {
  id: '/reset-password'
  path: '/reset-password'
  fullPath: '/reset-password'
  preLoaderRoute: typeof ResetPasswordRouteImport
  parentRoute: typeof rootRouteImport
}
'/services': {
  id: '/services'
  path: '/services'
  fullPath: '/services'
  preLoaderRoute: typeof ServicesRouteImport
  parentRoute: typeof rootRouteImport
}
'/shop': {
  id: '/shop'
  path: '/shop'
  fullPath: '/shop'
  preLoaderRoute: typeof ShopRouteImport
  parentRoute: typeof rootRouteImport
}
'/_authenticated/admin': {
  id: '/_authenticated/admin'
  path: '/admin'
  fullPath: '/admin'
  preLoaderRoute: typeof AuthenticatedAdminRouteImport
  parentRoute: typeof AuthenticatedRouteRoute
}
'/api/offline-status': {
  id: '/api/offline-status'
  path: '/api/offline-status'
  fullPath: '/api/offline-status'
  preLoaderRoute: typeof ApiOfflineStatusRouteImport
  parentRoute: typeof rootRouteImport
}
'/api/offline-sync': {
  id: '/api/offline-sync'
  path: '/api/offline-sync'
  fullPath: '/api/offline-sync'
  preLoaderRoute: typeof ApiOfflineSyncRouteImport
  parentRoute: typeof rootRouteImport
}
```

```
'/api/scheduled-notifications': {
  id: '/api/scheduled-notifications'
  path: '/api/scheduled-notifications'
  fullPath: '/api/scheduled-notifications'
  preLoaderRoute: typeof ApiScheduledNotificationsRouteImport
  parentRoute: typeof rootRouteImport
}
'/_authenticated/admin/': {
  id: '/_authenticated/admin/'
  path: '/'
  fullPath: '/admin/'
  preLoaderRoute: typeof AuthenticatedAdminIndexRouteImport
  parentRoute: typeof AuthenticatedAdminRoute
}
'/_authenticated/admin/activity-log': {
  id: '/_authenticated/admin/activity-log'
  path: '/activity-log'
  fullPath: '/admin/activity-log'
  preLoaderRoute: typeof AuthenticatedAdminActivityLogRouteImport
  parentRoute: typeof AuthenticatedAdminRoute
}
'/_authenticated/admin/appointments': {
  id: '/_authenticated/admin/appointments'
  path: '/appointments'
  fullPath: '/admin/appointments'
  preLoaderRoute: typeof AuthenticatedAdminAppointmentsRouteImport
  parentRoute: typeof AuthenticatedAdminRoute
}
'/_authenticated/admin/archive': {
  id: '/_authenticated/admin/archive'
  path: '/archive'
  fullPath: '/admin/archive'
  preLoaderRoute: typeof AuthenticatedAdminArchiveRouteImport
  parentRoute: typeof AuthenticatedAdminRoute
}
'/_authenticated/admin/availability': {
  id: '/_authenticated/admin/availability'
  path: '/availability'
  fullPath: '/admin/availability'
  preLoaderRoute: typeof AuthenticatedAdminAvailabilityRouteImport
  parentRoute: typeof AuthenticatedAdminRoute
}
'/_authenticated/admin/blocked-numbers': {
  id: '/_authenticated/admin/blocked-numbers'
  path: '/blocked-numbers'
```

```
    fullPath: '/admin/blocked-numbers'
    preLoaderRoute: typeof AuthenticatedAdminBlockedNumbersRouteImport
    parentRoute: typeof AuthenticatedAdminRoute
  }
  '/_authenticated/admin/customers': {
    id: '/_authenticated/admin/customers'
    path: '/customers'
    fullPath: '/admin/customers'
    preLoaderRoute: typeof AuthenticatedAdminCustomersRouteImport
    parentRoute: typeof AuthenticatedAdminRoute
  }
  '/_authenticated/admin/mechanics': {
    id: '/_authenticated/admin/mechanics'
    path: '/mechanics'
    fullPath: '/admin/mechanics'
    preLoaderRoute: typeof AuthenticatedAdminMechanicsRouteImport
    parentRoute: typeof AuthenticatedAdminRoute
  }
  '/_authenticated/admin/motorcycles': {
    id: '/_authenticated/admin/motorcycles'
    path: '/motorcycles'
    fullPath: '/admin/motorcycles'
    preLoaderRoute: typeof AuthenticatedAdminMotorcyclesRouteImport
    parentRoute: typeof AuthenticatedAdminRoute
  }
  '/_authenticated/admin/notifications': {
    id: '/_authenticated/admin/notifications'
    path: '/notifications'
    fullPath: '/admin/notifications'
    preLoaderRoute: typeof AuthenticatedAdminNotificationsRouteImport
    parentRoute: typeof AuthenticatedAdminRoute
  }
  '/_authenticated/admin/parts': {
    id: '/_authenticated/admin/parts'
    path: '/parts'
    fullPath: '/admin/parts'
    preLoaderRoute: typeof AuthenticatedAdminPartsRouteImport
    parentRoute: typeof AuthenticatedAdminRoute
  }
  '/_authenticated/admin/prices': {
    id: '/_authenticated/admin/prices'
    path: '/prices'
    fullPath: '/admin/prices'
    preLoaderRoute: typeof AuthenticatedAdminPricesRouteImport
    parentRoute: typeof AuthenticatedAdminRoute
```

```
    }
    '/_authenticated/admin/reports': {
      id: '/_authenticated/admin/reports'
      path: '/reports'
      fullPath: '/admin/reports'
      preLoaderRoute: typeof AuthenticatedAdminReportsRouteImport
      parentRoute: typeof AuthenticatedAdminRoute
    }
    '/_authenticated/admin/schedule-blocks': {
      id: '/_authenticated/admin/schedule-blocks'
      path: '/schedule-blocks'
      fullPath: '/admin/schedule-blocks'
      preLoaderRoute: typeof AuthenticatedAdminScheduleBlocksRouteImport
      parentRoute: typeof AuthenticatedAdminRoute
    }
    '/_authenticated/admin/services': {
      id: '/_authenticated/admin/services'
      path: '/services'
      fullPath: '/admin/services'
      preLoaderRoute: typeof AuthenticatedAdminServicesRouteImport
      parentRoute: typeof AuthenticatedAdminRoute
    }
    '/_authenticated/admin/settings': {
      id: '/_authenticated/admin/settings'
      path: '/settings'
      fullPath: '/admin/settings'
      preLoaderRoute: typeof AuthenticatedAdminSettingsRouteImport
      parentRoute: typeof AuthenticatedAdminRoute
    }
  }
}

interface AuthenticatedAdminRouteChildren {
  AuthenticatedAdminActivityLogRoute: typeof
AuthenticatedAdminActivityLogRoute
  AuthenticatedAdminAppointmentsRoute: typeof
AuthenticatedAdminAppointmentsRoute
  AuthenticatedAdminArchiveRoute: typeof AuthenticatedAdminArchiveRoute
  AuthenticatedAdminAvailabilityRoute: typeof
AuthenticatedAdminAvailabilityRoute
  AuthenticatedAdminBlockedNumbersRoute: typeof
AuthenticatedAdminBlockedNumbersRoute
  AuthenticatedAdminCustomersRoute: typeof
AuthenticatedAdminCustomersRoute
  AuthenticatedAdminMechanicsRoute: typeof
```

```
AuthenticatedAdminMechanicsRoute
  AuthenticatedAdminMotorcyclesRoute: typeof
AuthenticatedAdminMotorcyclesRoute
  AuthenticatedAdminNotificationsRoute: typeof
AuthenticatedAdminNotificationsRoute
  AuthenticatedAdminPartsRoute: typeof AuthenticatedAdminPartsRoute
  AuthenticatedAdminPricesRoute: typeof AuthenticatedAdminPricesRoute
  AuthenticatedAdminReportsRoute: typeof AuthenticatedAdminReportsRoute
  AuthenticatedAdminScheduleBlocksRoute: typeof
AuthenticatedAdminScheduleBlocksRoute
  AuthenticatedAdminServicesRoute: typeof AuthenticatedAdminServicesRoute
  AuthenticatedAdminSettingsRoute: typeof AuthenticatedAdminSettingsRoute
  AuthenticatedAdminIndexRoute: typeof AuthenticatedAdminIndexRoute
}

const AuthenticatedAdminRouteChildren: AuthenticatedAdminRouteChildren = {
  AuthenticatedAdminActivityLogRoute: AuthenticatedAdminActivityLogRoute,
  AuthenticatedAdminAppointmentsRoute:
AuthenticatedAdminAppointmentsRoute,
  AuthenticatedAdminArchiveRoute: AuthenticatedAdminArchiveRoute,
  AuthenticatedAdminAvailabilityRoute: AuthenticatedAdminAvailabilityRoute,
  AuthenticatedAdminBlockedNumbersRoute:
AuthenticatedAdminBlockedNumbersRoute,
  AuthenticatedAdminCustomersRoute: AuthenticatedAdminCustomersRoute,
  AuthenticatedAdminMechanicsRoute: AuthenticatedAdminMechanicsRoute,
  AuthenticatedAdminMotorcyclesRoute: AuthenticatedAdminMotorcyclesRoute,
  AuthenticatedAdminNotificationsRoute: AuthenticatedAdminNotificationsRoute,
  AuthenticatedAdminPartsRoute: AuthenticatedAdminPartsRoute,
  AuthenticatedAdminPricesRoute: AuthenticatedAdminPricesRoute,
  AuthenticatedAdminReportsRoute: AuthenticatedAdminReportsRoute,
  AuthenticatedAdminScheduleBlocksRoute:
AuthenticatedAdminScheduleBlocksRoute,
  AuthenticatedAdminServicesRoute: AuthenticatedAdminServicesRoute,
  AuthenticatedAdminSettingsRoute: AuthenticatedAdminSettingsRoute,
  AuthenticatedAdminIndexRoute: AuthenticatedAdminIndexRoute,
}

const AuthenticatedAdminRouteWithChildren =

AuthenticatedAdminRoute._addFileChildren(AuthenticatedAdminRouteChildren)

interface AuthenticatedRouteRouteChildren {
  AuthenticatedAdminRoute: typeof AuthenticatedAdminRouteWithChildren
}
```

```
const AuthenticatedRouteRouteChildren: AuthenticatedRouteRouteChildren = {
  AuthenticatedAdminRoute: AuthenticatedAdminRouteWithChildren,
}

const AuthenticatedRouteRouteWithChildren =
  AuthenticatedRouteRoute._addFileChildren(AuthenticatedRouteRouteChildren)

const rootRouteChildren: RootRouteChildren = {
  IndexRoute: IndexRoute,
  AuthenticatedRouteRoute: AuthenticatedRouteRouteWithChildren,
  AuthRoute: AuthRoute,
  BookRoute: BookRoute,
  MyAppointmentRoute: MyAppointmentRoute,
  ResetPasswordRoute: ResetPasswordRoute,
  ServicesRoute: ServicesRoute,
  ShopRoute: ShopRoute,
  ApiOfflineStatusRoute: ApiOfflineStatusRoute,
  ApiOfflineSyncRoute: ApiOfflineSyncRoute,
  ApiScheduledNotificationsRoute: ApiScheduledNotificationsRoute,
}
export const routeTree = rootRouteImport
  ._addFileChildren(rootRouteChildren)
  ._addFileTypes<FileRouteTypes>()

import type { getRouter } from './router.tsx'
import type { startInstance } from './start.ts'
declare module '@tanstack/react-start' {
  interface Register {
    ssr: true
    router: Awaited<ReturnType<typeof getRouter>>
    config: Awaited<ReturnType<typeof startInstance.getOptions>>
  }
}
```

src/server.ts

```
import "./lib/error-capture";

import { consumeLastCapturedError } from "./lib/error-capture";
import { renderErrorPage } from "./lib/error-page";

type ServerEntry = {
  fetch: (request: Request, env: unknown, ctx: unknown) => Promise<Response>
| Response;
};

let serverEntryPromise: Promise<ServerEntry> | undefined;

async function getServerEntry(): Promise<ServerEntry> {
  if (!serverEntryPromise) {
    serverEntryPromise = import("@tanstack/react-start/server-entry").then(
      (m) => (m.default ?? m) as ServerEntry,
    );
  }
  return serverEntryPromise;
}

// h3 swallows in-handler throws into a normal 500 Response with body
// {"unhandled":true,"message":"HTTPError"} — try/catch alone never fires for
those.
async function normalizeCatastrophicSsrResponse(
  request: Request,
  response: Response,
): Promise<Response> {
  if (response.status < 500) return response;
  const contentType = response.headers.get("content-type") ?? "";
  if (!contentType.includes("application/json")) return response;

  const body = await response.clone().text();
  if (!isH3SwallowedErrorBody(body)) return response;

  const error = consumeLastCapturedError() ?? new Error(`h3 swallowed SSR
error: ${body}`);
  console.error(error);
  await alertPageFailure(request, error);
  return new Response(renderErrorPage(), {
    status: 500,
    headers: { "content-type": "text/html; charset=utf-8" },
  });
```

```
}

/**
 * Tells the admin that a visitor got the error page, at most once an hour while
 * the alert is unread. Waits at most 2 seconds so the visitor is not kept waiting.
 */
async function alertPageFailure(request: Request, error: unknown) {
  try {
    const { alertAdmin, redactPersonalDetails } = await
import("./lib/admin-alerts.server");
    // The path only: query strings can hold reference codes and phone numbers.
    const path = new URL(request.url).pathname;
    const detail = error instanceof Error ?
redactPersonalDetails(error.message).slice(0, 300) : "";
    await Promise.race([
      alertAdmin({
        type: "server_error",
        title: "A page failed to load",
        message: `A visitor got the error page at ${path}${detail ? `. Server error:
${detail}` : "."}`,
        onceWithinMinutes: 60,
      }),
      new Promise((resolve) => setTimeout(resolve, 2_000)),
    ]);
  } catch (alertError) {
    console.error("[Admin alert] page failure alert failed", alertError);
  }
}

function isH3SwallowedErrorBody(body: string): boolean {
  try {
    const payload = JSON.parse(body) as { unhandled?: unknown; message?:
unknown };
    return payload.unhandled === true && payload.message === "HTTPError";
  } catch {
    return false;
  }
}

export default {
  async fetch(request: Request, env: unknown, ctx: unknown) {
    try {
      const handler = await getServerEntry();
      const response = await handler.fetch(request, env, ctx);
      return await normalizeCatastrophicSsrResponse(request, response);
```

```
    } catch (error) {
      console.error(error);
      await alertPageFailure(request, error);
      return new Response(renderErrorPage(), {
        status: 500,
        headers: { "content-type": "text/html; charset=utf-8" },
      });
    }
  },
};
```

src/start.ts

```
import { createStart, createCsrfMiddleware, createMiddleware } from
"@tanstack/react-start";

import { renderErrorPage } from "./lib/error-page";
import { attachSupabaseAuth } from "@/integrations/supabase/auth-attacher";

const errorMiddleware = createMiddleware().server(async ({ next }) => {
  try {
    return await next();
  } catch (error) {
    if (error != null && typeof error === "object" && "statusCode" in error) {
      throw error;
    }
    console.error(error);
    return new Response(renderErrorPage(), {
      status: 500,
      headers: { "content-type": "text/html; charset=utf-8" },
    });
  }
});

// Start installs this automatically when src/start.ts is absent; defining the
// file opts out, so re-add it explicitly to keep server functions protected
// from cross-site requests.
const csrfMiddleware = createCsrfMiddleware({
  filter: (ctx) => ctx.handlerType === "serverFn",
});

export const startInstance = createStart(() => ({
  functionMiddleware: [attachSupabaseAuth],
  requestMiddleware: [errorMiddleware, csrfMiddleware],
}));
```

src/styles.css

```
@import "tailwindcss" source(none);
@source "../src";
@import "tw-animate-css";
/* Bundled with the app rather than loaded from Google, so text looks the same
offline. */
@import "@fontsource/barlow/400.css";
@import "@fontsource/barlow/500.css";
@import "@fontsource/barlow/600.css";
@import "@fontsource/barlow/700.css";
@import "@fontsource/barlow-condensed/500.css";
@import "@fontsource/barlow-condensed/600.css";
@import "@fontsource/barlow-condensed/700.css";
@import "@fontsource/barlow-condensed/800.css";

@theme {
  --font-display: "Barlow Condensed", "Arial Narrow", sans-serif;
  --font-sans: "Barlow", ui-sans-serif, system-ui, sans-serif;
}

@custom-variant dark (&:is(.dark *));

/*
 * Design system definition.
 *
 * The @theme inline block maps CSS custom properties to Tailwind utility
 * classes (e.g. --color-primary -> bg-primary, text-primary).
 *
 * The :root and .dark blocks define the actual color values using oklch.
 * All colors MUST use oklch format.
 *
 * To add a new semantic color:
 * 1. Add the variable to :root (light value) and .dark (dark value)
 * 2. Register it in @theme inline as --color-<name>: var(--<name>)
 */

@theme inline {
  --radius-sm: calc(var(--radius) - 4px);
  --radius-md: calc(var(--radius) - 2px);
  --radius-lg: var(--radius);
  --radius-xl: calc(var(--radius) + 4px);
  --radius-2xl: calc(var(--radius) + 8px);
  --radius-3xl: calc(var(--radius) + 12px);
  --radius-4xl: calc(var(--radius) + 16px);
```

```
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --color-card: var(--card);
  --color-card-foreground: var(--card-foreground);
  --color-popover: var(--popover);
  --color-popover-foreground: var(--popover-foreground);
  --color-primary: var(--primary);
  --color-primary-foreground: var(--primary-foreground);
  --color-secondary: var(--secondary);
  --color-secondary-foreground: var(--secondary-foreground);
  --color-muted: var(--muted);
  --color-muted-foreground: var(--muted-foreground);
  --color-accent: var(--accent);
  --color-accent-foreground: var(--accent-foreground);
  --color-destructive: var(--destructive);
  --color-destructive-foreground: var(--destructive-foreground);
  --color-border: var(--border);
  --color-input: var(--input);
  --color-ring: var(--ring);
  --color-ring-offset-background: var(--background);
  --color-chart-1: var(--chart-1);
  --color-chart-2: var(--chart-2);
  --color-chart-3: var(--chart-3);
  --color-chart-4: var(--chart-4);
  --color-chart-5: var(--chart-5);
  --color-sidebar: var(--sidebar);
  --color-sidebar-foreground: var(--sidebar-foreground);
  --color-sidebar-primary: var(--sidebar-primary);
  --color-sidebar-primary-foreground: var(--sidebar-primary-foreground);
  --color-sidebar-accent: var(--sidebar-accent);
  --color-sidebar-accent-foreground: var(--sidebar-accent-foreground);
  --color-sidebar-border: var(--sidebar-border);
  --color-sidebar-ring: var(--sidebar-ring);
}

:root {
  --radius: 0.5rem;
  --background: oklch(0.985 0.005 90);
  --foreground: oklch(0.22 0.016 65);
  --card: oklch(1 0 0);
  --card-foreground: oklch(0.22 0.016 65);
  --popover: oklch(1 0 0);
  --popover-foreground: oklch(0.22 0.016 65);
  --primary: oklch(0.63 0.16 82);
  --primary-foreground: oklch(0.99 0.003 90);
```

```
  --secondary: oklch(0.94 0.012 78);
  --secondary-foreground: oklch(0.25 0.016 65);
  --muted: oklch(0.94 0.01 78);
  --muted-foreground: oklch(0.48 0.018 70);
  --accent: oklch(0.6 0.2 45);
  --accent-foreground: oklch(0.99 0.003 90);
  --destructive: oklch(0.53 0.22 25);
  --destructive-foreground: oklch(0.99 0.003 90);
  --border: oklch(0.85 0.012 75);
  --input: oklch(0.87 0.012 75);
  --ring: oklch(0.63 0.16 82);
  --chart-1: oklch(0.63 0.16 82);
  --chart-2: oklch(0.6 0.2 45);
  --chart-3: oklch(0.55 0.1 150);
  --chart-4: oklch(0.56 0.14 260);
  --chart-5: oklch(0.61 0.15 20);
  --sidebar: oklch(0.96 0.008 80);
  --sidebar-foreground: oklch(0.22 0.016 65);
  --sidebar-primary: oklch(0.63 0.16 82);
  --sidebar-primary-foreground: oklch(0.99 0.003 90);
  --sidebar-accent: oklch(0.91 0.014 78);
  --sidebar-accent-foreground: oklch(0.22 0.016 65);
  --sidebar-border: oklch(0.85 0.012 75);
  --sidebar-ring: oklch(0.63 0.16 82);
}

.dark {
  --background: oklch(0.16 0.012 60);
  --foreground: oklch(0.97 0.004 90);
  --card: oklch(0.21 0.014 65);
  --card-foreground: oklch(0.97 0.004 90);
  --popover: oklch(0.19 0.013 65);
  --popover-foreground: oklch(0.97 0.004 90);
  --primary: oklch(0.84 0.18 92);
  --primary-foreground: oklch(0.18 0.02 70);
  --secondary: oklch(0.27 0.016 65);
  --secondary-foreground: oklch(0.96 0.005 90);
  --muted: oklch(0.26 0.013 65);
  --muted-foreground: oklch(0.72 0.015 80);
  --accent: oklch(0.68 0.2 45);
  --accent-foreground: oklch(0.16 0.012 60);
  --destructive: oklch(0.58 0.22 25);
  --destructive-foreground: oklch(0.98 0.003 90);
  --border: oklch(0.32 0.015 70);
  --input: oklch(0.3 0.015 70);
```

```
  --ring: oklch(0.84 0.18 92);
  --chart-1: oklch(0.84 0.18 92);
  --chart-2: oklch(0.68 0.2 45);
  --chart-3: oklch(0.72 0.09 150);
  --chart-4: oklch(0.62 0.14 260);
  --chart-5: oklch(0.75 0.14 20);
  --sidebar: oklch(0.19 0.013 65);
  --sidebar-foreground: oklch(0.95 0.005 90);
  --sidebar-primary: oklch(0.84 0.18 92);
  --sidebar-primary-foreground: oklch(0.18 0.02 70);
  --sidebar-accent: oklch(0.27 0.016 65);
  --sidebar-accent-foreground: oklch(0.96 0.005 90);
  --sidebar-border: oklch(0.3 0.015 70);
  --sidebar-ring: oklch(0.84 0.18 92);
  color-scheme: dark;
}

@layer base {
  * {
    border-color: var(--color-border);
  }

  body {
    background-color: var(--color-background);
    color: var(--color-foreground);
    -webkit-text-size-adjust: 100%;
    text-size-adjust: 100%;
  }
}

@layer base {
  body {
    font-family: var(--font-sans);
  }
  h1,
  h2,
  h3,
  h4,
  .font-display {
    font-family: var(--font-display);
    letter-spacing: 0.01em;
  }
}

@utility checker-strip {
```

```
  background-image:
    linear-gradient(45deg, var(--color-foreground) 25%, transparent 25%),
    linear-gradient(-45deg, var(--color-foreground) 25%, transparent 25%),
    linear-gradient(45deg, transparent 75%, var(--color-foreground) 75%),
    linear-gradient(-45deg, transparent 75%, var(--color-foreground) 75%);
  background-size: 14px 14px;
  background-position:
    0 0,
    0 7px,
    7px -7px,
    -7px 0;
  opacity: 0.85;
}

@utility text-gradient-race {
  background-image: linear-gradient(100deg, var(--color-primary),
var(--color-accent));
  background-clip: text;
  color: transparent;
}

/* Shared public-page shell: compact gutters on phones, comfortable breathing
 * room on laptops, and a cap that prevents ultra-wide screens from stretching
 * page content too far. */
@utility site-container {
  width: 100%;
  max-width: 90rem;
  margin-inline: auto;
  padding-inline: clamp(1rem, 2.5vw, 3rem);
}

@media (max-width: 767px) {
  /* Controls do not need the delayed double-tap gesture on touch screens.
   * This also makes dense admin actions feel immediate without affecting
   * pinch zoom for the rest of the page. */
  button,
  [role="button"],
  [role="combobox"],
  input,
  textarea,
  select {
    touch-action: manipulation;
  }
}
```

supabase/.temp/linked-project.json

```
{
  "ref": "egqukkwjuagvgyuqdart",
  "name": "tat-shop",
  "organization_id": "muiedhskswfgxgwcopbk",
  "organization_slug": "muiedhskswfgxgwcopbk"
}
```

supabase/config.toml

```
project_id = "egqukkwjuagvgyuqdart"
```

supabase/migrations/20260816152359_1aadbf74-b753-439f-b1eb-4877adbc55dd.sql

---

```
-- roles
DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname = 'app_role' AND
typnamespace = 'public'::regnamespace) THEN
    CREATE TYPE public.app_role AS ENUM ('admin','staff');
  END IF;
END $$;

CREATE TABLE IF NOT EXISTS public.user_roles (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid NOT NULL REFERENCES auth.users(id) ON DELETE
CASCADE,
  role public.app_role NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (user_id, role)
);
GRANT SELECT ON public.user_roles TO authenticated;
GRANT ALL ON public.user_roles TO service_role;
ALTER TABLE public.user_roles ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "users read own roles" ON public.user_roles;
CREATE POLICY "users read own roles" ON public.user_roles FOR SELECT
TO authenticated USING (auth.uid() = user_id);

CREATE OR REPLACE FUNCTION public.has_role(_user_id uuid, _role
public.app_role)
RETURNS boolean LANGUAGE sql STABLE SECURITY DEFINER SET
search_path = public AS $$
  SELECT EXISTS (SELECT 1 FROM public.user_roles WHERE user_id =
_user_id AND role = _role)
$$;

CREATE OR REPLACE FUNCTION public.update_updated_at_column()
RETURNS trigger LANGUAGE plpgsql SET search_path = public AS $$
BEGIN NEW.updated_at = now(); RETURN NEW; END; $$;

-- services
CREATE TABLE IF NOT EXISTS public.services (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  description text,
  price numeric(10,2) NOT NULL DEFAULT 0,
```

```
  duration_minutes integer NOT NULL DEFAULT 60,
  category text NOT NULL DEFAULT 'general',
  image_url text,
  is_active boolean NOT NULL DEFAULT true,
  sort_order integer NOT NULL DEFAULT 0,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);
GRANT SELECT ON public.services TO anon;
GRANT SELECT, INSERT, UPDATE, DELETE ON public.services TO
authenticated;
GRANT ALL ON public.services TO service_role;
ALTER TABLE public.services ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "services public read" ON public.services;
CREATE POLICY "services public read" ON public.services FOR SELECT TO
anon, authenticated USING (true);
DROP POLICY IF EXISTS "services admin write" ON public.services;
CREATE POLICY "services admin write" ON public.services FOR ALL TO
authenticated USING (public.has_role(auth.uid(),'admin')) WITH CHECK
(public.has_role(auth.uid(),'admin'));
DROP TRIGGER IF EXISTS services_updated ON public.services;
CREATE TRIGGER services_updated BEFORE UPDATE ON public.services
FOR EACH ROW EXECUTE FUNCTION public.update_updated_at_column();

-- products
CREATE TABLE IF NOT EXISTS public.products (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  category text NOT NULL DEFAULT 'part',
  brand text,
  description text,
  price numeric(10,2) NOT NULL DEFAULT 0,
  image_url text,
  in_stock boolean NOT NULL DEFAULT true,
  is_featured boolean NOT NULL DEFAULT false,
  is_active boolean NOT NULL DEFAULT true,
  sort_order integer NOT NULL DEFAULT 0,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);
GRANT SELECT ON public.products TO anon;
GRANT SELECT, INSERT, UPDATE, DELETE ON public.products TO
authenticated;
GRANT ALL ON public.products TO service_role;
ALTER TABLE public.products ENABLE ROW LEVEL SECURITY;
```

```
DROP POLICY IF EXISTS "products public read" ON public.products;
CREATE POLICY "products public read" ON public.products FOR SELECT TO
anon, authenticated USING (true);
DROP POLICY IF EXISTS "products admin write" ON public.products;
CREATE POLICY "products admin write" ON public.products FOR ALL TO
authenticated USING (public.has_role(auth.uid(),'admin')) WITH CHECK
(public.has_role(auth.uid(),'admin'));
DROP TRIGGER IF EXISTS products_updated ON public.products;
CREATE TRIGGER products_updated BEFORE UPDATE ON public.products
FOR EACH ROW EXECUTE FUNCTION public.update_updated_at_column();

-- time slots
CREATE TABLE IF NOT EXISTS public.time_slots (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  start_time time NOT NULL,
  end_time time NOT NULL,
  capacity integer NOT NULL DEFAULT 2,
  is_active boolean NOT NULL DEFAULT true,
  created_at timestamptz NOT NULL DEFAULT now()
);
GRANT SELECT ON public.time_slots TO anon;
GRANT SELECT, INSERT, UPDATE, DELETE ON public.time_slots TO
authenticated;
GRANT ALL ON public.time_slots TO service_role;
ALTER TABLE public.time_slots ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "slots public read" ON public.time_slots;
CREATE POLICY "slots public read" ON public.time_slots FOR SELECT TO
anon, authenticated USING (true);
DROP POLICY IF EXISTS "slots admin write" ON public.time_slots;
CREATE POLICY "slots admin write" ON public.time_slots FOR ALL TO
authenticated USING (public.has_role(auth.uid(),'admin')) WITH CHECK
(public.has_role(auth.uid(),'admin'));

-- schedule blocks
CREATE TABLE IF NOT EXISTS public.schedule_blocks (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  block_date date NOT NULL,
  start_time time,
  reason text,
  created_at timestamptz NOT NULL DEFAULT now()
);
GRANT SELECT ON public.schedule_blocks TO anon;
GRANT SELECT, INSERT, UPDATE, DELETE ON public.schedule_blocks TO
authenticated;
GRANT ALL ON public.schedule_blocks TO service_role;
```

```
ALTER TABLE public.schedule_blocks ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "blocks public read" ON public.schedule_blocks;
CREATE POLICY "blocks public read" ON public.schedule_blocks FOR SELECT
TO anon, authenticated USING (true);
DROP POLICY IF EXISTS "blocks admin write" ON public.schedule_blocks;
CREATE POLICY "blocks admin write" ON public.schedule_blocks FOR ALL TO
authenticated USING (public.has_role(auth.uid(),'admin')) WITH CHECK
(public.has_role(auth.uid(),'admin'));

-- crew
CREATE TABLE IF NOT EXISTS public.crew_members (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  role text NOT NULL DEFAULT 'Mechanic',
  phone text,
  is_active boolean NOT NULL DEFAULT true,
  created_at timestamptz NOT NULL DEFAULT now()
);
GRANT SELECT, INSERT, UPDATE, DELETE ON public.crew_members TO
authenticated;
GRANT ALL ON public.crew_members TO service_role;
ALTER TABLE public.crew_members ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "crew admin all" ON public.crew_members;
CREATE POLICY "crew admin all" ON public.crew_members FOR ALL TO
authenticated USING (public.has_role(auth.uid(),'admin')) WITH CHECK
(public.has_role(auth.uid(),'admin'));

-- appointments
CREATE TABLE IF NOT EXISTS public.appointments (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  reference_code text NOT NULL UNIQUE,
  customer_name text NOT NULL,
  phone text NOT NULL,
  email text,
  moto_brand text NOT NULL,
  moto_model text NOT NULL,
  moto_variant text,
  moto_year integer,
  plate_number text NOT NULL,
  appointment_date date NOT NULL,
  start_time time NOT NULL,
  status text NOT NULL DEFAULT 'pending',
  notes text,
  admin_notes text,
  total_estimate numeric(10,2) NOT NULL DEFAULT 0,
```

```
  assigned_crew_id uuid REFERENCES public.crew_members(id) ON DELETE
SET NULL,
  is_archived boolean NOT NULL DEFAULT false,
  terms_accepted boolean NOT NULL DEFAULT false,
  cancelled_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX IF NOT EXISTS appointments_date_idx ON
public.appointments (appointment_date, start_time);
GRANT SELECT, INSERT, UPDATE, DELETE ON public.appointments TO
authenticated;
GRANT ALL ON public.appointments TO service_role;
ALTER TABLE public.appointments ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "appointments admin all" ON public.appointments;
CREATE POLICY "appointments admin all" ON public.appointments FOR ALL
TO authenticated USING (public.has_role(auth.uid(),'admin')) WITH CHECK
(public.has_role(auth.uid(),'admin'));

CREATE TABLE IF NOT EXISTS public.appointment_services (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  appointment_id uuid NOT NULL REFERENCES public.appointments(id) ON
DELETE CASCADE,
  service_id uuid REFERENCES public.services(id) ON DELETE SET NULL,
  service_name text NOT NULL,
  price numeric(10,2) NOT NULL DEFAULT 0
);
GRANT SELECT, INSERT, UPDATE, DELETE ON public.appointment_services
TO authenticated;
GRANT ALL ON public.appointment_services TO service_role;
ALTER TABLE public.appointment_services ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "appt services admin all" ON
public.appointment_services;
CREATE POLICY "appt services admin all" ON public.appointment_services
FOR ALL TO authenticated USING (public.has_role(auth.uid(),'admin')) WITH
CHECK (public.has_role(auth.uid(),'admin'));

DROP TRIGGER IF EXISTS appointments_updated ON public.appointments;
CREATE TRIGGER appointments_updated BEFORE UPDATE ON
public.appointments FOR EACH ROW EXECUTE FUNCTION
public.update_updated_at_column();

-- notifications
CREATE TABLE IF NOT EXISTS public.notifications (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  type text NOT NULL DEFAULT 'new_appointment',
  title text NOT NULL,
  message text,
  appointment_id uuid REFERENCES public.appointments(id) ON DELETE
CASCADE,
  is_read boolean NOT NULL DEFAULT false,
  created_at timestamptz NOT NULL DEFAULT now()
);
GRANT SELECT, INSERT, UPDATE, DELETE ON public.notifications TO
authenticated;
GRANT ALL ON public.notifications TO service_role;
ALTER TABLE public.notifications ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "notifications admin all" ON public.notifications;
CREATE POLICY "notifications admin all" ON public.notifications FOR ALL TO
authenticated USING (public.has_role(auth.uid(),'admin')) WITH CHECK
(public.has_role(auth.uid(),'admin'));

DO $$
BEGIN
  IF EXISTS (SELECT 1 FROM pg_publication_tables WHERE pubname =
'supabase_realtime' AND tablename = 'notifications') THEN
    ALTER PUBLICATION supabase_realtime DROP TABLE public.notifications;
  END IF;
  ALTER PUBLICATION supabase_realtime ADD TABLE public.notifications;
END $$;

DO $$
BEGIN
  IF EXISTS (SELECT 1 FROM pg_publication_tables WHERE pubname =
'supabase_realtime' AND tablename = 'appointments') THEN
    ALTER PUBLICATION supabase_realtime DROP TABLE
public.appointments;
  END IF;
  ALTER PUBLICATION supabase_realtime ADD TABLE public.appointments;
END $$;

-- seed slots (Mon-Sat 08:00-17:00, 1 hour)
INSERT INTO public.time_slots (start_time, end_time, capacity) VALUES
('08:00','09:00',2),('09:00','10:00',2),('10:00','11:00',2),('11:00','12:00',2),
('13:00','14:00',2),('14:00','15:00',2),('15:00','16:00',2),('16:00','17:00',2)
ON CONFLICT DO NOTHING;

INSERT INTO public.services (name, description, price, duration_minutes,
category, sort_order) VALUES
('Preventive Maintenance Service (PMS)','Full check-up: oil change, brake, chain,
```

```
and fluid inspection.',850.00,90,'maintenance',1),
('Change Oil','Engine oil and filter replacement using your preferred
brand.',450.00,45,'maintenance',2),
('Suspension Tuning / Rebuild','Ohlins-grade fork and shock service, oil change
and re-valving.',2500.00,180,'suspension',3),
('Tire Change & Balancing','Tire mounting, balancing and valve
replacement.',600.00,60,'tires',4),
('Brake Service','Pad replacement, caliper clean and brake fluid
bleed.',750.00,60,'brakes',5),
('Engine Overhaul','Top or bottom-end overhaul with parts
assessment.',6500.00,480,'engine',6),
('Electrical Diagnostics','Wiring, charging system and lighting
troubleshooting.',500.00,60,'electrical',7),
('Carburetor / FI Cleaning','Deep clean and tuning for smoother throttle
response.',900.00,90,'engine',8)
ON CONFLICT DO NOTHING;

INSERT INTO public.products (name, category, brand, description, price,
in_stock, is_featured, sort_order) VALUES
('Ohlins Rear Shock Absorber','part','Ohlins','Premium gas-charged rear shock
with adjustable preload.',24500.00,true,true,1),
('Racing Clutch Assembly','part','Yoshimura','Heavy duty clutch kit for aggressive
riding.',4800.00,true,true,2),
('Performance Exhaust Pipe','part','Akrapovic','Slip-on stainless exhaust with
deep race tone.',13500.00,true,true,3),
('Braided Brake Hose','part','Galespeed','Stainless braided hose for sharper
brake feel.',2100.00,true,false,4),
('CNC Adjustable Levers','accessory','Rizoma','Anodized aluminum brake and
clutch levers.',1850.00,true,true,5),
('Full-Face Racing Helmet','accessory','HJC','DOT/ICC certified helmet with
pinlock visor.',7900.00,true,true,6),
('Motocross Riding Gloves','accessory','Fox','Breathable gloves with silicone grip
print.',1250.00,true,false,7),
('Motorcycle Tail Bag','accessory','Komine','Waterproof 20L expandable tail
bag.',3400.00,true,false,8),
('Honda CRF150L','motorcycle','Honda','Dual sport 150cc, ideal for trail and
city.',169900.00,true,true,9),
('Kawasaki KLX150','motorcycle','Kawasaki','Lightweight trail bike, brand new
unit.',159000.00,true,true,10),
('Yamaha WR155R','motorcycle','Yamaha','VVA-equipped enduro with racing
DNA.',189000.00,true,false,11),
('Suzuki Raider R150 Fi','motorcycle','Suzuki','Underbone legend with fuel
injection.',129900.00,true,false,12)
ON CONFLICT DO NOTHING;
```

```
INSERT INTO public.crew_members (name, role, phone) VALUES
('Mang Tonio','Head Mechanic','09171234567'),
('JR Salazar','Suspension Specialist','09181234567'),
('Kevin Dela Cruz','Service Technician','09191234567')
ON CONFLICT DO NOTHING;

REVOKE EXECUTE ON FUNCTION public.has_role(uuid, public.app_role)
FROM anon;
REVOKE EXECUTE ON FUNCTION public.update_updated_at_column() FROM
anon, authenticated;
REVOKE EXECUTE ON FUNCTION public.has_role(uuid, public.app_role)
FROM PUBLIC;
REVOKE EXECUTE ON FUNCTION public.update_updated_at_column() FROM
PUBLIC;
GRANT EXECUTE ON FUNCTION public.has_role(uuid, public.app_role) TO
authenticated, service_role;
```

supabase/migrations/20260816152418_d84da632-d75d-4e40-bb9e-ebd93be253cf.sql

---

```
REVOKE EXECUTE ON FUNCTION public.has_role(uuid, public.app_role)
FROM anon;
REVOKE EXECUTE ON FUNCTION public.update_updated_at_column() FROM
anon, authenticated;
```

supabase/migrations/20260816152436_f6440707-f2a5-4d75-92b2-eb0b2feb3f33.sql

---

```
REVOKE EXECUTE ON FUNCTION public.has_role(uuid, public.app_role)
FROM PUBLIC;
REVOKE EXECUTE ON FUNCTION public.update_updated_at_column() FROM
PUBLIC;
GRANT EXECUTE ON FUNCTION public.has_role(uuid, public.app_role) TO
authenticated, service_role;
```

supabase/migrations/20260818222449_d76fe9f7-c81f-40c7-915c-1d3658470a09.sql

---

```
-- roles
DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname = 'app_role' AND
typnamespace = 'public'::regnamespace) THEN
    CREATE TYPE public.app_role AS ENUM ('admin','staff');
  END IF;
END $$;

CREATE TABLE IF NOT EXISTS public.user_roles (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid NOT NULL REFERENCES auth.users(id) ON DELETE
CASCADE,
  role public.app_role NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (user_id, role)
);
GRANT SELECT ON public.user_roles TO authenticated;
GRANT ALL ON public.user_roles TO service_role;
ALTER TABLE public.user_roles ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "users read own roles" ON public.user_roles;
CREATE POLICY "users read own roles" ON public.user_roles FOR SELECT
TO authenticated USING (auth.uid() = user_id);

CREATE OR REPLACE FUNCTION public.has_role(_user_id uuid, _role
public.app_role)
RETURNS boolean LANGUAGE sql STABLE SECURITY DEFINER SET
search_path = public AS $$
  SELECT EXISTS (SELECT 1 FROM public.user_roles WHERE user_id =
_user_id AND role = _role)
$$;

CREATE OR REPLACE FUNCTION public.update_updated_at_column()
RETURNS trigger LANGUAGE plpgsql SET search_path = public AS $$
BEGIN NEW.updated_at = now(); RETURN NEW; END; $$;

-- services
CREATE TABLE IF NOT EXISTS public.services (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  description text,
  price numeric(10,2) NOT NULL DEFAULT 0,
```

```
  duration_minutes integer NOT NULL DEFAULT 60,
  category text NOT NULL DEFAULT 'general',
  image_url text,
  is_active boolean NOT NULL DEFAULT true,
  sort_order integer NOT NULL DEFAULT 0,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);
GRANT SELECT ON public.services TO anon;
GRANT SELECT, INSERT, UPDATE, DELETE ON public.services TO
authenticated;
GRANT ALL ON public.services TO service_role;
ALTER TABLE public.services ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "services public read" ON public.services;
CREATE POLICY "services public read" ON public.services FOR SELECT TO
anon, authenticated USING (true);
DROP POLICY IF EXISTS "services admin write" ON public.services;
CREATE POLICY "services admin write" ON public.services FOR ALL TO
authenticated USING (public.has_role(auth.uid(),'admin')) WITH CHECK
(public.has_role(auth.uid(),'admin'));
DROP TRIGGER IF EXISTS services_updated ON public.services;
CREATE TRIGGER services_updated BEFORE UPDATE ON public.services
FOR EACH ROW EXECUTE FUNCTION public.update_updated_at_column();

-- products
CREATE TABLE IF NOT EXISTS public.products (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  category text NOT NULL DEFAULT 'part',
  brand text,
  description text,
  price numeric(10,2) NOT NULL DEFAULT 0,
  image_url text,
  in_stock boolean NOT NULL DEFAULT true,
  is_featured boolean NOT NULL DEFAULT false,
  is_active boolean NOT NULL DEFAULT true,
  sort_order integer NOT NULL DEFAULT 0,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);
GRANT SELECT ON public.products TO anon;
GRANT SELECT, INSERT, UPDATE, DELETE ON public.products TO
authenticated;
GRANT ALL ON public.products TO service_role;
ALTER TABLE public.products ENABLE ROW LEVEL SECURITY;
```

```
DROP POLICY IF EXISTS "products public read" ON public.products;
CREATE POLICY "products public read" ON public.products FOR SELECT TO
anon, authenticated USING (true);
DROP POLICY IF EXISTS "products admin write" ON public.products;
CREATE POLICY "products admin write" ON public.products FOR ALL TO
authenticated USING (public.has_role(auth.uid(),'admin')) WITH CHECK
(public.has_role(auth.uid(),'admin'));
DROP TRIGGER IF EXISTS products_updated ON public.products;
CREATE TRIGGER products_updated BEFORE UPDATE ON public.products
FOR EACH ROW EXECUTE FUNCTION public.update_updated_at_column();
CREATE UNIQUE INDEX IF NOT EXISTS products_name_idx ON
public.products (name);

-- time slots
CREATE TABLE IF NOT EXISTS public.time_slots (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  start_time time NOT NULL,
  end_time time NOT NULL,
  capacity integer NOT NULL DEFAULT 2,
  is_active boolean NOT NULL DEFAULT true,
  created_at timestamptz NOT NULL DEFAULT now()
);
GRANT SELECT ON public.time_slots TO anon;
GRANT SELECT, INSERT, UPDATE, DELETE ON public.time_slots TO
authenticated;
GRANT ALL ON public.time_slots TO service_role;
ALTER TABLE public.time_slots ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "slots public read" ON public.time_slots;
CREATE POLICY "slots public read" ON public.time_slots FOR SELECT TO
anon, authenticated USING (true);
DROP POLICY IF EXISTS "slots admin write" ON public.time_slots;
CREATE POLICY "slots admin write" ON public.time_slots FOR ALL TO
authenticated USING (public.has_role(auth.uid(),'admin')) WITH CHECK
(public.has_role(auth.uid(),'admin'));

-- schedule blocks
CREATE TABLE IF NOT EXISTS public.schedule_blocks (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  block_date date NOT NULL,
  start_time time,
  reason text,
  created_at timestamptz NOT NULL DEFAULT now()
);
GRANT SELECT ON public.schedule_blocks TO anon;
GRANT SELECT, INSERT, UPDATE, DELETE ON public.schedule_blocks TO
```

```
authenticated;
GRANT ALL ON public.schedule_blocks TO service_role;
ALTER TABLE public.schedule_blocks ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "blocks public read" ON public.schedule_blocks;
CREATE POLICY "blocks public read" ON public.schedule_blocks FOR SELECT
TO anon, authenticated USING (true);
DROP POLICY IF EXISTS "blocks admin write" ON public.schedule_blocks;
CREATE POLICY "blocks admin write" ON public.schedule_blocks FOR ALL TO
authenticated USING (public.has_role(auth.uid(),'admin')) WITH CHECK
(public.has_role(auth.uid(),'admin'));

-- crew
CREATE TABLE IF NOT EXISTS public.crew_members (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  role text NOT NULL DEFAULT 'Mechanic',
  phone text,
  is_active boolean NOT NULL DEFAULT true,
  created_at timestamptz NOT NULL DEFAULT now()
);
GRANT SELECT, INSERT, UPDATE, DELETE ON public.crew_members TO
authenticated;
GRANT ALL ON public.crew_members TO service_role;
ALTER TABLE public.crew_members ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "crew admin all" ON public.crew_members;
CREATE POLICY "crew admin all" ON public.crew_members FOR ALL TO
authenticated USING (public.has_role(auth.uid(),'admin')) WITH CHECK
(public.has_role(auth.uid(),'admin'));

-- appointments
CREATE TABLE IF NOT EXISTS public.appointments (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  reference_code text NOT NULL UNIQUE,
  customer_name text NOT NULL,
  phone text NOT NULL,
  email text,
  moto_brand text NOT NULL,
  moto_model text NOT NULL,
  moto_variant text,
  moto_year integer,
  plate_number text NOT NULL,
  appointment_date date NOT NULL,
  start_time time NOT NULL,
  status text NOT NULL DEFAULT 'pending',
  notes text,
```

```
  admin_notes text,
  total_estimate numeric(10,2) NOT NULL DEFAULT 0,
  assigned_crew_id uuid REFERENCES public.crew_members(id) ON DELETE
SET NULL,
  is_archived boolean NOT NULL DEFAULT false,
  terms_accepted boolean NOT NULL DEFAULT false,
  cancelled_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX IF NOT EXISTS appointments_date_idx ON
public.appointments (appointment_date, start_time);
GRANT SELECT, INSERT, UPDATE, DELETE ON public.appointments TO
authenticated;
GRANT ALL ON public.appointments TO service_role;
ALTER TABLE public.appointments ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "appointments admin all" ON public.appointments;
CREATE POLICY "appointments admin all" ON public.appointments FOR ALL
TO authenticated USING (public.has_role(auth.uid(),'admin')) WITH CHECK
(public.has_role(auth.uid(),'admin'));

CREATE TABLE IF NOT EXISTS public.appointment_services (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  appointment_id uuid NOT NULL REFERENCES public.appointments(id) ON
DELETE CASCADE,
  service_id uuid REFERENCES public.services(id) ON DELETE SET NULL,
  service_name text NOT NULL,
  price numeric(10,2) NOT NULL DEFAULT 0
);
GRANT SELECT, INSERT, UPDATE, DELETE ON public.appointment_services
TO authenticated;
GRANT ALL ON public.appointment_services TO service_role;
ALTER TABLE public.appointment_services ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "appt services admin all" ON
public.appointment_services;
CREATE POLICY "appt services admin all" ON public.appointment_services
FOR ALL TO authenticated USING (public.has_role(auth.uid(),'admin')) WITH
CHECK (public.has_role(auth.uid(),'admin'));

DROP TRIGGER IF EXISTS appointments_updated ON public.appointments;
CREATE TRIGGER appointments_updated BEFORE UPDATE ON
public.appointments FOR EACH ROW EXECUTE FUNCTION
public.update_updated_at_column();

-- notifications
```

```
CREATE TABLE IF NOT EXISTS public.notifications (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  type text NOT NULL DEFAULT 'new_appointment',
  title text NOT NULL,
  message text,
  appointment_id uuid REFERENCES public.appointments(id) ON DELETE
CASCADE,
  is_read boolean NOT NULL DEFAULT false,
  created_at timestamptz NOT NULL DEFAULT now()
);
GRANT SELECT, INSERT, UPDATE, DELETE ON public.notifications TO
authenticated;
GRANT ALL ON public.notifications TO service_role;
ALTER TABLE public.notifications ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "notifications admin all" ON public.notifications;
CREATE POLICY "notifications admin all" ON public.notifications FOR ALL TO
authenticated USING (public.has_role(auth.uid(),'admin')) WITH CHECK
(public.has_role(auth.uid(),'admin'));

DO $$
BEGIN
  IF EXISTS (SELECT 1 FROM pg_publication_tables WHERE pubname =
'supabase_realtime' AND tablename = 'notifications') THEN
    ALTER PUBLICATION supabase_realtime DROP TABLE public.notifications;
  END IF;
  ALTER PUBLICATION supabase_realtime ADD TABLE public.notifications;
END $$;

DO $$
BEGIN
  IF EXISTS (SELECT 1 FROM pg_publication_tables WHERE pubname =
'supabase_realtime' AND tablename = 'appointments') THEN
    ALTER PUBLICATION supabase_realtime DROP TABLE
public.appointments;
  END IF;
  ALTER PUBLICATION supabase_realtime ADD TABLE public.appointments;
END $$;

-- seed slots (Mon-Sat 08:00-17:00, 1 hour)
INSERT INTO public.time_slots (start_time, end_time, capacity) VALUES
('08:00','09:00',2),('09:00','10:00',2),('10:00','11:00',2),('11:00','12:00',2),
('13:00','14:00',2),('14:00','15:00',2),('15:00','16:00',2),('16:00','17:00',2)
ON CONFLICT (start_time) DO NOTHING;

INSERT INTO public.services (name, description, price, duration_minutes,
```

```
category, sort_order) VALUES
('Preventive Maintenance Service (PMS)','Full check-up: oil change, brake, chain, and fluid inspection.',850.00,90,'maintenance',1),
('Change Oil','Engine oil and filter replacement using your preferred brand.',450.00,45,'maintenance',2),
('Suspension Tuning / Rebuild','Ohlins-grade fork and shock service, oil change and re-valving.',2500.00,180,'suspension',3),
('Tire Change & Balancing','Tire mounting, balancing and valve replacement.',600.00,60,'tires',4),
('Brake Service','Pad replacement, caliper clean and brake fluid bleed.',750.00,60,'brakes',5),
('Engine Overhaul','Top or bottom-end overhaul with parts assessment.',6500.00,480,'engine',6),
('Electrical Diagnostics','Wiring, charging system and lighting troubleshooting.',500.00,60,'electrical',7),
('Carburetor / FI Cleaning','Deep clean and tuning for smoother throttle response.',900.00,90,'engine',8)
ON CONFLICT DO NOTHING;

INSERT INTO public.products (name, category, brand, description, price, in_stock, is_featured, sort_order) VALUES
('Ohlins Rear Shock Absorber','part','Ohlins','Premium gas-charged rear shock with adjustable preload.',24500.00,true,true,1),
('Racing Clutch Assembly','part','Yoshimura','Heavy duty clutch kit for aggressive riding.',4800.00,true,true,2),
('Performance Exhaust Pipe','part','Akrapovic','Slip-on stainless exhaust with deep race tone.',13500.00,true,true,3),
('Braided Brake Hose','part','Galespeed','Stainless braided hose for sharper brake feel.',2100.00,true,false,4),
('CNC Adjustable Levers','accessory','Rizoma','Anodized aluminum brake and clutch levers.',1850.00,true,true,5),
('Full-Face Racing Helmet','accessory','HJC','DOT/ICC certified helmet with pinlock visor.',7900.00,true,true,6),
('Motocross Riding Gloves','accessory','Fox','Breathable gloves with silicone grip print.',1250.00,true,false,7),
('Motorcycle Tail Bag','accessory','Komine','Waterproof 20L expandable tail bag.',3400.00,true,false,8),
('Honda CRF150L','motorcycle','Honda','Dual sport 150cc, ideal for trail and city.',169900.00,true,true,9),
('Honda CB150R','motorcycle','Honda','Streetfighter 150cc with aggressive styling.',129000.00,true,true,10),
('Honda Beat','motorcycle','Honda','Popular underbone 110cc for daily commuting.',79000.00,true,true,11),
('Honda PCX160','motorcycle','Honda','Popular underbone 158cc for daily commuting.',119000.00,true,true,12),
```

```
('Yamaha Mio','motorcycle','Yamaha','125cc underbone, the king of city
rides.',78000.00,true,true,13),
('Yamaha NMAX','motorcycle','Yamaha','155cc automatic scooter for urban
commuters.',125000.00,true,true,14),
('Yamaha MT-15','motorcycle','Yamaha','Naked 155cc with aggressive
styling.',135000.00,true,true,15),
('Yamaha R15','motorcycle','Yamaha','155cc sport bike with VVA
engine.',149000.00,true,false,16),
('Suzuki Raider R150 Fi','motorcycle','Suzuki','Underbone legend with fuel
injection.',129900.00,true,true,17),
('Suzuki GSX-S150','motorcycle','Suzuki','150cc sporty
streetfighter.',119000.00,true,true,18),
('Kawasaki KLX150','motorcycle','Kawasaki','Lightweight trail bike, brand new
unit.',159000.00,true,true,19),
('Kawasaki Z150','motorcycle','Kawasaki','150cc street naked with sharp
styling.',124000.00,true,false,20),
('KTM 125 Duke','motorcycle','KTM','125cc naked with race-bred
handling.',185000.00,true,false,21),
('Royal Enfield Classic 350','motorcycle','Royal Enfield','350cc classic cruiser with
modern fuel injection.',225000.00,true,false,22),
('TVS Apache RTR 160','motorcycle','TVS','160cc sporty naked with race-tuned
engine.',98000.00,true,false,23)
ON CONFLICT (name) DO NOTHING;

INSERT INTO public.crew_members (name, role, phone) VALUES
('Mang Tonio','Head Mechanic','09171234567'),
('JR Salazar','Suspension Specialist','09181234567'),
('Kevin Dela Cruz','Service Technician','09191234567')
ON CONFLICT DO NOTHING;

REVOKE EXECUTE ON FUNCTION public.has_role(uuid, public.app_role)
FROM anon;
REVOKE EXECUTE ON FUNCTION public.update_updated_at_column() FROM
anon, authenticated;
REVOKE EXECUTE ON FUNCTION public.has_role(uuid, public.app_role)
FROM PUBLIC;
REVOKE EXECUTE ON FUNCTION public.update_updated_at_column() FROM
PUBLIC;
GRANT EXECUTE ON FUNCTION public.has_role(uuid, public.app_role) TO
authenticated, service_role;
```

supabase/migrations/20260819150000_archive_soft_delete.sql

```
-- Add is_active column to schedule_blocks for archive support
ALTER TABLE public.schedule_blocks
  ADD COLUMN IF NOT EXISTS is_active boolean NOT NULL DEFAULT true;

-- Backfill existing rows (they are implicitly active)
UPDATE public.schedule_blocks
  SET is_active = true
  WHERE is_active IS DISTINCT FROM true;
```

supabase/migrations/20260819160000_fix_trigger_permissions.sql

---

```
-- Fix: authenticated role was revoked from update_updated_at_column function,
-- causing triggers that call it (e.g. appointments_updated) to fail on UPDATE.
GRANT EXECUTE ON FUNCTION public.update_updated_at_column() TO
authenticated;
```

supabase/migrations/20260819170000_fix_time_slots_duplicates.sql

---

```
-- Add unique constraint on time_slots.start_time to prevent duplicates
-- First clean up any duplicates that exist from the seed data running multiple
times
DELETE FROM public.time_slots
WHERE ctid IN (
  SELECT ctid
  FROM (
    SELECT ctid,
         ROW_NUMBER() OVER (PARTITION BY start_time ORDER BY
created_at) AS rn
    FROM public.time_slots
  ) t
  WHERE rn > 1
);

-- Add unique constraint on start_time
ALTER TABLE public.time_slots
ADD CONSTRAINT time_slots_start_time_key UNIQUE (start_time);
```

supabase/migrations/20260819180000_create_admin_account.sql

```
-- Create the pre-seeded admin account (email: admin@gmail.com, password:
admin123)
-- Uses a pre-computed bcrypt hash so pgcrypto is not required at migration time

-- Only insert if the user doesn't already exist
INSERT INTO auth.users (
  instance_id,
  id,
  aud,
  role,
  email,
  encrypted_password,
  email_confirmed_at,
  invited_at,
  created_at,
  updated_at,
  raw_app_meta_data,
  raw_user_meta_data,
  is_super_admin
)
SELECT
  '00000000-0000-0000-0000-000000000000'::uuid AS instance_id,
  gen_random_uuid() AS id,
  'authenticated' AS aud,
  'authenticated' AS role,
  'admin@gmail.com' AS email,
  '$2a$12$fRGSzQw5vY/AjVSh9t0K3evMKOtcvSTEls1/URDIUbH4OH53/Q1c6'
AS encrypted_password,
  now() AS email_confirmed_at,
  now() AS invited_at,
  now() AS created_at,
  now() AS updated_at,
  '{"provider":"email","providers":["email"]}'::jsonb AS raw_app_meta_data,
  '{}'::jsonb AS raw_user_meta_data,
  false AS is_super_admin
WHERE NOT EXISTS (
  SELECT 1 FROM auth.users WHERE email = 'admin@gmail.com'
);

-- Grant admin role to the user
INSERT INTO public.user_roles (user_id, role)
SELECT id, 'admin'
FROM auth.users
```

```
WHERE email = 'admin@gmail.com'
ON CONFLICT (user_id, role) DO NOTHING;
```

supabase/migrations/20260819190000_price_history_table.sql

```
-- Price change history tracking
CREATE TABLE IF NOT EXISTS public.price_history (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  table_name text NOT NULL,
  item_id uuid NOT NULL,
  item_name text NOT NULL,
  old_price numeric(10,2) NOT NULL,
  new_price numeric(10,2) NOT NULL,
  reason text,
  changed_by uuid REFERENCES auth.users(id),
  created_at timestamptz NOT NULL DEFAULT now()
);

GRANT SELECT, INSERT, UPDATE, DELETE ON public.price_history TO
authenticated;
GRANT ALL ON public.price_history TO service_role;

ALTER TABLE public.price_history ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "price history admin all" ON public.price_history;
CREATE POLICY "price history admin all" ON public.price_history
  FOR ALL TO authenticated
  USING (public.has_role(auth.uid(), 'admin'))
  WITH CHECK (public.has_role(auth.uid(), 'admin'));

CREATE INDEX IF NOT EXISTS price_history_table_item_idx
  ON public.price_history (table_name, item_id);
```

supabase/migrations/20260819193343_expand_ph_motorcycle_catalog.sql

---

```
-- Expand Philippines motorcycle catalog: add localized brands and models
-- These represent popular motorcycle models in the Philippine market

INSERT INTO public.products (name, category, brand, description, price,
in_stock, is_featured, sort_order)
SELECT * FROM (
  VALUES
  ('Honda CB150R','motorcycle','Honda','Streetfighter 150cc with aggressive
styling.',129000.00,true,true,10),
  ('Honda Beat','motorcycle','Honda','Popular underbone 110cc for daily
commuting.',79000.00,true,true,11),
  ('Honda PCX160','motorcycle','Honda','Popular underbone 158cc for daily
commuting.',119000.00,true,true,12),
  ('Yamaha Mio','motorcycle','Yamaha','125cc underbone, the king of city
rides.',78000.00,true,true,13),
  ('Yamaha NMAX','motorcycle','Yamaha','155cc automatic scooter for urban
commuters.',125000.00,true,true,14),
  ('Yamaha MT-15','motorcycle','Yamaha','Naked 155cc with aggressive
styling.',135000.00,true,true,15),
  ('Yamaha R15','motorcycle','Yamaha','155cc sport bike with VVA
engine.',149000.00,true,false,16),
  ('Suzuki GSX-S150','motorcycle','Suzuki','150cc sporty
streetfighter.',119000.00,true,true,18),
  ('Kawasaki Z150','motorcycle','Kawasaki','150cc street naked with sharp
styling.',124000.00,true,false,20),
  ('KTM 125 Duke','motorcycle','KTM','125cc naked with race-bred
handling.',185000.00,true,false,21),
  ('Royal Enfield Classic 350','motorcycle','Royal Enfield','350cc classic cruiser
with modern fuel injection.',225000.00,true,false,22),
  ('TVS Apache RTR 160','motorcycle','TVS','160cc sporty naked with race-tuned
engine.',98000.00,true,false,23)
) AS v(name, category, brand, description, price, in_stock, is_featured,
sort_order)
WHERE NOT EXISTS (
  SELECT 1 FROM public.products p
  WHERE p.name = v.name AND p.category = 'motorcycle'
);

-- Rename Yamaha WR155R (less common PH model) to Yamaha FZ-S
(popular PH model)
UPDATE public.products
SET name = 'Yamaha FZ-S', description = '250cc naked with sharp styling.'
WHERE name = 'Yamaha WR155R' AND brand = 'Yamaha' AND category =
```

'motorcycle';

supabase/migrations/20260819195215_cleanup_duplicate_motorcycles.sql

---

```
-- Clean up duplicate motorcycle products, keeping the earliest created
DELETE FROM public.products a
USING public.products b
WHERE a.id != b.id
  AND a.name = b.name
  AND a.category = 'motorcycle'
  AND a.category = b.category
  AND a.created_at > b.created_at;
```

supabase/migrations/20260820113112_crew_scheduling_schema.sql

---

```
-- Crew scheduling: weekly rosters and availability exceptions

-- crew_schedules: weekly recurring work schedule per crew member
CREATE TABLE IF NOT EXISTS public.crew_schedules (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  crew_id uuid NOT NULL REFERENCES public.crew_members(id) ON DELETE
CASCADE,
  day_of_week integer NOT NULL CHECK (day_of_week BETWEEN 0 AND 6),
  start_time time NOT NULL,
  end_time time NOT NULL,
  is_working boolean NOT NULL DEFAULT true,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (crew_id, day_of_week)
);
GRANT SELECT, INSERT, UPDATE, DELETE ON public.crew_schedules TO
authenticated;
GRANT ALL ON public.crew_schedules TO service_role;
ALTER TABLE public.crew_schedules ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "crew_schedules admin all" ON
public.crew_schedules;
CREATE POLICY "crew_schedules admin all" ON public.crew_schedules FOR
ALL TO authenticated USING (public.has_role(auth.uid(),'admin')) WITH CHECK
(public.has_role(auth.uid(),'admin'));

-- crew_availability_exceptions: temporary unavailability (leave, sick, day off,
emergency)
CREATE TABLE IF NOT EXISTS public.crew_availability_exceptions (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  crew_id uuid NOT NULL REFERENCES public.crew_members(id) ON DELETE
CASCADE,
  start_date date NOT NULL,
  end_date date NOT NULL,
  start_time time,
  end_time time,
  reason text NOT NULL CHECK (reason IN ('leave', 'sick', 'day_off', 'emergency',
'special_schedule')),
  note text,
  is_all_day boolean NOT NULL DEFAULT true,
  created_at timestamptz NOT NULL DEFAULT now()
);
GRANT SELECT, INSERT, UPDATE, DELETE ON
public.crew_availability_exceptions TO authenticated;
```

```
GRANT ALL ON public.crew_availability_exceptions TO service_role;
ALTER TABLE public.crew_availability_exceptions ENABLE ROW LEVEL
SECURITY;
DROP POLICY IF EXISTS "crew_exceptions admin all" ON
public.crew_availability_exceptions;
CREATE POLICY "crew_exceptions admin all" ON
public.crew_availability_exceptions FOR ALL TO authenticated USING
(public.has_role(auth.uid(),'admin')) WITH CHECK
(public.has_role(auth.uid(),'admin'));

-- Add updated_at trigger to crew_schedules
DROP TRIGGER IF EXISTS crew_schedules_updated ON
public.crew_schedules;
CREATE TRIGGER crew_schedules_updated BEFORE UPDATE ON
public.crew_schedules FOR EACH ROW EXECUTE FUNCTION
public.update_updated_at_column();
```

supabase/migrations/20260820161821_crew_schedules_specific_dates.sql

---

```
-- Extend crew_schedules to support specific date scheduling (for one-off
schedule overrides)
ALTER TABLE public.crew_schedules
  ADD COLUMN IF NOT EXISTS schedule_date date,
  ADD COLUMN IF NOT EXISTS effective_from date,
  ADD COLUMN IF NOT EXISTS effective_to date;

-- Index for efficient date-specific schedule lookups
CREATE INDEX IF NOT EXISTS crew_schedules_date_idx ON
public.crew_schedules (schedule_date);
CREATE INDEX IF NOT EXISTS crew_schedules_effective_idx ON
public.crew_schedules (effective_from, effective_to);

COMMENT ON COLUMN public.crew_schedules.schedule_date IS 'Specific
date for one-off schedule override. If NULL, the schedule is a weekly recurring
pattern based on day_of_week.';
COMMENT ON COLUMN public.crew_schedules.effective_from IS 'Start of a
date range during which this weekly pattern is active. If NULL, always active.';
COMMENT ON COLUMN public.crew_schedules.effective_to IS 'End of a date
range during which this weekly pattern is active. If NULL, no end date.';
```

supabase/migrations/20260820162150_crew_schedules_note.sql

```
-- Add note column to crew_schedules for schedule-specific notes (e.g.,
"unavailable" status)
ALTER TABLE public.crew_schedules ADD COLUMN IF NOT EXISTS note text;
COMMENT ON COLUMN public.crew_schedules.note IS 'Optional note for the
schedule entry, e.g. reason for unavailability.';
```

supabase/migrations/20260820163024_crew_schedules_nullable_times.sql

---

```
-- Allow NULL start_time/end_time for non-working schedule overrides
ALTER TABLE public.crew_schedules ALTER COLUMN start_time DROP NOT
NULL;
ALTER TABLE public.crew_schedules ALTER COLUMN end_time DROP NOT
NULL;
```

supabase/migrations/20260820190000_crew_schedules_date_override_unique.sql

---

```
-- Add partial unique index for date-specific schedule overrides
-- Weekly schedules (schedule_date IS NULL) use the existing UNIQUE (crew_id, day_of_week)
-- Date-specific overrides (schedule_date IS NOT NULL) use this new partial unique index

CREATE UNIQUE INDEX IF NOT EXISTS crew_schedules_date_override_idx
ON public.crew_schedules (crew_id, schedule_date)
WHERE schedule_date IS NOT NULL;
```

supabase/migrations/20260820214127_remove_seeded_weekly_schedules.sql

---

```
-- Remove seeded weekly crew schedules (Mon-Sat 8:00-17:00) so mechanics
-- only appear on days that are explicitly scheduled via the admin UI.
DELETE FROM public.crew_schedules
WHERE schedule_date IS NULL
  AND start_time = '08:00:00'::time
  AND end_time = '17:00:00'::time
  AND is_working = true
  AND day_of_week IN (1, 2, 3, 4, 5, 6);
```

supabase/migrations/20260820224500_fix_crew_schedules_upsert_conflict.sql

---

```
-- Fix ON CONFLICT target for date-specific schedule upserts.
-- Problem: crew_schedules_date_override_idx was a PARTIAL unique index
-- (WHERE schedule_date IS NOT NULL), which Postgres cannot use for ON CONFLICT.
--
-- Solution:
--  1. Replace the partial date_override index with a NON-PARTIAL unique index
--     on (crew_id, schedule_date).  Postgres treats NULL as distinct in unique
--     indexes, so multiple weekly schedules (NULL schedule_date) per crew_id
--     are still allowed.
--  2. Replace the table-level UNIQUE (crew_id, day_of_week) with a PARTIAL
--     unique index that only covers weekly schedules (schedule_date IS NULL).
--     This allows date-specific overrides to coexist with weekly schedules
--     even when they share the same day_of_week.

-- Step 1: Drop the table-level unique constraint (auto-generated index)
ALTER TABLE public.crew_schedules
  DROP CONSTRAINT IF EXISTS crew_schedules_crew_id_day_of_week_key;

-- Step 1a: Create partial unique index for weekly schedules only
CREATE UNIQUE INDEX IF NOT EXISTS crew_schedules_weekly_unique
  ON public.crew_schedules (crew_id, day_of_week)
  WHERE schedule_date IS NULL;

-- Step 2: Drop the old PARTIAL date_override index
DROP INDEX IF EXISTS crew_schedules_date_override_idx;

-- Step 2a: Create NON-PARTIAL unique index for date-specific schedules.
-- NULL schedule_date values are treated as distinct, so weekly schedules
-- (NULL) are allowed to coexist.
CREATE UNIQUE INDEX IF NOT EXISTS crew_schedules_date_override_idx
  ON public.crew_schedules (crew_id, schedule_date);
```

supabase/migrations/20260821150500_fix_crew_schedules_updated_at.sql

---

```
-- Add missing updated_at column to crew_schedules (trigger references it but column was absent)
ALTER TABLE public.crew_schedules ADD COLUMN IF NOT EXISTS updated_at timestamptz NOT NULL DEFAULT now();
```

supabase/migrations/20260821161200_create_products_storage_bucket.sql

---

```
-- Create storage bucket for product images
INSERT INTO storage.buckets (id, name, public)
VALUES ('products', 'products', true)
ON CONFLICT (id) DO NOTHING;

-- Allow authenticated admin users to upload/select/update/delete objects in the products bucket
INSERT INTO storage.objects (bucket_id, name, owner, created_at, updated_at)
SELECT 'products', '', NULL, now(), now()
WHERE FALSE;

-- Policies for the products storage bucket
DROP POLICY IF EXISTS "products admin upload" ON storage.objects;
CREATE POLICY "products admin upload" ON storage.objects
FOR INSERT TO authenticated WITH CHECK (bucket_id = 'products' AND public.has_role(auth.uid(), 'admin'));

DROP POLICY IF EXISTS "products admin update" ON storage.objects;
CREATE POLICY "products admin update" ON storage.objects
FOR UPDATE TO authenticated USING (bucket_id = 'products' AND public.has_role(auth.uid(), 'admin'));

DROP POLICY IF EXISTS "products admin delete" ON storage.objects;
CREATE POLICY "products admin delete" ON storage.objects
FOR DELETE TO authenticated USING (bucket_id = 'products' AND public.has_role(auth.uid(), 'admin'));

DROP POLICY IF EXISTS "products public read" ON storage.objects;
CREATE POLICY "products public read" ON storage.objects
FOR SELECT TO anon, authenticated USING (bucket_id = 'products');
```

supabase/migrations/20260821185300_dedupe_services_add_name_unique.sql

---

```
-- Deduplicate services by name (cleanup) and add a unique index to prevent
future duplicates
-- Two seed migrations (20260816152359 and 20260818222449) both insert the
same services
-- without a unique constraint on name, creating duplicate rows with different
UUIDs.

DELETE FROM public.services a
USING public.services b
WHERE a.id != b.id
  AND a.name = b.name
  AND a.created_at > b.created_at;

CREATE UNIQUE INDEX IF NOT EXISTS services_name_idx ON
public.services (name);
```

supabase/migrations/20260821194500_dedupe_products_services.sql

---

```
-- Clean up duplicate products and services, and ensure unique indexes exist

-- Products: remove duplicates by name, keeping the earliest created_at
DELETE FROM public.products a
USING public.products b
WHERE a.id != b.id
  AND a.name = b.name
  AND a.created_at > b.created_at;

-- Services: remove duplicates by name, keeping the earliest created_at
DELETE FROM public.services a
USING public.services b
WHERE a.id != b.id
  AND a.name = b.name
  AND a.created_at > b.created_at;

-- Ensure unique indexes exist on name columns
CREATE UNIQUE INDEX IF NOT EXISTS products_name_idx ON
public.products (name);
CREATE UNIQUE INDEX IF NOT EXISTS services_name_idx ON
public.services (name);
```

supabase/migrations/20260822125500_dedupe_crew_members_add_name_unique.sql

---

```
-- Deduplicate crew_members and add unique index on name to prevent future duplicates.

-- 1. Delete duplicate crew_members, keeping the one with the smallest id per name
DELETE FROM public.crew_members a
USING public.crew_members b
WHERE a.name = b.name
  AND a.id > b.id;

-- 2. Add unique index on name to prevent future duplicates
CREATE UNIQUE INDEX IF NOT EXISTS crew_members_name_idx
  ON public.crew_members (name);

-- 3. Add updated_at column and trigger for consistency
ALTER TABLE public.crew_members
  ADD COLUMN IF NOT EXISTS updated_at timestamptz NOT NULL DEFAULT now();

DROP TRIGGER IF EXISTS update_crew_members_updated_at ON public.crew_members;
CREATE TRIGGER update_crew_members_updated_at
  BEFORE UPDATE ON public.crew_members
  FOR EACH ROW
  EXECUTE FUNCTION public.update_updated_at_column();
```

supabase/migrations/20260822133700_create_blocked_numbers_table.sql

---

```
-- Blocked phone numbers for the online booking form.
-- When a customer with a blocked number tries to book, the request is rejected.

CREATE TABLE IF NOT EXISTS public.blocked_numbers (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  phone text NOT NULL,
  reason text,
  created_at timestamptz NOT NULL DEFAULT now(),
  created_by uuid REFERENCES auth.users(id) ON DELETE SET NULL
);

-- Normalize phone to lowercase for case-insensitive uniqueness
CREATE UNIQUE INDEX IF NOT EXISTS blocked_numbers_phone_idx
  ON public.blocked_numbers (LOWER(phone));

GRANT SELECT, INSERT, UPDATE, DELETE ON public.blocked_numbers TO
authenticated;
GRANT ALL ON public.blocked_numbers TO service_role;
ALTER TABLE public.blocked_numbers ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "blocked_numbers admin all" ON
public.blocked_numbers;
CREATE POLICY "blocked_numbers admin all" ON public.blocked_numbers
  FOR ALL TO authenticated
  USING (public.has_role(auth.uid(), 'admin'))
  WITH CHECK (public.has_role(auth.uid(), 'admin'));
```

supabase/migrations/20260823123700_seed_default_crew_schedules.sql

---

```
-- Re-seed default weekly crew schedules (Mon–Sat 08:00–17:00) for every
active
-- mechanic after remove_seeded_weekly_schedules.sql ran (which deleted
nothing on
-- a fresh DB). Without these schedules the booking calendar shows zero
selectable
-- dates because every date is filtered out by the hasMechanic check in book.tsx.
INSERT INTO public.crew_schedules (crew_id, day_of_week, start_time,
end_time, is_working)
SELECT cm.id, v.dow, '08:00:00'::time, '17:00:00'::time, true
FROM public.crew_members cm
CROSS JOIN (VALUES (1), (2), (3), (4), (5), (6)) AS v(dow)
WHERE cm.is_active = true
  AND NOT EXISTS (
    SELECT 1
    FROM public.crew_schedules cs
    WHERE cs.crew_id = cm.id
      AND cs.day_of_week = v.dow
      AND cs.schedule_date IS NULL
  );
```

supabase/migrations/20260823140000_remove_default_weekly_crew_schedules.sql

---

```
-- The availability admin UI assigns crews to individual dates. Remove the
-- recurring defaults that could make a mechanic appear assigned when no
-- date-specific assignment was created.
DELETE FROM public.crew_schedules
WHERE schedule_date IS NULL
  AND start_time = '08:00:00'::time
  AND end_time = '17:00:00'::time
  AND is_working = true
  AND day_of_week IN (1, 2, 3, 4, 5, 6);
```

supabase/migrations/20260823150000_add_booking_duration_capacity_guard.sql

---

```
-- Persist appointment durations so availability remains correct even if a
-- service is later edited or archived.
ALTER TABLE public.appointment_services
  ADD COLUMN IF NOT EXISTS duration_minutes integer NOT NULL
DEFAULT 60
  CHECK (duration_minutes > 0);

UPDATE public.appointment_services aps
SET duration_minutes = services.duration_minutes
FROM public.services services
WHERE aps.service_id = services.id;

ALTER TABLE public.appointments
  ADD COLUMN IF NOT EXISTS booking_duration_minutes integer NOT NULL
DEFAULT 75
  CHECK (booking_duration_minutes > 0);

UPDATE public.appointments appointments
SET booking_duration_minutes = COALESCE(
  (
    SELECT SUM(aps.duration_minutes + 15)
    FROM public.appointment_services aps
    WHERE aps.appointment_id = appointments.id
  ),
  75
);

-- An atomic database guard prevents two final availability checks from booking
-- the same crew member into overlapping time ranges.
CREATE EXTENSION IF NOT EXISTS btree_gist;

ALTER TABLE public.appointments
  DROP CONSTRAINT IF EXISTS appointments_crew_time_no_overlap;

ALTER TABLE public.appointments
  ADD CONSTRAINT appointments_crew_time_no_overlap
  EXCLUDE USING gist (
    assigned_crew_id WITH =,
    tsrange(
      appointment_date + start_time,
      appointment_date + start_time + make_interval(mins =>
booking_duration_minutes),
```

```
    '[)'
  ) WITH &&
)
WHERE (
  is_archived = false
  AND status IN ('pending', 'confirmed', 'in_progress')
);
```

supabase/migrations/20260824090000_add_blocked_numbers_archiving.sql

---

```
-- Keep removed blocked numbers available for restoration in the admin archive.
ALTER TABLE public.blocked_numbers
  ADD COLUMN IF NOT EXISTS is_archived boolean NOT NULL DEFAULT false;

-- Existing blocked numbers remain active unless explicitly archived.
UPDATE public.blocked_numbers
  SET is_archived = false
  WHERE is_archived IS DISTINCT FROM false;

CREATE INDEX IF NOT EXISTS blocked_numbers_is_archived_idx
  ON public.blocked_numbers (is_archived);
```

supabase/migrations/20260824100000_create_booking_atomic.sql

```
-- A booking must either be fully recorded (appointment, services and staff
-- notification) or not be created at all. PostgreSQL executes this function
-- as one transaction, so any failed insert rolls the entire booking back.
CREATE OR REPLACE FUNCTION public.create_booking_atomic(
  p_reference_code text,
  p_customer_name text,
  p_phone text,
  p_email text,
  p_moto_brand text,
  p_moto_model text,
  p_moto_variant text,
  p_moto_year integer,
  p_plate_number text,
  p_appointment_date date,
  p_start_time time,
  p_notes text,
  p_total_estimate numeric,
  p_booking_duration_minutes integer,
  p_assigned_crew_id uuid,
  p_services jsonb,
  p_notification_title text,
  p_notification_message text
)
RETURNS TABLE (appointment_id uuid, reference_code text)
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment_id uuid;
  v_reference_code text;
BEGIN
  IF jsonb_typeof(p_services) <> 'array' OR jsonb_array_length(p_services) = 0
THEN
    RAISE EXCEPTION 'A booking must include at least one service.';
  END IF;

  INSERT INTO public.appointments (
    reference_code,
    customer_name,
    phone,
    email,
    moto_brand,
```

```
  moto_model,
  moto_variant,
  moto_year,
  plate_number,
  appointment_date,
  start_time,
  notes,
  total_estimate,
  booking_duration_minutes,
  terms_accepted,
  assigned_crew_id
)
VALUES (
  p_reference_code,
  p_customer_name,
  p_phone,
  p_email,
  p_moto_brand,
  p_moto_model,
  p_moto_variant,
  p_moto_year,
  p_plate_number,
  p_appointment_date,
  p_start_time,
  p_notes,
  p_total_estimate,
  p_booking_duration_minutes,
  true,
  p_assigned_crew_id
)
RETURNING id, reference_code INTO v_appointment_id, v_reference_code;

INSERT INTO public.appointment_services (
  appointment_id,
  service_id,
  service_name,
  price,
  duration_minutes
)
SELECT
  v_appointment_id,
  service.service_id,
  service.service_name,
  service.price,
  service.duration_minutes
```

```
  FROM jsonb_to_recordset(p_services) AS service(
    service_id uuid,
    service_name text,
    price numeric,
    duration_minutes integer
  );

  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES (
    'new_appointment',
    p_notification_title,
    p_notification_message,
    v_appointment_id
  );

  RETURN QUERY SELECT v_appointment_id, v_reference_code;
END;
$$;

-- This RPC is called only by the server-side Supabase service-role client.
REVOKE ALL ON FUNCTION public.create_booking_atomic(
  text, text, text, text, text, text, text, integer, text, date, time, text,
  numeric, integer, uuid, jsonb, text, text
) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.create_booking_atomic(
  text, text, text, text, text, text, text, integer, text, date, time, text,
  numeric, integer, uuid, jsonb, text, text
) TO service_role;
```

supabase/migrations/20260824110000_normalize_customer_phone_numbers.sql

---

```
-- Store customer and blocked-number mobile values in one canonical E.164
-- format. The application accepts both local 09XXXXXXXXX and
+639XXXXXXXXX
-- input, then saves +639XXXXXXXXX.
WITH normalized AS (
  SELECT
    id,
    CASE
      WHEN regexp_replace(phone, '[^0-9]', '', 'g') ~ '^09[0-9]{9}$'
        THEN '+63' || substring(regexp_replace(phone, '[^0-9]', '', 'g') FROM 2)
      WHEN regexp_replace(phone, '[^0-9]', '', 'g') ~ '^639[0-9]{9}$'
        THEN '+' || regexp_replace(phone, '[^0-9]', '', 'g')
    END AS canonical_phone
  FROM public.appointments
)
UPDATE public.appointments AS appointment
SET phone = normalized.canonical_phone
FROM normalized
WHERE appointment.id = normalized.id
  AND normalized.canonical_phone IS NOT NULL
  AND appointment.phone IS DISTINCT FROM normalized.canonical_phone;

WITH normalized AS (
  SELECT
    id,
    CASE
      WHEN regexp_replace(phone, '[^0-9]', '', 'g') ~ '^09[0-9]{9}$'
        THEN '+63' || substring(regexp_replace(phone, '[^0-9]', '', 'g') FROM 2)
      WHEN regexp_replace(phone, '[^0-9]', '', 'g') ~ '^639[0-9]{9}$'
        THEN '+' || regexp_replace(phone, '[^0-9]', '', 'g')
    END AS canonical_phone
  FROM public.blocked_numbers
)
UPDATE public.blocked_numbers AS blocked_number
SET phone = normalized.canonical_phone
FROM normalized
WHERE blocked_number.id = normalized.id
  AND normalized.canonical_phone IS NOT NULL
  AND blocked_number.phone IS DISTINCT FROM
normalized.canonical_phone;

-- Keep legacy malformed rows readable during the migration, while ensuring
```

```
-- every newly written or updated phone number uses the canonical format.
ALTER TABLE public.appointments
  DROP CONSTRAINT IF EXISTS appointments_phone_e164_check;
ALTER TABLE public.appointments
  ADD CONSTRAINT appointments_phone_e164_check
  CHECK (phone ~ '^[+]639[0-9]{9}$') NOT VALID;

ALTER TABLE public.blocked_numbers
  DROP CONSTRAINT IF EXISTS blocked_numbers_phone_e164_check;
ALTER TABLE public.blocked_numbers
  ADD CONSTRAINT blocked_numbers_phone_e164_check
  CHECK (phone ~ '^[+]639[0-9]{9}$') NOT VALID;
```

supabase/migrations/20260824120000_add_public_endpoint_rate_limits.sql

```
-- Persistent limits for unauthenticated booking endpoints. The application
-- calls this only with its service-role client, so the counters are not
-- exposed through the public Supabase API.
CREATE TABLE IF NOT EXISTS public.public_endpoint_rate_limits (
  scope text NOT NULL CHECK (char_length(scope) BETWEEN 1 AND 64),
  subject text NOT NULL CHECK (char_length(subject) BETWEEN 1 AND 128),
  window_started_at timestamptz NOT NULL DEFAULT now(),
  request_count integer NOT NULL DEFAULT 1 CHECK (request_count > 0),
  updated_at timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (scope, subject)
);

GRANT ALL ON public.public_endpoint_rate_limits TO service_role;
ALTER TABLE public.public_endpoint_rate_limits ENABLE ROW LEVEL
SECURITY;

CREATE OR REPLACE FUNCTION public.enforce_public_rate_limit(
  p_scope text,
  p_subject text,
  p_limit integer,
  p_window_seconds integer
)
RETURNS boolean
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_allowed boolean;
BEGIN
  IF p_limit < 1 OR p_window_seconds < 1 THEN
    RAISE EXCEPTION 'Rate-limit configuration must be positive.';
  END IF;

  -- Keep the small counter table bounded without relying on a scheduled job.
  DELETE FROM public.public_endpoint_rate_limits
  WHERE scope = p_scope
    AND window_started_at < now() - make_interval(secs =>
GREATEST(p_window_seconds * 2, 86400));

  INSERT INTO public.public_endpoint_rate_limits (
    scope,
    subject,
```

```
    window_started_at,
    request_count,
    updated_at
  )
  VALUES (p_scope, p_subject, now(), 1, now())
  ON CONFLICT (scope, subject) DO UPDATE
  SET
    window_started_at = CASE
      WHEN public_endpoint_rate_limits.window_started_at
        <= now() - make_interval(secs => p_window_seconds)
        THEN now()
      ELSE public_endpoint_rate_limits.window_started_at
    END,
    request_count = CASE
      WHEN public_endpoint_rate_limits.window_started_at
        <= now() - make_interval(secs => p_window_seconds)
        THEN 1
      ELSE public_endpoint_rate_limits.request_count + 1
    END,
    updated_at = now()
  RETURNING request_count <= p_limit INTO v_allowed;

  RETURN COALESCE(v_allowed, false);
END;
$$;

REVOKE ALL ON FUNCTION public.enforce_public_rate_limit(text, text, integer,
integer) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.enforce_public_rate_limit(text, text,
integer, integer)
  TO service_role;
```

supabase/migrations/20260824130000_add_booking_idempotency.sql

---

```
-- A client-generated request ID makes a retry safe when a booking succeeds but
-- its response is lost. Each ID may create at most one appointment.
ALTER TABLE public.appointments
  ADD COLUMN IF NOT EXISTS booking_request_id uuid;

CREATE UNIQUE INDEX IF NOT EXISTS
appointments_booking_request_id_unique
  ON public.appointments (booking_request_id)
  WHERE booking_request_id IS NOT NULL;

DROP FUNCTION IF EXISTS public.create_booking_atomic(
  text, text, text, text, text, text, text, integer, text, date, time, text,
  numeric, integer, uuid, jsonb, text, text
);

CREATE OR REPLACE FUNCTION public.create_booking_atomic(
  p_reference_code text,
  p_booking_request_id uuid,
  p_customer_name text,
  p_phone text,
  p_email text,
  p_moto_brand text,
  p_moto_model text,
  p_moto_variant text,
  p_moto_year integer,
  p_plate_number text,
  p_appointment_date date,
  p_start_time time,
  p_notes text,
  p_total_estimate numeric,
  p_booking_duration_minutes integer,
  p_assigned_crew_id uuid,
  p_services jsonb,
  p_notification_title text,
  p_notification_message text
)
RETURNS TABLE (appointment_id uuid, reference_code text)
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment_id uuid;
```

```
  v_reference_code text;
BEGIN
  IF p_booking_request_id IS NULL THEN
    RAISE EXCEPTION 'A booking request ID is required.';
  END IF;

  IF jsonb_typeof(p_services) <> 'array' OR jsonb_array_length(p_services) = 0
THEN
    RAISE EXCEPTION 'A booking must include at least one service.';
  END IF;

  INSERT INTO public.appointments (
    reference_code,
    booking_request_id,
    customer_name,
    phone,
    email,
    moto_brand,
    moto_model,
    moto_variant,
    moto_year,
    plate_number,
    appointment_date,
    start_time,
    notes,
    total_estimate,
    booking_duration_minutes,
    terms_accepted,
    assigned_crew_id
  )
  VALUES (
    p_reference_code,
    p_booking_request_id,
    p_customer_name,
    p_phone,
    p_email,
    p_moto_brand,
    p_moto_model,
    p_moto_variant,
    p_moto_year,
    p_plate_number,
    p_appointment_date,
    p_start_time,
    p_notes,
    p_total_estimate,
```

```
    p_booking_duration_minutes,
    true,
    p_assigned_crew_id
  )
  ON CONFLICT DO NOTHING
  RETURNING appointments.id, appointments.reference_code INTO
v_appointment_id, v_reference_code;

  IF v_appointment_id IS NULL THEN
    SELECT appointments.id, appointments.reference_code
    INTO v_appointment_id, v_reference_code
    FROM public.appointments AS appointments
    WHERE appointments.booking_request_id = p_booking_request_id;

    IF FOUND THEN
      RETURN QUERY SELECT v_appointment_id, v_reference_code;
      RETURN;
    END IF;

    -- The conflict belongs to capacity or a rare reference-code collision,
    -- not this request ID. Preserve the app's existing retryable response.
    RAISE EXCEPTION 'The requested schedule is no longer available.' USING
ERRCODE = '23P01';
  END IF;

  INSERT INTO public.appointment_services (
    appointment_id,
    service_id,
    service_name,
    price,
    duration_minutes
  )
  SELECT
    v_appointment_id,
    service.service_id,
    service.service_name,
    service.price,
    service.duration_minutes
  FROM jsonb_to_recordset(p_services) AS service(
    service_id uuid,
    service_name text,
    price numeric,
    duration_minutes integer
  );
```

```
  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES (
    'new_appointment',
    p_notification_title,
    p_notification_message,
    v_appointment_id
  );

  RETURN QUERY SELECT v_appointment_id, v_reference_code;
END;
$$;

REVOKE ALL ON FUNCTION public.create_booking_atomic(
  text, uuid, text, text, text, text, text, text, integer, text, date, time,
  text, numeric, integer, uuid, jsonb, text, text
) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.create_booking_atomic(
  text, uuid, text, text, text, text, text, text, integer, text, date, time,
  text, numeric, integer, uuid, jsonb, text, text
) TO service_role;
```

supabase/migrations/20260828120000_fix_create_booking_atomic_reference_ambiguity.sql

---

```
-- The function's OUT column name `reference_code` is also a column on
-- appointments. Qualify every reference so PostgreSQL does not treat it as an
-- ambiguous PL/pgSQL variable during a final booking write.
CREATE OR REPLACE FUNCTION public.create_booking_atomic(
  p_reference_code text,
  p_booking_request_id uuid,
  p_customer_name text,
  p_phone text,
  p_email text,
  p_moto_brand text,
  p_moto_model text,
  p_moto_variant text,
  p_moto_year integer,
  p_plate_number text,
  p_appointment_date date,
  p_start_time time,
  p_notes text,
  p_total_estimate numeric,
  p_booking_duration_minutes integer,
  p_assigned_crew_id uuid,
  p_services jsonb,
  p_notification_title text,
  p_notification_message text
)
RETURNS TABLE (appointment_id uuid, reference_code text)
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment_id uuid;
  v_reference_code text;
BEGIN
  IF p_booking_request_id IS NULL THEN
    RAISE EXCEPTION 'A booking request ID is required.';
  END IF;

  IF jsonb_typeof(p_services) <> 'array' OR jsonb_array_length(p_services) = 0
THEN
    RAISE EXCEPTION 'A booking must include at least one service.';
  END IF;
```

```
  INSERT INTO public.appointments AS appointments (
    reference_code,
    booking_request_id,
    customer_name,
    phone,
    email,
    moto_brand,
    moto_model,
    moto_variant,
    moto_year,
    plate_number,
    appointment_date,
    start_time,
    notes,
    total_estimate,
    booking_duration_minutes,
    terms_accepted,
    assigned_crew_id
  )
  VALUES (
    p_reference_code,
    p_booking_request_id,
    p_customer_name,
    p_phone,
    p_email,
    p_moto_brand,
    p_moto_model,
    p_moto_variant,
    p_moto_year,
    p_plate_number,
    p_appointment_date,
    p_start_time,
    p_notes,
    p_total_estimate,
    p_booking_duration_minutes,
    true,
    p_assigned_crew_id
  )
  ON CONFLICT DO NOTHING
  RETURNING appointments.id, appointments.reference_code INTO
v_appointment_id, v_reference_code;

  IF v_appointment_id IS NULL THEN
    SELECT appointments.id, appointments.reference_code
    INTO v_appointment_id, v_reference_code
```

```
    FROM public.appointments AS appointments
    WHERE appointments.booking_request_id = p_booking_request_id;

    IF FOUND THEN
      RETURN QUERY SELECT v_appointment_id, v_reference_code;
      RETURN;
    END IF;

    RAISE EXCEPTION 'The requested schedule is no longer available.' USING
ERRCODE = '23P01';
  END IF;

  INSERT INTO public.appointment_services (
    appointment_id,
    service_id,
    service_name,
    price,
    duration_minutes
  )
  SELECT
    v_appointment_id,
    service.service_id,
    service.service_name,
    service.price,
    service.duration_minutes
  FROM jsonb_to_recordset(p_services) AS service(
    service_id uuid,
    service_name text,
    price numeric,
    duration_minutes integer
  );

  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES (
    'new_appointment',
    p_notification_title,
    p_notification_message,
    v_appointment_id
  );

  RETURN QUERY SELECT v_appointment_id, v_reference_code;
END;
$$;

REVOKE ALL ON FUNCTION public.create_booking_atomic(
```

```
  text, uuid, text, text, text, text, text, text, integer, text, date, time,
  text, numeric, integer, uuid, jsonb, text, text
) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.create_booking_atomic(
  text, uuid, text, text, text, text, text, text, integer, text, date, time,
  text, numeric, integer, uuid, jsonb, text, text
) TO service_role;
```

supabase/migrations/20260830120000_separate_deactivation_from_archiving.sql

---

```
-- A record can be deactivated (not visible to customers) without being archived.
ALTER TABLE public.services
  ADD COLUMN IF NOT EXISTS is_archived boolean NOT NULL DEFAULT false;

ALTER TABLE public.products
  ADD COLUMN IF NOT EXISTS is_archived boolean NOT NULL DEFAULT false;

ALTER TABLE public.crew_members
  ADD COLUMN IF NOT EXISTS is_archived boolean NOT NULL DEFAULT false;

-- Before this migration, inactive records were the archive. Preserve that existing meaning.
UPDATE public.services
SET is_archived = true
WHERE is_active = false AND is_archived = false;

UPDATE public.products
SET is_archived = true
WHERE is_active = false AND is_archived = false;

UPDATE public.crew_members
SET is_archived = true
WHERE is_active = false AND is_archived = false;

CREATE INDEX IF NOT EXISTS services_is_archived_idx ON public.services (is_archived);
CREATE INDEX IF NOT EXISTS products_is_archived_idx ON public.products (is_archived);
CREATE INDEX IF NOT EXISTS crew_members_is_archived_idx ON public.crew_members (is_archived);
```

supabase/migrations/20260902120000_create_admin_activity_logs.sql

---

```
-- Keep an auditable record of data-changing actions performed by
administrators.
CREATE TABLE IF NOT EXISTS public.admin_activity_logs (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  actor_id uuid REFERENCES auth.users(id) ON DELETE SET NULL,
  actor_email text,
  action text NOT NULL,
  resource_type text NOT NULL,
  target_label text NOT NULL,
  summary text NOT NULL,
  changed_fields text[] NOT NULL DEFAULT ARRAY[]::text[],
  activity_date date NOT NULL DEFAULT ((now() AT TIME ZONE
'Asia/Manila')::date),
  activity_time time NOT NULL DEFAULT ((now() AT TIME ZONE
'Asia/Manila')::time),
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE INDEX IF NOT EXISTS admin_activity_logs_created_at_idx
  ON public.admin_activity_logs (created_at DESC);
CREATE INDEX IF NOT EXISTS admin_activity_logs_filter_idx
  ON public.admin_activity_logs (activity_date DESC, activity_time DESC, action,
resource_type);

GRANT SELECT ON public.admin_activity_logs TO authenticated;
GRANT ALL ON public.admin_activity_logs TO service_role;

ALTER TABLE public.admin_activity_logs ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS "admin activity logs read" ON
public.admin_activity_logs;
CREATE POLICY "admin activity logs read" ON public.admin_activity_logs
  FOR SELECT TO authenticated
  USING (public.has_role(auth.uid(), 'admin'));

CREATE OR REPLACE FUNCTION public.purge_expired_admin_activity_logs()
RETURNS void
LANGUAGE sql
SECURITY DEFINER
SET search_path = public
AS $$
  DELETE FROM public.admin_activity_logs
  WHERE created_at < now() - interval '50 days';
$$;
```

```
REVOKE ALL ON FUNCTION public.purge_expired_admin_activity_logs()
FROM PUBLIC;

CREATE OR REPLACE FUNCTION public.record_admin_activity()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public, auth
AS $$
DECLARE
  v_actor_id uuid := auth.uid();
  v_claims text := current_setting('request.jwt.claims', true);
  v_actor_email text;
  v_old jsonb := CASE WHEN TG_OP IN ('UPDATE', 'DELETE') THEN
to_jsonb(OLD) ELSE '{}'::jsonb END;
  v_new jsonb := CASE WHEN TG_OP IN ('INSERT', 'UPDATE') THEN
to_jsonb(NEW) ELSE '{}'::jsonb END;
  v_record jsonb;
  v_action text;
  v_target text;
  v_changed_fields text[] := ARRAY[]::text[];
BEGIN
  -- Public/server-side activity (such as a customer booking) is not an admin
action.
  IF v_actor_id IS NULL OR NOT public.has_role(v_actor_id, 'admin') THEN
    IF TG_OP = 'DELETE' THEN
      RETURN OLD;
    END IF;
    RETURN NEW;
  END IF;

  IF v_claims IS NOT NULL AND v_claims <> '' THEN
    v_actor_email := nullif(v_claims::jsonb ->> 'email', '');
  END IF;

  IF TG_OP = 'UPDATE' THEN
    SELECT COALESCE(array_agg(changed.key ORDER BY changed.key),
ARRAY[]::text[])
    INTO v_changed_fields
    FROM jsonb_object_keys(v_new) AS changed(key)
    WHERE changed.key NOT IN ('created_at', 'updated_at')
      AND v_old -> changed.key IS DISTINCT FROM v_new -> changed.key;

    -- Do not record a no-op update or an update that only changed a timestamp.
```

```
    IF cardinality(v_changed_fields) = 0 THEN
      RETURN NEW;
    END IF;

    IF v_new ->> 'is_archived' IS DISTINCT FROM v_old ->> 'is_archived' THEN
      v_action := CASE WHEN (v_new ->> 'is_archived')::boolean THEN 'archived'
ELSE 'restored' END;
    ELSIF v_new ->> 'is_active' IS DISTINCT FROM v_old ->> 'is_active' THEN
      v_action := CASE WHEN (v_new ->> 'is_active')::boolean THEN 'activated'
ELSE 'deactivated' END;
    ELSIF v_new ->> 'status' IS DISTINCT FROM v_old ->> 'status' THEN
      v_action := 'status changed';
    ELSE
      v_action := 'updated';
    END IF;
  ELSIF TG_OP = 'INSERT' THEN
    v_action := 'created';
  ELSE
    v_action := 'deleted';
  END IF;

  v_record := CASE WHEN TG_OP = 'DELETE' THEN v_old ELSE v_new END;
  v_target := COALESCE(
    nullif(v_record ->> 'reference_code', ''),
    nullif(v_record ->> 'name', ''),
    nullif(v_record ->> 'phone', ''),
    nullif(v_record ->> 'service_name', ''),
    nullif(v_record ->> 'block_date', ''),
    nullif(v_record ->> 'id', ''),
    'Record'
  );

  PERFORM public.purge_expired_admin_activity_logs();

  INSERT INTO public.admin_activity_logs (
    actor_id,
    actor_email,
    action,
    resource_type,
    target_label,
    summary,
    changed_fields
  )
  VALUES (
    v_actor_id,
```

```
    v_actor_email,
    v_action,
    initcap(replace(TG_TABLE_NAME, '_', ' ')),
    v_target,
    format('%s %s: %s', initcap(replace(TG_TABLE_NAME, '_', ' ')), v_action,
v_target),
    v_changed_fields
  );

  IF TG_OP = 'DELETE' THEN
    RETURN OLD;
  END IF;
  RETURN NEW;
END;
$$;

REVOKE ALL ON FUNCTION public.record_admin_activity() FROM PUBLIC;

DROP TRIGGER IF EXISTS admin_activity_services ON public.services;
CREATE TRIGGER admin_activity_services
  AFTER INSERT OR UPDATE OR DELETE ON public.services
  FOR EACH ROW EXECUTE FUNCTION public.record_admin_activity();

DROP TRIGGER IF EXISTS admin_activity_products ON public.products;
CREATE TRIGGER admin_activity_products
  AFTER INSERT OR UPDATE OR DELETE ON public.products
  FOR EACH ROW EXECUTE FUNCTION public.record_admin_activity();

DROP TRIGGER IF EXISTS admin_activity_crew_members ON
public.crew_members;
CREATE TRIGGER admin_activity_crew_members
  AFTER INSERT OR UPDATE OR DELETE ON public.crew_members
  FOR EACH ROW EXECUTE FUNCTION public.record_admin_activity();

DROP TRIGGER IF EXISTS admin_activity_appointments ON
public.appointments;
CREATE TRIGGER admin_activity_appointments
  AFTER INSERT OR UPDATE OR DELETE ON public.appointments
  FOR EACH ROW EXECUTE FUNCTION public.record_admin_activity();

DROP TRIGGER IF EXISTS admin_activity_schedule_blocks ON
public.schedule_blocks;
CREATE TRIGGER admin_activity_schedule_blocks
  AFTER INSERT OR UPDATE OR DELETE ON public.schedule_blocks
  FOR EACH ROW EXECUTE FUNCTION public.record_admin_activity();
```

```
DROP TRIGGER IF EXISTS admin_activity_blocked_numbers ON
public.blocked_numbers;
CREATE TRIGGER admin_activity_blocked_numbers
  AFTER INSERT OR UPDATE OR DELETE ON public.blocked_numbers
  FOR EACH ROW EXECUTE FUNCTION public.record_admin_activity();

DROP TRIGGER IF EXISTS admin_activity_time_slots ON public.time_slots;
CREATE TRIGGER admin_activity_time_slots
  AFTER INSERT OR UPDATE OR DELETE ON public.time_slots
  FOR EACH ROW EXECUTE FUNCTION public.record_admin_activity();

DROP TRIGGER IF EXISTS admin_activity_crew_schedules ON
public.crew_schedules;
CREATE TRIGGER admin_activity_crew_schedules
  AFTER INSERT OR UPDATE OR DELETE ON public.crew_schedules
  FOR EACH ROW EXECUTE FUNCTION public.record_admin_activity();

DROP TRIGGER IF EXISTS admin_activity_crew_availability_exceptions ON
public.crew_availability_exceptions;
CREATE TRIGGER admin_activity_crew_availability_exceptions
  AFTER INSERT OR UPDATE OR DELETE ON
public.crew_availability_exceptions
  FOR EACH ROW EXECUTE FUNCTION public.record_admin_activity();

DROP TRIGGER IF EXISTS admin_activity_price_history ON
public.price_history;
CREATE TRIGGER admin_activity_price_history
  AFTER INSERT OR UPDATE OR DELETE ON public.price_history
  FOR EACH ROW EXECUTE FUNCTION public.record_admin_activity();

DROP TRIGGER IF EXISTS admin_activity_notifications ON public.notifications;
CREATE TRIGGER admin_activity_notifications
  AFTER INSERT OR UPDATE OR DELETE ON public.notifications
  FOR EACH ROW EXECUTE FUNCTION public.record_admin_activity();

-- Delete expired records on every new activity. Schedule a daily cleanup as well
when pg_cron is enabled.
DO $$
BEGIN
  IF EXISTS (SELECT 1 FROM pg_extension WHERE extname = 'pg_cron')
THEN
    PERFORM cron.unschedule(jobid)
    FROM cron.job
    WHERE jobname = 'purge-admin-activity-logs';
```

```
    PERFORM cron.schedule(
      'purge-admin-activity-logs',
      '15 0 * * *',
      'SELECT public.purge_expired_admin_activity_logs();'
    );
  END IF;
EXCEPTION
  WHEN undefined_function OR undefined_table OR insufficient_privilege THEN
    RAISE NOTICE 'Daily activity-log cleanup could not be scheduled;
insert-trigger cleanup remains enabled.';
END;
$$;
```

supabase/migrations/20260902130000_harden_activity_log_retention.sql

---

```
-- Enforce the retention window when an authorized administrator views the log,
-- in addition to the insert-trigger and optional daily pg_cron cleanup.
CREATE OR REPLACE FUNCTION public.purge_expired_admin_activity_logs()
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public, auth
AS $$
BEGIN
  -- The function is also invoked by a trigger and a scheduled database job,
  -- where auth.uid() is null. Direct signed-in callers must be administrators.
  IF auth.uid() IS NOT NULL AND NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;

  DELETE FROM public.admin_activity_logs
  WHERE created_at < now() - interval '50 days';
END;
$$;

REVOKE ALL ON FUNCTION public.purge_expired_admin_activity_logs()
FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.purge_expired_admin_activity_logs()
TO authenticated;
```

supabase/migrations/20260902140000_expand_admin_activity_coverage.sql

---

```
-- Cover supporting records that can also be changed from administrator tools.
DROP TRIGGER IF EXISTS admin_activity_appointment_services ON
public.appointment_services;
CREATE TRIGGER admin_activity_appointment_services
  AFTER INSERT OR UPDATE OR DELETE ON public.appointment_services
  FOR EACH ROW EXECUTE FUNCTION public.record_admin_activity();

DROP TRIGGER IF EXISTS admin_activity_user_roles ON public.user_roles;
CREATE TRIGGER admin_activity_user_roles
  AFTER INSERT OR UPDATE OR DELETE ON public.user_roles
  FOR EACH ROW EXECUTE FUNCTION public.record_admin_activity();
```

supabase/migrations/20260904110000_secure_public_data_and_remove_staff_role.sql

---

```
-- Keep public catalogue reads limited to records that are actually visible on
-- the public website. Scheduling inputs are now available only to server-side
-- booking functions that use the service role.
DROP POLICY IF EXISTS "services public read" ON public.services;
CREATE POLICY "services public read" ON public.services
  FOR SELECT TO anon
  USING (is_active = true AND is_archived = false);

DROP POLICY IF EXISTS "products public read" ON public.products;
CREATE POLICY "products public read" ON public.products
  FOR SELECT TO anon
  USING (is_active = true AND is_archived = false);

DROP POLICY IF EXISTS "slots public read" ON public.time_slots;
DROP POLICY IF EXISTS "blocks public read" ON public.schedule_blocks;
REVOKE SELECT ON public.time_slots FROM anon;
REVOKE SELECT ON public.schedule_blocks FROM anon;

-- The application has no staff-level console. Remove that unused assignment
-- and replace the active role enum with the single role the application uses.
DELETE FROM public.user_roles WHERE role::text = 'staff';

ALTER TYPE public.app_role RENAME TO app_role_legacy;
CREATE TYPE public.app_role AS ENUM ('admin');

ALTER TABLE public.user_roles
  ALTER COLUMN role TYPE public.app_role
  USING role::text::public.app_role;

-- Existing RLS policies keep their function binding after the rename. Preserve
-- that legacy binding for the applied migration history, then expose a clean
-- app_role-based function for all new code and policies.
ALTER FUNCTION public.has_role(uuid, public.app_role_legacy) RENAME TO
has_role_legacy;

CREATE OR REPLACE FUNCTION public.has_role_legacy(
  _user_id uuid,
  _role public.app_role_legacy
)
RETURNS boolean
LANGUAGE sql
STABLE
```

```
SECURITY DEFINER
SET search_path = public
AS $$
  SELECT EXISTS (
    SELECT 1
    FROM public.user_roles
    WHERE user_id = _user_id
      AND role::text = _role::text
  )
$$;

CREATE OR REPLACE FUNCTION public.has_role(
  _user_id uuid,
  _role public.app_role
)
RETURNS boolean
LANGUAGE sql
STABLE
SECURITY DEFINER
SET search_path = public
AS $$
  SELECT EXISTS (
    SELECT 1
    FROM public.user_roles
    WHERE user_id = _user_id
      AND role = _role
  )
$$;

REVOKE ALL ON FUNCTION public.has_role(uuid, public.app_role) FROM
PUBLIC;
GRANT EXECUTE ON FUNCTION public.has_role(uuid, public.app_role) TO
authenticated, service_role;

-- Records application-level admin actions that do not mutate a tracked table,
-- such as signing in/out or exporting a report. Data changes remain captured by
-- the table triggers created in the activity-log migration.
CREATE OR REPLACE FUNCTION public.record_admin_activity_event(
  p_action text,
  p_resource_type text,
  p_target_label text,
  p_summary text,
  p_changed_fields text[] DEFAULT ARRAY[]::text[]
)
RETURNS uuid
```

```
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public, auth
AS $$
DECLARE
  v_actor_id uuid := auth.uid();
  v_actor_email text;
  v_claims text := current_setting('request.jwt.claims', true);
  v_log_id uuid;
BEGIN
  IF v_actor_id IS NULL OR NOT public.has_role(v_actor_id, 'admin') THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;

  IF v_claims IS NOT NULL AND v_claims <> '' THEN
    v_actor_email := nullif(v_claims::jsonb ->> 'email', '');
  END IF;

  IF btrim(p_action) = '' OR btrim(p_resource_type) = '' OR btrim(p_target_label) =
'' OR btrim(p_summary) = '' THEN
    RAISE EXCEPTION 'Activity event details are required';
  END IF;

  PERFORM public.purge_expired_admin_activity_logs();

  INSERT INTO public.admin_activity_logs (
    actor_id,
    actor_email,
    action,
    resource_type,
    target_label,
    summary,
    changed_fields
  )
  VALUES (
    v_actor_id,
    v_actor_email,
    left(btrim(p_action), 80),
    left(btrim(p_resource_type), 120),
    left(btrim(p_target_label), 180),
    left(btrim(p_summary), 500),
    COALESCE(p_changed_fields, ARRAY[]::text[])
  )
  RETURNING id INTO v_log_id;
```

```
  RETURN v_log_id;
END;
$$;

REVOKE ALL ON FUNCTION public.record_admin_activity_event(text, text, text,
text, text[]) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.record_admin_activity_event(text,
text, text, text, text[]) TO authenticated;
```

supabase/migrations/20260904120000_add_admin_pagination_queries.sql

---

```
-- Keep growing admin screens responsive by aggregating and paging in
Postgres
-- instead of transferring every historical record to the browser.

CREATE INDEX IF NOT EXISTS appointments_admin_list_idx
  ON public.appointments (is_archived, appointment_date DESC, start_time);
CREATE INDEX IF NOT EXISTS appointments_customer_history_idx
  ON public.appointments (phone, appointment_date DESC, start_time);
CREATE INDEX IF NOT EXISTS appointment_services_appointment_id_idx
  ON public.appointment_services (appointment_id);
CREATE INDEX IF NOT EXISTS services_category_idx ON public.services
(category);
CREATE INDEX IF NOT EXISTS products_archive_list_idx
  ON public.products (is_archived, category, sort_order);
CREATE INDEX IF NOT EXISTS crew_members_archive_list_idx
  ON public.crew_members (is_archived, name);
CREATE INDEX IF NOT EXISTS schedule_blocks_archive_list_idx
  ON public.schedule_blocks (is_active, block_date DESC);
CREATE INDEX IF NOT EXISTS blocked_numbers_archive_list_idx
  ON public.blocked_numbers (is_archived, created_at DESC);

CREATE OR REPLACE FUNCTION public.get_admin_customers_page(
  p_search text DEFAULT NULL,
  p_limit integer DEFAULT 25,
  p_offset integer DEFAULT 0
)
RETURNS jsonb
LANGUAGE plpgsql
STABLE
SECURITY INVOKER
SET search_path = public, auth
AS $$
DECLARE
  v_limit integer := LEAST(GREATEST(COALESCE(p_limit, 25), 1), 100);
  v_offset integer := GREATEST(COALESCE(p_offset, 0), 0);
  v_search text := NULLIF(BTRIM(p_search), '');
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;

  RETURN (
    WITH customers AS (
```

```
      SELECT
        a.phone,
        (ARRAY_AGG(a.customer_name ORDER BY a.appointment_date DESC,
a.created_at DESC))[1] AS customer_name,
        (ARRAY_AGG(a.email ORDER BY a.appointment_date DESC,
a.created_at DESC))[1] AS email,
        COUNT(*)::integer AS visits,
        COALESCE(SUM(a.total_estimate) FILTER (WHERE a.status =
'completed'), 0) AS completed_spend,
        MAX(a.appointment_date) AS last_booking,
        ARRAY_AGG(DISTINCT CONCAT_WS(' ', a.moto_brand, a.moto_model) ||
' (' || a.plate_number || ')') AS units
      FROM public.appointments a
      GROUP BY a.phone
    ),
    filtered AS (
      SELECT *
      FROM customers c
      WHERE v_search IS NULL
        OR CONCAT_WS(' ', c.customer_name, c.phone, c.email) ILIKE '%' ||
v_search || '%'
    ),
    paged AS (
      SELECT *
      FROM filtered
      ORDER BY last_booking DESC, customer_name ASC
      LIMIT v_limit OFFSET v_offset
    )
    SELECT jsonb_build_object(
      'total', (SELECT COUNT(*) FROM filtered),
      'rows', COALESCE((SELECT jsonb_agg(TO_JSONB(paged)) FROM paged),
'[]'::jsonb)
    )
  );
END;
$$;

CREATE OR REPLACE FUNCTION public.get_admin_report_page(
  p_report_kind text,
  p_from date,
  p_to date,
  p_status text DEFAULT NULL,
  p_category text DEFAULT NULL,
  p_service_name text DEFAULT NULL,
  p_limit integer DEFAULT 25,
```

```
  p_offset integer DEFAULT 0
)
RETURNS jsonb
LANGUAGE plpgsql
STABLE
SECURITY INVOKER
SET search_path = public, auth
AS $$
DECLARE
  v_limit integer := LEAST(GREATEST(COALESCE(p_limit, 25), 1), 250);
  v_offset integer := GREATEST(COALESCE(p_offset, 0), 0);
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;
  IF p_report_kind NOT IN ('bookings', 'services') THEN
    RAISE EXCEPTION 'Unknown report type';
  END IF;
  IF p_from IS NULL OR p_to IS NULL OR p_from > p_to THEN
    RAISE EXCEPTION 'A valid report date range is required';
  END IF;

  IF p_report_kind = 'bookings' THEN
    RETURN (
      WITH filtered AS (
        SELECT a.id, a.reference_code, a.customer_name, a.appointment_date,
a.status, a.total_estimate
        FROM public.appointments a
        WHERE a.is_archived = false
          AND a.appointment_date BETWEEN p_from AND p_to
          AND (p_status IS NULL OR a.status = p_status)
          AND (
            (p_category IS NULL AND p_service_name IS NULL)
            OR EXISTS (
              SELECT 1
              FROM public.appointment_services aps
              LEFT JOIN public.services s ON s.id = aps.service_id
              WHERE aps.appointment_id = a.id
                AND (p_category IS NULL OR COALESCE(s.category,
'Uncategorized') = p_category)
                AND (p_service_name IS NULL OR aps.service_name =
p_service_name)
            )
          )
      ),
```

```
    grouped_status AS (
      SELECT status, COUNT(*)::integer AS total FROM filtered GROUP BY
status
    ),
    paged AS (
      SELECT * FROM filtered
      ORDER BY appointment_date DESC, reference_code ASC
      LIMIT v_limit OFFSET v_offset
    )
    SELECT jsonb_build_object(
      'total', (SELECT COUNT(*) FROM filtered),
      'rows', COALESCE((SELECT jsonb_agg(TO_JSONB(paged)) FROM
paged), '[]'::jsonb),
      'metrics', jsonb_build_object(
        'total_bookings', (SELECT COUNT(*) FROM filtered),
        'completed_bookings', (SELECT COUNT(*) FROM filtered WHERE status
= 'completed'),
        'completed_value', (SELECT COALESCE(SUM(total_estimate) FILTER
(WHERE status = 'completed'), 0) FROM filtered),
        'status_counts', COALESCE((SELECT jsonb_object_agg(status, total)
FROM grouped_status), '{}'::jsonb)
      )
    )
  );
 END IF;

 RETURN (
  WITH filtered AS (
    SELECT
      aps.appointment_id,
      aps.service_id,
      aps.service_name,
      aps.price,
      a.reference_code,
      a.customer_name,
      a.appointment_date,
      a.status,
      COALESCE(s.category, 'Uncategorized') AS category
    FROM public.appointment_services aps
    JOIN public.appointments a ON a.id = aps.appointment_id
    LEFT JOIN public.services s ON s.id = aps.service_id
    WHERE a.is_archived = false
      AND a.appointment_date BETWEEN p_from AND p_to
      AND (p_status IS NULL OR a.status = p_status)
      AND (p_category IS NULL OR COALESCE(s.category, 'Uncategorized') =
```

```
p_category)
       AND (p_service_name IS NULL OR aps.service_name = p_service_name)
   ),
   grouped_status AS (
     SELECT status, COUNT(*)::integer AS total FROM filtered GROUP BY
status
   ),
   paged AS (
     SELECT * FROM filtered
     ORDER BY appointment_date DESC, reference_code ASC, service_name
ASC
     LIMIT v_limit OFFSET v_offset
   )
   SELECT jsonb_build_object(
     'total', (SELECT COUNT(*) FROM filtered),
     'rows', COALESCE((SELECT jsonb_agg(TO_JSONB(paged)) FROM paged),
'[]'::jsonb),
     'metrics', jsonb_build_object(
       'total_bookings', (SELECT COUNT(DISTINCT appointment_id) FROM
filtered),
       'completed_rows', (SELECT COUNT(*) FROM filtered WHERE status =
'completed'),
       'completed_value', (SELECT COALESCE(SUM(price) FILTER (WHERE
status = 'completed'), 0) FROM filtered),
       'status_counts', COALESCE((SELECT jsonb_object_agg(status, total)
FROM grouped_status), '{}'::jsonb)
     )
   )
 );
END;
$$;

REVOKE ALL ON FUNCTION public.get_admin_customers_page(text, integer,
integer) FROM PUBLIC;
REVOKE ALL ON FUNCTION public.get_admin_report_page(text, date, date,
text, text, text, integer, integer) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.get_admin_customers_page(text,
integer, integer) TO authenticated;
GRANT EXECUTE ON FUNCTION public.get_admin_report_page(text, date,
date, text, text, text, integer, integer) TO authenticated;
```

supabase/migrations/20260905120000_add_activity_log_security_context.sql

---

```
-- Store the record affected by an admin action and the request IP when the
-- Supabase gateway provides it. Historical rows remain valid with NULL values.
ALTER TABLE public.admin_activity_logs
  ADD COLUMN IF NOT EXISTS record_id uuid,
  ADD COLUMN IF NOT EXISTS ip_address text;

CREATE INDEX IF NOT EXISTS admin_activity_logs_record_id_idx
  ON public.admin_activity_logs (record_id);

CREATE OR REPLACE FUNCTION public.admin_activity_request_ip()
RETURNS text
LANGUAGE plpgsql
STABLE
AS $$
DECLARE
  v_headers jsonb;
  v_forwarded_for text;
BEGIN
  v_headers := COALESCE(
    NULLIF(current_setting('request.headers', true), '')::jsonb,
    '{}'::jsonb
  );
  v_forwarded_for := COALESCE(
    v_headers ->> 'x-vercel-forwarded-for',
    v_headers ->> 'cf-connecting-ip',
    v_headers ->> 'x-real-ip',
    v_headers ->> 'x-forwarded-for'
  );

  RETURN NULLIF(btrim(split_part(COALESCE(v_forwarded_for, ''), ',', 1)), '');
EXCEPTION
  WHEN others THEN
    -- Logging must not block an admin action if a platform omits request headers.
    RETURN NULL;
END;
$$;

REVOKE ALL ON FUNCTION public.admin_activity_request_ip() FROM
PUBLIC;
GRANT EXECUTE ON FUNCTION public.admin_activity_request_ip() TO
authenticated, service_role;
```

```
CREATE OR REPLACE FUNCTION public.record_admin_activity()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public, auth
AS $$
DECLARE
  v_actor_id uuid := auth.uid();
  v_claims text := current_setting('request.jwt.claims', true);
  v_actor_email text;
  v_old jsonb := CASE WHEN TG_OP IN ('UPDATE', 'DELETE') THEN
to_jsonb(OLD) ELSE '{}'::jsonb END;
  v_new jsonb := CASE WHEN TG_OP IN ('INSERT', 'UPDATE') THEN
to_jsonb(NEW) ELSE '{}'::jsonb END;
  v_record jsonb;
  v_action text;
  v_target text;
  v_record_id uuid;
  v_changed_fields text[] := ARRAY[]::text[];
BEGIN
  -- Public/server-side activity (such as a customer booking) is not an admin
action.
  IF v_actor_id IS NULL OR NOT public.has_role(v_actor_id, 'admin') THEN
    IF TG_OP = 'DELETE' THEN
      RETURN OLD;
    END IF;
    RETURN NEW;
  END IF;

  IF v_claims IS NOT NULL AND v_claims <> '' THEN
    v_actor_email := nullif(v_claims::jsonb ->> 'email', '');
  END IF;

  IF TG_OP = 'UPDATE' THEN
    SELECT COALESCE(array_agg(changed.key ORDER BY changed.key),
ARRAY[]::text[])
    INTO v_changed_fields
    FROM jsonb_object_keys(v_new) AS changed(key)
    WHERE changed.key NOT IN ('created_at', 'updated_at')
      AND v_old -> changed.key IS DISTINCT FROM v_new -> changed.key;

    -- Do not record a no-op update or an update that only changed a timestamp.
    IF cardinality(v_changed_fields) = 0 THEN
      RETURN NEW;
    END IF;
```

```
    IF v_new ->> 'is_archived' IS DISTINCT FROM v_old ->> 'is_archived' THEN
      v_action := CASE WHEN (v_new ->> 'is_archived')::boolean THEN 'archived'
ELSE 'restored' END;
    ELSIF v_new ->> 'is_active' IS DISTINCT FROM v_old ->> 'is_active' THEN
      v_action := CASE WHEN (v_new ->> 'is_active')::boolean THEN 'activated'
ELSE 'deactivated' END;
    ELSIF v_new ->> 'status' IS DISTINCT FROM v_old ->> 'status' THEN
      v_action := 'status changed';
    ELSE
      v_action := 'updated';
    END IF;
  ELSIF TG_OP = 'INSERT' THEN
    v_action := 'created';
  ELSE
    v_action := 'deleted';
  END IF;

  v_record := CASE WHEN TG_OP = 'DELETE' THEN v_old ELSE v_new END;
  IF COALESCE(v_record ->> 'id', '') ~*
'^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$' THEN
    v_record_id := (v_record ->> 'id')::uuid;
  END IF;
  v_target := COALESCE(
    nullif(v_record ->> 'reference_code', ''),
    nullif(v_record ->> 'name', ''),
    nullif(v_record ->> 'phone', ''),
    nullif(v_record ->> 'service_name', ''),
    nullif(v_record ->> 'block_date', ''),
    nullif(v_record ->> 'id', ''),
    'Record'
  );

  PERFORM public.purge_expired_admin_activity_logs();

  INSERT INTO public.admin_activity_logs (
    actor_id,
    actor_email,
    action,
    resource_type,
    target_label,
    summary,
    changed_fields,
    record_id,
    ip_address
```

```
  )
  VALUES (
    v_actor_id,
    v_actor_email,
    v_action,
    initcap(replace(TG_TABLE_NAME, '_', ' ')),
    v_target,
    format('%s %s: %s', initcap(replace(TG_TABLE_NAME, '_', ' ')), v_action,
v_target),
    v_changed_fields,
    v_record_id,
    public.admin_activity_request_ip()
  );

  IF TG_OP = 'DELETE' THEN
    RETURN OLD;
  END IF;
  RETURN NEW;
END;
$$;

CREATE OR REPLACE FUNCTION public.record_admin_activity_event(
  p_action text,
  p_resource_type text,
  p_target_label text,
  p_summary text,
  p_changed_fields text[] DEFAULT ARRAY[]::text[]
)
RETURNS uuid
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public, auth
AS $$
DECLARE
  v_actor_id uuid := auth.uid();
  v_actor_email text;
  v_claims text := current_setting('request.jwt.claims', true);
  v_log_id uuid;
BEGIN
  IF v_actor_id IS NULL OR NOT public.has_role(v_actor_id, 'admin') THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;

  IF v_claims IS NOT NULL AND v_claims <> '' THEN
    v_actor_email := nullif(v_claims::jsonb ->> 'email', '');
```

```
  END IF;

  IF btrim(p_action) = '' OR btrim(p_resource_type) = '' OR btrim(p_target_label) =
'' OR btrim(p_summary) = '' THEN
    RAISE EXCEPTION 'Activity event details are required';
  END IF;

  PERFORM public.purge_expired_admin_activity_logs();

  INSERT INTO public.admin_activity_logs (
    actor_id,
    actor_email,
    action,
    resource_type,
    target_label,
    summary,
    changed_fields,
    ip_address
  )
  VALUES (
    v_actor_id,
    v_actor_email,
    left(btrim(p_action), 80),
    left(btrim(p_resource_type), 120),
    left(btrim(p_target_label), 180),
    left(btrim(p_summary), 500),
    COALESCE(p_changed_fields, ARRAY[]::text[]),
    public.admin_activity_request_ip()
  )
  RETURNING id INTO v_log_id;

  RETURN v_log_id;
END;
$$;
```

supabase/migrations/20260905121000_align_booking_duration_buffers.sql

---

```
-- A booking reserves the selected service work plus one 15-minute arrival
-- buffer and one 15-minute post-service buffer, regardless of service count.
ALTER TABLE public.appointments
  ALTER COLUMN booking_duration_minutes SET DEFAULT 90;

UPDATE public.appointments appointments
SET booking_duration_minutes = COALESCE(
  (
    SELECT SUM(COALESCE(appointment_services.duration_minutes, 60)) + 30
    FROM public.appointment_services
    WHERE appointment_services.appointment_id = appointments.id
  ),
  90
)
WHERE appointments.is_archived = false
  AND appointments.appointment_date >= CURRENT_DATE
  AND appointments.status IN ('pending', 'confirmed', 'in_progress');

COMMENT ON COLUMN public.appointments.booking_duration_minutes IS
  'Selected service duration plus a 15-minute arrival buffer and a 15-minute
post-service buffer.';
```

supabase/migrations/20260906120000_create_shop_settings.sql

```
-- One persisted configuration record powers the admin settings dialog and the
-- customer-facing booking rules. Keeping it as a singleton avoids a shop-wide
-- setting being accidentally duplicated by separate administrators.
CREATE TABLE IF NOT EXISTS public.shop_settings (
  id boolean PRIMARY KEY DEFAULT true CHECK (id),
  shop_name text NOT NULL DEFAULT 'Fake Rider Motorparts',
  address text NOT NULL DEFAULT 'Purok Bangkal Sta. Cruz, Baclayon, Bohol,
Philippines',
  contact_number text NOT NULL DEFAULT '0916 126 3317',
  contact_email text NOT NULL DEFAULT 'joemartato4@gmail.com',
  booking_terms text NOT NULL DEFAULT 'Bookings are subject to shop
confirmation. Please arrive 15 minutes before your slot. Late arrivals beyond 30
minutes may be rescheduled. Quoted prices are starting rates; parts and
additional labor are billed separately. The shop is not liable for personal items left
on the unit.

Cancellations must be made at least 48 hours before the schedule.

We use your name, contact details, motorcycle details, selected services, and
notes only to manage this booking, contact you about it, and provide shop
services. We do not sell your information.',
  logo_url text,
  minimum_booking_lead_hours integer NOT NULL DEFAULT 48 CHECK
(minimum_booking_lead_hours BETWEEN 0 AND 168),
  max_advance_booking_days integer NOT NULL DEFAULT 30 CHECK
(max_advance_booking_days BETWEEN 1 AND 365),
  default_appointment_duration_minutes integer NOT NULL DEFAULT 90
CHECK (default_appointment_duration_minutes BETWEEN 15 AND 600),
  allow_same_day_appointments boolean NOT NULL DEFAULT false,
  cancellation_notice_hours integer NOT NULL DEFAULT 48 CHECK
(cancellation_notice_hours BETWEEN 0 AND 168),
  rescheduling_notice_hours integer NOT NULL DEFAULT 48 CHECK
(rescheduling_notice_hours BETWEEN 0 AND 168),
  notify_confirmation boolean NOT NULL DEFAULT true,
  notify_cancellation boolean NOT NULL DEFAULT true,
  notify_reminder boolean NOT NULL DEFAULT true,
  reminder_hours_before integer NOT NULL DEFAULT 24 CHECK
(reminder_hours_before BETWEEN 1 AND 168),
  notify_admin boolean NOT NULL DEFAULT true,
  updated_at timestamptz NOT NULL DEFAULT now()
);

INSERT INTO public.shop_settings (id)
```

```
VALUES (true)
ON CONFLICT (id) DO NOTHING;

GRANT SELECT ON public.shop_settings TO anon, authenticated;
GRANT INSERT, UPDATE ON public.shop_settings TO authenticated;
GRANT ALL ON public.shop_settings TO service_role;

ALTER TABLE public.shop_settings ENABLE ROW LEVEL SECURITY;

DROP POLICY IF EXISTS "shop settings public read" ON public.shop_settings;
CREATE POLICY "shop settings public read" ON public.shop_settings
  FOR SELECT TO anon, authenticated
  USING (true);

DROP POLICY IF EXISTS "shop settings admin insert" ON public.shop_settings;
CREATE POLICY "shop settings admin insert" ON public.shop_settings
  FOR INSERT TO authenticated
  WITH CHECK (public.has_role(auth.uid(), 'admin'));

DROP POLICY IF EXISTS "shop settings admin update" ON
public.shop_settings;
CREATE POLICY "shop settings admin update" ON public.shop_settings
  FOR UPDATE TO authenticated
  USING (public.has_role(auth.uid(), 'admin'))
  WITH CHECK (public.has_role(auth.uid(), 'admin'));

DROP TRIGGER IF EXISTS shop_settings_updated ON public.shop_settings;
CREATE TRIGGER shop_settings_updated
  BEFORE UPDATE ON public.shop_settings
  FOR EACH ROW EXECUTE FUNCTION public.update_updated_at_column();

-- Public URLs are needed for the customer-facing logo. Only administrators
-- can place, replace, or remove files from this bucket.
INSERT INTO storage.buckets (id, name, public, file_size_limit,
allowed_mime_types)
VALUES (
  'shop-assets',
  'shop-assets',
  true,
  5242880,
  ARRAY['image/jpeg', 'image/png', 'image/webp']
)
ON CONFLICT (id) DO UPDATE
  SET public = EXCLUDED.public,
      file_size_limit = EXCLUDED.file_size_limit,
```

```
    allowed_mime_types = EXCLUDED.allowed_mime_types;

DROP POLICY IF EXISTS "shop assets public read" ON storage.objects;
CREATE POLICY "shop assets public read" ON storage.objects
  FOR SELECT TO public
  USING (bucket_id = 'shop-assets');

DROP POLICY IF EXISTS "shop assets admin insert" ON storage.objects;
CREATE POLICY "shop assets admin insert" ON storage.objects
  FOR INSERT TO authenticated
  WITH CHECK (bucket_id = 'shop-assets' AND public.has_role(auth.uid(),
'admin'));

DROP POLICY IF EXISTS "shop assets admin update" ON storage.objects;
CREATE POLICY "shop assets admin update" ON storage.objects
  FOR UPDATE TO authenticated
  USING (bucket_id = 'shop-assets' AND public.has_role(auth.uid(), 'admin'))
  WITH CHECK (bucket_id = 'shop-assets' AND public.has_role(auth.uid(),
'admin'));

DROP POLICY IF EXISTS "shop assets admin delete" ON storage.objects;
CREATE POLICY "shop assets admin delete" ON storage.objects
  FOR DELETE TO authenticated
  USING (bucket_id = 'shop-assets' AND public.has_role(auth.uid(), 'admin'));
```

supabase/migrations/20260906130000_expand_default_booking_terms.sql

---

```
-- Upgrade the initial built-in copy without replacing terms an administrator has
already edited.
UPDATE public.shop_settings
SET booking_terms = 'Bookings are subject to shop confirmation. Please arrive
15 minutes before your slot. Late arrivals beyond 30 minutes may be
rescheduled. Quoted prices are starting rates; parts and additional labor are
billed separately. The shop is not liable for personal items left on the unit.

Cancellations must be made at least 48 hours before the schedule.

We use your name, contact details, motorcycle details, selected services, and
notes only to manage this booking, contact you about it, and provide shop
services. We do not sell your information.'
WHERE booking_terms = 'Bookings are subject to shop confirmation. Please
arrive 15 minutes before your appointment. Cancellations must be made before
the required notice period.';
```

supabase/migrations/20260907120000_add_product_stock_quantities.sql

---

```
-- Store the real inventory count while retaining the existing boolean for public
availability UI.
ALTER TABLE public.products
  ADD COLUMN IF NOT EXISTS stock_quantity integer NOT NULL DEFAULT 0;

UPDATE public.products
SET stock_quantity = CASE WHEN in_stock THEN 1 ELSE 0 END;

DO $$
BEGIN
  IF NOT EXISTS (
    SELECT 1
    FROM pg_constraint
    WHERE conname = 'products_stock_quantity_nonnegative'
  ) THEN
    ALTER TABLE public.products
      ADD CONSTRAINT products_stock_quantity_nonnegative CHECK
(stock_quantity >= 0);
  END IF;
END
$$;

CREATE OR REPLACE FUNCTION public.sync_product_stock_status()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = public
AS $$
BEGIN
  NEW.stock_quantity := GREATEST(COALESCE(NEW.stock_quantity, 0), 0);
  NEW.in_stock := NEW.stock_quantity > 0;
  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS products_sync_stock_status ON public.products;
CREATE TRIGGER products_sync_stock_status
  BEFORE INSERT OR UPDATE ON public.products
  FOR EACH ROW
  EXECUTE FUNCTION public.sync_product_stock_status();
```

supabase/migrations/20260908090000_add_services_to_booking_report_rows.sql

---

```
-- Include the services selected for each appointment in booking-volume report
rows.
CREATE OR REPLACE FUNCTION public.get_admin_report_page(
  p_report_kind text,
  p_from date,
  p_to date,
  p_status text DEFAULT NULL,
  p_category text DEFAULT NULL,
  p_service_name text DEFAULT NULL,
  p_limit integer DEFAULT 25,
  p_offset integer DEFAULT 0
)
RETURNS jsonb
LANGUAGE plpgsql
STABLE
SECURITY INVOKER
SET search_path = public, auth
AS $$
DECLARE
  v_limit integer := LEAST(GREATEST(COALESCE(p_limit, 25), 1), 250);
  v_offset integer := GREATEST(COALESCE(p_offset, 0), 0);
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;
  IF p_report_kind NOT IN ('bookings', 'services') THEN
    RAISE EXCEPTION 'Unknown report type';
  END IF;
  IF p_from IS NULL OR p_to IS NULL OR p_from > p_to THEN
    RAISE EXCEPTION 'A valid report date range is required';
  END IF;

  IF p_report_kind = 'bookings' THEN
    RETURN (
      WITH filtered AS (
        SELECT
          a.id,
          a.reference_code,
          a.customer_name,
          a.appointment_date,
          a.status,
          a.total_estimate,
```

```
        COALESCE(
          (
            SELECT string_agg(aps.service_name, ', ' ORDER BY
aps.service_name)
            FROM public.appointment_services aps
            WHERE aps.appointment_id = a.id
          ),
          '—'
        ) AS service
      FROM public.appointments a
      WHERE a.is_archived = false
        AND a.appointment_date BETWEEN p_from AND p_to
        AND (p_status IS NULL OR a.status = p_status)
        AND (
          (p_category IS NULL AND p_service_name IS NULL)
          OR EXISTS (
            SELECT 1
            FROM public.appointment_services aps
            LEFT JOIN public.services s ON s.id = aps.service_id
            WHERE aps.appointment_id = a.id
              AND (p_category IS NULL OR COALESCE(s.category,
'Uncategorized') = p_category)
              AND (p_service_name IS NULL OR aps.service_name =
p_service_name)
          )
        )
    ),
    grouped_status AS (
      SELECT status, COUNT(*)::integer AS total FROM filtered GROUP BY
status
    ),
    paged AS (
      SELECT * FROM filtered
      ORDER BY appointment_date DESC, reference_code ASC
      LIMIT v_limit OFFSET v_offset
    )
    SELECT jsonb_build_object(
      'total', (SELECT COUNT(*) FROM filtered),
      'rows', COALESCE((SELECT jsonb_agg(TO_JSONB(paged)) FROM
paged), '[]'::jsonb),
      'metrics', jsonb_build_object(
        'total_bookings', (SELECT COUNT(*) FROM filtered),
        'completed_bookings', (SELECT COUNT(*) FROM filtered WHERE status
= 'completed'),
        'completed_value', (SELECT COALESCE(SUM(total_estimate) FILTER
```

```
(WHERE status = 'completed'), 0) FROM filtered),
        'status_counts', COALESCE((SELECT jsonb_object_agg(status, total)
FROM grouped_status), '{}'::jsonb)
       )
     )
   );
  END IF;

  RETURN (
    WITH filtered AS (
      SELECT
        aps.appointment_id,
        aps.service_id,
        aps.service_name,
        aps.price,
        a.reference_code,
        a.customer_name,
        a.appointment_date,
        a.status,
        COALESCE(s.category, 'Uncategorized') AS category
      FROM public.appointment_services aps
      JOIN public.appointments a ON a.id = aps.appointment_id
      LEFT JOIN public.services s ON s.id = aps.service_id
      WHERE a.is_archived = false
        AND a.appointment_date BETWEEN p_from AND p_to
        AND (p_status IS NULL OR a.status = p_status)
        AND (p_category IS NULL OR COALESCE(s.category, 'Uncategorized') =
p_category)
        AND (p_service_name IS NULL OR aps.service_name = p_service_name)
    ),
    grouped_status AS (
      SELECT status, COUNT(*)::integer AS total FROM filtered GROUP BY
status
    ),
    paged AS (
      SELECT * FROM filtered
      ORDER BY appointment_date DESC, reference_code ASC, service_name
ASC
      LIMIT v_limit OFFSET v_offset
    )
    SELECT jsonb_build_object(
      'total', (SELECT COUNT(*) FROM filtered),
      'rows', COALESCE((SELECT jsonb_agg(TO_JSONB(paged)) FROM paged),
'[]'::jsonb),
      'metrics', jsonb_build_object(
```

```
        'total_bookings', (SELECT COUNT(DISTINCT appointment_id) FROM
filtered),
        'completed_rows', (SELECT COUNT(*) FROM filtered WHERE status =
'completed'),
        'completed_value', (SELECT COALESCE(SUM(price) FILTER (WHERE
status = 'completed'), 0) FROM filtered),
        'status_counts', COALESCE((SELECT jsonb_object_agg(status, total)
FROM grouped_status), '{}'::jsonb)
      )
    )
  );
END;
$$;
```

supabase/migrations/20260908120000_add_appointment_rescheduling.sql

---

```
-- Preserve the appointment chain when a customer moves an appointment to a
new slot.
ALTER TABLE public.appointments
  ADD COLUMN IF NOT EXISTS rescheduled_from_appointment_id uuid
    REFERENCES public.appointments(id) ON DELETE SET NULL,
  ADD COLUMN IF NOT EXISTS rescheduled_to_appointment_id uuid
    REFERENCES public.appointments(id) ON DELETE SET NULL;

CREATE INDEX IF NOT EXISTS appointments_rescheduled_from_idx
  ON public.appointments (rescheduled_from_appointment_id);
CREATE INDEX IF NOT EXISTS appointments_rescheduled_to_idx
  ON public.appointments (rescheduled_to_appointment_id);

-- A rescheduled appointment is still a real appointment for capacity checks;
-- the replaced record is excluded through its rescheduled_to link.
ALTER TABLE public.appointments
  DROP CONSTRAINT IF EXISTS appointments_crew_time_no_overlap;

ALTER TABLE public.appointments
  ADD CONSTRAINT appointments_crew_time_no_overlap
  EXCLUDE USING gist (
    assigned_crew_id WITH =,
    tsrange(
      appointment_date + start_time,
      appointment_date + start_time + make_interval(mins =>
booking_duration_minutes),
      '[)'
    ) WITH &&
  )
  WHERE (
    is_archived = false
    AND rescheduled_to_appointment_id IS NULL
    AND status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
  );

CREATE OR REPLACE FUNCTION
public.create_rescheduled_booking_atomic(
  p_reference_code text,
  p_booking_request_id uuid,
  p_customer_name text,
  p_phone text,
  p_email text,
  p_moto_brand text,
```

```
  p_moto_model text,
  p_moto_variant text,
  p_moto_year integer,
  p_plate_number text,
  p_appointment_date date,
  p_start_time time,
  p_notes text,
  p_total_estimate numeric,
  p_booking_duration_minutes integer,
  p_assigned_crew_id uuid,
  p_services jsonb,
  p_notification_title text,
  p_notification_message text,
  p_original_appointment_id uuid
)
RETURNS TABLE (appointment_id uuid, reference_code text)
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment_id uuid;
  v_reference_code text;
  v_original_status text;
BEGIN
  IF p_booking_request_id IS NULL THEN
    RAISE EXCEPTION 'A booking request ID is required.';
  END IF;

  IF jsonb_typeof(p_services) <> 'array' OR jsonb_array_length(p_services) = 0
THEN
    RAISE EXCEPTION 'A booking must include at least one service.';
  END IF;

  -- Lock the original before checking it so two simultaneous reschedule
  -- requests cannot both create replacements.
  SELECT appointments.status
    INTO v_original_status
  FROM public.appointments AS appointments
  WHERE appointments.id = p_original_appointment_id
    AND appointments.is_archived = false
    AND appointments.rescheduled_to_appointment_id IS NULL
  FOR UPDATE;

  IF v_original_status IS NULL OR v_original_status NOT IN ('pending',
```

```
'confirmed', 'rescheduled') THEN
    RAISE EXCEPTION 'This appointment is no longer available for
rescheduling.';
  END IF;

  INSERT INTO public.appointments AS appointments (
    reference_code,
    booking_request_id,
    customer_name,
    phone,
    email,
    moto_brand,
    moto_model,
    moto_variant,
    moto_year,
    plate_number,
    appointment_date,
    start_time,
    status,
    notes,
    total_estimate,
    booking_duration_minutes,
    terms_accepted,
    assigned_crew_id,
    rescheduled_from_appointment_id
  )
  VALUES (
    p_reference_code,
    p_booking_request_id,
    p_customer_name,
    p_phone,
    p_email,
    p_moto_brand,
    p_moto_model,
    p_moto_variant,
    p_moto_year,
    p_plate_number,
    p_appointment_date,
    p_start_time,
    'rescheduled',
    p_notes,
    p_total_estimate,
    p_booking_duration_minutes,
    true,
    p_assigned_crew_id,
```

```
    p_original_appointment_id
  )
  RETURNING appointments.id, appointments.reference_code INTO
v_appointment_id, v_reference_code;

  INSERT INTO public.appointment_services (
    appointment_id,
    service_id,
    service_name,
    price,
    duration_minutes
  )
  SELECT
    v_appointment_id,
    service.service_id,
    service.service_name,
    service.price,
    service.duration_minutes
  FROM jsonb_to_recordset(p_services) AS service(
    service_id uuid,
    service_name text,
    price numeric,
    duration_minutes integer
  );

  UPDATE public.appointments
  SET status = 'rescheduled',
      rescheduled_to_appointment_id = v_appointment_id
  WHERE id = p_original_appointment_id;

  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES (
    'rescheduled_appointment',
    p_notification_title,
    p_notification_message,
    v_appointment_id
  );

  RETURN QUERY SELECT v_appointment_id, v_reference_code;
END;
$$;

REVOKE ALL ON FUNCTION public.create_rescheduled_booking_atomic(
  text, uuid, text, text, text, text, text, text, integer, text, date, time,
  text, numeric, integer, uuid, jsonb, text, text, uuid
```

```
) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.create_rescheduled_booking_atomic(
  text, uuid, text, text, text, text, text, text, integer, text, date, time,
  text, numeric, integer, uuid, jsonb, text, text, uuid
) TO service_role;
```

supabase/migrations/20260908130000_rescheduled_booking_starts_pending.sql

---

```
-- A moved appointment is a new booking and must follow the normal approval flow.
-- Keep the original record as "rescheduled" for history, but start its replacement as "pending".
CREATE OR REPLACE FUNCTION public.create_rescheduled_booking_atomic(
  p_reference_code text,
  p_booking_request_id uuid,
  p_customer_name text,
  p_phone text,
  p_email text,
  p_moto_brand text,
  p_moto_model text,
  p_moto_variant text,
  p_moto_year integer,
  p_plate_number text,
  p_appointment_date date,
  p_start_time time,
  p_notes text,
  p_total_estimate numeric,
  p_booking_duration_minutes integer,
  p_assigned_crew_id uuid,
  p_services jsonb,
  p_notification_title text,
  p_notification_message text,
  p_original_appointment_id uuid
)
RETURNS TABLE (appointment_id uuid, reference_code text)
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment_id uuid;
  v_reference_code text;
  v_original_status text;
BEGIN
  IF p_booking_request_id IS NULL THEN
    RAISE EXCEPTION 'A booking request ID is required.';
  END IF;

  IF jsonb_typeof(p_services) <> 'array' OR jsonb_array_length(p_services) = 0
```

```
THEN
    RAISE EXCEPTION 'A booking must include at least one service.';
  END IF;

  -- Lock the original before checking it so two simultaneous reschedule
  -- requests cannot both create replacements.
  SELECT appointments.status
    INTO v_original_status
  FROM public.appointments AS appointments
  WHERE appointments.id = p_original_appointment_id
    AND appointments.is_archived = false
    AND appointments.rescheduled_to_appointment_id IS NULL
  FOR UPDATE;

  IF v_original_status IS NULL OR v_original_status NOT IN ('pending',
'confirmed', 'rescheduled') THEN
    RAISE EXCEPTION 'This appointment is no longer available for
rescheduling.';
  END IF;

  INSERT INTO public.appointments AS appointments (
    reference_code,
    booking_request_id,
    customer_name,
    phone,
    email,
    moto_brand,
    moto_model,
    moto_variant,
    moto_year,
    plate_number,
    appointment_date,
    start_time,
    status,
    notes,
    total_estimate,
    booking_duration_minutes,
    terms_accepted,
    assigned_crew_id,
    rescheduled_from_appointment_id
  )
  VALUES (
    p_reference_code,
    p_booking_request_id,
    p_customer_name,
```

```
    p_phone,
    p_email,
    p_moto_brand,
    p_moto_model,
    p_moto_variant,
    p_moto_year,
    p_plate_number,
    p_appointment_date,
    p_start_time,
    'pending',
    p_notes,
    p_total_estimate,
    p_booking_duration_minutes,
    true,
    p_assigned_crew_id,
    p_original_appointment_id
  )
  RETURNING appointments.id, appointments.reference_code INTO
v_appointment_id, v_reference_code;

  INSERT INTO public.appointment_services (
    appointment_id,
    service_id,
    service_name,
    price,
    duration_minutes
  )
  SELECT
    v_appointment_id,
    service.service_id,
    service.service_name,
    service.price,
    service.duration_minutes
  FROM jsonb_to_recordset(p_services) AS service(
    service_id uuid,
    service_name text,
    price numeric,
    duration_minutes integer
  );

  UPDATE public.appointments
  SET status = 'rescheduled',
      rescheduled_to_appointment_id = v_appointment_id
  WHERE id = p_original_appointment_id;
```

```
  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES (
    'rescheduled_appointment',
    p_notification_title,
    p_notification_message,
    v_appointment_id
  );

  RETURN QUERY SELECT v_appointment_id, v_reference_code;
END;
$$;
```

supabase/migrations/20260909110000_create_atomic_admin_appointment_update.sql

---

```
-- Keep an administrator's appointment edit atomic. This prevents a service edit
-- from being saved without the matching schedule, crew, total, and duration
update.
CREATE OR REPLACE FUNCTION public.update_appointment_details_atomic(
  p_appointment_id uuid,
  p_service_ids uuid[],
  p_appointment_date date,
  p_start_time time,
  p_assigned_crew_id uuid,
  p_status text,
  p_admin_notes text
)
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_service_count integer;
  v_total_estimate numeric(10,2);
  v_duration_minutes integer;
BEGIN
  IF COALESCE(cardinality(p_service_ids), 0) = 0 THEN
    RAISE EXCEPTION 'Select at least one service.';
  END IF;

  IF p_status NOT IN (
    'pending', 'confirmed', 'in_progress', 'completed', 'rescheduled', 'cancelled',
'no_show'
  ) THEN
    RAISE EXCEPTION 'Choose a valid appointment status.';
  END IF;

  SELECT count(*)
    INTO v_service_count
  FROM public.services
  WHERE id = ANY(p_service_ids)
    AND is_active = true
    AND is_archived = false;

  IF v_service_count <> cardinality(p_service_ids) THEN
    RAISE EXCEPTION 'One or more selected services are no longer available.';
```

```
END IF;

IF p_assigned_crew_id IS NOT NULL AND NOT EXISTS (
  SELECT 1
  FROM public.crew_members
  WHERE id = p_assigned_crew_id
    AND is_active = true
    AND is_archived = false
) THEN
  RAISE EXCEPTION 'The selected crew member is no longer available.';
END IF;

SELECT
  COALESCE(sum(price), 0),
  COALESCE(sum(duration_minutes), 0) + 30
INTO v_total_estimate, v_duration_minutes
FROM public.services
WHERE id = ANY(p_service_ids);

UPDATE public.appointments
SET appointment_date = p_appointment_date,
    start_time = p_start_time,
    assigned_crew_id = p_assigned_crew_id,
    status = p_status,
    admin_notes = NULLIF(btrim(p_admin_notes), ''),
    total_estimate = v_total_estimate,
    booking_duration_minutes = v_duration_minutes
WHERE id = p_appointment_id
  AND is_archived = false;

IF NOT FOUND THEN
  RAISE EXCEPTION 'This appointment is no longer available.';
END IF;

DELETE FROM public.appointment_services
WHERE appointment_id = p_appointment_id;

INSERT INTO public.appointment_services (
  appointment_id,
  service_id,
  service_name,
  price,
  duration_minutes
)
SELECT
```

```
    p_appointment_id,
    service.id,
    service.name,
    service.price,
    service.duration_minutes
  FROM public.services AS service
  WHERE service.id = ANY(p_service_ids);
END;
$$;

REVOKE ALL ON FUNCTION public.update_appointment_details_atomic(
  uuid, uuid[], date, time, uuid, text, text
) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.update_appointment_details_atomic(
  uuid, uuid[], date, time, uuid, text, text
) TO authenticated;
```

supabase/migrations/20260909120000_enforce_appointment_crew_assignment.sql

---

```
-- Confirmed appointments are assigned to a free, scheduled crew member
unless
-- an administrator has explicitly selected a crew option (including Unassigned).
DROP FUNCTION IF EXISTS public.update_appointment_details_atomic(
  uuid, uuid[], date, time, uuid, text, text
);

CREATE FUNCTION public.update_appointment_details_atomic(
  p_appointment_id uuid,
  p_service_ids uuid[],
  p_appointment_date date,
  p_start_time time,
  p_assigned_crew_id uuid,
  p_crew_assignment_manual boolean,
  p_status text,
  p_admin_notes text
)
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_service_count integer;
  v_total_estimate numeric(10,2);
  v_duration_minutes integer;
  v_assigned_crew_id uuid := p_assigned_crew_id;
BEGIN
  IF COALESCE(cardinality(p_service_ids), 0) = 0 THEN
    RAISE EXCEPTION 'Select at least one service.';
  END IF;

  IF p_status NOT IN (
    'pending', 'confirmed', 'in_progress', 'completed', 'rescheduled', 'cancelled',
'rejected', 'no_show'
  ) THEN
    RAISE EXCEPTION 'Choose a valid appointment status.';
  END IF;

  SELECT count(*)
    INTO v_service_count
  FROM public.services
```

```
  WHERE id = ANY(p_service_ids)
    AND is_active = true
    AND is_archived = false;

  IF v_service_count <> cardinality(p_service_ids) THEN
    RAISE EXCEPTION 'One or more selected services are no longer available.';
  END IF;

  SELECT
    COALESCE(sum(price), 0),
    COALESCE(sum(duration_minutes), 0) + 30
  INTO v_total_estimate, v_duration_minutes
  FROM public.services
  WHERE id = ANY(p_service_ids);

  -- When confirming without a manual crew choice, select the first active
  -- mechanic whose scheduled shift, exceptions and current appointments allow
  -- the entire service period.
  IF p_status = 'confirmed' AND v_assigned_crew_id IS NULL AND NOT
p_crew_assignment_manual THEN
    SELECT crew.id
      INTO v_assigned_crew_id
    FROM public.crew_members AS crew
    WHERE crew.is_active = true
      AND crew.is_archived = false
      AND EXISTS (
        SELECT 1
        FROM public.crew_schedules AS schedule
        WHERE schedule.crew_id = crew.id
          AND schedule.schedule_date = p_appointment_date
          AND schedule.is_working = true
          AND schedule.start_time IS NOT NULL
          AND schedule.end_time IS NOT NULL
          AND schedule.start_time <= p_start_time
          AND schedule.end_time >= p_start_time + make_interval(mins =>
v_duration_minutes)
      )
      AND NOT EXISTS (
        SELECT 1
        FROM public.crew_availability_exceptions AS exception
        WHERE exception.crew_id = crew.id
          AND exception.start_date <= p_appointment_date
          AND exception.end_date >= p_appointment_date
          AND (
            exception.is_all_day = true
```

```
          OR (
            exception.start_time IS NOT NULL
            AND exception.end_time IS NOT NULL
            AND tsrange(
              p_appointment_date + p_start_time,
              p_appointment_date + p_start_time + make_interval(mins =>
v_duration_minutes),
              '[)'
            ) && tsrange(
              p_appointment_date + exception.start_time,
              p_appointment_date + exception.end_time,
              '[)'
            )
          )
        )
      )
      AND NOT EXISTS (
        SELECT 1
        FROM public.appointments AS appointment
        WHERE appointment.id <> p_appointment_id
          AND appointment.assigned_crew_id = crew.id
          AND appointment.appointment_date = p_appointment_date
          AND appointment.is_archived = false
          AND appointment.rescheduled_to_appointment_id IS NULL
          AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
          AND tsrange(
            appointment.appointment_date + appointment.start_time,
            appointment.appointment_date + appointment.start_time
              + make_interval(mins => appointment.booking_duration_minutes),
            '[)'
          ) && tsrange(
            p_appointment_date + p_start_time,
            p_appointment_date + p_start_time + make_interval(mins =>
v_duration_minutes),
            '[)'
          )
      )
    ORDER BY crew.name, crew.id
    LIMIT 1;

    IF v_assigned_crew_id IS NULL THEN
      RAISE EXCEPTION 'No crew member is available for the selected
appointment period.';
    END IF;
```

```
  END IF;

  -- Manual crew assignments must meet those exact same availability rules.
  IF v_assigned_crew_id IS NOT NULL AND NOT EXISTS (
    SELECT 1
    FROM public.crew_members AS crew
    WHERE crew.id = v_assigned_crew_id
      AND crew.is_active = true
      AND crew.is_archived = false
      AND EXISTS (
        SELECT 1
        FROM public.crew_schedules AS schedule
        WHERE schedule.crew_id = crew.id
          AND schedule.schedule_date = p_appointment_date
          AND schedule.is_working = true
          AND schedule.start_time IS NOT NULL
          AND schedule.end_time IS NOT NULL
          AND schedule.start_time <= p_start_time
          AND schedule.end_time >= p_start_time + make_interval(mins =>
v_duration_minutes)
      )
      AND NOT EXISTS (
        SELECT 1
        FROM public.crew_availability_exceptions AS exception
        WHERE exception.crew_id = crew.id
          AND exception.start_date <= p_appointment_date
          AND exception.end_date >= p_appointment_date
          AND (
            exception.is_all_day = true
            OR (
              exception.start_time IS NOT NULL
              AND exception.end_time IS NOT NULL
              AND tsrange(
                p_appointment_date + p_start_time,
                p_appointment_date + p_start_time + make_interval(mins =>
v_duration_minutes),
                '[)'
              ) && tsrange(
                p_appointment_date + exception.start_time,
                p_appointment_date + exception.end_time,
                '[)'
              )
            )
          )
      )
```

```
    AND NOT EXISTS (
      SELECT 1
      FROM public.appointments AS appointment
      WHERE appointment.id <> p_appointment_id
        AND appointment.assigned_crew_id = crew.id
        AND appointment.appointment_date = p_appointment_date
        AND appointment.is_archived = false
        AND appointment.rescheduled_to_appointment_id IS NULL
        AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
        AND tsrange(
          appointment.appointment_date + appointment.start_time,
          appointment.appointment_date + appointment.start_time
            + make_interval(mins => appointment.booking_duration_minutes),
          '[)'
        ) && tsrange(
          p_appointment_date + p_start_time,
          p_appointment_date + p_start_time + make_interval(mins =>
v_duration_minutes),
          '[)'
        )
    )
  ) THEN
    RAISE EXCEPTION 'Choose a crew member who is free for the full
appointment period.';
  END IF;

  UPDATE public.appointments
  SET appointment_date = p_appointment_date,
      start_time = p_start_time,
      assigned_crew_id = v_assigned_crew_id,
      status = p_status,
      admin_notes = NULLIF(btrim(p_admin_notes), ''),
      total_estimate = v_total_estimate,
      booking_duration_minutes = v_duration_minutes
  WHERE id = p_appointment_id
    AND is_archived = false;

  IF NOT FOUND THEN
    RAISE EXCEPTION 'This appointment is no longer available.';
  END IF;

  DELETE FROM public.appointment_services
  WHERE appointment_id = p_appointment_id;
```

```
  INSERT INTO public.appointment_services (
    appointment_id,
    service_id,
    service_name,
    price,
    duration_minutes
  )
  SELECT
    p_appointment_id,
    service.id,
    service.name,
    service.price,
    service.duration_minutes
  FROM public.services AS service
  WHERE service.id = ANY(p_service_ids);
END;
$$;

REVOKE ALL ON FUNCTION public.update_appointment_details_atomic(
  uuid, uuid[], date, time, uuid, boolean, text, text
) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.update_appointment_details_atomic(
  uuid, uuid[], date, time, uuid, boolean, text, text
) TO authenticated;
```

supabase/migrations/20260910100000_review_pending_appointment.sql

```
-- Allow administrators to approve or reject a pending appointment directly
-- from the notifications inbox. Confirmation revalidates the appointment's
-- crew assignment against the current schedule and availability rules.
CREATE OR REPLACE FUNCTION public.review_pending_appointment(
  p_appointment_id uuid,
  p_decision text
)
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment public.appointments%ROWTYPE;
  v_assigned_crew_id uuid;
  v_duration_minutes integer;
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Only administrators can review appointments.';
  END IF;

  IF p_decision IS NULL OR lower(btrim(p_decision)) NOT IN ('confirmed',
'rejected') THEN
    RAISE EXCEPTION 'Choose either confirmed or rejected.';
  END IF;

  -- Lock the row so two notification actions cannot review the same booking.
  SELECT appointment.*
    INTO v_appointment
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
  FOR UPDATE;

  IF NOT FOUND THEN
    RAISE EXCEPTION 'This appointment is no longer available.';
  END IF;

  IF v_appointment.status <> 'pending' THEN
    RAISE EXCEPTION 'This appointment has already been reviewed.';
  END IF;

  IF lower(btrim(p_decision)) = 'rejected' THEN
```

```
    UPDATE public.appointments
    SET status = 'rejected',
        assigned_crew_id = NULL
    WHERE id = v_appointment.id;
    RETURN;
  END IF;

  v_duration_minutes := COALESCE(v_appointment.booking_duration_minutes,
75);

  -- Prefer the crew member reserved during booking, but fall back to another
  -- currently available member when schedules or availability have changed.
  SELECT crew.id
    INTO v_assigned_crew_id
  FROM public.crew_members AS crew
  WHERE crew.is_active = true
    AND crew.is_archived = false
    AND EXISTS (
      SELECT 1
      FROM public.crew_schedules AS schedule
      WHERE schedule.crew_id = crew.id
        AND schedule.schedule_date = v_appointment.appointment_date
        AND schedule.is_working = true
        AND schedule.start_time IS NOT NULL
        AND schedule.end_time IS NOT NULL
        AND schedule.start_time <= v_appointment.start_time
        AND schedule.end_time >= v_appointment.start_time + make_interval(mins
=> v_duration_minutes)
    )
    AND NOT EXISTS (
      SELECT 1
      FROM public.crew_availability_exceptions AS exception
      WHERE exception.crew_id = crew.id
        AND exception.start_date <= v_appointment.appointment_date
        AND exception.end_date >= v_appointment.appointment_date
        AND (
          exception.is_all_day = true
          OR (
            exception.start_time IS NOT NULL
            AND exception.end_time IS NOT NULL
            AND tsrange(
              v_appointment.appointment_date + v_appointment.start_time,
              v_appointment.appointment_date + v_appointment.start_time
                + make_interval(mins => v_duration_minutes),
              '[)'
```

```
          ) && tsrange(
            v_appointment.appointment_date + exception.start_time,
            v_appointment.appointment_date + exception.end_time,
            '[)'
          )
        )
      )
    )
    AND NOT EXISTS (
      SELECT 1
      FROM public.appointments AS other
      WHERE other.id <> v_appointment.id
        AND other.assigned_crew_id = crew.id
        AND other.appointment_date = v_appointment.appointment_date
        AND other.is_archived = false
        AND other.rescheduled_to_appointment_id IS NULL
        AND other.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
        AND tsrange(
          other.appointment_date + other.start_time,
          other.appointment_date + other.start_time
            + make_interval(mins => COALESCE(other.booking_duration_minutes,
75)),
          '[)'
        ) && tsrange(
          v_appointment.appointment_date + v_appointment.start_time,
          v_appointment.appointment_date + v_appointment.start_time
            + make_interval(mins => v_duration_minutes),
          '[)'
        )
    )
  ORDER BY (crew.id = v_appointment.assigned_crew_id) DESC, crew.name,
crew.id
  LIMIT 1;

  IF v_assigned_crew_id IS NULL THEN
    RAISE EXCEPTION 'No crew member is available for the selected
appointment period.';
  END IF;

  UPDATE public.appointments
  SET status = 'confirmed',
      assigned_crew_id = v_assigned_crew_id
  WHERE id = v_appointment.id;
END;
$$;
```

```
REVOKE ALL ON FUNCTION public.review_pending_appointment(uuid, text)
FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.review_pending_appointment(uuid,
text) TO authenticated;
```

supabase/migrations/20260910110000_add_pending_reschedule_requests.sql

---

```
-- Keep the current appointment reserved while a customer-requested move is
-- reviewed. The target date/time becomes real only after an administrator
-- confirms it.
ALTER TABLE public.appointments
  ADD COLUMN IF NOT EXISTS pending_reschedule_request_id uuid,
  ADD COLUMN IF NOT EXISTS pending_reschedule_date date,
  ADD COLUMN IF NOT EXISTS pending_reschedule_start_time time,
  ADD COLUMN IF NOT EXISTS pending_reschedule_reason text;

CREATE UNIQUE INDEX IF NOT EXISTS
appointments_pending_reschedule_request_id_unique
  ON public.appointments (pending_reschedule_request_id)
  WHERE pending_reschedule_request_id IS NOT NULL;

CREATE OR REPLACE FUNCTION public.submit_reschedule_request(
  p_appointment_id uuid,
  p_request_id uuid,
  p_appointment_date date,
  p_start_time time,
  p_reason text
)
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment public.appointments%ROWTYPE;
BEGIN
  IF p_request_id IS NULL OR p_appointment_date IS NULL OR p_start_time IS
NULL
     OR NULLIF(btrim(p_reason), '') IS NULL THEN
    RAISE EXCEPTION 'A date, time, and rescheduling reason are required.';
  END IF;

  SELECT appointment.*
    INTO v_appointment
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
  FOR UPDATE;
```

```
  IF NOT FOUND THEN
    RAISE EXCEPTION 'This appointment is no longer available.';
  END IF;

  IF v_appointment.pending_reschedule_request_id = p_request_id THEN
    RETURN;
  END IF;
  IF v_appointment.pending_reschedule_request_id IS NOT NULL THEN
    RAISE EXCEPTION 'A reschedule request is already awaiting review.';
  END IF;
  IF v_appointment.status NOT IN ('pending', 'confirmed')
     OR v_appointment.rescheduled_to_appointment_id IS NOT NULL THEN
    RAISE EXCEPTION 'This appointment can no longer be rescheduled.';
  END IF;

  UPDATE public.appointments
  SET pending_reschedule_request_id = p_request_id,
      pending_reschedule_date = p_appointment_date,
      pending_reschedule_start_time = p_start_time,
      pending_reschedule_reason = NULLIF(btrim(p_reason), '')
  WHERE id = v_appointment.id;

  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES (
    'reschedule_request',
    'Reschedule request ' || v_appointment.reference_code,
    v_appointment.customer_name || ' requested ' || to_char(p_appointment_date,
'Mon FMDD, YYYY')
      || ' at ' || to_char(p_start_time, 'HH12:MI AM') || '. Reason: ' || btrim(p_reason),
    v_appointment.id
  );
END;
$$;

CREATE OR REPLACE FUNCTION public.review_reschedule_request(
  p_appointment_id uuid,
  p_decision text
)
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment public.appointments%ROWTYPE;
```

```
  v_assigned_crew_id uuid;
  v_duration_minutes integer;
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Only administrators can review reschedule requests.';
  END IF;
  IF lower(btrim(COALESCE(p_decision, ''))) NOT IN ('confirmed', 'rejected')
THEN
    RAISE EXCEPTION 'Choose either confirmed or rejected.';
  END IF;

  SELECT appointment.*
    INTO v_appointment
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
  FOR UPDATE;

  IF NOT FOUND OR v_appointment.pending_reschedule_request_id IS NULL
THEN
    RAISE EXCEPTION 'This reschedule request is no longer available.';
  END IF;

  IF lower(btrim(p_decision)) = 'rejected' THEN
    UPDATE public.appointments
    SET pending_reschedule_request_id = NULL,
        pending_reschedule_date = NULL,
        pending_reschedule_start_time = NULL,
        pending_reschedule_reason = NULL
    WHERE id = v_appointment.id;
    RETURN;
  END IF;

  v_duration_minutes := COALESCE(v_appointment.booking_duration_minutes,
75);

  IF EXISTS (
    SELECT 1
    FROM public.schedule_blocks AS block
    WHERE block.block_date = v_appointment.pending_reschedule_date
      AND block.is_active = true
      AND (
        block.start_time IS NULL
        OR (
          block.start_time IS NOT NULL
```

```
        AND (
          (COALESCE(block.reason, '') NOT LIKE 'RANGE:%'
            AND block.start_time = v_appointment.pending_reschedule_start_time)
          OR (COALESCE(block.reason, '') LIKE 'RANGE:%'
            AND tsrange(
              v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
              v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
                + make_interval(mins => v_duration_minutes),
              '[)'
            ) && tsrange(
              v_appointment.pending_reschedule_date + block.start_time,
              v_appointment.pending_reschedule_date
                + substring(block.reason FROM '^RANGE:([0-9]{2}:[0-9]{2})')::time,
              '[)'
            ))
        )
      )
    )
  ) THEN
    RAISE EXCEPTION 'The requested time is no longer available because the
shop schedule is blocked.';
  END IF;

  SELECT crew.id
    INTO v_assigned_crew_id
  FROM public.crew_members AS crew
  WHERE crew.is_active = true
    AND crew.is_archived = false
    AND EXISTS (
      SELECT 1
      FROM public.time_slots AS slot
      WHERE slot.is_active = true
        AND slot.start_time = v_appointment.pending_reschedule_start_time
        AND slot.capacity > (
          SELECT count(*)
          FROM public.appointments AS other
          WHERE other.id <> v_appointment.id
            AND other.appointment_date = v_appointment.pending_reschedule_date
            AND other.is_archived = false
            AND other.rescheduled_to_appointment_id IS NULL
            AND other.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
            AND tsrange(
              other.appointment_date + other.start_time,
```

```
            other.appointment_date + other.start_time
              + make_interval(mins =>
COALESCE(other.booking_duration_minutes, 75)), '[)'
          ) && tsrange(
            v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
            v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
              + make_interval(mins => v_duration_minutes), '[)'
          )
      )
    )
    AND EXISTS (
      SELECT 1 FROM public.crew_schedules AS schedule
      WHERE schedule.crew_id = crew.id
        AND schedule.schedule_date = v_appointment.pending_reschedule_date
        AND schedule.is_working = true
        AND schedule.start_time <= v_appointment.pending_reschedule_start_time
        AND schedule.end_time >= v_appointment.pending_reschedule_start_time
          + make_interval(mins => v_duration_minutes)
    )
    AND NOT EXISTS (
      SELECT 1 FROM public.crew_availability_exceptions AS exception
      WHERE exception.crew_id = crew.id
        AND exception.start_date <= v_appointment.pending_reschedule_date
        AND exception.end_date >= v_appointment.pending_reschedule_date
        AND (exception.is_all_day OR (
          exception.start_time IS NOT NULL AND exception.end_time IS NOT NULL
          AND tsrange(
            v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
            v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
              + make_interval(mins => v_duration_minutes), '[)'
          ) && tsrange(
            v_appointment.pending_reschedule_date + exception.start_time,
            v_appointment.pending_reschedule_date + exception.end_time, '[)'
          )
        ))
    )
    AND NOT EXISTS (
      SELECT 1 FROM public.appointments AS other
      WHERE other.id <> v_appointment.id
        AND other.assigned_crew_id = crew.id
        AND other.appointment_date = v_appointment.pending_reschedule_date
```

```
        AND other.is_archived = false
        AND other.rescheduled_to_appointment_id IS NULL
        AND other.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
        AND tsrange(
          other.appointment_date + other.start_time,
          other.appointment_date + other.start_time
            + make_interval(mins => COALESCE(other.booking_duration_minutes,
75)), '[)'
        ) && tsrange(
          v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
          v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
            + make_interval(mins => v_duration_minutes), '[)'
        )
    )
  ORDER BY (crew.id = v_appointment.assigned_crew_id) DESC, crew.name,
crew.id
  LIMIT 1;

  IF v_assigned_crew_id IS NULL THEN
    RAISE EXCEPTION 'No crew member is available for the requested
appointment period.';
  END IF;

  UPDATE public.appointments
  SET appointment_date = v_appointment.pending_reschedule_date,
      start_time = v_appointment.pending_reschedule_start_time,
      assigned_crew_id = v_assigned_crew_id,
      status = 'confirmed',
      pending_reschedule_request_id = NULL,
      pending_reschedule_date = NULL,
      pending_reschedule_start_time = NULL,
      pending_reschedule_reason = NULL
  WHERE id = v_appointment.id;
END;
$$;

REVOKE ALL ON FUNCTION public.submit_reschedule_request(uuid, uuid,
date, time, text) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.submit_reschedule_request(uuid,
uuid, date, time, text) TO service_role;
REVOKE ALL ON FUNCTION public.review_reschedule_request(uuid, text)
FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.review_reschedule_request(uuid, text)
```

TO authenticated;

supabase/migrations/20260910120000_prevent_duplicate_customer_service_appointments.sql

---

```
-- A customer cannot hold two active appointments for the same service on the
-- same day. Keep this as a deferred constraint trigger so multi-step atomic
-- appointment edits can replace services in one transaction without a false
-- positive. The customer-facing preflight check lives in booking.functions.ts;
-- this trigger is the concurrency-safe enforcement point.

CREATE OR REPLACE FUNCTION
public.prevent_duplicate_active_customer_service_appointments()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = public
AS $$
DECLARE
  v_appointment_id uuid;
  v_phone text;
  v_appointment_date date;
BEGIN
  v_appointment_id := CASE
    WHEN TG_TABLE_NAME = 'appointments' THEN NEW.id
    ELSE NEW.appointment_id
  END;

  SELECT appointment.phone, appointment.appointment_date
  INTO v_phone, v_appointment_date
  FROM public.appointments AS appointment
  WHERE appointment.id = v_appointment_id
    AND appointment.is_archived = false
    AND appointment.rescheduled_to_appointment_id IS NULL
    AND appointment.status NOT IN ('completed', 'cancelled', 'rejected');

  -- The changed appointment is not active, has been deleted as part of an
  -- update, or no longer exists. It cannot violate the active-booking rule.
  IF NOT FOUND THEN
    RETURN NEW;
  END IF;

  IF EXISTS (
    SELECT 1
    FROM public.appointment_services AS current_service
    JOIN public.appointments AS other_appointment
      ON other_appointment.phone = v_phone
     AND other_appointment.appointment_date = v_appointment_date
```

```
    AND other_appointment.id <> v_appointment_id
    AND other_appointment.is_archived = false
    AND other_appointment.rescheduled_to_appointment_id IS NULL
    AND other_appointment.status NOT IN ('completed', 'cancelled', 'rejected')
   JOIN public.appointment_services AS other_service
     ON other_service.appointment_id = other_appointment.id
    AND (
      other_service.service_id = current_service.service_id
      OR (
        other_service.service_id IS NULL
        AND current_service.service_id IS NULL
        AND other_service.service_name = current_service.service_name
      )
    )
   WHERE current_service.appointment_id = v_appointment_id
  ) THEN
    RAISE EXCEPTION
      'This customer already has an active appointment for one or more selected
services on this date. Complete, cancel, or reject it before scheduling another.'
      USING ERRCODE = '23505';
  END IF;

  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS
appointments_prevent_duplicate_active_customer_services ON
public.appointments;
CREATE CONSTRAINT TRIGGER
appointments_prevent_duplicate_active_customer_services
  AFTER INSERT OR UPDATE ON public.appointments
  DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW
  EXECUTE FUNCTION
public.prevent_duplicate_active_customer_service_appointments();

DROP TRIGGER IF EXISTS
appointment_services_prevent_duplicate_active_customer_services ON
public.appointment_services;
CREATE CONSTRAINT TRIGGER
appointment_services_prevent_duplicate_active_customer_services
  AFTER INSERT OR UPDATE ON public.appointment_services
  DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW
```

```
  EXECUTE FUNCTION
public.prevent_duplicate_active_customer_service_appointments();

CREATE INDEX IF NOT EXISTS appointments_active_customer_date_idx
  ON public.appointments (phone, appointment_date)
  WHERE is_archived = false
    AND rescheduled_to_appointment_id IS NULL
    AND status NOT IN ('completed', 'cancelled', 'rejected');

CREATE INDEX IF NOT EXISTS
appointment_services_service_appointment_idx
  ON public.appointment_services (service_id, appointment_id);
```

supabase/migrations/20260910121000_include_duplicate_appointment_details.sql

---

```
-- Return the conflicting service and reserved schedule in the concurrency-safe
-- database error as well, so a race condition receives the same useful feedback
-- as the booking preflight check.

CREATE OR REPLACE FUNCTION
public.prevent_duplicate_active_customer_service_appointments()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = public
AS $$
DECLARE
  v_appointment_id uuid;
  v_phone text;
  v_appointment_date date;
  v_service_name text;
  v_start_time time;
BEGIN
  v_appointment_id := CASE
    WHEN TG_TABLE_NAME = 'appointments' THEN NEW.id
    ELSE NEW.appointment_id
  END;

  SELECT appointment.phone, appointment.appointment_date
  INTO v_phone, v_appointment_date
  FROM public.appointments AS appointment
  WHERE appointment.id = v_appointment_id
    AND appointment.is_archived = false
    AND appointment.rescheduled_to_appointment_id IS NULL
    AND appointment.status NOT IN ('completed', 'cancelled', 'rejected');

  IF NOT FOUND THEN
    RETURN NEW;
  END IF;

  SELECT current_service.service_name, other_appointment.start_time
  INTO v_service_name, v_start_time
  FROM public.appointment_services AS current_service
  JOIN public.appointments AS other_appointment
    ON other_appointment.phone = v_phone
   AND other_appointment.appointment_date = v_appointment_date
   AND other_appointment.id <> v_appointment_id
   AND other_appointment.is_archived = false
```

```
   AND other_appointment.rescheduled_to_appointment_id IS NULL
   AND other_appointment.status NOT IN ('completed', 'cancelled', 'rejected')
  JOIN public.appointment_services AS other_service
    ON other_service.appointment_id = other_appointment.id
   AND (
     other_service.service_id = current_service.service_id
     OR (
       other_service.service_id IS NULL
       AND current_service.service_id IS NULL
       AND other_service.service_name = current_service.service_name
     )
   )
  WHERE current_service.appointment_id = v_appointment_id
  LIMIT 1;

  IF FOUND THEN
    RAISE EXCEPTION
      'You already have an appointment for "%" on % at %.',
      v_service_name,
      to_char(v_appointment_date, 'FMDay, FMMonth FMDD, YYYY'),
      to_char(v_start_time, 'FMHH12:MI AM')
      USING ERRCODE = '23505';
  END IF;

  RETURN NEW;
END;
$$;
```

supabase/migrations/20260910122000_fix_duplicate_service_trigger_row_types.sql

---

```
-- The original duplicate-service trigger was attached to both appointments
-- and appointment_services. PostgreSQL caches a trigger function's NEW row
-- shape, so one shared function cannot safely access table-specific columns.
-- Keep the validation in one helper and use a small trigger wrapper per table.

CREATE OR REPLACE FUNCTION
public.assert_no_duplicate_active_customer_service_appointment(
  p_appointment_id uuid
)
RETURNS void
LANGUAGE plpgsql
SET search_path = public
AS $$
DECLARE
  v_phone text;
  v_appointment_date date;
  v_service_name text;
  v_start_time time;
BEGIN
  SELECT appointment.phone, appointment.appointment_date
  INTO v_phone, v_appointment_date
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
    AND appointment.rescheduled_to_appointment_id IS NULL
    AND appointment.status NOT IN ('completed', 'cancelled', 'rejected');

  IF NOT FOUND THEN
    RETURN;
  END IF;

  SELECT current_service.service_name, other_appointment.start_time
  INTO v_service_name, v_start_time
  FROM public.appointment_services AS current_service
  JOIN public.appointments AS other_appointment
    ON other_appointment.phone = v_phone
   AND other_appointment.appointment_date = v_appointment_date
   AND other_appointment.id <> p_appointment_id
   AND other_appointment.is_archived = false
   AND other_appointment.rescheduled_to_appointment_id IS NULL
   AND other_appointment.status NOT IN ('completed', 'cancelled', 'rejected')
  JOIN public.appointment_services AS other_service
```

```
   ON other_service.appointment_id = other_appointment.id
  AND (
    other_service.service_id = current_service.service_id
    OR (
      other_service.service_id IS NULL
      AND current_service.service_id IS NULL
      AND other_service.service_name = current_service.service_name
    )
  )
  WHERE current_service.appointment_id = p_appointment_id
  LIMIT 1;

  IF FOUND THEN
    RAISE EXCEPTION
      'You already have an appointment for "%" on % at %.',
      v_service_name,
      to_char(v_appointment_date, 'FMDay, FMMonth FMDD, YYYY'),
      to_char(v_start_time, 'FMHH12:MI AM')
      USING ERRCODE = '23505';
  END IF;
END;
$$;

CREATE OR REPLACE FUNCTION
public.prevent_duplicate_active_customer_service_from_appointment()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = public
AS $$
BEGIN
  PERFORM
public.assert_no_duplicate_active_customer_service_appointment(NEW.id);
  RETURN NEW;
END;
$$;

CREATE OR REPLACE FUNCTION
public.prevent_duplicate_active_customer_service_from_service()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = public
AS $$
BEGIN
  PERFORM public.assert_no_duplicate_active_customer_service_appointment(
NEW.appointment_id);
```

```
  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS
appointments_prevent_duplicate_active_customer_services ON
public.appointments;
CREATE CONSTRAINT TRIGGER
appointments_prevent_duplicate_active_customer_services
  AFTER INSERT OR UPDATE ON public.appointments
  DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW
  EXECUTE FUNCTION
public.prevent_duplicate_active_customer_service_from_appointment();

DROP TRIGGER IF EXISTS
appointment_services_prevent_duplicate_active_customer_services ON
public.appointment_services;
CREATE CONSTRAINT TRIGGER
appointment_services_prevent_duplicate_active_customer_services
  AFTER INSERT OR UPDATE ON public.appointment_services
  DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW
  EXECUTE FUNCTION
public.prevent_duplicate_active_customer_service_from_service();
```

supabase/migrations/20260911100000_limit_and_track_appointment_reschedules.sql

---

```
-- A reschedule changes an existing appointment in place. Keep a durable count so
-- both the customer portal and the admin console share the same three-move limit.
ALTER TABLE public.appointments
  ADD COLUMN IF NOT EXISTS reschedule_count integer NOT NULL DEFAULT 0,
  ADD COLUMN IF NOT EXISTS last_reschedule_rejected_at timestamptz,
  ADD COLUMN IF NOT EXISTS last_reschedule_rejection_message text;

ALTER TABLE public.appointments
  DROP CONSTRAINT IF EXISTS appointments_reschedule_count_range;

ALTER TABLE public.appointments
  ADD CONSTRAINT appointments_reschedule_count_range
  CHECK (reschedule_count BETWEEN 0 AND 3);

CREATE OR REPLACE FUNCTION public.track_appointment_reschedule()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = public
AS $$
BEGIN
  IF NEW.appointment_date IS DISTINCT FROM OLD.appointment_date
     OR NEW.start_time IS DISTINCT FROM OLD.start_time THEN
    IF OLD.reschedule_count >= 3 THEN
      RAISE EXCEPTION 'This appointment has reached the maximum of 3 reschedules.';
    END IF;

    NEW.reschedule_count := OLD.reschedule_count + 1;
    NEW.last_reschedule_rejected_at := NULL;
    NEW.last_reschedule_rejection_message := NULL;
  ELSIF OLD.pending_reschedule_request_id IS NULL
     AND NEW.pending_reschedule_request_id IS NOT NULL THEN
    -- A fresh request replaces an earlier rejection notice.
    NEW.last_reschedule_rejected_at := NULL;
    NEW.last_reschedule_rejection_message := NULL;
  ELSIF OLD.pending_reschedule_request_id IS NOT NULL
     AND NEW.pending_reschedule_request_id IS NULL THEN
    -- Rejecting a request clears only the pending request fields; the original
    -- appointment date and time stay reserved and are therefore unchanged here.
```

```
    NEW.last_reschedule_rejected_at := now();
    NEW.last_reschedule_rejection_message :=
      'Reschedule Request Rejected. Your requested reschedule was not
approved. Your original appointment remains unchanged and reserved.';
  END IF;

  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS appointments_track_reschedule ON
public.appointments;
CREATE TRIGGER appointments_track_reschedule
  BEFORE UPDATE OF appointment_date, start_time,
pending_reschedule_request_id
  ON public.appointments
  FOR EACH ROW
  EXECUTE FUNCTION public.track_appointment_reschedule();

CREATE OR REPLACE FUNCTION public.submit_reschedule_request(
  p_appointment_id uuid,
  p_request_id uuid,
  p_appointment_date date,
  p_start_time time,
  p_reason text
)
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment public.appointments%ROWTYPE;
BEGIN
  IF p_request_id IS NULL OR p_appointment_date IS NULL OR p_start_time IS
NULL
     OR NULLIF(btrim(p_reason), '') IS NULL THEN
    RAISE EXCEPTION 'A date, time, and rescheduling reason are required.';
  END IF;

  SELECT appointment.*
    INTO v_appointment
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
```

```
  FOR UPDATE;

  IF NOT FOUND THEN
    RAISE EXCEPTION 'This appointment is no longer available.';
  END IF;

  IF v_appointment.pending_reschedule_request_id = p_request_id THEN
    RETURN;
  END IF;
  IF v_appointment.pending_reschedule_request_id IS NOT NULL THEN
    RAISE EXCEPTION 'A reschedule request is already awaiting review.';
  END IF;
  IF v_appointment.reschedule_count >= 3 THEN
    RAISE EXCEPTION 'This appointment has reached the maximum of 3
reschedules.';
  END IF;
  IF v_appointment.status NOT IN ('pending', 'confirmed')
     OR v_appointment.rescheduled_to_appointment_id IS NOT NULL THEN
    RAISE EXCEPTION 'This appointment can no longer be rescheduled.';
  END IF;

  UPDATE public.appointments
  SET pending_reschedule_request_id = p_request_id,
      pending_reschedule_date = p_appointment_date,
      pending_reschedule_start_time = p_start_time,
      pending_reschedule_reason = NULLIF(btrim(p_reason), '')
  WHERE id = v_appointment.id;

  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES (
    'reschedule_request',
    'Reschedule request ' || v_appointment.reference_code,
    v_appointment.customer_name || ' requested ' || to_char(p_appointment_date,
'Mon FMDD, YYYY')
      || ' at ' || to_char(p_start_time, 'HH12:MI AM') || '. Reason: ' || btrim(p_reason),
    v_appointment.id
  );
END;
$$;

REVOKE ALL ON FUNCTION public.submit_reschedule_request(uuid, uuid,
date, time, text) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.submit_reschedule_request(uuid,
uuid, date, time, text) TO service_role;
```

supabase/migrations/20260912090000_add_structured_appointment_names.sql

---

```
-- Store the individual customer name fields while retaining customer_name as
-- the backwards-compatible full-name value used by existing reports and views.
ALTER TABLE public.appointments
  ADD COLUMN IF NOT EXISTS first_name text,
  ADD COLUMN IF NOT EXISTS middle_name text,
  ADD COLUMN IF NOT EXISTS last_name text;

-- Give existing appointments editable starting values. The original full name
-- remains untouched, so no historical information is lost when a legacy name
-- cannot be split perfectly.
UPDATE public.appointments
SET first_name = COALESCE(first_name, split_part(btrim(customer_name), ' ',
1)),
    middle_name = COALESCE(
      middle_name,
      NULLIF(
        regexp_replace(
          regexp_replace(btrim(customer_name), '^[^[:space:]]+[[:space:]]*', ''),
          '[[:space:]]*[^[:space:]]+$',
          ''
        ),
        ''
      )
    ),
    last_name = COALESCE(
      last_name,
      CASE
        WHEN btrim(customer_name) LIKE '% %' THEN
regexp_replace(btrim(customer_name), '^.*[[:space:]]+', '')
        ELSE NULL
      END
    )
WHERE first_name IS NULL OR middle_name IS NULL OR last_name IS NULL;

CREATE FUNCTION public.create_booking_atomic(
  p_reference_code text,
  p_booking_request_id uuid,
  p_customer_name text,
  p_phone text,
  p_email text,
  p_moto_brand text,
  p_moto_model text,
```

```
  p_moto_variant text,
  p_moto_year integer,
  p_plate_number text,
  p_appointment_date date,
  p_start_time time,
  p_notes text,
  p_total_estimate numeric,
  p_booking_duration_minutes integer,
  p_assigned_crew_id uuid,
  p_services jsonb,
  p_notification_title text,
  p_notification_message text,
  p_first_name text,
  p_middle_name text,
  p_last_name text
)
RETURNS TABLE (appointment_id uuid, reference_code text)
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment_id uuid;
  v_reference_code text;
BEGIN
  IF COALESCE(btrim(p_first_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$'
     OR COALESCE(btrim(p_middle_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$'
     OR COALESCE(btrim(p_last_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$' THEN
    RAISE EXCEPTION 'First name, middle name, and last name must contain
letters only.';
  END IF;

  SELECT created.appointment_id, created.reference_code
    INTO v_appointment_id, v_reference_code
  FROM public.create_booking_atomic(
    p_reference_code,
    p_booking_request_id,
    p_customer_name,
    p_phone,
    p_email,
    p_moto_brand,
    p_moto_model,
    p_moto_variant,
    p_moto_year,
    p_plate_number,
```

```
    p_appointment_date,
    p_start_time,
    p_notes,
    p_total_estimate,
    p_booking_duration_minutes,
    p_assigned_crew_id,
    p_services,
    p_notification_title,
    p_notification_message
  ) AS created;

  UPDATE public.appointments
  SET first_name = btrim(p_first_name),
      middle_name = btrim(p_middle_name),
      last_name = btrim(p_last_name),
      customer_name = concat_ws(' ', btrim(p_first_name), btrim(p_middle_name),
btrim(p_last_name))
  WHERE id = v_appointment_id;

  RETURN QUERY SELECT v_appointment_id, v_reference_code;
END;
$$;

REVOKE ALL ON FUNCTION public.create_booking_atomic(
  text, uuid, text, text, text, text, text, text, integer, text, date, time,
  text, numeric, integer, uuid, jsonb, text, text, text, text, text
) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.create_booking_atomic(
  text, uuid, text, text, text, text, text, text, integer, text, date, time,
  text, numeric, integer, uuid, jsonb, text, text, text, text, text
) TO service_role;

CREATE FUNCTION public.update_appointment_details_atomic(
  p_appointment_id uuid,
  p_service_ids uuid[],
  p_appointment_date date,
  p_start_time time,
  p_assigned_crew_id uuid,
  p_crew_assignment_manual boolean,
  p_status text,
  p_admin_notes text,
  p_first_name text,
  p_middle_name text,
  p_last_name text
)
```

```
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
BEGIN
  IF COALESCE(btrim(p_first_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$'
     OR COALESCE(btrim(p_middle_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$'
     OR COALESCE(btrim(p_last_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$' THEN
    RAISE EXCEPTION 'First name, middle name, and last name must contain
letters only.';
  END IF;

  PERFORM public.update_appointment_details_atomic(
    p_appointment_id,
    p_service_ids,
    p_appointment_date,
    p_start_time,
    p_assigned_crew_id,
    p_crew_assignment_manual,
    p_status,
    p_admin_notes
  );

  UPDATE public.appointments
  SET first_name = btrim(p_first_name),
      middle_name = btrim(p_middle_name),
      last_name = btrim(p_last_name),
      customer_name = concat_ws(' ', btrim(p_first_name), btrim(p_middle_name),
btrim(p_last_name))
  WHERE id = p_appointment_id
    AND is_archived = false;
END;
$$;

REVOKE ALL ON FUNCTION public.update_appointment_details_atomic(
  uuid, uuid[], date, time, uuid, boolean, text, text, text, text, text
) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.update_appointment_details_atomic(
  uuid, uuid[], date, time, uuid, boolean, text, text, text, text, text
) TO authenticated;
```

supabase/migrations/20260912100000_add_motorcycle_catalog_specifications.sql

---

```
-- Motorcycle catalog fields. Engine displacement is nullable only for electric
motorcycles.
ALTER TABLE public.products
  ADD COLUMN IF NOT EXISTS engine_cc numeric(6, 1),
  ADD COLUMN IF NOT EXISTS fuel_type text,
  ADD COLUMN IF NOT EXISTS transmission text;

DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_constraint WHERE conname =
'products_engine_cc_positive') THEN
    ALTER TABLE public.products
      ADD CONSTRAINT products_engine_cc_positive CHECK (engine_cc IS
NULL OR engine_cc > 0);
  END IF;

  IF NOT EXISTS (SELECT 1 FROM pg_constraint WHERE conname =
'products_fuel_type_valid') THEN
    ALTER TABLE public.products
      ADD CONSTRAINT products_fuel_type_valid
      CHECK (fuel_type IS NULL OR fuel_type IN ('FI', 'Carbureted', 'Electric'));
  END IF;

  IF NOT EXISTS (SELECT 1 FROM pg_constraint WHERE conname =
'products_transmission_valid') THEN
    ALTER TABLE public.products
      ADD CONSTRAINT products_transmission_valid
      CHECK (transmission IS NULL OR transmission IN ('Manual',
'Semi-Automatic', 'Automatic'));
  END IF;
END
$$;

-- Verified Philippine-market model baseline. Entries are model-level; trim-only
variants
-- with identical powertrain data share a catalog row. Sources: Honda PH, Suzuki
PH, RE PH.
INSERT INTO public.products (
  name, category, brand, engine_cc, fuel_type, transmission, is_active, in_stock,
sort_order
)
VALUES
```

('Honda ADV160', 'motorcycle', 'Honda', 156.9, 'FI', 'Automatic', true, true, 100),
('Honda AirBlade160', 'motorcycle', 'Honda', 156.9, 'FI', 'Automatic', true, true, 101),
('Honda Beat', 'motorcycle', 'Honda', 109.5, 'FI', 'Automatic', true, true, 102),
('Honda CB1000R', 'motorcycle', 'Honda', 998.0, 'FI', 'Manual', true, true, 103),
('Honda CB150R', 'motorcycle', 'Honda', 149.2, 'FI', 'Manual', true, true, 104),
('Honda CB150X', 'motorcycle', 'Honda', 149.2, 'FI', 'Manual', true, true, 105),
('Honda CB500F', 'motorcycle', 'Honda', 471.0, 'FI', 'Manual', true, true, 106),
('Honda CB500X', 'motorcycle', 'Honda', 471.0, 'FI', 'Manual', true, true, 107),
('Honda CB650R', 'motorcycle', 'Honda', 649.0, 'FI', 'Manual', true, true, 108),
('Honda CBR150R', 'motorcycle', 'Honda', 149.2, 'FI', 'Manual', true, true, 109),
('Honda CBR500R', 'motorcycle', 'Honda', 471.0, 'FI', 'Manual', true, true, 110),
('Honda CBR650R', 'motorcycle', 'Honda', 649.0, 'FI', 'Manual', true, true, 111),
('Honda CL500', 'motorcycle', 'Honda', 471.0, 'FI', 'Manual', true, true, 112),
('Honda Click125', 'motorcycle', 'Honda', 124.9, 'FI', 'Automatic', true, true, 113),
('Honda Click160', 'motorcycle', 'Honda', 156.9, 'FI', 'Automatic', true, true, 114),
('Honda CRF150L', 'motorcycle', 'Honda', 149.2, 'FI', 'Manual', true, true, 115),
('Honda CRF300L', 'motorcycle', 'Honda', 286.0, 'FI', 'Manual', true, true, 116),
('Honda CRF300 Rally', 'motorcycle', 'Honda', 286.0, 'FI', 'Manual', true, true, 117),
('Honda Dio', 'motorcycle', 'Honda', 109.5, 'FI', 'Automatic', true, true, 118),
('Honda EM1 e:', 'motorcycle', 'Honda', null, 'Electric', 'Automatic', true, true, 119),
('Honda Gold Wing', 'motorcycle', 'Honda', 1833.0, 'FI', 'Automatic', true, true, 120),
('Honda NAVi', 'motorcycle', 'Honda', 109.2, 'Carbureted', 'Automatic', true, true, 121),
('Honda PCX160', 'motorcycle', 'Honda', 156.9, 'FI', 'Automatic', true, true, 122),
('Honda Rebel500', 'motorcycle', 'Honda', 471.0, 'FI', 'Manual', true, true, 123),
('Honda RS125 Fi', 'motorcycle', 'Honda', 124.9, 'FI', 'Semi-Automatic', true, true, 124),
('Honda Supra GTR150', 'motorcycle', 'Honda', 149.2, 'FI', 'Semi-Automatic', true, true, 125),
('Honda TMX125 Alpha', 'motorcycle', 'Honda', 124.1, 'Carbureted', 'Manual', true, true, 126),
('Honda TMX Supremo', 'motorcycle', 'Honda', 149.0, 'Carbureted', 'Manual', true, true, 127),
('Honda Wave RSX', 'motorcycle', 'Honda', 109.1, 'FI', 'Semi-Automatic', true, true, 128),
('Honda X-ADV', 'motorcycle', 'Honda', 745.0, 'FI', 'Automatic', true, true, 129),
('Honda XL750 Transalp', 'motorcycle', 'Honda', 755.0, 'FI', 'Manual', true, true, 130),
('Honda XRM125 DS', 'motorcycle', 'Honda', 124.9, 'FI', 'Semi-Automatic', true, true, 131),
('Honda XR150L', 'motorcycle', 'Honda', 149.2, 'Carbureted', 'Manual', true, true,

132),

('Kawasaki KLX150', 'motorcycle', 'Kawasaki', 144.0, 'Carbureted', 'Manual', true, true, 200),
('Kawasaki Z150', 'motorcycle', 'Kawasaki', 144.0, 'Carbureted', 'Manual', true, true, 201),

('KTM 125 Duke', 'motorcycle', 'KTM', 124.7, 'FI', 'Manual', true, true, 300),
('KTM 200 Duke', 'motorcycle', 'KTM', 199.5, 'FI', 'Manual', true, true, 301),
('KTM 390 Adventure', 'motorcycle', 'KTM', 373.0, 'FI', 'Manual', true, true, 302),
('KTM 390 Duke', 'motorcycle', 'KTM', 373.0, 'FI', 'Manual', true, true, 303),
('KTM 790 Adventure', 'motorcycle', 'KTM', 799.0, 'FI', 'Manual', true, true, 304),
('KTM 790 Duke', 'motorcycle', 'KTM', 799.0, 'FI', 'Manual', true, true, 305),
('KTM RC 200', 'motorcycle', 'KTM', 199.5, 'FI', 'Manual', true, true, 306),
('KTM RC 390', 'motorcycle', 'KTM', 373.0, 'FI', 'Manual', true, true, 307),

('Royal Enfield Bear 650', 'motorcycle', 'Royal Enfield', 648.0, 'FI', 'Manual', true, true, 400),
('Royal Enfield Bullet 350', 'motorcycle', 'Royal Enfield', 349.0, 'FI', 'Manual', true, true, 401),
('Royal Enfield Classic 350', 'motorcycle', 'Royal Enfield', 349.0, 'FI', 'Manual', true, true, 402),
('Royal Enfield Classic 650', 'motorcycle', 'Royal Enfield', 648.0, 'FI', 'Manual', true, true, 403),
('Royal Enfield Continental GT 650', 'motorcycle', 'Royal Enfield', 648.0, 'FI', 'Manual', true, true, 404),
('Royal Enfield Goan Classic 350', 'motorcycle', 'Royal Enfield', 349.0, 'FI', 'Manual', true, true, 405),
('Royal Enfield Guerrilla 450', 'motorcycle', 'Royal Enfield', 452.0, 'FI', 'Manual', true, true, 406),
('Royal Enfield Himalayan 410', 'motorcycle', 'Royal Enfield', 411.0, 'FI', 'Manual', true, true, 407),
('Royal Enfield Himalayan 450', 'motorcycle', 'Royal Enfield', 452.0, 'FI', 'Manual', true, true, 408),
('Royal Enfield Hunter 350', 'motorcycle', 'Royal Enfield', 349.0, 'FI', 'Manual', true, true, 409),
('Royal Enfield Interceptor 650', 'motorcycle', 'Royal Enfield', 648.0, 'FI', 'Manual', true, true, 410),
('Royal Enfield Meteor 350', 'motorcycle', 'Royal Enfield', 349.0, 'FI', 'Manual', true, true, 411),
('Royal Enfield Scram 411', 'motorcycle', 'Royal Enfield', 411.0, 'FI', 'Manual', true, true, 412),
('Royal Enfield Shotgun 650', 'motorcycle', 'Royal Enfield', 648.0, 'FI', 'Manual', true, true, 413),
('Royal Enfield Super Meteor 650', 'motorcycle', 'Royal Enfield', 648.0, 'FI',

'Manual', true, true, 414),

('Suzuki Access', 'motorcycle', 'Suzuki', 124.0, 'FI', 'Automatic', true, true, 500),
('Suzuki Access Ride Connect Edition', 'motorcycle', 'Suzuki', 124.0, 'FI', 'Automatic', true, true, 501),
('Suzuki Avenis', 'motorcycle', 'Suzuki', 124.0, 'FI', 'Automatic', true, true, 502),
('Suzuki Burgman 15', 'motorcycle', 'Suzuki', 149.0, 'FI', 'Automatic', true, true, 503),
('Suzuki Burgman 400', 'motorcycle', 'Suzuki', 400.0, 'FI', 'Automatic', true, true, 504),
('Suzuki Burgman Street', 'motorcycle', 'Suzuki', 124.0, 'FI', 'Automatic', true, true, 505),
('Suzuki Burgman Street 125 EX', 'motorcycle', 'Suzuki', 124.0, 'FI', 'Automatic', true, true, 506),
('Suzuki DR160', 'motorcycle', 'Suzuki', 162.0, 'FI', 'Manual', true, true, 507),
('Suzuki DR-Z4S', 'motorcycle', 'Suzuki', 398.0, 'FI', 'Manual', true, true, 508),
('Suzuki DR-Z4SM', 'motorcycle', 'Suzuki', 398.0, 'FI', 'Manual', true, true, 509),
('Suzuki GIXXER 155', 'motorcycle', 'Suzuki', 155.0, 'FI', 'Manual', true, true, 510),
('Suzuki GIXXER 250', 'motorcycle', 'Suzuki', 249.0, 'FI', 'Manual', true, true, 511),
('Suzuki GIXXER SF 155', 'motorcycle', 'Suzuki', 155.0, 'FI', 'Manual', true, true, 512),
('Suzuki GIXXER SF250', 'motorcycle', 'Suzuki', 249.0, 'FI', 'Manual', true, true, 513),
('Suzuki GN160', 'motorcycle', 'Suzuki', 155.0, 'Carbureted', 'Manual', true, true, 514),
('Suzuki GSX-8R', 'motorcycle', 'Suzuki', 776.0, 'FI', 'Manual', true, true, 515),
('Suzuki GSX-8S', 'motorcycle', 'Suzuki', 776.0, 'FI', 'Manual', true, true, 516),
('Suzuki GSX-8T', 'motorcycle', 'Suzuki', 776.0, 'FI', 'Manual', true, true, 517),
('Suzuki GSX-8TT', 'motorcycle', 'Suzuki', 776.0, 'FI', 'Manual', true, true, 518),
('Suzuki GSX-R1000', 'motorcycle', 'Suzuki', 999.0, 'FI', 'Manual', true, true, 519),
('Suzuki GSX-R1000R', 'motorcycle', 'Suzuki', 999.8, 'FI', 'Manual', true, true, 520),
('Suzuki GSX-S1000', 'motorcycle', 'Suzuki', 999.0, 'FI', 'Manual', true, true, 521),
('Suzuki GSX-S1000GT', 'motorcycle', 'Suzuki', 999.0, 'FI', 'Manual', true, true, 522),
('Suzuki GSX-S1000GX', 'motorcycle', 'Suzuki', 999.0, 'FI', 'Manual', true, true, 523),
('Suzuki GSX-S150', 'motorcycle', 'Suzuki', 147.3, 'FI', 'Manual', true, true, 524),
('Suzuki Hayabusa', 'motorcycle', 'Suzuki', 1340.0, 'FI', 'Manual', true, true, 525),
('Suzuki Raider J Crossover', 'motorcycle', 'Suzuki', 113.0, 'FI', 'Semi-Automatic', true, true, 526),

```
  ('Suzuki Raider PRO', 'motorcycle', 'Suzuki', 147.3, 'FI', 'Manual', true, true,
527),
  ('Suzuki Raider R150', 'motorcycle', 'Suzuki', 147.3, 'FI', 'Manual', true, true,
528),
  ('Suzuki Raider R150 Fi', 'motorcycle', 'Suzuki', 147.3, 'FI', 'Manual', true, true,
529),
  ('Suzuki RM-Z250', 'motorcycle', 'Suzuki', 249.0, 'FI', 'Manual', true, true, 530),
  ('Suzuki RM-Z450', 'motorcycle', 'Suzuki', 449.0, 'FI', 'Manual', true, true, 531),
  ('Suzuki Skydrive Sport', 'motorcycle', 'Suzuki', 113.0, 'FI', 'Automatic', true, true,
532),
  ('Suzuki Smash Carb', 'motorcycle', 'Suzuki', 109.7, 'Carbureted',
'Semi-Automatic', true, true, 533),
  ('Suzuki Smash Fi', 'motorcycle', 'Suzuki', 113.0, 'FI', 'Semi-Automatic', true,
true, 534),
  ('Suzuki SV650', 'motorcycle', 'Suzuki', 645.0, 'FI', 'Manual', true, true, 535),
  ('Suzuki V-STROM 1050DE', 'motorcycle', 'Suzuki', 1037.0, 'FI', 'Manual', true,
true, 536),
  ('Suzuki V-STROM 160', 'motorcycle', 'Suzuki', 162.0, 'FI', 'Manual', true, true,
537),
  ('Suzuki V-STROM 250 SX', 'motorcycle', 'Suzuki', 249.0, 'FI', 'Manual', true,
true, 538),
  ('Suzuki V-STROM 800DE', 'motorcycle', 'Suzuki', 776.0, 'FI', 'Manual', true,
true, 539),

  ('TVS Apache RTR 160', 'motorcycle', 'TVS', 159.7, 'Carbureted', 'Manual', true,
true, 600),

  ('Yamaha FZ-S', 'motorcycle', 'Yamaha', 149.0, 'FI', 'Manual', true, true, 700),
  ('Yamaha MT-15', 'motorcycle', 'Yamaha', 155.0, 'FI', 'Manual', true, true, 701),
  ('Yamaha Mio', 'motorcycle', 'Yamaha', 125.0, 'FI', 'Automatic', true, true, 702),
  ('Yamaha NMAX', 'motorcycle', 'Yamaha', 155.0, 'FI', 'Automatic', true, true,
703),
  ('Yamaha R15', 'motorcycle', 'Yamaha', 155.0, 'FI', 'Manual', true, true, 704)
ON CONFLICT (name) DO UPDATE
SET
  category = EXCLUDED.category,
  brand = EXCLUDED.brand,
  engine_cc = EXCLUDED.engine_cc,
  fuel_type = EXCLUDED.fuel_type,
  transmission = EXCLUDED.transmission,
  is_active = EXCLUDED.is_active;

CREATE INDEX IF NOT EXISTS products_motorcycle_catalog_sort_idx
  ON public.products (brand, name)
  WHERE category = 'motorcycle' AND is_archived = false;
```

supabase/migrations/20260912110000_correct_legacy_motorcycle_catalog_row.sql

---

```
-- Correct a legacy catalog entry that was saved with the wrong manufacturer and no specifications.
-- The S 1000 R is a BMW roadster, not a Honda model.
UPDATE public.products
SET
  name = 'BMW S 1000 R',
  brand = 'BMW',
  engine_cc = 999.0,
  fuel_type = 'FI',
  transmission = 'Manual',
  is_active = true
WHERE category = 'motorcycle'
  AND lower(name) = 'honda s1000r';
```

supabase/migrations/20260912120000_create_service_configurations.sql

---

```
-- Service rules allow the admin to price and time work by motorcycle attributes.
CREATE TABLE public.service_configurations (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  service_id uuid NOT NULL REFERENCES public.services(id) ON DELETE
CASCADE,
  cc_category text NOT NULL,
  fuel_type text NOT NULL,
  transmission text NOT NULL,
  duration_minutes integer NOT NULL CHECK (duration_minutes BETWEEN 15
AND 480),
  price numeric(10,2) NOT NULL CHECK (price >= 0),
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);

CREATE INDEX service_configurations_service_id_idx ON
public.service_configurations (service_id);

GRANT SELECT, INSERT, UPDATE, DELETE ON public.service_configurations
TO authenticated;
GRANT ALL ON public.service_configurations TO service_role;

ALTER TABLE public.service_configurations ENABLE ROW LEVEL SECURITY;
CREATE POLICY "service configurations admin all" ON
public.service_configurations
  FOR ALL TO authenticated
  USING (public.has_role(auth.uid(), 'admin'))
  WITH CHECK (public.has_role(auth.uid(), 'admin'));

CREATE TRIGGER service_configurations_updated
  BEFORE UPDATE ON public.service_configurations
  FOR EACH ROW EXECUTE FUNCTION public.update_updated_at_column();

-- Model overrides are service-level adjustments that take precedence over the
-- applicable configuration for a particular brand and model.
CREATE TABLE public.service_model_overrides (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  service_id uuid NOT NULL REFERENCES public.services(id) ON DELETE
CASCADE,
  brand text NOT NULL,
  model text NOT NULL,
  duration_minutes integer NOT NULL CHECK (duration_minutes BETWEEN 15
AND 480),
```

```
  price numeric(10,2) NOT NULL CHECK (price >= 0),
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (service_id, brand, model)
);

CREATE INDEX service_model_overrides_service_id_idx ON
public.service_model_overrides (service_id);

GRANT SELECT, INSERT, UPDATE, DELETE ON
public.service_model_overrides TO authenticated;
GRANT ALL ON public.service_model_overrides TO service_role;

ALTER TABLE public.service_model_overrides ENABLE ROW LEVEL
SECURITY;
CREATE POLICY "service model overrides admin all" ON
public.service_model_overrides
  FOR ALL TO authenticated
  USING (public.has_role(auth.uid(), 'admin'))
  WITH CHECK (public.has_role(auth.uid(), 'admin'));

CREATE TRIGGER service_model_overrides_updated
  BEFORE UPDATE ON public.service_model_overrides
  FOR EACH ROW EXECUTE FUNCTION public.update_updated_at_column();

-- Include configuration edits in the existing admin activity trail.
CREATE TRIGGER admin_activity_service_configurations
  AFTER INSERT OR UPDATE OR DELETE ON public.service_configurations
  FOR EACH ROW EXECUTE FUNCTION public.record_admin_activity();

CREATE TRIGGER admin_activity_service_model_overrides
  AFTER INSERT OR UPDATE OR DELETE ON public.service_model_overrides
  FOR EACH ROW EXECUTE FUNCTION public.record_admin_activity();
```

supabase/migrations/20260913110000_add_price_configuration_history_and_archive_dates.sql

---

```
-- Price changes can now identify the exact service or product configuration
-- that changed, while retaining compatibility with existing item-level history.
ALTER TABLE public.price_history
  ADD COLUMN IF NOT EXISTS configuration_id uuid,
  ADD COLUMN IF NOT EXISTS configuration_label text;

UPDATE public.price_history
SET configuration_label = CASE
  WHEN table_name = 'services' THEN 'Base service'
  WHEN table_name = 'products' THEN 'Default item'
  ELSE 'Default configuration'
END
WHERE configuration_label IS NULL;

CREATE INDEX IF NOT EXISTS price_history_configuration_idx
  ON public.price_history (table_name, item_id, configuration_id, created_at
DESC);

-- Preserve the actual archive event rather than relying on updated_at, which
-- can change later when a record is restored or edited.
ALTER TABLE public.services
  ADD COLUMN IF NOT EXISTS archived_at timestamptz;

ALTER TABLE public.products
  ADD COLUMN IF NOT EXISTS archived_at timestamptz;

UPDATE public.services
SET archived_at = COALESCE(archived_at, updated_at, created_at)
WHERE is_archived = true;

UPDATE public.products
SET archived_at = COALESCE(archived_at, updated_at, created_at)
WHERE is_archived = true;

CREATE OR REPLACE FUNCTION public.set_catalog_archive_timestamp()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = public
AS $$
BEGIN
  IF NEW.is_archived = true AND OLD.is_archived = false THEN
    NEW.archived_at = now();
```

```
  ELSIF NEW.is_archived = false AND OLD.is_archived = true THEN
    NEW.archived_at = NULL;
  END IF;
  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS services_archive_timestamp ON public.services;
CREATE TRIGGER services_archive_timestamp
  BEFORE UPDATE ON public.services
  FOR EACH ROW EXECUTE FUNCTION
public.set_catalog_archive_timestamp();

DROP TRIGGER IF EXISTS products_archive_timestamp ON public.products;
CREATE TRIGGER products_archive_timestamp
  BEFORE UPDATE ON public.products
  FOR EACH ROW EXECUTE FUNCTION
public.set_catalog_archive_timestamp();

REVOKE EXECUTE ON FUNCTION public.set_catalog_archive_timestamp()
FROM PUBLIC;
```

supabase/migrations/20260913130000_add_booking_motorcycle_configuration.sql

---

```
-- Preserve the motorcycle attributes used to resolve service price and duration.
ALTER TABLE public.appointments
  ADD COLUMN IF NOT EXISTS moto_cc integer,
  ADD COLUMN IF NOT EXISTS moto_fuel_type text,
  ADD COLUMN IF NOT EXISTS moto_transmission text;

-- Replace the legacy signatures so PostgREST has one unambiguous RPC target.
DROP FUNCTION IF EXISTS public.create_booking_atomic(
  text, uuid, text, text, text, text, text, text, integer, text, date, time,
  text, numeric, integer, uuid, jsonb, text, text, text, text, text
);
DROP FUNCTION IF EXISTS public.create_booking_atomic(
  text, uuid, text, text, text, text, text, text, integer, text, date, time,
  text, numeric, integer, uuid, jsonb, text, text
);

CREATE FUNCTION public.create_booking_atomic(
  p_reference_code text,
  p_booking_request_id uuid,
  p_customer_name text,
  p_phone text,
  p_email text,
  p_moto_brand text,
  p_moto_model text,
  p_moto_cc integer,
  p_moto_fuel_type text,
  p_moto_transmission text,
  p_moto_variant text,
  p_moto_year integer,
  p_plate_number text,
  p_appointment_date date,
  p_start_time time,
  p_notes text,
  p_total_estimate numeric,
  p_booking_duration_minutes integer,
  p_assigned_crew_id uuid,
  p_services jsonb,
  p_notification_title text,
  p_notification_message text,
  p_first_name text,
  p_middle_name text,
```

```
  p_last_name text
)
RETURNS TABLE (appointment_id uuid, reference_code text)
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment_id uuid;
  v_reference_code text;
BEGIN
  IF p_booking_request_id IS NULL THEN
    RAISE EXCEPTION 'A booking request ID is required.';
  END IF;

  IF COALESCE(btrim(p_first_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$'
     OR COALESCE(btrim(p_middle_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$'
     OR COALESCE(btrim(p_last_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$' THEN
    RAISE EXCEPTION 'First name, middle name, and last name must contain
letters only.';
  END IF;

  IF p_moto_cc IS NULL OR p_moto_cc < 50 OR p_moto_cc > 2000
     OR p_moto_fuel_type NOT IN ('FI', 'Carburetor')
     OR p_moto_transmission NOT IN ('Manual', 'Semi-Automatic', 'Automatic')
THEN
    RAISE EXCEPTION 'A valid motorcycle configuration is required.';
  END IF;

  IF jsonb_typeof(p_services) <> 'array' OR jsonb_array_length(p_services) = 0
THEN
    RAISE EXCEPTION 'A booking must include at least one service.';
  END IF;

  INSERT INTO public.appointments (
    reference_code,
    booking_request_id,
    customer_name,
    first_name,
    middle_name,
    last_name,
    phone,
    email,
    moto_brand,
    moto_model,
```

```
    moto_cc,
    moto_fuel_type,
    moto_transmission,
    moto_variant,
    moto_year,
    plate_number,
    appointment_date,
    start_time,
    notes,
    total_estimate,
    booking_duration_minutes,
    terms_accepted,
    assigned_crew_id
  )
  VALUES (
    p_reference_code,
    p_booking_request_id,
    concat_ws(' ', btrim(p_first_name), btrim(p_middle_name),
btrim(p_last_name)),
    btrim(p_first_name),
    btrim(p_middle_name),
    btrim(p_last_name),
    p_phone,
    p_email,
    p_moto_brand,
    p_moto_model,
    p_moto_cc,
    p_moto_fuel_type,
    p_moto_transmission,
    p_moto_variant,
    p_moto_year,
    p_plate_number,
    p_appointment_date,
    p_start_time,
    p_notes,
    p_total_estimate,
    p_booking_duration_minutes,
    true,
    p_assigned_crew_id
  )
  ON CONFLICT DO NOTHING
  RETURNING appointments.id, appointments.reference_code INTO
v_appointment_id, v_reference_code;

  IF v_appointment_id IS NULL THEN
```

```
    SELECT appointments.id, appointments.reference_code
      INTO v_appointment_id, v_reference_code
    FROM public.appointments AS appointments
    WHERE appointments.booking_request_id = p_booking_request_id;

    IF FOUND THEN
      RETURN QUERY SELECT v_appointment_id, v_reference_code;
      RETURN;
    END IF;

    RAISE EXCEPTION 'The requested schedule is no longer available.' USING
ERRCODE = '23P01';
  END IF;

  INSERT INTO public.appointment_services (
    appointment_id,
    service_id,
    service_name,
    price,
    duration_minutes
  )
  SELECT
    v_appointment_id,
    service.service_id,
    service.service_name,
    service.price,
    service.duration_minutes
  FROM jsonb_to_recordset(p_services) AS service(
    service_id uuid,
    service_name text,
    price numeric,
    duration_minutes integer
  );

  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES ('new_appointment', p_notification_title, p_notification_message,
v_appointment_id);

  RETURN QUERY SELECT v_appointment_id, v_reference_code;
END;
$$;

REVOKE ALL ON FUNCTION public.create_booking_atomic(
  text, uuid, text, text, text, text, text, integer, text, text, text, integer,
  text, date, time, text, numeric, integer, uuid, jsonb, text, text, text, text, text
```

```
) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.create_booking_atomic(
  text, uuid, text, text, text, text, text, integer, text, text, text, integer,
  text, date, time, text, numeric, integer, uuid, jsonb, text, text, text, text, text
) TO service_role;
```

supabase/migrations/20260913140000_seed_complete_service_configuration_matrix.sql

---

```
-- Every service receives a predictable rate and estimated duration for each
-- supported engine-size, fuel, and transmission combination. The matrix keeps
-- the service's base price/duration as the 110cc FI automatic baseline, then
-- scales larger engines and the extra labour of carbureted/manual systems.

CREATE UNIQUE INDEX IF NOT EXISTS
service_configurations_service_rule_key
  ON public.service_configurations (service_id, cc_category, fuel_type,
transmission);

WITH cc_bands (cc_category, price_factor, duration_factor) AS (
  VALUES
    ('110', 1.00::numeric, 1.00::numeric),
    ('115', 1.00::numeric, 1.00::numeric),
    ('125', 1.05::numeric, 1.00::numeric),
    ('150', 1.15::numeric, 1.10::numeric),
    ('160', 1.20::numeric, 1.10::numeric),
    ('250', 1.35::numeric, 1.20::numeric),
    ('400', 1.55::numeric, 1.30::numeric),
    ('500', 1.70::numeric, 1.40::numeric),
    ('600', 1.85::numeric, 1.50::numeric),
    ('650', 1.90::numeric, 1.55::numeric),
    ('700', 2.00::numeric, 1.65::numeric),
    ('900', 2.20::numeric, 1.80::numeric),
    ('1000', 2.40::numeric, 2.00::numeric)
),
fuel_types (fuel_type, price_factor, duration_factor) AS (
  VALUES
    ('FI', 1.00::numeric, 1.00::numeric),
    ('Carbureted', 1.10::numeric, 1.10::numeric)
),
transmissions (transmission, price_factor, duration_factor) AS (
  VALUES
    ('Automatic', 1.00::numeric, 1.00::numeric),
    ('Semi-Auto', 1.02::numeric, 1.00::numeric),
    ('Manual', 1.05::numeric, 1.05::numeric)
)
INSERT INTO public.service_configurations (
  service_id,
  cc_category,
  fuel_type,
  transmission,
```

```
  duration_minutes,
  price
)
SELECT
  service.id,
  cc_bands.cc_category,
  fuel_types.fuel_type,
  transmissions.transmission,
  LEAST(
    480,
    GREATEST(
      15,
      CEIL(
        (
          COALESCE(service.duration_minutes, 60)
          * cc_bands.duration_factor
          * fuel_types.duration_factor
          * transmissions.duration_factor
        ) / 15.0
      ) * 15
    )
  )::integer,
  GREATEST(
    50::numeric,
    ROUND(
      (
        service.price
        * cc_bands.price_factor
        * fuel_types.price_factor
        * transmissions.price_factor
      ) / 50.0
    ) * 50
  )::numeric(10, 2)
FROM public.services AS service
CROSS JOIN cc_bands
CROSS JOIN fuel_types
CROSS JOIN transmissions
WHERE service.is_active = true
  AND service.is_archived = false
ON CONFLICT (service_id, cc_category, fuel_type, transmission) DO UPDATE
SET
  duration_minutes = EXCLUDED.duration_minutes,
  price = EXCLUDED.price,
  updated_at = now();
```

supabase/migrations/20260913150000_simplify_service_configurations_to_cc_categories.sql

---

```
-- Replace granular engine-size rules with the booking categories used by the
-- customer flow: Small (50–150cc), Mid (151–499cc), and Big (500cc+).
-- Mid includes 401–499cc to ensure every valid catalog motorcycle has a rule.

DELETE FROM public.service_configurations;

WITH cc_categories (cc_category, price, duration_factor) AS (
  VALUES
    ('Small', 50.00::numeric, 1.00::numeric),
    ('Mid', 100.00::numeric, 1.25::numeric),
    ('Big', 150.00::numeric, 1.50::numeric)
),
fuel_types (fuel_type, extra_minutes) AS (
  VALUES
    ('FI', 0),
    ('Carbureted', 15)
),
transmissions (transmission, extra_minutes) AS (
  VALUES
    ('Automatic', 0),
    ('Semi-Auto', 0),
    ('Manual', 15)
)
INSERT INTO public.service_configurations (
  service_id,
  cc_category,
  fuel_type,
  transmission,
  duration_minutes,
  price
)
SELECT
  service.id,
  cc_categories.cc_category,
  fuel_types.fuel_type,
  transmissions.transmission,
  LEAST(
    480,
    GREATEST(
      15,
      CEIL(
        (
```

```
        COALESCE(service.duration_minutes, 60) * cc_categories.duration_factor
        + fuel_types.extra_minutes
        + transmissions.extra_minutes
      ) / 15.0
    ) * 15
  )
 )::integer,
 cc_categories.price
FROM public.services AS service
CROSS JOIN cc_categories
CROSS JOIN fuel_types
CROSS JOIN transmissions
WHERE service.is_active = true
  AND service.is_archived = false;
```

supabase/migrations/20260913160000_remove_legacy_service_configurations.sql

---

```
-- Service pricing is now represented by services.price and optional
-- service_model_overrides. Remove the obsolete CC/fuel/transmission table.
DROP TABLE IF EXISTS public.service_configurations CASCADE;
```

supabase/migrations/20260913170000_add_price_history_actor_email.sql

---

```
-- Keep a readable administrator identifier with each price change. The UUID
-- remains available for audit joins; the email is captured at write time so
-- history remains useful without exposing auth.users to the client.
ALTER TABLE public.price_history
  ADD COLUMN IF NOT EXISTS changed_by_email text;

CREATE OR REPLACE FUNCTION public.set_price_history_actor()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
BEGIN
  NEW.changed_by = COALESCE(auth.uid(), NEW.changed_by);
  NEW.changed_by_email = COALESCE(auth.jwt() ->> 'email',
NEW.changed_by_email);
  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS price_history_actor ON public.price_history;
CREATE TRIGGER price_history_actor
  BEFORE INSERT ON public.price_history
  FOR EACH ROW EXECUTE FUNCTION public.set_price_history_actor();

REVOKE EXECUTE ON FUNCTION public.set_price_history_actor() FROM
PUBLIC;
```

supabase/migrations/20260914100000_reschedule_approval_creates_linked_booking.sql

---

```
-- A customer-confirmed reschedule reserves a fresh replacement reference
code.
-- Admin approval then uses that same code to create the linked booking while
-- preserving the original booking as audit history.

ALTER TABLE public.appointments
  ADD COLUMN IF NOT EXISTS pending_reschedule_reference_code text;

CREATE UNIQUE INDEX IF NOT EXISTS
appointments_pending_reschedule_reference_code_unique
  ON public.appointments (pending_reschedule_reference_code)
  WHERE pending_reschedule_reference_code IS NOT NULL;

CREATE OR REPLACE FUNCTION public.submit_reschedule_request(
  p_appointment_id uuid,
  p_request_id uuid,
  p_appointment_date date,
  p_start_time time,
  p_reason text
)
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment public.appointments%ROWTYPE;
  v_reference_code text;
  v_reference_available boolean := false;
  v_attempt integer;
  v_character integer;
  v_reference_alphabet constant text :=
'ABCDEFGHJKLMNPQRSTUVWXYZ23456789';
BEGIN
  IF p_request_id IS NULL OR p_appointment_date IS NULL OR p_start_time IS
NULL
     OR NULLIF(btrim(p_reason), '') IS NULL THEN
    RAISE EXCEPTION 'A date, time, and rescheduling reason are required.';
  END IF;

  SELECT appointment.*
    INTO v_appointment
```

```
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
  FOR UPDATE;

  IF NOT FOUND THEN
    RAISE EXCEPTION 'This appointment is no longer available.';
  END IF;
  IF v_appointment.pending_reschedule_request_id IS NOT NULL
     AND v_appointment.pending_reschedule_request_id <> p_request_id THEN
    RAISE EXCEPTION 'A reschedule request is already awaiting review.';
  END IF;
  IF v_appointment.reschedule_count >= 3 THEN
    RAISE EXCEPTION 'This appointment has reached the maximum of 3
reschedules.';
  END IF;
  IF v_appointment.status NOT IN ('pending', 'confirmed')
     OR v_appointment.rescheduled_to_appointment_id IS NOT NULL THEN
    RAISE EXCEPTION 'This appointment can no longer be rescheduled.';
  END IF;

  -- Preserve an existing reserved reference on a retry. Older pending requests
  -- without one receive a reservation the first time they are retried.
  v_reference_code := v_appointment.pending_reschedule_reference_code;
  IF v_reference_code IS NULL THEN
    FOR v_attempt IN 1..10 LOOP
      v_reference_code := 'FRM-';
      FOR v_character IN 1..6 LOOP
        v_reference_code := v_reference_code || substr(
          v_reference_alphabet,
          1 + floor(random() * length(v_reference_alphabet))::integer,
          1
        );
      END LOOP;

      SELECT NOT EXISTS (
        SELECT 1
        FROM public.appointments
        WHERE reference_code = v_reference_code
           OR pending_reschedule_reference_code = v_reference_code
      ) INTO v_reference_available;
      EXIT WHEN v_reference_available;
    END LOOP;

    IF NOT v_reference_available THEN
```

```
      RAISE EXCEPTION 'Could not generate a unique booking reference. Please
try again.';
    END IF;
  END IF;

  UPDATE public.appointments
  SET pending_reschedule_request_id = p_request_id,
      pending_reschedule_date = p_appointment_date,
      pending_reschedule_start_time = p_start_time,
      pending_reschedule_reference_code = v_reference_code,
      pending_reschedule_reason = NULLIF(btrim(p_reason), '')
  WHERE id = v_appointment.id;

  IF v_appointment.pending_reschedule_request_id IS NULL THEN
    INSERT INTO public.notifications (type, title, message, appointment_id)
    VALUES (
      'reschedule_request',
      'Reschedule request ' || v_appointment.reference_code,
      v_appointment.customer_name || ' requested ' ||
to_char(p_appointment_date, 'Mon FMDD, YYYY')
        || ' at ' || to_char(p_start_time, 'HH12:MI AM') || '. New reference: '
        || v_reference_code || '. Reason: ' || btrim(p_reason),
      v_appointment.id
    );
  END IF;
END;
$$;

CREATE OR REPLACE FUNCTION public.track_appointment_reschedule()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = public
AS $$
BEGIN
  IF NEW.appointment_date IS DISTINCT FROM OLD.appointment_date
     OR NEW.start_time IS DISTINCT FROM OLD.start_time THEN
    IF OLD.reschedule_count >= 3 THEN
      RAISE EXCEPTION 'This appointment has reached the maximum of 3
reschedules.';
    END IF;

    NEW.reschedule_count := OLD.reschedule_count + 1;
    NEW.last_reschedule_rejected_at := NULL;
    NEW.last_reschedule_rejection_message := NULL;
  ELSIF OLD.pending_reschedule_request_id IS NULL
```

```
    AND NEW.pending_reschedule_request_id IS NOT NULL THEN
   -- A fresh request replaces an earlier rejection notice.
   NEW.last_reschedule_rejected_at := NULL;
   NEW.last_reschedule_rejection_message := NULL;
  ELSIF OLD.pending_reschedule_request_id IS NOT NULL
    AND NEW.pending_reschedule_request_id IS NULL THEN
   IF NEW.status = 'rescheduled' AND NEW.rescheduled_to_appointment_id IS
NOT NULL THEN
     -- Approval clears the pending fields while linking a new appointment; it
     -- must not be recorded as a rejected request.
     NEW.last_reschedule_rejected_at := NULL;
     NEW.last_reschedule_rejection_message := NULL;
   ELSE
     -- Rejecting a request leaves the original appointment unchanged.
     NEW.last_reschedule_rejected_at := now();
     NEW.last_reschedule_rejection_message :=
       'Reschedule Request Rejected. Your requested reschedule was not
approved. Your original appointment remains unchanged and reserved.';
   END IF;
  END IF;

  RETURN NEW;
END;
$$;

CREATE OR REPLACE FUNCTION public.review_reschedule_request(
  p_appointment_id uuid,
  p_decision text
)
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment public.appointments%ROWTYPE;
  v_assigned_crew_id uuid;
  v_duration_minutes integer;
  v_new_appointment_id uuid;
  v_new_reference_code text;
  v_reason text;
  v_original_shop_notes text;
  v_new_shop_notes text;
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
```

```
    RAISE EXCEPTION 'Only administrators can review reschedule requests.';
  END IF;
  IF lower(btrim(COALESCE(p_decision, ''))) NOT IN ('confirmed', 'rejected')
THEN
    RAISE EXCEPTION 'Choose either confirmed or rejected.';
  END IF;

  -- Lock the original request so it can produce at most one replacement.
  SELECT appointment.*
    INTO v_appointment
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
  FOR UPDATE;

  IF NOT FOUND OR v_appointment.pending_reschedule_request_id IS NULL
THEN
    RAISE EXCEPTION 'This reschedule request is no longer available.';
  END IF;

  IF lower(btrim(p_decision)) = 'rejected' THEN
    UPDATE public.appointments
    SET pending_reschedule_request_id = NULL,
        pending_reschedule_date = NULL,
        pending_reschedule_start_time = NULL,
        pending_reschedule_reference_code = NULL,
        pending_reschedule_reason = NULL
    WHERE id = v_appointment.id;
    RETURN;
  END IF;

  IF v_appointment.reschedule_count >= 3 THEN
    RAISE EXCEPTION 'This appointment has reached the maximum of 3
reschedules.';
  END IF;
  IF v_appointment.pending_reschedule_date IS NULL
     OR v_appointment.pending_reschedule_start_time IS NULL
     OR NULLIF(btrim(v_appointment.pending_reschedule_reason), '') IS NULL
THEN
    RAISE EXCEPTION 'The reschedule request is missing its requested
schedule or reason.';
  END IF;

  v_duration_minutes := COALESCE(v_appointment.booking_duration_minutes,
75);
```

```
  v_reason := btrim(v_appointment.pending_reschedule_reason);

  IF EXISTS (
    SELECT 1
    FROM public.schedule_blocks AS block
    WHERE block.block_date = v_appointment.pending_reschedule_date
      AND block.is_active = true
      AND (
        block.start_time IS NULL
        OR (
          block.start_time IS NOT NULL
          AND (
            (COALESCE(block.reason, '') NOT LIKE 'RANGE:%'
              AND block.start_time = v_appointment.pending_reschedule_start_time)
            OR (COALESCE(block.reason, '') LIKE 'RANGE:%'
              AND tsrange(
                v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
                v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
                  + make_interval(mins => v_duration_minutes),
                '[)'
              ) && tsrange(
                v_appointment.pending_reschedule_date + block.start_time,
                v_appointment.pending_reschedule_date
                  + substring(block.reason FROM '^RANGE:([0-9]{2}:[0-9]{2})')::time,
                '[)'
              ))
          )
        )
      )
  ) THEN
    RAISE EXCEPTION 'The requested time is no longer available because the
shop schedule is blocked.';
  END IF;

  -- Re-check capacity, crew schedules, and crew availability immediately before
  -- creating the replacement booking.
  SELECT crew.id
    INTO v_assigned_crew_id
  FROM public.crew_members AS crew
  WHERE crew.is_active = true
    AND crew.is_archived = false
    AND EXISTS (
      SELECT 1
```

```
      FROM public.time_slots AS slot
      WHERE slot.is_active = true
        AND slot.start_time = v_appointment.pending_reschedule_start_time
        AND slot.capacity > (
          SELECT count(*)
          FROM public.appointments AS other
          WHERE other.id <> v_appointment.id
            AND other.appointment_date = v_appointment.pending_reschedule_date
            AND other.is_archived = false
            AND other.rescheduled_to_appointment_id IS NULL
            AND other.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
            AND tsrange(
              other.appointment_date + other.start_time,
              other.appointment_date + other.start_time
                + make_interval(mins =>
COALESCE(other.booking_duration_minutes, 75)),
              '[)'
            ) && tsrange(
              v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
              v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
                + make_interval(mins => v_duration_minutes),
              '[)'
            )
        )
    )
    AND EXISTS (
      SELECT 1
      FROM public.crew_schedules AS schedule
      WHERE schedule.crew_id = crew.id
        AND schedule.schedule_date = v_appointment.pending_reschedule_date
        AND schedule.is_working = true
        AND schedule.start_time <= v_appointment.pending_reschedule_start_time
        AND schedule.end_time >= v_appointment.pending_reschedule_start_time
          + make_interval(mins => v_duration_minutes)
    )
    AND NOT EXISTS (
      SELECT 1
      FROM public.crew_availability_exceptions AS exception
      WHERE exception.crew_id = crew.id
        AND exception.start_date <= v_appointment.pending_reschedule_date
        AND exception.end_date >= v_appointment.pending_reschedule_date
        AND (exception.is_all_day OR (
          exception.start_time IS NOT NULL AND exception.end_time IS NOT NULL
```

```
        AND tsrange(
          v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
          v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
            + make_interval(mins => v_duration_minutes),
          '[)'
        ) && tsrange(
          v_appointment.pending_reschedule_date + exception.start_time,
          v_appointment.pending_reschedule_date + exception.end_time,
          '[)'
        )
      ))
  )
  AND NOT EXISTS (
    SELECT 1
    FROM public.appointments AS other
    WHERE other.id <> v_appointment.id
      AND other.assigned_crew_id = crew.id
      AND other.appointment_date = v_appointment.pending_reschedule_date
      AND other.is_archived = false
      AND other.rescheduled_to_appointment_id IS NULL
      AND other.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
      AND tsrange(
        other.appointment_date + other.start_time,
        other.appointment_date + other.start_time
          + make_interval(mins => COALESCE(other.booking_duration_minutes,
75)),
        '[)'
      ) && tsrange(
        v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
        v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
          + make_interval(mins => v_duration_minutes),
        '[)'
      )
  )
  ORDER BY (crew.id = v_appointment.assigned_crew_id) DESC, crew.name,
crew.id
  LIMIT 1;

  IF v_assigned_crew_id IS NULL THEN
    RAISE EXCEPTION 'No crew member is available for the requested
appointment period.';
```

```
  END IF;

  -- The customer receives this reserved code immediately after confirming the
  -- request. Approval must create the replacement with the same reference.
  v_new_reference_code := v_appointment.pending_reschedule_reference_code;
  IF v_new_reference_code IS NULL
     OR v_new_reference_code !~ '^FRM-[A-Z0-9]{6}$' THEN
    RAISE EXCEPTION 'The reschedule request is missing its replacement
reference.';
  END IF;

  v_original_shop_notes := concat_ws(
    E'\n',
    NULLIF(btrim(v_appointment.admin_notes), ''),
    'Rescheduled to ' || v_new_reference_code,
    'Reason for rescheduling: ' || v_reason
  );
  v_new_shop_notes := concat_ws(
    E'\n',
    'Rescheduled from ' || v_appointment.reference_code,
    'Reason for rescheduling: ' || v_reason
  );

  INSERT INTO public.appointments AS replacement (
    reference_code,
    booking_request_id,
    customer_name,
    first_name,
    middle_name,
    last_name,
    phone,
    email,
    moto_brand,
    moto_model,
    moto_cc,
    moto_fuel_type,
    moto_transmission,
    moto_variant,
    moto_year,
    plate_number,
    appointment_date,
    start_time,
    status,
    notes,
    admin_notes,
```

```
  total_estimate,
  booking_duration_minutes,
  terms_accepted,
  assigned_crew_id,
  reschedule_count,
  rescheduled_from_appointment_id
)
VALUES (
  v_new_reference_code,
  gen_random_uuid(),
  v_appointment.customer_name,
  v_appointment.first_name,
  v_appointment.middle_name,
  v_appointment.last_name,
  v_appointment.phone,
  v_appointment.email,
  v_appointment.moto_brand,
  v_appointment.moto_model,
  v_appointment.moto_cc,
  v_appointment.moto_fuel_type,
  v_appointment.moto_transmission,
  v_appointment.moto_variant,
  v_appointment.moto_year,
  v_appointment.plate_number,
  v_appointment.pending_reschedule_date,
  v_appointment.pending_reschedule_start_time,
  'confirmed',
  v_appointment.notes,
  v_new_shop_notes,
  v_appointment.total_estimate,
  v_duration_minutes,
  v_appointment.terms_accepted,
  v_assigned_crew_id,
  v_appointment.reschedule_count + 1,
  v_appointment.id
)
RETURNING replacement.id INTO v_new_appointment_id;

INSERT INTO public.appointment_services (
  appointment_id,
  service_id,
  service_name,
  price,
  duration_minutes
)
```

```
  SELECT
    v_new_appointment_id,
    service.service_id,
    service.service_name,
    service.price,
    service.duration_minutes
  FROM public.appointment_services AS service
  WHERE service.appointment_id = v_appointment.id;

  UPDATE public.appointments
  SET status = 'rescheduled',
      admin_notes = v_original_shop_notes,
      reschedule_count = v_appointment.reschedule_count + 1,
      rescheduled_to_appointment_id = v_new_appointment_id,
      pending_reschedule_request_id = NULL,
      pending_reschedule_date = NULL,
      pending_reschedule_start_time = NULL,
      pending_reschedule_reference_code = NULL,
      pending_reschedule_reason = NULL
  WHERE id = v_appointment.id;

  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES (
    'rescheduled_appointment',
    'Rescheduled booking ' || v_new_reference_code,
    v_appointment.customer_name || ' rescheduled from ' ||
v_appointment.reference_code
      || ' to ' || v_new_reference_code || '. Reason: ' || v_reason,
    v_new_appointment_id
  );
END;
$$;

REVOKE ALL ON FUNCTION public.review_reschedule_request(uuid, text)
FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.review_reschedule_request(uuid, text)
TO authenticated;
```

supabase/migrations/20260915000000_add_reschedule_history_tracking.sql

---

```
-- A dedicated audit trail for each reschedule decision (confirmed or rejected).
-- The admin console reads this table to show the date, time, and reason of every
-- reschedule attempt on an appointment, in chronological order.

CREATE TABLE IF NOT EXISTS public.appointment_reschedule_history (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  appointment_id uuid NOT NULL REFERENCES public.appointments(id) ON
DELETE CASCADE,
  new_appointment_id uuid REFERENCES public.appointments(id) ON DELETE
SET NULL,
  reschedule_number integer NOT NULL CHECK (reschedule_number
BETWEEN 1 AND 3),
  from_date date NOT NULL,
  from_start_time time NOT NULL,
  to_date date,
  to_start_time time,
  reason text NOT NULL,
  decision text NOT NULL CHECK (decision IN ('confirmed', 'rejected')),
  approved_at timestamptz NOT NULL DEFAULT now(),
  approved_by uuid REFERENCES auth.users(id) ON DELETE SET NULL
);

CREATE INDEX IF NOT EXISTS
appointment_reschedule_history_appointment_id_idx
  ON public.appointment_reschedule_history (appointment_id);

CREATE INDEX IF NOT EXISTS
appointment_reschedule_history_new_appointment_id_idx
  ON public.appointment_reschedule_history (new_appointment_id);

CREATE INDEX IF NOT EXISTS
appointment_reschedule_history_reschedule_number_idx
  ON public.appointment_reschedule_history (appointment_id,
reschedule_number);

ALTER TABLE public.appointment_reschedule_history ENABLE ROW LEVEL
SECURITY;

DROP POLICY IF EXISTS "reschedule history admin all" ON
public.appointment_reschedule_history;
CREATE POLICY "reschedule history admin all" ON
public.appointment_reschedule_history
```

```
  FOR ALL TO authenticated
  USING (public.has_role(auth.uid(), 'admin'))
  WITH CHECK (public.has_role(auth.uid(), 'admin'));

GRANT SELECT ON public.appointment_reschedule_history TO authenticated;
GRANT ALL ON public.appointment_reschedule_history TO service_role;

-- Re-surface the review function so every decision is audited in the history table.
-- The function body is reproduced from 20260914100000 with two INSERT
statements added:
-- one in the reject branch and one at the end of the confirm branch.
CREATE OR REPLACE FUNCTION public.review_reschedule_request(
  p_appointment_id uuid,
  p_decision text
)
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment public.appointments%ROWTYPE;
  v_assigned_crew_id uuid;
  v_duration_minutes integer;
  v_new_appointment_id uuid;
  v_new_reference_code text;
  v_reason text;
  v_original_shop_notes text;
  v_new_shop_notes text;
  v_reschedule_number integer;
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Only administrators can review reschedule requests.';
  END IF;
  IF lower(btrim(COALESCE(p_decision, ''))) NOT IN ('confirmed', 'rejected')
THEN
    RAISE EXCEPTION 'Choose either confirmed or rejected.';
  END IF;

  -- Lock the original request so it can produce at most one replacement.
  SELECT appointment.*
    INTO v_appointment
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
```

```
  FOR UPDATE;

  IF NOT FOUND OR v_appointment.pending_reschedule_request_id IS NULL
THEN
    RAISE EXCEPTION 'This reschedule request is no longer available.';
  END IF;

  v_reschedule_number := v_appointment.reschedule_count + 1;

  IF lower(btrim(p_decision)) = 'rejected' THEN
    INSERT INTO public.appointment_reschedule_history (
      appointment_id,
      new_appointment_id,
      reschedule_number,
      from_date,
      from_start_time,
      to_date,
      to_start_time,
      reason,
      decision,
      approved_by
    ) VALUES (
      v_appointment.id,
      NULL,
      v_reschedule_number,
      v_appointment.appointment_date,
      v_appointment.start_time,
      v_appointment.pending_reschedule_date,
      v_appointment.pending_reschedule_start_time,
      btrim(v_appointment.pending_reschedule_reason),
      'rejected',
      auth.uid()
    );

    UPDATE public.appointments
    SET pending_reschedule_request_id = NULL,
        pending_reschedule_date = NULL,
        pending_reschedule_start_time = NULL,
        pending_reschedule_reference_code = NULL,
        pending_reschedule_reason = NULL
    WHERE id = v_appointment.id;
    RETURN;
  END IF;

  IF v_appointment.reschedule_count >= 3 THEN
```

```
    RAISE EXCEPTION 'This appointment has reached the maximum of 3
reschedules.';
  END IF;
  IF v_appointment.pending_reschedule_date IS NULL
     OR v_appointment.pending_reschedule_start_time IS NULL
     OR NULLIF(btrim(v_appointment.pending_reschedule_reason), '') IS NULL
THEN
    RAISE EXCEPTION 'The reschedule request is missing its requested
schedule or reason.';
  END IF;

  v_duration_minutes := COALESCE(v_appointment.booking_duration_minutes,
75);
  v_reason := btrim(v_appointment.pending_reschedule_reason);

  IF EXISTS (
    SELECT 1
    FROM public.schedule_blocks AS block
    WHERE block.block_date = v_appointment.pending_reschedule_date
      AND block.is_active = true
      AND (
        block.start_time IS NULL
        OR (
          block.start_time IS NOT NULL
          AND (
            (COALESCE(block.reason, '') NOT LIKE 'RANGE:%'
             AND block.start_time = v_appointment.pending_reschedule_start_time)
            OR (COALESCE(block.reason, '') LIKE 'RANGE:%'
             AND tsrange(
               v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
               v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
                 + make_interval(mins => v_duration_minutes),
               '[)'
             ) && tsrange(
               v_appointment.pending_reschedule_date + block.start_time,
               v_appointment.pending_reschedule_date
                 + substring(block.reason FROM '^RANGE:([0-9]{2}:[0-9]{2})')::time,
               '[)'
             ))
          )
        )
      )
  ) THEN
```

```
    RAISE EXCEPTION 'The requested time is no longer available because the
shop schedule is blocked.';
  END IF;

  -- Re-check capacity, crew schedules, and crew availability immediately before
  -- creating the replacement booking.
  SELECT crew.id
    INTO v_assigned_crew_id
  FROM public.crew_members AS crew
  WHERE crew.is_active = true
    AND crew.is_archived = false
    AND EXISTS (
      SELECT 1
      FROM public.time_slots AS slot
      WHERE slot.is_active = true
        AND slot.start_time = v_appointment.pending_reschedule_start_time
        AND slot.capacity > (
          SELECT count(*)
          FROM public.appointments AS other
          WHERE other.id <> v_appointment.id
            AND other.appointment_date = v_appointment.pending_reschedule_date
            AND other.is_archived = false
            AND other.rescheduled_to_appointment_id IS NULL
            AND other.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
          AND tsrange(
            other.appointment_date + other.start_time,
            other.appointment_date + other.start_time
              + make_interval(mins => COALESCE(other.booking_duration_minutes,
75)), '[)'
        ) && tsrange(
          v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
          v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
            + make_interval(mins => v_duration_minutes), '[)'
        )
      )
    )
    AND EXISTS (
      SELECT 1
      FROM public.crew_schedules AS schedule
      WHERE schedule.crew_id = crew.id
        AND schedule.schedule_date = v_appointment.pending_reschedule_date
        AND schedule.is_working = true
        AND schedule.start_time <= v_appointment.pending_reschedule_start_time
```

```
      AND schedule.end_time >= v_appointment.pending_reschedule_start_time
        + make_interval(mins => v_duration_minutes)
    )
    AND NOT EXISTS (
      SELECT 1
      FROM public.crew_availability_exceptions AS exception
      WHERE exception.crew_id = crew.id
        AND exception.start_date <= v_appointment.pending_reschedule_date
        AND exception.end_date >= v_appointment.pending_reschedule_date
        AND (exception.is_all_day OR (
          exception.start_time IS NOT NULL AND exception.end_time IS NOT NULL
          AND tsrange(
            v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
            v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
              + make_interval(mins => v_duration_minutes), '[)'
          ) && tsrange(
            v_appointment.pending_reschedule_date + exception.start_time,
            v_appointment.pending_reschedule_date + exception.end_time, '[)'
          )
        ))
    )
    AND NOT EXISTS (
      SELECT 1
      FROM public.appointments AS other
      WHERE other.id <> v_appointment.id
        AND other.assigned_crew_id = crew.id
        AND other.appointment_date = v_appointment.pending_reschedule_date
        AND other.is_archived = false
        AND other.rescheduled_to_appointment_id IS NULL
        AND other.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
        AND tsrange(
          other.appointment_date + other.start_time,
          other.appointment_date + other.start_time
            + make_interval(mins => COALESCE(other.booking_duration_minutes,
75)), '[)'
        ) && tsrange(
          v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
          v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
            + make_interval(mins => v_duration_minutes), '[)'
        )
    )
```

```
  ORDER BY (crew.id = v_appointment.assigned_crew_id) DESC, crew.name,
crew.id
  LIMIT 1;

  IF v_assigned_crew_id IS NULL THEN
    RAISE EXCEPTION 'No crew member is available for the requested
appointment period.';
  END IF;

  -- The customer receives this reserved code immediately after confirming the
  -- request. Approval must create the replacement with the same reference.
  v_new_reference_code := v_appointment.pending_reschedule_reference_code;
  IF v_new_reference_code IS NULL
     OR v_new_reference_code !~ '^FRM-[A-Z0-9]{6}$' THEN
    RAISE EXCEPTION 'The reschedule request is missing its replacement
reference.';
  END IF;

  v_original_shop_notes := concat_ws(
    E'\n',
    NULLIF(btrim(v_appointment.admin_notes), ''),
    'Rescheduled to ' || v_new_reference_code,
    'Reason for rescheduling: ' || v_reason
  );
  v_new_shop_notes := concat_ws(
    E'\n',
    'Rescheduled from ' || v_appointment.reference_code,
    'Reason for rescheduling: ' || v_reason
  );

  INSERT INTO public.appointments AS replacement (
    reference_code,
    booking_request_id,
    customer_name,
    first_name,
    middle_name,
    last_name,
    phone,
    email,
    moto_brand,
    moto_model,
    moto_cc,
    moto_fuel_type,
    moto_transmission,
    moto_variant,
```

```
  moto_year,
  plate_number,
  appointment_date,
  start_time,
  status,
  notes,
  admin_notes,
  total_estimate,
  booking_duration_minutes,
  terms_accepted,
  assigned_crew_id,
  reschedule_count,
  rescheduled_from_appointment_id
)
VALUES (
  v_new_reference_code,
  gen_random_uuid(),
  v_appointment.customer_name,
  v_appointment.first_name,
  v_appointment.middle_name,
  v_appointment.last_name,
  v_appointment.phone,
  v_appointment.email,
  v_appointment.moto_brand,
  v_appointment.moto_model,
  v_appointment.moto_cc,
  v_appointment.moto_fuel_type,
  v_appointment.moto_transmission,
  v_appointment.moto_variant,
  v_appointment.moto_year,
  v_appointment.plate_number,
  v_appointment.pending_reschedule_date,
  v_appointment.pending_reschedule_start_time,
  'confirmed',
  v_appointment.notes,
  v_new_shop_notes,
  v_appointment.total_estimate,
  v_duration_minutes,
  v_appointment.terms_accepted,
  v_assigned_crew_id,
  v_appointment.reschedule_count + 1,
  v_appointment.id
)
RETURNING replacement.id INTO v_new_appointment_id;
```

```
  INSERT INTO public.appointment_services (
    appointment_id,
    service_id,
    service_name,
    price,
    duration_minutes
  )
  SELECT
    v_new_appointment_id,
    service.service_id,
    service.service_name,
    service.price,
    service.duration_minutes
  FROM public.appointment_services AS service
  WHERE service.appointment_id = v_appointment.id;

  UPDATE public.appointments
  SET status = 'rescheduled',
      admin_notes = v_original_shop_notes,
      reschedule_count = v_appointment.reschedule_count + 1,
      rescheduled_to_appointment_id = v_new_appointment_id,
      pending_reschedule_request_id = NULL,
      pending_reschedule_date = NULL,
      pending_reschedule_start_time = NULL,
      pending_reschedule_reference_code = NULL,
      pending_reschedule_reason = NULL
  WHERE id = v_appointment.id;

  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES (
    'rescheduled_appointment',
    'Rescheduled booking ' || v_new_reference_code,
    v_appointment.customer_name || ' rescheduled from ' ||
v_appointment.reference_code
      || ' to ' || v_new_reference_code || '. Reason: ' || v_reason,
    v_new_appointment_id
  );

  INSERT INTO public.appointment_reschedule_history (
    appointment_id,
    new_appointment_id,
    reschedule_number,
    from_date,
    from_start_time,
    to_date,
```

```
    to_start_time,
    reason,
    decision,
    approved_by
  ) VALUES (
    v_appointment.id,
    v_new_appointment_id,
    v_reschedule_number,
    v_appointment.appointment_date,
    v_appointment.start_time,
    v_appointment.pending_reschedule_date,
    v_appointment.pending_reschedule_start_time,
    v_reason,
    'confirmed',
    auth.uid()
  );
END;
$$;

REVOKE ALL ON FUNCTION public.review_reschedule_request(uuid, text)
FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.review_reschedule_request(uuid, text)
TO authenticated;
```

supabase/migrations/20260915110000_separate_motorcycle_catalog_from_products.sql

---

```
-- Keep the service motorcycle catalog independent from shop inventory.
-- Appointment snapshots intentionally remain untouched: they preserve the
-- configuration used for an already-created booking and are required when a
-- customer reschedules that appointment.

CREATE TABLE IF NOT EXISTS public.motorcycle_catalog (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  brand text,
  model text NOT NULL,
  engine_cc numeric(6, 1),
  fuel_type text,
  transmission text,
  is_active boolean NOT NULL DEFAULT true,
  is_archived boolean NOT NULL DEFAULT false,
  sort_order integer NOT NULL DEFAULT 0,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now(),
  archived_at timestamptz
);

-- Preserve every existing catalog row, including archived records and their
-- identifiers, before removing motorcycle rows from the shop-products table.
INSERT INTO public.motorcycle_catalog (
  id,
  brand,
  model,
  engine_cc,
  fuel_type,
  transmission,
  is_active,
  is_archived,
  sort_order,
  created_at,
  updated_at,
  archived_at
)
SELECT
  id,
  brand,
  name,
  engine_cc,
  fuel_type,
```

```
  transmission,
  is_active,
  is_archived,
  sort_order,
  created_at,
  updated_at,
  archived_at
FROM public.products
WHERE category = 'motorcycle'
ON CONFLICT (id) DO UPDATE
SET
  brand = EXCLUDED.brand,
  model = EXCLUDED.model,
  engine_cc = EXCLUDED.engine_cc,
  fuel_type = EXCLUDED.fuel_type,
  transmission = EXCLUDED.transmission,
  is_active = EXCLUDED.is_active,
  is_archived = EXCLUDED.is_archived,
  sort_order = EXCLUDED.sort_order,
  created_at = EXCLUDED.created_at,
  updated_at = EXCLUDED.updated_at,
  archived_at = EXCLUDED.archived_at;

ALTER TABLE public.motorcycle_catalog
  DROP CONSTRAINT IF EXISTS motorcycle_catalog_engine_cc_positive,
  DROP CONSTRAINT IF EXISTS motorcycle_catalog_fuel_type_valid,
  DROP CONSTRAINT IF EXISTS motorcycle_catalog_transmission_valid;

ALTER TABLE public.motorcycle_catalog
  ADD CONSTRAINT motorcycle_catalog_engine_cc_positive
    CHECK (engine_cc IS NULL OR engine_cc > 0),
  ADD CONSTRAINT motorcycle_catalog_fuel_type_valid
    CHECK (fuel_type IS NULL OR fuel_type IN ('FI', 'Carbureted', 'Electric')),
  ADD CONSTRAINT motorcycle_catalog_transmission_valid
    CHECK (transmission IS NULL OR transmission IN ('Manual',
'Semi-Automatic', 'Automatic'));

CREATE UNIQUE INDEX IF NOT EXISTS
motorcycle_catalog_brand_model_unique
  ON public.motorcycle_catalog (brand, model)
  NULLS NOT DISTINCT;

CREATE INDEX IF NOT EXISTS motorcycle_catalog_active_list_idx
  ON public.motorcycle_catalog (brand, model)
  WHERE is_active = true AND is_archived = false;
```

```
CREATE INDEX IF NOT EXISTS motorcycle_catalog_archive_list_idx
  ON public.motorcycle_catalog (is_archived, brand, model);

GRANT SELECT ON public.motorcycle_catalog TO anon;
GRANT SELECT, INSERT, UPDATE, DELETE ON public.motorcycle_catalog
TO authenticated;
GRANT ALL ON public.motorcycle_catalog TO service_role;

ALTER TABLE public.motorcycle_catalog ENABLE ROW LEVEL SECURITY;

DROP POLICY IF EXISTS "motorcycle catalog public read" ON
public.motorcycle_catalog;
CREATE POLICY "motorcycle catalog public read" ON
public.motorcycle_catalog
  FOR SELECT TO anon
  USING (is_active = true AND is_archived = false);

DROP POLICY IF EXISTS "motorcycle catalog admin write" ON
public.motorcycle_catalog;
CREATE POLICY "motorcycle catalog admin write" ON
public.motorcycle_catalog
  FOR ALL TO authenticated
  USING (public.has_role(auth.uid(), 'admin'))
  WITH CHECK (public.has_role(auth.uid(), 'admin'));

DROP TRIGGER IF EXISTS motorcycle_catalog_updated ON
public.motorcycle_catalog;
CREATE TRIGGER motorcycle_catalog_updated
  BEFORE UPDATE ON public.motorcycle_catalog
  FOR EACH ROW EXECUTE FUNCTION public.update_updated_at_column();

DROP TRIGGER IF EXISTS motorcycle_catalog_archive_timestamp ON
public.motorcycle_catalog;
CREATE TRIGGER motorcycle_catalog_archive_timestamp
  BEFORE UPDATE ON public.motorcycle_catalog
  FOR EACH ROW EXECUTE FUNCTION
public.set_catalog_archive_timestamp();

DROP TRIGGER IF EXISTS admin_activity_motorcycle_catalog ON
public.motorcycle_catalog;
CREATE TRIGGER admin_activity_motorcycle_catalog
  AFTER INSERT OR UPDATE OR DELETE ON public.motorcycle_catalog
  FOR EACH ROW EXECUTE FUNCTION public.record_admin_activity();
```

```
-- Motorcycle records now live exclusively in public.motorcycle_catalog.
DELETE FROM public.products WHERE category = 'motorcycle';

DROP INDEX IF EXISTS public.products_motorcycle_catalog_sort_idx;

ALTER TABLE public.products
  DROP CONSTRAINT IF EXISTS products_engine_cc_positive,
  DROP CONSTRAINT IF EXISTS products_fuel_type_valid,
  DROP CONSTRAINT IF EXISTS products_transmission_valid,
  DROP COLUMN IF EXISTS engine_cc,
  DROP COLUMN IF EXISTS fuel_type,
  DROP COLUMN IF EXISTS transmission;

ALTER TABLE public.products
  DROP CONSTRAINT IF EXISTS products_shop_category_check;

ALTER TABLE public.products
  ADD CONSTRAINT products_shop_category_check
  CHECK (category IN ('part', 'accessory'));
```

supabase/migrations/20260917100000_harden_booking_capacity_and_reschedule_slots.sql

---

```
-- Keep the final booking write authoritative. The application preflights
-- availability for a fast customer response, while this trigger serializes
-- overlapping requests for a date so different free crew members cannot bypass
-- a shared slot capacity during a race.

CREATE OR REPLACE FUNCTION public.booking_slot_capacity(p_start_time
time)
RETURNS integer
LANGUAGE sql
STABLE
SET search_path = public
AS $$
  SELECT slot.capacity
  FROM public.time_slots AS slot
  WHERE slot.is_active = true
    AND (
      slot.start_time = p_start_time
      OR slot.start_time = make_time(extract(hour FROM p_start_time)::integer, 0,
0)
    )
  ORDER BY (slot.start_time = p_start_time) DESC
  LIMIT 1;
$$;

CREATE OR REPLACE FUNCTION
public.enforce_active_appointment_capacity()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = public
AS $$
DECLARE
  v_capacity integer;
  v_overlapping_appointments integer;
  v_start_minutes integer;
  v_hour_start time;
BEGIN
  -- Archived, replaced, or terminal appointments do not consume capacity.
  IF NEW.is_archived = true
     OR NEW.rescheduled_to_appointment_id IS NOT NULL
     OR NEW.status NOT IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
THEN
    RETURN NEW;
```

```
  END IF;

  v_start_minutes := extract(hour FROM NEW.start_time)::integer * 60
    + extract(minute FROM NEW.start_time)::integer;
  IF v_start_minutes < 480
     OR v_start_minutes >= 1020
     OR mod(v_start_minutes - 480, 30) <> 0
     OR v_start_minutes + NEW.booking_duration_minutes > 1020 THEN
    RAISE EXCEPTION 'The appointment must fit within 8:00 AM to 5:00 PM in
30-minute start intervals.'
      USING ERRCODE = '23P01';
  END IF;

  v_hour_start := make_time(extract(hour FROM NEW.start_time)::integer, 0, 0);

  -- All new or changed appointments on the same day use one transaction lock.
  -- This also covers overlapping starts that belong to adjacent hourly rows.
  PERFORM pg_advisory_xact_lock(hashtext('booking-capacity:' ||
NEW.appointment_date::text));

  SELECT slot.capacity
    INTO v_capacity
  FROM public.time_slots AS slot
  WHERE slot.is_active = true
    AND (slot.start_time = NEW.start_time OR slot.start_time = v_hour_start)
  ORDER BY (slot.start_time = NEW.start_time) DESC
  LIMIT 1
  FOR UPDATE;

  IF COALESCE(v_capacity, 0) <= 0 THEN
    RAISE EXCEPTION 'The requested time slot is not available.' USING
ERRCODE = '23P01';
  END IF;

  SELECT count(*)
    INTO v_overlapping_appointments
  FROM public.appointments AS other
  WHERE other.id <> NEW.id
    AND other.appointment_date = NEW.appointment_date
    AND other.is_archived = false
    AND other.rescheduled_to_appointment_id IS NULL
    AND other.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
    AND tsrange(
      other.appointment_date + other.start_time,
      other.appointment_date + other.start_time
```

```
      + make_interval(mins => other.booking_duration_minutes),
    '[)'
  ) && tsrange(
    NEW.appointment_date + NEW.start_time,
    NEW.appointment_date + NEW.start_time
      + make_interval(mins => NEW.booking_duration_minutes),
    '[)'
  );

 IF v_overlapping_appointments >= v_capacity THEN
  RAISE EXCEPTION 'The requested schedule is no longer available.' USING
ERRCODE = '23P01';
 END IF;

 RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS appointments_enforce_active_capacity ON
public.appointments;
CREATE TRIGGER appointments_enforce_active_capacity
 BEFORE INSERT OR UPDATE OF
  appointment_date,
  start_time,
  booking_duration_minutes,
  status,
  is_archived,
  rescheduled_to_appointment_id
 ON public.appointments
 FOR EACH ROW
 EXECUTE FUNCTION public.enforce_active_appointment_capacity();

REVOKE ALL ON FUNCTION public.booking_slot_capacity(time) FROM
PUBLIC;
GRANT EXECUTE ON FUNCTION public.booking_slot_capacity(time) TO
authenticated, service_role;

-- The booking grid maps an hourly time_slots configuration to both :00 and
-- :30 starts. The previous review function looked up only an exact row, which
-- rejected a valid :30 reschedule during admin approval. Revalidate the same
-- hourly mapping here; the trigger above remains the concurrency-safe final
-- capacity check when the replacement row is inserted.
CREATE OR REPLACE FUNCTION public.review_reschedule_request(
 p_appointment_id uuid,
 p_decision text
```

```
)
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment public.appointments%ROWTYPE;
  v_assigned_crew_id uuid;
  v_duration_minutes integer;
  v_new_appointment_id uuid;
  v_new_reference_code text;
  v_reason text;
  v_original_shop_notes text;
  v_new_shop_notes text;
  v_reschedule_number integer;
  v_slot_capacity integer;
  v_overlapping_appointments integer;
  v_start_minutes integer;
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Only administrators can review reschedule requests.';
  END IF;
  IF lower(btrim(COALESCE(p_decision, ''))) NOT IN ('confirmed', 'rejected')
THEN
    RAISE EXCEPTION 'Choose either confirmed or rejected.';
  END IF;

  SELECT appointment.*
    INTO v_appointment
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
  FOR UPDATE;

  IF NOT FOUND OR v_appointment.pending_reschedule_request_id IS NULL
THEN
    RAISE EXCEPTION 'This reschedule request is no longer available.';
  END IF;

  v_reschedule_number := v_appointment.reschedule_count + 1;

  IF lower(btrim(p_decision)) = 'rejected' THEN
    INSERT INTO public.appointment_reschedule_history (
      appointment_id,
```

```
      new_appointment_id,
      reschedule_number,
      from_date,
      from_start_time,
      to_date,
      to_start_time,
      reason,
      decision,
      approved_by
    ) VALUES (
      v_appointment.id,
      NULL,
      v_reschedule_number,
      v_appointment.appointment_date,
      v_appointment.start_time,
      v_appointment.pending_reschedule_date,
      v_appointment.pending_reschedule_start_time,
      btrim(v_appointment.pending_reschedule_reason),
      'rejected',
      auth.uid()
    );

    UPDATE public.appointments
    SET pending_reschedule_request_id = NULL,
        pending_reschedule_date = NULL,
        pending_reschedule_start_time = NULL,
        pending_reschedule_reference_code = NULL,
        pending_reschedule_reason = NULL
    WHERE id = v_appointment.id;
    RETURN;
  END IF;

  IF v_appointment.reschedule_count >= 3 THEN
    RAISE EXCEPTION 'This appointment has reached the maximum of 3
reschedules.';
  END IF;
  IF v_appointment.pending_reschedule_date IS NULL
     OR v_appointment.pending_reschedule_start_time IS NULL
     OR NULLIF(btrim(v_appointment.pending_reschedule_reason), '') IS NULL
THEN
    RAISE EXCEPTION 'The reschedule request is missing its requested
schedule or reason.';
  END IF;

  v_duration_minutes := COALESCE(v_appointment.booking_duration_minutes,
```

```
75);
  v_reason := btrim(v_appointment.pending_reschedule_reason);
  v_start_minutes := extract(hour FROM
v_appointment.pending_reschedule_start_time)::integer * 60
    + extract(minute FROM
v_appointment.pending_reschedule_start_time)::integer;
  IF extract(dow FROM v_appointment.pending_reschedule_date) = 0
     OR v_start_minutes < 480
     OR v_start_minutes >= 1020
     OR mod(v_start_minutes - 480, 30) <> 0
     OR v_start_minutes + v_duration_minutes > 1020 THEN
    RAISE EXCEPTION 'The requested schedule is outside the shop booking
hours.';
  END IF;

  SELECT
public.booking_slot_capacity(v_appointment.pending_reschedule_start_time)
    INTO v_slot_capacity;
  IF COALESCE(v_slot_capacity, 0) <= 0 THEN
    RAISE EXCEPTION 'The requested time slot is no longer available.';
  END IF;

  SELECT count(*)
    INTO v_overlapping_appointments
  FROM public.appointments AS other
  WHERE other.id <> v_appointment.id
    AND other.appointment_date = v_appointment.pending_reschedule_date
    AND other.is_archived = false
    AND other.rescheduled_to_appointment_id IS NULL
    AND other.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
    AND tsrange(
      other.appointment_date + other.start_time,
      other.appointment_date + other.start_time
        + make_interval(mins => COALESCE(other.booking_duration_minutes,
75)),
      '[)'
    ) && tsrange(
      v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
      v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
        + make_interval(mins => v_duration_minutes),
      '[)'
    );
  IF v_overlapping_appointments >= v_slot_capacity THEN
```

```
    RAISE EXCEPTION 'The requested time is no longer available.';
  END IF;

  IF EXISTS (
    SELECT 1
    FROM public.schedule_blocks AS block
    WHERE block.block_date = v_appointment.pending_reschedule_date
      AND block.is_active = true
      AND (
        block.start_time IS NULL
        OR (
          block.start_time IS NOT NULL
          AND (
            (COALESCE(block.reason, '') NOT LIKE 'RANGE:%'
              AND block.start_time = v_appointment.pending_reschedule_start_time)
            OR (COALESCE(block.reason, '') LIKE 'RANGE:%'
              AND tsrange(
                v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
                v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
                  + make_interval(mins => v_duration_minutes),
                '[)'
              ) && tsrange(
                v_appointment.pending_reschedule_date + block.start_time,
                v_appointment.pending_reschedule_date
                  + substring(block.reason FROM '^RANGE:([0-9]{2}:[0-9]{2})')::time,
                '[)'
              ))
          )
        )
      )
  ) THEN
    RAISE EXCEPTION 'The requested time is no longer available because the
shop schedule is blocked.';
  END IF;

  SELECT crew.id
    INTO v_assigned_crew_id
  FROM public.crew_members AS crew
  WHERE crew.is_active = true
    AND crew.is_archived = false
    AND EXISTS (
      SELECT 1
      FROM public.crew_schedules AS schedule
```

```
      WHERE schedule.crew_id = crew.id
        AND schedule.schedule_date = v_appointment.pending_reschedule_date
        AND schedule.is_working = true
        AND schedule.start_time IS NOT NULL
        AND schedule.end_time IS NOT NULL
        AND schedule.start_time <= v_appointment.pending_reschedule_start_time
        AND schedule.end_time >= v_appointment.pending_reschedule_start_time
          + make_interval(mins => v_duration_minutes)
    )
    AND NOT EXISTS (
      SELECT 1
      FROM public.crew_availability_exceptions AS exception
      WHERE exception.crew_id = crew.id
        AND exception.start_date <= v_appointment.pending_reschedule_date
        AND exception.end_date >= v_appointment.pending_reschedule_date
        AND (
          exception.is_all_day
          OR (
            exception.start_time IS NOT NULL
            AND exception.end_time IS NOT NULL
            AND tsrange(
              v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
              v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
                + make_interval(mins => v_duration_minutes),
              '[)'
            ) && tsrange(
              v_appointment.pending_reschedule_date + exception.start_time,
              v_appointment.pending_reschedule_date + exception.end_time,
              '[)'
            )
          )
        )
    )
    AND NOT EXISTS (
      SELECT 1
      FROM public.appointments AS other
      WHERE other.id <> v_appointment.id
        AND other.assigned_crew_id = crew.id
        AND other.appointment_date = v_appointment.pending_reschedule_date
        AND other.is_archived = false
        AND other.rescheduled_to_appointment_id IS NULL
        AND other.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
        AND tsrange(
```

```
          other.appointment_date + other.start_time,
          other.appointment_date + other.start_time
            + make_interval(mins => COALESCE(other.booking_duration_minutes,
75)),
          '[)'
        ) && tsrange(
          v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
          v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
            + make_interval(mins => v_duration_minutes),
          '[)'
        )
    )
  ORDER BY (crew.id = v_appointment.assigned_crew_id) DESC, crew.name,
crew.id
  LIMIT 1;

  IF v_assigned_crew_id IS NULL THEN
    RAISE EXCEPTION 'No crew member is available for the requested
appointment period.';
  END IF;

  v_new_reference_code := v_appointment.pending_reschedule_reference_code;
  IF v_new_reference_code IS NULL
     OR v_new_reference_code !~ '^FRM-[A-Z0-9]{6}$' THEN
    RAISE EXCEPTION 'The reschedule request is missing its replacement
reference.';
  END IF;

  v_original_shop_notes := concat_ws(
    E'\n',
    NULLIF(btrim(v_appointment.admin_notes), ''),
    'Rescheduled to ' || v_new_reference_code,
    'Reason for rescheduling: ' || v_reason
  );
  v_new_shop_notes := concat_ws(
    E'\n',
    'Rescheduled from ' || v_appointment.reference_code,
    'Reason for rescheduling: ' || v_reason
  );

  INSERT INTO public.appointments AS replacement (
    reference_code,
    booking_request_id,
```

```
  customer_name,
  first_name,
  middle_name,
  last_name,
  phone,
  email,
  moto_brand,
  moto_model,
  moto_cc,
  moto_fuel_type,
  moto_transmission,
  moto_variant,
  moto_year,
  plate_number,
  appointment_date,
  start_time,
  status,
  notes,
  admin_notes,
  total_estimate,
  booking_duration_minutes,
  terms_accepted,
  assigned_crew_id,
  reschedule_count,
  rescheduled_from_appointment_id
) VALUES (
  v_new_reference_code,
  gen_random_uuid(),
  v_appointment.customer_name,
  v_appointment.first_name,
  v_appointment.middle_name,
  v_appointment.last_name,
  v_appointment.phone,
  v_appointment.email,
  v_appointment.moto_brand,
  v_appointment.moto_model,
  v_appointment.moto_cc,
  v_appointment.moto_fuel_type,
  v_appointment.moto_transmission,
  v_appointment.moto_variant,
  v_appointment.moto_year,
  v_appointment.plate_number,
  v_appointment.pending_reschedule_date,
  v_appointment.pending_reschedule_start_time,
  'confirmed',
```

```
    v_appointment.notes,
    v_new_shop_notes,
    v_appointment.total_estimate,
    v_duration_minutes,
    v_appointment.terms_accepted,
    v_assigned_crew_id,
    v_appointment.reschedule_count + 1,
    v_appointment.id
  )
  RETURNING replacement.id INTO v_new_appointment_id;

  INSERT INTO public.appointment_services (
    appointment_id,
    service_id,
    service_name,
    price,
    duration_minutes
  )
  SELECT
    v_new_appointment_id,
    service.service_id,
    service.service_name,
    service.price,
    service.duration_minutes
  FROM public.appointment_services AS service
  WHERE service.appointment_id = v_appointment.id;

  UPDATE public.appointments
  SET status = 'rescheduled',
      admin_notes = v_original_shop_notes,
      reschedule_count = v_appointment.reschedule_count + 1,
      rescheduled_to_appointment_id = v_new_appointment_id,
      pending_reschedule_request_id = NULL,
      pending_reschedule_date = NULL,
      pending_reschedule_start_time = NULL,
      pending_reschedule_reference_code = NULL,
      pending_reschedule_reason = NULL
  WHERE id = v_appointment.id;

  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES (
    'rescheduled_appointment',
    'Rescheduled booking ' || v_new_reference_code,
    v_appointment.customer_name || ' rescheduled from ' ||
v_appointment.reference_code
```

```
      || ' to ' || v_new_reference_code || '. Reason: ' || v_reason,
    v_new_appointment_id
  );

  INSERT INTO public.appointment_reschedule_history (
    appointment_id,
    new_appointment_id,
    reschedule_number,
    from_date,
    from_start_time,
    to_date,
    to_start_time,
    reason,
    decision,
    approved_by
  ) VALUES (
    v_appointment.id,
    v_new_appointment_id,
    v_reschedule_number,
    v_appointment.appointment_date,
    v_appointment.start_time,
    v_appointment.pending_reschedule_date,
    v_appointment.pending_reschedule_start_time,
    v_reason,
    'confirmed',
    auth.uid()
  );
END;
$$;

REVOKE ALL ON FUNCTION public.review_reschedule_request(uuid, text)
FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.review_reschedule_request(uuid, text)
TO authenticated;

-- Lookup, notice validation, cancellation, and its notification must be one
-- transaction. A plain update after the public lookup could otherwise cancel
-- an appointment that was rescheduled in the short interval before the write,
-- or leave a cancelled booking without its staff notification.
CREATE OR REPLACE FUNCTION public.cancel_public_appointment(
  p_appointment_id uuid,
  p_cancellation_notice_hours integer
)
RETURNS void
LANGUAGE plpgsql
```

```
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment public.appointments%ROWTYPE;
BEGIN
  IF p_appointment_id IS NULL
     OR p_cancellation_notice_hours IS NULL
     OR p_cancellation_notice_hours < 0
     OR p_cancellation_notice_hours > 168 THEN
    RAISE EXCEPTION 'Invalid cancellation request.';
  END IF;

  SELECT appointment.*
    INTO v_appointment
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
  FOR UPDATE;

  IF NOT FOUND THEN
    RAISE EXCEPTION 'This appointment is no longer available.';
  END IF;
  IF v_appointment.rescheduled_to_appointment_id IS NOT NULL THEN
    RAISE EXCEPTION 'This appointment has already been rescheduled. Please
use the new reference.';
  END IF;
  IF v_appointment.pending_reschedule_request_id IS NOT NULL THEN
    RAISE EXCEPTION 'A reschedule request is already awaiting the shop''s
review.';
  END IF;
  IF v_appointment.status NOT IN ('pending', 'confirmed', 'rescheduled') THEN
    RAISE EXCEPTION 'This appointment can no longer be cancelled online.
Please call the shop.';
  END IF;
  IF (v_appointment.appointment_date + v_appointment.start_time)
       - (now() AT TIME ZONE 'Asia/Manila')
       < make_interval(hours => p_cancellation_notice_hours) THEN
    RAISE EXCEPTION 'Cancellations need % hours notice. Please call the shop
instead.',
      p_cancellation_notice_hours;
  END IF;

  UPDATE public.appointments
  SET status = 'cancelled',
```

```
    cancelled_at = now()
  WHERE id = v_appointment.id;

  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES (
    'cancelled_appointment',
    'Cancelled ' || v_appointment.reference_code,
    v_appointment.customer_name || ' cancelled their '
      || to_char(v_appointment.appointment_date, 'Mon FMDD, YYYY') || '
appointment.',
    v_appointment.id
  );
END;
$$;

REVOKE ALL ON FUNCTION public.cancel_public_appointment(uuid, integer)
FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.cancel_public_appointment(uuid,
integer) TO service_role;
```

supabase/migrations/20260918000000_remove_motorcycle_specifications_from_booking.sql

---

```
-- Motorcycle identity is limited to brand and model. Service price and duration
-- come from an exact model override when present, otherwise the service
default.

DROP FUNCTION IF EXISTS public.create_booking_atomic(
  text, uuid, text, text, text, text, text, integer, text, text, text, integer,
  text, date, time, text, numeric, integer, uuid, jsonb, text, text, text, text, text
);

CREATE FUNCTION public.create_booking_atomic(
  p_reference_code text,
  p_booking_request_id uuid,
  p_customer_name text,
  p_phone text,
  p_email text,
  p_moto_brand text,
  p_moto_model text,
  p_moto_variant text,
  p_moto_year integer,
  p_plate_number text,
  p_appointment_date date,
  p_start_time time,
  p_notes text,
  p_total_estimate numeric,
  p_booking_duration_minutes integer,
  p_assigned_crew_id uuid,
  p_services jsonb,
  p_notification_title text,
  p_notification_message text,
  p_first_name text,
  p_middle_name text,
  p_last_name text
)
RETURNS TABLE (appointment_id uuid, reference_code text)
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment_id uuid;
  v_reference_code text;
BEGIN
```

```
  IF p_booking_request_id IS NULL THEN
    RAISE EXCEPTION 'A booking request ID is required.';
  END IF;
  IF COALESCE(btrim(p_first_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$'
     OR COALESCE(btrim(p_middle_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$'
     OR COALESCE(btrim(p_last_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$' THEN
    RAISE EXCEPTION 'First name, middle name, and last name must contain
letters only.';
  END IF;
  IF jsonb_typeof(p_services) <> 'array' OR jsonb_array_length(p_services) = 0
THEN
    RAISE EXCEPTION 'A booking must include at least one service.';
  END IF;

  INSERT INTO public.appointments (
    reference_code, booking_request_id, customer_name, first_name,
middle_name,
    last_name, phone, email, moto_brand, moto_model, moto_variant, moto_year,
    plate_number, appointment_date, start_time, notes, total_estimate,
    booking_duration_minutes, terms_accepted, assigned_crew_id
  ) VALUES (
    p_reference_code, p_booking_request_id,
    concat_ws(' ', btrim(p_first_name), btrim(p_middle_name),
btrim(p_last_name)),
    btrim(p_first_name), btrim(p_middle_name), btrim(p_last_name), p_phone,
p_email,
    p_moto_brand, p_moto_model, p_moto_variant, p_moto_year,
p_plate_number,
    p_appointment_date, p_start_time, p_notes, p_total_estimate,
    p_booking_duration_minutes, true, p_assigned_crew_id
  )
  ON CONFLICT DO NOTHING
  RETURNING appointments.id, appointments.reference_code INTO
v_appointment_id, v_reference_code;

  IF v_appointment_id IS NULL THEN
    SELECT appointments.id, appointments.reference_code
      INTO v_appointment_id, v_reference_code
    FROM public.appointments
    WHERE appointments.booking_request_id = p_booking_request_id;
    IF FOUND THEN
      RETURN QUERY SELECT v_appointment_id, v_reference_code;
      RETURN;
    END IF;
    RAISE EXCEPTION 'The requested schedule is no longer available.' USING
```

```
ERRCODE = '23P01';
  END IF;

  INSERT INTO public.appointment_services (
    appointment_id, service_id, service_name, price, duration_minutes
  )
  SELECT v_appointment_id, service.service_id, service.service_name,
    service.price, service.duration_minutes
  FROM jsonb_to_recordset(p_services) AS service(
    service_id uuid, service_name text, price numeric, duration_minutes integer
  );

  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES ('new_appointment', p_notification_title, p_notification_message,
v_appointment_id);

  RETURN QUERY SELECT v_appointment_id, v_reference_code;
END;
$$;

REVOKE ALL ON FUNCTION public.create_booking_atomic(
  text, uuid, text, text, text, text, text, text, integer, text, date, time,
  text, numeric, integer, uuid, jsonb, text, text, text, text, text
) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.create_booking_atomic(
  text, uuid, text, text, text, text, text, text, integer, text, date, time,
  text, numeric, integer, uuid, jsonb, text, text, text, text, text
) TO service_role;

CREATE OR REPLACE FUNCTION public.review_reschedule_request(
  p_appointment_id uuid,
  p_decision text
)
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment public.appointments%ROWTYPE;
  v_assigned_crew_id uuid;
  v_duration_minutes integer;
  v_new_appointment_id uuid;
  v_new_reference_code text;
  v_reason text;
```

```
  v_original_shop_notes text;
  v_new_shop_notes text;
  v_reschedule_number integer;
  v_slot_capacity integer;
  v_overlapping_appointments integer;
  v_start_minutes integer;
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Only administrators can review reschedule requests.';
  END IF;
  IF lower(btrim(COALESCE(p_decision, ''))) NOT IN ('confirmed', 'rejected')
THEN
    RAISE EXCEPTION 'Choose either confirmed or rejected.';
  END IF;

  SELECT appointment.* INTO v_appointment
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id AND appointment.is_archived =
false
  FOR UPDATE;
  IF NOT FOUND OR v_appointment.pending_reschedule_request_id IS NULL
THEN
    RAISE EXCEPTION 'This reschedule request is no longer available.';
  END IF;

  v_reschedule_number := v_appointment.reschedule_count + 1;
  IF lower(btrim(p_decision)) = 'rejected' THEN
    INSERT INTO public.appointment_reschedule_history (
      appointment_id, new_appointment_id, reschedule_number, from_date,
      from_start_time, to_date, to_start_time, reason, decision, approved_by
    ) VALUES (
      v_appointment.id, NULL, v_reschedule_number,
v_appointment.appointment_date,
      v_appointment.start_time, v_appointment.pending_reschedule_date,
      v_appointment.pending_reschedule_start_time,
btrim(v_appointment.pending_reschedule_reason),
      'rejected', auth.uid()
    );
    UPDATE public.appointments
    SET pending_reschedule_request_id = NULL,
        pending_reschedule_date = NULL,
        pending_reschedule_start_time = NULL,
        pending_reschedule_reference_code = NULL,
        pending_reschedule_reason = NULL
    WHERE id = v_appointment.id;
```

```
    RETURN;
  END IF;

  IF v_appointment.reschedule_count >= 3 THEN
    RAISE EXCEPTION 'This appointment has reached the maximum of 3
reschedules.';
  END IF;
  IF v_appointment.pending_reschedule_date IS NULL
     OR v_appointment.pending_reschedule_start_time IS NULL
     OR NULLIF(btrim(v_appointment.pending_reschedule_reason), '') IS NULL
THEN
    RAISE EXCEPTION 'The reschedule request is missing its requested
schedule or reason.';
  END IF;

  v_duration_minutes := COALESCE(v_appointment.booking_duration_minutes,
75);
  v_reason := btrim(v_appointment.pending_reschedule_reason);
  v_start_minutes := extract(hour FROM
v_appointment.pending_reschedule_start_time)::integer * 60
    + extract(minute FROM
v_appointment.pending_reschedule_start_time)::integer;
  IF extract(dow FROM v_appointment.pending_reschedule_date) = 0
     OR v_start_minutes < 480 OR v_start_minutes >= 1020
     OR mod(v_start_minutes - 480, 30) <> 0
     OR v_start_minutes + v_duration_minutes > 1020 THEN
    RAISE EXCEPTION 'The requested schedule is outside the shop booking
hours.';
  END IF;

  SELECT
public.booking_slot_capacity(v_appointment.pending_reschedule_start_time)
    INTO v_slot_capacity;
  IF COALESCE(v_slot_capacity, 0) <= 0 THEN
    RAISE EXCEPTION 'The requested time slot is no longer available.';
  END IF;

  SELECT count(*) INTO v_overlapping_appointments
  FROM public.appointments AS other
  WHERE other.id <> v_appointment.id
    AND other.appointment_date = v_appointment.pending_reschedule_date
    AND other.is_archived = false
    AND other.rescheduled_to_appointment_id IS NULL
    AND other.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
    AND tsrange(
```

```
      other.appointment_date + other.start_time,
      other.appointment_date + other.start_time
        + make_interval(mins => COALESCE(other.booking_duration_minutes,
75)), '[)'
    ) && tsrange(
      v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
      v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
        + make_interval(mins => v_duration_minutes), '[)'
    );
  IF v_overlapping_appointments >= v_slot_capacity THEN
    RAISE EXCEPTION 'The requested time is no longer available.';
  END IF;

  IF EXISTS (
    SELECT 1 FROM public.schedule_blocks AS block
    WHERE block.block_date = v_appointment.pending_reschedule_date
      AND block.is_active = true
      AND (
        block.start_time IS NULL
        OR (COALESCE(block.reason, '') NOT LIKE 'RANGE:%'
          AND block.start_time = v_appointment.pending_reschedule_start_time)
        OR (COALESCE(block.reason, '') LIKE 'RANGE:%'
          AND tsrange(
            v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
            v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
              + make_interval(mins => v_duration_minutes), '[)'
          ) && tsrange(
            v_appointment.pending_reschedule_date + block.start_time,
            v_appointment.pending_reschedule_date
              + substring(block.reason FROM '^RANGE:([0-9]{2}:[0-9]{2})')::time, '[)'
          ))
      )
  ) THEN
    RAISE EXCEPTION 'The requested time is no longer available because the
shop schedule is blocked.';
  END IF;

  SELECT crew.id INTO v_assigned_crew_id
  FROM public.crew_members AS crew
  WHERE crew.is_active = true AND crew.is_archived = false
    AND EXISTS (
```

```
    SELECT 1 FROM public.crew_schedules AS schedule
    WHERE schedule.crew_id = crew.id
      AND schedule.schedule_date = v_appointment.pending_reschedule_date
      AND schedule.is_working = true
      AND schedule.start_time IS NOT NULL AND schedule.end_time IS NOT
NULL
      AND schedule.start_time <= v_appointment.pending_reschedule_start_time
      AND schedule.end_time >= v_appointment.pending_reschedule_start_time
        + make_interval(mins => v_duration_minutes)
  )
  AND NOT EXISTS (
    SELECT 1 FROM public.crew_availability_exceptions AS exception
    WHERE exception.crew_id = crew.id
      AND exception.start_date <= v_appointment.pending_reschedule_date
      AND exception.end_date >= v_appointment.pending_reschedule_date
      AND (
        exception.is_all_day
        OR (exception.start_time IS NOT NULL AND exception.end_time IS NOT
NULL
          AND tsrange(
            v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
            v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
              + make_interval(mins => v_duration_minutes), '[)'
          ) && tsrange(
            v_appointment.pending_reschedule_date + exception.start_time,
            v_appointment.pending_reschedule_date + exception.end_time, '[)'
          ))
      )
  )
  AND NOT EXISTS (
    SELECT 1 FROM public.appointments AS other
    WHERE other.id <> v_appointment.id
      AND other.assigned_crew_id = crew.id
      AND other.appointment_date = v_appointment.pending_reschedule_date
      AND other.is_archived = false
      AND other.rescheduled_to_appointment_id IS NULL
      AND other.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
      AND tsrange(
        other.appointment_date + other.start_time,
        other.appointment_date + other.start_time
          + make_interval(mins => COALESCE(other.booking_duration_minutes,
75)), '[)'
      ) && tsrange(
```

```
          v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
          v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
            + make_interval(mins => v_duration_minutes), '[)'
        )
    )
  ORDER BY (crew.id = v_appointment.assigned_crew_id) DESC, crew.name,
crew.id
  LIMIT 1;
  IF v_assigned_crew_id IS NULL THEN
    RAISE EXCEPTION 'No crew member is available for the requested
appointment period.';
  END IF;

  v_new_reference_code := v_appointment.pending_reschedule_reference_code;
  IF v_new_reference_code IS NULL OR v_new_reference_code !~
'^FRM-[A-Z0-9]{6}$' THEN
    RAISE EXCEPTION 'The reschedule request is missing its replacement
reference.';
  END IF;

  v_original_shop_notes := concat_ws(E'\n',
NULLIF(btrim(v_appointment.admin_notes), ''),
    'Rescheduled to ' || v_new_reference_code, 'Reason for rescheduling: ' ||
v_reason);
  v_new_shop_notes := concat_ws(E'\n', 'Rescheduled from ' ||
v_appointment.reference_code,
    'Reason for rescheduling: ' || v_reason);

  INSERT INTO public.appointments AS replacement (
    reference_code, booking_request_id, customer_name, first_name,
middle_name,
    last_name, phone, email, moto_brand, moto_model, moto_variant, moto_year,
    plate_number, appointment_date, start_time, status, notes, admin_notes,
    total_estimate, booking_duration_minutes, terms_accepted,
assigned_crew_id,
    reschedule_count, rescheduled_from_appointment_id
  ) VALUES (
    v_new_reference_code, gen_random_uuid(), v_appointment.customer_name,
    v_appointment.first_name, v_appointment.middle_name,
v_appointment.last_name,
    v_appointment.phone, v_appointment.email, v_appointment.moto_brand,
    v_appointment.moto_model, v_appointment.moto_variant,
v_appointment.moto_year,
```

```
    v_appointment.plate_number, v_appointment.pending_reschedule_date,
    v_appointment.pending_reschedule_start_time, 'confirmed',
v_appointment.notes,
    v_new_shop_notes, v_appointment.total_estimate, v_duration_minutes,
    v_appointment.terms_accepted, v_assigned_crew_id,
    v_appointment.reschedule_count + 1, v_appointment.id
  ) RETURNING replacement.id INTO v_new_appointment_id;

  INSERT INTO public.appointment_services (
    appointment_id, service_id, service_name, price, duration_minutes
  )
  SELECT v_new_appointment_id, service.service_id, service.service_name,
    service.price, service.duration_minutes
  FROM public.appointment_services AS service
  WHERE service.appointment_id = v_appointment.id;

  UPDATE public.appointments
  SET status = 'rescheduled', admin_notes = v_original_shop_notes,
      reschedule_count = v_appointment.reschedule_count + 1,
      rescheduled_to_appointment_id = v_new_appointment_id,
      pending_reschedule_request_id = NULL, pending_reschedule_date = NULL,
      pending_reschedule_start_time = NULL,
pending_reschedule_reference_code = NULL,
      pending_reschedule_reason = NULL
  WHERE id = v_appointment.id;

  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES (
    'rescheduled_appointment', 'Rescheduled booking ' || v_new_reference_code,
    v_appointment.customer_name || ' rescheduled from ' ||
v_appointment.reference_code
      || ' to ' || v_new_reference_code || '. Reason: ' || v_reason,
    v_new_appointment_id
  );
  INSERT INTO public.appointment_reschedule_history (
    appointment_id, new_appointment_id, reschedule_number, from_date,
    from_start_time, to_date, to_start_time, reason, decision, approved_by
  ) VALUES (
    v_appointment.id, v_new_appointment_id, v_reschedule_number,
    v_appointment.appointment_date, v_appointment.start_time,
    v_appointment.pending_reschedule_date,
v_appointment.pending_reschedule_start_time,
    v_reason, 'confirmed', auth.uid()
  );
END;
```

```
$$;

REVOKE ALL ON FUNCTION public.review_reschedule_request(uuid, text)
FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.review_reschedule_request(uuid, text)
TO authenticated;

ALTER TABLE public.motorcycle_catalog
  DROP CONSTRAINT IF EXISTS motorcycle_catalog_engine_cc_positive,
  DROP CONSTRAINT IF EXISTS motorcycle_catalog_fuel_type_valid,
  DROP CONSTRAINT IF EXISTS motorcycle_catalog_transmission_valid,
  DROP COLUMN IF EXISTS engine_cc,
  DROP COLUMN IF EXISTS fuel_type,
  DROP COLUMN IF EXISTS transmission;

ALTER TABLE public.appointments
  DROP COLUMN IF EXISTS moto_cc,
  DROP COLUMN IF EXISTS moto_fuel_type,
  DROP COLUMN IF EXISTS moto_transmission;
```

supabase/migrations/20260918100000_remove_customer_email_from_bookings.sql

---

```
-- Customer contact is phone-only. Authentication/account emails remain in the
-- users/auth domain and are not touched by this migration.

-- Keep the full reschedule-review safeguards intact while removing the copied
-- customer email field from replacement appointments.
DO $$
DECLARE
  v_definition text;
BEGIN
  SELECT pg_get_functiondef('public.review_reschedule_request(uuid,
text)'::regprocedure)
    INTO v_definition;
  v_definition := replace(v_definition, E'    email,\n', '');
  v_definition := replace(v_definition, E'    v_appointment.email,\n', '');
  EXECUTE v_definition;
END;
$$;

DROP FUNCTION IF EXISTS public.create_booking_atomic(
  text, uuid, text, text, text, text, text, text, integer, text, date, time,
  text, numeric, integer, uuid, jsonb, text, text, text, text, text
);

CREATE FUNCTION public.create_booking_atomic(
  p_reference_code text, p_booking_request_id uuid, p_customer_name text,
  p_phone text, p_moto_brand text, p_moto_model text, p_moto_variant text,
  p_moto_year integer, p_plate_number text, p_appointment_date date,
  p_start_time time, p_notes text, p_total_estimate numeric,
  p_booking_duration_minutes integer, p_assigned_crew_id uuid, p_services
jsonb,
  p_notification_title text, p_notification_message text, p_first_name text,
  p_middle_name text, p_last_name text
)
RETURNS TABLE (appointment_id uuid, reference_code text)
LANGUAGE plpgsql SECURITY INVOKER SET search_path = public
AS $$
DECLARE v_appointment_id uuid; v_reference_code text;
BEGIN
  IF p_booking_request_id IS NULL THEN RAISE EXCEPTION 'A booking
request ID is required.'; END IF;
  IF COALESCE(btrim(p_first_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$'
     OR COALESCE(btrim(p_middle_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$'
```

```
    OR COALESCE(btrim(p_last_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$' THEN
    RAISE EXCEPTION 'First name, middle name, and last name must contain
letters only.';
  END IF;
  IF jsonb_typeof(p_services) <> 'array' OR jsonb_array_length(p_services) = 0
THEN
    RAISE EXCEPTION 'A booking must include at least one service.';
  END IF;
  INSERT INTO public.appointments (
    reference_code, booking_request_id, customer_name, first_name,
middle_name,
    last_name, phone, moto_brand, moto_model, moto_variant, moto_year,
    plate_number, appointment_date, start_time, notes, total_estimate,
    booking_duration_minutes, terms_accepted, assigned_crew_id
  ) VALUES (
    p_reference_code, p_booking_request_id,
    concat_ws(' ', btrim(p_first_name), btrim(p_middle_name),
btrim(p_last_name)),
    btrim(p_first_name), btrim(p_middle_name), btrim(p_last_name), p_phone,
    p_moto_brand, p_moto_model, p_moto_variant, p_moto_year,
p_plate_number,
    p_appointment_date, p_start_time, p_notes, p_total_estimate,
    p_booking_duration_minutes, true, p_assigned_crew_id
  ) ON CONFLICT DO NOTHING
  RETURNING appointments.id, appointments.reference_code INTO
v_appointment_id, v_reference_code;
  IF v_appointment_id IS NULL THEN
    SELECT appointments.id, appointments.reference_code INTO
v_appointment_id, v_reference_code
    FROM public.appointments WHERE appointments.booking_request_id =
p_booking_request_id;
    IF FOUND THEN RETURN QUERY SELECT v_appointment_id,
v_reference_code; RETURN; END IF;
    RAISE EXCEPTION 'The requested schedule is no longer available.' USING
ERRCODE = '23P01';
  END IF;
  INSERT INTO public.appointment_services (appointment_id, service_id,
service_name, price, duration_minutes)
  SELECT v_appointment_id, service.service_id, service.service_name,
service.price, service.duration_minutes
  FROM jsonb_to_recordset(p_services) AS service(
    service_id uuid, service_name text, price numeric, duration_minutes integer
  );
  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES ('new_appointment', p_notification_title, p_notification_message,
```

```
v_appointment_id);
  RETURN QUERY SELECT v_appointment_id, v_reference_code;
END;
$$;

REVOKE ALL ON FUNCTION public.create_booking_atomic(
  text, uuid, text, text, text, text, text, integer, text, date, time, text,
  numeric, integer, uuid, jsonb, text, text, text, text, text
) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.create_booking_atomic(
  text, uuid, text, text, text, text, text, integer, text, date, time, text,
  numeric, integer, uuid, jsonb, text, text, text, text, text
) TO service_role;

CREATE OR REPLACE FUNCTION public.get_admin_customers_page(
  p_search text DEFAULT NULL, p_limit integer DEFAULT 25, p_offset integer
DEFAULT 0
)
RETURNS jsonb
LANGUAGE plpgsql STABLE SECURITY INVOKER SET search_path = public,
auth
AS $$
DECLARE
  v_limit integer := LEAST(GREATEST(COALESCE(p_limit, 25), 1), 100);
  v_offset integer := GREATEST(COALESCE(p_offset, 0), 0);
  v_search text := NULLIF(BTRIM(p_search), '');
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;
  RETURN (
    WITH customers AS (
      SELECT a.phone,
        (ARRAY_AGG(a.customer_name ORDER BY a.appointment_date DESC,
a.created_at DESC))[1] AS customer_name,
        COUNT(*)::integer AS visits,
        COALESCE(SUM(a.total_estimate) FILTER (WHERE a.status =
'completed'), 0) AS completed_spend,
        MAX(a.appointment_date) AS last_booking,
        ARRAY_AGG(DISTINCT CONCAT_WS(' ', a.moto_brand, a.moto_model) ||
' (' || a.plate_number || ')') AS units
      FROM public.appointments a GROUP BY a.phone
    ), filtered AS (
      SELECT * FROM customers c
      WHERE v_search IS NULL OR CONCAT_WS(' ', c.customer_name,
```

```
c.phone) ILIKE '%' || v_search || '%'
    ), paged AS (
      SELECT * FROM filtered ORDER BY last_booking DESC, customer_name
ASC LIMIT v_limit OFFSET v_offset
    )
    SELECT jsonb_build_object(
      'total', (SELECT COUNT(*) FROM filtered),
      'rows', COALESCE((SELECT jsonb_agg(TO_JSONB(paged)) FROM paged),
'[]'::jsonb)
    )
  );
END;
$$;

ALTER TABLE public.appointments DROP COLUMN IF EXISTS email
CASCADE;

-- The unused legacy reschedule RPC depended on the removed customer email
-- column. Current rescheduling uses submit_reschedule_request instead.
DROP FUNCTION IF EXISTS public.create_rescheduled_booking_atomic(
  text, uuid, text, text, text, text, text, text, integer, text, date, time,
  text, numeric, integer, uuid, jsonb, text, text, uuid
);
```

supabase/migrations/20260918110000_make_booking_middle_name_optional.sql

---

```
-- Public booking captures last name, first name, and an optional middle name.
-- Administrative customer records and their display order are intentionally unchanged.

CREATE OR REPLACE FUNCTION public.create_booking_atomic(
  p_reference_code text, p_booking_request_id uuid, p_customer_name text,
  p_phone text, p_moto_brand text, p_moto_model text, p_moto_variant text,
  p_moto_year integer, p_plate_number text, p_appointment_date date,
  p_start_time time, p_notes text, p_total_estimate numeric,
  p_booking_duration_minutes integer, p_assigned_crew_id uuid, p_services jsonb,
  p_notification_title text, p_notification_message text, p_first_name text,
  p_middle_name text, p_last_name text
)
RETURNS TABLE (appointment_id uuid, reference_code text)
LANGUAGE plpgsql SECURITY INVOKER SET search_path = public
AS $$
DECLARE v_appointment_id uuid; v_reference_code text;
BEGIN
  IF p_booking_request_id IS NULL THEN RAISE EXCEPTION 'A booking request ID is required.'; END IF;
  IF COALESCE(btrim(p_first_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$'
     OR COALESCE(btrim(p_last_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$' THEN
    RAISE EXCEPTION 'First and last names must contain letters only.';
  END IF;
  IF COALESCE(btrim(p_middle_name), '') <> ''
     AND btrim(p_middle_name) !~ '^[[:alpha:]]+( [[:alpha:]]+)*$' THEN
    RAISE EXCEPTION 'Middle name must contain letters only.';
  END IF;
  IF jsonb_typeof(p_services) <> 'array' OR jsonb_array_length(p_services) = 0 THEN
    RAISE EXCEPTION 'A booking must include at least one service.';
  END IF;

  INSERT INTO public.appointments (
    reference_code, booking_request_id, customer_name, first_name, middle_name,
    last_name, phone, moto_brand, moto_model, moto_variant, moto_year,
    plate_number, appointment_date, start_time, notes, total_estimate,
    booking_duration_minutes, terms_accepted, assigned_crew_id
  ) VALUES (
    p_reference_code, p_booking_request_id,
```

```
    concat_ws(' ', btrim(p_first_name), NULLIF(btrim(p_middle_name), ''),
btrim(p_last_name)),
    btrim(p_first_name), NULLIF(btrim(p_middle_name), ''), btrim(p_last_name),
p_phone,
    p_moto_brand, p_moto_model, p_moto_variant, p_moto_year,
p_plate_number,
    p_appointment_date, p_start_time, p_notes, p_total_estimate,
    p_booking_duration_minutes, true, p_assigned_crew_id
  ) ON CONFLICT DO NOTHING
  RETURNING appointments.id, appointments.reference_code INTO
v_appointment_id, v_reference_code;
  IF v_appointment_id IS NULL THEN
    SELECT appointments.id, appointments.reference_code INTO
v_appointment_id, v_reference_code
    FROM public.appointments WHERE appointments.booking_request_id =
p_booking_request_id;
    IF FOUND THEN RETURN QUERY SELECT v_appointment_id,
v_reference_code; RETURN; END IF;
    RAISE EXCEPTION 'The requested schedule is no longer available.' USING
ERRCODE = '23P01';
  END IF;

  INSERT INTO public.appointment_services (appointment_id, service_id,
service_name, price, duration_minutes)
  SELECT v_appointment_id, service.service_id, service.service_name,
service.price, service.duration_minutes
  FROM jsonb_to_recordset(p_services) AS service(
    service_id uuid, service_name text, price numeric, duration_minutes integer
  );
  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES ('new_appointment', p_notification_title, p_notification_message,
v_appointment_id);
  RETURN QUERY SELECT v_appointment_id, v_reference_code;
END;
$$;

REVOKE ALL ON FUNCTION public.create_booking_atomic(
  text, uuid, text, text, text, text, text, integer, text, date, time, text,
  numeric, integer, uuid, jsonb, text, text, text, text, text
) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.create_booking_atomic(
  text, uuid, text, text, text, text, text, integer, text, date, time, text,
  numeric, integer, uuid, jsonb, text, text, text, text, text
) TO service_role;
```

supabase/migrations/20260920090000_add_shop_operating_hours.sql

---

```
-- Customer bookings must fall entirely within these administrator-managed
-- operating hours. The booking grid supports half-hour starts only.
ALTER TABLE public.shop_settings
  ADD COLUMN IF NOT EXISTS opening_time time NOT NULL DEFAULT
'08:00:00',
  ADD COLUMN IF NOT EXISTS closing_time time NOT NULL DEFAULT
'17:00:00';

ALTER TABLE public.shop_settings
  DROP CONSTRAINT IF EXISTS shop_settings_operating_hours_valid;

ALTER TABLE public.shop_settings
  ADD CONSTRAINT shop_settings_operating_hours_valid
  CHECK (
    opening_time < closing_time
    AND EXTRACT(MINUTE FROM opening_time) IN (0, 30)
    AND EXTRACT(MINUTE FROM closing_time) IN (0, 30)
    AND EXTRACT(SECOND FROM opening_time) = 0
    AND EXTRACT(SECOND FROM closing_time) = 0
  );
```

supabase/migrations/20260920093000_enforce_configured_shop_hours.sql

---

```
-- Keep the database's final booking guard aligned with the operating hours
-- configured in shop_settings. This protects all appointment writes, even if
-- they do not come through the customer booking page.
CREATE OR REPLACE FUNCTION
public.enforce_active_appointment_capacity()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = public
AS $$
DECLARE
  v_capacity integer;
  v_overlapping_appointments integer;
  v_start_minutes integer;
  v_opening_minutes integer;
  v_closing_minutes integer;
  v_hour_start time;
  v_opening_time time;
  v_closing_time time;
BEGIN
  -- Archived, replaced, or terminal appointments do not consume capacity.
  IF NEW.is_archived = true
     OR NEW.rescheduled_to_appointment_id IS NOT NULL
     OR NEW.status NOT IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
THEN
    RETURN NEW;
  END IF;

  SELECT opening_time, closing_time
    INTO v_opening_time, v_closing_time
  FROM public.shop_settings
  WHERE id = true;

  v_opening_time := COALESCE(v_opening_time, '08:00:00'::time);
  v_closing_time := COALESCE(v_closing_time, '17:00:00'::time);
  v_opening_minutes := extract(hour FROM v_opening_time)::integer * 60
    + extract(minute FROM v_opening_time)::integer;
  v_closing_minutes := extract(hour FROM v_closing_time)::integer * 60
    + extract(minute FROM v_closing_time)::integer;
  v_start_minutes := extract(hour FROM NEW.start_time)::integer * 60
    + extract(minute FROM NEW.start_time)::integer;

  IF v_start_minutes < v_opening_minutes
     OR v_start_minutes >= v_closing_minutes
```

```
    OR mod(v_start_minutes - v_opening_minutes, 30) <> 0
    OR v_start_minutes + NEW.booking_duration_minutes > v_closing_minutes
THEN
   RAISE EXCEPTION 'The appointment must fit within the configured shop
hours in 30-minute start intervals.'
     USING ERRCODE = '23P01';
  END IF;

  v_hour_start := make_time(extract(hour FROM NEW.start_time)::integer, 0, 0);

  -- All new or changed appointments on the same day use one transaction lock.
  -- This also covers overlapping starts that belong to adjacent hourly rows.
  PERFORM pg_advisory_xact_lock(hashtext('booking-capacity:' ||
NEW.appointment_date::text));

  SELECT slot.capacity
    INTO v_capacity
  FROM public.time_slots AS slot
  WHERE slot.is_active = true
    AND (slot.start_time = NEW.start_time OR slot.start_time = v_hour_start)
  ORDER BY (slot.start_time = NEW.start_time) DESC
  LIMIT 1
  FOR UPDATE;

  IF COALESCE(v_capacity, 0) <= 0 THEN
    RAISE EXCEPTION 'The requested time slot is not available.' USING
ERRCODE = '23P01';
  END IF;

  SELECT count(*)
    INTO v_overlapping_appointments
  FROM public.appointments AS other
  WHERE other.id <> NEW.id
    AND other.appointment_date = NEW.appointment_date
    AND other.is_archived = false
    AND other.rescheduled_to_appointment_id IS NULL
    AND other.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
    AND tsrange(
      other.appointment_date + other.start_time,
      other.appointment_date + other.start_time
        + make_interval(mins => other.booking_duration_minutes),
      '[)'
    ) && tsrange(
      NEW.appointment_date + NEW.start_time,
      NEW.appointment_date + NEW.start_time
```

```
        + make_interval(mins => NEW.booking_duration_minutes),
      '[)'
    );

  IF v_overlapping_appointments >= v_capacity THEN
    RAISE EXCEPTION 'The requested schedule is no longer available.' USING
ERRCODE = '23P01';
  END IF;

  RETURN NEW;
END;
$$;
```

supabase/migrations/20260920100000_add_archive_retention.sql

```
-- Keep archived records recoverable for 30 days, then remove them
permanently.
-- Catalog tables already record archive timestamps; the remaining archive types
-- need the same timestamp so the retention rule is based on the archive event.
ALTER TABLE public.appointments
  ADD COLUMN IF NOT EXISTS archived_at timestamptz;

ALTER TABLE public.crew_members
  ADD COLUMN IF NOT EXISTS archived_at timestamptz;

ALTER TABLE public.schedule_blocks
  ADD COLUMN IF NOT EXISTS archived_at timestamptz;

ALTER TABLE public.blocked_numbers
  ADD COLUMN IF NOT EXISTS archived_at timestamptz;

-- Older records did not retain the moment they were archived. Start their
-- 30-day recovery period when this policy is enabled rather than deleting them
-- sooner based on an unrelated created/updated timestamp.
UPDATE public.appointments
SET archived_at = now()
WHERE is_archived = true
  AND archived_at IS NULL;

UPDATE public.crew_members
SET archived_at = now()
WHERE is_archived = true
  AND archived_at IS NULL;

UPDATE public.schedule_blocks
SET archived_at = now()
WHERE is_active = false
  AND archived_at IS NULL;

UPDATE public.blocked_numbers
SET archived_at = now()
WHERE is_archived = true
  AND archived_at IS NULL;

CREATE INDEX IF NOT EXISTS appointments_archived_retention_idx
  ON public.appointments (archived_at)
  WHERE is_archived = true;
```

```
CREATE INDEX IF NOT EXISTS services_archived_retention_idx
  ON public.services (archived_at)
  WHERE is_archived = true;

CREATE INDEX IF NOT EXISTS products_archived_retention_idx
  ON public.products (archived_at)
  WHERE is_archived = true;

CREATE INDEX IF NOT EXISTS motorcycle_catalog_archived_retention_idx
  ON public.motorcycle_catalog (archived_at)
  WHERE is_archived = true;

CREATE INDEX IF NOT EXISTS crew_members_archived_retention_idx
  ON public.crew_members (archived_at)
  WHERE is_archived = true;

CREATE INDEX IF NOT EXISTS schedule_blocks_archived_retention_idx
  ON public.schedule_blocks (archived_at)
  WHERE is_active = false;

CREATE INDEX IF NOT EXISTS blocked_numbers_archived_retention_idx
  ON public.blocked_numbers (archived_at)
  WHERE is_archived = true;

-- Reuse the catalog archive timestamp trigger for archive types that use
-- is_archived. The function also clears the timestamp when a record is
-- restored, giving a later archive a fresh 30-day retention window.
DROP TRIGGER IF EXISTS appointments_archive_timestamp ON
public.appointments;
CREATE TRIGGER appointments_archive_timestamp
  BEFORE UPDATE OF is_archived ON public.appointments
  FOR EACH ROW EXECUTE FUNCTION
public.set_catalog_archive_timestamp();

DROP TRIGGER IF EXISTS crew_members_archive_timestamp ON
public.crew_members;
CREATE TRIGGER crew_members_archive_timestamp
  BEFORE UPDATE OF is_archived ON public.crew_members
  FOR EACH ROW EXECUTE FUNCTION
public.set_catalog_archive_timestamp();

DROP TRIGGER IF EXISTS blocked_numbers_archive_timestamp ON
public.blocked_numbers;
CREATE TRIGGER blocked_numbers_archive_timestamp
  BEFORE UPDATE OF is_archived ON public.blocked_numbers
```

```
  FOR EACH ROW EXECUTE FUNCTION
public.set_catalog_archive_timestamp();

CREATE OR REPLACE FUNCTION
public.set_schedule_block_archive_timestamp()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = public
AS $$
BEGIN
  IF NEW.is_active = false AND OLD.is_active = true THEN
    NEW.archived_at = now();
  ELSIF NEW.is_active = true AND OLD.is_active = false THEN
    NEW.archived_at = NULL;
  END IF;
  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS schedule_blocks_archive_timestamp ON
public.schedule_blocks;
CREATE TRIGGER schedule_blocks_archive_timestamp
  BEFORE UPDATE OF is_active ON public.schedule_blocks
  FOR EACH ROW EXECUTE FUNCTION
public.set_schedule_block_archive_timestamp();

CREATE OR REPLACE FUNCTION public.purge_expired_archived_records()
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
BEGIN
  DELETE FROM public.appointments
  WHERE is_archived = true
    AND archived_at < now() - interval '30 days';

  DELETE FROM public.services
  WHERE is_archived = true
    AND archived_at < now() - interval '30 days';

  DELETE FROM public.products
  WHERE is_archived = true
    AND archived_at < now() - interval '30 days';
```

```
  DELETE FROM public.motorcycle_catalog
  WHERE is_archived = true
    AND archived_at < now() - interval '30 days';

  DELETE FROM public.crew_members
  WHERE is_archived = true
    AND archived_at < now() - interval '30 days';

  DELETE FROM public.schedule_blocks
  WHERE is_active = false
    AND archived_at < now() - interval '30 days';

  DELETE FROM public.blocked_numbers
  WHERE is_archived = true
    AND archived_at < now() - interval '30 days';
END;
$$;

REVOKE ALL ON FUNCTION public.purge_expired_archived_records() FROM
PUBLIC;

-- Run the cleanup once per day. The guard retains compatibility with local
-- Supabase environments where pg_cron is not installed or not available.
DO $$
BEGIN
  IF EXISTS (SELECT 1 FROM pg_extension WHERE extname = 'pg_cron')
THEN
    PERFORM cron.unschedule(jobid)
    FROM cron.job
    WHERE jobname = 'purge-expired-archives';

    PERFORM cron.schedule(
      'purge-expired-archives',
      '30 0 * * *',
      'SELECT public.purge_expired_archived_records();'
    );
  END IF;
EXCEPTION
  WHEN undefined_function OR undefined_table OR insufficient_privilege THEN
    RAISE NOTICE 'pg_cron unavailable; archive retention must be run by an
external scheduler.';
END;
$$;
```

supabase/migrations/20260921113000_add_customer_cancellation_threshold_notification.sql

---

```
-- Alert administrators when a customer reaches three customer-initiated
-- cancellations in one Manila calendar month. This is an alert only: it does
-- not block the customer or alter any booking permissions.

CREATE INDEX IF NOT EXISTS appointments_cancelled_phone_month_idx
  ON public.appointments (phone, cancelled_at)
  WHERE status = 'cancelled' AND cancelled_at IS NOT NULL;

CREATE OR REPLACE FUNCTION public.cancel_public_appointment(
  p_appointment_id uuid,
  p_cancellation_notice_hours integer
)
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment public.appointments%ROWTYPE;
  v_month_start timestamptz;
  v_month_end timestamptz;
  v_month_label text;
  v_monthly_cancellation_count integer;
BEGIN
  IF p_appointment_id IS NULL
     OR p_cancellation_notice_hours IS NULL
     OR p_cancellation_notice_hours < 0
     OR p_cancellation_notice_hours > 168 THEN
    RAISE EXCEPTION 'Invalid cancellation request.';
  END IF;

  SELECT appointment.*
    INTO v_appointment
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
  FOR UPDATE;

  IF NOT FOUND THEN
    RAISE EXCEPTION 'This appointment is no longer available.';
  END IF;
  IF v_appointment.rescheduled_to_appointment_id IS NOT NULL THEN
```

```
    RAISE EXCEPTION 'This appointment has already been rescheduled. Please
use the new reference.';
  END IF;
  IF v_appointment.pending_reschedule_request_id IS NOT NULL THEN
    RAISE EXCEPTION 'A reschedule request is already awaiting the shop''s
review.';
  END IF;
  IF v_appointment.status NOT IN ('pending', 'confirmed', 'rescheduled') THEN
    RAISE EXCEPTION 'This appointment can no longer be cancelled online.
Please call the shop.';
  END IF;
  IF (v_appointment.appointment_date + v_appointment.start_time)
       - (now() AT TIME ZONE 'Asia/Manila')
       < make_interval(hours => p_cancellation_notice_hours) THEN
    RAISE EXCEPTION 'Cancellations need % hours notice. Please call the shop
instead.',
      p_cancellation_notice_hours;
  END IF;

  -- Serialize same-customer cancellations for the month so concurrent requests
  -- cannot produce duplicate third-cancellation alerts.
  v_month_start := date_trunc('month', now() AT TIME ZONE 'Asia/Manila') AT
TIME ZONE 'Asia/Manila';
  v_month_end := v_month_start + interval '1 month';
  v_month_label := to_char(now() AT TIME ZONE 'Asia/Manila', 'FMMonth
YYYY');
  PERFORM pg_advisory_xact_lock(
    hashtext(
      'customer-cancellation-threshold:'
      || COALESCE(v_appointment.phone, '')
      || ':'
      || to_char(v_month_start AT TIME ZONE 'Asia/Manila', 'YYYY-MM')
    )
  );

  UPDATE public.appointments
  SET status = 'cancelled',
      cancelled_at = now()
  WHERE id = v_appointment.id;

  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES (
    'cancelled_appointment',
    'Cancelled ' || v_appointment.reference_code,
    v_appointment.customer_name || ' cancelled their '
```

```
      || to_char(v_appointment.appointment_date, 'Mon FMDD, YYYY') || '
appointment.',
    v_appointment.id
  );

  SELECT count(*)
    INTO v_monthly_cancellation_count
  FROM public.appointments AS appointment
  WHERE appointment.phone = v_appointment.phone
    AND appointment.status = 'cancelled'
    AND appointment.cancelled_at >= v_month_start
    AND appointment.cancelled_at < v_month_end;

  -- Alert exactly once when the threshold is reached; later cancellations in
  -- the same month retain normal cancellation notifications without flooding
  -- the admin queue.
  IF v_monthly_cancellation_count = 3 THEN
    INSERT INTO public.notifications (type, title, message, appointment_id)
    VALUES (
      'customer_cancellation_threshold',
      'Customer cancellation threshold reached',
      COALESCE(NULLIF(btrim(v_appointment.customer_name), ''),
v_appointment.phone)
        || ' (' || v_appointment.phone || ') has cancelled 3 appointments in '
        || v_month_label
        || '. Review the customer''s cancellation history and decide whether to block
the customer or take no action.',
      v_appointment.id
    );
  END IF;
END;
$$;

REVOKE ALL ON FUNCTION public.cancel_public_appointment(uuid, integer)
FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.cancel_public_appointment(uuid,
integer) TO service_role;
```

supabase/migrations/20260922090000_fix_reschedule_approval_after_email_removal.sql

---

```
-- `appointments.email` was deliberately removed in 20260918100000. The prior
-- migration tried to edit this RPC's source text, but the active INSERT used a
-- different line layout and retained the obsolete column reference. Define the
-- approval path explicitly against the current appointment schema instead.
CREATE OR REPLACE FUNCTION public.review_reschedule_request(
  p_appointment_id uuid,
  p_decision text
)
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment public.appointments%ROWTYPE;
  v_assigned_crew_id uuid;
  v_duration_minutes integer;
  v_new_appointment_id uuid;
  v_new_reference_code text;
  v_reason text;
  v_original_shop_notes text;
  v_new_shop_notes text;
  v_reschedule_number integer;
  v_slot_capacity integer;
  v_overlapping_appointments integer;
  v_start_minutes integer;
  v_opening_time time;
  v_closing_time time;
  v_opening_minutes integer;
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Only administrators can review reschedule requests.';
  END IF;
  IF lower(btrim(COALESCE(p_decision, ''))) NOT IN ('confirmed', 'rejected')
THEN
    RAISE EXCEPTION 'Choose either confirmed or rejected.';
  END IF;

  SELECT appointment.*
    INTO v_appointment
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id
```

```
    AND appointment.is_archived = false
  FOR UPDATE;

  IF NOT FOUND OR v_appointment.pending_reschedule_request_id IS NULL
THEN
    RAISE EXCEPTION 'This reschedule request is no longer available.';
  END IF;

  v_reschedule_number := v_appointment.reschedule_count + 1;

  IF lower(btrim(p_decision)) = 'rejected' THEN
    INSERT INTO public.appointment_reschedule_history (
      appointment_id,
      new_appointment_id,
      reschedule_number,
      from_date,
      from_start_time,
      to_date,
      to_start_time,
      reason,
      decision,
      approved_by
    ) VALUES (
      v_appointment.id,
      NULL,
      v_reschedule_number,
      v_appointment.appointment_date,
      v_appointment.start_time,
      v_appointment.pending_reschedule_date,
      v_appointment.pending_reschedule_start_time,
      btrim(v_appointment.pending_reschedule_reason),
      'rejected',
      auth.uid()
    );

    UPDATE public.appointments
    SET pending_reschedule_request_id = NULL,
        pending_reschedule_date = NULL,
        pending_reschedule_start_time = NULL,
        pending_reschedule_reference_code = NULL,
        pending_reschedule_reason = NULL
    WHERE id = v_appointment.id;
    RETURN;
  END IF;
```

```
  IF v_appointment.reschedule_count >= 3 THEN
    RAISE EXCEPTION 'This appointment has reached the maximum of 3
reschedules.';
  END IF;
  IF v_appointment.pending_reschedule_date IS NULL
     OR v_appointment.pending_reschedule_start_time IS NULL
     OR NULLIF(btrim(v_appointment.pending_reschedule_reason), '') IS NULL
THEN
    RAISE EXCEPTION 'The reschedule request is missing its requested
schedule or reason.';
  END IF;

  v_duration_minutes := COALESCE(v_appointment.booking_duration_minutes,
75);
  v_reason := btrim(v_appointment.pending_reschedule_reason);
  v_start_minutes := extract(hour FROM
v_appointment.pending_reschedule_start_time)::integer * 60
    + extract(minute FROM
v_appointment.pending_reschedule_start_time)::integer;

  SELECT settings.opening_time, settings.closing_time
    INTO v_opening_time, v_closing_time
  FROM public.shop_settings AS settings
  ORDER BY settings.id
  LIMIT 1;
  v_opening_time := COALESCE(v_opening_time, '08:00'::time);
  v_closing_time := COALESCE(v_closing_time, '17:00'::time);
  v_opening_minutes := extract(hour FROM v_opening_time)::integer * 60
    + extract(minute FROM v_opening_time)::integer;

  IF extract(dow FROM v_appointment.pending_reschedule_date) = 0
     OR v_appointment.pending_reschedule_start_time < v_opening_time
     OR v_appointment.pending_reschedule_start_time >= v_closing_time
     OR mod(v_start_minutes - v_opening_minutes, 30) <> 0
     OR v_appointment.pending_reschedule_start_time + make_interval(mins =>
v_duration_minutes)
        > v_closing_time THEN
    RAISE EXCEPTION 'The requested schedule is outside the shop booking
hours.';
  END IF;

  SELECT
public.booking_slot_capacity(v_appointment.pending_reschedule_start_time)
    INTO v_slot_capacity;
  IF COALESCE(v_slot_capacity, 0) <= 0 THEN
```

```
    RAISE EXCEPTION 'The requested time slot is no longer available.';
  END IF;

  SELECT count(*)
    INTO v_overlapping_appointments
  FROM public.appointments AS other
  WHERE other.id <> v_appointment.id
    AND other.appointment_date = v_appointment.pending_reschedule_date
    AND other.is_archived = false
    AND other.rescheduled_to_appointment_id IS NULL
    AND other.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
    AND tsrange(
      other.appointment_date + other.start_time,
      other.appointment_date + other.start_time
        + make_interval(mins => COALESCE(other.booking_duration_minutes,
75)),
      '[)'
    ) && tsrange(
      v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
      v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
        + make_interval(mins => v_duration_minutes),
      '[)'
    );
  IF v_overlapping_appointments >= v_slot_capacity THEN
    RAISE EXCEPTION 'The requested time is no longer available.';
  END IF;

  IF EXISTS (
    SELECT 1
    FROM public.schedule_blocks AS block
    WHERE block.block_date = v_appointment.pending_reschedule_date
      AND block.is_active = true
      AND (
        block.start_time IS NULL
        OR (
          block.start_time IS NOT NULL
          AND (
            (COALESCE(block.reason, '') NOT LIKE 'RANGE:%'
              AND block.start_time = v_appointment.pending_reschedule_start_time)
            OR (COALESCE(block.reason, '') LIKE 'RANGE:%'
              AND tsrange(
                v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
```

```
              v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
                + make_interval(mins => v_duration_minutes),
              '[)'
            ) && tsrange(
              v_appointment.pending_reschedule_date + block.start_time,
              v_appointment.pending_reschedule_date
                + substring(block.reason FROM '^RANGE:([0-9]{2}:[0-9]{2})')::time,
              '[)'
            ))
        )
      )
    )
  ) THEN
    RAISE EXCEPTION 'The requested time is no longer available because the
shop schedule is blocked.';
  END IF;

  SELECT crew.id
    INTO v_assigned_crew_id
  FROM public.crew_members AS crew
  WHERE crew.is_active = true
    AND crew.is_archived = false
    AND EXISTS (
      SELECT 1
      FROM public.crew_schedules AS schedule
      WHERE schedule.crew_id = crew.id
        AND schedule.schedule_date = v_appointment.pending_reschedule_date
        AND schedule.is_working = true
        AND schedule.start_time IS NOT NULL
        AND schedule.end_time IS NOT NULL
        AND schedule.start_time <= v_appointment.pending_reschedule_start_time
        AND schedule.end_time >= v_appointment.pending_reschedule_start_time
          + make_interval(mins => v_duration_minutes)
    )
    AND NOT EXISTS (
      SELECT 1
      FROM public.crew_availability_exceptions AS exception
      WHERE exception.crew_id = crew.id
        AND exception.start_date <= v_appointment.pending_reschedule_date
        AND exception.end_date >= v_appointment.pending_reschedule_date
        AND (
          exception.is_all_day
          OR (
            exception.start_time IS NOT NULL
```

```
          AND exception.end_time IS NOT NULL
          AND tsrange(
            v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
            v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
              + make_interval(mins => v_duration_minutes),
            '[)'
          ) && tsrange(
            v_appointment.pending_reschedule_date + exception.start_time,
            v_appointment.pending_reschedule_date + exception.end_time,
            '[)'
          )
        )
      )
    )
    AND NOT EXISTS (
      SELECT 1
      FROM public.appointments AS other
      WHERE other.id <> v_appointment.id
        AND other.assigned_crew_id = crew.id
        AND other.appointment_date = v_appointment.pending_reschedule_date
        AND other.is_archived = false
        AND other.rescheduled_to_appointment_id IS NULL
        AND other.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
        AND tsrange(
          other.appointment_date + other.start_time,
          other.appointment_date + other.start_time
            + make_interval(mins => COALESCE(other.booking_duration_minutes,
75)),
          '[)'
        ) && tsrange(
          v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
          v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
            + make_interval(mins => v_duration_minutes),
          '[)'
        )
    )
  ORDER BY (crew.id = v_appointment.assigned_crew_id) DESC, crew.name,
crew.id
  LIMIT 1;

  IF v_assigned_crew_id IS NULL THEN
```

```
    RAISE EXCEPTION 'No crew member is available for the requested
appointment period.';
  END IF;

  v_new_reference_code := v_appointment.pending_reschedule_reference_code;
  IF v_new_reference_code IS NULL
     OR v_new_reference_code !~ '^FRM-[A-Z0-9]{6}$' THEN
    RAISE EXCEPTION 'The reschedule request is missing its replacement
reference.';
  END IF;

  v_original_shop_notes := concat_ws(
    E'\n',
    NULLIF(btrim(v_appointment.admin_notes), ''),
    'Rescheduled to ' || v_new_reference_code,
    'Reason for rescheduling: ' || v_reason
  );
  v_new_shop_notes := concat_ws(
    E'\n',
    'Rescheduled from ' || v_appointment.reference_code,
    'Reason for rescheduling: ' || v_reason
  );

  INSERT INTO public.appointments AS replacement (
    reference_code,
    booking_request_id,
    customer_name,
    first_name,
    middle_name,
    last_name,
    phone,
    moto_brand,
    moto_model,
    moto_variant,
    moto_year,
    plate_number,
    appointment_date,
    start_time,
    status,
    notes,
    admin_notes,
    total_estimate,
    booking_duration_minutes,
    terms_accepted,
    assigned_crew_id,
```

```
  reschedule_count,
  rescheduled_from_appointment_id
) VALUES (
  v_new_reference_code,
  gen_random_uuid(),
  v_appointment.customer_name,
  v_appointment.first_name,
  v_appointment.middle_name,
  v_appointment.last_name,
  v_appointment.phone,
  v_appointment.moto_brand,
  v_appointment.moto_model,
  v_appointment.moto_variant,
  v_appointment.moto_year,
  v_appointment.plate_number,
  v_appointment.pending_reschedule_date,
  v_appointment.pending_reschedule_start_time,
  'confirmed',
  v_appointment.notes,
  v_new_shop_notes,
  v_appointment.total_estimate,
  v_duration_minutes,
  v_appointment.terms_accepted,
  v_assigned_crew_id,
  v_appointment.reschedule_count + 1,
  v_appointment.id
)
RETURNING replacement.id INTO v_new_appointment_id;

INSERT INTO public.appointment_services (
  appointment_id,
  service_id,
  service_name,
  price,
  duration_minutes
)
SELECT
  v_new_appointment_id,
  service.service_id,
  service.service_name,
  service.price,
  service.duration_minutes
FROM public.appointment_services AS service
WHERE service.appointment_id = v_appointment.id;
```

```
  UPDATE public.appointments
  SET status = 'rescheduled',
      admin_notes = v_original_shop_notes,
      reschedule_count = v_appointment.reschedule_count + 1,
      rescheduled_to_appointment_id = v_new_appointment_id,
      pending_reschedule_request_id = NULL,
      pending_reschedule_date = NULL,
      pending_reschedule_start_time = NULL,
      pending_reschedule_reference_code = NULL,
      pending_reschedule_reason = NULL
  WHERE id = v_appointment.id;

  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES (
    'rescheduled_appointment',
    'Rescheduled booking ' || v_new_reference_code,
    v_appointment.customer_name || ' rescheduled from ' ||
v_appointment.reference_code
      || ' to ' || v_new_reference_code || '. Reason: ' || v_reason,
    v_new_appointment_id
  );

  INSERT INTO public.appointment_reschedule_history (
    appointment_id,
    new_appointment_id,
    reschedule_number,
    from_date,
    from_start_time,
    to_date,
    to_start_time,
    reason,
    decision,
    approved_by
  ) VALUES (
    v_appointment.id,
    v_new_appointment_id,
    v_reschedule_number,
    v_appointment.appointment_date,
    v_appointment.start_time,
    v_appointment.pending_reschedule_date,
    v_appointment.pending_reschedule_start_time,
    v_reason,
    'confirmed',
    auth.uid()
  );
```

```
END;
$$;

REVOKE ALL ON FUNCTION public.review_reschedule_request(uuid, text)
FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.review_reschedule_request(uuid, text)
TO authenticated;
```

supabase/migrations/20260922091000_restore_reschedule_request_tracker.sql

---

```
-- Restore the established tracker behavior after the approval-RPC correction.
-- A confirmed reschedule clears its request while linking a replacement; that
-- is not a rejected request.
CREATE OR REPLACE FUNCTION public.track_appointment_reschedule()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = public
AS $$
BEGIN
  IF NEW.appointment_date IS DISTINCT FROM OLD.appointment_date
     OR NEW.start_time IS DISTINCT FROM OLD.start_time THEN
    IF OLD.reschedule_count >= 3 THEN
      RAISE EXCEPTION 'This appointment has reached the maximum of 3
reschedules.';
    END IF;

    NEW.reschedule_count := OLD.reschedule_count + 1;
    NEW.last_reschedule_rejected_at := NULL;
    NEW.last_reschedule_rejection_message := NULL;
  ELSIF OLD.pending_reschedule_request_id IS NULL
     AND NEW.pending_reschedule_request_id IS NOT NULL THEN
    NEW.last_reschedule_rejected_at := NULL;
    NEW.last_reschedule_rejection_message := NULL;
  ELSIF OLD.pending_reschedule_request_id IS NOT NULL
     AND NEW.pending_reschedule_request_id IS NULL THEN
    IF NEW.status = 'rescheduled' AND NEW.rescheduled_to_appointment_id IS
NOT NULL THEN
      NEW.last_reschedule_rejected_at := NULL;
      NEW.last_reschedule_rejection_message := NULL;
    ELSE
      NEW.last_reschedule_rejected_at := now();
      NEW.last_reschedule_rejection_message :=
        'Reschedule Request Rejected. Your requested reschedule was not
approved. Your original appointment remains unchanged and reserved.';
    END IF;
  END IF;

  RETURN NEW;
END;
$$;
```

supabase/migrations/20260922100000_add_multi_day_booking_continuations.sql

---

```
-- A multi-day service remains one customer appointment, while each later work
-- period has its own reservation so it participates in capacity calculations.
CREATE TABLE IF NOT EXISTS public.appointment_continuations (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  appointment_id uuid NOT NULL REFERENCES public.appointments(id) ON
DELETE CASCADE,
  segment_number integer NOT NULL CHECK (segment_number > 0),
  appointment_date date NOT NULL,
  start_time time NOT NULL,
  booking_duration_minutes integer NOT NULL CHECK
(booking_duration_minutes > 0),
  assigned_crew_id uuid NOT NULL REFERENCES public.crew_members(id),
  created_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (appointment_id, segment_number)
);

CREATE INDEX IF NOT EXISTS appointment_continuations_schedule_idx
  ON public.appointment_continuations (appointment_date, start_time);

CREATE INDEX IF NOT EXISTS
appointment_continuations_crew_schedule_idx
  ON public.appointment_continuations (assigned_crew_id, appointment_date,
start_time);

GRANT SELECT, INSERT, UPDATE, DELETE ON
public.appointment_continuations TO authenticated;
GRANT ALL ON public.appointment_continuations TO service_role;
ALTER TABLE public.appointment_continuations ENABLE ROW LEVEL
SECURITY;
DROP POLICY IF EXISTS "appointment continuations admin all" ON
public.appointment_continuations;
CREATE POLICY "appointment continuations admin all"
  ON public.appointment_continuations FOR ALL TO authenticated
  USING (public.has_role(auth.uid(), 'admin'))
  WITH CHECK (public.has_role(auth.uid(), 'admin'));

CREATE OR REPLACE FUNCTION
public.enforce_active_appointment_continuation_capacity()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = public
AS $$
```

```
DECLARE
  v_parent public.appointments%ROWTYPE;
  v_opening_time time;
  v_closing_time time;
  v_opening_minutes integer;
  v_closing_minutes integer;
  v_start_minutes integer;
  v_hour_start time;
  v_capacity integer;
  v_overlapping_reservations integer;
BEGIN
  SELECT * INTO v_parent FROM public.appointments WHERE id =
NEW.appointment_id;
  IF NOT FOUND THEN
    RAISE EXCEPTION 'The parent appointment no longer exists.' USING
ERRCODE = '23503';
  END IF;

  -- A cancelled, archived, or replaced appointment releases every later-day
  -- reservation without destroying the audit trail.
  IF v_parent.is_archived = true
     OR v_parent.rescheduled_to_appointment_id IS NOT NULL
     OR v_parent.status NOT IN ('pending', 'confirmed', 'in_progress',
'rescheduled') THEN
    RETURN NEW;
  END IF;

  SELECT opening_time, closing_time INTO v_opening_time, v_closing_time
  FROM public.shop_settings WHERE id = true;
  v_opening_time := COALESCE(v_opening_time, '08:00:00'::time);
  v_closing_time := COALESCE(v_closing_time, '17:00:00'::time);
  v_opening_minutes := extract(hour FROM v_opening_time)::integer * 60
    + extract(minute FROM v_opening_time)::integer;
  v_closing_minutes := extract(hour FROM v_closing_time)::integer * 60
    + extract(minute FROM v_closing_time)::integer;
  v_start_minutes := extract(hour FROM NEW.start_time)::integer * 60
    + extract(minute FROM NEW.start_time)::integer;

  IF extract(dow FROM NEW.appointment_date) = 0
     OR v_start_minutes < v_opening_minutes
     OR v_start_minutes >= v_closing_minutes
     OR mod(v_start_minutes - v_opening_minutes, 30) <> 0
     OR v_start_minutes + NEW.booking_duration_minutes > v_closing_minutes
THEN
    RAISE EXCEPTION 'The continuation must fit within the configured shop
```

```
hours in 30-minute start intervals.'
      USING ERRCODE = '23P01';
  END IF;

  IF NOT EXISTS (
    SELECT 1
    FROM public.crew_members AS crew
    JOIN public.crew_schedules AS schedule ON schedule.crew_id = crew.id
    WHERE crew.id = NEW.assigned_crew_id
      AND crew.is_active = true
      AND crew.is_archived = false
      AND schedule.schedule_date = NEW.appointment_date
      AND schedule.is_working = true
      AND schedule.start_time <= NEW.start_time
      AND schedule.end_time >= NEW.start_time + make_interval(mins =>
NEW.booking_duration_minutes)
  ) THEN
    RAISE EXCEPTION 'No mechanic is available for the continuation schedule.'
USING ERRCODE = '23P01';
  END IF;

  IF EXISTS (
    SELECT 1
    FROM public.crew_availability_exceptions AS exception
    WHERE exception.crew_id = NEW.assigned_crew_id
      AND exception.start_date <= NEW.appointment_date
      AND exception.end_date >= NEW.appointment_date
      AND (
        exception.is_all_day
        OR (
          exception.start_time IS NOT NULL
          AND exception.end_time IS NOT NULL
          AND tsrange(
            NEW.appointment_date + NEW.start_time,
            NEW.appointment_date + NEW.start_time
              + make_interval(mins => NEW.booking_duration_minutes),
            '[)'
          ) && tsrange(
            NEW.appointment_date + exception.start_time,
            NEW.appointment_date + exception.end_time,
            '[)'
          )
        )
      )
  ) THEN
```

```
    RAISE EXCEPTION 'The assigned mechanic is unavailable for the
continuation schedule.'
      USING ERRCODE = '23P01';
  END IF;

  IF EXISTS (
    SELECT 1
    FROM public.schedule_blocks AS block
    WHERE block.block_date = NEW.appointment_date
      AND block.is_active = true
      AND (
        block.start_time IS NULL
        OR (
          COALESCE(block.reason, '') NOT LIKE 'RANGE:%'
          AND block.start_time = NEW.start_time
        )
        OR (
          COALESCE(block.reason, '') LIKE 'RANGE:%'
          AND tsrange(
            NEW.appointment_date + NEW.start_time,
            NEW.appointment_date + NEW.start_time
              + make_interval(mins => NEW.booking_duration_minutes),
            '[)'
          ) && tsrange(
            NEW.appointment_date + block.start_time,
            NEW.appointment_date
              + substring(block.reason FROM '^RANGE:([0-9]{2}:[0-9]{2})')::time,
            '[)'
          )
        )
      )
  ) THEN
    RAISE EXCEPTION 'The shop is closed for the continuation schedule.' USING
ERRCODE = '23P01';
  END IF;

  v_hour_start := make_time(extract(hour FROM NEW.start_time)::integer, 0, 0);
  PERFORM pg_advisory_xact_lock(hashtext('booking-capacity:' ||
NEW.appointment_date::text));

  SELECT slot.capacity INTO v_capacity
  FROM public.time_slots AS slot
  WHERE slot.is_active = true
    AND (slot.start_time = NEW.start_time OR slot.start_time = v_hour_start)
  ORDER BY (slot.start_time = NEW.start_time) DESC
```

```
  LIMIT 1
  FOR UPDATE;

  IF COALESCE(v_capacity, 0) <= 0 THEN
    RAISE EXCEPTION 'The requested continuation time slot is not available.'
USING ERRCODE = '23P01';
  END IF;

  SELECT count(*) INTO v_overlapping_reservations
  FROM (
    SELECT appointment.assigned_crew_id, appointment.appointment_date,
appointment.start_time,
      appointment.booking_duration_minutes
    FROM public.appointments AS appointment
    WHERE appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
      AND appointment.appointment_date = NEW.appointment_date
    UNION ALL
    SELECT continuation.assigned_crew_id, continuation.appointment_date,
continuation.start_time,
      continuation.booking_duration_minutes
    FROM public.appointment_continuations AS continuation
    JOIN public.appointments AS appointment ON appointment.id =
continuation.appointment_id
    WHERE continuation.id <> NEW.id
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
      AND continuation.appointment_date = NEW.appointment_date
  ) AS reservation
  WHERE tsrange(
    reservation.appointment_date + reservation.start_time,
    reservation.appointment_date + reservation.start_time
      + make_interval(mins => reservation.booking_duration_minutes),
    '[)'
  ) && tsrange(
    NEW.appointment_date + NEW.start_time,
    NEW.appointment_date + NEW.start_time + make_interval(mins =>
NEW.booking_duration_minutes),
    '[)'
  );
```

```
  IF v_overlapping_reservations >= v_capacity THEN
    RAISE EXCEPTION 'The requested continuation schedule is no longer
available.' USING ERRCODE = '23P01';
  END IF;

  IF EXISTS (
    SELECT 1
    FROM public.appointments AS appointment
    WHERE appointment.assigned_crew_id = NEW.assigned_crew_id
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
      AND appointment.appointment_date = NEW.appointment_date
      AND tsrange(
        appointment.appointment_date + appointment.start_time,
        appointment.appointment_date + appointment.start_time
          + make_interval(mins => appointment.booking_duration_minutes),
        '[)'
      ) && tsrange(
        NEW.appointment_date + NEW.start_time,
        NEW.appointment_date + NEW.start_time
          + make_interval(mins => NEW.booking_duration_minutes),
        '[)'
      )
  ) OR EXISTS (
    SELECT 1
    FROM public.appointment_continuations AS continuation
    JOIN public.appointments AS appointment ON appointment.id =
continuation.appointment_id
    WHERE continuation.id <> NEW.id
      AND continuation.assigned_crew_id = NEW.assigned_crew_id
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
      AND continuation.appointment_date = NEW.appointment_date
      AND tsrange(
        continuation.appointment_date + continuation.start_time,
        continuation.appointment_date + continuation.start_time
          + make_interval(mins => continuation.booking_duration_minutes),
        '[)'
      ) && tsrange(
        NEW.appointment_date + NEW.start_time,
        NEW.appointment_date + NEW.start_time
```

```
        + make_interval(mins => NEW.booking_duration_minutes),
       '[)'
     )
  ) THEN
    RAISE EXCEPTION 'The assigned mechanic is no longer available for the
continuation schedule.'
      USING ERRCODE = '23P01';
  END IF;

  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS
appointment_continuations_enforce_active_capacity
  ON public.appointment_continuations;
CREATE TRIGGER appointment_continuations_enforce_active_capacity
  BEFORE INSERT OR UPDATE OF appointment_date, start_time,
booking_duration_minutes, assigned_crew_id
  ON public.appointment_continuations
  FOR EACH ROW EXECUTE FUNCTION
public.enforce_active_appointment_continuation_capacity();

-- Existing single-day appointments must also see continuation reservations
when
-- checking capacity, otherwise a later booking could overlap a carried-forward
-- work period.
CREATE OR REPLACE FUNCTION
public.enforce_active_appointment_capacity()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = public
AS $$
DECLARE
  v_capacity integer;
  v_overlapping_appointments integer;
  v_start_minutes integer;
  v_opening_minutes integer;
  v_closing_minutes integer;
  v_hour_start time;
  v_opening_time time;
  v_closing_time time;
BEGIN
  IF NEW.is_archived = true
     OR NEW.rescheduled_to_appointment_id IS NOT NULL
```

```
    OR NEW.status NOT IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
THEN
   RETURN NEW;
  END IF;

  SELECT opening_time, closing_time INTO v_opening_time, v_closing_time
  FROM public.shop_settings WHERE id = true;
  v_opening_time := COALESCE(v_opening_time, '08:00:00'::time);
  v_closing_time := COALESCE(v_closing_time, '17:00:00'::time);
  v_opening_minutes := extract(hour FROM v_opening_time)::integer * 60
    + extract(minute FROM v_opening_time)::integer;
  v_closing_minutes := extract(hour FROM v_closing_time)::integer * 60
    + extract(minute FROM v_closing_time)::integer;
  v_start_minutes := extract(hour FROM NEW.start_time)::integer * 60
    + extract(minute FROM NEW.start_time)::integer;

  IF v_start_minutes < v_opening_minutes
     OR v_start_minutes >= v_closing_minutes
     OR mod(v_start_minutes - v_opening_minutes, 30) <> 0
     OR v_start_minutes + NEW.booking_duration_minutes > v_closing_minutes
THEN
   RAISE EXCEPTION 'The appointment must fit within the configured shop
hours in 30-minute start intervals.'
     USING ERRCODE = '23P01';
  END IF;

  v_hour_start := make_time(extract(hour FROM NEW.start_time)::integer, 0, 0);
  PERFORM pg_advisory_xact_lock(hashtext('booking-capacity:' ||
NEW.appointment_date::text));

  SELECT slot.capacity INTO v_capacity
  FROM public.time_slots AS slot
  WHERE slot.is_active = true
    AND (slot.start_time = NEW.start_time OR slot.start_time = v_hour_start)
  ORDER BY (slot.start_time = NEW.start_time) DESC
  LIMIT 1 FOR UPDATE;

  IF COALESCE(v_capacity, 0) <= 0 THEN
   RAISE EXCEPTION 'The requested time slot is not available.' USING
ERRCODE = '23P01';
  END IF;

  SELECT count(*) INTO v_overlapping_appointments
  FROM (
    SELECT appointment.appointment_date, appointment.start_time,
```

```
appointment.booking_duration_minutes
    FROM public.appointments AS appointment
    WHERE appointment.id <> NEW.id
      AND appointment.appointment_date = NEW.appointment_date
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
    UNION ALL
    SELECT continuation.appointment_date, continuation.start_time,
continuation.booking_duration_minutes
    FROM public.appointment_continuations AS continuation
    JOIN public.appointments AS appointment ON appointment.id =
continuation.appointment_id
    WHERE continuation.appointment_date = NEW.appointment_date
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
  ) AS reservation
  WHERE tsrange(
    reservation.appointment_date + reservation.start_time,
    reservation.appointment_date + reservation.start_time
      + make_interval(mins => reservation.booking_duration_minutes),
    '[)'
  ) && tsrange(
    NEW.appointment_date + NEW.start_time,
    NEW.appointment_date + NEW.start_time + make_interval(mins =>
NEW.booking_duration_minutes),
    '[)'
  );

  IF v_overlapping_appointments >= v_capacity THEN
    RAISE EXCEPTION 'The requested schedule is no longer available.' USING
ERRCODE = '23P01';
  END IF;
  RETURN NEW;
END;
$$;

DROP FUNCTION IF EXISTS public.create_booking_atomic(
  text, uuid, text, text, text, text, text, integer, text, date, time, text,
  numeric, integer, uuid, jsonb, text, text, text, text, text
);
```

```
CREATE FUNCTION public.create_booking_atomic(
  p_reference_code text, p_booking_request_id uuid, p_customer_name text,
  p_phone text, p_moto_brand text, p_moto_model text, p_moto_variant text,
  p_moto_year integer, p_plate_number text, p_appointment_date date,
  p_start_time time, p_notes text, p_total_estimate numeric,
  p_booking_duration_minutes integer, p_assigned_crew_id uuid, p_services
jsonb,
  p_continuation_segments jsonb, p_notification_title text,
p_notification_message text,
  p_first_name text, p_middle_name text, p_last_name text
)
RETURNS TABLE (appointment_id uuid, reference_code text)
LANGUAGE plpgsql SECURITY INVOKER SET search_path = public
AS $$
DECLARE v_appointment_id uuid; v_reference_code text;
BEGIN
  IF p_booking_request_id IS NULL THEN RAISE EXCEPTION 'A booking
request ID is required.'; END IF;
  IF COALESCE(btrim(p_first_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$'
     OR COALESCE(btrim(p_last_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$' THEN
    RAISE EXCEPTION 'First and last names must contain letters only.';
  END IF;
  IF COALESCE(btrim(p_middle_name), '') <> ''
     AND btrim(p_middle_name) !~ '^[[:alpha:]]+( [[:alpha:]]+)*$' THEN
    RAISE EXCEPTION 'Middle name must contain letters only.';
  END IF;
  IF jsonb_typeof(p_services) <> 'array' OR jsonb_array_length(p_services) = 0
THEN
    RAISE EXCEPTION 'A booking must include at least one service.';
  END IF;
  IF jsonb_typeof(COALESCE(p_continuation_segments, '[]'::jsonb)) <> 'array'
THEN
    RAISE EXCEPTION 'Continuation segments must be an array.';
  END IF;

  INSERT INTO public.appointments (
    reference_code, booking_request_id, customer_name, first_name,
middle_name,
    last_name, phone, moto_brand, moto_model, moto_variant, moto_year,
    plate_number, appointment_date, start_time, notes, total_estimate,
    booking_duration_minutes, terms_accepted, assigned_crew_id
  ) VALUES (
    p_reference_code, p_booking_request_id,
    concat_ws(' ', btrim(p_first_name), NULLIF(btrim(p_middle_name), ''),
btrim(p_last_name)),
```

```
    btrim(p_first_name), NULLIF(btrim(p_middle_name), ''), btrim(p_last_name),
p_phone,
    p_moto_brand, p_moto_model, p_moto_variant, p_moto_year,
p_plate_number,
    p_appointment_date, p_start_time, p_notes, p_total_estimate,
    p_booking_duration_minutes, true, p_assigned_crew_id
  ) ON CONFLICT DO NOTHING
  RETURNING appointments.id, appointments.reference_code INTO
v_appointment_id, v_reference_code;

  IF v_appointment_id IS NULL THEN
    SELECT appointments.id, appointments.reference_code INTO
v_appointment_id, v_reference_code
    FROM public.appointments WHERE appointments.booking_request_id =
p_booking_request_id;
    IF FOUND THEN RETURN QUERY SELECT v_appointment_id,
v_reference_code; RETURN; END IF;
    RAISE EXCEPTION 'The requested schedule is no longer available.' USING
ERRCODE = '23P01';
  END IF;

  INSERT INTO public.appointment_services (appointment_id, service_id,
service_name, price, duration_minutes)
  SELECT v_appointment_id, service.service_id, service.service_name,
service.price, service.duration_minutes
  FROM jsonb_to_recordset(p_services) AS service(
    service_id uuid, service_name text, price numeric, duration_minutes integer
  );

  INSERT INTO public.appointment_continuations (
    appointment_id, segment_number, appointment_date, start_time,
    booking_duration_minutes, assigned_crew_id
  )
  SELECT v_appointment_id, segment.segment_number,
segment.appointment_date, segment.start_time,
    segment.booking_duration_minutes, segment.assigned_crew_id
  FROM jsonb_to_recordset(COALESCE(p_continuation_segments, '[]'::jsonb))
AS segment(
    segment_number integer, appointment_date date, start_time time,
    booking_duration_minutes integer, assigned_crew_id uuid
  );

  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES ('new_appointment', p_notification_title, p_notification_message,
v_appointment_id);
```

```
  RETURN QUERY SELECT v_appointment_id, v_reference_code;
END;
$$;

REVOKE ALL ON FUNCTION public.create_booking_atomic(
  text, uuid, text, text, text, text, text, integer, text, date, time, text,
  numeric, integer, uuid, jsonb, jsonb, text, text, text, text, text
) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.create_booking_atomic(
  text, uuid, text, text, text, text, text, integer, text, date, time, text,
  numeric, integer, uuid, jsonb, jsonb, text, text, text, text, text
) TO service_role;
```

supabase/migrations/20260923100000_align_admin_appointment_contact_validation.sql

---

```
-- Keep administrative appointment edits consistent with public booking:
-- middle names are optional and mobile numbers are stored in canonical E.164
form.

DROP FUNCTION IF EXISTS public.update_appointment_details_atomic(
  uuid, uuid[], date, time, uuid, boolean, text, text, text, text, text
);

CREATE FUNCTION public.update_appointment_details_atomic(
  p_appointment_id uuid,
  p_service_ids uuid[],
  p_appointment_date date,
  p_start_time time,
  p_assigned_crew_id uuid,
  p_crew_assignment_manual boolean,
  p_status text,
  p_admin_notes text,
  p_first_name text,
  p_middle_name text,
  p_last_name text,
  p_phone text
)
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
BEGIN
  IF COALESCE(btrim(p_first_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$'
     OR COALESCE(btrim(p_last_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$' THEN
    RAISE EXCEPTION 'First and last names must contain letters only.';
  END IF;

  IF COALESCE(btrim(p_middle_name), '') <> ''
     AND btrim(p_middle_name) !~ '^[[:alpha:]]+( [[:alpha:]]+)*$' THEN
    RAISE EXCEPTION 'Middle name must contain letters only.';
  END IF;

  IF COALESCE(btrim(p_phone), '') !~ '^\+639[0-9]{9}$' THEN
    RAISE EXCEPTION 'Enter a valid Philippine mobile number.';
  END IF;
```

```
  PERFORM public.update_appointment_details_atomic(
    p_appointment_id,
    p_service_ids,
    p_appointment_date,
    p_start_time,
    p_assigned_crew_id,
    p_crew_assignment_manual,
    p_status,
    p_admin_notes
  );

  UPDATE public.appointments
  SET first_name = btrim(p_first_name),
      middle_name = NULLIF(btrim(p_middle_name), ''),
      last_name = btrim(p_last_name),
      phone = btrim(p_phone),
      customer_name = concat_ws(
        ' ',
        btrim(p_first_name),
        NULLIF(btrim(p_middle_name), ''),
        btrim(p_last_name)
      )
  WHERE id = p_appointment_id
    AND is_archived = false;
END;
$$;

REVOKE ALL ON FUNCTION public.update_appointment_details_atomic(
  uuid, uuid[], date, time, uuid, boolean, text, text, text, text, text, text
) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.update_appointment_details_atomic(
  uuid, uuid[], date, time, uuid, boolean, text, text, text, text, text, text
) TO authenticated;
```

supabase/migrations/20260924100000_add_appointment_service_progress.sql

---

```
-- Persist actual service timing and the small amount of state needed to make
-- service-progress notifications durable across refreshes and admin sessions.
ALTER TABLE public.appointments
  ADD COLUMN IF NOT EXISTS service_started_at timestamptz,
  ADD COLUMN IF NOT EXISTS service_ended_at timestamptz,
  ADD COLUMN IF NOT EXISTS arrival_notification_snooze_count integer NOT
NULL DEFAULT 0,
  ADD COLUMN IF NOT EXISTS arrival_notification_snoozed_until timestamptz,
  ADD COLUMN IF NOT EXISTS completion_notification_snoozed_until
timestamptz;

DO $$
BEGIN
  IF NOT EXISTS (
    SELECT 1
    FROM pg_constraint
    WHERE conname = 'appointments_arrival_notification_snooze_count_check'
      AND conrelid = 'public.appointments'::regclass
  ) THEN
    ALTER TABLE public.appointments
      ADD CONSTRAINT appointments_arrival_notification_snooze_count_check
      CHECK (arrival_notification_snooze_count BETWEEN 0 AND 3);
  END IF;
END;
$$;

DO $$
BEGIN
  IF NOT EXISTS (
    SELECT 1
    FROM pg_constraint
    WHERE conname = 'appointments_service_progress_timestamp_check'
      AND conrelid = 'public.appointments'::regclass
  ) THEN
    ALTER TABLE public.appointments
      ADD CONSTRAINT appointments_service_progress_timestamp_check
      CHECK (
        service_ended_at IS NULL
        OR (service_started_at IS NOT NULL AND service_ended_at >=
service_started_at)
      );
  END IF;
```

```
END;
$$;

COMMENT ON COLUMN public.appointments.service_started_at IS
  'Actual server-recorded start time of the appointment service.';
COMMENT ON COLUMN public.appointments.service_ended_at IS
  'Actual server-recorded completion time of the appointment service.';

CREATE INDEX IF NOT EXISTS appointments_service_progress_arrival_idx
  ON public.appointments (appointment_date, start_time)
  WHERE is_archived = false AND status = 'confirmed' AND service_started_at
IS NULL;

CREATE INDEX IF NOT EXISTS
appointments_service_progress_completion_idx
  ON public.appointments (service_started_at)
  WHERE is_archived = false
    AND status = 'in_progress'
    AND service_started_at IS NOT NULL
    AND service_ended_at IS NULL;

-- Only one currently actionable alert of each progress type may exist for an
-- appointment. This is the final guard against concurrent admin sessions or
-- overlapping client polling intervals creating duplicate notifications.
CREATE UNIQUE INDEX IF NOT EXISTS
notifications_active_service_progress_unique
  ON public.notifications (appointment_id, type)
  WHERE is_read = false
    AND type IN ('service_arrival', 'service_completion');

-- A manual status edit must have the same timestamp behavior as an action
-- selected from the notification inbox.
CREATE OR REPLACE FUNCTION
public.track_appointment_service_progress()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = public
AS $$
BEGIN
  IF NEW.status IS NOT DISTINCT FROM OLD.status THEN
    RETURN NEW;
  END IF;

  IF NEW.status = 'in_progress' THEN
    IF OLD.service_ended_at IS NOT NULL THEN
```

```
      RAISE EXCEPTION 'A completed service cannot be restarted.';
    END IF;

    NEW.service_started_at := COALESCE(NEW.service_started_at, now());
    NEW.service_ended_at := NULL;
  ELSIF NEW.status = 'completed' THEN
    IF NEW.service_started_at IS NULL THEN
      RAISE EXCEPTION 'Start the service before marking the appointment
completed.';
    END IF;

    NEW.service_ended_at := COALESCE(NEW.service_ended_at, now());
  ELSIF NEW.status = 'no_show' THEN
    IF NEW.service_started_at IS NOT NULL THEN
      RAISE EXCEPTION 'An appointment with a recorded service start cannot be
marked as no-show.';
    END IF;
    IF NEW.arrival_notification_snooze_count < 3 THEN
      RAISE EXCEPTION 'An appointment can be marked as no-show after three
arrival snoozes.';
    END IF;
  ELSIF OLD.service_started_at IS NOT NULL
    AND NEW.status IN ('pending', 'confirmed', 'rescheduled', 'cancelled',
'rejected') THEN
    RAISE EXCEPTION 'A started service cannot return to a pre-service
appointment status.';
  END IF;

  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS appointments_track_service_progress ON
public.appointments;
CREATE TRIGGER appointments_track_service_progress
  BEFORE UPDATE OF status ON public.appointments
  FOR EACH ROW EXECUTE FUNCTION
public.track_appointment_service_progress();

-- Resolve outstanding workflow alerts when an appointment is changed outside
-- the dedicated progress RPC (for example, from Edit Appointment).
CREATE OR REPLACE FUNCTION
public.resolve_appointment_service_progress_notifications()
RETURNS trigger
LANGUAGE plpgsql
```

```
SECURITY DEFINER
SET search_path = public
AS $$
BEGIN
  IF NEW.is_archived = true
     OR NEW.status NOT IN ('confirmed', 'in_progress') THEN
    UPDATE public.notifications
    SET is_read = true
    WHERE appointment_id = NEW.id
      AND is_read = false
      AND type IN ('service_arrival', 'service_completion');
  ELSIF NEW.status = 'in_progress' THEN
    UPDATE public.notifications
    SET is_read = true
    WHERE appointment_id = NEW.id
      AND is_read = false
      AND type = 'service_arrival';
  END IF;

  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS
appointments_resolve_service_progress_notifications ON public.appointments;
CREATE TRIGGER appointments_resolve_service_progress_notifications
  AFTER UPDATE OF status, is_archived ON public.appointments
  FOR EACH ROW EXECUTE FUNCTION
public.resolve_appointment_service_progress_notifications();

-- Keep activity logs focused on the administrator's decisions. The appointment
-- update trigger still records starts, snoozes, no-shows, and completions;
-- this only suppresses automatic inbox rows created by the scheduler.
DROP TRIGGER IF EXISTS admin_activity_notifications ON public.notifications;
CREATE TRIGGER admin_activity_notifications
  AFTER INSERT OR UPDATE ON public.notifications
  FOR EACH ROW
  WHEN (NEW.type NOT IN ('service_arrival', 'service_completion'))
  EXECUTE FUNCTION public.record_admin_activity();

DROP TRIGGER IF EXISTS admin_activity_notifications_delete ON
public.notifications;
CREATE TRIGGER admin_activity_notifications_delete
  AFTER DELETE ON public.notifications
  FOR EACH ROW
```

```
  WHEN (OLD.type NOT IN ('service_arrival', 'service_completion'))
  EXECUTE FUNCTION public.record_admin_activity();

-- Insert alerts only once they are due. This function is intentionally
-- idempotent and uses server time, so browser clocks and repeated polls cannot
-- change the workflow's timing.
CREATE OR REPLACE FUNCTION
public.sync_service_progress_notifications()
RETURNS integer
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_created integer := 0;
  v_now timestamptz := now();
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Only administrators can synchronize service progress
notifications.';
  END IF;

  WITH inserted AS (
    INSERT INTO public.notifications (type, title, message, appointment_id)
    SELECT
      'service_arrival',
      'Customer expected: ' || appointment.reference_code,
      appointment.customer_name || ' is scheduled to arrive for service now.',
      appointment.id
    FROM public.appointments AS appointment
    WHERE appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status = 'confirmed'
      AND appointment.service_started_at IS NULL
      AND (appointment.appointment_date + appointment.start_time) AT TIME
ZONE 'Asia/Manila' <= v_now
      AND COALESCE(appointment.arrival_notification_snoozed_until, v_now) <=
v_now
      AND NOT EXISTS (
        SELECT 1
        FROM public.notifications AS notification
        WHERE notification.appointment_id = appointment.id
          AND notification.type = 'service_arrival'
          AND notification.is_read = false
      )
```

```
    ON CONFLICT DO NOTHING
    RETURNING 1
  )
  SELECT count(*) INTO v_created FROM inserted;

  WITH inserted AS (
    INSERT INTO public.notifications (type, title, message, appointment_id)
    SELECT
      'service_completion',
      'Service duration ended: ' || appointment.reference_code,
      'The expected service duration for ' || appointment.customer_name || ' has
ended.',
      appointment.id
    FROM public.appointments AS appointment
    JOIN LATERAL (
      SELECT sum(appointment_service.duration_minutes)::integer AS
service_duration_minutes
      FROM public.appointment_services AS appointment_service
      WHERE appointment_service.appointment_id = appointment.id
    ) AS service_duration ON service_duration.service_duration_minutes > 0
    WHERE appointment.is_archived = false
      AND appointment.status = 'in_progress'
      AND appointment.service_started_at IS NOT NULL
      AND appointment.service_ended_at IS NULL
      AND appointment.service_started_at
        + make_interval(mins => service_duration.service_duration_minutes) <=
v_now
      AND COALESCE(appointment.completion_notification_snoozed_until,
v_now) <= v_now
      AND NOT EXISTS (
        SELECT 1
        FROM public.notifications AS notification
        WHERE notification.appointment_id = appointment.id
          AND notification.type = 'service_completion'
          AND notification.is_read = false
      )
    ON CONFLICT DO NOTHING
    RETURNING 1
  )
  SELECT v_created + count(*) INTO v_created FROM inserted;

  RETURN v_created;
END;
$$;
```

```
REVOKE ALL ON FUNCTION public.sync_service_progress_notifications()
FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.sync_service_progress_notifications()
TO authenticated;

-- Keep every operator decision and its notification state update in one row
-- lock-protected transaction. The five-minute value is the requested snooze
-- period; all service duration calculations come from persisted services.
CREATE OR REPLACE FUNCTION
public.manage_appointment_service_progress(
  p_appointment_id uuid,
  p_action text
)
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment public.appointments%ROWTYPE;
  v_action text := lower(btrim(COALESCE(p_action, '')));
  v_snooze_interval interval := interval '5 minutes';
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Only administrators can update service progress.';
  END IF;

  IF v_action NOT IN (
    'start', 'snooze_arrival', 'no_show', 'complete', 'snooze_completion'
  ) THEN
    RAISE EXCEPTION 'Choose a valid service progress action.';
  END IF;

  SELECT appointment.*
    INTO v_appointment
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
  FOR UPDATE;

  IF NOT FOUND THEN
    RAISE EXCEPTION 'This appointment is no longer available.';
  END IF;

  IF v_action = 'start' THEN
```

```
    IF v_appointment.status NOT IN ('pending', 'confirmed')
       OR v_appointment.service_started_at IS NOT NULL THEN
      RAISE EXCEPTION 'This appointment cannot be started.';
    END IF;

    UPDATE public.appointments
    SET status = 'in_progress',
        service_started_at = COALESCE(service_started_at, now()),
        service_ended_at = NULL,
        arrival_notification_snoozed_until = NULL
    WHERE id = v_appointment.id;

  ELSIF v_action = 'snooze_arrival' THEN
    IF v_appointment.status <> 'confirmed' OR v_appointment.service_started_at
IS NOT NULL THEN
      RAISE EXCEPTION 'This arrival reminder is no longer available.';
    END IF;
    IF v_appointment.arrival_notification_snooze_count >= 3 THEN
      RAISE EXCEPTION 'The maximum of three arrival snoozes has been
reached.';
    END IF;

    UPDATE public.appointments
    SET arrival_notification_snooze_count = arrival_notification_snooze_count +
1,
        arrival_notification_snoozed_until = now() + v_snooze_interval
    WHERE id = v_appointment.id;

    UPDATE public.notifications
    SET is_read = true
    WHERE appointment_id = v_appointment.id
      AND type = 'service_arrival'
      AND is_read = false;

  ELSIF v_action = 'no_show' THEN
    IF v_appointment.status <> 'confirmed' OR v_appointment.service_started_at
IS NOT NULL THEN
      RAISE EXCEPTION 'This appointment cannot be marked as no-show.';
    END IF;
    IF v_appointment.arrival_notification_snooze_count < 3 THEN
      RAISE EXCEPTION 'The appointment can be marked as no-show after three
arrival snoozes.';
    END IF;

    UPDATE public.appointments
```

```
    SET status = 'no_show',
        arrival_notification_snoozed_until = NULL
    WHERE id = v_appointment.id;

  ELSIF v_action = 'complete' THEN
    IF v_appointment.status <> 'in_progress'
       OR v_appointment.service_started_at IS NULL
       OR v_appointment.service_ended_at IS NOT NULL THEN
      RAISE EXCEPTION 'This service cannot be marked as completed.';
    END IF;

    UPDATE public.appointments
    SET status = 'completed',
        service_ended_at = COALESCE(service_ended_at, now()),
        completion_notification_snoozed_until = NULL
    WHERE id = v_appointment.id;

  ELSE
    IF v_appointment.status <> 'in_progress'
       OR v_appointment.service_started_at IS NULL
       OR v_appointment.service_ended_at IS NOT NULL THEN
      RAISE EXCEPTION 'This completion reminder is no longer available.';
    END IF;

    UPDATE public.appointments
    SET completion_notification_snoozed_until = now() + v_snooze_interval
    WHERE id = v_appointment.id;

    UPDATE public.notifications
    SET is_read = true
    WHERE appointment_id = v_appointment.id
      AND type = 'service_completion'
      AND is_read = false;
  END IF;
END;
$$;

REVOKE ALL ON FUNCTION
public.manage_appointment_service_progress(uuid, text) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION
public.manage_appointment_service_progress(uuid, text) TO authenticated;
```

supabase/migrations/20260924110000_harden_service_progress_notification_race.sql

---

```
-- Serialize due-notification creation with manual service-progress actions.
-- SKIP LOCKED lets the next poll retry a row currently being started or
-- completed without creating an alert from stale appointment state.
CREATE OR REPLACE FUNCTION
public.sync_service_progress_notifications()
RETURNS integer
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_created integer := 0;
  v_now timestamptz := now();
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Only administrators can synchronize service progress
notifications.';
  END IF;

  WITH due_appointments AS (
    SELECT appointment.id, appointment.reference_code,
appointment.customer_name
    FROM public.appointments AS appointment
    WHERE appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status = 'confirmed'
      AND appointment.service_started_at IS NULL
      AND (appointment.appointment_date + appointment.start_time) AT TIME
ZONE 'Asia/Manila' <= v_now
      AND COALESCE(appointment.arrival_notification_snoozed_until, v_now) <=
v_now
      AND NOT EXISTS (
        SELECT 1
        FROM public.notifications AS notification
        WHERE notification.appointment_id = appointment.id
          AND notification.type = 'service_arrival'
          AND notification.is_read = false
      )
    FOR UPDATE OF appointment SKIP LOCKED
  ),
  inserted AS (
    INSERT INTO public.notifications (type, title, message, appointment_id)
```

```
    SELECT
      'service_arrival',
      'Customer expected: ' || due.reference_code,
      due.customer_name || ' is scheduled to arrive for service now.',
      due.id
    FROM due_appointments AS due
    ON CONFLICT DO NOTHING
    RETURNING 1
  )
  SELECT count(*) INTO v_created FROM inserted;

  WITH due_appointments AS (
    SELECT
      appointment.id,
      appointment.reference_code,
      appointment.customer_name,
      appointment.service_started_at,
      service_duration.service_duration_minutes
    FROM public.appointments AS appointment
    JOIN LATERAL (
      SELECT sum(appointment_service.duration_minutes)::integer AS
service_duration_minutes
      FROM public.appointment_services AS appointment_service
      WHERE appointment_service.appointment_id = appointment.id
    ) AS service_duration ON service_duration.service_duration_minutes > 0
    WHERE appointment.is_archived = false
      AND appointment.status = 'in_progress'
      AND appointment.service_started_at IS NOT NULL
      AND appointment.service_ended_at IS NULL
      AND appointment.service_started_at
        + make_interval(mins => service_duration.service_duration_minutes) <=
v_now
      AND COALESCE(appointment.completion_notification_snoozed_until,
v_now) <= v_now
      AND NOT EXISTS (
        SELECT 1
        FROM public.notifications AS notification
        WHERE notification.appointment_id = appointment.id
          AND notification.type = 'service_completion'
          AND notification.is_read = false
      )
    FOR UPDATE OF appointment SKIP LOCKED
  ),
  inserted AS (
    INSERT INTO public.notifications (type, title, message, appointment_id)
```

```
    SELECT
      'service_completion',
      'Service duration ended: ' || due.reference_code,
      'The expected service duration for ' || due.customer_name || ' has ended.',
      due.id
    FROM due_appointments AS due
    ON CONFLICT DO NOTHING
    RETURNING 1
  )
  SELECT v_created + count(*) INTO v_created FROM inserted;

  RETURN v_created;
END;
$$;

REVOKE ALL ON FUNCTION public.sync_service_progress_notifications()
FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.sync_service_progress_notifications()
TO authenticated;
```

supabase/migrations/20260924120000_standardize_local_philippine_mobile_numbers.sql

---

```
-- Application phone values are always stored and compared as local Philippine
-- mobile numbers (09XXXXXXXXX). International formatting belongs only at a
-- future Supabase Auth phone-auth boundary, never in application records.

CREATE OR REPLACE FUNCTION
public.normalize_philippine_mobile(p_phone text)
RETURNS text
LANGUAGE sql
IMMUTABLE
STRICT
SET search_path = public
AS $$
  SELECT CASE
    WHEN value.digits ~ '^09[0-9]{9}$' THEN value.digits
    WHEN value.digits ~ '^639[0-9]{9}$' THEN '0' || substring(value.digits FROM 3)
    ELSE NULL
  END
  FROM (SELECT regexp_replace(p_phone, '[^0-9]', '', 'g') AS digits) AS value;
$$;

-- The previous constraints required +639XXXXXXXXX. Remove them before
-- migrating recognized historical values back to the application format.
ALTER TABLE public.appointments
  DROP CONSTRAINT IF EXISTS appointments_phone_e164_check;
ALTER TABLE public.blocked_numbers
  DROP CONSTRAINT IF EXISTS blocked_numbers_phone_e164_check;

-- NOT VALID retains access to any irrecoverable historical values while every
-- new insert or phone update is enforced in the local format.
ALTER TABLE public.appointments
  ADD CONSTRAINT appointments_phone_local_check
  CHECK (phone ~ '^09[0-9]{9}$') NOT VALID;
ALTER TABLE public.blocked_numbers
  ADD CONSTRAINT blocked_numbers_phone_local_check
  CHECK (phone ~ '^09[0-9]{9}$') NOT VALID;
ALTER TABLE public.crew_members
  ADD CONSTRAINT crew_members_phone_local_check
  CHECK (phone IS NULL OR phone ~ '^09[0-9]{9}$') NOT VALID;

-- Keep direct database writes aligned with the application normalizer as a
-- final integrity boundary. Existing +639 values are accepted only to be
-- converted before the local-format constraints run.
```

```
CREATE OR REPLACE FUNCTION
public.normalize_philippine_mobile_before_write()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = public
AS $$
BEGIN
  IF NEW.phone IS NULL THEN
    RETURN NEW;
  END IF;

  NEW.phone := public.normalize_philippine_mobile(NEW.phone);
  IF NEW.phone IS NULL THEN
    RAISE EXCEPTION 'Enter a valid Philippine mobile number.' USING
ERRCODE = '22023';
  END IF;

  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS normalize_appointment_phone_before_write ON
public.appointments;
CREATE TRIGGER normalize_appointment_phone_before_write
  BEFORE INSERT OR UPDATE OF phone ON public.appointments
  FOR EACH ROW EXECUTE FUNCTION
public.normalize_philippine_mobile_before_write();

DROP TRIGGER IF EXISTS normalize_blocked_number_phone_before_write
ON public.blocked_numbers;
CREATE TRIGGER normalize_blocked_number_phone_before_write
  BEFORE INSERT OR UPDATE OF phone ON public.blocked_numbers
  FOR EACH ROW EXECUTE FUNCTION
public.normalize_philippine_mobile_before_write();

DROP TRIGGER IF EXISTS normalize_crew_member_phone_before_write ON
public.crew_members;
CREATE TRIGGER normalize_crew_member_phone_before_write
  BEFORE INSERT OR UPDATE OF phone ON public.crew_members
  FOR EACH ROW EXECUTE FUNCTION
public.normalize_philippine_mobile_before_write();

-- The admin edit RPC receives the already-normalized client value, but keeps
-- the same database-side validation as public booking and other write paths.
CREATE OR REPLACE FUNCTION public.update_appointment_details_atomic(
```

```
  p_appointment_id uuid,
  p_service_ids uuid[],
  p_appointment_date date,
  p_start_time time,
  p_assigned_crew_id uuid,
  p_crew_assignment_manual boolean,
  p_status text,
  p_admin_notes text,
  p_first_name text,
  p_middle_name text,
  p_last_name text,
  p_phone text
)
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
BEGIN
  IF COALESCE(btrim(p_first_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$'
     OR COALESCE(btrim(p_last_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$' THEN
    RAISE EXCEPTION 'First and last names must contain letters only.';
  END IF;

  IF COALESCE(btrim(p_middle_name), '') <> ''
     AND btrim(p_middle_name) !~ '^[[:alpha:]]+( [[:alpha:]]+)*$' THEN
    RAISE EXCEPTION 'Middle name must contain letters only.';
  END IF;

  IF COALESCE(btrim(p_phone), '') !~ '^09[0-9]{9}$' THEN
    RAISE EXCEPTION 'Enter a valid Philippine mobile number.';
  END IF;

  PERFORM public.update_appointment_details_atomic(
    p_appointment_id,
    p_service_ids,
    p_appointment_date,
    p_start_time,
    p_assigned_crew_id,
    p_crew_assignment_manual,
    p_status,
    p_admin_notes
  );

  UPDATE public.appointments
```

```
  SET first_name = btrim(p_first_name),
      middle_name = NULLIF(btrim(p_middle_name), ''),
      last_name = btrim(p_last_name),
      phone = btrim(p_phone),
      customer_name = concat_ws(
        ' ',
        btrim(p_first_name),
        NULLIF(btrim(p_middle_name), ''),
        btrim(p_last_name)
      )
  WHERE id = p_appointment_id
    AND is_archived = false;
END;
$$;

REVOKE ALL ON FUNCTION public.update_appointment_details_atomic(
  uuid, uuid[], date, time, uuid, boolean, text, text, text, text, text, text
) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.update_appointment_details_atomic(
  uuid, uuid[], date, time, uuid, boolean, text, text, text, text, text, text
) TO authenticated;

-- These updates are deliberately last. Appointment updates queue existing
-- audit/updated-at trigger events, and PostgreSQL does not permit subsequent
-- table alterations while those events are pending in this transaction.
UPDATE public.appointments
SET phone = public.normalize_philippine_mobile(phone)
WHERE public.normalize_philippine_mobile(phone) IS NOT NULL
  AND phone IS DISTINCT FROM public.normalize_philippine_mobile(phone);

UPDATE public.blocked_numbers
SET phone = public.normalize_philippine_mobile(phone)
WHERE public.normalize_philippine_mobile(phone) IS NOT NULL
  AND phone IS DISTINCT FROM public.normalize_philippine_mobile(phone);

UPDATE public.crew_members
SET phone = public.normalize_philippine_mobile(phone)
WHERE phone IS NOT NULL
  AND public.normalize_philippine_mobile(phone) IS NOT NULL
  AND phone IS DISTINCT FROM public.normalize_philippine_mobile(phone);
```

supabase/migrations/20260925100000_support_customer_name_segments.sql

---

```
-- Customer name parts can contain space-separated words, hyphens, and
straight
-- or typographic apostrophes. Keep the same rule in public booking and admin
-- edits so a valid customer booking remains editable later.
CREATE OR REPLACE FUNCTION
public.is_valid_customer_name_part(p_name text)
RETURNS boolean
LANGUAGE sql
IMMUTABLE
STRICT
SET search_path = public
AS $$
  SELECT btrim(p_name) ~ '^[[:alpha:]]+(([[:space:]]+|[-'’])[[:alpha:]]+)*$';
$$;

CREATE OR REPLACE FUNCTION public.create_booking_atomic(
  p_reference_code text, p_booking_request_id uuid, p_customer_name text,
  p_phone text, p_moto_brand text, p_moto_model text, p_moto_variant text,
  p_moto_year integer, p_plate_number text, p_appointment_date date,
  p_start_time time, p_notes text, p_total_estimate numeric,
  p_booking_duration_minutes integer, p_assigned_crew_id uuid, p_services
jsonb,
  p_continuation_segments jsonb, p_notification_title text,
p_notification_message text,
  p_first_name text, p_middle_name text, p_last_name text
)
RETURNS TABLE (appointment_id uuid, reference_code text)
LANGUAGE plpgsql SECURITY INVOKER SET search_path = public
AS $$
DECLARE v_appointment_id uuid; v_reference_code text;
BEGIN
  IF p_booking_request_id IS NULL THEN RAISE EXCEPTION 'A booking
request ID is required.'; END IF;
  IF NOT public.is_valid_customer_name_part(p_first_name)
     OR NOT public.is_valid_customer_name_part(p_last_name) THEN
    RAISE EXCEPTION 'First and last names must contain letters, spaces,
hyphens, or apostrophes only.';
  END IF;
  IF COALESCE(btrim(p_middle_name), '') <> ''
     AND NOT public.is_valid_customer_name_part(p_middle_name) THEN
    RAISE EXCEPTION 'Middle name must contain letters, spaces, hyphens, or
apostrophes only.';
```

```
  END IF;
  IF jsonb_typeof(p_services) <> 'array' OR jsonb_array_length(p_services) = 0
THEN
    RAISE EXCEPTION 'A booking must include at least one service.';
  END IF;
  IF jsonb_typeof(COALESCE(p_continuation_segments, '[]'::jsonb)) <> 'array'
THEN
    RAISE EXCEPTION 'Continuation segments must be an array.';
  END IF;

  INSERT INTO public.appointments (
    reference_code, booking_request_id, customer_name, first_name,
middle_name,
    last_name, phone, moto_brand, moto_model, moto_variant, moto_year,
    plate_number, appointment_date, start_time, notes, total_estimate,
    booking_duration_minutes, terms_accepted, assigned_crew_id
  ) VALUES (
    p_reference_code, p_booking_request_id,
    concat_ws(' ', btrim(p_first_name), NULLIF(btrim(p_middle_name), ''),
btrim(p_last_name)),
    btrim(p_first_name), NULLIF(btrim(p_middle_name), ''), btrim(p_last_name),
p_phone,
    p_moto_brand, p_moto_model, p_moto_variant, p_moto_year,
p_plate_number,
    p_appointment_date, p_start_time, p_notes, p_total_estimate,
    p_booking_duration_minutes, true, p_assigned_crew_id
  ) ON CONFLICT DO NOTHING
  RETURNING appointments.id, appointments.reference_code INTO
v_appointment_id, v_reference_code;

  IF v_appointment_id IS NULL THEN
    SELECT appointments.id, appointments.reference_code INTO
v_appointment_id, v_reference_code
    FROM public.appointments WHERE appointments.booking_request_id =
p_booking_request_id;
    IF FOUND THEN RETURN QUERY SELECT v_appointment_id,
v_reference_code; RETURN; END IF;
    RAISE EXCEPTION 'The requested schedule is no longer available.' USING
ERRCODE = '23P01';
  END IF;

  INSERT INTO public.appointment_services (appointment_id, service_id,
service_name, price, duration_minutes)
  SELECT v_appointment_id, service.service_id, service.service_name,
service.price, service.duration_minutes
```

```
  FROM jsonb_to_recordset(p_services) AS service(
    service_id uuid, service_name text, price numeric, duration_minutes integer
  );

  INSERT INTO public.appointment_continuations (
    appointment_id, segment_number, appointment_date, start_time,
    booking_duration_minutes, assigned_crew_id
  )
  SELECT v_appointment_id, segment.segment_number,
segment.appointment_date, segment.start_time,
    segment.booking_duration_minutes, segment.assigned_crew_id
  FROM jsonb_to_recordset(COALESCE(p_continuation_segments, '[]'::jsonb))
AS segment(
    segment_number integer, appointment_date date, start_time time,
    booking_duration_minutes integer, assigned_crew_id uuid
  );

  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES ('new_appointment', p_notification_title, p_notification_message,
v_appointment_id);
  RETURN QUERY SELECT v_appointment_id, v_reference_code;
END;
$$;

CREATE OR REPLACE FUNCTION public.update_appointment_details_atomic(
  p_appointment_id uuid,
  p_service_ids uuid[],
  p_appointment_date date,
  p_start_time time,
  p_assigned_crew_id uuid,
  p_crew_assignment_manual boolean,
  p_status text,
  p_admin_notes text,
  p_first_name text,
  p_middle_name text,
  p_last_name text,
  p_phone text
)
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
BEGIN
  IF NOT public.is_valid_customer_name_part(p_first_name)
```

```
     OR NOT public.is_valid_customer_name_part(p_last_name) THEN
    RAISE EXCEPTION 'First and last names must contain letters, spaces,
hyphens, or apostrophes only.';
  END IF;

  IF COALESCE(btrim(p_middle_name), '') <> ''
     AND NOT public.is_valid_customer_name_part(p_middle_name) THEN
    RAISE EXCEPTION 'Middle name must contain letters, spaces, hyphens, or
apostrophes only.';
  END IF;

  IF COALESCE(btrim(p_phone), '') !~ '^09[0-9]{9}$' THEN
    RAISE EXCEPTION 'Enter a valid Philippine mobile number.';
  END IF;

  PERFORM public.update_appointment_details_atomic(
    p_appointment_id,
    p_service_ids,
    p_appointment_date,
    p_start_time,
    p_assigned_crew_id,
    p_crew_assignment_manual,
    p_status,
    p_admin_notes
  );

  UPDATE public.appointments
  SET first_name = btrim(p_first_name),
      middle_name = NULLIF(btrim(p_middle_name), ''),
      last_name = btrim(p_last_name),
      phone = btrim(p_phone),
      customer_name = concat_ws(
        ' ',
        btrim(p_first_name),
        NULLIF(btrim(p_middle_name), ''),
        btrim(p_last_name)
      )
  WHERE id = p_appointment_id
    AND is_archived = false;
END;
$$;
```

supabase/migrations/20260925110000_constrain_service_progress_notification_timeline.sql

---

```
-- Keep service-progress alerts tied to the active service timeline. Arrival
-- alerts expire at the scheduled service end unless an admin explicitly
-- snoozed them; completion alerts remain active until the service is completed.

CREATE OR REPLACE FUNCTION
public.resolve_appointment_service_progress_notifications()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
  v_now timestamptz := now();
  v_service_duration_minutes integer := 0;
  v_scheduled_start timestamptz;
  v_expected_completion timestamptz;
BEGIN
  SELECT COALESCE(sum(service.duration_minutes), 0)::integer
    INTO v_service_duration_minutes
  FROM public.appointment_services AS service
  WHERE service.appointment_id = NEW.id;

  v_scheduled_start := (NEW.appointment_date + NEW.start_time) AT TIME
ZONE 'Asia/Manila';
  v_expected_completion := CASE
    WHEN NEW.service_started_at IS NOT NULL
      THEN NEW.service_started_at + make_interval(mins =>
v_service_duration_minutes)
    ELSE NULL
  END;

  UPDATE public.notifications AS notification
  SET is_read = true
  WHERE notification.appointment_id = NEW.id
    AND notification.is_read = false
    AND notification.type IN ('service_arrival', 'service_completion')
    AND (
      NEW.is_archived = true
      OR NEW.rescheduled_to_appointment_id IS NOT NULL
      OR (
        notification.type = 'service_arrival'
        AND (
```

```
        NEW.status <> 'confirmed'
        OR NEW.service_started_at IS NOT NULL
        OR v_scheduled_start > v_now
        OR (
          NEW.arrival_notification_snoozed_until IS NULL
          AND (
            v_service_duration_minutes <= 0
            OR v_scheduled_start + make_interval(mins =>
v_service_duration_minutes) <= v_now
          )
        )
      )
    )
    OR (
      notification.type = 'service_completion'
      AND (
        NEW.status <> 'in_progress'
        OR NEW.service_started_at IS NULL
        OR NEW.service_ended_at IS NOT NULL
        OR v_service_duration_minutes <= 0
        OR v_expected_completion > v_now
      )
    )
  );

  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS
appointments_resolve_service_progress_notifications ON public.appointments;
CREATE TRIGGER appointments_resolve_service_progress_notifications
  AFTER UPDATE OF status, is_archived, appointment_date, start_time,
service_started_at, service_ended_at
  ON public.appointments
  FOR EACH ROW EXECUTE FUNCTION
public.resolve_appointment_service_progress_notifications();

CREATE OR REPLACE FUNCTION
public.sync_service_progress_notifications()
RETURNS integer
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
```

```
DECLARE
  v_changed integer := 0;
  v_now timestamptz := now();
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Only administrators can synchronize service progress
notifications.';
  END IF;

  -- Lock appointments before their related notification rows. This matches the
  -- action RPC's locking order and lets a concurrent start, snooze, no-show,
  -- or completion win without stale rows or deadlocks.
  WITH timeline AS (
    SELECT
      appointment.id,
      appointment.status,
      appointment.is_archived,
      appointment.rescheduled_to_appointment_id,
      appointment.service_started_at,
      appointment.service_ended_at,
      appointment.arrival_notification_snoozed_until,
      appointment.completion_notification_snoozed_until,
      (appointment.appointment_date + appointment.start_time) AT TIME ZONE
'Asia/Manila'
        AS scheduled_start,
      COALESCE(service_duration.service_duration_minutes, 0) AS
service_duration_minutes
    FROM public.appointments AS appointment
    LEFT JOIN LATERAL (
      SELECT sum(service.duration_minutes)::integer AS
service_duration_minutes
      FROM public.appointment_services AS service
      WHERE service.appointment_id = appointment.id
    ) AS service_duration ON true
    WHERE EXISTS (
      SELECT 1
      FROM public.notifications AS notification
      WHERE notification.appointment_id = appointment.id
        AND notification.is_read = false
        AND notification.type IN ('service_arrival', 'service_completion')
    )
    FOR UPDATE OF appointment SKIP LOCKED
  ),
  resolved AS (
    UPDATE public.notifications AS notification
```

```
    SET is_read = true
    FROM timeline
    WHERE notification.appointment_id = timeline.id
      AND notification.is_read = false
      AND notification.type IN ('service_arrival', 'service_completion')
      AND (
        timeline.is_archived = true
        OR timeline.rescheduled_to_appointment_id IS NOT NULL
        OR (
          notification.type = 'service_arrival'
          AND (
            timeline.status <> 'confirmed'
            OR timeline.service_started_at IS NOT NULL
            OR timeline.scheduled_start > v_now
            OR (
              timeline.arrival_notification_snoozed_until IS NULL
              AND (
                timeline.service_duration_minutes <= 0
                OR timeline.scheduled_start
                  + make_interval(mins => timeline.service_duration_minutes) <=
v_now
              )
            )
          )
        )
        OR (
          notification.type = 'service_completion'
          AND (
            timeline.status <> 'in_progress'
            OR timeline.service_started_at IS NULL
            OR timeline.service_ended_at IS NOT NULL
            OR timeline.service_duration_minutes <= 0
            OR timeline.service_started_at
              + make_interval(mins => timeline.service_duration_minutes) > v_now
          )
        )
      )
    RETURNING 1
  )
  SELECT count(*) INTO v_changed FROM resolved;

  WITH due_appointments AS (
    SELECT appointment.id, appointment.reference_code,
appointment.customer_name
    FROM public.appointments AS appointment
```

```
    JOIN LATERAL (
      SELECT sum(service.duration_minutes)::integer AS
service_duration_minutes
      FROM public.appointment_services AS service
      WHERE service.appointment_id = appointment.id
    ) AS service_duration ON service_duration.service_duration_minutes > 0
    WHERE appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status = 'confirmed'
      AND appointment.service_started_at IS NULL
      AND (appointment.appointment_date + appointment.start_time) AT TIME
ZONE 'Asia/Manila' <= v_now
      AND COALESCE(appointment.arrival_notification_snoozed_until, v_now) <=
v_now
      -- Unsnoozed arrival alerts are relevant only inside the scheduled
      -- service window. A snoozed alert intentionally outlives that window.
      AND (
        appointment.arrival_notification_snoozed_until IS NOT NULL
        OR (appointment.appointment_date + appointment.start_time) AT TIME
ZONE 'Asia/Manila'
          + make_interval(mins => service_duration.service_duration_minutes) >
v_now
      )
      AND NOT EXISTS (
        SELECT 1
        FROM public.notifications AS notification
        WHERE notification.appointment_id = appointment.id
          AND notification.type = 'service_arrival'
          AND notification.is_read = false
      )
    FOR UPDATE OF appointment SKIP LOCKED
  ),
  inserted AS (
    INSERT INTO public.notifications (type, title, message, appointment_id)
    SELECT
      'service_arrival',
      'Customer expected: ' || due.reference_code,
      due.customer_name || ' is scheduled to arrive for service now.',
      due.id
    FROM due_appointments AS due
    ON CONFLICT DO NOTHING
    RETURNING 1
  )
  SELECT v_changed + count(*) INTO v_changed FROM inserted;
```

```
  WITH due_appointments AS (
    SELECT appointment.id, appointment.reference_code,
appointment.customer_name
    FROM public.appointments AS appointment
    JOIN LATERAL (
      SELECT sum(service.duration_minutes)::integer AS
service_duration_minutes
      FROM public.appointment_services AS service
      WHERE service.appointment_id = appointment.id
    ) AS service_duration ON service_duration.service_duration_minutes > 0
    WHERE appointment.is_archived = false
      AND appointment.status = 'in_progress'
      AND appointment.service_started_at IS NOT NULL
      AND appointment.service_ended_at IS NULL
      AND appointment.service_started_at
        + make_interval(mins => service_duration.service_duration_minutes) <=
v_now
      AND COALESCE(appointment.completion_notification_snoozed_until,
v_now) <= v_now
      AND NOT EXISTS (
        SELECT 1
        FROM public.notifications AS notification
        WHERE notification.appointment_id = appointment.id
          AND notification.type = 'service_completion'
          AND notification.is_read = false
      )
    FOR UPDATE OF appointment SKIP LOCKED
  ),
  inserted AS (
    INSERT INTO public.notifications (type, title, message, appointment_id)
    SELECT
      'service_completion',
      'Service duration ended: ' || due.reference_code,
      'The expected service duration for ' || due.customer_name || ' has ended.',
      due.id
    FROM due_appointments AS due
    ON CONFLICT DO NOTHING
    RETURNING 1
  )
  SELECT v_changed + count(*) INTO v_changed FROM inserted;

  RETURN v_changed;
END;
$$;
```

```
REVOKE ALL ON FUNCTION public.sync_service_progress_notifications()
FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.sync_service_progress_notifications()
TO authenticated;
```

supabase/migrations/20260925120000_transfer_admin_account.sql

---

```
-- Transfer the sole application administrator to the verified Supabase Auth
-- user. The prior account was a bootstrap account and must not retain access.
--
-- Historical audit rows intentionally keep their captured email/summary. Their
-- UUID foreign keys are cleared when the old auth user is removed, rather than
-- being reassigned to the new administrator and misattributing past actions.
DO $$
DECLARE
  v_target_admin_id constant uuid := '7a9e497f-8c3f-44a7-ab4e-c9247c50eb74';
BEGIN
  IF NOT EXISTS (SELECT 1 FROM auth.users WHERE id = v_target_admin_id)
THEN
    RAISE EXCEPTION 'The replacement administrator does not exist in
Supabase Auth.';
  END IF;

  INSERT INTO public.user_roles (user_id, role)
  VALUES (v_target_admin_id, 'admin')
  ON CONFLICT (user_id, role) DO NOTHING;

  -- price_history.changed_by has a restrictive foreign key. It is nullable and
  -- retains changed_by_email, so clear only this blocking pointer before the
  -- old Auth account is deleted. Other dependent audit tables use ON DELETE
  -- SET NULL and retain their historical metadata automatically.
  UPDATE public.price_history
  SET changed_by = NULL
  WHERE changed_by IN (
    SELECT user_id
    FROM public.user_roles
    WHERE role = 'admin'
      AND user_id <> v_target_admin_id
  );

  -- Deleting the former Admin Auth account revokes its credentials and cascades
  -- the obsolete role row. Current RLS policies and functions resolve access
  -- through user_roles, so the replacement account is the only administrator.
  DELETE FROM auth.users AS former_admin
  WHERE former_admin.id <> v_target_admin_id
    AND EXISTS (
      SELECT 1
      FROM public.user_roles AS role_assignment
      WHERE role_assignment.user_id = former_admin.id
        AND role_assignment.role = 'admin'
```

```
    );
END;
$$;
```

supabase/migrations/20260926100000_add_report_total_value.sql

---

```
-- Keep report totals aligned with the visible rows for every status and
-- service filter. Completed value remains a separate completed-only metric.
CREATE OR REPLACE FUNCTION public.get_admin_report_page(
  p_report_kind text,
  p_from date,
  p_to date,
  p_status text DEFAULT NULL,
  p_category text DEFAULT NULL,
  p_service_name text DEFAULT NULL,
  p_limit integer DEFAULT 25,
  p_offset integer DEFAULT 0
)
RETURNS jsonb
LANGUAGE plpgsql
STABLE
SECURITY INVOKER
SET search_path = public, auth
AS $$
DECLARE
  v_limit integer := LEAST(GREATEST(COALESCE(p_limit, 25), 1), 250);
  v_offset integer := GREATEST(COALESCE(p_offset, 0), 0);
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;
  IF p_report_kind NOT IN ('bookings', 'services') THEN
    RAISE EXCEPTION 'Unknown report type';
  END IF;
  IF p_from IS NULL OR p_to IS NULL OR p_from > p_to THEN
    RAISE EXCEPTION 'A valid report date range is required';
  END IF;

  IF p_report_kind = 'bookings' THEN
    RETURN (
      WITH filtered AS (
        SELECT
          a.id,
          a.reference_code,
          a.customer_name,
          a.appointment_date,
          a.status,
          a.total_estimate,
          COALESCE(
```

```
          (
            SELECT string_agg(aps.service_name, ', ' ORDER BY
aps.service_name)
            FROM public.appointment_services aps
            WHERE aps.appointment_id = a.id
          ),
          '—'
        ) AS service
      FROM public.appointments a
      WHERE a.is_archived = false
        AND a.appointment_date BETWEEN p_from AND p_to
        AND (p_status IS NULL OR a.status = p_status)
        AND (
          (p_category IS NULL AND p_service_name IS NULL)
          OR EXISTS (
            SELECT 1
            FROM public.appointment_services aps
            LEFT JOIN public.services s ON s.id = aps.service_id
            WHERE aps.appointment_id = a.id
              AND (p_category IS NULL OR COALESCE(s.category,
'Uncategorized') = p_category)
              AND (p_service_name IS NULL OR aps.service_name =
p_service_name)
          )
        )
    ),
    grouped_status AS (
      SELECT status, COUNT(*)::integer AS total FROM filtered GROUP BY
status
    ),
    paged AS (
      SELECT * FROM filtered
      ORDER BY appointment_date DESC, reference_code ASC
      LIMIT v_limit OFFSET v_offset
    )
    SELECT jsonb_build_object(
      'total', (SELECT COUNT(*) FROM filtered),
      'rows', COALESCE((SELECT jsonb_agg(TO_JSONB(paged)) FROM
paged), '[]'::jsonb),
      'metrics', jsonb_build_object(
        'total_bookings', (SELECT COUNT(*) FROM filtered),
        'completed_bookings', (SELECT COUNT(*) FROM filtered WHERE status
= 'completed'),
        'completed_value',
          (SELECT COALESCE(SUM(total_estimate) FILTER (WHERE status =
```

```
'completed'), 0) FROM filtered),
        'total_value', (SELECT COALESCE(SUM(total_estimate), 0) FROM
filtered),
        'status_counts',
          COALESCE((SELECT jsonb_object_agg(status, total) FROM
grouped_status), '{}'::jsonb)
      )
    )
  );
 END IF;

 RETURN (
  WITH filtered AS (
    SELECT
      aps.appointment_id,
      aps.service_id,
      aps.service_name,
      aps.price,
      a.reference_code,
      a.customer_name,
      a.appointment_date,
      a.status,
      COALESCE(s.category, 'Uncategorized') AS category
    FROM public.appointment_services aps
    JOIN public.appointments a ON a.id = aps.appointment_id
    LEFT JOIN public.services s ON s.id = aps.service_id
    WHERE a.is_archived = false
      AND a.appointment_date BETWEEN p_from AND p_to
      AND (p_status IS NULL OR a.status = p_status)
      AND (p_category IS NULL OR COALESCE(s.category, 'Uncategorized') =
p_category)
      AND (p_service_name IS NULL OR aps.service_name = p_service_name)
  ),
  grouped_status AS (
    SELECT status, COUNT(*)::integer AS total FROM filtered GROUP BY
status
  ),
  paged AS (
    SELECT * FROM filtered
    ORDER BY appointment_date DESC, reference_code ASC, service_name
ASC
    LIMIT v_limit OFFSET v_offset
  )
  SELECT jsonb_build_object(
    'total', (SELECT COUNT(*) FROM filtered),
```

```
    'rows', COALESCE((SELECT jsonb_agg(TO_JSONB(paged)) FROM paged),
'[]'::jsonb),
    'metrics', jsonb_build_object(
      'total_bookings', (SELECT COUNT(DISTINCT appointment_id) FROM
filtered),
      'completed_rows', (SELECT COUNT(*) FROM filtered WHERE status =
'completed'),
      'completed_value',
        (SELECT COALESCE(SUM(price) FILTER (WHERE status = 'completed'),
0) FROM filtered),
      'total_value', (SELECT COALESCE(SUM(price), 0) FROM filtered),
      'status_counts',
        COALESCE((SELECT jsonb_object_agg(status, total) FROM
grouped_status), '{}'::jsonb)
    )
  )
 );
END;
$$;

REVOKE ALL ON FUNCTION public.get_admin_report_page(text, date, date,
text, text, text, integer, integer) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.get_admin_report_page(text, date,
date, text, text, text, integer, integer) TO authenticated;
```

supabase/migrations/20260926200000_add_engine_overhaul_500cc_model_overrides.sql

---

```
-- Configure the Engine Overhaul service for catalog models verified by
-- 500cc_and_up_models.pdf. The source list excludes named "500" models that
-- are actually below the 500cc threshold.
--
-- The migration only applies to active, non-archived catalog rows whose brand
-- and model match the source list after harmless spacing/punctuation
-- normalization. Unmatched and ambiguous catalog entries are intentionally
-- left unchanged.

DO $$
DECLARE
  v_engine_overhaul_id uuid;
BEGIN
  SELECT id
  INTO v_engine_overhaul_id
  FROM public.services
  WHERE name = 'Engine Overhaul'
    AND is_archived = false
  ORDER BY created_at
  LIMIT 1;

  IF v_engine_overhaul_id IS NULL THEN
    RAISE EXCEPTION 'Engine Overhaul service is required before creating
model overrides';
  END IF;

  WITH source_model_groups (brand, models) AS (
    VALUES
      ('Honda', ARRAY['CBR600RR', 'CBR650R', 'CBR1000RR', 'CBR1000RR-R',
'Rebel1100', 'CB650R', 'CB1000R', 'NC750X', 'X-ADV', 'Africa Twin', 'Africa Twin
1100', 'Gold Wing']::text[]),
      ('Yamaha', ARRAY['MT-07', 'MT-09', 'MT-10', 'Tenere 700', 'XSR700',
'XSR900', 'VMAX', 'TMAX', 'Bolt', 'R6', 'R1']::text[]),
      ('Suzuki', ARRAY['GSX-R600', 'GSX-R750', 'GSX-R1000', 'V-Strom650',
'V-Strom800', 'V-Strom1050', 'SV650', 'SV1000', 'DR650', 'Boulevard M109R',
'Boulevard C50', 'Boulevard C90', 'Hayabusa', 'Katana', 'GSX-S1000']::text[]),
      ('Kawasaki', ARRAY['KLR650', 'Ninja650', 'Ninja ZX-6R', 'Ninja ZX-10R',
'Ninja H2', 'Z650', 'Z750', 'Z800', 'Z900', 'Z1000', 'Versys650', 'Versys1000',
'Vulcan S', 'Vulcan900', 'Vulcan1700']::text[]),
      ('Kymco', ARRAY['AK550']::text[]),
      ('CFMOTO', ARRAY['650NK', '675NK', '800NK', '500SR', '675SR-R',
'750SR-S', '650MT', '700MT', '800MT', '800MT Sport', '800MT Explore',
```

'800MT-X', '1000MT-X', '700CL-X', '700CL-X Heritage', '700CL-X Sport', '700CL-X ADV', '650GT', '650TR', '1250TR']::text[]),
('Benelli', ARRAY['TNT600i', 'TNT600GT', 'BN600', 'Leoncino500', 'Leoncino500 Trail', 'Leoncino800', 'TRK502', 'TRK502X', 'TRK702', 'TRK702X', '502C', '752S', '752C', '1200GT', '1200GT SE']::text[]),
('KTM', ARRAY['Duke690', 'Duke790', 'Duke890', 'Duke990', 'Duke1290', 'Adventure790', 'Adventure890', 'Adventure1050', 'Adventure1090', 'Adventure1190', 'Adventure1290', '500 EXC', '690 Enduro', '690 SMC', '500 EXC-F']::text[]),
('Bristol', ARRAY['Bobber650', 'Cafe500', 'Scrambler500', 'Adventure500', 'Bullet500', 'Maxie500', 'Veloce500', 'Veloce650']::text[]),
('Royal Enfield', ARRAY['Bullet500', 'Classic500', 'Classic500 FI', 'Classic650', 'Thunderbird500', 'Thunderbird X500', 'Interceptor650', 'Continental GT650', 'ContinentalGT', 'Super Meteor650', 'Shotgun650', 'Bear650', 'Electra500']::text[]),
('SYM', ARRAY['Maxsym600i']::text[]),
('Zontes', ARRAY['703F', '703R', '703RR', '703T']::text[]),
('Piaggio', ARRAY['MP3 500', 'MP3 530']::text[]),
('BMW Motorrad', ARRAY['F650GS', 'F700GS', 'F750GS', 'F800GS', 'F800R', 'F800GT', 'F850GS', 'F850GSA', 'F900R', 'F900XR', 'R850R', 'R1100R', 'R1150R', 'R1200GS', 'R1200GSA', 'R1250GS', 'R1250GSA', 'R1250R', 'R1250RS', 'R1300GS', 'R1300GSA', 'R nineT', 'R12', 'S1000RR', 'S1000R', 'S1000XR', 'S1000', 'K1200R', 'K1300R', 'K1600GT', 'K1600GTL', 'K1600B', 'K1600 Grand America']::text[]),
('Ducati', ARRAY['Monster600', 'Monster620', 'Monster695', 'Monster696', 'Monster796', 'Monster821', 'Monster937', 'Monster1100', 'Monster1200', 'Monster1200R', 'MonsterSP', 'Scrambler800', 'Scrambler Icon', 'Scrambler1100', 'Scrambler1100 Sport', 'Panigale899', 'Panigale959', 'Panigale V2', 'Panigale V4', 'Panigale V4R', '848', '1098', '1198', '1299', 'Streetfighter848', 'Streetfighter1098', 'Streetfighter V2', 'Streetfighter V4', 'Multistrada950', 'Multistrada1200', 'Multistrada1260', 'Multistrada V2', 'Multistrada V4', 'Diavel', 'Diavel1260', 'Diavel V4', 'XDiavel', 'Hypermotard796', 'Hypermotard821', 'Hypermotard939', 'Hypermotard950', 'DesertX', 'SuperSport', 'SuperSport950']::text[]),
('Harley-Davidson', ARRAY['Sportster883', 'Sportster1200', 'Iron883', 'Iron1200', 'Forty-Eight', 'Forty-Eight Special', 'Nightster', 'Nightster Special', 'Street500', 'Street750', 'Street Bob', 'Street Bob114', 'Low Rider', 'Low Rider S', 'Low Rider ST', 'Fat Bob', 'Fat Bob114', 'Fat Boy', 'Fat Boy114', 'Heritage Classic', 'Heritage Classic114', 'Breakout', 'Breakout114', 'Softail Standard', 'Deluxe', 'Slim', 'Road King', 'Road King Special', 'Road Glide', 'Road Glide Special', 'Road Glide Limited', 'Street Glide', 'Street Glide Special', 'Electra Glide Ultra Limited', 'Pan America1250', 'Pan America1250 Special', 'Sportster S', 'LiveWire']::text[]),
('Triumph', ARRAY['Bonneville', 'Bonneville T100', 'Bonneville T120', 'Bonneville Bobber', 'Bobber1200', 'Speedmaster', 'Scrambler900', 'Scrambler1200', 'Street Scrambler', 'Street Twin', 'Speed Twin900', 'Speed Twin1200', 'Street Triple675', 'Street Triple765', 'Street Triple R', 'Street Triple

```
RS', 'Speed Triple1050', 'Speed Triple1200', 'Daytona675', 'Daytona765',
'Daytona955i', 'Tiger800', 'Tiger850', 'Tiger900', 'Tiger1200', 'Tiger Explorer1200',
'Tiger Sport660', 'Tiger Sport850', 'Tiger Sport1050', 'Rocket III', 'Rocket3',
'Trident660', 'Thruxton900', 'Thruxton1200', 'SprintST', 'SprintGT',
'Daytona1200']::text[]),
      ('Aprilia', ARRAY['RS660', 'RSV1000', 'RSV4', 'Tuono660', 'Tuono V4', 'Tuono
V4 Factory', 'Shiver750', 'Shiver900', 'Dorsoduro750', 'Dorsoduro900',
'Caponord1200', 'Mana850', 'Tuareg660', 'Pegaso650']::text[])
  ),
  source_models AS (
    SELECT source_model_groups.brand, unnest(source_model_groups.models)
AS model
    FROM source_model_groups
  ),
  matching_catalog_models AS (
    SELECT DISTINCT catalog.brand, catalog.model
    FROM public.motorcycle_catalog AS catalog
    INNER JOIN source_models AS source
      ON regexp_replace(lower(trim(catalog.brand)), '[^a-z0-9]+', '', 'g') =
        regexp_replace(lower(source.brand), '[^a-z0-9]+', '', 'g')
     AND (
       regexp_replace(lower(trim(catalog.model)), '[^a-z0-9]+', '', 'g') =
         regexp_replace(lower(source.model), '[^a-z0-9]+', '', 'g')
       OR regexp_replace(lower(trim(catalog.model)), '[^a-z0-9]+', '', 'g') =
          regexp_replace(lower(catalog.brand || source.model), '[^a-z0-9]+', '', 'g')
     )
    WHERE catalog.is_active = true
      AND catalog.is_archived = false
  )
  INSERT INTO public.service_model_overrides (
    service_id,
    brand,
    model,
    price,
    duration_minutes
  )
  SELECT
    v_engine_overhaul_id,
    matching_catalog_models.brand,
    matching_catalog_models.model,
    1800.00,
    480
  FROM matching_catalog_models
  ON CONFLICT (service_id, brand, model) DO UPDATE
  SET
```

```
    price = EXCLUDED.price,
    duration_minutes = EXCLUDED.duration_minutes
  WHERE public.service_model_overrides.price IS DISTINCT FROM
EXCLUDED.price
     OR public.service_model_overrides.duration_minutes IS DISTINCT FROM
EXCLUDED.duration_minutes;
END
$$;
```

supabase/migrations/20260927090000_replace_service_catalog_from_updated_details.sql

---

```
-- Replace the active service catalog with
Motorcycle_Service_Details_Updated.pdf.
-- Preserve the Engine Overhaul record so its existing model-specific prices stay
-- attached to the same service ID and historical bookings retain their snapshots.

DO $$
DECLARE
  v_engine_overhaul_id uuid;
BEGIN
  SELECT id
  INTO v_engine_overhaul_id
  FROM public.services
  WHERE name = 'Engine Overhaul'
    AND is_archived = false
  ORDER BY created_at
  LIMIT 1;

  IF v_engine_overhaul_id IS NULL THEN
    RAISE EXCEPTION 'Engine Overhaul service is required before replacing the
service catalog';
  END IF;

  -- Archive rather than delete legacy services. This preserves foreign-key
  -- relationships and the price/duration snapshots on historical appointments.
  UPDATE public.services
  SET
    is_active = false,
    is_archived = true,
    archived_at = COALESCE(archived_at, now())
  WHERE id <> v_engine_overhaul_id
    AND (is_active = true OR is_archived = false);

  -- Non-Engine-Overhaul overrides belong to the retired catalog and must not
  -- remain available to the booking or price-management flows.
  DELETE FROM public.service_model_overrides
  WHERE service_id <> v_engine_overhaul_id;

  -- Keep Engine Overhaul in the source document's relative display position
  -- without changing its preserved default price, duration, or model overrides.
  UPDATE public.services
  SET sort_order = 340
  WHERE id = v_engine_overhaul_id;
```

```
WITH replacement_services (
  name,
  description,
  price,
  duration_minutes,
  category,
  sort_order
) AS (
  VALUES
    ('Change Oil', 'Drains old engine oil and refills the engine with the correct oil to reduce friction and protect moving parts.', 50.00, 60, 'maintenance', 10),
    ('Motorcycle Tune-Up', 'Checks basic engine and performance items for rough running, difficult starting, reduced power, or rideability concerns.', 150.00, 150, 'maintenance', 20),
    ('Brake Service', 'Inspects braking concerns and carries out routine brake work where practical; additional confirmed work may require more time or cost.', 50.00, 90, 'brakes', 30),
    ('Clutch Adjustment', 'Adjusts clutch control and checks operation for difficult shifting, incorrect lever feel, or poor engagement.', 50.00, 75, 'drivetrain', 40),
    ('Full CVT Service', 'Inspects and cleans serviceable automatic-CVT components, checks their condition, and reports worn parts billed separately.', 500.00, 210, 'drivetrain', 50),
    ('Chain Cleaning', 'Cleans dirt, dust, and sticky buildup from the drive chain and checks its general condition.', 50.00, 75, 'drivetrain', 60),
    ('Chain Lubrication', 'Applies suitable lubricant to a dry, squeaky, or noisy drive chain to support smooth operation and reduce wear.', 50.00, 70, 'drivetrain', 70),
    ('Chain Adjustment', 'Adjusts drive-chain slack and checks rear-wheel alignment for a chain that is too loose, too tight, or noisy.', 50.00, 75, 'drivetrain', 80),
    ('Chain Replacement', 'Installs a compatible replacement chain, adjusts tension, and checks operation; the chain itself may be billed separately.', 100.00, 90, 'drivetrain', 90),
    ('Sprocket Replacement', 'Replaces a worn or damaged sprocket and checks the drive-chain fit.', 100.00, 90, 'drivetrain', 100),
    ('Chain and Sprocket Replacement', 'Replaces worn or damaged chain and sprockets as a set, adjusts tension, and checks alignment; parts may be billed separately.', 150.00, 120, 'drivetrain', 110),
    ('Tire Change', 'Removes a tire and fits a compatible replacement; tires, tubes, and other parts may be billed separately.', 70.00, 90, 'tires', 120),
    ('Tire Balancing', 'Checks and balances the wheel to address vibration caused by uneven weight, where suitable equipment is available.', 50.00, 75, 'tires', 130),
    ('Wheel Alignment', 'Checks wheel alignment and makes supported
```

adjustments when the motorcycle does not track straight.', 50.00, 75, 'tires', 140),
    ('Wheel Bearing Replacement', 'Checks wheel operation and replaces confirmed worn or damaged bearings.', 50.00, 120, 'tires', 150),
    ('Front Fork Service', 'Inspects and services front forks for leaks, noise, poor damping, or incorrect feel, including seal replacement where appropriate.', 150.00, 150, 'suspension', 160),
    ('Rear Shock Service', 'Checks and services the rear shock where its design supports servicing; replacement may be required for some units.', 100.00, 120, 'suspension', 170),
    ('Suspension Tuning', 'Checks available suspension settings and adjusts them where the motorcycle supports adjustment.', 200.00, 90, 'suspension', 180),
    ('Suspension Rebuild', 'Disassembles, checks, and rebuilds a serviceable worn or leaking suspension unit; additional parts or work may be needed.', 150.00, 210, 'suspension', 190),
    ('Steering Bearing Replacement', 'Checks steering bearings and replaces worn bearings that cause stiffness, looseness, catching, or knocking.', 250.00, 210, 'suspension', 200),
    ('Battery Testing', 'Tests battery charge retention and identifies whether further charging-system checks are needed.', 20.00, 60, 'electrical', 210),
    ('Battery Replacement', 'Installs a compatible replacement battery after confirming a failed or unreliable battery; the battery cost may be separate.', 50.00, 75, 'electrical', 220),
    ('Electrical Diagnostics FI', 'Checks relevant electrical connections and components for fuel-injected motorcycles with warning lights, starting trouble, or intermittent functions.', 150.00, 120, 'electrical', 230),
    ('Electrical Wiring Repair', 'Checks related wiring and connections and repairs accessible electrical faults; difficult faults may take longer.', 200.00, 150, 'electrical', 240),
    ('ECU Diagnostic Scan', 'Scans compatible motorcycles for stored error codes and explains what the results may indicate.', 250.00, 90, 'electrical', 250),
    ('Spark Plug Replacement', 'Checks and replaces a compatible spark plug for hard starting, uneven running, or misfiring.', 50.00, 75, 'engine', 260),
    ('Air Filter Replacement', 'Accesses the filter housing and installs a suitable replacement filter when needed.', 50.00, 75, 'engine', 270),
    ('Carburetor Cleaning', 'Cleans serviceable carburetor passages and components for hard starting, stalling, uneven idle, or poor running.', 100.00, 120, 'engine', 280),
    ('Carburetor Tuning', 'Checks and adjusts carburetor settings for rough idle or poor running; worn parts may require separate work.', 100.00, 90, 'engine', 290),
    ('Throttle Body Cleaning', 'Cleans the throttle body and accessible air passages on fuel-injected motorcycles; injector cleaning is separate work.', 250.00, 120, 'engine', 300),
    ('Fuel System Cleaning', 'Checks and cleans fuel-delivery components included in the shop procedure; injector removal may be separate work.', 100.00, 120, 'engine', 310),

('Radiator Cleaning', 'Carefully cleans external radiator fins to remove dirt, leaves, or mud that can restrict airflow.', 100.00, 90, 'engine', 320),
('Engine Diagnostic', 'Performs appropriate checks and tests to identify the cause of engine noise, power loss, rough running, smoke, or similar concerns.', 150.00, 120, 'engine', 330),
('Motorcycle General Inspection', 'Checks key areas including brakes, tires, lights, chain, fluid levels, and visible leaks, then explains recommended next steps.', 500.00, 90, 'inspection', 350),
('Headlight/Lighting Replacement', 'Checks the bulb and basic connection, then replaces a faulty compatible lighting component when needed; parts may be separate.', 250.00, 75, 'electrical', 360),
('Starter System Diagnosis & Repair', 'Checks battery connections, starter relay, wiring, and starter motor for clicking, no-start, or intermittent-starting concerns.', 150.00, 120, 'electrical', 370),
('Charging System Repair', 'Checks and repairs a faulty charging-system connection or serviceable component; parts may be separate.', 100.00, 150, 'electrical', 380),
('Charging System Testing', 'Tests the battery and charging output to determine whether the motorcycle is charging correctly.', 100.00, 75, 'electrical', 390),
('Horn Replacement', 'Checks the horn and its connection, then replaces a faulty compatible horn when needed; the replacement part may be separate.', 100.00, 75, 'electrical', 400),
('Fuel Pump Inspection/Repair', 'Checks the fuel pump and related connections for hard starting, stalling, or suspected fuel-delivery faults.', 100.00, 120, 'engine', 410),
('Water Pump Inspection/Repair', 'Inspects the water pump and related parts for coolant leaks or overheating and recommends repair or replacement as needed.', 200.00, 150, 'engine', 420),
('Transmission Repair', 'Diagnoses difficult shifting, gear slipping, and abnormal drivetrain noises; major internal repairs can take longer and cost more.', 250.00, 270, 'drivetrain', 430),
('Brake Shoe Replacement', 'Removes the wheel, replaces worn drum-brake shoes, and adjusts the drum for proper stopping performance.', 70.00, 90, 'brakes', 440),
('Brake Pad Replacement', 'Replaces worn disc-brake pads and checks disc condition before reassembly.', 50.00, 75, 'brakes', 450),
('Magneto Cleaning', 'Cleans magneto ignition contacts and components and checks ignition output for hard starting, weak spark, or rough idle.', 50.00, 60, 'engine', 460),
('Crankshaft Bearing Replacement', 'Opens the crankcase, inspects the crankshaft assembly, and replaces worn main bearings; additional internal parts may be needed.', 700.00, 300, 'engine', 470),
('CVT Roller / Fly Weight Replacement', 'Opens the CVT cover, replaces worn rollers or fly weights, and checks belt condition.', 150.00, 105, 'drivetrain',

```
480),
    ('Handlebar Switch Replacement', 'Checks switch wiring and replaces a faulty handlebar switch assembly for affected horn, signal, lighting, or starter controls.', 100.00, 75, 'electrical', 490),
    ('Throttle Cable Replacement/Adjustment', 'Adjusts or replaces a stiff, sticky, loose, or slow-returning throttle cable and checks smooth operation.', 50.00, 60, 'engine', 500),
    ('Muffler/Exhaust Replacement', 'Removes and installs a compatible muffler or exhaust, then checks for leaks; the replacement part may be separate.', 100.00, 90, 'engine', 510),
    ('Racing Carburetor Installation', 'Installs and base-tunes a customer-supplied performance carburetor; additional fine-tuning may require a follow-up visit.', 300.00, 150, 'upgrades', 520),
    ('Frame/Crash Slider Installation', 'Fits and secures customer-supplied frame or swingarm sliders to the appropriate mounting points.', 100.00, 60, 'upgrades', 530),
    ('Fan Spinner Installation', 'Fits and secures a customer-supplied decorative fan spinner on the front-wheel axle.', 50.00, 30, 'upgrades', 540),
    ('Accessory/Upgrade Parts Installation General', 'Fits and secures other customer-supplied accessories or upgrades, such as mirrors, guards, grips, or lighting kits.', 0.00, 120, 'upgrades', 550)
  )
  INSERT INTO public.services (
    name,
    description,
    price,
    duration_minutes,
    category,
    is_active,
    is_archived,
    archived_at,
    sort_order
  )
  SELECT
    name,
    description,
    price,
    duration_minutes,
    category,
    true,
    false,
    NULL,
    sort_order
  FROM replacement_services
  ON CONFLICT (name) DO UPDATE
```

```
  SET
    description = EXCLUDED.description,
    price = EXCLUDED.price,
    duration_minutes = EXCLUDED.duration_minutes,
    category = EXCLUDED.category,
    is_active = true,
    is_archived = false,
    archived_at = NULL,
    sort_order = EXCLUDED.sort_order;
END
$$;
```

supabase/migrations/20260927100000_schedule_durable_admin_notifications.sql

---

```
-- Make service-progress notifications durable background work. Booking and
-- reschedule notifications are already created by their database transactions;
-- this scheduler covers due arrival and completion alerts when no admin is
-- signed in. Unread notifications are never automatically resolved.

DROP TRIGGER IF EXISTS
appointments_resolve_service_progress_notifications ON public.appointments;

CREATE OR REPLACE FUNCTION
public.enqueue_due_service_progress_notifications()
RETURNS integer
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
  v_created integer := 0;
  v_now timestamptz := now();
BEGIN
  -- Lock appointments before their notification rows so a concurrent admin
  -- action wins without producing a stale or duplicate alert.
  WITH due_appointments AS (
    SELECT appointment.id, appointment.reference_code,
appointment.customer_name
    FROM public.appointments AS appointment
    JOIN LATERAL (
      SELECT sum(service.duration_minutes)::integer AS
service_duration_minutes
      FROM public.appointment_services AS service
      WHERE service.appointment_id = appointment.id
    ) AS service_duration ON service_duration.service_duration_minutes > 0
    WHERE appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status = 'confirmed'
      AND appointment.service_started_at IS NULL
      AND (appointment.appointment_date + appointment.start_time) AT TIME
ZONE 'Asia/Manila' <= v_now
      AND COALESCE(appointment.arrival_notification_snoozed_until, v_now) <=
v_now
      AND NOT EXISTS (
        SELECT 1
        FROM public.notifications AS notification
```

```
      WHERE notification.appointment_id = appointment.id
        AND notification.type = 'service_arrival'
        AND notification.is_read = false
    )
  FOR UPDATE OF appointment SKIP LOCKED
),
inserted AS (
  INSERT INTO public.notifications (type, title, message, appointment_id)
  SELECT
    'service_arrival',
    'Customer expected: ' || due.reference_code,
    due.customer_name || ' is scheduled to arrive for service now.',
    due.id
  FROM due_appointments AS due
  ON CONFLICT DO NOTHING
  RETURNING 1
)
SELECT count(*) INTO v_created FROM inserted;

WITH due_appointments AS (
  SELECT appointment.id, appointment.reference_code,
appointment.customer_name
  FROM public.appointments AS appointment
  JOIN LATERAL (
    SELECT sum(service.duration_minutes)::integer AS
service_duration_minutes
    FROM public.appointment_services AS service
    WHERE service.appointment_id = appointment.id
  ) AS service_duration ON service_duration.service_duration_minutes > 0
  WHERE appointment.is_archived = false
    AND appointment.status = 'in_progress'
    AND appointment.service_started_at IS NOT NULL
    AND appointment.service_ended_at IS NULL
    AND appointment.service_started_at
      + make_interval(mins => service_duration.service_duration_minutes) <=
v_now
    AND COALESCE(appointment.completion_notification_snoozed_until,
v_now) <= v_now
    AND NOT EXISTS (
      SELECT 1
      FROM public.notifications AS notification
      WHERE notification.appointment_id = appointment.id
        AND notification.type = 'service_completion'
        AND notification.is_read = false
    )
```

```
    FOR UPDATE OF appointment SKIP LOCKED
  ),
  inserted AS (
    INSERT INTO public.notifications (type, title, message, appointment_id)
    SELECT
      'service_completion',
      'Service duration ended: ' || due.reference_code,
      'The expected service duration for ' || due.customer_name || ' has ended.',
      due.id
    FROM due_appointments AS due
    ON CONFLICT DO NOTHING
    RETURNING 1
  )
  SELECT v_created + count(*) INTO v_created FROM inserted;

  RETURN v_created;
END;
$$;

REVOKE ALL ON FUNCTION
public.enqueue_due_service_progress_notifications() FROM PUBLIC;
REVOKE ALL ON FUNCTION
public.enqueue_due_service_progress_notifications() FROM anon,
authenticated;
GRANT EXECUTE ON FUNCTION
public.enqueue_due_service_progress_notifications() TO service_role;

-- Keep the existing admin RPC as an immediate, authenticated catch-up path;
-- the background job uses the private function above while admins are away.
CREATE OR REPLACE FUNCTION
public.sync_service_progress_notifications()
RETURNS integer
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Only administrators can synchronize service progress
notifications.';
  END IF;

  RETURN public.enqueue_due_service_progress_notifications();
END;
$$;
```

```
REVOKE ALL ON FUNCTION public.sync_service_progress_notifications()
FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.sync_service_progress_notifications()
TO authenticated;

-- Mark an alert read only inside the explicit workflow action that handles it.
-- Status changes from other workflows leave untouched notification records
-- intact rather than dismissing them automatically.
CREATE OR REPLACE FUNCTION
public.manage_appointment_service_progress(
  p_appointment_id uuid,
  p_action text
)
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment public.appointments%ROWTYPE;
  v_action text := lower(btrim(COALESCE(p_action, '')));
  v_snooze_interval interval := interval '5 minutes';
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Only administrators can update service progress.';
  END IF;

  IF v_action NOT IN (
    'start', 'snooze_arrival', 'no_show', 'complete', 'snooze_completion'
  ) THEN
    RAISE EXCEPTION 'Choose a valid service progress action.';
  END IF;

  SELECT appointment.*
    INTO v_appointment
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
  FOR UPDATE;

  IF NOT FOUND THEN
    RAISE EXCEPTION 'This appointment is no longer available.';
  END IF;
```

```
  IF v_action = 'start' THEN
    IF v_appointment.status NOT IN ('pending', 'confirmed')
       OR v_appointment.service_started_at IS NOT NULL THEN
      RAISE EXCEPTION 'This appointment cannot be started.';
    END IF;

    UPDATE public.appointments
    SET status = 'in_progress',
        service_started_at = COALESCE(service_started_at, now()),
        service_ended_at = NULL,
        arrival_notification_snoozed_until = NULL
    WHERE id = v_appointment.id;

    UPDATE public.notifications
    SET is_read = true
    WHERE appointment_id = v_appointment.id
      AND type = 'service_arrival'
      AND is_read = false;

  ELSIF v_action = 'snooze_arrival' THEN
    IF v_appointment.status <> 'confirmed' OR v_appointment.service_started_at
IS NOT NULL THEN
      RAISE EXCEPTION 'This arrival reminder is no longer available.';
    END IF;
    IF v_appointment.arrival_notification_snooze_count >= 3 THEN
      RAISE EXCEPTION 'The maximum of three arrival snoozes has been
reached.';
    END IF;

    UPDATE public.appointments
    SET arrival_notification_snooze_count = arrival_notification_snooze_count +
1,
        arrival_notification_snoozed_until = now() + v_snooze_interval
    WHERE id = v_appointment.id;

    UPDATE public.notifications
    SET is_read = true
    WHERE appointment_id = v_appointment.id
      AND type = 'service_arrival'
      AND is_read = false;

  ELSIF v_action = 'no_show' THEN
    IF v_appointment.status <> 'confirmed' OR v_appointment.service_started_at
IS NOT NULL THEN
      RAISE EXCEPTION 'This appointment cannot be marked as no-show.';
```

```
    END IF;
    IF v_appointment.arrival_notification_snooze_count < 3 THEN
      RAISE EXCEPTION 'The appointment can be marked as no-show after three
arrival snoozes.';
    END IF;

    UPDATE public.appointments
    SET status = 'no_show',
        arrival_notification_snoozed_until = NULL
    WHERE id = v_appointment.id;

    UPDATE public.notifications
    SET is_read = true
    WHERE appointment_id = v_appointment.id
      AND type = 'service_arrival'
      AND is_read = false;

  ELSIF v_action = 'complete' THEN
    IF v_appointment.status <> 'in_progress'
       OR v_appointment.service_started_at IS NULL
       OR v_appointment.service_ended_at IS NOT NULL THEN
      RAISE EXCEPTION 'This service cannot be marked as completed.';
    END IF;

    UPDATE public.appointments
    SET status = 'completed',
        service_ended_at = COALESCE(service_ended_at, now()),
        completion_notification_snoozed_until = NULL
    WHERE id = v_appointment.id;

    UPDATE public.notifications
    SET is_read = true
    WHERE appointment_id = v_appointment.id
      AND type = 'service_completion'
      AND is_read = false;

  ELSE
    IF v_appointment.status <> 'in_progress'
       OR v_appointment.service_started_at IS NULL
       OR v_appointment.service_ended_at IS NOT NULL THEN
      RAISE EXCEPTION 'This completion reminder is no longer available.';
    END IF;

    UPDATE public.appointments
    SET completion_notification_snoozed_until = now() + v_snooze_interval
```

```
    WHERE id = v_appointment.id;

    UPDATE public.notifications
    SET is_read = true
    WHERE appointment_id = v_appointment.id
      AND type = 'service_completion'
      AND is_read = false;
  END IF;
END;
$$;

REVOKE ALL ON FUNCTION
public.manage_appointment_service_progress(uuid, text) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION
public.manage_appointment_service_progress(uuid, text) TO authenticated;

-- pg_cron runs in the database, independently of browser sessions. Enable the
-- supported extension and replace any prior job so this remains idempotent.
CREATE EXTENSION IF NOT EXISTS pg_cron;

DO $$
BEGIN
  PERFORM cron.unschedule(jobid)
  FROM cron.job
  WHERE jobname = 'enqueue-due-service-progress-notifications';

  PERFORM cron.schedule(
    'enqueue-due-service-progress-notifications',
    '* * * * *',
    'SELECT public.enqueue_due_service_progress_notifications();'
  );
END;
$$;
```

supabase/migrations/20260927110000_remove_reschedule_loop_variable_shadowing.sql

---

```
-- The loop variables are declared by PL/pgSQL's FOR statements. Declaring
-- them again makes db lint report shadowed, unused locals without changing the
-- rescheduling workflow.
CREATE OR REPLACE FUNCTION public.submit_reschedule_request(
  p_appointment_id uuid,
  p_request_id uuid,
  p_appointment_date date,
  p_start_time time,
  p_reason text
)
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment public.appointments%ROWTYPE;
  v_reference_code text;
  v_reference_available boolean := false;
  v_reference_alphabet constant text :=
'ABCDEFGHJKLMNPQRSTUVWXYZ23456789';
BEGIN
  IF p_request_id IS NULL OR p_appointment_date IS NULL OR p_start_time IS
NULL
     OR NULLIF(btrim(p_reason), '') IS NULL THEN
    RAISE EXCEPTION 'A date, time, and rescheduling reason are required.';
  END IF;

  SELECT appointment.*
    INTO v_appointment
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
  FOR UPDATE;

  IF NOT FOUND THEN
    RAISE EXCEPTION 'This appointment is no longer available.';
  END IF;
  IF v_appointment.pending_reschedule_request_id IS NOT NULL
     AND v_appointment.pending_reschedule_request_id <> p_request_id THEN
    RAISE EXCEPTION 'A reschedule request is already awaiting review.';
  END IF;
```

```
  IF v_appointment.reschedule_count >= 3 THEN
    RAISE EXCEPTION 'This appointment has reached the maximum of 3
reschedules.';
  END IF;
  IF v_appointment.status NOT IN ('pending', 'confirmed')
     OR v_appointment.rescheduled_to_appointment_id IS NOT NULL THEN
    RAISE EXCEPTION 'This appointment can no longer be rescheduled.';
  END IF;

  v_reference_code := v_appointment.pending_reschedule_reference_code;
  IF v_reference_code IS NULL THEN
    FOR v_attempt IN 1..10 LOOP
      v_reference_code := 'FRM-';
      FOR v_character IN 1..6 LOOP
        v_reference_code := v_reference_code || substr(
          v_reference_alphabet,
          1 + floor(random() * length(v_reference_alphabet))::integer,
          1
        );
      END LOOP;

      SELECT NOT EXISTS (
        SELECT 1
        FROM public.appointments
        WHERE reference_code = v_reference_code
           OR pending_reschedule_reference_code = v_reference_code
      ) INTO v_reference_available;
      EXIT WHEN v_reference_available;
    END LOOP;

    IF NOT v_reference_available THEN
      RAISE EXCEPTION 'Could not generate a unique booking reference. Please
try again.';
    END IF;
  END IF;

  UPDATE public.appointments
  SET pending_reschedule_request_id = p_request_id,
      pending_reschedule_date = p_appointment_date,
      pending_reschedule_start_time = p_start_time,
      pending_reschedule_reference_code = v_reference_code,
      pending_reschedule_reason = NULLIF(btrim(p_reason), '')
  WHERE id = v_appointment.id;

  IF v_appointment.pending_reschedule_request_id IS NULL THEN
```

```
    INSERT INTO public.notifications (type, title, message, appointment_id)
    VALUES (
      'reschedule_request',
      'Reschedule request ' || v_appointment.reference_code,
      v_appointment.customer_name || ' requested ' ||
to_char(p_appointment_date, 'Mon FMDD, YYYY')
        || ' at ' || to_char(p_start_time, 'HH12:MI AM') || '. New reference: '
        || v_reference_code || '. Reason: ' || btrim(p_reason),
      v_appointment.id
    );
  END IF;
END;
$$;
```

supabase/migrations/20260928100000_notify_multiple_active_service_bookings.sql

---

```
-- Notify administrators when a customer has two or more active bookings for
-- the same service on different dates. The alert is generated at commit time
-- so public bookings, reschedules, and admin edits all use the final state of
-- the transaction.

CREATE TABLE IF NOT EXISTS public.customer_service_booking_alert_state (
  customer_phone text NOT NULL,
  service_key text NOT NULL,
  is_threshold_active boolean NOT NULL DEFAULT false,
  alert_cycle integer NOT NULL DEFAULT 0 CHECK (alert_cycle >= 0),
  updated_at timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (customer_phone, service_key)
);

CREATE TABLE IF NOT EXISTS public.customer_service_booking_alerts (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_phone text NOT NULL,
  service_key text NOT NULL,
  alert_cycle integer NOT NULL CHECK (alert_cycle > 0),
  active_appointment_ids uuid[] NOT NULL CHECK
(cardinality(active_appointment_ids) >= 2),
  notification_id uuid REFERENCES public.notifications(id) ON DELETE SET
NULL,
  created_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (customer_phone, service_key, alert_cycle, active_appointment_ids)
);

-- These tables are an internal deduplication ledger. Only the trigger function
-- writes them; administrators continue to use the existing notifications UI.
REVOKE ALL ON TABLE public.customer_service_booking_alert_state FROM
anon, authenticated;
REVOKE ALL ON TABLE public.customer_service_booking_alerts FROM anon,
authenticated;
GRANT ALL ON TABLE public.customer_service_booking_alert_state TO
service_role;
GRANT ALL ON TABLE public.customer_service_booking_alerts TO
service_role;
ALTER TABLE public.customer_service_booking_alert_state ENABLE ROW
LEVEL SECURITY;
ALTER TABLE public.customer_service_booking_alerts ENABLE ROW LEVEL
SECURITY;
```

```
CREATE OR REPLACE FUNCTION
public.refresh_customer_service_booking_alert(
  p_customer_phone text,
  p_service_id uuid,
  p_service_name text,
  p_preferred_appointment_id uuid,
  p_customer_name text
)
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
  v_service_key text;
  v_service_label text;
  v_customer_label text;
  v_active_appointment_ids uuid[];
  v_active_booking_count integer;
  v_distinct_booking_date_count integer;
  v_schedule_details text;
  v_notification_appointment_id uuid;
  v_alert_cycle integer;
  v_threshold_was_active boolean;
  v_notification_id uuid;
BEGIN
  IF NULLIF(btrim(p_customer_phone), '') IS NULL
     OR (p_service_id IS NULL AND NULLIF(btrim(p_service_name), '') IS NULL)
THEN
    RETURN;
  END IF;

  -- A service ID is the canonical identity. The name fallback supports legacy
  -- appointment-service rows that do not have a service ID.
  v_service_key := COALESCE(
    'id:' || p_service_id::text,
    'name:' || lower(btrim(p_service_name))
  );
  v_service_label := COALESCE(
    NULLIF(btrim(p_service_name), ''),
    (SELECT service.name FROM public.services AS service WHERE service.id
= p_service_id),
    'this service'
  );
```

```
  -- Serializing per customer lets concurrent bookings see the committed
  -- booking that reached the threshold, while avoiding duplicate alerts.
  PERFORM pg_advisory_xact_lock(
    hashtext('customer-service-booking-alert:' || btrim(p_customer_phone))
  );

  WITH active_bookings AS (
    SELECT DISTINCT
      appointment.id,
      appointment.appointment_date,
      appointment.start_time,
      appointment.customer_name
    FROM public.appointments AS appointment
    JOIN public.appointment_services AS appointment_service
      ON appointment_service.appointment_id = appointment.id
    WHERE appointment.phone = btrim(p_customer_phone)
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status NOT IN ('completed', 'cancelled', 'rejected')
      AND (
        appointment_service.service_id = p_service_id
        OR (
          p_service_id IS NULL
          AND appointment_service.service_id IS NULL
          AND appointment_service.service_name = p_service_name
        )
      )
  )
  SELECT
    COALESCE(array_agg(appointment.id ORDER BY appointment.id),
ARRAY[]::uuid[]),
    count(*)::integer,
    count(DISTINCT appointment.appointment_date)::integer,
    string_agg(
      to_char(appointment.appointment_date, 'Mon FMDD, YYYY')
        || ' at ' || to_char(appointment.start_time, 'FMHH12:MI AM'),
      '; ' ORDER BY appointment.appointment_date, appointment.start_time,
appointment.id
    ),
    COALESCE(
      max(appointment.customer_name) FILTER (WHERE appointment.id =
p_preferred_appointment_id),
      max(appointment.customer_name),
      NULLIF(btrim(p_customer_name), ''),
      btrim(p_customer_phone)
```

```
    ),
    COALESCE(
      min(appointment.id) FILTER (WHERE appointment.id =
p_preferred_appointment_id),
      (array_agg(
        appointment.id
        ORDER BY appointment.appointment_date, appointment.start_time,
appointment.id
      ))[1]
    )
  INTO
    v_active_appointment_ids,
    v_active_booking_count,
    v_distinct_booking_date_count,
    v_schedule_details,
    v_customer_label,
    v_notification_appointment_id
  FROM active_bookings AS appointment;

  INSERT INTO public.customer_service_booking_alert_state (
    customer_phone,
    service_key
  )
  VALUES (btrim(p_customer_phone), v_service_key)
  ON CONFLICT (customer_phone, service_key) DO NOTHING;

  SELECT state.is_threshold_active, state.alert_cycle
  INTO v_threshold_was_active, v_alert_cycle
  FROM public.customer_service_booking_alert_state AS state
  WHERE state.customer_phone = btrim(p_customer_phone)
    AND state.service_key = v_service_key
  FOR UPDATE;

  IF v_active_booking_count < 2 OR v_distinct_booking_date_count < 2 THEN
    UPDATE public.customer_service_booking_alert_state
    SET is_threshold_active = false,
        updated_at = now()
    WHERE customer_phone = btrim(p_customer_phone)
      AND service_key = v_service_key;
    RETURN;
  END IF;

  -- A new cycle begins only after the customer has dropped below two active
  -- bookings. Within a cycle, each distinct set of booking IDs is alerted once.
  IF NOT v_threshold_was_active THEN
```

```
    v_alert_cycle := v_alert_cycle + 1;

    UPDATE public.customer_service_booking_alert_state
    SET is_threshold_active = true,
        alert_cycle = v_alert_cycle,
        updated_at = now()
    WHERE customer_phone = btrim(p_customer_phone)
      AND service_key = v_service_key;
  END IF;

  INSERT INTO public.customer_service_booking_alerts (
    customer_phone,
    service_key,
    alert_cycle,
    active_appointment_ids
  )
  VALUES (
    btrim(p_customer_phone),
    v_service_key,
    v_alert_cycle,
    v_active_appointment_ids
  )
  ON CONFLICT (customer_phone, service_key, alert_cycle,
active_appointment_ids) DO NOTHING
  RETURNING id INTO v_notification_id;

  -- A NULL returned ID means this exact active-booking set has already been
  -- reported during the current threshold cycle.
  IF v_notification_id IS NULL THEN
    RETURN;
  END IF;

  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES (
    'multiple_active_service_bookings',
    v_active_booking_count || ' active ' || v_service_label || ' bookings for ' ||
v_customer_label,
    v_customer_label || ' (' || btrim(p_customer_phone) || ') has '
      || v_active_booking_count || ' active bookings for ' || v_service_label || ': '
      || v_schedule_details || '.',
    v_notification_appointment_id
  )
  RETURNING id INTO v_notification_id;

  UPDATE public.customer_service_booking_alerts
```

```
  SET notification_id = v_notification_id
  WHERE customer_phone = btrim(p_customer_phone)
    AND service_key = v_service_key
    AND alert_cycle = v_alert_cycle
    AND active_appointment_ids = v_active_appointment_ids;
END;
$$;

CREATE OR REPLACE FUNCTION
public.sync_customer_service_booking_alert()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
  v_appointment_id uuid;
  v_customer_phone text;
  v_customer_name text;
  v_previous_customer_phone text;
  v_previous_customer_name text;
  v_service record;
BEGIN
  IF TG_TABLE_NAME = 'appointments' THEN
    IF TG_OP = 'DELETE' THEN
      v_appointment_id := OLD.id;
      v_customer_phone := OLD.phone;
      v_customer_name := OLD.customer_name;
    ELSE
      v_appointment_id := NEW.id;
      v_customer_phone := NEW.phone;
      v_customer_name := NEW.customer_name;
      IF TG_OP = 'UPDATE' AND OLD.phone IS DISTINCT FROM NEW.phone
THEN
        v_previous_customer_phone := OLD.phone;
        v_previous_customer_name := OLD.customer_name;
      END IF;
    END IF;

    FOR v_service IN
      SELECT DISTINCT appointment_service.service_id,
appointment_service.service_name
      FROM public.appointment_services AS appointment_service
      WHERE appointment_service.appointment_id = v_appointment_id
    LOOP
```

```
    -- Refresh both customer scopes if an administrator corrects a phone
    -- number. The lexical order prevents two concurrent corrections from
    -- taking the customer advisory locks in opposite directions.
    IF v_previous_customer_phone IS NOT NULL
       AND COALESCE(v_previous_customer_phone, '') <
COALESCE(v_customer_phone, '') THEN
      PERFORM public.refresh_customer_service_booking_alert(
        v_previous_customer_phone,
        v_service.service_id,
        v_service.service_name,
        v_appointment_id,
        v_previous_customer_name
      );
    END IF;

    PERFORM public.refresh_customer_service_booking_alert(
      v_customer_phone,
      v_service.service_id,
      v_service.service_name,
      v_appointment_id,
      v_customer_name
    );

    IF v_previous_customer_phone IS NOT NULL
       AND COALESCE(v_previous_customer_phone, '') >=
COALESCE(v_customer_phone, '') THEN
      PERFORM public.refresh_customer_service_booking_alert(
        v_previous_customer_phone,
        v_service.service_id,
        v_service.service_name,
        v_appointment_id,
        v_previous_customer_name
      );
    END IF;
  END LOOP;
 ELSE
  IF TG_OP = 'DELETE' THEN
    v_appointment_id := OLD.appointment_id;
  ELSE
    v_appointment_id := NEW.appointment_id;
  END IF;

  SELECT appointment.phone, appointment.customer_name
  INTO v_customer_phone, v_customer_name
  FROM public.appointments AS appointment
```

```
    WHERE appointment.id = v_appointment_id;

    -- Cascading deletion can remove the parent before this deferred trigger
    -- executes. It does not occur in the normal soft-archive workflow; if it
    -- does, there is no remaining active booking to alert for this row.
    IF NOT FOUND THEN
      IF TG_OP = 'DELETE' THEN
        RETURN OLD;
      END IF;
      RETURN NEW;
    END IF;

    IF TG_OP <> 'DELETE' THEN
      PERFORM public.refresh_customer_service_booking_alert(
        v_customer_phone,
        NEW.service_id,
        NEW.service_name,
        v_appointment_id,
        v_customer_name
      );
    END IF;

    IF TG_OP <> 'INSERT' THEN
      PERFORM public.refresh_customer_service_booking_alert(
        v_customer_phone,
        OLD.service_id,
        OLD.service_name,
        v_appointment_id,
        v_customer_name
      );
    END IF;
  END IF;

  IF TG_OP = 'DELETE' THEN
    RETURN OLD;
  END IF;
  RETURN NEW;
END;
$$;

REVOKE ALL ON FUNCTION
public.refresh_customer_service_booking_alert(text, uuid, text, uuid, text) FROM
PUBLIC;
REVOKE ALL ON FUNCTION public.sync_customer_service_booking_alert()
FROM PUBLIC;
```

```
DROP TRIGGER IF EXISTS
appointments_sync_customer_service_booking_alert ON public.appointments;
CREATE CONSTRAINT TRIGGER
appointments_sync_customer_service_booking_alert
  AFTER INSERT OR UPDATE OR DELETE ON public.appointments
  DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW
  EXECUTE FUNCTION public.sync_customer_service_booking_alert();

DROP TRIGGER IF EXISTS
appointment_services_sync_customer_service_booking_alert ON
public.appointment_services;
CREATE CONSTRAINT TRIGGER
appointment_services_sync_customer_service_booking_alert
  AFTER INSERT OR UPDATE OR DELETE ON public.appointment_services
  DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW
  EXECUTE FUNCTION public.sync_customer_service_booking_alert();
```

supabase/migrations/20260928110000_add_customer_no_show_history_warnings.sql

---

```
-- Record the customer's no-show history when a booking is created, then use
-- that immutable snapshot to show administrators a warning on the booking.
-- The history count intentionally includes archived appointments: archiving a
-- record must not erase a customer's recorded no-shows.

ALTER TABLE public.appointments
  ADD COLUMN IF NOT EXISTS no_show_count_at_booking integer NOT NULL
DEFAULT 0
  CHECK (no_show_count_at_booking >= 0);

CREATE OR REPLACE FUNCTION
public.capture_customer_no_show_count_at_booking()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
BEGIN
  SELECT count(*)::integer
  INTO NEW.no_show_count_at_booking
  FROM public.appointments AS appointment
  WHERE appointment.phone = NEW.phone
    -- A row can have only one status, so cancelled and completed appointments
    -- are never included in this recorded no-show history.
    AND appointment.status = 'no_show';

  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS appointments_capture_customer_no_show_count
ON public.appointments;
CREATE TRIGGER appointments_capture_customer_no_show_count
  BEFORE INSERT ON public.appointments
  FOR EACH ROW
  EXECUTE FUNCTION public.capture_customer_no_show_count_at_booking();

CREATE OR REPLACE FUNCTION
public.notify_customer_no_show_threshold()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
```

```
SET search_path = public
AS $$
DECLARE
  v_no_show_count integer;
BEGIN
  -- This supports both the normal admin workflow and any trusted import that
  -- creates an already-recorded no-show. Repeated updates to a no-show row
do
  -- not create duplicate alerts.
  IF NEW.status <> 'no_show'
     OR (TG_OP = 'UPDATE' AND OLD.status = 'no_show') THEN
    RETURN NEW;
  END IF;

  -- Concurrent status changes for the same customer must see one another
  -- before deciding whether this is the threshold-crossing no-show.
  PERFORM pg_advisory_xact_lock(
    hashtext('customer-no-show-threshold:' || COALESCE(NEW.phone, ''))
  );

  SELECT count(*)::integer
  INTO v_no_show_count
  FROM public.appointments AS appointment
  WHERE appointment.phone = NEW.phone
    AND appointment.status = 'no_show';

  -- Notify once when the customer exceeds two recorded no-shows. If the count
  -- later falls below the threshold and reaches it again, the new crossing is
  -- correctly reported again.
  IF v_no_show_count = 3 THEN
    INSERT INTO public.notifications (type, title, message, appointment_id)
    VALUES (
      'customer_no_show_threshold',
      'Customer no-show threshold reached',
      COALESCE(NULLIF(btrim(NEW.customer_name), ''), NEW.phone)
        || ' (' || NEW.phone || ') now has ' || v_no_show_count
        || ' recorded no-shows. Review the customer''s booking history before
accepting future work.',
      NEW.id
    );
  END IF;

  RETURN NEW;
END;
$$;
```

```
DROP TRIGGER IF EXISTS appointments_notify_customer_no_show_threshold
ON public.appointments;
CREATE TRIGGER appointments_notify_customer_no_show_threshold
  AFTER INSERT OR UPDATE OF status ON public.appointments
  FOR EACH ROW
  EXECUTE FUNCTION public.notify_customer_no_show_threshold();

REVOKE ALL ON FUNCTION
public.capture_customer_no_show_count_at_booking() FROM PUBLIC;
REVOKE ALL ON FUNCTION public.notify_customer_no_show_threshold()
FROM PUBLIC;
```

supabase/migrations/20260928120000_enforce_single_active_admin_session.sql

---

```
-- Allow one active Supabase Auth session per administrator. The database lease
-- is bound to the signed JWT session_id claim, so copying requests, refreshing
-- the page, or bypassing the UI cannot grant a second session admin access.

CREATE TABLE IF NOT EXISTS public.admin_session_leases (
  admin_id uuid PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  session_id uuid NOT NULL,
  expires_at timestamptz NOT NULL,
  last_seen_at timestamptz NOT NULL DEFAULT now(),
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS public.admin_session_handover_tokens (
  token uuid PRIMARY KEY,
  admin_id uuid NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  session_id uuid NOT NULL,
  expires_at timestamptz NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE INDEX IF NOT EXISTS admin_session_handover_tokens_expiry_idx
  ON public.admin_session_handover_tokens (expires_at);

ALTER TABLE public.admin_session_leases ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.admin_session_handover_tokens ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON TABLE public.admin_session_leases FROM anon, authenticated;
REVOKE ALL ON TABLE public.admin_session_handover_tokens FROM anon, authenticated;
GRANT ALL ON TABLE public.admin_session_leases TO service_role;
GRANT ALL ON TABLE public.admin_session_handover_tokens TO service_role;

CREATE OR REPLACE FUNCTION public.current_admin_auth_session_id()
RETURNS uuid
LANGUAGE sql
STABLE
SECURITY DEFINER
```

```
SET search_path = public, auth
AS $$
  SELECT NULLIF(auth.jwt() ->> 'session_id', '')::uuid;
$$;

CREATE OR REPLACE FUNCTION
public.is_current_admin_session(p_admin_id uuid)
RETURNS boolean
LANGUAGE sql
STABLE
SECURITY DEFINER
SET search_path = public, auth
AS $$
  SELECT
    p_admin_id IS NOT NULL
    AND p_admin_id = auth.uid()
    AND EXISTS (
      SELECT 1
      FROM public.user_roles AS role
      WHERE role.user_id = p_admin_id
        AND role.role::text = 'admin'
    )
    AND EXISTS (
      SELECT 1
      FROM public.admin_session_leases AS lease
      JOIN auth.sessions AS session
        ON session.id = lease.session_id
       AND session.user_id = lease.admin_id
      WHERE lease.admin_id = p_admin_id
        AND lease.session_id = public.current_admin_auth_session_id()
        AND lease.expires_at > now()
    );
$$;

-- Existing RLS policies created before the role enum migration still point to
-- has_role_legacy. Update both role helpers so every protected admin path uses
-- the same server-side session restriction.
CREATE OR REPLACE FUNCTION public.has_role_legacy(
  _user_id uuid,
  _role public.app_role_legacy
)
RETURNS boolean
LANGUAGE sql
STABLE
SECURITY DEFINER
```

```
SET search_path = public
AS $$
  SELECT EXISTS (
    SELECT 1
    FROM public.user_roles
    WHERE user_id = _user_id
      AND role::text = _role::text
  )
  AND public.is_current_admin_session(_user_id);
$$;

CREATE OR REPLACE FUNCTION public.has_role(
  _user_id uuid,
  _role public.app_role
)
RETURNS boolean
LANGUAGE sql
STABLE
SECURITY DEFINER
SET search_path = public
AS $$
  SELECT EXISTS (
    SELECT 1
    FROM public.user_roles
    WHERE user_id = _user_id
      AND role = _role
  )
  AND public.is_current_admin_session(_user_id);
$$;

CREATE OR REPLACE FUNCTION public.claim_admin_session(
  p_handover_token uuid DEFAULT NULL
)
RETURNS boolean
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public, auth
AS $$
DECLARE
  v_admin_id uuid := auth.uid();
  v_session_id uuid := public.current_admin_auth_session_id();
  v_existing_lease public.admin_session_leases%ROWTYPE;
  v_has_valid_handover boolean := false;
BEGIN
  IF v_admin_id IS NULL
```

```
    OR v_session_id IS NULL
    OR NOT EXISTS (
      SELECT 1
      FROM public.user_roles AS role
      WHERE role.user_id = v_admin_id
        AND role.role::text = 'admin'
    ) THEN
   RAISE EXCEPTION 'Only administrators can start an admin session.';
  END IF;

  -- Serialize all claim, renewal, and release decisions for one account.
  PERFORM pg_advisory_xact_lock(hashtext('admin-session-lease:' ||
v_admin_id::text));
  DELETE FROM public.admin_session_handover_tokens WHERE expires_at
<= now();

  SELECT lease.*
  INTO v_existing_lease
  FROM public.admin_session_leases AS lease
  WHERE lease.admin_id = v_admin_id
  FOR UPDATE;

  IF FOUND
    AND v_existing_lease.session_id <> v_session_id
    AND v_existing_lease.expires_at > now()
    AND EXISTS (
      SELECT 1
      FROM auth.sessions AS session
      WHERE session.id = v_existing_lease.session_id
        AND session.user_id = v_admin_id
    ) THEN
   IF p_handover_token IS NOT NULL THEN
     SELECT true
     INTO v_has_valid_handover
     FROM public.admin_session_handover_tokens AS handover
     WHERE handover.token = p_handover_token
       AND handover.admin_id = v_admin_id
       AND handover.session_id = v_existing_lease.session_id
       AND handover.expires_at > now()
     FOR UPDATE;
   END IF;

   IF NOT COALESCE(v_has_valid_handover, false) THEN
     RAISE EXCEPTION
       'An administrator session is already active. Sign out from the active device
```

```
or wait for it to expire before signing in here.';
    END IF;

    DELETE FROM public.admin_session_handover_tokens
    WHERE token = p_handover_token;
  END IF;

  INSERT INTO public.admin_session_leases (
    admin_id,
    session_id,
    expires_at,
    last_seen_at
  )
  VALUES (
    v_admin_id,
    v_session_id,
    now() + interval '3 minutes',
    now()
  )
  ON CONFLICT (admin_id) DO UPDATE
  SET session_id = EXCLUDED.session_id,
      expires_at = EXCLUDED.expires_at,
      last_seen_at = EXCLUDED.last_seen_at;

  RETURN true;
END;
$$;

CREATE OR REPLACE FUNCTION public.renew_admin_session()
RETURNS boolean
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public, auth
AS $$
DECLARE
  v_admin_id uuid := auth.uid();
  v_session_id uuid := public.current_admin_auth_session_id();
BEGIN
  IF v_admin_id IS NULL OR v_session_id IS NULL THEN
    RETURN false;
  END IF;

  UPDATE public.admin_session_leases AS lease
  SET expires_at = now() + interval '3 minutes',
      last_seen_at = now()
```

```
  WHERE lease.admin_id = v_admin_id
    AND lease.session_id = v_session_id
    AND lease.expires_at > now()
    AND EXISTS (
      SELECT 1
      FROM auth.sessions AS session
      WHERE session.id = v_session_id
        AND session.user_id = v_admin_id
    );

  RETURN FOUND;
END;
$$;

CREATE OR REPLACE FUNCTION public.release_admin_session()
RETURNS boolean
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public, auth
AS $$
DECLARE
  v_admin_id uuid := auth.uid();
  v_session_id uuid := public.current_admin_auth_session_id();
BEGIN
  IF v_admin_id IS NULL OR v_session_id IS NULL THEN
    RETURN false;
  END IF;

  DELETE FROM public.admin_session_handover_tokens
  WHERE admin_id = v_admin_id
    AND session_id = v_session_id;

  DELETE FROM public.admin_session_leases
  WHERE admin_id = v_admin_id
    AND session_id = v_session_id;

  RETURN FOUND;
END;
$$;

CREATE OR REPLACE FUNCTION public.create_admin_session_handover()
RETURNS uuid
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public, auth
```

```
AS $$
DECLARE
  v_admin_id uuid := auth.uid();
  v_session_id uuid := public.current_admin_auth_session_id();
  v_token uuid := gen_random_uuid();
BEGIN
  IF NOT public.is_current_admin_session(v_admin_id) THEN
    RAISE EXCEPTION 'Your administrator session is no longer active. Please
sign in again.';
  END IF;

  DELETE FROM public.admin_session_handover_tokens
  WHERE expires_at <= now()
     OR (admin_id = v_admin_id AND session_id = v_session_id);

  INSERT INTO public.admin_session_handover_tokens (token, admin_id,
session_id, expires_at)
  VALUES (v_token, v_admin_id, v_session_id, now() + interval '2 minutes');

  RETURN v_token;
END;
$$;

CREATE OR REPLACE FUNCTION public.validate_admin_session()
RETURNS boolean
LANGUAGE sql
STABLE
SECURITY DEFINER
SET search_path = public
AS $$
  SELECT public.is_current_admin_session(auth.uid());
$$;

REVOKE ALL ON FUNCTION public.current_admin_auth_session_id() FROM
PUBLIC;
REVOKE ALL ON FUNCTION public.is_current_admin_session(uuid) FROM
PUBLIC;
REVOKE ALL ON FUNCTION public.claim_admin_session(uuid) FROM
PUBLIC;
REVOKE ALL ON FUNCTION public.renew_admin_session() FROM PUBLIC;
REVOKE ALL ON FUNCTION public.release_admin_session() FROM PUBLIC;
REVOKE ALL ON FUNCTION public.create_admin_session_handover() FROM
PUBLIC;
REVOKE ALL ON FUNCTION public.validate_admin_session() FROM PUBLIC;
REVOKE ALL ON FUNCTION public.has_role(uuid, public.app_role) FROM
```

```
PUBLIC;
REVOKE ALL ON FUNCTION public.has_role_legacy(uuid,
public.app_role_legacy) FROM PUBLIC;

GRANT EXECUTE ON FUNCTION public.claim_admin_session(uuid) TO
authenticated;
GRANT EXECUTE ON FUNCTION public.renew_admin_session() TO
authenticated;
GRANT EXECUTE ON FUNCTION public.release_admin_session() TO
authenticated;
GRANT EXECUTE ON FUNCTION public.create_admin_session_handover() TO
authenticated;
GRANT EXECUTE ON FUNCTION public.validate_admin_session() TO
authenticated;
GRANT EXECUTE ON FUNCTION public.has_role(uuid, public.app_role) TO
authenticated, service_role;
GRANT EXECUTE ON FUNCTION public.has_role_legacy(uuid,
public.app_role_legacy) TO authenticated, service_role;
```

supabase/migrations/20260928130000_restore_legacy_booking_rpc_compatibility.sql

---

```
-- Keep the pre-multi-day customer client compatible while a frontend release
-- rolls out. That client does not send p_continuation_segments, so PostgREST
-- otherwise reports create_booking_atomic as missing even though the current
-- 22-argument function is present.
CREATE OR REPLACE FUNCTION public.create_booking_atomic(
  p_reference_code text,
  p_booking_request_id uuid,
  p_customer_name text,
  p_phone text,
  p_moto_brand text,
  p_moto_model text,
  p_moto_variant text,
  p_moto_year integer,
  p_plate_number text,
  p_appointment_date date,
  p_start_time time,
  p_notes text,
  p_total_estimate numeric,
  p_booking_duration_minutes integer,
  p_assigned_crew_id uuid,
  p_services jsonb,
  p_notification_title text,
  p_notification_message text,
  p_first_name text,
  p_middle_name text,
  p_last_name text
)
RETURNS TABLE (appointment_id uuid, reference_code text)
LANGUAGE sql
SECURITY INVOKER
SET search_path = public
AS $$
  SELECT created.appointment_id, created.reference_code
  FROM public.create_booking_atomic(
    p_reference_code,
    p_booking_request_id,
    p_customer_name,
    p_phone,
    p_moto_brand,
    p_moto_model,
    p_moto_variant,
    p_moto_year,
```

```
    p_plate_number,
    p_appointment_date,
    p_start_time,
    p_notes,
    p_total_estimate,
    p_booking_duration_minutes,
    p_assigned_crew_id,
    p_services,
    '[]'::jsonb,
    p_notification_title,
    p_notification_message,
    p_first_name,
    p_middle_name,
    p_last_name
  ) AS created;
$$;

REVOKE ALL ON FUNCTION public.create_booking_atomic(
  text, uuid, text, text, text, text, text, integer, text, date, time, text,
  numeric, integer, uuid, jsonb, text, text, text, text, text
) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.create_booking_atomic(
  text, uuid, text, text, text, text, text, integer, text, date, time, text,
  numeric, integer, uuid, jsonb, text, text, text, text, text
) TO service_role;

NOTIFY pgrst, 'reload schema';
```

supabase/migrations/20260928140000_fix_uuid_aggregate_in_booking_alert.sql

---

```
-- PostgreSQL has no built-in min(uuid) aggregate. The original alert trigger
-- executed it for every appointment-service insert, which rolled back normal
-- booking creation before the reference code could be returned.

CREATE OR REPLACE FUNCTION
public.refresh_customer_service_booking_alert(
  p_customer_phone text,
  p_service_id uuid,
  p_service_name text,
  p_preferred_appointment_id uuid,
  p_customer_name text
)
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
  v_service_key text;
  v_service_label text;
  v_customer_label text;
  v_active_appointment_ids uuid[];
  v_active_booking_count integer;
  v_distinct_booking_date_count integer;
  v_schedule_details text;
  v_notification_appointment_id uuid;
  v_alert_cycle integer;
  v_threshold_was_active boolean;
  v_notification_id uuid;
BEGIN
  IF NULLIF(btrim(p_customer_phone), '') IS NULL
     OR (p_service_id IS NULL AND NULLIF(btrim(p_service_name), '') IS NULL)
THEN
    RETURN;
  END IF;

  v_service_key := COALESCE(
    'id:' || p_service_id::text,
    'name:' || lower(btrim(p_service_name))
  );
  v_service_label := COALESCE(
    NULLIF(btrim(p_service_name), ''),
```

```
    (SELECT service.name FROM public.services AS service WHERE service.id
= p_service_id),
    'this service'
  );

  PERFORM pg_advisory_xact_lock(
    hashtext('customer-service-booking-alert:' || btrim(p_customer_phone))
  );

  WITH active_bookings AS (
    SELECT DISTINCT
      appointment.id,
      appointment.appointment_date,
      appointment.start_time,
      appointment.customer_name
    FROM public.appointments AS appointment
    JOIN public.appointment_services AS appointment_service
      ON appointment_service.appointment_id = appointment.id
    WHERE appointment.phone = btrim(p_customer_phone)
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status NOT IN ('completed', 'cancelled', 'rejected')
      AND (
        appointment_service.service_id = p_service_id
        OR (
          p_service_id IS NULL
          AND appointment_service.service_id IS NULL
          AND appointment_service.service_name = p_service_name
        )
      )
  )
  SELECT
    COALESCE(array_agg(appointment.id ORDER BY appointment.id),
ARRAY[]::uuid[]),
    count(*)::integer,
    count(DISTINCT appointment.appointment_date)::integer,
    string_agg(
      to_char(appointment.appointment_date, 'Mon FMDD, YYYY')
        || ' at ' || to_char(appointment.start_time, 'FMHH12:MI AM'),
      '; ' ORDER BY appointment.appointment_date, appointment.start_time,
appointment.id
    ),
    COALESCE(
      max(appointment.customer_name) FILTER (WHERE appointment.id =
p_preferred_appointment_id),
```

```
      max(appointment.customer_name),
      NULLIF(btrim(p_customer_name), ''),
      btrim(p_customer_phone)
    ),
    COALESCE(
      (
        array_agg(appointment.id ORDER BY appointment.appointment_date,
appointment.start_time, appointment.id)
          FILTER (WHERE appointment.id = p_preferred_appointment_id)
      )[1],
      (
        array_agg(appointment.id ORDER BY appointment.appointment_date,
appointment.start_time, appointment.id)
      )[1]
    )
  INTO
    v_active_appointment_ids,
    v_active_booking_count,
    v_distinct_booking_date_count,
    v_schedule_details,
    v_customer_label,
    v_notification_appointment_id
  FROM active_bookings AS appointment;

  INSERT INTO public.customer_service_booking_alert_state (customer_phone,
service_key)
  VALUES (btrim(p_customer_phone), v_service_key)
  ON CONFLICT (customer_phone, service_key) DO NOTHING;

  SELECT state.is_threshold_active, state.alert_cycle
  INTO v_threshold_was_active, v_alert_cycle
  FROM public.customer_service_booking_alert_state AS state
  WHERE state.customer_phone = btrim(p_customer_phone)
    AND state.service_key = v_service_key
  FOR UPDATE;

  IF v_active_booking_count < 2 OR v_distinct_booking_date_count < 2 THEN
    UPDATE public.customer_service_booking_alert_state
    SET is_threshold_active = false,
        updated_at = now()
    WHERE customer_phone = btrim(p_customer_phone)
      AND service_key = v_service_key;
    RETURN;
  END IF;
```

```
  IF NOT v_threshold_was_active THEN
    v_alert_cycle := v_alert_cycle + 1;

    UPDATE public.customer_service_booking_alert_state
    SET is_threshold_active = true,
        alert_cycle = v_alert_cycle,
        updated_at = now()
    WHERE customer_phone = btrim(p_customer_phone)
      AND service_key = v_service_key;
  END IF;

  INSERT INTO public.customer_service_booking_alerts (
    customer_phone,
    service_key,
    alert_cycle,
    active_appointment_ids
  )
  VALUES (
    btrim(p_customer_phone),
    v_service_key,
    v_alert_cycle,
    v_active_appointment_ids
  )
  ON CONFLICT (customer_phone, service_key, alert_cycle,
active_appointment_ids) DO NOTHING
  RETURNING id INTO v_notification_id;

  IF v_notification_id IS NULL THEN
    RETURN;
  END IF;

  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES (
    'multiple_active_service_bookings',
    v_active_booking_count || ' active ' || v_service_label || ' bookings for ' ||
v_customer_label,
    v_customer_label || ' (' || btrim(p_customer_phone) || ') has '
      || v_active_booking_count || ' active bookings for ' || v_service_label || ': '
      || v_schedule_details || '.',
    v_notification_appointment_id
  )
  RETURNING id INTO v_notification_id;

  UPDATE public.customer_service_booking_alerts
  SET notification_id = v_notification_id
```

```
  WHERE customer_phone = btrim(p_customer_phone)
    AND service_key = v_service_key
    AND alert_cycle = v_alert_cycle
    AND active_appointment_ids = v_active_appointment_ids;
END;
$$;

REVOKE ALL ON FUNCTION
public.refresh_customer_service_booking_alert(text, uuid, text, uuid, text) FROM
PUBLIC;
GRANT EXECUTE ON FUNCTION
public.refresh_customer_service_booking_alert(text, uuid, text, uuid, text) TO
service_role;
```

supabase/migrations/20260928150000_fix_admin_session_validation.sql

---

```
-- A successful password login could claim a lease but then fail the protected
-- route's validation because it additionally queried auth.sessions. That
-- internal catalog is not required to prove a request owns its lease: Supabase
-- has already authenticated the JWT, and its session_id claim is compared to
-- the server-side lease. Removing that dependency prevents a valid new admin
-- login from being immediately signed out while preserving the one-session
-- restriction for all direct requests and UI routes.

DELETE FROM public.admin_session_leases
WHERE expires_at <= now();

CREATE OR REPLACE FUNCTION
public.is_current_admin_session(p_admin_id uuid)
RETURNS boolean
LANGUAGE sql
STABLE
SECURITY DEFINER
SET search_path = public, auth
AS $$
  SELECT
    p_admin_id IS NOT NULL
    AND p_admin_id = auth.uid()
    AND EXISTS (
      SELECT 1
      FROM public.user_roles AS role
      WHERE role.user_id = p_admin_id
        AND role.role::text = 'admin'
    )
    AND EXISTS (
      SELECT 1
      FROM public.admin_session_leases AS lease
      WHERE lease.admin_id = p_admin_id
        AND lease.session_id = public.current_admin_auth_session_id()
        AND lease.expires_at > now()
    );
$$;

CREATE OR REPLACE FUNCTION public.claim_admin_session(
  p_handover_token uuid DEFAULT NULL
)
RETURNS boolean
LANGUAGE plpgsql
SECURITY DEFINER
```

```
SET search_path = public, auth
AS $$
DECLARE
  v_admin_id uuid := auth.uid();
  v_session_id uuid := public.current_admin_auth_session_id();
  v_existing_lease public.admin_session_leases%ROWTYPE;
  v_has_valid_handover boolean := false;
BEGIN
  IF v_admin_id IS NULL
     OR v_session_id IS NULL
     OR NOT EXISTS (
       SELECT 1
       FROM public.user_roles AS role
       WHERE role.user_id = v_admin_id
         AND role.role::text = 'admin'
     ) THEN
    RAISE EXCEPTION 'Only administrators can start an admin session.';
  END IF;

  PERFORM pg_advisory_xact_lock(hashtext('admin-session-lease:' ||
v_admin_id::text));
  DELETE FROM public.admin_session_handover_tokens WHERE expires_at
<= now();

  SELECT lease.*
  INTO v_existing_lease
  FROM public.admin_session_leases AS lease
  WHERE lease.admin_id = v_admin_id
  FOR UPDATE;

  IF FOUND
     AND v_existing_lease.session_id <> v_session_id
     AND v_existing_lease.expires_at > now() THEN
    IF p_handover_token IS NOT NULL THEN
      SELECT true
      INTO v_has_valid_handover
      FROM public.admin_session_handover_tokens AS handover
      WHERE handover.token = p_handover_token
        AND handover.admin_id = v_admin_id
        AND handover.session_id = v_existing_lease.session_id
        AND handover.expires_at > now()
      FOR UPDATE;
    END IF;

    IF NOT COALESCE(v_has_valid_handover, false) THEN
```

```
      RAISE EXCEPTION
        'An administrator session is already active. Sign out from the active device
or wait for it to expire before signing in here.';
    END IF;

    DELETE FROM public.admin_session_handover_tokens
    WHERE token = p_handover_token;
  END IF;

  INSERT INTO public.admin_session_leases (
    admin_id,
    session_id,
    expires_at,
    last_seen_at
  )
  VALUES (
    v_admin_id,
    v_session_id,
    now() + interval '3 minutes',
    now()
  )
  ON CONFLICT (admin_id) DO UPDATE
  SET session_id = EXCLUDED.session_id,
      expires_at = EXCLUDED.expires_at,
      last_seen_at = EXCLUDED.last_seen_at;

  RETURN true;
END;
$$;

CREATE OR REPLACE FUNCTION public.renew_admin_session()
RETURNS boolean
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public, auth
AS $$
DECLARE
  v_admin_id uuid := auth.uid();
  v_session_id uuid := public.current_admin_auth_session_id();
BEGIN
  IF v_admin_id IS NULL OR v_session_id IS NULL THEN
    RETURN false;
  END IF;

  UPDATE public.admin_session_leases AS lease
```

```
  SET expires_at = now() + interval '3 minutes',
      last_seen_at = now()
  WHERE lease.admin_id = v_admin_id
    AND lease.session_id = v_session_id
    AND lease.expires_at > now();

  RETURN FOUND;
END;
$$;

NOTIFY pgrst, 'reload schema';
```

supabase/migrations/20260929120000_require_service_creation_fields.sql

---

```
-- Services may be created directly through PostgREST, so the required-field
-- rules enforced by the admin form must also be enforced by PostgreSQL.

-- Preserve older records while making them editable under the new requirement.
UPDATE public.services
SET description = 'Service details pending review.'
WHERE description IS NULL OR btrim(description) = '';

ALTER TABLE public.services
  ALTER COLUMN description SET NOT NULL,
  ALTER COLUMN category DROP DEFAULT,
  ALTER COLUMN price DROP DEFAULT,
  ALTER COLUMN duration_minutes DROP DEFAULT;

ALTER TABLE public.services
  DROP CONSTRAINT IF EXISTS services_name_required,
  DROP CONSTRAINT IF EXISTS services_category_required,
  DROP CONSTRAINT IF EXISTS services_description_required,
  DROP CONSTRAINT IF EXISTS services_price_valid,
  DROP CONSTRAINT IF EXISTS services_duration_valid,
  ADD CONSTRAINT services_name_required
    CHECK (char_length(btrim(name)) >= 2),
  ADD CONSTRAINT services_category_required
    CHECK (char_length(btrim(category)) >= 2),
  ADD CONSTRAINT services_description_required
    CHECK (char_length(btrim(description)) > 0),
  ADD CONSTRAINT services_price_valid
    CHECK (price >= 0),
  ADD CONSTRAINT services_duration_valid
    CHECK (duration_minutes BETWEEN 15 AND 480);
```

supabase/migrations/20261001100000_remove_booking_duration_buffer.sql

---

```
-- A reservation now covers only the selected service work. The old 30-minute
-- arrival/post-service buffer is no longer part of its persisted duration.
ALTER TABLE public.appointments
  ALTER COLUMN booking_duration_minutes SET DEFAULT 60;

ALTER TABLE public.shop_settings
  ALTER COLUMN default_appointment_duration_minutes SET DEFAULT 60;

-- The prior default represented a 60-minute service plus the removed buffer.
-- Leave any administrator-selected value alone.
UPDATE public.shop_settings
SET default_appointment_duration_minutes = 60
WHERE default_appointment_duration_minutes = 90;

COMMENT ON COLUMN public.appointments.booking_duration_minutes IS
  'Total duration of the selected service work, without an arrival or post-service
buffer.';

-- Preserve historical records, but release the buffer from upcoming active
-- reservations. Snapshot service durations are used so later catalog edits do
-- not alter an appointment that has already been booked. These updates only
-- shrink reservations, so bypass revalidation against any subsequently changed
-- slot configuration while the persisted values are normalized.
ALTER TABLE public.appointments DISABLE TRIGGER
appointments_enforce_active_capacity;
ALTER TABLE public.appointment_continuations
  DISABLE TRIGGER appointment_continuations_enforce_active_capacity;

WITH service_totals AS (
  SELECT appointment_id, SUM(duration_minutes)::integer AS duration_minutes
  FROM public.appointment_services
  GROUP BY appointment_id
)
UPDATE public.appointments AS appointment
SET booking_duration_minutes = service_totals.duration_minutes
FROM service_totals
WHERE appointment.id = service_totals.appointment_id
  AND appointment.appointment_date >= CURRENT_DATE
  AND appointment.is_archived = false
  AND appointment.rescheduled_to_appointment_id IS NULL
  AND appointment.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
  AND NOT EXISTS (
```

```
    SELECT 1
    FROM public.appointment_continuations AS continuation
    WHERE continuation.appointment_id = appointment.id
  );

-- Multi-day reservations already split all service time before the trailing
-- post-service buffer. Removing the final 30 minutes preserves the first-day
-- and intermediate service segments while releasing the buffer at the end.
WITH final_continuations AS (
  SELECT DISTINCT ON (continuation.appointment_id)
    continuation.id,
    continuation.booking_duration_minutes
  FROM public.appointment_continuations AS continuation
  JOIN public.appointments AS appointment ON appointment.id =
continuation.appointment_id
  WHERE appointment.appointment_date >= CURRENT_DATE
    AND appointment.is_archived = false
    AND appointment.rescheduled_to_appointment_id IS NULL
    AND appointment.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
  ORDER BY continuation.appointment_id, continuation.segment_number DESC
)
UPDATE public.appointment_continuations AS continuation
SET booking_duration_minutes = final_continuations.booking_duration_minutes -
30
FROM final_continuations
WHERE continuation.id = final_continuations.id
  AND final_continuations.booking_duration_minutes > 30;

-- The normalization updates can queue deferred appointment triggers. Flush
-- them before changing trigger state; PostgreSQL otherwise rejects ALTER
TABLE
-- with pending trigger events in this migration transaction.
SET CONSTRAINTS ALL IMMEDIATE;

ALTER TABLE public.appointment_continuations
  ENABLE TRIGGER appointment_continuations_enforce_active_capacity;
ALTER TABLE public.appointments ENABLE TRIGGER
appointments_enforce_active_capacity;

-- The database must perform the same final staffing checks as the application
-- so concurrent writes cannot assign one mechanic to two overlapping jobs.
-- Keep the database callers on this one predicate; the TypeScript equivalent
-- is evaluateCrewAvailability in src/lib/availability.ts.
CREATE OR REPLACE FUNCTION public.is_crew_available_for_appointment(
  p_crew_id uuid,
```

```
  p_appointment_date date,
  p_start_time time,
  p_duration_minutes integer,
  p_excluded_appointment_id uuid DEFAULT NULL,
  p_excluded_continuation_id uuid DEFAULT NULL
)
RETURNS boolean
LANGUAGE sql
SECURITY INVOKER
SET search_path = public
AS $$
  SELECT p_duration_minutes > 0
    AND EXISTS (
      SELECT 1
      FROM public.crew_members AS crew
      JOIN public.crew_schedules AS schedule ON schedule.crew_id = crew.id
      WHERE crew.id = p_crew_id
        AND crew.is_active = true
        AND crew.is_archived = false
        AND schedule.schedule_date = p_appointment_date
        AND schedule.is_working = true
        AND schedule.start_time IS NOT NULL
        AND schedule.end_time IS NOT NULL
        AND schedule.start_time <= p_start_time
        AND schedule.end_time >= p_start_time + make_interval(mins =>
p_duration_minutes)
    )
    AND NOT EXISTS (
      SELECT 1
      FROM public.crew_availability_exceptions AS exception
      WHERE exception.crew_id = p_crew_id
        AND exception.start_date <= p_appointment_date
        AND exception.end_date >= p_appointment_date
        AND (
          exception.is_all_day = true
          OR (
            exception.start_time IS NOT NULL
            AND exception.end_time IS NOT NULL
            AND tsrange(
              p_appointment_date + p_start_time,
              p_appointment_date + p_start_time + make_interval(mins =>
p_duration_minutes),
              '[)'
            ) && tsrange(
              p_appointment_date + exception.start_time,
```

```
            p_appointment_date + exception.end_time,
            '[)'
          )
        )
      )
    )
    AND NOT EXISTS (
      SELECT 1
      FROM public.appointments AS appointment
      WHERE (p_excluded_appointment_id IS NULL OR appointment.id <>
p_excluded_appointment_id)
        AND appointment.assigned_crew_id = p_crew_id
        AND appointment.appointment_date = p_appointment_date
        AND appointment.is_archived = false
        AND appointment.rescheduled_to_appointment_id IS NULL
        AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
        AND tsrange(
          appointment.appointment_date + appointment.start_time,
          appointment.appointment_date + appointment.start_time
            + make_interval(mins => appointment.booking_duration_minutes),
          '[)'
        ) && tsrange(
          p_appointment_date + p_start_time,
          p_appointment_date + p_start_time + make_interval(mins =>
p_duration_minutes),
          '[)'
        )
    )
    AND NOT EXISTS (
      SELECT 1
      FROM public.appointment_continuations AS continuation
      JOIN public.appointments AS appointment ON appointment.id =
continuation.appointment_id
      WHERE (p_excluded_continuation_id IS NULL OR continuation.id <>
p_excluded_continuation_id)
        AND continuation.assigned_crew_id = p_crew_id
        AND continuation.appointment_date = p_appointment_date
        AND appointment.is_archived = false
        AND appointment.rescheduled_to_appointment_id IS NULL
        AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
        AND tsrange(
          continuation.appointment_date + continuation.start_time,
          continuation.appointment_date + continuation.start_time
```

```
          + make_interval(mins => continuation.booking_duration_minutes),
        '[)'
      ) && tsrange(
        p_appointment_date + p_start_time,
        p_appointment_date + p_start_time + make_interval(mins =>
p_duration_minutes),
        '[)'
      )
  );
$$;

-- Continuations use the same final crew predicate as primary appointments.
-- Their capacity and shop-block checks remain specific to continuation rows.
CREATE OR REPLACE FUNCTION
public.enforce_active_appointment_continuation_capacity()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = public
AS $$
DECLARE
  v_parent public.appointments%ROWTYPE;
  v_opening_time time;
  v_closing_time time;
  v_opening_minutes integer;
  v_closing_minutes integer;
  v_start_minutes integer;
  v_hour_start time;
  v_capacity integer;
  v_overlapping_reservations integer;
BEGIN
  SELECT * INTO v_parent FROM public.appointments WHERE id =
NEW.appointment_id;
  IF NOT FOUND THEN
    RAISE EXCEPTION 'The parent appointment no longer exists.' USING
ERRCODE = '23503';
  END IF;

  IF v_parent.is_archived = true
     OR v_parent.rescheduled_to_appointment_id IS NOT NULL
     OR v_parent.status NOT IN ('pending', 'confirmed', 'in_progress',
'rescheduled') THEN
    RETURN NEW;
  END IF;

  SELECT opening_time, closing_time INTO v_opening_time, v_closing_time
```

```
  FROM public.shop_settings WHERE id = true;
  v_opening_time := COALESCE(v_opening_time, '08:00:00'::time);
  v_closing_time := COALESCE(v_closing_time, '17:00:00'::time);
  v_opening_minutes := extract(hour FROM v_opening_time)::integer * 60
    + extract(minute FROM v_opening_time)::integer;
  v_closing_minutes := extract(hour FROM v_closing_time)::integer * 60
    + extract(minute FROM v_closing_time)::integer;
  v_start_minutes := extract(hour FROM NEW.start_time)::integer * 60
    + extract(minute FROM NEW.start_time)::integer;

  IF extract(dow FROM NEW.appointment_date) = 0
     OR NEW.booking_duration_minutes <= 0
     OR v_start_minutes < v_opening_minutes
     OR v_start_minutes >= v_closing_minutes
     OR mod(v_start_minutes - v_opening_minutes, 30) <> 0
     OR v_start_minutes + NEW.booking_duration_minutes > v_closing_minutes
THEN
    RAISE EXCEPTION 'The continuation must fit within the configured shop
hours in 30-minute start intervals.'
      USING ERRCODE = '23P01';
  END IF;

  IF EXISTS (
    SELECT 1
    FROM public.schedule_blocks AS block
    WHERE block.block_date = NEW.appointment_date
      AND block.is_active = true
      AND (
        block.start_time IS NULL
        OR (
          COALESCE(block.reason, '') NOT LIKE 'RANGE:%'
          AND block.start_time = NEW.start_time
        )
        OR (
          COALESCE(block.reason, '') LIKE 'RANGE:%'
          AND tsrange(
            NEW.appointment_date + NEW.start_time,
            NEW.appointment_date + NEW.start_time
              + make_interval(mins => NEW.booking_duration_minutes),
            '[)'
          ) && tsrange(
            NEW.appointment_date + block.start_time,
            NEW.appointment_date
              + substring(block.reason FROM '^RANGE:([0-9]{2}:[0-9]{2})')::time,
            '[)'
```

```
        )
      )
    )
  ) THEN
    RAISE EXCEPTION 'The shop is closed for the continuation schedule.' USING
ERRCODE = '23P01';
  END IF;

  PERFORM pg_advisory_xact_lock(hashtext('booking-capacity:' ||
NEW.appointment_date::text));
  v_hour_start := make_time(extract(hour FROM NEW.start_time)::integer, 0, 0);

  SELECT slot.capacity INTO v_capacity
  FROM public.time_slots AS slot
  WHERE slot.is_active = true
    AND (slot.start_time = NEW.start_time OR slot.start_time = v_hour_start)
  ORDER BY (slot.start_time = NEW.start_time) DESC
  LIMIT 1
  FOR UPDATE;

  IF COALESCE(v_capacity, 0) <= 0 THEN
    RAISE EXCEPTION 'The requested continuation time slot is not available.'
USING ERRCODE = '23P01';
  END IF;

  SELECT count(*) INTO v_overlapping_reservations
  FROM (
    SELECT appointment.appointment_date, appointment.start_time,
appointment.booking_duration_minutes
    FROM public.appointments AS appointment
    WHERE appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
      AND appointment.appointment_date = NEW.appointment_date
    UNION ALL
    SELECT continuation.appointment_date, continuation.start_time,
continuation.booking_duration_minutes
    FROM public.appointment_continuations AS continuation
    JOIN public.appointments AS appointment ON appointment.id =
continuation.appointment_id
    WHERE continuation.id <> NEW.id
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
```

```
'rescheduled')
      AND continuation.appointment_date = NEW.appointment_date
  ) AS reservation
  WHERE tsrange(
    reservation.appointment_date + reservation.start_time,
    reservation.appointment_date + reservation.start_time
      + make_interval(mins => reservation.booking_duration_minutes),
    '[)'
  ) && tsrange(
    NEW.appointment_date + NEW.start_time,
    NEW.appointment_date + NEW.start_time + make_interval(mins =>
NEW.booking_duration_minutes),
    '[)'
  );

  IF v_overlapping_reservations >= v_capacity THEN
    RAISE EXCEPTION 'The requested continuation schedule is no longer
available.' USING ERRCODE = '23P01';
  END IF;

  IF NOT public.is_crew_available_for_appointment(
    NEW.assigned_crew_id,
    NEW.appointment_date,
    NEW.start_time,
    NEW.booking_duration_minutes,
    NULL,
    NEW.id
  ) THEN
    RAISE EXCEPTION 'The assigned mechanic is no longer available for the
continuation schedule.'
      USING ERRCODE = '23P01';
  END IF;

  RETURN NEW;
END;
$$;

-- Keep administrative edits in sync with customer booking: the saved duration
-- and every crew/capacity validation now use only selected service durations.
CREATE OR REPLACE FUNCTION public.update_appointment_details_atomic(
  p_appointment_id uuid,
  p_service_ids uuid[],
  p_appointment_date date,
  p_start_time time,
  p_assigned_crew_id uuid,
```

```
  p_crew_assignment_manual boolean,
  p_status text,
  p_admin_notes text
)
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_service_count integer;
  v_total_estimate numeric(10,2);
  v_duration_minutes integer;
  v_assigned_crew_id uuid := p_assigned_crew_id;
BEGIN
  IF COALESCE(cardinality(p_service_ids), 0) = 0 THEN
    RAISE EXCEPTION 'Select at least one service.';
  END IF;

  IF p_status NOT IN (
    'pending', 'confirmed', 'in_progress', 'completed', 'rescheduled', 'cancelled',
'rejected', 'no_show'
  ) THEN
    RAISE EXCEPTION 'Choose a valid appointment status.';
  END IF;

  SELECT count(*)
    INTO v_service_count
  FROM public.services
  WHERE id = ANY(p_service_ids)
    AND is_active = true
    AND is_archived = false;

  IF v_service_count <> cardinality(p_service_ids) THEN
    RAISE EXCEPTION 'One or more selected services are no longer available.';
  END IF;

  SELECT
    COALESCE(sum(price), 0),
    COALESCE(sum(duration_minutes), 0)
  INTO v_total_estimate, v_duration_minutes
  FROM public.services
  WHERE id = ANY(p_service_ids);

  -- Serialize selection with other schedule writes for this date. The trigger
```

```
  -- below repeats the lock as the final guard for every write path.
  PERFORM pg_advisory_xact_lock(hashtext('booking-capacity:' ||
p_appointment_date::text));

  -- When confirming without a manual crew choice, select the first mechanic
  -- accepted by the common final-availability predicate.
  IF p_status = 'confirmed' AND v_assigned_crew_id IS NULL AND NOT
p_crew_assignment_manual THEN
    SELECT crew.id
      INTO v_assigned_crew_id
    FROM public.crew_members AS crew
    WHERE crew.is_active = true
      AND crew.is_archived = false
      AND public.is_crew_available_for_appointment(
        crew.id,
        p_appointment_date,
        p_start_time,
        v_duration_minutes,
        p_appointment_id
      )
    ORDER BY crew.name, crew.id
    LIMIT 1;

    IF v_assigned_crew_id IS NULL THEN
      RAISE EXCEPTION 'No crew member is available for the selected
appointment period.';
    END IF;
  END IF;

  -- Manual crew assignments use that same predicate.
  IF v_assigned_crew_id IS NOT NULL
     AND NOT public.is_crew_available_for_appointment(
       v_assigned_crew_id,
       p_appointment_date,
       p_start_time,
       v_duration_minutes,
       p_appointment_id
     ) THEN
    RAISE EXCEPTION 'Choose a crew member who is free for the full
appointment period.';
  END IF;

  UPDATE public.appointments
  SET appointment_date = p_appointment_date,
      start_time = p_start_time,
```

```
    assigned_crew_id = v_assigned_crew_id,
    status = p_status,
    admin_notes = NULLIF(btrim(p_admin_notes), ''),
    total_estimate = v_total_estimate,
    booking_duration_minutes = v_duration_minutes
  WHERE id = p_appointment_id
   AND is_archived = false;

  IF NOT FOUND THEN
   RAISE EXCEPTION 'This appointment is no longer available.';
  END IF;

  DELETE FROM public.appointment_services
  WHERE appointment_id = p_appointment_id;

  INSERT INTO public.appointment_services (
   appointment_id,
   service_id,
   service_name,
   price,
   duration_minutes
  )
  SELECT
   p_appointment_id,
   service.id,
   service.name,
   service.price,
   service.duration_minutes
  FROM public.services AS service
  WHERE service.id = ANY(p_service_ids);
END;
$$;

-- The browser and server make an optimistic availability decision. This
-- trigger is the transaction-safe final guard: it serializes schedule writes
-- for a date and prevents one crew member from being assigned to overlapping
-- appointment or continuation work, even when slot capacity is greater than
-- one or two requests arrive at the same time.
CREATE OR REPLACE FUNCTION
public.enforce_active_appointment_crew_conflict()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = public
AS $$
BEGIN
```

```
  IF NEW.is_archived = true
     OR NEW.rescheduled_to_appointment_id IS NOT NULL
     OR NEW.assigned_crew_id IS NULL
     OR NEW.status NOT IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
THEN
    RETURN NEW;
  END IF;

  PERFORM pg_advisory_xact_lock(hashtext('booking-capacity:' ||
NEW.appointment_date::text));

  IF NOT public.is_crew_available_for_appointment(
    NEW.assigned_crew_id,
    NEW.appointment_date,
    NEW.start_time,
    NEW.booking_duration_minutes,
    NEW.id
  ) THEN
    RAISE EXCEPTION 'The assigned mechanic is no longer available for the
requested schedule.'
      USING ERRCODE = '23P01';
  END IF;

  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS appointments_enforce_active_crew_conflict ON
public.appointments;
CREATE TRIGGER appointments_enforce_active_crew_conflict
  BEFORE INSERT OR UPDATE OF
    appointment_date,
    start_time,
    booking_duration_minutes,
    assigned_crew_id,
    status,
    is_archived,
    rescheduled_to_appointment_id
  ON public.appointments
  FOR EACH ROW
  EXECUTE FUNCTION public.enforce_active_appointment_crew_conflict();

NOTIFY pgrst, 'reload schema';
```

supabase/migrations/20261004090000_add_booking_confirmation_emails.sql

---

```
-- Customer bookings now require an email address. Existing records remain
-- nullable because they were created before email collection was restored.
ALTER TABLE public.appointments
  ADD COLUMN IF NOT EXISTS email text;

ALTER TABLE public.appointments
  DROP CONSTRAINT IF EXISTS appointments_email_format_check;

ALTER TABLE public.appointments
  ADD CONSTRAINT appointments_email_format_check
  CHECK (
    email IS NULL
    OR btrim(email) ~* '^[^[:space:]@]+@[^[:space:]@]+\.[^[:space:]@]+$'
  );

-- Keep an approval-created email durable and idempotent. The application
-- claims and sends these rows using its server-only mail credentials.
CREATE TABLE IF NOT EXISTS public.appointment_confirmation_emails (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  appointment_id uuid NOT NULL UNIQUE REFERENCES
public.appointments(id) ON DELETE CASCADE,
  recipient_email text NOT NULL,
  status text NOT NULL DEFAULT 'pending'
    CHECK (status IN ('pending', 'sending', 'sent')),
  attempt_count integer NOT NULL DEFAULT 0 CHECK (attempt_count >= 0),
  last_error text,
  sent_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);

REVOKE ALL ON TABLE public.appointment_confirmation_emails FROM anon,
authenticated;
GRANT ALL ON TABLE public.appointment_confirmation_emails TO
service_role;
ALTER TABLE public.appointment_confirmation_emails ENABLE ROW LEVEL
SECURITY;

CREATE OR REPLACE FUNCTION
public.queue_appointment_confirmation_email()
RETURNS trigger
LANGUAGE plpgsql
```

```
SECURITY DEFINER
SET search_path = public
AS $$
BEGIN
  -- Pending booking requests do not send customer email. A row is queued only
  -- when an appointment is actually approved into the confirmed state.
  IF NEW.status <> 'confirmed' OR NULLIF(btrim(NEW.email), '') IS NULL THEN
    RETURN NEW;
  END IF;
  IF TG_OP = 'UPDATE' AND OLD.status IS NOT DISTINCT FROM 'confirmed'
THEN
    RETURN NEW;
  END IF;

  INSERT INTO public.appointment_confirmation_emails (appointment_id,
recipient_email)
  VALUES (NEW.id, lower(btrim(NEW.email)))
  ON CONFLICT (appointment_id) DO NOTHING;
  RETURN NEW;
END;
$$;

CREATE OR REPLACE FUNCTION
public.inherit_rescheduled_appointment_email()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
BEGIN
  IF NEW.rescheduled_from_appointment_id IS NOT NULL
     AND NULLIF(btrim(NEW.email), '') IS NULL THEN
    SELECT email
      INTO NEW.email
    FROM public.appointments
    WHERE id = NEW.rescheduled_from_appointment_id;
  END IF;
  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS appointments_inherit_rescheduled_email ON
public.appointments;
CREATE TRIGGER appointments_inherit_rescheduled_email
  BEFORE INSERT ON public.appointments
```

```
  FOR EACH ROW
  EXECUTE FUNCTION public.inherit_rescheduled_appointment_email();

DROP TRIGGER IF EXISTS appointments_queue_confirmation_email ON
public.appointments;
DROP TRIGGER IF EXISTS appointments_queue_confirmation_email_on_insert
ON public.appointments;
CREATE TRIGGER appointments_queue_confirmation_email
  AFTER UPDATE OF status ON public.appointments
  FOR EACH ROW
  EXECUTE FUNCTION public.queue_appointment_confirmation_email();

CREATE TRIGGER appointments_queue_confirmation_email_on_insert
  AFTER INSERT ON public.appointments
  FOR EACH ROW
  EXECUTE FUNCTION public.queue_appointment_confirmation_email();

-- This overload is used by the current booking server function. The previous
-- signature stays in place for any browser session still running an older
-- deployment while the new form rolls out.
CREATE FUNCTION public.create_booking_atomic(
  p_reference_code text, p_booking_request_id uuid, p_customer_name text,
  p_phone text, p_email text, p_moto_brand text, p_moto_model text,
  p_moto_variant text, p_moto_year integer, p_plate_number text,
  p_appointment_date date, p_start_time time, p_notes text,
  p_total_estimate numeric, p_booking_duration_minutes integer,
  p_assigned_crew_id uuid, p_services jsonb, p_continuation_segments jsonb,
  p_notification_title text, p_notification_message text, p_first_name text,
  p_middle_name text, p_last_name text
)
RETURNS TABLE (appointment_id uuid, reference_code text)
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment_id uuid;
  v_reference_code text;
BEGIN
  IF p_booking_request_id IS NULL THEN
    RAISE EXCEPTION 'A booking request ID is required.';
  END IF;
  IF COALESCE(btrim(p_first_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$'
     OR COALESCE(btrim(p_last_name), '') !~ '^[[:alpha:]]+( [[:alpha:]]+)*$' THEN
    RAISE EXCEPTION 'First and last names must contain letters only.';
```

```
  END IF;
  IF COALESCE(btrim(p_middle_name), '') <> ''
     AND btrim(p_middle_name) !~ '^[[:alpha:]]+( [[:alpha:]]+)*$' THEN
    RAISE EXCEPTION 'Middle name must contain letters only.';
  END IF;
  IF COALESCE(btrim(p_email), '') !~*
'^[^[:space:]@]+@[^[:space:]@]+\.[^[:space:]@]+$' THEN
    RAISE EXCEPTION 'Enter a valid email address.';
  END IF;
  IF jsonb_typeof(p_services) <> 'array' OR jsonb_array_length(p_services) = 0
THEN
    RAISE EXCEPTION 'A booking must include at least one service.';
  END IF;
  IF jsonb_typeof(COALESCE(p_continuation_segments, '[]'::jsonb)) <> 'array'
THEN
    RAISE EXCEPTION 'Continuation segments must be an array.';
  END IF;

  INSERT INTO public.appointments (
    reference_code, booking_request_id, customer_name, first_name,
middle_name,
    last_name, phone, email, moto_brand, moto_model, moto_variant, moto_year,
    plate_number, appointment_date, start_time, status, notes, total_estimate,
    booking_duration_minutes, terms_accepted, assigned_crew_id
  ) VALUES (
    p_reference_code,
    p_booking_request_id,
    concat_ws(' ', btrim(p_first_name), NULLIF(btrim(p_middle_name), ''),
btrim(p_last_name)),
    btrim(p_first_name),
    NULLIF(btrim(p_middle_name), ''),
    btrim(p_last_name),
    p_phone,
    lower(btrim(p_email)),
    p_moto_brand,
    p_moto_model,
    p_moto_variant,
    p_moto_year,
    p_plate_number,
    p_appointment_date,
    p_start_time,
    'pending',
    p_notes,
    p_total_estimate,
    p_booking_duration_minutes,
```

```
    true,
    p_assigned_crew_id
  ) ON CONFLICT DO NOTHING
  RETURNING appointments.id, appointments.reference_code INTO
v_appointment_id, v_reference_code;

  IF v_appointment_id IS NULL THEN
    SELECT appointments.id, appointments.reference_code
      INTO v_appointment_id, v_reference_code
    FROM public.appointments
    WHERE appointments.booking_request_id = p_booking_request_id;
    IF FOUND THEN
      RETURN QUERY SELECT v_appointment_id, v_reference_code;
      RETURN;
    END IF;
    RAISE EXCEPTION 'The requested schedule is no longer available.' USING
ERRCODE = '23P01';
  END IF;

  INSERT INTO public.appointment_services (
    appointment_id, service_id, service_name, price, duration_minutes
  )
  SELECT v_appointment_id, service.service_id, service.service_name,
service.price,
    service.duration_minutes
  FROM jsonb_to_recordset(p_services) AS service(
    service_id uuid, service_name text, price numeric, duration_minutes integer
  );

  INSERT INTO public.appointment_continuations (
    appointment_id, segment_number, appointment_date, start_time,
    booking_duration_minutes, assigned_crew_id
  )
  SELECT v_appointment_id, segment.segment_number,
segment.appointment_date,
    segment.start_time, segment.booking_duration_minutes,
segment.assigned_crew_id
  FROM jsonb_to_recordset(COALESCE(p_continuation_segments, '[]'::jsonb))
AS segment(
    segment_number integer, appointment_date date, start_time time,
    booking_duration_minutes integer, assigned_crew_id uuid
  );

  INSERT INTO public.notifications (type, title, message, appointment_id)
  VALUES ('new_appointment', p_notification_title, p_notification_message,
```

```
v_appointment_id);

  RETURN QUERY SELECT v_appointment_id, v_reference_code;
END;
$$;

REVOKE ALL ON FUNCTION public.create_booking_atomic(
  text, uuid, text, text, text, text, text, text, integer, text, date, time,
  text, numeric, integer, uuid, jsonb, jsonb, text, text, text, text, text
) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.create_booking_atomic(
  text, uuid, text, text, text, text, text, text, integer, text, date, time,
  text, numeric, integer, uuid, jsonb, jsonb, text, text, text, text, text
) TO service_role;

NOTIFY pgrst, 'reload schema';
```

supabase/migrations/20261004100000_secure_email_reference_recovery.sql

---

```
-- Reference recovery is email-only. Verification codes are stored only as
-- hashes and are consumed atomically before reference codes are emailed.
-- Normalize pre-existing addresses so exact email lookups work consistently.
UPDATE public.appointments
SET email = lower(btrim(email))
WHERE email IS NOT NULL
  AND email IS DISTINCT FROM lower(btrim(email));

CREATE INDEX IF NOT EXISTS appointments_active_email_created_idx
  ON public.appointments (email, created_at DESC)
  WHERE is_archived = false AND email IS NOT NULL;

CREATE TABLE IF NOT EXISTS
public.appointment_reference_recovery_challenges (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  email text NOT NULL,
  code_hash text NOT NULL CHECK (code_hash ~ '^[a-f0-9]{64}$'),
  expires_at timestamptz NOT NULL,
  attempt_count integer NOT NULL DEFAULT 0 CHECK (attempt_count >= 0
AND attempt_count <= 5),
  verified_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE INDEX IF NOT EXISTS
appointment_reference_recovery_challenges_email_created_idx
  ON public.appointment_reference_recovery_challenges (email, created_at
DESC);

REVOKE ALL ON TABLE public.appointment_reference_recovery_challenges
FROM anon, authenticated;
GRANT ALL ON TABLE public.appointment_reference_recovery_challenges TO
service_role;
ALTER TABLE public.appointment_reference_recovery_challenges ENABLE
ROW LEVEL SECURITY;

CREATE OR REPLACE FUNCTION
public.consume_reference_recovery_challenge(
  p_email text,
  p_code_hash text
)
RETURNS text
```

```
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_challenge public.appointment_reference_recovery_challenges%ROWTYPE;
BEGIN
  SELECT challenge.*
    INTO v_challenge
  FROM public.appointment_reference_recovery_challenges AS challenge
  WHERE challenge.email = lower(btrim(p_email))
    AND challenge.verified_at IS NULL
  ORDER BY challenge.created_at DESC
  LIMIT 1
  FOR UPDATE;

  IF NOT FOUND OR v_challenge.expires_at <= now() OR
v_challenge.attempt_count >= 5 THEN
    RETURN 'expired';
  END IF;

  IF v_challenge.code_hash <> p_code_hash THEN
    UPDATE public.appointment_reference_recovery_challenges
    SET attempt_count = attempt_count + 1,
        expires_at = CASE WHEN attempt_count + 1 >= 5 THEN now() ELSE
expires_at END
    WHERE id = v_challenge.id;
    RETURN 'invalid';
  END IF;

  UPDATE public.appointment_reference_recovery_challenges
  SET verified_at = now()
  WHERE id = v_challenge.id;

  RETURN 'verified';
END;
$$;

REVOKE ALL ON FUNCTION
public.consume_reference_recovery_challenge(text, text) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION
public.consume_reference_recovery_challenge(text, text) TO service_role;

NOTIFY pgrst, 'reload schema';
```

supabase/migrations/20261006090000_add_motorcycle_cc_categories.sql

---

```
-- Keep a model's engine displacement in the catalog and derive its category
-- in the database so clients cannot save an inconsistent classification.
ALTER TABLE public.motorcycle_catalog
  ADD COLUMN IF NOT EXISTS engine_cc numeric(6, 1);

ALTER TABLE public.motorcycle_catalog
  DROP CONSTRAINT IF EXISTS motorcycle_catalog_engine_cc_positive;

ALTER TABLE public.motorcycle_catalog
  ADD CONSTRAINT motorcycle_catalog_engine_cc_positive
  CHECK (engine_cc IS NULL OR engine_cc > 0);

ALTER TABLE public.motorcycle_catalog
  ADD COLUMN IF NOT EXISTS cc_category text GENERATED ALWAYS AS (
    CASE
      WHEN engine_cc IS NULL THEN NULL
      WHEN engine_cc <= 125 THEN 'small_bike'
      ELSE 'big_bike'
    END
  ) STORED;

COMMENT ON COLUMN public.motorcycle_catalog.cc_category IS
  'Derived from engine_cc: small_bike for 125cc and below; big_bike above
125cc.';

NOTIFY pgrst, 'reload schema';
```

supabase/migrations/20261006100000_replace_service_model_overrides_with_large_bike_pricing.sql

---

```
-- Services.price and services.duration_minutes are the Small Bike (<=125cc)
-- defaults. A service can optionally define one Large Bike (>125cc) price and
-- duration; model-specific exceptions are intentionally retired.
ALTER TABLE public.services
  ADD COLUMN IF NOT EXISTS large_bike_price numeric(10, 2),
  ADD COLUMN IF NOT EXISTS large_bike_duration_minutes integer;

ALTER TABLE public.services
  DROP CONSTRAINT IF EXISTS services_large_bike_configuration_complete;

ALTER TABLE public.services
  ADD CONSTRAINT services_large_bike_configuration_complete
  CHECK (
    (large_bike_price IS NULL AND large_bike_duration_minutes IS NULL)
    OR (
      large_bike_price > price
      AND large_bike_duration_minutes BETWEEN 15 AND 480
      AND large_bike_duration_minutes >= duration_minutes
    )
  );

COMMENT ON COLUMN public.services.price IS
  'Small Bike service price for motorcycles at 125cc and below.';
COMMENT ON COLUMN public.services.duration_minutes IS
  'Small Bike service duration for motorcycles at 125cc and below.';
COMMENT ON COLUMN public.services.large_bike_price IS
  'Large Bike service price for motorcycles above 125cc; must exceed the Small
Bike price.';
COMMENT ON COLUMN public.services.large_bike_duration_minutes IS
  'Large Bike service duration for motorcycles above 125cc.';

DROP TABLE IF EXISTS public.service_model_overrides;

-- The authenticated admin RPC has an overload that supplies the customer
-- name and phone, but it delegates the schedule and price calculation to this
-- eight-argument implementation. Keep that shared calculation category-aware
-- so customer booking and admin editing always snapshot the same values.
CREATE OR REPLACE FUNCTION public.update_appointment_details_atomic(
  p_appointment_id uuid,
  p_service_ids uuid[],
  p_appointment_date date,
  p_start_time time,
```

```
  p_assigned_crew_id uuid,
  p_crew_assignment_manual boolean,
  p_status text,
  p_admin_notes text
)
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_service_count integer;
  v_total_estimate numeric(10,2);
  v_duration_minutes integer;
  v_assigned_crew_id uuid := p_assigned_crew_id;
  v_appointment_id uuid;
  v_motorcycle_category text;
BEGIN
  IF COALESCE(cardinality(p_service_ids), 0) = 0 THEN
    RAISE EXCEPTION 'Select at least one service.';
  END IF;

  IF p_status NOT IN (
    'pending', 'confirmed', 'in_progress', 'completed', 'rescheduled', 'cancelled',
'rejected', 'no_show'
  ) THEN
    RAISE EXCEPTION 'Choose a valid appointment status.';
  END IF;

  SELECT appointment.id, catalog.cc_category
    INTO v_appointment_id, v_motorcycle_category
  FROM public.appointments AS appointment
  LEFT JOIN public.motorcycle_catalog AS catalog
    ON lower(btrim(catalog.brand)) = lower(btrim(appointment.moto_brand))
   AND lower(btrim(catalog.model)) = lower(btrim(appointment.moto_model))
   AND catalog.is_archived = false
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
  LIMIT 1;

  IF v_appointment_id IS NULL THEN
    RAISE EXCEPTION 'This appointment is no longer available.';
  END IF;
  IF v_motorcycle_category IS NULL
     OR v_motorcycle_category NOT IN ('small_bike', 'big_bike') THEN
```

```
    RAISE EXCEPTION 'The appointment motorcycle needs an engine CC
classification before its services can be updated.';
  END IF;

  SELECT count(*)
    INTO v_service_count
  FROM public.services
  WHERE id = ANY(p_service_ids)
    AND is_active = true
    AND is_archived = false;

  IF v_service_count <> cardinality(p_service_ids) THEN
    RAISE EXCEPTION 'One or more selected services are no longer available.';
  END IF;

  IF v_motorcycle_category = 'big_bike' AND EXISTS (
    SELECT 1
    FROM public.services
    WHERE id = ANY(p_service_ids)
      AND (large_bike_price IS NULL OR large_bike_duration_minutes IS NULL)
  ) THEN
    RAISE EXCEPTION 'Every selected service needs a Large Bike price and
duration.';
  END IF;

  SELECT
    COALESCE(sum(CASE WHEN v_motorcycle_category = 'big_bike' THEN
large_bike_price ELSE price END), 0),
    COALESCE(sum(CASE WHEN v_motorcycle_category = 'big_bike' THEN
large_bike_duration_minutes ELSE duration_minutes END), 0)
  INTO v_total_estimate, v_duration_minutes
  FROM public.services
  WHERE id = ANY(p_service_ids);

  PERFORM pg_advisory_xact_lock(hashtext('booking-capacity:' ||
p_appointment_date::text));

  IF p_status = 'confirmed' AND v_assigned_crew_id IS NULL AND NOT
p_crew_assignment_manual THEN
    SELECT crew.id
      INTO v_assigned_crew_id
    FROM public.crew_members AS crew
    WHERE crew.is_active = true
      AND crew.is_archived = false
      AND public.is_crew_available_for_appointment(
```

```
       crew.id, p_appointment_date, p_start_time, v_duration_minutes,
p_appointment_id
     )
   ORDER BY crew.name, crew.id
   LIMIT 1;

   IF v_assigned_crew_id IS NULL THEN
     RAISE EXCEPTION 'No crew member is available for the selected
appointment period.';
   END IF;
  END IF;

  IF v_assigned_crew_id IS NOT NULL
    AND NOT public.is_crew_available_for_appointment(
      v_assigned_crew_id, p_appointment_date, p_start_time,
v_duration_minutes, p_appointment_id
    ) THEN
   RAISE EXCEPTION 'Choose a crew member who is free for the full
appointment period.';
  END IF;

  UPDATE public.appointments
  SET appointment_date = p_appointment_date,
     start_time = p_start_time,
     assigned_crew_id = v_assigned_crew_id,
     status = p_status,
     admin_notes = NULLIF(btrim(p_admin_notes), ''),
     total_estimate = v_total_estimate,
     booking_duration_minutes = v_duration_minutes
  WHERE id = p_appointment_id
   AND is_archived = false;

  DELETE FROM public.appointment_services
  WHERE appointment_id = p_appointment_id;

  INSERT INTO public.appointment_services (
   appointment_id, service_id, service_name, price, duration_minutes
  )
  SELECT
   p_appointment_id,
   service.id,
   service.name,
   CASE WHEN v_motorcycle_category = 'big_bike' THEN
service.large_bike_price ELSE service.price END,
   CASE WHEN v_motorcycle_category = 'big_bike' THEN
```

```
service.large_bike_duration_minutes ELSE service.duration_minutes END
  FROM public.services AS service
  WHERE service.id = ANY(p_service_ids);
END;
$$;

REVOKE ALL ON FUNCTION public.update_appointment_details_atomic(
  uuid, uuid[], date, time, uuid, boolean, text, text
) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.update_appointment_details_atomic(
  uuid, uuid[], date, time, uuid, boolean, text, text
) TO authenticated;

NOTIFY pgrst, 'reload schema';
```

supabase/migrations/20261007100000_seed_complete_motorcycle_catalog.sql

---

```
-- Import the PDF supplied for the Motorcycle Catalog task (1995-2026).
-- CC is kept in its own column; cc_category is generated by the preceding
-- migration, so its Small Bike / Large Bike boundary cannot drift from CC.
-- Four source models have distinct CC versions. Their displayed model label
-- includes CC so the existing brand+model booking identity remains
unambiguous.
-- The sole electric entry has no combustion CC. It is retained for catalog
-- completeness, but inactive because it has no requested CC price category.

CREATE TEMP TABLE _motorcycle_catalog_source (
  brand text NOT NULL,
  model text NOT NULL,
  legacy_model text NOT NULL,
  engine_cc numeric(6, 1),
  legacy_preference integer NOT NULL,
  is_active boolean NOT NULL
) ON COMMIT DROP;

INSERT INTO _motorcycle_catalog_source (
  brand, model, legacy_model, engine_cc, legacy_preference, is_active
)
VALUES
  ('Honda', 'ADV 150', 'ADV 150', 149, 1, true),
  ('Honda', 'ADV 160', 'ADV 160', 157, 1, true),
  ('Honda', 'Africa Twin (1084cc)', 'Africa Twin', 1084, 0, true),
  ('Honda', 'Africa Twin (998cc)', 'Africa Twin', 998, 1, true),
  ('Honda', 'AirBlade 125', 'AirBlade 125', 125, 1, true),
  ('Honda', 'AirBlade 150', 'AirBlade 150', 149, 1, true),
  ('Honda', 'AirBlade 160', 'AirBlade 160', 157, 1, true),
  ('Honda', 'Beat (110cc)', 'Beat', 110, 0, true),
  ('Honda', 'Beat (125cc)', 'Beat', 125, 1, true),
  ('Honda', 'CB1000R', 'CB1000R', 998, 1, true),
  ('Honda', 'CB150R', 'CB150R', 149, 1, true),
  ('Honda', 'CB150X', 'CB150X', 149, 1, true),
  ('Honda', 'CB500F', 'CB500F', 471, 1, true),
  ('Honda', 'CB500X', 'CB500X', 471, 1, true),
  ('Honda', 'CB650R', 'CB650R', 649, 1, true),
  ('Honda', 'CBR1000RR', 'CBR1000RR', 999, 1, true),
  ('Honda', 'CBR1000RR-R Fireblade', 'CBR1000RR-R Fireblade', 999, 1, true),
  ('Honda', 'CBR150R', 'CBR150R', 149, 1, true),
  ('Honda', 'CBR500R', 'CBR500R', 471, 1, true),
  ('Honda', 'CBR650R', 'CBR650R', 649, 1, true),
```

('Honda', 'CL500', 'CL500', 471, 1, true),
('Honda', 'Click 110i', 'Click 110i', 108, 1, true),
('Honda', 'Click 125i', 'Click 125i', 125, 1, true),
('Honda', 'Click 150i', 'Click 150i', 150, 1, true),
('Honda', 'Click 160', 'Click 160', 157, 1, true),
('Honda', 'CRF150L', 'CRF150L', 149, 1, true),
('Honda', 'CRF250L', 'CRF250L', 249, 1, true),
('Honda', 'CRF300 Rally', 'CRF300 Rally', 286, 1, true),
('Honda', 'CRF300L', 'CRF300L', 286, 1, true),
('Honda', 'Dio 110', 'Dio 110', 109, 1, true),
('Honda', 'Gold Wing', 'Gold Wing', 1833, 1, true),
('Honda', 'NC750X', 'NC750X', 745, 1, true),
('Honda', 'NX500', 'NX500', 471, 1, true),
('Honda', 'PCX 125', 'PCX 125', 125, 1, true),
('Honda', 'PCX 150', 'PCX 150', 149, 1, true),
('Honda', 'PCX 160', 'PCX 160', 157, 1, true),
('Honda', 'Rebel 1100', 'Rebel 1100', 1084, 1, true),
('Honda', 'Rebel 300', 'Rebel 300', 286, 1, true),
('Honda', 'Rebel 500', 'Rebel 500', 471, 1, true),
('Honda', 'RS125', 'RS125', 124, 1, true),
('Honda', 'Supra GTR 150', 'Supra GTR 150', 149, 1, true),
('Honda', 'TMX 125', 'TMX 125', 125, 1, true),
('Honda', 'TMX Supremo', 'TMX Supremo', 125, 1, true),
('Honda', 'Wave 100', 'Wave 100', 100, 1, true),
('Honda', 'Wave 110', 'Wave 110', 110, 1, true),
('Honda', 'Wave 125', 'Wave 125', 125, 1, true),
('Honda', 'Wave RSX', 'Wave RSX', 110, 1, true),
('Honda', 'X-ADV', 'X-ADV', 745, 1, true),
('Honda', 'XL750 Transalp', 'XL750 Transalp', 755, 1, true),
('Honda', 'XR150L', 'XR150L', 149, 1, true),
('Honda', 'XRM 110', 'XRM 110', 109, 1, true),
('Honda', 'XRM 125', 'XRM 125', 124, 1, true),
('Yamaha', 'Aerox 155', 'Aerox 155', 155, 1, true),
('Yamaha', 'Bolt', 'Bolt', 942, 1, true),
('Yamaha', 'Fino', 'Fino', 125, 1, true),
('Yamaha', 'FZ1', 'FZ1', 998, 1, true),
('Yamaha', 'FZ150i', 'FZ150i', 149, 1, true),
('Yamaha', 'Lexi', 'Lexi', 125, 1, true),
('Yamaha', 'Mio Fazzio', 'Mio Fazzio', 125, 1, true),
('Yamaha', 'Mio Gear', 'Mio Gear', 125, 1, true),
('Yamaha', 'Mio Gravis', 'Mio Gravis', 125, 1, true),
('Yamaha', 'Mio i125', 'Mio i125', 125, 1, true),
('Yamaha', 'Mio Soul i', 'Mio Soul i', 115, 1, true),
('Yamaha', 'Mio Soul i 125', 'Mio Soul i 125', 125, 1, true),
('Yamaha', 'Mio Sporty', 'Mio Sporty', 115, 1, true),

('Yamaha', 'MT-03', 'MT-03', 321, 1, true),
('Yamaha', 'MT-07', 'MT-07', 689, 1, true),
('Yamaha', 'MT-09', 'MT-09', 890, 1, true),
('Yamaha', 'MT-10', 'MT-10', 998, 1, true),
('Yamaha', 'MT-15', 'MT-15', 155, 1, true),
('Yamaha', 'NMAX', 'NMAX', 155, 1, true),
('Yamaha', 'R15', 'R15', 149, 1, true),
('Yamaha', 'R15 V3', 'R15 V3', 155, 1, true),
('Yamaha', 'R15M', 'R15M', 155, 1, true),
('Yamaha', 'R3', 'R3', 321, 1, true),
('Yamaha', 'Sight', 'Sight', 115, 1, true),
('Yamaha', 'Sniper 135', 'Sniper 135', 134, 1, true),
('Yamaha', 'Sniper 150', 'Sniper 150', 150, 1, true),
('Yamaha', 'Sniper 155', 'Sniper 155', 155, 1, true),
('Yamaha', 'Tenere 700', 'Tenere 700', 689, 1, true),
('Yamaha', 'TMAX (530cc)', 'TMAX', 530, 1, true),
('Yamaha', 'TMAX (560cc)', 'TMAX', 560, 0, true),
('Yamaha', 'Tracer 7', 'Tracer 7', 689, 1, true),
('Yamaha', 'Tracer 9', 'Tracer 9', 890, 1, true),
('Yamaha', 'V-Star 250', 'V-Star 250', 249, 1, true),
('Yamaha', 'XMAX', 'XMAX', 300, 1, true),
('Yamaha', 'XSR155', 'XSR155', 155, 1, true),
('Yamaha', 'XSR700', 'XSR700', 689, 1, true),
('Yamaha', 'XSR900', 'XSR900', 889, 1, true),
('Yamaha', 'YBR 125', 'YBR 125', 125, 1, true),
('Yamaha', 'YTX 125', 'YTX 125', 125, 1, true),
('Yamaha', 'YZF-R1', 'YZF-R1', 998, 1, true),
('Yamaha', 'YZF-R6', 'YZF-R6', 599, 1, true),
('Yamaha', 'YZF-R7', 'YZF-R7', 689, 1, true),
('Suzuki', 'Access', 'Access', 124, 1, true),
('Suzuki', 'Address', 'Address', 113, 1, true),
('Suzuki', 'Avenis', 'Avenis', 124, 1, true),
('Suzuki', 'Bandit 1200', 'Bandit 1200', 1157, 1, true),
('Suzuki', 'Bandit 1250', 'Bandit 1250', 1255, 1, true),
('Suzuki', 'Bandit 600', 'Bandit 600', 599, 1, true),
('Suzuki', 'Bandit 650', 'Bandit 650', 656, 1, true),
('Suzuki', 'Best 125', 'Best 125', 124, 1, true),
('Suzuki', 'Boulevard C50', 'Boulevard C50', 805, 1, true),
('Suzuki', 'Boulevard M109', 'Boulevard M109', 1783, 1, true),
('Suzuki', 'Burgman 250', 'Burgman 250', 249, 1, true),
('Suzuki', 'Burgman 400', 'Burgman 400', 400, 1, true),
('Suzuki', 'Burgman 650', 'Burgman 650', 638, 1, true),
('Suzuki', 'Burgman Street', 'Burgman Street', 124, 1, true),
('Suzuki', 'Burgman Street 125 EX', 'Burgman Street 125 EX', 124, 1, true),
('Suzuki', 'DR-Z125', 'DR-Z125', 124, 1, true),

('Suzuki', 'DR-Z400', 'DR-Z400', 398, 1, true),
('Suzuki', 'DR150', 'DR150', 149, 1, true),
('Suzuki', 'DR160', 'DR160', 162, 1, true),
('Suzuki', 'DR200', 'DR200', 199, 1, true),
('Suzuki', 'DR650', 'DR650', 644, 1, true),
('Suzuki', 'GD110', 'GD110', 113, 1, true),
('Suzuki', 'Gixxer 155', 'Gixxer 155', 155, 1, true),
('Suzuki', 'Gixxer 250', 'Gixxer 250', 249, 1, true),
('Suzuki', 'Gixxer SF 155', 'Gixxer SF 155', 155, 1, true),
('Suzuki', 'Gixxer SF250', 'Gixxer SF250', 249, 1, true),
('Suzuki', 'GN125', 'GN125', 124, 1, true),
('Suzuki', 'GN160', 'GN160', 162, 1, true),
('Suzuki', 'GSX-8R', 'GSX-8R', 776, 1, true),
('Suzuki', 'GSX-8S', 'GSX-8S', 776, 1, true),
('Suzuki', 'GSX-R1000', 'GSX-R1000', 999, 1, true),
('Suzuki', 'GSX-R600', 'GSX-R600', 599, 1, true),
('Suzuki', 'GSX-R750', 'GSX-R750', 750, 1, true),
('Suzuki', 'GSX-S1000', 'GSX-S1000', 999, 1, true),
('Suzuki', 'GSX-S1000GT', 'GSX-S1000GT', 999, 1, true),
('Suzuki', 'GSX-S750', 'GSX-S750', 749, 1, true),
('Suzuki', 'Hayabusa (1299cc)', 'Hayabusa', 1299, 1, true),
('Suzuki', 'Hayabusa (1340cc)', 'Hayabusa', 1340, 0, true),
('Suzuki', 'Hayate', 'Hayate', 113, 1, true),
('Suzuki', 'Raider 125', 'Raider 125', 125, 1, true),
('Suzuki', 'Raider J Crossover', 'Raider J Crossover', 113, 1, true),
('Suzuki', 'Raider R150', 'Raider R150', 147, 1, true),
('Suzuki', 'Skydrive 125', 'Skydrive 125', 124, 1, true),
('Suzuki', 'Skydrive Sport', 'Skydrive Sport', 113, 1, true),
('Suzuki', 'Smash 110', 'Smash 110', 113, 1, true),
('Suzuki', 'Smash 115', 'Smash 115', 115, 1, true),
('Suzuki', 'SV650', 'SV650', 645, 1, true),
('Suzuki', 'V-Strom 1050', 'V-Strom 1050', 1037, 1, true),
('Suzuki', 'V-Strom 160', 'V-Strom 160', 162, 1, true),
('Suzuki', 'V-Strom 250 SX', 'V-Strom 250 SX', 249, 1, true),
('Suzuki', 'V-Strom 650', 'V-Strom 650', 645, 1, true),
('Suzuki', 'V-Strom 800DE', 'V-Strom 800DE', 776, 1, true),
('Kawasaki', 'Barako', 'Barako', 175, 1, true),
('Kawasaki', 'Barako II', 'Barako II', 177, 1, true),
('Kawasaki', 'Barako III FI', 'Barako III FI', 177, 1, true),
('Kawasaki', 'Curve', 'Curve', 112, 1, true),
('Kawasaki', 'ER-6f', 'ER-6f', 649, 1, true),
('Kawasaki', 'ER-6n', 'ER-6n', 649, 1, true),
('Kawasaki', 'Fury', 'Fury', 125, 1, true),
('Kawasaki', 'Fury RR', 'Fury RR', 125, 1, true),
('Kawasaki', 'KLR650', 'KLR650', 652, 1, true),

('Kawasaki', 'KLX125', 'KLX125', 125, 1, true),
('Kawasaki', 'KLX150', 'KLX150', 144, 1, true),
('Kawasaki', 'KLX150L', 'KLX150L', 144, 1, true),
('Kawasaki', 'KLX230', 'KLX230', 233, 1, true),
('Kawasaki', 'Ninja 250', 'Ninja 250', 249, 1, true),
('Kawasaki', 'Ninja 300', 'Ninja 300', 296, 1, true),
('Kawasaki', 'Ninja 400', 'Ninja 400', 399, 1, true),
('Kawasaki', 'Ninja 650', 'Ninja 650', 649, 1, true),
('Kawasaki', 'Ninja ZX-10R', 'Ninja ZX-10R', 998, 1, true),
('Kawasaki', 'Ninja ZX-14R', 'Ninja ZX-14R', 1441, 1, true),
('Kawasaki', 'Ninja ZX-25R', 'Ninja ZX-25R', 249, 1, true),
('Kawasaki', 'Ninja ZX-6R', 'Ninja ZX-6R', 636, 1, true),
('Kawasaki', 'Versys 1000', 'Versys 1000', 1043, 1, true),
('Kawasaki', 'Versys 650', 'Versys 650', 649, 1, true),
('Kawasaki', 'Versys-X 250', 'Versys-X 250', 249, 1, true),
('Kawasaki', 'Vulcan 1700', 'Vulcan 1700', 1700, 1, true),
('Kawasaki', 'Vulcan 900', 'Vulcan 900', 903, 1, true),
('Kawasaki', 'Vulcan S', 'Vulcan S', 649, 1, true),
('Kawasaki', 'W175', 'W175', 177, 1, true),
('Kawasaki', 'Z1000', 'Z1000', 1043, 1, true),
('Kawasaki', 'Z250', 'Z250', 249, 1, true),
('Kawasaki', 'Z300', 'Z300', 296, 1, true),
('Kawasaki', 'Z400', 'Z400', 399, 1, true),
('Kawasaki', 'Z650', 'Z650', 649, 1, true),
('Kawasaki', 'Z900', 'Z900', 948, 1, true),
('Kawasaki', 'ZZR1200', 'ZZR1200', 1164, 1, true),
('Kawasaki', 'ZZR600', 'ZZR600', 599, 1, true),
('RUSI', 'Adventure 110', 'Adventure 110', 110, 1, true),
('RUSI', 'Classic 125', 'Classic 125', 125, 1, true),
('RUSI', 'Classic 250', 'Classic 250', 249, 1, true),
('RUSI', 'Euro 100', 'Euro 100', 100, 1, true),
('RUSI', 'Flash 125', 'Flash 125', 125, 1, true),
('RUSI', 'Gala 150', 'Gala 150', 150, 1, true),
('RUSI', 'Gemini 125', 'Gemini 125', 125, 1, true),
('RUSI', 'Magma 110', 'Magma 110', 110, 1, true),
('RUSI', 'Magma 125', 'Magma 125', 125, 1, true),
('RUSI', 'Motoscout 125', 'Motoscout 125', 125, 1, true),
('RUSI', 'Pioneer 100', 'Pioneer 100', 100, 1, true),
('RUSI', 'Rapid 150', 'Rapid 150', 150, 1, true),
('RUSI', 'Raptor 150', 'Raptor 150', 150, 1, true),
('RUSI', 'RFI 125', 'RFI 125', 125, 1, true),
('RUSI', 'RFI 175', 'RFI 175', 175, 1, true),
('RUSI', 'Royal 125', 'Royal 125', 125, 1, true),
('RUSI', 'Saber 150', 'Saber 150', 150, 1, true),
('RUSI', 'Vista 150', 'Vista 150', 150, 1, true),

('MotorStar', 'Café 400', 'Café 400', 400, 1, true),
('MotorStar', 'Magnum 150', 'Magnum 150', 150, 1, true),
('MotorStar', 'Max 200', 'Max 200', 200, 1, true),
('MotorStar', 'Moto R150', 'Moto R150', 150, 1, true),
('MotorStar', 'Motoposh 110', 'Motoposh 110', 110, 1, true),
('MotorStar', 'Motoposh 125', 'Motoposh 125', 125, 1, true),
('MotorStar', 'MSX100', 'MSX100', 100, 1, true),
('MotorStar', 'MSX125', 'MSX125', 125, 1, true),
('MotorStar', 'Star X200R', 'Star X200R', 200, 1, true),
('MotorStar', 'V200', 'V200', 200, 1, true),
('MotorStar', 'X200R II', 'X200R II', 200, 1, true),
('MotorStar', 'Xplorer 150', 'Xplorer 150', 150, 1, true),
('MotorStar', 'Xplorer 200R', 'Xplorer 200R', 200, 1, true),
('MotorStar', 'Z200R', 'Z200R', 200, 1, true),
('MotorStar', 'Z250', 'Z250', 250, 1, true),
('Skygo', 'Archer', 'Archer', 107, 1, true),
('Skygo', 'Blink 125', 'Blink 125', 125, 1, true),
('Skygo', 'Boss 150', 'Boss 150', 145, 1, true),
('Skygo', 'Boss 200', 'Boss 200', 200, 1, true),
('Skygo', 'Duke', 'Duke', 107, 1, true),
('Skygo', 'Earl 150', 'Earl 150', 145, 1, true),
('Skygo', 'Earl 250', 'Earl 250', 250, 1, true),
('Skygo', 'Hero', 'Hero', 125, 1, true),
('Skygo', 'King 150', 'King 150', 145, 1, true),
('Skygo', 'KP V150', 'KP V150', 145, 1, true),
('Skygo', 'Prince 125', 'Prince 125', 124, 1, true),
('Skygo', 'Stallion 150-SM', 'Stallion 150-SM', 150, 1, true),
('Skygo', 'Wizard 125', 'Wizard 125', 124, 1, true),
('Skygo', 'Wizard 175', 'Wizard 175', 175, 1, true),
('Kymco', 'Active 110', 'Active 110', 111, 1, true),
('Kymco', 'Active 125', 'Active 125', 124, 1, true),
('Kymco', 'Active 50', 'Active 50', 50, 1, true),
('Kymco', 'Agility 125', 'Agility 125', 124, 1, true),
('Kymco', 'Agility 50', 'Agility 50', 49, 1, true),
('Kymco', 'AK 550', 'AK 550', 550, 1, true),
('Kymco', 'Dink 150', 'Dink 150', 149, 1, true),
('Kymco', 'Downtown 300i', 'Downtown 300i', 298, 1, true),
('Kymco', 'Downtown 350i', 'Downtown 350i', 321, 1, true),
('Kymco', 'DT X360', 'DT X360', 320, 1, true),
('Kymco', 'Grand Dink 250', 'Grand Dink 250', 249, 1, true),
('Kymco', 'K-XCT 300i', 'K-XCT 300i', 299, 1, true),
('Kymco', 'KRV 180', 'KRV 180', 175, 1, true),
('Kymco', 'Like 125', 'Like 125', 124, 1, true),
('Kymco', 'Like 150i', 'Like 150i', 150, 1, true),
('Kymco', 'People 125', 'People 125', 124, 1, true),

('Kymco', 'People 150', 'People 150', 149, 1, true),
('Kymco', 'Super 8 125', 'Super 8 125', 124, 1, true),
('Kymco', 'Super 8 150', 'Super 8 150', 149, 1, true),
('Kymco', 'Venox 250', 'Venox 250', 249, 1, true),
('Kymco', 'Xciting 400i', 'Xciting 400i', 399, 1, true),
('Kymco', 'Xciting S 400i', 'Xciting S 400i', 399, 1, true),
('CFMOTO', '1000MT-X', '1000MT-X', 997, 1, true),
('CFMOTO', '125 ST Papio', '125 ST Papio', 126, 1, true),
('CFMOTO', '150 Aura', '150 Aura', 149, 1, true),
('CFMOTO', '150NK', '150NK', 149, 1, true),
('CFMOTO', '150SC', '150SC', 149, 1, true),
('CFMOTO', '250NK', '250NK', 249, 1, true),
('CFMOTO', '250SR', '250SR', 249, 1, true),
('CFMOTO', '300CL-X', '300CL-X', 292, 1, true),
('CFMOTO', '300NK', '300NK', 292, 1, true),
('CFMOTO', '300SR', '300SR', 292, 1, true),
('CFMOTO', '400GT', '400GT', 400, 1, true),
('CFMOTO', '400NK', '400NK', 400, 1, true),
('CFMOTO', '450CL-C', '450CL-C', 449, 1, true),
('CFMOTO', '450CL-C Bobber', '450CL-C Bobber', 449, 1, true),
('CFMOTO', '450MT', '450MT', 449, 1, true),
('CFMOTO', '450NK', '450NK', 449, 1, true),
('CFMOTO', '450SR', '450SR', 449, 1, true),
('CFMOTO', '450SR-S', '450SR-S', 449, 1, true),
('CFMOTO', '500SR VOOM', '500SR VOOM', 449, 1, true),
('CFMOTO', '650GT', '650GT', 649, 1, true),
('CFMOTO', '650MT', '650MT', 649, 1, true),
('CFMOTO', '650NK', '650NK', 649, 1, true),
('CFMOTO', '675SR-R', '675SR-R', 675, 1, true),
('CFMOTO', '700CL-X', '700CL-X', 693, 1, true),
('CFMOTO', '700MT', '700MT', 693, 1, true),
('CFMOTO', '750SR-S', '750SR-S', 749, 1, true),
('CFMOTO', '800MT', '800MT', 799, 1, true),
('CFMOTO', '800MT Explore', '800MT Explore', 799, 1, true),
('CFMOTO', '800MT Sport', '800MT Sport', 799, 1, true),
('CFMOTO', '800MT-X', '800MT-X', 799, 1, true),
('CFMOTO', '800NK', '800NK', 799, 1, true),
('CFMOTO', 'ST Papio', 'ST Papio', 126, 1, true),
('TVS', 'Apache RR 310', 'Apache RR 310', 312, 1, true),
('TVS', 'Apache RTR 160', 'Apache RTR 160', 159, 1, true),
('TVS', 'Apache RTR 180', 'Apache RTR 180', 177, 1, true),
('TVS', 'Apache RTR 200 4V', 'Apache RTR 200 4V', 197, 1, true),
('TVS', 'Jupiter', 'Jupiter', 109, 1, true),
('TVS', 'Jupiter 125', 'Jupiter 125', 124, 1, true),
('TVS', 'NTORQ 125', 'NTORQ 125', 124, 1, true),

('TVS', 'Radeon', 'Radeon', 109, 1, true),
('TVS', 'Raider 125', 'Raider 125', 124, 1, true),
('TVS', 'Ronin', 'Ronin', 225, 1, true),
('TVS', 'Sport', 'Sport', 109, 1, true),
('TVS', 'Star City', 'Star City', 109, 1, true),
('TVS', 'Star City Plus', 'Star City Plus', 109, 1, true),
('TVS', 'Victor', 'Victor', 109, 1, true),
('TVS', 'Wego', 'Wego', 109, 1, true),
('TVS', 'XL100', 'XL100', 99, 1, true),
('TVS', 'Zest 110', 'Zest 110', 109, 1, true),
('Keeway', 'Cafe Racer 152', 'Cafe Racer 152', 149, 1, true),
('Keeway', 'CR 152', 'CR 152', 149, 1, true),
('Keeway', 'K-Light 202', 'K-Light 202', 197, 1, true),
('Keeway', 'K-Light 250V', 'K-Light 250V', 249, 1, true),
('Keeway', 'RKF 125', 'RKF 125', 125, 1, true),
('Keeway', 'RKF 200', 'RKF 200', 197, 1, true),
('Keeway', 'RKS 125', 'RKS 125', 124, 1, true),
('Keeway', 'RKS 150', 'RKS 150', 149, 1, true),
('Keeway', 'RKS 200', 'RKS 200', 197, 1, true),
('Keeway', 'Sixties 125', 'Sixties 125', 125, 1, true),
('Keeway', 'Superlight 125', 'Superlight 125', 125, 1, true),
('Keeway', 'Superlight 200', 'Superlight 200', 197, 1, true),
('Keeway', 'TX 125', 'TX 125', 124, 1, true),
('Keeway', 'TX 200', 'TX 200', 197, 1, true),
('Keeway', 'V302C', 'V302C', 298, 1, true),
('Keeway', 'Vieste 125', 'Vieste 125', 125, 1, true),
('Keeway', 'Vieste 300', 'Vieste 300', 278, 1, true),
('Benelli', '180S', '180S', 175, 1, true),
('Benelli', '302R', '302R', 300, 1, true),
('Benelli', '302S', '302S', 300, 1, true),
('Benelli', '502C', '502C', 500, 1, true),
('Benelli', '752S', '752S', 754, 1, true),
('Benelli', 'BN 125', 'BN 125', 125, 1, true),
('Benelli', 'BN 251', 'BN 251', 249, 1, true),
('Benelli', 'Imperiale 400', 'Imperiale 400', 374, 1, true),
('Benelli', 'Leoncinco 500', 'Leoncinco 500', 500, 1, true),
('Benelli', 'Leoncino 250', 'Leoncino 250', 249, 1, true),
('Benelli', 'TNT 15', 'TNT 15', 148, 1, true),
('Benelli', 'TNT 150i', 'TNT 150i', 150, 1, true),
('Benelli', 'TNT 180S', 'TNT 180S', 175, 1, true),
('Benelli', 'TNT 25', 'TNT 25', 249, 1, true),
('Benelli', 'TNT 600', 'TNT 600', 600, 1, true),
('Benelli', 'TNT 899', 'TNT 899', 898, 1, true),
('Benelli', 'Tornado 125', 'Tornado 125', 125, 1, true),
('Benelli', 'Tornado 550', 'Tornado 550', 554, 1, true),

('Benelli', 'Tornado Tre 1130', 'Tornado Tre 1130', 1130, 1, true),
('Benelli', 'TRK 251', 'TRK 251', 249, 1, true),
('Benelli', 'TRK 502', 'TRK 502', 500, 1, true),
('Benelli', 'TRK 502 X', 'TRK 502 X', 500, 1, true),
('Benelli', 'TRK 702', 'TRK 702', 698, 1, true),
('Benelli', 'TRK 702 X', 'TRK 702 X', 698, 1, true),
('KTM', '125 Duke', '125 Duke', 125, 1, true),
('KTM', '1290 Super Adventure', '1290 Super Adventure', 1301, 1, true),
('KTM', '1390 Super Adventure', '1390 Super Adventure', 1350, 1, true),
('KTM', '1390 Super Duke R', '1390 Super Duke R', 1350, 1, true),
('KTM', '200 Duke', '200 Duke', 199, 1, true),
('KTM', '250 Duke', '250 Duke', 248, 1, true),
('KTM', '350 EXC-F', '350 EXC-F', 349, 1, true),
('KTM', '390 Adventure', '390 Adventure', 373, 1, true),
('KTM', '390 Duke', '390 Duke', 373, 1, true),
('KTM', '390 Enduro R', '390 Enduro R', 399, 1, true),
('KTM', '390 SMC R', '390 SMC R', 399, 1, true),
('KTM', '450 EXC-F', '450 EXC-F', 449, 1, true),
('KTM', '500 EXC-F', '500 EXC-F', 510, 1, true),
('KTM', '690 Duke', '690 Duke', 690, 1, true),
('KTM', '690 SMC R', '690 SMC R', 692, 1, true),
('KTM', '790 Adventure', '790 Adventure', 799, 1, true),
('KTM', '790 Duke', '790 Duke', 799, 1, true),
('KTM', '890 Adventure', '890 Adventure', 889, 1, true),
('KTM', '890 Duke R', '890 Duke R', 889, 1, true),
('KTM', '990 Duke', '990 Duke', 947, 1, true),
('KTM', '990 RC R', '990 RC R', 947, 1, true),
('KTM', 'RC 125', 'RC 125', 125, 1, true),
('KTM', 'RC 200', 'RC 200', 199, 1, true),
('KTM', 'RC 390', 'RC 390', 373, 1, true),
('Bristol', 'AD200', 'AD200', 200, 1, true),
('Bristol', 'Bobber 250', 'Bobber 250', 249, 1, true),
('Bristol', 'BR400i', 'BR400i', 400, 1, true),
('Bristol', 'Cafe Racer 250', 'Cafe Racer 250', 249, 1, true),
('Bristol', 'Classic 250', 'Classic 250', 249, 1, true),
('Bristol', 'Maxie 400', 'Maxie 400', 400, 1, true),
('Bristol', 'Raptor 500', 'Raptor 500', 500, 1, true),
('Bristol', 'Vantage 400', 'Vantage 400', 400, 1, true),
('Bristol', 'Venturi 400', 'Venturi 400', 400, 1, true),
('Royal Enfield', 'Bear 650', 'Bear 650', 648, 1, true),
('Royal Enfield', 'Bullet 350', 'Bullet 350', 346, 1, true),
('Royal Enfield', 'Classic 350', 'Classic 350', 346, 1, true),
('Royal Enfield', 'Classic 650', 'Classic 650', 648, 1, true),
('Royal Enfield', 'Continental GT 650', 'Continental GT 650', 648, 1, true),
('Royal Enfield', 'Goan Classic 350', 'Goan Classic 350', 349, 1, true),

('Royal Enfield', 'Guerrilla 450', 'Guerrilla 450', 452, 1, true),
('Royal Enfield', 'Himalayan 411', 'Himalayan 411', 411, 1, true),
('Royal Enfield', 'Himalayan 450', 'Himalayan 450', 452, 1, true),
('Royal Enfield', 'Hunter 350', 'Hunter 350', 349, 1, true),
('Royal Enfield', 'Interceptor 650', 'Interceptor 650', 648, 1, true),
('Royal Enfield', 'Meteor 350', 'Meteor 350', 349, 1, true),
('Royal Enfield', 'Scram 411', 'Scram 411', 411, 1, true),
('Royal Enfield', 'Shotgun 650', 'Shotgun 650', 648, 1, true),
('Royal Enfield', 'Super Meteor 650', 'Super Meteor 650', 648, 1, true),
('SYM', 'Allo 125', 'Allo 125', 124, 1, true),
('SYM', 'Allo 150', 'Allo 150', 149, 1, true),
('SYM', 'Allo 50', 'Allo 50', 49, 1, true),
('SYM', 'Bonus 100', 'Bonus 100', 100, 1, true),
('SYM', 'Bonus 110', 'Bonus 110', 107, 1, true),
('SYM', 'Citycom 125', 'Citycom 125', 124, 1, true),
('SYM', 'Citycom 300', 'Citycom 300', 278, 1, true),
('SYM', 'Crox 125', 'Crox 125', 124, 1, true),
('SYM', 'Crox 50', 'Crox 50', 49, 1, true),
('SYM', 'Cruisym 125', 'Cruisym 125', 124, 1, true),
('SYM', 'Cruisym 250i', 'Cruisym 250i', 249, 1, true),
('SYM', 'Cruisym 300', 'Cruisym 300', 278, 1, true),
('SYM', 'Cruisym Alpha 125', 'Cruisym Alpha 125', 124, 1, true),
('SYM', 'Jet 14 125', 'Jet 14 125', 124, 1, true),
('SYM', 'Jet X 125', 'Jet X 125', 124, 1, true),
('SYM', 'Maxsym 400', 'Maxsym 400', 399, 1, true),
('SYM', 'Maxsym TL 500', 'Maxsym TL 500', 465, 1, true),
('SYM', 'NH-T 200', 'NH-T 200', 183, 1, true),
('SYM', 'VF3i 185', 'VF3i 185', 183, 1, true),
('SYM', 'Wolf 125', 'Wolf 125', 124, 1, true),
('Bajaj', '125 Classic', '125 Classic', 124, 1, true),
('Bajaj', 'Avenger 180', 'Avenger 180', 178, 1, true),
('Bajaj', 'Avenger 200', 'Avenger 200', 200, 1, true),
('Bajaj', 'Avenger 220', 'Avenger 220', 220, 1, true),
('Bajaj', 'Chetak 145', 'Chetak 145', 145, 1, true),
('Bajaj', 'Chetak Electric', 'Chetak Electric', NULL, 1, false),
('Bajaj', 'CT100', 'CT100', 99, 1, true),
('Bajaj', 'CT110', 'CT110', 115, 1, true),
('Bajaj', 'CT125X', 'CT125X', 124, 1, true),
('Bajaj', 'Discover 100', 'Discover 100', 94, 1, true),
('Bajaj', 'Discover 110', 'Discover 110', 115, 1, true),
('Bajaj', 'Discover 125', 'Discover 125', 124, 1, true),
('Bajaj', 'Discover 150', 'Discover 150', 145, 1, true),
('Bajaj', 'Dominar 250', 'Dominar 250', 248, 1, true),
('Bajaj', 'Dominar 400', 'Dominar 400', 373, 1, true),
('Bajaj', 'Platina 100', 'Platina 100', 102, 1, true),

('Bajaj', 'Platina 110', 'Platina 110', 115, 1, true),
('Bajaj', 'Pulsar 125', 'Pulsar 125', 124, 1, true),
('Bajaj', 'Pulsar 135 LS', 'Pulsar 135 LS', 135, 1, true),
('Bajaj', 'Pulsar 150', 'Pulsar 150', 149, 1, true),
('Bajaj', 'Pulsar 180', 'Pulsar 180', 178, 1, true),
('Bajaj', 'Pulsar 220F', 'Pulsar 220F', 220, 1, true),
('Bajaj', 'Pulsar NS160', 'Pulsar NS160', 160, 1, true),
('Bajaj', 'Pulsar NS200', 'Pulsar NS200', 199, 1, true),
('Bajaj', 'Pulsar RS200', 'Pulsar RS200', 199, 1, true),
('Bajaj', 'Wind 125', 'Wind 125', 125, 1, true),
('Bajaj', 'XCD 125', 'XCD 125', 125, 1, true),
('Zontes', '310 T1', '310 T1', 312, 1, true),
('Zontes', '310 V', '310 V', 312, 1, true),
('Zontes', '310 X1', '310 X1', 312, 1, true),
('Zontes', '368G', '368G', 368, 1, true),
('Zontes', '703F', '703F', 699, 1, true),
('Zontes', '703RR', '703RR', 699, 1, true),
('Zontes', 'E-Mantis 125', 'E-Mantis 125', 124, 1, true),
('Zontes', 'GK350', 'GK350', 348, 1, true),
('Zontes', 'RA1', 'RA1', 125, 1, true),
('Zontes', 'ZT125', 'ZT125', 124, 1, true),
('Zontes', 'ZT125-G1 Scrambler', 'ZT125-G1 Scrambler', 124, 1, true),
('Zontes', 'ZT125-M', 'ZT125-M', 124, 1, true),
('Zontes', 'ZT125-U1', 'ZT125-U1', 124, 1, true),
('Zontes', 'ZT250', 'ZT250', 249, 1, true),
('Zontes', 'ZT310-M', 'ZT310-M', 309, 1, true),
('Zontes', 'ZT350-GK', 'ZT350-GK', 348, 1, true),
('Zontes', 'ZT350-T', 'ZT350-T', 348, 1, true),
('Vespa', '946 125', '946 125', 125, 1, true),
('Vespa', 'ET4 125', 'ET4 125', 124, 1, true),
('Vespa', 'ET4 150', 'ET4 150', 150, 1, true),
('Vespa', 'ET4 50', 'ET4 50', 49, 1, true),
('Vespa', 'GTS 125', 'GTS 125', 124, 1, true),
('Vespa', 'GTS 150', 'GTS 150', 155, 1, true),
('Vespa', 'GTS 250', 'GTS 250', 244, 1, true),
('Vespa', 'GTS 300', 'GTS 300', 278, 1, true),
('Vespa', 'GTV 300', 'GTV 300', 278, 1, true),
('Vespa', 'LX 125', 'LX 125', 124, 1, true),
('Vespa', 'LX 150', 'LX 150', 150, 1, true),
('Vespa', 'LX 50', 'LX 50', 49, 1, true),
('Vespa', 'Primavera 125', 'Primavera 125', 124, 1, true),
('Vespa', 'Primavera 150', 'Primavera 150', 155, 1, true),
('Vespa', 'Primavera 50', 'Primavera 50', 49, 1, true),
('Vespa', 'S 125', 'S 125', 124, 1, true),
('Vespa', 'S 150', 'S 150', 150, 1, true),

('Vespa', 'S 50', 'S 50', 49, 1, true),
('Vespa', 'Sprint 125', 'Sprint 125', 124, 1, true),
('Vespa', 'Sprint 150', 'Sprint 150', 155, 1, true),
('Vespa', 'Sprint 50', 'Sprint 50', 49, 1, true),
('Piaggio', 'Beverly 250', 'Beverly 250', 244, 1, true),
('Piaggio', 'Beverly 300', 'Beverly 300', 278, 1, true),
('Piaggio', 'Beverly 350', 'Beverly 350', 330, 1, true),
('Piaggio', 'Beverly 400', 'Beverly 400', 399, 1, true),
('Piaggio', 'Liberty 125', 'Liberty 125', 124, 1, true),
('Piaggio', 'Liberty 150', 'Liberty 150', 155, 1, true),
('Piaggio', 'Liberty 50', 'Liberty 50', 49, 1, true),
('Piaggio', 'Medley 125', 'Medley 125', 125, 1, true),
('Piaggio', 'Medley 150', 'Medley 150', 155, 1, true),
('Piaggio', 'MP3 300', 'MP3 300', 278, 1, true),
('Piaggio', 'MP3 400', 'MP3 400', 399, 1, true),
('Piaggio', 'MP3 500', 'MP3 500', 493, 1, true),
('Piaggio', 'MP3 530', 'MP3 530', 530, 1, true),
('Piaggio', 'X10 125', 'X10 125', 124, 1, true),
('Piaggio', 'X10 350', 'X10 350', 330, 1, true),
('Piaggio', 'X9 250', 'X9 250', 244, 1, true),
('Piaggio', 'X9 500', 'X9 500', 460, 1, true),
('Piaggio', 'Zip 50', 'Zip 50', 49, 1, true),
('BMW Motorrad', 'C 400 GT', 'C 400 GT', 350, 1, true),
('BMW Motorrad', 'C 650 GT', 'C 650 GT', 647, 1, true),
('BMW Motorrad', 'F 650 CS', 'F 650 CS', 652, 1, true),
('BMW Motorrad', 'F 650 GS', 'F 650 GS', 652, 1, true),
('BMW Motorrad', 'F 700 GS', 'F 700 GS', 798, 1, true),
('BMW Motorrad', 'F 750 GS', 'F 750 GS', 853, 1, true),
('BMW Motorrad', 'F 800 GS', 'F 800 GS', 798, 1, true),
('BMW Motorrad', 'F 850 GS', 'F 850 GS', 853, 1, true),
('BMW Motorrad', 'F 900 GS', 'F 900 GS', 895, 1, true),
('BMW Motorrad', 'F 900 XR', 'F 900 XR', 895, 1, true),
('BMW Motorrad', 'G 310 GS', 'G 310 GS', 313, 1, true),
('BMW Motorrad', 'G 310 R', 'G 310 R', 313, 1, true),
('BMW Motorrad', 'K 1200 S', 'K 1200 S', 1157, 1, true),
('BMW Motorrad', 'K 1300 S', 'K 1300 S', 1293, 1, true),
('BMW Motorrad', 'K 1600 B', 'K 1600 B', 1649, 1, true),
('BMW Motorrad', 'K 1600 GT', 'K 1600 GT', 1649, 1, true),
('BMW Motorrad', 'M 1000 RR', 'M 1000 RR', 999, 1, true),
('BMW Motorrad', 'R 1100 GS', 'R 1100 GS', 1085, 1, true),
('BMW Motorrad', 'R 1150 GS', 'R 1150 GS', 1130, 1, true),
('BMW Motorrad', 'R 1200 GS', 'R 1200 GS', 1170, 1, true),
('BMW Motorrad', 'R 1250 GS', 'R 1250 GS', 1254, 1, true),
('BMW Motorrad', 'R 1300 GS', 'R 1300 GS', 1300, 1, true),
('BMW Motorrad', 'R 18', 'R 18', 1802, 1, true),

('BMW Motorrad', 'R nineT', 'R nineT', 1170, 1, true),
('BMW Motorrad', 'S 1000 R', 'S 1000 R', 999, 1, true),
('BMW Motorrad', 'S 1000 RR', 'S 1000 RR', 999, 1, true),
('BMW Motorrad', 'S 1000 XR', 'S 1000 XR', 999, 1, true),
('Ducati', 'DesertX', 'DesertX', 937, 1, true),
('Ducati', 'Diavel 1200', 'Diavel 1200', 1198, 1, true),
('Ducati', 'Diavel 1260', 'Diavel 1260', 1262, 1, true),
('Ducati', 'Diavel V4', 'Diavel V4', 1158, 1, true),
('Ducati', 'Hypermotard 821', 'Hypermotard 821', 821, 1, true),
('Ducati', 'Hypermotard 939', 'Hypermotard 939', 937, 1, true),
('Ducati', 'Hypermotard 950', 'Hypermotard 950', 937, 1, true),
('Ducati', 'Monster 1200', 'Monster 1200', 1198, 1, true),
('Ducati', 'Monster 600', 'Monster 600', 583, 1, true),
('Ducati', 'Monster 620', 'Monster 620', 618, 1, true),
('Ducati', 'Monster 696', 'Monster 696', 696, 1, true),
('Ducati', 'Monster 797', 'Monster 797', 803, 1, true),
('Ducati', 'Monster 937', 'Monster 937', 937, 1, true),
('Ducati', 'Multistrada 1200', 'Multistrada 1200', 1198, 1, true),
('Ducati', 'Multistrada 1260', 'Multistrada 1260', 1262, 1, true),
('Ducati', 'Multistrada 950', 'Multistrada 950', 937, 1, true),
('Ducati', 'Multistrada V2', 'Multistrada V2', 937, 1, true),
('Ducati', 'Multistrada V4', 'Multistrada V4', 1158, 1, true),
('Ducati', 'Panigale V2', 'Panigale V2', 955, 1, true),
('Ducati', 'Panigale V4', 'Panigale V4', 1103, 1, true),
('Ducati', 'Panigale V4 R', 'Panigale V4 R', 998, 1, true),
('Ducati', 'Scrambler 400 X', 'Scrambler 400 X', 399, 1, true),
('Ducati', 'Scrambler 800', 'Scrambler 800', 803, 1, true),
('Ducati', 'Streetfighter V2', 'Streetfighter V2', 955, 1, true),
('Ducati', 'Streetfighter V4', 'Streetfighter V4', 1103, 1, true),
('Ducati', 'Supersport 939', 'Supersport 939', 937, 1, true),
('Ducati', 'Supersport 950', 'Supersport 950', 937, 1, true),
('Harley-Davidson', 'Fat Bob 114', 'Fat Bob 114', 1868, 1, true),
('Harley-Davidson', 'Fat Boy 114', 'Fat Boy 114', 1868, 1, true),
('Harley-Davidson', 'Forty-Eight', 'Forty-Eight', 1202, 1, true),
('Harley-Davidson', 'Heritage Classic 114', 'Heritage Classic 114', 1868, 1, true),
('Harley-Davidson', 'Iron 883', 'Iron 883', 883, 1, true),
('Harley-Davidson', 'Low Rider S', 'Low Rider S', 1923, 1, true),
('Harley-Davidson', 'Low Rider ST', 'Low Rider ST', 1923, 1, true),
('Harley-Davidson', 'Nightster', 'Nightster', 975, 1, true),
('Harley-Davidson', 'Pan America 1250', 'Pan America 1250', 1252, 1, true),
('Harley-Davidson', 'Road Glide', 'Road Glide', 1868, 1, true),
('Harley-Davidson', 'Road King', 'Road King', 1746, 1, true),
('Harley-Davidson', 'Softail Standard', 'Softail Standard', 1746, 1, true),
('Harley-Davidson', 'Sportster 1200', 'Sportster 1200', 1202, 1, true),
('Harley-Davidson', 'Sportster 883', 'Sportster 883', 883, 1, true),

```
  ('Harley-Davidson', 'Sportster S', 'Sportster S', 1252, 1, true),
  ('Harley-Davidson', 'Street 500', 'Street 500', 494, 1, true),
  ('Harley-Davidson', 'Street 750', 'Street 750', 749, 1, true),
  ('Harley-Davidson', 'Street Bob 114', 'Street Bob 114', 1868, 1, true),
  ('Harley-Davidson', 'Street Glide', 'Street Glide', 1868, 1, true),
  ('Harley-Davidson', 'Tri Glide Ultra', 'Tri Glide Ultra', 1868, 1, true),
  ('Harley-Davidson', 'Ultra Limited', 'Ultra Limited', 1923, 1, true),
  ('Triumph', 'Bonneville T100', 'Bonneville T100', 900, 1, true),
  ('Triumph', 'Bonneville T120', 'Bonneville T120', 1200, 1, true),
  ('Triumph', 'Daytona 660', 'Daytona 660', 660, 1, true),
  ('Triumph', 'Daytona 675', 'Daytona 675', 675, 1, true),
  ('Triumph', 'Rocket 3', 'Rocket 3', 2458, 1, true),
  ('Triumph', 'Rocket III', 'Rocket III', 2294, 1, true),
  ('Triumph', 'Scrambler 400 X', 'Scrambler 400 X', 398, 1, true),
  ('Triumph', 'Speed 400', 'Speed 400', 398, 1, true),
  ('Triumph', 'Speed Triple 1050', 'Speed Triple 1050', 1050, 1, true),
  ('Triumph', 'Speed Triple 1200 RS', 'Speed Triple 1200 RS', 1160, 1, true),
  ('Triumph', 'Speed Twin 1200', 'Speed Twin 1200', 1200, 1, true),
  ('Triumph', 'Speed Twin 900', 'Speed Twin 900', 900, 1, true),
  ('Triumph', 'Street Triple 675', 'Street Triple 675', 675, 1, true),
  ('Triumph', 'Street Triple 765', 'Street Triple 765', 765, 1, true),
  ('Triumph', 'Street Twin', 'Street Twin', 900, 1, true),
  ('Triumph', 'Thruxton 1200', 'Thruxton 1200', 1200, 1, true),
  ('Triumph', 'Thruxton 900', 'Thruxton 900', 865, 1, true),
  ('Triumph', 'Tiger 1200', 'Tiger 1200', 1160, 1, true),
  ('Triumph', 'Tiger 800', 'Tiger 800', 799, 1, true),
  ('Triumph', 'Tiger 900', 'Tiger 900', 888, 1, true),
  ('Triumph', 'Tiger Sport 1050', 'Tiger Sport 1050', 1050, 1, true),
  ('Triumph', 'Tiger Sport 660', 'Tiger Sport 660', 660, 1, true),
  ('Triumph', 'Trident 660', 'Trident 660', 660, 1, true);

-- Convert prior catalog labels such as "Honda ADV160" to the PDF's single
-- canonical entry before upserting. Appointment snapshots are updated in the
-- same migration, so reschedules and admin edits still resolve their catalog row.
CREATE TEMP TABLE _motorcycle_catalog_legacy_mapping ON COMMIT
DROP AS
WITH candidates AS (
  SELECT
    catalog.id,
    catalog.brand AS old_brand,
    catalog.model AS old_model,
    source.brand AS new_brand,
    source.model AS new_model,
    source.engine_cc AS new_engine_cc,
    source.is_active AS new_is_active,
```

```
    row_number() OVER (
      PARTITION BY catalog.id
      ORDER BY source.legacy_preference, source.engine_cc NULLS LAST,
source.model
    ) AS candidate_rank
  FROM public.motorcycle_catalog AS catalog
  JOIN _motorcycle_catalog_source AS source
    ON regexp_replace(lower(btrim(catalog.brand)), '[^a-z0-9]+', '', 'g')
       = regexp_replace(lower(btrim(source.brand)), '[^a-z0-9]+', '', 'g')
   AND (
     regexp_replace(lower(btrim(catalog.model)), '[^a-z0-9]+', '', 'g')
       = regexp_replace(lower(btrim(source.model)), '[^a-z0-9]+', '', 'g')
     OR regexp_replace(lower(btrim(catalog.model)), '[^a-z0-9]+', '', 'g')
       = regexp_replace(lower(btrim(source.legacy_model)), '[^a-z0-9]+', '', 'g')
     OR regexp_replace(lower(btrim(catalog.model)), '[^a-z0-9]+', '', 'g')
       = regexp_replace(lower(btrim(source.brand || source.legacy_model)),
'[^a-z0-9]+', '', 'g')
   )
)
SELECT
  id, old_brand, old_model, new_brand, new_model, new_engine_cc,
new_is_active
FROM candidates
WHERE candidate_rank = 1;

UPDATE public.appointments AS appointment
SET
  moto_brand = mapping.new_brand,
  moto_model = mapping.new_model
FROM _motorcycle_catalog_legacy_mapping AS mapping
WHERE lower(btrim(appointment.moto_brand)) =
lower(btrim(mapping.old_brand))
  AND lower(btrim(appointment.moto_model)) =
lower(btrim(mapping.old_model));

UPDATE public.motorcycle_catalog AS catalog
SET
  brand = mapping.new_brand,
  model = mapping.new_model,
  engine_cc = mapping.new_engine_cc,
  is_active = mapping.new_is_active,
  is_archived = false,
  archived_at = NULL
FROM _motorcycle_catalog_legacy_mapping AS mapping
WHERE catalog.id = mapping.id
```

```
  AND NOT EXISTS (
    SELECT 1
    FROM public.motorcycle_catalog AS existing
    WHERE existing.id <> catalog.id
      AND existing.brand = mapping.new_brand
      AND existing.model = mapping.new_model
  );

INSERT INTO public.motorcycle_catalog (
  brand, model, engine_cc, is_active, is_archived, archived_at
)
SELECT brand, model, engine_cc, is_active, false, NULL
FROM _motorcycle_catalog_source
ON CONFLICT (brand, model) DO UPDATE
SET
  engine_cc = EXCLUDED.engine_cc,
  is_active = EXCLUDED.is_active,
  is_archived = false,
  archived_at = NULL;

-- A customer-facing catalog item must be classifiable for service pricing.
-- This also keeps a future direct database write from exposing an item that
-- availability and booking would correctly reject.
UPDATE public.motorcycle_catalog
SET is_active = false
WHERE is_active = true
  AND engine_cc IS NULL;

ALTER TABLE public.motorcycle_catalog
  DROP CONSTRAINT IF EXISTS
motorcycle_catalog_active_requires_engine_cc;

ALTER TABLE public.motorcycle_catalog
  ADD CONSTRAINT motorcycle_catalog_active_requires_engine_cc
  CHECK (NOT is_active OR engine_cc IS NOT NULL);

COMMENT ON COLUMN public.motorcycle_catalog.cc_category IS
  'Derived from engine_cc: small_bike for 125cc and below; Large Bike above
125cc.';

NOTIFY pgrst, 'reload schema';
```

supabase/migrations/20261007120000_add_appointment_products_financial_reporting.sql

---

```
-- A booking records only the customer's product preference. The actual items,
-- quantities, and selling prices are captured by an administrator at service
-- time so financial reports use durable price snapshots rather than the
-- current catalog price.
ALTER TABLE public.appointments
  ADD COLUMN IF NOT EXISTS wants_products boolean NOT NULL DEFAULT
false;

CREATE TABLE IF NOT EXISTS public.appointment_products (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  appointment_id uuid NOT NULL REFERENCES public.appointments(id) ON
DELETE CASCADE,
  product_id uuid NOT NULL REFERENCES public.products(id),
  product_name text NOT NULL CHECK (btrim(product_name) <> ''),
  quantity integer NOT NULL CHECK (quantity BETWEEN 1 AND 999),
  unit_price numeric(10, 2) NOT NULL CHECK (unit_price >= 0),
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (appointment_id, product_id)
);

CREATE INDEX IF NOT EXISTS appointment_products_appointment_id_idx
  ON public.appointment_products (appointment_id);

REVOKE ALL ON TABLE public.appointment_products FROM anon;
GRANT SELECT, INSERT, UPDATE, DELETE ON TABLE
public.appointment_products TO authenticated;
GRANT ALL ON TABLE public.appointment_products TO service_role;
ALTER TABLE public.appointment_products ENABLE ROW LEVEL SECURITY;

DROP POLICY IF EXISTS "appointment products admin all" ON
public.appointment_products;
CREATE POLICY "appointment products admin all"
  ON public.appointment_products
  FOR ALL TO authenticated
  USING (public.has_role(auth.uid(), 'admin'))
  WITH CHECK (public.has_role(auth.uid(), 'admin'));

DROP TRIGGER IF EXISTS appointment_products_updated ON
public.appointment_products;
CREATE TRIGGER appointment_products_updated
  BEFORE UPDATE ON public.appointment_products
```

```
  FOR EACH ROW EXECUTE FUNCTION public.update_updated_at_column();

-- Preserve the current booking RPC for older clients and expose a distinct,
-- unambiguous signature for the current form's product preference.
CREATE FUNCTION public.create_booking_atomic(
  p_reference_code text, p_booking_request_id uuid, p_customer_name text,
  p_phone text, p_email text, p_moto_brand text, p_moto_model text,
  p_moto_variant text, p_moto_year integer, p_plate_number text,
  p_appointment_date date, p_start_time time, p_notes text,
  p_total_estimate numeric, p_booking_duration_minutes integer,
  p_assigned_crew_id uuid, p_services jsonb, p_continuation_segments jsonb,
  p_notification_title text, p_notification_message text, p_first_name text,
  p_middle_name text, p_last_name text, p_wants_products boolean
)
RETURNS TABLE (appointment_id uuid, reference_code text)
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_appointment_id uuid;
  v_reference_code text;
BEGIN
  IF p_wants_products IS NULL THEN
    RAISE EXCEPTION 'Choose whether you would like to avail shop products.';
  END IF;

  -- Preserve the original preference on a retry. The browser retries with
  -- the same request id when a response is lost, so serializing that id keeps
  -- the preference just as immutable as the appointment details.
  PERFORM pg_advisory_xact_lock(hashtext('booking-request:' ||
p_booking_request_id::text));
  SELECT appointment.id, appointment.reference_code
    INTO v_appointment_id, v_reference_code
  FROM public.appointments AS appointment
  WHERE appointment.booking_request_id = p_booking_request_id;
  IF FOUND THEN
    RETURN QUERY SELECT v_appointment_id, v_reference_code;
    RETURN;
  END IF;

  SELECT created.appointment_id, created.reference_code
    INTO v_appointment_id, v_reference_code
  FROM public.create_booking_atomic(
    p_reference_code, p_booking_request_id, p_customer_name, p_phone,
```

```
p_email,
    p_moto_brand, p_moto_model, p_moto_variant, p_moto_year,
p_plate_number,
    p_appointment_date, p_start_time, p_notes, p_total_estimate,
    p_booking_duration_minutes, p_assigned_crew_id, p_services,
    p_continuation_segments, p_notification_title, p_notification_message,
    p_first_name, p_middle_name, p_last_name
  ) AS created;

  UPDATE public.appointments
  SET wants_products = p_wants_products
  WHERE id = v_appointment_id;

  RETURN QUERY SELECT v_appointment_id, v_reference_code;
END;
$$;

REVOKE ALL ON FUNCTION public.create_booking_atomic(
  text, uuid, text, text, text, text, text, text, integer, text, date, time,
  text, numeric, integer, uuid, jsonb, jsonb, text, text, text, text, text, boolean
) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.create_booking_atomic(
  text, uuid, text, text, text, text, text, text, integer, text, date, time,
  text, numeric, integer, uuid, jsonb, jsonb, text, text, text, text, text, boolean
) TO service_role;

-- This admin-only transaction changes the service snapshot, appointment data,
-- product lines, and stock quantities together. It prevents stale stock from
-- being oversold and preserves the catalog name/price snapshot already used.
CREATE FUNCTION public.update_appointment_financial_details_atomic(
  p_appointment_id uuid,
  p_service_ids uuid[],
  p_appointment_date date,
  p_start_time time,
  p_assigned_crew_id uuid,
  p_crew_assignment_manual boolean,
  p_status text,
  p_admin_notes text,
  p_first_name text,
  p_middle_name text,
  p_last_name text,
  p_phone text,
  p_products jsonb
)
RETURNS void
```

```
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public, auth
AS $$
DECLARE
  v_wants_products boolean;
  v_requested_count integer;
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Only administrators can update appointment products.';
  END IF;
  IF jsonb_typeof(COALESCE(p_products, '[]'::jsonb)) <> 'array'
     OR jsonb_array_length(COALESCE(p_products, '[]'::jsonb)) > 50 THEN
    RAISE EXCEPTION 'Products must be a list of up to 50 items.';
  END IF;

  -- Reuse the single source of truth for service pricing, appointment editing,
  -- crew availability, and status validation.
  PERFORM public.update_appointment_details_atomic(
    p_appointment_id, p_service_ids, p_appointment_date, p_start_time,
    p_assigned_crew_id, p_crew_assignment_manual, p_status, p_admin_notes,
    p_first_name, p_middle_name, p_last_name, p_phone
  );

  SELECT wants_products
    INTO v_wants_products
  FROM public.appointments
  WHERE id = p_appointment_id
    AND is_archived = false
  FOR UPDATE;
  IF NOT FOUND THEN
    RAISE EXCEPTION 'This appointment is no longer available.';
  END IF;

  WITH requested AS (
    SELECT product_id, quantity, unit_price
    FROM jsonb_to_recordset(COALESCE(p_products, '[]'::jsonb))
      AS item(product_id uuid, quantity integer, unit_price numeric)
  )
  SELECT count(*) INTO v_requested_count FROM requested;

  IF v_requested_count > 0 AND NOT v_wants_products THEN
    RAISE EXCEPTION 'This customer declined shop products for this
appointment.';
  END IF;
```

```
  IF v_requested_count > 0 AND p_status NOT IN ('in_progress', 'completed')
THEN
    RAISE EXCEPTION 'Products can only be added while service is in progress
or completed.';
  END IF;

  IF EXISTS (
    WITH requested AS (
      SELECT product_id, quantity, unit_price
      FROM jsonb_to_recordset(COALESCE(p_products, '[]'::jsonb))
        AS item(product_id uuid, quantity integer, unit_price numeric)
    )
    SELECT 1
    FROM requested
    GROUP BY product_id
    HAVING count(*) > 1
       OR min(quantity) IS NULL
       OR min(quantity) < 1
       OR max(quantity) > 999
       OR min(unit_price) IS NULL
       OR min(unit_price) < 0
  ) THEN
    RAISE EXCEPTION 'Each product must be unique and have a valid quantity
and price.';
  END IF;

  -- Lock both old and newly requested inventory rows before checking stock or
  -- calculating differences. This serializes concurrent appointment edits.
  PERFORM 1
  FROM public.products AS product
  WHERE product.id IN (
    SELECT item.product_id
    FROM jsonb_to_recordset(COALESCE(p_products, '[]'::jsonb))
      AS item(product_id uuid, quantity integer, unit_price numeric)
    UNION
    SELECT appointment_product.product_id
    FROM public.appointment_products AS appointment_product
    WHERE appointment_product.appointment_id = p_appointment_id
  )
  ORDER BY product.id
  FOR UPDATE;

  IF EXISTS (
    WITH requested AS (
      SELECT product_id, quantity, unit_price
```

```
      FROM jsonb_to_recordset(COALESCE(p_products, '[]'::jsonb))
        AS item(product_id uuid, quantity integer, unit_price numeric)
    )
    SELECT 1
    FROM requested
    LEFT JOIN public.products AS product ON product.id = requested.product_id
    LEFT JOIN public.appointment_products AS existing
      ON existing.appointment_id = p_appointment_id
     AND existing.product_id = requested.product_id
    WHERE product.id IS NULL
       OR (existing.id IS NULL AND (product.is_archived OR NOT
product.is_active OR NOT product.in_stock))
       OR product.stock_quantity + COALESCE(existing.quantity, 0) <
requested.quantity
  ) THEN
    RAISE EXCEPTION 'One or more products are unavailable or do not have
enough stock.';
  END IF;

  -- Return stock for removed lines first, then reserve the requested quantity.
  UPDATE public.products AS product
  SET stock_quantity = product.stock_quantity + existing.quantity
  FROM public.appointment_products AS existing
  WHERE existing.appointment_id = p_appointment_id
    AND existing.product_id = product.id
    AND NOT EXISTS (
      SELECT 1
      FROM jsonb_to_recordset(COALESCE(p_products, '[]'::jsonb))
        AS item(product_id uuid, quantity integer, unit_price numeric)
      WHERE item.product_id = existing.product_id
    );

  UPDATE public.products AS product
  SET stock_quantity = product.stock_quantity + COALESCE(existing.quantity, 0)
- requested.quantity
  FROM jsonb_to_recordset(COALESCE(p_products, '[]'::jsonb))
    AS requested(product_id uuid, quantity integer, unit_price numeric)
  LEFT JOIN public.appointment_products AS existing
    ON existing.appointment_id = p_appointment_id
   AND existing.product_id = requested.product_id
  WHERE product.id = requested.product_id;

  INSERT INTO public.appointment_products (
    appointment_id, product_id, product_name, quantity, unit_price
  )
```

```
  SELECT p_appointment_id, requested.product_id, product.name,
requested.quantity, requested.unit_price
  FROM jsonb_to_recordset(COALESCE(p_products, '[]'::jsonb))
    AS requested(product_id uuid, quantity integer, unit_price numeric)
  JOIN public.products AS product ON product.id = requested.product_id
  ON CONFLICT (appointment_id, product_id) DO UPDATE
    SET quantity = EXCLUDED.quantity,
        unit_price = EXCLUDED.unit_price,
        updated_at = now();

  DELETE FROM public.appointment_products AS existing
  WHERE existing.appointment_id = p_appointment_id
    AND NOT EXISTS (
      SELECT 1
      FROM jsonb_to_recordset(COALESCE(p_products, '[]'::jsonb))
        AS item(product_id uuid, quantity integer, unit_price numeric)
      WHERE item.product_id = existing.product_id
    );
END;
$$;

REVOKE ALL ON FUNCTION
public.update_appointment_financial_details_atomic(
  uuid, uuid[], date, time, uuid, boolean, text, text, text, text, text, text, jsonb
) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION
public.update_appointment_financial_details_atomic(
  uuid, uuid[], date, time, uuid, boolean, text, text, text, text, text, text, jsonb
) TO authenticated;

-- The financial report includes saved service and product subtotals. The
-- `services` branch remains service-focused for compatibility with existing
-- filtering, while the booking branch is the financial summary shown in UI.
CREATE OR REPLACE FUNCTION public.get_admin_report_page(
  p_report_kind text,
  p_from date,
  p_to date,
  p_status text DEFAULT NULL,
  p_category text DEFAULT NULL,
  p_service_name text DEFAULT NULL,
  p_limit integer DEFAULT 25,
  p_offset integer DEFAULT 0
)
RETURNS jsonb
LANGUAGE plpgsql
```

```
STABLE
SECURITY INVOKER
SET search_path = public, auth
AS $$
DECLARE
  v_limit integer := LEAST(GREATEST(COALESCE(p_limit, 25), 1), 250);
  v_offset integer := GREATEST(COALESCE(p_offset, 0), 0);
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;
  IF p_report_kind NOT IN ('bookings', 'services') THEN
    RAISE EXCEPTION 'Unknown report type';
  END IF;
  IF p_from IS NULL OR p_to IS NULL OR p_from > p_to THEN
    RAISE EXCEPTION 'A valid report date range is required';
  END IF;

  IF p_report_kind = 'bookings' THEN
    RETURN (
      WITH filtered AS (
        SELECT
          appointment.id,
          appointment.reference_code,
          appointment.customer_name,
          appointment.appointment_date,
          appointment.status,
          service_totals.service,
          service_totals.service_subtotal,
          product_totals.products,
          product_totals.product_subtotal,
          service_totals.service_subtotal + product_totals.product_subtotal AS
total_amount,
          service_totals.service_subtotal AS total_estimate
        FROM public.appointments AS appointment
        CROSS JOIN LATERAL (
          SELECT
            COALESCE(string_agg(service.service_name, ', ' ORDER BY
service.service_name), '—') AS service,
            COALESCE(sum(service.price), 0)::numeric AS service_subtotal
          FROM public.appointment_services AS service
          WHERE service.appointment_id = appointment.id
        ) AS service_totals
        CROSS JOIN LATERAL (
          SELECT
```

```
          COALESCE(string_agg(product.product_name || ' × ' || product.quantity, ',
' ORDER BY product.product_name), '—') AS products,
          COALESCE(sum(product.quantity * product.unit_price), 0)::numeric AS
product_subtotal
        FROM public.appointment_products AS product
        WHERE product.appointment_id = appointment.id
      ) AS product_totals
      WHERE appointment.is_archived = false
        AND appointment.appointment_date BETWEEN p_from AND p_to
        AND (p_status IS NULL OR appointment.status = p_status)
        AND (
          (p_category IS NULL AND p_service_name IS NULL)
          OR EXISTS (
            SELECT 1
            FROM public.appointment_services AS selected_service
            LEFT JOIN public.services AS catalog_service ON catalog_service.id =
selected_service.service_id
            WHERE selected_service.appointment_id = appointment.id
              AND (p_category IS NULL OR COALESCE(catalog_service.category,
'Uncategorized') = p_category)
              AND (p_service_name IS NULL OR selected_service.service_name =
p_service_name)
          )
        )
    ),
    grouped_status AS (
      SELECT status, count(*)::integer AS total FROM filtered GROUP BY status
    ),
    paged AS (
      SELECT * FROM filtered
      ORDER BY appointment_date DESC, reference_code ASC
      LIMIT v_limit OFFSET v_offset
    )
    SELECT jsonb_build_object(
      'total', (SELECT count(*) FROM filtered),
      'rows', COALESCE((SELECT jsonb_agg(to_jsonb(paged)) FROM paged),
'[]'::jsonb),
      'metrics', jsonb_build_object(
        'total_bookings', (SELECT count(*) FROM filtered),
        'completed_bookings', (SELECT count(*) FROM filtered WHERE status =
'completed'),
        'completed_value', (SELECT COALESCE(sum(total_amount) FILTER
(WHERE status = 'completed'), 0) FROM filtered),
        'total_value', (SELECT COALESCE(sum(total_amount), 0) FROM filtered),
        'service_total', (SELECT COALESCE(sum(service_subtotal), 0) FROM
```

```
filtered),
        'product_total', (SELECT COALESCE(sum(product_subtotal), 0) FROM
filtered),
        'status_counts', COALESCE((SELECT jsonb_object_agg(status, total)
FROM grouped_status), '{}'::jsonb)
      )
    )
  );
 END IF;

 RETURN (
  WITH filtered AS (
    SELECT
      appointment_service.appointment_id,
      appointment_service.service_id,
      appointment_service.service_name,
      appointment_service.price,
      appointment.reference_code,
      appointment.customer_name,
      appointment.appointment_date,
      appointment.status,
      COALESCE(catalog_service.category, 'Uncategorized') AS category
    FROM public.appointment_services AS appointment_service
    JOIN public.appointments AS appointment ON appointment.id =
appointment_service.appointment_id
    LEFT JOIN public.services AS catalog_service ON catalog_service.id =
appointment_service.service_id
    WHERE appointment.is_archived = false
      AND appointment.appointment_date BETWEEN p_from AND p_to
      AND (p_status IS NULL OR appointment.status = p_status)
      AND (p_category IS NULL OR COALESCE(catalog_service.category,
'Uncategorized') = p_category)
      AND (p_service_name IS NULL OR appointment_service.service_name =
p_service_name)
  ),
  grouped_status AS (
    SELECT status, count(*)::integer AS total FROM filtered GROUP BY status
  ),
  paged AS (
    SELECT * FROM filtered
    ORDER BY appointment_date DESC, reference_code ASC, service_name
ASC
    LIMIT v_limit OFFSET v_offset
  )
  SELECT jsonb_build_object(
```

```
      'total', (SELECT count(*) FROM filtered),
      'rows', COALESCE((SELECT jsonb_agg(to_jsonb(paged)) FROM paged),
'[]'::jsonb),
      'metrics', jsonb_build_object(
        'total_bookings', (SELECT count(DISTINCT appointment_id) FROM
filtered),
        'completed_rows', (SELECT count(*) FROM filtered WHERE status =
'completed'),
        'completed_value', (SELECT COALESCE(sum(price) FILTER (WHERE
status = 'completed'), 0) FROM filtered),
        'total_value', (SELECT COALESCE(sum(price), 0) FROM filtered),
        'status_counts', COALESCE((SELECT jsonb_object_agg(status, total)
FROM grouped_status), '{}'::jsonb)
      )
    )
  );
END;
$$;

REVOKE ALL ON FUNCTION public.get_admin_report_page(text, date, date,
text, text, text, integer, integer) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.get_admin_report_page(text, date,
date, text, text, text, integer, integer) TO authenticated;

NOTIFY pgrst, 'reload schema';
```

supabase/migrations/20261008100000_harden_appointment_crew_availability.sql

---

```
-- The booking page preflights availability for a fast response. This final
-- database guard keeps an appointment correct if two requests race each other
-- or a schedule is changed between the preflight and the insert.
CREATE OR REPLACE FUNCTION
public.enforce_active_appointment_capacity()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = public
AS $$
DECLARE
  v_capacity integer;
  v_overlapping_reservations integer;
  v_start_minutes integer;
  v_opening_minutes integer;
  v_closing_minutes integer;
  v_opening_time time;
  v_closing_time time;
  v_hour_start time;
BEGIN
  -- Archived, replaced, and terminal appointments release both their slot
  -- capacity and their crew assignment.
  IF NEW.is_archived = true
     OR NEW.rescheduled_to_appointment_id IS NOT NULL
     OR NEW.status NOT IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
THEN
    RETURN NEW;
  END IF;

  SELECT opening_time, closing_time
    INTO v_opening_time, v_closing_time
  FROM public.shop_settings
  WHERE id = true;
  v_opening_time := COALESCE(v_opening_time, '08:00:00'::time);
  v_closing_time := COALESCE(v_closing_time, '17:00:00'::time);
  v_opening_minutes := extract(hour FROM v_opening_time)::integer * 60
    + extract(minute FROM v_opening_time)::integer;
  v_closing_minutes := extract(hour FROM v_closing_time)::integer * 60
    + extract(minute FROM v_closing_time)::integer;
  v_start_minutes := extract(hour FROM NEW.start_time)::integer * 60
    + extract(minute FROM NEW.start_time)::integer;

  IF extract(dow FROM NEW.appointment_date) = 0
```

```
    OR NEW.booking_duration_minutes IS NULL
    OR NEW.booking_duration_minutes <= 0
    OR v_start_minutes < v_opening_minutes
    OR v_start_minutes >= v_closing_minutes
    OR mod(v_start_minutes - v_opening_minutes, 30) <> 0
    OR v_start_minutes + NEW.booking_duration_minutes > v_closing_minutes
THEN
   RAISE EXCEPTION 'The appointment must fit within the configured shop
hours on a Monday through Saturday.'
     USING ERRCODE = '23P01';
  END IF;

  -- Serialize every active reservation for one date. The lock is deliberately
  -- acquired before both capacity and crew-overlap checks, so a transaction
  -- that waited on it sees a previously committed reservation.
  PERFORM pg_advisory_xact_lock(hashtext('booking-capacity:' ||
NEW.appointment_date::text));

  v_hour_start := make_time(extract(hour FROM NEW.start_time)::integer, 0, 0);
  SELECT slot.capacity
    INTO v_capacity
  FROM public.time_slots AS slot
  WHERE slot.is_active = true
    AND (slot.start_time = NEW.start_time OR slot.start_time = v_hour_start)
  ORDER BY (slot.start_time = NEW.start_time) DESC
  LIMIT 1
  FOR UPDATE;

  IF COALESCE(v_capacity, 0) <= 0 THEN
    RAISE EXCEPTION 'The requested time slot is not available.' USING
ERRCODE = '23P01';
  END IF;

  IF EXISTS (
    SELECT 1
    FROM public.schedule_blocks AS block
    WHERE block.block_date = NEW.appointment_date
      AND block.is_active = true
      AND (
        block.start_time IS NULL
        OR (
          COALESCE(block.reason, '') !~ '^RANGE:[0-9]{2}:[0-9]{2}'
          AND block.start_time = NEW.start_time
        )
        OR (
```

```
        COALESCE(block.reason, '') ~ '^RANGE:[0-9]{2}:[0-9]{2}'
        AND tsrange(
          NEW.appointment_date + NEW.start_time,
          NEW.appointment_date + NEW.start_time
            + make_interval(mins => NEW.booking_duration_minutes),
          '[)'
        ) && tsrange(
          NEW.appointment_date + block.start_time,
          NEW.appointment_date
            + substring(block.reason FROM '^RANGE:([0-9]{2}:[0-9]{2})')::time,
          '[)'
        )
      )
    )
  ) THEN
    RAISE EXCEPTION 'The shop is closed for the requested schedule.' USING
ERRCODE = '23P01';
  END IF;

  IF NEW.assigned_crew_id IS NOT NULL THEN
    IF NOT EXISTS (
      SELECT 1
      FROM public.crew_members AS crew
      JOIN public.crew_schedules AS schedule ON schedule.crew_id = crew.id
      WHERE crew.id = NEW.assigned_crew_id
        AND crew.is_active = true
        AND crew.is_archived = false
        AND schedule.schedule_date = NEW.appointment_date
        AND schedule.is_working = true
        AND schedule.start_time IS NOT NULL
        AND schedule.end_time IS NOT NULL
        AND schedule.start_time <= NEW.start_time
        AND schedule.end_time >= NEW.start_time
          + make_interval(mins => NEW.booking_duration_minutes)
    ) THEN
      RAISE EXCEPTION 'The assigned crew member is not scheduled for the
requested appointment period.'
        USING ERRCODE = '23P01';
    END IF;

    IF EXISTS (
      SELECT 1
      FROM public.crew_availability_exceptions AS exception
      WHERE exception.crew_id = NEW.assigned_crew_id
        AND exception.start_date <= NEW.appointment_date
```

```
        AND exception.end_date >= NEW.appointment_date
        AND (
          exception.is_all_day
          OR (
            exception.start_time IS NOT NULL
            AND exception.end_time IS NOT NULL
            AND tsrange(
              NEW.appointment_date + NEW.start_time,
              NEW.appointment_date + NEW.start_time
                + make_interval(mins => NEW.booking_duration_minutes),
              '[)'
            ) && tsrange(
              NEW.appointment_date + exception.start_time,
              NEW.appointment_date + exception.end_time,
              '[)'
            )
          )
        )
    ) THEN
      RAISE EXCEPTION 'The assigned crew member is unavailable for the
requested appointment period.'
        USING ERRCODE = '23P01';
    END IF;
  END IF;

  SELECT count(*)
    INTO v_overlapping_reservations
  FROM (
    SELECT appointment.appointment_date, appointment.start_time,
appointment.booking_duration_minutes
    FROM public.appointments AS appointment
    WHERE appointment.id <> NEW.id
      AND appointment.appointment_date = NEW.appointment_date
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
    UNION ALL
    SELECT continuation.appointment_date, continuation.start_time,
continuation.booking_duration_minutes
    FROM public.appointment_continuations AS continuation
    JOIN public.appointments AS appointment ON appointment.id =
continuation.appointment_id
    WHERE continuation.appointment_date = NEW.appointment_date
      AND appointment.is_archived = false
```

```
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
  ) AS reservation
  WHERE tsrange(
    reservation.appointment_date + reservation.start_time,
    reservation.appointment_date + reservation.start_time
      + make_interval(mins => reservation.booking_duration_minutes),
    '[)'
  ) && tsrange(
    NEW.appointment_date + NEW.start_time,
    NEW.appointment_date + NEW.start_time + make_interval(mins =>
NEW.booking_duration_minutes),
    '[)'
  );

  IF v_overlapping_reservations >= v_capacity THEN
    RAISE EXCEPTION 'The requested schedule is no longer available.' USING
ERRCODE = '23P01';
  END IF;

  -- Capacity alone is not enough: two slots can overlap while there is still
  -- aggregate capacity for another crew. A crew ending at exactly the new
  -- start time is allowed because all reservation ranges are [start, end).
  IF NEW.assigned_crew_id IS NOT NULL AND EXISTS (
    SELECT 1
    FROM (
      SELECT appointment.assigned_crew_id, appointment.appointment_date,
        appointment.start_time, appointment.booking_duration_minutes
      FROM public.appointments AS appointment
      WHERE appointment.id <> NEW.id
        AND appointment.assigned_crew_id = NEW.assigned_crew_id
        AND appointment.appointment_date = NEW.appointment_date
        AND appointment.is_archived = false
        AND appointment.rescheduled_to_appointment_id IS NULL
        AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
      UNION ALL
      SELECT continuation.assigned_crew_id, continuation.appointment_date,
        continuation.start_time, continuation.booking_duration_minutes
      FROM public.appointment_continuations AS continuation
      JOIN public.appointments AS appointment ON appointment.id =
continuation.appointment_id
      WHERE continuation.assigned_crew_id = NEW.assigned_crew_id
        AND continuation.appointment_date = NEW.appointment_date
```

```
        AND appointment.is_archived = false
        AND appointment.rescheduled_to_appointment_id IS NULL
        AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
    ) AS crew_reservation
    WHERE tsrange(
      crew_reservation.appointment_date + crew_reservation.start_time,
      crew_reservation.appointment_date + crew_reservation.start_time
        + make_interval(mins => crew_reservation.booking_duration_minutes),
      '[)'
    ) && tsrange(
      NEW.appointment_date + NEW.start_time,
      NEW.appointment_date + NEW.start_time + make_interval(mins =>
NEW.booking_duration_minutes),
      '[)'
    )
  ) THEN
    RAISE EXCEPTION 'The assigned crew member is no longer available for the
requested appointment period.'
      USING ERRCODE = '23P01';
  END IF;

  RETURN NEW;
END;
$$;

-- A crew-only reassignment must run through the same guard. Earlier trigger
-- definitions omitted assigned_crew_id from UPDATE OF, which allowed an
-- otherwise-valid appointment to be moved onto a busy or unscheduled crew.
DROP TRIGGER IF EXISTS appointments_enforce_active_capacity ON
public.appointments;
CREATE TRIGGER appointments_enforce_active_capacity
  BEFORE INSERT OR UPDATE OF
    appointment_date,
    start_time,
    booking_duration_minutes,
    assigned_crew_id,
    status,
    is_archived,
    rescheduled_to_appointment_id
  ON public.appointments
  FOR EACH ROW
  EXECUTE FUNCTION public.enforce_active_appointment_capacity();
```

supabase/migrations/20261008110000_add_appointment_confirmation_sms.sql

---

```
-- SMS notifications are queued only after an appointment has actually been
-- confirmed. The server claims and delivers them with the TextBee API key;
-- browser clients cannot read or modify this queue.
CREATE TABLE IF NOT EXISTS public.appointment_confirmation_sms (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  appointment_id uuid NOT NULL UNIQUE REFERENCES
public.appointments(id) ON DELETE CASCADE,
  recipient_phone text NOT NULL,
  status text NOT NULL DEFAULT 'pending'
    CHECK (status IN ('pending', 'sending', 'sent')),
  attempt_count integer NOT NULL DEFAULT 0 CHECK (attempt_count >= 0),
  last_error text,
  sent_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);

REVOKE ALL ON TABLE public.appointment_confirmation_sms FROM anon,
authenticated;
GRANT ALL ON TABLE public.appointment_confirmation_sms TO service_role;
ALTER TABLE public.appointment_confirmation_sms ENABLE ROW LEVEL
SECURITY;

CREATE OR REPLACE FUNCTION
public.queue_appointment_confirmation_sms()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
BEGIN
  -- A pending or rejected booking must never notify the customer. For a
  -- reschedule, the replacement appointment is inserted as confirmed, which
  -- queues its own notification after the approval transaction succeeds.
  IF NEW.status <> 'confirmed' OR NULLIF(btrim(NEW.phone), '') IS NULL THEN
    RETURN NEW;
  END IF;
  IF TG_OP = 'UPDATE' AND OLD.status IS NOT DISTINCT FROM 'confirmed'
THEN
    RETURN NEW;
  END IF;
```

```
  INSERT INTO public.appointment_confirmation_sms (appointment_id,
recipient_phone)
  VALUES (NEW.id, btrim(NEW.phone))
  ON CONFLICT (appointment_id) DO NOTHING;
  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS appointments_queue_confirmation_sms ON
public.appointments;
DROP TRIGGER IF EXISTS appointments_queue_confirmation_sms_on_insert
ON public.appointments;
CREATE TRIGGER appointments_queue_confirmation_sms
  AFTER UPDATE OF status ON public.appointments
  FOR EACH ROW
  EXECUTE FUNCTION public.queue_appointment_confirmation_sms();

CREATE TRIGGER appointments_queue_confirmation_sms_on_insert
  AFTER INSERT ON public.appointments
  FOR EACH ROW
  EXECUTE FUNCTION public.queue_appointment_confirmation_sms();
```

supabase/migrations/20261008120000_add_service_completion_customer_notifications.sql

---

```
-- Completion notifications are separate from booking-confirmation notices so
-- each channel is delivered at most once after a service is actually completed.
CREATE TABLE IF NOT EXISTS public.appointment_completion_notifications (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  appointment_id uuid NOT NULL REFERENCES public.appointments(id) ON
DELETE CASCADE,
  channel text NOT NULL CHECK (channel IN ('email', 'sms')),
  recipient text NOT NULL,
  status text NOT NULL DEFAULT 'pending'
    CHECK (status IN ('pending', 'sending', 'sent')),
  attempt_count integer NOT NULL DEFAULT 0 CHECK (attempt_count >= 0),
  last_error text,
  sent_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (appointment_id, channel)
);

REVOKE ALL ON TABLE public.appointment_completion_notifications FROM
anon, authenticated;
GRANT ALL ON TABLE public.appointment_completion_notifications TO
service_role;
ALTER TABLE public.appointment_completion_notifications ENABLE ROW
LEVEL SECURITY;

-- Completion is valid only from an active service. This protects the editor
-- and API paths alike, and records a durable completion timestamp for notices.
CREATE OR REPLACE FUNCTION
public.validate_appointment_completion_transition()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = public
AS $$
BEGIN
  IF NEW.status = 'completed' AND OLD.status IS DISTINCT FROM 'completed'
THEN
    IF OLD.status <> 'in_progress' OR OLD.service_started_at IS NULL OR
OLD.service_ended_at IS NOT NULL THEN
      RAISE EXCEPTION 'Start the service before marking the appointment
completed.';
    END IF;
    NEW.service_ended_at := COALESCE(NEW.service_ended_at, now());
```

```
  END IF;
  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS appointments_validate_completion_transition ON
public.appointments;
CREATE TRIGGER appointments_validate_completion_transition
  BEFORE UPDATE OF status ON public.appointments
  FOR EACH ROW
  EXECUTE FUNCTION public.validate_appointment_completion_transition();

CREATE OR REPLACE FUNCTION
public.queue_appointment_completion_notifications()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
BEGIN
  IF NEW.status <> 'completed' OR OLD.status IS NOT DISTINCT FROM
'completed' THEN
    RETURN NEW;
  END IF;

  IF NULLIF(btrim(NEW.email), '') IS NOT NULL THEN
    INSERT INTO public.appointment_completion_notifications (appointment_id,
channel, recipient)
    VALUES (NEW.id, 'email', lower(btrim(NEW.email)))
    ON CONFLICT (appointment_id, channel) DO NOTHING;
  END IF;

  IF NULLIF(btrim(NEW.phone), '') IS NOT NULL THEN
    INSERT INTO public.appointment_completion_notifications (appointment_id,
channel, recipient)
    VALUES (NEW.id, 'sms', btrim(NEW.phone))
    ON CONFLICT (appointment_id, channel) DO NOTHING;
  END IF;
  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS appointments_queue_completion_notifications ON
public.appointments;
CREATE TRIGGER appointments_queue_completion_notifications
```

```
AFTER UPDATE OF status ON public.appointments
FOR EACH ROW
EXECUTE FUNCTION public.queue_appointment_completion_notifications();
```

supabase/migrations/20261008130000_limit_textbee_sms_to_demo_phone.sql

---

```
-- TextBee trial delivery is intentionally restricted to the approved demo
-- recipient. Other customer numbers still receive email notifications and
-- have their SMS row recorded as skipped without an error or API request.
ALTER TABLE public.appointment_confirmation_sms
  DROP CONSTRAINT IF EXISTS
appointment_confirmation_sms_status_check;
ALTER TABLE public.appointment_confirmation_sms
  ADD CONSTRAINT appointment_confirmation_sms_status_check
  CHECK (status IN ('pending', 'sending', 'sent', 'skipped'));

ALTER TABLE public.appointment_completion_notifications
  DROP CONSTRAINT IF EXISTS
appointment_completion_notifications_status_check;
ALTER TABLE public.appointment_completion_notifications
  ADD CONSTRAINT appointment_completion_notifications_status_check
  CHECK (status IN ('pending', 'sending', 'sent', 'skipped'));
```

supabase/migrations/20261008140000_add_completed_financial_report.sql

---

```
-- Financial reporting recognizes revenue only when a service is completed.
-- Service and product values use the appointment snapshots, never current
-- catalog prices, so historical totals remain accurate after price changes.
CREATE FUNCTION public.get_completed_financial_report_page(
  p_from date,
  p_to date,
  p_category text DEFAULT NULL,
  p_service_name text DEFAULT NULL,
  p_limit integer DEFAULT 25,
  p_offset integer DEFAULT 0
)
RETURNS jsonb
LANGUAGE plpgsql
STABLE
SECURITY INVOKER
SET search_path = public, auth
AS $$
DECLARE
  v_limit integer := LEAST(GREATEST(COALESCE(p_limit, 25), 1), 250);
  v_offset integer := GREATEST(COALESCE(p_offset, 0), 0);
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;
  IF p_from IS NULL OR p_to IS NULL OR p_from > p_to THEN
    RAISE EXCEPTION 'A valid report date range is required';
  END IF;

  RETURN (
    WITH completed_appointments AS (
      SELECT
        appointment.id,
        appointment.reference_code,
        COALESCE(
          (appointment.service_ended_at AT TIME ZONE 'Asia/Manila')::date,
          appointment.appointment_date
        ) AS transaction_date
      FROM public.appointments AS appointment
      WHERE appointment.is_archived = false
        AND appointment.status = 'completed'
    ),
    filtered AS (
      SELECT
```

```
        appointment.id,
        appointment.reference_code,
        appointment.transaction_date,
        service_totals.services,
        service_totals.service_amount,
        product_totals.products,
        product_totals.product_amount,
        service_totals.service_amount + product_totals.product_amount AS
total_amount
      FROM completed_appointments AS appointment
      CROSS JOIN LATERAL (
        SELECT
          COALESCE(string_agg(service.service_name, ', ' ORDER BY
service.service_name), '-') AS services,
          COALESCE(sum(service.price), 0)::numeric AS service_amount
        FROM public.appointment_services AS service
        WHERE service.appointment_id = appointment.id
      ) AS service_totals
      CROSS JOIN LATERAL (
        SELECT
          COALESCE(
            string_agg(product.product_name || ' x ' || product.quantity, ', ' ORDER
BY product.product_name),
            '-'
          ) AS products,
          COALESCE(sum(product.quantity * product.unit_price), 0)::numeric AS
product_amount
        FROM public.appointment_products AS product
        WHERE product.appointment_id = appointment.id
      ) AS product_totals
      WHERE appointment.transaction_date BETWEEN p_from AND p_to
        AND (
          (p_category IS NULL AND p_service_name IS NULL)
          OR EXISTS (
            SELECT 1
            FROM public.appointment_services AS selected_service
            LEFT JOIN public.services AS catalog_service ON catalog_service.id =
selected_service.service_id
            WHERE selected_service.appointment_id = appointment.id
              AND (p_category IS NULL OR COALESCE(catalog_service.category,
'Uncategorized') = p_category)
              AND (p_service_name IS NULL OR selected_service.service_name =
p_service_name)
          )
        )
```

```
    ),
    scoped_services AS (
      SELECT
        service.appointment_id,
        service.service_name,
        service.price,
        COALESCE(catalog_service.category, 'Uncategorized') AS category
      FROM public.appointment_services AS service
      JOIN filtered AS appointment ON appointment.id = service.appointment_id
      LEFT JOIN public.services AS catalog_service ON catalog_service.id =
service.service_id
    ),
    scoped_products AS (
      SELECT product.appointment_id, product.product_name, product.quantity,
product.unit_price
      FROM public.appointment_products AS product
      JOIN filtered AS appointment ON appointment.id = product.appointment_id
    ),
    paged AS (
      SELECT *
      FROM filtered
      ORDER BY transaction_date DESC, reference_code ASC
      LIMIT v_limit OFFSET v_offset
    )
    SELECT jsonb_build_object(
      'total', (SELECT count(*) FROM filtered),
      'rows', COALESCE((SELECT jsonb_agg(to_jsonb(paged)) FROM paged),
'[]'::jsonb),
      'metrics', jsonb_build_object(
        'total_revenue', (SELECT COALESCE(sum(total_amount), 0) FROM
filtered),
        'service_revenue', (SELECT COALESCE(sum(service_amount), 0) FROM
filtered),
        'product_revenue', (SELECT COALESCE(sum(product_amount), 0) FROM
filtered),
        'completed_transactions', (SELECT count(*) FROM filtered),
        'total_completed_services', (SELECT count(*) FROM scoped_services),
        'total_products_sold', (SELECT COALESCE(sum(quantity), 0) FROM
scoped_products),
        'average_transaction_amount', (
          SELECT COALESCE(avg(total_amount), 0) FROM filtered
        )
      ),
      'service_performance', COALESCE((
        SELECT jsonb_agg(to_jsonb(performance) ORDER BY
```

```
performance.revenue DESC, performance.service_name)
        FROM (
          SELECT
            service_name,
            count(*)::integer AS completed_count,
            COALESCE(sum(price), 0)::numeric AS revenue
          FROM scoped_services
          GROUP BY service_name
        ) AS performance
      ), '[]'::jsonb),
      'product_sales', COALESCE((
        SELECT jsonb_agg(to_jsonb(sales) ORDER BY sales.total_sales DESC,
sales.product_name, sales.unit_price)
        FROM (
          SELECT
            product_name,
            sum(quantity)::integer AS quantity_sold,
            unit_price,
            COALESCE(sum(quantity * unit_price), 0)::numeric AS total_sales
          FROM scoped_products
          GROUP BY product_name, unit_price
        ) AS sales
      ), '[]'::jsonb),
      'daily_revenue', COALESCE((
        SELECT jsonb_agg(to_jsonb(day_revenue) ORDER BY
day_revenue.period)
        FROM (
          SELECT
            transaction_date::text AS period,
            count(*)::integer AS completed_transactions,
            COALESCE(sum(service_amount), 0)::numeric AS service_revenue,
            COALESCE(sum(product_amount), 0)::numeric AS product_revenue,
            COALESCE(sum(total_amount), 0)::numeric AS total_revenue
          FROM filtered
          GROUP BY transaction_date
        ) AS day_revenue
      ), '[]'::jsonb),
      'monthly_revenue', COALESCE((
        SELECT jsonb_agg(to_jsonb(month_revenue) ORDER BY
month_revenue.period)
        FROM (
          SELECT
            date_trunc('month', transaction_date::timestamp)::date::text AS period,
            count(*)::integer AS completed_transactions,
            COALESCE(sum(service_amount), 0)::numeric AS service_revenue,
```

```
          COALESCE(sum(product_amount), 0)::numeric AS product_revenue,
          COALESCE(sum(total_amount), 0)::numeric AS total_revenue
        FROM filtered
        GROUP BY date_trunc('month', transaction_date::timestamp)::date
      ) AS month_revenue
    ), '[]'::jsonb)
  )
 );
END;
$$;

REVOKE ALL ON FUNCTION public.get_completed_financial_report_page(
  date, date, text, text, integer, integer
) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.get_completed_financial_report_page(
  date, date, text, text, integer, integer
) TO authenticated;

NOTIFY pgrst, 'reload schema';
```

supabase/migrations/20261011140000_allow_products_before_service_start.sql

---

```
-- Product lines may be prepared after approval, before a mechanic starts the
-- service. Keep the same atomic stock checks and price snapshots, while
-- rejecting unapproved and terminal appointments.
CREATE OR REPLACE FUNCTION
public.update_appointment_financial_details_atomic(
  p_appointment_id uuid,
  p_service_ids uuid[],
  p_appointment_date date,
  p_start_time time,
  p_assigned_crew_id uuid,
  p_crew_assignment_manual boolean,
  p_status text,
  p_admin_notes text,
  p_first_name text,
  p_middle_name text,
  p_last_name text,
  p_phone text,
  p_products jsonb
)
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public, auth
AS $$
DECLARE
  v_wants_products boolean;
  v_requested_count integer;
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Only administrators can update appointment products.';
  END IF;
  IF jsonb_typeof(COALESCE(p_products, '[]'::jsonb)) <> 'array'
     OR jsonb_array_length(COALESCE(p_products, '[]'::jsonb)) > 50 THEN
    RAISE EXCEPTION 'Products must be a list of up to 50 items.';
  END IF;

  PERFORM public.update_appointment_details_atomic(
    p_appointment_id, p_service_ids, p_appointment_date, p_start_time,
    p_assigned_crew_id, p_crew_assignment_manual, p_status, p_admin_notes,
    p_first_name, p_middle_name, p_last_name, p_phone
  );
```

```
  SELECT wants_products
    INTO v_wants_products
  FROM public.appointments
  WHERE id = p_appointment_id
    AND is_archived = false
  FOR UPDATE;
  IF NOT FOUND THEN
    RAISE EXCEPTION 'This appointment is no longer available.';
  END IF;

  WITH requested AS (
    SELECT product_id, quantity, unit_price
    FROM jsonb_to_recordset(COALESCE(p_products, '[]'::jsonb))
      AS item(product_id uuid, quantity integer, unit_price numeric)
  )
  SELECT count(*) INTO v_requested_count FROM requested;

  IF v_requested_count > 0 AND NOT v_wants_products THEN
    RAISE EXCEPTION 'This customer declined shop products for this
appointment.';
  END IF;
  IF v_requested_count > 0 AND p_status NOT IN ('confirmed', 'in_progress',
'completed') THEN
    RAISE EXCEPTION 'Products can be added after the appointment is
confirmed, before or during service.';
  END IF;

  IF EXISTS (
    WITH requested AS (
      SELECT product_id, quantity, unit_price
      FROM jsonb_to_recordset(COALESCE(p_products, '[]'::jsonb))
        AS item(product_id uuid, quantity integer, unit_price numeric)
    )
    SELECT 1
    FROM requested
    GROUP BY product_id
    HAVING count(*) > 1
       OR min(quantity) IS NULL
       OR min(quantity) < 1
       OR max(quantity) > 999
       OR min(unit_price) IS NULL
       OR min(unit_price) < 0
  ) THEN
    RAISE EXCEPTION 'Each product must be unique and have a valid quantity
and price.';
```

```
  END IF;

  -- Lock existing and requested inventory in a stable order. This prevents
  -- parallel appointment edits from overselling a product.
  PERFORM 1
  FROM public.products AS product
  WHERE product.id IN (
    SELECT item.product_id
    FROM jsonb_to_recordset(COALESCE(p_products, '[]'::jsonb))
      AS item(product_id uuid, quantity integer, unit_price numeric)
    UNION
    SELECT appointment_product.product_id
    FROM public.appointment_products AS appointment_product
    WHERE appointment_product.appointment_id = p_appointment_id
  )
  ORDER BY product.id
  FOR UPDATE;

  IF EXISTS (
    WITH requested AS (
      SELECT product_id, quantity, unit_price
      FROM jsonb_to_recordset(COALESCE(p_products, '[]'::jsonb))
        AS item(product_id uuid, quantity integer, unit_price numeric)
    )
    SELECT 1
    FROM requested
    LEFT JOIN public.products AS product ON product.id = requested.product_id
    LEFT JOIN public.appointment_products AS existing
      ON existing.appointment_id = p_appointment_id
     AND existing.product_id = requested.product_id
    WHERE product.id IS NULL
       OR (existing.id IS NULL AND (product.is_archived OR NOT
product.is_active OR NOT product.in_stock))
       OR product.stock_quantity + COALESCE(existing.quantity, 0) <
requested.quantity
  ) THEN
    RAISE EXCEPTION 'One or more products are unavailable or do not have
enough stock.';
  END IF;

  UPDATE public.products AS product
  SET stock_quantity = product.stock_quantity + existing.quantity
  FROM public.appointment_products AS existing
  WHERE existing.appointment_id = p_appointment_id
    AND existing.product_id = product.id
```

```
    AND NOT EXISTS (
      SELECT 1
      FROM jsonb_to_recordset(COALESCE(p_products, '[]'::jsonb))
        AS item(product_id uuid, quantity integer, unit_price numeric)
      WHERE item.product_id = existing.product_id
    );

  UPDATE public.products AS product
  SET stock_quantity = product.stock_quantity + COALESCE(existing.quantity, 0)
- requested.quantity
  FROM jsonb_to_recordset(COALESCE(p_products, '[]'::jsonb))
    AS requested(product_id uuid, quantity integer, unit_price numeric)
  LEFT JOIN public.appointment_products AS existing
    ON existing.appointment_id = p_appointment_id
   AND existing.product_id = requested.product_id
  WHERE product.id = requested.product_id;

  INSERT INTO public.appointment_products (
    appointment_id, product_id, product_name, quantity, unit_price
  )
  SELECT p_appointment_id, requested.product_id, product.name,
requested.quantity, requested.unit_price
  FROM jsonb_to_recordset(COALESCE(p_products, '[]'::jsonb))
    AS requested(product_id uuid, quantity integer, unit_price numeric)
  JOIN public.products AS product ON product.id = requested.product_id
  ON CONFLICT (appointment_id, product_id) DO UPDATE
    SET quantity = EXCLUDED.quantity,
        unit_price = EXCLUDED.unit_price,
        updated_at = now();

  DELETE FROM public.appointment_products AS existing
  WHERE existing.appointment_id = p_appointment_id
    AND NOT EXISTS (
      SELECT 1
      FROM jsonb_to_recordset(COALESCE(p_products, '[]'::jsonb))
        AS item(product_id uuid, quantity integer, unit_price numeric)
      WHERE item.product_id = existing.product_id
    );
END;
$$;

-- A pre-service product allocation reserves stock. If that appointment ends
-- before service begins, return the reservation and clear its unsold lines.
CREATE OR REPLACE FUNCTION
public.release_pre_service_product_reservations()
```

```
RETURNS trigger
LANGUAGE plpgsql
SET search_path = public
AS $$
BEGIN
  IF OLD.status IN ('pending', 'confirmed')
     AND NEW.status IN ('cancelled', 'rejected', 'rescheduled', 'no_show')
     AND NEW.status IS DISTINCT FROM OLD.status THEN
    PERFORM 1
    FROM public.products AS product
    JOIN public.appointment_products AS appointment_product
      ON appointment_product.product_id = product.id
    WHERE appointment_product.appointment_id = OLD.id
    ORDER BY product.id
    FOR UPDATE;

    UPDATE public.products AS product
    SET stock_quantity = product.stock_quantity + appointment_product.quantity
    FROM public.appointment_products AS appointment_product
    WHERE appointment_product.appointment_id = OLD.id
      AND appointment_product.product_id = product.id;

    DELETE FROM public.appointment_products
    WHERE appointment_id = OLD.id;
  END IF;

  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS
appointments_release_pre_service_product_reservations
  ON public.appointments;

CREATE TRIGGER appointments_release_pre_service_product_reservations
  BEFORE UPDATE OF status ON public.appointments
  FOR EACH ROW
  EXECUTE FUNCTION public.release_pre_service_product_reservations();

REVOKE ALL ON FUNCTION
public.update_appointment_financial_details_atomic(
  uuid, uuid[], date, time, uuid, boolean, text, text, text, text, text, text, jsonb
) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION
public.update_appointment_financial_details_atomic(
```

```
  uuid, uuid[], date, time, uuid, boolean, text, text, text, text, text, text, jsonb
) TO authenticated;

NOTIFY pgrst, 'reload schema';
```

supabase/migrations/20261014100000_add_appointment_action_verification.sql

---

```
-- Cancelling or rescheduling online requires a one-time code emailed to the
-- address on the booking. A correct code issues a short-lived verification
-- token for that one appointment and action. Codes and tokens are stored only
-- as SHA-256 hashes, and the table is reachable only through the service role.
CREATE TABLE IF NOT EXISTS public.appointment_action_verifications (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  appointment_id uuid NOT NULL REFERENCES public.appointments(id) ON
DELETE CASCADE,
  action text NOT NULL CHECK (action IN ('cancel', 'reschedule')),
  code_hash text NOT NULL CHECK (code_hash ~ '^[a-f0-9]{64}$'),
  expires_at timestamptz NOT NULL,
  attempt_count integer NOT NULL DEFAULT 0 CHECK (attempt_count >= 0
AND attempt_count <= 5),
  verified_at timestamptz,
  token_hash text UNIQUE CHECK (token_hash IS NULL OR token_hash ~
'^[a-f0-9]{64}$'),
  token_expires_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now(),
  CHECK ((verified_at IS NULL) = (token_hash IS NULL)),
  CHECK ((token_hash IS NULL) = (token_expires_at IS NULL))
);

CREATE INDEX IF NOT EXISTS appointment_action_verifications_lookup_idx
  ON public.appointment_action_verifications (appointment_id, action, created_at
DESC);

REVOKE ALL ON TABLE public.appointment_action_verifications FROM anon,
authenticated;
GRANT ALL ON TABLE public.appointment_action_verifications TO
service_role;
ALTER TABLE public.appointment_action_verifications ENABLE ROW LEVEL
SECURITY;

CREATE OR REPLACE FUNCTION public.verify_appointment_action_code(
  p_appointment_id uuid,
  p_action text,
  p_code_hash text,
  p_token_hash text,
  p_token_ttl_seconds integer
)
RETURNS text
LANGUAGE plpgsql
```

```
SECURITY INVOKER
SET search_path = public
AS $$
DECLARE
  v_challenge public.appointment_action_verifications%ROWTYPE;
BEGIN
  IF p_token_ttl_seconds < 60 OR p_token_ttl_seconds > 7200 THEN
    RAISE EXCEPTION 'Verification token lifetime must be between 1 minute and
2 hours.';
  END IF;

  -- Only the newest unverified code for this appointment and action counts.
  SELECT challenge.*
    INTO v_challenge
  FROM public.appointment_action_verifications AS challenge
  WHERE challenge.appointment_id = p_appointment_id
    AND challenge.action = p_action
    AND challenge.verified_at IS NULL
  ORDER BY challenge.created_at DESC
  LIMIT 1
  FOR UPDATE;

  IF NOT FOUND OR v_challenge.expires_at <= now() OR
v_challenge.attempt_count >= 5 THEN
    RETURN 'expired';
  END IF;

  IF v_challenge.code_hash <> p_code_hash THEN
    UPDATE public.appointment_action_verifications
    SET attempt_count = attempt_count + 1,
        expires_at = CASE WHEN attempt_count + 1 >= 5 THEN now() ELSE
expires_at END
    WHERE id = v_challenge.id;
    RETURN 'invalid';
  END IF;

  UPDATE public.appointment_action_verifications
  SET verified_at = now(),
      token_hash = p_token_hash,
      token_expires_at = now() + make_interval(secs => p_token_ttl_seconds)
  WHERE id = v_challenge.id;

  RETURN 'verified';
END;
$$;
```

```
REVOKE ALL ON FUNCTION public.verify_appointment_action_code(uuid, text,
text, text, integer)
  FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.verify_appointment_action_code(uuid,
text, text, text, integer)
  TO service_role;

NOTIFY pgrst, 'reload schema';
```

supabase/migrations/20261015100000_protect_completed_appointments_and_lock_down_functions.sql

---

```
-- 1. Functions that bypass row-level security are no longer callable with the
--    public API key.
-- 2. Completed appointments are kept: they cannot be archived or deleted, and
--    the financial report counts every completed appointment.
-- 3. The daily archive and activity-log cleanups are scheduled.

-- Scheduled jobs run as the role that schedules them, and the functions and
-- trigger below are owned by it. Stop before changing anything unless that is
-- postgres (the SQL editor and the Supabase CLI both apply migrations as
postgres).
DO $$
BEGIN
  IF current_user <> 'postgres' THEN
    RAISE EXCEPTION 'Apply this migration as postgres (current role: %).',
current_user;
  END IF;
END;
$$;

-- 1. Function access ------------------------------------------------------

-- Supabase grants EXECUTE on new functions to anon and authenticated
directly,
-- so the earlier REVOKE ... FROM PUBLIC statements left these callable by
anyone
-- holding the public key. No public page calls them: public requests go through
-- server functions that use the service role. Trigger functions are skipped
-- because they cannot be called directly.
DO $$
DECLARE
  v_function regprocedure;
BEGIN
  FOR v_function IN
    SELECT p.oid::regprocedure
    FROM pg_proc AS p
    WHERE p.pronamespace = 'public'::regnamespace
      AND p.prosecdef
      AND p.prorettype <> 'trigger'::regtype
  LOOP
    EXECUTE format('REVOKE EXECUTE ON FUNCTION %s FROM PUBLIC,
anon', v_function);
  END LOOP;
```

```
END;
$$;

-- These run only from triggers, the scheduler, or server code using the
-- service role, so signed-in users do not need them either. The admin-session
-- and activity-log functions stay available to the signed-in admin console.
REVOKE EXECUTE ON FUNCTION
public.refresh_customer_service_booking_alert(text, uuid, text, uuid, text)
  FROM PUBLIC, anon, authenticated;
REVOKE EXECUTE ON FUNCTION public.purge_expired_archived_records()
  FROM PUBLIC, anon, authenticated;
REVOKE EXECUTE ON FUNCTION
public.verify_appointment_action_code(uuid, text, text, text, integer)
  FROM PUBLIC, anon, authenticated;

-- 2. Completed appointments ------------------------------------------------

CREATE OR REPLACE FUNCTION public.protect_completed_appointments()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = public
AS $$
BEGIN
  IF TG_OP = 'DELETE' THEN
    IF OLD.status = 'completed' THEN
      RAISE EXCEPTION 'Completed appointments are kept for financial records
and cannot be deleted.';
    END IF;
    RETURN OLD;
  END IF;

  IF NEW.is_archived
     AND NOT OLD.is_archived
     AND (OLD.status = 'completed' OR NEW.status = 'completed') THEN
    RAISE EXCEPTION 'Completed appointments are kept for financial records
and cannot be archived.';
  END IF;
  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS appointments_protect_completed ON
public.appointments;
CREATE TRIGGER appointments_protect_completed
  BEFORE DELETE OR UPDATE OF is_archived, status ON
```

```
public.appointments
  FOR EACH ROW EXECUTE FUNCTION
public.protect_completed_appointments();

-- The 30-day archive cleanup leaves completed appointments in place, including
-- any archived before archiving them was blocked.
CREATE OR REPLACE FUNCTION public.purge_expired_archived_records()
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
BEGIN
  DELETE FROM public.appointments
  WHERE is_archived = true
    AND status <> 'completed'
    AND archived_at < now() - interval '30 days';

  DELETE FROM public.services
  WHERE is_archived = true
    AND archived_at < now() - interval '30 days';

  DELETE FROM public.products
  WHERE is_archived = true
    AND archived_at < now() - interval '30 days';

  DELETE FROM public.motorcycle_catalog
  WHERE is_archived = true
    AND archived_at < now() - interval '30 days';

  DELETE FROM public.crew_members
  WHERE is_archived = true
    AND archived_at < now() - interval '30 days';

  DELETE FROM public.schedule_blocks
  WHERE is_active = false
    AND archived_at < now() - interval '30 days';

  DELETE FROM public.blocked_numbers
  WHERE is_archived = true
    AND archived_at < now() - interval '30 days';
END;
$$;

-- Revenue is recognized for every completed appointment, archived or not, so
```

```
-- archiving can never change a past period's totals.
CREATE OR REPLACE FUNCTION
public.get_completed_financial_report_page(
  p_from date,
  p_to date,
  p_category text DEFAULT NULL,
  p_service_name text DEFAULT NULL,
  p_limit integer DEFAULT 25,
  p_offset integer DEFAULT 0
)
RETURNS jsonb
LANGUAGE plpgsql
STABLE
SECURITY INVOKER
SET search_path = public, auth
AS $$
DECLARE
  v_limit integer := LEAST(GREATEST(COALESCE(p_limit, 25), 1), 250);
  v_offset integer := GREATEST(COALESCE(p_offset, 0), 0);
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;
  IF p_from IS NULL OR p_to IS NULL OR p_from > p_to THEN
    RAISE EXCEPTION 'A valid report date range is required';
  END IF;

  RETURN (
    WITH completed_appointments AS (
      SELECT
        appointment.id,
        appointment.reference_code,
        COALESCE(
          (appointment.service_ended_at AT TIME ZONE 'Asia/Manila')::date,
          appointment.appointment_date
        ) AS transaction_date
      FROM public.appointments AS appointment
      WHERE appointment.status = 'completed'
    ),
    filtered AS (
      SELECT
        appointment.id,
        appointment.reference_code,
        appointment.transaction_date,
        service_totals.services,
```

```
        service_totals.service_amount,
        product_totals.products,
        product_totals.product_amount,
        service_totals.service_amount + product_totals.product_amount AS
total_amount
      FROM completed_appointments AS appointment
      CROSS JOIN LATERAL (
        SELECT
          COALESCE(string_agg(service.service_name, ', ' ORDER BY
service.service_name), '-') AS services,
          COALESCE(sum(service.price), 0)::numeric AS service_amount
        FROM public.appointment_services AS service
        WHERE service.appointment_id = appointment.id
      ) AS service_totals
      CROSS JOIN LATERAL (
        SELECT
          COALESCE(
            string_agg(product.product_name || ' x ' || product.quantity, ', ' ORDER
BY product.product_name),
            '-'
          ) AS products,
          COALESCE(sum(product.quantity * product.unit_price), 0)::numeric AS
product_amount
        FROM public.appointment_products AS product
        WHERE product.appointment_id = appointment.id
      ) AS product_totals
      WHERE appointment.transaction_date BETWEEN p_from AND p_to
        AND (
          (p_category IS NULL AND p_service_name IS NULL)
          OR EXISTS (
            SELECT 1
            FROM public.appointment_services AS selected_service
            LEFT JOIN public.services AS catalog_service ON catalog_service.id =
selected_service.service_id
            WHERE selected_service.appointment_id = appointment.id
              AND (p_category IS NULL OR COALESCE(catalog_service.category,
'Uncategorized') = p_category)
              AND (p_service_name IS NULL OR selected_service.service_name =
p_service_name)
          )
        )
    ),
    scoped_services AS (
      SELECT
        service.appointment_id,
```

```
        service.service_name,
        service.price,
        COALESCE(catalog_service.category, 'Uncategorized') AS category
      FROM public.appointment_services AS service
      JOIN filtered AS appointment ON appointment.id = service.appointment_id
      LEFT JOIN public.services AS catalog_service ON catalog_service.id =
service.service_id
    ),
    scoped_products AS (
      SELECT product.appointment_id, product.product_name, product.quantity,
product.unit_price
      FROM public.appointment_products AS product
      JOIN filtered AS appointment ON appointment.id = product.appointment_id
    ),
    paged AS (
      SELECT *
      FROM filtered
      ORDER BY transaction_date DESC, reference_code ASC
      LIMIT v_limit OFFSET v_offset
    )
    SELECT jsonb_build_object(
      'total', (SELECT count(*) FROM filtered),
      'rows', COALESCE((SELECT jsonb_agg(to_jsonb(paged)) FROM paged),
'[]'::jsonb),
      'metrics', jsonb_build_object(
        'total_revenue', (SELECT COALESCE(sum(total_amount), 0) FROM
filtered),
        'service_revenue', (SELECT COALESCE(sum(service_amount), 0) FROM
filtered),
        'product_revenue', (SELECT COALESCE(sum(product_amount), 0) FROM
filtered),
        'completed_transactions', (SELECT count(*) FROM filtered),
        'total_completed_services', (SELECT count(*) FROM scoped_services),
        'total_products_sold', (SELECT COALESCE(sum(quantity), 0) FROM
scoped_products),
        'average_transaction_amount', (
          SELECT COALESCE(avg(total_amount), 0) FROM filtered
        )
      ),
      'service_performance', COALESCE((
        SELECT jsonb_agg(to_jsonb(performance) ORDER BY
performance.revenue DESC, performance.service_name)
        FROM (
          SELECT
            service_name,
```

```
          count(*)::integer AS completed_count,
          COALESCE(sum(price), 0)::numeric AS revenue
        FROM scoped_services
        GROUP BY service_name
      ) AS performance
    ), '[]'::jsonb),
    'product_sales', COALESCE((
      SELECT jsonb_agg(to_jsonb(sales) ORDER BY sales.total_sales DESC,
sales.product_name, sales.unit_price)
      FROM (
        SELECT
          product_name,
          sum(quantity)::integer AS quantity_sold,
          unit_price,
          COALESCE(sum(quantity * unit_price), 0)::numeric AS total_sales
        FROM scoped_products
        GROUP BY product_name, unit_price
      ) AS sales
    ), '[]'::jsonb),
    'daily_revenue', COALESCE((
      SELECT jsonb_agg(to_jsonb(day_revenue) ORDER BY
day_revenue.period)
      FROM (
        SELECT
          transaction_date::text AS period,
          count(*)::integer AS completed_transactions,
          COALESCE(sum(service_amount), 0)::numeric AS service_revenue,
          COALESCE(sum(product_amount), 0)::numeric AS product_revenue,
          COALESCE(sum(total_amount), 0)::numeric AS total_revenue
        FROM filtered
        GROUP BY transaction_date
      ) AS day_revenue
    ), '[]'::jsonb),
    'monthly_revenue', COALESCE((
      SELECT jsonb_agg(to_jsonb(month_revenue) ORDER BY
month_revenue.period)
      FROM (
        SELECT
          date_trunc('month', transaction_date::timestamp)::date::text AS period,
          count(*)::integer AS completed_transactions,
          COALESCE(sum(service_amount), 0)::numeric AS service_revenue,
          COALESCE(sum(product_amount), 0)::numeric AS product_revenue,
          COALESCE(sum(total_amount), 0)::numeric AS total_revenue
        FROM filtered
        GROUP BY date_trunc('month', transaction_date::timestamp)::date
```

```
        ) AS month_revenue
      ), '[]'::jsonb)
    )
  );
END;
$$;

-- 3. Scheduled cleanups ---------------------------------------------------

-- The original schedules were only created when pg_cron was already enabled,
-- but pg_cron was enabled by a later migration. pg_cron runs in UTC: 16:15 and
-- 16:30 UTC are 12:15 AM and 12:30 AM Philippine time.
DO $$
BEGIN
  PERFORM cron.unschedule(jobid)
  FROM cron.job
  WHERE jobname IN ('purge-admin-activity-logs', 'purge-expired-archives');

  PERFORM cron.schedule(
    'purge-admin-activity-logs',
    '15 16 * * *',
    'SELECT public.purge_expired_admin_activity_logs();'
  );
  PERFORM cron.schedule(
    'purge-expired-archives',
    '30 16 * * *',
    'SELECT public.purge_expired_archived_records();'
  );
END;
$$;

NOTIFY pgrst, 'reload schema';
```

supabase/migrations/20261015120000_hold_reschedule_requests_limit_bookings_balance_mechanics.sql

---

```
-- Booking rules:
-- 1. A pending reschedule request holds its requested time until it is
--    reviewed. It counts as a reservation that does not have a mechanic yet.
-- 2. A mobile number, and an email address, can each have at most 3 upcoming
--    appointments at a time.
-- 3. The mechanic with the fewest bookings that day is assigned.
-- 4. The duplicate rule compares plate numbers instead of mobile numbers, so
--    one customer can book the same service for two motorcycles on one day.

-- Scheduled jobs and the functions below must belong to postgres.
DO $$
BEGIN
  IF current_user <> 'postgres' THEN
    RAISE EXCEPTION 'Apply this migration as postgres (current role: %).',
current_user;
  END IF;
END;
$$;

-- Helpers ---------------------------------------------------------------

-- Plate numbers compare without spaces, dashes, or letter case.
CREATE OR REPLACE FUNCTION public.normalize_plate_number(p_plate
text)
RETURNS text
LANGUAGE sql
IMMUTABLE
SET search_path = public
AS $$
  SELECT upper(regexp_replace(COALESCE(p_plate, ''), '[^A-Za-z0-9]', '', 'g'));
$$;

-- [bookings, booked minutes] for one mechanic on one day. Arrays sort element
-- by element, so ORDER BY on this puts the least busy mechanic first.
CREATE OR REPLACE FUNCTION public.crew_daily_workload(
  p_crew_id uuid,
  p_date date,
  p_excluded_appointment_id uuid DEFAULT NULL
)
RETURNS integer[]
LANGUAGE sql
STABLE
```

```
SET search_path = public
AS $$
  SELECT ARRAY[count(*)::integer, COALESCE(sum(work.minutes), 0)::integer]
  FROM (
    SELECT appointment.booking_duration_minutes AS minutes
    FROM public.appointments AS appointment
    WHERE appointment.assigned_crew_id = p_crew_id
      AND appointment.appointment_date = p_date
      AND appointment.id IS DISTINCT FROM p_excluded_appointment_id
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
    UNION ALL
    SELECT continuation.booking_duration_minutes
    FROM public.appointment_continuations AS continuation
    JOIN public.appointments AS appointment ON appointment.id =
continuation.appointment_id
    WHERE continuation.assigned_crew_id = p_crew_id
      AND continuation.appointment_date = p_date
      AND appointment.id IS DISTINCT FROM p_excluded_appointment_id
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
  ) AS work;
$$;

REVOKE ALL ON FUNCTION public.normalize_plate_number(text) FROM
PUBLIC, anon;
GRANT EXECUTE ON FUNCTION public.normalize_plate_number(text) TO
authenticated, service_role;
REVOKE ALL ON FUNCTION public.crew_daily_workload(uuid, date, uuid)
FROM PUBLIC, anon;
GRANT EXECUTE ON FUNCTION public.crew_daily_workload(uuid, date, uuid)
TO authenticated, service_role;

-- 1. Reschedule requests hold their time ---------------------------------

CREATE OR REPLACE FUNCTION
public.enforce_active_appointment_capacity()
 RETURNS trigger
 LANGUAGE plpgsql
 SET search_path TO 'public'
AS $function$
```

```
DECLARE
  v_capacity integer;
  v_overlapping_reservations integer;
  v_start_minutes integer;
  v_opening_minutes integer;
  v_closing_minutes integer;
  v_opening_time time;
  v_closing_time time;
  v_hour_start time;
BEGIN
  -- Archived, replaced, and terminal appointments release both their slot
  -- capacity and their crew assignment.
  IF NEW.is_archived = true
     OR NEW.rescheduled_to_appointment_id IS NOT NULL
     OR NEW.status NOT IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
THEN
    RETURN NEW;
  END IF;

  SELECT opening_time, closing_time
    INTO v_opening_time, v_closing_time
  FROM public.shop_settings
  WHERE id = true;
  v_opening_time := COALESCE(v_opening_time, '08:00:00'::time);
  v_closing_time := COALESCE(v_closing_time, '17:00:00'::time);
  v_opening_minutes := extract(hour FROM v_opening_time)::integer * 60
    + extract(minute FROM v_opening_time)::integer;
  v_closing_minutes := extract(hour FROM v_closing_time)::integer * 60
    + extract(minute FROM v_closing_time)::integer;
  v_start_minutes := extract(hour FROM NEW.start_time)::integer * 60
    + extract(minute FROM NEW.start_time)::integer;

  IF extract(dow FROM NEW.appointment_date) = 0
     OR NEW.booking_duration_minutes IS NULL
     OR NEW.booking_duration_minutes <= 0
     OR v_start_minutes < v_opening_minutes
     OR v_start_minutes >= v_closing_minutes
     OR mod(v_start_minutes - v_opening_minutes, 30) <> 0
     OR v_start_minutes + NEW.booking_duration_minutes > v_closing_minutes
THEN
    RAISE EXCEPTION 'The appointment must fit within the configured shop
hours on a Monday through Saturday.'
      USING ERRCODE = '23P01';
  END IF;
```

```
  -- Serialize every active reservation for one date. The lock is deliberately
  -- acquired before both capacity and crew-overlap checks, so a transaction
  -- that waited on it sees a previously committed reservation.
  PERFORM pg_advisory_xact_lock(hashtext('booking-capacity:' ||
NEW.appointment_date::text));

  v_hour_start := make_time(extract(hour FROM NEW.start_time)::integer, 0, 0);
  SELECT slot.capacity
    INTO v_capacity
  FROM public.time_slots AS slot
  WHERE slot.is_active = true
    AND (slot.start_time = NEW.start_time OR slot.start_time = v_hour_start)
  ORDER BY (slot.start_time = NEW.start_time) DESC
  LIMIT 1
  FOR UPDATE;

  IF COALESCE(v_capacity, 0) <= 0 THEN
    RAISE EXCEPTION 'The requested time slot is not available.' USING
ERRCODE = '23P01';
  END IF;

  IF EXISTS (
    SELECT 1
    FROM public.schedule_blocks AS block
    WHERE block.block_date = NEW.appointment_date
      AND block.is_active = true
      AND (
        block.start_time IS NULL
        OR (
          COALESCE(block.reason, '') !~ '^RANGE:[0-9]{2}:[0-9]{2}'
          AND block.start_time = NEW.start_time
        )
        OR (
          COALESCE(block.reason, '') ~ '^RANGE:[0-9]{2}:[0-9]{2}'
          AND tsrange(
            NEW.appointment_date + NEW.start_time,
            NEW.appointment_date + NEW.start_time
              + make_interval(mins => NEW.booking_duration_minutes),
            '[)'
          ) && tsrange(
            NEW.appointment_date + block.start_time,
            NEW.appointment_date
              + substring(block.reason FROM '^RANGE:([0-9]{2}:[0-9]{2})')::time,
            '[)'
          )
```

```
      )
    )
  ) THEN
    RAISE EXCEPTION 'The shop is closed for the requested schedule.' USING
ERRCODE = '23P01';
  END IF;

  IF NEW.assigned_crew_id IS NOT NULL THEN
    IF NOT EXISTS (
      SELECT 1
      FROM public.crew_members AS crew
      JOIN public.crew_schedules AS schedule ON schedule.crew_id = crew.id
      WHERE crew.id = NEW.assigned_crew_id
        AND crew.is_active = true
        AND crew.is_archived = false
        AND schedule.schedule_date = NEW.appointment_date
        AND schedule.is_working = true
        AND schedule.start_time IS NOT NULL
        AND schedule.end_time IS NOT NULL
        AND schedule.start_time <= NEW.start_time
        AND schedule.end_time >= NEW.start_time
          + make_interval(mins => NEW.booking_duration_minutes)
    ) THEN
      RAISE EXCEPTION 'The assigned crew member is not scheduled for the
requested appointment period.'
        USING ERRCODE = '23P01';
    END IF;

    IF EXISTS (
      SELECT 1
      FROM public.crew_availability_exceptions AS exception
      WHERE exception.crew_id = NEW.assigned_crew_id
        AND exception.start_date <= NEW.appointment_date
        AND exception.end_date >= NEW.appointment_date
        AND (
          exception.is_all_day
          OR (
            exception.start_time IS NOT NULL
            AND exception.end_time IS NOT NULL
            AND tsrange(
              NEW.appointment_date + NEW.start_time,
              NEW.appointment_date + NEW.start_time
                + make_interval(mins => NEW.booking_duration_minutes),
              '[)'
            ) && tsrange(
```

```
            NEW.appointment_date + exception.start_time,
            NEW.appointment_date + exception.end_time,
            '[)'
          )
        )
      )
    ) THEN
      RAISE EXCEPTION 'The assigned crew member is unavailable for the
requested appointment period.'
        USING ERRCODE = '23P01';
    END IF;
  END IF;

  SELECT count(*)
    INTO v_overlapping_reservations
  FROM (
    SELECT appointment.appointment_date, appointment.start_time,
appointment.booking_duration_minutes
    FROM public.appointments AS appointment
    WHERE appointment.id <> NEW.id
      AND appointment.appointment_date = NEW.appointment_date
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
    UNION ALL
    SELECT continuation.appointment_date, continuation.start_time,
continuation.booking_duration_minutes
    FROM public.appointment_continuations AS continuation
    JOIN public.appointments AS appointment ON appointment.id =
continuation.appointment_id
    WHERE continuation.appointment_date = NEW.appointment_date
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
    UNION ALL
    -- A pending reschedule request holds its requested time until it is
    -- reviewed. The appointment's own request, and the request being approved
    -- (when this row is its replacement), do not count against it.
    SELECT hold.pending_reschedule_date, hold.pending_reschedule_start_time,
      COALESCE(hold.booking_duration_minutes, 75)
    FROM public.appointments AS hold
    WHERE hold.pending_reschedule_request_id IS NOT NULL
      AND hold.pending_reschedule_date = NEW.appointment_date
```

```
      AND hold.pending_reschedule_start_time IS NOT NULL
      AND hold.id <> NEW.id
      AND hold.id IS DISTINCT FROM NEW.rescheduled_from_appointment_id
      AND hold.is_archived = false
      AND hold.rescheduled_to_appointment_id IS NULL
      AND hold.status IN ('pending', 'confirmed')
  ) AS reservation
  WHERE tsrange(
    reservation.appointment_date + reservation.start_time,
    reservation.appointment_date + reservation.start_time
      + make_interval(mins => reservation.booking_duration_minutes),
    '[)'
  ) && tsrange(
    NEW.appointment_date + NEW.start_time,
    NEW.appointment_date + NEW.start_time + make_interval(mins =>
NEW.booking_duration_minutes),
    '[)'
  );

  IF v_overlapping_reservations >= v_capacity THEN
    RAISE EXCEPTION 'The requested schedule is no longer available.' USING
ERRCODE = '23P01';
  END IF;

  -- Capacity alone is not enough: two slots can overlap while there is still
  -- aggregate capacity for another crew. A crew ending at exactly the new
  -- start time is allowed because all reservation ranges are [start, end).
  IF NEW.assigned_crew_id IS NOT NULL AND EXISTS (
    SELECT 1
    FROM (
      SELECT appointment.assigned_crew_id, appointment.appointment_date,
        appointment.start_time, appointment.booking_duration_minutes
      FROM public.appointments AS appointment
      WHERE appointment.id <> NEW.id
        AND appointment.assigned_crew_id = NEW.assigned_crew_id
        AND appointment.appointment_date = NEW.appointment_date
        AND appointment.is_archived = false
        AND appointment.rescheduled_to_appointment_id IS NULL
        AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
      UNION ALL
      SELECT continuation.assigned_crew_id, continuation.appointment_date,
        continuation.start_time, continuation.booking_duration_minutes
      FROM public.appointment_continuations AS continuation
      JOIN public.appointments AS appointment ON appointment.id =
```

```
continuation.appointment_id
      WHERE continuation.assigned_crew_id = NEW.assigned_crew_id
        AND continuation.appointment_date = NEW.appointment_date
        AND appointment.is_archived = false
        AND appointment.rescheduled_to_appointment_id IS NULL
        AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
    ) AS crew_reservation
    WHERE tsrange(
      crew_reservation.appointment_date + crew_reservation.start_time,
      crew_reservation.appointment_date + crew_reservation.start_time
        + make_interval(mins => crew_reservation.booking_duration_minutes),
      '[)'
    ) && tsrange(
      NEW.appointment_date + NEW.start_time,
      NEW.appointment_date + NEW.start_time + make_interval(mins =>
NEW.booking_duration_minutes),
      '[)'
    )
  ) THEN
    RAISE EXCEPTION 'The assigned crew member is no longer available for the
requested appointment period.'
      USING ERRCODE = '23P01';
  END IF;

  RETURN NEW;
END;
$function$;

CREATE OR REPLACE FUNCTION
public.enforce_active_appointment_continuation_capacity()
 RETURNS trigger
 LANGUAGE plpgsql
 SET search_path TO 'public'
AS $function$
DECLARE
  v_parent public.appointments%ROWTYPE;
  v_opening_time time;
  v_closing_time time;
  v_opening_minutes integer;
  v_closing_minutes integer;
  v_start_minutes integer;
  v_hour_start time;
  v_capacity integer;
  v_overlapping_reservations integer;
```

```
BEGIN
  SELECT * INTO v_parent FROM public.appointments WHERE id =
NEW.appointment_id;
  IF NOT FOUND THEN
    RAISE EXCEPTION 'The parent appointment no longer exists.' USING
ERRCODE = '23503';
  END IF;

  IF v_parent.is_archived = true
     OR v_parent.rescheduled_to_appointment_id IS NOT NULL
     OR v_parent.status NOT IN ('pending', 'confirmed', 'in_progress',
'rescheduled') THEN
    RETURN NEW;
  END IF;

  SELECT opening_time, closing_time INTO v_opening_time, v_closing_time
  FROM public.shop_settings WHERE id = true;
  v_opening_time := COALESCE(v_opening_time, '08:00:00'::time);
  v_closing_time := COALESCE(v_closing_time, '17:00:00'::time);
  v_opening_minutes := extract(hour FROM v_opening_time)::integer * 60
    + extract(minute FROM v_opening_time)::integer;
  v_closing_minutes := extract(hour FROM v_closing_time)::integer * 60
    + extract(minute FROM v_closing_time)::integer;
  v_start_minutes := extract(hour FROM NEW.start_time)::integer * 60
    + extract(minute FROM NEW.start_time)::integer;

  IF extract(dow FROM NEW.appointment_date) = 0
     OR NEW.booking_duration_minutes <= 0
     OR v_start_minutes < v_opening_minutes
     OR v_start_minutes >= v_closing_minutes
     OR mod(v_start_minutes - v_opening_minutes, 30) <> 0
     OR v_start_minutes + NEW.booking_duration_minutes > v_closing_minutes
THEN
    RAISE EXCEPTION 'The continuation must fit within the configured shop
hours in 30-minute start intervals.'
      USING ERRCODE = '23P01';
  END IF;

  IF EXISTS (
    SELECT 1
    FROM public.schedule_blocks AS block
    WHERE block.block_date = NEW.appointment_date
      AND block.is_active = true
      AND (
        block.start_time IS NULL
```

```
        OR (
          COALESCE(block.reason, '') NOT LIKE 'RANGE:%'
          AND block.start_time = NEW.start_time
        )
        OR (
          COALESCE(block.reason, '') LIKE 'RANGE:%'
          AND tsrange(
            NEW.appointment_date + NEW.start_time,
            NEW.appointment_date + NEW.start_time
              + make_interval(mins => NEW.booking_duration_minutes),
            '[)'
          ) && tsrange(
            NEW.appointment_date + block.start_time,
            NEW.appointment_date
              + substring(block.reason FROM '^RANGE:([0-9]{2}:[0-9]{2})')::time,
            '[)'
          )
        )
      )
  ) THEN
    RAISE EXCEPTION 'The shop is closed for the continuation schedule.' USING
ERRCODE = '23P01';
  END IF;

  PERFORM pg_advisory_xact_lock(hashtext('booking-capacity:' ||
NEW.appointment_date::text));
  v_hour_start := make_time(extract(hour FROM NEW.start_time)::integer, 0, 0);

  SELECT slot.capacity INTO v_capacity
  FROM public.time_slots AS slot
  WHERE slot.is_active = true
    AND (slot.start_time = NEW.start_time OR slot.start_time = v_hour_start)
  ORDER BY (slot.start_time = NEW.start_time) DESC
  LIMIT 1
  FOR UPDATE;

  IF COALESCE(v_capacity, 0) <= 0 THEN
    RAISE EXCEPTION 'The requested continuation time slot is not available.'
USING ERRCODE = '23P01';
  END IF;

  SELECT count(*) INTO v_overlapping_reservations
  FROM (
    SELECT appointment.appointment_date, appointment.start_time,
appointment.booking_duration_minutes
```

```
    FROM public.appointments AS appointment
    WHERE appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
      AND appointment.appointment_date = NEW.appointment_date
    UNION ALL
    SELECT continuation.appointment_date, continuation.start_time,
continuation.booking_duration_minutes
    FROM public.appointment_continuations AS continuation
    JOIN public.appointments AS appointment ON appointment.id =
continuation.appointment_id
    WHERE continuation.id <> NEW.id
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
      AND continuation.appointment_date = NEW.appointment_date
    UNION ALL
    -- Pending reschedule requests hold their requested times too.
    SELECT hold.pending_reschedule_date, hold.pending_reschedule_start_time,
      COALESCE(hold.booking_duration_minutes, 75)
    FROM public.appointments AS hold
    WHERE hold.pending_reschedule_request_id IS NOT NULL
      AND hold.pending_reschedule_date = NEW.appointment_date
      AND hold.pending_reschedule_start_time IS NOT NULL
      AND hold.is_archived = false
      AND hold.rescheduled_to_appointment_id IS NULL
      AND hold.status IN ('pending', 'confirmed')
  ) AS reservation
  WHERE tsrange(
    reservation.appointment_date + reservation.start_time,
    reservation.appointment_date + reservation.start_time
      + make_interval(mins => reservation.booking_duration_minutes),
    '[)'
  ) && tsrange(
    NEW.appointment_date + NEW.start_time,
    NEW.appointment_date + NEW.start_time + make_interval(mins =>
NEW.booking_duration_minutes),
    '[)'
  );

  IF v_overlapping_reservations >= v_capacity THEN
    RAISE EXCEPTION 'The requested continuation schedule is no longer
available.' USING ERRCODE = '23P01';
```

```
  END IF;

  IF NOT public.is_crew_available_for_appointment(
    NEW.assigned_crew_id,
    NEW.appointment_date,
    NEW.start_time,
    NEW.booking_duration_minutes,
    NULL,
    NEW.id
  ) THEN
    RAISE EXCEPTION 'The assigned mechanic is no longer available for the
continuation schedule.'
      USING ERRCODE = '23P01';
  END IF;

  RETURN NEW;
END;
$function$;

-- Setting or moving a request checks that its time still has room: the slot
-- limit counts every overlapping reservation, and a free mechanic must remain
-- for each reservation that has none yet (as in the booking screen).
CREATE OR REPLACE FUNCTION
public.enforce_reschedule_request_capacity()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = public
AS $$
DECLARE
  v_date date := NEW.pending_reschedule_date;
  v_start time := NEW.pending_reschedule_start_time;
  v_duration integer := COALESCE(NEW.booking_duration_minutes, 75);
  v_capacity integer;
  v_overlapping integer;
  v_unassigned integer;
  v_free_crew integer;
BEGIN
  IF NEW.pending_reschedule_request_id IS NULL
     OR v_date IS NULL
     OR v_start IS NULL
     OR NEW.is_archived
     OR NEW.rescheduled_to_appointment_id IS NOT NULL
     OR NEW.status NOT IN ('pending', 'confirmed') THEN
    RETURN NEW;
  END IF;
```

```
  IF OLD.pending_reschedule_request_id IS NOT DISTINCT FROM
NEW.pending_reschedule_request_id
     AND OLD.pending_reschedule_date IS NOT DISTINCT FROM v_date
     AND OLD.pending_reschedule_start_time IS NOT DISTINCT FROM v_start
THEN
    RETURN NEW;
  END IF;

  PERFORM pg_advisory_xact_lock(hashtext('booking-capacity:' || v_date::text));

  SELECT slot.capacity
    INTO v_capacity
  FROM public.time_slots AS slot
  WHERE slot.is_active = true
    AND (
      slot.start_time = v_start
      OR slot.start_time = make_time(extract(hour FROM v_start)::integer, 0, 0)
    )
  ORDER BY (slot.start_time = v_start) DESC
  LIMIT 1;

  SELECT count(*), count(*) FILTER (WHERE reservation.crew_id IS NULL)
    INTO v_overlapping, v_unassigned
  FROM (
    SELECT appointment.start_time, appointment.booking_duration_minutes AS
minutes,
      appointment.assigned_crew_id AS crew_id
    FROM public.appointments AS appointment
    WHERE appointment.id <> NEW.id
      AND appointment.appointment_date = v_date
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
    UNION ALL
    SELECT continuation.start_time, continuation.booking_duration_minutes,
      continuation.assigned_crew_id
    FROM public.appointment_continuations AS continuation
    JOIN public.appointments AS parent ON parent.id =
continuation.appointment_id
    WHERE continuation.appointment_date = v_date
      AND parent.id <> NEW.id
      AND parent.is_archived = false
      AND parent.rescheduled_to_appointment_id IS NULL
      AND parent.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
```

```
    UNION ALL
    SELECT hold.pending_reschedule_start_time,
COALESCE(hold.booking_duration_minutes, 75),
      NULL::uuid
    FROM public.appointments AS hold
    WHERE hold.id <> NEW.id
      AND hold.pending_reschedule_request_id IS NOT NULL
      AND hold.pending_reschedule_date = v_date
      AND hold.pending_reschedule_start_time IS NOT NULL
      AND hold.is_archived = false
      AND hold.rescheduled_to_appointment_id IS NULL
      AND hold.status IN ('pending', 'confirmed')
  ) AS reservation
  WHERE tsrange(
    v_date + reservation.start_time,
    v_date + reservation.start_time + make_interval(mins => reservation.minutes),
    '[)'
  ) && tsrange(v_date + v_start, v_date + v_start + make_interval(mins =>
v_duration), '[)');

  IF COALESCE(v_capacity, 0) - v_overlapping < 1 THEN
    RAISE EXCEPTION 'The requested time is no longer available. Please
choose another time.'
      USING ERRCODE = '23P01';
  END IF;

  SELECT count(*)
    INTO v_free_crew
  FROM public.crew_members AS crew
  WHERE crew.is_active = true
    AND crew.is_archived = false
    AND public.is_crew_available_for_appointment(crew.id, v_date, v_start,
v_duration, NEW.id);

  IF v_free_crew - v_unassigned < 1 THEN
    RAISE EXCEPTION 'No mechanic is available at the requested time. Please
choose another time.'
      USING ERRCODE = '23P01';
  END IF;

  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS
```

```
appointments_enforce_reschedule_request_capacity ON public.appointments;
CREATE TRIGGER appointments_enforce_reschedule_request_capacity
  BEFORE UPDATE OF pending_reschedule_request_id,
pending_reschedule_date, pending_reschedule_start_time
  ON public.appointments
  FOR EACH ROW EXECUTE FUNCTION
public.enforce_reschedule_request_capacity();

CREATE OR REPLACE FUNCTION
public.review_reschedule_request(p_appointment_id uuid, p_decision text)
 RETURNS void
 LANGUAGE plpgsql
 SET search_path TO 'public'
AS $function$
DECLARE
  v_appointment public.appointments%ROWTYPE;
  v_assigned_crew_id uuid;
  v_duration_minutes integer;
  v_new_appointment_id uuid;
  v_new_reference_code text;
  v_reason text;
  v_original_shop_notes text;
  v_new_shop_notes text;
  v_reschedule_number integer;
  v_slot_capacity integer;
  v_overlapping_appointments integer;
  v_start_minutes integer;
  v_opening_time time;
  v_closing_time time;
  v_opening_minutes integer;
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Only administrators can review reschedule requests.';
  END IF;
  IF lower(btrim(COALESCE(p_decision, ''))) NOT IN ('confirmed', 'rejected')
THEN
    RAISE EXCEPTION 'Choose either confirmed or rejected.';
  END IF;

  SELECT appointment.*
    INTO v_appointment
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
  FOR UPDATE;
```

```
  IF NOT FOUND OR v_appointment.pending_reschedule_request_id IS NULL
THEN
    RAISE EXCEPTION 'This reschedule request is no longer available.';
  END IF;

  v_reschedule_number := v_appointment.reschedule_count + 1;

  IF lower(btrim(p_decision)) = 'rejected' THEN
    INSERT INTO public.appointment_reschedule_history (
      appointment_id,
      new_appointment_id,
      reschedule_number,
      from_date,
      from_start_time,
      to_date,
      to_start_time,
      reason,
      decision,
      approved_by
    ) VALUES (
      v_appointment.id,
      NULL,
      v_reschedule_number,
      v_appointment.appointment_date,
      v_appointment.start_time,
      v_appointment.pending_reschedule_date,
      v_appointment.pending_reschedule_start_time,
      btrim(v_appointment.pending_reschedule_reason),
      'rejected',
      auth.uid()
    );

    UPDATE public.appointments
    SET pending_reschedule_request_id = NULL,
        pending_reschedule_date = NULL,
        pending_reschedule_start_time = NULL,
        pending_reschedule_reference_code = NULL,
        pending_reschedule_reason = NULL
    WHERE id = v_appointment.id;
    RETURN;
  END IF;

  IF v_appointment.reschedule_count >= 3 THEN
    RAISE EXCEPTION 'This appointment has reached the maximum of 3
```

```
reschedules.';
  END IF;
  IF v_appointment.pending_reschedule_date IS NULL
     OR v_appointment.pending_reschedule_start_time IS NULL
     OR NULLIF(btrim(v_appointment.pending_reschedule_reason), '') IS NULL
THEN
    RAISE EXCEPTION 'The reschedule request is missing its requested
schedule or reason.';
  END IF;

  v_duration_minutes := COALESCE(v_appointment.booking_duration_minutes,
75);
  v_reason := btrim(v_appointment.pending_reschedule_reason);
  v_start_minutes := extract(hour FROM
v_appointment.pending_reschedule_start_time)::integer * 60
    + extract(minute FROM
v_appointment.pending_reschedule_start_time)::integer;

  SELECT settings.opening_time, settings.closing_time
    INTO v_opening_time, v_closing_time
  FROM public.shop_settings AS settings
  ORDER BY settings.id
  LIMIT 1;
  v_opening_time := COALESCE(v_opening_time, '08:00'::time);
  v_closing_time := COALESCE(v_closing_time, '17:00'::time);
  v_opening_minutes := extract(hour FROM v_opening_time)::integer * 60
    + extract(minute FROM v_opening_time)::integer;

  IF extract(dow FROM v_appointment.pending_reschedule_date) = 0
     OR v_appointment.pending_reschedule_start_time < v_opening_time
     OR v_appointment.pending_reschedule_start_time >= v_closing_time
     OR mod(v_start_minutes - v_opening_minutes, 30) <> 0
     OR v_appointment.pending_reschedule_start_time + make_interval(mins =>
v_duration_minutes)
        > v_closing_time THEN
    RAISE EXCEPTION 'The requested schedule is outside the shop booking
hours.';
  END IF;

  SELECT
public.booking_slot_capacity(v_appointment.pending_reschedule_start_time)
    INTO v_slot_capacity;
  IF COALESCE(v_slot_capacity, 0) <= 0 THEN
    RAISE EXCEPTION 'The requested time slot is no longer available.';
  END IF;
```

```
  SELECT count(*)
    INTO v_overlapping_appointments
  FROM public.appointments AS other
  WHERE other.id <> v_appointment.id
    AND other.appointment_date = v_appointment.pending_reschedule_date
    AND other.is_archived = false
    AND other.rescheduled_to_appointment_id IS NULL
    AND other.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
    AND tsrange(
      other.appointment_date + other.start_time,
      other.appointment_date + other.start_time
        + make_interval(mins => COALESCE(other.booking_duration_minutes,
75)),
      '[)'
    ) && tsrange(
      v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
      v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
        + make_interval(mins => v_duration_minutes),
      '[)'
    );
  -- Other pending reschedule requests hold their requested times too.
  v_overlapping_appointments := v_overlapping_appointments + (
    SELECT count(*)
    FROM public.appointments AS hold
    WHERE hold.id <> v_appointment.id
      AND hold.pending_reschedule_request_id IS NOT NULL
      AND hold.pending_reschedule_date =
v_appointment.pending_reschedule_date
      AND hold.pending_reschedule_start_time IS NOT NULL
      AND hold.is_archived = false
      AND hold.rescheduled_to_appointment_id IS NULL
      AND hold.status IN ('pending', 'confirmed')
      AND tsrange(
        hold.pending_reschedule_date + hold.pending_reschedule_start_time,
        hold.pending_reschedule_date + hold.pending_reschedule_start_time
          + make_interval(mins => COALESCE(hold.booking_duration_minutes,
75)),
        '[)'
      ) && tsrange(
        v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
        v_appointment.pending_reschedule_date +
```

```
v_appointment.pending_reschedule_start_time + make_interval(mins =>
v_duration_minutes),
        '[)'
      )
  );
  IF v_overlapping_appointments >= v_slot_capacity THEN
    RAISE EXCEPTION 'The requested time is no longer available.';
  END IF;

  IF EXISTS (
    SELECT 1
    FROM public.schedule_blocks AS block
    WHERE block.block_date = v_appointment.pending_reschedule_date
      AND block.is_active = true
      AND (
        block.start_time IS NULL
        OR (
          block.start_time IS NOT NULL
          AND (
            (COALESCE(block.reason, '') NOT LIKE 'RANGE:%'
              AND block.start_time = v_appointment.pending_reschedule_start_time)
            OR (COALESCE(block.reason, '') LIKE 'RANGE:%'
              AND tsrange(
                v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
                v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
                  + make_interval(mins => v_duration_minutes),
                '[)'
              ) && tsrange(
                v_appointment.pending_reschedule_date + block.start_time,
                v_appointment.pending_reschedule_date
                  + substring(block.reason FROM '^RANGE:([0-9]{2}:[0-9]{2})')::time,
                '[)'
              ))
          )
        )
      )
  ) THEN
    RAISE EXCEPTION 'The requested time is no longer available because the
shop schedule is blocked.';
  END IF;

  SELECT crew.id
    INTO v_assigned_crew_id
```

```
  FROM public.crew_members AS crew
  WHERE crew.is_active = true
    AND crew.is_archived = false
    AND EXISTS (
      SELECT 1
      FROM public.crew_schedules AS schedule
      WHERE schedule.crew_id = crew.id
        AND schedule.schedule_date = v_appointment.pending_reschedule_date
        AND schedule.is_working = true
        AND schedule.start_time IS NOT NULL
        AND schedule.end_time IS NOT NULL
        AND schedule.start_time <= v_appointment.pending_reschedule_start_time
        AND schedule.end_time >= v_appointment.pending_reschedule_start_time
          + make_interval(mins => v_duration_minutes)
    )
    AND NOT EXISTS (
      SELECT 1
      FROM public.crew_availability_exceptions AS exception
      WHERE exception.crew_id = crew.id
        AND exception.start_date <= v_appointment.pending_reschedule_date
        AND exception.end_date >= v_appointment.pending_reschedule_date
        AND (
          exception.is_all_day
          OR (
            exception.start_time IS NOT NULL
            AND exception.end_time IS NOT NULL
            AND tsrange(
              v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
              v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
                + make_interval(mins => v_duration_minutes),
              '[)'
            ) && tsrange(
              v_appointment.pending_reschedule_date + exception.start_time,
              v_appointment.pending_reschedule_date + exception.end_time,
              '[)'
            )
          )
        )
    )
    AND NOT EXISTS (
      SELECT 1
      FROM public.appointments AS other
      WHERE other.id <> v_appointment.id
```

```
      AND other.assigned_crew_id = crew.id
      AND other.appointment_date = v_appointment.pending_reschedule_date
      AND other.is_archived = false
      AND other.rescheduled_to_appointment_id IS NULL
      AND other.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
      AND tsrange(
        other.appointment_date + other.start_time,
        other.appointment_date + other.start_time
          + make_interval(mins => COALESCE(other.booking_duration_minutes,
75)),
        '[)'
      ) && tsrange(
        v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time,
        v_appointment.pending_reschedule_date +
v_appointment.pending_reschedule_start_time
          + make_interval(mins => v_duration_minutes),
        '[)'
      )
  )
  -- The mechanic with the fewest bookings that day takes the new schedule.
  ORDER BY public.crew_daily_workload(
    crew.id,
    v_appointment.pending_reschedule_date,
    v_appointment.id
  ), crew.name, crew.id
  LIMIT 1;

  IF v_assigned_crew_id IS NULL THEN
    RAISE EXCEPTION 'No crew member is available for the requested
appointment period.';
  END IF;

  v_new_reference_code := v_appointment.pending_reschedule_reference_code;
  IF v_new_reference_code IS NULL
     OR v_new_reference_code !~ '^FRM-[A-Z0-9]{6}$' THEN
    RAISE EXCEPTION 'The reschedule request is missing its replacement
reference.';
  END IF;

  v_original_shop_notes := concat_ws(
    E'\n',
    NULLIF(btrim(v_appointment.admin_notes), ''),
    'Rescheduled to ' || v_new_reference_code,
    'Reason for rescheduling: ' || v_reason
```

```
);
v_new_shop_notes := concat_ws(
  E'\n',
  'Rescheduled from ' || v_appointment.reference_code,
  'Reason for rescheduling: ' || v_reason
);

INSERT INTO public.appointments AS replacement (
  reference_code,
  booking_request_id,
  customer_name,
  first_name,
  middle_name,
  last_name,
  phone,
  moto_brand,
  moto_model,
  moto_variant,
  moto_year,
  plate_number,
  appointment_date,
  start_time,
  status,
  notes,
  admin_notes,
  total_estimate,
  booking_duration_minutes,
  terms_accepted,
  assigned_crew_id,
  reschedule_count,
  rescheduled_from_appointment_id
) VALUES (
  v_new_reference_code,
  gen_random_uuid(),
  v_appointment.customer_name,
  v_appointment.first_name,
  v_appointment.middle_name,
  v_appointment.last_name,
  v_appointment.phone,
  v_appointment.moto_brand,
  v_appointment.moto_model,
  v_appointment.moto_variant,
  v_appointment.moto_year,
  v_appointment.plate_number,
  v_appointment.pending_reschedule_date,
```

```
  v_appointment.pending_reschedule_start_time,
  'confirmed',
  v_appointment.notes,
  v_new_shop_notes,
  v_appointment.total_estimate,
  v_duration_minutes,
  v_appointment.terms_accepted,
  v_assigned_crew_id,
  v_appointment.reschedule_count + 1,
  v_appointment.id
)
RETURNING replacement.id INTO v_new_appointment_id;

INSERT INTO public.appointment_services (
  appointment_id,
  service_id,
  service_name,
  price,
  duration_minutes
)
SELECT
  v_new_appointment_id,
  service.service_id,
  service.service_name,
  service.price,
  service.duration_minutes
FROM public.appointment_services AS service
WHERE service.appointment_id = v_appointment.id;

UPDATE public.appointments
SET status = 'rescheduled',
    admin_notes = v_original_shop_notes,
    reschedule_count = v_appointment.reschedule_count + 1,
    rescheduled_to_appointment_id = v_new_appointment_id,
    pending_reschedule_request_id = NULL,
    pending_reschedule_date = NULL,
    pending_reschedule_start_time = NULL,
    pending_reschedule_reference_code = NULL,
    pending_reschedule_reason = NULL
WHERE id = v_appointment.id;

INSERT INTO public.notifications (type, title, message, appointment_id)
VALUES (
  'rescheduled_appointment',
  'Rescheduled booking ' || v_new_reference_code,
```

```
    v_appointment.customer_name || ' rescheduled from ' ||
v_appointment.reference_code
      || ' to ' || v_new_reference_code || '. Reason: ' || v_reason,
    v_new_appointment_id
  );

  INSERT INTO public.appointment_reschedule_history (
    appointment_id,
    new_appointment_id,
    reschedule_number,
    from_date,
    from_start_time,
    to_date,
    to_start_time,
    reason,
    decision,
    approved_by
  ) VALUES (
    v_appointment.id,
    v_new_appointment_id,
    v_reschedule_number,
    v_appointment.appointment_date,
    v_appointment.start_time,
    v_appointment.pending_reschedule_date,
    v_appointment.pending_reschedule_start_time,
    v_reason,
    'confirmed',
    auth.uid()
  );
END;
$function$;

-- 2. At most 3 upcoming appointments per mobile number and per email --------

CREATE OR REPLACE FUNCTION
public.enforce_customer_upcoming_appointment_limit()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = public
AS $$
DECLARE
  v_limit constant integer := 3;
  v_today date := (now() AT TIME ZONE 'Asia/Manila')::date;
  v_phone text := COALESCE(public.normalize_philippine_mobile(NEW.phone),
btrim(NEW.phone));
```

```
  v_email text := NULLIF(lower(btrim(COALESCE(NEW.email, ''))), '');
BEGIN
  -- A replacement created by an approved reschedule takes over an existing
  -- appointment, so it is not limited.
  IF NEW.rescheduled_from_appointment_id IS NOT NULL
     OR NEW.is_archived
     OR NEW.status NOT IN ('pending', 'confirmed', 'in_progress')
     OR NEW.appointment_date < v_today THEN
    RETURN NEW;
  END IF;

  PERFORM
pg_advisory_xact_lock(hashtext('customer-upcoming-appointments:' ||
v_phone));
  IF (
    SELECT count(*)
    FROM public.appointments AS appointment
    WHERE appointment.phone = v_phone
      AND appointment.id <> NEW.id
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress')
      AND appointment.appointment_date >= v_today
  ) >= v_limit THEN
    RAISE EXCEPTION 'This mobile number already has % upcoming
appointments, the most allowed at one time. Please cancel one or wait until one
is done before booking another.', v_limit
      USING ERRCODE = '23514';
  END IF;

  IF v_email IS NOT NULL THEN
    PERFORM
pg_advisory_xact_lock(hashtext('customer-upcoming-appointments:' || v_email));
    IF (
      SELECT count(*)
      FROM public.appointments AS appointment
      WHERE lower(btrim(appointment.email)) = v_email
        AND appointment.id <> NEW.id
        AND appointment.is_archived = false
        AND appointment.rescheduled_to_appointment_id IS NULL
        AND appointment.status IN ('pending', 'confirmed', 'in_progress')
        AND appointment.appointment_date >= v_today
    ) >= v_limit THEN
      RAISE EXCEPTION 'This email address already has % upcoming
appointments, the most allowed at one time. Please cancel one or wait until one
```

```
is done before booking another.', v_limit
        USING ERRCODE = '23514';
    END IF;
  END IF;

  RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS appointments_enforce_customer_upcoming_limit
ON public.appointments;
CREATE TRIGGER appointments_enforce_customer_upcoming_limit
  BEFORE INSERT ON public.appointments
  FOR EACH ROW EXECUTE FUNCTION
public.enforce_customer_upcoming_appointment_limit();

-- 3. The least busy mechanic ----------------------------------------------

CREATE OR REPLACE FUNCTION
public.review_pending_appointment(p_appointment_id uuid, p_decision text)
 RETURNS void
 LANGUAGE plpgsql
 SET search_path TO 'public'
AS $function$
DECLARE
  v_appointment public.appointments%ROWTYPE;
  v_assigned_crew_id uuid;
  v_duration_minutes integer;
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Only administrators can review appointments.';
  END IF;

  IF p_decision IS NULL OR lower(btrim(p_decision)) NOT IN ('confirmed',
'rejected') THEN
    RAISE EXCEPTION 'Choose either confirmed or rejected.';
  END IF;

  -- Lock the row so two notification actions cannot review the same booking.
  SELECT appointment.*
    INTO v_appointment
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
  FOR UPDATE;
```

```
  IF NOT FOUND THEN
    RAISE EXCEPTION 'This appointment is no longer available.';
  END IF;

  IF v_appointment.status <> 'pending' THEN
    RAISE EXCEPTION 'This appointment has already been reviewed.';
  END IF;

  IF lower(btrim(p_decision)) = 'rejected' THEN
    UPDATE public.appointments
    SET status = 'rejected',
        assigned_crew_id = NULL
    WHERE id = v_appointment.id;
    RETURN;
  END IF;

  v_duration_minutes := COALESCE(v_appointment.booking_duration_minutes,
75);

  -- Prefer the crew member reserved during booking, but fall back to another
  -- currently available member when schedules or availability have changed.
  SELECT crew.id
    INTO v_assigned_crew_id
  FROM public.crew_members AS crew
  WHERE crew.is_active = true
    AND crew.is_archived = false
    AND EXISTS (
      SELECT 1
      FROM public.crew_schedules AS schedule
      WHERE schedule.crew_id = crew.id
        AND schedule.schedule_date = v_appointment.appointment_date
        AND schedule.is_working = true
        AND schedule.start_time IS NOT NULL
        AND schedule.end_time IS NOT NULL
        AND schedule.start_time <= v_appointment.start_time
        AND schedule.end_time >= v_appointment.start_time + make_interval(mins
=> v_duration_minutes)
    )
    AND NOT EXISTS (
      SELECT 1
      FROM public.crew_availability_exceptions AS exception
      WHERE exception.crew_id = crew.id
        AND exception.start_date <= v_appointment.appointment_date
        AND exception.end_date >= v_appointment.appointment_date
```

```
      AND (
        exception.is_all_day = true
        OR (
          exception.start_time IS NOT NULL
          AND exception.end_time IS NOT NULL
          AND tsrange(
            v_appointment.appointment_date + v_appointment.start_time,
            v_appointment.appointment_date + v_appointment.start_time
              + make_interval(mins => v_duration_minutes),
            '[)'
          ) && tsrange(
            v_appointment.appointment_date + exception.start_time,
            v_appointment.appointment_date + exception.end_time,
            '[)'
          )
        )
      )
    )
    AND NOT EXISTS (
      SELECT 1
      FROM public.appointments AS other
      WHERE other.id <> v_appointment.id
        AND other.assigned_crew_id = crew.id
        AND other.appointment_date = v_appointment.appointment_date
        AND other.is_archived = false
        AND other.rescheduled_to_appointment_id IS NULL
        AND other.status IN ('pending', 'confirmed', 'in_progress', 'rescheduled')
        AND tsrange(
          other.appointment_date + other.start_time,
          other.appointment_date + other.start_time
            + make_interval(mins => COALESCE(other.booking_duration_minutes,
75)),
          '[)'
        ) && tsrange(
          v_appointment.appointment_date + v_appointment.start_time,
          v_appointment.appointment_date + v_appointment.start_time
            + make_interval(mins => v_duration_minutes),
          '[)'
        )
    )
  ORDER BY (crew.id = v_appointment.assigned_crew_id) DESC,
    public.crew_daily_workload(crew.id, v_appointment.appointment_date,
v_appointment.id),
    crew.name,
    crew.id
```

```
  LIMIT 1;

  IF v_assigned_crew_id IS NULL THEN
    RAISE EXCEPTION 'No crew member is available for the selected
appointment period.';
  END IF;

  UPDATE public.appointments
  SET status = 'confirmed',
      assigned_crew_id = v_assigned_crew_id
  WHERE id = v_appointment.id;
END;
$function$;

CREATE OR REPLACE FUNCTION
public.update_appointment_details_atomic(p_appointment_id uuid,
p_service_ids uuid[], p_appointment_date date, p_start_time time without time
zone, p_assigned_crew_id uuid, p_crew_assignment_manual boolean, p_status
text, p_admin_notes text)
 RETURNS void
 LANGUAGE plpgsql
 SET search_path TO 'public'
AS $function$
DECLARE
  v_service_count integer;
  v_total_estimate numeric(10,2);
  v_duration_minutes integer;
  v_assigned_crew_id uuid := p_assigned_crew_id;
  v_appointment_id uuid;
  v_motorcycle_category text;
BEGIN
  IF COALESCE(cardinality(p_service_ids), 0) = 0 THEN
    RAISE EXCEPTION 'Select at least one service.';
  END IF;

  IF p_status NOT IN (
    'pending', 'confirmed', 'in_progress', 'completed', 'rescheduled', 'cancelled',
'rejected', 'no_show'
  ) THEN
    RAISE EXCEPTION 'Choose a valid appointment status.';
  END IF;

  SELECT appointment.id, catalog.cc_category
    INTO v_appointment_id, v_motorcycle_category
  FROM public.appointments AS appointment
```

```
  LEFT JOIN public.motorcycle_catalog AS catalog
    ON lower(btrim(catalog.brand)) = lower(btrim(appointment.moto_brand))
   AND lower(btrim(catalog.model)) = lower(btrim(appointment.moto_model))
   AND catalog.is_archived = false
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
  LIMIT 1;

  IF v_appointment_id IS NULL THEN
    RAISE EXCEPTION 'This appointment is no longer available.';
  END IF;
  IF v_motorcycle_category IS NULL
     OR v_motorcycle_category NOT IN ('small_bike', 'big_bike') THEN
    RAISE EXCEPTION 'The appointment motorcycle needs an engine CC
classification before its services can be updated.';
  END IF;

  SELECT count(*)
    INTO v_service_count
  FROM public.services
  WHERE id = ANY(p_service_ids)
    AND is_active = true
    AND is_archived = false;

  IF v_service_count <> cardinality(p_service_ids) THEN
    RAISE EXCEPTION 'One or more selected services are no longer available.';
  END IF;

  IF v_motorcycle_category = 'big_bike' AND EXISTS (
    SELECT 1
    FROM public.services
    WHERE id = ANY(p_service_ids)
      AND (large_bike_price IS NULL OR large_bike_duration_minutes IS NULL)
  ) THEN
    RAISE EXCEPTION 'Every selected service needs a Large Bike price and
duration.';
  END IF;

  SELECT
    COALESCE(sum(CASE WHEN v_motorcycle_category = 'big_bike' THEN
large_bike_price ELSE price END), 0),
    COALESCE(sum(CASE WHEN v_motorcycle_category = 'big_bike' THEN
large_bike_duration_minutes ELSE duration_minutes END), 0)
  INTO v_total_estimate, v_duration_minutes
  FROM public.services
```

```
  WHERE id = ANY(p_service_ids);

  PERFORM pg_advisory_xact_lock(hashtext('booking-capacity:' ||
p_appointment_date::text));

  IF p_status = 'confirmed' AND v_assigned_crew_id IS NULL AND NOT
p_crew_assignment_manual THEN
    SELECT crew.id
      INTO v_assigned_crew_id
    FROM public.crew_members AS crew
    WHERE crew.is_active = true
      AND crew.is_archived = false
      AND public.is_crew_available_for_appointment(
        crew.id, p_appointment_date, p_start_time, v_duration_minutes,
p_appointment_id
      )
    ORDER BY public.crew_daily_workload(crew.id, p_appointment_date,
p_appointment_id),
      crew.name,
      crew.id
    LIMIT 1;

    IF v_assigned_crew_id IS NULL THEN
      RAISE EXCEPTION 'No crew member is available for the selected
appointment period.';
    END IF;
  END IF;

  IF v_assigned_crew_id IS NOT NULL
     AND NOT public.is_crew_available_for_appointment(
       v_assigned_crew_id, p_appointment_date, p_start_time,
v_duration_minutes, p_appointment_id
     ) THEN
    RAISE EXCEPTION 'Choose a crew member who is free for the full
appointment period.';
  END IF;

  UPDATE public.appointments
  SET appointment_date = p_appointment_date,
      start_time = p_start_time,
      assigned_crew_id = v_assigned_crew_id,
      status = p_status,
      admin_notes = NULLIF(btrim(p_admin_notes), ''),
      total_estimate = v_total_estimate,
      booking_duration_minutes = v_duration_minutes
```

```
  WHERE id = p_appointment_id
    AND is_archived = false;

  DELETE FROM public.appointment_services
  WHERE appointment_id = p_appointment_id;

  INSERT INTO public.appointment_services (
    appointment_id, service_id, service_name, price, duration_minutes
  )
  SELECT
    p_appointment_id,
    service.id,
    service.name,
    CASE WHEN v_motorcycle_category = 'big_bike' THEN
service.large_bike_price ELSE service.price END,
    CASE WHEN v_motorcycle_category = 'big_bike' THEN
service.large_bike_duration_minutes ELSE service.duration_minutes END
  FROM public.services AS service
  WHERE service.id = ANY(p_service_ids);
END;
$function$;

-- 4. Duplicate rule by plate number ------------------------------------

CREATE OR REPLACE FUNCTION public.assert_no_duplicate_active_custome
r_service_appointment(p_appointment_id uuid)
 RETURNS void
 LANGUAGE plpgsql
 SET search_path TO 'public'
AS $function$
DECLARE
  v_plate text;
  v_plate_label text;
  v_appointment_date date;
  v_service_name text;
  v_start_time time;
BEGIN
  SELECT public.normalize_plate_number(appointment.plate_number),
    btrim(appointment.plate_number),
    appointment.appointment_date
  INTO v_plate, v_plate_label, v_appointment_date
  FROM public.appointments AS appointment
  WHERE appointment.id = p_appointment_id
    AND appointment.is_archived = false
    AND appointment.rescheduled_to_appointment_id IS NULL
```

```
    AND appointment.status NOT IN ('completed', 'cancelled', 'rejected');

  IF NOT FOUND OR v_plate = '' THEN
    RETURN;
  END IF;

  SELECT current_service.service_name, other_appointment.start_time
  INTO v_service_name, v_start_time
  FROM public.appointment_services AS current_service
  JOIN public.appointments AS other_appointment
    ON public.normalize_plate_number(other_appointment.plate_number) =
v_plate
   AND other_appointment.appointment_date = v_appointment_date
   AND other_appointment.id <> p_appointment_id
   AND other_appointment.is_archived = false
   AND other_appointment.rescheduled_to_appointment_id IS NULL
   AND other_appointment.status NOT IN ('completed', 'cancelled', 'rejected')
  JOIN public.appointment_services AS other_service
    ON other_service.appointment_id = other_appointment.id
   AND (
     other_service.service_id = current_service.service_id
     OR (
       other_service.service_id IS NULL
       AND current_service.service_id IS NULL
       AND other_service.service_name = current_service.service_name
     )
   )
  WHERE current_service.appointment_id = p_appointment_id
  LIMIT 1;

  IF FOUND THEN
    RAISE EXCEPTION
      'The motorcycle with plate number % already has an appointment for "%" on
% at %.',
      v_plate_label,
      v_service_name,
      to_char(v_appointment_date, 'FMDay, FMMonth FMDD, YYYY'),
      to_char(v_start_time, 'FMHH12:MI AM')
      USING ERRCODE = '23505';
  END IF;
END;
$function$;

NOTIFY pgrst, 'reload schema';
```

supabase/migrations/20261018100000_dashboard_summary_and_safe_crew_shift_changes.sql

---

```
-- 1. The admin dashboard counts in the database. Loading every appointment into
--    the browser stops working past the API's 1,000-row limit, and the newest
--    appointments were the ones left out. Revenue uses the financial report's
--    calculation, so the dashboard and Reports always agree.
-- 2. Removing or editing a crew shift moves that mechanic's bookings on that day
--    to another free mechanic. If no one else is free for a booking, nothing
--    changes and the booking references are reported.

-- 1. Dashboard ------------------------------------------------------------

CREATE OR REPLACE FUNCTION public.get_admin_dashboard_summary(
  p_from date,
  p_to date,
  p_previous_from date,
  p_previous_to date
)
RETURNS jsonb
LANGUAGE plpgsql
STABLE
SECURITY INVOKER
SET search_path = public, auth
AS $$
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;
  IF p_from IS NULL OR p_to IS NULL OR p_from > p_to
     OR p_previous_from IS NULL OR p_previous_to IS NULL OR p_previous_from > p_previous_to THEN
    RAISE EXCEPTION 'A valid dashboard date range is required';
  END IF;

  RETURN (
    WITH scoped AS (
      SELECT
        appointment.status,
        appointment.phone,
        appointment.appointment_date BETWEEN p_from AND p_to AS in_current,
        appointment.appointment_date BETWEEN p_previous_from AND p_previous_to AS in_previous,
```

```
      -- A rescheduled booking's old record is replaced by its new appointment.
      appointment.status NOT IN ('cancelled', 'rejected', 'no_show')
        AND appointment.rescheduled_to_appointment_id IS NULL AS is_active
    FROM public.appointments AS appointment
    -- Completed appointments always count, as in the financial report.
    WHERE (appointment.is_archived = false OR appointment.status =
'completed')
      AND (
        appointment.appointment_date BETWEEN p_from AND p_to
        OR appointment.appointment_date BETWEEN p_previous_from AND
p_previous_to
      )
  ),
  -- Waiting for the shop whatever their date: new bookings to approve and
  -- reschedule requests to review.
  open_requests AS (
    SELECT
      count(*) FILTER (
        WHERE appointment.status = 'pending'
          AND appointment.pending_reschedule_request_id IS NULL
      ) AS new_bookings,
      count(*) FILTER (
        WHERE appointment.pending_reschedule_request_id IS NOT NULL
      ) AS reschedule_requests
    FROM public.appointments AS appointment
    WHERE appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND (
        appointment.status = 'pending'
        OR (
          appointment.pending_reschedule_request_id IS NOT NULL
          AND appointment.status IN ('pending', 'confirmed')
        )
      )
  )
  SELECT jsonb_build_object(
    'current', jsonb_build_object(
      'appointments', count(*) FILTER (WHERE in_current AND is_active),
      'customers', count(DISTINCT phone) FILTER (WHERE in_current AND
is_active),
      'revenue', (
        public.get_completed_financial_report_page(p_from, p_to, NULL, NULL, 1,
0)
          -> 'metrics' ->> 'total_revenue'
      )::numeric,
```

```
        'statuses', jsonb_build_object(
          'pending', count(*) FILTER (WHERE in_current AND status = 'pending'),
          'confirmed', count(*) FILTER (WHERE in_current AND status =
'confirmed'),
          'in_progress', count(*) FILTER (WHERE in_current AND status =
'in_progress'),
          'completed', count(*) FILTER (WHERE in_current AND status =
'completed'),
          'rescheduled', count(*) FILTER (WHERE in_current AND status =
'rescheduled'),
          'cancelled', count(*) FILTER (WHERE in_current AND status IN
('cancelled', 'rejected')),
          'no_show', count(*) FILTER (WHERE in_current AND status = 'no_show')
        )
      ),
      'previous', jsonb_build_object(
        'appointments', count(*) FILTER (WHERE in_previous AND is_active),
        'customers', count(DISTINCT phone) FILTER (WHERE in_previous AND
is_active),
        'revenue', (
          public.get_completed_financial_report_page(p_previous_from,
p_previous_to, NULL, NULL, 1, 0)
            -> 'metrics' ->> 'total_revenue'
        )::numeric
      ),
      'needs_action', (
        SELECT jsonb_build_object(
          'new_bookings', open_requests.new_bookings,
          'reschedule_requests', open_requests.reschedule_requests
        )
        FROM open_requests
      )
    )
    FROM scoped
  );
END;
$$;

-- Appointment counts for the dashboard chart, per day or per month.
CREATE OR REPLACE FUNCTION public.get_admin_dashboard_chart(
  p_from date,
  p_to date,
  p_by_month boolean DEFAULT false
)
RETURNS jsonb
```

```
LANGUAGE plpgsql
STABLE
SECURITY INVOKER
SET search_path = public, auth
AS $$
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;
  IF p_from IS NULL OR p_to IS NULL OR p_from > p_to THEN
    RAISE EXCEPTION 'A valid dashboard date range is required';
  END IF;

  RETURN COALESCE((
    SELECT jsonb_agg(to_jsonb(bucket) ORDER BY bucket.period)
    FROM (
      SELECT
        (
          CASE
            WHEN p_by_month THEN date_trunc('month',
appointment.appointment_date::timestamp)::date
            ELSE appointment.appointment_date
          END
        )::text AS period,
        count(*) FILTER (WHERE appointment.status = 'completed') AS completed,
        count(*) FILTER (WHERE appointment.status = 'confirmed') AS confirmed,
        count(*) FILTER (
          WHERE appointment.status IN ('cancelled', 'rejected', 'no_show')
        ) AS cancelled,
        count(*) FILTER (WHERE appointment.status = 'in_progress') AS
in_progress
      FROM public.appointments AS appointment
      WHERE (appointment.is_archived = false OR appointment.status =
'completed')
        AND appointment.appointment_date BETWEEN p_from AND p_to
      GROUP BY 1
    ) AS bucket
  ), '[]'::jsonb);
END;
$$;

REVOKE ALL ON FUNCTION public.get_admin_dashboard_summary(date,
date, date, date) FROM PUBLIC, anon;
GRANT EXECUTE ON FUNCTION
public.get_admin_dashboard_summary(date, date, date, date)
```

```
  TO authenticated, service_role;
REVOKE ALL ON FUNCTION public.get_admin_dashboard_chart(date, date,
boolean) FROM PUBLIC, anon;
GRANT EXECUTE ON FUNCTION public.get_admin_dashboard_chart(date,
date, boolean)
  TO authenticated, service_role;

-- 2. Crew shift changes ------------------------------------------------

-- Removes a date shift (p_remove) or changes its hours. Bookings, and
later-day
-- parts of multi-day bookings, that the mechanic can no longer do that day go
-- to the free mechanic with the fewest bookings. Returns the bookings moved.
CREATE OR REPLACE FUNCTION public.change_crew_schedule(
  p_schedule_id uuid,
  p_remove boolean,
  p_start_time time DEFAULT NULL,
  p_end_time time DEFAULT NULL
)
RETURNS jsonb
LANGUAGE plpgsql
SECURITY INVOKER
SET search_path = public, auth
AS $$
DECLARE
  v_schedule public.crew_schedules%ROWTYPE;
  v_work record;
  v_replacement_id uuid;
  v_replacement_name text;
  v_moved jsonb := '[]'::jsonb;
  v_stuck text[] := ARRAY[]::text[];
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;

  SELECT * INTO v_schedule FROM public.crew_schedules WHERE id =
p_schedule_id;
  IF NOT FOUND THEN
    RAISE EXCEPTION 'This shift no longer exists. Refresh the list and try again.'
      USING ERRCODE = 'P0002';
  END IF;
  IF v_schedule.schedule_date IS NULL THEN
    RAISE EXCEPTION 'Only shifts on a specific date can be changed here.';
  END IF;
```

```
  IF NOT p_remove AND (p_start_time IS NULL OR p_end_time IS NULL OR
p_start_time >= p_end_time) THEN
    RAISE EXCEPTION 'The shift must end after it starts.';
  END IF;

  -- The lock every booking change on this date takes, so no booking can claim
  -- this mechanic while the shift changes.
  PERFORM pg_advisory_xact_lock(hashtext('booking-capacity:' ||
v_schedule.schedule_date::text));

  IF p_remove THEN
    DELETE FROM public.crew_schedules WHERE id = v_schedule.id;
  ELSE
    UPDATE public.crew_schedules
    SET start_time = p_start_time,
        end_time = p_end_time,
        is_working = true
    WHERE id = v_schedule.id;
  END IF;

  FOR v_work IN
    SELECT
      'appointment' AS kind,
      appointment.id AS appointment_id,
      NULL::uuid AS continuation_id,
      appointment.reference_code,
      appointment.customer_name,
      appointment.start_time,
      appointment.booking_duration_minutes AS duration_minutes
    FROM public.appointments AS appointment
    WHERE appointment.assigned_crew_id = v_schedule.crew_id
      AND appointment.appointment_date = v_schedule.schedule_date
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
      AND NOT public.is_crew_available_for_appointment(
        v_schedule.crew_id,
        v_schedule.schedule_date,
        appointment.start_time,
        appointment.booking_duration_minutes,
        appointment.id
      )
    UNION ALL
    SELECT
```

```
      'continuation',
      appointment.id,
      continuation.id,
      appointment.reference_code,
      appointment.customer_name,
      continuation.start_time,
      continuation.booking_duration_minutes
    FROM public.appointment_continuations AS continuation
    JOIN public.appointments AS appointment ON appointment.id =
continuation.appointment_id
    WHERE continuation.assigned_crew_id = v_schedule.crew_id
      AND continuation.appointment_date = v_schedule.schedule_date
      AND appointment.is_archived = false
      AND appointment.rescheduled_to_appointment_id IS NULL
      AND appointment.status IN ('pending', 'confirmed', 'in_progress',
'rescheduled')
      AND NOT public.is_crew_available_for_appointment(
        v_schedule.crew_id,
        v_schedule.schedule_date,
        continuation.start_time,
        continuation.booking_duration_minutes,
        NULL,
        continuation.id
      )
    ORDER BY start_time, reference_code
  LOOP
    SELECT crew.id, crew.name
      INTO v_replacement_id, v_replacement_name
    FROM public.crew_members AS crew
    WHERE crew.id <> v_schedule.crew_id
      AND crew.is_active = true
      AND crew.is_archived = false
      AND public.is_crew_available_for_appointment(
        crew.id,
        v_schedule.schedule_date,
        v_work.start_time,
        v_work.duration_minutes,
        v_work.appointment_id,
        v_work.continuation_id
      )
    ORDER BY public.crew_daily_workload(crew.id, v_schedule.schedule_date,
v_work.appointment_id),
      crew.name,
      crew.id
    LIMIT 1;
```

```
    IF v_replacement_id IS NULL THEN
      v_stuck := array_append(v_stuck, v_work.reference_code);
      CONTINUE;
    END IF;

    IF v_work.kind = 'appointment' THEN
      UPDATE public.appointments
      SET assigned_crew_id = v_replacement_id
      WHERE id = v_work.appointment_id;
    ELSE
      UPDATE public.appointment_continuations
      SET assigned_crew_id = v_replacement_id
      WHERE id = v_work.continuation_id;
    END IF;

    v_moved := v_moved || jsonb_build_array(jsonb_build_object(
      'reference_code', v_work.reference_code,
      'customer_name', v_work.customer_name,
      'start_time', v_work.start_time,
      'mechanic', v_replacement_name
    ));
  END LOOP;

  -- Undo everything, including the shift change, rather than leave a booking
  -- with a mechanic who is not working.
  IF cardinality(v_stuck) > 0 THEN
    RAISE EXCEPTION 'No other mechanic is free for %.',
array_to_string(v_stuck, ', ')
      USING HINT = 'crew_shift_conflict';
  END IF;

  RETURN jsonb_build_object('moved', v_moved);
END;
$$;

REVOKE ALL ON FUNCTION public.change_crew_schedule(uuid, boolean,
time, time) FROM PUBLIC, anon;
GRANT EXECUTE ON FUNCTION public.change_crew_schedule(uuid,
boolean, time, time)
  TO authenticated, service_role;

NOTIFY pgrst, 'reload schema';
```

supabase/migrations/20261021100000_report_filter_multiple_services.sql

---

```
-- The financial report can be filtered by several services at once. A
-- transaction matches when it includes any of the chosen services. An empty or
-- null list means all services, as before.

DROP FUNCTION IF EXISTS public.get_completed_financial_report_page(date,
date, text, text, integer, integer);

CREATE FUNCTION public.get_completed_financial_report_page(
  p_from date,
  p_to date,
  p_category text DEFAULT NULL,
  p_service_names text[] DEFAULT NULL,
  p_limit integer DEFAULT 25,
  p_offset integer DEFAULT 0
)
RETURNS jsonb
LANGUAGE plpgsql
STABLE
SECURITY INVOKER
SET search_path = public, auth
AS $$
DECLARE
  v_limit integer := LEAST(GREATEST(COALESCE(p_limit, 25), 1), 250);
  v_offset integer := GREATEST(COALESCE(p_offset, 0), 0);
BEGIN
  IF auth.uid() IS NULL OR NOT public.has_role(auth.uid(), 'admin') THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;
  IF p_from IS NULL OR p_to IS NULL OR p_from > p_to THEN
    RAISE EXCEPTION 'A valid report date range is required';
  END IF;

  RETURN (
    WITH completed_appointments AS (
      SELECT
        appointment.id,
        appointment.reference_code,
        COALESCE(
          (appointment.service_ended_at AT TIME ZONE 'Asia/Manila')::date,
          appointment.appointment_date
        ) AS transaction_date
      FROM public.appointments AS appointment
      WHERE appointment.status = 'completed'
```

```
  ),
  filtered AS (
    SELECT
      appointment.id,
      appointment.reference_code,
      appointment.transaction_date,
      service_totals.services,
      service_totals.service_amount,
      product_totals.products,
      product_totals.product_amount,
      service_totals.service_amount + product_totals.product_amount AS
total_amount
    FROM completed_appointments AS appointment
    CROSS JOIN LATERAL (
      SELECT
        COALESCE(string_agg(service.service_name, ', ' ORDER BY
service.service_name), '-') AS services,
        COALESCE(sum(service.price), 0)::numeric AS service_amount
      FROM public.appointment_services AS service
      WHERE service.appointment_id = appointment.id
    ) AS service_totals
    CROSS JOIN LATERAL (
      SELECT
        COALESCE(
          string_agg(product.product_name || ' x ' || product.quantity, ', ' ORDER
BY product.product_name),
          '-'
        ) AS products,
        COALESCE(sum(product.quantity * product.unit_price), 0)::numeric AS
product_amount
      FROM public.appointment_products AS product
      WHERE product.appointment_id = appointment.id
    ) AS product_totals
    WHERE appointment.transaction_date BETWEEN p_from AND p_to
      AND (
        (p_category IS NULL AND COALESCE(cardinality(p_service_names), 0) =
0)
        OR EXISTS (
          SELECT 1
          FROM public.appointment_services AS selected_service
          LEFT JOIN public.services AS catalog_service ON catalog_service.id =
selected_service.service_id
          WHERE selected_service.appointment_id = appointment.id
            AND (p_category IS NULL OR COALESCE(catalog_service.category,
'Uncategorized') = p_category)
```

```
            AND (
              COALESCE(cardinality(p_service_names), 0) = 0
              OR selected_service.service_name = ANY(p_service_names)
            )
          )
        )
    ),
    scoped_services AS (
      SELECT
        service.appointment_id,
        service.service_name,
        service.price,
        COALESCE(catalog_service.category, 'Uncategorized') AS category
      FROM public.appointment_services AS service
      JOIN filtered AS appointment ON appointment.id = service.appointment_id
      LEFT JOIN public.services AS catalog_service ON catalog_service.id =
service.service_id
    ),
    scoped_products AS (
      SELECT product.appointment_id, product.product_name, product.quantity,
product.unit_price
      FROM public.appointment_products AS product
      JOIN filtered AS appointment ON appointment.id = product.appointment_id
    ),
    paged AS (
      SELECT *
      FROM filtered
      ORDER BY transaction_date DESC, reference_code ASC
      LIMIT v_limit OFFSET v_offset
    )
    SELECT jsonb_build_object(
      'total', (SELECT count(*) FROM filtered),
      'rows', COALESCE((SELECT jsonb_agg(to_jsonb(paged)) FROM paged),
'[]'::jsonb),
      'metrics', jsonb_build_object(
        'total_revenue', (SELECT COALESCE(sum(total_amount), 0) FROM
filtered),
        'service_revenue', (SELECT COALESCE(sum(service_amount), 0) FROM
filtered),
        'product_revenue', (SELECT COALESCE(sum(product_amount), 0) FROM
filtered),
        'completed_transactions', (SELECT count(*) FROM filtered),
        'total_completed_services', (SELECT count(*) FROM scoped_services),
        'total_products_sold', (SELECT COALESCE(sum(quantity), 0) FROM
scoped_products),
```

```
      'average_transaction_amount', (
        SELECT COALESCE(avg(total_amount), 0) FROM filtered
      )
    ),
    'service_performance', COALESCE((
      SELECT jsonb_agg(to_jsonb(performance) ORDER BY
performance.revenue DESC, performance.service_name)
      FROM (
        SELECT
          service_name,
          count(*)::integer AS completed_count,
          COALESCE(sum(price), 0)::numeric AS revenue
        FROM scoped_services
        GROUP BY service_name
      ) AS performance
    ), '[]'::jsonb),
    'product_sales', COALESCE((
      SELECT jsonb_agg(to_jsonb(sales) ORDER BY sales.total_sales DESC,
sales.product_name, sales.unit_price)
      FROM (
        SELECT
          product_name,
          sum(quantity)::integer AS quantity_sold,
          unit_price,
          COALESCE(sum(quantity * unit_price), 0)::numeric AS total_sales
        FROM scoped_products
        GROUP BY product_name, unit_price
      ) AS sales
    ), '[]'::jsonb),
    'daily_revenue', COALESCE((
      SELECT jsonb_agg(to_jsonb(day_revenue) ORDER BY
day_revenue.period)
      FROM (
        SELECT
          transaction_date::text AS period,
          count(*)::integer AS completed_transactions,
          COALESCE(sum(service_amount), 0)::numeric AS service_revenue,
          COALESCE(sum(product_amount), 0)::numeric AS product_revenue,
          COALESCE(sum(total_amount), 0)::numeric AS total_revenue
        FROM filtered
        GROUP BY transaction_date
      ) AS day_revenue
    ), '[]'::jsonb),
    'monthly_revenue', COALESCE((
      SELECT jsonb_agg(to_jsonb(month_revenue) ORDER BY
```

```
month_revenue.period)
        FROM (
          SELECT
            date_trunc('month', transaction_date::timestamp)::date::text AS period,
            count(*)::integer AS completed_transactions,
            COALESCE(sum(service_amount), 0)::numeric AS service_revenue,
            COALESCE(sum(product_amount), 0)::numeric AS product_revenue,
            COALESCE(sum(total_amount), 0)::numeric AS total_revenue
          FROM filtered
          GROUP BY date_trunc('month', transaction_date::timestamp)::date
        ) AS month_revenue
      ), '[]'::jsonb)
    )
  );
END;
$$;

REVOKE ALL ON FUNCTION public.get_completed_financial_report_page(date,
date, text, text[], integer, integer)
  FROM PUBLIC, anon;
GRANT EXECUTE ON FUNCTION
public.get_completed_financial_report_page(date, date, text, text[], integer,
integer)
  TO authenticated, service_role;

NOTIFY pgrst, 'reload schema';
```

supabase/migrations/20261026100000_drop_unused_booking_function_versions.sql

---

```
-- Drops the one booking function version nothing calls.
--
-- create_booking_atomic exists with 22, 23 and 24 parameters, and that is on
-- purpose: each newer version adds one parameter and calls the previous one
-- (24 with p_wants_products -> 23 with p_email -> 22, which does the work).
-- update_appointment_details_atomic works the same way (12 -> 8). Those
stay.
--
-- The 21-parameter version was a bridge for a frontend release that did not
-- send p_continuation_segments (migration 20260928130000). Bookings now
go
-- through the server with all 24 parameters, so it is no longer used.
DROP FUNCTION IF EXISTS public.create_booking_atomic(
  text, uuid, text, text, text, text, text, integer, text, date, time without time zone,
  text, numeric, integer, uuid, jsonb, text, text, text, text, text
);
```

supabase/migrations/20261026110000_appointment_reminders_and_scheduled_notifications.sql

---

```
-- Appointment reminders, and a scheduled run that sends them and retries
-- customer texts and emails that failed.
--
-- Every 5 minutes pg_cron asks the site to do this work by posting to the
-- address in the Vault secret scheduled_notifications_url, with the Vault
-- secret scheduled_notifications_secret as a bearer token. The site compares
-- it with its SCHEDULED_NOTIFICATIONS_SECRET. Until both secrets are
saved,
-- nothing is posted and the admin is told the run is not set up.

CREATE EXTENSION IF NOT EXISTS pg_net WITH SCHEMA extensions;

-- One row per appointment and channel, like
appointment_completion_notifications.
CREATE TABLE IF NOT EXISTS public.appointment_reminders (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  appointment_id uuid NOT NULL REFERENCES public.appointments(id) ON
DELETE CASCADE,
  channel text NOT NULL CHECK (channel IN ('email', 'sms')),
  recipient text NOT NULL,
  status text NOT NULL DEFAULT 'pending'
    CHECK (status IN ('pending', 'sending', 'sent', 'skipped')),
  attempt_count integer NOT NULL DEFAULT 0 CHECK (attempt_count >= 0),
  last_error text,
  sent_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (appointment_id, channel)
);

-- Only the server, with the service role, reads and writes reminders.
ALTER TABLE public.appointment_reminders ENABLE ROW LEVEL
SECURITY;
REVOKE ALL ON TABLE public.appointment_reminders FROM anon,
authenticated;

-- When the scheduled run last started and finished, so a stopped run is noticed.
CREATE TABLE IF NOT EXISTS public.scheduled_notification_runs (
  id boolean PRIMARY KEY DEFAULT true CHECK (id),
  last_started_at timestamptz,
  last_finished_at timestamptz,
  last_error text
```

```
);

ALTER TABLE public.scheduled_notification_runs ENABLE ROW LEVEL
SECURITY;
REVOKE ALL ON TABLE public.scheduled_notification_runs FROM anon,
authenticated;

-- Counting from now gives a newly set up run 30 minutes before it is reported
-- as stopped.
INSERT INTO public.scheduled_notification_runs (id, last_finished_at)
VALUES (true, now())
ON CONFLICT (id) DO NOTHING;

-- Queues a reminder for each confirmed appointment that starts within the
-- reminder window set in Settings. Appointments confirmed in the last 12 hours
-- are left out: their confirmation message has the same details.
CREATE OR REPLACE FUNCTION
public.queue_due_appointment_reminders()
RETURNS integer
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
  v_enabled boolean;
  v_hours integer;
  v_queued integer;
BEGIN
  SELECT notify_reminder, reminder_hours_before
    INTO v_enabled, v_hours
  FROM public.shop_settings
  WHERE id;
  IF NOT COALESCE(v_enabled, false) THEN
    RETURN 0;
  END IF;

  WITH due AS (
    SELECT a.id, a.email, a.phone
    FROM public.appointments a
    WHERE a.status = 'confirmed'
      AND NOT a.is_archived
      AND (a.appointment_date + a.start_time) AT TIME ZONE 'Asia/Manila' >
now()
      AND (a.appointment_date + a.start_time) AT TIME ZONE 'Asia/Manila'
        <= now() + make_interval(hours => v_hours)
```

```
    AND NOT EXISTS (
      SELECT 1
      FROM public.appointment_confirmation_emails c
      WHERE c.appointment_id = a.id
        AND c.created_at > now() - interval '12 hours'
    )
    AND NOT EXISTS (
      SELECT 1
      FROM public.appointment_confirmation_sms c
      WHERE c.appointment_id = a.id
        AND c.created_at > now() - interval '12 hours'
    )
  )
  INSERT INTO public.appointment_reminders (appointment_id, channel,
recipient)
  SELECT id, 'email', lower(btrim(email)) FROM due WHERE
NULLIF(btrim(email), '') IS NOT NULL
  UNION ALL
  SELECT id, 'sms', phone FROM due WHERE NULLIF(btrim(phone), '') IS NOT
NULL
  ON CONFLICT (appointment_id, channel) DO NOTHING;

  GET DIAGNOSTICS v_queued = ROW_COUNT;
  RETURN v_queued;
END;
$$;

REVOKE ALL ON FUNCTION public.queue_due_appointment_reminders()
FROM PUBLIC, anon, authenticated;
GRANT EXECUTE ON FUNCTION public.queue_due_appointment_reminders()
TO service_role;

-- Asks the site to send due reminders and retry failed customer messages.
CREATE OR REPLACE FUNCTION public.request_scheduled_notifications()
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
  v_url text;
  v_secret text;
BEGIN
  SELECT decrypted_secret INTO v_url
  FROM vault.decrypted_secrets
```

```
  WHERE name = 'scheduled_notifications_url';
  SELECT decrypted_secret INTO v_secret
  FROM vault.decrypted_secrets
  WHERE name = 'scheduled_notifications_secret';
  IF NULLIF(v_url, '') IS NULL OR NULLIF(v_secret, '') IS NULL THEN
    RETURN;
  END IF;

  PERFORM net.http_post(
    url := v_url,
    headers := jsonb_build_object(
      'Content-Type', 'application/json',
      'Authorization', 'Bearer ' || v_secret
    ),
    body := '{}'::jsonb,
    timeout_milliseconds := 55000
  );
END;
$$;

REVOKE ALL ON FUNCTION public.request_scheduled_notifications() FROM
PUBLIC, anon, authenticated;

-- Tells the admin when the scheduled run is not set up or has stopped, with at
-- most one unread alert at a time.
CREATE OR REPLACE FUNCTION
public.alert_if_scheduled_notifications_stopped()
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
  v_configured boolean;
  v_last_finished timestamptz;
BEGIN
  SELECT last_finished_at INTO v_last_finished
  FROM public.scheduled_notification_runs
  WHERE id;
  IF v_last_finished > now() - interval '30 minutes' THEN
    RETURN;
  END IF;
  IF EXISTS (
    SELECT 1
    FROM public.notifications
```

```
    WHERE type = 'scheduled_notifications_stopped'
      AND NOT is_read
  ) THEN
    RETURN;
  END IF;

  SELECT count(*) = 2 INTO v_configured
  FROM vault.decrypted_secrets
  WHERE name IN ('scheduled_notifications_url',
'scheduled_notifications_secret')
    AND NULLIF(decrypted_secret, '') IS NOT NULL;

  INSERT INTO public.notifications (type, title, message)
  VALUES (
    'scheduled_notifications_stopped',
    CASE
      WHEN v_configured THEN 'Automatic reminders and retries have stopped'
      ELSE 'Automatic reminders and retries are not set up'
    END,
    CASE
      WHEN v_configured THEN
        'They last finished '
          || COALESCE(
            to_char(v_last_finished AT TIME ZONE 'Asia/Manila', 'Mon FMDD, YYYY
FMHH12:MI AM'),
            'never'
          )
          || '. Appointment reminders are not being sent and failed customer texts
and emails '
          || 'are not being retried. Check that the site is online and that its '
          || 'SCHEDULED_NOTIFICATIONS_SECRET matches the one saved in
the database.'
      ELSE
        'Appointment reminders are not being sent and failed customer texts and
emails are not '
          || 'being retried until the site address and secret for scheduled notifications
are '
          || 'saved in the database.'
    END
  );
END;
$$;

REVOKE ALL ON FUNCTION public.alert_if_scheduled_notifications_stopped()
FROM PUBLIC, anon, authenticated;
```

```
DO $$
BEGIN
  PERFORM cron.unschedule(jobid)
  FROM cron.job
  WHERE jobname IN ('request-scheduled-notifications',
'alert-stopped-scheduled-notifications');

  PERFORM cron.schedule(
    'request-scheduled-notifications',
    '*/5 * * * *',
    'SELECT public.request_scheduled_notifications();'
  );
  PERFORM cron.schedule(
    'alert-stopped-scheduled-notifications',
    '*/15 * * * *',
    'SELECT public.alert_if_scheduled_notifications_stopped();'
  );
END;
$$;
```

tsconfig.json

```
{
  "include": ["src/**/*.ts", "src/**/*.tsx", "vite.config.ts", "eslint.config.js"],
  "compilerOptions": {
    "target": "ES2022",
    "jsx": "react-jsx",
    "module": "ESNext",
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "types": ["vite/client"],

    "moduleResolution": "Bundler",
    "allowImportingTsExtensions": true,
    "verbatimModuleSyntax": false,
    "noEmit": true,

    "skipLibCheck": true,
    "strict": true,
    "noUnusedLocals": false,
    "noUnusedParameters": false,
    "noFallthroughCasesInSwitch": true,
    "noImplicitOverride": true,
    "noImplicitReturns": true,
    "noPropertyAccessFromIndexSignature": true,
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true,
    "noUncheckedSideEffectImports": true,
    "paths": {
      "@/*": ["./src/*"]
    }
  }
}
```

vite.config.ts

```
import { defineConfig, loadEnv, type ViteDevServer } from "vite";
import { tanstackStart } from "@tanstack/react-start/plugin/vite";
import { nitro } from "nitro/vite";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";

// The offline copy runs with --mode offline (npm run offline), which loads
// .env.offline over .env. The browser then reaches the local Supabase through
// this server at /supabase, so phones on the laptop's hotspot only need the
// app's address. Port 4000 keeps its browser data apart from npm run dev.
export default defineConfig(({ mode }) => ({
  ...(mode === "offline" && {
    server: {
      host: true,
      port: 4000,
      strictPort: true,
      // The local database calls /api/scheduled-notifications from Docker,
      // where this laptop is host.docker.internal (offline/setup.sql).
      allowedHosts: ["host.docker.internal"],
      proxy: {
        "/supabase": {
          target: loadEnv(mode, process.cwd(), "")["SUPABASE_URL"] ??
"http://127.0.0.1:54321",
          changeOrigin: true,
          ws: true,
          rewrite: (path: string) => path.replace(/^\/supabase/, ""),
        },
      },
    },
  }),
  resolve: {
    tsconfigPaths: true,
  },
  environments: {
    ssr: {
      build: {
        rollupOptions: {
          output: {
            inlineDynamicImports: true,
          },
        },
      },
    },
```

```
  },
  plugins: [
    tanstackStart({
      server: { entry: "server" },
    }),
    nitro({ preset: "vercel" }),
    react(),
    tailwindcss(),
    mode === "offline" && {
      // Nitro's dev server answers every path before Vite does, except paths
      // another middleware is mounted on. Mounting these pass-throughs lets
      // /supabase reach the proxy above, and /src reach Vite for phones: over
      // the hotspot's plain http address, browsers don't mark module requests
      // as scripts, so Nitro would answer the logo's JSON import with a 404 page.
      name: "offline-pass-through",
      configureServer(server: ViteDevServer) {
        for (const path of ["/supabase", "/src"]) {
          server.middlewares.use(path, (_req, _res, next) => next());
        }
      },
    },
  ],
}));
```